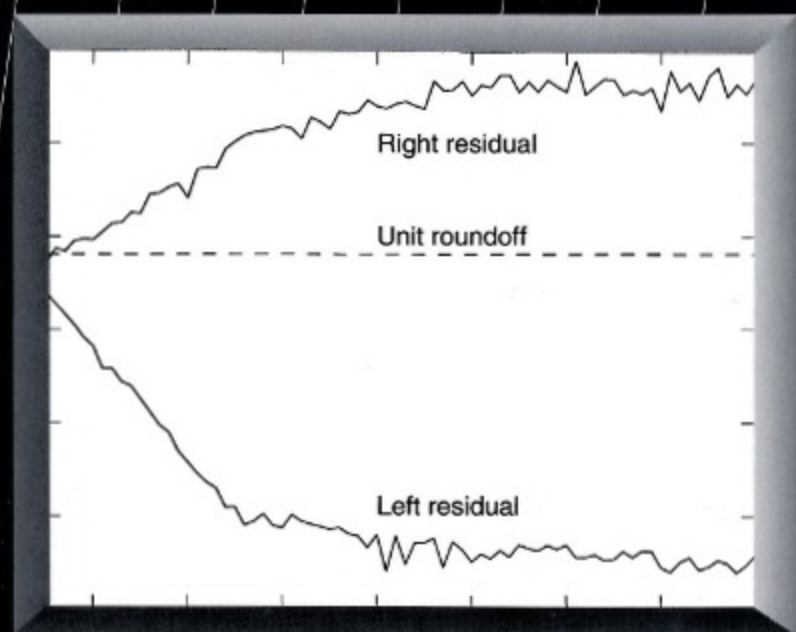


Nicholas J. Higham

Accuracy and Stability of Numerical Algorithms

SECOND EDITION



siam

Nicholas J. Higham

University of Manchester
Manchester, England

Accuracy and Stability of Numerical Algorithms

SECOND EDITION



Society for Industrial and Applied Mathematics
Philadelphia

Copyright © 2002 by the Society for Industrial and Applied Mathematics.

10 9 8 7 6 5 4 3 2 1

All rights reserved. Printed in the United States of America. No part of this book may be reproduced, stored, or transmitted in any manner without the written permission of the publisher. For information, write to the Society for Industrial and Applied Mathematics, 3600 University City Science Center, Philadelphia, PA 19104-2688.

Library of Congress Cataloging-in-Publication Data

Higham, Nicholas J., 1961-

Accuracy and stability of numerical algorithms / Nicholas J. Higham.—
2nd ed.

p. cm.

Includes bibliographical references.

ISBN 0-89871-521-0

1. Numerical analysis—Data processing. 2. Computer algorithms. I.
Title.

QA297 .H53 2002

519.4'0285'51—dc21

2002075848

Dedicated to
Alan M. Turing
and
James H. Wilkinson

Contents

List of Figures	xvii
List of Tables	xix
Preface to Second Edition	xxi
Preface to First Edition	xxv
About the Dedication	xxix
1 Principles of Finite Precision Computation	1
1.1 Notation and Background	2
1.2 Relative Error and Significant Digits	3
1.3 Sources of Errors	5
1.4 Precision Versus Accuracy	6
1.5 Backward and Forward Errors	6
1.6 Conditioning	8
1.7 Cancellation	9
1.8 Solving a Quadratic Equation	10
1.9 Computing the Sample Variance	11
1.10 Solving Linear Equations	12
1.10.1 GEPP Versus Cramer’s Rule	13
1.11 Accumulation of Rounding Errors	14
1.12 Instability Without Cancellation	14
1.12.1 The Need for Pivoting	15
1.12.2 An Innocuous Calculation?	15
1.12.3 An Infinite Sum	16
1.13 Increasing the Precision	17
1.14 Cancellation of Rounding Errors	19
1.14.1 Computing $(e^x - 1)/x$	19
1.14.2 QR Factorization	21
1.15 Rounding Errors Can Be Beneficial	22
1.16 Stability of an Algorithm Depends on the Problem	24
1.17 Rounding Errors Are Not Random	25
1.18 Designing Stable Algorithms	26
1.19 Misconceptions	28
1.20 Rounding Errors in Numerical Analysis	28
1.21 Notes and References	28
Problems	31

2	Floating Point Arithmetic	35
2.1	Floating Point Number System	36
2.2	Model of Arithmetic	40
2.3	IEEE Arithmetic	41
2.4	Aberrant Arithmetics	43
2.5	Exact Subtraction	45
2.6	Fused Multiply-Add Operation	46
2.7	Choice of Base and Distribution of Numbers	47
2.8	Statistical Distribution of Rounding Errors	48
2.9	Alternative Number Systems	49
2.10	Elementary Functions	50
2.11	Accuracy Tests	51
2.12	Notes and References	52
	Problems	57
3	Basics	61
3.1	Inner and Outer Products	62
3.2	The Purpose of Rounding Error Analysis	65
3.3	Running Error Analysis	65
3.4	Notation for Error Analysis	67
3.5	Matrix Multiplication	69
3.6	Complex Arithmetic	71
3.7	Miscellany	73
3.8	Error Analysis Demystified	74
3.9	Other Approaches	76
3.10	Notes and References	76
	Problems	77
4	Summation	79
4.1	Summation Methods	80
4.2	Error Analysis	81
4.3	Compensated Summation	83
4.4	Other Summation Methods	88
4.5	Statistical Estimates of Accuracy	88
4.6	Choice of Method	89
4.7	Notes and References	90
	Problems	91
5	Polynomials	93
5.1	Horner's Method	94
5.2	Evaluating Derivatives	96
5.3	The Newton Form and Polynomial Interpolation	99
5.4	Matrix Polynomials	102
5.5	Notes and References	102
	Problems	104

6	Norms	105
6.1	Vector Norms	106
6.2	Matrix Norms	107
6.3	The Matrix p -Norm	112
6.4	Singular Value Decomposition	114
6.5	Notes and References	114
	Problems	115
7	Perturbation Theory for Linear Systems	119
7.1	Normwise Analysis	120
7.2	Componentwise Analysis	122
7.3	Scaling to Minimize the Condition Number	125
7.4	The Matrix Inverse	127
7.5	Extensions	128
7.6	Numerical Stability	129
7.7	Practical Error Bounds	130
7.8	Perturbation Theory by Calculus	132
7.9	Notes and References	132
	Problems	134
8	Triangular Systems	139
8.1	Backward Error Analysis	140
8.2	Forward Error Analysis	142
8.3	Bounds for the Inverse	147
8.4	A Parallel Fan-In Algorithm	149
8.5	Notes and References	151
	8.5.1 LAPACK	153
	Problems	153
9	LU Factorization and Linear Equations	157
9.1	Gaussian Elimination and Pivoting Strategies	158
9.2	LU Factorization	160
9.3	Error Analysis	163
9.4	The Growth Factor	166
9.5	Diagonally Dominant and Banded Matrices	170
9.6	Tridiagonal Matrices	174
9.7	More Error Bounds	176
9.8	Scaling and Choice of Pivoting Strategy	177
9.9	Variants of Gaussian Elimination	179
9.10	A Posteriori Stability Tests	180
9.11	Sensitivity of the LU Factorization	181
9.12	Rank-Revealing LU Factorizations	182
9.13	Historical Perspective	183
9.14	Notes and References	187
	9.14.1 LAPACK	191
	Problems	192

10 Cholesky Factorization	195
10.1 Symmetric Positive Definite Matrices	196
10.1.1 Error Analysis	197
10.2 Sensitivity of the Cholesky Factorization	201
10.3 Positive Semidefinite Matrices	201
10.3.1 Perturbation Theory	203
10.3.2 Error Analysis	205
10.4 Matrices with Positive Definite Symmetric Part	208
10.5 Notes and References	209
10.5.1 LAPACK	210
Problems	211
11 Symmetric Indefinite and Skew-Symmetric Systems	213
11.1 Block LDL ^T Factorization for Symmetric Matrices	214
11.1.1 Complete Pivoting	215
11.1.2 Partial Pivoting	216
11.1.3 Rook Pivoting	219
11.1.4 Tridiagonal Matrices	221
11.2 Aasen's Method	222
11.2.1 Aasen's Method Versus Block LDL ^T Factorization	224
11.3 Block LDL ^T Factorization for Skew-Symmetric Matrices	225
11.4 Notes and References	226
11.4.1 LAPACK	228
Problems	228
12 Iterative Refinement	231
12.1 Behaviour of the Forward Error	232
12.2 Iterative Refinement Implies Stability	235
12.3 Notes and References	240
12.3.1 LAPACK	242
Problems	242
13 Block LU Factorization	245
13.1 Block Versus Partitioned LU Factorization	246
13.2 Error Analysis of Partitioned LU Factorization	249
13.3 Error Analysis of Block LU Factorization	250
13.3.1 Block Diagonal Dominance	251
13.3.2 Symmetric Positive Definite Matrices	255
13.4 Notes and References	256
13.4.1 LAPACK	257
Problems	257
14 Matrix Inversion	259
14.1 Use and Abuse of the Matrix Inverse	260
14.2 Inverting a Triangular Matrix	262
14.2.1 Unblocked Methods	262
14.2.2 Block Methods	265
14.3 Inverting a Full Matrix by LU Factorization	267

14.3.1	Method A	267
14.3.2	Method B	268
14.3.3	Method C	269
14.3.4	Method D	270
14.3.5	Summary	271
14.4	Gauss–Jordan Elimination	273
14.5	Parallel Inversion Methods	278
14.6	The Determinant	279
14.6.1	Hyman’s Method	280
14.7	Notes and References	281
14.7.1	LAPACK	282
	Problems	283
15	Condition Number Estimation	287
15.1	How to Estimate Componentwise Condition Numbers	288
15.2	The p -Norm Power Method	289
15.3	LAPACK 1-Norm Estimator	292
15.4	Block 1-Norm Estimator	294
15.5	Other Condition Estimators	295
15.6	Condition Numbers of Tridiagonal Matrices	299
15.7	Notes and References	301
15.7.1	LAPACK	303
	Problems	303
16	The Sylvester Equation	305
16.1	Solving the Sylvester Equation	307
16.2	Backward Error	308
16.2.1	The Lyapunov Equation	311
16.3	Perturbation Result	313
16.4	Practical Error Bounds	315
16.5	Extensions	316
16.6	Notes and References	317
16.6.1	LAPACK	318
	Problems	318
17	Stationary Iterative Methods	321
17.1	Survey of Error Analysis	323
17.2	Forward Error Analysis	325
17.2.1	Jacobi’s Method	328
17.2.2	Successive Overrelaxation	329
17.3	Backward Error Analysis	330
17.4	Singular Systems	331
17.4.1	Theoretical Background	331
17.4.2	Forward Error Analysis	333
17.5	Stopping an Iterative Method	335
17.6	Notes and References	337
	Problems	337

18 Matrix Powers	339
18.1 Matrix Powers in Exact Arithmetic	340
18.2 Bounds for Finite Precision Arithmetic	346
18.3 Application to Stationary Iteration	351
18.4 Notes and References	351
Problems	352
19 QR Factorization	353
19.1 Householder Transformations	354
19.2 QR Factorization	355
19.3 Error Analysis of Householder Computations	357
19.4 Pivoting and Row-Wise Stability	362
19.5 Aggregated Householder Transformations	363
19.6 Givens Rotations	365
19.7 Iterative Refinement	368
19.8 Gram–Schmidt Orthogonalization	369
19.9 Sensitivity of the QR Factorization	373
19.10 Notes and References	374
19.10.1 LAPACK	377
Problems	378
20 The Least Squares Problem	381
20.1 Perturbation Theory	382
20.2 Solution by QR Factorization	384
20.3 Solution by the Modified Gram–Schmidt Method	386
20.4 The Normal Equations	386
20.5 Iterative Refinement	388
20.6 The Seminormal Equations	391
20.7 Backward Error	392
20.8 Weighted Least Squares Problems	395
20.9 The Equality Constrained Least Squares Problem	396
20.9.1 Perturbation Theory	396
20.9.2 Methods	397
20.10 Proof of Wedin’s Theorem	400
20.11 Notes and References	402
20.11.1 LAPACK	405
Problems	405
21 Underdetermined Systems	407
21.1 Solution Methods	408
21.2 Perturbation Theory and Backward Error	409
21.3 Error Analysis	411
21.4 Notes and References	413
21.4.1 LAPACK	414
Problems	414

22 Vandermonde Systems	415
22.1 Matrix Inversion	416
22.2 Primal and Dual Systems	418
22.3 Stability	423
22.3.1 Forward Error	424
22.3.2 Residual	425
22.3.3 Dealing with Instability	426
22.4 Notes and References	428
Problems	430
23 Fast Matrix Multiplication	433
23.1 Methods	434
23.2 Error Analysis	438
23.2.1 Winograd's Method	439
23.2.2 Strassen's Method	440
23.2.3 Bilinear Noncommutative Algorithms	443
23.2.4 The 3M Method	444
23.3 Notes and References	446
Problems	448
24 The Fast Fourier Transform and Applications	451
24.1 The Fast Fourier Transform	452
24.2 Circulant Linear Systems	454
24.3 Notes and References	456
Problems	457
25 Nonlinear Systems and Newton's Method	459
25.1 Newton's Method	460
25.2 Error Analysis	461
25.3 Special Cases and Experiments	462
25.4 Conditioning	464
25.5 Stopping an Iterative Method	467
25.6 Notes and References	468
Problems	469
26 Automatic Error Analysis	471
26.1 Exploiting Direct Search Optimization	472
26.2 Direct Search Methods	474
26.3 Examples of Direct Search	477
26.3.1 Condition Estimation	477
26.3.2 Fast Matrix Inversion	478
26.3.3 Roots of a Cubic	479
26.4 Interval Analysis	481
26.5 Other Work	484
26.6 Notes and References	486
Problems	487

27 Software Issues in Floating Point Arithmetic	489
27.1 Exploiting IEEE Arithmetic	490
27.2 Subtleties of Floating Point Arithmetic	493
27.3 Cray Peculiarities	493
27.4 Compilers	494
27.5 Determining Properties of Floating Point Arithmetic	494
27.6 Testing a Floating Point Arithmetic	495
27.7 Portability	496
27.7.1 Arithmetic Parameters	496
27.7.2 2×2 Problems in LAPACK	497
27.7.3 Numerical Constants	498
27.7.4 Models of Floating Point Arithmetic	498
27.8 Avoiding Underflow and Overflow	499
27.9 Multiple Precision Arithmetic	501
27.10 Extended and Mixed Precision BLAS	503
27.11 Patriot Missile Software Problem	503
27.12 Notes and References	504
Problems	505
28 A Gallery of Test Matrices	511
28.1 The Hilbert and Cauchy Matrices	512
28.2 Random Matrices	515
28.3 “Randsvd” Matrices	517
28.4 The Pascal Matrix	518
28.5 Tridiagonal Toeplitz Matrices	521
28.6 Companion Matrices	522
28.7 Notes and References	523
28.7.1 LAPACK	525
Problems	525
A Solutions to Problems	527
B Acquiring Software	573
B.1 Internet	574
B.2 Netlib	574
B.3 MATLAB	575
B.4 NAG Library and NAGWare F95 Compiler	575
C Program Libraries	577
C.1 Basic Linear Algebra Subprograms	578
C.2 EISPACK	579
C.3 LINPACK	579
C.4 LAPACK	579
C.4.1 Structure of LAPACK	580
D The Matrix Computation Toolbox	583
Bibliography	587

CONTENTS	xv
Name Index	657
Subject Index	667

List of Figures

1.1	Backward and forward errors for $y = f(x)$	7
1.2	Mixed forward-backward error for $y = f(x)$	8
1.3	Forward errors $\ x - \hat{x}\ _\infty / \ x\ _\infty$ and relative residuals $\ b - A\hat{x}\ _\infty / (\ A\ _\infty \ \hat{x}\ _\infty)$ versus precision.	18
1.4	Absolute error versus precision, $t = -\log_2 u$, in evaluating (1.8). . .	19
1.5	Relative errors $\ A_k - \hat{A}_k\ _2 / \ A\ _2$ for Givens QR factorization. . .	23
1.6	Values of rational function $r(x)$ computed by Horner's rule.	26
2.1	Relative distance from x to the next larger machine number ($\beta = 2$, $t = 24$), displaying wobbling precision.	40
4.1	Recovering the rounding error.	84
4.2	Errors $ y(1) - \hat{y}_n $ for Euler's method with and without compensated summation.	87
5.1	Computed polynomial values and running and a priori bounds for Horner's method.	97
6.1	Plots of p versus $\ A\ _p$, for $1 \leq p \leq 15$	113
9.1	Illustration of rook pivoting.	159
9.2	Upper bounds for growth factors ρ_n for partial pivoting, rook pivoting, and complete pivoting.	169
9.3	Maximum growth factors and average number of comparisons for 15000 random matrices of dimension $n = 100:100:1500$	171
9.4	A banded matrix.	173
14.1	Residuals for inverses computed by MATLAB's <code>inv</code> function. . . .	262
15.1	Underestimation ratio for Algorithm 15.4 for 5×5 matrix $A(\theta)$. . .	295
17.1	Forward and backward errors for SOR iteration.	323
18.1	A typical hump for a convergent, nonnormal matrix.	341
18.2	Diverging powers of a nilpotent matrix, C_{14}	341
18.3	Infinity norms of powers of 2×2 matrix J in (18.2).	343
18.4	Computed powers of <code>chebspec</code> matrices.	349
18.5	Pseudospectra for <code>chebspec</code> matrices.	350
18.6	Pseudospectrum for SOR iteration matrix.	351

19.1	Householder matrix P times vector x	355
19.2	Givens rotation, $y = G(i, j, \theta)x$	365
23.1	Exponent versus time for matrix multiplication.	437
23.2	Errors for Strassen's method with two random matrices of dimension $n = 1024$	444
24.1	Error in FFT followed by inverse FFT.	454
25.1	Newton's method with $\bar{u} = u$ and $\bar{u} = u^2$	465
26.1	The possible steps in one iteration of the MDS method when $n = 2$	475
27.1	Rational function r	491
27.2	Error in evaluating rational function r	492
28.1	<code>spy(rem(pascal(32),2))</code>	522
28.2	Pseudospectra of <code>compan(poly(A))</code>	524
28.3	Pseudospectra of 32×32 pentadiagonal Toeplitz matrices.	526
A.1	$\log(1 + x)$ evaluated using MATLAB's <code>log</code> and using the formula (A.1).	529

List of Tables

1.1	Computed approximations $\hat{f}_n = fl((1 + 1/n)^n)$ to $e = 2.71828\dots$	15
1.2	Computed values of $(e^x - 1)/x$ from Algorithms 1 and 2.	21
1.3	Results from GE without pivoting on an upper Hessenberg matrix.	25
2.1	Floating point arithmetic parameters.	37
2.2	IEEE arithmetic exceptions and default results.	42
2.3	Test arithmetics.	51
2.4	Sine test.	52
2.5	Exponentiation test.	52
4.1	Mean square errors for nonnegative x_i	89
6.1	Constants α_{pq} such that $\ x\ _p \leq \alpha_{pq}\ x\ _q$, $x \in \mathbb{C}^n$	109
6.2	Constants α_{pq} such that $\ A\ _p \leq \alpha_{pq}\ A\ _q$, $A \in \mathbb{C}^{m \times n}$	109
7.1	Four classes of perturbations and the corresponding condition numbers.	125
7.2	Backward and forward stability.	130
9.1	Classes of matrices for which $\rho_n = O(1)$ for GE without pivoting. .	166
9.2	Times for solution of a linear system of order n	185
9.3	Records for largest dense linear systems solved.	191
12.1	$\omega_{ A , b }$ values for $A = \texttt{gallery}(\text{'orthog'}, 25)$	239
12.2	$\omega_{ A , b }$ values for $A = \texttt{gallery}(\text{'clement'}, 50)$	240
12.3	$\omega_{ A , b }$ values for $A = \texttt{gfpp}(50)$	240
13.1	Stability of block and point LU factorization.	256
14.1	Backward errors $\eta_{A,b}(\hat{x})$ for the ∞ -norm.	260
14.2	Mflop rates for inverting a triangular matrix on a Cray 2.	267
14.3	Mflop rates for inverting a full matrix on a Cray 2.	272
14.4	Times (minutes and seconds) for inverting an $n \times n$ matrix.	272
14.5	Additional timings for inverting an $n \times n$ matrix.	272
14.6	GJE for $Ux = b$	276
17.1	Dates of publication of selected iterative methods.	322
17.2	Results for Jacobi method, $a = 1/2 - 8^{-j}$	328
17.3	Results for Jacobi method, $a = -(1/2 - 8^{-j})$	329

19.1	Backward errors for QR factorization with no pivoting, row sorting, and column pivoting on matrix (19.16).	363
20.1	LS backward errors and residual for Vandermonde system.	394
21.1	Backward errors for underdetermined Vandermonde system.	413
22.1	Bounds and estimates for $\kappa(V_n)$	418
22.2	Parameters in the three-term recurrence (22.6).	422
22.3	Results for dual Chebyshev–Vandermonde-like system.	427
27.1	Results from Cholesky factorization.	493
27.2	Effect of extended run time on Patriot missile operation.	504
28.1	MATLAB functions for generating matrices discussed in this book.	513
28.2	Condition numbers of Hilbert and Pascal matrices.	514

Preface to Second Edition

*We dare not lengthen this book much more,
lest it be out of moderation and should
stir up men's apathy because of its size.*

— AELFRIC, schoolteacher of Cerne Abbas,
later Abbot of Eynsham (c. 995–1020)

In the nearly seven years since I finished writing the first edition of this book research on the accuracy and stability of numerical algorithms has continued to flourish and mature. Our understanding of algorithms has steadily improved, and in some areas new or improved algorithms have been derived.

Three developments during this period deserve particular note. First, the widespread adoption of electronic publication of journals and the increased practice of posting technical reports and preprints on the Web have both made research results more quickly available than before. Second, the inclusion of routines from state-of-the-art numerical software libraries such as LAPACK in packages such as MATLAB* and Maple† has brought the highest-quality algorithms to a very wide audience. Third, IEEE arithmetic is now ubiquitous—indeed, it is hard to find a computer whose arithmetic does not comply with the standard.

This new edition is a major revision of the book that brings it fully up to date, expands the coverage, and includes numerous improvements to the original material. The changes reflect my own experiences in using the book, as well as suggestions received from readers.

The changes to the book can be summarized as follows.

New Chapters

- *Symmetric Indefinite and Skew-Symmetric Systems* (Chapter 11). A greatly expanded treatment is given of symmetric indefinite systems (previously contained in the chapter *Cholesky Factorization*) and a new section treats skew-symmetric systems.
- *Nonlinear Systems and Newton's Method* (Chapter 25). Results on the limiting accuracy and limiting residual of Newton's method are given under general assumptions that permit the use of extended precision in calculating residuals. The conditioning of nonlinear systems, and termination criteria for iterative methods, are also investigated.

*MATLAB is a registered trademark of The MathWorks, Inc.

†Maple is a registered trademark of Waterloo Maple Software.

New Sections

- *Fused Multiply-Add Operation* (§2.6). The advantages of this operation, which is included in the Intel IA-64 architecture, are discussed, along with some subtle issues that it raises.
- *Elementary Functions* (§2.10). We explain why it is difficult to compute elementary functions in a way that satisfies all the natural requirements, and give pointers to relevant work.
- *Matrix Polynomials* (§5.4). How to evaluate three different matrix generalizations of a scalar polynomial is discussed.
- *More Error Bounds* (§9.7). Some additional backward and forward error bounds for Gaussian elimination (GE) without pivoting are given, leading to the new result that GE is row-wise backward stable for row diagonally dominant matrices.
- *Variants of Gaussian Elimination* (§9.9). Some lesser-known variants of GE with partial pivoting are described.
- *Rank-Revealing LU Factorizations* (§9.12). This section explains why LU factorization with an appropriate pivoting strategy leads to a factorization that is usually rank revealing.
- *Parallel Inversion Methods* (§14.5). Several methods for matrix inversion on parallel machines are described, including the Schulz iteration, which is of wider interest.
- *Block 1-Norm Estimator* (§15.4). An improved version of the LAPACK condition estimator, implemented in MATLAB's `condst` function, is outlined.
- *Pivoting and Row-Wise Stability* (§19.4). The behaviour of Householder QR factorization for matrices whose rows are poorly scaled is examined. The backward error result herein is the only one I know that requires a particular choice of sign when constructing Householder matrices.
- *Weighted Least Squares Problems* (§20.8). Building on §19.4, an overall row-wise backward error result is given for solution of the least squares problem by Householder QR factorization with column pivoting.
- *The Equality Constrained Least Squares Problem* (§20.9). This section treats the least squares problem subject to linear equality constraints. It gives a perturbation result and describes three classes of methods (the method of weighting, null space methods, and elimination methods) and their numerical stability.
- *Extended and Mixed Precision BLAS* (§27.10). A brief description is given of these important new aids to carrying out extended precision computations in a portable way.

Other Changes

In the error analysis of QR factorization in the first edition of the book, backward error bounds were given in normwise form and in a componentwise form that essentially provided columnwise bounds. I now give just columnwise bounds, as they are the natural result of the analysis and trivially imply both normwise and componentwise bounds. The basic lemma on construction of the Householder vector has been modified so that most of the ensuing results apply for either choice of sign in constructing the vector. These and other results are expressed using the error constant $\tilde{\gamma}_n$, which replaces the more clumsy notation γ_{cn} used in the first edition (see §3.4).

Rook pivoting is a pivoting strategy that is applicable to both GE for general matrices and block LDL^T factorization for symmetric indefinite matrices, and it is of pedagogical interest because it is intermediate between partial pivoting and complete pivoting in both cost and stability. Rook pivoting is described in detail and its merits for practical computation are explained. A thorough discussion is given of the choice of pivoting strategy for GE and of the effects on the method of scaling. Some new error bounds are included, as well as several other results that help to provide a comprehensive picture of current understanding of GE.

This new edition has a more thorough treatment of block LDL^T factorization for symmetric indefinite matrices, including recent error analysis, rook pivoting, and Bunch's pivoting strategy for tridiagonal matrices. Aasen's method and Bunch's block LDL^T factorization method for skew-symmetric matrices are also treated.

Strengthened error analysis includes results for Gauss–Jordan elimination (Theorem 14.5, Corollary 14.7), fast solution of Vandermonde systems (Corollary 22.7), the fast Fourier transform (FFT) (Theorem 24.2), and solution of circulant linear systems via the FFT (Theorem 24.3).

All the numerical experiments have been redone in the latest version, 6.1, of MATLAB. The figures have been regenerated and their design improved, where possible. Discussions of LAPACK reflect the current release, 3.0.

A major effort has gone into updating the bibliography, with the aim of referring to the most recent results and ensuring that the latest editions of books are referenced and papers are cited in their final published form. Over 190 works published since the first edition are cited. See page 587 for a histogram that shows the distribution of dates of publication of the works cited.

In revising the book I took the opportunity to rewrite and rearrange material, improve the index, and fine tune the typesetting (in particular, using ideas of Knuth [745, 1999, Chap. 33]). Several research problems from the first edition have been solved and are now incorporated into the text, and new research problems and general problems have been added.

In small ways the emphasis of the book has been changed. For example, when the first edition was written IEEE arithmetic was not so prevalent, so a number of results were stated with the proviso that a guard digit was present. Now it is implicitly assumed throughout that the arithmetic is “well behaved” and unfortunate consequences of lack of a guard digit are given less prominence.

A final change concerns the associated MATLAB toolbox. The Test Matrix Toolbox from the first edition is superseded by the new Matrix Computation Tool-

box, described in Appendix D. The new toolbox includes functions implementing a number of algorithms in the book—in particular, GE with rook pivoting and block LDL^T factorization for symmetric and skew-symmetric matrices. The toolbox should be of use for both teaching and research.

I am grateful to Bill Gragg, Beresford Parlett, Colin Percival, Siegfried Rump, Françoise Tisseur, Nick Trefethen, and Tjalling Ypma for comments that influenced the second edition.

It has been a pleasure working once again with the SIAM publication staff, in particular Linda Thiel, Sara Triller Murphy, Marianne Will, and my copy editor, Beth Gallagher.

Research leading to this book has been supported by grants from the Engineering and Physical Sciences Research Council and by a Royal Society Leverhulme Trust Senior Research Fellowship.

The tools used to prepare the book were the same as for the first edition, except that for T_EX-related tasks I used MikT_EX (<http://www.miktex.org/>), including its excellent YAP previewer.

Manchester
February 2002

Nicholas J. Higham

Preface to First Edition

It has been 30 years since the publication of Wilkinson's books *Rounding Errors in Algebraic Processes* [1232, 1963] and *The Algebraic Eigenvalue Problem* [1233, 1965]. These books provided the first thorough analysis of the effects of rounding errors on numerical algorithms, and they rapidly became highly influential classics in numerical analysis. Although a number of more recent books have included analysis of rounding errors, none has treated the subject in the same depth as Wilkinson.

This book gives a thorough, up-to-date treatment of the behaviour of numerical algorithms in finite precision arithmetic. It combines algorithmic derivations, perturbation theory, and rounding error analysis. Software practicalities are emphasized throughout, with particular reference to LAPACK. The best available error bounds, some of them new, are presented in a unified format with a minimum of jargon. Historical perspective is given to provide insight into the development of the subject, and further information is provided in the many quotations. Perturbation theory is treated in detail, because of its central role in revealing problem sensitivity and providing error bounds. The book is unique in that algorithmic derivations and motivation are given succinctly, and implementation details minimized, so that attention can be concentrated on accuracy and stability results. The book was designed to be a comprehensive reference and contains extensive citations to the research literature.

Although the book's main audience is specialists in numerical analysis, it will be of use to all computational scientists and engineers who are concerned about the accuracy of their results. Much of the book can be understood with only a basic grounding in numerical analysis and linear algebra.

This first two chapters are very general. Chapter 1 describes fundamental concepts of finite precision arithmetic, giving many examples for illustration and dispelling some misconceptions. Chapter 2 gives a thorough treatment of floating point arithmetic and may well be the single most useful chapter in the book. In addition to describing models of floating point arithmetic and the IEEE standard, it explains how to exploit "low-level" features not represented in the models and contains a large set of informative exercises.

In the rest of the book the focus is, inevitably, on numerical linear algebra, because it is in this area that rounding errors are most influential and have been most extensively studied. However, I found that it was impossible to cover the whole of numerical linear algebra in a single volume. The main omission is the area of eigenvalue and singular value computations, which is still the subject of intensive research and requires a book of its own to summarize algorithms, perturbation theory, and error analysis. This book is therefore certainly not a replacement

for *The Algebraic Eigenvalue Problem*.

Two reasons why rounding error analysis can be hard to understand are that, first, there is no standard notation and, second, error analyses are often cluttered with re-derivations of standard results. In this book I have used notation that I find nearly always to be the most convenient for error analysis: the key ingredient is the symbol $\gamma_n = nu/(1 - nu)$, explained in §3.1. I have also summarized many basic error analysis results (for example, in Chapters 3 and 8) and made use of them throughout the book. I like to think of these basic results as analogues of the Fortran BLAS (Basic Linear Algebra Subprograms): once available in a standard form they can be used as black boxes and need not be reinvented.

A number of the topics included here have not been treated in depth in previous numerical analysis textbooks. These include floating point summation, block LU factorization, condition number estimation, the Sylvester equation, powers of matrices, finite precision behaviour of stationary iterative methods, Vandermonde systems, and fast matrix multiplication, each of which has its own chapter. But there are also some notable omissions. I would have liked to include a chapter on Toeplitz systems, but this is an area in which stability and accuracy are incompletely understood and where knowledge of the underlying applications is required to guide the investigation. The important problems of updating and downdating matrix factorizations when the matrix undergoes a “small” change have also been omitted due to lack of time and space. A further omission is analysis of parallel algorithms for all the problems considered in the book (though blocked and partitioned algorithms and one particular parallel method for triangular systems are treated). Again, there are relatively few results and this is an area of active research.

Throughout the history of numerical linear algebra, theoretical advances have gone hand in hand with software development. This tradition has continued with LAPACK (1987–), a project to develop a state-of-the-art Fortran package for solving linear equations and eigenvalue problems. LAPACK has enjoyed a synergy with research that has led to a number of important breakthroughs in the design and analysis of algorithms, from the standpoints of both performance and accuracy. A key feature of this book is that it provides the material needed to understand the numerical properties of many of the algorithms in LAPACK, the exceptions being the routines for eigenvalue and singular value problems. In particular, the error bounds computed by the LAPACK linear equation solvers are explained, the LAPACK condition estimator is described in detail, and some of the software issues confronted by the LAPACK developers are highlighted. Chapter 27 examines the influence of floating point arithmetic on general numerical software, offering salutary stories, useful techniques, and brief descriptions of relevant codes.

This book has been written with numerical analysis courses in mind, although it is not designed specifically as a textbook. It would be a suitable reference for an advanced course (for example, for a graduate course on numerical linear algebra following the syllabus recommended by the ILAS Education Committee [661, 1993]), and could be used by instructors at all levels as a supplementary text from which to draw examples, historical perspective, statements of results, and exercises. The exercises (actually labelled “problems”) are an important part of the book, and many of them have not, to my knowledge, appeared in textbooks before.

Where appropriate I have indicated the source of an exercise; a name without a citation means that the exercise came from private communication or unpublished notes. Research problems given at the end of some sets of exercises emphasize that most of the areas covered are still active.

In addition to surveying and unifying existing results (including some that have not appeared in the mainstream literature) and sometimes improving upon their presentation or proof, this book contains new results. Some of particular note are as follows.

1. The error analysis in §5.3 for evaluation of the Newton interpolating polynomial.
2. The forward error analysis for iterative refinement in §12.1.
3. The error analysis of Gauss–Jordan elimination in §14.4.
4. The unified componentwise error analysis of QR factorization methods in Chapter 19, and the corresponding analysis of their use for solving the least squares problem in Chapter 20.
5. Theorem 21.4, which shows the backward stability of the QR factorization method for computing the minimum 2-norm solution to an underdetermined system.

The Notes and References are an integral part of each chapter. In addition to containing references, historical information, and further details, they include material not covered elsewhere in the chapter, and should always be consulted, in conjunction with the index, to obtain the complete picture.

I have included relatively few numerical examples except in the first chapter. There are two reasons. One is to reduce the length of the book. The second reason is because today it is so easy for the reader to perform experiments in MATLAB or some other interactive system. To this end I have made available the Test Matrix Toolbox, which contains MATLAB M-files for many of the algorithms and special matrices described in the book; see Appendix D.

This book has been designed to be as easy to use as possible. There are thorough name and subject indexes, page headings show chapter and section titles and numbers, and there is extensive cross-referencing. I have adopted the unusual policy of giving with (nearly) every citation not only its numerical location in the bibliography but also the names of the authors and the year of publication. This provides as much information as possible in a citation and reduces the need for the reader to turn to the bibliography.

A B^IB^TE^X database `acc-stab-num-alg.bib` containing all the references in the bibliography is available over the Internet from the bibnet project (which can be accessed via netlib, described in §B.2).

Special care has been taken to minimize the number of typographical and other errors, but no doubt some remain. I will be happy to receive notification of errors, as well as comments and suggestions for improvement.

Acknowledgements

Three books, in addition to Wilkinson's, have strongly influenced my research in numerical linear algebra and have provided inspiration for this book: Golub and Van Loan's *Matrix Computations* [509, 1996] (first edition 1983), Parlett's *The Symmetric Eigenvalue Problem* [926, 1998] (first published 1980) and Stewart's *Introduction to Matrix Computations* [1065, 1973]. Knuth's *The Art of Computer Programming* books (1973–) [743], [744], have also influenced my style and presentation.

Jim Demmel has contributed greatly to my understanding of the subject of this book and provided valuable technical help and suggestions. The first two chapters owe much to the work of Velvel Kahan; I am grateful to him for giving me access to unpublished notes and for suggesting improvements to early versions of Chapters 2 and 27. Des Higham read various drafts of the book, offering sound advice and finding improvements that had eluded me.

Other people who have given valuable help, suggestions, or advice are

Zhaojun Bai, Brad Baxter, Åke Björck, Martin Campbell-Kelly, Shivkumar Chandrasekaran, Alan Edelman, Warren Ferguson, Philip Gill, Gene Golub, George Hall, Sven Hammarling, Andrzej Kielbasiński, Philip Knight, Beresford Parlett, David Silvester, Michael Saunders, Ian Smith, Doron Swade, Nick Trefethen, Jack Williams, and Hongyuan Zha.

David Carlisle provided invaluable help and advice concerning $\text{\LaTeX} 2_{\epsilon}$.

Working with SIAM on the publication of this book was a pleasure. Special thanks go to Nancy Abbott (design), Susan Ciambrano (acquisition), Ed Ciburso (production), Beth Gallagher (copy editing), Corey Gray (production), Mary Rose Muccie (copy editing and indexing), Colleen Robishaw (design), and Sam Young (production).

Research leading to this book has been supported by grants from the Engineering and Physical Sciences Research Council, by a Nuffield Science Research Fellowship from the Nuffield Foundation, and by a NATO Collaborative Research Grant held with J. W. Demmel. I was fortunate to be able to make extensive use of the libraries of the University of Manchester, the University of Dundee, Stanford University, and the University of California, Berkeley.

This book was typeset in $\text{\LaTeX} 2_{\epsilon}$ using the `book` document style. The references were prepared in $\text{\BIBTeX}$ and the index with `MakeIndex`. It is difficult to imagine how I could have written the book without these wonderful tools. I used the “big” software from the `emTeX` distribution, running on a 486DX workstation. I used text editors The Semware Editor (Semware Corporation) and GNU Emacs (Free Software Foundation) and checked spelling with PC-Write (Quicksoft).

About the Dedication

This book is dedicated to the memory of two remarkable English mathematicians, James Hardy Wilkinson (1919–1986), FRS, and Alan Mathison Turing (1912–1954), FRS, both of whom made immense contributions to scientific computation.

Turing’s achievements include his paper “On Computable Numbers, with an Application to the Entscheidungsproblem”, which answered Hilbert’s decidability question using the abstract device now known as a Turing machine [1164, 1936]; his work at Bletchley Park during World War II on breaking the ciphers of the Enigma machine; his 1945 report proposing a design for the Automatic Computing Engine (ACE) at the National Physical Laboratory [1165, 1945]; his 1948 paper on LU factorization and its rounding error analysis [1166, 1948]; his consideration of fundamental questions in artificial intelligence (including his proposal of the “Turing test”); and, during the last part of his life, spent at the University of Manchester, his work on morphogenesis (the development of structure and form in an organism). Turing is remembered through the Turing Award of the Association for Computing Machinery (ACM), which has been awarded yearly since 1966 [3, 1987]. For more about Turing, read the superb biography by Hodges [631, 1983], described by a reviewer as “one of the finest pieces of scholarship to appear in the history of computing” [201, 1984]. Hodges maintains the Alan Turing Home Page at <http://www.turing.org.uk/turing/>

Wilkinson, like Turing a Cambridge-trained mathematician, was Turing’s assistant at the National Physical Laboratory. When Turing left, Wilkinson managed the group that built the Pilot ACE, contributing to the design and construction of the machine and its software. Subsequently, he used the machine to develop and study a variety of numerical methods. He developed backward error analysis in the 1950s and 1960s, publishing the books *Rounding Errors in Algebraic Processes* [1232, 1963][‡] (REAP) and *The Algebraic Eigenvalue Problem* [1233, 1965][§] (AEP), both of which rapidly achieved the status of classics. (AEP was reprinted in paperback in 1988 and, after being out of print for many years, REAP is now also available in paperback.) The AEP was described by the late Professor Leslie Fox as “almost certainly the most important and widely read title in numerical analysis”. Wilkinson also contributed greatly to the development of mathematical software. The volume *Handbook for Automatic Computation, Volume II: Linear Algebra* [1246, 1971], co-edited with Reinsch, contains high-quality, properly documented software and has strongly influenced subsequent software projects such as the NAG Library, EISPACK, LINPACK, and LAPACK.

Wilkinson received the 1970 Turing Award. In his Turing Award lecture he

[‡]REAP has been translated into Polish [1235, 1967] and German [1237, 1969].

[§]AEP has been translated into Russian [1238, 1970].

described life with Turing at the National Physical Laboratory in the 1940s [1240, 1971].

Wilkinson is remembered through SIAM's James H. Wilkinson Prize in Numerical Analysis and Scientific Computing, awarded every 4 years; the Wilkinson Prize for Numerical Software, awarded by Argonne National Laboratory, the National Physical Laboratory, and the Numerical Algorithms Group; and the Wilkinson Fellowship in Scientific Computing at Argonne National Laboratory. For more about Wilkinson see the biographical memoir by Fox [439, 1987], Fox's article [438, 1978], Parlett's essay [925, 1990], the prologue and epilogue of the proceedings [279, 1990] of a conference held in honour of Wilkinson at the National Physical Laboratory in 1987, and the tributes in [29, 1987]. Lists of Wilkinson's publications are given in [439, 1987] and in the special volume of the journal *Linear Algebra and Its Applications* (88/89, April 1987) published in his memory.

Chapter 1

Principles of Finite Precision Computation

Numerical precision is the very soul of science.

— SIR D'ARCY WENTWORTH THOMPSON, *On Growth and Form* (1942)

*There will always be a small but steady demand for error-analysts to . . .
expose bad algorithms' big errors and, more important,
supplant bad algorithms with provably good ones.*

— WILLIAM M. KAHAN, *Interval Arithmetic Options in the
Proposed IEEE Floating Point Arithmetic Standard* (1980)

*Since none of the numbers which we take out from logarithmic and
trigonometric tables admit of absolute precision,
but are all to a certain extent approximate only,
the results of all calculations performed
by the aid of these numbers can only be approximately true . . .*

*It may happen, that in special cases the
effect of the errors of the tables is so augmented that
we may be obliged to reject a method,
otherwise the best, and substitute another in its place.*

— CARL FRIEDRICH GAUSS¹, *Theoria Motus* (1809)

*Backward error analysis is no panacea;
it may explain errors but not excuse them.*

— HEWLETT-PACKARD, *HP-15C Advanced Functions Handbook* (1982)

¹Cited in Goldstine [500, 1977, p. 258].

This book is concerned with the effects of finite precision arithmetic on numerical algorithms², particularly those in numerical linear algebra. Central to any understanding of high-level algorithms is an appreciation of the basic concepts of finite precision arithmetic. This opening chapter briskly imparts the necessary background material. Various examples are used for illustration, some of them familiar (such as the quadratic equation) but several less well known. Common misconceptions and myths exposed during the chapter are highlighted towards the end, in §1.19.

This chapter has few prerequisites and few assumptions are made about the nature of the finite precision arithmetic (for example, the base, number of digits, or mode of rounding, or even whether it is floating point arithmetic). The second chapter deals in detail with the specifics of floating point arithmetic.

A word of warning: some of the examples from §1.12 onward are special ones chosen to illustrate particular phenomena. You may never see in practice the extremes of behaviour shown here. Let the examples show you what can happen, but do not let them destroy your confidence in finite precision arithmetic!

1.1. Notation and Background

We describe the notation used in the book and briefly set up definitions needed for this chapter.

Generally, we use

capital letters	A, B, C, Δ, Λ	for matrices,
subscripted lower case letters	$a_{ij}, b_{ij}, c_{ij}, \delta_{ij}, \lambda_{ij}$	for matrix elements,
lower case letters	x, y, z, c, g, h	for vectors,
lower case Greek letters	$\alpha, \beta, \gamma, \theta, \pi$	for scalars,

following the widely used convention originally introduced by Householder [644, 1964].

The vector space of all real $m \times n$ matrices is denoted by $\mathbb{R}^{m \times n}$ and the vector space of real n -vectors by $\mathbb{R}^n$. Similarly, $\mathbb{C}^{m \times n}$ denotes the vector space of complex $m \times n$ matrices. A superscript “ T ” denotes transpose and a superscript “ $*$ ” conjugate transpose.

Algorithms are expressed using a pseudocode based on the MATLAB language [576, 2000], [824]. Comments begin with the % symbol.

Submatrices are specified with the colon notation, as used in MATLAB and Fortran 90/95: $A(p:q, r:s)$ denotes the submatrix of A formed by the intersection of rows p to q and columns r to s . As a special case, a lone colon as the row or column specifier means to take all entries in that row or column; thus $A(:, j)$ is the j th column of A , and $A(i, :)$ the i th row. The values taken by an integer variable are also described using the colon notation: “ $i = 1:n$ ” means the same as “ $i = 1, 2, \dots, n$ ”.

Evaluation of an expression in floating point arithmetic is denoted $fl(\cdot)$, and we assume that the basic arithmetic operations $\text{op} = +, -, *, /$ satisfy

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u. \quad (1.1)$$

²For the purposes of this book an algorithm is a MATLAB program; cf. Smale [1046, 1990].

Here, u is the *unit roundoff* (or machine precision), which is typically of order 10^{-8} or 10^{-16} in single and double precision computer arithmetic, respectively, and between 10^{-10} and 10^{-12} on pocket calculators. For more on floating point arithmetic see Chapter 2.

Computed quantities (and, in this chapter only, arbitrary approximations) wear a hat. Thus $\hat{x}$ denotes the computed approximation to x .

Definitions are often (but not always) indicated by “:=” or “=:”, with the colon next to the object being defined.

We make use of the floor and ceiling functions: $\lfloor x \rfloor$ is the largest integer less than or equal to x , and $\lceil x \rceil$ is the smallest integer greater than or equal to x .

The normal distribution with mean μ and variance σ^2 is denoted by $N(\mu, \sigma^2)$.

We measure the cost of algorithms in flops. A *flop* is an elementary floating point operation: $+$, $-$, $/$, or $*$. We normally state only the highest-order terms of flop counts. Thus, when we say that an algorithm for $n \times n$ matrices requires $2n^3/3$ flops, we really mean $2n^3/3 + O(n^2)$ flops.

Other definitions and notation are introduced when needed.

Except where stated otherwise, all our numerical experiments were carried out in MATLAB 6.1 (R12.1) [824] on a Pentium III machine under Windows 98 or Windows ME, sometimes in conjunction with the Symbolic Math Toolbox [825]. Thus our computations were done in IEEE standard floating point arithmetic with unit roundoff $u = 2^{-53} \approx 1.1 \times 10^{-16}$. Sometimes, we simulate single precision arithmetic ($u \approx 6 \times 10^{-8}$) by rounding the result of every elementary operation to single precision (using the function `chop` from the Matrix Computation Toolbox—see Appendix D).

1.2. Relative Error and Significant Digits

Let $\hat{x}$ be an approximation to a real number x . The most useful measures of the accuracy of $\hat{x}$ are its *absolute error*

$$E_{\text{abs}}(\hat{x}) = |x - \hat{x}|,$$

and its *relative error*

$$E_{\text{rel}}(\hat{x}) = \frac{|x - \hat{x}|}{|x|}$$

(which is undefined if $x = 0$). An equivalent definition of relative error is $E_{\text{rel}}(\hat{x}) = |\rho|$, where $\hat{x} = x(1 + \rho)$. Some authors omit the absolute values from these definitions. When the sign is important we will simply talk about “the error $x - \hat{x}$ ”.

In scientific computation, where answers to problems can vary enormously in magnitude, it is usually the relative error that is of interest, because it is scale independent: scaling $x \rightarrow \alpha x$ and $\hat{x} \rightarrow \alpha \hat{x}$ leaves $E_{\text{rel}}(\hat{x})$ unchanged.

Relative error is connected with the notion of correct significant digits (or correct significant figures). The significant digits in a number are the first nonzero digit and all succeeding digits. Thus 1.7320 has five significant digits, while 0.0491 has only three. What is meant by *correct* significant digits in a number that approximates another seems intuitively clear, but a precise definition is problematic, as we explain in a moment. First, note that for a number $\hat{x}$ with p significant

digits there are only $p + 1$ possible answers to the question, “How many correct significant digits does $\hat{x}$ have?” (assuming $\hat{x}$ is not a constant such as 2.0 that is known exactly). Therefore the number of correct significant digits is a fairly crude measure of accuracy in comparison with the relative error. For example, in the following two cases $\hat{x}$ agrees with x to three but not four significant digits by any reasonable definition, yet the relative errors differ by a factor of about 44:

$$\begin{aligned} x &= 1.00000, & \hat{x} &= 1.00499, & E_{\text{rel}}(\hat{x}) &= 4.99 \times 10^{-3}, \\ x &= 9.00000, & \hat{x} &= 8.99899, & E_{\text{rel}}(\hat{x}) &= 1.12 \times 10^{-4}. \end{aligned}$$

Here is a possible definition of correct significant digits: *an approximation $\hat{x}$ to x has p correct significant digits if $\hat{x}$ and x round to the same number to p significant digits. Rounding is the act of replacing a given number by the nearest p significant digit number, with some rule for breaking ties when there are two nearest. This definition of correct significant digits is mathematically elegant and agrees with intuition most of the time. But consider the numbers*

$$x = 0.9949, \quad \hat{x} = 0.9951.$$

According to the definition $\hat{x}$ does not have two correct significant digits ($x \rightarrow 0.99$, $\hat{x} \rightarrow 1.0$), but does have one and three correct significant digits! A definition of correct significant digits that does not suffer from the latter anomaly states that *$\hat{x}$ agrees with x to p significant digits if $|x - \hat{x}|$ is less than half a unit in the p th significant digit of x* . However, this definition implies that 0.123 and 0.127 agree to two significant digits, whereas many people would say that they agree to less than two significant digits.

In summary, while the number of correct significant digits provides a useful way in which to think about the accuracy of an approximation, the relative error is a more precise measure (and is base independent). Whenever we give an approximate answer to a problem we should aim to state an estimate or bound for the relative error.

When x and $\hat{x}$ are vectors the relative error is most often defined with a norm, as $\|x - \hat{x}\|/\|x\|$. For the commonly used norms $\|x\|_{\infty} := \max_i |x_i|$, $\|x\|_1 := \sum_i |x_i|$, and $\|x\|_2 := (x^T x)^{1/2}$, the inequality $\|x - \hat{x}\|/\|x\| \leq \frac{1}{2} \times 10^{-p}$ implies that components $\hat{x}_i$ with $|\hat{x}_i| \approx \|x\|$ have about p correct significant decimal digits, but for the smaller components the inequality merely bounds the absolute error.

A relative error that puts the individual relative errors on an equal footing is the *componentwise relative error*

$$\max_i \frac{|x_i - \hat{x}_i|}{|x_i|},$$

which is widely used in error analysis and perturbation analysis (see Chapter 7, for example).

As an interesting aside we mention the “tablemaker’s dilemma”. Suppose you are tabulating the values of a transcendental function such as the sine function and a particular entry is evaluated as 0.124|500000000 correct to a few digits in the last place shown, where the vertical bar follows the final significant digit to be tabulated. Should the final significant digit be 4 or 5? The answer depends

on whether there is a nonzero trailing digit, and there is no simple bound on how many digits we have to compute in order to answer the question.

1.3. Sources of Errors

There are three main sources of errors in numerical computation: rounding, data uncertainty, and truncation.

Rounding errors, which are an unavoidable consequence of working in finite precision arithmetic, are largely what this book is about. The remainder of this chapter gives basic insight into rounding errors and their effects.

Uncertainty in the data is always a possibility when we are solving practical problems. It may arise in several ways:

- from errors of measurement or estimation (possibly large: data in engineering and economics [835, 1999], for example, is usually accurate to only a few digits),
- from errors in storing the data on the computer (rounding errors—tiny),
- from the result of errors (big or small) in an earlier computation if the data is itself the solution to another problem.

The effects of errors in the data are generally easier to understand than the effects of rounding errors committed during a computation, because data errors can be analysed using perturbation theory for the problem at hand, while intermediate rounding errors require an analysis specific to the given method. This book contains perturbation theory for most of the problems considered, for example, in Chapters 7 (linear systems), 20 (the least squares problem), 21 (underdetermined systems), and 25 (nonlinear systems).

Analysing truncation errors, or discretization errors, is one of the major tasks of the numerical analyst. Many standard numerical methods (for example, the trapezium rule for quadrature, Euler's method for differential equations, and Newton's method for nonlinear equations) can be derived by taking finitely many terms of a Taylor series. The terms omitted constitute the truncation error, and for many methods the size of this error depends on a parameter (often called h , "the step-size") whose appropriate value is a compromise between obtaining a small error and a fast computation.

Because the emphasis of this book is on finite precision computation, with virtually no mention of truncation errors, it would be easy for the reader to gain the impression that the study of numerical methods is dominated by the study of rounding errors. This is certainly not the case. Trefethen explains it well when he discusses how to define numerical analysis [1153, 1992]:

Rounding errors and instability are important, and numerical analysts will always be the experts in these subjects and at pains to ensure that the unwary are not tripped up by them. But our central mission is to compute quantities that are typically uncomputable, from an analytic point of view, and to do it with lightning speed.

In this quotation “uncomputable” means that approximations are necessary, and thus Trefethen’s point is that developing good approximations is a more fundamental task than analysing the effects of rounding errors on those approximations.

A possible way to avoid rounding and truncation errors (but not data errors) is to try to solve a problem using a symbolic manipulation package, such as Maple [815] (perhaps via MATLAB’s Symbolic Math Toolbox [825]) or Mathematica³ [818], [1253, 1999]. Indeed, we have used this approach to compute “exact answers” in some of our numerical experiments. While we acknowledge the value of symbolic manipulation as part of the toolkit of the scientific problem solver, we do not study it in this book.

1.4. Precision Versus Accuracy

The terms accuracy and precision are often confused or used interchangeably, but it is worth making a distinction between them. *Accuracy* refers to the absolute or relative error of an approximate quantity. *Precision* is the accuracy with which the basic arithmetic operations $+$, $-$, $*$, $/$ are performed, and for floating point arithmetic is measured by the unit roundoff u (see (1.1)). Accuracy and precision are the same for the scalar computation $c = a * b$, but accuracy can be much worse than precision in the solution of a linear system of equations, for example.

It is important to realize that *accuracy is not limited by precision*, at least in theory. This may seem surprising, and may even appear to contradict many of the results in this book. However, arithmetic of a given precision can be used to simulate arithmetic of arbitrarily high precision, as explained in §27.9. (The catch is that such simulation is too expensive to be of practical use for routine computation.) In all our error analyses there is an implicit assumption that the given arithmetic is not being used to simulate arithmetic of a higher precision.

1.5. Backward and Forward Errors

Suppose that an approximation $\hat{y}$ to $y = f(x)$ is computed in an arithmetic of precision u , where f is a real scalar function of a real scalar variable. How should we measure the “quality” of $\hat{y}$?

In most computations we would be happy with a tiny relative error, $E_{\text{rel}}(\hat{y}) \approx u$, but this cannot always be achieved. Instead of focusing on the relative error of $\hat{y}$ we can ask, “For what set of data have we actually solved our problem?”, that is, for what Δx do we have $\hat{y} = f(x + \Delta x)$? In general, there may be many such Δx , so we should ask for the smallest one. The value of $|\Delta x|$ (or $\min |\Delta x|$), possibly divided by $|x|$, is called the *backward error*. The absolute and relative errors of $\hat{y}$ are called *forward errors*, to distinguish them from the backward error. Figure 1.1 illustrates these concepts.

The process of bounding the backward error of a computed solution is called *backward error analysis*, and its motivation is twofold. First, it interprets rounding errors as being equivalent to perturbations in the data. The data frequently

³Mathematica is a registered trademark of Wolfram Research Inc.

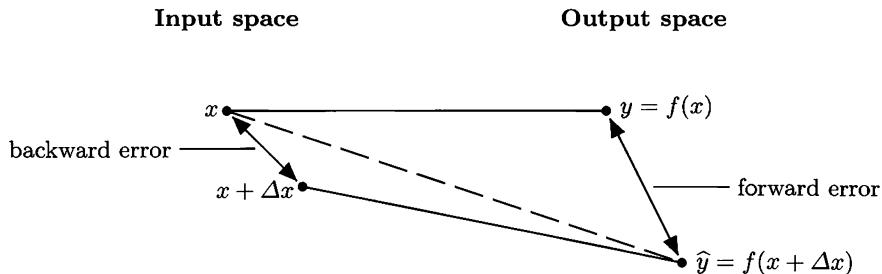


Figure 1.1. Backward and forward errors for $y = f(x)$. Solid line = exact; dotted line = computed.

contains uncertainties due to measurements, previous computations, or errors committed in storing numbers on the computer, as explained in §1.3. If the backward error is no larger than these uncertainties then the computed solution can hardly be criticized—it may be the solution we are seeking, for all we know. The second attraction of backward error analysis is that it reduces the question of bounding or estimating the forward error to perturbation theory, which for many problems is well understood (and only has to be developed once, for the given problem, and not for each method). We discuss perturbation theory in the next section.

A method for computing $y = f(x)$ is called *backward stable* if, for any x , it produces a computed $\hat{y}$ with a small backward error, that is, $\hat{y} = f(x + \Delta x)$ for some small Δx . The definition of “small” will be context dependent. In general, a given problem has several methods of solution, some of which are backward stable and some not.

As an example, assumption (1.1) says that the computed result of the operation $x \pm y$ is the exact result for perturbed data $x(1 + \delta)$ and $y(1 + \delta)$ with $|\delta| \leq u$; thus addition and subtraction are, by assumption, backward stable operations.

Most routines for computing the cosine function do not satisfy $\hat{y} = \cos(x + \Delta x)$ with a relatively small Δx , but only the weaker relation $\hat{y} + \Delta y = \cos(x + \Delta x)$, with relatively small Δy and Δx . A result of the form

$$\hat{y} + \Delta y = f(x + \Delta x), \quad |\Delta y| \leq \epsilon |y|, \quad |\Delta x| \leq \eta |x| \quad (1.2)$$

is known as a *mixed forward-backward error* result and is illustrated in Figure 1.2. Provided that ϵ and η are sufficiently small, (1.2) says that *the computed value $\hat{y}$ scarcely differs from the value $\hat{y} + \Delta y$ that would have been produced by an input $x + \Delta x$ scarcely different from the actual input x* . Even more simply, $\hat{y}$ is almost the right answer for almost the right data.

In general, an algorithm is called *numerically stable* if it is stable in the mixed forward-backward error sense of (1.2) (hence a backward stable algorithm can certainly be called numerically stable). Note that this definition is specific to problems where rounding errors are the dominant form of errors. The term *stability* has different meanings in other areas of numerical analysis.

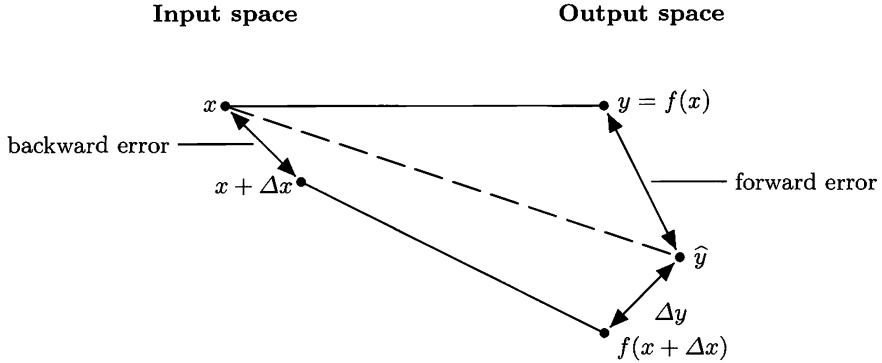


Figure 1.2. *Mixed forward-backward error for $y = f(x)$. Solid line = exact; dotted line = computed.*

1.6. Conditioning

The relationship between forward and backward error for a problem is governed by the conditioning of the problem, that is, the sensitivity of the solution to perturbations in the data. Continuing the $y = f(x)$ example of the previous section, let an approximate solution $\hat{y}$ satisfy $\hat{y} = f(x + \Delta x)$. Then, assuming for simplicity that f is twice continuously differentiable,

$$\hat{y} - y = f(x + \Delta x) - f(x) = f'(x)\Delta x + \frac{f''(x + \theta\Delta x)}{2!}(\Delta x)^2, \quad \theta \in (0, 1),$$

and we can bound or estimate the right-hand side. This expansion leads to the notion of condition number. Since

$$\frac{\hat{y} - y}{y} = \left(\frac{xf'(x)}{f(x)} \right) \frac{\Delta x}{x} + O((\Delta x)^2),$$

the quantity

$$c(x) = \left| \frac{xf'(x)}{f(x)} \right|$$

measures, for small Δx , the relative change in the output for a given relative change in the input, and it is called the (*relative*) *condition number* of f . If x or f is a vector then the condition number is defined in a similar way using norms, and it measures the *maximum* relative change, which is attained for some, but not all, vectors Δx .

As an example, consider the function $f(x) = \log x$. The condition number is $c(x) = |1/\log x|$, which is large for $x \approx 1$. This means that a small relative change in x can produce a much larger relative change in $\log x$ for $x \approx 1$. The reason is that a small relative change in x produces a small *absolute* change in $f(x) = \log x$ (since $f(x + \Delta x) \approx f(x) + f'(x)\Delta x = f(x) + \Delta x/x$), and that change in $\log x$ may be large in a relative sense.

When backward error, forward error, and the condition number are defined in a consistent fashion we have the useful rule of thumb that

$$\text{forward error} \lesssim \text{condition number} \times \text{backward error},$$

with approximate equality possible. One way to interpret this rule of thumb is to say that *the computed solution to an ill-conditioned problem can have a large forward error*. For even if the computed solution has a small backward error, this error can be amplified by a factor as large as the condition number when passing to the forward error.

One further definition is useful. If a method produces answers with forward errors of similar magnitude to those produced by a backward stable method, then it is called *forward stable*. Such a method need not be backward stable itself. Backward stability implies forward stability, but not vice versa. An example of a method that is forward stable but not backward stable is Cramer's rule for solving a 2×2 linear system, which is discussed in §1.10.1.

1.7. Cancellation

Cancellation is what happens when two nearly equal numbers are subtracted. It is often, but not always, a bad thing. Consider the function $f(x) = (1 - \cos x)/x^2$. With $x = 1.2 \times 10^{-5}$ the value of $\cos x$ rounded to 10 significant figures is

$$c = 0.9999\ 9999\ 99,$$

so that

$$1 - c = 0.0000\ 0000\ 01.$$

Then $(1 - c)/x^2 = 10^{-10}/1.44 \times 10^{-10} = 0.6944\dots$, which is clearly wrong given the fact that $0 \leq f(x) < 1/2$ for all $x \neq 0$. A 10-significant-figure approximation to $\cos x$ is therefore not sufficient to yield a value of $f(x)$ with even one correct figure. The problem is that $1 - c$ has only 1 significant figure. The subtraction $1 - c$ is *exact*, but this subtraction produces a result of the same size as the error in c . In other words, the subtraction elevates the importance of the earlier error. In this particular example it is easy to rewrite $f(x)$ to avoid the cancellation. Since $\cos x = 1 - 2\sin^2(x/2)$,

$$f(x) = \frac{1}{2} \left(\frac{\sin(x/2)}{x/2} \right)^2.$$

Evaluating this second formula for $f(x)$ with a 10-significant-figure approximation to $\sin(x/2)$ yields $f(x) = 0.5$, which is correct to 10 significant figures.

To gain more insight into the cancellation phenomenon consider the subtraction (in exact arithmetic) $\hat{x} = \hat{a} - \hat{b}$, where $\hat{a} = a(1 + \Delta a)$ and $\hat{b} = b(1 + \Delta b)$. The terms Δa and Δb are relative errors or uncertainties in the data, perhaps attributable to previous computations. With $x = a - b$ we have

$$\left| \frac{x - \hat{x}}{x} \right| = \left| \frac{-a\Delta a + b\Delta b}{a - b} \right| \leq \max(|\Delta a|, |\Delta b|) \frac{|a| + |b|}{|a - b|}.$$

The relative error bound for $\hat{x}$ is large when $|a - b| \ll |a| + |b|$, that is, when there is heavy cancellation in the subtraction. This analysis shows that subtractive cancellation causes relative errors or uncertainties already present in $\hat{a}$ and $\hat{b}$ to be magnified. In other words, subtractive cancellation *brings earlier errors into prominence*.

It is important to realize that cancellation is not *always* a bad thing. There are several reasons. First, the numbers being subtracted may be error free, as when they are from initial data that is known exactly. The computation of divided differences, for example, involves many subtractions, but half of them involve the initial data and are harmless for suitable orderings of the points (see §5.3 and §22.3). The second reason is that cancellation may be a symptom of intrinsic ill conditioning of a problem, and may therefore be unavoidable. Third, the effect of cancellation depends on the role that the result plays in the remaining computation. For example, if $x \gg y \approx z > 0$ then the cancellation in the evaluation of $x + (y - z)$ is harmless.

1.8. Solving a Quadratic Equation

Mathematically, the problem of solving the (real) quadratic equation $ax^2 + bx + c = 0$ is trivial: there are two roots (if $a \neq 0$), given by

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}. \quad (1.3)$$

Numerically, the problem is more challenging, as neither the successful evaluation of (1.3) nor the accuracy of the computed roots can be taken for granted.

The easiest issue to deal with is the choice of formula for computing the roots. If $b^2 \gg |4ac|$ then $\sqrt{b^2 - 4ac} \approx |b|$, and so for one choice of sign the formula (1.3) suffers massive cancellation. This is damaging cancellation because one of the arguments, $fl(\sqrt{b^2 - 4ac})$, is inexact, so the subtraction brings into prominence the earlier rounding errors. How to avoid the cancellation is well known: obtain the larger root (in absolute value), x_1 , from

$$x_1 = \frac{-(b + \text{sign}(b)\sqrt{b^2 - 4ac})}{2a},$$

and the other from the equation $x_1 x_2 = c/a$.

Unfortunately, there is a more pernicious source of cancellation: the subtraction $b^2 - 4ac$. Accuracy is lost here when $b^2 \approx 4ac$ (the case of nearly equal roots), and no algebraic rearrangement can avoid the cancellation. The only way to guarantee accurate computed roots is to use extended precision (or some trick tantamount to the use of extended precision) in the evaluation of $b^2 - 4ac$.

Another potential difficulty is underflow and overflow. If we apply the formula (1.3) in IEEE single precision arithmetic (described in §2.3) to the equation $10^{20}x^2 - 3 \cdot 10^{20}x + 2 \cdot 10^{20} = 0$ then overflow occurs, since the maximum floating point number is of order 10^{38} ; the roots, however, are innocuous: $x = 1$ and $x = 2$. Dividing through the equation by $\max(|a|, |b|, |c|) = 3 \cdot 10^{20}$ cures the problem, but this strategy is ineffective for the equation $10^{-20}x^2 - 3x + 2 \cdot 10^{20} = 0$, whose roots

are 10^{20} and $2 \cdot 10^{20}$. In the latter equation we need to scale the variable: defining $x = 10^{20}y$ gives $10^{20}y^2 - 3 \cdot 10^{20}y + 2 \cdot 10^{20} = 0$, which is the first equation we considered. These ideas can be built into a general scaling strategy (see the Notes and References), but the details are nontrivial.

As this discussion indicates, not only is it difficult to devise an accurate and robust algorithm for solving a quadratic equation, but it is a nontrivial task to prepare specifications that define precisely what “accurate” and “robust” mean for a given system of floating point arithmetic.

1.9. Computing the Sample Variance

In statistics the sample variance of n numbers $x_1, \dots, x_n$ is defined as

$$s_n^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad (1.4)$$

where the sample mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Computing s_n^2 from this formula requires two passes through the data, one to compute $\bar{x}$ and the other to accumulate the sum of squares. A two-pass computation is undesirable for large data sets or when the sample variance is to be computed as the data is generated. An alternative formula, found in many statistics textbooks, uses about the same number of operations but requires only one pass through the data:

$$s_n^2 = \frac{1}{n-1} \left(\sum_{i=1}^n x_i^2 - \frac{1}{n} \left(\sum_{i=1}^n x_i \right)^2 \right). \quad (1.5)$$

This formula is very poor in the presence of rounding errors because it computes the sample variance as the difference of two positive numbers, and therefore can suffer severe cancellation that leaves the computed answer dominated by roundoff. In fact, the computed answer can be negative, an event aptly described by Chan, Golub, and LeVeque [214, 1983] as “a blessing in disguise since this at least alerts the programmer that disastrous cancellation has occurred”. In contrast, the original formula (1.4) always yields a very accurate (and nonnegative) answer, unless n is large (see Problem 1.10). Surprisingly, current calculators from more than one manufacturer (but not Hewlett-Packard) appear to use the one-pass formula, and they list it in their manuals.

As an example, if $x = [10000, 10001, 10002]^T$ then, in single precision arithmetic ($u \approx 6 \times 10^{-8}$), the sample variance is computed as 1.0 by the two-pass formula (relative error 0) but 0.0 by the one-pass formula (relative error 1). It might be argued that this data should be shifted by some estimate of the mean before applying the one-pass formula ($x_i \rightarrow x_i - d$, $i = 1:n$, which does not change s_n^2), but a good estimate is not always available and there are alternative one-pass algorithms that will always produce an acceptably accurate answer. For example,

instead of accumulating $\sum_i x_i$ and $\sum_i x_i^2$ we can accumulate

$$M_k := \frac{1}{k} \sum_{i=1}^k x_i \quad \text{and} \quad Q_k := \sum_{i=1}^k (x_i - M_k)^2 = \sum_{i=1}^k x_i^2 - \frac{1}{k} \left(\sum_{i=1}^k x_i \right)^2,$$

which can be done via the updating formulae

$$M_1 = x_1, \quad M_k = M_{k-1} + \frac{x_k - M_{k-1}}{k}, \quad k = 2:n, \quad (1.6a)$$

$$Q_1 = 0, \quad Q_k = Q_{k-1} + \frac{(k-1)(x_k - M_{k-1})^2}{k}, \quad k = 2:n, \quad (1.6b)$$

after which $s_n^2 = Q_n/(n-1)$. Note that the only subtractions in these recurrences are relatively harmless ones that involve the data x_i . For the numerical example above, (1.6) produces the exact answer. The updating formulae (1.6) are numerically stable, though their error bound is not as small as the one for the two-pass formula (it is proportional to the condition number κ_N in Problem 1.7).

The problem of computing the sample variance illustrates well how mathematically equivalent formulae can have different numerical stability properties.

1.10. Solving Linear Equations

For an approximate solution y to a linear system $Ax = b$ ($A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$) the forward error is defined as $\|x - y\|/\|x\|$, for some appropriate norm. Another measure of the quality of y , more or less important depending on the circumstances, is the size of the residual $r = b - Ay$. When the linear system comes from an interpolation problem, for example, we are probably more interested in how closely Ay represents b than in the accuracy of y . The residual is scale dependent: multiply A and b by α , and r is multiplied by α . One way to obtain a scale-independent quantity is to divide by $\|A\| \|y\|$, yielding the *relative residual*

$$\rho(y) := \frac{\|b - Ay\|}{\|A\| \|y\|}.$$

The importance of the relative residual is explained by the following result, which was probably first proved by Wilkinson (see the Notes and References). We use the 2-norm, defined by $\|x\|_2 = (x^T x)^{1/2}$ and $\|A\|_2 = \max_{x \neq 0} \|Ax\|_2 / \|x\|_2$.

Lemma 1.1. *With the notation above, and for the 2-norm,*

$$\rho(y) = \min \left\{ \frac{\|\Delta A\|_2}{\|A\|_2} : (A + \Delta A)y = b \right\}.$$

Proof. If $(A + \Delta A)y = b$ then $r := b - Ay = \Delta Ay$, so $\|r\|_2 \leq \|\Delta A\|_2 \|y\|_2$, giving

$$\frac{\|\Delta A\|_2}{\|A\|_2} \geq \frac{\|r\|_2}{\|A\|_2 \|y\|_2} = \rho(y). \quad (1.7)$$

On the other hand, $(A + \Delta A)y = b$ for $\Delta A = ry^T / (y^T y)$ and $\|\Delta A\|_2 = \|r\|_2 / \|y\|_2$, so the bound (1.7) is attainable. $\square$

Lemma 1.1 says that $\rho(y)$ measures how much A (but not b) must be perturbed in order for y to be the exact solution to the perturbed system, that is, $\rho(y)$ equals a normwise relative backward error. If the data A and b are uncertain and $\rho(y)$ is no larger than this uncertainty (e.g., $\rho(y) = O(u)$) then the approximate solution y must be regarded as very satisfactory. For other problems the backward error may not be as easy to compute as it is for a general linear system, as we will see for the Sylvester equation (§16.2), the least squares problem (§20.7), and the problem of minimum norm solution of an underdetermined system (§21.2).

To illustrate these concepts we consider two specific linear equation solvers: Gaussian elimination with partial pivoting (GEPP) and Cramer's rule.

1.10.1. GEPP Versus Cramer's Rule

Cramer's rule says that the components of the solution to a linear system $Ax = b$ are given by $x_i = \det(A_i(b))/\det(A)$, where $A_i(b)$ denotes A with its i th column replaced by b . These formulae are a prime example of a method that is mathematically elegant, but useless for solving practical problems. The two flaws in Cramer's rule are its computational expense and its numerical instability. The computational expense needs little comment and is, fortunately, explained in most modern linear algebra textbooks (for example, Strang [1092, 1993] cautions the student that "it would be crazy to solve equations that way"). The numerical instability is less well known, but not surprising. It is present even for $n = 2$, as a numerical example shows.

We formed a 2×2 system $Ax = b$ with condition number $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 \approx 10^{13}$, and solved the system by both Cramer's rule and GEPP in MATLAB (unit roundoff $u \approx 1.1 \times 10^{-16}$). The results were as follows, where $r = b - A\hat{x}$:

Cramer's rule		GEPP	
$\hat{x}$	$r/(\ A\ _2 \ \hat{x}\ _2)$	$\hat{x}$	$r/(\ A\ _2 \ \hat{x}\ _2)$
1.0000	1.5075×10^{-7}	1.0002	-4.5689×10^{-17}
2.0001	1.9285×10^{-7}	2.0004	-2.1931×10^{-17}

The scaled residual for GEPP is pleasantly small—of order the unit roundoff. That for Cramer's rule is 10 orders of magnitude larger, showing that the computed solution $\hat{x}$ from Cramer's rule does not closely satisfy the equations, or, equivalently, does not solve a nearby system. The solutions themselves are similar, both being accurate to three significant figures in each component but incorrect in the fourth significant figure. This is the accuracy we would expect from GEPP because of the rule of thumb "forward error $\lesssim$ condition number $\times$ backward error". That Cramer's rule is as accurate as GEPP in this example, despite its large residual, is perhaps surprising, but it is explained by the fact that Cramer's rule is forward stable for $n = 2$; see Problem 1.9. For general n , the accuracy and stability of Cramer's rule depend on the method used to evaluate the determinants, and satisfactory bounds are not known even for the case where the determinants are evaluated by GEPP.

The small residual produced by GEPP in this example is typical: error analysis shows that GEPP is guaranteed to produce a relative residual of order u when $n = 2$ (see §9.3). To see how remarkable a property this is, consider the rounded

version of the exact solution: $z = fl(x) = x + \Delta x$, where $\|\Delta x\|_2 \leq u\|x\|_2$. The residual of z satisfies $\|b - Az\|_2 = \|-A\Delta x\|_2 \leq u\|A\|_2\|x\|_2 \approx u\|A\|_2\|z\|_2$. Thus *the computed solution from GEPP has about as small a residual as the rounded exact solution, irrespective of its accuracy.*

Expressed another way, the errors in GEPP are highly correlated so as to produce a small residual. To emphasize this point, the vector $[1.0006, 2.0012]$, which agrees with the exact solution of the above problem to five significant figures (and therefore is more accurate than the solution produced by GEPP), has a relative residual $\|r\|_2/(\|A\|_2\|\hat{x}\|_2)$ of order 10^{-6} .

1.11. Accumulation of Rounding Errors

Since the first electronic computers were developed in the 1940s, comments along the following lines have often been made: “The enormous speed of current machines means that in a typical problem many millions of floating point operations are performed. This in turn means that rounding errors can potentially accumulate in a disastrous way.” This sentiment is true, but misleading. Most often, instability is caused not by the accumulation of millions of rounding errors, but by the insidious growth of just a few rounding errors.

As an example, let us approximate $e = \exp(1)$ by taking finite n in the definition $e := \lim_{n \rightarrow \infty} (1 + 1/n)^n$. Table 1.1 gives results computed in single precision ($u \approx 6 \times 10^{-8}$).

The approximations are poor, degrading as n approaches the reciprocal of the machine precision. For n a power of 10, $1/n$ has a nonterminating binary expansion. When $1 + 1/n$ is formed for n a large power of 10, only a few significant digits from $1/n$ are retained in the sum. The subsequent exponentiation to the power n , even if done exactly, must produce an inaccurate approximation to e (indeed, doing the exponentiation in double precision does not change any of the numbers shown in Table 1.1). Therefore a single rounding error is responsible for the poor results in Table 1.1.

There is a way to compute $(1 + 1/n)^n$ more accurately, using only single precision arithmetic; it is the subject of Problem 1.5.

Strassen’s method for fast matrix multiplication provides another example of the unpredictable relation between the number of arithmetic operations and the error. If we evaluate $fl(AB)$ by Strassen’s method, for $n \times n$ matrices A and B , and we look at the error as a function of the recursion threshold $n_0 \leq n$, we find that while the number of operations decreases as n_0 decreases from n to 8, the error typically *increases*; see §23.2.2.

1.12. Instability Without Cancellation

It is tempting to assume that calculations free from subtractive cancellation must be accurate and stable, especially if they involve only a small number of operations. The three examples in this section show the fallacy of this assumption.

Table 1.1. *Computed approximations $\hat{f}_n = fl((1 + 1/n)^n)$ to $e = 2.71828\dots$*

n	$\hat{f}_n$	$ e - \hat{f}_n $
10^1	2.593743	1.25×10^{-1}
10^2	2.704811	1.35×10^{-2}
10^3	2.717051	1.23×10^{-3}
10^4	2.718597	3.15×10^{-4}
10^5	2.721962	3.68×10^{-3}
10^6	2.595227	1.23×10^{-1}
10^7	3.293968	5.76×10^{-1}

1.12.1. The Need for Pivoting

Suppose we wish to compute an LU factorization

$$A = \begin{bmatrix} \epsilon & -1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l_{21} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix}, \quad 0 < \epsilon \ll 1.$$

Clearly, $u_{11} = \epsilon$, $u_{12} = -1$, $l_{21} = \epsilon^{-1}$, and $u_{22} = 1 - l_{21}u_{12} = 1 + \epsilon^{-1}$. In floating point arithmetic, if ϵ is sufficiently small then $\hat{u}_{22} = fl(1 + \epsilon^{-1})$ evaluates to ϵ^{-1} . Assuming l_{21} is computed exactly, we then have

$$A - \hat{L}\hat{U} = \begin{bmatrix} \epsilon & -1 \\ 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ \epsilon^{-1} & 1 \end{bmatrix} \begin{bmatrix} \epsilon & -1 \\ 0 & \epsilon^{-1} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Thus the computed LU factors fail completely to reproduce A . Notice that there is no subtraction in the formation of $\hat{L}$ and $\hat{U}$. Furthermore, the matrix A is very well conditioned ($\kappa_\infty(A) = 4/(1 + \epsilon)$). The problem, of course, is with the choice of ϵ as the pivot. The partial pivoting strategy would interchange the two rows of A before factorizing it, resulting in a stable factorization.

1.12.2. An Innocuous Calculation?

For any $x \geq 0$ the following computation leaves x unchanged:

```

for  $i = 1:60$ 
     $x = \sqrt{x}$ 
end
for  $i = 1:60$ 
     $x = x^2$ 
end

```

Since the computation involves no subtractions and all the intermediate numbers lie between 1 and x , we might expect it to return an accurate approximation to x in floating point arithmetic.

On the HP 48G calculator, starting with $x = 100$ the algorithm produces $x = 1.0$. In fact, for any x , the calculator computes, in place of $f(x) = x$, the

function

$$\hat{f}(x) = \begin{cases} 0, & 0 \leq x < 1, \\ 1, & x \geq 1. \end{cases}$$

The calculator is producing a completely inaccurate approximation to $f(x)$ in just 120 operations on nonnegative numbers. How can this happen?

The positive numbers x representable on the HP 48G satisfy $10^{-499} \leq x \leq 9.999 \dots \times 10^{499}$. If we define $r(x) = x^{1/2^{60}}$ then, for any machine number $x \geq 1$,

$$\begin{aligned} 1 &\leq r(x) < r(10^{500}) = 10^{500/2^{60}} \\ &= e^{500 \cdot 2^{-60} \cdot \log 10} < e^{10^{-15}} \\ &= 1 + 10^{-15} + \frac{1}{2} \cdot 10^{-30} + \dots, \end{aligned}$$

which rounds to 1, since the HP 48G works to about 12 decimal digits. Thus for $x > 1$, the repeated square roots reduce x to 1.0, which the squarings leave unchanged.

For $0 < x < 1$ we have

$$x \leq \underbrace{0.99 \dots 9}_{12}$$

on a 12-digit calculator, so we would expect the square root to satisfy

$$\begin{aligned} \sqrt{x} &\leq (1 - 10^{-12})^{1/2} = 1 - \frac{1}{2} \cdot 10^{-12} - \frac{1}{8} \cdot 10^{-24} - \dots \\ &= 0.\underbrace{99 \dots 9}_{12} 4 \underbrace{99 \dots 9}_{11} 87499 \dots \end{aligned}$$

This upper bound rounds to the 12-significant-digit number $0.99 \dots 9$. Hence after the 60 square roots we have on the calculator a number $x \leq 0.99 \dots 9$. The 60 squarings are represented by $s(x) = x^{2^{60}}$, and

$$\begin{aligned} s(x) &\leq s(0.99 \dots 9) = (1 - 10^{-12})^{2^{60}} \\ &= 10^{2^{60} \log(1 - 10^{-12}) \log_{10} e} \\ &\approx 10^{-2^{60} \cdot 10^{-12} \cdot \log_{10} e} \\ &\approx 3.6 \times 10^{-500708}. \end{aligned}$$

Because it is smaller than the smallest positive representable number, this result is set to zero on the calculator—a process known as *underflow*. (The converse situation, in which a result exceeds the largest representable number, is called *overflow*.)

The conclusion is that there is nothing wrong with the calculator. This innocuous-looking calculation simply exhausts the precision and range of a machine with 12 digits of precision and a 3-digit exponent.

1.12.3. An Infinite Sum

It is well known that $\sum_{k=1}^{\infty} k^{-2} = \pi^2/6 = 1.6449\ 3406\ 6848 \dots$. Suppose we were not aware of this identity and wished to approximate the sum numerically. The most obvious strategy is to evaluate the sum for increasing k until the computed

sum does not change. In single precision this yields the value 1.6447 2532, which is first attained at $k = 4096$. This agrees with the exact infinite sum to just four significant digits out of a possible nine.

The explanation for the poor accuracy is that we are summing the numbers from largest to smallest, and the small numbers are unable to contribute to the sum. For $k = 4096$ we are forming $s + 4096^{-2} = s + 2^{-24}$, where $s \approx 1.6$. Single precision corresponds to a 24-bit significand, so the term we are adding to s “drops off the end” of the computer word, as do all successive terms.

The simplest cure for this inaccuracy is to sum in the opposite order: from smallest to largest. Unfortunately, this requires knowledge of how many terms to take before the summation begins. With 10^9 terms we obtain the computed sum 1.6449 3406, which is correct to eight significant digits.

For much more on summation, see Chapter 4.

1.13. Increasing the Precision

When the only source of errors is rounding, a common technique for estimating the accuracy of an answer is to recompute it at a higher precision and to see how many digits of the original and the (presumably) more accurate answer agree. We would intuitively expect any desired accuracy to be achievable by computing at a high enough precision. This is certainly the case for algorithms possessing an error bound proportional to the precision, which includes all the algorithms described in the subsequent chapters of this book. However, since an error bound is not necessarily attained, there is no guarantee that a result computed in t -digit precision will be more accurate than one computed in s -digit precision, for a given $t > s$; in particular, for a very ill conditioned problem both results could have no correct digits.

For illustration, consider the system $Ax = b$, where A is the inverse of the 5×5 Hilbert matrix and $b_i = (-1)^i i$. (For details of the matrices used in this experiment see Chapter 28.) We solved the system in varying precisions with unit roundoffs $u = 2^{-t}$, $t = 15:40$, corresponding to about 4 to 12 decimal places of accuracy. (This was accomplished in MATLAB by using the function `chop` from the Matrix Computation Toolbox to round the result of every arithmetic operation to t bits; see Appendix D.) The algorithm used was Gaussian elimination (without pivoting), which is perfectly stable for this symmetric positive definite matrix. The upper plot of Figure 1.3 shows t against the relative errors $\|x - \hat{x}\|_\infty / \|x\|_\infty$ and the relative residuals $\|b - A\hat{x}\|_\infty / (\|A\|_\infty \|\hat{x}\|_\infty)$. The lower plot of Figure 1.3 gives corresponding results for $A = P_5 + 5I$, where P_5 is the Pascal matrix of order 5. The condition numbers $\kappa_\infty(A)$ are 1.62×10^2 for the inverse Hilbert matrix and 9.55×10^5 for the shifted Pascal matrix. In both cases the general trend is that increasing the precision decreases the residual and relative error, but the behaviour is not monotonic. The reason for the pronounced oscillating behaviour of the relative error (but not the residual) for the inverse Hilbert matrix is not clear.

An example in which increasing the precision by several bits does not improve

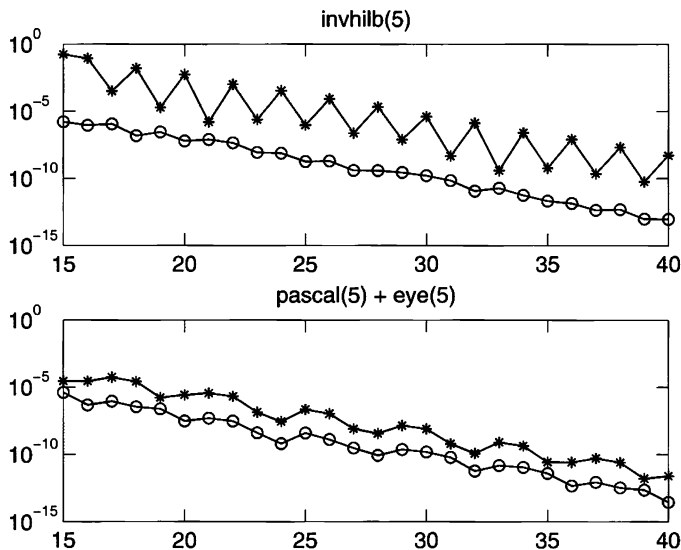


Figure 1.3. Forward errors $\|x - \hat{x}\|_\infty / \|x\|_\infty$ (“*”) and relative residuals $\|b - A\hat{x}\|_\infty / (\|A\|_\infty \|\hat{x}\|_\infty)$ (“o”) versus precision $t = -\log_2 u$ on the x axis.

the accuracy is the evaluation of

$$y = x + a \sin(bx), \quad x = \frac{1}{7}, \quad a = 10^{-8}, \quad b = 2^{24}. \quad (1.8)$$

Figure 1.4 plots t versus the absolute error, for precisions $u = 2^{-t}$, $t = 10:40$. Since $a \sin(bx) \approx -8.55 \times 10^{-9}$, for t less than about 20 the error is dominated by the error in representing $x = 1/7$. For $22 \leq t \leq 31$ the accuracy is (exactly) constant! The plateau over the range $22 \leq t \leq 31$ is caused by a fortuitous rounding error in the addition: in the binary representation of the exact answer the 23rd to 32nd digits are 1s, and in the range of t of interest the final rounding produces a number with a 1 in the 22nd bit and zeros beyond, yielding an unexpectedly small error that affects only bits 33 onwards.

A more contrived example in which increasing the precision has no beneficial effect on the accuracy is the following evaluation of $z = f(x)$:

```

y = abs(3(x - 0.5) - 0.5)/25
if y = 0
    z = 1
else
    z = ey    % Store to inhibit extended precision evaluation.
    z = (z - 1)/y
end

```

In exact arithmetic, $z = f(2/3) = 1$, but in MATLAB $\hat{z} = fl(f(2/3)) = 0.0$ in both (simulated) single precision and double precision arithmetic. A further example is provided by the “innocuous calculation” of §1.12.2, in which a step function is computed in place of $f(x) = x$ for a wide range of precisions.

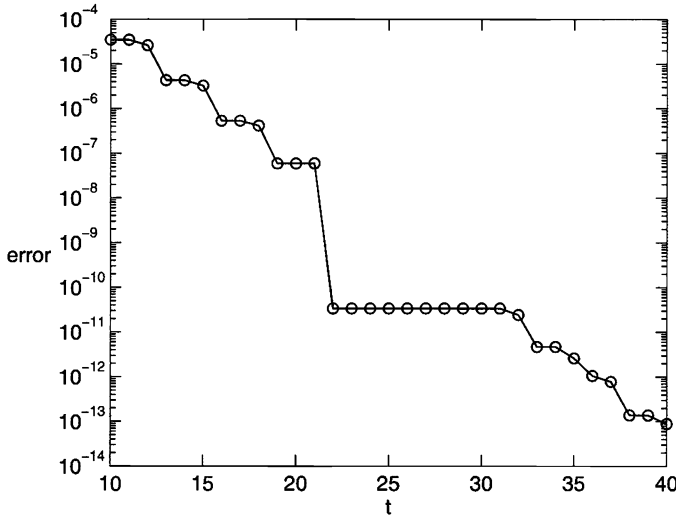


Figure 1.4. Absolute error versus precision, $t = -\log_2 u$, in evaluating (1.8).

It is worth stressing that *how* precision is increased can greatly affect the results obtained. Increasing the precision without preserving important properties such as monotonicity of rounding can vitiate an otherwise reliable algorithm. Increasing the precision without maintaining a correct relationship among the precisions in different parts of an algorithm can also be harmful to the accuracy.

1.14. Cancellation of Rounding Errors

It is not unusual for rounding errors to cancel in stable algorithms, with the result that the final computed answer is much more accurate than the intermediate quantities. This phenomenon is not universally appreciated, perhaps because we tend to look at the intermediate numbers in an algorithm only when something is wrong, not when the computed answer is satisfactory. We describe two examples. The first is a very short and rather unusual computation, while the second involves a well-known algorithm for computing a standard matrix decomposition.

1.14.1. Computing $(e^x - 1)/x$

Consider the function $f(x) = (e^x - 1)/x = \sum_{i=0}^{\infty} x^i/(i+1)!$, which arises in various applications. The obvious way to evaluate f is via the algorithm

```
% Algorithm 1.
if x = 0
    f = 1
else
    f = (ex - 1)/x
end
```

This algorithm suffers severe cancellation for $|x| \ll 1$, causing it to produce an inaccurate answer (0 instead of 1, if x is small enough). Here is an alternative:

```
% Algorithm 2.
y = e^x
if y = 1
    f = 1
else
    f = (y - 1)/log y
end
```

At first sight this algorithm seems perverse, since it evaluates both $\exp$ and $\log$ instead of just $\exp$. Some results computed in MATLAB are shown in Table 1.2. All the results for Algorithm 2 are correct in all the significant figures shown, except for $x = 10^{-15}$, when the last digit should be 1. On the other hand, Algorithm 1 returns answers that become less and less accurate as x decreases.

To gain insight we look at the numbers in a particular computation with $x = 9 \times 10^{-8}$ and $u = 2^{-24} \approx 6 \times 10^{-8}$, for which the correct answer is 1.00000005 to the significant digits shown. For Algorithm 1 we obtain a completely inaccurate result, as expected:

$$fl\left(\frac{e^x - 1}{x}\right) \equiv fl\left(\frac{1.19209290 \times 10^{-7}}{9.00000000 \times 10^{-8}}\right) = 1.32454766.$$

Algorithm 2 produces a result correct in all but the last digit:

$$fl\left(\frac{e^x - 1}{\log e^x}\right) \equiv fl\left(\frac{1.19209290 \times 10^{-7}}{1.19209282 \times 10^{-7}}\right) = 1.00000006.$$

Here are the quantities that would be obtained by Algorithm 2 in exact arithmetic (correct to the significant digits shown):

$$\frac{e^x - 1}{\log e^x} \equiv \frac{9.00000041 \times 10^{-8}}{9.00000001 \times 10^{-8}} = 1.00000005.$$

We see that Algorithm 2 obtains very inaccurate values of $e^x - 1$ and $\log e^x$, but the ratio of the two quantities it computes is very accurate. Conclusion: errors cancel in the division in Algorithm 2.

A short error analysis explains this striking cancellation of errors. We assume that the $\exp$ and $\log$ functions are both computed with a relative error not exceeding the unit roundoff u . The algorithm first computes $\hat{y} = e^x(1 + \delta)$, $|\delta| \leq u$. If $\hat{y} = 1$ then $e^x(1 + \delta) = 1$, so

$$x = -\log(1 + \delta) = -\delta + \delta^2/2 - \delta^3/3 + \cdots, \quad |\delta| \leq u,$$

which implies that the correctly rounded value of $f(x) = 1 + x/2 + x^2/6 + \cdots$ is 1, and so f has been evaluated correctly, to the working precision. If $\hat{y} \neq 1$ then, using (1.1),

$$\hat{f} = fl((\hat{y} - 1)/\log \hat{y}) = \frac{(\hat{y} - 1)(1 + \epsilon_1)}{\log \hat{y}(1 + \epsilon_2)}(1 + \epsilon_3), \quad (1.9)$$

Table 1.2. *Computed values of $(e^x - 1)/x$ from Algorithms 1 and 2.*

x	Algorithm 1	Algorithm 2
10^{-5}	1.000005000006965	1.000005000016667
10^{-6}	1.000000499962184	1.000000500000167
10^{-7}	1.000000049433680	1.000000050000002
10^{-8}	$9.99999939225290 \times 10^{-1}$	1.000000005000000
10^{-9}	1.000000082740371	1.000000000500000
10^{-10}	1.000000082740371	1.000000000050000
10^{-11}	1.000000082740371	1.000000000005000
10^{-12}	1.000088900582341	1.000000000000500
10^{-13}	$9.992007221626408 \times 10^{-1}$	1.000000000000050
10^{-14}	$9.992007221626408 \times 10^{-1}$	1.000000000000005
10^{-15}	1.110223024625156	1.000000000000000
10^{-16}	0	1

where $|\epsilon_i| \leq u$, $i = 1:3$. Thus $\hat{f}$ is a very accurate approximation to

$$g(\hat{y}) := \frac{\hat{y} - 1}{\log \hat{y}}.$$

Note that $\hat{y} = e^x(1 + \delta) =: e^{\hat{x}}$, where $\hat{x} = x + \delta + O(\delta^2)$. To find how $g(\hat{y})$ compares with $g(y) = f(x)$ for $y \approx 1$ we write, using the series expansion of f ,

$$\begin{aligned}
 g(\hat{y}) - f(x) &= f(\hat{x}) - f(x) = \frac{\hat{x} - x}{2} + O(\hat{x} - x)^2 \\
 &= \frac{\delta}{2} + O(\hat{x} - x)^2 \\
 &\approx \frac{\delta}{2} g(y) = \frac{\delta}{2} f(x).
 \end{aligned}$$

From (1.9) it follows that $\hat{f}$ approximates f with relative error at most about $3.5u$.

The details of the analysis obscure the simplicity of the underlying explanation. The expression $(e^x - 1)/x$ cannot be accurately evaluated for a given $x \approx 0$ in floating point arithmetic, while the expression $(y - 1)/\log y$ can be accurately evaluated for a given $y \approx 1$. Since these functions are slowly varying near $x = 0$ ($y = 1$), evaluating $(y - 1)/\log y$ with an accurate, if inexact, approximation to $y = e^x \approx 1$ produces an accurate result.

Finally, we note that f is the divided difference of e^x at x and 0: $(e^x - e^0)/(x - 0)$, and Algorithm 2 rewrites f as the reciprocal of the divided difference of $\log$ at y and 1. This example illustrates the principle that we should recognize divided differences (which may be in disguise) and take care in their evaluation when the arguments are close.

1.14.2. QR Factorization

Any matrix $A \in \mathbb{R}^{m \times n}$, $m \geq n$, has a QR factorization $A = QR$, where $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R \in \mathbb{R}^{n \times n}$ is upper trapezoidal ($r_{ij} = 0$ for $i > j$).

One way of computing the QR factorization is to premultiply A by a sequence of Givens rotations—orthogonal matrices G that differ from the identity matrix only in a 2×2 principal submatrix, which has the form

$$\begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}.$$

With $A_1 := A$, a sequence of matrices A_k satisfying $A_k = G_k A_{k-1}$ is generated. Each A_k has one more zero than the last, so $A_p = R$ for $p = n(m - (n + 1)/2)$. To be specific, we will assume that the zeros are introduced in the order $(n, 1)$, $(n - 1, 1), \dots, (2, 1)$; $(n, 2), \dots, (3, 2)$; and so on.

For a particular 10×6 matrix A , Figure 1.5 plots the relative errors $\|A_k - \hat{A}_k\|_2 / \|A\|_2$, where $\hat{A}_k$ denotes the matrix computed in single precision arithmetic ($u \approx 6 \times 10^{-8}$). We see that many of the intermediate matrices are very inaccurate, but the final computed $\hat{R}$ has an acceptably small relative error, of order u . Clearly, there is heavy cancellation of errors on the last few stages of the computation. This matrix $A \in \mathbb{R}^{10 \times 6}$ was specially chosen, following a suggestion of Wilkinson [1244, 1985], as a full matrix such that $\|A\|_2 \approx 1$ and A_{10} has the form

$$A_{10} = \begin{bmatrix} \tilde{a}_{11} & \tilde{a}_{12} & \tilde{A}(1, 3:n) \\ 0 & 1 & \tilde{A}(2, 3:n) \\ 0 & y & \tilde{A}(3:m, 3:n) \end{bmatrix}, \quad \|y\|_2 \approx 2u.$$

Because y is at the roundoff level, the computed $\hat{y}$ is the result of severe subtractive cancellation and so is dominated by rounding errors. Consequently, the computed Givens rotations $\tilde{G}_{10}, \dots, \tilde{G}_{17}$, whose purpose is to zero the vector $\hat{y}$, and which are determined by ratios involving the elements of $\hat{y}$, bear little relation to their exact counterparts, causing $\hat{A}_k$ to differ greatly from A_k for $k = 11, 12, \dots$.

To shed further light on this behaviour, we note that the Givens QR factorization is perfectly backward stable; that is, the computed $\hat{R}$ is the exact R factor of $A + \Delta A$, where $\|\Delta A\|_2 \leq cu\|A\|_2$, with c a modest constant depending on the dimensions (Theorem 19.10). By invoking a perturbation result for the QR factorization (namely (19.35a)) we conclude that $\|R - \hat{R}\|_2 / \|A\|_2$ is bounded by a multiple of $\kappa_2(A)u$. Our example is constructed so that $\kappa_2(A)$ is small (≈ 24), so we know a priori that the graph in Figure 1.5 must eventually dip down to the unit roundoff level.

We also note that $\|Q - \hat{Q}\|_2$ is of order u in this example, as again we can show it must be from perturbation theory. Since Q is a product of Givens rotations, this means that even though some of the intermediate Givens rotations are very inaccurate, their product is highly accurate, so in the formation of Q , too, there is extensive cancellation of rounding errors.

1.15. Rounding Errors Can Be Beneficial

An old method for computing the largest eigenvalue (in absolute value) of a matrix A and the corresponding eigenvector is the power method, which consists of repeatedly multiplying a given starting vector by A . With scaling to avoid underflow and overflow, the process in its simplest form is

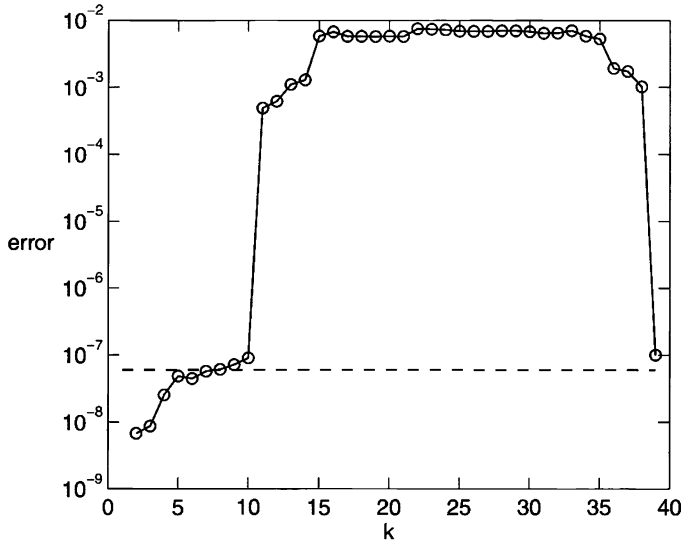


Figure 1.5. Relative errors $\|A_k - \hat{A}_k\|_2 / \|A\|_2$ for Givens QR factorization. The dotted line is the unit roundoff level.

```
% Choose a starting vector x.
while not converged
    x := Ax
    x := x / ||x||_∞
end
```

The theory says that if A has a unique eigenvalue of largest modulus and x is not deficient in the direction of the corresponding eigenvector v , then the power method converges to a multiple of v (at a linear rate).

Consider the matrix

$$A = \begin{bmatrix} 0.4 & -0.6 & 0.2 \\ -0.3 & 0.7 & -0.4 \\ -0.1 & -0.4 & 0.5 \end{bmatrix},$$

which has eigenvalues 0, 0.4394, and 1.161 (correct to the digits shown) and an eigenvector $[1, 1, 1]^T$ corresponding to the eigenvalue zero. If we take $[1, 1, 1]^T$ as the starting vector for the power method then, in principle, the zero vector is produced in one step, and we obtain no indication of the desired dominant eigenvalue–eigenvector pair. However, when we carry out the computation in MATLAB, the first step produces a vector with elements of order 10^{-16} and we obtain after 38 iterations a good approximation to the dominant eigenpair. The explanation is that the matrix A cannot be stored exactly in binary floating point arithmetic. The computer actually works with $A + \Delta A$ for a tiny perturbation ΔA , and the dominant eigenvalue and eigenvector of $A + \Delta A$ are very good approximations to those of A . The starting vector $[1, 1, 1]^T$ contains a nonzero (though tiny) component of the dominant eigenvector of $A + \Delta A$. This component grows rapidly

under multiplication by $A + \Delta A$, helped by rounding errors in the multiplication, until convergence to the dominant eigenvector is obtained.

Perhaps an even more striking example of beneficial effects of rounding errors is in inverse iteration, which is just the power method applied to the shifted and inverted matrix $(A - \mu I)^{-1}$. The shift μ is usually an approximate eigenvalue. The closer μ is to an eigenvalue, the more nearly singular $A - \mu I$ is, and hence the larger the error in computing $y = (A - \mu I)^{-1}x$ (which is done by solving $(A - \mu I)y = x$). However, it can be shown that the error lies almost entirely in the direction of the required eigenvector, and so is harmless; see, for example, Parlett [926, 1998, §4.3] or Golub and Van Loan [509, 1996, §7.6.1].

1.16. Stability of an Algorithm Depends on the Problem

An algorithm can be stable as a means for solving one problem but unstable when applied to another problem. One example is the modified Gram–Schmidt method, which is stable when used to solve the least squares problem but can give poor results when used to compute an orthonormal basis of a matrix (see §§19.8 and 20.3).

A lesser known and much simpler example is Gaussian elimination (GE) without pivoting for computing the determinant of an upper Hessenberg matrix. A square matrix A is upper Hessenberg if $a_{ij} = 0$ for $i > j + 1$. GE transforms A to upper triangular form by $n - 1$ row eliminations, one for each of the boxed entries in this 4×4 illustration:

$$A = \begin{bmatrix} \times & \times & \times & \times \\ \boxed{\times} & \times & \times & \times \\ 0 & \boxed{\times} & \times & \times \\ 0 & 0 & \boxed{\times} & \times \end{bmatrix} \longrightarrow \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} = U.$$

The determinant of A is given by the product of the diagonal elements of U . It is easy to show that this is a stable way to evaluate $\det(A)$, even though arbitrarily large multipliers may arise during the elimination. Note, first, that, if $A^{(k)}$ denotes the matrix at the start of the k th stage ($A^{(1)} = A$), then

$$u_{kk} = a_{kk}^{(k)} = a_{kk}^{(k-1)} - \frac{a_{k,k-1}^{(k-1)} a_{k-1,k}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} = a_{kk} - \frac{a_{k,k-1} a_{k-1,k}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}},$$

because the k th row of $A^{(k-1)}$ is the same as the k th row of A . In floating point arithmetic the model (1.1) shows that the computed $\hat{a}_{ij}^{(k)}$ satisfy

$$\begin{aligned} \hat{u}_{kk} = \hat{a}_{kk}^{(k)} &= \left(a_{kk} - \frac{a_{k,k-1} \hat{a}_{k-1,k}^{(k-1)}}{\hat{a}_{k-1,k-1}^{(k-1)}} (1 + \epsilon_1^{(k)}) (1 + \epsilon_2^{(k)}) \right) (1 + \epsilon_3^{(k)}) \\ &= a_{kk} (1 + \epsilon_3^{(k)}) - \frac{[a_{k,k-1} (1 + \epsilon_1^{(k)}) (1 + \epsilon_2^{(k)}) (1 + \epsilon_3^{(k)})] \hat{a}_{k-1,k}^{(k-1)}}{\hat{a}_{k-1,k-1}^{(k-1)}}, \end{aligned}$$

where $|\epsilon_i^{(k)}| \leq u$, $i = 1:3$. This equation says that the computed diagonal elements $\hat{u}_{kk}$ are the exact diagonal elements corresponding not to A , but to a

Table 1.3. *Results from GE without pivoting on an upper Hessenberg matrix.*

	Exact	Computed	Relative error
x :	$\begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$	$\begin{bmatrix} 2.3842 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$	1.3842
$\det(A)$:	2.0000	2.0000	1.9209×10^{-8}

matrix obtained from A by changing the diagonal elements to $a_{kk}(1 + \epsilon_3^{(k)})$ and the subdiagonal elements to $a_{k,k-1}(1 + \epsilon_1^{(k)})(1 + \epsilon_2^{(k)})(1 + \epsilon_3^{(k)})$. In other words, the computed $\hat{u}_{kk}$ are exact for a matrix differing negligibly from A . The computed determinant $\hat{d}$, which is given by

$$\hat{d} = fl(\hat{u}_{11} \dots \hat{u}_{nn}) = \hat{u}_{11} \dots \hat{u}_{nn}(1 + \eta_1) \dots (1 + \eta_n), \quad |\eta_i| \leq u,$$

is therefore a tiny relative perturbation of the determinant of a matrix differing negligibly from A , so this method for evaluating $\det(A)$ is numerically stable (in the mixed forward-backward error sense of (1.2)).

However, if we use GE without pivoting to solve an upper Hessenberg linear system then large multipliers can cause the solution process to be unstable. If we try to extend the analysis above we find that the computed LU factors (as opposed to just the diagonal of U) do not, as a whole, necessarily correspond to a small perturbation of A .

A numerical example illustrates these ideas. Let

$$A = \begin{bmatrix} \alpha & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

We took $\alpha = 10^{-7}$ and $b = Ae$ ($e = [1, 1, 1, 1]^T$) and used GE without pivoting in single precision arithmetic ($u \approx 6 \times 10^{-8}$) to solve $Ax = b$ and compute $\det(A)$. The computed and exact answers are shown to five significant figures in Table 1.3. Not surprisingly, the computed determinant is very accurate. But the computed solution to $Ax = b$ has no correct figures in its first component. This reflects instability of the algorithm rather than ill conditioning of the problem because the condition number $\kappa_\infty(A) = 16$. The source of the instability is the large first multiplier, $a_{21}/a_{11} = 10^7$.

1.17. Rounding Errors Are Not Random

Rounding errors, and their accumulated effect on a computation, are not random. This fact underlies the success of many computations, including some of those described earlier in this chapter. The validity of statistical analysis of rounding

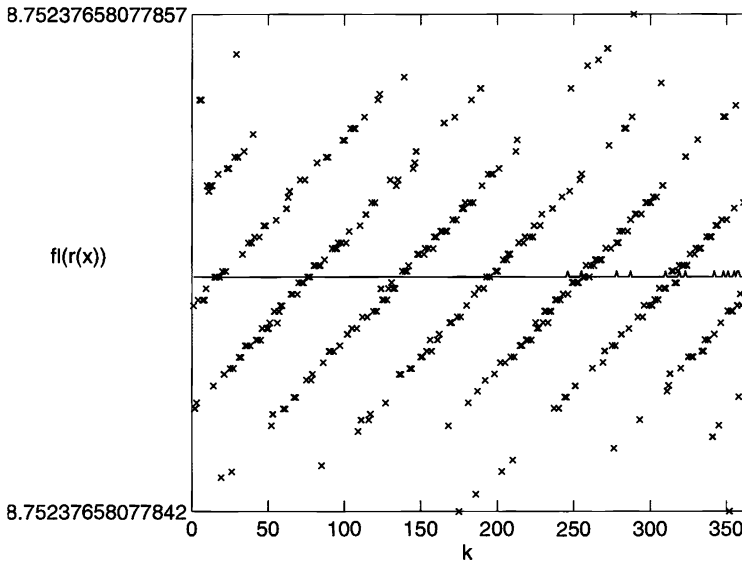


Figure 1.6. Values of rational function $r(x)$ computed by Horner's rule (marked as "x"), for $x = 1.606 + (k - 1)2^{-52}$; solid line is the "exact" $r(x)$.

errors is discussed in §2.8. Here we simply give a revealing numerical example (due to W. Kahan).

Define the rational function

$$r(x) = \frac{622 - x(751 - x(324 - x(59 - 4x)))}{112 - x(151 - x(72 - x(14 - x)))},$$

which is expressed in a form corresponding to evaluation of the quartic polynomials in the numerator and denominator by Horner's rule. We evaluated $r(x)$ by Horner's rule in double precision arithmetic for 361 consecutive floating point numbers starting with $a = 1.606$, namely $x = a + (k - 1)2^{-52}$, $k = 1:361$; the function $r(x)$ is virtually constant on this interval. Figure 1.6 plots the computed function values together with a much more accurate approximation to $r(x)$ (computed from a continued fraction representation). The striking pattern formed by the values computed by Horner's rule shows clearly that the rounding errors in this example are not random.

1.18. Designing Stable Algorithms

There is no simple recipe for designing numerically stable algorithms. While this helps to keep numerical analysts in business (even in proving each other's algorithms to be unstable!) it is not good news for computational scientists in general. The best advice is to *be aware* of the need for numerical stability when designing an algorithm and not to concentrate solely on other issues, such as computational cost and parallelizability.

A few guidelines can be given.

1. Try to avoid subtracting quantities contaminated by error (though such subtractions may be unavoidable).
2. Minimize the size of intermediate quantities relative to the final solution. The reason is that if intermediate quantities are very large then the final answer may be the result of damaging subtractive cancellation. Looked at another way, large intermediate numbers swamp the initial data, resulting in loss of information. The classic example of an algorithm where this consideration is important is Gaussian elimination (§9.3), but an even simpler one is recursive summation (§4.2).
3. Look for different formulations of a computation that are mathematically but not numerically equivalent. For example, the classical Gram–Schmidt method is unstable, but a trivial modification produces the stable modified Gram–Schmidt (MGS) method (§19.8). There are two ways of using the MGS method to solve a least squares problem, the more obvious of which is unstable (§20.3).
4. It is advantageous to express update formulae as

$$\text{new_value} = \text{old_value} + \text{small_correction}$$

if the small correction can be computed with many correct significant figures. Numerical methods are often naturally expressed in this form; examples include methods for solving ordinary differential equations, where the correction is proportional to a stepsize, and Newton’s method for solving a nonlinear system. A classic example of the use of this update strategy is in iterative refinement for improving the computed solution to a linear system $Ax = b$, in which by computing residuals $r = b - Ay$ in extended precision and solving update equations that have the residuals as right-hand sides a highly accurate solution can be computed; see Chapter 12. For another example (in which the correction is not necessarily small), see Problem 2.8.

5. Use only well-conditioned transformations of the problem. In matrix computations this amounts to multiplying by orthogonal matrices instead of nonorthogonal, and possibly, ill-conditioned matrices, where possible. See §6.2 for a simple explanation of this advice in terms of norms.
6. Take precautions to avoid unnecessary overflow and underflow (see §27.8).

Concerning the second point, good advice is to look at the numbers generated during a computation. This was common practice in the early days of electronic computing. On some machines it was unavoidable because the contents of the store were displayed on lights or monitor tubes! Wilkinson gained much insight into numerical stability by inspecting the progress of an algorithm, and sometimes altering its course (for an iterative process with parameters): “Speaking for myself I gained a great deal of experience from user participation, and it was this that led to my own conversion to backward error analysis” [1243, 1980, pp. 112–113] (see also [1227, 1955]). It is ironic that with the wealth of facilities we now have for tracking the progress of numerical algorithms (multiple windows in colour,

graphical tools, fast printers) we often glean less than Wilkinson and his co-workers did from mere paper tape and lights.

1.19. Misconceptions

Several common misconceptions and myths have been dispelled in this chapter (none of them for the first time—see the Notes and References). We highlight them in the following list.

1. Cancellation in the subtraction of two nearly equal numbers is always a bad thing (§1.7).
2. Rounding errors can overwhelm a computation only if vast numbers of them accumulate (§1.11).
3. A short computation free from cancellation, underflow, and overflow must be accurate (§1.12).
4. Increasing the precision at which a computation is performed increases the accuracy of the answer (§1.13).
5. The final computed answer from an algorithm cannot be more accurate than any of the intermediate quantities; that is, errors cannot cancel (§1.14).
6. Rounding errors can only hinder, not help, the success of a computation (§1.15).

1.20. Rounding Errors in Numerical Analysis

Inevitably, much of this book is concerned with numerical linear algebra, because this is the area of numerical analysis in which the effects of rounding errors are most important and have been most studied. In nonlinear problems rounding errors are often not a major concern because other forms of error dominate. Nevertheless the effects of rounding errors have been investigated in many areas of numerical analysis. Throughout the book we give pointers to the general literature (usually in the Notes and References sections), and particular topics (e.g., quadrature) can be tracked down via the index.

1.21. Notes and References

The term “correct significant digits” is rarely defined in textbooks; it is apparently assumed that the definition is obvious. One of the earliest books on numerical analysis, by Scarborough [1014, 1950] (first edition 1930), is noteworthy for containing theorems describing the relationship between correct significant digits and relative error.

The first definition of correct significant digits in §1.2 is suggested by Hildebrand [627, 1974, §1.4], who notes its weaknesses.

For a formal proof and further explanation of the fact that precision does not limit accuracy see Priest [955, 1992].

It is possible to develop formal definitions of numerical stability, either with respect to a particular problem, as is frequently done in research papers, or for a very general class of problems, as is done, for example, by de Jong [301, 1977]. Except in §7.6, we do not give formal definitions of stability in this book, preferring instead to adapt informally the basic notions of backward and forward stability to each problem, and thereby to minimize the amount of notation and abstraction.

Backward error analysis was systematically developed, exploited, and popularized by Wilkinson in the 1950s and 1960s in his research papers and, in particular, through his books [1232, 1963], [1233, 1965] (for more about the books see the Notes and References for Chapter 2). Backward error ideas had earlier appeared implicitly in papers by von Neumann and Goldstine [1200, 1947] and Turing [1166, 1948], both of which deal with the solution of linear systems, and explicitly in an unpublished technical report of Givens [490, 1954] on the solution of the symmetric eigenproblem by reduction to tridiagonal form followed by the use of Sturm sequences. The concept of backward error is not limited to numerical linear algebra. It is used, for example, in the numerical solution of differential equations; see Coomes, Koçak, and Palmer [269, 1995], Eirola [386, 1993], Enright [390, 1989], Sanz-Serna and Larsson [1010, 1993], and Shampine [1030, 1994, §2.2],

Conditioning of problems has been studied by numerical analysts since the 1940s, but the first general theory was developed by Rice [985, 1966]. In numerical linear algebra, developing condition numbers is part of the subject of perturbation theory, on which there is a large literature.

The solution of a quadratic equation is a classic problem in numerical analysis. In 1969 Forsythe [428, 1969] pointed out “the near absence of algorithms to solve even a quadratic equation in a satisfactory way on actually used digital computer systems” and he presented specifications suggested by Kahan for a satisfactory solver. Similar, but less technical, presentations are given by Forsythe [427, 1969], Forsythe, Malcolm, and Moler [430, 1977, §2.6], and Young and Gregory [1272, 1972, §§1.2, 3.4]. Kahan [688, 1972] and Sterbenz [1062, 1974] both present algorithms for solving a quadratic equation, accompanied by error analysis.

For more details of algorithms for computing the sample variance and their error analysis, see Chan and Lewis [215, 1979], Chan, Golub, and LeVeque [214, 1983], Barlow [71, 1991], and the references therein. Good general references on computational aspects of statistics are Kennedy and Gentle [723, 1980] and Thisted [1135, 1988].

The issues of conditioning and numerical stability play a role in any discipline in which finite precision computation is performed, but the understanding of these issues is less well developed in some disciplines than in others. In geometric computation, for example, there has been much interest since the late 1980s in the accuracy and robustness of geometric algorithms; see Milenkovic [848, 1988], Hoffmann [632, 1989], Priest [954, 1991], [955, 1992], and Shewchuk [1038, 1997].

It was after discovering Lemma 1.1 that Wilkinson began to develop backward error analysis systematically in the 1950s. He explains that in solving eigenproblems $Ax = \lambda x$ by deflation, the residual of the computed solution, $r := A\bar{x} - \bar{\lambda}\bar{x}$ (with the normalization $\bar{x}^T \bar{x} = 1$), was “always at noise level relative to A ” [1245, 1986]. He continues, “After some years’ experience of this I happened, almost by accident, to observe that ... $(A - r\bar{x}^T)\bar{x} = \bar{\lambda}\bar{x}$... In other words $\bar{\lambda}$ and $\bar{x}$ were

exact for a matrix $A - r\bar{x}^T$ and since $\|r\bar{x}^T\|_2 = \|r\|_2$, this meant that they were exact for a matrix differing from A at the noise level of the computer.” For further details see [1245, 1986] or [1244, 1985].

The numerical stability of Cramer’s rule for 2×2 systems has been investigated by Moler [862, 1974] and Stummel [1095, 1981, §3.3].

The example in §1.12.2 is taken from the *HP-15C Advanced Functions Handbook* [570, 1982], and a similar example is given by Kahan [690, 1980]. For another approach to analysing this “innocuous calculation” see Problem 3.11. The “ $f(2/3)$ ” example in §1.13 is also taken from [690, 1980], in which Kahan states three “anti-theorems” that are included among our misconceptions in §1.19.

The example (1.8) is adapted from an example of Sterbenz [1062, 1974, p. 220], who devotes a section to discussing the effects of rerunning a computation at higher precision.

The function $\text{expm1} := e^x - 1$ is provided in some floating point processors and mathematics libraries as a more accurate alternative to forming e^x and subtracting 1 [1125, 1992]. It is important in the computation of $\sinh$ and $\tanh$, for example (since $\sinh x = e^{-x}(e^{2x} - 1)/2$). Of course, it is also possible to take the converse approach and express the exponential in terms of trigonometric functions: the expression $(e^x - 1)/x = (e^x + 1) \tanh(x/2)/x$ provides an accurate, if generally expensive, way to evaluate the function investigated in §1.14.1 if an accurate $\tanh$ routine is available. Algorithm 2 in §1.14.1 is due to Kahan [690, 1980].

The instability and stability of GE without pivoting applied to an upper Hessenberg matrix (§1.16) was first pointed out and explained by Wilkinson [1228, 1960]; Parlett [923, 1965] also gives a lucid discussion. In the 1950s and 1960s, prior to the development of the QR algorithm, various methods were proposed for the nonsymmetric eigenvalue problem that involved transforming a matrix to Hessenberg form H and then finding the zeros of the characteristic polynomial $\det(H - \lambda I)$. The most successful method of this type was Laguerre’s iteration, described by Parlett [922, 1964], and used in conjunction with Hyman’s method for evaluating $\det(H - \lambda I)$. Hyman’s method is described in §14.6.1.

Classic papers dispensing good advice on the dangers inherent in numerical computation are the “pitfalls” papers by Stegun and Abramowitz [1061, 1956] and Forsythe [429, 1970]. The book *Numerical Methods That Work* by Acton [4, 1970] must also be mentioned as a fount of hard-earned practical advice on numerical computation (look carefully and you will see that the front cover includes a faint image of the word “Usually” before “Work”). If it is not obvious to you that the equation $x^2 - 10x + 1 = 0$ is best thought of as a nearly linear equation for the smaller root, you will benefit from reading Acton (see p. 58). Everyone should read Acton’s “Interlude: What *Not* to Compute” (pp. 245–257). Acton’s more recent work [5, 1996] dispenses further advice.

Finally, we mention the paper “How to Get Meaningless Answers in Scientific Computation (and What to Do About It)” by Fox [437, 1971]. Fox, a contemporary of Wilkinson, founded the Oxford Computing Laboratory and was for many years Professor of Numerical Analysis at Oxford. In this paper he gives numerous examples in which incorrect answers are obtained from plausible numerical methods (many of the examples involve truncation errors as well as rounding errors). The section titles provide a list of reasons why you might compute worthless

answers:

- Your problem might be ill conditioned.
- Your method might be unstable.
- You expect too much “analysis” from the computer⁴.
- Your intuition fails you.
- You accept consistency too easily.
- A successful method may fail in slightly different circumstances.
- Your test examples may be too special.

Fox estimates [437, 1971, p. 296] that “about 80 per cent of all the results printed from the computer are in error to a much greater extent than the user would believe.”

Problems

*The road to wisdom?
Well, it's plain and simple to express:
Err
and err
and err again
but less
and less
and less.*

— PIET HEIN, *Grooks* (1966)

1.1. In error analysis it is sometimes convenient to bound $\tilde{E}_{\text{rel}}(\hat{x}) = |x - \hat{x}|/|\hat{x}|$ instead of $E_{\text{rel}}(\hat{x}) = |x - \hat{x}|/|x|$. Obtain inequalities between $E_{\text{rel}}(\hat{x})$ and $\tilde{E}_{\text{rel}}(\hat{x})$.

1.2. (Skeel and Keiper [1044, 1993, §1.2]) The number $y = e^{\pi\sqrt{163}}$ was evaluated at t -digit precision for several values of t , yielding the values shown in the following table, which are in error by at most one unit in the least significant digit (the first two values are padded with trailing zeros):

t	y
10	2625374126000000000
15	262537412640769000
20	262537412640768744.00
25	262537412640768744.0000000
30	262537412640768743.999999999999

Does it follow that the last digit before the decimal point is 4?

1.3. Show how to rewrite the following expressions to avoid cancellation for the indicated arguments.

⁴This reason refers to using an inappropriate convergence test in an iterative process.

1. $\sqrt{x+1} - 1, x \approx 0$.
2. $\sin x - \sin y, x \approx y$.
3. $x^2 - y^2, x \approx y$.
4. $(1 - \cos x)/\sin x, x \approx 0$.
5. $c = (a^2 + b^2 - 2ab \cos \theta)^{1/2}, a \approx b, |\theta| \ll 1$.

1.4. Give stable formulae for computing the square root $x + iy$ of a complex number $a + ib$.

1.5. [570, 1982] Show how to compute $\log(1+x)$ accurately for all $x > -1$, including for small $|x|$. Assume that the log function is computed with a relative error not exceeding u . (Hint: adapt the technique used in §1.14.1.) Hence, by writing $(1 + 1/n)^n = \exp(n \log(1 + 1/n))$, show how to compute $(1 + 1/n)^n$ accurately for large n .

1.6. (Smith [1051, 1975]) Type the following numbers into your pocket calculator, and look at them upside down (you or the calculator):

07734	The famous “___ world” program
38079	Object
318808	Name
35007	Adjective
$57738.57734 \times 10^{40}$	Exclamation on finding a bug
3331	A high-quality floating point arithmetic
$\sqrt{31,438,449}$	Fallen tree trunks

1.7. A condition number for the sample variance (1.4), here denoted by $V(x) : \mathbb{R}^n \rightarrow \mathbb{R}$, can be defined by

$$\kappa_C := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{|V(x) - V(x + \Delta x)|}{\epsilon V(x)} : |\Delta x_i| \leq \epsilon |x_i|, i = 1:n \right\}.$$

Show that

$$\kappa_C = 2 \frac{\sum_{i=1}^n |x_i - \bar{x}| |x_i|}{(n-1)V(x)}.$$

This condition number measures perturbations in x componentwise. A corresponding normwise condition number is

$$\kappa_N := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{|V(x) - V(x + \Delta x)|}{\epsilon V(x)} : \|\Delta x\|_2 \leq \epsilon \|x\|_2 \right\}.$$

Show that

$$\kappa_N = 2 \frac{\|x\|_2}{\sqrt{(n-1)V(x)}} = 2 \left(1 + \frac{n}{n-1} \frac{\bar{x}^2}{V(x)} \right)^{1/2} \geq \kappa_C.$$

1.8. (Kahan, Muller, [875, 1989], Francois and Muller [442, 1991]) Consider the recurrence

$$x_{k+1} = 111 - (1130 - 3000/x_{k-1})/x_k, \quad x_0 = 11/2, \quad x_1 = 61/11.$$

In exact arithmetic the x_k form a monotonically increasing sequence that converges to 6. Implement the recurrence on your computer or pocket calculator and compare the computed x_{34} with the true value 5.998 (to four correct significant figures). Explain what you see.

The following questions require knowledge of material from later chapters.

1.9. Cramer's rule solves a 2×2 system $Ax = b$ according to

$$\begin{aligned}d &= a_{11}a_{22} - a_{21}a_{12}, \\x_1 &= (b_1a_{22} - b_2a_{12})/d, \\x_2 &= (a_{11}b_2 - a_{21}b_1)/d.\end{aligned}$$

Show that, assuming d is computed exactly (this assumption has little effect on the final bounds), the computed solution $\hat{x}$ satisfies

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_3 \operatorname{cond}(A, x), \quad \|b - Ax\|_\infty \leq \gamma_3 \operatorname{cond}(A^{-1}) \|b\|_\infty,$$

where $\gamma_3 = 3u/(1 - 3u)$, $\operatorname{cond}(A, x) = \| |A^{-1}| |A| |x| \|_\infty / \|x\|_\infty$, and $\operatorname{cond}(A) = \| |A^{-1}| |A| \|_\infty$. This forward error bound is as small as that for a backward stable method (see §§7.2, 7.6), so Cramer's rule is forward stable for 2×2 systems.

1.10. Show that the computed sample variance $\hat{V} = fl(V(x))$ produced by the two-pass formula (1.4) satisfies

$$\frac{|V - \hat{V}|}{V} \leq (n + 3)u + O(u^2).$$

(Note that this error bound does not involve the condition numbers κ_C or κ_N from Problem 1.7, at least in the first-order term. This is a rare instance of an algorithm that determines the answer more accurately than the data warrants!)

Chapter 2

Floating Point Arithmetic

*From 1946–1948 a great deal of quite detailed coding was done.
The subroutines for floating-point arithmetic were ...
produced by Alway and myself in 1947 ...
They were almost certainly the earliest floating-point subroutines.*

— J. H. WILKINSON, *Turing's Work at the National Physical Laboratory* ... (1980)

*MATLAB's creator Cleve Moler used to advise foreign visitors
not to miss the country's two most awesome spectacles:
the Grand Canyon, and meetings of IEEE p754.*

— MICHAEL L. OVERTON, *Numerical Computing
with IEEE Floating Point Arithmetic* (2001)

*Arithmetic on Cray computers is interesting because it is driven by a
motivation for the highest possible floating-point performance ...
Addition on Cray computers does not have a guard digit,
and multiplication is even less accurate than addition ...
At least Cray computers serve to keep numerical analysts on their toes!*

— DAVID GOLDBERG⁵, *Computer Arithmetic* (1996)

*It is rather conventional to obtain a "realistic" estimate
of the possible overall error due to k roundoffs,
when k is fairly large,
by replacing k by $\sqrt{k}$ in an expression for (or an estimate of)
the maximum resultant error.*

— F. B. HILDEBRAND, *Introduction to Numerical Analysis* (1974)

⁵In Hennessy and Patterson [562, 1996, §A.12].

2.1. Floating Point Number System

A floating point number system $F \subset \mathbb{R}$ is a subset of the real numbers whose elements have the form

$$y = \pm m \times \beta^{e-t}. \quad (2.1)$$

The system F is characterized by four integer parameters:

- the *base* β (sometimes called the *radix*),
- the *precision* t , and
- the *exponent range* $e_{\min} \leq e \leq e_{\max}$.

The *significand*⁶ m is an integer satisfying $0 \leq m \leq \beta^t - 1$. To ensure a unique representation for each nonzero $y \in F$ it is assumed that $m \geq \beta^{t-1}$ if $y \neq 0$, so that the system is *normalized*. The number 0 is a special case in that it does not have a normalized representation. The *range* of the nonzero floating point numbers in F is given by $\beta^{e_{\min}-1} \leq |y| \leq \beta^{e_{\max}}(1 - \beta^{-t})$. Values of the parameters for some machines of historical interest are given in Table 2.1 (the unit roundoff u is defined on page 38).

Note that an alternative (and more common) way of expressing y is

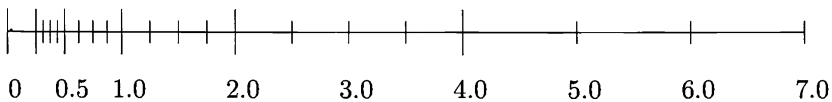
$$y = \pm \beta^e \left(\frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_t}{\beta^t} \right) = \pm \beta^e \times .d_1 d_2 \dots d_t, \quad (2.2)$$

where each digit d_i satisfies $0 \leq d_i \leq \beta - 1$, and $d_1 \neq 0$ for normalized numbers. We prefer the more concise representation (2.1), which we usually find easier to work with. This “nonpositional” representation has pedagogical advantages, being entirely integer based and therefore simpler to grasp. In the representation (2.2), d_1 is called the *most significant digit* and d_t the *least significant digit*.

It is important to realize that the floating point numbers are not equally spaced. If $\beta = 2$, $t = 3$, $e_{\min} = -1$, and $e_{\max} = 3$ then the nonnegative floating point numbers are

0, 0.25, 0.3125, 0.3750, 0.4375, 0.5, 0.625, 0.750, 0.875,
1.0, 1.25, 1.50, 1.75, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0, 6.0, 7.0.

They can be represented pictorially as follows:



⁶The significand is often called the mantissa, but strictly speaking the term mantissa should be used only in conjunction with logarithms.

Table 2.1. *Floating point arithmetic parameters.*

Machine and arithmetic	β	t	$e_{\min}$	$e_{\max}$	u
Cray-1 single	2	48	-8192	8191	4×10^{-15}
Cray-1 double	2	96	-8192	8191	1×10^{-29}
DEC VAX G format, double	2	53	-1023	1023	1×10^{-16}
DEC VAX D format, double	2	56	-127	127	1×10^{-17}
HP 28 and 48G calculators	10	12	-499	499	5×10^{-12}
IBM 3090 single	16	6	-64	63	5×10^{-7}
IBM 3090 double	16	14	-64	63	1×10^{-16}
IBM 3090 extended	16	28	-64	63	2×10^{-33}
IEEE single	2	24	-125	128	6×10^{-8}
IEEE double	2	53	-1021	1024	1×10^{-16}
IEEE extended (typical)	2	64	-16381	16384	5×10^{-20}

Notice that the spacing of the floating point numbers jumps by a factor 2 at each power of 2. The spacing can be characterized in terms of *machine epsilon*, which is the distance ϵ_M from 1.0 to the next larger floating point number. Clearly, $\epsilon_M = \beta^{1-t}$, and this is the spacing of the floating point numbers between 1.0 and β ; the spacing of the numbers between 1.0 and $1/\beta$ is $\beta^{-t} = \epsilon_M/\beta$. The spacing at an arbitrary $x \in F$ is estimated by the following lemma.

Lemma 2.1. *The spacing between a normalized floating point number x and an adjacent normalized floating point number is at least $\beta^{-1}\epsilon_M|x|$ and at most $\epsilon_M|x|$.*

Proof. See Problem 2.2. $\square$

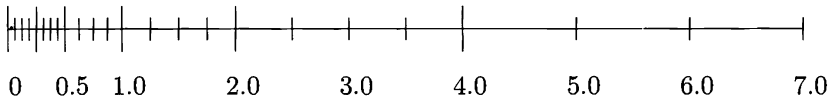
The system F can be extended by including *subnormal numbers* (also known as *denormalized numbers*), which, in the notation of (2.1), are the numbers

$$y = \pm m \times \beta^{e_{\min}-t}, \quad 0 < m < \beta^{t-1},$$

which have the minimum exponent and are not normalized (equivalently, in (2.2) $e = e_{\min}$ and the most significant digit $d_1 = 0$). The subnormal numbers have fewer digits of precision than the normalized numbers. The smallest positive normalized floating point number is $\lambda = \beta^{e_{\min}-1}$ while the smallest positive subnormal number is $\mu = \beta^{e_{\min}-t} = \lambda\epsilon_M$. The subnormal numbers fill the gap between λ and 0 and are equally spaced, with spacing the same as that of the numbers of F between λ and $\beta\lambda$, namely $\lambda\epsilon_M = \beta^{e_{\min}-t}$. For example, in the system illustrated above with $\beta = 2$, $t = 3$, $e_{\min} = -1$, and $e_{\max} = 3$, we have $\lambda = 2^{-2}$ and $\mu = 2^{-4}$, the subnormal numbers are

$$0.0625, 0.125, 0.1875,$$

and the complete number system can be represented as



Let $G \subset \mathbb{R}$ denote all real numbers of the form (2.1) with no restriction on the exponent e . If $x \in \mathbb{R}$ then $fl(x)$ denotes an element of G nearest to x , and the transformation $x \rightarrow fl(x)$ is called *rounding*. There are several ways to break ties when x is equidistant from two floating point numbers, including taking $fl(x)$ to be the number of larger magnitude (round away from zero) or the one with an even last digit d_t (round to even); the latter rule enjoys impeccable statistics [160, 1973]. For more on tie-breaking strategies see the Notes and References. Note that rounding is monotonic: $x \geq y$ implies $fl(x) \geq fl(y)$.

Although we have defined fl as a mapping onto G , we are only interested in the cases where it produces a result in F . We say that $fl(x)$ *overflows* if $|fl(x)| > \max\{|y| : y \in F\}$ and *underflows* if $0 < |fl(x)| < \min\{|y| : 0 \neq y \in F\}$. When subnormal numbers are included in F , underflow is said to be *gradual*.

The following result shows that every real number x lying in the range of F can be approximated by an element of F with a relative error no larger than $u = \frac{1}{2}\beta^{1-t}$. The quantity u is called the *unit roundoff*. It is the most useful quantity associated with F and is ubiquitous in the world of rounding error analysis.

Theorem 2.2. *If $x \in \mathbb{R}$ lies in the range of F then*

$$fl(x) = x(1 + \delta), \quad |\delta| < u. \quad (2.3)$$

Proof. We can assume that $x > 0$. Writing the real number x in the form

$$x = \mu \times \beta^{e-t}, \quad \beta^{t-1} \leq \mu < \beta^t,$$

we see that x lies between the adjacent floating point numbers $y_1 = \lfloor \mu \rfloor \beta^{e-t}$ and $y_2 = \lceil \mu \rceil \beta^{e-t}$. (Strictly, if $\lceil \mu \rceil = \beta^t$ then the floating point representation (2.1) of y_2 is $\lceil \mu \rceil / \beta \cdot \beta^{e-t+1}$.) Thus $fl(x) = y_1$ or y_2 and we have

$$|fl(x) - x| \leq \frac{|y_2 - y_1|}{2} \leq \frac{\beta^{e-t}}{2}.$$

Hence

$$\left| \frac{fl(x) - x}{x} \right| \leq \frac{\frac{1}{2}\beta^{e-t}}{\mu \times \beta^{e-t}} \leq \frac{1}{2}\beta^{1-t} = u.$$

The last inequality is strict unless $\mu = \beta^{t-1}$, in which case $x = fl(x)$, and hence the inequality of the theorem is strict. $\square$

Theorem 2.2 says that $fl(x)$ is equal to x multiplied by a factor very close to 1. The representation $1 + \delta$ for the factor is the standard choice, but it is not the only possibility. For example, we could write the factor as e^α , with a bound on $|\alpha|$ a little less than u (cf. the *rp* notation in §3.4).

The following modified version of this theorem can also be useful.

Theorem 2.3. *If $x \in \mathbb{R}$ lies in the range of F then*

$$fl(x) = \frac{x}{1 + \delta}, \quad |\delta| \leq u.$$

Proof. See Problem 2.4. $\square$

The widely used IEEE standard arithmetic (described in §2.3) has $\beta = 2$ and supports two precisions. Single precision has $t = 24$, $e_{\min} = -125$, $e_{\max} = 128$, and $u = 2^{-24} \approx 5.96 \times 10^{-8}$. Double precision has $t = 53$, $e_{\min} = -1021$, $e_{\max} = 1024$, and $u = 2^{-53} \approx 1.11 \times 10^{-16}$. IEEE arithmetic uses round to even.

It is easy to see that

$$\begin{aligned} x = \left(\beta^{t-1} + \frac{1}{2} \right) \times \beta^e &\Rightarrow \left| \frac{fl(x) - x}{x} \right| \approx \frac{1}{2} \beta^{1-t}, \\ x = \left(\beta^t - \frac{1}{2} \right) \times \beta^e &\Rightarrow \left| \frac{fl(x) - x}{x} \right| \approx \frac{1}{2} \beta^{-t}. \end{aligned}$$

Hence, while the relative error in representing x is bounded by $\frac{1}{2}\beta^{1-t}$ (as it must be, by Theorem 2.2), the relative error varies with x by as much as a factor β . This phenomenon is called *wobbling precision* and is one of the reasons why small bases (in particular, $\beta = 2$) are favoured. The effect of wobbling precision is clearly displayed in Figure 2.1, which plots machine numbers x versus the relative distance from x to the next larger machine number, for $1 \leq x \leq 16$ in IEEE single precision arithmetic. In this plot, the relative distances range from about $2^{-23} \approx 1.19 \times 10^{-7}$ just to the right of a power of 2 to about $2^{-24} \approx 5.96 \times 10^{-8}$ just to the left of a power of 2 (see Lemma 2.1).

The notion of *ulp*, or “unit in last place”, is sometimes used when describing the accuracy of a floating point result. One ulp of the normalized floating point number $y = \pm \beta^e \times .d_1 d_2 \dots d_t$ is $\text{ulp}(y) = \beta^e \times .00 \dots 01 = \beta^{e-t}$. If x is any real number we can say that y and x agree to $|y - x| / \text{ulp}(y)$ ulps in y . This measure of accuracy “wobbles” when y changes from a power of β to the next smaller floating point number, since $\text{ulp}(y)$ decreases by a factor β .

In MATLAB the permanent variable `eps` represents the machine epsilon, ϵ_M (not the unit roundoff as is sometimes thought), while `realmax` and `realmin` represent the largest positive and smallest positive normalized floating point number, respectively. On a machine using IEEE standard double precision arithmetic MATLAB returns

```
>> eps
ans =
    2.2204e-016
```

```
>> realmax
ans =
    1.7977e+308
```

```
>> realmin
ans =
    2.2251e-308
```

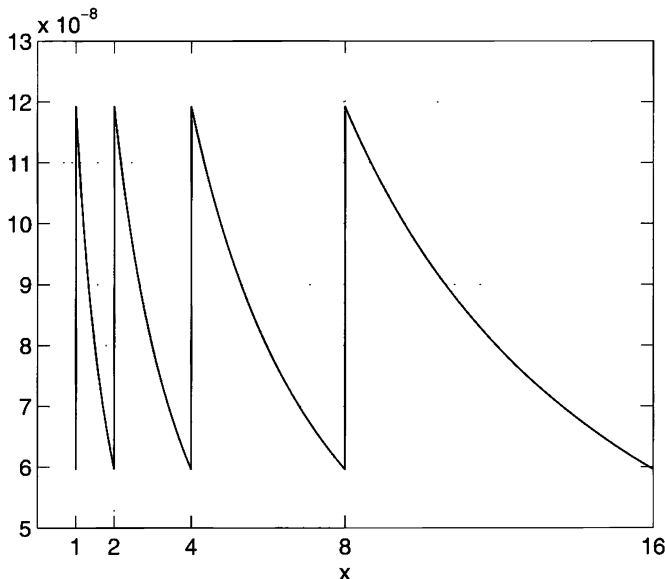



Figure 2.1. Relative distance from x to the next larger machine number ($\beta = 2$, $t = 24$), displaying wobbling precision.

2.2. Model of Arithmetic

To carry out rounding error analysis of an algorithm we need to make some assumptions about the accuracy of the basic arithmetic operations. The most common assumptions are embodied in the following model, in which $x, y \in F$:

STANDARD MODEL

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u, \quad \text{op} = +, -, *, /. \quad (2.4)$$

It is normal to assume that (2.4) holds also for the square root operation.

Note that now we are using $fl(\cdot)$ with an argument that is an arithmetic expression to denote the computed value of that expression. The model says that the computed value of $x \text{ op } y$ is “as good as” the rounded exact answer, in the sense that the relative error bound is the same in both cases. However, the model does not require that $\delta = 0$ when $x \text{ op } y \in F$ —a condition which obviously does hold for the rounded exact answer—so the model does not capture all the features we might require of floating point arithmetic. This model is valid for most computers, and, in particular, holds for IEEE standard arithmetic. Cases in which the model is not valid are described in §2.4.

The following modification of (2.4) can also be used (cf. Theorem 2.3):

$$fl(x \text{ op } y) = \frac{x \text{ op } y}{1 + \delta}, \quad |\delta| \leq u. \quad (2.5)$$

All the error analysis in this book is carried out under the model (2.4), sometimes making use of (2.5). Most results proved using the standard model remain true with the weaker model (2.6) described below, possibly subject to slight increases in the constants. We identify problems for which the choice of model significantly affects the results that can be proved.

2.3. IEEE Arithmetic

IEEE standard 754, published in 1985 [655, 1985], defines a binary floating point arithmetic system. It is the result of several years' work by a working group of a subcommittee of the IEEE Computer Society Computer Standards Committee.

Among the design principles of the standard were that it should encourage experts to develop robust, efficient, and portable numerical programs, enable the handling of arithmetic exceptions, and provide for the development of transcendental functions and very high precision arithmetic.

The standard specifies floating point number formats, the results of the basic floating point operations and comparisons, rounding modes, floating point exceptions and their handling, and conversion between different arithmetic formats. Square root is included as a basic operation. The standard says nothing about exponentiation or transcendental functions such as $\exp$ and $\cos$.

Two main floating point formats are defined:

Type	Size	Significand	Exponent	Unit roundoff	Range
Single	32 bits	23+1 bits	8 bits	$2^{-24} \approx 5.96 \times 10^{-8}$	$10^{\pm 38}$
Double	64 bits	52+1 bits	11 bits	$2^{-53} \approx 1.11 \times 10^{-16}$	$10^{\pm 308}$

In both formats one bit is reserved as a sign bit. Since the floating point numbers are normalized, the most significant bit is always 1 and is not stored (except for the denormalized numbers described below). This *hidden bit* accounts for the “+1” in the table.

The standard specifies that all arithmetic operations are to be performed as if they were first calculated to infinite precision and then rounded according to one of four modes. The default rounding mode is to round to the nearest representable number, with rounding to even (zero least significant bit) in the case of a tie. With this default mode, the model (2.4) is obviously satisfied. Note that computing with a single guard bit (see §2.4) will not always give the same answer as computing the exact result and then rounding. But the use of a second guard bit and a third *sticky* bit (the logical OR of all succeeding bits) enables the rounded exact result to be computed. Rounding to plus or minus infinity is also supported by the standard; this facilitates the implementation of interval arithmetic. The fourth supported mode is rounding to zero (truncation, or chopping).

IEEE arithmetic is a closed system: every arithmetic operation produces a result, whether it is mathematically expected or not, and exceptional operations raise a signal. The default results are shown in Table 2.2. The default response to an exception is to set a flag and continue, but it is also possible to take a trap (pass control to a trap handler).

A NaN is a special bit pattern that cannot be generated in the course of unexceptional operations because it has a reserved exponent field. Since the significand

Table 2.2. *IEEE arithmetic exceptions and default results.*

Exception type	Example	Default result
Invalid operation	$0/0, 0 \times \infty, \sqrt{-1}$	NaN (Not a Number)
Overflow		$\pm\infty$
Divide by zero	Finite nonzero/0	$\pm\infty$
Underflow		Subnormal numbers
Inexact	Whenever $fl(x \text{ op } y) \neq x \text{ op } y$	Correctly rounded result

is arbitrary, subject to being nonzero, a NaN can have something about its provenance encoded in it, and this information can be used for retrospective diagnostics. A NaN is generated by operations such as $0/0$, $0 \times \infty$, ∞/∞ , $(+\infty) + (-\infty)$, and $\sqrt{-1}$. One creative use of the NaN is to denote uninitialized or missing data. Arithmetic operations involving a NaN return a NaN as the answer. A NaN compares as unordered and unequal with everything including itself (a NaN can be tested with the predicate $x \neq x$ or with the IEEE recommended function `isnan`, if provided).

Zero is represented by a zero exponent field and a zero significand, with the sign bit providing distinct representations for $+0$ and -0 . The standard defines comparisons so that $+0 = -0$. Signed zeros provide an elegant way to handle branch cuts in complex arithmetic; for details, see Kahan [694, 1987].

The infinity symbol is represented by a zero significand and the same exponent field as a NaN; the sign bit distinguishes between $\pm\infty$. The infinity symbol obeys the usual mathematical conventions regarding infinity, such as $\infty + \infty = \infty$, $(-1) \times \infty = -\infty$, and (finite)/ $\infty = 0$.

The standard requires subnormal numbers to be represented, instead of being flushed to zero as in many systems. This support of gradual underflow makes it easier to write reliable numerical software; see Demmel [308, 1984].

The standard may be implemented in hardware or software. The first hardware implementation was the Intel 8087 floating point coprocessor, which was produced in 1981 and implements an early draft of the standard (the 8087 very nearly conforms to the present standard). This chip, together with its bigger and more recent brothers the Intel 80287, 80387, 80486 and the Pentium series, is used in IBM PC compatibles (the 80486DX and Pentiums are general-purpose chips that incorporate a floating point coprocessor). Virtually all modern processors implement IEEE arithmetic.

The IEEE standard defines minimum requirements for two extended number formats: *single extended* and *double extended*. The double extended format has at least 79 bits, with at least 63 bits in the significand and at least 15 in the exponent; it therefore surpasses the double format in both precision and range, having unit roundoff $u \leq 5.42 \times 10^{-20}$ and range at least $10^{\pm 4932}$. The purpose of the extended precision formats is not to provide for higher precision computation per se, but to enable double precision results to be computed more reliably (by avoiding intermediate overflow and underflow) and more accurately (by reducing the effect of cancellation) than would otherwise be possible. In particular, extended

precision makes it easier to write accurate routines to evaluate the elementary functions, as explained by Hough [640, 1981].

A double extended format of 80 bits is supported by the Intel chips mentioned above and the Motorola 680x0 chips (used on early Apple Macintoshes); these chips, in fact, normally do *all* their floating point arithmetic in 80-bit arithmetic (even for arguments of the single or double format). However, double extended is not supported by Sun SPARCstations or machines that use the PowerPC or DEC Alpha chips. Furthermore, the extended format (when available) is supported little by compilers and packages such as Mathematica and Maple. Kahan [698, 1994] notes that “What you do not use, you are destined to lose”, and encourages the development of benchmarks to measure accuracy and related attributes. He also explains that

For now the 10-byte Extended format is a tolerable compromise between the value of extra-precise arithmetic and the price of implementing it to run fast; very soon two more bytes of precision will become tolerable, and ultimately a 16-byte format ... That kind of gradual evolution towards wider precision was already in view when IEEE Standard 754 for Floating-Point Arithmetic was framed.

A possible side effect of the use of an extended format is the phenomenon of *double rounding*, whereby a result computed “as if to infinite precision” (as specified by the standard) is rounded first to the extended format and then to the destination format. Double rounding (which is allowed by the standard) can give a different result from that obtained by rounding directly to the destination format, and so can lead to subtle differences between the results obtained with different implementations of IEEE arithmetic (see Problem 2.9).

An IEEE Standard 854, which generalizes the binary standard 754, was published in 1987 [656, 1987]. It is a standard for floating point arithmetic that is independent of word length and base (although in fact only bases 2 and 10 are provided in the standard, since the drafting committee “could find no valid technical reason for allowing other radices, and found several reasons for not allowing them” [251, 1988]). Base 10 IEEE 854 arithmetic is implemented in the HP-71B calculator.

2.4. Aberrant Arithmetics

In the past, not all computer floating point arithmetics adhered to the model (2.4). The most common reason for noncompliance with the model was that the arithmetic lacked a guard digit in subtraction. The role of a guard digit is easily explained with a simple example.

Consider a floating point arithmetic system with base $\beta = 2$ and $t = 3$ digits in the significand. Subtracting from 1.0 the next smaller floating number, we have, in binary notation,

$$\begin{array}{rcl} 2^1 \times 0.100 - & \longrightarrow & 2^1 \times 0.100 - \\ 2^0 \times 0.111 & & 2^1 \times 0.0111 \\ & & \hline & & 2^1 \times 0.0001 = 2^{-2} \times 0.100 \end{array}$$

Notice that to do the subtraction we had to line up the binary points, thereby unnormalizing the second number and using, temporarily, a fourth digit in the significand, known as a *guard digit*. Some old machines did not have a guard digit. Without a guard digit in our example we would compute as follows, assuming the extra digits are simply discarded:

$$\begin{array}{rcl} 2^1 \times 0.100 - & \longrightarrow & 2^1 \times 0.100 - \\ 2^0 \times 0.111 & & 2^1 \times 0.011 \quad (\text{last digit dropped}) \\ \hline & & 2^1 \times 0.001 = 2^{-1} \times 0.100 \end{array}$$

The computed answer is too big by a factor 2 and so has relative error 1! For machines without a guard digit it is not true that

$$fl(x \pm y) = (x \pm y)(1 + \delta), \quad |\delta| \leq u,$$

but it is true that

$$fl(x \pm y) = x(1 + \alpha) \pm y(1 + \beta), \quad \alpha\beta = 0, \quad |\alpha| + |\beta| \leq u.$$

Our model of floating point arithmetic becomes

NO GUARD DIGIT MODEL

$$fl(x \pm y) = x(1 + \alpha) \pm y(1 + \beta), \quad |\alpha|, |\beta| \leq u, \quad (2.6a)$$

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u, \quad \text{op} = *, /, \quad (2.6b)$$

where we have stated a weaker condition on α and β that is generally easier to work with.

Notable examples of machines that lacked guard digits are several models of Cray computers (Cray 1, 2, X-MP, Y-MP, and C90). On these computers subtracting any power of 2 from the next smaller floating point number gives an answer that is either a factor of 2 too large (as in the example above—e.g., Cray X-MP or Y-MP) or is zero (Cray 2). In 1992 Cray announced that it would produce systems that use IEEE standard double precision arithmetic by 1995.

The lack of a guard digit is a serious drawback. It causes many algorithms that would otherwise work perfectly to fail some of the time, including Kahan's version of Heron's formula in the next section and compensated summation (see §4.3). Throughout the book we assume a guard digit is used, unless otherwise stated.

Kahan has made these interesting historical comments about guard digits [696, 1990]:

CRAYs are not the first machines to compute differences blighted by lack of a guard digit. The earliest IBM '360s, from 1964 to 1967, subtracted and multiplied without a hexadecimal guard digit until SHARE, the IBM mainframe user group, discovered why the consequential anomalies were intolerable and so compelled a guard digit to be retrofitted. The earliest Hewlett-Packard financial calculator, the HP-80, had a similar problem. Even now, many a calculator (but not Hewlett-Packard's) lacks a guard digit.

2.5. Exact Subtraction

It is an interesting and sometimes important fact that floating point subtraction is exact if the numbers being subtracted are sufficiently close. The following result about exact subtraction holds for any base β .

Theorem 2.4 (Ferguson). *If x and y are floating point numbers for which $e(x - y) \leq \min(e(x), e(y))$, where $e(x)$ denotes the exponent of x in its normalized floating point representation, then $fl(x - y) = x - y$ (assuming $x - y$ does not underflow).*

Proof. From the condition of the theorem the exponents of x and y differ by at most 1. If the exponents are the same then $fl(x - y)$ is computed exactly, so suppose the exponents differ by 1, which can happen only when x and y have the same sign. Scale and interchange x and y if necessary so that $\beta^{-1} \leq y < 1 \leq x < \beta$, where β is the base. Now x is represented in base β as $x_1.x_2 \dots x_t$ and the exact difference $z = x - y$ is of the form

$$\begin{array}{r} x_1.x_2 \dots x_t - \\ 0.y_1 \dots y_{t-1}y_t \\ \hline z_1.z_2 \dots z_t z_{t+1} \end{array}$$

But $e(x - y) \leq e(y)$ and $y < 1$, so $z_1 = 0$. The algorithm for computing z forms $z_1.z_2 \dots z_{t+1}$ and then rounds to t digits; since z has at most t significant digits this rounding introduces no error, and thus z is computed exactly. $\square$

The next result is a corollary of the previous one but is more well known. It is worth stating as a separate theorem because the conditions of the theorem are so elegant and easy to check (being independent of the base), and because this weaker theorem is sufficient for many purposes.

Theorem 2.5 (Sterbenz). *If x and y are floating point numbers with $y/2 \leq x \leq 2y$ then $fl(x - y) = x - y$ (assuming $x - y$ does not underflow).* $\square$

With gradual underflow, the condition “assuming $x - y$ does not underflow” can be removed from Theorems 2.4 and 2.5 (see Problem 2.19).

Theorem 2.5 is vital in proving that certain special algorithms work. A good example involves Heron’s formula for the area A of a triangle with sides of length a , b , and c :

$$A = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = (a+b+c)/2.$$

This formula is inaccurate for needle-shaped triangles: if $a \approx b + c$ then $s \approx a$ and the term $s - a$ suffers severe cancellation. A way around this difficulty, devised by Kahan, is to rename a , b , and c so that $a \geq b \geq c$ and then evaluate

$$A = \frac{1}{4} \sqrt{(a + (b + c))(c - (a - b))(c + (a - b))(a + (b - c))}. \quad (2.7)$$

The parentheses are essential! Kahan has shown that this formula gives the area with a relative error bounded by a modest multiple of the unit roundoff [496, 1991, Thm. 3], [692, 1983], [701, 1997], [702, 2001] (see Problem 2.23). If a guard digit is not used in subtraction, however, the computed result can be very inaccurate.

2.6. Fused Multiply-Add Operation

Some computers have a *fused multiply-add* (FMA) operation that enables a floating point multiplication followed by an addition or subtraction, $x * y + z$ or $x * y - z$, to be performed as though it were a single floating point operation. An FMA operation therefore commits just one rounding error:

$$fl(x \pm y * z) = (x \pm y * z)(1 + \delta), \quad |\delta| \leq u.$$

The Intel IA-64 architecture, as first implemented on the Intel Itanium chip, has an FMA instruction [273, 1999], as did the IBM RISC System/6000 and IBM Power PC before it. The Itanium's FMA instruction enables a multiply and an addition to be performed in the same number of cycles as one multiplication or addition, so it is advantageous for speed as well as accuracy.

An FMA is a welcome capability in that it enables the number of rounding errors in many algorithms to be approximately halved. Indeed, by using FMAs an inner product $x^T y$ between two n -vectors can be computed with just n rounding errors instead of the usual $2n - 1$, and any algorithm whose dominant operation is an inner product benefits accordingly.

Opportunities to exploit an FMA are frequent. Consider, for example, Newton's method for solving $f(x) = a - 1/x = 0$. The method is

$$\begin{aligned} x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{a - 1/x_k}{x_k^{-2}} \\ &= x_k + (1 - x_k a)x_k, \end{aligned}$$

and its quadratic convergence can be seen from $1 - x_{k+1}a = (1 - x_k a)^2$. This method was used on early computers to implement reciprocation in terms of multiplication and thence division as $a/b = a * (1/b)$ [551, 1946], [1222, 1997], and this technique is still in use [714, 1997]. Since the computation of x_{k+1} from x_k can be expressed as two multiply-adds, the method is attractive when an FMA is available; an FMA also has the advantage of enabling a correctly rounded quotient to be achieved more easily [562, 1996, §A.7]. The floating point divide algorithms for the IA-64 architecture use this Newton-based approach [273, 1999].

An FMA makes it possible to compute an exact representation of the product of two floating point numbers x and y : by computing $\hat{a} = fl(xy)$ and $\hat{b} = fl(xy - \hat{a})$ with two FMAs, $\hat{a} + \hat{b} \equiv xy$ (see Problem 2.26). Furthermore, clever use of an FMA enables highly accurate computation of, for example, the determinant of a 2×2 matrix (see Problem 2.27).

However, the presence of an FMA introduces some tricky programming language issues [700, 1996]. If a programmer writes `a*d + c*b` how is this expression to be compiled for a machine with an FMA? There are three possibilities—two using the FMA and one without it—and they can all yield different answers. An example of where this question is particularly important is the product of complex numbers

$$(x + iy)^*(x + iy) = x^2 + y^2 + i(xy - yx).$$

The product is obviously real, and the right-hand side evaluates as real in IEEE arithmetic, but if an FMA is employed in evaluating $xy - yx$ then a nonreal result will generally be obtained.

In the course of solving the quadratic $ax^2 - 2bx + c = 0$ for x , the expression $\sqrt{b^2 - ac}$ must be computed. Can the discriminant under the square root evaluate to a negative number when $b^2 \geq ac$? In correctly rounded arithmetic the answer is no: the monotonicity of rounding implies $fl(b^2) - fl(ac) \geq 0$ and the floating point subtraction of these incurs a small relative error and hence produces a nonnegative result. However, evaluation as $fl(fl(b^2) - ac)$ using an FMA can produce a negative result (for example, if $b^2 = ac$ and $fl(b^2) < b^2$).

In conclusion, as Kahan [700, 1996] puts it, “[FMAs] should not be used indiscriminately”. Unfortunately, compiler writers, in their efforts to obtain maximum performance, may not give programmers the capability to inhibit FMAs in those subexpressions where their use can have undesirable effects.

2.7. Choice of Base and Distribution of Numbers

What base β is best for a floating point number system? Most modern computers use base 2. Most hand-held calculators use base 10, since it makes the calculator easier for the user to understand (how would you explain to a naive user that 0.1 is not exactly representable on a base 2 calculator?). IBM mainframes traditionally have used base 16. Even base 3 has been tried—in an experimental machine called SETUN, built at Moscow State University in the late 1950s [1208, 1960].

Several criteria can be used to guide the choice of base. One is the impact of wobbling precision: as we saw at the end of §2.1, the spread of representation errors is smallest for small bases. Another possibility is to measure the worst-case representation error or the mean square representation error. The latter quantity depends on the assumed distribution of the numbers that are represented. Brent [160, 1973] shows that for the logarithmic distribution the worst-case error and the mean square error are both minimal for (normalized) base 2, provided that the most significant bit is not stored explicitly.

The logarithmic distribution is defined by the property that the proportion of base β numbers with leading significant digit n is

$$\log_{\beta}(n+1) - \log_{\beta} n = \log_{\beta}\left(1 + \frac{1}{n}\right).$$

It appears that in practice real numbers *are* logarithmically distributed. In 1938, Benford [102, 1938] noticed, as had Newcomb [889, 1881] before him, that the early pages of logarithm tables showed greater signs of wear than the later ones. (He was studying dirty books!) This prompted him to carry out a survey of 20,229 “real-life” numbers, whose decimal representations he found matched the logarithmic distribution closely.

The observed logarithmic distribution of leading significant digits has not been fully explained. Some proposed explanations are based on the assumption that the actual distribution is scale invariant, but this assumption is equivalent to the observation to be explained [1170, 1984]. Barlow [67, 1981], [68, 1981], [70, 1988] and Turner [1169, 1982], [1170, 1984] give useful insight by showing that if uniformly distributed numbers are multiplied together, then the resulting distribution converges to the logarithmic one; see also Boyle [157, 1994]. Furthermore, it is an interesting result that the leading significant digits of the numbers q^k ,

$k = 0, 1, 2, \dots$, are logarithmically distributed if q is positive and is not a rational power of 10; when $q = 2$ and the digit is 7 this is Gelfand's problem [939, 1981, pp. 50–51].

The nature of the logarithmic distribution is striking. For decimal numbers, the digits 1 to 9 are not equally likely to be a leading significant digit. The probabilities are as follows:

1	2	3	4	5	6	7	8	9
0.301	0.176	0.125	0.097	0.079	0.067	0.058	0.051	0.046

As an example, here is the leading significant digit distribution for the elements of the inverse of one random 100×100 matrix from the normal $N(0, 1)$ distribution:

1	2	3	4	5	6	7	8	9
0.334	0.163	0.100	0.087	0.077	0.070	0.063	0.056	0.051

For an entertaining survey of work on the distribution of leading significant digits see Raimi [968, 1976] (and also the popular article [967, 1969]).

2.8. Statistical Distribution of Rounding Errors

Most rounding error analyses, including all the ones in this book, are designed to produce worst-case bounds for the error. The analyses ignore the signs of rounding errors and are often the result of many applications of the triangle inequality and the submultiplicative inequality. Consequently, although the bounds may well give much insight into a method, they tend to be pessimistic if regarded as error estimates.

Statistical statements about the effect of rounding on a numerical process can be obtained from statistical analysis coupled with probabilistic models of the rounding errors. For example, a well-known rule of thumb is that a more realistic error estimate for a numerical method is obtained by replacing the dimension-dependent constants in a rounding error bound by their square root; thus if the bound is $f(n)u$, the rule of thumb says that the error is typically of order $\sqrt{f(n)}u$ (see, for example, Wilkinson [1232, 1963, pp. 26, 102]). This rule of thumb can be supported by assuming that the rounding errors are independent random variables and applying the central limit theorem. Statistical analysis of rounding errors goes back to one of the first papers on rounding error analysis, by Goldstine and von Neumann [501, 1951].

As we noted in §1.17, rounding errors are *not* random. See Problem 2.10 for an example of how two rounding errors cancel in one particular class of computations. Forsythe [424, 1959] points out that rounding errors do not necessarily behave like independent random variables and proposes a random form of rounding (intended for computer testing) to which statistical analysis is applicable.

Henrici [564, 1962], [565, 1963], [566, 1964] assumes models for rounding errors and then derives the probability distribution of the overall error, mainly in the context of difference methods for differential equations. Hull and Swenson [650, 1966] give an insightful discussion of probabilistic models, pointing out that “There is no claim that ordinary rounding and chopping are random processes, or that successive errors are independent. The question to be decided is whether or

not these particular probabilistic models of the processes will adequately describe what actually happens” (see also the ensuing note by Henrici [567, 1966]). Kahan [699, 1996] notes that “The fact that rounding errors are neither random nor uncorrelated will not in itself preclude the possibility of modelling them usefully by uncorrelated random variables. What will jeopardize the utility of such models is their failure to mimic important properties that actual rounding errors possess.” In the last sentence Kahan is referring to results such as Theorem 2.5.

Several authors have investigated the distribution of rounding errors under the assumption that the significands of the operands are from a logarithmic distribution and for different modes of rounding; see Barlow and Bareiss [72, 1985] and the references therein.

Other work concerned with statistical modelling of rounding errors includes that of Tienari [1137, 1970] and Linnainmaa [789, 1975]; see also the discussion of the CESTAC and PRECISE systems in §26.5.

2.9. Alternative Number Systems

The floating point format is not the only means for representing numbers in finite precision arithmetic. Various alternatives have been proposed, though none has achieved widespread use.

A particularly elegant system is the “level index arithmetic” of Clenshaw, Olver, and Turner, in which a number $x > 1$ is represented by $\xi = l + f$, where $f \in [0, 1]$ and

$$x = e^{e^{\dots e^f}}, \quad \text{or} \quad f = \ln(\ln(\dots(\ln x)\dots)),$$

where the exponentiation or logarithm is performed l times (l is the “level”). If $0 < x < 1$, then x is represented by the reciprocal of the representation for $1/x$. An obvious feature of the level index system is that it can represent much larger and smaller numbers than the floating point system, for similar word lengths. A price to be paid is that as the size of the number increases the precision with which it is represented shrinks. If l is sufficiently large then adjacent level index numbers are so far apart that even their exponents in base 10 differ. For very readable introductions to level index arithmetic see Clenshaw and Olver [240, 1984] and Turner [1171, 1991], and for more details see Clenshaw, Olver, and Turner [241, 1989]. Level index arithmetic is somewhat controversial in that there is disagreement about its advantages and disadvantages with respect to floating point arithmetic; see Demmel [310, 1987]. A number system involving levels has also been proposed by Matsui and Iri [826, 1981]; in their system, the number of bits allocated to the significand and exponent is allowed to vary (within a fixed word size).

Other number systems include those of Swartzlander and Alexopolous [1115, 1975], Matula and Kornerup [831, 1985], and Hamada [539, 1987]. For summaries of alternatives to floating point arithmetic see the section “Alternatives to Floating-Point—Some Candidates”, in [241, 1989], and Knuth [744, 1998, Chap. 4].

2.10. Elementary Functions

The term elementary functions refers to the base 2, e and 10 logarithms and exponentials, and the trigonometric and hyperbolic functions and their inverses. These functions are much harder to implement than the elementary operations $+$, $-$, $*$, $/$ and square root and, as noted earlier, the IEEE arithmetic standard does not have anything to say about elementary functions.

In the past, elementary function libraries were of variable quality, but with increased demand for high-quality floating point arithmetic, the ability now to build on IEEE standard elementary operations, and continuing research into elementary function evaluation, the quality of algorithms and libraries is improving.

Ideally, we would like the computed value of an elementary function at a machine number to be the correctly rounded result. The tablemaker's dilemma (see §1.2) casts doubt on whether this goal is achievable. However, Lindemann's result that the exponential of a nonzero algebraic number cannot be algebraic implies that the exponential of a nonzero *machine* number cannot be a machine number or half-way between two machine numbers. Therefore for a given machine number x and precision t there is a finite m so that computing m digits of e^x is sufficient to determine e^x correctly rounded to t digits. Similar arguments apply to the other elementary functions, and so the tablemaker's dilemma does not occur in the context of floating point arithmetic [778, 1998]. In extensive computations, Lefèvre, Muller, and Tisserand [778, 1998], [777, 2001] have determined the maximum value of m over all double precision IEEE arithmetic numbers ($t = 53$) for each of the elementary functions. They find that m is usually close to $2t = 106$. This knowledge of the worst-case m makes the goal of correctly rounded results achievable.

Other desirable properties of computed elementary functions are preservation of monotonicity (e.g., $x < y \Rightarrow e^x < e^y$), preservation of symmetries (e.g., $\sin(x) = -\sin(-x)$), and preservation of mathematical relations and identities (e.g., $\sin(x) \in [-1, 1]$ and $\cos^2(x) + \sin^2(x) = 1$). Unfortunately, these requirements can conflict. Lefèvre, Muller, and Tisserand [778, 1998] note that in IEEE single precision arithmetic the machine number closest to $\arctan(2^{30})$ is slightly greater than $\pi/2$, so a correctly rounded approximation to $\arctan(2^{30})$ necessarily violates the requirement that $\arctan(x) \in [-\pi/2, \pi/2]$.

Describing methods for evaluating elementary functions is outside the scope of this book. Details can be found in the excellent book by Muller [876, 1997], which is the best overall reference for this topic. Algorithms for evaluating elementary functions in IEEE arithmetic are developed by Tang [1121, 1989], [1123, 1990], [1125, 1992], Gal and Bachelis [451, 1991], and Ziv [1285, 1991]. Tang [1124, 1991] gives a very readable description of table lookup algorithms for evaluating elementary functions, which are used in a number of current chips and libraries.

Algorithms for evaluating complex elementary functions that exploit exception handling and assume the availability of algorithms for the real elementary functions are presented by Hull, Fairgrieve, and Tang [649, 1994]. For details of how elementary functions are evaluated on many of today's pocket calculators see Schelin [1016, 1983], as well as Muller [876, 1997].

Table 2.3. *Test arithmetics.*

Hardware	Software	$ 3 \times (4/3 - 1) - 1 $ ^a
Casio fx-140 (≈ 1979)		1×10^{-9}
Casio fx-992VB (≈ 1990)		1×10^{-13}
HP 48G (1993)		1×10^{-11}
Sharp EL-5020 (1994)		0.0^b
Pentium III	MATLAB 6.1 (2001)	$2.2 \dots \times 10^{-16}$
486DX	WATFOR-77 ^c V3.0 (1988)	$2.2 \dots \times 10^{-16}$
486DX	FTN 90 ^d (1993)	$2.2 \dots \times 10^{-16}$
486DX	MS-DOS QBasic 1.1	$1.1 \dots \times 10^{-19}$ ^e

^aIntegers in the test expression are typed as real constants 3.0, etc., for the Fortran tests.

^b 1×10^{-9} if $4/3$ is stored and recalled from memory.

^cWATCOM Systems Inc.

^dSalford Software/Numerical Algorithms Group, Version 1.2.

^e $2.2 \dots \times 10^{-16}$ if $4/3$ is stored and recalled from a variable.

2.11. Accuracy Tests

How can you test the accuracy of the floating point arithmetic on a computer or pocket calculator? There is no easy way, though a few software packages are available to help with the tasks in specific programming languages (see §27.6). There are, however, a few quick and easy tests that *may* reveal weaknesses. The following list is far from comprehensive and good performance on these tests does not imply that an arithmetic is correct. Results from the tests are given in Tables 2.4–2.5 for the selected floating point arithmetics described in Table 2.3, with incorrect digits in boldface and underlined>. Double precision was used for the compiled languages. The last column of Table 2.3 gives an estimate of the unit roundoff (see Problem 2.14). The estimate produced by QBasic indicates that the compiler used extended precision in evaluating the estimate.

1. (Cody [249, 1982]) Evaluate $\sin(22) = -8.8513\ 0929\ 0403\ 8759\ 2169 \times 10^{-3}$ (shown correct to 21 digits). This is a difficult test for the range reduction used in the sine evaluation (which brings the argument within the range $[-\pi/2, \pi/2]$, and which necessarily uses an approximate value of π), since 22 is close to an integer multiple of π .
2. (Cody [249, 1982]) Evaluate $2.5^{125} = 5.5271\ 4787\ 5260\ 4445\ 6025 \times 10^{49}$ (shown correct to 21 digits). One way to evaluate $z = x^y$ is as $z = \exp(y \log x)$. But to obtain z correct to within a few ulps it is not sufficient to compute $\exp$ and $\log$ correct to within a few ulps; in other words, the composition of two functions evaluated to high accuracy is not necessarily obtained to the same accuracy. To examine this particular case, write

$$w := y \log x, \quad z = \exp(w).$$

If $w \rightarrow w + \Delta w$ then $z \rightarrow z + \Delta z$, where $z + \Delta z = \exp(w + \Delta w) =$

Table 2.4. *Sine test.*

Machine	sin(22)
Exact	$-8.8513\ 0929\ 0403\ 8759 \times 10^{-3}$
Casio fx-140	$-8.8513\ \underline{62} \times 10^{-3}$
Casio fx-992VB	$-8.8513\ 0929\ \underline{096} \times 10^{-3}$
HP 48G	$-8.8513\ 0929\ 040 \times 10^{-3}$
Sharp EL-5020	$-8.8513\ \underline{0915\ 4} \times 10^{-3}$
MATLAB 6.1	$-8.8513\ 0929\ 0403\ 876 \times 10^{-3}$
WATFOR-77	$-8.8513\ 0929\ 0403\ \underline{880} \times 10^{-3}$
FTN 90	$-8.8513\ 0929\ 0403\ 876 \times 10^{-3}$
QBasic	$-8.8513\ 0929\ 0403\ 876 \times 10^{-3}$

Table 2.5. *Exponentiation test. No entry for last column means same value as previous column.*

Machine	2.5^{125}	$\exp(125 \log(2.5))$
Exact	$5.5271\ 4787\ 5260\ 4446 \times 10^{49}$	$5.5271\ 4787\ 5260\ 4446 \times 10^{49}$
Casio fx-140	$5.5271\ \underline{477} \times 10^{49}$	$5.5271\ \underline{463} \times 10^{49}$
Casio fx-992VB	$5.5271\ 4787\ 526 \times 10^{49}$	
HP 48G	$5.5271\ 4787\ 526 \times 10^{49}$	$5.5271\ 4787\ \underline{377} \times 10^{49}$
Sharp EL-5020	$5.5271\ 4787\ \underline{3} \times 10^{49}$	$5.5271\ \underline{4796\ 2} \times 10^{49}$
MATLAB 6.1	$5.5271\ 4787\ 5260\ \underline{444} \times 10^{49}$	$5.5271\ 4787\ 5260\ \underline{459} \times 10^{49}$
WATFOR-77	$5.5271\ 4787\ 5260\ \underline{450} \times 10^{49}$	$5.5271\ 4787\ 5260\ \underline{460} \times 10^{49}$
FTN 90	$5.5271\ 4787\ 5260\ 445 \times 10^{49}$	$5.5271\ 4787\ 5260\ \underline{459} \times 10^{49}$
QBasic	$5.5271\ 4787\ 5260\ \underline{444} \times 10^{49}$	

$\exp(w)\exp(\Delta w) \approx \exp(w)(1 + \Delta w)$, so $\Delta z/z \approx \Delta w$. In other words, the relative error of z depends on the absolute error of w and hence on the size of w . To obtain z correct to within a few ulps it is necessary to use extra precision in calculating the logarithm and exponential [256, 1980, Chap. 7].

- (Karpinski [715, 1985]) A simple test for the presence of a guard digit on a pocket calculator is to evaluate the expressions

$$9/27 * 3 - 1, \quad 9/27 * 3 - 0.5 - 0.5,$$

which are given in a form that can be typed directly into most four-function calculators. If the results are equal then a guard digit is present. Otherwise there is probably no guard digit (we cannot be completely sure from this simple test). To test for a guard digit on a computer it is best to run one of the diagnostic codes described in §27.5.

2.12. Notes and References

The classic reference on floating point arithmetic, and on all aspects of rounding error analysis, is Wilkinson's *Rounding Errors in Algebraic Processes* (REAP) [1232,

1963]. Wilkinson was uniquely qualified to write such a book, for not only was he the leading expert in rounding error analysis, but he was one of the architects and builders of the Automatic Computing Engine (ACE) at the National Physical Laboratory [1226, 1954]. The Pilot (prototype) ACE first operated in May 1950, and an engineered version was later sold commercially as the DEUCE Computer by the English Electric Company. Wilkinson and his colleagues were probably the first to write subroutines for floating point arithmetic, and this enabled them to accumulate practical experience of floating point arithmetic much earlier than anyone else [395, 1976], [1243, 1980].

In REAP, Wilkinson gives equal consideration to fixed point and floating point arithmetic. In fixed point arithmetic, all numbers are constrained to lie in a range such as $[-1, 1]$, as if the exponent were frozen in the floating point representation (2.1). Preliminary analysis and the introduction of scale factors during the computation are needed to keep numbers in the permitted range. We consider only floating point arithmetic in this book. REAP, together with Wilkinson's second book, *The Algebraic Eigenvalue Problem* (AEP) [1233, 1965], has been immensely influential in the areas of floating point arithmetic and rounding error analysis.

Wilkinson's books were preceded by the paper "Error Analysis of Floating-Point Computation" [1228, 1960], in which he presents the model (2.4) for floating point arithmetic and applies the model to several algorithms for the eigenvalue problem. This paper has hardly dated and is still well worth reading.

Another classic book devoted entirely to floating point arithmetic is Sterbenz's *Floating-Point Computation* [1062, 1974]. It contains a thorough treatment of low-level details of floating point arithmetic, with particular reference to IBM 360 and IBM 7090 machines. It also contains a good chapter on rounding error analysis and an interesting collection of exercises. R. W. Hamming has said of this book, "Nobody should ever have to know that much about floating-point arithmetic. But I'm afraid sometimes you might" [942, 1988]. Although Sterbenz's book is now dated in some respects, it remains a useful reference.

A third important reference on floating point arithmetic is Knuth's *Seminumerical Algorithms* [744, 1998, Sec. 4.2], from his *Art of Computer Programming* series. Knuth's lucid presentation includes historical comments and challenging exercises (with solutions).

The first analysis of floating point arithmetic was given by Samelson and Bauer [1009, 1953]. Later in the same decade Carr [205, 1959] gave a detailed discussion of error bounds for the basic arithmetic operations.

An up-to-date and very readable reference on floating point arithmetic is the survey paper by Goldberg [496, 1991], which includes a detailed discussion of IEEE arithmetic. A less mathematical, more hardware-oriented discussion can be found in the appendix "Computer Arithmetic" written by Goldberg that appears in the book on computer architecture by Hennessy and Patterson [562, 1996].

A fascinating historical perspective on the development of computer floating point arithmetics, including background to the development of the IEEE standard, can be found in the textbook by Patterson and Hennessy [929, 1998, §4.12].

The idea of representing floating point numbers in the form (2.1) is found, for example, in the work of Forsythe [428, 1969], Matula [830, 1970], and Dekker [302, 1971].

An alternative definition of $fl(x)$ is the nearest $y \in G$ satisfying $|y| \leq |x|$. This operation is called *chopping*, and does not satisfy our definition of rounding. Chopped arithmetic is used in the IBM/370 floating point system.

The difference between chopping and rounding (to nearest) is highlighted by a discrepancy in the index of the Vancouver Stock Exchange in the early 1980s [963, 1983]. The exchange established an index in January 1982, with the initial value of 1000. By November 1983 the index had been hitting lows in the 520s, despite the exchange apparently performing well. The index was recorded to three decimal places and it was discovered that the computer program calculating the index was chopping instead of rounding to produce the final value. Since the index was recalculated thousands of times a day, each time with a nonpositive final error, the bias introduced by chopping became significant. Upon recalculation with rounding the index almost doubled!

When there is a tie in rounding, two possible strategies are to round to the number with an even last digit and to round to the one with an odd last digit. Both are stable forms of rounding in the sense that

$$fl(((x + y) - y) + y) = fl((x + y) - y),$$

as shown by Reiser and Knuth [982, 1975], [744, 1998, Sec. 4.2.2, Thm. D]. For other rules, such as round away from zero, repeated subtraction and addition of the same number can yield an increasing sequence, a phenomenon known as *drift*. For bases 2 and 10 rounding to even is preferred to rounding to odd. After rounding to even a subsequent rounding to one less place does not involve a tie. Thus we have the rounding sequence 2.445, 2.44, 2.4 with round to even, but 2.445, 2.45, 2.5 with round to odd. For base 2, round to even causes computations to produce integers more often [706, 1979] as a consequence of producing a zero least significant bit. Rounding to even in the case of ties seems to have first been suggested by Scarborough in the first edition (1930) of [1014, 1950]. Hunt [651, 1997] notes that in the presentation of meteorological temperature measurements round to odd is used as a tie-breaking strategy, and he comments that this may be to avoid the temperatures 0.5°C and 32.5°F being rounding down and the incorrect impression being given that it is freezing.

Predictions based on the growth in the size of mathematical models solved as the memory and speed of computers increase suggest that floating point arithmetic with unit roundoff $u \approx 10^{-32}$ will be needed for some applications on future supercomputers [57, 1989].

The model (2.4) does not fully describe any floating point arithmetic. It is merely a tool for error analysis—one that has been remarkably successful in view of our current understanding of the numerical behaviour of algorithms. There have been various attempts to devise formal models of floating point arithmetic, by specifying sets of axioms in terms of which error analysis can be performed. Some attempts are discussed in §27.7.4. No model yet proposed has been truly successful. Priest [955, 1992] conjectures that the task of “encapsulating all that we wish to know about floating point arithmetic in a single set of axioms” is impossible, and he gives some motivation for this conjecture.

Under the model (2.4), floating point arithmetic is not associative with respect to any of the four basic operations: $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$, $\text{op} = +, -, *, /$,

where $a \boxed{\text{op}} b := fl(a \text{ op } b)$. Nevertheless, floating point arithmetic enjoys some algebraic structure, and it is possible to carry out error analysis in the “ $\boxed{\text{op}}$ algebra”. Fortunately, it was recognized by Wilkinson and others in the 1950s that this laboured approach is unnecessarily complicated, and that it is much better to work with the exact equations satisfied by the computed quantities. As Parlett [925, 1990] notes, though, “There have appeared a number of ponderous tomes that do manage to abstract the computer’s numbers into a formal structure and burden us with more jargon.”

A draft proposal of IEEE Standard 754 is defined and described in [657, 1981]. That article, together with others in the same issue of the journal *Computer*, provides a very readable description of IEEE arithmetic. In particular, an excellent discussion of gradual underflow is given by Coonen [270, 1981]. A draft proposal of IEEE Standard 854 is presented, with discussion, in [253, 1984].

W. M. Kahan of the University of California at Berkeley received the 1989 ACM Turing Award for his contributions to computer architecture and numerical analysis, and in particular for his work on IEEE floating point arithmetic standards 754 and 854. For interesting comments by Kahan on the development of IEEE arithmetic and other floating point matters, see [1029, 1998], [1251, 1997],

An interesting examination of the implications of the IEEE standard for high-level languages such as Fortran is given by Fateman [405, 1982]. Topics discussed include trap handling and how to exploit NaNs. For an overview of hardware implementations of IEEE arithmetic, and software support for it, see Cody [251, 1988].

Producing a fast and correct implementation of IEEE arithmetic is a difficult task. Correctness is especially important for a microprocessor (as opposed to a software) implementation, because of the logistical difficulties of correcting errors when they are found. In late 1994, much publicity was generated by the discovery of a flaw in the floating point divide instruction of Intel’s Pentium chip. Because of some missing entries in a lookup table on the chip, the FPDIV instruction could give as few as four correct significant decimal digits for double precision floating point arguments with certain special bit patterns [1036, 1994], [380, 1997]. The flaw had been discovered by Intel in the summer of 1994 during ongoing testing of the Pentium processor, but it had not been publically announced. In October 1994, a mathematician doing research into prime numbers independently discovered the flaw and reported it to the user community. Largely because of the way in which Intel responded to the discovery of the flaw, the story was reported in national newspapers (e.g., the *New York Times* [816, 1994]) and generated voluminous discussion on Internet newsgroups (notably `comp.sys.intel`). Intel corrected the bug in 1994 and, several weeks after the bug was first reported, offered to replace faulty chips. For a very readable account of the Pentium FPDIV bug story, see Moler [866, 1995]. To emphasize that bugs in implementations of floating point arithmetic are far from rare, we mention that the Calculator application in Microsoft Windows 3.1 evaluates $fl(2.01 - 2.00) = 0.0$.

Computer chip designs can be tested in two main ways: by software simulations and by applying formal verification techniques. Formal verification aims to prove mathematically that the chip design is correct, and this approach has been in use by chip manufacturers for some time [491, 1995]. Some relevant references are

Barrett [79, 1989] for the Inmos T800 (or Shepherd and Wilson [1037, 1989] for a more informal overview), and Moore, Lynch, and Kaufmann [871, 1998] and Russinoff [1003, 1998] for AMD processors.

For low-level details of computer arithmetic several textbooks are available. We mention only the recent and very thorough book by Parhami [921, 2000].

The floating point operation op ($\text{op} = +, -, *, \text{ or } /$) is *monotonic* if $fl(a \text{ op } b) \leq fl(c \text{ op } d)$ whenever a, b, c , and d are floating point numbers for which $a \text{ op } b \leq c \text{ op } d$ and neither $fl(a \text{ op } b)$ nor $fl(c \text{ op } d)$ overflows. IEEE arithmetic is monotonic, as is any correctly rounded arithmetic. Monotonic arithmetic is important in the bisection algorithm for finding the eigenvalues of a symmetric tridiagonal matrix; see Demmel, Dhillon, and Ren [320, 1995], who give rigorous correctness proofs of some bisection implementations in floating point arithmetic. Ferguson and Brightman [410, 1991] derive conditions that ensure that an approximation to a monotonic function preserves the monotonicity on a set of floating point numbers.

On computers of the 1950s, (fixed point) multiplication was slower than (fixed point) addition by up to an order of magnitude [773, 1980, Apps. 2, 3]. For most modern computers it is a rule of thumb that a floating point addition and multiplication take about the same amount of time, while a floating point division is 2–10 times slower, and a square root operation (in hardware) is 1–2 times slower than a division.

During the design of the IBM 7030, Sweeney [1116, 1965] collected statistics on the floating point additions carried out by selected application programs on an IBM 704. He found that 11% of all instructions traced were floating point additions. Details were recorded of the shifting needed to align floating point numbers prior to addition, and the results were used in the design of the shifter on the IBM 7030.

The word *bit*, meaning binary digit, first appeared in print in a 1948 paper of Claude E. Shannon, but the term was apparently coined by John W. Tukey [1160, 1984]. The word *byte*, meaning a group of (usually eight) bits, did not appear in print until 1959 [173, 1981].

The earliest reference we know for Theorem 2.5 is Sterbenz [1062, 1974, Thm. 4.3.1]. Theorem 2.4 is due to Ferguson [409, 1995], who proves a more general version of the theorem that allows for trailing zero digits in x and y . A variation in which the condition is $0 \leq y \leq x \leq y + \beta^e$, where $e = \min\{j : \beta^j \geq y\}$, is stated by Ziv [1285, 1991] and can be proved in a similar way.

For more on the choice of base, see Cody [255, 1973] and Kuki and Cody [754, 1973]. Buchholz's paper *Fingers or Fists?* [172, 1959] on binary versus decimal representation of numbers on a computer deserves mention for its clever title, though the content is only of historical interest.

The model (2.4) ignores the possibility of underflow and overflow. To take underflow into account the model must be modified to

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta) + \eta, \quad \text{op} = +, -, *, /. \quad (2.8)$$

As before, $|\delta| \leq u$. If underflow is gradual, as in IEEE arithmetic, then $|\eta| \leq \frac{1}{2}\beta^{e_{\min}-t} = \lambda u$, which is half the spacing between the subnormal numbers ($\lambda = \beta^{e_{\min}-1}$ is the smallest positive normalized floating point number); if underflows

are flushed to zero then $|\eta| \leq \lambda$. Only one of δ and η is nonzero: δ if no underflow occurs, otherwise η . With gradual underflow the *absolute error* of an underflowed result is no greater than the smallest (bound for the) absolute error that arises from an operation $fl(x \text{ op } y)$ in which the arguments and result are normalized. Moreover, with gradual underflow we can take $\eta = 0$ for $\text{op} = +, -$ (see Problem 2.19). For more details, and a thorough discussion of how error analysis of standard algorithms is affected by using the model (2.8), see the perceptive paper by Demmel [308, 1984]. Another relevant reference is Neumaier [884, 1985].

Hauser [553, 1996] gives a thorough and very readable treatment of exception handling, covering underflow, overflow, indeterminate or undefined cases, and their support in software.

An important problem not considered in this chapter is the conversion of numbers between decimal and binary representations. These conversions are needed whenever numbers are read into a computer or printed out. They tend to be taken for granted, but if not done carefully they can lead to puzzling behaviour, such as a number read in as 0.1 being printed out as 0.099...9. To be precise, the problems of interest are (a) convert a number represented in decimal notation to the best binary floating point representation of a given precision, and (b) given a binary floating point number, print a correctly rounded decimal representation, either to a given number of significant digits or to the smallest number of significant digits that allows the number to be re-read without loss of accuracy. Algorithms for solving these problems are given by Clinger [247, 1990] and Steele and White [1060, 1990]; Gay [470, 1990] gives some improvements to the algorithms and C code implementing them. Precise requirements for binary–decimal conversion are specified in the IEEE arithmetic standard. A program for testing the correctness of binary–decimal conversion routines is described by Paxson [930, 1991]. Early references on base conversion are Goldberg [497, 1967] and Matula [829, 1968], [830, 1970]. It is interesting to note that, in Fortran or C, where the output format for a “print” statement can be precisely specified, most compilers will, for an (in)appropriate choice of format, print a decimal string that contains many more significant digits than are determined by the floating point number whose value is being represented.

Other authors who have analysed various aspects of floating (and fixed) point arithmetic include Diamond [339, 1978], Urabe [1174, 1968], and Feldstein, Goodman, and co-authors [510, 1975], [407, 1982], [511, 1985], [408, 1986]. For a survey of computer arithmetic up until 1976 that includes a number of references not given here, see Garner [460, 1976].

Problems

*The exercise had warmed my blood,
and I was beginning to enjoy myself amazingly.*

— JOHN BUCHAN, *The Thirty-Nine Steps* (1915)

2.1. How many normalized numbers and how many subnormal numbers are there in the system F defined in (2.1) with $e_{\min} \leq e \leq e_{\max}$? What are the figures for IEEE single and double precision (base 2)?

2.2. Prove Lemma 2.1.

2.3. In IEEE arithmetic how many double precision numbers are there between any two adjacent nonzero single precision numbers?

2.4. Prove Theorem 2.3.

2.5. Show that

$$0.1 = \sum_{i=1}^{\infty} (2^{-4i} + 2^{-4i-1})$$

and deduce that 0.1 has the base 2 representation $0.000\overline{1100}$ (repeating last 4 bits). Let $\hat{x} = fl(0.1)$ be the rounded version of 0.1 obtained in binary IEEE single precision arithmetic ($u = 2^{-24}$). Show that $(x - \hat{x})/x = -\frac{1}{4}u$.

2.6. What is the largest integer p such that all integers in the interval $[-p, p]$ are exactly representable in IEEE double precision arithmetic? What is the corresponding p for IEEE single precision arithmetic?

2.7. Which of the following statements is true in IEEE arithmetic, assuming that a and b are normalized floating point numbers and that no exception occurs in the stated operations?

1. $fl(a \text{ op } b) = fl(b \text{ op } a)$, $\text{op} = +, *$.
2. $fl(b - a) = -fl(a - b)$.
3. $fl(a + a) = fl(2 * a)$.
4. $fl(0.5 * a) = fl(a/2)$.
5. $fl((a + b) + c) = fl(a + (b + c))$.
6. $a \leq fl((a + b)/2) \leq b$, given that $a \leq b$.

2.8. Show that the inequalities $a \leq fl((a + b)/2) \leq b$, where a and b are floating point numbers with $a \leq b$, can be violated in base 10 arithmetic. Show that $a \leq fl(a + (b - a)/2) \leq b$ in base β arithmetic, for any β .

2.9. What is the result of the computation $\sqrt{1 - 2^{-53}}$ in IEEE double precision arithmetic, with and without double rounding from an extended format with a 64-bit significand?

2.10. A theorem of Kahan [496, 1991, Thm. 7] says that if $\beta = 2$ and the arithmetic rounds as specified in the IEEE standard, then for integers m and n with $|m| < 2^{t-1}$ and $n = 2^i + 2^j$ (some i, j), $fl((m/n) * n) = m$. Thus, for example, $fl((1/3) * 3) = 1$ (even though $fl(1/3) \neq 1/3$). The sequence of allowable n begins 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 16, 17, 18, 20, so Kahan's theorem covers many common cases. Test the theorem on your computer.

2.11. Investigate the leading significant digit distribution for numbers obtained as follows.

1. k^n , $n = 0:1000$ for $k = 2$ and 3.
2. $n!$, $n = 1:1000$.

3. The eigenvalues of a random symmetric matrix.
4. Physical constants from published tables.
5. From the front page of the *London Times* or the *New York Times*.

(Note that in writing a program for the first case you can form the powers of 2 or 3 in order, following each multiplication by a division by 10, as necessary, to keep the result in the range $[1, 10]$. Similarly for the second case.)

2.12. (Edelman [379, 1994]) Let x be a floating point number in IEEE double precision arithmetic satisfying $1 \leq x < 2$. Show that $fl(x * (1/x))$ is either 1 or $1 - \epsilon/2$, where $\epsilon = 2^{-52}$ (the machine epsilon).

2.13. (Edelman [379, 1994]) Consider IEEE double precision arithmetic. Find the smallest positive integer j such that $fl(x * (1/x)) \neq 1$, where $x = 1 + j\epsilon$, with $\epsilon = 2^{-52}$ (the machine epsilon).

2.14. Kahan has stated that “an (over-)estimate of u can be obtained for almost any machine by computing $|3 \times (4/3 - 1) - 1|$ using rounded floating-point for every operation”. Test this estimate against u on any machines available to you.

2.15. What is 0^0 in IEEE arithmetic?

2.16. Evaluate these expressions in any IEEE arithmetic environment available to you. Are the values returned what you would expect? (None of the results is specified by the IEEE standard.)

1. 1^∞ .
2. 2^∞ .
3. $\exp(\infty)$, $\exp(-\infty)$.
4. $\text{sign}(\text{NaN})$, $\text{sign}(-\text{NaN})$.
5. NaN^0 .
6. ∞^0 .
7. 1^{NaN} .
8. $\log(\infty)$, $\log(-\infty)$, $\log(0)$.

2.17. Let $x_{\max}$ denote the largest representable number in IEEE single or double precision arithmetic. In what circumstances does $2x_{\max}$ not overflow?

2.18. Can Theorem 2.4 be strengthened to say that $fl(x - y)$ is computed exactly whenever the exponents of $x \geq 0$ and $y \geq 0$ differ by at most 1?

2.19. (Hauser [553, 1996]) Show that with gradual underflow, if x and y are floating point numbers and $fl(x \pm y)$ underflows then $fl(x \pm y) = x \pm y$.

2.20. Two requirements that we might ask of a routine for computing $\sqrt{x}$ in floating point arithmetic are that the identities $\sqrt{x^2} = |x|$ and $(\sqrt{x})^2 = |x|$ be satisfied. Which, if either, of these is a reasonable requirement?

2.21. Are there any floating point values of x and y (excepting values both 0, or so huge or tiny as to cause overflow or underflow) for which the computed value of $x/\sqrt{x^2 + y^2}$ exceeds 1?

2.22. (Kahan) A natural way to compute the maximum of two numbers x and y is with the code

```
% max(x, y)
if x > y then
    max = x
else
    max = y
end
```

Does this code always produce the expected answer in IEEE arithmetic?

2.23. Prove that Kahan's formula (2.7) computes the area of a triangle accurately. (Hint: you will need one invocation of Theorem 2.5.)

2.24. (Kahan) Describe the result of the computation $y = (x + x) - x$ on a binary machine with a guard digit and one without a guard digit.

2.25. (Kahan) Let $f(x) = (((x - 0.5) + x) - 0.5) + x$. Show that if f is evaluated as shown in single or double precision binary IEEE arithmetic then $f(x) \neq 0$ for all floating point numbers x .

2.26. Show that if x and y are floating point numbers and $\hat{a} = fl(xy)$ and $\hat{b} = fl(xy - \hat{a})$ are computed using fused multiply-add operations, then $\hat{a} + \hat{b} \equiv xy$.

2.27. (Kahan) Show that with the use of a fused multiply-add operation the algorithm

```
w = bc
e = w - bc
x = (ad - w) + e
```

computes $x = \det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right)$ with high relative accuracy.

2.28. Suppose we have an iterative algorithm for computing $z = x/y$. Derive a convergence test that terminates the iteration (only) when full accuracy has been achieved. Assume the use of IEEE arithmetic with gradual underflow (use (2.8)).

Chapter 3

Basics

*A method of inverting the problem of round-off error is proposed
which we plan to employ in other contexts and
which suggests that it may be unwise to
separate the estimation of round-off error
from that due to observation and truncation.*

— WALLACE J. GIVENS, *Numerical Computation of the
Characteristic Values of a Real Symmetric Matrix* (1954)

The enjoyment of one's tools is an essential ingredient of successful work.

— DONALD E. KNUTH, *The Art of Computer Programming,
Volume 2, Seminumerical Algorithms* (1998)

*The subject of propagation of rounding error,
while of undisputed importance in numerical analysis,
is notorious for the difficulties which it presents when it is to be
taught in the classroom in such a manner that the student is
neither insulted by lack of mathematical content
nor bored by lack of transparency and clarity.*

— PETER HENRICI, *A Model for the Propagation
of Rounding Error in Floating Arithmetic* (1980)

*The two main classes of rounding error analysis are not,
as my audience might imagine, 'backwards' and 'forwards',
but rather 'one's own' and 'other people's'.
One's own is, of course, a model of lucidity;
that of others serves only to obscure the
essential simplicity of the matter in hand.*

— J. H. WILKINSON, *The State of the Art in Error Analysis* (1985)

Having defined a model for floating point arithmetic in the last chapter, we now apply the model to some basic matrix computations, beginning with inner products. This first application is simple enough to permit a short analysis, yet rich enough to illustrate the ideas of forward and backward error. It also raises the thorny question of what is the best notation to use in an error analysis. We introduce the “ γ_n ” notation, which we use widely, though not exclusively, in the book. The inner product analysis leads immediately to results for matrix–vector and matrix–matrix multiplication.

Also in this chapter we determine a model for rounding errors in complex arithmetic, derive some miscellaneous results of use in later chapters, and give a high-level overview of forward error analysis and backward error analysis.

3.1. Inner and Outer Products

Consider the inner product $s_n = x^T y$, where $x, y \in \mathbb{R}^n$. Since the order of evaluation of $s_n = x_1 y_1 + \cdots + x_n y_n$ affects the analysis (but not the final error *bounds*), we will assume that the evaluation is from left to right. (The effect of particular orderings is discussed in detail in Chapter 4, which considers the special case of summation.) In the following analysis, and throughout the book, a hat denotes a computed quantity.

Let $s_i = x_1 y_1 + \cdots + x_i y_i$ denote the i th partial sum. Using the standard model (2.4), we have

$$\begin{aligned}\hat{s}_1 &= fl(x_1 y_1) = x_1 y_1 (1 + \delta_1), \\ \hat{s}_2 &= fl(\hat{s}_1 + x_2 y_2) = (\hat{s}_1 + x_2 y_2 (1 + \delta_2))(1 + \delta_3) \\ &= (x_1 y_1 (1 + \delta_1) + x_2 y_2 (1 + \delta_2))(1 + \delta_3) \\ &= x_1 y_1 (1 + \delta_1)(1 + \delta_3) + x_2 y_2 (1 + \delta_2)(1 + \delta_3),\end{aligned}\tag{3.1}$$

where $|\delta_i| \leq u$, $i = 1:3$. For our purposes it is not necessary to distinguish between the different δ_i terms, so to simplify the expressions let us drop the subscripts on the δ_i and write $1 + \delta_i \equiv 1 \pm \delta$. Then

$$\begin{aligned}\hat{s}_3 &= fl(\hat{s}_2 + x_3 y_3) = (\hat{s}_2 + x_3 y_3 (1 \pm \delta))(1 \pm \delta) \\ &= (x_1 y_1 (1 \pm \delta)^2 + x_2 y_2 (1 \pm \delta)^2 + x_3 y_3 (1 \pm \delta))(1 \pm \delta) \\ &= x_1 y_1 (1 \pm \delta)^3 + x_2 y_2 (1 \pm \delta)^3 + x_3 y_3 (1 \pm \delta)^2.\end{aligned}$$

The pattern is clear. Overall, we have

$$\hat{s}_n = x_1 y_1 (1 \pm \delta)^n + x_2 y_2 (1 \pm \delta)^n + x_3 y_3 (1 \pm \delta)^{n-1} + \cdots + x_n y_n (1 \pm \delta)^2. \tag{3.2}$$

There are various ways to simplify this expression. A particularly elegant way is to use the following result.

Lemma 3.1. *If $|\delta_i| \leq u$ and $\rho_i = \pm 1$ for $i = 1:n$, and $nu < 1$, then*

$$\prod_{i=1}^n (1 + \delta_i)^{\rho_i} = 1 + \theta_n,$$

where

$$|\theta_n| \leq \frac{nu}{1 - nu} =: \gamma_n.$$

Proof. See Problem 3.1. $\square$

The θ_n and γ_n notation will be used throughout this book. Whenever we write γ_n there is an implicit assumption that $nu < 1$, which is true in virtually any circumstance that might arise with IEEE single or double precision arithmetic.

Applying the lemma to (3.2) we obtain

$$\hat{s}_n = x_1 y_1 (1 + \theta_n) + x_2 y_2 (1 + \theta'_n) + x_3 y_3 (1 + \theta_{n-1}) + \cdots + x_n y_n (1 + \theta_2). \quad (3.3)$$

This is a backward error result and may be interpreted as follows: the computed inner product is the exact one for a perturbed set of data $x_1, \dots, x_n$, $y_1(1 + \theta_n), y_2(1 + \theta'_n), \dots, y_n(1 + \theta_2)$ (alternatively, we could perturb the x_i and leave the y_i alone). Each relative perturbation is certainly bounded by $\gamma_n = nu/(1 - nu)$, so the perturbations are tiny.

The result (3.3) applies to one particular order of evaluation. It is easy to see that for any order of evaluation we have, using vector notation,

$$fl(x^T y) = (x + \Delta x)^T y = x^T (y + \Delta y), \quad |\Delta x| \leq \gamma_n |x|, \quad |\Delta y| \leq \gamma_n |y|, \quad (3.4)$$

where $|x|$ denotes the vector with elements $|x_i|$ and inequalities between vectors (and, later, matrices) hold componentwise.

A forward error bound follows immediately from (3.4):

$$|x^T y - fl(x^T y)| \leq \gamma_n \sum_{i=1}^n |x_i y_i| = \gamma_n |x|^T |y|. \quad (3.5)$$

If $y = x$, so that we are forming a sum of squares $x^T x$, this result shows that high relative accuracy is obtained. However, in general, high relative accuracy is not guaranteed if $|x^T y| \ll |x|^T |y|$.

It is easy to see that precisely the same results (3.3)–(3.5) hold when we use the no-guard-digit rounding error model (2.6). For example, expression (3.1) becomes $\hat{s}_2 = x_1 y_1 (1 + \delta_1)(1 + \delta_3) + x_2 y_2 (1 + \delta_2)(1 + \delta_4)$, where δ_4 has replaced a second occurrence of δ_3 , but this has no effect on the error bounds.

It is worth emphasizing that the constants in the bounds above can be reduced by focusing on particular implementations of an inner product. For example, if $n = 2m$ and we compute

$$\begin{aligned} s_1 &= x(1:m)^T y(1:m) \\ s_2 &= x(m+1:n)^T y(m+1:n) \\ s_n &= s_1 + s_2 \end{aligned}$$

then $|s_n - \widehat{s}_n| \leq \gamma_{n/2+1} |x^T| |y|$. By accumulating the inner product in two pieces we have almost halved the error bound. This idea can be generalized by breaking the inner product into k pieces, with each mini inner product of length n/k being evaluated separately and the results summed. The error bound is now $\gamma_{n/k+1} |x^T| |y|$, which achieves its minimal value of $\gamma_{2\sqrt{n}-1} |x^T| |y|$ for $k = \sqrt{n}$ (or, rather, we should take k to be the nearest integer to $\sqrt{n}$). But it is possible to do even better by using pairwise summation of the products $x_i y_i$ (this method is described in §4.1). With pairwise summation, the error bound becomes

$$|s_n - \widehat{s}_n| \leq \gamma_{\lceil \log_2 n \rceil + 1} |x^T| |y|.$$

Since many of the error analyses in this book are built upon the error analysis of inner products, it follows that the constants in these higher-level bounds can also be reduced by using one of these nonstandard inner product implementations. The main significance of this observation is that we should not attach too much significance to the precise values of the constants in error bounds.

Inner products are amenable to being calculated in extended precision. If the working precision involves a t -digit significand then the product of two floating point numbers has a significand of $2t - 1$ or $2t$ digits and so can be represented exactly with a $2t$ -digit significand. Some computers always form the $2t$ -digit product before rounding to t digits, thus allowing an inner product to be accumulated at $2t$ -digit precision at little or no extra cost, prior to a final rounding.

The extended precision computation can be expressed as $fl(fl_e(x^T y))$, where fl_e denotes computations with unit roundoff u_e ($u_e < u$). Defining $\widehat{s}_n = fl_e(x^T y)$, the analysis above holds with u replaced by u_e in (3.3) (and with the subscripts on the θ_i reduced by 1 if the multiplications are done exactly). For the final rounding we have

$$fl(fl_e(x^T y)) = \widehat{s}_n(1 + \delta), \quad |\delta| \leq u,$$

and so, overall,

$$|x^T y - fl(fl_e(x^T y))| \leq u |x^T y| + \frac{nu_e}{1 - nu_e} (1 + u) |x^T| |y|.$$

Hence, as long as $nu_e |x^T| |y| \leq u |x^T y|$, the computed inner product is about as good as the rounded exact inner product. The effect of using extended precision inner products in an algorithm is typically to enable a factor n to be removed from the overall error bound.

Because extended precision inner product calculations are machine dependent it has always been difficult to write portable programs that use them, but the widespread adoption of IEEE arithmetic and the development of the Extended and Mixed Precision BLAS (see §27.10) make portability much easier to achieve now. Most modern numerical codes (for example, those in EISPACK, LINPACK, and LAPACK) do not use extended precision inner products. One process in which these more accurate products are needed is the traditional formulation of iterative refinement, in which the aim is to improve the accuracy of the computed solution to a linear system (see Chapter 12).

We have seen that computation of an inner product is a backward stable process. What can be said for an outer product $A = xy^T$, where $x, y \in \mathbb{R}^n$? The

analysis is easy. We have $\hat{a}_{ij} = x_i y_j (1 + \delta_{ij})$, $|\delta_{ij}| \leq u$, so

$$\hat{A} = xy^T + \Delta, \quad |\Delta| \leq u|xy^T|. \quad (3.6)$$

This is a satisfying result, but the computation is not backward stable. In fact, $\hat{A} = (x + \Delta x)(y + \Delta y)^T$ does not hold for *any* Δx and Δy (let alone a small Δx and Δy) because $\hat{A}$ is not in general a rank-1 matrix.

This distinction between inner and outer products illustrates a general principle: a numerical process is more likely to be backward stable when the number of outputs is small compared with the number of inputs, so that there is an abundance of data onto which to “throw the backward error”. An inner product has the minimum number of outputs for its $2n$ scalar inputs, and an outer product has the maximum number (among standard linear algebra operations).

3.2. The Purpose of Rounding Error Analysis

Before embarking on further error analyses, it is worthwhile to consider what a rounding error analysis is designed to achieve. The purpose is to show the existence of an a priori bound for some appropriate measure of the effects of rounding errors on an algorithm. Whether a bound exists is the most important question. Ideally, the bound is small for all choices of problem data. If not, it should reveal features of the algorithm that characterize any potential instability and thereby suggest how the instability can be cured or avoided. For some unstable algorithms, however, there is no useful error bound. (For example, no bound is known for the loss of orthogonality due to rounding error in the classical Gram–Schmidt method; see §19.8.)

The constant terms in an error bound (those depending only on the problem dimensions) are the least important parts of it. As discussed in §2.8, the constants usually cause the bound to overestimate the actual error by orders of magnitude. It is not worth spending much effort to minimize constants because the achievable improvements are usually insignificant.

It *is* worth spending effort, though, to put error bounds in a concise, easily interpreted form. Part of the secret is in the choice of notation, which we discuss in §3.4, including the question of what symbols to choose for variables (see the discussion in Higham [611, 1998, §3.5]).

If sharp error estimates or bounds are desired they should be computed a posteriori, so that the actual rounding errors that occur can be taken into account. One approach is to use running error analysis, described in the next section. Other possibilities are to compute the backward error explicitly, as can be done for linear equation and least squares problems (see §§7.1, 7.2, 20.7, and 21.2), or to apply iterative refinement to obtain a correction that approximates the forward error (see Chapter 12).

3.3. Running Error Analysis

The forward error bound (3.5) is an a priori bound that does not depend on the actual rounding errors committed. We can derive a sharper, a posteriori bound by reworking the analysis. The inner product evaluation may be expressed as

```

 $s_0 = 0$ 
for  $i = 1:n$ 
     $s_i = s_{i-1} + x_i y_i$ 
end

```

Write the computed partial sums as $\widehat{s}_i = s_i + e_i$ and let $\widehat{z}_i := fl(x_i y_i)$. We have, using (2.5),

$$\widehat{z}_i = \frac{x_i y_i}{1 + \delta_i}, \quad |\delta_i| \leq u \quad \Rightarrow \quad \widehat{z}_i = x_i y_i - \delta_i \widehat{z}_i.$$

Similarly, $(1 + \epsilon_i)\widehat{s}_i = \widehat{s}_{i-1} + \widehat{z}_i$, where $|\epsilon_i| \leq u$, or

$$s_i + e_i + \epsilon_i \widehat{s}_i = s_{i-1} + e_{i-1} + x_i y_i - \delta_i \widehat{z}_i.$$

Hence $e_i = e_{i-1} - \epsilon_i \widehat{s}_i - \delta_i \widehat{z}_i$, which gives

$$|e_i| \leq |e_{i-1}| + u|\widehat{s}_i| + u|\widehat{z}_i|.$$

Since $e_0 = 0$, we have $|e_n| \leq u\mu_n$, where

$$\mu_i = \mu_{i-1} + |\widehat{s}_i| + |\widehat{z}_i|, \quad \mu_0 = 0.$$

Algorithm 3.2. Given $x, y \in \mathbb{R}^n$ this algorithm computes $s = fl(x^T y)$ and a number μ such that $|s - x^T y| \leq \mu$.

```

 $s = 0$ 
 $\mu = 0$ 
for  $i = 1:n$ 
     $z = x_i y_i$ 
     $s = s + z$ 
     $\mu = \mu + |s| + |z|$ 
end
 $\mu = \mu * u$ 

```

This type of computation, where an error bound is computed concurrently with the solution, is called *running error analysis*. The underlying idea is simple: we use the modified form (2.5) of the standard rounding error model to write

$$|x \text{ op } y - fl(x \text{ op } y)| \leq u|fl(x \text{ op } y)|,$$

which gives a bound for the error in $x \text{ op } y$ that is easily computed, since $fl(x \text{ op } y)$ is stored on the computer. Key features of a running error analysis are that few inequalities are involved in the derivation of the bound and that the actual computed intermediate quantities are used, enabling advantage to be taken of cancellation and zero operands. A running error bound is, therefore, usually smaller than an a priori one.

There are, of course, rounding errors in the computation of the running error bound, but their effect is negligible for $nu \ll 1$ (we do not need many correct significant digits in an error bound).

Running error analysis is a somewhat neglected practice nowadays, but it was widely used by Wilkinson in the early years of computing. It is applicable to almost any numerical algorithm. Wilkinson explains [1245, 1986]:

When doing running error analysis on the ACE at no time did I write down these expressions. I merely took an existing program (without any error analysis) and modified it as follows. As each arithmetic operation was performed I added the absolute value of the computed result (or of the dividend) into the accumulating error bound.

For more on the derivation of running error bounds see Wilkinson [1244, 1985] or [1245, 1986]. A running error analysis for Horner's method is given in §5.1.

3.4. Notation for Error Analysis

Another way to write (3.5) is

$$|x^T y - fl(x^T y)| \leq nu|x|^T|y| + O(u^2). \quad (3.7)$$

In general, which form of bound is preferable depends on the context. The use of first-order bounds such as (3.7) can simplify the algebra in an analysis. But there can be some doubt as to the size of the constant term concealed by the big-oh notation. Furthermore, in vector inequalities an $O(u^2)$ term hides the structure of the vector it is bounding and so can make interpretation of the result difficult; for example, the inequality $|x - y| \leq nu|x| + O(u^2)$ does not imply that y approximates every element of x with the same relative error (indeed the relative error could be infinite when $x_i = 0$, as far as we can tell from the bound).

In more complicated analyses based on Lemma 3.1 it is necessary to manipulate the $1 + \theta_k$ and γ_k terms. The next lemma provides the necessary rules.

Lemma 3.3. *For any positive integer k let θ_k denote a quantity bounded according to $|\theta_k| \leq \gamma_k = ku/(1 - ku)$. The following relations hold:*

$$\begin{aligned} (1 + \theta_k)(1 + \theta_j) &= 1 + \theta_{k+j}, \\ \frac{1 + \theta_k}{1 + \theta_j} &= \begin{cases} 1 + \theta_{k+j}, & j \leq k, \\ 1 + \theta_{k+2j}, & j > k, \end{cases} \\ \gamma_k \gamma_j &\leq \gamma_{\min(k,j)} \quad \text{for } \max(j, k)u \leq 1/2, \\ i\gamma_k &\leq \gamma_{ik}, \\ \gamma_k + u &\leq \gamma_{k+1}, \\ \gamma_k + \gamma_j + \gamma_k \gamma_j &\leq \gamma_{k+j}. \end{aligned}$$

Proof. See Problem 3.4. $\square$

Concerning the second rule in Lemma 3.3, we certainly have

$$\prod_{i=1}^k (1 + \delta_i)^{\pm 1} / \prod_{i=1}^j (1 + \delta_i)^{\pm 1} = 1 + \theta_{k+j},$$

but if we are given only the expression $(1 + \theta_k)/(1 + \theta_j)$ and the bounds for θ_k and θ_j , we cannot do better than to replace it by θ_{k+2j} for $j > k$.

In some error analyses it is not worth the trouble to keep precise track of the constants in the γ_k terms. The notation

$$\tilde{\gamma}_k = \frac{cku}{1 - cku} \quad (3.8)$$

is then useful, where c denotes a small integer constant whose exact value is unimportant. Thus we can write, for example, $3\gamma_n = \tilde{\gamma}_n$, $m\tilde{\gamma}_n = n\tilde{\gamma}_m = \tilde{\gamma}_{mn}$, and $\gamma_2 + \gamma_3(1 + \gamma_5) = \tilde{\gamma}_1$.

Another style of writing bounds is made possible by the following lemma.

Lemma 3.4. *If $|\delta_i| \leq u$ for $i = 1:n$ and $nu \leq 0.01$, then*

$$\prod_{i=1}^n (1 + \delta_i) = 1 + \eta_n,$$

where $|\eta_n| \leq 1.01nu$.

Proof. We have

$$|\eta_n| = \left| \prod_{i=1}^n (1 + \delta_i) - 1 \right| \leq (1 + u)^n - 1.$$

Since $1 + x \leq e^x$ for $x \geq 0$, we have $(1 + u)^n < \exp(nu)$, and so

$$\begin{aligned} (1 + u)^n - 1 &< nu + \frac{(nu)^2}{2!} + \frac{(nu)^3}{3!} + \cdots \\ &< nu \left(1 + \frac{nu}{2} + \left(\frac{nu}{2} \right)^2 + \left(\frac{nu}{2} \right)^3 + \cdots \right) \\ &= nu \frac{1}{1 - nu/2} \\ &\leq nu \frac{1}{0.995} < 1.01nu. \quad \square \end{aligned}$$

Note that this lemma is slightly stronger than the corresponding bound we can obtain from Lemma 3.1: $|\theta_n| \leq nu/(1 - nu) < nu/0.99 = 1.0101 \dots nu$.

Lemma 3.4 enables us to derive, as an alternative to (3.5),

$$|x^T y - fl(x^T y)| \leq 1.01nu|x|^T|y|. \quad (3.9)$$

A convenient device for keeping track of powers of $1 + \delta$ terms was introduced by Stewart [1065, 1973, App. 3]. His relative error counter $\langle k \rangle$ denotes a product

$$\langle k \rangle = \prod_{i=1}^k (1 + \delta_i)^{\rho_i}, \quad \rho_i = \pm 1, \quad |\delta_i| \leq u. \quad (3.10)$$

The counters can be manipulated using the rules

$$\begin{aligned} \langle j \rangle \langle k \rangle &= \langle j + k \rangle, \\ \frac{\langle j \rangle}{\langle k \rangle} &= \langle j - k \rangle. \end{aligned}$$

At the end of an analysis it is necessary to bound $|\langle k \rangle - 1|$, for which any of the techniques described above can be used.

Wilkinson explained in [1244, 1985] that he used a similar notation in his research, writing ψ^r for a product of r factors $1 + \delta_i$ with $|\delta_i| \leq u$. He also derived results for specific values of n before treating the general case—a useful trick of the trade!

An alternative notation to $fl(\cdot)$ to denote the rounded value of a number or the computed value of an expression is $[\cdot]$, suggested by Kahan. Thus, we would write $[a + [bc]]$ instead of $fl(a + fl(bc))$.

A completely different notation for error analysis was proposed by Olver [903, 1978], and subsequently used by him and several other authors. For scalars x and y of the same sign, Olver defines the *relative precision* rp as follows:

$$y \approx x; rp(\alpha) \quad \text{means that} \quad y = e^\delta x, \quad |\delta| \leq \alpha.$$

Since $e^\delta = 1 + \delta + O(\delta^2)$, this definition implies that the relative error in x as an approximation to y (or vice versa) is at most $\alpha + O(\alpha^2)$. But, unlike the usual definition of relative error, the relative precision possesses the properties of

$$\begin{aligned} \text{symmetry: } y \approx x; rp(\alpha) &\iff x \approx y; rp(\alpha), \\ \text{additivity: } y \approx x; rp(\alpha) \text{ and } z \approx y; rp(\beta) &\implies z \approx x; rp(\alpha + \beta). \end{aligned}$$

Proponents of relative precision claim that the symmetry and additivity properties make it easier to work with than the relative error.

Pryce [956, 1981] gives an excellent appraisal of relative precision, with many examples. He uses the additional notation $1(\delta)$ to mean a number θ with $\theta \approx 1; rp(\delta)$. The $1(\delta)$ notation is the analogue for relative precision of Stewart's $\langle k \rangle$ counter for relative error. In later papers, Pryce extends the rp notation to vectors and matrices and shows how it can be used in the error analysis of some basic matrix computations [957, 1984], [958, 1985].

Relative precision has not achieved wide use. The important thing for an error analyst is to settle on a comfortable notation that does not hinder the thinking process. It does not really matter which of the notations described in this section is used, as long as the final result is informative and expressed in a readable form.

3.5. Matrix Multiplication

Given error analysis for inner products it is straightforward to analyse matrix-vector and matrix-matrix products. Let $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, and $y = Ax$. The vector y can be formed as m inner products, $y_i = a_i^T x$, $i = 1:m$, where a_i^T is the i th row of A . From (3.4) we have

$$\hat{y}_i = (a_i + \Delta a_i)^T x, \quad |\Delta a_i| \leq \gamma_n |a_i|.$$

This gives the backward error result

$$\hat{y} = (A + \Delta A)x, \quad |\Delta A| \leq \gamma_n |A| \quad (A \in \mathbb{R}^{m \times n}), \quad (3.11)$$

which implies the forward error bound

$$|y - \hat{y}| \leq \gamma_n |A| |x|. \quad (3.12)$$

Normwise bounds readily follow (see Chapter 6 for norm definitions): we have

$$\|y - \hat{y}\|_p \leq \gamma_n \|A\|_p \|x\|_p, \quad p = 1, \infty,$$

and for the 2-norm, using Lemma 6.6,

$$\|y - \hat{y}\|_2 \leq \min(m, n)^{1/2} \gamma_n \|A\|_2 \|x\|_2.$$

This inner product formation of y can be expressed algorithmically as

```
% Sdot or inner product form.
y(1:m) = 0
for i = 1:m
    for j = 1:n
        y(i) = y(i) + a(i,j)x(j)
    end
end
```

The two loops can be interchanged to give

```
% Saxpy form.
y(1:m) = 0
for j = 1:n
    for i = 1:m
        y(i) = y(i) + a(i,j)x(j)
    end
end
```

The terms “sdot” and “saxpy” come from the BLAS (see §C.1). Sdot stands for (single precision) dot product, and saxpy for (single precision) α times x plus y . The question of interest is whether (3.11) and (3.12) hold for the saxpy form. They do: the saxpy algorithm still forms the inner products $a_i^T x$, but instead of forming each one in turn it evaluates them all “in parallel”, a term at a time. The key observation is that exactly the same operations are performed, and hence *exactly the same rounding errors are committed*—the only difference is the order in which the rounding errors are created.

This “rounding error equivalence” of algorithms that are mathematically identical but algorithmically different is one that occurs frequently in matrix computations. The equivalence is not always as easy to see as it is for matrix–vector products.

Now consider matrix multiplication: $C = AB$, where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Matrix multiplication is a triple loop procedure, with six possible loop orderings, one of which is

```
C(1:m, 1:p) = 0
for i = 1:m
```

```

    for  $j = 1:p$ 
      for  $k = 1:n$ 
         $C(i, j) = C(i, j) + A(i, k)B(k, j)$ 
      end
    end
  end
end

```

As for the matrix–vector product, all six versions commit the same rounding errors, so it suffices to consider any one of them. The “jik” and “jki” orderings both compute C a column at a time: $c_j = Ab_j$, where $c_j = C(:, j)$ and $b_j = B(:, j)$. From (3.11),

$$\hat{c}_j = (A + \Delta A_j)b_j, \quad |\Delta A_j| \leq \gamma_n |A|.$$

Hence the j th computed column of C has a small backward error: it is the exact j th column for slightly perturbed data. The same cannot be said for $\hat{C}$ as a whole (see Problem 3.5 for a possibly large backward error bound). However, we have the forward error bound

$$|C - \hat{C}| \leq \gamma_n |A| |B| \quad (A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times p}), \quad (3.13)$$

and the corresponding normwise bounds include

$$\|C - \hat{C}\|_p \leq \gamma_n \|A\|_p \|B\|_p, \quad p = 1, \infty, F.$$

The bound (3.13) falls short of the ideal bound $|C - \hat{C}| \leq \gamma_n |C|$, which says that each component of C is computed with high relative accuracy. Nevertheless (3.13) is the best bound we can expect, because it reflects the sensitivity of the product to componentwise relative perturbations in the data: for any i and j we can find a perturbation ΔA with $|\Delta A| \leq u|A|$ such that $|(A + \Delta A)B - AB|_{ij} = u(|A||B|)_{ij}$ (similarly for perturbations in B).

3.6. Complex Arithmetic

To carry out error analysis of algorithms in complex arithmetic we need a model for the basic arithmetic operations. Since complex arithmetic must be implemented using real arithmetic, the complex model is a consequence of the corresponding real one. We will assume that for complex numbers $x = a + ib$ and $y = c + id$ we compute

$$x \pm y = a \pm c + i(b \pm d), \quad (3.14a)$$

$$xy = ac - bd + i(ad + bc), \quad (3.14b)$$

$$x/y = \frac{ac + bd}{c^2 + d^2} + i \frac{bc - ad}{c^2 + d^2}. \quad (3.14c)$$

Lemma 3.5. *For $x, y \in \mathbb{C}$ the basic arithmetic operations computed according to (3.14) under the standard model (2.4) satisfy*

$$\begin{aligned} fl(x \pm y) &= (x \pm y)(1 + \delta), & |\delta| &\leq u, \\ fl(xy) &= xy(1 + \delta), & |\delta| &\leq \sqrt{2}\gamma_2, \\ fl(x/y) &= (x/y)(1 + \delta), & |\delta| &\leq \sqrt{2}\gamma_4. \end{aligned}$$

Proof. Throughout the proof, δ_i denotes a number bounded by $|\delta_i| \leq u$.
Addition/subtraction:

$$\begin{aligned} fl(x+y) &= (a+c)(1+\delta_1) + i(b+d)(1+\delta_2) \\ &= x+y + (a+c)\delta_1 + i(b+d)\delta_2, \end{aligned}$$

so

$$|fl(x+y) - (x+y)|^2 \leq (|a+c|^2 + |b+d|^2)u^2 = (|x+y|u)^2,$$

as required.

Multiplication:

$$\begin{aligned} fl(xy) &= (ac(1+\delta_1) - bd(1+\delta_2))(1+\delta_3) \\ &\quad + i(ad(1+\delta_4) + bc(1+\delta_5))(1+\delta_6) \\ &= ac(1+\theta_2) - bd(1+\theta'_2) + i(ad(1+\theta''_2) + bc(1+\theta'''_2)) \\ &= xy + e, \end{aligned}$$

where

$$\begin{aligned} |e|^2 &\leq \gamma_2^2((|ac| + |bd|)^2 + (|ad| + |bc|)^2) \\ &\leq 2\gamma_2^2(a^2 + b^2)(c^2 + d^2) \\ &= 2\gamma_2^2|xy|^2, \end{aligned}$$

as required.

Division:

$$\begin{aligned} fl(c^2 + d^2) &= (c^2(1+\delta_1) + d^2(1+\delta_2))(1+\delta_3) \\ &= c^2(1+\theta_2) + d^2(1+\theta'_2) \\ &= (c^2 + d^2)(1+\theta''_2). \end{aligned}$$

Then

$$\begin{aligned} fl(\operatorname{Re} x/y) &= \frac{(ac(1+\delta_4) + bd(1+\delta_5))(1+\delta_6)}{(c^2 + d^2)(1+\theta''_2)} \\ &= \frac{ac(1+\theta'''_2) + bd(1+\theta''''_2)}{(c^2 + d^2)(1+\theta''_2)} \\ &= \operatorname{Re} x/y + e_1, \end{aligned}$$

where, using Lemma 3.3,

$$|e_1| \leq \frac{|ac| + |bd|}{c^2 + d^2} \gamma_4.$$

Using the analogous formula for the error in $fl(\operatorname{Im} x/y)$,

$$\begin{aligned} |fl(x/y) - x/y|^2 &\leq \frac{(|ac| + |bd|)^2 + (|bc| + |ad|)^2}{(c^2 + d^2)^2} \gamma_4^2 \\ &\leq \frac{2(a^2 + b^2)(c^2 + d^2)}{(c^2 + d^2)^2} \gamma_4^2 \\ &= 2\gamma_4^2 |x/y|^2, \end{aligned}$$

which completes the proof. $\square$

It is worth stressing that δ in Lemma 3.5 is a complex number, so we cannot conclude from the lemma that the real and imaginary parts of $fl(x \text{ op } y)$ are obtained to high relative accuracy—only that they are obtained to high accuracy relative to $|x \text{ op } y|$.

As explained in §27.8, the formula (3.14c) is not recommended for practical use since it is susceptible to overflow. For the alternative formula (27.1), which avoids overflow, similar analysis to that in the proof of Lemma 3.5 shows that

$$fl(x/y) = (x/y)(1 + \delta), \quad |\delta| \leq \sqrt{2}\gamma_7.$$

Bounds for the rounding errors in the basic complex arithmetic operations are rarely given in the literature. Indeed, virtually all published error analyses in matrix computations are for real arithmetic. However, because the bounds of Lemma 3.5 are of the same form as for the standard model (2.4) for real arithmetic, most results for real arithmetic (including virtually all those in this book) are valid for complex arithmetic, provided that the constants are increased appropriately.

3.7. Miscellany

In this section we give some miscellaneous results that will be needed in later chapters. The first three results provide convenient ways to bound the effect of perturbations in a matrix product. The first two results use norms and the third, components.

Lemma 3.6. *If $X_j + \Delta X_j \in \mathbb{R}^{n \times n}$ satisfies $\|\Delta X_j\| \leq \delta_j \|X_j\|$ for all j for a consistent norm, then*

$$\left\| \prod_{j=0}^m (X_j + \Delta X_j) - \prod_{j=0}^m X_j \right\| \leq \left(\prod_{j=0}^m (1 + \delta_j) - 1 \right) \prod_{j=0}^m \|X_j\|.$$

Proof. The proof is a straightforward induction, which we leave as an exercise (Problem 3.9). $\square$

The second lemma is a variation on the first, with an analogous proof that employs the inequality in Problem 6.5. It is useful when analysing Householder transformations (see Chapter 19).

Lemma 3.7. *If $X_j + \Delta X_j \in \mathbb{R}^{n \times n}$ satisfies $\|\Delta X_j\|_F \leq \delta_j \|X_j\|_2$ for all j , then*

$$\left\| \prod_{j=0}^m (X_j + \Delta X_j) - \prod_{j=0}^m X_j \right\|_F \leq \left(\prod_{j=0}^m (1 + \delta_j) - 1 \right) \prod_{j=0}^m \|X_j\|_2. \quad \square$$

In comparing the last lemma with Lemma 3.6 for the Frobenius norm, note that the product $\|X_1\|_2 \dots \|X_k\|_2$ can be much smaller than $\|X_1\|_F \dots \|X_k\|_F$; the extreme case occurs when the X_i are orthogonal.

A componentwise result is entirely analogous.

Lemma 3.8. *If $X_j + \Delta X_j \in \mathbb{R}^{n \times n}$ satisfies $|\Delta X_j| \leq \delta_j |X_j|$ for all j , then*

$$\left| \prod_{j=0}^m (X_j + \Delta X_j) - \prod_{j=0}^m X_j \right| \leq \left(\prod_{j=0}^m (1 + \delta_j) - 1 \right) \prod_{j=0}^m |X_j|. \quad \square$$

The final result describes the computation of the “rank-1 update” $y = (I - ab^T)x$, which is an operation arising in various algorithms, including the Gram–Schmidt method and Householder QR factorization.

Lemma 3.9. *Let $a, b, x \in \mathbb{R}^n$ and let $y = (I - ab^T)x$ be computed as $\hat{y} = fl(x - a(b^T x))$. Then $\hat{y} = y + \Delta y$, where*

$$|\Delta y| \leq \gamma_{n+3}(I + |a||b^T|)|x|,$$

so that

$$\|\Delta y\|_2 \leq \gamma_{n+3}(1 + \|a\|_2 \|b\|_2) \|x\|_2.$$

Proof. Consider first the computation of $w = a(b^T x)$. We have

$$\begin{aligned} \hat{w} &:= (a + \Delta a)b^T(x + \Delta x), & |\Delta a| &\leq u|a|, & |\Delta x| &\leq \gamma_n|x|, \\ &= a(b^T x) + a(b^T \Delta x) + \Delta a b^T(x + \Delta x) \\ &=: w + \Delta w, \end{aligned}$$

where

$$|\Delta w| \leq (\gamma_n + u(1 + \gamma_n))|a||b^T||x|.$$

Finally, $\hat{y} = fl(x - \hat{w})$ satisfies

$$\hat{y} = x - a(b^T x) - \Delta w + \Delta y_1, \quad |\Delta y_1| \leq u(|x| + |\hat{w}|),$$

and

$$|-\Delta w + \Delta y_1| \leq u(|x| + |a||b^T||x|) + (1 + u)(\gamma_n + u(1 + \gamma_n))|a||b^T||x|.$$

Hence $\hat{y} = y + \Delta y$, where

$$\begin{aligned} |\Delta y| &\leq [uI + (2u + u^2 + \gamma_n + 2u\gamma_n + u^2\gamma_n)|a||b^T|]|x| \\ &\leq \gamma_{n+3}(I + |a||b^T|)|x|. \quad \square \end{aligned}$$

3.8. Error Analysis Demystified

The principles underlying an error analysis can easily be obscured by the details. It is therefore instructive to examine the basic mechanism of forward and backward error analyses. We outline a general framework that reveals the essential simplicity.

Consider the problem of computing $z = f(a)$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Any algorithm for computing z can be expressed as follows. Let $x_1 = a$ and $x_{k+1} = g_k(x_k)$, $k = 1:p$, where

$$x_{k+1} = \begin{bmatrix} x_k \\ \xi_k \end{bmatrix}, \quad \xi_k \in \mathbb{R}.$$

The k th stage of the algorithm represents a single floating point operation and x_k contains the original data together with all the intermediate quantities computed so far. Finally, $z = \bar{I}x_{p+1}$, where $\bar{I}$ is comprised of a subset of the columns of the identity matrix (so that each z_i is a component of x_{p+1}). In floating point arithmetic we have

$$\hat{x}_{k+1} = g_k(\hat{x}_k) + \Delta x_{k+1},$$

where Δx_{k+1} represents the rounding errors on the k th stage and should be easy to bound. We assume that the functions g_k are continuously differentiable and denote the Jacobian of g_k at a by J_k . Then, to first order,

$$\begin{aligned}\hat{x}_2 &= g_1(a) + \Delta x_2, \\ \hat{x}_3 &= g_2(\hat{x}_2) + \Delta x_3 = g_2(g_1(a) + \Delta x_2) + \Delta x_3 \\ &= g_2(g_1(a)) + J_2\Delta x_2 + \Delta x_3, \\ \hat{x}_4 &= g_3(\hat{x}_3) + \Delta x_4 = g_3(g_2(g_1(a)) + J_2\Delta x_2 + \Delta x_3) + \Delta x_4 \\ &= g_3(g_2(g_1(a))) + J_3J_2\Delta x_2 + J_3\Delta x_3 + \Delta x_4.\end{aligned}$$

The pattern is clear: for the final $\hat{z} = \hat{x}_{p+1}$ we have

$$\begin{aligned}\hat{z} &= \bar{I}[g_p(\dots g_2(g_1(a))\dots) + J_p\dots J_2\Delta x_2 + J_p\dots J_3\Delta x_3 + \dots \\ &\quad + J_p\Delta x_p + \Delta x_{p+1}] \\ &= f(a) + \bar{I}[J_p\dots J_2, \dots, J_p, I] \begin{bmatrix} \Delta x_2 \\ \Delta x_3 \\ \vdots \\ \Delta x_{p+1} \end{bmatrix} =: f(a) + Jh.\end{aligned}$$

In a forward error analysis we bound $f(a) - \hat{z}$, which requires bounds for (products of) the Jacobians J_k . In a backward error analysis we write, again to first order,

$$f(a) + Jh = \hat{z} = f(a + \Delta a) = f(a) + J_f\Delta a,$$

where J_f is the Jacobian of f . So we need to solve, for Δa ,

$$\underbrace{J_f}_{m \times n} \underbrace{\Delta a}_{n \times 1} = \underbrace{J}_{m \times q} \underbrace{h}_{q \times 1}, \quad q = p(n + (p + 1)/2).$$

In most matrix problems there are fewer outputs than inputs ($m < n$), so this is an underdetermined system. For a normwise backward error analysis we want a solution of minimum norm. For a componentwise backward error analysis, in which we may want (for example) to minimize ϵ subject to $|\Delta a| \leq \epsilon|a|$, we can write

$$Jh = J_f D \cdot D^{-1} \Delta a =: Bc, \quad D = \text{diag}(a_i),$$

and then we want the solution c of minimal ∞ -norm.

The conclusions are that forward error analysis corresponds to bounding derivatives and that backward error analysis corresponds to solving a large underdetermined linear system for a solution of minimal norm. In principal, therefore, error analysis is straightforward! Complicating factors in practice are that the Jacobians

J_k may be difficult to obtain explicitly, that an error bound has to be expressed in a form that can easily be interpreted, and that we may want to keep track of higher-order terms.

3.9. Other Approaches

In this book we do not describe all possible approaches to error analysis. Some others are mentioned in this section.

Linearized rounding error bounds can be developed by applying equations that describe, to first order, the propagation of absolute or relative errors in the elementary operations $+$, $-$, $*$, $/$. The basics of this approach are given in many textbooks (see, for example, Dahlquist and Björck [289, 1974, §2.2] or Stoer and Bulirsch [1086, 1980, §1.3]), but for a thorough treatment see Stummel [1094, 1980], [1095, 1981]. Ziv [1286, 1995] shows that linearized bounds can be turned into true bounds by increasing them by a factor that depends on the algorithm.

Rounding error analysis can be phrased in terms of graphs. This appears to have been first suggested by McCracken and Dorn [833, 1964], who use “process graphs” to represent a numerical computation and thereby to analyse the propagation of rounding errors. Subsequent more detailed treatments include those of Bauer [93, 1974], Miller [852, 1976], and Yalamov [1263, 1995]. The work on graphs falls under the heading of automatic error analysis (for more on which see Chapter 26) because processing of the large graphs required to represent practical computations is impractical by hand. Linnainmaa [790, 1976] shows how to compute the Taylor series expansion of the forward error in an algorithm in terms of the individual rounding errors, and he presents a graph framework for the computation.

Some authors have taken a computational complexity approach to error analysis, by aiming to minimize the number of rounding error terms in a forward error bound, perhaps by rearranging a computation. Because this approach ignores the possibility of cancellation of rounding errors, the results need to be interpreted with care. See Aggarwal and Burgmeier [7, 1979] and Tsao [1162, 1983].

3.10. Notes and References

The use of Lemma 3.1 for rounding error analysis appears to originate with the original German edition [1085, 1972] of a book by Stoer and Bulirsch [1086, 1980]. The lemma is also used, with $\rho_i \equiv 1$, by Shampine and Allen [1031, 1973, p. 18].

The notation $\tilde{\gamma}_n$ in (3.8) was suggested to the author by Beresford Parlett and replaces the more clumsy notation γ_{cn} used in the first edition of this book.

Lemma 3.4 is given by Forsythe and Moler [431, 1967, p. 92]. Wilkinson made frequent use of a slightly different version of Lemma 3.4 in which the assumption is $nu < 0.1$ and the bound for $|\eta_n|$ is $1.06nu$ (see, e.g., [1233, 1965, p. 113]).

A straightforward notation for rounding errors that is subsumed by the notation described in this chapter is suggested by Scherer and Zeller [1017, 1980].

Ziv [1284, 1982] proposes the relative error measure

$$d(x, y) = \|x - y\| / \max(\|x\|, \|y\|)$$

for vectors x and y and explains some of its favourable properties for error analysis.

Wilkinson [1233, 1965, p. 447] gives error bounds for complex arithmetic; Olver [904, 1983] does the same in the relative precision framework. Demmel [308, 1984] gives error bounds that extend those in Lemma 3.5 by taking into account the possibility of underflow.

Henrici [568, 1980] gives a brief, easy-to-read introduction to the use of the model (2.4) for analysing the propagation of rounding errors in a general algorithm. He uses a set notation that is another possible notation to add to those in §3.4.

The perspective on error analysis in §3.8 was suggested by J. W. Demmel.

Problems

3.1. Prove Lemma 3.1.

3.2. (Kiełbasiński and Schwetlick [734, 1988], [735, 1992]) Show that if $\rho_i \equiv 1$ in Lemma 3.1 then the stronger bound $|\theta_n| \leq nu/(1 - \frac{1}{2}nu)$ holds for $nu < 2$.

3.3. One algorithm for evaluating a continued fraction

$$a_0 + \frac{b_0}{a_1 + \frac{b_1}{a_2 + \cdots + \frac{b_n}{a_{n+1}}}}$$

is

```

 $q_{n+1} = a_{n+1}$ 
for  $k = n: -1: 0$ 
     $q_k = a_k + b_k/q_{k+1}$ 
end

```

Derive a running error bound for this algorithm.

3.4. Prove Lemma 3.3.

3.5. (Backward error result for matrix multiplication.) Let $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times n}$ both be nonsingular. Show that $fl(AB) = (A + \Delta A)B$, where $|\Delta A| \leq \gamma_n |A| |B| |B|^{-1}|$, and derive a corresponding bound in which B is perturbed.

3.6. (Backward error definition for matrix multiplication.) Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ be of full rank and suppose $C \approx AB$. Define the componentwise backward error

$$\omega = \min \{ \epsilon : C = (A + \Delta A)(B + \Delta B), \quad |\Delta A| \leq \epsilon E, \quad |\Delta B| \leq \epsilon F \},$$

where E and F have nonnegative entries. Show that

$$\omega \geq \max_{i,j} \left(\sqrt{1 + \frac{|r_{ij}|}{g_{ij}}} - 1 \right),$$

where $R = C - AB$ and $G = EF$. Explain why the definition of ω makes sense only when A and B have full rank. Define a mixed backward/forward error applicable to the general case.

3.7. Give analogues of the backward error results (3.4) and (3.11) for complex x , y , and A .

3.8. Which is the more accurate way to compute $x^2 - y^2$: as $x^2 - y^2$ or as $(x + y)(x - y)$? (Note that this computation arises when squaring a complex number.)

3.9. Prove Lemma 3.6.

3.10. Let $A_1, \dots, A_k \in \mathbb{R}^{n \times n}$. Show that

$$\|A_1 \dots A_k - fl(A_1 \dots A_k)\|_F \leq (kn^2u + O(u^2))\|A_1\|_2 \dots \|A_k\|_2.$$

3.11. (Kahan [690, 1980]) Consider this MATLAB function, which returns the absolute value of its first argument $x \in \mathbb{R}^n$:

```
function z = absolute(x, m)
y = x.^2;
for i=1:m
    y = sqrt(y);
end
z = y;
for i=1:m-1
    z = z.^2;
end
```

Here is some output from a Pentium III workstation (which uses IEEE standard double precision arithmetic):

```
>> x = [.25 .5 .75 1.25 1.5 2]; z = absolute(x, 50); [x; z]
ans =
    0.2500    0.5000    0.7500    1.2500    1.5000    2.0000
    0.2375    0.4724    0.7316    1.1331    1.4550    1.8682
```

Give an error analysis to explain the results.

3.12. Consider the quadrature rule

$$I(f) := \int_a^b f(x) dx \approx \sum_{i=1}^n w_i f(x_i) =: J(f),$$

where the weights w_i and nodes x_i are assumed to be floating point numbers. Assuming that the sum is evaluated in left-to-right order and that

$$fl(f(x_i)) = f(x_i)(1 + \eta_i), \quad |\eta_i| \leq \eta,$$

obtain and interpret a bound for $|I(f) - \widehat{J}(f)|$, where $\widehat{J}(f) = fl(J(f))$.

Chapter 4

Summation

*I do hate sums.
There is no greater mistake than to call arithmetic an exact science.
There are . . . hidden laws of Number
which it requires a mind like mine to perceive.
For instance, if you add a sum from the bottom up,
and then again from the top down,
the result is always different.*
— MRS. LA TOUCHE⁷

*Joseph Fourier introduced this delimited $\sum$ -notation in 1820,
and it soon took the mathematical world by storm.*
— RONALD L. GRAHAM, DONALD E. KNUTH, and
OREN PATASHNIK, *Concrete Mathematics* (1994)

*One of the major difficulties in a practical [error] analysis
is that of description.
An ounce of analysis follows a pound of preparation.*
— BERESFORD N. PARLETT, *Matrix Eigenvalue Problems* (1965)

⁷Quoted in *Mathematical Gazette* [819, 1924].

Sums of floating point numbers are ubiquitous in scientific computing. They occur when evaluating inner products, means, variances, norms, and all kinds of non-linear functions. Although at first sight summation might appear to offer little scope for algorithmic ingenuity, the usual “recursive summation” (with various orderings) is just one of a variety of possible techniques. We describe several summation methods and their error analyses in this chapter. No one method is uniformly more accurate than the others, but some guidelines can be given on the choice of method in particular cases.

4.1. Summation Methods

In most circumstances in scientific computing we would naturally translate a sum $\sum_{i=1}^n x_i$ into code of the form

```
s = 0
for i = 1:n
    s = s + x_i
end
```

This is known as *recursive summation*. Since the individual rounding errors depend on the operands being summed, the accuracy of the computed sum $\hat{s}$ varies with the ordering of the x_i . (Hence Mrs. La Touche, quoted at the beginning of the chapter, was correct if we interpret her remarks as applying to floating point arithmetic.) Two interesting choices of ordering are the increasing order $|x_1| \leq |x_2| \leq \dots \leq |x_n|$, and the decreasing order $|x_1| \geq |x_2| \geq \dots \geq |x_n|$.

Another method is *pairwise summation* (also known as cascade summation, or fan-in summation), in which the x_i are summed in pairs according to

$$y_i = x_{2i-1} + x_{2i}, \quad i = 1 : \left\lfloor \frac{n}{2} \right\rfloor \quad (y_{(n+1)/2} = x_n \text{ if } n \text{ is odd}),$$

and this pairwise summation process is repeated recursively on the y_i , $i = 1 : \lfloor (n+1)/2 \rfloor$. The sum is obtained in $\lceil \log_2 n \rceil$ stages. For $n = 6$, for example, pairwise summation forms

$$S_6 = ((x_1 + x_2) + (x_3 + x_4)) + (x_5 + x_6).$$

Pairwise summation is attractive for parallel computing, because each of the $\lceil \log_2 n \rceil$ stages can be done in parallel [629, 1988, §5.2.2].

A third summation method is the *insertion* method. First, the x_i are sorted by order of increasing magnitude (alternatively, some other ordering could be used). Then $x_1 + x_2$ is formed, and the sum is inserted into the list $x_2, \dots, x_n$, maintaining the increasing order. The process is repeated recursively until the final sum is obtained. In particular cases the insertion method reduces to one of the other two. For example, if $x_i = 2^{i-1}$, the insertion method is equivalent to recursive summation, since the insertion is always to the bottom of the list:

$$1 \ 2 \ 4 \ 8 \quad \rightarrow \quad \underline{3} \ 4 \ 8 \quad \rightarrow \quad \underline{7} \ 8 \quad \rightarrow \quad 15.$$

On the other hand, if $1 \leq x_1 < x_2 < \dots < x_n \leq 2$, every insertion is to the end of the list, and the method is equivalent to pairwise summation if n is a power of 2; for example, if $0 < \epsilon < 1/2$,

$$1, 1+\epsilon, 1+2\epsilon, 1+3\epsilon \rightarrow 1+2\epsilon, 1+3\epsilon, \underline{2+\epsilon} \rightarrow 2+\epsilon, \underline{2+5\epsilon} \rightarrow 4+6\epsilon.$$

To choose between these methods we need error analysis, which we develop in the next section.

4.2. Error Analysis

Error analysis can be done individually for the recursive, pairwise, and insertion summation methods, but it is more profitable to recognize that each is a special case of a general algorithm and to analyse that algorithm.

Algorithm 4.1. Given numbers $x_1, \dots, x_n$ this algorithm computes $S_n = \sum_{i=1}^n x_i$.

```

Let  $\mathcal{S} = \{x_1, \dots, x_n\}$ .
repeat while  $\mathcal{S}$  contains more than one element
    Remove two numbers  $x$  and  $y$  from  $\mathcal{S}$ 
    and add their sum  $x + y$  to  $\mathcal{S}$ .
end
Assign the remaining element of  $\mathcal{S}$  to  $S_n$ .

```

Note that since there are n numbers to be added and hence $n - 1$ additions to be performed, there must be precisely $n - 1$ executions of the repeat loop.

First, let us check that the previous methods are special cases of Algorithm 4.1. Recursive summation (with any desired ordering) is obtained by taking x at each stage to be the sum computed on the previous stage of the algorithm. Pairwise summation is obtained by $\lceil \log_2 n \rceil$ groups of executions of the repeat loop, in each group of which the members of $\mathcal{S}$ are broken into pairs, each of which is summed. Finally, the insertion method is, by definition, a special case of Algorithm 4.1.

Now for the error analysis. Express the i th execution of the repeat loop as $T_i = x_{i_1} + y_{i_1}$. The computed sums satisfy (using (2.5))

$$\widehat{T}_i = \frac{x_{i_1} + y_{i_1}}{1 + \delta_i}, \quad |\delta_i| \leq u, \quad i = 1:n-1. \quad (4.1)$$

The local error introduced in forming $\widehat{T}_i$ is $\delta_i \widehat{T}_i$. The overall error is the sum of the local errors (since summation is a linear process), so overall we have

$$E_n := S_n - \widehat{S}_n = \sum_{i=1}^{n-1} \delta_i \widehat{T}_i. \quad (4.2)$$

The smallest possible error bound is therefore

$$|E_n| \leq u \sum_{i=1}^{n-1} |\widehat{T}_i|. \quad (4.3)$$

(This is actually in the form of a running error bound, because it contains the computed quantities—see §3.3.) It is easy to see that $|\hat{T}_i| \leq \sum_{j=1}^n |x_j| + O(u)$ for each i , and so we have also the weaker bound

$$|E_n| \leq (n-1)u \sum_{i=1}^n |x_i| + O(u^2). \quad (4.4)$$

This is a forward error bound. A backward error result showing that $\hat{S}_n$ is the exact sum of terms $x_i(1 + \epsilon_i)$ with $|\epsilon_i| \leq \gamma_{n-1}$ can be deduced from (4.1), using the fact that no number x_i takes part in more than $n-1$ additions.

The following criterion is apparent from (4.2) and (4.3):

In designing or choosing a summation method to achieve high accuracy, the aim should be to minimize the absolute values of the intermediate sums T_i .

The aim specified in this criterion is surprisingly simple to state. However, even if we concentrate on a specific set of data the aim is difficult to achieve, because minimizing the bound in (4.3) is known to be NP-hard [708, 2000]. Some insight can be gained by specializing to recursive summation.

For recursive summation, $T_{i-1} = S_i := \sum_{j=1}^i x_j$, and we would like to choose the ordering of the x_i to minimize $\sum_{i=2}^n |\hat{S}_i|$. This is a combinatorial optimization problem that is too expensive to solve in the context of summation. A reasonable compromise is to determine the ordering sequentially by minimizing, in turn, $|x_1|$, $|\hat{S}_2|$, $\dots$, $|\hat{S}_{n-1}|$. This ordering strategy, which we denote by Psum, can be implemented with $O(n \log n)$ comparisons. If we are willing to give up the property that the ordering is influenced by the signs of the x_i we can instead use the increasing ordering, which in general will lead to a larger value of $\sum_{i=2}^n |\hat{S}_i|$ than that for the Psum ordering. If all the x_i have the same sign then all these orderings are equivalent. Therefore *when summing nonnegative numbers by recursive summation the increasing ordering is the best ordering, in the sense of having the smallest a priori forward error bound.*

How does the decreasing ordering fit into the picture? For the summation of positive numbers this ordering has little to recommend it. The bound (4.3) is no smaller, and potentially much larger, than it is for the increasing ordering. Furthermore, in a sum of positive terms that vary widely in magnitude the decreasing ordering may not allow the smaller terms to contribute to the sum (which is why the harmonic sum $\sum_{i=1}^n 1/i$ “converges” in floating point arithmetic as $n \rightarrow \infty$). However, consider the example with $n = 4$ and

$$x = [1, \quad M, \quad 2M, \quad -3M], \quad (4.5)$$

where M is a floating point number so large that $fl(1+M) = M$ (thus $M > u^{-1}$). The three orderings considered so far produce the following results:

$$\begin{aligned} \text{Increasing:} \quad & \hat{S}_n = fl(1 + M + 2M - 3M) = 0, \\ \text{Psum:} \quad & \hat{S}_n = fl(1 + M - 3M + 2M) = 0, \\ \text{Decreasing:} \quad & \hat{S}_n = fl(-3M + 2M + M + 1) = 1. \end{aligned}$$

Thus the decreasing ordering sustains no rounding errors and produces the exact answer, while both the increasing and Psum orderings yield computed sums with relative error 1. The reason why the decreasing ordering performs so well in this example is that it adds the “1” after the inevitable heavy cancellation has taken place, rather than before, and so retains the important information in this term. If we evaluate the term $\mu = \sum_{i=2}^n |\hat{S}_i|$ in the error bound (4.3) for example (4.5) we find

$$\text{Increasing: } \mu = 4M, \quad \text{Psum: } \mu = 3M, \quad \text{Decreasing: } \mu = M + 1,$$

so (4.3) “predicts” that the decreasing ordering will produce the most accurate answer, but the bound it provides is extremely pessimistic since there are no rounding errors in this instance.

Extrapolating from this example, we conclude that the decreasing ordering is likely to yield greater accuracy than the increasing or Psum orderings whenever there is heavy cancellation in the sum, that is, whenever $|\sum_{i=1}^n x_i| \ll \sum_{i=1}^n |x_i|$.

Turning to the insertion method, a good explanation of the insertion strategy is that it attempts to minimize, one at a time, the terms $|\hat{T}_1|, \dots, |\hat{T}_{n-1}|$ in the error bound (4.3). Indeed, if the x_i are all nonnegative the insertion method minimizes this bound over all instances of Algorithm 4.1.

Finally, we note that a stronger form of the bound (4.4) holds for pairwise summation. It can be deduced from (4.3) or derived directly, as follows. Assume for simplicity that $n = 2^r$. Unlike in recursive summation each addend takes part in the same number of additions, $\log_2 n$. Therefore we have a relation of the form

$$\hat{S}_n = \sum_{i=1}^n x_i \prod_{k=1}^{\log_2 n} (1 + \delta_k^{(i)}), \quad |\delta_k^{(i)}| \leq u,$$

which leads to the bound

$$|E_n| \leq \gamma_{\log_2 n} \sum_{i=1}^n |x_i|. \quad (4.6)$$

Since it is proportional to $\log_2 n$ rather than n , this is a smaller bound than (4.4), which is the best bound of this form that holds in general for Algorithm 4.1.

4.3. Compensated Summation

We have left to last the *compensated summation* method, which is recursive summation with a correction term cleverly designed to diminish the rounding errors. Compensated summation is worth considering whenever an accurate sum is required and computations are already taking place at the highest precision supported by the hardware or the programming language in use.

In 1951 Gill [488, 1951] noticed that the rounding error in the sum of two numbers could be estimated by subtracting one of the numbers from the sum, and he made use of this estimate in a Runge-Kutta code in a program library for the EDSAC computer. Gill’s estimate is valid for fixed point arithmetic only. Kahan [686, 1965] and Møller [870, 1965] both extended the idea to floating point arithmetic. Møller shows how to estimate $a + b - fl(a + b)$ in chopped arithmetic,

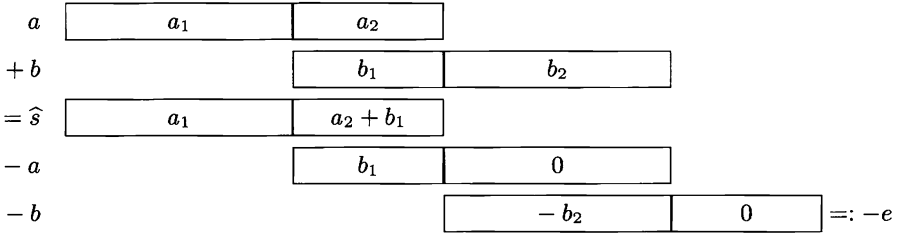


Figure 4.1. Recovering the rounding error.

while Kahan uses a slightly simpler estimate to derive the compensated summation method for computing $\sum_{i=1}^n x_i$.

The estimate used by Kahan is perhaps best explained with the aid of a diagram. Let a and b be floating point numbers with $|a| \geq |b|$, let $\hat{s} = fl(a + b)$, and consider Figure 4.1, which uses boxes to represent the significands of a and b . The figure suggests that if we evaluate

$$e = -[((a + b) - a) - b] = (a - \hat{s}) + b$$

in floating point arithmetic, in the order indicated by the parentheses, then the computed $\hat{e}$ will be a good estimate of the error $(a + b) - \hat{s}$. In fact, for rounded floating point arithmetic in base 2, we have

$$a + b = \hat{s} + \hat{e}, \quad (4.7)$$

that is, the computed $\hat{e}$ represents the error exactly. This result (which does not hold for all bases) is proved by Dekker [302, 1971, Thm. 4.7], Knuth [744, 1998, Thm. C, §4.2.2], and Linnainmaa [788, 1974, Thm. 3]. Note that there is no point in computing $fl(\hat{s} + \hat{e})$, since $\hat{s}$ is already the best floating point representation of $a + b$! Note also that this result implies, in particular, that the error $(a + b) - \hat{s}$ is a floating point number; for a short proof of this fact see Problem 4.6.

Kahan's compensated summation method employs the correction e on every step of a recursive summation. After each partial sum is formed, the correction is computed and immediately added to the next term x_i before that term is added to the partial sum. Thus the idea is to capture the rounding errors and feed them back into the summation. The method may be written as follows.

Algorithm 4.2 (compensated summation). Given floating point numbers $x_1, \dots, x_n$ this algorithm forms the sum $s = \sum_{i=1}^n x_i$ by compensated summation.

```

s = 0; e = 0
for i = 1:n
    temp = s
    y = xi + e
    s = temp + y
    e = (temp - s) + y    % Evaluate in the order shown.
end

```

The compensated summation method has two weaknesses: $\hat{e}$ is not necessarily the exact correction, since (4.7) is based on the assumption that $|a| \geq |b|$, and the addition $y = x_i + e$ is not performed exactly. Nevertheless, the use of the corrections brings a benefit in the form of an improved error bound. Knuth [744, 1998, Ex. 19, §4.2.2] shows that the computed sum $\hat{S}_n$ satisfies

$$\hat{S}_n = \sum_{i=1}^n (1 + \mu_i)x_i, \quad |\mu_i| \leq 2u + O(nu^2), \quad (4.8)$$

which is an almost ideal backward error result (a more detailed version of Knuth's proof is given by Goldberg [496, 1991]).

In [688, 1972] and [689, 1973] Kahan describes a variation of compensated summation in which the final sum is also corrected (thus " $s = s + e$ " is appended to the algorithm above). Kahan states in [688, 1972] and proves in [689, 1973] that (4.8) holds with the stronger bound $|\mu_i| \leq 2u + O((n-i+1)u^2)$. The proofs of (4.8) given by Knuth and Kahan are similar; they use the model (2.4) with a subtle induction and some intricate algebraic manipulation.

The forward error bound corresponding to (4.8) is

$$|E_n| \leq (2u + O(nu^2)) \sum_{i=1}^n |x_i|. \quad (4.9)$$

As long as $nu \leq 1$, the constant in this bound is independent of n , and so the bound is a significant improvement over the bounds (4.4) for recursive summation and (4.6) for pairwise summation. Note, however, that if $\sum_{i=1}^n |x_i| \gg |\sum_{i=1}^n x_i|$, compensated summation is not guaranteed to yield a small relative error.

Another version of compensated summation has been investigated by several authors: Jankowski, Smoktunowicz, and Woźniakowski [670, 1983], Jankowski and Woźniakowski [672, 1985], Kielbasiński [731, 1973], Neumaier [883, 1974], and Nickel [892, 1970]. Here, instead of immediately feeding each correction back into the summation, the corrections are accumulated separately by recursive summation and then the global correction is added to the computed sum. For this version of compensated summation Kielbasiński [731, 1973] and Neumaier [883, 1974] show that

$$\hat{S}_n = \sum_{i=1}^n (1 + \mu_i)x_i, \quad |\mu_i| \leq 2u + n^2u^2, \quad (4.10)$$

provided $nu \leq 0.1$; this is weaker than (4.8) in that the second-order term has an extra factor n . If $n^2u \leq 0.1$ then in (4.10), $|\mu_i| \leq 2.1u$. Jankowski, Smoktunowicz, and Woźniakowski [670, 1983] show that, by using a divide and conquer implementation of compensated summation, the range of n for which $|\mu_i| \leq cu$ holds in (4.10) can be extended, at the cost of a slight increase in the size of the constant c .

Neither the correction formula (4.7) nor the result (4.8) for compensated summation holds under the no-guard-digit model of floating point arithmetic. Indeed, Kahan [696, 1990] constructs an example where compensated summation fails to achieve (4.9) on certain Cray machines, but he states that such failure is extremely rare. In [688, 1972] and [689, 1973] Kahan gives a modification of the compensated summation algorithm in which the assignment " $e = (temp - s) + y$ " is replaced by

```

f = 0
if sign(temp) = sign(y), f = (0.46 * s - s) + s, end
e = ((temp - f) - (s - f)) + y

```

Kahan shows in [689, 1973] that the modified algorithm achieves (4.8) “on all North American machines with floating hardware” and explains that “The mysterious constant 0.46, which could perhaps be any number between 0.25 and 0.50, and the fact that the proof requires a consideration of known machines designs, indicate that this algorithm is not an advance in computer science.”

Viten'ko [1199, 1968] shows that under the no-guard-digit model (2.6) the summation method with the optimal error bound (in a certain sense defined in [1199, 1968]) is pairwise summation. This does not contradict Kahan's result because Kahan uses properties of the floating point arithmetic beyond those in the no-guard-digit model.

A good illustration of the benefits of compensated summation is provided by Euler's method for the ordinary differential equation initial value problem $y' = f(x, y)$, $y(a)$ given, which generates an approximate solution according to $y_{k+1} = y_k + hf_k$, $y_0 = y(a)$. We solved the equation $y' = -y$ with $y(0) = 1$ over $[0, 1]$ using n steps of Euler's method ($nh = 1$), with n ranging from 10 to 10^8 . With compensated summation we replace the statements $x = x + h$, $y = y + h * f(x, y)$ by (with the initialization $cx = 0$, $cy = 0$)

```

dx = h + cx
new_x = x + dx
cx = (x - new_x) + dx
x = new_x

```

```

dy = h * f(x, y) + cy
new_y = y + dy
cy = (y - new_y) + dy
y = new_y

```

Figure 4.2 shows the errors $e_n = |y(1) - \hat{y}_n|$, where $\hat{y}_n$ is the computed approximation to $y(1)$. The computations were done in Fortran 90 in single precision arithmetic on a Sun SPARCstation ($u \approx 6 \times 10^{-8}$). Since Euler's method has global error of order h , the error curve on the plot should be approximately a straight line. For the standard implementation of Euler's method the errors e_n start to increase steadily beyond $n = 20000$ because of the influence of rounding errors. With compensated summation the errors e_n are much less affected by rounding errors and do not grow in the range of n shown (for $n = 10^8$, e_n is about 10 times larger than it would be in exact arithmetic). Plots of U-shaped curves showing total error against stepsize are common in numerical analysis textbooks (see, e.g., Forsythe, Malcolm, and Moler [430, 1977, p. 119] and Shampine [1030, 1994, p. 259]), but the textbooks rarely point out that the “U” can be flattened out by compensated summation.

The cost of applying compensated summation in an ordinary differential equation solver is almost negligible if the function f is at all expensive to evaluate. But, of course, the benefits it brings are noticeable only when a vast number of

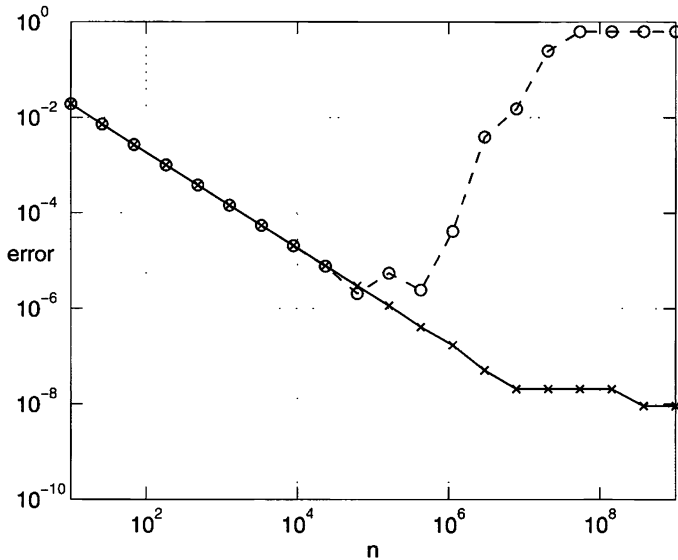


Figure 4.2. Errors $|y(1) - \hat{y}_n|$ for Euler's method with ("x") and without ("o") compensated summation.

integration steps are taken. Very-long-term integrations are undertaken in celestial mechanics, where roundoff can affect the ability to track planetary orbits. Researchers in astronomy use compensated summation, and other techniques, to combat roundoff. An example application is a 3 million year integration of the planets in the solar system by Quinn, Tremaine, and Duncan [965, 1991]; it used a linear multistep method of order 13 with a constant stepsize of 0.75 days and took 65 days of machine time on a Silicon Graphics 4D-25 workstation. See also Quinn and Tremaine [964, 1990] and Quinlan [962, 1994].

Finally, we describe an even more ingenious algorithm called *doubly compensated summation*, derived by Priest [955, 1992] from a related algorithm of Kahan. It is compensated summation with two extra applications of the correction process⁸ and it requires 10 instead of 4 additions per step. The algorithm is tantamount to simulating double precision arithmetic with single precision arithmetic; it requires that the summands first be sorted into decreasing order, which removes the need for certain logical tests that would otherwise be necessary.

Algorithm 4.3 (doubly compensated summation). Given floating point numbers $x_1, \dots, x_n$ this algorithm forms the sum $s_n = \sum_{i=1}^n x_i$ by doubly compensated summation. All expressions should be evaluated in the order specified by the parentheses.

Sort the x_i so that $|x_1| \geq |x_2| \geq \dots \geq |x_n|$.
 $s_1 = x_1$; $c_1 = 0$
 for $k = 2$: n

⁸The algorithm should perhaps be called triply compensated summation, but we adopt Priest's terminology.


```

 $y_k = c_{k-1} + x_k$ 
 $u_k = x_k - (y_k - c_{k-1})$ 
 $t_k = y_k + s_{k-1}$ 
 $v_k = y_k - (t_k - s_{k-1})$ 
 $z_k = u_k + v_k$ 
 $s_k = t_k + z_k$ 
 $c_k = z_k - (s_k - t_k)$ 
end

```

Priest [955, 1992, §4.1] analyses this algorithm for t -digit base β arithmetic that satisfies certain reasonable assumptions—ones which are all satisfied by IEEE arithmetic. He shows that if $n \leq \beta^{t-3}$ then the computed sum $\hat{s}_n$ satisfies

$$|s_n - \hat{s}_n| \leq 2u|s_n|,$$

that is, the computed sum is accurate virtually to full precision.

4.4. Other Summation Methods

We mention briefly two further classes of summation algorithms. The first builds the sum in a series of accumulators, which are themselves added to give the sum. As originally described by Wolfe [1252, 1964] each accumulator holds a partial sum lying in a different interval. Each term x_i is added to the lowest-level accumulator; if that accumulator overflows it is added to the next-highest one and then reset to zero, and this cascade continues until no overflow occurs. Modifications of Wolfe's algorithm are presented by Malcolm [808, 1971] and Ross [993, 1965]. Malcolm [808, 1971] gives a detailed error analysis to show that his method achieves a relative error of order u . A drawback of the algorithm is that it is strongly machine dependent. An interesting and crucial feature of Malcolm's algorithm is that on the final step the accumulators are summed by recursive summation in order of *decreasing* absolute value, which in this particular situation precludes severe loss of significant digits and guarantees a small relative error.

Another class of algorithms, referred to as “distillation algorithms” by Kahan [695, 1987], work as follows: given $x_i = fl(x_i)$, $i = 1:n$, they iteratively construct floating point numbers $x_1^{(k)}, \dots, x_n^{(k)}$ such that $\sum_{i=1}^n x_i^{(k)} = \sum_{i=1}^n x_i$, terminating when $x_n^{(k)}$ approximates $\sum_{i=1}^n x_i$ with relative error at most u . Kahan states that these algorithms appear to have average run times of order at least $n \log n$. Anderson [23, 1999] gives a very readable description of a distillation algorithm and offers comments on earlier algorithms, including those of Bohlender [145, 1977], Leuprecht and Oberaigner [782, 1982], and Pichat [940, 1972]. See also Kahan [695, 1987] and Priest [955, 1992, pp. 66–69] for further details and references.

4.5. Statistical Estimates of Accuracy

The rounding error bounds presented above can be very pessimistic, because they account for the worst-case propagation of errors. An alternative way to compare

Table 4.1. *Mean square errors for nonnegative x_i .*

Distrib.	Increasing	Random	Decreasing	Insertion	Pairwise
Unif(0, 2μ)	$0.20\mu^2 n^3 \sigma^2$	$0.33\mu^2 n^3 \sigma^2$	$0.53\mu^2 n^3 \sigma^2$	$2.6\mu^2 n^2 \sigma^2$	$2.7\mu^2 n^2 \sigma^2$
Exp(μ)	$0.13\mu^2 n^3 \sigma^2$	$0.33\mu^2 n^3 \sigma^2$	$0.63\mu^2 n^3 \sigma^2$	$2.6\mu^2 n^2 \sigma^2$	$4.0\mu^2 n^2 \sigma^2$

summation methods is through statistical estimates of the error, which may be more representative of the average case. A statistical analysis of three summation methods has been given by Robertazzi and Schwartz [987, 1988] for the case of nonnegative x_i . They assume that the relative errors in floating point addition are statistically independent and have zero mean and finite variance σ^2 . Two distributions of nonnegative x_i are considered: the uniform distribution on $[0, 2\mu]$, and the exponential distribution with mean μ . Making various simplifying assumptions Robertazzi and Schwartz estimate the mean square error (that is, the variance of the absolute error) of the computed sums from recursive summation with random, increasing, and decreasing orderings, and from insertion summation and pairwise summation (with the increasing ordering). Their results for the summation of n numbers are given in Table 4.1.

The results show that for recursive summation the ordering affects only the constant in the mean square error, with the increasing ordering having the smallest constant and the decreasing ordering the largest; since the x_i are nonnegative, this is precisely the ranking given by the rounding error bound (4.3). The insertion and pairwise summation methods have mean square errors proportional to n^2 rather than n^3 for recursive summation, and the insertion method has a smaller constant than pairwise summation. This is also consistent with the rounding error analysis, in which for nonnegative x_i the insertion method satisfies an error bound no larger than pairwise summation and the latter method has an error bound with a smaller constant than for recursive summation ($\log_2 n$ versus n).

4.6. Choice of Method

There is a wide variety of summation methods to choose from. For each method the error can vary greatly with the data, within the freedom afforded by the error bounds; numerical experiments show that, given any two of the methods, data can be found for which either method is more accurate than the other [600, 1993]. However, some specific advice on the choice of method can be given.

1. If high accuracy is important, consider implementing recursive summation in higher precision; if feasible this may be less expensive (and more accurate) than using one of the alternative methods at the working precision. What can be said about the accuracy of the sum computed at higher precision? If $S_n = \sum_{i=1}^n x_i$ is computed by recursive summation at double precision (unit roundoff u^2) and then rounded to single precision, an error bound of the form $|S_n - \hat{S}_n| \leq u|\hat{S}_n| + nu^2 \sum_{i=1}^n |x_i|$ holds. Hence a relative error of order u is guaranteed if $nu \sum_{i=1}^n |x_i| \leq |S_n|$. Priest [955, 1992, pp. 62–

63] shows that if the x_i are sorted in decreasing order of magnitude before being summed in double precision, then $|S_n - \hat{S}_n| \leq 2u|S_n|$ holds provided only that $n \leq \beta^{t-3}$ for t -digit base β arithmetic satisfying certain reasonable assumptions. Therefore the decreasing ordering may be worth arranging if there is a lot of cancellation in the sum. An alternative to extra precision computation is doubly compensated summation, which is the only other method described here that guarantees a small relative error in the computed sum.

2. For most of the methods the errors are, in the worst case, proportional to n . If n is very large, pairwise summation (error constant $\log_2 n$) and compensated summation (error constant of order 1) are attractive.
3. If the x_i all have the same sign then all the methods yield a relative error of at most nu and compensated summation guarantees perfect relative accuracy (as long as $nu \leq 1$). For recursive summation of one-signed data, the increasing ordering has the smallest error bound (4.3) and the insertion method minimizes this error bound over all instances of Algorithm 4.1.
4. For sums with heavy cancellation ($\sum_{i=1}^n |x_i| \gg |\sum_{i=1}^n x_i|$), recursive summation with the decreasing ordering is attractive, although it cannot be guaranteed to achieve the best accuracy.

Considerations of computational cost and the way in which the data are generated may rule out some of the methods. Recursive summation in the natural order, pairwise summation, and compensated summation can be implemented in $O(n)$ operations for general x_i , but the other methods are more expensive since they require searching or sorting. Furthermore, in an application such as the numerical solution of ordinary differential equations, where x_k is not known until $\sum_{i=1}^{k-1} x_i$ has been formed, sorting and searching may be impossible.

4.7. Notes and References

This chapter is based on Higham [600, 1993]. Analysis of Algorithm 4.1 can also be found in Espelid [394, 1995].

The earliest error analysis of summation is that of Wilkinson for recursive summation in [1228, 1960], [1232, 1963].

Pairwise summation was first discussed by McCracken and Dorn [833, 1964, pp. 61–63], Babuška [43, 1969], and Linz [792, 1970]. Caprani [203, 1971] shows how to implement the method on a serial machine using temporary storage of size $\lfloor \log_2 n \rfloor + 1$ (without overwriting the x_i).

The use of compensated summation with a Runge–Kutta formula is described by Vitasek [1198, 1969]. See also Butcher [189, 1987, pp. 118–120] and the experiments of Linnainmaa [788, 1974]. Davis and Rabinowitz [295, 1984, §4.2.1] discuss pairwise summation and compensated summation in the context of quadrature.

Demmel [319, 2001] analyses how much extra precision is needed in recursive summation with the (approximately) decreasing ordering in order to guarantee a computed result correct to working precision.

Problems

4.1. Define and evaluate a condition number $C(x)$ for the summation $S_n(x) = \sum_{i=1}^n x_i$. When does the condition number take the value 1?

4.2. (Wilkinson [1232, 1963, p. 19]) Show that the bounds (4.3) and (4.4) are nearly attainable for recursive summation. (Hint: assume $u = 2^{-t}$, set $n = 2^r$ ($r \ll t$), and define

$$\begin{aligned} x(1) &= 1, \\ x(2) &= 1 - 2^{-t}, \\ x(3:4) &= 1 - 2^{1-t}, \\ x(5:8) &= 1 - 2^{2-t}, \\ &\vdots \\ x(2^{r-1} + 1:2^r) &= 1 - 2^{r-1-t}. \end{aligned}$$

4.3. Let $S_n = \sum_{i=1}^n x_i$ be computed by recursive summation in the natural order. Show that

$$\hat{S}_n = (x_1 + x_2)(1 + \theta_{n-1}) + \sum_{i=3}^n x_i(1 + \theta_{n-i+1}), \quad |\theta_k| \leq \gamma_k = \frac{ku}{1 - ku},$$

and hence that $E_n = \hat{S}_n - S_n$ satisfies

$$|E_n| \leq (|x_1| + |x_2|)\gamma_{n-1} + \sum_{i=3}^n |x_i|\gamma_{n-i+1}.$$

Which ordering of the x_i minimizes this bound?

4.4. Let M be a floating point number so large that $fl(10 + M) = M$. What are the possible values of $fl(\sum_{i=1}^6 x_i)$, where $\{x_i\}_{i=1}^6 = \{1, 2, 3, 4, M, -M\}$ and the sum is evaluated by recursive summation?

4.5. The “ $\pm$ ” method for computing $S_n = \sum_{i=1}^n x_i$ is defined as follows: form the sum of the positive numbers, S_+ , and the sum of the nonpositive numbers, S_- , separately, by any method, and then form $S_n = S_- + S_+$. Discuss the pros and cons of this method.

4.6. (Shewchuk [1038, 1997]) Consider correctly rounded binary arithmetic. Show that if a and b are floating point numbers then $\text{err}(a, b) = a + b - fl(a + b)$ satisfies $|\text{err}(a, b)| \leq \min(|a|, |b|)$. Hence show that, barring overflow in $fl(a + b)$, $\text{err}(a, b)$ is a floating point number.

4.7. Let $\{x_i\}$ be a convergent sequence with limit ξ . Aitken’s Δ^2 -method (Aitken extrapolation) generates a transformed sequence $\{y_i\}$ defined by

$$y_i := x_i - \frac{(x_{i+1} - x_i)^2}{x_{i+2} - 2x_{i+1} + x_i}.$$

Under suitable conditions (typically that $\{x_i\}$ is linearly convergent), the y_i converge to ξ faster than the x_i . Which of the following expressions should be used to evaluate the denominator in the formula for y_i ?

- (a) $(x_{i+2} - 2x_{i+1}) + x_i$.
- (b) $(x_{i+2} - x_{i+1}) - (x_{i+1} - x_i)$.
- (c) $(x_{i+2} + x_i) - 2x_{i+1}$.

4.8. Analyse the accuracy of the following method for evaluating $S_n = \sum_{i=1}^n x_i$:

$$S_n = \log \prod_{i=1}^n e^{x_i}.$$

4.9. In numerical methods for quadrature and for solving ordinary differential equation initial value problems it is often necessary to evaluate a function on an equally spaced grid of points on a range $[a, b]$: $x_i := a + ih$, $i = 0:n$, where $h = (b - a)/n$. Compare the accuracy of the following ways to form x_i . Assume that a and b , but not necessarily h , are floating point numbers.

- (a) $x_i = x_{i-1} + h$ ($x_0 = a$).
- (b) $x_i = a + ih$.
- (c) $x_i = a(1 - i/n) + (i/n)b$.

Note that (a) is typically used without comment in, for example, a Newton–Cotes quadrature rule or a Runge–Kutta method with fixed stepsize.

4.10. (RESEARCH PROBLEM) Priest [955, 1992, pp. 61–62] has proved that if $|x_1| \geq |x_2| \geq |x_3|$ then compensated summation computes the sum $x_1 + x_2 + x_3$ with a relative error of order u (under reasonable assumptions on the arithmetic, such as the presence of a guard digit). He also gives the example

$$x_1 = 2^{t+1}, \quad x_2 = 2^{t+1} - 2, \quad x_3 = x_4 = x_5 = x_6 = -(2^t - 1),$$

for which the exact sum is 2 but compensated summation computes 0 in IEEE single precision arithmetic ($t = 24$). What is the smallest n for which compensated summation applied to $x_1, \dots, x_n$ ordered by decreasing absolute value can produce a computed sum with large relative error?

Chapter 5

Polynomials

The polynomial $(z - 1)(z - 2) \dots (z - 20)$ is not a 'difficult' polynomial per se . . .

The 'difficulty' with the polynomial $\prod (z - i)$ is that of evaluating the explicit polynomial accurately.

If one already knows the roots, then the polynomial can be evaluated without any loss of accuracy.

— J. H. WILKINSON, *The Perfidious Polynomial* (1984)

I first used backward error analysis in connection with simple programs for computing zeros of polynomials soon after the PILOT ACE came into use.

— J. H. WILKINSON, *The State of the Art in Error Analysis* (1985)

The Fundamental Theorem of Algebra asserts that every polynomial equation over the complex field has a root.

It is almost beneath the dignity of such a majestic theorem to mention that in fact it has precisely n roots.

— J. H. WILKINSON, *The Perfidious Polynomial* (1984)

It can happen . . . that a particular polynomial can be evaluated accurately by nested multiplication, whereas evaluating the same polynomial by an economical method may produce relatively inaccurate results.

— C. T. FIKE, *Computer Evaluation of Mathematical Functions* (1968)

Two common tasks associated with polynomials are evaluation and interpolation: given the polynomial find its values at certain arguments, and given the values at certain arguments find the polynomial. We consider Horner's rule for evaluation and the Newton divided difference polynomial for interpolation. A third task not considered here is finding the zeros of a polynomial. Much research was devoted to polynomial zero finding up until the late 1960s; indeed, Wilkinson devotes a quarter of *Rounding Errors in Algebraic Processes* [1232, 1963] to the topic. Since the development of the QR algorithm for finding matrix eigenvalues there has been less demand for polynomial zero finding, since the problem either arises as, or can be converted to (see §28.6 and [383, 1995], [1144, 1994]), the matrix eigenvalue problem.

5.1. Horner's Method

The standard method for evaluating a polynomial

$$p(x) = a_0 + a_1x + \cdots + a_nx^n \quad (5.1)$$

is Horner's method (also known as Horner's rule and nested multiplication), which consists of the following recurrence:

$$\begin{aligned} q_n(x) &= a_n \\ \text{for } i &= n-1: -1: 0 \\ q_i(x) &= xq_{i+1}(x) + a_i \\ \text{end} \\ p(x) &= q_0(x) \end{aligned}$$

The cost is $2n$ flops, which is n less than the more obvious method of evaluation that explicitly forms powers of x (see Problem 5.2).

To analyse the rounding errors in Horner's method it is convenient to use the relative error counter notation $\langle k \rangle$ (see (3.10)). We have

$$\begin{aligned} \hat{q}_{n-1} &= (x\hat{q}_n\langle 1 \rangle + a_{n-1})\langle 1 \rangle \\ &= xa_n\langle 2 \rangle + a_{n-1}\langle 1 \rangle, \\ \hat{q}_{n-2} &= (x\hat{q}_{n-1}\langle 1 \rangle + a_{n-2})\langle 1 \rangle \\ &= x^2a_n\langle 4 \rangle + xa_{n-1}\langle 3 \rangle + a_{n-2}\langle 1 \rangle. \end{aligned}$$

It is easy to either guess or prove by induction that

$$\begin{aligned} \hat{q}_0 &= a_0\langle 1 \rangle + a_1x\langle 3 \rangle + \cdots + a_{n-1}x^{n-1}\langle 2n-1 \rangle + a_nx^n\langle 2n \rangle \\ &= (1 + \theta_1)a_0 + (1 + \theta_3)a_1x + \cdots + (1 + \theta_{2n-1})a_{n-1}x^{n-1} \\ &\quad + (1 + \theta_{2n})a_nx^n, \end{aligned} \quad (5.2)$$

where we have used Lemma 3.1, and where $|\theta_k| \leq ku/(1 - ku) =: \gamma_k$. This result shows that Horner's method has a small backward error: the computed $\hat{q}_0$ is the exact value at x of a polynomial obtained by making relative perturbations of size at most γ_{2n} to the coefficients of $p(x)$.

A forward error bound is easily obtained: from (5.2) we have

$$|p(x) - \hat{q}_0| \leq \gamma_{2n} \sum_{i=0}^n |a_i| |x|^i = \gamma_{2n} \tilde{p}(|x|), \quad (5.3)$$

where $\tilde{p}(x) = \sum_{i=0}^n |a_i| x^i$. The relative error is bounded according to

$$\frac{|p(x) - \hat{q}_0|}{|p(x)|} \leq \gamma_{2n} \frac{\tilde{p}(|x|)}{|p(x)|} =: \gamma_{2n} \psi(p, x).$$

Clearly, the factor $\psi(p, x)$ can be arbitrarily large. However, $\psi(p, x) = 1$ if $a_i \geq 0$ for all i and $x \geq 0$, or if $(-1)^i a_i \geq 0$ for all i and $x \leq 0$.

In a practical computation we may wish to compute an error bound along with $\hat{q}_0$. The bound (5.3) is entirely adequate for theoretical purposes and can itself be computed by Horner's method. However, it lacks sharpness for two reasons. First, the bound is the result of replacing each γ_k by γ_{2n} . Second, and more importantly, it is an a priori bound and so takes no account of the actual rounding errors that occur. We can derive a sharper, a posteriori bound by a running error analysis.

For the i th step of Horner's method we can write

$$(1 + \epsilon_i) \hat{q}_i = x \hat{q}_{i+1} (1 + \delta_i) + a_i, \quad |\delta_i|, |\epsilon_i| \leq u, \quad (5.4)$$

where we have used both (2.4) and (2.5). Defining $\hat{q}_i =: q_i + f_i$, we have

$$q_i + f_i + \epsilon_i \hat{q}_i = x(q_{i+1} + f_{i+1}) + x \hat{q}_{i+1} \delta_i + a_i,$$

or

$$f_i = x f_{i+1} + x \hat{q}_{i+1} \delta_i - \epsilon_i \hat{q}_i, \quad f_n = 0.$$

Hence

$$|f_i| \leq |x| |f_{i+1}| + u(|x| |\hat{q}_{i+1}| + |\hat{q}_i|).$$

Since $f_n = 0$, we have $|f_i| \leq u \pi_i$, where

$$\pi_i = |x| \pi_{i+1} + |x| |\hat{q}_{i+1}| + |\hat{q}_i|, \quad \pi_n = 0.$$

We can slightly reduce the cost of evaluating the majorizing sequence π_i by working with $\mu_i := \frac{1}{2}(\pi_i + |\hat{q}_i|)$, which satisfies the recurrence

$$\mu_i = |x| \mu_{i+1} + |\hat{q}_i|, \quad \mu_n = \frac{1}{2} |\hat{q}_n|.$$

We can now furnish Horner's method with a running error bound.

Algorithm 5.1. This algorithm evaluates $y = fl(p(x))$ by Horner's method, where $p(x) = \sum_{i=0}^n a_i x^i$. It also evaluates a quantity μ such that $|y - p(x)| \leq \mu$.

```

y = a_n
μ = |y|/2
for i = n - 1: -1: 0
    y = xy + a_i
    μ = |x|μ + |y|
end
μ = u(2μ - |y|)

```

Cost: $4n$ flops.

It is worth commenting on the case where one or more of the a_i and x is complex. The analysis leading to Algorithm 5.1 is still valid for complex data, but we need to remember that the error bounds for $fl(xopy)$ are not the same as for real arithmetic. In view of Lemma 3.5, it suffices to replace the last line of the algorithm by $\mu = \sqrt{2}\gamma_2(2\mu - |y|)$. An increase in speed of the algorithm, with only a slight worsening of the bound, can be obtained by replacing $|y| = ((\operatorname{Re} y)^2 + (\operatorname{Im} y)^2)^{1/2}$ by $|\operatorname{Re} y| + |\operatorname{Im} y|$ (and, of course, $|x|$ should be evaluated once and for all before entering the loop).

One use of Algorithm 5.1 is to provide a stopping criterion for a polynomial zero-finder: if $|fl(p(x))|$ is of the same order as the error bound μ , then further iteration serves no purpose, for as far as we can tell, x could be an exact zero.

As a numerical example, for the expanded form of $p(x) = (x + 1)^{32}$ we found in MATLAB that

$$fl(p(-1)) \equiv 0, \quad \mu = 2.4 \times 10^{-7}, \quad \gamma_{2n}\tilde{p}(|x|) = 1.5 \times 10^{-5},$$

and for $p(x)$ the Chebyshev polynomial of degree 32,

$$fl(p(0.5)) \approx 0.5000, \quad \mu = 3.3 \times 10^{-10}, \quad \gamma_{2n}\tilde{p}(|x|) = 1.0 \times 10^{-8}.$$

In these two cases, the running error bound is, respectively, 62 and 31 times smaller than the a priori one.

In another experiment we evaluated the expanded form of $p(x) = (x - 2)^3$ in simulated single precision in MATLAB ($u \approx 6 \times 10^{-8}$) for 200 equally spaced points near $x = 2$. The polynomial values, the error, and the a priori and running error bounds are all plotted in Figure 5.1. The running error bound is about seven times smaller than the a priori one.

5.2. Evaluating Derivatives

Suppose now that we wish to evaluate derivatives of p . We could simply differentiate (5.1) as many times as necessary and apply Horner's method to each expression, but there is a more efficient way. Observe that if we define

$$q(x) = q_1 + q_2x + \cdots + q_nx^{n-1}, \quad r = q_0,$$

where the $q_i = q_i(\alpha)$ are generated by Horner's method for $p(\alpha)$, then

$$p(x) = (x - \alpha)q(x) + r.$$

In other words, Horner's method carries out the process of synthetic division. Clearly, $p'(\alpha) = q(\alpha)$. It is worth noting in passing that for $x \neq \alpha$,

$$q(x) = \frac{p(x) - p(\alpha)}{x - \alpha},$$

that is, q is a divided difference. If we repeat synthetic division recursively on $q(x)$, we will be evaluating the coefficients in the Taylor expansion

$$p(x) = p(\alpha) + (x - \alpha)p'(\alpha) + \frac{(x - \alpha)^2}{2!}p''(\alpha) + \cdots + \frac{(x - \alpha)^n}{n!}p^{(n)}(\alpha),$$

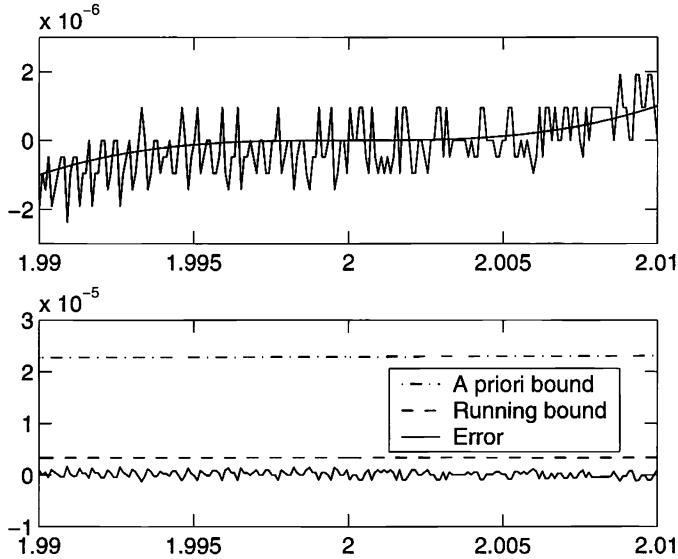


Figure 5.1. *Computed polynomial values (top) and running and a priori bounds (bottom) for Horner's method.*

and after a final scaling by factorials, we will obtain the derivatives of p at α . The resulting algorithm is quite short.

Algorithm 5.2. This algorithm evaluates the polynomial $p(x) = \sum_{i=0}^n a_i x^i$ and its first k derivatives at α , returning $y_i = p^{(i)}(\alpha)$, $i = 0:k$.

```

 $y_0 = a_n$ 
 $y(1:k) = 0$ 
for  $j = n - 1:-1:0$ 
  for  $i = \min(k, n - j):-1:1$ 
     $y_i = \alpha y_i + y_{i-1}$ 
  end
   $y_0 = \alpha y_0 + a_j$ 
end
 $m = 1$ 
for  $j = 2:k$ 
   $m = m * j$ 
   $y_j = m * y_j$ 
end

```

Cost: $nk + 2(k + n) - k^2/2$ flops.

How is the error bounded for the derivatives in Algorithm 5.2? To answer this question with the minimum of algebra, we express the algorithm in matrix notation. Horner's method for evaluating $p(\alpha)$ is equivalent to solution of the

bidiagonal system

$$U_{n+1}q := \begin{bmatrix} 1 & -\alpha & & & \\ & 1 & -\alpha & & \\ & & 1 & \ddots & \\ & & & \ddots & -\alpha \\ & & & & 1 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ \vdots \\ \vdots \\ q_n \end{bmatrix} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ \vdots \\ a_n \end{bmatrix} =: a.$$

By considering (5.4), we see that

$$(U_{n+1} + \Delta_1)\hat{q} = a, \quad |\Delta_1| \leq u|U_{n+1}|.$$

Hence

$$|q - \hat{q}| \leq u|U_{n+1}^{-1}||U_{n+1}||q| + O(u^2). \quad (5.5)$$

The recurrence for $r_0 = p'(\alpha)$ can be expressed as $U_n r = q(1:n)$, where $r = r(0:n-1)$, so

$$(U_n + \Delta_2)\hat{r} = \hat{q}(1:n), \quad |\Delta_2| \leq u|U_n|.$$

Hence

$$\begin{aligned} \hat{r} &= (U_n^{-1} - U_n^{-1}\Delta_2 U_n^{-1})(q(1:n) + (\hat{q}(1:n) - q(1:n))) + O(u^2) \\ &= r - U_n^{-1}\Delta_2 r + U_n^{-1}(\hat{q}(1:n) - q(1:n)) + O(u^2). \end{aligned}$$

This gives, using (5.5),

$$|r - \hat{r}| \leq u|U_n^{-1}||U_n||r| + u|U_n^{-1}||U_n^{-1}||U_n||q(1:n)| + O(u^2). \quad (5.6)$$

Now

$$\begin{aligned} |U_n^{-1}| &= \begin{bmatrix} 1 & |\alpha| & |\alpha|^2 & \dots & |\alpha|^{n-1} \\ & 1 & |\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & |\alpha| \\ & & & & 1 \end{bmatrix}, \\ |U_n^{-1}||U_n| &= \begin{bmatrix} 1 & 2|\alpha| & 2|\alpha|^2 & \dots & 2|\alpha|^{n-1} \\ & 1 & 2|\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & 2|\alpha| \\ & & & & 1 \end{bmatrix}, \\ |U_n^{-1}||U_n^{-1}||U_n| &= \begin{bmatrix} 1 & 3|\alpha| & 5|\alpha|^2 & \dots & (2n-1)|\alpha|^{n-1} \\ & 1 & 3|\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & 3|\alpha| \\ & & & & 1 \end{bmatrix}. \end{aligned}$$

By looking at the form of r and q , we find from (5.6) that

$$\begin{aligned} |p'(\alpha) - \widehat{r}_0| &\leq 2u \sum_{k=1}^n k^2 |a_k| |\alpha|^{k-1} + O(u^2) \\ &\leq 2nu \sum_{k=1}^n k |a_k| |\alpha|^{k-1} + O(u^2) \\ &=: 2nu \widetilde{p}'(\alpha) + O(u^2). \end{aligned} \quad (5.7)$$

This is essentially the same form of bound as for $p(\alpha)$ in (5.3). Analogous bounds hold for all derivatives.

5.3. The Newton Form and Polynomial Interpolation

An alternative to the monomial representation of a polynomial is the Newton form

$$p(x) = \sum_{i=0}^n c_i \prod_{j=0}^{i-1} (x - \alpha_j), \quad (5.8)$$

which is commonly used for polynomial interpolation. The interpolation problem is to choose p so that $p(\alpha_i) = f_i$, $i = 0:n$, and the numbers c_i are known as divided differences. Assuming that the points α_j are distinct, the divided differences may be computed from a standard recurrence:

```

 $c^{(0)}(0:n) = f(0:n)$ 
for  $k = 0:n-1$ 
  for  $j = n:-1:k+1$ 
     $c_j^{(k+1)} = (c_j^{(k)} - c_{j-1}^{(k)}) / (\alpha_j - \alpha_{j-k-1})$ 
  end
end
 $c = c^{(n)}$ 

```

Cost: $3n^2/2$ flops.

Two questions are of interest: how accurate are the computed $\widehat{c}_j$, and what is the effect of rounding errors on the polynomial values obtained by evaluating the Newton form? To answer the first question we express the recurrence in matrix-vector form:

$$c^{(0)} = f, \quad c^{(k+1)} = L_k c^{(k)}, \quad k = 0:n-1,$$

where $L_k = D_k^{-1} M_k$ is lower bidiagonal, with

$$D_k = \text{diag}(\text{ones}(1:k+1), \alpha_{k+1} - \alpha_0, \alpha_{k+2} - \alpha_1, \dots, \alpha_n - \alpha_{n-k-1}),$$

$$M_k = \begin{bmatrix} I_k & & & & \\ & 1 & & & \\ & -1 & 1 & & \\ & & -1 & \ddots & \\ & & & \ddots & \ddots \\ & & & & -1 & 1 \end{bmatrix}.$$

It is straightforward to show that

$$\widehat{c}^{(k+1)} = G_k L_k \widehat{c}^{(k)}, \quad (5.9)$$

where $G_k = \text{diag}(\text{ones}(1:k+1), \eta_{k,k+2}, \dots, \eta_{k,n+1})$, where each η_{ij} is of the form $\eta_{ij} = (1 + \delta_1)(1 + \delta_2)(1 + \delta_3)$, $|\delta_i| \leq u$. Hence

$$\widehat{c} = (L_{n-1} + \Delta L_{n-1}) \dots (L_0 + \Delta L_0) f, \quad |\Delta L_i| \leq \gamma_3 |L_i|. \quad (5.10)$$

From Lemma 3.8,

$$\begin{aligned} |c - \widehat{c}| &= |(L_{n-1} \dots L_0 - (L_{n-1} + \Delta L_{n-1}) \dots (L_0 + \Delta L_0)) f| \\ &\leq ((1 + \gamma_3)^n - 1) |L_{n-1}| \dots |L_0| |f| \\ &= ((1 - 3u)^{-n} - 1) |L_{n-1}| \dots |L_0| |f|. \end{aligned} \quad (5.11)$$

To interpret the bound, note first that merely rounding the data ($f_i \rightarrow f_i(1 + \delta_i)$, $|\delta_i| \leq u$) can cause an error Δc as large as $e_{\text{round}} = u|L||f|$, where $L = L_{n-1} \dots L_0$, so errors of at least this size are inevitable. Since $|L_{n-1}| \dots |L_0| \geq |L_{n-1} \dots L_0| = |L|$, the error in the computed divided differences can be larger than e_{round} only if there is much subtractive cancellation in the product $L = L_{n-1} \dots L_0$. If $\alpha_0 < \alpha_1 < \dots < \alpha_n$ then each L_i is positive on the diagonal and nonpositive on the first subdiagonal; therefore $|L_{n-1}| \dots |L_0| = |L_{n-1} \dots L_0| = |L|$, and we have the very satisfactory bound $|c - \widehat{c}| \leq ((1 - 3u)^{-n} - 1)|L||f|$. This same bound holds if the α_i are arranged in decreasing order.

To examine how well the computed Newton form reproduces the f_i we “unwind” the analysis above. From (5.9) we have

$$\widehat{c}^{(k)} = L_k^{-1} G_k^{-1} \widehat{c}^{(k+1)} = (L_k^{-1} + \widetilde{\Delta L_k}) \widehat{c}^{(k+1)}, \quad |\widetilde{\Delta L_k}| \leq \gamma_3 |L_k^{-1}|.$$

By invoking Lemma 3.8 again, we obtain

$$|f - L^{-1} \widehat{c}| \leq ((1 - 3u)^{-n} - 1) |L_0^{-1}| \dots |L_{n-1}^{-1}| |\widehat{c}|. \quad (5.12)$$

If $\alpha_0 < \alpha_1 < \dots < \alpha_n$ then $L_i^{-1} \geq 0$ for all i , and we obtain the very satisfactory bound $|f - L^{-1} \widehat{c}| \leq ((1 - 3u)^{-n} - 1) |L^{-1}| |\widehat{c}|$. Again, the same bound holds for points arranged in decreasing order.

In practice it is found that even when the computed divided differences are very inaccurate, the computed interpolating polynomial may still reproduce the original data well. The bounds (5.11) and (5.12) provide insight into this observed behaviour by showing that $c - \widehat{c}$ and $f - L^{-1} \widehat{c}$ can be large only when there is much cancellation in the products $L_{n-1} \dots L_0 f$ and $L_0^{-1} \dots L_{n-1}^{-1} c$, respectively.

The analysis has shown that the ordering $\alpha_0 < \alpha_1 < \dots < \alpha_n$ yields “optimal” error bounds for the divided differences and the residual, and so may be a good choice of ordering of interpolation points. However, if the aim is to minimize $|p(x) - fl(p(x))|$ for a given $x \neq \alpha_j$, then other orderings need to be considered. An ordering with some theoretical support is the Leja ordering, which is defined by the equations [975, 1990]

$$\alpha_0 = \max_i |\alpha_i|, \quad (5.13a)$$

$$\prod_{k=0}^{j-1} |\alpha_j - \alpha_k| = \max_{i \geq j} \prod_{k=0}^{j-1} |\alpha_i - \alpha_k|, \quad j = 1: n-1. \quad (5.13b)$$

For a given set of $n + 1$ points α_i , the Leja ordering can be computed in n^2 flops (see Problem 5.4).

We give a numerical example to illustrate the analysis. Let $n = 16$ and let $\alpha_0 < \dots < \alpha_n$ be equally spaced points on $[-1, 1]$. Working in simulated single precision with $u = 2^{-24} \approx 6 \times 10^{-8}$, we computed divided differences for two different vectors f . Error statistics were computed by regarding the solutions computed in double precision as exact. We define the ratios

$$\rho_1 = \max_i \frac{(|L_{n-1}| \dots |L_0| |f|)_i}{|c_i|}, \quad \rho_2 = \max_i \frac{(|L_0^{-1}| \dots |L_{n-1}^{-1}| |\hat{c}|)_i}{|f_i|}.$$

(1) For f_i from the normal $N(0, 1)$ distribution the divided differences range in magnitude from 1 to 10^5 , and their relative errors range from 0 (the first divided difference, f_0 , is always exact) to 3×10^{-7} . The ratio $\rho_1 = 16.3$, so (5.11) provides a reasonably sharp bound for the error in $\hat{c}$. The relative errors when f is reconstructed from the computed divided differences range between 0 and 3×10^{-1} (it makes little difference whether the reconstruction is done in single or double precision). Again, this is predicted by the analysis, in this case by (5.12), because $\rho_2 = 2 \times 10^7$. For the Leja ordering, the divided differences are computed with about the same accuracy, but f is reconstructed much more accurately, with maximum relative error 7×10^{-6} ($\rho_1 = 1 \times 10^3$, $\rho_2 = 8 \times 10^4$).

(2) For $f_i = \exp(\alpha_i)$, the situation is reversed: we obtain inaccurate divided differences but an accurate reconstruction of f . The divided differences range in magnitude from 10^{-4} to 10^{-1} , and their relative errors are as large as 1, but the relative errors in the reconstructed f are all less than 10^{-7} . Again, the error bounds predict this behaviour: $\rho_1 = 6 \times 10^8$, $\rho_2 = 1.02$. The Leja ordering performs similarly.

The natural way to evaluate the polynomial (5.8) for a given x is by a generalization of Horner's method:

```

 $q_n(x) = c_n$ 
for  $i = n - 1 : -1 : 0$ 
     $q_i(x) = (x - \alpha_i)q_{i+1}(x) + c_i$ 
end
 $p(x) = q_0(x)$ 

```

A straightforward analysis shows that (cf. (5.2))

$$\begin{aligned} \hat{q}_0 &= c_0 \langle 1 \rangle + (x - \alpha_0) c_1 \langle 4 \rangle + (x - \alpha_0)(x - \alpha_1) c_2 \langle 7 \rangle + \dots \\ &\quad + (x - \alpha_0) \dots (x - \alpha_{n-2}) c_{n-1} \langle 3n - 2 \rangle \\ &\quad + (x - \alpha_0) \dots (x - \alpha_{n-1}) c_n \langle 3n \rangle. \end{aligned}$$

Hence the computed $\hat{q}_0$ is the exact value corresponding to a polynomial with slightly perturbed divided differences. The corresponding forward error bound is

$$|p(x) - \hat{q}_0| \leq \gamma_{3n} \sum_{i=0}^n |c_i| \prod_{j=0}^{i-1} |x - \alpha_j|.$$

5.4. Matrix Polynomials

The scalar polynomial (5.1) can be generalized to a matrix polynomial in three ways: to a matrix polynomial with scalar coefficients and matrix argument,

$$P_1(x) = a_0I + a_1X + \cdots + a_nX^n, \quad a_i \in \mathbb{C}, \quad X \in \mathbb{C}^{m \times m};$$

to a matrix polynomial with matrix coefficients and scalar argument,

$$P_2(x) = A_0 + A_1x + \cdots + A_nx^n, \quad A_i \in \mathbb{C}^{m \times m}, \quad x \in \mathbb{C};$$

and to a matrix polynomial with matrix coefficients and matrix argument,

$$P_3(x) = A_0 + A_1X + \cdots + A_nX^n, \quad A_i \in \mathbb{C}^{m \times m}, \quad X \in \mathbb{C}^{m \times m}. \quad (5.14)$$

(We could also consider an analogue of P_3 in which the coefficient matrices A_i appear to the right of the powers of X .) Each type of polynomial arises in applications: P_1 in the approximation of matrix functions such as the exponential and logarithm [868, 1978], [509, 1996, Chap. 11], [615, 2001]; P_2 in the polynomial eigenvalue problem [1139, 2001]; and P_3 in quasi-birth-death processes in Markov chains and in numerical solution of the polynomial eigenvalue problem [617, 2000].

Evaluation of P_2 (or, more commonly, P_2y for some vector y) is straightforward. To evaluate P_1 and P_3 Horner's method can be used, but for P_1 Horner's method is not necessarily of optimal efficiency, even for a single evaluation. In fact, a method of Paterson and Stockmeyer can evaluate P_1 in a number of matrix multiplications proportional to $\sqrt{n}$, at the cost of extra storage proportional to $m^2\sqrt{n}$ elements [928, 1973], [509, 1996, §11.2.4]. The storage can be reduced to a small constant multiple of n^2 by a modification of Van Loan, which slightly increases the cost [1179, 1979].

Error analysis of Horner's method and of the Paterson–Stockmeyer method for evaluating P_1 and P_3 is straightforward; see Problem 5.6.

5.5. Notes and References

Backward and forward error analysis for Horner's rule was given by Wilkinson [1232, 1963, pp. 36–37, 49–50]; our results are simply Wilkinson's presented in a different notation. The analysis has been redone by many other authors, sometimes without reference to Wilkinson's results. Another early reference, which gives a forward error bound only, is McCracken and Dorn [833, 1964, §3.5],

For more on running error analysis see §3.3.

Müller [877, 1983] gives a first-order error analysis for the evaluation of the divided difference form of a polynomial. Olver [905, 1986] derives a posteriori error bounds for the Horner scheme with derivatives (Algorithm 5.2), phrasing them in terms of his relative precision notation. Stewart [1063, 1971] analyses synthetic division, using a matrix-oriented approach similar to that in §5.2.

The relative merits of the monomial and Chebyshev representations of a polynomial are investigated, with respect to accuracy of evaluation, by Newbery [888, 1974] and Schonfelder and Razaz [1020, 1980]. Clenshaw [239, 1955] showed how Horner's method could be extended to evaluate a polynomial expressed in the

Chebyshev form $p(x) = \sum_{i=0}^n a_i T_i(x)$, where T_i is the Chebyshev polynomial of degree i . Error analysis of Clenshaw's method, and variations of it, are given by Gentleman [472, 1969], Newbery [887, 1973], and Oliver [901, 1977], [902, 1979]. Clenshaw's scheme can be generalized to expansions in terms of arbitrary orthogonal polynomials; see Smith [1050, 1965] and Algorithm 22.8.

Running error bounds for Horner's method were included in algorithms of Kahan and Farkas [703, 1963], [704, 1963] without explanation. Adams [6, 1967] derives the bounds and extends them to evaluation of a real polynomial at a complex argument. Algorithm 5.1 is given in [6, 1967], and also in the classic paper by Peters and Wilkinson [937, 1971], which describes many aspects of the solution of polynomial equations. Wilkinson's paper "The Perfidious Polynomial" [1247, 1984] (for which he was awarded the Chauvenet Prize) is highly recommended as a beautifully written introduction to backward error analysis in general and error analysis for polynomials in particular.

There seems to be little work on choosing the ordering of interpolation points to minimize the effect of rounding errors on the construction or evaluation of the interpolating polynomial. Werner [1216, 1984] examines experimentally the effect of different orderings on the computed value of an interpolating polynomial at a single point, for several forms of interpolating polynomial.

The Leja ordering, which was proposed by Leja in a 1957 paper, is analysed in detail by Reichel [975, 1990]. He shows that an appropriately defined condition number for the Newton form of interpolating polynomial grows at a slower than exponential rate in the degree n for Leja points, which are points taken from a given compact set that satisfy the condition (5.13). For more on the numerical benefits of the Leja ordering see §22.3.3.

Eğecioglu, Gallopoulos, and Koç [396, 1990] give a parallel algorithm, based on the parallel prefix operation, for computing divided differences and evaluating the interpolating polynomial and they give a rounding error analysis of the algorithm.

If a polynomial is to be evaluated many times at different arguments it may be worthwhile to expend some effort transforming it to a form that can be evaluated more cheaply than by a straightforward application of Horner's rule. For example, the quartic

$$p(x) = a_4 x^4 + a_3 x^3 + a_2 x^2 + a_1 x + a_0, \quad a_4 \neq 0,$$

can be rewritten as [744, 1998, Sec. 4.6.4]

$$p(x) = ((y + x + \alpha_2)y + \alpha_3)\alpha_4, \quad y = (x + \alpha_0)x + \alpha_1,$$

where the coefficients α_i are given by

$$\alpha_0 = \frac{1}{2}(a_3/a_4 - 1), \quad \beta = a_2/a_4 - \alpha_0(\alpha_0 + 1), \quad \alpha_1 = a_1/a_4 - \alpha_0\beta, \\ \alpha_2 = \beta - 2\alpha_1, \quad \alpha_3 = a_0/a_4 - \alpha_1(\alpha_1 + \alpha_2), \quad \alpha_4 = a_4.$$

Once the α_i have been computed, $p(x)$ can be evaluated in three multiplications and five additions, as compared with the four multiplications and four additions required by Horner's rule. If a multiplication takes longer than an addition, the transformed polynomial should be cheaper to evaluate. For polynomials of degree

$n > 4$ there exist evaluation schemes that require strictly less than the $2n$ total additions and multiplications required by Horner's rule; see Knuth [741, 1962], [744, 1998, Sec. 4.6.4] and Fike [412, 1967]. One application in which such schemes have been used is in evaluating polynomial approximations in an elementary function library [451, 1991]. Little seems to be known about the numerical stability of fast polynomial evaluation schemes; see Problem 5.7.

Problems

5.1. Give an alternative derivation of Algorithm 5.2 by differentiating the Horner recurrence and rescaling the iterates.

5.2. Give an error analysis for the following “beginner's” algorithm for evaluating $p(x) = a_0 + a_1x + \cdots + a_nx^n$:

```

 $q(x) = a_0; y = 1$ 
for  $i = 1:n$ 
     $y = xy$ 
     $q(x) = q(x) + a_iy$ 
end
 $p(x) = q(x)$ 

```

5.3. Let $p(x) = a_0 + a_1x + \cdots + a_nx^n$ and $n = 2m$. Then

$$\begin{aligned} p(x) &= (a_0 + a_2x^2 + \cdots + a_{2m}x^{2m}) + (a_1x + a_3x^3 + \cdots + a_{2m-1}x^{2m-1}) \\ &= a_0 + a_2y + \cdots + a_{2m}y^m + x(a_1 + a_3y + \cdots + a_{2m-1}y^{m-1}), \end{aligned}$$

where $y = x^2$. Obtain an error bound for $fl(p(x))$ when p is evaluated using this splitting (using Horner's rule on each part).

5.4. Write down an algorithm for computing the Leja ordering (5.13) in n^2 flops.

5.5. If the polynomial $p(x) = \sum_{i=0}^n a_i x^i$ has roots $x_1, \dots, x_n$, it can be evaluated from the root product form $p(x) = a_n \prod_{i=1}^n (x - x_i)$. Give an error analysis for this evaluation.

5.6. Show that the computed polynomial $\hat{P}_3$ from Horner's method applied to P_3 in (5.14) satisfies

$$\|P_3 - \hat{P}_3\| \leq n(m+1)u\tilde{p}_3(\|X\|) + O(u^2),$$

where $\tilde{p}_3(x) = \sum_{i=0}^n \|A_k\| x^k$ and the norm is the 1-norm or the ∞ -norm.

5.7. (RESEARCH PROBLEM) Investigate the numerical stability of fast polynomial evaluation schemes (see the Notes and References) by both rounding error analysis and numerical experiments. For a brief empirical study see Miller [851, 1975, §10].

Chapter 6

Norms

*While it is true that all norms are equivalent theoretically,
only a homely one like the ∞ -norm is truly useful numerically.*
— J. H. WILKINSON⁹, *Lecture at Stanford University* (1984)

*Matrix norms are defined in many different ways in the older literature,
but the favorite was the Euclidean norm of the matrix
considered as a vector in n^2 -space.
Wedderburn (1934) calls this the absolute value of the matrix
and traces the idea back to Peano in 1887.*
— ALSTON S. HOUSEHOLDER,
The Theory of Matrices in Numerical Analysis (1964)

⁹Quoted in Fox [439, 1987].

Norms are an indispensable tool in numerical linear algebra. Their ability to compress the mn numbers in an $m \times n$ matrix into a single scalar measure of size enables perturbation results and rounding error analyses to be expressed in a concise and easily interpreted form. In problems that are badly scaled, or contain a structure such as sparsity, it is often better to measure matrices and vectors componentwise. But norms remain a valuable instrument for the error analyst, and in this chapter we describe some of their most useful and interesting properties.

6.1. Vector Norms

A vector norm is a function $\|\cdot\| : \mathbb{C}^n \rightarrow \mathbb{R}$ satisfying the following conditions:

1. $\|x\| \geq 0$ with equality iff $x = 0$.
2. $\|\alpha x\| = |\alpha| \|x\|$ for all $\alpha \in \mathbb{C}$, $x \in \mathbb{C}^n$.
3. $\|x + y\| \leq \|x\| + \|y\|$ for all $x, y \in \mathbb{C}^n$ (the triangle inequality).

The three most useful norms in error analysis and in numerical computation are

$$\begin{aligned}\|x\|_1 &= \sum_{i=1}^n |x_i|, \quad \text{"Manhattan" or "taxi cab" norm,} \\ \|x\|_2 &= \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2} = (x^*x)^{1/2}, \quad \text{Euclidean length,} \\ \|x\|_\infty &= \max_{1 \leq i \leq n} |x_i|.\end{aligned}$$

These are all special cases of the Hölder p -norm:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad p \geq 1.$$

The 2-norm has two properties that make it particularly useful for theoretical purposes. First, it is invariant under unitary transformations, for if $Q^*Q = I$, then $\|Qx\|_2^2 = x^*Q^*Qx = x^*x = \|x\|_2^2$. Second, the 2-norm is differentiable for all x , with gradient vector $\nabla\|x\|_2 = x/\|x\|_2$.

A fundamental inequality for vectors is the Hölder inequality (see, for example, [547, 1967, App. 1])

$$|x^*y| \leq \|x\|_p \|y\|_q, \quad \frac{1}{p} + \frac{1}{q} = 1. \quad (6.1)$$

This is an equality when $p, q > 1$ if the vectors $(|x_i|^p)$ and $(|y_i|^q)$ are linearly dependent and $x_i y_i$ lies on the same ray in the complex plane for all i ; equality is also possible when $p = 1$ and $p = \infty$, as is easily verified. The special case with $p = q = 2$ is called the Cauchy-Schwarz inequality:

$$|x^*y| \leq \|x\|_2 \|y\|_2.$$

For an arbitrary vector norm $\|\cdot\|$ the *dual norm* is defined by

$$\|x\|_D = \max_{z \neq 0} \frac{|z^*x|}{\|z\|}. \quad (6.2)$$

It follows from the Hölder inequality that the dual of the p -norm is the q -norm, where $p^{-1} + q^{-1} = 1$. The definition of dual norm yields, trivially, the general Hölder inequality $|x^*y| \leq \|x\| \|y\|_D$. For a proof of the reassuring result that the dual of the dual norm is the original norm (the “duality theorem”) see Horn and Johnson [636, 1985, Thm. 5.5.14].

In some analyses we need the vector z *dual to* y , which is defined by the property

$$z^*y = \|z\|_D \|y\| = 1. \quad (6.3)$$

That such a vector z exists is a consequence of the duality theorem (see [636, 1985, Cor. 5.5.15]).

How much two p -norms of a vector can differ is shown by the attainable inequalities [462, 1983, pp. 27–28], [498, 1983, Lem. 1.1]

$$\|x\|_{p_2} \leq \|x\|_{p_1} \leq n^{(\frac{1}{p_1} - \frac{1}{p_2})} \|x\|_{p_2}, \quad p_1 \leq p_2. \quad (6.4)$$

The p -norms have the properties that $\|x\|$ depends only on the absolute value of x , and the norm is an increasing function of the absolute values of the entries of x . These properties are important enough to warrant a definition.

Definition 6.1. A norm on $\mathbb{C}^n$ is

1. *monotone* if $|x| \leq |y| \Rightarrow \|x\| \leq \|y\|$ for all $x, y \in \mathbb{C}^n$, and
2. *absolute* if $\| |x| \| = \|x\|$ for all $x \in \mathbb{C}^n$.

The following nonobvious theorem shows that these two properties are equivalent.

Theorem 6.2 (Bauer, Stoer, and Witzgall). *A norm on $\mathbb{C}^n$ is monotone if and only if it is absolute.*

Proof. See Horn and Johnson [636, 1985, Thm. 5.5.10], or Stewart and Sun [1083, 1990, Thm. 2.1.3]. $\square$

6.2. Matrix Norms

A matrix norm is a function $\|\cdot\| : \mathbb{C}^{m \times n} \rightarrow \mathbb{R}$ satisfying obvious analogues of the three vector norm properties. The simplest example is the Frobenius norm

$$\|A\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \right)^{1/2} = (\text{trace}(A^*A))^{1/2}$$

(which is sometimes called the Euclidean norm and denoted $\|\cdot\|_E$).

A very important class of matrix norms are those *subordinate* to vector norms. Given a vector norm on $\mathbb{C}^n$, the corresponding subordinate matrix norm on $\mathbb{C}^{m \times n}$ is defined by

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}, \quad (6.5)$$

or, equivalently,

$$\|A\| = \max_{\|x\|=1} \|Ax\|.$$

(Strictly speaking, this definition uses two different norms: one on $\mathbb{C}^m$ in the numerator of (6.5) and one on $\mathbb{C}^n$ in the denominator. Thus the norm used in the definition is assumed to form a family defined on $\mathbb{C}^s$ for any s .)

For the 1-, 2-, and ∞ -vector norms it can be shown that

$$\begin{aligned} \|A\|_1 &= \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|, & \text{“max column sum”,} \\ \|A\|_\infty &= \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|, & \text{“max row sum”,} \\ \|A\|_2 &= (\rho(A^*A))^{1/2} = \sigma_{\max}(A), & \text{spectral norm,} \end{aligned}$$

where the spectral radius

$$\rho(B) = \max\{|\lambda| : \det(B - \lambda I) = 0\},$$

and where $\sigma_{\max}(A)$ denotes the largest singular value of A . To remember the formulae for the 1- and ∞ -norms, note that 1 is a vertical symbol (for columns) and ∞ is a horizontal symbol (for rows).

From the expression for the ∞ -norm it is immediate that

$$\|A\|_\infty = \| |A| e \|_\infty, \quad e = [1, 1, \dots, 1]^T,$$

where $|A| = (|a_{ij}|)$. This useful relation will be employed frequently in later chapters.

A norm is *consistent* if it satisfies $\|AB\| \leq \|A\| \|B\|$ whenever the product AB is defined. The Frobenius norm and all subordinate norms are consistent. An example of a norm that is not consistent is the “max norm” $\|A\| = \max_{i,j} |a_{ij}|$. The best bound that holds for all $A \in \mathbb{C}^{m \times n}$ and $B \in \mathbb{C}^{n \times p}$ is $\|AB\| \leq n \|A\| \|B\|$, with equality when $a_{ij} \equiv 1$ and $b_{ij} \equiv 1$.

A norm for which $\|UAV\| = \|A\|$ for all unitary U and V is called a *unitarily invariant norm*. These norms have an interesting theory, which we will not explore here (see [107, 1997, Chap. 4], [637, 1991, §3.5], or [1083, 1990, §2.3]). Only two unitarily invariant norms will be needed for our analysis: the 2-norm and the Frobenius norm. That these two norms are unitarily invariant follows easily from the formulae above. For any unitarily invariant norm, the useful property holds that $\|A^*\| = \|A\|$. The 2-norm satisfies the additional relation $\|A^*A\|_2 = \|A\|_2^2$.

The unitary invariance of the 2- and Frobenius norms has implications for error analysis, for it means that multiplication by unitary matrices does not magnify

Table 6.1. Constants α_{pq} such that $\|x\|_p \leq \alpha_{pq}\|x\|_q$, $x \in \mathbb{C}^n$.

		q		
		1	2	∞
p	1	1	$\sqrt{n}$	n
	2	1	1	$\sqrt{n}$
	∞	1	1	1

Table 6.2. Constants α_{pq} such that $\|A\|_p \leq \alpha_{pq}\|A\|_q$, $A \in \mathbb{C}^{m \times n}$. Here, $\|A\|_M := \max_{i,j} |a_{ij}|$ and $\|A\|_S := \sum_{i,j} |a_{ij}|$.

		q					
		1	2	∞	F	M	S
p	1	1	$\sqrt{m}$	m	$\sqrt{m}$	m	1
	2	$\sqrt{n}$	1	$\sqrt{m}$	1	$\sqrt{mn}$	1
	∞	n	$\sqrt{n}$	1	$\sqrt{n}$	n	1
	F	$\sqrt{n}$	$\sqrt{\text{rank}(A)}$	$\sqrt{m}$	1	$\sqrt{mn}$	1
	M	1	1	1	1	1	1
	S	n	$\sqrt{mn \text{rank}(A)}$	m	$\sqrt{mn}$	mn	1

errors. For example, if $A \in \mathbb{C}^{n \times n}$ is contaminated by errors E and Q is unitary, then

$$Q(A + E)Q^* = QAQ^* + F,$$

and $\|F\|_2 = \|QEQ^*\|_2 = \|E\|_2$. In contrast, if we do a general, nonsingular similarity transformation

$$X(A + E)X^{-1} = XAX^{-1} + G,$$

then $\|G\|_2 = \|XEX^{-1}\|_2 \leq \kappa_2(X)\|E\|_2$, where

$$\kappa(X) = \|X\| \|X^{-1}\|$$

is the *condition number* of X . The condition number satisfies $\kappa(X) \geq 1$ ($\kappa_F(X) \geq \sqrt{n}$) and can be arbitrarily large. Justification for calling κ a condition number is found in Theorem 6.4 below.

In perturbation theory and error analysis it is often necessary to switch between norms. Therefore inequalities that bound one norm in terms of another are required. It is well known that on a finite-dimensional space any two norms differ by at most a constant that depends only on the dimension (so-called norm equivalence). Tables 6.1 and 6.2 give attainable inequalities for the vector and matrix norms of most interest.

The definition of subordinate matrix norm can be generalized by permitting different norms on the input and output space:

$$\|A\|_{\alpha, \beta} = \max_{x \neq 0} \frac{\|Ax\|_{\beta}}{\|x\|_{\alpha}}. \quad (6.6)$$

Note that, in general, the submultiplicative property $\|AB\|_{\alpha,\beta} \leq \|A\|_{\alpha,\beta}\|B\|_{\alpha,\beta}$ does not hold, but we do have

$$\|AB\|_{\alpha,\beta} \leq \|A\|_{\gamma,\beta}\|B\|_{\alpha,\gamma}, \quad (6.7)$$

for any third vector norm $\|\cdot\|_\gamma$. The choice $\alpha = 1$ and $\beta = \infty$ produces the max norm, mentioned above, $\|A\|_{1,\infty} = \max_{i,j} |a_{ij}|$.

At least two important results about matrix norms hold for this mixed subordinate norm. The first is a simple formula for the matrix condition number of a nonsingular $A \in \mathbb{C}^{n \times n}$, defined by

$$\kappa_{\alpha,\beta}(A) := \lim_{\epsilon \rightarrow 0} \sup_{\|\Delta A\|_{\alpha,\beta} \leq \epsilon \|A\|_{\alpha,\beta}} \left(\frac{\|(A + \Delta A)^{-1} - A^{-1}\|_{\beta,\alpha}}{\epsilon \|A^{-1}\|_{\beta,\alpha}} \right).$$

Note that this definition uses the $\|\cdot\|_{\alpha,\beta}$ norm on the data space and the $\|\cdot\|_{\beta,\alpha}$ norm on the solution space, as is natural.

We need the following lemma.

Lemma 6.3. *Given vector norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ and vectors $x \in \mathbb{C}^n$, $y \in \mathbb{C}^m$ such that $\|x\|_\alpha = \|y\|_\beta = 1$, there exists a matrix $B \in \mathbb{C}^{m \times n}$ with $\|B\|_{\alpha,\beta} = 1$ such that $Bx = y$.*

Proof. Recall that the dual of the α -norm is defined by $\|z\|_\alpha^D = \max_{\|w\|_\alpha=1} |z^*w|$. Let z be a vector dual to x , so that $z^*x = \|z\|_\alpha^D \|x\|_\alpha = 1$, and hence $\|z\|_\alpha^D = 1$. Let $B = yz^*$. Then $Bx = y$ and

$$\|B\|_{\alpha,\beta} = \max_{\|w\|_\alpha=1} \|yz^*w\|_\beta = \|y\|_\beta \max_{\|w\|_\alpha=1} |z^*w| = \|y\|_\beta \|z\|_\alpha^D = 1,$$

as required. $\square$

Theorem 6.4. *For nonsingular $A \in \mathbb{C}^{n \times n}$, the matrix condition number $\kappa_{\alpha,\beta}(A)$ satisfies*

$$\kappa_{\alpha,\beta}(A) = \|A\|_{\alpha,\beta} \|A^{-1}\|_{\beta,\alpha}. \quad (6.8)$$

Proof. In view of the expansion

$$(A + \Delta A)^{-1} - A^{-1} = -A^{-1} \Delta A A^{-1} + O(\|\Delta A\|^2),$$

the result is proved if we can show that

$$\sup_{\|\Delta A\|_{\alpha,\beta} \leq 1} \|A^{-1} \Delta A A^{-1}\|_{\beta,\alpha} = \|A^{-1}\|_{\beta,\alpha}^2. \quad (6.9)$$

That (6.9) holds with the equality replaced by " $\leq$ " follows from two applications of (6.7). To show the opposite inequality, we have

$$\|A^{-1} \Delta A A^{-1}\|_{\beta,\alpha} = \max_{\|y\|_\beta=1} \|A^{-1} \Delta A A^{-1} y\|_\alpha \geq \|A^{-1} \Delta A x\|_\alpha \|A^{-1}\|_{\beta,\alpha}, \quad (6.10)$$

where, for the lower bound, we have chosen y so that $\|A^{-1}y\|_\alpha = \|A^{-1}\|_{\beta,\alpha}$, and where $A^{-1}y = \|A^{-1}\|_{\beta,\alpha} x$ with $\|x\|_\alpha = 1$. Now, from Lemma 6.3, there

exists a matrix ΔA with $\|\Delta A\|_{\alpha,\beta} = 1$ such that $\Delta A x = y$. In (6.10) this gives $\|A^{-1}\Delta A A^{-1}\|_{\beta,\alpha} \geq \|A^{-1}\|_{\beta,\alpha}^2$, as required. $\square$

The next result concerns the relative distance to singularity for a matrix $A \in \mathbb{C}^{n \times n}$:

$$\text{dist}_{\alpha,\beta}(A) := \min \left\{ \frac{\|\Delta A\|_{\alpha,\beta}}{\|A\|_{\alpha,\beta}} : A + \Delta A \text{ singular} \right\}.$$

It states that the relative distance to singularity is the reciprocal of the condition number.

Theorem 6.5 (Gastinel, Kahan). *For nonsingular $A \in \mathbb{C}^{n \times n}$, we have*

$$\text{dist}_{\alpha,\beta}(A) = (\|A\|_{\alpha,\beta} \|A^{-1}\|_{\beta,\alpha})^{-1} = \kappa_{\alpha,\beta}(A)^{-1}.$$

Proof. If $A + \Delta A$ is singular, then there exists $x \neq 0$ such that $(A + \Delta A)x = 0$. Hence

$$\|x\|_{\alpha} = \|A^{-1}\Delta A x\|_{\alpha} \leq \|A^{-1}\|_{\beta,\alpha} \|\Delta A x\|_{\beta} \leq \|A^{-1}\|_{\beta,\alpha} \|\Delta A\|_{\alpha,\beta} \|x\|_{\alpha},$$

giving

$$\frac{\|\Delta A\|_{\alpha,\beta}}{\|A\|_{\alpha,\beta}} \geq \kappa_{\alpha,\beta}(A)^{-1}.$$

To show that a suitable perturbation achieves this lower bound, let y be such that $\|y\|_{\beta} = 1$ and $\|A^{-1}y\|_{\alpha} = \|A^{-1}\|_{\beta,\alpha}$, and write $x = A^{-1}y$. By Lemma 6.3 there exists B with $\|B\|_{\alpha,\beta} = 1$ such that $Bx/\|x\|_{\alpha} = -y$. Letting $\Delta A = B/\|x\|_{\alpha}$ we have $\|\Delta A\|_{\alpha,\beta}/\|A\|_{\alpha,\beta} = \kappa_{\alpha,\beta}(A)^{-1}$, and $A + \Delta A$ is singular because $(A + \Delta A)A^{-1}y = 0$. $\square$

The next lemma collects some results that will be needed in later chapters.

Lemma 6.6. *Let $A, B \in \mathbb{R}^{m \times n}$.*

(a) *If $\|a_j\|_2 \leq \|b_j\|_2$, $j = 1:n$, then*

$$\|A\|_F \leq \|B\|_F, \quad \|A\|_2 \leq \sqrt{\text{rank}(B)} \|B\|_2, \quad |A| \leq ee^T |B|.$$

(b) *If $|A| \leq B$ then $\|A\|_2 \leq \|B\|_2$.*

(c) *If $|A| \leq |B|$ then $\|A\|_2 \leq \sqrt{\text{rank}(B)} \|B\|_2$.*

(d) $\|A\|_2 \leq \| |A| \|_2 \leq \sqrt{\text{rank}(A)} \|A\|_2$.

Proof. The first inequality of (a) is trivial. For the second, we have, using Table 6.2,

$$\|A\|_2 \leq \|A\|_F \leq \|B\|_F \leq \sqrt{\text{rank}(B)} \|B\|_2.$$

The third inequality follows from

$$|a_j| \leq \|a_j\|_2 e \leq \|b_j\|_2 e \leq \|b_j\|_1 e = (e^T |b_j|) e = (ee^T) |b_j|.$$

For (b) we have

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 \leq \max_{\|x\|_2=1} \| |A| |x| \|_2 \leq \max_{\|x\|_2=1} \|B|x\|_2 = \|B\|_2.$$

Finally, (c) is a special case of (a), and (d) is a special case of (b) and (c). $\square$

Note that the second inequality in (a) is sharp: there is equality if $A = B$ has rank 1 and in the case $A = ee^T$, $B = \sqrt{n}I$, in which B has full rank.

6.3. The Matrix p -Norm

The matrix p -norm is the norm subordinate to the Hölder p -norm:

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}, \quad p \geq 1. \quad (6.11)$$

Formulae for $\|A\|_p$ are known only for $p = 1, 2, \infty$. For other values of p , how to estimate or compute $\|A\|_p$ is an interesting problem, the study of which, as well as being interesting in its own right, yields insight into the properties of the 1, 2, and ∞ norms.

By taking $x = e_j$ in (6.11), using (6.4), and using (6.21) below, we can derive the bounds, for $A \in \mathbb{C}^{m \times n}$,

$$\max_j \|A(:, j)\|_p \leq \|A\|_p \leq n^{1-1/p} \max_j \|A(:, j)\|_p, \quad (6.12)$$

$$\max_i \|A(i, :)\|_{p/(p-1)} \leq \|A\|_p \leq m^{1/p} \max_i \|A(i, :)\|_{p/(p-1)}. \quad (6.13)$$

Matrix norms can be compared using the following elegant result of Schneider and Strang [1018, 1962] (see also [636, 1985, Thm. 5.6.18]): if $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ denote two vector norms and the corresponding subordinate matrix norms, then for $A \in \mathbb{C}^{m \times n}$

$$\max_{A \neq 0} \frac{\|A\|_\alpha}{\|A\|_\beta} = \left(\max_{0 \neq x \in \mathbb{C}^m} \frac{\|x\|_\alpha}{\|x\|_\beta} \right) \left(\max_{0 \neq x \in \mathbb{C}^n} \frac{\|x\|_\beta}{\|x\|_\alpha} \right). \quad (6.14)$$

From (6.4) and (6.14), we have, when $m = n$,

$$\max_{A \neq 0} \frac{\|A\|_{p_1}}{\|A\|_{p_2}} = n \left(\frac{1}{\min(p_1, p_2)} - \frac{1}{\max(p_1, p_2)} \right). \quad (6.15)$$

Note that, unlike for vectors, $p_1 < p_2$ does not imply $\|A\|_{p_1} \geq \|A\|_{p_2}$. The result (6.15) implies, for example, that for all $p \geq 1$

$$\frac{\|A\|_1}{n^{1-1/p}} \leq \|A\|_p \leq n^{1-1/p} \|A\|_1, \quad (6.16)$$

$$\frac{\|A\|_2}{n^{1/p-1/2}} \leq \|A\|_p \leq n^{1/p-1/2} \|A\|_2. \quad (6.17)$$

Upper bounds for $\|A\|_p$ that do not involve m or n can be obtained from the interesting property that $\log \|A\|_p$ is a convex function of $1/p$ for $p \geq 1$

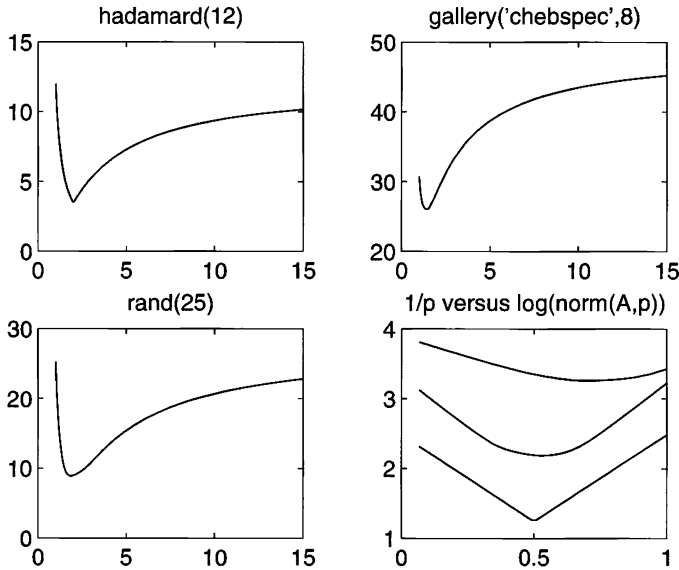


Figure 6.1. Plots of p versus $\|A\|_p$, for $1 \leq p \leq 15$. Fourth plot shows $1/p$ versus $\log \|A\|_p$ for the matrices in the first three plots.

(see Figure 6.1), which is a consequence of the Riesz–Thorin theorem [548, 1952, pp. 214, 219], [489, 1991]. The convexity implies that if $f(\alpha) = \|A\|_{1/\alpha}$, then for $0 \leq \alpha, \beta \leq 1$,

$$\log f(\theta\alpha + (1-\theta)\beta) \leq \theta \log f(\alpha) + (1-\theta) \log f(\beta), \quad 0 \leq \theta \leq 1.$$

Writing $p_1 = 1/\alpha$ and $p_2 = 1/\beta$, this inequality can be expressed as

$$\|A\|_p \leq \|A\|_{p_1}^\theta \|A\|_{p_2}^{1-\theta}, \quad p = \frac{p_1 p_2}{(1-\theta)p_1 + \theta p_2}, \quad (6.18)$$

$$1 \leq p_1, p_2 \leq \infty, \quad 0 \leq \theta \leq 1.$$

Two interesting special cases are

$$\|A\|_p \leq \|A\|_1^{1/p} \|A\|_\infty^{1-1/p} \quad (6.19)$$

and

$$\|A\|_p \leq \|A\|_1^{2/p-1} \|A\|_2^{2-2/p}, \quad 1 \leq p \leq 2. \quad (6.20)$$

Note that (6.19) includes the well-known inequality $\|A\|_2 \leq \sqrt{\|A\|_1 \|A\|_\infty}$.

Two further results that are familiar for $p = 1, 2, \infty$ are

$$\|A^*\|_p = \|A\|_q, \quad \frac{1}{p} + \frac{1}{q} = 1 \quad (6.21)$$

(see also Problem 6.3), and

$$\left\| \begin{bmatrix} 0 & A \\ A^* & 0 \end{bmatrix} \right\|_p = \max(\|A\|_p, \|A\|_q).$$

The bounds (6.16) and (6.17) imply that given the ability to compute $\|A\|_1$, $\|A\|_2$, and $\|A\|_\infty$ we can estimate $\|A\|_p$ correct to within a factor $n^{1/4}$. These a priori estimates are at their best when p is close to 1, 2, or ∞ , but in general they will not provide even one correct significant digit. The bound in (6.18) can be much smaller than the other upper bounds given above, but how tight it is depends on how nearly $\log \|A\|_p$ is linear in p . Numerical methods are needed to obtain better estimates; these are developed in Chapter 15.

6.4. Singular Value Decomposition

Any matrix $A \in \mathbb{C}^{m \times n}$ has a *singular value decomposition* (SVD)

$$A = U \Sigma V^*, \quad \Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p) \in \mathbb{C}^{m \times n}, \quad p = \min(m, n),$$

where $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$ and $U \in \mathbb{C}^{m \times m}$, $V \in \mathbb{C}^{n \times n}$ are both unitary. The σ_i are the *singular values* of A and the columns of U and V are the *left* and *right singular vectors* of A , respectively.

The rank of A is equal to the number of nonzero singular values. If A is real, U and V can be taken to be real.

For any unitarily invariant norm, $\|A\| = \|\Sigma\|$, and hence

$$\|A\|_2 = \sigma_1(A), \quad \|A\|_F = \left(\sum_{i=1}^n \sigma_i^2 \right)^{1/2}. \quad (6.22)$$

The SVD is an extremely useful tool in numerical linear algebra. Aside from exploiting these norm relations, our main use of the SVD in this book is in Chapter 20 on the least squares problem.

6.5. Notes and References

The matrix condition number appears to have been first introduced explicitly by Turing [1166, 1948], who defined, for example, the N -condition number of $A \in \mathbb{R}^{n \times n}$ as $n^{-1}N(A)N(A^{-1})$, where $N(\cdot)$ is Turing's notation for the Frobenius norm. Todd [1140, 1968] gives a short survey of the matrix condition number with many references.

Theorem 6.2 was originally proved by Bauer, Stoer, and Witzgall, in a paper that contains many interesting results on monotonic norms [95, 1961].

Tables of constants in inequalities between different norms have been given by various authors; see, for example, Stone [1088, 1962] and Zielke [1282, 1988].

Our development of the mixed subordinate norm $\|\cdot\|_{\alpha,\beta}$ is based on that of D. J. Higham [573, 1995].

Theorem 6.5 is proved by Kahan [687, 1966, pp. 775–776], who attributes it to Gastinel but gives no reference. For the 2-norm, this result goes back to a paper by Eckart and Young [370, 1936]. Theorem 6.5 is an instance of a relationship that holds for many problems: the condition number is the reciprocal of the distance to the nearest singular problem (one with an infinite condition number). This relationship applies to matrix inversion, eigenvalue and eigenvector computation,

polynomial zero-finding, and pole assignment in linear control systems. For an in-depth study see Demmel [309, 1987].

Schneider and Weinberger [1019, 1998] study classes of matrices for which (6.14) is attained in the case of Hölder p -norms.

Direct proofs of inequality (6.19) can be found in Kato [716, 1976, p. 29] and Todd [1143, 1977, pp. 25–26]. The inequality does not seem to be well known.

For historical comments on the development of norms in numerical analysis, see Householder [644, 1964, Chap. 2] and Stewart and Sun [1083, 1990, Chap. 2].

For more details on the SVD see Golub and Van Loan [509, 1996, §2.5.3], Stewart and Sun [1083, 1990, pp. 30–34], and Horn and Johnson [636, 1985, §7.3], [637, 1991, §3.1]. The history of the SVD is described by Stewart [1074, 1993] and Horn and Johnson [637, 1991, §3.0].

Problems

*Problems worthy
of attack
prove their worth
by hitting back.*

— PIET HEIN, *Grooks* (1966)

6.1. Prove the inequalities given in Tables 6.1 and 6.2. Show that each inequality in Table 6.2 (except the one for $\alpha_{S,2}$) is attainable for a matrix of the form $A = xy^T$, where $x, y \in \{e, e_j\}$, where $e = [1, 1, \dots, 1]^T$. Show that equality in $\|A\|_S \leq \alpha_{S,2} \|A\|_2$ is attained for square real matrices A iff A is a scalar multiple of a Hadamard matrix (see §9.4 for the definition of a Hadamard matrix), and for square complex matrices if $a_{rs} = \exp(2\pi i(r-1)(s-1)/n)$ (this is a Vandermonde matrix based on the roots of unity).

6.2. Let $x, y \in \mathbb{C}^{n \times n}$. Show that, for any subordinate matrix norm, $\|xy^*\| = \|x\| \|y\|_D$.

6.3. Show that a subordinate matrix norm $\|\cdot\|$ on $\mathbb{C}^{n \times n}$ satisfies

$$\|A\| = \max_{x, y \neq 0} \frac{\operatorname{Re} y^* A x}{\|y\|_D \|x\|}.$$

Deduce that $\|A^*\| = \|A\|_D$, where the latter norm is the norm subordinate to the vector norm $\|\cdot\|_D$.

*From ancient times until now the
study of magic squares has flourished as a kind of cult,
often with occult trappings, whose initiates range from
such eminent mathematicians as Arthur Cayley and Oswald Veblen
to laymen such as Benjamin Franklin.*

— MARTIN GARDNER, *More Mathematical Puzzles and Diversions* (1961)

6.4. Let $M_n \in \mathbb{R}^{n \times n}$ denote a magic square matrix, that is, an $n \times n$ matrix containing the integers from 1 to n^2 arranged in such a way that the row and column sums are all the same. Let μ_n denote the magic sum of M_n (thus, $\mu_n = n(n^2 + 1)/2$). Show that $\|M_n\|_p = \mu_n$ for all $1 \leq p \leq \infty$. (This result is a

special case of an apparently little-known result of Stoer and Witzgall, which states that the norm of a doubly stochastic matrix is 1 for any norm subordinate to a permutation-invariant absolute vector norm [1087, 1962].)

6.5. Show that $\|ABC\|_F \leq \|A\|_2 \|B\|_F \|C\|_2$ for any A , B , and C such that the product is defined. (This result remains true when the Frobenius norm is replaced by any unitarily invariant norm [637, 1991, p. 211].)

6.6. Show that for any nonsingular $A \in \mathbb{C}^{n \times n}$,

$$\kappa_{\alpha, \beta}(A) = \frac{\max_{x \neq 0} \frac{\|Ax\|_{\beta}}{\|x\|_{\alpha}}}{\min_{x \neq 0} \frac{\|Ax\|_{\beta}}{\|x\|_{\alpha}}}.$$

6.7. Show that for any $A \in \mathbb{C}^{n \times n}$ and any consistent matrix norm, $\rho(A) \leq \|A\|$, where ρ is the spectral radius.

6.8. Show that for any $A \in \mathbb{C}^{n \times n}$ and $\delta > 0$ there is a consistent norm $\|\cdot\|$ (which depends on A and δ) such that $\|A\| \leq \rho(A) + \delta$, where ρ is the spectral radius. Hence show that if $\rho(A) < 1$ then there is a consistent norm $\|\cdot\|$ such that $\|A\| < 1$.

6.9. Let $A \in \mathbb{C}^{m \times n}$. Use (6.22) to obtain a bound of the form $c_1 \|A\|_2 \leq \|A\|_F \leq c_2 \|A\|_2$, where c_1 and c_2 are constants that depend on n . When is there equality in the upper bound? When is there equality in the lower bound?

6.10. Show that

$$\left\| \begin{bmatrix} I & F \\ 0 & I \end{bmatrix} \right\|_2 = \sqrt{\frac{2 + \|F\|_2^2 + \|F\|_2 \sqrt{4 + \|F\|_2^2}}{2}}.$$

Deduce that when $\|F\|_2 = 1$, the norm is $(1 + \sqrt{5})/2$, the golden ratio.

6.11. Let $A \in \mathbb{C}^{n \times n}$. Prove that (a) $\|A\|_{1, \beta} = \max_j \|A(:, j)\|_{\beta}$, and (b) $\|A\|_{\alpha, \infty} = \max_i \|A(i, :)^*\|_{\alpha}^D$. What is $\|A\|_{1, \infty}$?

6.12. (Tao [1128, 1984]) Show that if A is Hermitian positive definite then

$$\|A\|_{\infty, 1} = \max\{x^* A x : \|x\|_{\infty} = 1\}.$$

(Rohn [992, 1995] shows that the problem of computing $\|A\|_{\infty, 1}$ is NP-hard.)

6.13. Prove that if $H \in \mathbb{R}^{n \times n}$ is a Hadamard matrix then

$$\|H\|_p = \max\{n^{1/p}, n^{1-1/p}\}.$$

(See §9.4 for the definition of a Hadamard matrix.)

6.14. Show that if $A \in \mathbb{R}^{m \times n}$ has at most μ nonzeros per row then

$$\max_j \|A(:, j)\|_p \leq \|A\|_p \leq \mu^{1-1/p} \max_j \|A(:, j)\|_p, \quad (6.23)$$

while if A has at most μ nonzeros per column then

$$\max_i \|A(i, :)\|_q \leq \|A\|_p \leq \mu^{1/p} \max_i \|A(i, :)\|_q, \quad (6.24)$$

where $p^{-1} + q^{-1} = 1$. (These inequalities generalize (6.12) and (6.13).)

6.15. Show that if $A \in \mathbb{C}^{n \times n}$ then for any p -norm ($1 \leq p \leq \infty$),

$$\|A\|_p \leq \| |A| \|_p \leq n^{\min(1/p, 1-1/p)} \|A\|_p \leq \sqrt{n} \|A\|_p.$$

6.16. Define the function $\nu : \mathbb{C}^n \rightarrow \mathbb{R}$ by

$$\nu(x) = \sum_{i=1}^n (|\operatorname{Re} x_i| + |\operatorname{Im} x_i|).$$

Is ν a vector norm on $\mathbb{C}^n$? Derive an explicit expression for

$$\nu(A) = \max_{\nu(x)=1} \nu(Ax), \quad A \in \mathbb{C}^{n \times n}.$$

(For the relevance of ν see §27.8.)

Chapter 7

Perturbation Theory for Linear Systems

Our hero is the intrepid, yet sensitive matrix A .

Our villain is E , who keeps perturbing A .

When A is perturbed he puts on a crumpled hat: $\tilde{A} = A + E$.

— G. W. STEWART and JI-GUANG SUN, *Matrix Perturbation Theory* (1990)

The expression 'ill-conditioned' is sometimes used merely as a term of abuse applicable to matrices or equations . . .

It is characteristic of ill-conditioned sets of equations that small percentage errors in the coefficients given may lead to large percentage errors in the solution.

— A. M. TURING, *Rounding-Off Errors in Matrix Processes* (1948)

In this chapter we are concerned with a linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$. In the context of uncertain data or inexact arithmetic there are three important questions:

- (1) How much does x change if we perturb A and b ; that is, how sensitive is the solution to perturbations in the data?
- (2) How much do we have to perturb the data A and b for an approximate solution y to be the exact solution of the perturbed system—in other words, what is the backward error of y ?
- (3) What bound should we compute in practice for the forward error of a given approximate solution?

To answer these questions we need both normwise and componentwise perturbation theory.

7.1. Normwise Analysis

First, we present some classical normwise perturbation results. We denote by $\|\cdot\|$ any vector norm and the corresponding subordinate matrix norm. As usual, $\kappa(A) = \|A\| \|A^{-1}\|$ is the matrix condition number. Throughout this chapter the matrix E and the vector f are arbitrary and represent tolerances against which the perturbations are measured (their role becomes clear when we consider componentwise results).

Our first result makes precise the intuitive feeling that if the residual is small then we have a “good” approximate solution.

Theorem 7.1 (Rigal and Gaches). *The normwise backward error*

$$\eta_{E,f}(y) := \min\{\epsilon : (A + \Delta A)y = b + \Delta b, \quad \|\Delta A\| \leq \epsilon \|E\|, \quad \|\Delta b\| \leq \epsilon \|f\|\} \quad (7.1)$$

is given by

$$\eta_{E,f}(y) = \frac{\|r\|}{\|E\| \|y\| + \|f\|}, \quad (7.2)$$

where $r = b - Ay$.

Proof. It is straightforward to show that the right-hand side of (7.2) is a lower bound for $\eta_{E,f}(y)$. This lower bound is attained for the perturbations

$$\Delta A_{\min} = \frac{\|E\| \|y\|}{\|E\| \|y\| + \|f\|} r z^T, \quad \Delta b_{\min} = -\frac{\|f\|}{\|E\| \|y\| + \|f\|} r, \quad (7.3)$$

where z is a vector dual to y (see §6.1). $\square$

For the particular choice $E = A$ and $f = b$, $\eta_{E,f}(y)$ is called the *normwise relative backward error*.

The next result measures the sensitivity of the system.

Theorem 7.2. *Let $Ax = b$ and $(A + \Delta A)y = b + \Delta b$, where $\|\Delta A\| \leq \epsilon \|E\|$ and $\|\Delta b\| \leq \epsilon \|f\|$, and assume that $\epsilon \|A^{-1}\| \|E\| < 1$. Then*

$$\frac{\|x - y\|}{\|x\|} \leq \frac{\epsilon}{1 - \epsilon \|A^{-1}\| \|E\|} \left(\frac{\|A^{-1}\| \|f\|}{\|x\|} + \|A^{-1}\| \|E\| \right), \quad (7.4)$$

and this bound is attainable to first order in ϵ .

Proof. The bound (7.4) follows easily from the equation $A(y - x) = \Delta b - \Delta Ax + \Delta A(x - y)$. It is attained to first order in ϵ for $\Delta A = \epsilon \|E\| \|x\| wv^T$ and $\Delta b = -\epsilon \|f\| w$, where $\|w\| = 1$, $\|A^{-1}w\| = \|A^{-1}\|$, and v is a vector dual to x . $\square$

Associated with the way of measuring perturbations used in these two theorems is the normwise condition number

$$\kappa_{E,f}(A, x) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{\|\Delta x\|}{\epsilon \|x\|} : (A + \Delta A)(x + \Delta x) = b + \Delta b, \right. \\ \left. \|\Delta A\| \leq \epsilon \|E\|, \|\Delta b\| \leq \epsilon \|f\| \right\}.$$

Because the bound of Theorem 7.2 is sharp, it follows that

$$\kappa_{E,f}(A, x) = \frac{\|A^{-1}\| \|f\|}{\|x\|} + \|A^{-1}\| \|E\|. \quad (7.5)$$

For the choice $E = A$ and $f = b$ we have $\kappa(A) \leq \kappa_{E,f}(A, x) \leq 2\kappa(A)$, and the bound (7.4) can be weakened slightly to yield the familiar form

$$\frac{\|x - y\|}{\|x\|} \leq \frac{2\epsilon\kappa(A)}{1 - \epsilon\kappa(A)}.$$

A numerical example illustrates the above results. Let A be the 6×6 Vandermonde matrix with (i, j) element $j^{2(i-1)}$ and let $b = e_1$ be the first unit vector, so that x is the first column of A^{-1} . We take y to be the approximate solution to $Ax = b$ computed by Gaussian elimination with partial pivoting. Computations are performed in MATLAB ($u \approx 1.1 \times 10^{-16}$). We find that $\eta_{A,b}(y) = 4.16 \times 10^{-20}$ for the ∞ -norm, and $\kappa_\infty(A) = 1.93 \times 10^8$. This is an admirably small backward error, but it may be uninformative for two reasons. First, the elements of A vary over 7 orders of magnitude, so while our backward error perturbations are small compared with the large elements of A , we may be making large perturbations in the small elements (indeed we *are* in this particular example). Second, we are perturbing the zero elements of b (as can be seen from (7.3) together with the fact that for this example the residual r has no zero entries); this is unsatisfactory if we wish to regard y as the first column of the inverse of a perturbed matrix.

Next, let $b = Ae$, where $e = [1, 1, \dots, 1]^T$, and let z be the solution to the perturbed system $(A + \Delta A)z = b$, where $\Delta A = \text{tol}|A|$ with $\text{tol} = 10^{-10}$; thus we are perturbing each element of A by a small relative amount. We find that

$$\frac{\|x - z\|_\infty}{\|x\|_\infty} = 1.00 \times 10^{-10}, \quad (7.6)$$

while the corresponding bound from (7.4) with $\epsilon = \text{tol}$, $E = A$, and $f = 0$ is 1.97×10^{-2} . Thus the normwise forward error bound is extremely pessimistic for this special choice of perturbation.

To obtain a more satisfactory backward error measure and a sharper perturbation bound in this example, we need componentwise analysis.

7.2. Componentwise Analysis

The *componentwise backward error* is defined as

$$\omega_{E,f}(y) = \min\{\epsilon : (A + \Delta A)y = b + \Delta b, \quad |\Delta A| \leq \epsilon E, \quad |\Delta b| \leq \epsilon f\}, \quad (7.7)$$

where E and f are now assumed to have nonnegative entries. Absolute values and inequalities between matrices or vectors are understood to hold componentwise. In this definition each element of a perturbation is measured relative to its individual tolerance, so, unlike in the normwise definition, we are making full use of the $n^2 + n$ parameters in E and f .

How should E and f be chosen? There are four main choices of interest. The most common choice of tolerances is $E = |A|$ and $f = |b|$, which yields the *componentwise relative backward error*. For this choice

$$a_{ij} = 0 \Rightarrow \Delta a_{ij} = 0 \quad \text{and} \quad b_i = 0 \Rightarrow \Delta b_i = 0$$

in (7.7), and so if $\omega_{E,f}(y)$ is small then y solves a problem that is close to the original one in the sense of componentwise relative perturbations and has the same sparsity pattern. Another attractive property of the componentwise relative backward error is that it is insensitive to the scaling of the system: if $Ax = b$ is scaled to $(S_1 A S_2)(S_2^{-1}x) = S_1 b$, where S_1 and S_2 are diagonal, and y is scaled to $S_2^{-1}y$, then ω remains unchanged.

The choice $E = |A|ee^T$, $f = |b|$ gives a *row-wise backward error*. The constraint $|\Delta A| \leq \epsilon E$ is now $|\Delta a_{ij}| \leq \epsilon \alpha_i$, where α_i is the 1-norm of the i th row of A , so perturbations to the i th row of A are being measured relative to the norm of that row (and similarly for b). A *columnwise backward error* can be formulated in a similar way, by taking $E = ee^T|A|$ and $f = \|b\|_1 e$ —here, perturbations to the j th column of A (or to b) are measured relative to the 1-norm of that column.

The fourth natural choice of tolerances is $E = \|A\|ee^T$ and $f = \|b\|e$, for which $\omega_{E,f}(y)$ is the same as the normwise backward error $\eta_{E,f}(y)$, up to a constant.

As for the normwise backward error in Theorem 7.1, there is a simple formula for $\omega_{E,f}(y)$.

Theorem 7.3 (Oettli and Prager). *The componentwise backward error is given by*

$$\omega_{E,f}(y) = \max_i \frac{|r_i|}{(E|y| + f)_i}, \quad (7.8)$$

where $r = b - Ay$, and $\xi/0$ is interpreted as zero if $\xi = 0$ and infinity otherwise.

Proof. It is easy to show that the right-hand side of (7.8) is a lower bound for $\omega(y)$, and that this bound is attained for the perturbations

$$\Delta A = D_1 E D_2, \quad \Delta b = -D_1 f, \quad (7.9)$$

where $D_1 = \text{diag}(r_i/(E|y| + f)_i)$ and $D_2 = \text{diag}(\text{sign}(y_i))$. $\square$

The next result gives a forward error bound corresponding to the componentwise backward error.

Theorem 7.4. Let $Ax = b$ and $(A + \Delta A)y = b + \Delta b$, where $|\Delta A| \leq \epsilon E$ and $|\Delta b| \leq \epsilon f$, and assume that $\epsilon \| |A^{-1}|E \| < 1$, where $\|\cdot\|$ is an absolute norm. Then

$$\frac{\|x - y\|}{\|x\|} \leq \frac{\epsilon}{1 - \epsilon \| |A^{-1}|E \|} \frac{\| |A^{-1}|(|E|x| + f) \|}{\|x\|}, \quad (7.10)$$

and for the ∞ -norm this bound is attainable to first order in ϵ .

Proof. The bound (7.10) follows easily from the equation $A(y - x) = \Delta b - \Delta Ax + \Delta A(x - y)$. For the ∞ -norm the bound is attained, to first order in ϵ , for $\Delta A = \epsilon D_1 E D_2$ and $\Delta b = -\epsilon D_1 f$, where $D_2 = \text{diag}(\text{sign}(x_i))$ and $D_1 = \text{diag}(\xi_j)$, where $\xi_j = \text{sign}(A^{-1})_{kj}$ and $\| |A^{-1}|(|E|x| + f) \|_\infty = (|A^{-1}|(|E|x| + f))_k$. $\square$

Theorem 7.4 implies that the condition number

$$\text{cond}_{E,f}(A, x) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{\| \Delta x \|_\infty}{\epsilon \| x \|_\infty} : (A + \Delta A)(x + \Delta x) = b + \Delta b, \right. \\ \left. |\Delta A| \leq \epsilon E, \quad |\Delta b| \leq \epsilon f \right\}$$

is given by

$$\text{cond}_{E,f}(A, x) = \frac{\| |A^{-1}|(|E|x| + f) \|_\infty}{\|x\|_\infty}. \quad (7.11)$$

This condition number depends on x or, equivalently, on the right-hand side b . A worst-case measure of sensitivity applicable to all x is

$$\text{cond}_{E,f}(A) = \max_x \text{cond}_{E,f}(A, x), \quad (7.12)$$

where in practice we can take any convenient approximation to the maximum that is correct to within a constant factor.

For the special case $E = |A|$ and $f = |b|$ we have the condition numbers introduced by Skeel [1040, 1979]:

$$\text{cond}(A, x) := \frac{\| |A^{-1}||A||x| \|_\infty}{\|x\|_\infty}, \quad (7.13)$$

$$\text{cond}(A) := \text{cond}(A, e) = \| |A^{-1}||A| \|_\infty \leq \kappa_\infty(A). \quad (7.14)$$

These differ from $\text{cond}_{|A|,|b|}(A, x)$ and $\text{cond}_{|A|,|b|}(A)$, respectively, by at most a factor 2.

How does cond compare with κ ? Since $\text{cond}(A)$ is invariant under row scaling $Ax = b \rightarrow (DA)x = Db$, where D is diagonal, it can be arbitrarily smaller than $\kappa_\infty(A)$. In fact, it is straightforward to show that

$$\min\{ \kappa_\infty(DA) : D \text{ diagonal} \} = \text{cond}(A), \quad (7.15)$$

where the optimal scaling D_R equilibrates the rows of A , that is, $D_R A$ has rows of unit 1-norm ($D_R |A| e = e$)

Chandrasekaran and Ipsen [217, 1995] note the following inequalities. First, with D_R as just defined,

$$\frac{\kappa_\infty(A)}{\kappa_\infty(D_R)} \leq \text{cond}(A) \leq \kappa_\infty(A) \quad (7.16)$$

(these inequalities imply (7.15)). Thus $\text{cond}(A)$ can be much smaller than $\kappa_\infty(A)$ only when the rows of A are badly scaled. Second, if D_C equilibrates the columns of A ($e^T|A|D_C = e^T$) then

$$\frac{\kappa_1(A)}{n\kappa_\infty(D_C)} \min_j \frac{\|A^{-1}e_j\|_\infty}{\|A^{-1}\|_1} \leq \text{cond}(A, x) \leq \kappa_\infty(A).$$

These inequalities show that $\text{cond}(A, x)$ can be much smaller than $\kappa_\infty(A)$ only when the columns of either A or A^{-1} are badly scaled.

Returning to the numerical example of §7.1, we find that $\omega_{E,f}(y) = 5.76 \times 10^{-15}$ for $E = |A|$ and $f = |b|$ or $f = 0$. This tells us that if we measure changes to A in a componentwise relative sense, then for y to be the first column of the inverse of a perturbed matrix we must make relative changes to A of just over one order of magnitude larger than the unit roundoff. For the perturbed system, Theorem 7.4 with $\epsilon = \text{tol}$, $E = |A|$, and $f = 0$ gives the bound

$$\frac{\|x - z\|_\infty}{\|x\|_\infty} \leq 4.37 \times 10^{-7},$$

which is five orders of magnitude smaller than the normwise bound from Theorem 7.2, though still three orders of magnitude larger than the actual forward error (7.6).

An example of Kahan [687, 1966] is also instructive. Let

$$A = \begin{bmatrix} 2 & -1 & 1 \\ -1 & \epsilon & \epsilon \\ 1 & \epsilon & \epsilon \end{bmatrix}, \quad b = \begin{bmatrix} 2(1 + \epsilon) \\ -\epsilon \\ \epsilon \end{bmatrix}, \quad (7.17)$$

where $0 < \epsilon \ll 1$, so that $x = [\epsilon, -1, 1]^T$. The normwise condition number $\kappa_\infty(A)$ is $2(1 + \epsilon^{-1})$, so the system is very sensitive to arbitrary perturbations in A and b . Moreover,

$$|A^{-1}||A| = \begin{bmatrix} 1 & \epsilon & \epsilon \\ \frac{2\epsilon + 1}{2\epsilon} & 1 & 1 \\ \frac{2\epsilon + 1}{2\epsilon} & 1 & 1 \end{bmatrix},$$

so $\text{cond}(A) = 3 + (2\epsilon)^{-1}$, which implies that the system is also very sensitive to componentwise perturbations for some right-hand sides. However, $\text{cond}(A, x) = 5/2 + \epsilon$, so *for this particular b* the system is very well conditioned under componentwise perturbations.

A word is in order concerning the choice of condition number. Every condition number for a linear system is defined with respect to a particular class of perturbations. It is important to use the right condition number for the occasion. For example, if $\hat{x}$ is a computed solution to $Ax = b$ and we know its normwise backward error $\eta_{A,b}(\hat{x})$, then it is the condition number $\kappa(A)$ that appears in the relevant forward error bound (multiplying $\eta_{A,b}(\hat{x})$) and therefore tells us something about the accuracy of $\hat{x}$. The componentwise condition number $\text{cond}(A, x)$ is relevant only if we are dealing with the componentwise relative backward error,

Table 7.1. *Four classes of perturbations and the corresponding condition numbers. The terms $\text{cond}(A, x)$ and $\text{cond}(A)$ are defined in (7.13) and (7.14).*

	Componentwise relative	Row-wise	Columnwise	Normwise
E	$ A $	$ A ee^T$	$ee^T A $	$\ A\ _\infty ee^T$
f	$ b $	$ b $	$\ b\ _1 e$	$\ b\ _\infty e$
$\text{cond}_{E,f}(A, x)$	$\text{cond}(A, x)$	$\text{cond}(A) \frac{\ x\ _1}{\ x\ _\infty}$	$\ A^{-1}\ _\infty \frac{\ A \ x\ _1}{\ x\ _\infty}$	$\kappa_\infty(A)$
$\text{cond}_{E,f}(A)$	$\text{cond}(A)$	$n \text{cond}(A)$	$\kappa_\infty(A)$	$\kappa_\infty(A)$

$\omega_{|A|,|b|}(\hat{x})$. Looked at another way, each algorithm has an associated error analysis that determines the condition number relevant to that algorithm.

Table 7.1 summarizes the four main classes of perturbations and the corresponding condition numbers $\text{cond}_{E,f}(A, x)$ and $\text{cond}_{E,f}(A)$. The row-wise and columnwise condition numbers in the table (which are lower bounds that are at most a constant factor smaller than the actual condition numbers) follow from Problem 7.6.

7.3. Scaling to Minimize the Condition Number

In the last section we noted the invariance of $\text{cond}(A)$ under row scaling, which contrasts with the strong dependence of $\kappa_\infty(A)$ upon the row scaling. The opportunity to scale the rows or columns of A arises in various applications, so we now take a closer look at the effect of scaling on the normwise condition number.

First, we consider one-sided scaling, by giving a generalization of a well-known result of van der Sluis [1176, 1969]. It shows that, for one-sided scaling in a Hölder p -norm, equilibrating the rows or columns is a nearly optimal strategy. We state the result for rectangular matrices A , for which we define $\kappa_p(A) := \|A\|_p \|A^+\|_p$, where A^+ is the pseudo-inverse of A (see Problem 20.3).

Theorem 7.5 (van der Sluis). *Let $A \in \mathbb{R}^{m \times n}$, let $\mathcal{D}_k \subset \mathbb{R}^{k \times k}$ denote the set of nonsingular diagonal matrices, and define*

$$D_C := \text{diag}(\|A(:, j)\|_p)^{-1}, \quad D_R := \text{diag}(\|A(i, :)\|_p)^{-1}.$$

Then

$$\kappa_p(AD_C) \leq n^{1-1/p} \min_{D \in \mathcal{D}_n} \kappa_p(AD) \quad \text{if } \text{rank}(A) = n, \quad (7.18)$$

$$\kappa_p(D_RA) \leq m^{1/p} \min_{D \in \mathcal{D}_m} \kappa_p(DA) \quad \text{if } \text{rank}(A) = m. \quad (7.19)$$

Proof. For any $X \in \mathbb{R}^{m \times n}$ we have, from (6.12),

$$\max_j \|A(:, j)\|_p \leq \|A\|_p \leq n^{1-1/p} \max_j \|A(:, j)\|_p. \quad (7.20)$$

Therefore

$$\|AD_C\|_p \leq n^{1-1/p}. \quad (7.21)$$

Now, for any $D \in \mathcal{D}_n$,

$$\begin{aligned} \|D_C^{-1}A^+\|_p &= \|D_C^{-1}D \cdot D^{-1}A^+\|_p \\ &\leq \max_j (|d_{jj}| \|A(:,j)\|_p) \|D^{-1}A^+\|_p \\ &\leq \|AD\|_p \|D^{-1}A^+\|_p = \kappa_p(AD), \end{aligned} \quad (7.22)$$

using the first inequality in (7.20). Multiplying (7.21) and (7.22) and minimizing over D , we obtain (7.18). Inequality (7.19) follows by noting that $\kappa_p(DA) = \kappa_q(A^T D)$, where $p^{-1} + q^{-1} = 1$ (see (6.21)). $\square$

For $p = \infty$, (7.19) confirms what we already know from (7.15) and (7.16) for square matrices: that in the ∞ -norm, row equilibration is an optimal row scaling strategy. Similarly, for $p = 1$, column equilibration is the best column scaling, by (7.18). Theorem 7.5 is usually stated for the 2-norm, for which it shows that row and column equilibration produce condition numbers within factors $\sqrt{m}$ and $\sqrt{n}$, respectively, of the minimum 2-norm condition numbers achievable by row and column scaling.

As a corollary of Theorem 7.5 we have the result that among two-sided diagonal scalings of a symmetric positive definite matrix, the one that gives A a unit diagonal is not far from optimal.

Corollary 7.6 (van der Sluis). *Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite and let $D_* = \text{diag}(a_{ii}^{-1/2})$. Then*

$$\kappa_2(D_*AD_*) \leq n \min_{D \in \mathcal{D}_n} \kappa_2(DAD). \quad (7.23)$$

Proof. Let $A = R^T R$ be a Cholesky factorization, note that $\kappa_2(DAD) = \kappa_2(RD)^2$, and apply Theorem 7.5 to RD . $\square$

Is the scaling D_* in Corollary 7.6 ever optimal? Forsythe and Straus [432, 1955] show that it is optimal if A is symmetric positive definite with property A (that is, there exists a permutation matrix P such that PAP^T can be expressed as a block 2×2 matrix whose $(1,1)$ and $(2,2)$ blocks are diagonal). Thus, for example, any symmetric positive definite tridiagonal matrix with unit diagonal is optimally scaled.

We note that by using (6.23) in place of (7.20), the inequalities of Theorem 7.5 and Corollary 7.6 can be strengthened by replacing m and n with the maximum number of nonzeros per column and row, respectively.

Here is an independent result for the Frobenius norm.

Theorem 7.7 (Stewart and Sun). *Let $A = [a_1, \dots, a_n] \in \mathbb{R}^{n \times n}$ be nonsingular, with $B := A^{-1} = [b_1, \dots, b_n]^T$, and let $D_C = \text{diag}((\|b_j\|_2 / \|a_j\|_2)^{1/2})$. Then*

$$\sum_j \|a_j\|_2 \|b_j\|_2 = \kappa_F(AD_C) = \min_{D \in \mathcal{D}_n} \kappa_F(AD).$$

Proof. For $D = \text{diag}(d_i) \in \mathcal{D}_n$, we have, using the Cauchy–Schwarz inequality,

$$\kappa_F(AD) = \left(\sum_j d_j^2 \|a_j\|_2^2 \right)^{1/2} \left(\sum_j d_j^{-2} \|b_j\|_2^2 \right)^{1/2} \geq \sum_j \|a_j\|_2 \|b_j\|_2,$$

with equality if $d_j \|a_j\|_2 = \alpha d_j^{-1} \|b_j\|_2$ for all j , for some $\alpha \neq 0$. There is equality for $d_j^2 = \|b_j\|_2 / \|a_j\|_2$. $\square$

As we have seen in this and the previous section, the minimum value of $\kappa_\infty(DA)$ is $\| |A^{-1}| |A| \|_\infty$. The next result shows that for two-sided scalings the matrix $|A^{-1}| |A|$ again features in the formula for the minimal condition number. A matrix is irreducible if it cannot be symmetrically permuted to block triangular form. A Perron vector of $B \geq 0$ is a nonnegative eigenvector corresponding to the eigenvalue $\rho(B)$, where ρ denotes the spectral radius.

Theorem 7.8 (Bauer). *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and suppose that $|A| |A^{-1}|$ and $|A^{-1}| |A|$ are irreducible. Then*

$$\min_{D_1, D_2 \in \mathcal{D}_n} \kappa_\infty(D_1 A D_2) = \rho(|A| |A^{-1}|). \quad (7.24)$$

The minimum is attained for $D_1 = \text{diag}(x)^{-1}$ and $D_2 = \text{diag}(|A^{-1}|x)$, where $x > 0$ is a right Perron vector of $|A| |A^{-1}|$ (so that $|A| |A^{-1}| = \rho(|A| |A^{-1}|)x$).

Proof. See Problem 7.10. $\square$

Rump [1002, 2002] shows that (7.24) holds for any nonsingular A if the minimum on the left-hand side is replaced by an infimum.

For the Kahan example (7.17),

$$\rho(|A^{-1}| |A|) \approx 2.62 + 1.79\epsilon \ll 3 + (2\epsilon)^{-1} = \| |A^{-1}| |A| \|_\infty,$$

and, in fact, $\kappa_\infty(DAD) = 3$ for $D = \text{diag}(\epsilon^{1/2}, \epsilon^{-1/2}, \epsilon^{-1/2})$, so a symmetric two-sided scaling is nearly optimal in this case.

7.4. The Matrix Inverse

We briefly discuss componentwise perturbation theory for the matrix inverse. With $X := A^{-1}$ and $X + \Delta X := (A + \Delta A)^{-1}$, a componentwise condition number is

$$\mu_E(A) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{\| \Delta X \|_\infty}{\epsilon \| X \|_\infty} : |\Delta A| \leq \epsilon E \right\} \leq \frac{\| |A^{-1}| E |A^{-1}| \|_\infty}{\| A^{-1} \|_\infty}. \quad (7.25)$$

In general, the inequality is strict, but there is equality when $|A^{-1}| = D_1 A^{-1} D_2$ for D_i of the form $\text{diag}(\pm 1)$ [444, 1992, Thm. 1.10], [479, 1982]. Another componentwise condition number is evaluated in Problem 7.11. We saw in Theorem 6.5 that the reciprocal of the normwise condition number for matrix inversion is the normwise relative distance to singularity. Is the same true for an appropriate componentwise condition number? The componentwise distance to singularity,

$$d_E(A) := \min \{ \epsilon : A + \Delta A \text{ singular, } |\Delta A| \leq \epsilon E \},$$

has been characterized by Rohn [990, 1989], [991, 1990] as

$$d_E(A) = \frac{1}{\max_{S_1, S_2} \rho_0(S_1 A^{-1} S_2 E)},$$

where the maximum is taken over all signature matrices $S_i = \text{diag}(\pm 1)$ and where

$$\rho_0(X) = \max\{|\lambda| : \lambda \text{ is a real eigenvalue of } X\}.$$

This formula involves 4^n eigenproblems and thus is computationally intractable (in fact it has been shown to be NP-hard by Poljak and Rohn [945, 1993]).

Demmel [313, 1992] shows by complexity arguments that there can be no simple relationship between $d_E(A)$ and the quantity $\| |A^{-1}|E \|_\infty$, which is an upper bound for $\mu_E(A)$. However, $d_E(A)$ behaves like $1/\rho(|A^{-1}|E)$, as

$$\frac{1}{\rho(|A^{-1}|E)} \leq d_E(A) \leq \frac{(3 + 2\sqrt{2})n}{\rho(|A^{-1}|E)}, \quad (7.26)$$

and the upper bound is almost sharp because examples are known for which $d_E(A) = n/\rho(|A^{-1}|E)$. The lower bound was obtained by Demmel [313, 1992], and the more difficult upper bound and the observation about its sharpness are due to Rump [999, 1999] (see also the references therein for the history of the bound). In the case $E = |A|$ these inequalities are aesthetically pleasing because $d_{|A|}(A)$ is invariant under two-sided diagonal scalings of A and $\rho(|A^{-1}||A|)$ is the minimum ∞ -norm condition number achievable by such scalings, as shown by Theorem 7.8.

7.5. Extensions

The componentwise analyses can be extended in three main ways.

(1) We can use more general measures of size for the data and the solution. Higham and Higham [575, 1992] measure ΔA , Δb , and Δx by

$$\nu_p((\Delta a_{ij}/e_{ij}) (\Delta b_i/f_i)), \quad \nu_p((\Delta x_i/g_i)),$$

where $\nu_p(A) = (\sum_{i,j} |a_{ij}|^p)^{1/p}$, $1 \leq p \leq \infty$, and the e_{ij} , f_i , and g_i are tolerances. They show that the corresponding backward error is given by the explicit formula

$$\left\| \left(\frac{r_j}{\|D_j \begin{bmatrix} y \\ -1 \end{bmatrix}\|_q} \right) \right\|_p,$$

where $r = b - Ay$, $D_j = \text{diag}(e_{j1}, \dots, e_{jn}, f_j)$, and $p^{-1} + q^{-1} = 1$; bounds for the corresponding condition number are also obtained. Theorem 7.3, and Theorem 7.4 with the ∞ -norm, correspond to $p = \infty$ and $g_i \equiv \|x\|_\infty$. If we take $p = \infty$ and $g = |x|$, we are measuring the change in the solution in a componentwise relative sense, as $\nu_p((\Delta x_i/g_i)) = \max_i |\Delta x_i|/|x_i|$, and the condition number is [575, 1992]

$$\| \text{diag}(|x_i|)^{-1} |A^{-1}| (E|x| + f) \|_\infty.$$

This latter case has also been considered by Rohn [989, 1989] and Gohberg and Koltracht [494, 1993]. It is also possible to obtain individual bounds for $|\Delta x_i|/|x_i|$,

$i = 1:n$, by refraining from taking norms in the analysis; see Chandrasekaran and Ipsen [217, 1995] and Problem 7.1.

(2) The backward error results and the perturbation theory can be extended to systems with multiple right-hand sides. For the general ν_p measure described in (1), the backward error can be computed by finding the minimum p -norm solutions to n underdetermined linear systems. For details, see Higham and Higham [575, 1992].

(3) Structure in A and b can be preserved in the analysis. For example, if A is symmetric or Toeplitz then its perturbation can be forced to be symmetric or Toeplitz too, while still using componentwise measures. References include Higham and Higham [574, 1992] and Gohberg and Koltracht [494, 1993] for linear structure, and Bartels and D. J. Higham [86, 1992] and Sun [1109, 1998] for Vandermonde structure. A symmetry-preserving normwise backward error for linear systems is explored by Bunch, Demmel, and Van Loan [180, 1989], while Smoktunowicz [1053, 1995] considers the componentwise case (see Problem 7.12). Symmetry-preserving normwise condition numbers for the matrix inverse and linear systems are treated by D. J. Higham [573, 1995], who shows that they cannot differ much from the general condition numbers. Rump [997, 1998] proves the same thing for componentwise condition numbers, and also shows that when symmetry is exploited in the definitions the componentwise condition number and componentwise distance to singularity can differ by an arbitrary factor.

7.6. Numerical Stability

The backward errors examined in this chapter lead to definitions of numerical stability of algorithms for solving linear systems. Precise and formal definitions of stability can be given, but there are so many possibilities, across different problems, that to define and name each one tends to cloud the issues of interest. We therefore adopt an informal approach.

A numerical method for solving a square, nonsingular linear system $Ax = b$ is *normwise backward stable* if it produces a computed solution such that $\eta_{A,b}(\hat{x})$ is of order the unit roundoff. How large we allow $\eta_{A,b}(\hat{x})/u$ to be, while still declaring the method backward stable, depends on the context. It is usually implicit in this definition that $\eta_{A,b}(\hat{x}) = O(u)$ for *all* A and b , and a method that yields $\eta_{A,b}(\hat{x}) = O(u)$ for a particular A and b is said to have performed in a normwise backward stable manner.

The significance of normwise backward stability is that the computed solution solves a slightly perturbed problem, and if the data A and b contain uncertainties bounded only normwise ($A \rightarrow A + \Delta A$ with $\|\Delta A\| = O(u\|A\|)$ and similarly for b), then $\hat{x}$ may be the exact solution to the problem we wanted to solve, for all we know.

Componentwise backward stability is defined in a similar way: we now require the componentwise backward error $\omega_{|A|,|b|}(\hat{x})$ to be of order u . This is a more stringent requirement than normwise backward stability. The rounding errors incurred by a method that is componentwise backward stable are in size and effect equivalent to the errors incurred in simply converting the data A and b to floating point numbers before the solution process begins.

Table 7.2. *Backward and forward stability.*

Componentwise backward stability $\Rightarrow$ Componentwise forward stability	
$\omega_{ A , b }(\hat{x}) = O(u)$	$\frac{\ x - \hat{x}\ }{\ x\ } = O(\text{cond}(A, x)u)$
$\Downarrow$	$\Downarrow$
Normwise backward stability $\Rightarrow$ Normwise forward stability	
$\eta_{A,b}(\hat{x}) = O(u)$	$\frac{\ x - \hat{x}\ }{\ x\ } = O(\kappa(A)u)$

If a method is normwise backward stable then, by Theorem 7.2, the forward error $\|x - \hat{x}\|/\|x\|$ is bounded by a multiple of $\kappa(A)u$. However, a method can produce a solution whose forward error is bounded in this way without the normwise backward error $\eta_{A,b}(\hat{x})$ being of order u . Hence it is useful to define a method for which $\|x - \hat{x}\|/\|x\| = O(\kappa(A)u)$ as *normwise forward stable*. By similar reasoning involving $\omega_{|A|,|b|}(\hat{x})$, we say a method is *componentwise forward stable* if $\|x - \hat{x}\|/\|x\| = O(\text{cond}(A, x)u)$. Table 7.2 summarizes the definitions and the relations between them. There are several examples in this book of linear-equation-solving algorithms that are forward stable but not backward stable: Cramer's rule for $n = 2$ (§1.10.1), Gauss–Jordan elimination (§14.4), and the seminormal equations method for underdetermined systems (§21.3).

Other definitions of numerical stability can be useful (for example, *row-wise backward stability* means that $\eta_{|A|e^T,|b|}(\hat{x}) = O(u)$), and they will be introduced when needed.

7.7. Practical Error Bounds

Suppose we have a computed approximation $\hat{x}$ to the solution of a linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$. What error bounds should we compute?

The backward error can be computed exactly, from the formulae

$$\begin{aligned}\eta_{E,f}(\hat{x}) &= \frac{\|r\|}{\|E\| \|\hat{x}\| + \|f\|}, \\ \omega_{E,f}(\hat{x}) &= \max_i \frac{|r_i|}{(E|\hat{x}| + f)_i},\end{aligned}\tag{7.27}$$

at the cost of one or two matrix–vector products, for $r = b - A\hat{x}$ and $E|\hat{x}|$. The only question is what to do if the denominator is so small as to cause overflow or division by zero in the expression for $\omega_{E,f}(\hat{x})$. This could happen, for example, when $E = |A|$ and $f = |b|$ and, for some i , $a_{ij}x_j = 0$ for all j , as is most likely in a sparse problem. LAPACK's `xyyRFS` (“refine solution”) routines apply iterative refinement in fixed precision, in an attempt to satisfy $\omega_{|A|,|b|} \leq u$. If the i th component of the denominator in (7.27) is less than `safe_min`/ u , where `safe_min` is the smallest number such that $1/\text{safe_min}$ does not overflow, then they add $(n + 1)\text{safe_min}$ to the i th components of the numerator and denominator. A more sophisticated strategy is advocated for sparse problems by Arioli, Demmel,

and Duff [30, 1989]. They suggest modifying the formula (7.27) for $\omega_{|A|,|b|}$ by replacing $|b_i|$ in the denominator by $\|A(i,:) \|_1 \|\hat{x}\|_\infty$ when the i th denominator is very small. See [30, 1989] for details and justification of this strategy.

Turning to the forward error, one approach is to evaluate the forward error bound from Theorem 7.2 or Theorem 7.4, with ϵ equal to the corresponding backward error. Because x in (7.10) is unknown, we should use the modified bound

$$\frac{\|x - \hat{x}\|_\infty}{\|\hat{x}\|_\infty} \leq \omega_{E,f}(\hat{x}) \frac{\|A^{-1}(|E|\hat{x}| + f)\|_\infty}{\|\hat{x}\|_\infty}. \quad (7.28)$$

If we have a particular E and f in mind for backward error reasons, then it is natural to use them in (7.28). However, the size of the forward error bound varies with E and f , so it is natural to ask which choice minimizes the bound.

Lemma 7.9. *The upper bound in (7.28) is at least as large as the upper bound in*

$$\frac{\|x - \hat{x}\|_\infty}{\|\hat{x}\|_\infty} \leq \frac{\|A^{-1}\|r\|_\infty}{\|\hat{x}\|_\infty}, \quad (7.29)$$

and is equal to it when $E|\hat{x}| + f$ is a multiple of $|r|$.

Proof. First note that $r = b - A\hat{x}$ implies $|x - \hat{x}| \leq |A^{-1}||r|$, which implies (7.29). Now, for $z \geq 0$,

$$|A^{-1}||r| = |A^{-1}| \left(z_i \frac{|r_i|}{z_i} \right) \leq \max_i \frac{|r_i|}{|z_i|} |A^{-1}|z,$$

with equality if z is a multiple of r . Taking $z = E|\hat{x}| + f$ gives

$$|A^{-1}||r| \leq \omega_{E,f}(\hat{x}) |A^{-1}|(|E|\hat{x}| + f|),$$

with equality when $E|\hat{x}| + f$ is a multiple of $|r|$. The truth of this statement is preserved when ∞ -norms are taken, so the result follows. $\square$

Since the bound (7.29) is obtained by taking absolute values in the equation $x - \hat{x} = A^{-1}r$, it is clearly the smallest possible such bound subject to ignoring signs in A^{-1} and r . It is reasonable to ask why we do not take $\|A^{-1}r\|_\infty / \|\hat{x}\|_\infty$ as our error bound. (Theoretically it is an exact bound!) One reason is that we cannot compute r or $\|A^{-1}r\|_\infty$ exactly. In place of r we compute $\hat{r} = fl(b - A\hat{x})$, and

$$\hat{r} = r + \Delta r, \quad |\Delta r| \leq \gamma_{n+1}(|A|\hat{x}| + |b|). \quad (7.30)$$

Therefore a strict bound, and one that should be used in practice in place of (7.29), is

$$\frac{\|x - \hat{x}\|_\infty}{\|\hat{x}\|_\infty} \leq \frac{\| |A^{-1}|(|\hat{r}| + \gamma_{n+1}(|A|\hat{x}| + |b|)) \|_\infty}{\|\hat{x}\|_\infty}. \quad (7.31)$$

Given an LU factorization of A this bound can be cheaply estimated without computing A^{-1} (see Chapter 15), and this is done by LAPACK's `xyyRFS` routines. Note also that if we did compute $A^{-1}r$ then we might as well apply a step of iterative refinement, which could well provide a more stable and accurate solution (see Chapter 12).

The LAPACK linear equation solvers estimate only one condition number: the standard condition number $\kappa_1(A)$ (or, rather, its reciprocal, referred to as `rcond`), which is returned by the `xyyCON` routines.

7.8. Perturbation Theory by Calculus

The perturbation results in this book are all derived algebraically, without any use of derivatives. Calculus can also be used to derive perturbation bounds, often in a straightforward fashion.

As a simple example, consider a linear system $A(t)x(t) = b(t)$, where $A(t) \in \mathbb{R}^{n \times n}$ and $x(t), b(t) \in \mathbb{R}^n$ are assumed to be continuously differentiable functions of t . Differentiating gives

$$\dot{A}(t)x(t) + A(t)\dot{x}(t) = \dot{b}(t),$$

or, dropping the t arguments,

$$\dot{x} = -A^{-1}\dot{A}x + A^{-1}\dot{b}.$$

Taking norms, we obtain

$$\frac{\|\dot{x}\|}{\|x\|} \leq \|A^{-1}\| \|\dot{A}\| + \|A^{-1}\| \frac{\|\dot{b}\|}{\|x\|} = \kappa(A) \left(\frac{\|\dot{A}\|}{\|A\|} + \frac{\|\dot{b}\|}{\|A\|\|x\|} \right). \quad (7.32)$$

This bound shows that $\kappa(A)$ is a key quantity in measuring the sensitivity of a linear system. A componentwise bound could have been obtained just as easily.

To convert the bound (7.32) to the more standard form of perturbation bound we can choose $A(t) = A + tE$, $b(t) = b + tf$ and write $x(\epsilon) = x(0) + \epsilon\dot{x}(0) + O(\epsilon^2)$, which leads to a first-order version of the bound (7.4).

The calculus technique is a useful addition to the armoury of the error analyst (it is used by Golub and Van Loan [509, 1996], for example), but the algebraic approach is preferable for deriving rigorous perturbation bounds of the standard forms.

7.9. Notes and References

This chapter draws on the survey paper Higham [604, 1994].

Theorem 7.3 is due to Oettli and Prager [898, 1964], and predates the normwise backward error result Theorem 7.1 of Rigoal and Gaches [986, 1967]. In addition to Theorem 7.1, Rigoal and Gaches give a more general result based on norms of blocks that includes Theorems 7.3 and 7.1 as special cases. Theorem 7.1 is also obtained by Kovarik [749, 1976].

Theorems 7.1 and 7.3 both remain valid when A is rectangular. Componentwise backward error for rectangular A was considered by Oettli, Prager, and Wilkinson [899, 1965], but their results are subsumed by those of Oettli and Prager [898, 1964] and Rigoal and Gaches [986, 1967].

For a linear system $Ax = b$ subject to componentwise perturbations, Oettli [897, 1965] shows how linear programming can be used to obtain bounds on the components of x when all solutions of the perturbed system lie in the same orthant. Cope and Rust [271, 1979] extend this approach by showing, in general, how to bound all the solutions that lie in a given orthant. This type of analysis can also be found in the book by Kuperman [758, 1971], which includes an independent derivation of Theorem 7.3. See also Hartfiel [550, 1980].

Theorem 7.4 is a straightforward generalization of a result of Skeel [1040, 1979, Thms. 2.1 and 2.2]. It is clear from Bauer's comments in [91, 1966] that the bound (7.10), with $E = |A|$ and $f = |b|$, was known to him, though he does not state the bound. This is the earliest reference we know in which componentwise analysis is used to derive forward perturbation bounds.

Theorem 7.8 is from Bauer [90, 1963]. Bauer actually states that equality holds in (7.24) for any A , but his proof of equality is valid only when $|A^{-1}||A|$ and $|A||A^{-1}|$ have positive Perron vectors. Businger [187, 1968] proves that a sufficient condition for the irreducibility condition of Theorem 7.8 to hold (which, of course, implies the positivity of the Perron vectors) is that there do not exist permutations P and Q such that PAQ is in block triangular form.

Theorem 7.7 is from Stewart and Sun [1083, 1990, Thm. 4.3.5].

Further results on scaling to minimize the condition number $\kappa(A)$ are given by Forsythe and Straus [432, 1955], Bauer [92, 1969], Golub and Varah [504, 1974], McCarthy and Strang [832, 1974], Shapiro [1033, 1982], [1034, 1985], [1035, 1991], and Watson [1209, 1991].

Chan and Foulser [213, 1988] introduce the idea of "effective conditioning" for linear systems, which takes into account the projections of b onto the range space of A . See Problem 7.5, and for an application to partial differential equations see Christiansen and Hansen [234, 1994].

For an example of how definitions of numerical stability for linear equation solvers can be extended to incorporate structure in the problem, see Bunch [179, 1987].

An interesting application of linear system perturbation analysis is to Markov chains. A discrete-time Markov chain can be represented by a square matrix P , where p_{ij} is the probability of a transition from state i to state j . Since state i must lead to some other state, $\sum_j p_{ij} = 1$, and these conditions can be written in matrix-vector form as

$$Pe = e. \quad (7.33)$$

A nonnegative matrix satisfying (7.33) is called a *stochastic matrix*. The initial state of the Markov chain can be defined by a vector z^T , where z_i denotes the probability that the i th state of the chain is occupied. Then the state of the chain at the next time unit is given by $z^T P$. The *steady state* or *stationary vector* of the chain is given by

$$\pi^T = \lim_{k \rightarrow \infty} z^T P^k.$$

An important question is the sensitivity of the individual components of the steady-state vector to perturbations in P . This is investigated, for example, by Ipsen and Meyer [665, 1994], who measure the perturbation matrix normwise, and by O'Kinneide [896, 1993], who measures the perturbation matrix componentwise. For a matrix-oriented development of Markov chain theory see Berman and Plemmons [106, 1994].

It is possible to develop probabilistic perturbation theory for linear systems and other problems by making assumptions about the statistical distribution of the perturbations. We do not consider this approach here (though see Problem 7.14), but refer the interested reader to the papers by Fletcher [416, 1985], Stewart [1072, 1990], and Weiss, Wasilkowski, Woźniakowski, and Shub [1214, 1986].

Problems

7.1. Under the conditions of Theorem 7.4, show that

$$|x - y| \leq \epsilon(I - \epsilon|A^{-1}|E)^{-1}|A^{-1}|(f + E|x|).$$

Hence derive a first-order bound for $|x_i - y_i|/|x_i|$.

7.2. Let $Ax = b$, where $A \in \mathbb{R}^{n \times n}$. Show that for any vector y and any subordinate matrix norm,

$$\frac{\|r\|}{\|A\| \|x\|} \leq \frac{\|x - y\|}{\|x\|} \leq \kappa(A) \frac{\|r\|}{\|A\| \|x\|},$$

where the residual $r = b - Ay$. Interpret this result.

7.3. Prove (7.16) and deduce (7.15).

7.4. Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite and let $A = DHD$, where $D = \text{diag}(a_{ii}^{1/2})$ (this is the scaling used in Corollary 7.6). Show that $\text{cond}(H) \leq \kappa_\infty(H) \leq n \text{cond}(H)$.

7.5. (Chan and Foulser [213, 1988]) Let $A \in \mathbb{R}^{n \times n}$ have the SVD $A = U\Sigma V^T$, where $\Sigma = \text{diag}(\sigma_i)$, $\sigma_1 \geq \dots \geq \sigma_n$, and define the projection matrix $P_k := U_k U_k^T$, where $U_k = U(:, n+1-k:n)$. Show that if $Ax = b$ and $A(x + \Delta x) = (b + \Delta b)$ then

$$\frac{\|\Delta x\|_2}{\|x\|_2} \leq \frac{\sigma_{n+1-k}}{\sigma_n} \frac{\|b\|_2}{\|P_k b\|_2} \frac{\|\Delta b\|_2}{\|b\|_2}, \quad k = 1:n.$$

What is the interpretation of this result?

7.6. (a) For the choice of tolerances $E = |A|ee^T$, $f = |b|$, corresponding to a row-wise backward error, show that

$$\text{cond}(A) \frac{\|x\|_1}{\|x\|_\infty} \leq \text{cond}_{E,f}(A, x) \leq 2 \text{cond}(A) \frac{\|x\|_1}{\|x\|_\infty}.$$

(b) For $E = ee^T|A|$ and $f = \|b\|_1 e$, corresponding to a columnwise backward error, show that

$$\|A^{-1}\|_\infty \frac{\| |A| |x| \|_1}{\|x\|_\infty} \leq \text{cond}_{E,f}(A, x) \leq 2 \|A^{-1}\|_\infty \frac{\| |A| |x| \|_1}{\|x\|_\infty}.$$

7.7. Show that

$$\begin{aligned} \omega_{|A|,|b|}(y) &\leq \omega_{|A|,0}(y) \leq \frac{2\omega_{|A|,|b|}(y)}{1 - \omega_{|A|,|b|}(y)}, \\ \eta_{A,b}(y) &\leq \eta_{A,0}(y) \leq \frac{2\eta_{A,b}(y)}{1 - \eta_{A,b}(y)}. \end{aligned}$$

7.8. Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Show that the normwise backward error defined in terms of a parameter $\theta \geq 0$ by

$$\eta_F(y) := \min\{ \|\Delta A, \theta \Delta b\|_F : (A + \Delta A)y = b + \Delta b \}$$

is given by

$$\eta_F(y) = \frac{\theta \|r\|_2}{\sqrt{\theta^2 \|y\|_2^2 + 1}},$$

where $r = b - Ay$.

7.9. Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. A componentwise condition number for the problem of computing $c^T x$, where $Ax = b$, can be defined by

$$\chi_{E,f}(A, x) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{|c^T \Delta x|}{\epsilon |c^T x|} : (A + \Delta A)(x + \Delta x) = b + \Delta b, \right. \\ \left. |\Delta A| \leq \epsilon E, \quad |\Delta b| \leq \epsilon f \right\}.$$

Obtain an explicit formula for $\chi_{E,f}(A, x)$. Show that $\chi_{E,f}(A, x) \geq 1$ if $E = |A|$ or $f = |b|$. Derive the corresponding normwise condition number $\psi_{E,f}(A, x)$, in which the constraints are $\|\Delta A\|_2 \leq \epsilon \|E\|_2$ and $\|\Delta b\|_2 \leq \epsilon \|f\|_2$.

7.10. (Bauer [90, 1963]) Let $A, B, C \in \mathbb{R}^{n \times n}$. (a) Prove that if B and C have positive elements then

$$\min_{D_1, D_2 \in \mathcal{D}_n} \|D_1 B D_2\|_\infty \|D_2^{-1} C D_1^{-1}\|_\infty = \rho(BC),$$

where $\mathcal{D}_n = \{\text{diag}(d_i) : d_i > 0, i = 1:n\}$. (Hint: consider $D_1 = \text{diag}(x_1)^{-1}$ and $D_2 = \text{diag}(C x_1)$, where $x_1 > 0$ is a right Perron vector of BC : $BC x_1 = \rho(BC) x_1$.)

(b) Deduce that if $|A|$ and $|A^{-1}|$ have positive entries, then

$$\min_{D_1, D_2 \in \mathcal{D}_n} \kappa_\infty(D_1 A D_2) = \rho(|A| |A^{-1}|).$$

(c) Show that for any nonsingular A ,

$$\inf_{D_1, D_2 \in \mathcal{D}_n} \kappa_\infty(D_1 A D_2) \leq \rho(|A| |A^{-1}|).$$

(d) Strengthen (b) by showing that for any nonsingular A such that $|A| |A^{-1}|$ and $|A^{-1}| |A|$ are irreducible,

$$\min_{D_1, D_2 \in \mathcal{D}_n} \kappa_\infty(D_1 A D_2) = \rho(|A| |A^{-1}|).$$

(e) What can you deduce about $\min_{D_1, D_2 \in \mathcal{D}_n} \kappa(D_1 A D_2)$ for the 1- and 2-norms?

7.11. (Bauer [91, 1966, p. 413], Rohn [989, 1989]) We can modify the definition of $\mu_E(A)$ in (7.25) by measuring ΔX componentwise relative to X , giving

$$\mu'_E(A) := \limsup_{\epsilon \rightarrow 0} \left\{ \max_{i,j} \frac{|\Delta x_{ij}|}{\epsilon |x_{ij}|} : |\Delta A| \leq \epsilon E \right\}$$

(where $X = A^{-1}$ and $X + \Delta X = (A + \Delta A)^{-1}$). Show that

$$\mu'_E(A) = \max_{i,j} \frac{(|A^{-1}| E |A^{-1}|)_{ij}}{(|A^{-1}|)_{ij}}.$$

7.12. Let $A \in \mathbb{R}^{n \times n}$ be symmetric and let y be an approximate solution to $Ax = b$. If y has a small backward error, so that y solves a nearby system, does it follow that y solves a nearby *symmetric* system? This problem answers the question for both the normwise and componentwise relative backward errors.

(a) (Bunch, Demmel, and Van Loan [180, 1989]) Show that if $(A + G)y = b$ then there exists $H = H^T$ such that $(A + H)y = b$ with $\|H\|_2 \leq \|G\|_2$ and $\|H\|_F \leq \sqrt{2}\|G\|_F$. (This result does not require $A = A^T$.)

(b) (Smoktunowicz [1053, 1995]) Show that if A is symmetric and diagonally dominant and $(A + G)y = b$ with $|G| \leq \epsilon|A|$, then there exists $H = H^T$ such that $(A + H)y = b$ with $|H| \leq 3\epsilon|A|$. (For a *general* symmetric A there may not exist such an H , as is easily shown by a 2×2 example [574, 1992].)

(c) (Smoktunowicz [1053, 1995]) Show that if A is symmetric positive definite and $(A + G)y = b$ with $|G| \leq \epsilon|A|$ then there exists $H = H^T$ such that $(A + H)y = b$ with $|H| \leq (2n - 1)\epsilon|A|$.

7.13. Suppose that $A \in \mathbb{R}^{n \times n}$ has w_i nonzeros in its i th row, $i = 1:n$. Show that the inequality (7.31) can be replaced by

$$\frac{\|x - \hat{x}\|_\infty}{\|\hat{x}\|_\infty} \leq \frac{\| |A^{-1}| (|\hat{r}| + \Gamma(|A||\hat{x}| + |b|)) \|_\infty}{\|\hat{x}\|_\infty},$$

where $\Gamma = \text{diag}(\gamma_{w_i+1})$. This bound is potentially much smaller than (7.31) for large, sparse matrices.

7.14. (D. J. Higham, after Fletcher [416, 1985]) Suppose the nonsingular, square matrix A is perturbed to $A + \Delta A$ and b to $b + \Delta b$. Then, to first order, the solution of $Ax = b$ is perturbed to $x + \Delta x$, where

$$\Delta x = -A^{-1}\Delta Ax + A^{-1}\Delta b.$$

Suppose that the perturbations have the form

$$\Delta a_{ij} = \epsilon_{ij}e_{ij}, \quad \Delta b_i = \delta_i f_i,$$

where the ϵ_{ij} and δ_i are independent random variables, each having zero mean and variance σ^2 . (As usual, the matrix E and vector f represent fixed tolerances.) Let $\mathcal{E}$ denote the expected value.

(a) Show that

$$\mathcal{E}(\|\Delta x\|_2^2) = \sigma^2 \| [A^{-1}][E][x] + [A^{-1}][f] \|_1,$$

where square brackets denote the operation of elementwise squaring: $[B]_{ij} = b_{ij}^2$.

(b) Hence explain why

$$\text{condexp}(A, x) := \frac{(\| [A^{-1}][E][x] + [A^{-1}][f] \|_1)^{1/2}}{\|x\|_2}$$

may be regarded as an “expected condition number” for the linear system $Ax = b$.

(c) For the case where $e_{ij} \equiv \|A\|_2$ and $f_j \equiv \|b\|_2$, compare $\text{condexp}(A, x)$ with the “worst-case” condition number $\kappa_{A,b}(A, x)$ for the 2-norm.

7.15. (Horn and Johnson [637, 1991, p. 331]) Prove that for any nonsingular $A \in \mathbb{R}^{n \times n}$,

$$\inf_{D_1, D_2 \in \mathcal{D}_n} \kappa_2(D_1 A D_2) \geq \|A \circ A^{-T}\|_2,$$

where $\circ$ is the Hadamard product ($A \circ B = (a_{ij}b_{ij})$) and $\mathcal{D}_n$ is defined as in Problem 7.10. (Hint: use the inequality $\|A \circ B\|_2 \leq \|A\|_2 \|B\|_2$.) Discuss the attainability of this bound.

Chapter 8

Triangular Systems

*In the end there is left the coefficient of one unknown and the constant term.
An elimination between this equation and
one from the previous set that contains two unknowns
yields an equation with the coefficient of
another unknown and another constant term, etc.
The quotient of the constant term by the unknown
yields the value of the unknown in each case.*

— JOHN V. ATANASOFF, *Computing Machine for the Solution of
Large Systems of Linear Algebraic Equations* (1940)

The solutions of triangular systems are usually computed to high accuracy.
*This fact ... cannot be proved in general, for counter examples exist.
However, it is true of many special kinds of triangular matrices and
the phenomenon has been observed in many others.
The practical consequences of this fact cannot be over-emphasized.*

— G. W. STEWART, *Introduction to Matrix Computations* (1973)

*In practice one almost invariably finds that
if L is ill-conditioned, so that $\|L\| \|L^{-1}\| \gg 1$,
then the computed solution of $Lx = b$ (or the computed inverse)
is far more accurate than [standard norm bounds] would suggest.*

— J. H. WILKINSON, *Rounding Errors in Algebraic Processes* (1963)

Triangular systems play a fundamental role in matrix computations. Many methods are built on the idea of reducing a problem to the solution of one or more triangular systems, including virtually all direct methods for solving linear systems. On serial computers triangular systems are universally solved by the standard back and forward substitution algorithms. For parallel computation there are several alternative methods, one of which we analyse in §8.4.

Backward error analysis for the substitution algorithms is straightforward and the conclusion is well known: the algorithms are extremely stable. The behaviour of the forward error, however, is intriguing, because the forward error is often surprisingly small—much smaller than we would predict from the normwise condition number κ , or, sometimes, even the componentwise condition number cond . The quotes from Stewart and Wilkinson at the start of this chapter emphasize the high accuracy that is frequently observed in practice. The analysis we give in this chapter provides a partial explanation for the observed accuracy of the substitution algorithms. In particular, it reveals three important but nonobvious properties:

- the accuracy of the computed solution from substitution depends strongly on the right-hand side;
- a triangular matrix may be much more or less ill conditioned than its transpose; and
- the use of pivoting in LU, QR, and Cholesky factorizations can greatly improve the conditioning of a resulting triangular system.

As well as deriving backward and forward error bounds, we show how to compute upper and lower bounds for the inverse of a triangular matrix.

8.1. Backward Error Analysis

Recall that for an upper triangular matrix $U \in \mathbb{R}^{n \times n}$ the system $Ux = b$ can be solved using the formula $x_i = (b_i - \sum_{j=i+1}^n u_{ij}x_j)/u_{ii}$, which yields the components of x in order from last to first.

Algorithm 8.1 (back substitution). Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$ this algorithm solves the system $Ux = b$.

```

 $x_n = b_n/u_{nn}$ 
for  $i = n - 1 : -1 : 1$ 
     $s = b_i$ 
    for  $j = i + 1 : n$ 
         $s = s - u_{ij}x_j$ 
    end
     $x_i = s/u_{ii}$ 
end

```

Cost: n^2 flops.

We will not state the analogous algorithm for solving a lower triangular system, forward substitution. All the results below for back substitution have obvious analogues for forward substitution. Throughout this chapter T denotes a matrix that can be upper or lower triangular.

To analyse the errors in substitution we need the following lemma.

Lemma 8.2. *Let $y = (c - \sum_{i=1}^{k-1} a_i b_i)/b_k$ be evaluated in floating point arithmetic according to*

```

s = c
for i = 1:k-1
    s = s - a_i b_i
end
y = s/b_k

```

Then the computed $\hat{y}$ satisfies

$$b_k \hat{y}(1 + \theta_k) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_i), \quad (8.1)$$

where $|\theta_i| \leq \gamma_i = iu/(1 - iu)$.

Proof. Analysis very similar to that leading to (3.2) shows that $\hat{s} := fl(c - \sum_{i=1}^{k-1} a_i b_i)$ satisfies

$$\hat{s} = c(1 + \delta_1) \dots (1 + \delta_{k-1}) - \sum_{i=1}^{k-1} a_i b_i (1 + \epsilon_i)(1 + \delta_i) \dots (1 + \delta_{k-1}),$$

where $|\epsilon_i|, |\delta_i| \leq u$. The final division yields, using (2.5), $\hat{y} = fl(\hat{s}/b_k) = \hat{s}/(b_k(1 + \delta_k))$, $|\delta_k| \leq u$, so that, after dividing through by $(1 + \delta_1) \dots (1 + \delta_{k-1})$, we have

$$b_k \hat{y} \frac{1 + \delta_k}{(1 + \delta_1) \dots (1 + \delta_{k-1})} = c - \sum_{i=1}^{k-1} a_i b_i \frac{1 + \epsilon_i}{(1 + \delta_1) \dots (1 + \delta_{i-1})}.$$

The result is obtained on invoking Lemma 3.1. $\square$

Two remarks are in order. First, we chose the particular form of (8.1), in which c is not perturbed, in order to obtain a backward error result for $Ux = b$ in which b is not perturbed. Second, we carefully kept track of the terms $1 + \delta_i$ in the proof, so as to obtain the best possible constants. Direct application of the lemma to Algorithm 8.1 yields a backward error result.

Theorem 8.3. *The computed solution $\hat{x}$ from Algorithm 8.1 satisfies*

$$(U + \Delta U)\hat{x} = b, \quad |\Delta u_{ij}| \leq \begin{cases} \gamma_{n-i+1}|u_{ii}|, & i = j, \\ \gamma_{|i-j|}|u_{ij}|, & i \neq j. \end{cases} \quad \square$$

Theorem 8.3 holds only for the particular ordering of arithmetic operations used in Algorithm 8.1. A result that holds for any ordering is a consequence of the next lemma.

Lemma 8.4. *If $y = (c - \sum_{i=1}^{k-1} a_i b_i)/b_k$ is evaluated in floating point arithmetic, then, no matter what the order of evaluation,*

$$b_k \hat{y}(1 + \theta_k^{(0)}) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_k^{(i)}),$$

where $|\theta_k^{(i)}| \leq \gamma_k$ for all i . If $b_k = 1$, so that there is no division, then $|\theta_k^{(i)}| \leq \gamma_{k-1}$ for all i .

Proof. The result is not hard to see after a little thought, but a formal proof is tedious to write down. Note that the ordering used in Lemma 8.2 is the one for which this lemma is least obvious! The last part of the lemma is useful when analysing unit lower triangular systems, and in various other contexts. $\square$

Theorem 8.5. *Let the triangular system $Tx = b$, where $T \in \mathbb{R}^{n \times n}$ is nonsingular, be solved by substitution, with any ordering. Then the computed solution $\hat{x}$ satisfies*

$$(T + \Delta T)\hat{x} = b, \quad |\Delta T| \leq \gamma_n |T|. \quad \square$$

In technical terms, this result says that $\hat{x}$ has a tiny componentwise relative backward error. In other words, the backward error is about as small as we could possibly hope.

In most of the remaining error analyses in this book, we will derive results that, like the one in Theorem 8.5, do not depend on the ordering of the arithmetic operations. Results of this type are more general, usually no less informative, and easier to derive, than ones that depend on the ordering. However, it is important to realise that *the actual error does depend on the ordering*, possibly strongly so for certain data. This point is clear from Chapter 4 on summation.

8.2. Forward Error Analysis

From Theorems 8.5 and 7.4 there follows the forward error bound

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \frac{\text{cond}(T, x) \gamma_n}{1 - \text{cond}(T) \gamma_n}, \quad (8.2)$$

where

$$\text{cond}(T, x) = \frac{\|T^{-1}\|_\infty \|x\|_\infty}{\|x\|_\infty}, \quad \text{cond}(T) = \|T^{-1}\|_\infty \|T\|_\infty.$$

This bound can, of course, be arbitrarily smaller than the corresponding bound involving $\kappa_\infty(T) = \|T\|_\infty \|T^{-1}\|_\infty$, for the reasons explained in Chapter 7. For further insight, note that, in terms of the traditional condition number, $\kappa(T)$, ill conditioning of a triangular matrix stems from two possible sources: variation in the size of the diagonal elements and rows with off-diagonal elements which are large relative to the diagonal elements. Significantly, because of its row scaling invariance, $\text{cond}(T, x)$ is susceptible only to the second source.

Despite its pleasing properties, $\text{cond}(T, x)$ can be arbitrarily large. This is illustrated by the upper triangular matrix

$$U(\alpha) = (u_{ij}), \quad u_{ij} = \begin{cases} 1, & i = j, \\ -\alpha, & i < j, \end{cases} \quad (8.3)$$

for which

$$(U(\alpha)^{-1})_{ij} = \begin{cases} 1, & i = j, \\ \alpha(1 + \alpha)^{j-i-1}, & j > i. \end{cases} \quad (8.4)$$

We have $\text{cond}(U(\alpha), e) = \text{cond}(U(\alpha)) \sim 2\alpha^{n-1}$ as $\alpha \rightarrow \infty$. Therefore we cannot assert that *all* triangular systems are solved to high accuracy. Nevertheless, for any T there is always at least one system for which high accuracy is obtained: the system $Tx = e_1$ if T is upper triangular, or $Tx = e_n$ if T is lower triangular. In both cases $\text{cond}(T, x) = 1$, and the solution comprises the computation of just a single scalar reciprocal.

To gain further insight we consider special classes of triangular matrices, beginning with one produced by certain standard factorizations with pivoting. In all the results below, the triangular matrices are assumed to be $n \times n$ and nonsingular, and $\hat{x}$ is the computed solution from substitution.

Lemma 8.6. *Suppose the upper triangular matrix $U \in \mathbb{R}^{n \times n}$ satisfies*

$$|u_{ii}| \geq |u_{ij}| \quad \text{for all } j > i. \quad (8.5)$$

Then the unit upper triangular matrix $W = |U^{-1}||U|$ satisfies $w_{ij} \leq 2^{j-i}$ for all $j > i$, and hence $\text{cond}(U) \leq 2^n - 1$.

Proof. We can write $W = |V^{-1}||V|$ where $V = D^{-1}U$ and $D = \text{diag}(u_{ii})$. The matrix V is unit upper triangular with $|v_{ij}| \leq 1$, and it is easy to show that $|(V^{-1})_{ij}| \leq 2^{j-i-1}$ for $j > i$. Thus, for $j > i$,

$$w_{ij} = \sum_{k=i}^j |(V^{-1})_{ik}| |v_{kj}| \leq 1 + \sum_{k=i+1}^j 2^{k-i-1} \cdot 1 = 2^{j-i}. \quad \square$$

Theorem 8.7. *Under the conditions of Lemma 8.6, the computed solution $\hat{x}$ to $Ux = b$ obtained by substitution satisfies*

$$|x_i - \hat{x}_i| \leq 2^{n-i+1} \gamma_n \max_{j \geq i} |\hat{x}_j|, \quad i = 1:n.$$

Proof. From Theorem 8.5 we have

$$|x - \hat{x}| = |U^{-1} \Delta U \hat{x}| \leq \gamma_n |U^{-1}| |U| |\hat{x}|.$$

Using Lemma 8.6 we obtain

$$|x_i - \hat{x}_i| \leq \gamma_n \sum_{j=i}^n w_{ij} |\hat{x}_j| \leq \gamma_n \max_{j \geq i} |\hat{x}_j| \sum_{j=i}^n 2^{j-i} \leq 2^{n-i+1} \gamma_n \max_{j \geq i} |\hat{x}_j|. \quad \square$$

Lemma 8.6 shows that for U satisfying (8.5), $\text{cond}(U)$ is bounded for fixed n , no matter how large $\kappa(U)$. The bounds for $|x_i - \hat{x}_i|$ in Theorem 8.7, although large if n is large and i is small, decay exponentially with increasing i —thus, later components of x are always computed to high accuracy relative to the elements already computed.

Analogues of Lemma 8.6 and Theorem 8.7 hold for lower triangular L satisfying

$$|l_{ii}| \geq |l_{ij}| \quad \text{for all } j < i. \quad (8.6)$$

Note, however, that if the upper triangular matrix T satisfies (8.5) then T^T does not necessarily satisfy (8.6). In fact, $\text{cond}(T^T)$ can be arbitrarily large, as shown by the example

$$T = \begin{bmatrix} 1 & 1 & 0 \\ 0 & \epsilon & \epsilon \\ 0 & 0 & 1 \end{bmatrix},$$

$$\text{cond}(T) = 5, \quad \text{cond}(T^T) = 1 + \frac{2}{\epsilon}.$$

An important conclusion is that a triangular system $Tx = b$ can be much more or less ill conditioned than the system $T^Ty = c$, even if T satisfies (8.5).

Lemma 8.6 and Theorem 8.7, or their lower triangular analogues, are applicable to

- the lower triangular matrices from Gaussian elimination with partial pivoting, rook pivoting, and complete pivoting;
- the upper triangular matrices from Gaussian elimination with rook pivoting and complete pivoting;
- the upper triangular matrices from the Cholesky and QR factorizations with complete pivoting and column pivoting, respectively.

A more specialized class of upper triangular matrices than those satisfying (8.5) is those that are row diagonally dominant, that is, $U \in \mathbb{R}^{n \times n}$ satisfying

$$|u_{ii}| \leq \sum_{j=i+1}^n |u_{ij}|, \quad i = 1:n-1.$$

Such matrices arise as the upper triangular matrices in the LU factorization without pivoting of row diagonally dominant matrices, and for them a much stronger result than Lemma 8.6 holds.

Lemma 8.8. *If the upper triangular matrix $U \in \mathbb{R}^{n \times n}$ is row diagonally dominant then $(|U^{-1}||U|)_{ij} \leq i + j - 1$ and $\text{cond}(U) \leq 2n - 1$.*

Proof. The proof follows that of Lemma 8.6, with D and V as defined there. Using the fact that U , and hence V , is row diagonally dominant we find that both

V and V^{-1} have elements bounded in magnitude by 1. The bound $|w_{ij}| \leq i+j-1$, where $W = |U^{-1}||U|$, is then immediate. Finally,

$$\begin{aligned}\|W\|_\infty &= \| |W|e \|_\infty = \| |V^{-1}||V|e \|_\infty \\ &\leq \| |V^{-1}| [2 \ 2 \ \dots \ 2 \ 1]^T \|_\infty \\ &= 2n - 1. \quad \square\end{aligned}$$

It follows from Lemma 8.8 and (8.2) that row diagonally dominant upper triangular systems are solved to essentially perfect normwise relative accuracy.

Next, we consider triangular T satisfying

$$t_{ii} > 0, \quad t_{ij} \leq 0 \quad \text{for all } i \neq j.$$

It is easy to see that such a matrix has an inverse with nonnegative elements, and hence is an M -matrix (for definitions of an M -matrix see the Notes and References). Associated with any square matrix A is the *comparison matrix*:

$$M(A) = (m_{ij}), \quad m_{ij} = \begin{cases} |a_{ii}|, & i = j, \\ -|a_{ij}|, & i \neq j. \end{cases} \quad (8.7)$$

For any nonsingular triangular T , $M(T)$ is an M -matrix. Furthermore, it is easy to show that $|T^{-1}| \leq M(T)^{-1}$ (see Theorem 8.12).

The following result shows that among all matrices R such that $|R| = |T|$, $R = M(T)$ is the one that maximizes $\text{cond}(R, x)$.

Lemma 8.9. *For any triangular T ,*

$$\text{cond}(T, x) \leq \text{cond}(M(T), x) = \| (2M(T)^{-1} \text{diag}(|t_{ii}|) - I)|x| \|_\infty / \|x\|_\infty.$$

Proof. The inequality follows from $|T^{-1}| \leq M(T)^{-1}$, together with $|T| = |M(T)|$. Since $M(T)^{-1} \geq 0$, we have

$$\begin{aligned}|M(T)^{-1}||M(T)| &= M(T)^{-1}(2 \text{diag}(|t_{ii}|) - M(T)) \\ &= 2M(T)^{-1} \text{diag}(|t_{ii}|) - I,\end{aligned}$$

which yields the equality. $\square$

If $T = M(T)$ has unit diagonal then, using Lemma 8.9,

$$\text{cond}(T) = \text{cond}(T, e) = \|2T^{-1} - I\|_\infty \approx 2 \frac{\kappa(T)}{\|T\|_\infty}.$$

This means, for example, that the system $U(1)x = b$ (see (8.3)), where $x = e$, is about as ill conditioned with respect to componentwise relative perturbations in $U(1)$ as it is with respect to normwise perturbations in $U(1)$.

The next result gives a forward error bound for substitution that is proved directly, without reference to the backward error result Theorem 8.5 (indeed, it cannot be obtained from that result!). The bound can be very weak, because $\|M(T)^{-1}\|$ can be arbitrarily larger than $\|T^{-1}\|$ (see Problem 8.2), but it yields a pleasing bound in the special case described in the corollary.

Theorem 8.10. *The computed solution $\hat{x}$ obtained from substitution applied to the triangular system $Tx = b$ of order n satisfies*

$$|x - \hat{x}| \leq ((n^2 + n + 1)u + O(u^2))M(T)^{-1}|b|.$$

Proof. Without loss of generality, suppose $T = L$ is lower triangular. The proof is by induction on the components of x . The result clearly holds for the first component. Assume that it holds for the first $n - 1$ components. An analogue of Lemma 8.4 shows that

$$l_{nn}\hat{x}_n = b_n(1 + \theta_{n+1}^{(0)}) - \sum_{j=1}^{n-1} l_{nj}\hat{x}_j(1 + \theta_{n+1}^{(j)}),$$

where $|\theta_{n+1}^{(j)}| \leq \gamma_{n+1}$ for all j . Subtracting from $l_{nn}x_n = b_n - \sum_{j=1}^{n-1} l_{nj}x_j$ gives

$$l_{nn}(x_n - \hat{x}_n) = -b_n\theta_{n+1}^{(0)} - \sum_{j=1}^{n-1} l_{nj}(x_j - \hat{x}_j) + \sum_{j=1}^{n-1} l_{nj}\hat{x}_j\theta_{n+1}^{(j)},$$

so that

$$|x_n - \hat{x}_n| \leq \gamma_{n+1} \frac{|b_n|}{|l_{nn}|} + \sum_{j=1}^{n-1} \frac{|l_{nj}|}{|l_{nn}|} |x_j - \hat{x}_j| + \gamma_{n+1} \sum_{j=1}^{n-1} \frac{|l_{nj}|}{|l_{nn}|} |\hat{x}_j|. \quad (8.8)$$

Write

$$M(L) = \begin{bmatrix} M_{11} & 0 \\ -m^T & m_{nn} \end{bmatrix}, \quad M(L)^{-1} = \begin{bmatrix} M_{11}^{-1} & 0 \\ m_{nn}^{-1}m^T M_{11}^{-1} & m_{nn}^{-1} \end{bmatrix}.$$

Then the inductive assumption can be written as $|\hat{x}(1:n-1) - x(1:n-1)| \leq \mu_{n-1}M_{11}^{-1}|b(1:n-1)|$, which implies $|\hat{x}(1:n-1)| \leq (\mu_{n-1} + 1)M_{11}^{-1}|b(1:n-1)|$. Hence (8.8) gives

$$\begin{aligned} |x_n - \hat{x}_n| &\leq \gamma_{n+1}m_{nn}^{-1}|b_n| + \mu_{n-1}m_{nn}^{-1}m^T M_{11}^{-1}|b(1:n-1)| \\ &\quad + \gamma_{n+1}(\mu_{n-1} + 1)m_{nn}^{-1}m^T M_{11}^{-1}|b(1:n-1)| \\ &\leq (\mu_{n-1} + \gamma_{n+1}(\mu_{n-1} + 1))(M(L)^{-1}|b|)_n. \end{aligned}$$

We have the recurrence $\mu_k = (1 + \gamma_{k+1})\mu_{k-1} + \gamma_{k+1} \leq (1 + \gamma_{n+1})\mu_{k-1} + \gamma_{n+1}$, $\mu_0 = u$, which yields

$$\mu_n \leq (1 + \gamma_{n+1})^n u + ((1 + \gamma_{n+1})^n - 1) \leq (n^2 + n + 1)u + O(u^2). \quad \square$$

Corollary 8.11. *The computed solution obtained from substitution applied to the triangular system $Tx = b$ of order n , where $T = M(T)$ and $b \geq 0$, satisfies*

$$|x - \hat{x}| \leq ((n^2 + n + 1)u + O(u^2))|x|. \quad \square$$

Corollary 8.11 shows that, when T is an M -matrix and the right-hand side is nonnegative, the solution is obtained to high relative accuracy in every component. The reason for the high accuracy is that for such a system there are no subtractions of like-signed numbers, so that each x_i is computed as a sum of nonnegative quantities. A consequence of the corollary is that the inverse of a triangular M -matrix can be computed to high relative accuracy.

Triangular systems of the type in Corollary 8.11 occur in linear equations obtained by discretizing certain elliptic partial differential equations, such as the Poisson equation on a rectangle, with zero boundary conditions and a positive forcing function: these problems yield symmetric positive definite M -matrices, and the LU factors of an M -matrix are themselves M -matrices. Such systems also occur when evaluating the bounds of the next section.

8.3. Bounds for the Inverse

In this section we describe bounds for the inverse of a triangular matrix and show how they can be used to bound condition numbers. All the bounds in this section have the property that they depend only on the absolute values of the elements of the matrix. The norm estimation methods of Chapter 15, on the other hand, do take account of the signs of the elements.

The bounds are all based on the following theorem, whose easy proof we omit.

Theorem 8.12. *If U is a nonsingular upper triangular matrix then*

$$|U^{-1}| \leq M(U)^{-1} \leq W(U)^{-1} \leq Z(U)^{-1},$$

where the upper triangular matrices $W(U)$ and $Z(U)$ are defined as follows:

$$w_{ij} = \begin{cases} |u_{ii}|, & i = j, \\ -\max_{i+1 \leq k \leq n} |u_{ik}|, & i < j, \end{cases}$$

$$z_{ij} = \begin{cases} \alpha, & i = j, \\ -\alpha\beta, & i < j, \end{cases}$$

where $\alpha = \min_k |u_{kk}|$, $\beta = \max_{i < j} |u_{ij}|/|u_{ii}|$. $\square$

Theorem 8.12 is a special case of results in the theory of M -matrices. For more general results couched in terms of matrix minorants and diagonal dominance, respectively, see Dahlquist [288, 1983] and Varga [1190, 1976]; see also Householder [644, 1964, Exercise 15, p. 58].

An obvious implication of the theorem is that for any vector z and any absolute norm

$$\| |U^{-1}|z \| \leq \| M(U)^{-1}z \| \leq \| W(U)^{-1}z \| \leq \| Z(U)^{-1}z \|.$$

By taking $z = |U|e$, $z = |U||x|$, and $z = e$, respectively, we obtain upper bounds for $\text{cond}(U)$, $\text{cond}(U, x)$, and $\kappa_\infty(U)$. The cost of computing these bounds is just the cost of solving a triangular system with coefficient matrix $M(U)$, $W(U)$, or

$Z(U)$, which is easily seen to be $O(n^2)$, $O(n)$, and $O(1)$ flops, respectively. By comparison, computing any of these condition numbers exactly costs $O(n^3)$ flops.

As an example, here is how to compute an upper bound for $\|T^{-1}\|_\infty$ in n^2 flops.

Algorithm 8.13. Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$, this algorithm computes $\mu = \|M(U)^{-1}\|_\infty \geq \|U^{-1}\|_\infty$.

```

 $y_n = 1/|u_{nn}|$ 
for  $i = n - 1 : -1 : 1$ 
   $s = 1$ 
  for  $j = i + 1 : n$ 
     $s = s + |u_{ij}|y_j$ 
  end
   $y_i = y_i/|u_{ii}|$ 
end
 $\mu = \|y\|_\infty$ 

```

How good are these upper bounds? We know from Problem 8.2 that the ratio $\|M(T)^{-1}\|/\|T^{-1}\|$ can be arbitrarily large, therefore any of the upper bounds can be arbitrarily poor. However, with suitable assumptions on T , more can be said.

It is easy to show that if T is bidiagonal then $|T^{-1}| = M(T)^{-1}$. Since a bidiagonal system can be solved in $O(n)$ flops, it follows that the three condition numbers of interest can each be computed exactly in $O(n)$ flops when T is bidiagonal.

As in the previous section, triangular matrices that result from a pivoting strategy also lead to a special result.

Theorem 8.14. Suppose the upper triangular matrix $U \in \mathbb{R}^{n \times n}$ satisfies

$$|u_{ii}| \geq |u_{ij}| \quad \text{for all } j > i.$$

Then, for the 1-, 2-, and ∞ -norms,

$$\frac{1}{\min_i |u_{ii}|} \leq \|U^{-1}\| \leq \|M(U)^{-1}\| \leq \|W(U)^{-1}\| \leq \|Z(U)^{-1}\| \leq \frac{2^{n-1}}{\min_i |u_{ii}|}. \quad (8.9)$$

Proof. The left-hand inequality is trivial. The right-hand inequality follows from the expression $\|Z(U)^{-1}\|_{1,\infty} = (\beta + 1)^{n-1}/\alpha$ (see Problem 8.5), together with $\|A\|_2 \leq \sqrt{\|A\|_1 \|A\|_\infty}$. $\square$

The inequalities from the second onwards in (8.9) are all equalities for the matrix with $u_{ii} = 1$ and $u_{ij} = -1$ ($j > i$). The question arises of whether equality is possible for the upper triangular matrices arising from QR factorization with column pivoting, which satisfy the inequalities (see Problem 19.5)

$$u_{kk}^2 \geq \sum_{i=k}^j u_{ij}^2, \quad j = k + 1 : n, \quad k = 1 : n. \quad (8.10)$$

That equality is possible is shown by the parametrized matrix of Kahan [687, 1966]

$$U_n(\theta) = \text{diag}(1, s, \dots, s^{n-1}) \begin{bmatrix} 1 & -c & -c & \dots & -c \\ & 1 & -c & \dots & -c \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & -c \\ & & & & 1 \end{bmatrix}, \quad (8.11)$$

where $c = \cos(\theta)$, $s = \sin(\theta)$. It is easily verified that $U_n(\theta)$ satisfies the inequalities (8.10)—as equalities, in fact. From (8.4), $U_n(\theta)^{-1} = (\beta_{ij})$ is given by

$$\beta_{ij} = \begin{cases} s^{1-j}, & i = j, \\ s^{1-j}c(c+1)^{j-i-1}, & i < j. \end{cases}$$

Thus as $\theta \rightarrow 0$, $s^{n-1}U_n(\theta)^{-1} \rightarrow [0, 0, \dots, 0, x]$, where $x = [2^{n-2}, 2^{n-1}, \dots, 1, 1]^T$, and hence, for small θ ,

$$\|U_n(\theta)^{-1}\|_{1,2,\infty} \approx \frac{2^{n-1}}{|u_{nn}|}.$$

It can also be verified that the matrix $\bar{U}_n(\theta) = (\bar{u}_{ij})$ defined by $\bar{u}_{ij} = (-1)^{j-i}|u_{ij}|$ satisfies, for small θ , $\|\bar{U}_n(\theta)\|^{-1} \approx 1/|u_{nn}|$, while $\|M(\bar{U}_n(\theta))\|^{-1} \approx 2^{n-1}/|u_{nn}|$. Hence the upper bounds for $\|U^{-1}\|$ can all be too big by a factor of order 2^{n-1} .

8.4. A Parallel Fan-In Algorithm

Substitution is not the only way to solve a triangular system. In this section we describe a different approach that has been suggested for parallel computation.

Any lower triangular matrix $L \in \mathbb{R}^{n \times n}$ can be factorized $L = L_1 L_2 \dots L_n$, where L_k differs from the identity matrix only in the k th column:

$$L_k = \begin{bmatrix} I_{k-1} & & & & \\ & l_{kk} & & & \\ & l_{k+1,k} & 1 & & \\ & \vdots & & \ddots & \\ & l_{nk} & & & 1 \end{bmatrix}. \quad (8.12)$$

The solution to a linear system $Lx = b$ may therefore be expressed as

$$x = L^{-1}b = M_n M_{n-1} \dots M_1 b, \quad (8.13)$$

where $M_i = L_i^{-1}$. When evaluated in the natural right-to-left order, this formula yields a trivial variation of a column-oriented version of substitution.

The fan-in algorithm evaluates the product (8.13) in $\lceil \log(n+1) \rceil$ steps by the fan-in operation (which is the operation used in pairwise summation: see §4.1). For example, for $n = 7$ the calculation is specified by

$$x = ((M_7 M_6)(M_5 M_4))((M_3 M_2)(M_1 b)),$$

where all the products appearing within a particular size of parenthesis can be evaluated in parallel. In general, the evaluation can be expressed as a binary tree of depth $\lceil \log(n+1) \rceil + 1$, with products M_1b and M_iM_{i-1} ($i = 3, 5, \dots, 2\lfloor (n-1)/2 \rfloor + 1$) at the top level and a single product yielding x at the bottom level. This algorithm was proposed and analysed by Sameh and Brent [1008, 1977], who show that it can be implemented in $\frac{1}{2} \log^2 n + O(\log n)$ time steps on $\frac{1}{68}n^3 + O(n^2)$ processors. The algorithm requires about $n^3/10$ operations and thus is of no interest for serial computation. Some pertinent comments on the practical significance of $\log n$ terms in complexity results are given by Edelman [377, 1993].

To derive an error bound while avoiding complicated notation that obscures the simplicity of the analysis, we take $n = 7$. The result we obtain is easily seen to be valid for all n . We will not be concerned with the precise values of constants, so we write c_n for a constant depending on n . We assume that the inverses $M_i = L_i^{-1}$ are formed exactly, because the errors in forming them affect only the constants. From the error analysis of matrix–vector and matrix–matrix multiplication (§3.5), we find that the computed solution $\hat{x}$ satisfies

$$\hat{x} = ((M_7M_6 + \Delta_{76})(M_5M_4 + \Delta_{54}) + \Delta_{7654})((M_3M_2 + \Delta_{32})(M_1 + \Delta_1)b), \quad (8.14)$$

where

$$\begin{aligned} |\Delta_{i,i-1}| &\leq c_n u |M_i| |M_{i-1}| + O(u^2), \quad i = 5, 7, \\ |\Delta_{7654}| &\leq c_n u (|M_7M_6| |M_5M_4| + |M_7M_6M_5M_4|) + O(u^2), \\ |\Delta_{32}| &\leq c_n u (|M_3| |M_2| + |M_3M_2|) + O(u^2), \\ |\Delta_1| &\leq c_n u |M_1| + O(u^2). \end{aligned}$$

Premultiplying (8.14) on the left by L , we find that the residual $r = L\hat{x} - b$ is a sum of terms of the form

$$L(M_7 \dots M_{j+1})\Delta_{j,\dots,k}M_{k-1} \dots M_1b = L_1 \dots L_j \Delta_{j,\dots,k} L_k \dots L_7 x.$$

All these terms share the same upper bound, which we derive for just one of them. For $j = 5$, $k = 4$ we have

$$\begin{aligned} |L_1 \dots L_5 \Delta_{54} L_4 \dots L_7 x| &\leq c_n u |L_1 \dots L_5| |M_5| |M_4| |L_4 \dots L_7 x| + O(u^2) \\ &= c_n u |L_1 \dots L_5| |L_6 L_7 L^{-1} L_1 \dots L_4| \\ &\quad \times |L_5 L_6 L_7 L^{-1} L_1 L_2 L_3| |L_4 \dots L_7 x| + O(u^2) \\ &\leq c_n u |L| |L^{-1}| |L| |L^{-1}| |L| |x| + O(u^2), \end{aligned}$$

where we have used the property that, for any $L \in \mathbb{R}^{n \times n}$, $|L_1| \dots |L_n| = |L|$. The overall residual bound is therefore of the form

$$|b - L\hat{x}| \leq d_n u |L| |L^{-1}| |L| |L^{-1}| |L| |x| + O(u^2), \quad (8.15)$$

or, on taking norms,

$$\|b - L\hat{x}\|_\infty \leq d_n u \| |L| |L^{-1}| |L| |L^{-1}| |L| |x| \|_\infty + O(u^2). \quad (8.16)$$

By considering the binary tree associated with the fan-in algorithm, and using the fact that the matrices at the i th level of the tree have at most 2^{i-1} nontrivial

columns, it is easy to see that we can take $d_n = an \log n$, where a is a constant of order 1.

It is not hard to find numerical examples where the bound in (8.16) is approximately attained (for $d_n = 1$) and greatly exceeds $u\|L\|_\infty\|\hat{x}\|_\infty$, which is the magnitude required for normwise backward stability. One way to construct such examples is to use direct search (see Chapter 26).

The key fact revealed by (8.16) is that the fan-in algorithm is only conditionally stable. In particular, the algorithm is normwise backward stable if L is well conditioned. A special case in which (8.15) simplifies is when L is an M -matrix and $b \geq 0$: Problem 8.4 shows that in this case $|L^{-1}||L||x| \leq (2n-1)|x|$, so (8.15) yields $|L\hat{x} - b| \leq (2n-1)^2 d_n u |L||x| + O(u^2)$, and we have componentwise backward stability (to first order).

We can obtain from (8.16) the result

$$(L + \Delta L)\hat{x} = b, \quad \|\Delta L\|_\infty \leq \alpha_n u \kappa_\infty(L)^2 \|L\|_\infty + O(u^2), \quad (8.17)$$

which was proved by Sameh and Brent [1008, 1977] (with $\alpha_n = \frac{1}{4}n^2 \log n + O(n \log n)$). However, (8.17) is a much weaker bound than (8.15) and (8.16). In particular, a diagonal scaling $Lx = b \rightarrow D_1 L D_2 \cdot D_2^{-1}x = D_1 b$ (where D_j is diagonal) leaves (8.15) (and, to a lesser extent, (8.16)) essentially unchanged, but can change the bound (8.17) by an arbitrary amount.

A forward error bound can be obtained directly from (8.14). We find that

$$\begin{aligned} |x - \hat{x}| &\leq d'_n u |M_7| |M_6| \dots |M_1| |b| + O(u^2) \\ &= d'_n u M(L)^{-1} |b| + O(u^2), \end{aligned} \quad (8.18)$$

where $M(L)$ is the comparison matrix (a bound of the same form as that in Theorem 8.10 for substitution—see the Notes and References and Problem 8.10), which can be weakened to

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq c_n u \frac{\|M(L)^{-1} |L||x|\|_\infty}{\|x\|_\infty} + O(u^2). \quad (8.19)$$

We also have the bound

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq d_n u \frac{\|(|L^{-1}||L|)^3 |x|\|_\infty}{\|x\|_\infty} + O(u^2), \quad (8.20)$$

which is an immediate consequence of (8.15). Either bound in (8.19) and (8.20) can be arbitrarily larger than the other, for fixed n . An example where (8.20) is the better bound (for large n) is provided by the matrix with $l_{ij} \equiv 1$, for which $|L^{-1}||L|$ has maximum element 2 and $M(L)^{-1}|L|$ has maximum element 2^{n-1} .

8.5. Notes and References

Section 8.2 is based on Higham [586, 1989]. Many of the results presented in §§8.2 and 8.3 have their origin in the work of Wilkinson. Indeed, these sections are effectively a unification and extension of Wilkinson's results in [1229, 1961], [1232, 1963], and [1233, 1965].

Classic references for Theorems 8.3 and 8.5 are Wilkinson [1229, 1961, p. 294], [1232, 1963, pp. 100–102], Forsythe and Moler [431, 1967, §21], and Stewart [1065, 1973, pp. 150, 408–410].

Analogues of Theorem 8.7 and Corollary 8.11 for matrix inversion are proved by Wilkinson in [1229, 1961, pp. 322–323], and Corollary 8.11 itself is proved in [1233, 1965, pp. 250–251]. Lemma 8.8 appears to have first been obtained by Peña [933, 1998], and it was also derived by Malyshev [811, 2000].

A matrix $A \in \mathbb{R}^{n \times n}$ is an M -matrix if $a_{ij} \leq 0$ for all $i \neq j$ and all the eigenvalues of A have nonnegative real part. This is one of many equivalent definitions [106, 1994, Chap. 6]. An M -matrix may be singular. A particularly useful characterization of a nonsingular M -matrix is a nonsingular matrix $A \in \mathbb{R}^{n \times n}$ for which $a_{ij} \leq 0$ for all $i \neq j$ and $A^{-1} \geq 0$. For more information on M -matrices see Berman and Plemmons [106, 1994] and Horn and Johnson [637, 1991, §2.5].

A result of the form of Theorem 8.10 holds for any triangular system solver that does not rely on algebraic cancellation—in particular, for the fan-in algorithm, as already seen in (8.18). See Problem 8.10 for a more precise formulation of this general result.

Stewart [1079, 1997] shows that $\text{cond}(U)$ is small if the upper triangular matrix U is rank-revealing in a certain sense, and he argues that the U produced by pivoted LU, Cholesky, and QR factorizations tend to be rank-revealing. His analysis provides further insight into why triangular systems are typically solved to high accuracy in practice.

The bounds in §8.3 have been investigated by various authors. The unified presentation given here is based on Higham [582, 1987]. Karasalo [710, 1974] derives an $O(n)$ flops algorithm for computing $\|M(T)^{-1}\|_F$. Manteuffel [814, 1981] derives the first two inequalities in Theorem 8.12, and Algorithm 8.13. A different derivation of the equations in Algorithm 8.13 is given by Jennings [674, 1982, §9]. The formulae given in Problem 8.5 are derived directly as upper bounds for $\|T^{-1}\|_{1,\infty}$ by Lemeire [781, 1975].

That $\|B^{-1}\|_\infty$ can be computed in $O(n)$ flops when B is bidiagonal, by exploiting the relation $\|B^{-1}\|_\infty = \|M(B)^{-1}e\|_\infty$, was first pointed out by Higham [579, 1986]. Demmel and Kahan [329, 1990] derive an estimate for the smallest singular value $\sigma_{\min}$ of a bidiagonal matrix B by using the inequality $\bar{\sigma} \leq \sigma_{\min}(B) \leq \sqrt{n}\bar{\sigma}$, where $\bar{\sigma} = \min(\|B^{-1}\|_\infty^{-1}, \|B^{-1}\|_1^{-1})$. They compute $\bar{\sigma}$ in $O(n)$ flops as

$$\bar{\sigma} = \min(\|M(B)^{-1}e\|_\infty^{-1}, \|M(B)^{-T}e\|_\infty^{-1}).$$

Section 8.4 is adapted from Higham [605, 1995], in which error analysis is given for several parallel methods for solving triangular systems.

The fan-in method is topical because the fan-in operation is a special case of the parallel prefix operation and several fundamental computations in linear algebra are amenable to a parallel prefix-based implementation, as discussed by Demmel [315, 1992], [316, 1993]. (For a particularly clear explanation of the parallel prefix operation see the textbook by Buchanan and Turner [171, 1992, §13.2].) The important question of the stability of the parallel prefix implementation of Sturm sequence evaluation for the symmetric tridiagonal eigenproblem is answered by Mathias [822, 1995]. Mathias shows that for positive definite matrices the relative error in a computed minor can be as large as a multiple of λ_n^{-3} , where λ_n is

the smallest eigenvalue of the matrix; the corresponding bound for serial evaluation involves λ_n^{-1} . The analogy with (8.20), where we also see a condition cubing effect, is intriguing.

Higham and Pothen [622, 1994] analyse the stability of the “partitioned inverse method” for parallel solution of sparse triangular systems with many right-hand sides. This method has been studied by several authors in the 1990s; see Alvarado, Pothen, and Schreiber [17, 1993] and the references therein. The idea of the method is to factor a sparse triangular matrix $L \in \mathbb{R}^{n \times n}$ as $L = L_1 L_2 \dots L_n = G_1 G_2 \dots G_m$, where each G_i is a product of consecutive L_j terms and $1 \leq m \leq n$, with m as small as possible subject to the G_i being sparse. Then the solution to $Lx = b$ is evaluated as

$$x = G_m^{-1} G_{m-1}^{-1} \dots G_1^{-1} b,$$

where each G_i^{-1} is formed explicitly and the product is evaluated from right to left. The advantage of this approach is that x can be computed in m serial steps of parallel matrix–vector multiplication.

8.5.1. LAPACK

Computational routine **xTRTRS** solves a triangular system with multiple right-hand sides; **xBTRTS** is an analogue for banded triangular matrices. There is no driver routine for triangular systems.

Problems

*Before you start an exercise session,
make sure you have a glass of water and
a mat or towel nearby.*

— MARIE HELVIN, *Model Tips for a Healthy Future* (1994)

8.1. Show that under the no-guard-digit model (2.6), Lemma 8.2 remains true if (8.1) is changed to

$$b_k \hat{y}(1 + \theta_k) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_{i+2}),$$

and that the corresponding modification of Theorem 8.5 has

$$(T + \Delta T)\hat{x} = b, \quad |\Delta T| \leq \gamma_{n+1}|T|.$$

8.2. Show that for a triangular matrix T the ratio $\|M(T)^{-1}\|/\|T^{-1}\|$ can be arbitrarily large.

8.3. Suppose the unit upper triangular matrix $U \in \mathbb{R}^{n \times n}$ satisfies $|u_{ij}| \leq 1$ for $j > i$. By using Theorem 8.10, show that the computed solution $\hat{x}$ from substitution on $Ux = b$ satisfies

$$|x_i - \hat{x}_i| \leq 2^{n-i}((n^2 + n + 1)u + O(u^2))\|b\|_\infty.$$

Compare with the result of applying Theorem 8.7.

8.4. Let $T \in \mathbb{R}^{n \times n}$ be triangular and suppose $T = M(T)$ and $Tx = b \geq 0$. Show that $|T^{-1}||T||x| \leq (2n-1)|x|$, and hence that $\text{cond}(T, x) \leq 2n-1$. This result shows that a triangular M -matrix system with a nonnegative right-hand side is very well conditioned with respect to componentwise relative perturbations, irrespective of the size of $\kappa(T)$ (and so leads to an alternative proof of Corollary 8.11).

8.5. Show that for a triangular $T \in \mathbb{R}^{n \times n}$, $\|Z(T)^{-1}\| = (\beta+1)^{n-1}/\alpha$ for both the 1- and ∞ -norms (α and β are defined in Theorem 8.12).

8.6. Write detailed algorithms for efficiently computing the quantities

$$\|M(U)^{-1}|z|\|_{\infty}, \quad \|W(U)^{-1}|z|\|_{\infty}.$$

8.7. Bounds from diagonal dominance. (a) Prove the following result (Ahlberg and Nilson [9, 1963], Varah [1188, 1975]): if $A \in \mathbb{R}^{n \times n}$ (not necessarily triangular) satisfies

$$\alpha_i := |a_{ii}| - \sum_{j \neq i} |a_{ij}| > 0, \quad i = 1:n$$

(that is, A is strictly diagonally dominant by rows), then

$$\|A^{-1}\|_{\infty} \leq \frac{1}{\min_i \alpha_i}.$$

(b) Hence show that (Varga [1190, 1976]) if $A \in \mathbb{R}^{n \times n}$ satisfies

$$\beta_i := |a_{ii}|d_i - \sum_{j \neq i} |a_{ij}|d_j > 0, \quad i = 1:n,$$

for some positive diagonal matrix $D = \text{diag}(d_i)$ (that is, AD is strictly diagonally dominant by rows), then

$$\|A^{-1}\|_{\infty} \leq \frac{\|D\|_{\infty}}{\min_i \beta_i}.$$

(c) Use part (b) to provide another derivation of the upper bound $\|M(T)^{-1}e\|_{\infty} \geq \|T^{-1}\|_{\infty}$.

8.8. (a) Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. For a given i and j , determine, if possible, α_{ij} such that $A + \alpha_{ij}e_i e_j^T$ is singular. Where is the “best” place to perturb A to make it singular?

(b) Let $T = U(1)$ in (8.3), so that, for example,

$$T_4 = \begin{bmatrix} 1 & -1 & -1 & -1 \\ & 1 & -1 & -1 \\ & & 1 & -1 \\ & & & 1 \end{bmatrix}.$$

Show that T_n is made singular by subtracting 2^{2-n} from a certain element of T_n .

8.9. (Zha [1279, 1993]) Show that if c and s are nonnegative (with $c^2 + s^2 = 1$) then the Kahan matrix $U_n(\theta)$ in (8.11) has $s^{n-2}\sqrt{1+c}$ as its second smallest singular value. (That there should be such an explicit formula is surprising; none is known for the smallest singular value.)

8.10. Consider a method for solving triangular systems $Tx = b$ that computes $x_i = f_i(T, b)$ where, for all i , f_i is a multivariate rational function in which the only divisions are by diagonal elements of L and such that when $T = M(T)$ and $b \geq 0$ there are no subtractions in the evaluation of f_i . Show that a bound holds of the form in Theorem 8.10, namely, for $T \in \mathbb{R}^{n \times n}$,

$$|x - \hat{x}| \leq (c_n u + O(u^2)) M(T)^{-1} |b|. \quad (8.21)$$

Give an example of a triangular system solver for which (8.21) is not satisfied.

Chapter 9

LU Factorization and Linear Equations

*It appears that Gauss and Doolittle applied the method only to symmetric equations.
More recent authors, for example, Aitken, Banachiewicz, Dwyer, and Crout ...
have emphasized the use of the method, or variations of it,
in connection with non-symmetric problems ...
Banachiewicz ... saw the point ...
that the basic problem is really one of matrix factorization,
or "decomposition" as he called it.*

— PAUL S. DWYER, *Linear Computations* (1951)

*Intolerable pivot-growth [with partial pivoting] is a phenomenon that happens
only to numerical analysts who are looking for that phenomenon.*

— WILLIAM M. KAHAN, *Numerical Linear Algebra* (1966)

*By 1949 the major components of the
Pilot ACE were complete and undergoing trials ...
During 1951 a programme for solving simultaneous
linear algebraic equations was used for the first time.
26th June, 1951 was a landmark in the history of the machine,
for on that day it first rivalled alternative computing methods
by yielding by 3 p.m. the solution to
a set of 17 equations submitted the same morning.*

— MICHAEL WOODGER, *The History and Present Use of
Digital Computers at the National Physical Laboratory* (1958).

*The closer one looks,
the more subtle and remarkable Gaussian elimination appears.*
— LLOYD N. TREFETHEN, *Three Mysteries of Gaussian Elimination* (1985)

9.1. Gaussian Elimination and Pivoting Strategies

We begin by giving a traditional description of Gaussian elimination (GE) for solving a linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular.

The strategy of GE is to reduce a problem we can't solve (a full linear system) to one that we can (a triangular system), using elementary row operations. There are $n - 1$ stages, beginning with $A^{(1)} := A$, $b^{(1)} := b$, and finishing with the upper triangular system $A^{(n)}x = b^{(n)}$.

At the k th stage we have converted the original system to $A^{(k)}x = b^{(k)}$, where

$$A^{(k)} = \begin{bmatrix} A_{11}^{(k)} & A_{12}^{(k)} \\ 0 & A_{22}^{(k)} \end{bmatrix},$$

with $A_{11}^{(k)} \in \mathbb{R}^{(k-1) \times (k-1)}$ upper triangular. The purpose of the k th stage of the elimination is to zero the elements below the diagonal in the k th column of $A^{(k)}$. This is accomplished by the operations

$$\begin{aligned} a_{ij}^{(k+1)} &= a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}, & i &= k+1:n, \quad j = k+1:n, \\ b_i^{(k+1)} &= b_i^{(k)} - m_{ik}b_k^{(k)}, & i &= k+1:n, \end{aligned}$$

where the multipliers $m_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$, $i = k+1:n$. At the end of the $(n-1)$ st stage we have the upper triangular system $A^{(n)}x = b^{(n)}$, which is solved by back substitution. For an $n \times n$ matrix, GE requires $2n^3/3$ flops.

There are two problems with the method as described. First, there is a breakdown with division by zero if $a_{kk}^{(k)} = 0$. Second, if we are working in finite precision and some multiplier m_{ik} is large, then there is a possible loss of significance: in the subtraction $a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}$, low-order digits of $a_{ij}^{(k)}$ could be lost. Losing these digits could correspond to making a relatively large change to the original matrix A . The simplest example of this phenomenon is for the matrix $\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix}$; here, $a_{22}^{(2)} = 1 - 1/\epsilon$, and $fl(a_{22}^{(2)}) = -1/\epsilon$ if $\epsilon < u$, which would be the exact answer if we changed a_{22} from 1 to 0.

These observations motivate the strategy of *partial pivoting*. At the start of the k th stage, the k th and r th rows are interchanged, where

$$|a_{rk}^{(k)}| := \max_{k \leq i \leq n} |a_{ik}^{(k)}|.$$

Partial pivoting ensures that the multipliers are nicely bounded:

$$|m_{ik}| \leq 1, \quad i = k+1:n.$$

A more expensive pivoting strategy, which interchanges both rows and columns, is *complete pivoting*.

At the start of the k th stage, rows k and r and columns k and s are interchanged, where

$$|a_{rs}^{(k)}| := \max_{k \leq i, j \leq n} |a_{ij}^{(k)}|.$$

Note that this requires $O(n^3)$ comparisons in total, compared with $O(n^2)$ for partial pivoting. Because of the searching overhead, and because partial pivoting

1	10	1	2	4	5
0	5	2	7	8	2
2	0	3	1	9	4
3	2	4	2	1	0
1	4	5	6	3	1
1	0	3	4	0	12

Figure 9.1. Illustration of how rook pivoting searches for the first pivot for a particular 6×6 matrix (with the positive integer entries shown). Each dot denotes a putative pivot that is tested to see if it is the largest in magnitude in both its row and its column.

works so well, complete pivoting is used only in special situations (see the Notes and References).

Also of interest, but much less well known, is the *rook pivoting* strategy, which chooses at each stage a pivot intermediate in size between the pivots that would be chosen by partial pivoting and complete pivoting. At the start of the k th stage, rows k and r and columns k and s are interchanged, where

$$|a_{rs}^{(k)}| = \max_{k \leq i \leq n} |a_{is}^{(k)}| = \max_{k \leq j \leq n} |a_{rj}^{(k)}|;$$

in other words, a pivot is chosen that is the largest in magnitude in both its column (as for partial pivoting) *and* its row. The pivot search can be coded as follows:

```

s0 = k
for p = 1, 2, ...
    rp = row index of first element of max. modulus among {ai,sp-1}i=kn
    if p > 1 and |arp,sp-1| = |arp-1,sp-1|
        take arp-1,sp-1 as pivot, quit
    end
    sp = col. index of first element of max. modulus among {arp,j}j=kn
    if |arp,sp| = |arp,sp-1| take arp,sp-1 as pivot, quit, end
end

```

The rook pivoting strategy takes its name from the fact that the search for a pivot corresponds to the moves of a rook in chess; see Figure 9.1. Note that in the pivot search, elements previously considered can be skipped; in the following discussion we will assume that this refinement is incorporated, though it may not be worthwhile in practice.

Clearly, the search for a pivot in rook pivoting involves at least twice as many comparisons as for partial pivoting, and if the whole submatrix has to be searched then the number of comparisons is the same as for complete pivoting. Foster [435, 1997, Thm. 5] shows that if the elements $\{a_{ij}^{(k)}\}_{i,j=k}^n$ are independent identically

distributed random variables from any continuous probability distribution then the expected number of comparisons in the pivot search for rook pivoting at stage k is at most $(n - k)e$ (where $e = \exp(1)$). If this statistical assumption is satisfied for each k then the overall number of comparisons is bounded by $(n - 1)ne/2$, which is of the same order as for partial pivoting ($(n - 1)n/2$ comparisons) and an order of magnitude less than for complete pivoting ($n^3/3 + O(n/2)$ comparisons). Numerical experiments show that the cost of rook pivoting is indeed usually a small multiple of the cost of partial pivoting and significantly less than the cost of complete pivoting (see Figure 9.3). However, rook pivoting *can* require $O(n^3)$ comparisons, as illustrated by any matrix of the form, illustrated for $n = 4$,

$$\begin{bmatrix} \theta_1 & \theta_2 & & \\ & \theta_3 & \theta_4 & \\ & & \theta_5 & \theta_6 \\ & & & \theta_7 \end{bmatrix}, \quad |\theta_1| < |\theta_2| < \cdots < |\theta_7|,$$

for which $n^3/4 + O(n^2)$ comparisons are required.

9.2. LU Factorization

Much insight into GE is obtained by expressing it in matrix notation. We can write

$$A^{(k+1)} = M_k A^{(k)} := \begin{bmatrix} I_{k-1} & & & & \\ & 1 & & & \\ & -m_{k+1,k} & 1 & & \\ & -m_{k+2,k} & & \ddots & \\ & \vdots & & & \ddots \\ & -m_{n,k} & & & & 1 \end{bmatrix} A^{(k)}.$$

The matrix M_k can be expressed compactly as $M_k = I - m_k e_k^T$, where e_k is the k th unit vector and $e_i^T m_k = 0$ for $i \leq k$. To invert M_k , just flip the signs of the multipliers: $M_k^{-1} = I + m_k e_k^T$. Overall,

$$M_{n-1} M_{n-2} \cdots M_1 A = A^{(n)} =: U,$$

and so

$$\begin{aligned} A &= M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1} U \\ &= (I + m_1 e_1^T)(I + m_2 e_2^T) \cdots (I + m_{n-1} e_{n-1}^T) U \\ &= \left(I + \sum_{i=1}^{n-1} m_i e_i^T \right) U \\ &= \begin{bmatrix} 1 & & & & \\ m_{21} & 1 & & & \\ \vdots & m_{32} & \ddots & & \\ \vdots & \vdots & & \ddots & \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & 1 \end{bmatrix} U =: LU. \end{aligned}$$

The conclusion is that GE computes an LU factorization of A : $A = LU$, where L is unit lower triangular and U is upper triangular.

We introduce the shorthand notation $A_k := A(1:k, 1:k)$.

Theorem 9.1. *There exists a unique LU factorization of $A \in \mathbb{R}^{n \times n}$ if and only if A_k is nonsingular for $k = 1:n-1$. If A_k is singular for some $1 \leq k \leq n-1$ then the factorization may exist, but if so it is not unique.*

Proof. Suppose A_k is nonsingular for $k = 1:n-1$. The existence of an LU factorization can be proved by examining the steps of GE, but a more elegant proof, which also gives uniqueness, can be obtained by an inductive bordering construction. Suppose A_{k-1} has the unique LU factorization $A_{k-1} = L_{k-1}U_{k-1}$ (this supposition clearly holds for $k-1=1$). We look for a factorization

$$A_k = \begin{bmatrix} A_{k-1} & b \\ c^T & a_{kk} \end{bmatrix} = \begin{bmatrix} L_{k-1} & 0 \\ l^T & 1 \end{bmatrix} \begin{bmatrix} U_{k-1} & u \\ 0 & u_{kk} \end{bmatrix} =: L_k U_k.$$

The equations to be satisfied are $L_{k-1}u = b$, $U_{k-1}^T l = c$, and $a_{kk} = l^T u + u_{kk}$. The matrices L_{k-1} and U_{k-1} are nonsingular, since $0 \neq \det(A_{k-1}) = \det(L_{k-1})\det(U_{k-1})$, so the equations have a unique solution, completing the induction.

We prove the converse, under the assumption that A is nonsingular; for the case A singular see Problem 9.1. Suppose an LU factorization exists. Then $A_k = L_k U_k$ for $k = 1:n$, which gives

$$\det(A_k) = \det(U_k) = u_{11} \dots u_{kk}. \quad (9.1)$$

Setting $k = n$ we find that $0 \neq \det(A) = u_{11} \dots u_{nn}$, and hence $\det(A_k) = u_{11} \dots u_{kk} \neq 0$, $k = 1:n-1$.

Examples that illustrate the last part of the theorem are $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$, which holds for any l , and $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$, which does not have an LU factorization. $\square$

Visually, the condition of Theorem 9.1 is (for $n = 5$) that the indicated submatrices must be nonsingular:

$$\begin{bmatrix} \boxed{\begin{matrix} \times & \times \\ \times & \times \end{matrix}} & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}.$$

From (9.1) follows the expression $u_{kk} = \det(A_k)/\det(A_{k-1})$. In fact, all the elements of L and U can be expressed by determinantal formulae (see, e.g., Gantmacher [453, 1959, p. 35] or Householder [644, 1964, p. 11]):

$$l_{ij} = \frac{\det(A([1:j-1, i], 1:j))}{\det(A_j)}, \quad i \geq j, \quad (9.2a)$$

$$u_{ij} = \frac{\det(A(1:i, [1:i-1, j]))}{\det(A_{i-1})}, \quad i \leq j. \quad (9.2b)$$

The effect of partial pivoting is easily seen by considering the case $n = 4$. We have

$$\begin{aligned} U &= M_3 P_3 M_2 P_2 M_1 P_1 A, \quad \text{where } P_k \text{ swaps rows } k, r \ (r \geq k), \\ &= M_3 \cdot \underbrace{P_3 M_2 P_3} \cdot \underbrace{P_3 P_2 M_1 P_2 P_3} \cdot \underbrace{P_3 P_2 P_1 A} \\ &=: M'_3 M'_2 M'_1 P A, \end{aligned}$$

where, for example, $M'_1 = P_3 P_2 (I - m_1 e_1^T) P_2 P_3 = I - (P_3 P_2 m_1) e_1^T$. For $k = 1, 2, 3$, M'_k is the same as M_k except the multipliers are interchanged. Hence, for $n = 4$, GE with partial pivoting (GEPP) applied to A is equivalent to GE without pivoting applied to the row-permuted matrix PA . This conclusion is true for any n : GEPP computes a factorization $PA = LU$. Similarly, GE with rook pivoting or complete pivoting computes a factorization $PAQ = LU$, where P and Q are permutation matrices.

Exploitation of the LU factorization streamlines both the error analysis and the practical solution of linear systems. Solution of $Ax = b$ breaks into a factorization phase, $PA = LU$ for partial pivoting ($O(n^3)$ flops), and a substitution phase, where the triangular systems $Ly = Pb$, $Ux = y$ are solved ($O(n^2)$ flops). If more than one system is to be solved with the same coefficient matrix but different right-hand sides, the factorization can be reused, with a consequent saving in work.

Computing an LU factorization $A = LU$ is equivalent to solving the equations

$$a_{ij} = \sum_{r=1}^{\min(i,j)} l_{ir} u_{rj}.$$

If these nonlinear equations are examined in the right order, they are easily solved. For added generality let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) and consider an LU factorization with $L \in \mathbb{R}^{m \times n}$ and $U \in \mathbb{R}^{n \times n}$ (L is lower trapezoidal: $l_{ij} = 0$ for $i < j$). Suppose we know the first $k-1$ columns of L and the first $k-1$ rows of U . Setting $l_{kk} = 1$,

$$a_{kj} = l_{k1}u_{1j} + \cdots + l_{k,k-1}u_{k-1,j} + \boxed{u_{kj}}, \quad j = k:n, \quad (9.3)$$

$$a_{ik} = l_{i1}u_{1k} + \cdots + \boxed{l_{ik}}u_{kk}, \quad i = k+1:m. \quad (9.4)$$

We can solve for the boxed elements in the k th row of U and then the k th column of L . This process is called Doolittle's method.

Algorithm 9.2 (Doolittle's method). This algorithm computes an LU factorization $A = LU \in \mathbb{R}^{m \times n}$, where $m \geq n$ (assuming the factorization exists), by Doolittle's method.

```

for  $k = 1:n$ 
  for  $j = k:n$ 
    (*)  $u_{kj} = a_{kj} - \sum_{i=1}^{k-1} l_{ki}u_{ij}$ 
  end
  for  $i = k+1:m$ 
    (**)  $l_{ik} = (a_{ik} - \sum_{j=1}^{k-1} l_{ij}u_{jk})/u_{kk}$ 
  end
end
```

Cost: $n^2(m - n/3)$ flops.

Doolittle's method is mathematically equivalent to GE without pivoting, for we have, in (9.3),

$$a_{kj} - l_{k1}u_{1j} - \cdots - l_{ks}u_{sj} \equiv a_{kj}^{(s+1)} \quad (j > k), \quad (9.5)$$

and similarly for (9.4). Had we chosen the normalization $u_{ii} \equiv 1$, we would have obtained the Crout method. The Crout and Doolittle methods are well suited to calculations by hand or with a desk calculator, because they obviate the need to store the intermediate quantities $a_{ij}^{(k)}$. They are also attractive when we can accumulate inner products in extended precision.

It is straightforward to incorporate partial pivoting into Doolittle's method (see, e.g., Stewart [1065, 1973, p. 138]). However, rook pivoting and complete pivoting cannot be incorporated without changing the method.

9.3. Error Analysis

The error analysis of GE is a combination of the error analyses of inner products and substitution. When this fact is realized, the analysis becomes straightforward. The key observation leading to this viewpoint is that all mathematically equivalent variants of GE satisfy a common error bound. To see why, first note the connection between standard GE, as we have described it, and the Doolittle method, as shown in (9.5). Whether the inner product in (9.5) is calculated as one operation, or whether its terms are calculated many operations apart, precisely the same rounding errors are sustained (assuming that extended precision accumulation of inner products is not used); all that changes is the moment when those rounding errors are committed. If we allow inner products to be reordered, so that, for example, the summation (*) in Algorithm 9.2 is calculated with the index i decreasing from $k - 1$ to 1, instead of increasing from 1 to $k - 1$, then the actual rounding errors are different, but a common bound holds for all orderings.

It suffices, then, to analyse the Doolittle method. It also suffices to analyse the method without pivoting, because GE with partial, rook, or complete pivoting is equivalent to GE without pivoting applied to a permuted matrix.

The assignments (*) and (**) in Algorithm 9.2 are of the form $y = (c - \sum_{i=1}^{k-1} a_i b_i)/b_k$, which is analysed in Lemma 8.4. Applying the lemma, we deduce that, no matter what the ordering of the inner products, the computed matrices $\hat{L}$ and $\hat{U}$ satisfy (with $\hat{l}_{kk} := 1$)

$$\begin{aligned} \left| a_{kj} - \sum_{i=1}^{k-1} \hat{l}_{ki} \hat{u}_{ij} - \hat{u}_{kj} \right| &\leq \gamma_k \sum_{i=1}^k |\hat{l}_{ki}| |\hat{u}_{ij}|, \quad j \geq k, \\ \left| a_{ik} - \sum_{j=1}^k \hat{l}_{ij} \hat{u}_{jk} \right| &\leq \gamma_k \sum_{j=1}^k |\hat{l}_{ij}| |\hat{u}_{jk}|, \quad i > k. \end{aligned}$$

These inequalities constitute a backward error result for LU factorization.

Theorem 9.3. *If GE applied to $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) runs to completion then the computed LU factors $\hat{L} \in \mathbb{R}^{m \times n}$ and $\hat{U} \in \mathbb{R}^{n \times n}$ satisfy*

$$\hat{L}\hat{U} = A + \Delta A, \quad |\Delta A| \leq \gamma_n |\hat{L}||\hat{U}|. \quad \square \quad (9.6)$$

With only a little more effort, a backward error result can be obtained for the solution of $Ax = b$.

Theorem 9.4. *Let $A \in \mathbb{R}^{n \times n}$ and suppose GE produces computed LU factors $\hat{L}$, $\hat{U}$, and a computed solution $\hat{x}$ to $Ax = b$. Then*

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq \gamma_{3n} |\hat{L}||\hat{U}|. \quad (9.7)$$

Proof. From Theorem 9.3, $\hat{L}\hat{U} = A + \Delta A_1$, $|\Delta A_1| \leq \gamma_n |\hat{L}||\hat{U}|$. By Theorem 8.5, substitution produces $\hat{y}$ and $\hat{x}$ satisfying

$$\begin{aligned} (\hat{L} + \Delta L)\hat{y} &= b, & |\Delta L| &\leq \gamma_n |\hat{L}|, \\ (\hat{U} + \Delta U)\hat{x} &= \hat{y}, & |\Delta U| &\leq \gamma_n |\hat{U}|. \end{aligned}$$

Thus

$$\begin{aligned} b &= (\hat{L} + \Delta L)(\hat{U} + \Delta U)\hat{x} \\ &= (A + \Delta A_1 + \hat{L}\Delta U + \Delta L\hat{U} + \Delta L\Delta U)\hat{x} \\ &= (A + \Delta A)\hat{x}, \end{aligned}$$

where

$$|\Delta A| \leq (3\gamma_n + \gamma_n^2) |\hat{L}||\hat{U}| \leq \gamma_{3n} |\hat{L}||\hat{U}|,$$

using Lemma 3.3. $\square$

How do we interpret Theorem 9.4? Ideally, we would like $|\Delta A| \leq u|A|$, which corresponds to the uncertainty introduced by rounding the elements of A , but because each element of A undergoes up to n arithmetic operations we cannot expect better than a bound $|\Delta A| \leq c_n u|A|$, where c_n is a constant of order n . Such a bound holds if $\hat{L}$ and $\hat{U}$ satisfy $|\hat{L}||\hat{U}| = |\hat{L}\hat{U}|$, which certainly holds if $\hat{L}$ and $\hat{U}$ are nonnegative, because then (9.6) gives

$$\begin{aligned} |\hat{L}||\hat{U}| &= |\hat{L}\hat{U}| = |A + \Delta A| \leq |A| + \gamma_n |\hat{L}||\hat{U}| \\ \Rightarrow |\hat{L}||\hat{U}| &\leq \frac{1}{1 - \gamma_n} |A|. \end{aligned} \quad (9.8)$$

Substituting into (9.7), we obtain

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq \frac{\gamma_{3n}}{1 - \gamma_n} |A| \quad (\hat{L}, \hat{U} \geq 0). \quad (9.9)$$

This result says that $\hat{x}$ has a small componentwise relative backward error.

One class of matrices that has nonnegative LU factors is defined as follows. $A \in \mathbb{R}^{n \times n}$ is *totally positive* (nonnegative) if the determinant of every square submatrix is positive (nonnegative). In particular, this definition requires that

a_{ij} and $\det(A)$ be positive or nonnegative. Some examples of totally nonnegative matrices are given in Chapter 28. If A is totally nonnegative then it has an LU factorization $A = LU$ in which L and U are totally nonnegative, so that $L \geq 0$ and $U \geq 0$ (see Problem 9.6); moreover, the computed factors $\hat{L}$ and $\hat{U}$ from GE are nonnegative for sufficiently small values of the unit roundoff u [300, 1977]. Inverses of totally nonnegative matrices also have the property that $|A| = |L||U|$ (see Problem 9.8). Note that the property of a matrix or its inverse being totally nonnegative is generally destroyed under row permutations. Hence for totally nonnegative matrices and their inverses it is best to use Gaussian elimination *without* pivoting.

One important fact that follows from (9.6) and (9.7) is that the stability of GE is determined not by the size of the multipliers $\hat{l}_{ij}$ but by the size of the matrix $|\hat{L}||\hat{U}|$. This matrix can be small when the multipliers are large, and large when the multipliers are of order 1 (as we will see in the next section).

To understand the stability of GE further we turn to norms. For GE without pivoting, the ratio $\|\hat{L}\|\hat{U}\|/\|A\|$ can be arbitrarily large. For example, for the matrix $\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix}$ the ratio is of order ϵ^{-1} . Assume then that partial pivoting is used. Then $|l_{ij}| \leq 1$ for all $i \geq j$, since the l_{ij} are the multipliers. And it is easy to show by induction that $|u_{ij}| \leq 2^{i-1} \max_{k \leq i} |a_{kj}|$. Hence, for partial pivoting, L is small and U is bounded relative to A .

Traditionally, backward error analysis for GE is expressed in terms of the *growth factor*

$$\rho_n = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

which involves all the elements $a_{ij}^{(k)}$ ($k = 1:n$) that occur during the elimination. Using the bound $|u_{ij}| = |a_{ij}^{(i)}| \leq \rho_n \max_{i,j} |a_{ij}|$ we obtain the following classic theorem.

Theorem 9.5 (Wilkinson). *Let $A \in \mathbb{R}^{n \times n}$ and suppose GE with partial pivoting produces a computed solution $\hat{x}$ to $Ax = b$. Then*

$$(A + \Delta A)\hat{x} = b, \quad \|\Delta A\|_\infty \leq n^2 \gamma_{3n} \rho_n \|A\|_\infty. \quad \square \quad (9.10)$$

We hasten to admit to using an illicit manoeuvre in the derivation of this theorem: we have used bounds for $\hat{L}$ and $\hat{U}$ that strictly are valid only for the exact L and U . We could instead have defined the growth factor in terms of the computed $\hat{a}_{ij}^{(k)}$, but then any bounds for the growth factor would involve the unit roundoff (similarly, we can only guarantee that $|\hat{l}_{ij}| \leq 1 + u$). Our breach of correctness is harmless for the purposes to which we will put the theorem.

The assumption in Theorem 9.5 that partial pivoting is used is not necessary: $\| |L| |U| \|_\infty$ can also be bounded in terms of the growth factor for GE without pivoting, as the next result shows.

Lemma 9.6. *If $A = LU \in \mathbb{R}^{n \times n}$ is an LU factorization produced by GE without pivoting then*

$$\| |L| |U| \|_\infty \leq (1 + 2(n^2 - n)\rho_n) \|A\|_\infty.$$

Table 9.1. *Classes of matrices for which $\rho_n = O(1)$ for GE without pivoting.*

Matrix property	Reference
Totally nonnegative	§9.3
Row or column diagonally dominant	§9.5
Symmetric positive definite	§10.1.1
Complex symmetric with positive definite Hermitian and skew-Hermitian parts	§10.4

Proof. Let l_j denote the j th column of L and u_i^T denote the i th row of U . Then

$$\tilde{A}^{(k+1)} = \tilde{A}^{(k)} - l_k u_k^T, \quad k = 1:n-1,$$

where $\tilde{A}^{(k)}$ denotes $A^{(k)}$ with the elements in rows $1:k-1$ set to zero. Thus

$$|l_k| |u_k^T| = |l_k u_k^T| \leq |\tilde{A}^{(k)}| + |\tilde{A}^{(k+1)}|$$

and hence

$$|L||U| = \sum_{k=1}^n |l_k| |u_k^T| \leq |A| + 2 \sum_{k=2}^n |\tilde{A}^{(k)}|.$$

Taking norms and using $|a_{ij}^{(k)}| \leq \rho_n \max_{i,j} |a_{ij}|$ gives

$$\| |L||U| \|_\infty \leq \|A\|_\infty + 2(n-1)n\rho_n \|A\|_\infty = (1+2(n-1)n\rho_n) \|A\|_\infty. \quad \square$$

It is also possible to bound the growth factor above in terms of $\| |L||U| \|_\infty$; see Problem 9.9.

Since the normwise backward stability of GE with or without pivoting is governed by the growth factor, we next turn our attention to analysis of the growth factor. As a prelude, Table 9.1 summarizes classes of matrices for which the growth factor for GE without pivoting is small, and for which it is therefore safe not to pivot.

9.4. The Growth Factor

It is easy to show that $\rho_n \leq 2^{n-1}$ for partial pivoting. Wilkinson notes that this upper bound is achieved for matrices of the form illustrated for $n = 4$ by

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & 1 \end{bmatrix}.$$

For these matrices, no interchanges are required by partial pivoting, and there is exponential growth of elements in the last column of the reduced matrices. In fact, this is just one of a nontrivial class of matrices for which partial pivoting achieves maximal growth. When necessary in the rest of this chapter, we denote the growth factor for partial pivoting by ρ_n^p and that for complete pivoting by ρ_n^c .

Theorem 9.7 (Higham and Higham). *All real $n \times n$ matrices A for which $\rho_n^p(A) = 2^{n-1}$ are of the form*

$$A = DM \begin{bmatrix} T & \vdots & \alpha d \\ 0 & \vdots & \end{bmatrix},$$

where $D = \text{diag}(\pm 1)$, M is unit lower triangular with $m_{ij} = -1$ for $i > j$, T is an arbitrary nonsingular upper triangular matrix of order $n-1$, $d = (1, 2, 4, \dots, 2^{n-1})^T$, and α is a scalar such that $\alpha := |a_{1n}| = \max_{i,j} |a_{ij}|$.

Proof. GEPP applied to a matrix A gives a factorization $B := PA = LU$, where P is a permutation matrix. It is easy to show that $|u_{ij}| \leq 2^{i-1} \max_{r \leq i} |b_{rj}|$, with equality for $i = s$ only if there is equality for $i = 1:s-1$. Thus $\rho_n = 2^{n-1}$ implies that the last column of U has the form αDd , and also that $|b_{1n}| = \max_{i,j} |b_{ij}|$. By considering the final column of B , and imposing the requirement that $|l_{ij}| \leq 1$, it is easy to show that the unit lower triangular matrix L must have the form $L = DMD$. It follows that at each stage of the reduction every multiplier is ± 1 ; hence no interchanges are performed, that is, $P = I$. The only requirement on T is that it be nonsingular, for if $t_{ii} = 0$ then the i th elimination stage would be skipped because of a zero pivot column and no growth would be produced on that stage. $\square$

Note that by varying the elements m_{ij} ($i > j$) and the vector d in Theorem 9.7 we can construct matrices for which ρ_n^p achieves any desired value between 1 and 2^{n-1} .

Despite the existence of matrices for which ρ_n is large with partial pivoting, the growth factor is almost invariably small in practice. For example, Wilkinson says "It is our experience that any substantial increase in size of elements of successive A_r is extremely uncommon even with partial pivoting ... No example which has arisen naturally has in my experience given an increase by a factor as large as 16" [1233, 1965, pp. 213–214].

Matrices that yield large growth factors and that arise, or could arise, in practical applications, are rare. Wright [1261, 1993] has found a class of two-point boundary value problems that, when solved by the multiple shooting method, yield a linear system for which partial pivoting suffers exponential growth. The matrix is block lower bidiagonal, except for a nonzero block in the top right-hand corner. Furthermore, Foster [434, 1994] shows that a quadrature method for solving a practically occurring Volterra integral equation gives rise to linear systems for which partial pivoting again gives large growth factors.

There exist some well-known matrices that give unusually large, but not exponential, growth. They can be found using the following theorem, which is applicable whatever the strategy for interchanging rows and columns in GE.

Theorem 9.8 (Higham and Higham). *Let $A \in \mathbb{C}^{n \times n}$ be nonsingular and set $\alpha = \max_{i,j} |a_{ij}|$, $\beta = \max_{i,j} |(A^{-1})_{ij}|$, and $\theta = (\alpha\beta)^{-1}$. Then $\theta \leq n$, and for any permutation matrices P and Q such that PAQ has an LU factorization, the growth factor ρ_n for GE without pivoting on PAQ satisfies $\rho_n \geq \theta$.*

Proof. The inequality $\theta \leq n$ follows from $\sum_{j=1}^n a_{ij}(A^{-1})_{ji} = 1$. Consider an LU factorization $PAQ = LU$ computed by GE. We have

$$\begin{aligned} |u_{nn}^{-1}| &= |e_n^T U^{-1} e_n| = |e_n^T U^{-1} L^{-1} e_n| = |e_n^T Q^T A^{-1} P^T e_n| \\ &= |(A^{-1})_{ij}| \quad \text{for some } i, j \\ &\leq \beta. \end{aligned} \quad (9.11)$$

Hence $\max_{i,j,k} |a_{ij}^{(k)}| \geq |u_{nn}| \geq \beta^{-1}$, and the result follows. $\square$

Note that $\theta^{-1} = \alpha\beta$ satisfies $\kappa_\infty(A)^{-1} \leq \theta^{-1} \leq n^2 \kappa_\infty(A)^{-1}$. Clearly, A has to be very well conditioned for the theorem to provide a lower bound θ near the maximum of n .

We apply the theorem to three noncontrived matrices that appear in practical applications.

(1) The matrix

$$S_n = \sqrt{\frac{2}{n+1}} \left(\sin\left(\frac{ij\pi}{n+1}\right) \right)_{i,j=1}^n \quad (9.12)$$

is the symmetric, orthogonal eigenvector matrix for the second difference matrix (the tridiagonal matrix with typical row $(-1, 2, -1)$ —see §28.5); it arises, for example, in the analysis of time series [24, 1971, §6.5]. Theorem 9.8 gives $\rho_n(S_n) \geq (n+1)/2$.

(2) A Hadamard matrix H_n is an $n \times n$ matrix with elements $h_{ij} = \pm 1$ and for which the rows of H_n are mutually orthogonal. Hadamard matrices exist only for certain n ; a necessary condition for their existence if $n > 2$ is that n is a multiple of 4. For more about Hadamard matrices see Hall [538, 1967, Chap. 14], Hedayat, Sloane, and Stufken [557, 1999, Chap. 7], Wallis [1205, 1993], and Wallis, Street, and Wallis [1206, 1972]. We have $H_n H_n^T = nI$, and so $H_n^{-1} = n^{-1} H_n^T$. Theorem 9.8 gives $\rho_n \geq n$.

(3) The next matrix is a complex Vandermonde matrix based on the roots of unity, which occurs in the evaluation of Fourier transforms (see §24.1):

$$V_n = \left(\exp(-2\pi i(r-1)(s-1)/n) \right)_{r,s=1}^n. \quad (9.13)$$

Since $V_n^{-1} = n^{-1} V_n^H$, Theorem 9.8 gives $\rho_n(V_n) \geq n$.

Note that each of these matrices is orthogonal or unitary (to within a row scaling in the case of the Hadamard matrix), so it is not necessary to apply GE to them! This may explain why growth factors of order n for these matrices have not been reported in the literature as occurring in practice.

To summarize, although there are practically occurring matrices for which partial pivoting yields a moderately large, or even exponentially large, growth factor, the growth factor is almost invariably found to be small. Explaining this fact remains one of the major unsolved problems in numerical analysis. The best attempt to date is by Trefethen and Schreiber [1157, 1990], who develop a statistical model of the average growth factor for partial pivoting and complete pivoting. Their model supports their empirical findings that for various distributions of random matrices the average growth factor (normalized by the standard deviation of the

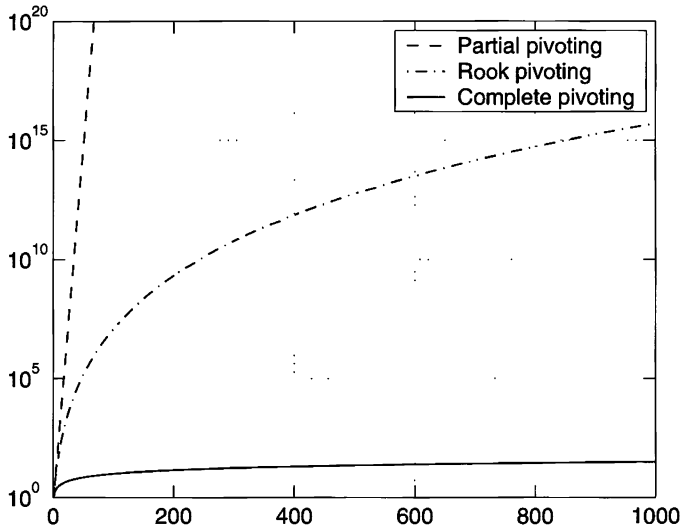


Figure 9.2. Upper bounds for growth factors ρ_n for partial pivoting, rook pivoting, and complete pivoting.

initial matrix elements) is close to $n^{2/3}$ for partial pivoting and $n^{1/2}$ for complete pivoting (for $n \leq 1024$). Extensive experiments by Edelman suggest that for random matrices from the normal $N(0, 1)$ distribution the unnormalized growth factor for partial pivoting grows like $n^{1/2}$ [382, 1995]. Trefethen and Bau [1156, 1997, Lecture 22] outline a possible strategy for explaining the growth factor behaviour: their idea is to show that for a matrix to produce a large growth factor with partial pivoting its column spaces must possess a certain skewness property and then to prove that this property holds with low probability for random matrices.

We turn now to complete pivoting. Wilkinson [1229, 1961, pp. 282–285] showed that

$$\rho_n^c \leq n^{1/2} (2 \cdot 3^{1/2} \dots n^{1/(n-1)})^{1/2} \sim c n^{1/2} n^{\frac{1}{4} \log n}, \quad (9.14)$$

and that this bound is not attainable. As Figure 9.2 illustrates, the bound is a much more slowly growing function than 2^{n-1} , but it can still be quite large (e.g., it is 3570 for $n = 100$). As for partial pivoting, in practice the growth factor is usually small. Wilkinson stated that “no matrix has been encountered in practice for which p_1/p_n was as large as 8” [1229, 1961, p. 285] and that “no matrix has yet been discovered for which $f(r) > r$ ” [1233, 1965, p. 213] (here, p_i is the $(n-i+1)$ st pivot and $f(r) := \rho_r^c$).

Cryer [282, 1968] defined

$$g(n) = \sup_{A \in \mathbb{R}^{n \times n}} \rho_n^c(A). \quad (9.15)$$

The following results are known:

- $g(2) = 2$ (trivial).
- $g(3) = 2\frac{1}{4}$; Tornheim [1149, 1965] and Cohen [257, 1974].

- $g(4) = 4$; Cryer [282, 1968] and Cohen [257, 1974].
- $g(5) < 5.005$; Cohen [257, 1974].

Tornheim [1149, 1965] (see also Cryer [282, 1968]) showed that $\rho_n^c(H_n) \geq n$ for any $n \times n$ Hadamard matrix H_n (a bound which, as we saw above, holds for any form of pivoting). For n such that a Hadamard matrix does not exist, the best known lower bound is $g(n) \geq \rho_n^c(S_n) = (n+1)/2$ (see (9.12)).

Cryer [282, 1968] conjectured that for real matrices $\rho_n^c(A) \leq n$, with equality if and only if A is a Hadamard matrix. The conjecture $\rho_n^c(A) \leq n$ became one of the most famous open problems in numerical analysis, and has been investigated by many mathematicians. The conjecture was finally shown to be false in 1991. Using a package LANCELOT [263, 1992] designed for large-scale nonlinear optimization, Gould [513, 1991] discovered a 13×13 matrix for which the growth factor is 13.0205 in IEEE double precision floating point arithmetic. Edelman subsequently showed, using the symbolic manipulation packages Mathematica and Maple, that a growth factor 13.02 can be achieved in exact arithmetic by making a small perturbation (of relative size 10^{-7}) to one element of Gould's matrix [374, 1992], [385, 1991]. A more striking counterexample to the conjecture is a matrix of order 25 for which $\rho_{25}^c = 32.986341$ [374, 1992]. Interesting problems remain, such as determining $\lim_{n \rightarrow \infty} g(n)/n$ and evaluating ρ_n^c for Hadamard matrices (see Problem 9.17).

For complex matrices the maximum growth factor is at least n for any n , since $\rho_n^c(V_n) \geq n$ (see (9.13)). The growth can exceed n , even for $n = 3$: Tornheim [1149, 1965] constructed the example

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & z & z^{-1} \\ 1 & z^{-1} & z \end{bmatrix}, \quad z = (-1 + i\sqrt{8})/3,$$

for which $\rho_3^c(A) = 3.079$.

Finally, we turn to rook pivoting. Foster [435, 1997] has shown that the growth factor for rook pivoting satisfies

$$\rho_n \leq 1.5n^{\frac{3}{4} \log n}. \quad (9.16)$$

This bound grows only slightly faster than that in (9.14) for complete pivoting and is much slower growing than that for partial pivoting; see Figure 9.2. Nothing is known about the largest attainable growth factor for rook pivoting (see Problem 9.18).

Figure 9.3 shows the results of an experiment in which, for each dimension $n = 100:100:1500$, 10 random matrices from the normal $N(0, 1)$ distribution and 10 from the uniform $[0, 1]$ distribution were factorized by LU factorization with partial pivoting, rook pivoting, and complete pivoting. The maximum growth factors and the number of comparisons (averaged in the case of rook pivoting) are plotted against n .

9.5. Diagonally Dominant and Banded Matrices

For matrices with certain special properties, more can be said about the behaviour of GE and, in particular, the size of the growth factor.

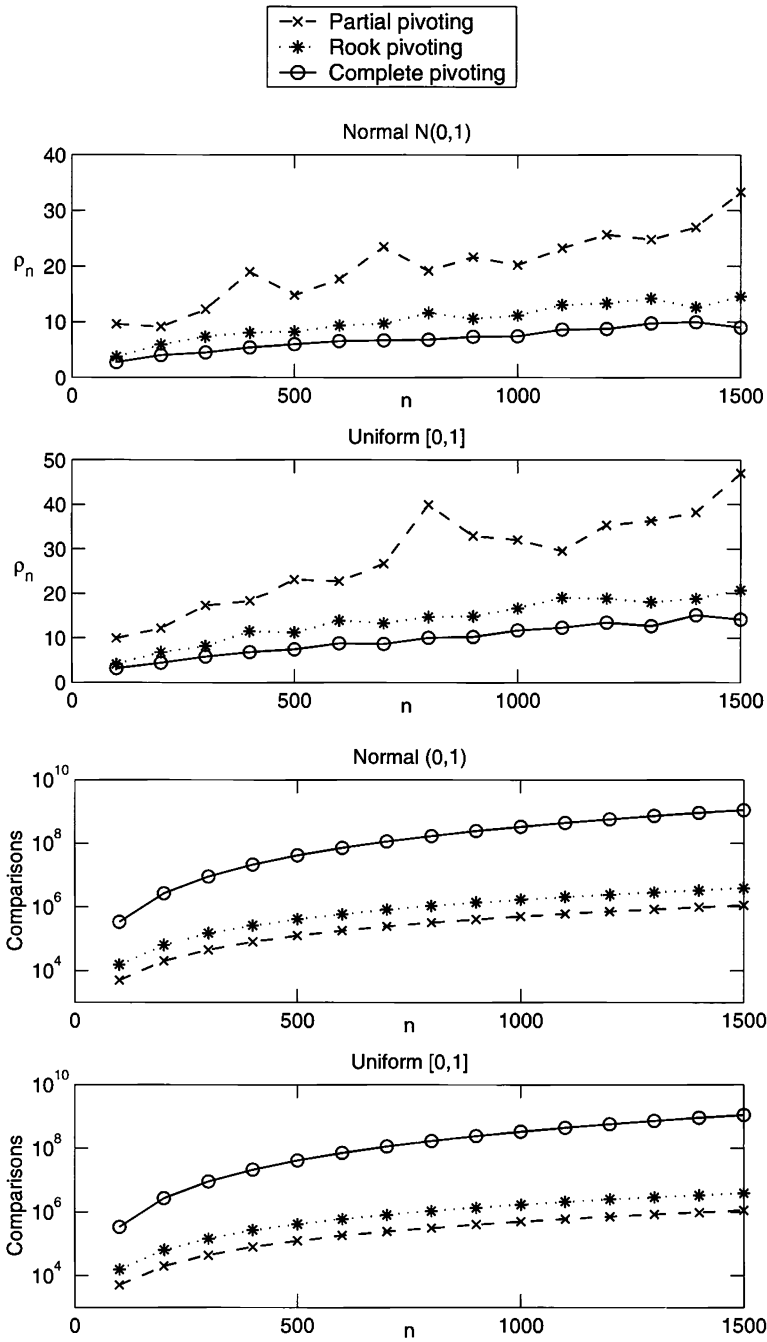


Figure 9.3. Maximum growth factors ρ_n (top) and average number of comparisons (bottom) for 15000 random matrices of dimension $n = 100:100:1500$.

As a first example, suppose $A \in \mathbb{C}^{n \times n}$ is *diagonally dominant by rows*,

$$\sum_{j \neq i} |a_{ij}| \leq |a_{ii}|, \quad i = 1:n,$$

or *diagonally dominant by columns*, that is, A^* is diagonally dominant by rows. Then GE without pivoting is perfectly stable.

Theorem 9.9 (Wilkinson). *Let $A \in \mathbb{C}^{n \times n}$ be nonsingular. If A is diagonally dominant by rows or columns then A has an LU factorization without pivoting and the growth factor $\rho_n \leq 2$. If A is diagonally dominant by columns then $|l_{ij}| \leq 1$ for all i and j in the LU factorization without pivoting (hence GEPP does not require any row interchanges).*

Proof. The result follows immediately from the more general Theorems 13.7 and 13.8 for block diagonally dominant matrices. (Note that the diagonal elements of A are nonzero, since otherwise the diagonal dominance would imply that A has a zero row or column, contradicting the nonsingularity of A . Therefore (13.17) holds.) $\square$

Note that for a matrix diagonally dominant by rows the multipliers can be arbitrarily large but, nevertheless, $\rho_n \leq 2$, so GE is perfectly stable. In fact, by writing $|L||U| = |AU^{-1}||U| \leq |A||U^{-1}||U|$ and invoking Lemma 8.8 we find that

$$\| |L||U| \|_{\infty} \leq (2n - 1) \|A\|_{\infty}. \quad (9.17)$$

This bound shows clearly that any large multipliers in L must be cancelled by correspondingly small elements of U .

A smaller bound for the growth factor also holds for an *upper Hessenberg matrix*: a matrix $H \in \mathbb{C}^{n \times n}$ for which $h_{ij} = 0$ for $i > j + 1$.

Theorem 9.10 (Wilkinson). *If $A \in \mathbb{C}^{n \times n}$ is upper Hessenberg then $\rho_n^p \leq n$.*

Proof. The structure of an upper Hessenberg H means that at each stage of GEPP we just add a multiple of the pivot row to the next row (after possibly swapping these two rows). That $\rho_n^p \leq n$ is a consequence of the following claim, which is easily proved by induction: at the start of stage k , row $k + 1$ of the reduced matrix is the same as row $k + 1$ of the original matrix, and the pivot row has elements of modulus at most k times the largest element of H . $\square$

A matrix $A \in \mathbb{C}^{n \times n}$ has *lower bandwidth* p and *upper bandwidth* q if $a_{ij} = 0$ for $i > j + p$ and $j > i + q$; see Figure 9.4. It is well known that in an LU factorization of a banded matrix the factors inherit A 's band structure: L has lower bandwidth p and U has upper bandwidth q . If partial pivoting is used then, in $PA = LU$, L has at most $p + 1$ nonzeros per column and U has upper bandwidth $p + q$. (For proofs of these properties see Golub and Van Loan [509, 1996, §4.3].) It is not hard to see that for a banded matrix, γ_n in Theorem 9.3 can be replaced by $\gamma_{\max(p+1, q+1)}$ and γ_{3n} in Theorem 9.4 can be replaced by $\gamma_{\max(p+1, q+1)} + \gamma_{p+q+1}$.

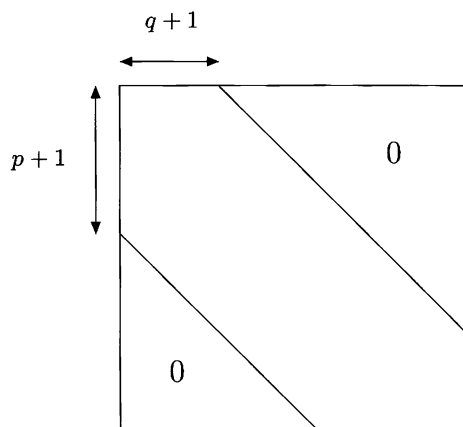


Figure 9.4. A banded matrix.

The following result bounds the growth factor for partial pivoting on a banded matrix.

Theorem 9.11 (Bohte). *If $A \in \mathbb{C}^{n \times n}$ has upper and lower bandwidths p then $\rho_n^p \leq 2^{2p-1} - (p-1)2^{p-2}$, and this bound is almost attainable when $n = 2p + 1$.*

Proof. See Bohte [146, 1975]. An example with $n = 9$ and $p = 4$ in which equality is almost attained is the matrix

$$A = \begin{bmatrix} -1 & -1 & -1 & -1 & 1 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 + \epsilon & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & -1 & -1 & -1 & -1 & 1 & 0 & 0 & 1 \\ 0 & 0 & -1 & -1 & -1 & -1 & 1 & 0 & 1 \\ 0 & 0 & 0 & -1 & -1 & -1 & -1 & 1 & 1 \\ 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & 1 \end{bmatrix},$$

where ϵ is an arbitrarily small positive number that ensures that rows 1 and 5 are interchanged on the first stage of the elimination, this being the only row interchange required. Ignoring terms in ϵ , the last column of U in $PA = LU$ is

$$[1, 1, 2, 4, 8, 16, 31, 60, 116]^T$$

and the growth factor is 116. $\square$

A special case of Theorem 9.11 is the easily verified result that for a tridiagonal matrix, $\rho_n^p \leq 2$. Hence GEPP achieves a small normwise backward error for tridiagonal matrices. In the next section we show that for several types of tridiagonal matrix GE *without pivoting* achieves a small *componentwise* backward error.

9.6. Tridiagonal Matrices

Consider the nonsingular tridiagonal matrix

$$A = \begin{bmatrix} d_1 & e_1 & & & \\ c_2 & d_2 & e_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & e_{n-1} \\ & & & c_n & d_n \end{bmatrix} \in \mathbb{R}^{n \times n},$$

and assume that A has an LU factorization $A = LU$, where

$$L = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & 1 & & \\ & & \ddots & \ddots & \\ & & & l_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_1 & e_1 & & & \\ & u_2 & e_2 & & \\ & & \ddots & \ddots & \\ & & & \ddots & e_{n-1} \\ & & & & u_n \end{bmatrix}. \quad (9.18)$$

GE for computing L and U is described by the recurrence relations

$$\left. \begin{aligned} u_1 &= d_1; \\ l_i &= c_i/u_{i-1} \\ u_i &= d_i - l_i e_{i-1} \end{aligned} \right\} i = 2:n. \quad (9.19)$$

For the computed quantities, we have

$$\begin{aligned} (1 + \epsilon_i) \hat{l}_i &= \frac{c_i}{\hat{u}_{i-1}}, & |\epsilon_i| &\leq u, \\ (1 + \theta_i) \hat{u}_i &= d_i - \hat{l}_i e_{i-1} (1 + \delta_i), & |\theta_i|, |\delta_i| &\leq u. \end{aligned}$$

Hence

$$\begin{aligned} |c_i - \hat{l}_i \hat{u}_{i-1}| &\leq u |\hat{l}_i \hat{u}_{i-1}|, \\ |d_i - \hat{l}_i e_{i-1} - \hat{u}_i| &\leq u (|\hat{l}_i e_{i-1}| + |\hat{u}_i|). \end{aligned}$$

In matrix terms these bounds may be written as

$$A = \hat{L}\hat{U} + \Delta A, \quad |\Delta A| \leq u |\hat{L}| |\hat{U}|. \quad (9.20)$$

If the LU factorization is used to solve a system $Ax = b$ by forward and back substitution then it is straightforward to show that the computed solution $\hat{x}$ satisfies

$$(\hat{L} + \Delta L)(\hat{U} + \Delta U)\hat{x} = b, \quad |\Delta L| \leq u |\hat{L}|, \quad |\Delta U| \leq (2u + u^2) |\hat{U}|. \quad (9.21)$$

Combining (9.20) and (9.21) we have, overall,

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq f(u) |\hat{L}| |\hat{U}|, \quad f(u) = 4u + 3u^2 + u^3. \quad (9.22)$$

The backward error result (9.22) applies to arbitrary nonsingular tridiagonal A having an LU factorization. We are interested in determining classes of tridiagonal A for which a bound of the form $|\Delta A| \leq g(u)|A|$ holds. Such a bound will hold if $|\hat{L}| |\hat{U}| = |\hat{L}\hat{U}|$, as noted in §9.3 (see (9.8)).

Three classes of matrices for which $|\hat{L}| |\hat{U}| = |\hat{L}\hat{U}|$ holds for the exact L and U are identified in the following theorem.

Theorem 9.12. *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and tridiagonal. If any of the following conditions hold then A has an LU factorization and $|L||U| = |LU|$:*

- (a) A is symmetric positive definite;
- (b) A is totally nonnegative, or equivalently, $L \geq 0$ and $U \geq 0$;
- (c) A is an M -matrix, or equivalently, L and U have positive diagonal elements and nonpositive off-diagonal elements;
- (d) A is sign equivalent to a matrix B of type (a)–(c), that is, $A = D_1 B D_2$, where $|D_1| = |D_2| = I$.

Proof. For (a), it is well known that a symmetric positive definite A has an LU factorization in which $U = DL^T$, where D is diagonal with positive diagonal elements. Hence $|L||U| = |L||D||L^T| = |LDL^T| = |LU|$, where the middle equality requires a little thought. In (b) and (c) the equivalences, and the existence of an LU factorization, follow from known results on totally nonnegative matrices [284, 1976] and M -matrices [106, 1994]; $|L||U| = |LU|$ is immediate from the sign properties of L and U . (d) is trivial. $\square$

For diagonally dominant tridiagonal matrices, $|L||U|$ is not equal to $|LU| = |A|$, but it cannot be much bigger.

Theorem 9.13. *Suppose $A \in \mathbb{R}^{n \times n}$ is nonsingular, tridiagonal, and diagonally dominant by rows or columns, and let A have the LU factorization $A = LU$. Then $|L||U| \leq 3|A|$.*

Proof. If $|i - j| = 1$ then $(|L||U|)_{ij} = |a_{ij}|$, so it suffices to consider the diagonal elements and show that (using the notation of (9.18))

$$|l_i e_{i-1}| + |u_i| \leq 3|d_i|.$$

The rest of the proof is for the case where A is diagonally dominant by rows; the proof for diagonal dominance by columns is similar.

First, we claim that $|e_i| \leq |u_i|$ for all i . The proof is by induction. For $i = 1$ the result is immediate, and if it is true for $i - 1$ then, from (9.19),

$$\begin{aligned} |u_i| &\geq |d_i| - |l_i||e_{i-1}| = |d_i| - \frac{|c_i|}{|u_{i-1}|}|e_{i-1}| \\ &\geq |d_i| - |c_i| \geq |e_i|, \end{aligned}$$

as required. Note that, similarly, $|u_i| \leq |d_i| + |c_i|$. Finally,

$$\begin{aligned} |l_i e_{i-1}| + |u_i| &= \left| \frac{c_i}{u_{i-1}} e_{i-1} \right| + |u_i| \leq |c_i| + |u_i| \\ &\leq |c_i| + (|d_i| + |c_i|) \\ &\leq 3|d_i|. \quad \square \end{aligned}$$

Theorem 9.14. *If the nonsingular tridiagonal matrix A is of type (a)–(d) in Theorem 9.12, and if the unit roundoff u is sufficiently small, then GE for solving $Ax = b$ succeeds and the computed solution $\hat{x}$ satisfies*

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq h(u)|A|, \quad h(u) = \frac{4u + 3u^2 + u^3}{1 - u}.$$

The same conclusion is true if A is diagonally dominant by rows or columns, with no restriction on u , provided the bound is multiplied by 3.

Proof. If u is sufficiently small then for types (a)–(c) the diagonal elements of $\hat{U}$ will be positive, since $\hat{u}_i \rightarrow u_i > 0$ as $u \rightarrow 0$. It is easy to see that $\hat{u}_i > 0$ for all i ensures that $|\hat{L}||\hat{U}| = |\hat{L}\hat{U}|$. The argument is similar for type (d). The result therefore follows from (9.22) and (9.8). The last part is trivial. $\square$

A corollary of Theorem 9.14 is that it is not necessary to pivot for the matrices specified in the theorem (and, indeed, pivoting could vitiate the result of the theorem). Note that large multipliers may occur under the conditions of the theorem, but they do not affect the stability. For example, consider the M -matrix

$$A = \begin{bmatrix} 2 & -2 & 0 \\ \epsilon - 2 & 2 & 0 \\ 0 & -1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ (\epsilon - 2)/2 & 1 & 0 \\ 0 & -1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 0 \\ 0 & \epsilon & 0 \\ 0 & 0 & 3 \end{bmatrix} = LU,$$

where $0 \leq \epsilon < 2$. The multiplier l_{32} is unbounded as $\epsilon \rightarrow 0$, but $|L||U| = |A|$ and GE performs very stably, as Theorem 9.14 shows it must.

9.7. More Error Bounds

From Theorem 9.4 it follows that the computed solution $\hat{x}$ from GE satisfies

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_{3n} \frac{\| |A^{-1}| |\hat{L}| |\hat{U}| |\hat{x}| \|_\infty}{\|x\|_\infty},$$

and this bound can be simplified to, for example,

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_{3n} \kappa_\infty(A) \frac{\| |\hat{L}| |\hat{U}| \|_\infty}{\|A\|_\infty}.$$

Given particular information about A it is often possible to obtain much stronger bounds. For example, if A is totally nonnegative then we know that GE produces a small componentwise relative backward error, as shown by (9.9), and hence $\|x - \hat{x}\|_\infty / \|x\|_\infty$ is bounded by a multiple of $\text{cond}(A, x)_u$.

Here is a line of attack that is fruitful in several situations. By Theorem 9.3 the computed LU factors satisfy $\hat{L}\hat{U} = A + \Delta A_0$, with $|\Delta A_0| \leq \gamma_n |\hat{L}||\hat{U}|$. Hence $\hat{L} = (A + \Delta A_0)\hat{U}^{-1}$ and

$$|\hat{L}||\hat{U}| \leq |A + \Delta A_0| \hat{U}^{-1} |\hat{U}| \leq (|A| + \gamma_n |\hat{L}||\hat{U}|) |\hat{U}^{-1}||\hat{U}|,$$

which gives, since $1 - \gamma_n |\hat{U}^{-1}||\hat{U}|$ is an M -matrix for $\gamma_n < 1$,

$$|\hat{L}||\hat{U}| \leq |A| \hat{U}^{-1} |\hat{U}| (1 - \gamma_n |\hat{U}^{-1}||\hat{U}|)^{-1}.$$

From Theorem 9.4, $(A + \Delta A)\hat{x} = b$, with

$$|\Delta A| \leq \gamma_{3n} |\hat{L}| |\hat{U}| \leq \gamma_{3n} |A| |\hat{U}^{-1}| |\hat{U}| (1 - \gamma_n |\hat{U}^{-1}| |\hat{U}|)^{-1}.$$

It follows that GE has a row-wise backward error bounded by a multiple of $\text{cond}(U)u$ (see Table 7.1). Correspondingly, the forward error is easily seen to satisfy

$$\begin{aligned} \frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} &\leq 3nu \frac{\|A^{-1}\| |A| |\hat{U}^{-1}| |\hat{U}| \|\hat{x}\|_\infty}{\|x\|_\infty} + O(u^2) \\ &\leq 3nu \text{cond}(A) \text{cond}(U) + O(u^2). \end{aligned} \quad (9.23)$$

In general, $\text{cond}(U)$ is unbounded, but from results in Chapter 8 we know that

- $\text{cond}(U) \leq 2^n - 1$ for rook pivoting and complete pivoting for general A (see Lemma 8.6),
- $\text{cond}(U) \leq 2n - 1$ for GE without pivoting applied to row diagonally dominant matrices (see Lemma 8.8).

An interesting conclusion is that a small row-wise backward error and (hence) a forward error bounded by a multiple of $\text{cond}(A)$ are guaranteed for row diagonally dominant matrices.

We continue the discussion of row-wise error bounds in the next section.

9.8. Scaling and Choice of Pivoting Strategy

Prior to solving a linear system $Ax = b$ by GE we are at liberty to scale the rows and the columns:

$$Ax = b \rightarrow D_1 A D_2 \cdot D_2^{-1} x = D_1 b, \quad \text{or} \quad A'y = c, \quad (9.24)$$

where D_1 and D_2 are nonsingular diagonal matrices. We apply GE to the scaled system $A'y = c$ and then recover x from $x = D_2 y$. Although scaling was used in some of the earliest published programs for GE [431, 1967], [836, 1962], how best to choose a scaling is still not well understood, and no single scaling algorithm can be guaranteed always to perform satisfactorily. Wilkinson's remark "We cannot decide whether equations are ill-conditioned without examining the way in which the coefficients were derived" [1233, 1965, p. 198] sums up the problem of scaling rather well.

The effect of scaling in GE without pivoting is easy to describe. If the elements of D_1 and D_2 are powers of the machine base β (so that the scaling is done without error) and GE produces $\hat{L}$ and $\hat{U}$ satisfying $A + \Delta A = \hat{L}\hat{U}$, then GE on $A' = D_1 A D_2$ produces $D_1 \hat{L} D_1^{-1}$ and $D_1 \hat{U} D_2$ satisfying $A' + D_1 \Delta A D_2 = D_1 \hat{L} D_1^{-1} \cdot D_1 \hat{U} D_2$. In other words, the rounding errors in GE scale in the same way as A . This is a result of Bauer [89, 1963] (see [431, 1967, Chap. 11] for a clear proof and discussion). With partial pivoting, however, the choice of pivots is affected by the row scaling (though not the column scaling), and in a way that is difficult to predict.

We can take a method-independent approach to scaling, by considering any method for solving $Ax = b$ that yields a solution satisfying

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq c_n \kappa_\infty(A) u,$$

with c_n a constant. For the scaled system (9.24) we have

$$\frac{\|D_2^{-1}(x - \hat{x})\|_\infty}{\|D_2^{-1}x\|_\infty} \leq c_n \kappa_\infty(D_1 A D_2) u,$$

so it is natural to choose D_1 and D_2 to minimize $\kappa_\infty(D_1 A D_2)$. As we saw in §7.3 (Theorem 7.8) the minimum possible value is $\rho(|A||A^{-1}|)$. However, a column scaling has the (usually) undesirable effect of changing the norm in which the error is measured. With row scaling only, the minimum value of $\kappa_\infty(D_1 A)$ is $\text{cond}(A) = \| |A^{-1}| |A| \|_\infty$, achieved when $D_1 A$ has rows of unit 1-norm (see (7.15)). Thus row equilibration yields a cond -bounded forward error.

Specializing to GE, we can say more. If A is row-equilibrated in the 1-norm then $|A|e = e$ and hence, from Theorem 9.4, the backward error matrix ΔA satisfies

$$|\Delta A| \leq \gamma_{3n} |\hat{L}| |\hat{U}| \leq \gamma_{3n} \|\hat{L}\| \|\hat{U}\|_\infty e e^T = \gamma_{3n} \|\hat{L}\| \|\hat{U}\|_\infty |A| e e^T.$$

In other words, if GE is normwise backward stable then it is *row-wise* backward stable (cf. Table 7.1), as noted by Skeel [1042, 1981].

We have already seen in §9.7 that a cond -bounded forward error is guaranteed for GE without pivoting on a row diagonally dominant matrix and for GE with rook pivoting or complete pivoting on an arbitrary matrix, provided that $\text{cond}(U)$ is of order 1. Further possibilities exist: if we LU factorize A^T instead of A (thus effectively carrying out column operations rather than row operations on A) then from (9.23) with the roles of L and U interchanged it follows that GEPP has a cond -bounded forward error if $\text{cond}(L^T)$ is of order 1. With a suitable scaling it is possible to do even better. Skeel [1040, 1979] shows that for $D_1 = \text{diag}(|A||x|)^{-1}$, GEPP on A is backward stable in the componentwise relative sense, and hence a forward error bound proportional to $\text{cond}(A, x) = \| |A^{-1}| |A| |x| \|_\infty / \|x\|_\infty$ holds; the catch is, of course, that the scaling depends on the unknown solution x ! Row equilibration can be regarded as approximating x by e in this “optimal” scaling.

Despite the variety of possible scalings and pivoting strategies and the catalogue of situations in which a particular pivoting strategy may have a smaller error bound than the usual theory predicts, in practice general linear systems are virtually always solved by unscaled GEPP. There are three main reasons:

1. Most users of GEPP feel that it performs well in practice.
2. GEPP has a long history of use in software libraries and packages (see page 188), and inertia makes it difficult to replace it. (For example, the LINPACK and LAPACK LU factorization routines use partial pivoting. LINPACK does not include scaling, while LAPACK's GEPP driver routine `xGESVX` offers an initial scaling as an option.)

3. A $\text{cond}(A, x)$ -bounded forward error *and* a small componentwise relative backward error are both achieved by following GEPP with fixed precision iterative refinement (under mild assumptions); see §12.2.

This last point explains why scaling, rook pivoting, and complete pivoting are not popular with numerical analysts for general linear systems. None of these techniques, with the exception of Skeel's optimal but implicitly defined scaling, guarantees a small componentwise relative backward error. Yet fixed precision refinement as a supplement to GEPP yields this ideal form of stability whenever it can reasonably be expected to and at negligible cost.

Some programs for GEPP incorporate row scaling *implicitly*. They compute row scale factors $d_1, \dots, d_n$, but, instead of applying GEPP to $\text{diag}(d_i)^{-1} \times A$, they apply it to A and choose as pivot row at the k th stage a row r for which $d_r |a_{rk}^{(k)}|$ is maximal. This type of scaling has the sole effect of influencing the choice of pivots. There is little justification for using it, and the best bound for the growth factor is 2^{n-1} multiplied by a product of terms d_{i_1}/d_{i_2} that can be large.

There is, however, one situation in which a form of implicit row scaling is beneficial. Consider the pivoting strategy that selects as the k th pivot an element $a_{rk}^{(k)}$ for which

$$\frac{|a_{rk}^{(k)}|}{\|A^{(k)}(r, k:n)\|_\infty} = \max_{i \geq k} \frac{|a_{ik}^{(k)}|}{\|A^{(k)}(i, k:n)\|_\infty}. \quad (9.25)$$

A result of Peña [932, 1996] shows that if there exists a permutation matrix P such that PA has an LU factorization $PA = LU$ with $|PA| = |L||U|$, then such a factorization will be produced by the pivoting scheme (9.25). This means that, unlike for partial pivoting, we can use the pivoting scheme (9.25) with impunity on totally nonnegative matrices and their inverses, row permutations of such matrices, and any matrix for which some row permutation has the “ $|PA| = |L||U|$ ” property. However, this pivoting strategy is as expensive as complete pivoting to implement, and for general A it is not guaranteed to produce a factorization as stable as that produced by partial pivoting.

9.9. Variants of Gaussian Elimination

Some variants of GE and partial pivoting have been proposed, mainly motivated by parallel computing and the aim of avoiding the sequential search down a column required by partial pivoting.

One idea is, at each element-zeroing step of GE, to interchange (if necessary) the pivot row and the row whose first element is to be zeroed to ensure that the multiplier is bounded by 1. In one particular algorithm, the first stage introduces zeros into elements $(2, 1)$, $(3, 1)$, $\dots$, $(n, 1)$, in this order, and the potential row interchanges are $1 \leftrightarrow 2$, $1 \leftrightarrow 3$, $\dots$, $1 \leftrightarrow n$, instead of just one row interchange as for partial pivoting. As well as saving on the pivot searching, this method has the advantage of permitting eliminations to be performed in parallel. For a 6×6 matrix we can represent the elimination as follows, where an integer k denotes

elements that can be eliminated in parallel on the k th stage:

$$\begin{bmatrix} \times & \times & \times & \times & \times & \times \\ 1 & \times & \times & \times & \times & \times \\ 2 & 3 & \times & \times & \times & \times \\ 3 & 4 & 5 & \times & \times & \times \\ 4 & 5 & 6 & 7 & \times & \times \\ 5 & 6 & 7 & 8 & 9 & \times \end{bmatrix}.$$

In general, there are $2n-3$ stages in each of which up to $n/2$ elements are eliminated in parallel. This algorithm is discussed by Wilkinson [1233, 1965, pp. 236–237] and Gallivan, Plemmons, and Sameh [452, 1990]. Sorensen [1057, 1985] derives a backward error bound for the factorization that is proportional to 4^n which, roughly, comprises a factor 2^{n-1} bounding the growth factor and a factor 2^{n-1} bounding “ L ”.

Another variant of GE is pairwise elimination, in which all row operations and row interchanges are between pairs of adjacent rows only. This method is discussed by Trefethen and Schreiber [1157, 1990] and is called Neville elimination by Gasca and Peña [461, 1992], who apply it to totally positive matrices. An error analysis of the method is given in [13, 1997].

Yet another idea is to avoid row interchanges altogether by adding plus or minus the row chosen by partial pivoting to the natural pivot row, thus ensuring that the multipliers are all bounded by 1. Mead, Renaut, and Welfert [838, 2001] show that the growth factor for this modified form of GEPP is bounded by 3^{n-1} , and that this bound is nearly attainable.

9.10. A Posteriori Stability Tests

Having solved a linear system by LU factorization we can compute the component-wise or normwise backward error at the cost of evaluating one or two matrix–vector products (see Theorems 7.1 and 7.3). In some situations, though, we may wish to assess the stability of a computed LU factorization before using it to solve one or more linear systems. One possibility is to compute the growth factor by monitoring the size of elements during the elimination, at a cost of $O(n^3)$ comparisons. This has been regarded as rather expensive, and more efficient ways to estimate ρ_n have been sought.

Businger [188, 1971] describes a way to obtain an upper bound for ρ_n in $O(n^2)$ operations. This approach is generalized by Erisman and Reid [393, 1974], who apply the Hölder inequality to the equation

$$a_{ij}^{(k+1)} = a_{ij} - \sum_{r=1}^k l_{ir} u_{rj}, \quad i, j > k,$$

to obtain the bound

$$\begin{aligned} |a_{ij}^{(k+1)}| &\leq |a_{ij}| + \|(l_{i1}, \dots, l_{ik})\|_p \|(u_{1j}, \dots, u_{kj})\|_q \\ &\leq \max_{i,j} |a_{ij}| + \max_i \|(l_{i1}, \dots, l_{i,i-1})\|_p \max_j \|(u_{1j}, \dots, u_{j-1,j})\|_q, \end{aligned} \quad (9.26)$$

where $p^{-1} + q^{-1} = 1$. In practice, $p = 1, 2, \infty$ are the values of interest. Barlow [69, 1986] notes that application of the Hölder inequality instead to

$$a_{ij}^{(k+1)} = \sum_{r=k+1}^{\min(i,j)} l_{ir} u_{rj}$$

yields a sometimes sharper bound.

It is interesting to note that in light of experience with the bound (9.26), Reid [980, 1987] recommends computing the growth factor explicitly in the context of sparse matrices, arguing that the expense is justified because (9.26) can be a very weak bound. See Erisman et al. [392, 1987] for some empirical results on the quality of the bound.

Chu and George [235, 1985] observe that the ∞ -norm of the matrix $|\widehat{L}||\widehat{U}|$ can be computed in $O(n^2)$ operations without forming the matrix explicitly, since

$$\| |\widehat{L}||\widehat{U}| \|_{\infty} = \| |\widehat{L}||\widehat{U}|e \|_{\infty} = \| |\widehat{L}|(|\widehat{U}|e) \|_{\infty}.$$

Thus one can cheaply compute a bound on $\|\Delta A\|_{\infty}$ from the componentwise backward error bounds in (9.6) and (9.7).

All the methods discussed in this section make use of an a priori error analysis to compute bounds on the backward error. Because the bounds do not take into account the statistical distribution of rounding errors, and because they involve somewhat pessimistic constant terms, they cannot be expected to be very sharp. Thus it is important not to forget that it is straightforward to compute the backward error itself: $A - \widehat{L}\widehat{U}$. Exact computation costs a prohibitively expensive $O(n^3)$ operations, but $\|A - \widehat{L}\widehat{U}\|_1$ can be *estimated* in $O(n^2)$ operations using the matrix norm estimator in Algorithm 15.4. Another possibility is to use a running error analysis, in which an error bound is computed concurrently with the factors (see §3.3).

9.11. Sensitivity of the LU Factorization

Although Theorem 9.3 bounds the backward error of the computed LU factors $\widehat{L}$ and $\widehat{U}$, it does not give any indication about the size of the forward errors $L - \widehat{L}$ and $U - \widehat{U}$. For most applications of the LU factorization it is the backward error and not the forward errors that matters, but it is still of some interest to know how big the forward errors can be. This is a question of perturbation theory and is answered by the next result.

Theorem 9.15 (Barrlund, Sun). *Let the nonsingular matrices $A \in \mathbb{R}^{n \times n}$ and $A + \Delta A$ have LU factorizations $A = LU$ and $A + \Delta A = (L + \Delta L)(U + \Delta U)$, and assume that $\|G\|_2 < 1$, where $G = L^{-1}\Delta A U^{-1}$. Then*

$$\max \left\{ \frac{\|\Delta L\|_F}{\|L\|_2}, \frac{\|\Delta U\|_F}{\|U\|_2} \right\} \leq \frac{\|G\|_F}{1 - \|G\|_2} \leq \frac{\|L^{-1}\|_2 \|U^{-1}\|_2 \|A\|_2}{1 - \|L^{-1}\|_2 \|U^{-1}\|_2 \|\Delta A\|_2} \frac{\|\Delta A\|_F}{\|A\|_F}. \quad (9.27)$$

Moreover, if $\rho(|\tilde{G}|) < 1$, where $\tilde{G} = (L + \Delta L)^{-1} \Delta A (U + \Delta U)^{-1}$, then

$$\begin{aligned} |\Delta L| &\leq |L + \Delta L| \text{stril}((I - |\tilde{G}|)^{-1} |\tilde{G}|), \\ |\Delta U| &\leq \text{triu}(|\tilde{G}|(I - |\tilde{G}|)^{-1}) |U + \Delta U|, \end{aligned}$$

where $\text{stril}(\cdot)$ and $\text{triu}(\cdot)$ denote, respectively, the strictly lower triangular part and the upper triangular part of their matrix arguments. $\square$

The normwise bounds (9.27) imply that $\chi(A) := \|L^{-1}\|_2 \|U^{-1}\|_2 \|A\|_2$ is an upper bound for the condition numbers of L and U under normwise perturbations. We have

$$\kappa_2(A) \leq \chi(A) \leq \min\{\kappa_2(L), \kappa_2(U)\} \kappa_2(A),$$

and the ratio $\chi(A)/\kappa_2(A)$ can be arbitrarily large (though if partial pivoting is used then $\kappa_2(L) \leq n2^{n-1}$).

The componentwise bounds in Theorem 9.15 are a little unsatisfactory in that they involve the unknown matrices ΔL and ΔU , but we can set these terms to zero and obtain a bound correct to first order.

9.12. Rank-Revealing LU Factorizations

In many applications it is useful to have a factorization of a matrix that “reveals the rank”. In exact arithmetic, a factorization produced by GE with pivoting is certainly rank-revealing if it has the form

$$PAQ = \begin{array}{c} r \\ n-r \end{array} \begin{array}{cc} r & n-r \\ \left[\begin{array}{cc} L_{11} & 0 \\ L_{21}^T & I \end{array} \right] \end{array} \begin{array}{cc} r & n-r \\ \left[\begin{array}{cc} U_{11} & U_{12} \\ 0 & 0 \end{array} \right] \end{array} \begin{array}{c} r \\ n-r \end{array},$$

with L_{11} and U_{11} nonsingular. It is easy to see that GE without pivoting or with partial pivoting does not in general produce such a factorization, but GE with rook pivoting or complete pivoting does. Formulating a definition of rank-revealing LU factorization for matrices that are not exactly rank-deficient is nontrivial, and there are several possible approaches. We will use the general definition of Demmel et al. [323, 1999] that a rank-revealing decomposition (RRD) of $A \in \mathbb{R}^{m \times n}$ is a factorization $A = XDY^T$, where $X \in \mathbb{R}^{m \times r}$, $D \in \mathbb{R}^{r \times r}$, and $Y \in \mathbb{R}^{n \times r}$, with $r \leq \min(m, n)$, D diagonal and nonsingular, and X and Y well conditioned. An RRD therefore concentrates the rank deficiency and ill condition of A into the diagonal matrix D . The existence of an RRD is clear, because the SVD is one (see §6.4).

Our interest here is in to what extent LU factorization with pivoting produces an RRD. If $PAQ = LU$ is an LU factorization with pivoting and A is nonsingular (but possibly ill conditioned) then we can take $X = L$, $D = \text{diag}(U)$ and $Y^T = D^{-1}U$. With no pivoting or with partial pivoting, Y^T can be an arbitrary upper triangular matrix and so an RRD is not obtained in general. With rook pivoting or complete pivoting X and Y^T are both unit triangular with off-diagonal elements bounded by 1 and hence

$$\kappa_\infty(X), \kappa_\infty(Y) \leq n2^{n-1}.$$

Moreover, in practice X and Y tend to be much better conditioned than these bounds suggest. Therefore GE with rook pivoting or complete pivoting is a means of computing an RRD, but with the remote possibility of failing to satisfy the definition by factors of order 2^n . This worst case is achievable. For the matrix $A(\theta) = U_n(\theta)^T U_n(\theta)$, where $U_n(\theta)$ is the Kahan matrix in (8.11), for small $\theta > 0$ we have $r = n$ and $\kappa_2(X) = \kappa_2(Y) \approx 2^{n-1}$.

Pan [914, 2000] shows the existence of a (guaranteed) RRD based on LU factorization and gives an algorithm for computing one that begins by computing an LU factorization with complete pivoting.

9.13. Historical Perspective

GE was the first numerical algorithm to be subjected to rounding error analysis, so it is instructive to follow the development of the error analysis from its beginnings in the 1940s.

In the 1940s there were three major papers giving error analyses of GE. Hotelling [639, 1943] presented a short forward error analysis of the LU factorization stage of GE. Under the assumptions that $|a_{ij}| \leq 1$ and $|b_i| \leq 1$ for all i and j and that the pivots are all of modulus unity, Hotelling derives a bound containing a factor 4^{n-1} for the error in the elements of the reduced upper triangular system. Hotelling's work was quoted approvingly by Bargmann, Montgomery, and von Neumann [64, 1946], who dismiss elimination methods for the solution of a linear system $Ax = b$ as being numerically unstable. Instead, they recommend computation of A^{-1} via the Newton–Schulz iteration described in §14.5 (which was also discussed by Hotelling). In one paragraph they outline the alleged shortcomings of elimination methods as follows:

In the elimination method a series of n compound operations is performed each of which depends on the preceding. An error at any stage affects all succeeding results and may become greatly magnified; this explains roughly why instability should be expected. It should be noticed that at each step a division is performed by a number whose size cannot be estimated in advance and which might be so small that any error in it would be greatly magnified by division. In fact such small divisors must occur if the determinant of the matrix is small and may occur even if it is not ... Another reason to expect instability is that once the variable x_n is obtained all the other variables are expressed in terms of it.

As Wilkinson [1242, 1974, p. 354] notes of this paragraph, “almost every statement in it is either wrong or misleading”.

Hotelling's result led to general pessimism about the practical effectiveness of GE for solving large systems of equations. Three papers published later in the same decade, as well as growing practical experience with the method, helped to restore confidence in GE.

Goldstine [499, 1972, p. 290] says of his discussions with von Neumann:

We did not feel it reasonable that so skilled a computer as Gauss would have fallen into the trap that Hotelling thought he had noted ... Von

Neumann remarked one day that even though errors may build up during one part of the computation, it was only relevant to ask how effective is the numerically obtained solution, not how close were some of the auxiliary numbers, calculated on the way to their correct counterparts. We sensed that at least for positive definite matrices the Gaussian procedure could be shown to be quite stable.

von Neumann and Goldstine [1200, 1947] subsequently gave a long and difficult *rigorous* fixed-point error analysis for the inversion of a symmetric positive definite matrix A via GE. They showed that the computed inverse $\hat{X}$ satisfies $\|A\hat{X} - I\|_2 \leq 14.2n^2\kappa_2(A)$. Parlett [925, 1990] explains that “the joy of this result was getting a polynomial in n , and the pain was obtaining 14.2, a number that reflects little more than the exigencies of the analysis.” Wilkinson [1239, 1971] gives an interesting critique of von Neumann and Goldstine’s paper and points out that the residual bound could hardly be improved using modern error analysis techniques. In a later paper [501, 1951], Goldstine and von Neumann gave a probabilistic analysis, which Goldstine summarizes as showing that “under reasonable probabilistic assumptions the error estimates of the previous paper could be reduced from a proportionality of n^2 to n ” [499, 1972, p. 291].

In his 1970 Turing Award Lecture [1240, 1971], Wilkinson recounts how in the early 1940s he solved a system of 12 linear equations on a desk calculator, obtaining a small residual. He goes on to describe a later experience:

It happened that some time after my arrival [at the National Physical Laboratory in 1946], a system of 18 equations arrived in Mathematics Division and after talking around it for some time we finally decided to abandon theorizing and to solve it ... The operation was manned by Fox, Goodwin, Turing, and me, and we decided on Gaussian elimination with complete pivoting ... Again the system was mildly ill-conditioned, the last equation had a coefficient of order 10^{-4} (the original coefficients being of order unity) and the residuals were again of order 10^{-10} , that is of the size corresponding to the exact solution rounded to ten decimals. It is interesting that in connection with this example we subsequently performed one or two steps of what would now be called “iterative refinement,” and this convinced us that the first solution had had almost six correct figures.

(Fox [439, 1987] notes that the computation referred to in this quotation took about two weeks using desk computing equipment!) In a subsequent paper, Fox, Huskey, and Wilkinson [440, 1948] presented empirical evidence in support of GE, commenting that “in our practical experience on matrices of orders up to the twentieth, some of them very ill-conditioned, the errors were in fact quite small”.

The experiences of Fox, Huskey, and Wilkinson prompted Turing to write a remarkable paper “Rounding-off errors in matrix processes” [1166, 1948]. In this paper, Turing made several important contributions. He formulated the LU (actually, the LDU) factorization of a matrix, proving the “if” part of Theorem 9.1 and showing that GE computes an LDU factorization. He introduced the term “condition number” and defined two matrix condition numbers, one of which is $n^{-1}N(A)N(A^{-1})$, where $N(A) = \|A\|_F$, the “ N -condition number of A ”. He used

Table 9.2. *Times for solution of a linear system of order n .*

Machine	Year	n	Time	Reference
Logarithm tables	c. 1884	29 ^a	7 weeks	[1076, 1995]
Desk computing equipment	c. 1946	18	2 weeks	[439, 1987]
Harvard Mark 1	1947	10	45 minutes	^b
IBM 602 Calculating Punch	1949	10	4 hours	[1195, 1949]
Pilot ACE	1951	17	over 3 hours	[1254, 1958]
Pilot ACE ^c	1954	30	1½ mins	[1254, 1958]
ACE	1958	30	5 seconds	[1254, 1958]
EDSAC 2	1960	31	4 seconds	[83, 1960]
EDSAC 2 ^d	1960	100	7 minutes	[83, 1960]

^aSymmetric positive definite system.

^b[142, 1948, p. 27], [552, 1948, p. 336].

^cWith magnetic drum store.

^dUsing magnetic tape for auxiliary storage.

the word “preconditioning” to mean improving the condition of a system of linear equations (a term that did not come into popular use until the 1970s). He described iterative refinement for linear systems. He exploited backward error ideas, for example by noting that “the triangular resolution obtained is an exact resolution of a matrix $A - S$, where $M(S) < \epsilon$ ” ($M(S) := \max_{i,j} |s_{ij}|$). Finally, and perhaps most importantly, he analysed GEPP for general matrices and obtained a bound for $\|x - \hat{x}\|_\infty$ that contains a term proportional to $\|A^{-1}\|_\infty^2$. (By making a trivial change in the analysis, namely replacing $A^{-1}b$ by x , Turing’s bound can be made proportional only to $\|A^{-1}\|_\infty$.) Turing also showed that the factor 4^{n-1} in Hotelling’s bound can be improved to 2^{n-1} and that still the bound is attained only in exceptional cases.

In a review of Turing’s paper, Bodewig [144, 1949] described the error bounds as “impractical” and advocated computing the residual of the computed solution and then determining “the exact correction by solving a new system.” That another researcher could miss the point of Turing’s analysis emphasizes how new the concept of rounding error analysis was in the 1940s.

Table 9.2 shows the time for solution of linear systems by GE on some early computing devices. The performance of modern computers on two linear system benchmarks is summarized by Dongarra [346, 2001] in a report that is regularly updated.

Douglas [353, 1959] presented a forward error analysis for GE applied to diagonally dominant tridiagonal systems arising in the solution of the heat equation by finite differences. He concluded that the whole procedure of solving this partial differential equation “is stable against round-off error”. It is surprising that Douglas’ paper is little known, because irrespective of the fact that his analysis can be simplified and clarified using modern techniques, his is one of the first truly positive rounding error results to be published.

A major breakthrough in the error analysis of GE came with Wilkinson’s pioneering backward error analysis, in which he proved Theorem 9.5 [1229, 1961],

[1232, 1963]. Apart from its simplicity and elegance and the realistic nature of the bounds, the main feature that distinguishes Wilkinson's analysis from the earlier error analyses of GE is that it bounds the normwise backward error rather than the forward error.

Wilkinson had been aware of the properties of the growth factor for partial pivoting long before developing his backward error analysis. In a 1954 paper [1225, 1954] he noted that

After m reductions the largest element is at most 2^m times as large as the largest original coefficient. It is possible to construct sets in which this factor is achieved but in practice an increase seldom takes place; more frequently the coefficients become progressively smaller, particularly if the equations are ill-conditioned.

This quote summarizes most of what we know today!

Four of the first textbooks to incorporate Wilkinson's analysis were those of Fox [436, 1964, pp. 161–174], Isaacson and Keller [667, 1966], Wendroff [1215, 1966], and Forsythe and Moler [431, 1967, Chap. 21]. Fox gives a simplified analysis for fixed-point arithmetic under the assumption that the growth factor is of order 1. Forsythe and Moler give a particularly readable backward error analysis that has been widely quoted.

Wilkinson's 1961 result is essentially the best that can be obtained by a normwise analysis. Subsequent work in error analysis for GE has mainly been concerned with bounding the backward error componentwise, as in Theorems 9.3 and 9.4. We note that Wilkinson could have given a componentwise bound for the backward perturbation ΔA , since most of his analysis is at the element level.

Chartres and Geuder [223, 1967] analyse the Doolittle version of GE. They derive a backward error result $(A + \Delta A)\hat{x} = b$, with a componentwise bound on ΔA ; although they do not recognize it, their bound can be written in the form $|\Delta A| \leq c_n u |\hat{L}| |\hat{U}|$.

Reid [979, 1971] shows that the assumption in Wilkinson's analysis that partial pivoting or complete pivoting is used is unnecessary. Without making any assumptions on the pivoting strategy, he derives for LU factorization the result $\hat{L}\hat{U} = A + \Delta A$, $|\Delta a_{ij}| \leq 3.01 \min(i-1, j) u \max_k |a_{ij}^{(k)}|$. Again, this is a componentwise bound. Erisman and Reid [393, 1974] note that for a sparse matrix, the term $\min(i-1, j)$ in Reid's bound can be replaced by m_{ij} , where m_{ij} is the number of multiplications required in the calculation of l_{ij} ($i > j$) or u_{ij} ($i \leq j$).

de Boor and Pinkus [300, 1977] give the result stated in Theorem 9.4. They refer to the original German edition [1085, 1972] of [1086, 1980] for a proof of the result and explain several advantages to be gained by working with a componentwise bound for ΔA , one of which is the strong result that ensues for totally nonnegative matrices. A result very similar to Theorem 9.4 is proved by Sauter [1012, 1978].

Skeel [1040, 1979] carried out a detailed componentwise error analysis of GE with a different flavour to the analysis given in this chapter. His aim was to understand the numerical stability of GE (in a precisely defined sense) and to determine the proper way to scale a system by examining the behaviour of the backward and forward errors under scaling (see §9.8). He later used this analysis to de-

rive important results about fixed precision iterative refinement (see Chapter 12). Skeel's work popularized the use of componentwise backward error analysis and componentwise perturbation theory.

The " $|\widehat{L}||\widehat{U}|$ " componentwise style of backward error analysis for GE has been well known for some time, as evidenced by its presence in the textbooks of Conte and de Boor [264, 1980], Golub and Van Loan [509, 1996] (also the 1983 first edition), and Stoer and Bulirsch [1086, 1980].

Forward error analyses have also been done for GE. The analyses are more complicated and more difficult to interpret than the backward error analyses. Olver and Wilkinson [906, 1982] derive a posteriori forward error bounds that require the computation of A^{-1} . Further results are given in a series of papers by Stummel [1096, 1982], [1097, 1985], [1098, 1985], [1099, 1985].

9.14. Notes and References

A variant of GE was used by the Chinese around the first century AD; the *Jiu Zhang Suanshu* (Nine Chapters of the Mathematical Art) contains a worked example for a system of five equations in five unknowns [680, 1991, pp. 156–177], [776, 1989].

Gauss, who was a great statistician and numerical analyst, developed his elimination method as a tool to help him prove results in linear regression theory. The first published appearance of GE is in his *Theoria Motus* (1809). Stewart [1076, 1995] gives a survey of Gauss's work on solving linear systems; see also the afterword in [463, 1995].

The traditional form of GE, as given at the start of this chapter, can be expressed algorithmically as

```

for  $k = 1:n$ 
  for  $j = k+1:n$ 
    for  $i = k+1:n$ 
       $a_{ij} = a_{ij} - (a_{ik}/a_{kk})a_{kj}$ 
    end
  end
end
end
```

This is identified as the *kji* form of GE. Altogether there are six possible orderings of the three loops. Doolittle's method (Algorithm 9.2) is the *ijk* or *jik* variant of Gaussian elimination. The choice of loop ordering does not affect the stability of the computation, but can greatly affect the efficiency of GE on a high-performance computer. For more on the different loop orderings of GE see Chapter 13; Dongarra, Gustavson, and Karp [344, 1984]; and the books by Dongarra, Duff, Sorensen, and van der Vorst [349, 1998] and Golub and Van Loan [509, 1996].

This chapter draws on the survey paper by Higham [593, 1990]. Theorems 9.7 and 9.8 are from Higham and Higham [616, 1989].

Myrick Hascall Doolittle (1830–1913) was a "computer of the United States coast and geodetic survey" [402, 1987]. Crout's method was published in an engineering journal in 1941 [281, 1941].

GE and its variants were known by various descriptive names in the early days of computing. These include the bordering method, the escalator method (for matrix inversion), the square root method (Cholesky factorization), and pivotal condensation. A good source for details of these methods is Faddeeva [399, 1959].

In a confidential 1948 report that “covers the general principles of both the design of the [Automatic Computing Engine] and the method of programming adopted for it”, Wilkinson gives a program implementing GE with partial pivoting and iterative refinement [1224, 1948, p. 111]. This was probably the first such program to be written—and for a machine that had not yet been built!

The terms “partial pivoting” and “complete pivoting” were introduced by Wilkinson in [1229, 1961]. The pivoting techniques themselves were in use in the 1940s and it is not clear who, if anyone, can be said to have invented them; indeed, von Neumann and Goldstine [1200, 1947, §4.2] refer to complete pivoting as the “customary procedure”.

Complete pivoting is currently used in several situations where the benefits it brings clearly outweigh the extra cost. It is used in LAPACK routine `xLASYS2` for solving Sylvester equations of dimensions 1 or 2 (see §16.6.1 and the solution to Problem 16.4) and in certain computations involving the generalized Schur decomposition. It is also used for computing a rank-revealing decomposition (see §9.12), for example as a first step in computing an accurate singular value decomposition; see Demmel et al. [323, 1999].

Rook pivoting for nonsymmetric matrices was introduced by Neal and Poole in [880, 1992]; see also [947, 2000]. Related pivoting strategies for symmetric indefinite matrices were introduced earlier by Fletcher [415, 1976] and subsequently by Ashcraft, Grimes, and Lewis [38, 1998]; see §11.1.3. Given the practical effectiveness of partial pivoting it is unlikely that rook pivoting will ever gain popular use for nonsymmetric matrices, and when the strongest possible guarantee of stability is required complete pivoting will always be preferred. Rook pivoting may find a niche for computing rank-revealing decompositions, since it is clearly better than partial pivoting for this purpose and no worse than complete pivoting in the worst case. For symmetric matrices rook pivoting-like strategies have additional advantages that make them of great practical interest; see §11.1.3.

There is a long history of published programs for GE, beginning with Crout routines of Forsythe [425, 1960], Thacher [1134, 1961], McKeeman [836, 1962], and Bowdler, Martin, Peters, and Wilkinson [154, 1966], all written in Algol 60 (which was the “official” language for publishing mathematical software in the 1960s and a strong competitor to Fortran for practical use at that time). The GE routines in LAPACK are the latest in a lineage beginning with the Fortran routines `decomp` and `solve` in Forsythe and Moler’s book [431, 1967], and continuing with routines by Moler [860, 1972], [861, 1972] (which achieve improved efficiency in Fortran by accessing arrays by column rather than by row), Forsythe, Malcolm, and Moler [430, 1977] (these routines incorporate condition estimation—see Chapter 15), and LINPACK [341, 1979].

LU factorization of totally nonnegative matrices has been investigated by Cryer [283, 1973], [284, 1976], Ando [26, 1987], and de Boer and Pinkus [300, 1977]. It is natural to ask whether we can test for total nonnegativity without computing all the minors. The answer is yes: for an $n \times n$ matrix total nonnegativity can be

tested in $O(n^3)$ operations, by testing the signs of the pivots in suitable elimination schemes, as shown by Cryer [284, 1976] and Gasca and Peña [461, 1992]. The latter paper employs Neville (pairwise) elimination (see §9.9). Note the analogy with positive definiteness, which holds for a symmetric matrix if and only if all the pivots in GE are positive. For an insightful survey of totally nonnegative matrices see Fallat [400, 2001].

The dilemma of whether to define the growth factor in terms of exact or computed quantities is faced by all authors; most make one choice or the other, and go on to derive, without comment, bounds that are strictly incorrect. Theorem 9.9, for example, bounds the exact growth factor; the computed one could conceivably violate the bound, but only by a tiny relative amount. van Veldhuizen [1184, 1977] shows that for a variation of partial pivoting that allows either a row or column interchange at each stage, the growth factor defined in terms of computed quantities is at most about $(1 + 3nu)2^{n-1}$, compared with the bound 2^{n-1} for the exact growth factor.

The idea of deriving error bounds for GE by analysing the equations obtained by solving $A = LU$ is exploited by Wilkinson [1241, 1974], who gives a general analysis that includes Cholesky factorization. This paper gives a concise summary of error analysis of factorization methods for linear equations and least squares problems.

Various authors have tabulated growth factors in extensive tests with random matrices. In tests during the development of LINPACK, the largest value observed was $\rho_{40}^p = 23$, occurring for a random matrix of 1s, 0s, and -1s [341, 1979, p. 1.21]. Macleod [803, 1989] recorded a value $\rho_{100}^p = 35.1$, which occurred for a symmetric matrix with elements from the uniform distribution on $[-1, 1]$. In one MATLAB test of 100,000 matrices of dimension 100 from the normal $N(0, 1)$ distribution, I found the largest growth factor to be $\rho_n^p = 20.12$.

Gould [513, 1991] used the optimization LANCELOT [263, 1992] to maximize the n th pivot for complete pivoting as a function of about $n^3/3$ variables comprising the intermediate elements $a_{ij}^{(k)}$ of the elimination; constraints were included that normalize the matrix A , describe the elimination equations, and impose the complete pivoting conditions. Gould's package found many local maxima, and many different starting values had to be tried in order to locate the matrix for which $\rho_{13}^c > 13$. In an earlier attempt at maximizing the growth factor, Day and Peterson [299, 1988] used a problem formulation in which the variables are the n^2 elements of A , which makes the constraints and objective function substantially more nonlinear than in Gould's formulation. Using the package NPSOL [484, 1986], they obtained "largest known" growth factors for $5 \leq n \leq 7$.

Theoretical progress into understanding the behaviour of the growth factor for complete pivoting has been made by Day and Peterson [299, 1988], Puschmann and Cortés [960, 1983], Puschmann and Nordio [961, 1985], and Edelman and Mascarenhas [382, 1995].

Probabilistic error analysis for GE is given by Barlow and Bareiss [73, 1985]. Yeung and Chan [1265, 1997] give a probabilistic analysis of GE without pivoting, obtaining the distributions of the entries of L and U for matrices from the normal $N(0, 1)$ distribution.

Barlow and Zha [77, 1998] consider an alternative definition of growth factor,

$\rho_n = \max_k \|A^{(k)}\|_2 / \|A\|_2$. They show that the maximum value of this growth factor for partial pivoting is about $2^{n/3}$ and that the maximum is attained at an orthogonal matrix.

A novel alternative to partial pivoting for stabilizing GE is proposed by Stewart [1066, 1974]. The idea is to modify the pivot element to make it suitably large, and undo this rank one change later using the Sherman–Morrison formula. Stewart gives error analysis that bounds the backward error for this modified form of GE.

Wilkinson proved Theorem 9.9 for matrices diagonally dominant by columns [1229, 1961, pp. 288–289]. Theorem 9.10 is proved in the same paper. That $\rho_n \leq 2$ for matrices diagonally dominant by rows does not appear to be well known, but it is proved by Wendroff [1215, 1966, pp. 122–123], for example.

In Theorem 9.9 it is assumed that A is nonsingular. It has long been known (and frequently rediscovered) that diagonal dominance provides sufficient conditions for nonsingularity: if a matrix is diagonally dominant with strict inequalities in all the defining inequalities, or if it is irreducible and at least one of the inequalities is strict, then the matrix is nonsingular [908, 1972, Thm. 6.2.6], [1190, 1976].

The results in §9.6 for tridiagonal matrices are taken from Higham [589, 1990]. Another method for solving tridiagonal systems is cyclic reduction, which was developed in the 1960s [190, 1970]. Error analysis given by Amodio and Mazzia [18, 1994] shows that cyclic reduction is normwise backward stable for diagonally dominant tridiagonal matrices.

The chapter “Scaling Equations and Unknowns” of Forsythe and Moler [431, 1967] is a perceptive, easy-to-understand treatment that is still well worth reading. Early efforts at matrix scaling for GE were directed to equilibrating either just the rows or the rows and columns simultaneously (so that all the rows and columns have similar norms). An algorithm with the latter aim is described by Curtis and Reid [286, 1972]. Other important references on scaling are the papers by van der Sluis [1177, 1970] and Stewart [1069, 1977], which employ normwise analysis, and those by Skeel [1040, 1979], [1042, 1981], which use componentwise analysis.

Much is known about the existence and stability of LU factorizations of M -matrices and related matrices. A is an H -matrix if the comparison matrix $M(A)$ (defined in (8.7)) is an M -matrix. Funderlic, Neumann, and Plemmons [449, 1982] prove the existence of an LU factorization for an H -matrix A that is generalized diagonally dominant, that is, DA is diagonally dominant by columns for some nonsingular diagonal matrix D ; they show that the growth factor satisfies $\rho_n \leq 2 \max_i |d_{ii}| / \min_i |d_{ii}|$. Neumann and Plemmons [886, 1984] obtain a growth factor bound for an inverse of an H -matrix. Ahac, Buoni, and Olesky [8, 1988] describe a novel column-pivoting scheme for which the growth factor can be bounded by n when A is an H -matrix; for more general related work see Peña [933, 1998].

The normwise bounds in Theorem 9.15 are due to Barrlund [81, 1991] and the componentwise ones to Sun [1103, 1992]. Similar bounds are given by Stewart [1075, 1993] and Sun [1104, 1992]. Perturbation bounds that can be much smaller than the normwise one in Theorem 9.15 are obtained by Chang and Paige [218, 1998] and Stewart [1078, 1997].

Interval arithmetic techniques (see §26.4) are worth considering if high accuracy

Table 9.3. *Records for largest dense linear systems solved (dimension n).*

Year	n	Computer	Time
1991	55,296	Connection Machine CM-2	4.4 days
1992/3	75,264	Intel iPSC/860	$2\frac{2}{3}$ days
1994	76,800	Connection Machine CM-5	4.1 days
1995	128,600	Intel Paragon	≈ 1 hour
1996	138,240	Hitachi SR2201/1024	2.2 hours
1997	235,000	Intel ASCI Red	1.8 hours
1998	259,200	SGI T3E1200	3.6 hours
1999	431,344	IBM ASCI Blue	6.9 hours
2000	518,096	IBM ASCI White	3.6 hours
2001	525,000	Compaq AlphaServer SC ES45	6.6 hours

or guaranteed accuracy is required when solving a linear system. We mention just two papers, those by Demmel and Krückeberg [330, 1985] and Rump [1001, 2001], which provide very readable introductions to the subject and contain further references.

Edelman [373, 1991], [377, 1993], [378, 1994] presents statistics and details of applications in which large dense linear algebra problems arise. He also discusses relevant issues such as what users expect of the computed solutions and how best to make use of parallel computers. Table 9.3 contains “world records” for linear systems from Edelman’s surveys and based on statistics from the *TOP500 Supercomputer Sites* Web site (<http://www.top500.org/> or <http://www.netlib.org/benchmark/top500.html>). A practical application in which very large systems arise is the solution of boundary integral equations, a major application being the analysis of radar cross sections; the resulting systems have coefficient matrices that are complex symmetric (but not Hermitian).

9.14.1. LAPACK

Driver routines `xGESV` (simple) and `xGESVX` (expert) use LU factorization with partial pivoting to solve a general system of linear equations with multiple right-hand sides. The expert driver incorporates iterative refinement, condition estimation, and backward and forward error estimation and has an option to scale the system $AX = B$ to $(D_R^{-1}AD_C^{-1})D_CX = D_R^{-1}B$ before solution, where $D_R = \text{diag}(r_i) = \text{diag}(\max_j |a_{ij}|)$ and $D_C = \text{diag}(\max_i r_i |a_{ij}|)$; the scaling is done by the routine `xGEEQU`. The LU factorization is computed by the routine `xGETRF`, which uses a partitioned outer product algorithm. The expert driver also returns the quantity $\|A\|/\|U\|$, where $\|A\| := \max_{i,j} |a_{ij}|$, which is an estimate of the reciprocal of the growth factor, $1/\rho_n^p$. A value much less than 1 signals that the condition estimate and forward error bound could be unreliable.

The auxiliary routine `xGETC2` implements LU factorization with complete pivoting.

For band matrices, the driver routines are `xGBSV` and `xGBSVX`, and for tridiagonal matrices, `xGTSV` and `xGTSVX`; they all use LU factorization with partial pivoting.

Problems

- 9.1.** (Completion of proof of Theorem 9.1.) Show that if a singular matrix $A \in \mathbb{R}^{n \times n}$ has a unique LU factorization then A_k is nonsingular for $k = 1:n-1$.
- 9.2.** Define $A(\sigma) = A - \sigma I$, where $\sigma \in \mathbb{R}$ and $A \in \mathbb{R}^{n \times n}$. For how many values of σ , at most, does $A(\sigma)$ fail to have an LU factorization without pivoting?
- 9.3.** Show that $A \in \mathbb{R}^{n \times n}$ has a unique LU factorization if 0 does not belong to the field of values $F(A) = \{z^*Az/(z^*z) : 0 \neq z \in \mathbb{C}^n\}$.
- 9.4.** State analogues of Theorems 9.3 and 9.4 for LU factorization with row and column interchanges: $PAQ = LU$.
- 9.5.** Give a 2×2 matrix A having an LU factorization $A = LU$ such that $|L||U| \leq c|A|$ does not hold for any c , yet $\| |L||U| \|_\infty / \|A\|_\infty$ is of order 1.
- 9.6.** Show that if $A \in \mathbb{R}^{n \times n}$ is nonsingular and totally nonnegative it has an LU factorization $A = LU$ with $L \geq 0$ and $U \geq 0$. (Hint: use the inequality

$$\det(A) \leq \det(A(1:p, 1:p)) \det(A(p+1:n, p+1:n)), \quad p = 1:n-1,$$

which holds for any totally nonnegative A [454, 1959, p. 100].) Deduce that the growth factor for GE without pivoting $\rho_n \equiv 1$.

- 9.7.** How many square submatrices, and how many rectangular submatrices, does an $n \times n$ matrix have?
- 9.8.** Show that if $A \in \mathbb{R}^{n \times n}$ is nonsingular and its inverse is totally nonnegative then it has an LU factorization $A = LU$ with $|A| = |L||U|$. (Use the fact that if C is totally nonnegative and nonsingular then $JC^{-1}J$ is totally nonnegative, where $J = \text{diag}((-1)^{i+1})$ (this can be proved using determinantal identities; see [26, 1987, Thm. 3.3]).)
- 9.9.** Show that for GE without pivoting $\rho_n \leq 1 + n\| |L||U| \|_\infty / \|A\|_\infty$.
- 9.10.** Suppose that GE without pivoting is applied to a linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular, and that all operations are performed exactly except for the division determining a single multiplier l_{ij} (where $i > j$ and $A = LU$), which is computed with relative error ϵ : $l_{ij} = \tilde{l}_{ij}(1 + \epsilon)$. Evaluate the difference $x - \hat{x}$ between the exact and computed solutions. (The answer allows us to see clearly the effect of a computational blunder, which could, for example, be the result of the malfunction of a computer's divide operation.)
- 9.11.** Show that θ in Theorem 9.8 satisfies

$$\theta(B) := \theta \left(\begin{bmatrix} A & A \\ A & -A \end{bmatrix} \right) = 2\theta(A).$$

Hence, for $g(n)$ defined in (9.15) and S_n in (9.12), deduce a larger lower bound than $g(2n) \geq \rho_n^c(S_{2n}) = (2n+1)/2$.

9.12. Explain the errors in the following criticism of GE with complete pivoting.

Gaussian elimination with complete pivoting maximizes the pivot at each stage of the elimination. Since the product of the pivots is the determinant (up to sign), which is fixed, making early pivots large forces later ones to be small. These small pivots will have large relative errors due to the accumulation of rounding errors during the algorithm, and dividing by them therefore introduces larger errors.

9.13. In sparse matrix computations the choice of pivot in GE has to be made with the aim of preserving sparsity as well as maintaining stability. In *threshold pivoting*, a pivot element is chosen from among those elements in column k that satisfy $|a_{ik}^{(k)}| \geq \tau \max_{m \geq k} |a_{mk}^{(k)}|$, where $\tau \in (0, 1]$ is a parameter (see, for example, Duff, Erisman, and Reid [360, 1986, §5.4]). Show that for threshold pivoting

$$\max_i |a_{ij}^{(k)}| \leq (1 + \tau^{-1})^{\mu_j} \max_i |a_{ij}|,$$

where μ_j is the number of nonzero entries in the j th column of U . Hence obtain a bound for ρ_n .

9.14. Let $A \in \mathbb{R}^{n \times n}$ be pre-pivoted for GEPP, that is, GEPP requires no row interchanges when applied to A . Show that GEPP computes the same LU factorization as (a) the first method described in §9.9 and (b) pairwise pivoting, with the natural pivoting strategy that ensures the multipliers are bounded by 1, applied to IIA , where $II = I(n; -1:1, :)$ is the row reversal permutation matrix.

9.15. (RESEARCH PROBLEM) Obtain sharp bounds for the growth factor for GEPP applied to (a) a matrix with lower bandwidth p and upper bandwidth q (thus generalizing Theorem 9.11), and (b) a quasi-tridiagonal matrix (an $n \times n$ matrix that is tridiagonal except for nonzero $(1, n)$ and $(n, 1)$ elements).

9.16. (RESEARCH PROBLEM) Explain why the growth factor for GEPP is almost always small in practice.

9.17. (RESEARCH PROBLEM) For GE with complete pivoting what is the value of $\lim_{n \rightarrow \infty} g(n)/n$ (see (9.15))? Is ρ_n^c equal to n for Hadamard matrices?

9.18. (RESEARCH PROBLEM) Let $g(n)$ be defined for GE with rook pivoting analogously to (9.15). Try to obtain the exact value of $g(n)$ for $n \leq 4$, say (cf. the bounds for complete pivoting on page 169), and to obtain lower bounds valid for all n . All that is currently known is the bound (9.16) and a slightly smaller implicitly defined bound from which it is derived in [435, 1997], together with the following lower bounds found by direct search (as in §26.1):

n	2	3	4	5
lower bound	2 (exact)	2.9	4.16	5.36

Chapter 10

Cholesky Factorization

The matrix of that equation system is negative definite—which is a positive definite system that has been multiplied through by -1 .

For all practical geometries the common finite difference Laplacian operator gives rise to these, the best of all possible matrices.

Just about any standard solution method will succeed, and many theorems are available for your pleasure.

— FORMAN S. ACTON, *Numerical Methods That Work* (1970)

Many years ago we made out of half a dozen transformers a simple and rather inaccurate machine for solving simultaneous equations—the solutions being represented as flux in the cores of the transformers.

During the course of our experiments we set the machine to solve the equations—

$$X + Y + Z = 1$$

$$X + Y + Z = 2$$

$$X + Y + Z = 3$$

The machine reacted sharply—it blew the main fuse and put all the lights out.

— B. V. BOWDEN, *The Organization of a Typical Machine* (1953)

There does seem to be some misunderstanding about the purpose of an a priori backward error analysis.

All too often, too much attention is paid to the precise error bound that has been established.

The main purpose of such an analysis is either to establish the essential numerical stability of an algorithm or to show why it is unstable and in doing so to expose what sort of change is necessary to make it stable.

The precise error bound is not of great importance.

— J. H. WILKINSON, *Numerical Linear Algebra on Digital Computers* (1974)

10.1. Symmetric Positive Definite Matrices

Symmetric positive definiteness is one of the highest accolades to which a matrix can aspire. Symmetry confers major advantages and simplifications in the eigenproblem and, as we will see in this chapter, positive definiteness permits economy and numerical stability in the solution of linear systems.

A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is *positive definite* if $x^T A x > 0$ for all nonzero $x \in \mathbb{R}^n$. Well-known equivalent conditions to $A = A^T$ being positive definite are

- $\det(A_k) > 0$, $k = 1:n$, where $A_k = A(1:k, 1:k)$ is the leading principal submatrix of order k .
- $\lambda_k(A) > 0$, $k = 1:n$, where λ_k denotes the k th largest eigenvalue.

The first of these conditions implies that A has an LU factorization, $A = LU$ (see Theorem 9.1). Another characterization of positive definiteness is that the pivots in LU factorization are positive, since $u_{kk} = \det(A_k) / \det(A_{k-1})$. By factoring out the diagonal of U and taking its square root, the LU factorization can be converted into a *Cholesky factorization*: $A = R^T R$, where R is upper triangular with positive diagonal elements. This factorization is so important that it merits a direct proof.

Theorem 10.1. *If $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite then there is a unique upper triangular $R \in \mathbb{R}^{n \times n}$ with positive diagonal elements such that $A = R^T R$.*

Proof. The proof is by induction. The result is clearly true for $n = 1$. Assume it is true for $n - 1$. The leading principal submatrix $A_{n-1} = A(1:n-1, 1:n-1)$ is positive definite, so it has a unique Cholesky factorization $A_{n-1} = R_{n-1}^T R_{n-1}$. We have a factorization

$$A = \begin{bmatrix} A_{n-1} & c \\ c^T & \alpha \end{bmatrix} = \begin{bmatrix} R_{n-1}^T & 0 \\ r^T & \beta \end{bmatrix} \begin{bmatrix} R_{n-1} & r \\ 0 & \beta \end{bmatrix} =: R^T R \quad (10.1)$$

if

$$R_{n-1}^T r = c, \quad (10.2)$$

$$r^T r + \beta^2 = \alpha. \quad (10.3)$$

Equation (10.2) has a unique solution since R_{n-1} is nonsingular. Then (10.3) gives $\beta^2 = \alpha - r^T r$. It remains to check that there is a unique real, positive β satisfying this equation. From the equation

$$0 < \det(A) = \det(R^T) \det(R) = \det(R_{n-1})^2 \beta^2$$

we see that $\beta^2 > 0$, hence there is a unique $\beta > 0$. $\square$

The proof of the theorem is constructive, and provides a way to compute the Cholesky factorization that builds R a column at a time. Alternatively, we can work directly from the equations

$$a_{ij} = \sum_{k=1}^i r_{ki} r_{kj}, \quad j \geq i,$$

which follow by equating (i, j) elements in $A = R^T R$. By solving these equations in the order $(1,1)$, $(1,2)$, $(2,2)$, $(1,3)$, $(2,3)$, $(3,3)$, $\dots$, (n,n) , we obtain the following algorithm.

Algorithm 10.2 (Cholesky factorization). Given a symmetric positive definite $A \in \mathbb{R}^{n \times n}$ this algorithm computes the Cholesky factorization $A = R^T R$.

```

for  $j = 1:n$ 
  for  $i = 1:j - 1$ 
     $r_{ij} = (a_{ij} - \sum_{k=1}^{i-1} r_{ki} r_{kj}) / r_{ii}$ 
  end
   $r_{jj} = (a_{jj} - \sum_{k=1}^{j-1} r_{kj}^2)^{1/2}$ 
end

```

Cost: $n^3/3$ flops (half the cost of LU factorization).

As for Gaussian elimination (GE), there are different algorithmic forms of Cholesky factorization. Algorithm 10.2 is the jik or “sdot” form. We describe the kij , outer product form in §10.3.

Given the Cholesky factorization $A = R^T R$, a linear system $Ax = b$ can be solved via the two triangular systems $R^T y = b$ and $Rx = y$.

If we define $D = \text{diag}(r_{ii}^2)$ then the Cholesky factorization $A = R^T R$ can be rewritten as $A = LDL^T$, where $L = R^T \text{diag}(r_{ii})^{-1}$ is unit lower triangular. The LDL^T factorization is sometimes preferred over the Cholesky factorization because it avoids the need to compute the n square roots that determine the r_{ii} . The LDL^T factorization is certainly preferred for solving tridiagonal systems, as it requires n fewer divisions than Cholesky factorization in the substitution stage. All the results for Cholesky factorization in this chapter have analogues for the LDL^T factorization. Block LDL^T factorization for indefinite matrices is discussed in §11.1.

10.1.1. Error Analysis

Error bounds for Cholesky factorization are derived in a similar way to those for LU factorization. Consider Algorithm 10.2. Using Lemma 8.4 we have

$$\left| a_{ij} - \sum_{k=1}^i \hat{r}_{ki} \hat{r}_{kj} \right| \leq \gamma_i \sum_{k=1}^i |\hat{r}_{ki}| |\hat{r}_{kj}|. \quad (10.4)$$

From a variation of Lemma 8.4 in which the division is replaced by a square root (see Problem 10.3), we have

$$\left| a_{jj} - \sum_{k=1}^j \hat{r}_{kj}^2 \right| \leq \gamma_{j+1} \sum_{k=1}^j \hat{r}_{kj}^2.$$

A backward error result is immediate.

Theorem 10.3. *If Cholesky factorization applied to the symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$ runs to completion then the computed factor $\hat{R}$ satisfies*

$$\hat{R}^T \hat{R} = A + \Delta A, \quad |\Delta A| \leq \gamma_{n+1} |\hat{R}^T| |\hat{R}|. \quad \square \quad (10.5)$$

Theorem 10.4. *Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite and suppose Cholesky factorization produces a computed factor $\hat{R}$ and a computed solution $\hat{x}$ to $Ax = b$. Then*

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq \gamma_{3n+1}|\hat{R}^T||\hat{R}|. \quad (10.6)$$

Proof. The proof is analogous to the proof of Theorem 9.4. $\square$

These results imply that Cholesky factorization enjoys perfect normwise backward stability. The key inequality is (using Lemma 6.6)

$$\| |R^T| |R| \|_2 = \| |R| \|_2^2 \leq n \|R\|_2^2 = n \|A\|_2,$$

whose analogue for the computed $\hat{R}$ is, from (10.5),

$$\| |\hat{R}^T| |\hat{R}| \|_2 \leq n(1 - n\gamma_{n+1})^{-1} \|A\|_2.$$

Thus (10.6) implies

$$\|\Delta A\|_2 \leq \| |\Delta A| \|_2 \leq \gamma_{3n+1} n(1 - n\gamma_{n+1})^{-1} \|A\|_2 \leq 4n(3n+1)u \|A\|_2, \quad (10.7)$$

where for the last inequality we have assumed that $\max((3n+1)u, n\gamma_{n+1}) < 1/2$. Another indicator of stability is that the growth factor for GE is exactly 1 (see Problem 10.4). It is important to realize that the multipliers can be arbitrarily large (consider, for example, $\begin{bmatrix} \theta^2 & \theta \\ \theta & 2 \end{bmatrix}$ as $\theta \rightarrow 0$). But, remarkably, for a positive definite matrix the size of the multipliers has no effect on stability.

Note that the perturbation ΔA in (10.6) is not symmetric, in general, because the backward error matrices for the triangular solves with R and R^T are not the transposes of each other. For conditions guaranteeing that a “small” symmetric ΔA can be found, see Problem 7.12.

The following rewritten version of Theorem 10.3 provides further insight into Cholesky factorization.

Theorem 10.5 (Demmel). *If Cholesky factorization applied to the symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$ runs to completion then the computed factor $\hat{R}$ satisfies*

$$\hat{R}^T \hat{R} = A + \Delta A, \quad |\Delta A| \leq (1 - \gamma_{n+1})^{-1} \gamma_{n+1} d d^T,$$

where $d_i = a_{ii}^{1/2}$.

Proof. Theorem 10.3 shows that $\hat{R}^T \hat{R} = A + \Delta A$ with $|\Delta A| \leq \gamma_{n+1} |\hat{R}^T| |\hat{R}|$. Denoting by $\hat{r}_i$ the i th column of $\hat{R}$, we have

$$\|\hat{r}_i\|_2^2 = \hat{r}_i^T \hat{r}_i = a_{ii} + \Delta a_{ii} \leq a_{ii} + \gamma_{n+1} |\hat{r}_i|^T |\hat{r}_i|,$$

so that $\|\hat{r}_i\|_2^2 \leq (1 - \gamma_{n+1})^{-1} a_{ii}$. Then, using the Cauchy–Schwarz inequality,

$$|\hat{r}_i^T \hat{r}_j| \leq \|\hat{r}_i\|_2 \|\hat{r}_j\|_2 \leq (1 - \gamma_{n+1})^{-1} (a_{ii} a_{jj})^{1/2},$$

giving

$$|\hat{R}^T| |\hat{R}| \leq (1 - \gamma_{n+1})^{-1} d d^T \quad (10.8)$$

and the required bound for ΔA . $\square$

Standard perturbation theory applied to Theorem 10.4 yields a bound of the form $\|x - \hat{x}\|/\|x\| \leq c_n u \kappa(A) + O(u^2)$. However, with the aid of Theorem 10.5 we can obtain a potentially much smaller bound. The idea is to write $A = DHD$ where $D = \text{diag}(A)^{1/2}$, so that H has unit diagonal. van der Sluis's result (Corollary 7.6) shows that

$$\kappa_2(H) \leq n \min_{F \text{ diagonal}} \kappa_2(FAF), \quad (10.9)$$

so D is nearly a condition-minimizing diagonal scaling. It follows that $\kappa_2(H) \leq n\kappa_2(A)$ and that $\kappa_2(H) \ll \kappa_2(A)$ is possible if A is badly scaled. Note that $1 \leq \|H\|_2 \leq n$, since H is positive definite with $h_{ii} \equiv 1$.

Theorem 10.6 (Demmel, Wilkinson). *Let $A = DHD \in \mathbb{R}^{n \times n}$ be symmetric positive definite, where $D = \text{diag}(A)^{1/2}$, and suppose Cholesky factorization successfully produces a computed solution $\hat{x}$ to $Ax = b$. Then the scaled error $D(x - \hat{x})$ satisfies*

$$\frac{\|D(x - \hat{x})\|_2}{\|Dx\|_2} \leq \frac{\kappa_2(H)\epsilon}{1 - \kappa_2(H)\epsilon}, \quad (10.10)$$

where $\epsilon = n(1 - \gamma_{n+1})^{-1}\gamma_{3n+1}$.

Proof. Straightforward analysis shows that (cf. the proof of Theorem 9.4) $(A + \Delta A)\hat{x} = b$, where

$$\Delta A = \Delta A_1 + \Delta_1 \hat{R} + \hat{R}^T \Delta_2 + \Delta_1 \Delta_2,$$

with $|\Delta A_1| \leq (1 - \gamma_{n+1})^{-1}\gamma_{n+1}dd^T$ (by Theorem 10.5) and $|\Delta_1| \leq \gamma_n |\hat{R}^T|$, $|\Delta_2| \leq \gamma_n |\hat{R}|$. Scaling with D , we have

$$(H + D^{-1}\Delta AD^{-1})D\hat{x} = D^{-1}b,$$

and standard perturbation theory gives

$$\frac{\|D(x - \hat{x})\|_2}{\|Dx\|_2} \leq \frac{\kappa_2(H)\|D^{-1}\Delta AD^{-1}\|_2}{1 - \kappa_2(H)\|D^{-1}\Delta AD^{-1}\|_2}.$$

But, using (10.8) and $\|D^{-1}dd^TD^{-1}\|_2 = \|ee^T\|_2 = n$, we have

$$\begin{aligned} \|D^{-1}\Delta AD^{-1}\|_2 &\leq \frac{\gamma_{n+1}}{1 - \gamma_{n+1}} \|D^{-1}dd^TD^{-1}\|_2 \\ &\quad + (2\gamma_n + \gamma_n^2) \|D^{-1}|\hat{R}^T||\hat{R}|D^{-1}\|_2 \\ &\leq n(1 - \gamma_{n+1})^{-1}\gamma_{3n+1}, \end{aligned}$$

using Lemma 3.3, which yields the result. $\square$

Care needs to be exercised when interpreting bounds that involve scaled quantities, but in this case the interpretation is relatively easy. Suppose that H is well conditioned and $\kappa_2(D)$ is large, which represents the artificial ill conditioning that the DHD scaling is designed to clarify. The vector $Dx = H^{-1}D^{-1}b$ is likely

to have components that do not vary much in magnitude. Theorem 10.6 then guarantees that we obtain the components of Dx to good relative accuracy and this means that the components of x (which will vary greatly in magnitude) are obtained to good relative accuracy.

So far, our results have all contained the proviso that Cholesky factorization runs to completion—in other words, the results assume that the argument of the square root is always positive. Wilkinson [1236, 1968] showed that success is guaranteed if $20n^{3/2}\kappa_2(A)u \leq 1$, that is, if A is not too ill conditioned. It would be nice to replace A in this condition by H , where $A = DHD$. Justification for doing so is that Algorithm 10.2 is scale invariant, in the sense that if we scale $A \leftarrow FAF$, where F is diagonal, then R scales to RF ; moreover, if F comprises powers of the machine base, then even the rounding errors scale according to F . The following theorem gives an appropriate modification of Wilkinson's condition.

Theorem 10.7 (Demmel). *Let $A = DHD \in \mathbb{R}^{n \times n}$ be symmetric positive definite, where $D = \text{diag}(A)^{1/2}$. If $\lambda_{\min}(H) > n\gamma_{n+1}/(1 - \gamma_{n+1})$ then Cholesky factorization applied to A succeeds (barring underflow and overflow) and produces a nonsingular $\hat{R}$. If $\lambda_{\min}(H) \leq -n\gamma_{n+1}/(1 - \gamma_{n+1})$ then the computation is certain to fail.*

Proof. The proof is by induction. Consider Algorithm 10.2. The first stage obviously succeeds and gives $\hat{r}_{11} > 0$, since $a_{11} > 0$. Suppose the algorithm has successfully completed $k - 1$ stages, producing a nonsingular $\hat{R}_{k-1}$, and consider equations (10.1)–(10.3) with n replaced by k . The k th stage can be completed, but may give a pure imaginary $\hat{\beta}$ (it will if $fl(\alpha - \hat{r}^T \hat{r}) < 0$). However, in the latter event, the error analysis of Theorem 10.5 is still valid! Thus we obtain $\hat{R}_k$ satisfying $\hat{R}_k^* \hat{R}_k = A_k + \Delta A_k$, $|\Delta A_k| \leq (1 - \gamma_{k+1})^{-1} \gamma_{k+1} d_k d_k^T$, where $d_k = [a_{11}^{1/2}, \dots, a_{kk}^{1/2}]^T$. Now, with $D_k = \text{diag}(d_k)$, we have

$$\begin{aligned} \lambda_{\min}(D_k^{-1}(A_k + \Delta A_k)D_k^{-1}) &= \lambda_{\min}(H_k + D_k^{-1}\Delta A_k D_k^{-1}) \\ &\geq \lambda_{\min}(H_k) - \|D_k^{-1}\Delta A_k D_k^{-1}\|_2 \\ &\geq \lambda_{\min}(H_k) - \frac{\gamma_{k+1}}{1 - \gamma_{k+1}} \|ee^T\|_2 \\ &\geq \lambda_{\min}(H) - k \frac{\gamma_{k+1}}{1 - \gamma_{k+1}} > 0, \end{aligned}$$

using the interlacing of the eigenvalues [509, 1996, Thm. 8.1.7] and the condition of the theorem. Hence $D_k^{-1}(A_k + \Delta A_k)D_k^{-1}$ is positive definite, and therefore so is the congruent matrix $A_k + \Delta A_k$, showing that $\hat{R}_k$ must be real and nonsingular, as required to complete the induction.

If Cholesky succeeds, then, by Theorem 10.5, $D^{-1}(A + \Delta A)D^{-1}$ is positive definite and so $0 < \lambda_{\min}(H) + \|D^{-1}\Delta A D^{-1}\|_2 \leq \lambda_{\min}(H) + n(1 - \gamma_{n+1})^{-1}\gamma_{n+1}$. Hence if $\lambda_{\min}(H) \leq -n\gamma_{n+1}/(1 - \gamma_{n+1})$ then the computation must fail. $\square$

Note that, since $\|H\|_2 \geq 1$, the condition for success of Cholesky factorization can be weakened slightly to $\kappa_2(H)n\gamma_{n+1}/(1 - \gamma_{n+1}) < 1$.

10.2. Sensitivity of the Cholesky Factorization

The Cholesky factorization has perturbation bounds that are similar to those for LU factorization, but of a simpler form thanks to the positive definiteness ($\|A^{-1}\|_2$ replaces $\|U^{-1}\|_2\|L^{-1}\|_2$ in the normwise bounds).

Theorem 10.8 (Sun). *Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite with the Cholesky factorization $A = R^T R$ and let ΔA be a symmetric matrix satisfying $\|A^{-1} \Delta A\|_2 < 1$. Then $A + \Delta A$ has the Cholesky factorization $A + \Delta A = (R + \Delta R)^T (R + \Delta R)$, where*

$$\frac{\|\Delta R\|_F}{\|R\|_p} \leq 2^{-1/2} \frac{\kappa_2(A)\epsilon}{1 - \kappa_2(A)\epsilon}, \quad \epsilon = \frac{\|\Delta A\|_F}{\|A\|_p}, \quad p = 2, F.$$

Moreover, if $\rho(|\tilde{G}|) < 1$, where $\tilde{G} = (R + \Delta R)^{-T} \Delta A (R + \Delta R)^{-1}$, then

$$|\Delta R| \leq \text{triu}(|\tilde{G}|(I - |\tilde{G}|)^{-1})|R + \Delta R|,$$

where $\text{triu}(\cdot)$ denotes the upper triangular part. $\square$

Note that the Cholesky factor of $A_k = A(1:k, 1:k)$ is R_k , and $\kappa_2(A_{k+1}) \geq \kappa_2(A_k)$ by the interlacing property of the eigenvalues. Hence if A_{k+1} (and hence A) is ill conditioned but A_k is well conditioned then R_k will be relatively insensitive to perturbations in A but the remaining columns of R will be much more sensitive.

10.3. Positive Semidefinite Matrices

If A is symmetric and positive semidefinite ($x^T A x \geq 0$ for all x) then a Cholesky factorization exists, but the theory and computation are more subtle than for positive definite A .

The questions of existence and uniqueness of a Cholesky factorization are answered by the following result.

Theorem 10.9. *Let $A \in \mathbb{R}^{n \times n}$ be positive semidefinite of rank r . (a) There exists at least one upper triangular R with nonnegative diagonal elements such that $A = R^T R$. (b) There is a permutation Π such that $\Pi^T A \Pi$ has a unique Cholesky factorization, which takes the form*

$$\Pi^T A \Pi = R^T R, \quad R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix}, \quad (10.11)$$

where R_{11} is $r \times r$ upper triangular with positive diagonal elements.

Proof. (a) Let the symmetric positive semidefinite square root X of A have the QR factorization $X = QR$ with $r_{ii} \geq 0$. Then $A = X^2 = X^T X = R^T Q^T Q R = R^T R$. (b) The algorithm with pivoting described below amounts to a constructive proof. $\square$

Note that the factorization in part (a) is not in general unique. For example,

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \equiv \begin{bmatrix} 0 & 0 \\ \cos \theta & \sin \theta \end{bmatrix} \begin{bmatrix} 0 & \cos \theta \\ 0 & \sin \theta \end{bmatrix}.$$

For practical computations a factorization of the form (10.11) is needed, because this factorization so conveniently displays the rank deficiency. Such a factorization can be computed using an outer product Cholesky algorithm, comprising $r = \text{rank}(A)$ stages. At the k th stage, a rank-1 matrix is subtracted from A so as to introduce zeros into positions $k:n$ in the k th row and column. Ignoring pivoting for the moment, at the start of the k th stage we have

$$A^{(k)} = (a_{ij}^{(k)}) = A - \sum_{i=1}^{k-1} r_i r_i^T = \begin{matrix} & \begin{matrix} k-1 & n-k+1 \end{matrix} \\ \begin{matrix} k-1 \\ n-k+1 \end{matrix} & \begin{bmatrix} 0 & 0 \\ 0 & A_k \end{bmatrix} \end{matrix}, \quad (10.12)$$

where $r_i^T = [0, \dots, 0, r_{ii}, \dots, r_{in}]$. The reduction is carried one stage further by computing

$$\begin{aligned} r_{kk} &= \sqrt{a_{kk}^{(k)}}, \\ r_{kj} &= a_{kj}^{(k)} / r_{kk}, \quad j = k+1:n, \\ a_{ij}^{(k+1)} &= a_{ij}^{(k)} - r_{ki} r_{kj}, \quad i, j = k+1:n. \end{aligned}$$

Overall we have,

$$A = \sum_{i=1}^r r_i r_i^T = R^T R, \quad R^T = [r_1, \dots, r_r].$$

To avoid breakdown when $a_{kk}^{(k)}$ vanishes (or is negative because of rounding errors), pivoting is incorporated into the algorithm as follows. At the start of the k th stage an element $a_{ss}^{(k)} > 0$ ($s \geq k$) is selected as pivot, and rows and columns k and s of A_k , and the k th and s th elements of r_i , $i = 1:k-1$, are interchanged. The overall effect is to compute the decomposition (10.11), where the permutation Π takes account of all the interchanges.

The standard form of pivoting is defined by

$$s = \min \left\{ j : a_{jj}^{(k)} = \max_{k \leq i \leq n} a_{ii}^{(k)} \right\}.$$

This is equivalent to complete pivoting in GE, since A_k is positive semidefinite so its largest element lies on the diagonal (see Problem 10.1). We note for later reference that this pivoting strategy produces a matrix R that satisfies (cf. Problem 19.5)

$$r_{kk}^2 \geq \sum_{i=k}^{\min(j,r)} r_{ij}^2, \quad j = k+1:n, \quad k = 1:r. \quad (10.13)$$

It will be convenient to denote by $\text{cp}(A) := \Pi^T A \Pi$ the permuted matrix obtained from the Cholesky algorithm with complete pivoting.

10.3.1. Perturbation Theory

In this section we analyse, for a positive semidefinite matrix, the effect on the Cholesky factorization of perturbations in the matrix. This perturbation theory will be used in the error analysis of the next section.

Throughout this section A is assumed to be an $n \times n$ positive semidefinite matrix of rank r whose leading principal submatrix of order r is positive definite. For $k = 1:r$ we will write

$$A = \begin{matrix} & k & n-k \\ \begin{matrix} k \\ n-k \end{matrix} & \begin{bmatrix} A_{11} & A_{12} \\ A_{12}^T & A_{22} \end{bmatrix} \end{matrix} \quad (10.14)$$

and other matrices will be partitioned conformally.

We have the identity

$$A = \begin{matrix} & k \\ \begin{matrix} k \\ n-k \end{matrix} & \begin{bmatrix} R_{11}^T \\ R_{12}^T \end{bmatrix} \end{matrix} [R_{11}, R_{12}] + \begin{bmatrix} 0 & 0 \\ 0 & S_k(A) \end{bmatrix}, \quad (10.15)$$

where R_{11} is the Cholesky factor of A_{11} , $R_{12} = R_{11}^{-T} A_{12}$, and

$$S_k(A) = A_{22} - A_{12}^T A_{11}^{-1} A_{12}$$

is the Schur complement of A_{11} in A . Note that $S_r(A) \equiv 0$, so that for $k = r$, (10.15) is the (unique) Cholesky factorization of A . The following lemma shows how $S_k(A)$ changes when A is perturbed.

Lemma 10.10. *Let E be symmetric and assume $A_{11} + E_{11}$ is nonsingular. Then*

$$S_k(A + E) = S_k(A) + E_{22} - (E_{12}^T W + W^T E_{12}) + W^T E_{11} W + O(\|E\|^2), \quad (10.16)$$

where $W = A_{11}^{-1} A_{12}$.

Proof. We can expand

$$(A_{11} + E_{11})^{-1} = A_{11}^{-1} - A_{11}^{-1} E_{11} A_{11}^{-1} + A_{11}^{-1} E_{11} A_{11}^{-1} E_{11} A_{11}^{-1} + O(\|E_{11}\|^3).$$

The result is obtained by substituting this expansion into $S_k(A + E) = (A_{22} + E_{22}) - (A_{12} + E_{12})^T (A_{11} + E_{11})^{-1} (A_{12} + E_{12})$, and collecting terms. $\square$

Lemma 10.10 shows that the sensitivity of $S_k(A)$ to perturbations in A is governed by the matrix $W = A_{11}^{-1} A_{12}$. The question arises of whether, for a given A , the potential $\|W\|_2^2$ magnification of E indicated by (10.16) is attainable. For the no-pivoting strategy, $\Pi = I$, the answer is trivially “yes”, since we can take $E = \begin{bmatrix} \gamma I & 0 \\ 0 & 0 \end{bmatrix}$, with $|\gamma|$ small, to obtain $\|S_k(A + E) - S_k(A)\|_2 = \|W\|_2^2 \|E\|_2 + O(\|E\|_2^2)$. For complete pivoting, however, the answer is complicated by the possibility that the sequence of pivots will be different for $A + E$ than for A , in which case Lemma 10.10 is not applicable. Fortunately, a mild assumption on A is enough to rule out this technical difficulty, for small $\|E\|_2$. In the next lemma we redefine $A := \text{cp}(A)$ in order to simplify the notation.

Lemma 10.11. *Let $A := \text{cp}(A)$. Suppose that*

$$(S_i(A))_{11} > (S_i(A))_{jj}, \quad j = 2:n-i, \quad i = 0:r-1 \quad (10.17)$$

(where $S_0(A) := A$). Then, for sufficiently small $\|E\|_2$, $A + E = \text{cp}(A + E)$. For $E = \begin{bmatrix} \gamma I & 0 \\ 0 & 0 \end{bmatrix}$, with $|\gamma|$ sufficiently small,

$$\|S_k(\text{cp}(A + E)) - S_k(A)\|_2 = \|W\|_2^2 \|E\|_2 + O(\|E\|_2^2).$$

Proof. Note that since $A = \text{cp}(A)$, (10.17) simply states that there are no ties in the pivoting strategy (since $(S_i(A))_{11} \equiv a_{i+1,i+1}^{(i+1)}$ in (10.12)). Lemma 10.10 shows that $S_i(A + E) = S_i(A) + O(\|E\|_2)$, and so, in view of (10.17), for sufficiently small $\|E\|_2$ we have

$$(S_i(A + E))_{11} > (S_i(A + E))_{jj}, \quad j = 2:n-i, \quad i = 0:r-1.$$

This shows that $A + E = \text{cp}(A + E)$. The last part then follows from Lemma 10.10. $\square$

We now examine the quantity $\|W\|_2 = \|A_{11}^{-1}A_{12}\|_2$. We show first that $\|W\|_2$ can be bounded in terms of the square root of the condition number of A_{11} .

Lemma 10.12. *If A , partitioned as in (10.14), is symmetric positive definite and A_{11} is positive definite then $\|A_{11}^{-1}A_{12}\|_2 \leq \sqrt{\|A_{11}^{-1}\|_2 \|A_{22}\|_2}$.*

Proof. Write $A_{11}^{-1}A_{12} = A_{11}^{-1/2}A_{11}^{-1/2}A_{12}$ and use $\|A_{11}^{-1/2}\|_2 = \|A_{11}^{-1}\|_2^{1/2}$, together with $\|A_{11}^{-1/2}A_{12}\|_2 = \|A_{12}^T A_{11}^{-1}A_{12}\|_2^{1/2} \leq \|A_{22}\|_2^{1/2}$, which follows from the fact that the Schur complement $A_{22} - A_{12}^T A_{11}^{-1}A_{12}$ is positive semidefinite. $\square$

Note that, by the arithmetic-geometric mean inequality $\sqrt{xy} \leq (x + y)/2$ ($x, y \geq 0$), we also have, from Lemma 10.12, $\|A_{11}^{-1}A_{12}\|_2 \leq (\|A_{11}^{-1}\|_2 + \|A_{22}\|_2)/2$.

The inequality of Lemma 10.12 is an equality for the positive semidefinite matrix

$$A = \begin{bmatrix} \alpha I_{k,k} & I_{k,n-k} \\ I_{n-k,k} & \alpha^{-1} I_{n-k,n-k} \end{bmatrix}, \quad \alpha > 0, \quad (10.18)$$

where $I_{p,q}$ is the $p \times q$ identity matrix. This example shows that $\|W\|_2$ can be arbitrarily large. However, for $A := \text{cp}(A)$, $\|W\|_2$ can be bounded solely in terms of n and k . The essence of the proof, in the next lemma, is that large elements in A_{11}^{-1} are countered by small elements in A_{12} . Hereafter we set $k = r$, the value of interest in the following sections.

Lemma 10.13. *Let $A := \text{cp}(A)$ and set $k = r$. Then*

$$\|A_{11}^{-1}A_{12}\|_{2,F} \leq \sqrt{\frac{1}{3}(n-r)(4r-1)}. \quad (10.19)$$

There is a parametrized family of rank- r matrices $A(\theta) = \text{cp}(A(\theta))$, $\theta \in (0, \frac{\pi}{2}]$, for which

$$\|A_{11}(\theta)^{-1}A_{12}(\theta)\|_{2,F} \rightarrow \sqrt{\frac{1}{3}(n-r)(4r-1)} \quad \text{as } \theta \rightarrow 0.$$

Proof. The proof is a straightforward computation. The matrix $A(\theta) := R(\theta)^T R(\theta)$, where

$$R(\theta) = \text{diag}(1, s, \dots, s^{r-1}) \begin{bmatrix} 1 & -c & -c & \dots & -c & -c & \dots & -c \\ & 1 & -c & \dots & -c & -c & \dots & -c \\ & & 1 & & \vdots & \vdots & & \vdots \\ & & & \ddots & \vdots & \vdots & & \vdots \\ & & & & 1 & -c & \dots & -c \end{bmatrix} \in \mathbb{R}^{r \times n}, \quad (10.20)$$

with $c = \cos \theta, s = \sin \theta$. This is the $r \times n$ version of Kahan's matrix (8.11). R satisfies the inequalities (10.13) (as equalities) and so $A(\theta) = \text{cp}(A(\theta))$. $\square$

We conclude this section with a “worst-case” example for the Cholesky factorization with complete pivoting. Let $U(\theta) = \text{diag}(r, r-1, \dots, 1)R(\theta)$, where $R(\theta)$ is given by (10.20), and define the rank- r matrix $C(\theta) = U(\theta)^T U(\theta)$. Then $C(\theta)$ satisfies the conditions of Lemma 10.11. Also,

$$\begin{aligned} \|W\|_2 &= \|C_{11}(\theta)^{-1} C_{12}(\theta)\|_2 = \|U_{11}(\theta)^{-1} U_{12}(\theta)\|_2 = \|R_{11}(\theta)^{-1} R_{12}(\theta)\|_2 \\ &\rightarrow \sqrt{\frac{1}{3}(n-r)(4^r-1)} \quad \text{as } \theta \rightarrow 0. \end{aligned}$$

Thus, from Lemma 10.11, for $E = \begin{bmatrix} \gamma^T & 0 \\ 0 & 0 \end{bmatrix}$, with $|\gamma|$ and θ sufficiently small,

$$\|S_r(\text{cp}(C(\theta) + E))\|_2 \approx \frac{1}{3}(n-r)(4^r-1)\|E\|_2.$$

This example can be interpreted as saying that in exact arithmetic the residual after an r -stage Cholesky factorization of a semidefinite matrix A can overestimate the distance of A from the rank- r semidefinite matrices by a factor as large as $(n-r)(4^r-1)/3$.

10.3.2. Error Analysis

In this section we present a backward error analysis for the Cholesky factorization of a positive semidefinite matrix. An important consideration is that a matrix of floating point numbers is very unlikely to be “exactly” positive semidefinite; errors in forming or storing A will almost certainly render the smallest eigenvalue nonzero, and possibly negative. Therefore error analysis for a rank- r positive semidefinite matrix may appear, at first sight, to be of limited applicability. One way around this difficulty is to state results for $A = \bar{A} + \Delta A$, where A is the matrix stored on the computer, $\bar{A}$ is positive semidefinite of rank r , and ΔA is a perturbation, which could represent the rounding errors in storing A , for example. However, if the perturbation ΔA is no larger than the backward error for the Cholesky factorization, then this extra formalism can be avoided by thinking of ΔA as being included in the backward error matrix. Hence for simplicity, we frame the error analysis for a positive semidefinite A .

The analysis makes no assumptions about the pivoting strategy, so A should be thought of as the pre-permuted matrix $\Pi^T A \Pi$.

Theorem 10.14. *Let A be an $n \times n$ symmetric positive semidefinite matrix of floating point numbers of rank $r \leq n$. Assume that $A_{11} = A(1:r, 1:r)$ is positive definite with*

$$\lambda_{\min}(H_{11}) > r\gamma_{r+1}/(1 - \gamma_{r+1}), \quad (10.21)$$

where $A_{11} = D_{11}H_{11}D_{11}$ and $D_{11} = \text{diag}(A_{11})^{1/2}$. Then, in floating point arithmetic, the Cholesky algorithm applied to A successfully completes r stages (barring underflow and overflow), and the computed $r \times n$ Cholesky factor $\hat{R}_r$ satisfies

$$\|A - \hat{R}_r^T \hat{R}_r\|_2 \leq 2r\gamma_{r+1}\|A\|_2(\|W\|_2 + 1)^2 + O(u^2), \quad (10.22)$$

where $W = A_{11}^{-1}A_{12}$.

Proof. First, note that condition (10.21) guarantees successful completion of the first r stages of the algorithm by Theorem 10.7.

Analysis very similar to that leading to Theorem 10.3 shows that

$$A + E = \hat{R}_r^T \hat{R}_r + \hat{A}^{(r+1)}, \quad (10.23)$$

where

$$\hat{R}_r = \begin{matrix} & r & n-r \\ r & \begin{bmatrix} \hat{R}_{11} & \hat{R}_{12} \end{bmatrix} \end{matrix}, \quad \hat{A}^{(r+1)} = \begin{matrix} & r & n-r \\ n-r & \begin{bmatrix} 0 & 0 \\ 0 & \hat{S}_{r+1} \end{bmatrix} \end{matrix},$$

and

$$|E| \leq \gamma_{r+1}(|\hat{R}_r^T| |\hat{R}_r| + |\hat{A}^{(r+1)}|). \quad (10.24)$$

Taking norms in (10.24) and using the inequality $\| |B| \|_2 \leq \sqrt{\text{rank}(B)} \|B\|_2$ from Lemma 6.6, we obtain

$$\begin{aligned} \|E\|_2 &\leq \gamma_{r+1}(r\|\hat{R}_r^T\|_2\|\hat{R}_r\|_2 + \sqrt{n-r}\|\hat{A}^{(r+1)}\|_2) \\ &= \gamma_{r+1}(r\|\hat{R}_r^T \hat{R}_r\|_2 + \sqrt{n-r}\|\hat{A}^{(r+1)}\|_2) \\ &= \gamma_{r+1}(r\|A + E - \hat{A}^{(r+1)}\|_2 + \sqrt{n-r}\|\hat{A}^{(r+1)}\|_2) \\ &\leq \gamma_{r+1}(r\|A\|_2 + r\|E\|_2 + n\|\hat{A}^{(r+1)}\|_2), \end{aligned}$$

which implies

$$\|E\|_2 \leq \frac{\gamma_{r+1}}{1 - r\gamma_{r+1}}(r\|A\|_2 + n\|\hat{A}^{(r+1)}\|_2). \quad (10.25)$$

Our aim is to obtain an a priori bound for $\|A - \hat{R}_r^T \hat{R}_r\|_2$. It is clear from (10.23)–(10.25) that to do this we have only to bound $\|\hat{A}^{(r+1)}\|_2$. To this end, we interpret (10.23) in such a way that the perturbation theory of §10.3.1 may be applied.

Equation (10.23) shows that $\hat{S}_{r+1}$ is the true Schur complement for the matrix $A + E$ and that $A_{11} + E_{11} = \hat{R}_{11}^T \hat{R}_{11}$ is positive definite. Hence we can use Lemma 10.10 to deduce that

$$\begin{aligned} \|\hat{A}^{(r+1)}\|_2 &= \|\hat{S}_{r+1}\|_2 \leq \|E_{22}\|_2 + 2\|E_{12}\|_2\|W\|_2 + \|W\|_2^2\|E_{11}\|_2 + O(u^2) \\ &\leq \|E\|_2(\|W\|_2 + 1)^2 + O(u^2). \end{aligned}$$

Substituting from (10.25) we find that

$$\|\hat{A}^{(r+1)}\|_2 \leq r\gamma_{r+1}\|A\|_2(\|W\|_2 + 1)^2 + O(u^2).$$

Finally, using (10.23) and (10.25), we obtain

$$\|A - \hat{R}_r^T \hat{R}_r\|_2 \leq 2r\gamma_{r+1}\|A\|_2(\|W\|_2 + 1)^2 + O(u^2). \quad \square$$

Theorem 10.14 is just about the best result that could have been expected, because the bound (10.22) is essentially the same as the bound obtained on taking norms in Lemma 10.10. In other words, (10.22) simply reflects the inherent mathematical sensitivity of $A - R^T R$ to small perturbations in A .

We turn now to the issue of stability. Ideally, for A as defined in Theorem 10.14, the computed Cholesky factor $\hat{R}_r$ produced after r stages of the algorithm would satisfy

$$\|A - \hat{R}_r^T \hat{R}_r\|_2 \leq c_n u \|A\|_2,$$

where c_n is a modest constant. Theorem 10.14 shows that stability depends on the size of $\|W\|_2 = \|A_{11}^{-1} A_{12}\|_2$ (to the extent that $\|W\|_2$ appears in a realistic bound for the backward error).

If no form of pivoting is used then $\|W\|_2$ can be arbitrarily large for fixed n (see (10.18)) and the Cholesky algorithm must in this case be classed as unstable. But for complete pivoting we have from Lemma 10.13 the upper bound $\|W\|_2 \leq (\frac{1}{3}(n-r)(4^r-1))^{1/2}$. Thus the Cholesky algorithm with complete pivoting is stable if r is small, but stability cannot be guaranteed, and seems unlikely in practice, if $\|W\|_2$ (and hence, necessarily, r and n) is large.

Numerical experiments show that $\|W\|_2$ is almost always small in practice (typically less than 10) [588, 1990]. However, it is easy to construct examples where $\|W\|_2$ is large. For example, if R is a Cholesky factor of A from complete pivoting then let $C = M(R)^T M(R)$, where $M(R)$ is the comparison matrix; C will usually have a much larger value of $\|W\|_2$ than A .

An important practical issue is when to terminate the Cholesky factorization of a semidefinite matrix. The LINPACK routine xCHDC proceeds with the factorization until a nonpositive pivot is encountered, that is, up to and including stage $k-1$, where k is the smallest integer for which

$$\hat{a}_{ii}^{(k)} \leq 0, \quad i = k:n. \quad (10.26)$$

Usually $k > r+1$, due to the effect of rounding errors.

A more sophisticated termination criterion is to stop as soon as

$$\|\hat{S}_k\| \leq \epsilon \|A\| \quad \text{or} \quad \hat{a}_{ii}^{(k)} \leq 0, \quad i = k:n, \quad (10.27)$$

for some readily computed norm and a suitable tolerance ϵ . This criterion terminates as soon as a stable factorization is achieved, avoiding unnecessary work in eliminating negligible elements in the computed Schur complement $\hat{S}_k$. Note that $\|\hat{S}_k\|$ is indeed a reliable order-of-magnitude estimate of the true residual, since

$\widehat{S}_k$ is the only nonzero block of $\widehat{A}^{(k)}$ and, by (10.23) and (10.25), $A - \widehat{R}_{k-1}^T \widehat{R}_{k-1} = \widehat{A}^{(k)} - E$ with $\|E\| = O(u)(\|A\| + \|\widehat{A}^{(k)}\|)$.

Another possible stopping criterion is

$$\max_{k \leq i \leq n} \widehat{a}_{ii}^{(k)} \leq \epsilon \widehat{a}_{11}^{(1)}. \quad (10.28)$$

This is related to (10.27) in that if A (pre-permuted) and $\widehat{A}_k$ are positive semidefinite then $a_{11}^{(1)} = \max_{i,j} |a_{ij}| \approx \|A\|_2$, and similarly $\max_{k \leq i \leq n} \widehat{a}_{ii}^{(k)} \approx \|\widehat{S}_k\|_2$. Note that (10.28) bounds $\kappa_2(\widehat{R}_{k-1})$, since if (10.28) holds first at the k th stage then, using Theorem 8.14,

$$\frac{\kappa_2(\widehat{R}_{k-1})}{(k-1)^{1/2} 2^{k-2}} \leq \frac{|\widehat{r}_{11}|}{|\widehat{r}_{k-1,k-1}|} = \left(\frac{\widehat{a}_{11}^{(1)}}{\widehat{a}_{k-1,k-1}^{(k-1)}} \right)^{1/2} \leq \epsilon^{-1/2}.$$

Practical experience shows that the criteria (10.27) and (10.28) with $\epsilon = nu$ both work well, and much more effectively than (10.26) [588, 1990]. We favour (10.28) because of its negligible cost.

10.4. Matrices with Positive Definite Symmetric Part

Any matrix $A \in \mathbb{R}^{n \times n}$ can be written

$$A \equiv A_S + A_K, \quad A_S = (A + A^T)/2, \quad A_K = (A - A^T)/2,$$

where A_S is the *symmetric part* of A and A_K is the *skew-symmetric part* of A . A number of results for symmetric positive definite matrices can be extended to matrices with positive definite symmetric part or, equivalently, to matrices for which $x^T A x > 0$ for all $x \neq 0$; these matrices are sometimes called nonsymmetric positive definite matrices.

A matrix with positive definite symmetric part clearly has nonsingular leading principal submatrices, and so has an LU factorization, $A = LU$. It can even be shown that pivots u_{ii} are positive. However, there is no guarantee that the factorization is stable without pivoting, as the example $\begin{bmatrix} \epsilon & 1 \\ -1 & \epsilon \end{bmatrix}$ shows. The standard error analysis for LU factorization applies (Theorems 9.3 and 9.4), and so the question is whether $|\widehat{L}||\widehat{U}|$ can be suitably bounded. Golub and Van Loan [508, 1979] show that, for the exact LU factors,

$$\| |L||U| \|_F \leq n \|A_S + A_K^T A_S^{-1} A_K\|_2. \quad (10.29)$$

Let $\chi(A) = \|A_S + A_K^T A_S^{-1} A_K\|_2 \|A_S^{-1}\|_2$, which is just $\kappa_2(A)$ when A is symmetric. Mathias [820, 1992] shows that $\| |\widehat{L}||\widehat{U}| \|_F$ (involving now the computed LU factors) is at most a factor $1 + 30un^{3/2}\chi(A)$ times larger than the upper bound in (10.29), and that the LU factorization (without pivoting) succeeds if $24n^{3/2}\chi(A)u \leq 1$.

These results show that it is safe not to pivot provided that the symmetric part of A is not too ill conditioned relative to the norm of the skew-symmetric part.

If A is symmetric ($A_K = 0$) then we recover the results for symmetric positive definite matrices.

For complex matrices A with positive definite Hermitian part $(A + A^*)/2$, or equivalently, $\operatorname{Re} x^*Ax > 0$ for all nonzero $x \in \mathbb{C}$, the results above can be extended. A particular class of such matrices is those of the form

$$A = B + iC, \quad B, C \in \mathbb{R}^{n \times n} \text{ both symmetric positive definite.} \quad (10.30)$$

These matrices arise in computational electrodynamics [1183, 2001]. They are nonsingular and have an LU factorization. George, Ikramov, and Kucherov [477, 2002] show that the growth factor $\rho_n < 3$, and so LU factorization without pivoting is perfectly normwise backward stable. An even smaller bound of about 1.28 holds when $B = I$, as shown in [660, 2000].

10.5. Notes and References

André-Louis Cholesky (1875–1918) was a French military officer involved in geodesy and surveying in Crete and North Africa. His work was published posthumously on his behalf by Benoit [103, 1924]. Biographies of Cholesky are given by Brezinski [164, 1996] and in [28, 1922]. In some books his name is misspelled “Choleski”. Discussions about Cholesky—in particular, concerning the pronunciation of his name!—can be found in the electronic mail magazine NA-Digest, volume 90, 1990, issues 7, 8, 10–12, and 24.

The properties of the Cholesky factorization are intimately associated with the properties of the Schur complement, as is apparent from some of the proofs in this chapter. The same is true for GE in general. An excellent survey of the Schur complement, containing historical comments, theory, and applications, is given by Cottle [274, 1974].

For results on the Cholesky factorization in Hilbert space see Power [952, 1986].

A book by George and Liu [478, 1981] is devoted to the many practical issues in the implementation of Cholesky factorization for the solution of sparse symmetric positive definite systems.

There is no floating point error analysis of Cholesky factorization in Wilkinson’s books, but he gives a detailed analysis in [1236, 1968], showing that $\hat{R}^T \hat{R} = A + E$, with $\|E\|_2 \leq 2.5n^{3/2}u\|A\|_2$. It is unfortunate that this paper is in a rather inaccessible proceedings, because it is a model of how to phrase and interpret an error analysis. Meinguet [840, 1983] and Sun [1104, 1992] give componentwise backward error bounds similar to those in Theorems 10.3 and 10.4. Kielbasiński [733, 1987] reworks Wilkinson’s analysis to improve the constant.

The fact that $\kappa_2(H)$ can replace the potentially much larger $\kappa_2(A)$ in the forward error bound for the Cholesky method was stated informally and without proof by Wilkinson [1236, 1968, p. 638]. Demmel [311, 1989] made this observation precise and explored its implications; Theorems 10.5, 10.6, and 10.7 are taken from [311, 1989].

The bounds in Theorem 10.8 are from Sun [1102, 1991], [1103, 1992]. Similar bounds are given by Stewart [1068, 1977], [1075, 1993], Barlund [81, 1991], and Sun [1104, 1992]. More refined perturbation bounds are derived and explored by Chang, Paige, and Stewart [220, 1996] and Stewart [1078, 1997]. Perturbation

results of a different flavour, including one for structured perturbations of the form of ΔA in Theorem 10.5, are given by Drmač, Omladič, and Veselić [356, 1994].

The perturbation and error analysis of §10.3 for semidefinite matrices is from Higham [588, 1990], wherein a perturbation result for the QR factorization with column pivoting is also given. For an application in optimization that makes use of Cholesky factorization with complete pivoting and the analysis of §10.3.1 see Forsgren, Gill, and Murray [420, 1995]. An extension of Theorem 10.14 to Toeplitz matrices is given by Stewart [1084, 1997], who shows that for these matrices pivoting is not needed in order for $\|W\|$ to be bounded in terms of n and r only.

Fletcher and Powell [418, 1974] describe several algorithms for updating an LDL^T factorization of a symmetric positive definite A when A is modified by a rank-1 matrix. They give detailed componentwise error analysis for some of the methods.

An excellent way to test whether a given symmetric matrix A is positive (semi)definite is to attempt to compute a Cholesky factorization. This test is less expensive than computing the eigenvalues and is numerically stable. Indeed, if the answer “yes” is obtained, it is the right answer for a nearby matrix, whereas if the answer is “no” then A must be close to an indefinite matrix. See Higham [583, 1988] for an application of this definiteness test. An algorithm for testing the definiteness of a Toeplitz matrix is developed by Cybenko and Van Loan [287, 1986], as part of a more complicated algorithm. According to Kerr [730, 1990], misconceptions of what is a sufficient condition for a matrix to be positive (semi)definite are rife in the engineering literature (for example, that it suffices to check the definiteness of all 2×2 submatrices). See also Problem 10.8. For some results on definiteness tests for Toeplitz matrices, see Makhoul [807, 1991].

That pivoting is not necessary for the class of complex symmetric matrices (10.30) was first noted by Higham [610, 1998], who also investigates the behaviour of block LDL^T factorization with the Bunch–Kaufman pivoting strategy (see §11.1) when applied to such matrices. The proof of the growth factor bound in [610, 1998] is incorrect; the subsequent proof that $\rho_n < 3$ in [477, 2002] is quite lengthy, and obtaining a sharp bound for the growth factor is an open problem (see Problem 10.12).

10.5.1. LAPACK

Driver routines `xPOSV` (simple) and `xPOSVX` (expert) use the Cholesky factorization to solve a symmetric (or Hermitian) positive definite system of linear equations with multiple right-hand sides. (There are corresponding routines for packed storage, in which one triangle of the matrix is stored in a one-dimensional array: `PP` replaces `P0` in the names.) The expert driver incorporates iterative refinement, condition estimation, and backward and forward error estimation and has an option to scale the system $AX = B$ to $(D^{-1}AD^{-1})DX = D^{-1}B$, where $D = \text{diag}(a_{ii}^{1/2})$. Modulo the rounding errors in computing and applying the scaling, the scaling has no effect on the accuracy of the solution prior to iterative refinement, in view of Theorem 10.6. The Cholesky factorization is computed by the routine `xPOTRF`, which uses a partitioned algorithm that computes R a block row at a time. The drivers `xPTSV` and `xPTSVX` for symmetric positive definite tridiagonal matrices use

LDL^T factorization. LAPACK does not currently contain a routine for Cholesky factorization of a positive semidefinite matrix, but there is such a routine in LINPACK (xCHDC).

Problems

10.1. Show that if $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite then

$$|a_{ij}| < \sqrt{a_{ii}a_{jj}} \quad \text{for all } i \neq j.$$

What does this statement imply about $\max_{i,j} |a_{ij}|$?

10.2. If A is a symmetric positive definite matrix, how would you compute $x^T A^{-1}x$?

10.3. Let $y = (c - \sum_{i=1}^{k-1} a_i b_i)^{1/2}$ be evaluated in floating point arithmetic in any order. Show that

$$\hat{y}^2(1 + \theta_{k+1}) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_{k-1}^{(i)}),$$

where $|\theta_{k-1}^{(i)}| \leq \gamma_{k-1}$ for all i , and $|\theta_{k+1}| \leq \gamma_{k+1}$.

10.4. Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite. Show that the reduced submatrix B of order $n - 1$ at the end of the first stage of GE is also symmetric positive definite. Deduce that $0 < a_{kk}^{(k)} \leq a_{kk}^{(k-1)} \leq \cdots \leq a_{kk}^{(1)} = a_{kk}$ and hence that the growth factor $\rho_n = 1$.

10.5. Show that the backward error result (10.6) for the solution of a symmetric positive definite linear system by Cholesky factorization implies

$$(A + \Delta A)\hat{x} = b, \quad \|\Delta A\|_M \leq \gamma_{3n+1}(1 - \gamma_{n+1})^{-1} \|A\|_M,$$

where $\|A\|_M = \max_{i,j} |a_{ij}|$ (which is not a consistent matrix norm—see §6.2). The significance of this result is that the bound for $\|\Delta A\|_M/\|A\|_M$ contains a linear polynomial in n , rather than the quadratic that appears for the 2-norm in (10.7).

10.6. Let $A = \text{cp}(A) \in \mathbb{R}^{n \times n}$ be positive semidefinite of rank r and suppose it has the Cholesky factorization (10.11) with $\Pi = I$. Show that $Z = [W, -I]^T$ is a basis for the null space of A , where $W = A_{11}^{-1}A_{12}$.

10.7. Prove that (10.13) holds for the Cholesky decomposition with complete pivoting.

10.8. Give an example of a symmetric matrix $A \in \mathbb{R}^{n \times n}$ for which the leading principal submatrices A_k satisfy $\det(A_k) \geq 0$, $k = 1:n$, but A is not positive semidefinite (recall that $\det(A_k) > 0$, $k = 1:n$, implies that A is positive definite). State a condition on the minors of A that is both necessary and sufficient for positive semidefiniteness.

10.9. Suppose the outer product Cholesky factorization algorithm terminates at the $(k+1)$ st stage (see (10.15)), with a negative pivot in the $(k+1, k+1)$ position. Show how to construct a direction of negative curvature for A (a vector p such that $p^T A p < 0$).

10.10. What is wrong with the following argument? A positive semidefinite matrix is the limit of a positive definite one as the smallest eigenvalue tends to zero. Theorem 10.3 shows that Cholesky factorization is stable for a positive definite matrix, and therefore, by continuity, it must be stable for a positive semidefinite matrix, implying that Theorem 10.14 is unnecessarily weak (since $\|W\|_2$ can be large).

10.11. Let

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{C}^{n \times n}$$

have positive definite Hermitian part. Show that the Schur complement $S = A_{22} - A_{21}^* A_{11}^{-1} A_{12}$ also has positive definite Hermitian part. In other words, show that GE preserves positive definiteness.

10.12. (RESEARCH PROBLEM) Investigate the sharpness of the bound $\rho_n < 3$ of [477, 2002] for matrices of the form (10.30).

Chapter 11

Symmetric Indefinite and Skew-Symmetric Systems

*Unfortunately, there is no stable scheme exactly analogous to
Gaussian elimination with partial pivoting;
one cannot construct an algorithm for which
there is a bound on the element growth of the sequence $A^{(k)}$
when at each stage only one column of $A^{(k)}$ is examined.*

— JAMES R. BUNCH and LINDA KAUFMAN,
*Some Stable Methods for Calculating Inertia and
Solving Symmetric Linear Systems* (1977)

*The comparison count is much less ...
than for the complete pivoting scheme,
and in practice this fact has had a
much larger impact than originally anticipated,
sometimes cutting the execution time by about 40%.*

— JAMES R. BUNCH and LINDA KAUFMAN,
*Some Stable Methods for Calculating Inertia and
Solving Symmetric Linear Systems* (1977)

*This fixed inertia property is why skew-symmetric matrices are
easier to decompose than symmetric indefinite matrices.*

— JAMES R. BUNCH, *A Note on the Stable Decomposition of
Skew-Symmetric Matrices* (1982)

A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is indefinite if $(x^T A x)(y^T A y) < 0$ for some $x, y \in \mathbb{R}^n$, or, equivalently, if A has both positive and negative eigenvalues. Linear systems with symmetric indefinite coefficient matrices arise in many applications, including least squares problems, optimization, and discretized incompressible Navier–Stokes equations. How can we solve $Ax = b$ efficiently?

Gaussian elimination with partial pivoting (GEPP) can be used to compute the factorization $PA = LU$, but it does not take advantage of the symmetry to reduce the cost and storage. We might try to construct a factorization $A = LDL^T$, where L is unit lower triangular and D is diagonal. But this factorization may not exist, even if symmetric pivoting is allowed, and if it does exist its computation may be unstable. For example, consider

$$A \equiv PAP^T = \begin{bmatrix} \epsilon & 1 \\ 1 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 0 \\ 0 & \epsilon - 1/\epsilon \end{bmatrix} \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}.$$

There is arbitrarily large element growth for $0 < \epsilon \ll 1$, and the factorization does not exist for $\epsilon = 0$.

For solving dense symmetric indefinite linear systems $Ax = b$, two types of factorization are used. The first is the block LDL^T factorization (or symmetric indefinite factorization)

$$PAP^T = LDL^T, \quad (11.1)$$

where P is a permutation matrix, L is unit lower triangular, and D is block diagonal with diagonal blocks of dimension 1 or 2. This factorization is essentially a symmetric block form of GE, with symmetric pivoting. Note that by Sylvester's inertia theorem, A and D have the same inertia¹⁰, which is easily determined from D (see Problem 11.2).

The second factorization is that produced by Aasen's method,

$$PAP^T = LTL^T,$$

where P is a permutation matrix, L is unit lower triangular with first column e_1 , and T is tridiagonal. Aasen's factorization is much less widely used than block LDL^T factorization, but it is mathematically interesting. We describe both factorizations in this chapter.

A matrix $A \in \mathbb{R}^{n \times n}$ is skew-symmetric if $A^T = -A$. In the final section of this chapter we describe a form of block LDL^T factorization specialized to skew-symmetric matrices.

11.1. Block LDL^T Factorization for Symmetric Matrices

If the symmetric matrix $A \in \mathbb{R}^{n \times n}$ is nonzero, we can find a permutation Π and an integer $s = 1$ or 2 so that

$$\Pi A \Pi^T = \begin{matrix} & s & n-s \\ \begin{matrix} s \\ n-s \end{matrix} & \begin{bmatrix} E & C^T \\ C & B \end{bmatrix} \end{matrix}, \quad (11.2)$$

¹⁰The inertia of a symmetric matrix is an ordered triple $\{i_+, i_-, i_0\}$, where i_+ is the number of positive eigenvalues, i_- the number of negative eigenvalues, and i_0 the number of zero eigenvalues.

with E nonsingular. Having chosen such a Π we can factorize

$$\Pi A \Pi^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & B - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}. \quad (11.3)$$

Repeating this process recursively on the $(n-s) \times (n-s)$ Schur complement

$$\tilde{A} = B - CE^{-1}C^T$$

yields the factorization (11.1). This factorization method is sometimes called the diagonal pivoting method, and it costs $n^3/3$ operations (the same cost as Cholesky factorization of a positive definite matrix) plus the cost of determining the permutations Π . It can be thought of as a generalization of Lagrange's method for reducing a quadratic form to diagonal form (devised by Lagrange in 1759 and rediscovered by Gauss in 1823) [857, 1961, p. 371].

We describe three different strategies for choosing the permutations.

11.1.1. Complete Pivoting

Bunch and Parlett [183, 1971] devised the following strategy for choosing Π . It suffices to describe the interchanges for the first stage of the factorization.

Algorithm 11.1 (Bunch–Parlett complete pivoting strategy). This algorithm determines the pivot for the first stage of the symmetric indefinite factorization applied to a symmetric matrix $A \in \mathbb{R}^{n \times n}$ using the complete pivoting strategy of Bunch and Parlett.

$\alpha = (1 + \sqrt{17})/8$ (≈ 0.64)
 $\mu_0 = \max_{i,j} |a_{ij}| =: |a_{pq}|$ ($q \geq p$), $\mu_1 = \max_i |a_{ii}| =: |a_{rr}|$
 if $\mu_1 \geq \alpha \mu_0$
 Use a_{rr} as a 1×1 pivot ($s = 1$, Π swaps rows and columns 1 and r).
 else
 Use $\begin{bmatrix} a_{pp} & a_{qp} \\ a_{qp} & a_{qq} \end{bmatrix}$ as a 2×2 pivot ($s = 2$, Π swaps rows and columns 1 and p , and 2 and q , in that order).
 end

Note that μ_1 is the best 1×1 pivot under symmetric permutations and μ_0 is the pivot that would be chosen by GE with complete pivoting. This strategy says “as long as there is a diagonal pivot element not much smaller than the complete pivot, choose it as a 1×1 pivot”, that is, “choose a 1×1 pivot whenever possible”. If the strategy dictates the use of a 2×2 pivot then that pivot E is indefinite (see Problem 11.2).

The parameter α is derived by minimizing a bound on the element growth. For the following analysis we assume that the interchanges have already been done. If $s = 1$ then

$$\tilde{a}_{ij} = b_{ij} - c_{i1} \frac{1}{e_{11}} c_{1j} \quad \Rightarrow \quad |\tilde{a}_{ij}| \leq \mu_0 + \frac{\mu_0^2}{\mu_1} \leq \left(1 + \frac{1}{\alpha}\right) \mu_0.$$

Now consider the case $s = 2$. The (i, j) element of the Schur complement $\tilde{A} = B - CE^{-1}C^T$ is

$$\tilde{a}_{ij} = b_{ij} - [c_{i1} \ c_{i2}] E^{-1} \begin{bmatrix} c_{j1} \\ c_{j2} \end{bmatrix}. \quad (11.4)$$

Now

$$E^{-1} = \begin{bmatrix} e_{11} & e_{12} \\ e_{21} & e_{22} \end{bmatrix}^{-1} = \frac{1}{\det(E)} \begin{bmatrix} e_{22} & -e_{12} \\ -e_{21} & e_{11} \end{bmatrix}$$

and, using the symmetry of E ,

$$\det(E) = e_{11}e_{22} - e_{21}^2 = e_{11}e_{22} - \mu_0^2 \leq \mu_1^2 - \mu_0^2 \leq (\alpha^2 - 1)\mu_0^2.$$

Assuming $\alpha \in (0, 1)$, we have $|\det(E)| \geq (1 - \alpha^2)\mu_0^2$. Thus

$$|E^{-1}| \leq \frac{1}{(1 - \alpha^2)\mu_0} \begin{bmatrix} \alpha & 1 \\ 1 & \alpha \end{bmatrix}.$$

Since $|c_{ij}| \leq \mu_0$, we obtain from (11.4)

$$|\tilde{a}_{ij}| \leq \mu_0 + \frac{2(1 + \alpha)\mu_0^2}{(1 - \alpha^2)\mu_0} = \left(1 + \frac{2}{1 - \alpha}\right) \mu_0.$$

To determine α we equate the maximum growth for two $s = 1$ steps with that for one $s = 2$ step:

$$\left(1 + \frac{1}{\alpha}\right)^2 = 1 + \frac{2}{1 - \alpha},$$

which reduces to the quadratic equation $4\alpha^2 - \alpha - 1 = 0$. We require the positive root

$$\alpha = \frac{1 + \sqrt{17}}{8} \approx 0.64.$$

The analysis guarantees the growth factor bound $\rho_n \leq (1 + \alpha^{-1})^{n-1} = (2.57)^{n-1}$, where ρ_n is defined in the same way as for GE. This bound is pessimistic, however; a much more detailed analysis by Bunch [175, 1971] shows that the growth factor is no more than $3.07(n - 1)^{0.446}$ times larger than the bound (9.14) for LU factorization with complete pivoting—a very satisfactory result. Strictly speaking, bounding the growth factor bounds only $\|D\|$, not $\|L\|$. But it is easy to show that for $s = 1$ and 2 no element of CE^{-1} exceeds $\max\{1/\alpha, 1/(1 - \alpha)\}$ in absolute value, and so $\|L\|$ is bounded independently of A .

Since complete pivoting requires the whole active submatrix to be searched at each stage, it needs up to $n^3/6$ comparisons, and so the method is rather expensive.

11.1.2. Partial Pivoting

Bunch and Kaufman [181, 1977] devised a pivoting strategy that requires only $O(n^2)$ comparisons. At each stage it searches at most two columns and so is analogous to partial pivoting for LU factorization. The strategy contains several logical tests. As before, we describe the pivoting for the first stage only. Recall that s denotes the size of the pivot block.

Algorithm 11.2 (Bunch–Kaufman partial pivoting strategy). This algorithm determines the pivot for the first stage of the symmetric indefinite factorization applied to a symmetric matrix $A \in \mathbb{R}^{n \times n}$ using the partial pivoting strategy of Bunch and Kaufman.

- $\alpha = (1 + \sqrt{17})/8$ (≈ 0.64)
 $\omega_1 =$ maximum magnitude of any subdiagonal entry in column 1.
 If $\omega_1 = 0$ there is nothing to do on this stage of the factorization.
 if $|a_{11}| \geq \alpha\omega_1$
 (1) Use a_{11} as a 1×1 pivot ($s = 1$, $\Pi = I$).
 else
 $r =$ row index of first (subdiagonal) entry of maximum magnitude in column 1.
 $\omega_r =$ maximum magnitude of any off-diagonal entry in column r .
 if $|a_{11}|\omega_r \geq \alpha\omega_1^2$
 (2) Use a_{11} as a 1×1 pivot ($s = 1$, $\Pi = I$).
 else if $|a_{rr}| \geq \alpha\omega_r$
 (3) Use a_{rr} as a 1×1 pivot ($s = 1$, Π swaps rows and columns 1 and r).
 else
 (4) Use $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$ as a 2×2 pivot ($s = 2$, Π swaps rows and columns 2 and r).
 end
 end
 end

To understand the Bunch–Kaufman pivoting strategy it helps to consider the matrix

$$\begin{bmatrix} a_{11} & \dots & a_{r1}(\omega_1) & \dots & \dots & \dots \\ \vdots & & \vdots & & & \\ a_{r1}(\omega_1) & \dots & a_{rr} & \dots & a_{ir}(\omega_r) & \dots \\ \vdots & & \vdots & & & \\ \vdots & & a_{ir}(\omega_r) & & & \\ \vdots & & \vdots & & & \end{bmatrix},$$

and to note that the pivot is one of a_{11} , a_{rr} , and $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$.

The parameter α is derived by element growth considerations. We consider each of the four cases in the algorithm in turn, noting that for cases (1) and (2) the elements of the Schur complement are given by¹¹

$$\tilde{a}_{ij} = a_{ij} - \frac{a_{i1}a_{1j}}{a_{11}}.$$

Case (1):

$$|\tilde{a}_{ij}| \leq |a_{ij}| + \frac{1}{\alpha}|a_{1j}| \leq \left(1 + \frac{1}{\alpha}\right) \max_{i,j} |a_{ij}|.$$

¹¹We commit a minor abuse of notation, in that in the rest of this section $\tilde{a}_{ij}$ should really be $\tilde{a}_{i-1,j-1}$ ($s = 1$) or $\tilde{a}_{i-2,j-2}$ ($s = 2$).

Case (2): Using symmetry,

$$|\tilde{a}_{ij}| \leq |a_{ij}| + \frac{\omega_1^2}{|a_{11}|} \leq |a_{ij}| + \frac{\omega_r}{\alpha} \leq \left(1 + \frac{1}{\alpha}\right) \max_{i,j} |a_{ij}|.$$

Case (3): The original a_{rr} is now the pivot, and $|a_{rr}| \geq \alpha\omega_r$, so

$$|\tilde{a}_{ij}| \leq |a_{ij}| + \frac{\omega_r}{|a_{rr}|} |a_{1j}| \leq \left(1 + \frac{1}{\alpha}\right) \max_{i,j} |a_{ij}|.$$

Case (4): This is where we use a 2×2 pivot, which, after the interchanges, is $E = (\Pi^T A \Pi)(1:2, 1:2) = \begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$. Now

$$|\det(E)| = |a_{r1}^2 - a_{11}a_{rr}| \geq \omega_1^2 - |a_{11}|\alpha\omega_r \geq \omega_1^2 - \alpha(\alpha\omega_1^2) = \omega_1^2(1 - \alpha^2).$$

The elements of the Schur complement $\tilde{A} = B - CE^{-1}C^T$ are given by

$$\tilde{a}_{ij} = b_{ij} - \det(E)^{-1} \begin{bmatrix} a_{i1} & a_{ir} \end{bmatrix} \begin{bmatrix} a_{rr} & -a_{r1} \\ -a_{r1} & a_{11} \end{bmatrix} \begin{bmatrix} a_{j1} \\ a_{jr} \end{bmatrix},$$

so

$$\begin{aligned} |\tilde{a}_{ij}| &\leq |b_{ij}| + (\omega_1^2(1 - \alpha^2))^{-1} \begin{bmatrix} \omega_1 & \omega_r \end{bmatrix} \begin{bmatrix} \alpha\omega_r & \omega_1 \\ \omega_1 & |a_{11}| \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_r \end{bmatrix} \\ &\leq |b_{ij}| + \frac{2\omega_1^2\omega_r(1 + \alpha)}{\omega_1^2(1 - \alpha^2)} = |b_{ij}| + \frac{2\omega_r}{1 - \alpha} \leq \left(1 + \frac{2}{1 - \alpha}\right) \max_{i,j} |a_{ij}|. \end{aligned}$$

This analysis shows that the bounds for the element growth for $s = 1$ and $s = 2$ are the same as the respective bounds for the complete pivoting strategy. Hence, using the same reasoning, we again choose $\alpha = (1 + \sqrt{17})/8$.

The growth factor for the partial pivoting strategy is bounded by $(2.57)^{n-1}$. As for GEPP, large growth factors do not seem to occur in practice. But unlike for GEPP, no example is known for which the bound is attained; see Problem 11.10.

To explain the numerical stability of partial pivoting we need the following general result on the stability of block LDL^T factorization.

Theorem 11.3. *Let block LDL^T factorization with any pivoting strategy be applied to a symmetric matrix $A \in \mathbb{R}^{n \times n}$ to yield the computed factorization $PAP^T \approx \hat{L}\hat{D}\hat{L}^T$, where P is a permutation matrix and D has diagonal blocks of dimension 1 or 2. Let $\hat{x}$ be the computed solution to $Ax = b$ obtained using the factorization. Assume that for all linear systems $Ey = f$ involving 2×2 pivots E the computed solution $\hat{x}$ satisfies*

$$(E + \Delta E)\hat{y} = f, \quad |\Delta E| \leq (cu + O(u^2))|E|, \quad (11.5)$$

where c is a constant. Then

$$P(A + \Delta A_1)P^T = \hat{L}\hat{D}\hat{L}^T, \quad (A + \Delta A_2)\hat{x} = b,$$

where

$$|\Delta A_i| \leq p(n)u(|A| + P^T|\hat{L}||\hat{D}||\hat{L}^T|P) + O(u^2), \quad i = 1:2,$$

with p a linear polynomial. $\square$

For partial pivoting the condition (11.5) can be shown to hold for the two most natural ways of solving the 2×2 systems: by GEPP and by use of the explicit inverse (as is done in LINPACK and LAPACK); see Problem 11.5. Thus Theorem 11.3 is applicable to partial pivoting, and the question is whether it implies backward stability, that is, whether the matrix $|\widehat{L}||\widehat{D}||\widehat{L}^T|$ is suitably bounded relative to A . If the elements of L were bounded by a constant then the inequality $\| |L||D||L^T| \|_\infty \leq \|L\|_\infty \|D\|_\infty \|L^T\|_\infty$ would immediately yield a satisfactory bound. However, for partial pivoting L is unbounded, as we now show by example. For $\epsilon > 0$, partial pivoting produces the factorization, with $P = I$,

$$A = \begin{bmatrix} 0 & \epsilon & 0 \\ \epsilon & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & & \\ 0 & 1 & \\ 1/\epsilon & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & \epsilon \\ \epsilon & 0 \\ & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1/\epsilon \\ & 1 & 0 \\ & & 1 \end{bmatrix} = LDL^T. \quad (11.6)$$

As $\epsilon \rightarrow 0$, $\|L\|_\infty \|D\|_\infty \|L^T\|_\infty / \|A\|_\infty \rightarrow \infty$. Nevertheless, it can be shown that for every A the matrix $|L||D||L^T|$ satisfies the bound [608, 1997]

$$\| |L||D||L^T| \|_M \leq 36n\rho_n \|A\|_M,$$

where $\|A\|_M = \max_{i,j} |a_{ij}|$ and ρ_n is the growth factor. The normwise stability of the Bunch–Kaufman partial pivoting strategy can be summarized as follows.

Theorem 11.4. *Let $A \in \mathbb{R}^{n \times n}$ be symmetric and let $\widehat{x}$ be a computed solution to the linear system $Ax = b$ produced by block LDL^T factorization with the Bunch–Kaufman partial pivoting strategy, where linear systems involving 2×2 pivots are solved by GEPP or by use of the explicit inverse. Then*

$$(A + \Delta A)\widehat{x} = b, \quad \|\Delta A\|_M \leq p(n)\rho_n u \|A\|_M + O(u^2),$$

where p is a quadratic. $\square$

11.1.3. Rook Pivoting

For solving linear systems, Theorem 11.4 shows that block LDL^T factorization with the Bunch–Kaufman partial pivoting strategy has satisfactory backward stability. But for certain other applications the possibly large L factor makes the factorization unsuitable. An example is a modified Cholesky factorization algorithm of Cheng and Higham [228, 1998], of interest in optimization and for constructing preconditioners. In this algorithm a block LDL^T factorization of a symmetric A is computed and then the D factor is perturbed to make it positive definite. The perturbation of D corresponds to a perturbation of A up to $\|L\|^2$ times larger, so it is essential in this application that $\|L\|$ is of order 1. A small $\|L\|$ can be ensured by a symmetric form of rook pivoting (cf. §9.1) proposed by Ashcraft, Grimes, and Lewis [38, 1998, §2.4]. This pivoting strategy is broadly similar to partial pivoting, but it has an iterative phase.

Algorithm 11.5 (symmetric rook pivoting). This algorithm determines the pivot for the first stage of the symmetric indefinite factorization applied to a symmetric matrix $A \in \mathbb{R}^{n \times n}$ using the rook pivoting strategy of Ashcraft, Grimes, and Lewis.

```

 $\alpha = (1 + \sqrt{17})/8$  ( $\approx 0.64$ )
 $\omega_1$  = maximum magnitude of any subdiagonal entry in column 1.
If  $\omega_1 = 0$  there is nothing to do on this stage of the factorization.
if  $|a_{11}| \geq \alpha\omega_1$ 
    Use  $a_{11}$  as a  $1 \times 1$  pivot ( $s = 1, \Pi = I$ ).
else
     $i = 1$ 
    repeat
         $r$  = row index of first (subdiagonal) entry of maximum
            magnitude in column  $i$ .
         $\omega_r$  = maximum magnitude of any off-diagonal entry in
            column  $r$ .
        if  $|a_{rr}| \geq \alpha\omega_r$ 
            Use  $a_{rr}$  as a  $1 \times 1$  pivot ( $s = 1, \Pi$  swaps rows and
                columns 1 and  $r$ ).
        else if  $\omega_i = \omega_r$ 
            Use  $\begin{bmatrix} a_{ii} & a_{ri} \\ a_{ri} & a_{rr} \end{bmatrix}$  as a  $2 \times 2$  pivot ( $s = 2, \Pi$  swaps
                rows and columns 1 and  $i$ , and 2 and  $r$ ).
        else
             $i = r, \omega_i = \omega_r$ 
        end
    until a pivot is chosen
end

```

The repeat loop in Algorithm 11.5 searches for an element a_{ri} that is simultaneously the largest in magnitude in the r th row and the i th column, and it uses this element to build a 2×2 pivot; the search terminates prematurely if a suitable 1×1 pivot is found. Note that the pivot test in case (2) of the partial pivoting strategy (Algorithm 11.2) is omitted in Algorithm 11.5—this is essential to obtain a bounded L [38, 1998].

Since the value of ω_i increases strictly from one pivot step to the next, the search in Algorithm 11.5 takes at most n steps. The overall cost of the searching is therefore between $O(n^2)$ and $O(n^3)$ comparisons. Matrices are known for which the entire Schur complement must be searched at each step, in which case the cost is $O(n^3)$ comparisons. However, probabilistic results (very similar to those for nonsymmetric rook pivoting) and experimental evidence suggest that usually only $O(n^2)$ comparisons are required [38, 1998].

The following properties are readily verified, using the property that any 2×2 pivot satisfies

$$\left| \begin{bmatrix} a_{ii} & a_{ri} \\ a_{ri} & a_{rr} \end{bmatrix}^{-1} \right| \leq \frac{1}{\omega_r(1 - \alpha^2)} \begin{bmatrix} \alpha & 1 \\ 1 & \alpha \end{bmatrix}.$$

1. Every entry of L is bounded by $\max\{1/(1 - \alpha), 1/\alpha\} \approx 2.78$.

2. Every 2×2 pivot block D_{ii} satisfies $\kappa_2(D_{ii}) \leq (1 + \alpha)/(1 - \alpha) \approx 4.56$.
3. The growth factor for the factorization satisfies the same bound as for partial pivoting. Also, Theorem 11.4 holds.

At the cost of a worst case $O(n^3)$ searching overhead, the symmetric rook pivoting strategy thus gives an L factor with elements of order 1 and produces well-conditioned 2×2 blocks of D .

As noted by Cheng [227, 1998], we can also derive a forward error bound analogous to (9.23) in §9.7. For any pivoting strategy satisfying the conditions of Theorem 11.3 we have

$$\begin{aligned} \frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} &\leq p(n)u(\text{cond}(A, x) + \|A^{-1}|P^T|\hat{L}|\hat{D}|\hat{L}^T|P\|_\infty) + O(u^2) \\ &\leq p(n)u \text{cond}(A) \text{cond}(|D||L^T|) + O(u^2). \end{aligned} \quad (11.7)$$

The term $\text{cond}(|D||L^T|)$ is unbounded for partial pivoting but is bounded exponentially in n for rook pivoting, in view of properties 1 and 2 above. In theory, then, rook pivoting seems more likely than partial pivoting to produce a $\text{cond}(A)$ -bounded forward error and hence to be insensitive to poor row (and hence column) scaling of A .

11.1.4. Tridiagonal Matrices

How should we solve a symmetric tridiagonal linear system $Ax = b$? Most commonly GEPP is used, which unfortunately destroys the symmetry and so does not reveal the inertia. A factorization $PA = LU$ is obtained, where L has at most one nonzero below the diagonal in each column and U has upper bandwidth 2 ($u_{ij} = 0$ for $j > i + 2$). Block LDL^T factorization using partial, rook, or complete pivoting exploits the symmetry, but these pivoting strategies do not preserve the bandwidth.

Bunch [177, 1974] suggested computing a block LDL^T factorization *without* interchanges, with a particular strategy for choosing the pivot size (1 or 2) at each stage of the factorization. Bunch's strategy for choosing the pivot size is fully defined by describing the choice of the first pivot.

Algorithm 11.6 (Bunch's pivoting strategy). This algorithm determines the pivot size, s , for the first stage of block LDL^T factorization applied to a symmetric tridiagonal matrix $A \in \mathbb{R}^{n \times n}$.

```

 $\alpha = (\sqrt{5} - 1)/2 \approx 0.62$ 
 $\sigma = \max\{|a_{ij}| : i, j = 1:n\}$  (compute once, at the start of the
factorization)
if  $\sigma|a_{11}| \geq \alpha a_{21}^2$ 
     $s = 1$ 
else
     $s = 2$ 
end

```

The result is a factorization

$$A = LDL^T, \quad (11.8)$$

where L is unit lower triangular and D is block diagonal with each diagonal block having dimension 1 or 2. The value of α is derived in a similar way as for the Bunch–Kaufman partial pivoting strategy, by equating growth bounds. The inertia of A is the same as that of D , which can be read from the (block) diagonal of D , since any 2×2 block can be shown to have one negative and one positive eigenvalue.

The numerical stability of this method is described in the next result.

Theorem 11.7. *Let block LDL^T factorization with the pivoting strategy of Algorithm 11.6 be applied to a symmetric tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ to yield the computed factorization $A \approx \hat{L}\hat{D}\hat{L}^T$, and let $\hat{x}$ be the computed solution to $Ax = b$ obtained using the factorization. Assume that all linear systems $Ey = f$ involving 2×2 pivots E are solved by GEPP or by use of the explicit inverse. Then*

$$A + \Delta A_1 = \hat{L}\hat{D}\hat{L}^T, \quad (A + \Delta A_2)\hat{x} = b,$$

where

$$\|\Delta A_i\|_M \leq cu\|A\|_M + O(u^2), \quad i = 1:2, \quad (11.9)$$

with c a constant. $\square$

Theorem 11.7 shows that block LDL^T factorization with the pivoting strategy of Algorithm 11.6 is a perfectly normwise backward stable way to factorize a symmetric tridiagonal matrix A and to solve a linear system $Ax = b$. Block LDL^T factorization therefore provides an attractive alternative to GEPP for solving such linear systems.

11.2. Aasen's Method

Aasen's method [1, 1971] factorizes a symmetric matrix $A \in \mathbb{R}^{n \times n}$ as

$$PAP^T = LTL^T,$$

where L is unit lower triangular with first column e_1 ,

$$T = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & \beta_{n-1} \\ & & & \beta_{n-1} & \alpha_n \end{bmatrix}$$

is tridiagonal, and P is a permutation matrix.

To derive Aasen's method, we initially ignore interchanges and assume that the first $i - 1$ columns of T and the first i columns of L are known. We show how to compute the i th column of T and the $(i + 1)$ st column of L . A key role is played by the matrix

$$H = TL^T, \quad (11.10)$$

which is easily seen to be upper Hessenberg. Equating i th columns in (11.10) we obtain

$$\begin{bmatrix} \underline{h_{1i}} \\ \underline{h_{2i}} \\ \vdots \\ \underline{h_{i-1,i}} \\ \underline{h_{ii}} \\ \underline{h_{i+1,i}} \\ 0 \\ \vdots \\ 0 \end{bmatrix} = T \begin{bmatrix} l_{i1} \\ l_{i2} \\ \vdots \\ l_{i,i-1} \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} \alpha_1 l_{i1} + \beta_1 l_{i2} \\ \beta_1 l_{i1} + \alpha_2 l_{i2} + \beta_2 l_{i3} \\ \vdots \\ \beta_{i-2} l_{i,i-2} + \alpha_{i-1} l_{i,i-1} + \beta_{i-1} \\ \beta_{i-1} l_{i,i-1} + \underline{\alpha_i} \\ \underline{\beta_i} \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \quad (11.11)$$

We use an underline to denote an unknown quantity to be determined.

The first $i-1$ equations in (11.11) are used to compute $h_{1i}, \dots, h_{i-1,i}$. The next two equations contain two unknowns each so cannot yet be used. The (i, i) and $(i+1, i)$ elements of the equation $A = LH$ give

$$a_{ii} = \sum_{j=1}^{i-1} l_{ij} h_{ji} + \underline{h_{ii}}, \quad (11.12)$$

$$a_{i+1,i} = \sum_{j=1}^i l_{i+1,j} h_{ji} + \underline{h_{i+1,i}}, \quad (11.13)$$

which we solve for h_{ii} and $h_{i+1,i}$. Now we can return to the last two nontrivial equations of (11.11) to obtain α_i and β_i . Finally, the i th column of the equation $A = LH$ yields

$$a_{ki} = \sum_{j=1}^{i+1} l_{kj} h_{ji}, \quad k = i+2:n,$$

which yields the elements below the diagonal in the $(i+1)$ st column of L :

$$l_{k,i+1} = \frac{a_{ki} - \sum_{j=1}^i l_{kj} h_{ji}}{h_{i+1,i}}, \quad k = i+2:n. \quad (11.14)$$

The factorization has thereby been advanced by one step.

The operation count for Aasen's method is the same as for block LDL^T factorization.

To ensure that the factorization does not break down, and to improve its numerical stability, interchanges are incorporated. Before the evaluation of (11.13) and (11.14) we compute $v_k = a_{ki} - \sum_{j=1}^i l_{kj} h_{ji}$, $k = i+1:n$, find r such that $|v_r| = \max\{|v_k| : k = i+1:n\}$, and then swap v_{i+1} and v_r and make corresponding interchanges in A and L . This partial pivoting strategy ensures that $|l_{ij}| \leq 1$ for $i > j$.

To solve a linear system $Ax = b$ using the factorization $PAP^T = LTL^T$ we solve in turn

$$Lz = Pb, \quad Ty = z, \quad L^T w = y, \quad x = Pw. \quad (11.15)$$

The system $Ty = z$ has a symmetric tridiagonal coefficient matrix that is indefinite in general. It is usually solved by GEPP, but the symmetry-preserving method of §11.1.4 can be used instead. The next result gives a backward error bound for Aasen's method; for simplicity, pivoting is ignored, that is, A is assumed to be "pre-pivoted".

Theorem 11.8. *Let $A \in \mathbb{R}^{n \times n}$ be symmetric and suppose Aasen's method produces computed factors $\widehat{L}, \widehat{T}$ and a computed solution $\widehat{x}$ to $Ax = b$. Then*

$$(A + \Delta A)\widehat{x} = b, \quad |\Delta A| \leq \gamma_{3n+1}|\widehat{L}||\widehat{T}||\widehat{L}^T| + \gamma_{2n+4}|\widehat{L}|\Pi^T|\widehat{M}||\widehat{U}||\widehat{L}^T|,$$

where $\Pi\widehat{T} \approx \widehat{M}\widehat{U}$ is the computed factorization produced by GEPP. Moreover,

$$\|\Delta A\|_\infty \leq (n-1)^2 \gamma_{15n+25} \|\widehat{T}\|_\infty. \quad \square$$

Theorem 11.8 shows that Aasen's method is a backward stable method for solving $Ax = b$ provided that the growth factor

$$\rho_n = \frac{\max_{i,j} |t_{ij}|}{\max_{i,j} |a_{ij}|}$$

is not too large.

Using the fact that the multipliers in Aasen's method with partial pivoting are bounded by 1, it is straightforward to show that if $\max_{i,j} |a_{ij}| = 1$ then T has a bound illustrated for $n = 5$ by

$$|T| \leq \begin{bmatrix} 1 & 1 & & & \\ 1 & 1 & 2 & & \\ & 2 & 4 & 8 & \\ & & 8 & 16 & 32 \\ & & & 32 & 64 \end{bmatrix}.$$

Hence

$$\rho_n \leq 4^{n-2}.$$

Whether this bound is attainable for $n \geq 4$ is an open question. Cheng [227, 1998] reports experiments using direct search optimization in which he obtained growth of 7.99 for $n = 4$ and 14.61 for $n = 5$, which are to be compared with the corresponding bounds of 16 and 64.

11.2.1. Aasen's Method Versus Block LDL^T Factorization

While block LDL^T of a symmetric matrix using the Bunch–Kaufman partial pivoting strategy is implemented in all the major program libraries, the only library we know to contain Aasen's method is the IMSL Fortran 90 library [546, 1997]. A comparison of the two methods in the mid 1970s found little difference between them in speed [87, 1976], but no thorough comparison on modern computer architectures has been published. See [38, 1998] for some further comments. The greater popularity of block LDL^T factorization may be due to the fact that it is generally easier to work with a block diagonal matrix with blocks of size at most 2 than with a tridiagonal one.

Note that since $|l_{ij}| \leq 1$ for Aasen's method with pivoting, the method is superior to block LDL^T factorization with partial pivoting for applications in which a bounded L is required.

11.3. Block LDL^T Factorization for Skew-Symmetric Matrices

Several properties follow from the definition of a skew-symmetric matrix $A \in \mathbb{R}^{n \times n}$: the diagonal is zero, the eigenvalues come in pure imaginary complex conjugate pairs, and the matrix is singular if n is odd. Because of the zero diagonal, a skew-symmetric matrix does not usually have an LU factorization, even if symmetric pivoting PAP^T (which preserves skew-symmetry) is allowed. To obtain a factorization that always exists and respects the skew-symmetry we must turn to a block LDL^T factorization. We consider a factorization

$$PAP^T = LDL^T, \quad (11.16)$$

where L is unit lower triangular and D is block diagonal with skew-symmetric diagonal blocks of dimension 1 or 2.

To begin the factorization we choose a permutation Π so that

$$\Pi A \Pi^T = \begin{matrix} s & n-s \\ s & \\ n-s & \end{matrix} \begin{bmatrix} E & -C^T \\ C & B \end{bmatrix}, \quad E = \begin{cases} 0, & s = 1, \\ \begin{bmatrix} 0 & -e_{21} \\ e_{21} & 0 \end{bmatrix}, & s = 2, \end{cases}$$

where we take $s = 1$ and $\Pi = I$ if the first column of A is zero, or else $s = 2$, in which case Π can be chosen so that E is nonsingular (we assume that A is nonzero). If $s = 1$ there is nothing to do; otherwise we factorize

$$\Pi A \Pi^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & B + CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}.$$

The Schur complement $B + CE^{-1}C^T$ (note the “+”) inherits skew-symmetry from A , and so the process can be repeated recursively, yielding on completion the factorization (11.16), with D having diagonal blocks 0 or of the form $\begin{bmatrix} 0 & -d_{i+1,i} \\ d_{i+1,i} & 0 \end{bmatrix}$ with $d_{i+1,i} \neq 0$. Bunch [178, 1982] proposed the following partial pivoting-type pivoting strategy.

Algorithm 11.9. (Bunch's pivoting strategy) This algorithm determines the pivot for the first stage of the block LDL^T factorization of a skew-symmetric matrix $A \in \mathbb{R}^{n \times n}$ using the pivoting strategy of Bunch.

if $\|A(2:n, 1)\|_\infty = 0$

 Use a_{11} as a 1×1 pivot ($s = 1$, $\Pi = I$, and no elimination required).

else

$|a_{pq}| = \max(\|A(2:n, 1)\|_\infty, \|A(2:n, 2)\|_\infty)$ ($p > q$)

 Use $\begin{bmatrix} 0 & -a_{pq} \\ a_{pq} & 0 \end{bmatrix}$ as a 2×2 pivot ($s = 2$, Π swaps rows and

 columns 1 and q , and 2 and p , in that order).

end

The numerical stability of the factorization is much easier to analyse than for the symmetric indefinite factorizations. It suffices to consider the 2×2 pivots, since 1×1 pivots involve no computation. We define $A := \Pi A \Pi^T$. Note first that a row of CE^{-1} has the form

$$\begin{bmatrix} a_{i1} & a_{i2} \end{bmatrix} \begin{bmatrix} 0 & 1/a_{21} \\ -1/a_{21} & 0 \end{bmatrix} = \begin{bmatrix} -a_{i2}/a_{21} & a_{i1}/a_{21} \end{bmatrix}.$$

The pivoting strategy ensures that both these elements are bounded in modulus by 1, so the same is therefore true for every element of L . Next, note that an element of $S = B + CE^{-1}C^T$ has the form

$$s_{i-2,j-2} = a_{ij} - \left(\frac{a_{i2}}{a_{21}} \right) a_{j1} + \left(\frac{a_{i1}}{a_{21}} \right) a_{j2}.$$

It follows that $|s_{ij}| \leq 3 \max_{i,j} |a_{ij}|$. Since the number of 2×2 pivots is at most $(n-2)/2$, whether n is odd or even, the growth factor for the whole elimination satisfies

$$\frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|} \leq (\sqrt{3})^{n-2} \approx (1.73)^{n-2}.$$

This bound is smaller than that for partial pivoting on a general matrix! In practice, the growth factor is usually much smaller than this bound.

The conclusion is that block LDL^T factorization with Bunch's pivoting strategy has excellent numerical stability properties. It is, of course, possible to incorporate rook pivoting or complete pivoting into the factorization to obtain even smaller worst-case growth factor bounds.

This factorization is not applicable to complex skew-Hermitian matrices ($A = -A^*$), because they do not have zero diagonal. Bunch [178, 1982] suggests applying one of the block LDL^T factorizations for Hermitian indefinite matrices to iA .

11.4. Notes and References

A major source of symmetric indefinite linear systems is the least squares problem, because the augmented system is symmetric indefinite; see Chapter 20. Other sources of such systems are interior methods for solving constrained optimization problems (see Forsgren, Gill, and Shinnerl [421, 1996], S. J. Wright [1262, 1997], and M. H. Wright [1259, 1992]) and linearly constrained optimization problems (see Gill, Murray, Saunders, and M. H. Wright [485, 1991] and Nocedal and S. J. Wright [894, 1999]).

The idea of using a block LDL^T factorization with some form of pivoting for symmetric indefinite matrices was first suggested by Kahan in 1965 [183, 1971]. Bunch and Parlett [183, 1971] developed the complete pivoting strategy and Bunch [175, 1971] proved its stability. Bunch [177, 1974] discusses a rather expensive partial pivoting strategy that requires repeated scalings. Bunch and Kaufman [181, 1977] found the efficient partial pivoting strategy presented here, which is the one now widely used, and Bunch, Kaufman, and Parlett [182, 1976] give an Algol code implementing the block LDL^T factorization with this pivoting strategy. Dongarra, Duff, Sorensen, and Van der Vorst [349, 1998, §5.4.5] show

how to develop a partitioned version of the block LDL^T factorization with partial pivoting.

Liu [794, 1987] shows how to incorporate a threshold into the Bunch–Kaufman partial pivoting strategy for sparse symmetric matrices; see also Duff et al. [361, 1991]. The partial pivoting strategy and variants of it described by Bunch and Kaufman [181, 1977] do not preserve band structure, but the fill-in depends on the number of 2×2 pivots, which is bounded by the number of negative eigenvalues (see Problem 11.2). Jones and Patrick [676, 1993], [677, 1994] show how to exploit this fact.

The complete and partial pivoting strategies of Bunch et al. use a fixed number of tests to determine each pivot. Another possibility is to prescribe growth bounds corresponding to 1×1 and 2×2 pivots and to search in some particular order for a pivot satisfying the bound. Fletcher [415, 1976] uses this approach to define a pivoting strategy that is very similar to rook pivoting and which usually requires only $O(n^2)$ operations. Duff, Reid, and co-workers apply the same approach to the block LDL^T factorization for sparse matrices, where sparsity considerations also influence the choice of pivot [366, 1979], [361, 1991]; their Fortran codes MA27 [364, 1982] and MA47 [365, 1995] implement the methods.

The backward error results Theorem 11.3 for block LDL^T factorization and Theorem 11.4 for partial pivoting are from Higham [608, 1997], who observed that despite the widespread use of partial pivoting no proof of stability had been given.

Gill, Murray, Ponceleón, and Saunders [483, 1992] show how for sparse, symmetric indefinite systems the block LDL^T factorization can be used to construct a (positive definite) preconditioner for an iterative method.

The work of Ashcraft, Grimes, and Lewis [38, 1998] was motivated by an optimization problem in which solving symmetric linear systems using the Bunch–Kaufman partial pivoting strategy led to convergence difficulties, which were traced to the fact that $\|L\|$ is unbounded. The theme of [38] is that pivoting strategies such as rook pivoting that bound $\|L\|$ lead to higher accuracy. A class of linear systems is given in [38] where rook pivoting provides more accurate solutions than partial pivoting, and the experimental results can be partially explained using the bound (11.7), as shown by Cheng [227, 1998].

Theorem 11.7 is from Higham [613, 1999].

Aasen [1, 1971] states without proof a backward error bound for the factorization produced by his method. Theorem 11.8 and a related bound for the factorization are derived by Higham [612, 1999].

Dax and Kaniel [298, 1977] propose computing a factorization $PAP^T = LDL^T$ for symmetric indefinite matrices by an extended form of Gaussian elimination in which extra row operations are used to “build up” a pivot element prior to the elimination operations; here, L is unit lower triangular and D is diagonal. A complete pivoting strategy for determining the permutation P is described in [298, 1977], and partial pivoting strategies are described in Dax [296, 1982].

Bunch [176, 1971] shows how to scale a symmetric matrix so that in every nonzero row and column the largest magnitude of an element is 1.

11.4.1. LAPACK

Driver routines **xSYSV** (simple) and **xSYSVX** (expert) use block LDL^T factorization with partial pivoting to solve a symmetric indefinite system of linear equations with multiple right-hand sides. For Hermitian matrices the corresponding routines are **xHESV** (simple) and **xHESVX** (expert). (Variants of these routines for packed storage have names in which **SP** replaces **SY** and **HP** replaces **HE**.) The expert drivers incorporate iterative refinement, condition estimation, and backward and forward error estimation. The factorization is computed by the routine **xSYTRF** or **xHETRF**.

Problems

11.1. Explain why if A is nonzero a nonsingular pivot E in (11.2) can be found.

11.2. Consider block LDL^T factorization applied to a symmetric matrix. Show that with the symmetric partial pivoting, rook pivoting, or complete pivoting strategies any 2×2 pivot is indefinite. Hence give a formula for the inertia in terms of the block sizes of the block diagonal factor. Show how to avoid overflow in computing the inverse of a 2×2 pivot.

11.3. Describe the effect of applying the block LDL^T factorization with partial pivoting to a 2×2 symmetric matrix.

11.4. What factorization is computed if the block LDL^T factorization with partial pivoting is applied to a symmetric positive definite matrix?

11.5. (Higham [608, 1997]) Show that the condition (11.5) is satisfied for the 2×2 pivots from partial pivoting if the system is solved by GEPP or by use of the explicit inverse.

11.6. Show that for matrices of the form generated by the MATLAB code

```
A = zeros(n);
A(n,1) = 2;
for i = 2:n-1
    A(i+1,i) = n-i+2;
end
A = A + A';
A(2,2) = n;
```

rook pivoting (Algorithm 11.5) requires the maximum number of comparisons on each stage of the factorization [38, 1998].

11.7. (Sorensen and Van Loan; see [349, 1998, §5.3.2]) Suppose the partial pivoting strategy in Algorithm 11.2 is modified by redefining

$$\omega_r = \|A(:, r)\|_\infty$$

(thus " $\omega_r^{\text{new}} = \max(\omega_r^{\text{old}}, |a_{rr}|)$ "). Show that the same growth factor bound holds as before and that for a positive definite matrix no interchanges are done and only 1×1 pivots are used.

11.8. For the matrix A in (11.6) what are the factorizations produced by block LDL^T with rook pivoting and complete pivoting?

11.9. A matrix of the form

$$A = \begin{bmatrix} H & B^T \\ B & -G \end{bmatrix},$$

where $H \in \mathbb{R}^{n \times n}$ and $G \in \mathbb{R}^{m \times m}$ are symmetric positive definite, has been called a *symmetric quasidefinite matrix* by Vanderbei [1186, 1995]. Show that (a) A is nonsingular, (b) for any permutation Π , $\Pi^T A \Pi$ has an LU factorization, (c) AS is nonsymmetric positive definite, where $S = \text{diag}(I, -I)$. (This last property reduces the question of the stability of an LDL^T factorization of A to that of the stability of the LU factorization of a nonsymmetric positive definite matrix, for which see §10.4. This reduction is pointed out and exploited by Gill, Saunders, and Shinnerl [487, 1996].)

11.10. (RESEARCH PROBLEM) Investigate the attainability of the growth factor bounds for block LDL^T factorization with

- (a) partial pivoting,
- (b) rook pivoting,
- (c) Bunch's partial pivoting strategy for skew-symmetric matrices.

Similarly, investigate the attainability of the growth factor bound for Aasen's method.

Chapter 12

Iterative Refinement

The ILLIAC's memory is sufficient to accommodate a system of 39 equations when used with Routine 51.

The additional length of Routine 100 restricts to 37 the number of equations that it can handle.

With 37 equations the operation time of Routine 100 is about 4 minutes per iteration.

— JAMES N. SNYDER, *On the Improvement of the Solutions to a Set of Simultaneous Linear Equations Using the ILLIAC* (1955)

In a short mantissa computing environment the presence of an iterative improvement routine can significantly widen the class of solvable $Ax = b$ problems.

— GENE H. GOLUB and CHARLES F. VAN LOAN, *Matrix Computations* (1996)

Most problems involve inexact input data and obtaining a highly accurate solution to an imprecise problem may not be justified.

— J. J. DONGARRA, J. R. BUNCH, C. B. MOLER, and G. W. STEWART, *LINPACK Users' Guide* (1979)

It is shown that even a single iteration of iterative refinement in single precision is enough to make Gaussian elimination stable in a very strong sense.

— ROBERT D. SKEEL, *Iterative Refinement Implies Numerical Stability for Gaussian Elimination* (1980)

Iterative refinement is an established technique for improving a computed solution $\hat{x}$ to a linear system $Ax = b$. The process consists of three steps:

1. Compute $r = b - A\hat{x}$.
2. Solve $Ad = r$.
3. Update $y = \hat{x} + d$.

(Repeat from step 1 if necessary, with $\hat{x}$ replaced by y .)

If there were no rounding errors in the computation of r , d , and y , then y would be the exact solution to the system. The idea behind iterative refinement is that if r and d are computed accurately enough then some improvement in the accuracy of the solution will be obtained. The economics of iterative refinement are favourable for solvers based on a factorization of A , because the factorization used to compute $\hat{x}$ can be reused in the second step of the refinement.

Traditionally, iterative refinement is used with Gaussian elimination (GE), and r is computed in extended precision before being rounded to working precision. Iterative refinement for GE was used in the 1940s on desk calculators, but the first thorough analysis of the method was given by Wilkinson in 1963 [1232, 1963]. The behaviour of iterative refinement for GE is usually summarized as follows: if double the working precision is used in the computation of r , and A is not too ill conditioned, then the iteration produces a solution correct to working precision and the rate of convergence depends on the condition number of A . In the next section we give a componentwise analysis of iterative refinement that confirms this summary and provides some further insight.

Iterative refinement is of interest not just for improving the accuracy or stability of an already stable linear system solver, but also for ameliorating instability in a less reliable solver. Examples of this usage are in Li and Demmel [786, 1998] and Dongarra, Eijkhout, and Luszczek [345, 2000], where sparse GE is performed without pivoting, for speed, and iterative refinement is used to regain stability.

12.1. Behaviour of the Forward Error

Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and let $\hat{x}$ be a computed solution to $Ax = b$. Define $x_1 = \hat{x}$ and consider the following iterative refinement process: $r_i = b - Ax_i$ (precision $\bar{u}$), solve $Ad_i = r_i$ (precision u), $x_{i+1} = x_i + d_i$ (precision u), $i = 1, 2, \dots$. For traditional iterative refinement, $\bar{u} = u^2$. Note that in this chapter subscripts specify members of a vector sequence, not vector elements.

We henceforth define r_i , d_i , and x_i to be the *computed* quantities (to avoid a profusion of hats). The only assumption we will make on the solver is that the computed solution $\hat{y}$ to a system $Ay = c$ satisfies

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq uW, \quad (12.1)$$

where W is a nonnegative matrix depending on A , n , and u (but not on b). Thus the solver need not be LU factorization or even a factorization method.

The page or so of analysis that follows is straightforward but tedious. The reader is invited to jump straight to (12.5), at least on first reading.

Consider first the computation of r_i . There are two stages. First, $s_i = fl(b - Ax_i) = b - Ax_i + \Delta s_i$ is formed in the (possibly) extended precision $\bar{u}$, so that $|\Delta s_i| \leq \bar{\gamma}_{n+1}(|b| + |A||x_i|)$ (cf. (3.11)), where $\bar{\gamma}_k \leq k\bar{u}/(1-k\bar{u})$. Second, the residual is rounded to the working precision: $r_i = fl(s_i) = s_i + f_i$, where $|f_i| \leq u|s_i|$. Hence

$$r_i = b - Ax_i + \Delta r_i, \quad |\Delta r_i| \leq u|b - Ax_i| + (1+u)\bar{\gamma}_{n+1}(|b| + |A||x_i|).$$

By writing $x_i = x + (x_i - x)$, we obtain the bound

$$|\Delta r_i| \leq [u + (1+u)\bar{\gamma}_{n+1}]|A||x - x_i| + 2(1+u)\bar{\gamma}_{n+1}|A||x|. \quad (12.2)$$

For the second step we have, by (12.1), $(A + \Delta A_i)d_i = r_i$, where $|\Delta A_i| \leq uW$. Now write

$$(A + \Delta A_i)^{-1} = (A(I + A^{-1}\Delta A_i))^{-1} =: (I + F_i)A^{-1},$$

where

$$|F_i| \leq u|A^{-1}|W + O(u^2). \quad (12.3)$$

Hence

$$d_i = (I + F_i)A^{-1}r_i = (I + F_i)(x - x_i + A^{-1}\Delta r_i). \quad (12.4)$$

For the last step,

$$\begin{aligned} x_{i+1} &= x_i + d_i + \Delta x_i, \\ |\Delta x_i| &\leq u|x_i + d_i| \leq u(|x - x_i| + |x| + |d_i|). \end{aligned}$$

Using (12.4) we have

$$x_{i+1} - x = F_i(x - x_i) + (I + F_i)A^{-1}\Delta r_i + \Delta x_i.$$

Hence

$$\begin{aligned} |x_{i+1} - x| &\leq |F_i||x - x_i| + (I + |F_i|)|A^{-1}||\Delta r_i| + u|x - x_i| + u|x| + u|d_i| \\ &\leq |F_i||x - x_i| + (I + |F_i|)|A^{-1}||\Delta r_i| + u|x - x_i| + u|x| \\ &\quad + u(I + |F_i|)(|x - x_i| + |A^{-1}||\Delta r_i|) \\ &= ((1+u)|F_i| + 2uI)|x - x_i| + (1+u)(I + |F_i|)|A^{-1}||\Delta r_i| + u|x|. \end{aligned}$$

Substituting the bound for $|\Delta r_i|$ from (12.2) gives

$$\begin{aligned} |x_{i+1} - x| &\leq ((1+u)|F_i| + 2uI)|x - x_i| \\ &\quad + (1+u)(u + (1+u)\bar{\gamma}_{n+1})(I + |F_i|)|A^{-1}||A||x - x_i| \\ &\quad + 2(1+u)^2\bar{\gamma}_{n+1}(I + |F_i|)|A^{-1}||A||x| + u|x| \\ &=: G_i|x - x_i| + g_i. \end{aligned} \quad (12.5)$$

Using (12.3), we estimate

$$\begin{aligned} G_i &\approx |F_i| + (u + \bar{\gamma}_{n+1})(I + |F_i|)|A^{-1}||A| \\ &\lesssim u|A^{-1}|W + (u + \bar{\gamma}_{n+1})(I + u|A^{-1}|W)|A^{-1}||A|, \\ g_i &\approx 2\bar{\gamma}_{n+1}(I + |F_i|)|A^{-1}||A||x| + u|x| \\ &\lesssim 2\bar{\gamma}_{n+1}(I + u|A^{-1}|W)|A^{-1}||A||x| + u|x|. \end{aligned}$$

As long as A is not too ill conditioned and the solver is not too unstable, we have $\|G_i\|_\infty < 1$, which means that the error contracts until we reach a point at which the g_i term becomes significant. The limiting normwise accuracy, that is, the minimum size of $\|x - x_i\|_\infty / \|x\|_\infty$, is roughly $\|g_i\|_\infty / \|x\|_\infty \approx 2n\bar{u} \operatorname{cond}(A, x) + u$. Moreover, if $2n\bar{u}(I + u|A^{-1}|W)|A^{-1}||A||x| \leq \mu u|x|$ for some μ , then we can expect to obtain a componentwise relative error of order μu , that is, $\min_i |x - x_i| \lesssim \mu u|x|$. Note that G_i is essentially independent of $\bar{u}$, which suggests that the rate of convergence of mixed and fixed precision iterative refinement will be similar; it is only the limiting accuracy that differs.

In the traditional use of iterative refinement, $\bar{u} = u^2$, and one way to summarize our findings is as follows.

Theorem 12.1 (mixed precision iterative refinement). *Let iterative refinement be applied to the nonsingular linear system $Ax = b$, using a solver satisfying (12.1) and with residuals computed in double the working precision. Let $\eta = u\| |A^{-1}|(|A| + W)\|_\infty$. Then, provided η is sufficiently less than 1, iterative refinement reduces the forward error by a factor approximately η at each stage, until $\|x - \hat{x}_i\|_\infty / \|x\|_\infty \approx u$.* $\square$

For LU factorization, Theorem 9.4 shows that we can take

$$uW \equiv \gamma_{3n} |\hat{L}||\hat{U}|, \quad (12.6)$$

where $\hat{L}$ and $\hat{U}$ are the computed LU factors. In this case Theorem 12.1 is stronger than the standard results in the literature, which have $\kappa_\infty(A)u$ in place of $\eta \approx u\| |A^{-1}|(|A| + 3n|\hat{L}||\hat{U}|)\|_\infty$. We can have $\eta \ll \kappa_\infty(A)u$, since η is independent of the row scaling of A (as long as the scaling does not cause changes in the pivot sequence). For example, if $|\hat{L}||\hat{U}| \approx |A|$ then $\eta \approx 3n \operatorname{cond}(A)u$, and $\operatorname{cond}(A)$ can be arbitrarily smaller than $\kappa_\infty(A)$.

Consider now the case where $\bar{u} = u$, which is called *fixed precision iterative refinement*. We have an analogue of Theorem 12.1.

Theorem 12.2 (fixed precision iterative refinement). *Let iterative refinement in fixed precision be applied to the nonsingular linear system $Ax = b$ of order n , using a solver satisfying (12.1). Let $\eta = u\| |A^{-1}|(|A| + W)\|_\infty$. Then, provided η is sufficiently less than 1, iterative refinement reduces the forward error by a factor approximately η at each stage, until $\|x - \hat{x}_i\|_\infty / \|x\|_\infty \lesssim 2n \operatorname{cond}(A, x)u$.* $\square$

The key difference between mixed and fixed precision iterative refinement is that in the latter case a relative error of order u is no longer ensured. But we do have a relative error bound of order $\operatorname{cond}(A, x)u$. This is a stronger bound than holds for the original computed solution $\hat{x}$, for which we can say only that

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \lesssim u \frac{\| |A^{-1}|W|x|\|_\infty}{\|x\|_\infty}.$$

In fact, a relative error bound of order $\operatorname{cond}(A, x)u$ is the best we can possibly expect if we do not use higher precision, because it corresponds to the uncertainty introduced by making componentwise relative perturbations to A of size u (see

Theorem 7.4); this level of uncertainty is usually present, because of errors in computing A or in rounding its elements to floating point form.

The gist of this discussion is that iterative refinement is beneficial even if residuals are computed only at the working precision. This fact became widely appreciated only after the publication of Skeel's 1980 paper [1041, 1980]. One reason for the delayed appreciation may be that comments such as that made by Forsythe and Moler, "It is absolutely essential that the residuals r_k be computed with a higher precision than that of the rest of the computation" [431, 1967, p. 49], were incorrectly read to mean that without the use of higher precision no advantage *at all* could be obtained from iterative refinement. In the next section we will see that fixed precision iterative refinement does more than just produce a $\text{cond}(A, x)u$ -bounded forward error for LU factorization—it brings componentwise backward stability as well.

12.2. Iterative Refinement Implies Stability

We saw in the last section that fixed precision iterative refinement can improve the accuracy of a computed solution to $Ax = b$. The question arises of what the refinement process does to the backward error. To answer this question we give a general backward error analysis that is applicable to a wide class of linear equation solvers. Throughout this section, "iterative refinement" means fixed precision iterative refinement.

We assume that the computed solution $\hat{x}$ to $Ax = b$ satisfies

$$|b - A\hat{x}| \leq u(g(A, b)|\hat{x}| + h(A, b)), \quad (12.7)$$

where $g : \mathbb{R}^{n \times (n+1)} \rightarrow \mathbb{R}^{n \times n}$ and $h : \mathbb{R}^{n \times (n+1)} \rightarrow \mathbb{R}^n$ have nonnegative entries. The functions g and h may depend on n and u as well as on the data A and b . We also assume that the residual $r = b - A\hat{x}$ is computed in such a way that

$$|\hat{r} - r| \leq ut(A, b, \hat{x}), \quad (12.8)$$

where $t : \mathbb{R}^{n \times (n+2)} \rightarrow \mathbb{R}^n$ is nonnegative. If r is computed in the conventional way, then we can take

$$t(A, b, \hat{x}) = \frac{\gamma_{n+1}}{u} (|A||\hat{x}| + |b|). \quad (12.9)$$

First we give an asymptotic result that does not make any further assumptions on the linear equation solver.

Theorem 12.3. *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. Suppose the linear system $Ax = b$ is solved in floating point arithmetic using a solver S together with one step of iterative refinement. Assume that the computed solution $\hat{x}$ produced by S satisfies (12.7) and that the computed residual $\hat{r}$ satisfies (12.8). Then the corrected solution $\hat{y}$ satisfies*

$$|b - A\hat{y}| \leq u(h(A, \hat{r}) + t(A, b, \hat{y}) + |A||\hat{y}|) + uq, \quad (12.10)$$

where $q = O(u)$ if $t(A, b, \hat{x}) - t(A, b, \hat{y}) = O(\|\hat{x} - \hat{y}\|_\infty)$.

Proof. The residual $r = b - A\hat{x}$ of the original computed solution $\hat{x}$ satisfies

$$|r| \leq u(g(A, b)|\hat{x}| + h(A, b)). \quad (12.11)$$

The computed residual is $\hat{r} = r + \Delta r$, where $|\Delta r| \leq ut(A, b, \hat{x})$. The computed correction $\hat{d}$ satisfies

$$A\hat{d} = \hat{r} + f_1, \quad |f_1| \leq u(g(A, \hat{r})|\hat{d}| + h(A, \hat{r})). \quad (12.12)$$

Finally, for the corrected solution we have

$$\hat{y} = fl(\hat{x} + \hat{d}) = \hat{x} + \hat{d} + f_2, \quad |f_2| \leq u(|\hat{x}| + |\hat{d}|). \quad (12.13)$$

Collecting together the above results we obtain

$$b - A\hat{y} = b - A\hat{x} - A\hat{d} - Af_2 = \hat{r} - \Delta r - A\hat{d} - Af_2 = -f_1 - \Delta r - Af_2.$$

Hence

$$\begin{aligned} |b - A\hat{y}| &\leq u(g(A, \hat{r})|\hat{d}| + h(A, \hat{r})) + ut(A, b, \hat{x}) + u|A|(|\hat{x}| + |\hat{d}|) \\ &= u(h(A, \hat{r}) + t(A, b, \hat{y}) + |A||\hat{y}|) + uq, \end{aligned} \quad (12.14)$$

where

$$q = t(A, b, \hat{x}) - t(A, b, \hat{y}) + g(A, \hat{r})|\hat{d}| + |A|(|\hat{x}| - |\hat{y}| + |\hat{d}|).$$

The claim about the order of q follows since $\hat{x} - \hat{y}$, $|\hat{x}| - |\hat{y}|$, and $\hat{d}$ are all of order u . $\square$

Theorem 12.3 shows that, to first order, the componentwise relative backward error $\omega_{|A|,|b|}$ will be small after one step of iterative refinement as long as $h(A, \hat{r})$ and $t(A, b, \hat{y})$ are bounded by a modest scalar multiple of $|A||\hat{y}| + |b|$. This is true for t if the residual is computed in the conventional way (see (12.9)), and in some cases we may take $h \equiv 0$, as shown below. Note that the function g of (12.7) does not appear in the first-order term of (12.10). This is the essential reason why iterative refinement improves stability: potential instability manifested in g is suppressed by the refinement stage.

A weakness of Theorem 12.3 is that the bound (12.10) is asymptotic. Since a strict bound for q is not given, it is difficult to draw firm conclusions about the size of $\omega_{|A|,|b|}$. The next result overcomes this drawback, at the cost of some specialization (and a rather long proof).

We introduce a measure of ill scaling of the vector $|B||x|$,

$$\sigma(B, x) = \frac{\max_i (|B||x|)_i}{\min_i (|B||x|)_i}.$$

Theorem 12.4. *Under the conditions of Theorem 12.3, suppose that $g(A, b) = G|A|$ and $h(A, b) = H|b|$, where $G, H \in \mathbb{R}^{n \times n}$ have nonnegative entries, and that the residual is computed in the conventional manner. Then there is a function*

$$f(t_1, t_2) \approx (t_2(t_1 + n + 1)/\text{cond}(A^{-1}) + 2(t_1 + n + 2)^2(1 + ut_2)^2)/(n + 1)$$

such that if

$$\text{cond}(A^{-1})\sigma(A, \hat{y}) \leq (f(\|G\|_\infty, \|H\|_\infty)u)^{-1}$$

then

$$|b - A\hat{y}| \leq 2\gamma_{n+1}|A||\hat{y}|.$$

Proof. As with the analysis in the previous section, this proof can be skipped without any real loss of understanding. From (12.14) in the proof of Theorem 12.3, using the formula (12.9) for t , we have

$$|b - A\hat{y}| \leq uH|\hat{r}| + \gamma_{n+1}|b| + (\gamma_{n+1} + u)|A||\hat{x}| + u(I + G)|A||\hat{d}|. \quad (12.15)$$

The inequality (12.11) implies

$$|b| - |A||\hat{x}| \leq |b - A\hat{x}| \leq u(G|A||\hat{x}| + H|b|),$$

or $(I - uH)|b| \leq (I + uG)|A||\hat{x}|$. If $u\|H\|_\infty < 1/2$ (say) then $I - uH$ is nonsingular with a nonnegative inverse satisfying $\|(I - uH)^{-1}\|_\infty \leq 2$ and we can solve for $|b|$ to obtain $|b| \leq (I - uH)^{-1}(I + uG)|A||\hat{x}|$. It follows from this relation and consideration of the rest of the proof that the simplifying step of replacing b by 0 in the analysis has little effect on the bounds—it merely produces unimportant perturbations in f in the statement of the theorem. Making this replacement in (12.15) and approximating $\gamma_{n+1} + u \approx \gamma_{n+1}$, we have

$$|b - A\hat{y}| \leq uH|\hat{r}| + \gamma_{n+1}|A||\hat{x}| + u(I + G)|A||\hat{d}|. \quad (12.16)$$

Our task is now to bound $|A||\hat{x}|$, $|\hat{r}|$, and $|A||\hat{d}|$ in terms of $|\hat{y}|$. By manipulating (12.13) we obtain the inequality

$$|\hat{x}| \leq (1 - u)^{-1}(|\hat{y}| + (1 + u)|\hat{d}|) \approx |\hat{y}| + |\hat{d}|. \quad (12.17)$$

Also, we can bound $|\hat{r}|$ by

$$|\hat{r}| \leq |r| + |\Delta r| \leq u(G|A||\hat{x}| + H|b|) + \gamma_{n+1}(|A||\hat{x}| + |b|),$$

and dropping the $|b|$ terms and using (12.17) gives

$$|\hat{r}| \leq (uG + \gamma_{n+1}I)|A||\hat{x}| \leq (uG + \gamma_{n+1}I)|A|(|\hat{y}| + |\hat{d}|). \quad (12.18)$$

Substituting from (12.17) and (12.18) into (12.16) we find

$$\begin{aligned} |b - A\hat{y}| &\leq (\gamma_{n+1}I + uH(uG + \gamma_{n+1}I))|A||\hat{y}| \\ &\quad + (\gamma_{n+1}I + u(I + G) + uH(uG + \gamma_{n+1}I))|A||\hat{d}| \\ &=: (\gamma_{n+1}I + M_1)|A||\hat{y}| + M_2|A||\hat{d}|, \end{aligned} \quad (12.19)$$

where

$$\begin{aligned} \|M_1\|_\infty &\leq u\|H\|_\infty(u\|G\|_\infty + \gamma_{n+1}), \\ \|M_2\|_\infty &\leq \gamma_{n+2} + u\|G\|_\infty + u\|H\|_\infty(u\|G\|_\infty + \gamma_{n+1}). \end{aligned}$$

Now from (12.12), making use of (12.18),

$$\begin{aligned} |\widehat{d}| &\leq |A^{-1}|(|\widehat{r}| + uG|A||\widehat{d}| + uH|\widehat{r}|) \\ &\leq |A^{-1}|((I + uH)(uG + \gamma_{n+1}I)|A|(|\widehat{y}| + |\widehat{d}|) + uG|A||\widehat{d}|). \end{aligned}$$

After premultiplying by $|A|$ this may be rearranged as

$$(I - uM_3)|A||\widehat{d}| \leq u|A||A^{-1}| M_4 |A||\widehat{y}|, \quad (12.20)$$

where

$$\begin{aligned} M_3 &= |A||A^{-1}|((I + uH)(G + (\gamma_{n+1}/u)I) + G), \\ M_4 &= (I + uH)(G + (\gamma_{n+1}/u)I). \end{aligned}$$

Using $\gamma_{n+1}/u \leq (n+1)/(1 - (n+1)u) \approx n+1$, we have the bounds

$$\begin{aligned} \|M_3\|_\infty &\leq \text{cond}(A^{-1})(\|G\|_\infty + n+1)(2 + u\|H\|_\infty), \\ \|M_4\|_\infty &\leq (\|G\|_\infty + n+1)(1 + u\|H\|_\infty). \end{aligned}$$

If $u\|M_3\|_\infty < 1/2$ (say) then $(I - uM_3)^{-1} \geq 0$ with $\|(I - uM_3)^{-1}\|_\infty \leq 2$ and we can rewrite (12.20) as

$$|A||\widehat{d}| \leq u(I - uM_3)^{-1}|A||A^{-1}| M_4 |A||\widehat{y}|. \quad (12.21)$$

Substituting this bound into (12.19) we obtain

$$\begin{aligned} |b - A\widehat{y}| &\leq (\gamma_{n+1}I + M_1 + uM_2(I - uM_3)^{-1}|A||A^{-1}|M_4) |A||\widehat{y}| \quad (12.22) \\ &=: (\gamma_{n+1}I + M_5)|A||\widehat{y}| \\ &\leq (\gamma_{n+1} + \|M_5\|_\infty \sigma(A, \widehat{y})) |A||\widehat{y}| \end{aligned}$$

(see Problem 12.1). Finally, we bound $\|M_5\|_\infty$. Writing $g = \|G\|_\infty$, $h = \|H\|_\infty$, we have

$$\begin{aligned} \|M_5\|_\infty &\leq u^2gh + uh\gamma_{n+1} + 2u(\gamma_{n+2} + ug + u^2gh + uh\gamma_{n+1}) \\ &\quad \times \text{cond}(A^{-1})(g + n+1)(1 + uh) \end{aligned}$$

and this expression is approximately bounded by $u^2(h(g+n+1) + 2(g+n+2)^2(1 + uh)^2 \text{cond}(A^{-1}))$. Requiring $\|M_5\|_\infty \sigma(A, \widehat{y})$ not to exceed γ_{n+1} leads to the result. $\square$

Theorem 12.4 says that as long as A is not too ill conditioned, $|A||\widehat{y}|$ is not too badly scaled ($\text{cond}(A^{-1})\sigma(A, \widehat{y})$ is not too large), and the solver is not too unstable ($f(\|G\|_\infty, \|H\|_\infty)$ is not too large), then $\omega_{|A|, |\widehat{y}|} \leq 2\gamma_{n+1}$ after one step of iterative refinement. Note that the term $\gamma_{n+1}|A||\widehat{y}|$ in (12.22) comes from the error bound for evaluation of the residual, so this bound for ω is about the smallest we could expect to prove.

Let us apply Theorem 12.4 to GE with or without pivoting. If there is pivoting, assume (without loss of generality) that no interchanges are required. Theorem 9.4 shows that we can take

$$g(A, b) \approx 3n|\widehat{L}||\widehat{U}|, \quad h(A, b) = 0,$$

Table 12.1. $\omega_{|A|,|b|}$ values for $A = \text{gallery}(\text{'orthog'}, 25)$.

$\text{cond}(A^{-1})\sigma(A, x) = 3.0\text{e}1$		
$\text{cond}(A) = 2.1\text{e}1, \quad \kappa_{\infty}(A) = 2.1\text{e}1$		
GEPP	GE	QR
3.5e-16	4.4e-7	3.4e-16
2.2e-16	7.2e-14	2.1e-16
	1.6e-16	

where $\hat{L}, \hat{U}$ are the computed LU factors of A . To apply Theorem 12.4 we use $A \approx \hat{L}\hat{U}$ and write

$$g(A, b) \approx 3n|\hat{L}||\hat{L}^{-1}A| \leq 3n|\hat{L}||\hat{L}^{-1}||A|,$$

which shows that we can take

$$G = 3n|\hat{L}||\hat{L}^{-1}|, \quad f(\|G\|_{\infty}, \|H\|_{\infty}) \approx 18n\|\hat{L}||\hat{L}^{-1}\|_{\infty}^2.$$

Without pivoting the growth factor-type term $\|\hat{L}||\hat{L}^{-1}\|_{\infty}$ is unbounded, but with partial pivoting it cannot exceed 2^n and is typically $O(n)$ [1157, 1990].

We can conclude that, for GE with partial pivoting (GEPP), one step of iterative refinement will usually be enough to yield a small componentwise relative backward error as long as A is not too ill conditioned and $|A||\hat{y}|$ is not too badly scaled. Without pivoting the same holds true with the added proviso that the computation of the original $\hat{x}$ must not be too unstable.

One interesting problem remains: to reconcile Theorem 12.4 with Theorem 12.2. Under the conditions of Theorem 12.4 the componentwise relative backward error is small after one step of iterative refinement, so the forward error is certainly bounded by a multiple of $\text{cond}(A, x)u$. How can this be shown (for GE) using the analysis of §12.1? An explanation is nontrivial—see Problem 12.2.

We will see applications of Theorems 12.3 and 12.4 to other types of linear equation solver in Chapters 19, 20, and 22.

Tables 12.1–12.3 show the performance of fixed precision iterative refinement for GE without pivoting, GEPP, and Householder QR factorization (see §19.7). The matrices are as follows: `gallery('clement')` is tridiagonal with zero diagonal entries, `gallery('orthog')` is a symmetric and orthogonal matrix, and `gfpp` (from the Matrix Computation Toolbox) is a matrix for which the growth factor for GEPP is maximal. In each case the right-hand side b was chosen as a random vector from the uniform distribution on $[0, 1]$. We report the componentwise relative backward errors for the initial solution and the refined iterates (refinement was terminated when $\omega_{|A|,|b|}(\hat{y}) \leq 2u$). GEPP performs as predicted by both our and Skeel's analyses. In fact, iterative refinement converges in one step even when $\theta(A, x) := \text{cond}(A^{-1})\sigma(A, x)$ exceeds u^{-1} in the examples reported and in most others we have tried. GE also achieves a small componentwise relative backward error, but can require more than one refinement step, even when $\theta(A, x)$ is small.

Table 12.2. $\omega_{|A|,|b|}$ values for $A = \text{gallery}(\text{'clement'}, 50)$.

$\text{cond}(A^{-1})\sigma(A, x) = 1.2\text{e}15$		
$\text{cond}(A) = 1.4\text{e}6, \quad \kappa_{\infty}(A) = 3.5\text{e}7$		
GEPP	GE	QR
1.0e-16	Fail	4.9e-9
		1.7e-16

Table 12.3. $\omega_{|A|,|b|}$ values for $A = \text{gfpp}(50)$.

$\text{cond}(A^{-1})\sigma(A, x) = 4.9\text{e}2$		
$\text{cond}(A) = 50, \quad \kappa_{\infty}(A) = 50$		
GEPP	GE	QR
1.7e-3	1.8e-3	1.8e-15
5.5e-17	5.5e-17	7.0e-17

12.3. Notes and References

Wilkinson [1232, 1963] gave a detailed analysis of iterative refinement in a kind of scaled fixed point arithmetic called block-floating arithmetic. Moler [859, 1967] extended the analysis to floating point arithmetic. Very readable analyses of iterative refinement are given in the books by Forsythe and Moler [431, 1967, §22] and Stewart [1065, 1973, §4.5].

As we mentioned in §9.14, as early as 1948 Wilkinson had written a program for the ACE to do GEPP and iterative refinement. Other early implementations of iterative refinement are in a code for the University of Illinois' ILLIAC by Snyder [1055, 1955], the Algol code of McKeeman [836, 1962], and the Algol codes in the Handbook [154, 1966], [817, 1966]. Some of the machines for which these codes were intended could accumulate inner products in extended precision, and so were well suited to mixed precision iterative refinement.

Interest in fixed precision iterative refinement was sparked by two papers that appeared in the late 1970s. Jankowski and Woźniakowski [671, 1977] proved that an arbitrary linear equation solver is made normwise backward stable by the use of fixed precision iterative refinement, as long as the solver is not too unstable to begin with and A is not too ill conditioned. Skeel [1041, 1980] analysed iterative refinement for GEPP and showed that one step of refinement yields a small componentwise relative backward error, as long as $\text{cond}(A^{-1})\sigma(A, x)$ is not too large.

The analysis in §12.1 is from Higham [607, 1997] and that in §12.2 is from Higham [596, 1991].

The results for GE in §12.2 are very similar to those of Skeel [1041, 1980]. The main differences are that Skeel's analysis covers an arbitrary number of refinement steps with residuals computed at working or double the working precision, his analysis is specific to GE, and his results involve $\sigma(A, x)$ rather than $\sigma(A, \hat{y})$.

The most general backward error results for iterative refinement are those of

Higham [607, 1997], which apply to an arbitrary solver over an arbitrary number of refinement steps with residuals computed at working or extended precision. The analysis in [607] suggests that, as for the forward error, the rate of reduction of the componentwise relative backward error is approximately the same for both fixed and mixed precision refinement.

The quantity $\sigma(A, x)$ appearing in Theorem 12.4 can be interpreted as follows. Consider a linear system $Ax = b$ for which $(|A||x|)_i = 0$ for some i . While the componentwise relative backward error $\omega_{|A|,|b|}(x)$ of the exact solution x is zero, an arbitrarily small change to a component x_j where $a_{ij} \neq 0$ yields $\omega_{|A|,|b|}(x + \Delta x) \geq 1$. Therefore solving $Ax = b$ to achieve a small componentwise relative backward error can be regarded as an ill-posed problem when $|A||x|$ has a zero component. The quantity $\sigma(A, x)$ reflects this ill-posedness because it is large when $|A||x|$ has a relatively small component.

For a lucid survey of both fixed and mixed precision iterative refinement and their applications, see Björck [123, 1990]. For particular applications of fixed precision iterative refinement see Govaerts [515, 2000], Govaerts and Pryce [514, 1990], and Jankowski and Woźniakowski [672, 1985].

By increasing the precision from one refinement iteration to the next it is possible to compute solutions to arbitrarily high accuracy, an idea first suggested by Stewart in an exercise [1065, 1973, pp. 206–207]. For algorithms, see Kielbasiński [732, 1981] and Smoktunowicz and Sokolnicka [1054, 1984].

In the past, mixed precision iterative refinement could not be implemented in a portable way when the working precision was already the highest precision supported by a compiler. This is the main reason why iterative refinement is not supported in LINPACK. (The LINPACK manual lists a subroutine that implements mixed precision iterative refinement for single precision data, but it is not part of LINPACK [341, 1979, pp. 1.8–1.10].) However, with the release of the Extended and Mixed Precision BLAS (see §27.10) and the portable reference implementation for IEEE arithmetic, and with the ubiquity of IEEE arithmetic, portable mixed precision iterative refinement is now achievable.

Two important practical issues when implementing iterative refinement concern storage and convergence. For either form of refinement, a copy of the matrix A needs to be kept in order to form the residual, and this necessitates an extra n^2 elements of storage. A test for terminating the refinement is needed. In addition to revealing when convergence has been achieved, it must signal lack of (sufficiently fast) convergence, which may certainly be experienced when A is very ill conditioned. In the LAPACK driver `xGESVX` for LU factorization with partial pivoting, fixed precision iterative refinement is terminated if the componentwise relative backward error $\omega = \omega_{|A|,|b|}(\hat{x}_i)$ satisfies

1. $\omega \leq u$,
2. ω has not decreased by a factor of at least 2 during the current iteration, or
3. five iterations have been performed.

These criteria were chosen to be robust in the face of different BLAS implementations and machine arithmetics. In an implementation of mixed precision iterative refinement it is more natural to test for convergence of the sequence $\{\hat{x}_i\}$, with a

test such as $\|\hat{x}_i - \hat{x}_{i-1}\|_\infty / \|\hat{x}_i\|_\infty \leq u$ (see, e.g., Forsythe and Moler [431, 1967, p. 65]). However, if A is so ill conditioned that Theorem 12.1 is not applicable, the sequence $\hat{x}_i$ could converge to a vector other than the solution. This behaviour is very unlikely, and Kahan [687, 1966] quotes a “prominent figure in the world of error-analysis” as saying “Anyone unlucky enough to encounter this sort of calamity has probably already been run over by a truck.”

A by-product of extended precision iterative refinement is an estimate of the condition number. Since the error decreases by a factor approximately $\eta = u \|A^{-1}\| \|\hat{L}\| \|\hat{U}\|_\infty$ on each iteration for LU factorization (Theorem 12.1), the relative change in x on the first iteration should be about η , that is, $\|d_1\|_\infty / \|\hat{x}\|_\infty \approx \eta \approx \kappa_\infty(A)u$. Now that reliable and inexpensive condition estimators are available (Chapter 15) this rough estimate is less important.

An unusual application of iterative refinement is to fault-tolerant computing. Boley et al. [148, 1994] propose solving $Ax = b$ by GEPP or QR factorization, performing one step of fixed precision iterative refinement and then testing whether the a priori residual bound in Theorem 12.4 is satisfied. If the bound is violated then a hardware fault may have occurred and special action is taken.

12.3.1. LAPACK

Iterative refinement is carried out by routines whose names end -RFS, and these routines are called by the expert drivers (name ending -SVX). Iterative refinement is available for all the standard matrix types except triangular matrices, for which the original computed solution already has a componentwise relative backward error of order u . As an example, the expert driver `xGESVX` uses LU factorization with partial pivoting and fixed precision iterative refinement to solve a general system of linear equations with multiple right-hand sides, and the refinement is actually carried out by the routine `xGERFS`.

Problems

12.1. Show that for $A \in \mathbb{R}^{m \times n}$ and $x \in \mathbb{R}^n$, $|A||x| \leq \sigma \|A\|_\infty |x|$, where $\sigma = \max_i |x_i| / \min_i |x_i|$.

12.2. Use the analysis of §12.1 to show that, under the conditions of Theorem 12.4, $\|x - \hat{x}_2\|_\infty / \|x\|_\infty$ is bounded by a multiple of $\text{cond}(A, x)u$ for GEPP after one step of fixed precision iterative refinement.

12.3. Investigate empirically the size of $\| |L| |L|^{-1} \|_\infty$ for L from GEPP.

12.4. (Demmel and Higham [324, 1992]) Suppose GEPP with fixed precision iterative refinement is applied to the multiple right-hand side system $AX = B$, and that refinement of the columns of X is done “in parallel”: $R = B - AX$, $AD = R$, $Y = X + D$. What can be said about the stability of the process if R is computed by conventional multiplication but the second step is done using a fast multiplication technique for which only (13.4) holds?

12.5. (RESEARCH PROBLEM [607, 1997]) Is one step of fixed precision iterative refinement sufficient to produce a componentwise relative backward error of order u

for Cholesky factorization applied to a symmetric positive definite system $Ax = b$, assuming $\text{cond}(A^{-1})\sigma(A, x)$ is not too large? Answer the same question for the diagonal pivoting method with partial pivoting applied to a symmetric system $Ax = b$.

Chapter 13

Block LU Factorization

Block algorithms are advantageous for at least two important reasons.

First, they work with blocks of data having b^2 elements, performing $O(b^3)$ operations.

The $O(b)$ ratio of work to storage means that processing elements with an $O(b)$ ratio of

computing speed to input/output bandwidth can be tolerated.

Second, these algorithms are usually rich in matrix multiplication.

This is an advantage because nearly every modern parallel machine is good at matrix multiplication.

— ROBERT S. SCHREIBER, *Block Algorithms for Parallel Machines* (1988)

It should be realized that, with partial pivoting, any matrix has a triangular factorization.

DECOMP actually works faster when zero pivots occur because they mean that the corresponding column is already in triangular form.

— GEORGE E. FORSYTHE, MICHAEL A. MALCOLM, and CLEVE B. MOLER, *Computer Methods for Mathematical Computations* (1977)

It was quite usual when dealing with very large matrices to perform an iterative process as follows:

the original matrix would be read from cards and the reduced matrix punched without more than a single row of the original matrix

being kept in store at any one time;

then the output hopper of the punch would be transferred to the card reader and the iteration repeated.

— MARTIN CAMPBELL-KELLY, *Programming the Pilot ACE* (1981)

13.1. Block Versus Partitioned LU Factorization

As we noted in Chapter 9 (Notes and References), Gaussian elimination (GE) comprises three nested loops that can be ordered in six ways, each yielding a different algorithmic variant of the method. These variants involve different computational kernels: inner product and saxpy operations (level-1 BLAS), or outer product and gaxpy operations (level-2 BLAS). To introduce matrix–matrix operations (level-3 BLAS), which are beneficial for high-performance computing, further manipulation beyond loop reordering is needed. We will use the following terminology, which emphasises an important distinction.

A *partitioned algorithm* is a scalar (or point) algorithm in which the operations have been grouped and reordered into matrix operations.

A *block algorithm* is a generalization of a scalar algorithm in which the basic scalar operations become matrix operations ($\alpha + \beta$, $\alpha\beta$, and α/β become $A + B$, AB , and AB^{-1}), and a matrix property based on the nonzero structure becomes the corresponding property blockwise (in particular, the scalars 0 and 1 become the zero matrix and the identity matrix, respectively). A *block factorization* is defined in a similar way and is usually what a block algorithm computes.

A partitioned version of the outer product form of LU factorization may be developed as follows. For $A \in \mathbb{R}^{n \times n}$ and a given block size r , write

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I_{n-r} \end{bmatrix} \begin{bmatrix} I_r & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & I_{n-r} \end{bmatrix}, \quad (13.1)$$

where A_{11} is $r \times r$.

Algorithm 13.1 (partitioned LU factorization). This algorithm computes an LU factorization $A = LU \in \mathbb{R}^{n \times n}$ using a partitioned outer product implementation, using block size r and the notation (13.1).

1. Factor $A_{11} = L_{11}U_{11}$.
2. Solve $L_{11}U_{12} = A_{12}$ for U_{12} .
3. Solve $L_{21}U_{11} = A_{21}$ for L_{21} .
4. Form $S = A_{22} - L_{21}U_{12}$.
5. Repeat steps 1–4 on S to obtain L_{22} and U_{22} .

Note that in step 4, $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$ is the Schur complement of A_{11} in A . Steps 2 and 3 require the solution of the multiple right-hand side triangular systems, so steps 2–4 are all level-3 BLAS operations. This partitioned algorithm does precisely the same arithmetic operations as any other variant of GE, but it does the operations in an order that permits them to be expressed as matrix operations.

A genuine block algorithm computes a *block LU factorization*, which is a factorization $A = LU \in \mathbb{R}^{n \times n}$, where L and U are block triangular and L has identity matrices on the diagonal:

$$L = \begin{bmatrix} I & & & \\ L_{21} & I & & \\ \vdots & & \ddots & \\ L_{m1} & \dots & L_{m,m-1} & I \end{bmatrix}, \quad U = \begin{bmatrix} U_{11} & U_{12} & \dots & U_{1m} \\ & U_{22} & & \vdots \\ & & \ddots & U_{m-1,m} \\ & & & U_{mm} \end{bmatrix}.$$

In general, the blocks can be of different dimensions. Note that this factorization is not the same as a standard LU factorization, because U is not triangular. However, the standard and block LU factorizations are related as follows: if $A = LU$ is a block LU factorization and each U_{ii} has an LU factorization $U_{ii} = \bar{L}_{ii}\bar{U}_{ii}$, then $A = L \operatorname{diag}(\bar{L}_{ii}) \cdot \operatorname{diag}(\bar{U}_{ii})U$ is an LU factorization. Conditions for the existence of a block LU factorization are easy to state.

Theorem 13.2. *The matrix $A = (A_{ij})_{i,j=1}^m \in \mathbb{R}^{n \times n}$ has a unique block LU factorization if and only if the first $m - 1$ leading principal block submatrices of A are nonsingular.*

Proof. The proof is entirely analogous to the proof of Theorem 9.1. $\square$

This theorem makes clear that a block LU factorization may exist when an LU factorization does not.

If $A_{11} \in \mathbb{R}^{r \times r}$ is nonsingular we can write

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} I & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ 0 & S \end{bmatrix}, \quad (13.2)$$

which describes one block step of an outer-product-based algorithm for computing a block LU factorization. Here, S is again the Schur complement of A_{11} in A . If the $(1, 1)$ block of S of appropriate dimension is nonsingular then we can factorize S in a similar manner, and this process can be continued recursively to obtain the complete block LU factorization. The overall algorithm can be expressed as follows.

Algorithm 13.3 (block LU factorization). This algorithm computes a block LU factorization $A = LU \in \mathbb{R}^{n \times n}$, using the notation (13.2).

1. $U_{11} = A_{11}$, $U_{12} = A_{12}$.
2. Solve $L_{21}A_{11} = A_{21}$ for L_{21} .
3. $S = A_{22} - L_{21}A_{12}$.
4. Compute the block LU factorization of S , recursively.

Given a block LU factorization of A , the solution to a system $Ax = b$ can be obtained by solving $Ly = b$ by forward substitution (since L is triangular) and solving $Ux = y$ by block back substitution. There is freedom in how step 2 of Algorithm 13.3 is accomplished, and how the linear systems with coefficient matrices U_{ii} that arise in the block back substitution are solved. The two main possibilities are as follows.

Implementation 1: A_{11} is factorized by GEPP. Step 2 and the solution of linear systems with U_{ii} are accomplished by substitution with the LU factors of A_{11} .

Implementation 2: A_{11}^{-1} is computed explicitly, so that step 2 becomes a matrix multiplication and $Ux = y$ is solved entirely by matrix-vector multiplications. This approach is attractive for parallel machines.

A particular case of partitioned LU factorization is *recursively partitioned LU factorization*. Assuming, for simplicity, that n is even, we write

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I_{n/2} \end{bmatrix} \begin{bmatrix} I_{n/2} & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & I_{n/2} \end{bmatrix}, \quad (13.3)$$

where each block is $n/2 \times n/2$. The algorithm is as follows.

Algorithm 13.4 (recursively partitioned LU factorization). This algorithm computes an LU factorization $A = LU \in \mathbb{R}^{n \times n}$ using a recursive partitioning, using the notation (13.3).

1. Recursively factorize $\begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} = \begin{bmatrix} L_{11} \\ L_{21} \end{bmatrix} U_{11}$.
2. Solve $L_{11}U_{12} = A_{12}$ for U_{12} .
3. Form $S = A_{22} - L_{21}U_{12}$.
4. Recursively factorize $S = L_{22}U_{22}$.

In contrast with Algorithm 13.1, this recursive algorithm does not require a block size to be chosen. Intuitively, the recursive algorithm maximizes the dimensions of the matrices that are multiplied in step 3: at the top level of the recursion two $n/2 \times n/2$ matrices are multiplied, at the next level two $n/4 \times n/4$ matrices, and so on. Toledo [1145, 1997] shows that Algorithm 13.4 transfers fewer words of data between primary and secondary computer memory than Algorithm 13.1 and shows that it outperforms Algorithm 13.1 on a range of computers. He also shows that the large matrix multiplications in Algorithm 13.4 enable it to benefit particularly well from the use of Strassen's fast matrix multiplication method (see §23.1).

What can be said about the numerical stability of partitioned and block LU factorization? Because the partitioned algorithms are just rearrangements of standard GE, the standard error analysis applies if the matrix operations are computed in the conventional way. However, if fast matrix multiplication techniques are used (for example, Strassen's method), the standard results are not applicable. Standard results are, in any case, not applicable to block LU factorization; its stability can be very different from that of LU factorization. Therefore we need error analysis for both partitioned and block LU factorization based on general assumptions that permit the use of fast matrix multiplication.

Unless otherwise stated, in this chapter an unsubscripted norm denotes $\|A\| := \max_{i,j} |a_{ij}|$. We make two assumptions about the underlying level-3 BLAS (matrix-matrix operations).

(1) If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ then the computed approximation $\hat{C}$ to $C = AB$ satisfies

$$\hat{C} = AB + \Delta C, \quad \|\Delta C\| \leq c_1(m, n, p)u\|A\|\|B\| + O(u^2), \quad (13.4)$$

where $c_1(m, n, p)$ denotes a constant depending on m , n , and p .

(2) The computed solution $\hat{X}$ to the triangular systems $TX = B$, where $T \in \mathbb{R}^{m \times m}$ and $B \in \mathbb{R}^{m \times p}$, satisfies

$$T\hat{X} = B + \Delta B, \quad \|\Delta B\| \leq c_2(m, p)u\|T\|\|\hat{X}\| + O(u^2). \quad (13.5)$$

For conventional multiplication and substitution, conditions (13.4) and (13.5) hold with $c_1(m, n, p) = n^2$ and $c_2(m, p) = m^2$. For implementations based on Strassen's method, (13.4) and (13.5) hold with c_1 and c_2 rather complicated functions of the dimensions m , n , p , and the threshold n_0 that determines the level of recursion (see Theorem 23.2 and [592, 1990]).

13.2. Error Analysis of Partitioned LU Factorization

An error analysis for partitioned LU factorization must answer two questions. The first is whether partitioned LU factorization becomes unstable in some fundamental way when fast matrix multiplication is used. The second is whether the constants in (13.4) and (13.5) are propagated stably into the final error bound (exponential growth of the constants would be disastrous).

We will analyse Algorithm 13.1 and will assume that the block level LU factorization is done in such a way that the computed LU factors of $A_{11} \in \mathbb{R}^{r \times r}$ satisfy

$$\widehat{L}_{11}\widehat{U}_{11} = A_{11} + \Delta A_{11}, \quad \|\Delta A_{11}\| \leq c_3(r)u\|\widehat{L}_{11}\|\|\widehat{U}_{11}\| + O(u^2). \quad (13.6)$$

Theorem 13.5 (Demmel and Higham). *Under the assumptions (13.4)–(13.6), the LU factors of $A \in \mathbb{R}^{n \times n}$ computed using the partitioned outer product form of LU factorization with block size r satisfy $\widehat{L}\widehat{U} = A + \Delta A$, where*

$$\|\Delta A\| \leq u(\delta(n, r)\|A\| + \theta(n, r)\|\widehat{L}\|\|\widehat{U}\|) + O(u^2), \quad (13.7)$$

and where

$$\begin{aligned} \delta(n, r) &= 1 + \delta(n - r, r), \quad \delta(r, r) = 0, \\ \theta(n, r) &= \max\{c_3(r), c_2(r, n - r), 1 + c_1(n - r, r, n - r) + \delta(n - r, r) \\ &\quad + \theta(n - r, r)\}, \quad \theta(r, r) = c_3(r). \end{aligned}$$

Proof. The proof is essentially inductive. To save clutter we will omit “ $+O(u^2)$ ” from each bound. For $n = r$, the result holds trivially. Consider the first block stage of the factorization, with the partitioning (13.1). The assumptions imply that

$$\widehat{L}_{11}\widehat{U}_{12} = A_{12} + \Delta A_{12}, \quad \|\Delta A_{12}\| \leq c_2(r, n - r)u\|\widehat{L}_{11}\|\|\widehat{U}_{12}\|, \quad (13.8)$$

$$\widehat{L}_{21}\widehat{U}_{11} = A_{21} + \Delta A_{21}, \quad \|\Delta A_{21}\| \leq c_2(r, n - r)u\|\widehat{L}_{21}\|\|\widehat{U}_{11}\|. \quad (13.9)$$

To obtain $S = A_{22} - L_{21}U_{12}$ we first compute $C = \widehat{L}_{21}\widehat{U}_{12}$, obtaining

$$\widehat{C} = \widehat{L}_{21}\widehat{U}_{12} + \Delta C, \quad \|\Delta C\| \leq c_1(n - r, r, n - r)u\|\widehat{L}_{21}\|\|\widehat{U}_{12}\|,$$

and then subtract from A_{22} , obtaining

$$\widehat{S} = A_{22} - \widehat{C} + F, \quad \|F\| \leq u(\|A_{22}\| + \|\widehat{C}\|). \quad (13.10)$$

It follows that

$$\widehat{S} = A_{22} - \widehat{L}_{21}\widehat{U}_{12} + \Delta S, \quad (13.11a)$$

$$\|\Delta S\| \leq u(\|A_{22}\| + \|\widehat{L}_{21}\|\|\widehat{U}_{12}\| + c_1(n - r, r, n - r)\|\widehat{L}_{21}\|\|\widehat{U}_{12}\|). \quad (13.11b)$$

The remainder of the algorithm consists of the computation of the LU factorization of $\widehat{S}$, and by our inductive assumption (13.7), the computed LU factors satisfy

$$\widehat{L}_{22}\widehat{U}_{22} = \widehat{S} + \Delta \widehat{S}, \quad (13.12a)$$

$$\|\Delta \widehat{S}\| \leq \delta(n - r, r)u\|\widehat{S}\| + \theta(n - r, r)u\|\widehat{L}_{22}\|\|\widehat{U}_{22}\|. \quad (13.12b)$$

Combining (13.11) and (13.12), and bounding $\|\widehat{S}\|$ using (13.10), we obtain

$$\widehat{L}_{21}\widehat{U}_{12} + \widehat{L}_{22}\widehat{U}_{22} = A_{22} + \Delta A_{22},$$

$$\begin{aligned} \|\Delta A_{22}\| &\leq u([1 + \delta(n-r, r)]\|A_{22}\| + [1 + c_1(n-r, r, n-r) + \delta(n-r, r)] \\ &\quad \times \|\widehat{L}_{21}\| \|\widehat{U}_{12}\| + \theta(n-r, r)\|\widehat{L}_{22}\| \|\widehat{U}_{22}\|). \end{aligned} \quad (13.13)$$

Collecting (13.6), (13.8), (13.9), and (13.13) we have $\widehat{L}\widehat{U} = A + \Delta A$, where bounds on $\|\Delta A_{ij}\|$ are given in the equations just mentioned. These bounds for the blocks of ΔA can be weakened slightly and expressed together in the more succinct form (13.7). $\square$

These recurrences for $\delta(n, r)$ and $\theta(n, r)$ show that the basic error constants in assumptions (13.4)–(13.6) combine additively at worst. Thus, the backward error analysis for the LU factorization is commensurate with the error analysis for the particular implementation of the BLAS3 employed in the partitioned factorization. In the case of the conventional BLAS3 we obtain a Wilkinson-style result for GE without pivoting, with $\theta(n, r) = O(n^3)$ (the growth factor is hidden in $\widehat{L}$ and $\widehat{U}$).

Although the above analysis is phrased in terms of the partitioned outer product form of LU factorization, the same result holds for other “*ijk*” partitioned forms (with slightly different constants), for example, the gaxpy or sdot forms and the recursive factorization (Algorithm 13.4). There is no difficulty in extending the analysis to cover partial pivoting and solution of $Ax = b$ using the computed LU factorization (see Problem 13.6).

13.3. Error Analysis of Block LU Factorization

Now we turn to block LU factorization. We assume that the computed matrices $\widehat{L}_{21}$ from step 2 of Algorithm 13.3 satisfy

$$\widehat{L}_{21}A_{11} = A_{21} + E_{21}, \quad \|E_{21}\| \leq c_4(n, r)u\|\widehat{L}_{21}\|\|A_{11}\| + O(u^2). \quad (13.14)$$

We also assume that when a system $U_{ii}x_i = d_i$ of order r is solved, the computed solution $\widehat{x}_i$ satisfies

$$(U_{ii} + \Delta U_{ii})\widehat{x}_i = d_i, \quad \|\Delta U_{ii}\| \leq c_5(r)u\|U_{ii}\| + O(u^2). \quad (13.15)$$

The assumptions (13.14) and (13.15) are satisfied for Implementation 1 of Algorithm 13.3 and are sufficient to prove the following result.

Theorem 13.6 (Demmel, Higham, and Schreiber). *Let $\widehat{L}$ and $\widehat{U}$ be the computed block LU factors of $A \in \mathbb{R}^{n \times n}$ from Algorithm 13.3 (with Implementation 1), and let $\widehat{x}$ be the computed solution to $Ax = b$. Under the assumptions (13.4), (13.14), and (13.15),*

$$\begin{aligned} \widehat{L}\widehat{U} &= A + \Delta A_1, & (A + \Delta A_2)\widehat{x} &= b, \\ \|\Delta A_i\| &\leq d_n u (\|A\| + \|\widehat{L}\| \|\widehat{U}\|) + O(u^2), & i &= 1:2, \end{aligned} \quad (13.16)$$

where the constant d_n is commensurate with those in the assumptions.

Proof. We omit the proof (see Demmel, Higham, and Schreiber [326, 1995] for details). It is similar to the proof of Theorem 13.5. $\square$

The bounds in Theorem 13.6 are valid also for other versions of block LU factorization obtained by “block loop reordering”, such as a block gaxpy based algorithm.

Theorem 13.6 shows that the stability of block LU factorization is determined by the ratio $\|\hat{L}\| \|\hat{U}\| / \|A\|$ (numerical experiments show that the bounds are, in fact, reasonably sharp). If this ratio is bounded by a modest function of n , then $\hat{L}$ and $\hat{U}$ are the true factors of a matrix close to A , and $\hat{x}$ solves a slightly perturbed system. However, $\|\hat{L}\| \|\hat{U}\|$ can exceed $\|A\|$ by an arbitrary factor, even if A is symmetric positive definite or diagonally dominant by rows. Indeed, $\|L\| \geq \|L_{21}\| = \|A_{21}A_{11}^{-1}\|$, using the partitioning (13.2), and this lower bound for $\|L\|$ can be arbitrarily large. In the following two subsections we investigate this instability more closely and show that $\|L\| \|U\|$ can be bounded in a useful way for particular classes of A . Without further comment we make the reasonable assumption that $\|L\| \|U\| \approx \|\hat{L}\| \|\hat{U}\|$, so that these bounds may be used in Theorem 13.6.

What can be said for Implementation 2? Suppose, for simplicity, that the inverses A_{11}^{-1} (which are used in step 2 of Algorithm 13.3 and in the block back substitution) are computed exactly. Then the best bounds of the forms (13.14) and (13.15) are

$$\begin{aligned} \hat{L}_{21}A_{11} &= A_{21} + \Delta A_{21}, & \|\Delta A_{21}\| &\leq c_4(n, r)u\kappa(A_{11})\|A_{21}\| + O(u^2), \\ (U_{ii} + \Delta U_{ii})\hat{x}_i &= d_i, & \|\Delta U_{ii}\| &\leq c_5(r)u\kappa(U_{ii})\|U_{ii}\| + O(u^2). \end{aligned}$$

Working from these results, we find that Theorem 13.6 still holds provided the first-order terms in the bounds in (13.16) are multiplied by $\max_i \kappa(\hat{U}_{ii})$. This suggests that Implementation 2 of Algorithm 13.3 can be much less stable than Implementation 1 when the diagonal blocks of U are ill conditioned, and this is confirmed by numerical experiments.

13.3.1. Block Diagonal Dominance

One class of matrices for which block LU factorization has long been known to be stable is block tridiagonal matrices that are diagonally dominant in an appropriate block sense. A general matrix $A \in \mathbb{R}^{n \times n}$ is *block diagonally dominant by columns* with respect to a given partitioning $A = (A_{ij})$ and a given norm if, for all j ,

$$\|A_{jj}^{-1}\|^{-1} - \sum_{i \neq j} \|A_{ij}\| =: \gamma_j \geq 0. \quad (13.17)$$

This definition implicitly requires that the diagonal blocks A_{jj} are all nonsingular. A is *block diagonally dominant by rows* if A^T is block diagonally dominant by columns. For the block size 1, the usual property of point diagonal dominance is obtained. Note that for the 1- and ∞ -norms diagonal dominance does not imply block diagonal dominance, nor does the reverse implication hold (see Problem 13.2). Throughout our analysis of block diagonal dominance we take the norm to be an arbitrary subordinate matrix norm.

First, we show that for block diagonally dominant matrices a block LU factorization exists, using the key property that block diagonal dominance is inherited by the Schur complements obtained in the course of the factorization. In the analysis we assume that A has m block rows and columns.

Theorem 13.7 (Demmel, Higham, and Schreiber). *Suppose $A \in \mathbb{R}^{n \times n}$ is nonsingular and block diagonally dominant by rows or columns with respect to a subordinate matrix norm in (13.17). Then A has a block LU factorization, and all the Schur complements arising in Algorithm 13.3 have the same kind of diagonal dominance as A .*

Proof. This proof is a generalization of Wilkinson's proof of the corresponding result for point diagonally dominant matrices [1229, 1961, pp. 288–289], [509, 1996, Thm. 3.4.3] (as is the proof of Theorem 13.8 below). We consider the case of block diagonal dominance by columns; the proof for row-wise diagonal dominance is analogous.

The first step of Algorithm 13.3 succeeds, since A_{11} is nonsingular, producing a matrix that we can write as

$$A^{(2)} = \begin{bmatrix} U_{11} & U_{12} \\ 0 & S \end{bmatrix}.$$

For $j = 2:m$ we have

$$\begin{aligned} \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij}^{(2)}\| &= \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij} - A_{i1}A_{11}^{-1}A_{1j}\| \\ &\leq \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij}\| + \|A_{1j}\| \|A_{11}^{-1}\| \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{i1}\| \\ &\leq \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij}\| + \|A_{1j}\| \|A_{11}^{-1}\| (\|A_{11}^{-1}\|^{-1} - \|A_{j1}\|), \quad \text{using (13.17),} \\ &= \sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij}\| + \|A_{1j}\| - \|A_{1j}\| \|A_{11}^{-1}\| \|A_{j1}\| \\ &\leq \|A_{jj}^{-1}\|^{-1} - \|A_{1j}\| \|A_{11}^{-1}\| \|A_{j1}\|, \quad \text{using (13.17),} \\ &= \min_{\|x\|=1} \|A_{jj}x\| - \|A_{1j}\| \|A_{11}^{-1}\| \|A_{j1}\| \\ &\leq \min_{\|x\|=1} \|(A_{jj} - A_{j1}A_{11}^{-1}A_{1j})x\| \\ &= \min_{\|x\|=1} \|A_{jj}^{(2)}x\|. \end{aligned} \tag{13.18}$$

Now if $A_{jj}^{(2)}$ is singular it follows that $\sum_{i=2, i \neq j}^m \|A_{ij}^{(2)}\| = 0$; therefore $A^{(2)}$, and hence also A , is singular, which is a contradiction. Thus $A_{jj}^{(2)}$ is nonsingular, and

(13.18) can be rewritten

$$\sum_{\substack{i=2 \\ i \neq j}}^m \|A_{ij}^{(2)}\| \leq \|A_{jj}^{(2)}\|^{-1},$$

showing that $A^{(2)}$ is block diagonally dominant by columns. The result follows by induction. $\square$

The next result allows us to bound $\|U\|$ for a block diagonally dominant matrix.

Theorem 13.8 (Demmel, Higham, and Schreiber). *Let A satisfy the conditions of Theorem 13.7. If $A^{(k)}$ denotes the matrix obtained after $k - 1$ steps of Algorithm 13.3, then*

$$\max_{k \leq i, j \leq m} \|A_{ij}^{(k)}\| \leq 2 \max_{1 \leq i, j \leq m} \|A_{ij}\|.$$

Proof. Let A be block diagonally dominant by columns (the proof for row diagonal dominance is similar). Then

$$\begin{aligned} \sum_{i=2}^m \|A_{ij}^{(2)}\| &= \sum_{i=2}^m \|A_{ij} - A_{i1}A_{11}^{-1}A_{1j}\| \\ &\leq \sum_{i=2}^m \|A_{ij}\| + \|A_{1j}\| \|A_{11}^{-1}\| \sum_{i=2}^m \|A_{i1}\| \\ &\leq \sum_{i=1}^m \|A_{ij}\|, \end{aligned}$$

using (13.17). By induction, using Theorem 13.7, it follows that $\sum_{i=k}^m \|A_{ij}^{(k)}\| \leq \sum_{i=1}^m \|A_{ij}\|$. This yields

$$\max_{k \leq i, j \leq m} \|A_{ij}^{(k)}\| \leq \max_{k \leq j \leq m} \sum_{i=k}^m \|A_{ij}^{(k)}\| \leq \max_{k \leq j \leq m} \sum_{i=1}^m \|A_{ij}\|.$$

From (13.17), $\sum_{i \neq j} \|A_{ij}\| \leq \|A_{jj}^{-1}\|^{-1} \leq \|A_{jj}\|$, so

$$\max_{k \leq i, j \leq m} \|A_{ij}^{(k)}\| \leq 2 \max_{k \leq j \leq m} \|A_{jj}\| \leq 2 \max_{1 \leq j \leq m} \|A_{jj}\| = 2 \max_{1 \leq i, j \leq m} \|A_{ij}\|. \quad \square$$

The implications of Theorems 13.7 and 13.8 for stability are as follows. Suppose A is block diagonally dominant by columns. Also, assume for the moment that the (subordinate) norm has the property that

$$\max_{i,j} \|A_{ij}\| \leq \|A\| \leq \sum_{i,j} \|A_{ij}\|, \quad (13.19)$$

which holds for any p -norm, for example. The subdiagonal blocks in the first block column of L are given by $L_{i1} = A_{i1}A_{11}^{-1}$ and so $\|[L_{21}^T, \dots, L_{m1}^T]^T\| \leq 1$, by (13.17) and (13.19). From Theorem 13.7 it follows that $\|[L_{j+1,j}^T, \dots, L_{mj}^T]^T\| \leq 1$

for $j = 2:m$. Since $U_{ij} = A_{ij}^{(i)}$ for $j \geq i$, Theorem 13.8 shows that $\|U_{ij}\| \leq 2\|A\|$ for each block of U (and $\|U_{ii}\| \leq \|A\|$). Therefore $\|L\| \leq m$ and $\|U\| \leq m^2\|A\|$, and so $\|L\|\|U\| \leq m^3\|A\|$. For particular norms the bounds on the blocks of L and U yield a smaller bound for $\|L\|$ and $\|U\|$. For example, for the 1-norm we have $\|L\|_1\|U\|_1 \leq 2m\|A\|_1$ and for the ∞ -norm $\|L\|_\infty\|U\|_\infty \leq 2m^2\|A\|_\infty$. We conclude that block LU factorization is stable if A is block diagonally dominant by columns with respect to any subordinate matrix norm satisfying (13.19).

Unfortunately, block LU factorization can be unstable when A is block diagonally dominant by rows, for although Theorem 13.8 guarantees that $\|U_{ij}\| \leq 2\|A\|$, $\|L\|$ can be arbitrarily large. This can be seen from the example

$$A = \begin{bmatrix} A_{11} & 0 \\ \frac{1}{2}I & I \end{bmatrix} = \begin{bmatrix} I & 0 \\ \frac{1}{2}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & 0 \\ 0 & I \end{bmatrix} = LU,$$

where A is block diagonally dominant by rows in any subordinate norm for any nonsingular matrix A_{11} . It is easy to confirm numerically that block LU factorization can be unstable on matrices of this form.

Next, we bound $\|L\|\|U\|$ for a general matrix and then specialize to point diagonal dominance. *From this point on we use the norm $\|A\| := \max_{i,j} |a_{ij}|$.* We partition A according to

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad A_{11} \in \mathbb{R}^{r \times r}, \quad (13.20)$$

and denote by ρ_n the growth factor for GE without pivoting. We assume that GE applied to A succeeds.

To bound $\|L\|$, we note that, under the partitioning (13.20), for the first block stage of Algorithm 13.3 we have $\|L_{21}\| = \|A_{21}A_{11}^{-1}\| \leq n\rho_n\kappa(A)$ (see Problem 13.4). Since the algorithm works recursively with the Schur complement S , and since every Schur complement satisfies $\kappa(S) \leq \rho_n\kappa(A)$ (see Problem 13.4), each subsequently computed subdiagonal block of L has norm at most $n\rho_n^2\kappa(A)$. Since U is composed of elements of A together with elements of Schur complements of A ,

$$\|U\| \leq \rho_n\|A\|. \quad (13.21)$$

Overall, then, for a general matrix $A \in \mathbb{R}^{n \times n}$,

$$\|L\|\|U\| \leq n\rho_n^2\kappa(A) \cdot \rho_n\|A\| = n\rho_n^3\kappa(A)\|A\|. \quad (13.22)$$

Thus, block LU factorization is stable for a general matrix A as long as GE is stable for A (that is, ρ_n is of order 1) and A is well conditioned.

If A is point diagonally dominant by columns then, since every Schur complement enjoys the same property, we have $\|L_{ij}\| \leq 1$ for $i > j$, by Problem 13.5. Hence $\|L\| = 1$. Furthermore, $\rho_n \leq 2$ (Theorem 9.9 or Theorem 13.8), giving $\|U\| \leq 2\|A\|$ by (13.21), and so

$$\|L\|\|U\| \leq 2\|A\|.$$

Thus block LU factorization is perfectly stable for a matrix point diagonally dominant by columns.

If A is point diagonally dominant by rows then the best we can do is to take $\rho_n \leq 2$ in (13.22), obtaining

$$\|L\| \|U\| \leq 8n\kappa(A)\|A\|. \quad (13.23)$$

Hence for point row diagonally dominant matrices, stability is guaranteed if A is well conditioned. This in turn is guaranteed if the row diagonal dominance amounts γ_j in the analogue of (13.17) for point row diagonal dominance are sufficiently large relative to $\|A\|$, because $\|A^{-1}\|_\infty \leq (\min_j \gamma_j)^{-1}$ (see Problem 8.7(a)).

13.3.2. Symmetric Positive Definite Matrices

Further useful results about the stability of block LU factorization can be derived for symmetric positive definite matrices. First, note that the existence of a block LU factorization is immediate for such matrices, since all their leading principal submatrices are nonsingular. Let A be a symmetric positive definite matrix, partitioned as

$$A = \begin{bmatrix} A_{11} & A_{21}^T \\ A_{21} & A_{22} \end{bmatrix}, \quad A_{11} \in \mathbb{R}^{r \times r}.$$

The definiteness implies certain relations among the submatrices A_{ij} that can be used to obtain a stronger bound for $\|L\|_2$ than can be deduced for a general matrix (cf. Problem 13.4).

Lemma 13.9. *If A is symmetric positive definite then $\|A_{21}A_{11}^{-1}\|_2 \leq \kappa_2(A)^{1/2}$.*

Proof. This lemma is a corollary of Lemma 10.12, but we give a separate proof. Let A have the Cholesky factorization

$$A = \begin{bmatrix} R_{11}^T & 0 \\ R_{12}^T & R_{22}^T \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}, \quad R_{11} \in \mathbb{R}^{r \times r}.$$

Then $A_{21}A_{11}^{-1} = R_{12}^T R_{11} \cdot R_{11}^{-1} R_{11}^{-T} = R_{12}^T R_{11}^{-T}$, so

$$\|A_{21}A_{11}^{-1}\|_2 \leq \|R_{12}\|_2 \|R_{11}^{-1}\|_2 \leq \|R\|_2 \|R^{-1}\|_2 = \kappa_2(R) = \kappa_2(A)^{1/2}. \quad \square$$

The following lemma is proved in a way similar to the second inequality in Problem 13.4.

Lemma 13.10. *If A is symmetric positive definite then the Schur complement $S = A_{22} - A_{21}A_{11}^{-1}A_{21}^T$ satisfies $\kappa_2(S) \leq \kappa_2(A)$.*

Using the same reasoning as in the last subsection, we deduce from these two lemmas that each subdiagonal block of L is bounded in 2-norm by $\kappa_2(A)^{1/2}$. Therefore $\|L\|_2 \leq 1 + m\kappa_2(A)^{1/2}$, where there are m block stages in the algorithm. Also, it can be shown that $\|U\|_2 \leq \sqrt{m}\|A\|_2$. Hence

$$\|L\|_2 \|U\|_2 \leq \sqrt{m}(1 + m\kappa_2(A)^{1/2})\|A\|_2. \quad (13.24)$$

Table 13.1. *Stability of block and point LU factorization. ρ_n is the growth factor for GE without pivoting.*

Matrix property	Block LU	Point LU
Symmetric positive definite	$\kappa(A)^{1/2}$	1
Block column diagonally dominant	1	ρ_n
Point column diagonally dominant	1	1
Block row diagonally dominant	$\rho_n^3 \kappa(A)$	ρ_n
Point row diagonally dominant	$\kappa(A)$	1
Arbitrary	$\rho_n^3 \kappa(A)$	ρ_n

It follows from Theorem 13.6 that when Algorithm 13.3 is applied to a symmetric positive definite matrix A , the backward errors for the LU factorization and the subsequent solution of a linear system are both bounded by

$$c_n \sqrt{mu} \|A\|_2 (2 + m\kappa_2(A)^{1/2}) + O(u^2). \quad (13.25)$$

Any resulting bound for $\|x - \hat{x}\|_2 / \|x\|_2$ will be proportional to $\kappa_2(A)^{3/2}$, rather than $\kappa_2(A)$ as for a stable method. This suggests that block LU factorization can lose up to 50% more digits of accuracy in x than a stable method for solving symmetric positive definite linear systems. The positive conclusion to be drawn, however, is that block LU factorization is guaranteed to be stable for a symmetric positive definite matrix that is well conditioned.

The stability results for block LU factorization are summarized in Table 13.1, which tabulates a bound for $\|A - \hat{L}\hat{U}\| / (c_n u \|A\|)$ for block and point LU factorization for the matrix properties considered in this chapter. The constant c_n incorporates any constants in the bound that depend polynomially on the dimension, so a value of 1 in the table indicates unconditional stability.

13.4. Notes and References

The distinction between a partitioned algorithm and a block algorithm is rarely made in the literature (exceptions include the papers by Schreiber [1021, 1988] and Demmel, Higham, and Schreiber [326, 1995]); the term “block algorithm” is frequently used to describe both types of algorithm. A partitioned algorithm might also be called a “blocked algorithm” (as is done by Dongarra, Duff, Sorensen, and van der Vorst [349, 1998]), but the similarity of this term to “block algorithm” can cause confusion and so we do not recommend this terminology. Note that in the particular case of matrix multiplication, partitioned and block algorithms are equivalent. Our treatment of partitioned LU factorization has focused on the stability aspects; for further details, particularly concerning implementation on high-performance computers, see Dongarra, Duff, Sorensen, and van der Vorst [349, 1998] and Golub and Van Loan [509, 1996].

Recursive LU factorization is now regarded as the most efficient way in which to implement LU factorization on machines with hierarchical memories [535, 1997], [1145, 1997], but it has not yet been incorporated into LAPACK.

Block LU factorization appears to have first been proposed for block tridiagonal matrices, which frequently arise in the discretization of partial differential equations. References relevant to this application include Isaacson and Keller [667, 1966, p. 59], Varah [1187, 1972], Bank and Rose [62, 1977], Mattheij [827, 1984], [828, 1984], and Concus, Golub, and Meurant [262, 1985].

For an application of block LU factorization to linear programming, see Eldersveld and Saunders [388, 1992].

Theorem 13.5 is from Demmel and Higham [324, 1992]. The results in §13.3 are from Demmel, Higham, and Schreiber [326, 1995], which extends earlier analysis of block LU factorization by Demmel and Higham [324, 1992].

Block diagonal dominance was introduced by Feingold and Varga [406, 1962], and has been used mainly in generalizations of the Gershgorin circle theorem. Varah [1187, 1972] obtained bounds on $\|L\|$ and $\|U\|$ for block diagonally dominant block tridiagonal matrices; see Problem 13.1.

Theorem 13.7 is obtained in the case of block diagonal dominance by rows with $\min_j \gamma_j > 0$ by Polman [946, 1987]; the proof in [946, 1987] makes use of the corresponding result for point diagonal dominance and thus differs from the proof we have given.

At the cost of a much more difficult proof, Lemma 13.9 can be strengthened to the attainable bound $\|A_{21}A_{11}^{-1}\|_2 \leq (\kappa_2(A)^{1/2} - \kappa_2(A)^{-1/2})/2$, as shown by Demmel [307, 1983, Thm. 4], but the weaker bound is sufficient for our purposes.

13.4.1. LAPACK

LAPACK does not implement block LU factorization, but its LU factorization (and related) routines for full matrices employ partitioned LU factorization in order to exploit the level-3 BLAS and thereby to be efficient on high-performance machines.

Problems

13.1. (Varah [1187, 1972]) Suppose A is block tridiagonal and has the block LU factorization $A = LU$ (so that L and U are block bidiagonal and $U_{i,i+1} = A_{i,i+1}$). Show that if A is block diagonally dominant by columns then

$$\|L_{i,i-1}\| \leq 1, \quad \|U_{ii}\| \leq \|A_{ii}\| + \|A_{i-1,i}\|,$$

while if A is block diagonally dominant by rows then

$$\|L_{i,i-1}\| \leq \|A_{i,i-1}\|/\|A_{i-1,i}\|, \quad \|U_{ii}\| \leq \|A_{ii}\| + \|A_{i,i-1}\|.$$

What can be deduced about the stability of the factorization for these two classes of matrices?

13.2. Show that for the 1- and ∞ -norms diagonal dominance does not imply block diagonal dominance, and vice versa.

13.3. If $A \in \mathbb{R}^{n \times n}$ is symmetric, has positive diagonal elements, and is block diagonally dominant by rows, must it be positive definite?

13.4. Let $A \in \mathbb{R}^{n \times n}$ be partitioned

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad A_{11} \in \mathbb{R}^{r \times r}, \quad (13.26)$$

with A_{11} nonsingular. Let $\|A\| := \max_{ij} |a_{ij}|$. Show that $\|A_{21}A_{11}^{-1}\| \leq n\rho_n\kappa(A)$, where ρ_n is the growth factor for GE without pivoting on A . Show that the Schur complement $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$ satisfies $\kappa(S) \leq \rho_n\kappa(A)$.

13.5. Let $A \in \mathbb{R}^{n \times n}$ be partitioned as in (13.26), with A_{11} nonsingular, and suppose that A is point diagonally dominant by columns. Show that $\|A_{21}A_{11}^{-1}\|_1 \leq 1$.

13.6. Show that under the conditions of Theorem 13.5 the computed solution to $Ax = b$ satisfies

$$(A + \Delta A)\hat{x} = b, \quad \|\Delta A\| \leq c_n u (\|A\| + \|\hat{L}\| \|\hat{U}\|) + O(u^2),$$

and the computed solution to the multiple right-hand side system $AX = B$ (where (13.5) is assumed to hold for the multiple right-hand side triangular solves) satisfies

$$\|A\hat{X} - B\| \leq c_n u (\|A\| + \|\hat{L}\| \|\hat{U}\|) \|\hat{X}\| + O(u^2).$$

In both cases, c_n is a constant depending on n and the block size.

13.7. Let $X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \in \mathbb{R}^{n \times n}$, where A is square and nonsingular. Show that

$$\det(X) = \det(A) \det(D - CA^{-1}B).$$

Assuming A, B, C, D are all $m \times m$, give a condition under which $\det(X) = \det(AD - CB)$.

13.8. By using a block LU factorization show that

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} + A^{-1}BS^{-1}CA^{-1} & -A^{-1}BS^{-1} \\ -S^{-1}CA^{-1} & S^{-1} \end{bmatrix},$$

where A is assumed to be nonsingular and $S = D - CA^{-1}B$.

13.9. Let $A \in \mathbb{R}^{n \times m}$, $B \in \mathbb{R}^{m \times n}$. Derive the expression

$$(I - AB)^{-1} = I + A(I - BA)^{-1}B$$

by considering block LU and block UL factorizations of $\begin{bmatrix} I & A \\ B & I \end{bmatrix}$. Deduce the Sherman–Morrison–Woodbury formula

$$(T - UW^{-1}V^T)^{-1} = T^{-1} + T^{-1}U(W - V^T T^{-1}U)^{-1}V^T T^{-1},$$

where $T \in \mathbb{R}^{n \times n}$, $U \in \mathbb{R}^{n \times r}$, $W \in \mathbb{R}^{r \times r}$, $V \in \mathbb{R}^{r \times n}$.

Chapter 14

Matrix Inversion

It is amusing to remark that we were so involved with matrix inversion that we probably talked of nothing else for months.

Just in this period Mrs. von Neumann acquired a big, rather wild but gentle Irish Setter puppy, which she called Inverse in honor of our work!

— HERMAN H. GOLDSTINE, *The Computer: From Pascal to von Neumann* (1972)

The most computationally intensive portion of the tasks assigned to the processors is integrating the KKR matrix inverse over the first Brillouin zone.

To evaluate the integral, hundreds or possibly thousands of complex double precision matrices of order between 80 and 300 must be formed and inverted. Each matrix corresponds to a different vertex of the tetrahedrons into which the Brillouin zone has been subdivided.

— M. T. HEATH, G. A. GEIST, and J. B. DRAKE,
Superconductivity Computations,
in Early Experience with the Intel iPSC/860
at Oak Ridge National Laboratory (1990)

Press $\boxed{1/x}$ to invert the matrix.
Note that matrix inversion can produce erroneous results if you are using ill-conditioned matrices.

— HEWLETT-PACKARD, *HP 48G Series User's Guide* (1993)

Almost anything you can do with A^{-1} can be done without it.

— GEORGE E. FORSYTHE and CLEVE B. MOLER,
Computer Solution of Linear Algebraic Systems (1967)

14.1. Use and Abuse of the Matrix Inverse

To most numerical analysts, matrix inversion is a sin. Forsythe, Malcolm, and Moler put it well when they say [430, 1977, p. 31] “In the vast majority of practical computational problems, it is unnecessary and inadvisable to actually compute A^{-1} .” The best example of a problem in which the matrix inverse should not be computed is the linear equations problem $Ax = b$. Computing the solution as $x = A^{-1} \times b$ requires $2n^3$ flops, assuming A^{-1} is computed by Gaussian elimination with partial pivoting (GEPP), whereas GEPP applied directly to the system costs only $2n^3/3$ flops.

Not only is the inversion approach three times more expensive, but it is much less stable. Suppose $X = A^{-1}$ is formed *exactly*, and that the only rounding errors are in forming $x = fl(Xb)$. Then $\hat{x} = (X + \Delta X)b$, where $|\Delta X| \leq \gamma_n |X|$, by (3.11). So $A\hat{x} = A(X + \Delta X)b = (I + A\Delta X)b$, and the best possible residual bound is

$$|b - A\hat{x}| \leq \gamma_n |A| |A^{-1}| |b|.$$

For GEPP, Theorem 9.4 yields

$$|b - A\hat{x}| \leq \gamma_{3n} |\hat{L}| |\hat{U}| |\hat{x}|.$$

Since it is usually true that $\|\hat{L}\|\hat{U}\|_\infty \approx \|A\|_\infty$ for GEPP, we see that the matrix inversion approach is likely to give a much larger residual than GEPP if A is ill conditioned and if $\|x\|_\infty \ll \| |A^{-1}| |b| \|_\infty$. For example, we solved fifty 25×25 systems $Ax = b$ in MATLAB, where the elements of x are taken from the normal $N(0, 1)$ distribution and A is random with $\kappa_2(A) = u^{-1/2} \approx 9 \times 10^7$. As Table 14.1 shows, the inversion approach provided much larger backward errors than GEPP in this experiment.

Given the inexpedience of matrix inversion, why devote a chapter to it? The answer is twofold. First, there *are* situations in which a matrix inverse must be computed. Examples are in statistics [63, 1974, §7.5], [806, 1984, §2.3], [834, 1989, p. 342 ff], where the inverse can convey important statistical information, in certain matrix iterations arising in eigenvalue-related problems [47, 1997], [193, 1987], [621, 1994], in the computation of matrix functions such as the square root [609, 1997] and the logarithm [231, 2001], and in numerical integrations arising in superconductivity computations [555, 1990] (see the quotation at the start of the chapter). Second, methods for matrix inversion display a wide variety of stability properties, making for instructive and challenging error analysis. (Indeed, the first major rounding error analysis to be published, that of von Neumann and Goldstine, was for matrix inversion—see §9.13).

Table 14.1. Backward errors $\eta_{A,b}(\hat{x})$ for the ∞ -norm.

	min	max
$x = A^{-1} \times b$	6.66e-12	1.69e-10
GEPP	3.44e-18	7.56e-17

Matrix inversion can be done in many different ways—in fact, there are more computationally distinct possibilities than for any other basic matrix computation. For example, in triangular matrix inversion different loop orderings are possible and either triangular matrix–vector multiplication, solution of a triangular system, or a rank-1 update of a rectangular matrix can be employed inside the outer loop. More generally, given a factorization $PA = LU$, two ways to evaluate A^{-1} are as $A^{-1} = U^{-1} \times L^{-1} \times P$, and as the solution to $UA^{-1} = L^{-1} \times P$. These methods generally achieve different levels of efficiency on high-performance computers, and they propagate rounding errors in different ways. We concentrate in this chapter on the numerical stability, but comment briefly on performance issues.

The quality of an approximation $Y \approx A^{-1}$ can be assessed by looking at the right and left residuals, $AY - I$ and $YA - I$, and the forward error, $Y - A^{-1}$. Suppose we perturb $A \rightarrow A + \Delta A$ with $|\Delta A| \leq \epsilon|A|$; thus, we are making relative perturbations of size at most ϵ to the elements of A . If $Y = (A + \Delta A)^{-1}$ then $(A + \Delta A)Y = Y(A + \Delta A) = I$, so that

$$|AY - I| = |\Delta AY| \leq \epsilon|A||Y|, \quad (14.1)$$

$$|YA - I| = |Y\Delta A| \leq \epsilon|Y||A|, \quad (14.2)$$

and, since $(A + \Delta A)^{-1} = A^{-1} - A^{-1}\Delta AA^{-1} + O(\epsilon^2)$,

$$|A^{-1} - Y| \leq \epsilon|A^{-1}||A||A^{-1}| + O(\epsilon^2). \quad (14.3)$$

(Note that (14.3) can also be derived from (14.1) or (14.2).) The bounds (14.1)–(14.3) represent “ideal” bounds for a computed approximation Y to A^{-1} , if we regard ϵ as a small multiple of the unit roundoff u . We will show that, for triangular matrix inversion, appropriate methods do indeed achieve (14.1) or (14.2) (but not both) and (14.3).

It is important to note that neither (14.1), (14.2), nor (14.3) implies that $Y + \Delta Y = (A + \Delta A)^{-1}$ with $\|\Delta A\|_\infty \leq \epsilon\|A\|_\infty$ and $\|\Delta Y\|_\infty \leq \epsilon\|Y\|_\infty$, that is, Y need not be close to the inverse of a matrix near to A , even in the norm sense. Indeed, such a result would imply that both the left and right residuals are bounded in norm by $(2\epsilon + \epsilon^2)\|A\|_\infty\|Y\|_\infty$, and this is not the case for any of the methods we will consider.

To illustrate the latter point we give a numerical example. Define the matrix $A_n \in \mathbb{R}^{n \times n}$ as `triu(qr(vand(n)))`, in MATLAB notation (`vand` is a routine from the Matrix Computation Toolbox—see Appendix D); in other words, A_n is the upper triangular QR factor of the $n \times n$ Vandermonde matrix based on equispaced points on $[0, 1]$. We inverted A_n , for $n = 5:80$, using MATLAB’s `inv` function, which uses GEPP. The left and right normwise relative residuals

$$\text{res}_L = \frac{\|\hat{X}A - I\|_\infty}{\|\hat{X}\|_\infty\|A\|_\infty}, \quad \text{res}_R = \frac{\|A\hat{X} - I\|_\infty}{\|A\|_\infty\|\hat{X}\|_\infty},$$

are plotted in Figure 14.1. We see that while the left residual is always less than the unit roundoff, the right residual becomes large as n increases. These matrices are very ill conditioned (singular to working precision for $n \geq 20$), yet it is still reasonable to expect a small residual, and we will prove in §14.3.2 that the left residual must be small, independent of the condition number.

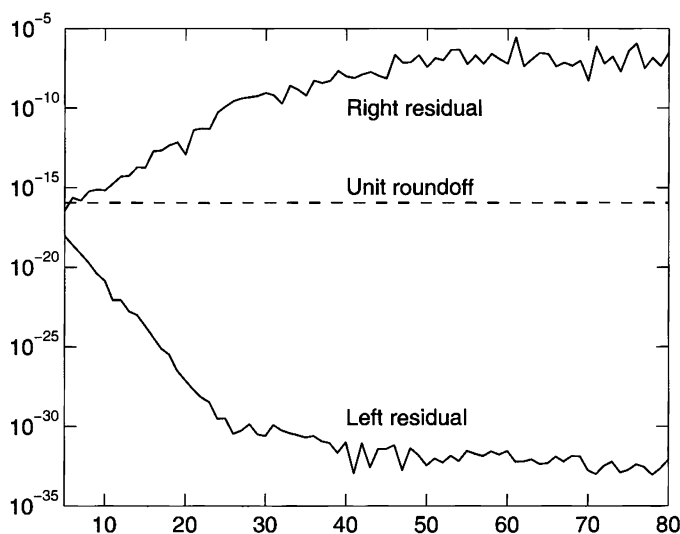


Figure 14.1. *Residuals for inverses computed by MATLAB's inv function.*

In most of this chapter we are not concerned with the precise values of constants (§14.4 is the exception); thus c_n denotes a constant of order n . To simplify the presentation we introduce a special notation. Let $A_i \in \mathbb{R}^{m_i \times n_i}$, $i = 1:k$, be matrices such that the product $A_1 A_2 \dots A_k$ is defined and let

$$p = \sum_{i=1}^{k-1} n_i.$$

Then $\Delta(A_1, A_2, \dots, A_k) \in \mathbb{R}^{m_1 \times n_k}$ denotes a matrix bounded according to

$$|\Delta(A_1, A_2, \dots, A_k)| \leq c_p u |A_1| |A_2| \dots |A_k|.$$

This notation is chosen so that if $\hat{C} = fl(A_1 A_2 \dots A_k)$, with the product evaluated in any order, then

$$\hat{C} = A_1 A_2 \dots A_k + \Delta(A_1, A_2, \dots, A_k).$$

14.2. Inverting a Triangular Matrix

We consider the inversion of a lower triangular matrix $L \in \mathbb{R}^{n \times n}$, treating unblocked and blocked methods separately. We do not make a distinction between partitioned and block methods in this section. All the results in this and the next section are from Du Croz and Higham [357, 1992].

14.2.1. Unblocked Methods

We focus our attention on two “ j ” methods that compute L^{-1} a column at a time. Analogous “ i ” and “ k ” methods exist, which compute L^{-1} row-wise or use outer products, respectively, and we comment on them at the end of the section.

The first method computes each column of $X = L^{-1}$ independently, using forward substitution. We write it as follows, to facilitate comparison with the second method.

Method 1.

```

for  $j = 1:n$ 
   $x_{jj} = l_{jj}^{-1}$ 
   $X(j+1:n, j) = -x_{jj}L(j+1:n, j)$ 
  Solve  $L(j+1:n, j+1:n)X(j+1:n, j) = X(j+1:n, j)$ ,
  by forward substitution.
end

```

In BLAS terminology, this method is dominated by n calls to the level-2 BLAS routine **xTRSV** (TRiangular SolVe).

The second method computes the columns in the reverse order. On the j th step it multiplies by the previously computed inverse $L(j+1:n, j+1:n)^{-1}$ instead of solving a system with coefficient matrix $L(j+1:n, j+1:n)$.

Method 2.

```

for  $j = n:-1:1$ 
   $x_{jj} = l_{jj}^{-1}$ 
   $X(j+1:n, j) = X(j+1:n, j+1:n)L(j+1:n, j)$ 
   $X(j+1:n, j) = -x_{jj}X(j+1:n, j)$ 
end

```

Method 2 uses n calls to the level-2 BLAS routine **xTRMV** (TRiangular Matrix times Vector). On most high-performance machines **xTRMV** can be implemented to run faster than **xTRSV**, so Method 2 is generally preferable to Method 1 from the point of view of efficiency (see the performance figures at the end of §14.2.2). We now compare the stability of the two methods.

Theorem 8.5 shows that the j th column of the computed $\hat{X}$ from Method 1 satisfies

$$(L + \Delta L_j)\hat{x}_j = e_j, \quad |\Delta L_j| \leq c_n u |L|.$$

It follows that we have the componentwise residual bound

$$|L\hat{X} - I| \leq c_n u |L| |\hat{X}| \quad (14.4)$$

and the componentwise forward error bound

$$|\hat{X} - L^{-1}| \leq c_n u |L^{-1}| |L| |\hat{X}|. \quad (14.5)$$

Since $\hat{X} = L^{-1} + O(u)$, (14.5) can be written as

$$|\hat{X} - L^{-1}| \leq c_n u |L^{-1}| |L| |L^{-1}| + O(u^2), \quad (14.6)$$

which is invariant under row and column scaling of L . If we take norms we obtain normwise relative error bounds that are either row or column scaling independent: from (14.6) we have

$$\frac{\|\hat{X} - L^{-1}\|_\infty}{\|L^{-1}\|_\infty} \leq c_n u \operatorname{cond}(L^{-1}) + O(u^2), \quad (14.7)$$

and the same bound holds with $\text{cond}(L^{-1})$ replaced by $\text{cond}(L)$.

Notice that (14.4) is a bound for the *right residual*, $L\hat{X} - I$. This is because Method 1 is derived by solving $LX = I$. Conversely, Method 2 can be derived by solving $XL = I$, which suggests that we should look for a bound on the *left residual* for this method.

Lemma 14.1. *The computed inverse $\hat{X}$ from Method 2 satisfies*

$$|\hat{X}L - I| \leq c_n u |\hat{X}| |L|. \quad (14.8)$$

Proof. The proof is by induction on n , the case $n = 1$ being trivial. Assume the result is true for $n - 1$ and write

$$L = \begin{bmatrix} \alpha & 0 \\ y & M \end{bmatrix}, \quad X = L^{-1} = \begin{bmatrix} \beta & 0 \\ z & N \end{bmatrix},$$

where $\alpha, \beta \in \mathbb{R}$, $y, z \in \mathbb{R}^{n-1}$, and $M, N \in \mathbb{R}^{(n-1) \times (n-1)}$. Method 2 computes the first column of X by solving $XL = I$ according to

$$\beta = \alpha^{-1}, \quad z = -\beta Ny.$$

In floating point arithmetic we obtain

$$\begin{aligned} \hat{\beta} &= \alpha^{-1}(1 + \delta), & |\delta| &\leq u, \\ \hat{z} &= -\hat{\beta}\hat{N}y + \Delta(\hat{\beta}, \hat{N}, y). \end{aligned}$$

Thus

$$\begin{aligned} \hat{\beta}\alpha &= 1 + \delta, \\ \hat{z}\alpha + \hat{N}y &= -\delta\hat{N}y + \alpha\Delta(\hat{\beta}, \hat{N}, y). \end{aligned}$$

This may be written as

$$\begin{aligned} |\hat{X}L - I|(1:n, 1) &\leq \begin{bmatrix} u \\ u|\hat{N}||y| + c_n u(1 + u)|\hat{N}||y| \end{bmatrix} \\ &\leq c'_n u (|\hat{X}||L|)(1:n, 1). \end{aligned}$$

By assumption, the corresponding inequality holds for the $(2:n, 2:n)$ submatrices and so the result is proved. $\square$

Lemma 14.1 shows that Method 2 has a left residual analogue of the right residual bound (14.4) for Method 1. Since there is, in general, no reason to choose between a small right residual and a small left residual, our conclusion is that Methods 1 and 2 have equally good numerical stability properties.

More generally, it can be shown that all three i , j , and k inversion variants that can be derived from the equations $LX = I$ produce identical rounding errors under suitable implementations, and all satisfy the same right residual bound; likewise, the three variants corresponding to the equation $XL = I$ all satisfy the same left residual bound. The LINPACK routine `xTRDI` uses a k variant derived from $XL = I$; the LINPACK routines `xGEDI` and `xPODI` contain analogous code for inverting an upper triangular matrix (but the *LINPACK Users' Guide* [341, 1979, Chaps. 1 and 3] describes a different variant from the one used in the code).

14.2.2. Block Methods

Let the lower triangular matrix $L \in \mathbb{R}^{n \times n}$ be partitioned in block form as

$$L = \begin{bmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ \vdots & & \ddots & \\ L_{N1} & \dots & \dots & L_{NN} \end{bmatrix}, \quad (14.9)$$

where we place no restrictions on the block sizes, other than to require the diagonal blocks to be square. The most natural block generalizations of Methods 1 and 2 are as follows. Here, we use the notation $L_{p:q,r:s}$ to denote the submatrix comprising the intersection of block rows p to q and block columns r to s of L .

Method 1B.

```
for  $j = 1:N$ 
   $X_{jj} = L_{jj}^{-1}$  (by Method 1)
   $X_{j+1:N,j} = -L_{j+1:N,j} X_{jj}$ 
  Solve  $L_{j+1:N,j+1:N} X_{j+1:N,j} = X_{j+1:N,j}$ ,
  by forward substitution
end
```

Method 2B.

```
for  $j = N:-1:1$ 
   $X_{jj} = L_{jj}^{-1}$  (by Method 2)
   $X_{j+1:N,j} = X_{j+1:N,j+1:N} L_{j+1:N,j}$ 
   $X_{j+1:N,j} = -X_{j+1:N,j} X_{jj}$ 
end
```

One can argue that Method 1B carries out the same arithmetic operations as Method 1, although possibly in a different order, and that it therefore satisfies the same error bound (14.4). For completeness, we give a direct proof.

Lemma 14.2. *The computed inverse $\hat{X}$ from Method 1B satisfies*

$$|L\hat{X} - I| \leq c_n u |L| |\hat{X}|. \quad (14.10)$$

Proof. Equating block columns in (14.10), we obtain the N independent inequalities

$$|L\hat{X}_{1:N,j} - I_{1:N,j}| \leq c_n u |L| |\hat{X}_{1:N,j}|, \quad j = 1:N. \quad (14.11)$$

It suffices to verify the inequality with $j = 1$. Write

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}, \quad X = \begin{bmatrix} X_{11} & 0 \\ X_{21} & X_{22} \end{bmatrix},$$

where $L_{11}, X_{11} \in \mathbb{R}^{r \times r}$ and L_{11} is the $(1,1)$ block in the partitioning of (14.9). X_{11} is computed by Method 1 and so, from (14.4),

$$|L_{11}\hat{X}_{11} - I| \leq c_r u |L_{11}| |\hat{X}_{11}| = c_r u (|L| |\hat{X}|)_{11}. \quad (14.12)$$

X_{21} is computed by forming $T = -L_{21}X_{11}$ and solving $L_{22}X_{21} = T$. The computed $\hat{X}_{21}$ satisfies

$$L_{22}\hat{X}_{21} + \Delta(L_{22}, \hat{X}_{21}) = -L_{21}\hat{X}_{11} + \Delta(L_{21}, \hat{X}_{11}).$$

Hence

$$\begin{aligned} |L_{21}\hat{X}_{11} + L_{22}\hat{X}_{21}| &\leq c_n u (|L_{21}||\hat{X}_{11}| + |L_{22}||\hat{X}_{21}|) \\ &= c_n u (|L||\hat{X}|)_{21}. \end{aligned} \quad (14.13)$$

Together, inequalities (14.12) and (14.13) are equivalent to (14.11) with $j = 1$, as required. $\square$

We can attempt a similar analysis for Method 2B. With the same notation as above, X_{21} is computed as $X_{21} = -X_{22}L_{21}X_{11}$. Thus

$$\hat{X}_{21} = -\hat{X}_{22}L_{21}\hat{X}_{11} + \Delta(\hat{X}_{22}, L_{21}, \hat{X}_{11}). \quad (14.14)$$

To bound the left residual we have to postmultiply by L_{11} and use the fact that X_{11} is computed by Method 2:

$$\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}(I + \Delta(\hat{X}_{11}, L_{11})) = \Delta(\hat{X}_{22}, L_{21}, \hat{X}_{11})L_{11}.$$

This leads to a bound of the form

$$|\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}| \leq c_n u |\hat{X}_{22}||L_{21}||\hat{X}_{11}||L_{11}|,$$

which would be of the desired form in (14.8) were it not for the factor $|\hat{X}_{11}||L_{11}|$. This analysis suggests that the left residual is not guaranteed to be small. Numerical experiments confirm that the left and right residuals can be large simultaneously for Method 2B, although examples are quite hard to find [357, 1992]; therefore the method must be regarded as unstable when the block size exceeds 1.

The reason for the instability is that there are *two* plausible block generalizations of Method 2 and we have chosen an unstable one that does not carry out the same arithmetic operations as Method 2. If we perform a solve with L_{jj} instead of multiplying by X_{jj} we obtain the second variation, which is used by LAPACK's `xTRTRI`.

Method 2C.

for $j = N:-1:1$

$$X_{jj} = L_{jj}^{-1} \text{ (by Method 2)}$$

$$X_{j+1:N,j} = X_{j+1:N,j+1:N}L_{j+1:N,j}$$

$$\text{Solve } X_{j+1:N,j}L_{jj} = -X_{j+1:N,j} \text{ by back substitution.}$$

end

For this method, the analogue of (14.14) is

$$\hat{X}_{21}L_{11} + \Delta(\hat{X}_{21}, L_{11}) = -\hat{X}_{22}L_{21} + \Delta(\hat{X}_{22}, L_{21}),$$

which yields

$$|\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}| \leq c_n u (|\hat{X}_{21}||L_{11}| + |\hat{X}_{22}||L_{21}|).$$

Hence Method 2C enjoys a very satisfactory residual bound.

Table 14.2. *Mflop rates for inverting a triangular matrix on a Cray 2.*

		$n = 128$	$n = 256$	$n = 512$	$n = 1024$
Unblocked:	Method 1	95	162	231	283
	Method 2	114	211	289	330
	k variant	114	157	178	191
Blocked: (block size 64)	Method 1B	125	246	348	405
	Method 2C	129	269	378	428
	k variant	148	263	344	383

Lemma 14.3. *The computed inverse $\hat{X}$ from Method 2C satisfies*

$$|\hat{X}L - I| \leq c_n u |\hat{X}| |L|. \quad \square$$

In summary, block versions of Methods 1 and 2 are available that have the same residual bounds as the point methods. However, in general, there is no guarantee that stability properties remain unchanged when we convert a point method to block form, as shown by Method 2B.

In Table 14.2 we present some performance figures for inversion of a lower triangular matrix on a Cray 2. These clearly illustrate the possible gains in efficiency from using block methods, and also the advantage of Method 2 over Method 1. For comparison, the performance of a k variant is also shown (both k variants run at the same rate). The performance characteristics of the i variants are similar to those of the j variants, except that since they are row oriented rather than column oriented, they are liable to be slowed down by memory-bank conflicts, page thrashing, or cache missing.

14.3. Inverting a Full Matrix by LU Factorization

Next, we consider four methods for inverting a full matrix $A \in \mathbb{R}^{n \times n}$, given an LU factorization computed by GEPP. We assume, without loss of generality, that there are no row interchanges. We write the *computed* LU factors as L and U . Recall that $A + \Delta A = LU$, with $|\Delta A| \leq c_n u |L| |U|$ (Theorem 9.3).

14.3.1. Method A

Perhaps the most frequently described method for computing $X = A^{-1}$ is the following one.

Method A.

for $j = 1:n$

 Solve $Ax_j = e_j$.

end

Compared with the methods to be described below, Method A has the disadvantages of requiring more temporary storage and of not having a convenient

partitioned version. However, it is simple to analyse. From Theorem 9.4 we have

$$(A + \Delta A_j)\hat{x}_j = e_j, \quad |\Delta A_j| \leq c'_n u |L||U|, \quad (14.15)$$

and so

$$|A\hat{X} - I| \leq c'_n u |L||U||\hat{X}|. \quad (14.16)$$

This bound departs from the form (14.1) only in that $|A|$ is replaced by its upper bound $|L||U| + O(u)$. The forward error bound corresponding to (14.16) is

$$|\hat{X} - A^{-1}| \leq c'_n u |A^{-1}||L||U||\hat{X}|. \quad (14.17)$$

Note that (14.15) says that $\hat{x}_j$ is the j th column of the inverse of a matrix close to A , but it is a different perturbation ΔA_j for each column. It is not true that $\hat{X}$ itself is the inverse of a matrix close to A , unless A is well conditioned.

14.3.2. Method B

Next, we consider the method used by LINPACK's `xGEDI`, LAPACK's `xGETRI`, and MATLAB's `inv` function.

Method B.

Compute U^{-1} and then solve for X the equation $XL = U^{-1}$.

To analyse this method we will assume that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices that obtains the columns of U^{-1} in the order 1 to n . Then the computed inverse $X_U \approx U^{-1}$ will satisfy the residual bound

$$|X_U U - I| \leq c_n u |X_U||U|.$$

We also assume that the triangular solve from the right with L is done by back substitution. The computed $\hat{X}$ therefore satisfies $\hat{X}L = X_U + \Delta(\hat{X}, L)$ and so

$$\hat{X}(A + \Delta A) = \hat{X}LU = X_U U + \Delta(\hat{X}, L)U.$$

This leads to the residual bound

$$\begin{aligned} |\hat{X}A - I| &\leq c_n u (|U^{-1}||U| + 2|\hat{X}||L||U|) \\ &\leq c'_n u |\hat{X}||L||U|, \end{aligned} \quad (14.18)$$

which is the left residual analogue of (14.16). From (14.18) we obtain the forward error bound

$$|\hat{X} - A^{-1}| \leq c'_n u |\hat{X}||L||U||A^{-1}|.$$

Note that Methods A and B are equivalent, in the sense that Method A solves for X the equation $LUX = I$ while Method B solves $XLU = I$. Thus the two methods carry out analogous operations but in different orders. It follows that the methods must satisfy analogous residual bounds, and so (14.18) can be deduced from (14.16).

We mention in passing that the LINPACK manual states that for Method B a bound holds of the form $\|A\hat{X} - I\| \leq d_n u \|A\| \|\hat{X}\|$ [341, 1979, p. 1.20]. This is incorrect, although counterexamples are rare; it is the *left* residual that is bounded this way, as follows from (14.18).

14.3.3. Method C

The next method that we consider is from Du Croz and Higham [357, 1992]. It solves the equation $UXL = I$, computing X a partial row and column at a time. To derive the method partition

$$X = \begin{bmatrix} x_{11} & x_{12}^T \\ x_{21} & X_{22} \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 \\ l_{21} & L_{22} \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12}^T \\ 0 & U_{22} \end{bmatrix},$$

where the $(1, 1)$ blocks are scalars, and assume that the trailing submatrix X_{22} is already known. Then the rest of X is computed according to

$$\begin{aligned} x_{21} &= -X_{22}l_{21}, \\ x_{12}^T &= -u_{12}^T X_{22}/u_{11}, \\ x_{11} &= 1/u_{11} - x_{12}^T l_{21}. \end{aligned}$$

The method can also be derived by forming the product $X = U^{-1} \times L^{-1}$ using the representation of L and U as a product of elementary matrices (and diagonal matrices in the case of U). In detail the method is as follows.

Method C.

for $k = n: -1: 1$

$$\begin{aligned} X(k+1:n, k) &= -X(k+1:n, k+1:n)L(k+1:n, k) \\ X(k, k+1:n) &= -U(k, k+1:n)X(k+1:n, k+1:n)/u_{kk} \\ x_{kk} &= 1/u_{kk} - X(k, k+1:n)L(k+1:n, k) \end{aligned}$$

end

The method can be implemented so that X overwrites L and U , with the aid of a work vector of length n (or a work array to hold a block row or column in the partitioned case). Because most of the work is performed by matrix–vector (or matrix–matrix) multiplication, Method C is likely to be the fastest of those considered in this section on many machines. (Some performance figures are given at the end of the section.)

A straightforward error analysis of Method C shows that the computed $\hat{X}$ satisfies

$$|U\hat{X}L - I| \leq c_n u |U| |\hat{X}| |L|. \quad (14.19)$$

We will refer to $U\hat{X}L - I$ as a “mixed residual”. From (14.19) we can obtain bounds on the left and right residual that are weaker than those in (14.18) and (14.16) by a factor $|U^{-1}||U|$ on the left or $|L||L^{-1}|$ on the right, respectively. We also obtain from (14.19) the forward error bound

$$|\hat{X} - A^{-1}| \leq c_n u |U^{-1}||U| |\hat{X}| |L| |L^{-1}|,$$

which is (14.17) with $|A^{-1}|$ replaced by its upper bound $|U^{-1}||L^{-1}| + O(u)$ and the factors reordered.

The LINPACK routine `xsIDI` uses a special case of Method C in conjunction with block LDL^T factorization with partial pivoting to invert a symmetric indefinite matrix; see Du Croz and Higham [357, 1992] for details.

14.3.4. Method D

The next method is based on another natural way to form A^{-1} and is used by LAPACK's xPOTRI, which inverts a symmetric positive definite matrix.

Method D.

Compute L^{-1} and U^{-1} and then form $A^{-1} = U^{-1} \times L^{-1}$.

The advantage of this method is that no extra workspace is needed; U^{-1} and L^{-1} can overwrite U and L , and can then be overwritten by their product. However, Method D is significantly slower on some machines than Methods B or C, because it uses a smaller average vector length for vector operations.

To analyse Method D we will assume initially that L^{-1} is computed by Method 2 (or Method 2C) and, as for Method B above, that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices. We have

$$\hat{X} = X_U X_L + \Delta(X_U, X_L). \quad (14.20)$$

Since $A = LU - \Delta A$,

$$\hat{X}A = X_U X_L LU - X_U X_L \Delta A + \Delta(X_U, X_L)A. \quad (14.21)$$

Rewriting the first term of the right-hand side using $X_L L = I + \Delta(X_L, L)$, and similarly for U , we obtain

$$\hat{X}A - I = \Delta(X_U, U) + X_U \Delta(X_L, L)U - X_U X_L \Delta A + \Delta(X_U, X_L)A, \quad (14.22)$$

and so

$$\begin{aligned} |\hat{X}A - I| &\leq c'_n u (|U^{-1}||U| + 2|U^{-1}||L^{-1}||L||U| + |U^{-1}||L^{-1}||A|) \\ &\leq c''_n u |U^{-1}||L^{-1}||L||U|. \end{aligned} \quad (14.23)$$

This bound is weaker than (14.18), since $|\hat{X}| \leq |U^{-1}||L^{-1}| + O(u)$. Note, however, that the term $\Delta(X_U, X_L)A$ in the residual (14.22) is an unavoidable consequence of forming $X_U X_L$, and so the bound (14.23) is essentially the best possible.

The analysis above assumes that X_L and X_U both have small left residuals. If they both have small right residuals, as when they are computed using Method 1, then it is easy to see that a bound analogous to (14.23) holds for the right residual $A\hat{X} - I$. If X_L has a small left residual and X_U has a small right residual (or vice versa) then it does not seem possible to derive a bound of the form (14.23). However, we have

$$|X_L L - I| = |L^{-1}(L X_L - I)L| \leq |L^{-1}||L X_L - I||L|, \quad (14.24)$$

and since L is unit lower triangular with $|l_{ij}| \leq 1$, we have $|(L^{-1})_{ij}| \leq 2^{n-1}$, which places a bound on how much the left and right residuals of X_L can differ. Furthermore, since the matrices L from GEPP tend to be well conditioned ($\kappa_\infty(L) \ll n2^{n-1}$), and since our numerical experience is that large residuals tend to occur only for ill-conditioned matrices, we would expect the left and right residuals of X_L almost always to be of similar size. We conclude that even in the

“conflicting residuals” case, Method D will, in practice, usually satisfy (14.23) or its right residual counterpart, according to whether X_U has a small left or right residual respectively. Similar comments apply to Method B when U^{-1} is computed by a method yielding a small right residual.

These considerations are particularly pertinent when we consider Method D specialized to symmetric positive definite matrices and the Cholesky factorization $A = R^T R$. Now A^{-1} is obtained by computing $X_R = R^{-1}$ and then forming $A^{-1} = X_R X_R^T$; this is the method used in the LINPACK routine `xPODI` [341, 1979, Chap. 3]. If X_R has a small right residual then X_R^T has a small left residual, so in this application we naturally encounter conflicting residuals. Fortunately, the symmetry and definiteness of the problem help us to obtain a satisfactory residual bound. The analysis parallels the derivation of (14.23), so it suffices to show how to treat the term $X_R X_R^T R^T R$ (cf. (14.21)), where R now denotes the computed Cholesky factor. Assuming $R X_R = I + \Delta(R, X_R)$, and using (14.24) with L replaced by R , we have

$$\begin{aligned} X_R X_R^T R^T R &= X_R (I + \Delta(R, X_R)^T) R \\ &= I + F + X_R \Delta(R, X_R)^T R, \quad |F| \leq |R^{-1}| |\Delta(R, X_R)| |R|, \\ &= I + G, \end{aligned}$$

and

$$|G| \leq c_n u (|R^{-1}| |R| |R^{-1}| |R| + |R^{-1}| |R^{-T}| |R^T| |R|).$$

From the inequality $\| |B| \|_2 \leq \sqrt{n} \|B\|_2$ for $B \in \mathbb{R}^{n \times n}$ (see Lemma 6.6), together with $\|A\|_2 = \|R\|_2^2 + O(u)$, it follows that

$$\|G\|_2 \leq 2n^2 c_n u \|A\|_2 \|A^{-1}\|_2,$$

and thus overall we have a bound of the form

$$\|\hat{X}A - I\|_2 \leq d_n u \|A\|_2 \|\hat{X}\|_2.$$

Since $\hat{X}$ and A are symmetric the same bound holds for the right residual.

14.3.5. Summary

In terms of the error bounds, there is little to choose between Methods A, B, C, and D. Numerical results reported in [357, 1992] show good agreement with the bounds. Therefore the choice of method can be based on other criteria, such as performance and the use of working storage. Table 14.3 gives some performance figures for a Cray 2, covering both blocked (partitioned) and unblocked forms of Methods B, C, and D.

On a historical note, Tables 14.4 and 14.5 give timings for matrix inversion on some early computing devices; times for two more modern machines are given for comparison. The inversion methods used for the timings on the early computers in Table 14.4 are not known, but are probably methods from this section or the next.

Table 14.3. *Mflop rates for inverting a full matrix on a Cray 2.*

		$n = 64$	$n = 128$	$n = 256$	$n = 512$
Unblocked:	Method B	118	229	310	347
	Method C	125	235	314	351
	Method D	70	166	267	329
Blocked: (block size 64)	Method B	142	259	353	406
	Method C	144	264	363	415
	Method D	70	178	306	390

Table 14.4. *Times (minutes and seconds) for inverting an $n \times n$ matrix. Source for DEUCE, Pegasus, and Mark 1 timings: [200, 1981].*

n	DEUCE (English Electric) 1955	Pegasus (Ferranti) 1956	Manchester Mark 1 1951	HP 48G Calculator 1993	Sun SPARC- station ELC 1991
8	20s	26s	—	4s	.004s
16	58s	2m 37s	—	18s	.01s
24	3m 36s	7m 57s	8m	48s	.02s
32	7m 44s	17m 52s	16m	—	.04s

Table 14.5. *Additional timings for inverting an $n \times n$ matrix.*

Machine	Year	n	Time	Reference
Aiken Relay Calculator	1948	38	59½ hours	[858, 1948]
IBM 602 Calculating Punch	1949	10	8 hours	[1195, 1949]
SEAC (National Bureau of Standards)	1954	49	3 hours	[1141, 1954]
Datatron	1957	80 ^a	2½ hours	[847, 1957]
IBM 704	1957	115 ^b	19m 30s	[354, 1957]

^aBlock tridiagonal matrix, using an inversion method designed for such matrices.^bSymmetric positive definite matrix.

14.4. Gauss–Jordan Elimination

Whereas Gaussian elimination (GE) reduces a matrix to triangular form by elementary operations, Gauss–Jordan elimination (GJE) reduces it all the way to diagonal form. GJE is usually presented as a method for matrix inversion, but it can also be regarded as a method for solving linear equations. We will take the method in its latter form, since it simplifies the error analysis. Error bounds for matrix inversion are obtained by taking unit vectors for the right-hand sides.

At the k th stage of GJE, all off-diagonal elements in the k th column are eliminated, instead of just those below the diagonal, as in GE. Since the elements in the lower triangle (including the pivots on the diagonal) are identical to those that occur in GE, Theorem 9.1 tells us that GJE succeeds if all the leading principal submatrices of A are nonsingular. With no form of pivoting GJE is unstable in general, for the same reasons that GE is unstable. Partial and complete pivoting are easily incorporated.

Algorithm 14.4 (Gauss–Jordan elimination). This algorithm solves the linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular, by GJE with partial pivoting.

```

for  $k = 1:n$ 
  Find  $r$  such that  $|a_{rk}| = \max_{i \geq k} |a_{ik}|$ .
   $A(k, k:n) \leftrightarrow A(r, k:n)$ ,  $b(k) \leftrightarrow b(r)$  % Swap rows  $k$  and  $r$ .
   $row\_ind = [1:k-1, k+1:n]$  % Row indices of elements to eliminate.
   $m = A(row\_ind, k)/A(k, k)$  % Multipliers.
   $A(row\_ind, k:n) = A(row\_ind, k:n) - m * A(k, k:n)$ 
  (*)  $b(row\_ind) = b(row\_ind) - m * b(k)$ 
end
 $x_i = b_i/a_{ii}$ ,  $i = 1:n$ 

```

Cost: n^3 flops.

Note that for matrix inversion we repeat statement (*) for $b = e_1, e_2, \dots, e_n$ in turn, and if we exploit the structure of these right-hand sides the total cost is $2n^3$ flops—the same as for inversion by LU factorization.

The numerical stability properties of GJE are rather subtle and error analysis is trickier than for GE. An observation that simplifies the analysis is that we can consider the algorithm as comprising two stages. The first stage is identical to GE and forms $M_{n-1}M_{n-2} \dots M_1 A = U$, where U is upper triangular. The second stage reduces U to diagonal form by elementary operations:

$$N_n N_{n-1} \dots N_2 U = D, \quad N_k = I - n_k e_k^T, \quad e_i^T n_k = 0, \quad i \geq k.$$

The solution x is formed as $x = D^{-1}N_n \dots N_2 y$, where $y = M_{n-1} \dots M_1 b$. The rounding errors in the first stage are precisely the same as those in GE, so we will ignore them for the moment (that is, we will consider L , U , and y to be exact) and consider the second stage. We will assume, for simplicity, that there are no row interchanges. As with GE, this is equivalent to assuming that all row interchanges are done at the start of the algorithm.

Define $U_{k+1} = N_k \dots N_2 U$ (so $U_2 = U$) and note that N_k and U_k have the forms

$$U_k = \begin{bmatrix} D_{k-1} & [v_k & W_k] \\ 0 & Z_k \end{bmatrix}, \quad D_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, \text{ diagonal},$$

$$N_k = \begin{bmatrix} I_{k-1} & [n_k & 0] \\ 0 & I_{n-k+1} \end{bmatrix},$$

where $n_k = [-u_{1k}/u_{kk}, \dots, -u_{k-1,k}/u_{kk}]^T$. The computed matrices obviously satisfy

$$\hat{U}_{k+1} = \hat{N}_k \hat{U}_k + \Delta_k, \quad (14.25a)$$

$$\Delta_k = \begin{bmatrix} 0_{k-1} & \Delta^{(k)} \\ 0 & 0_{n-k+1} \end{bmatrix}, \quad |\Delta_k| \leq \gamma_3 |\hat{N}_k| |\hat{U}_k| \quad (14.25b)$$

(to be precise, $\hat{N}_k$ is defined as N_k but with $(\hat{N}_k)_{ik} = -\hat{u}_{ik}/\hat{u}_{kk}$). Similarly, with $x_{k+1} = \hat{N}_k \dots \hat{N}_2 y$, we have

$$\hat{x}_{k+1} = \hat{N}_k \hat{x} + f_k, \quad e_i^T f_k = 0, \quad i \geq k, \quad |f_k| \leq \gamma_3 |\hat{N}_k| |\hat{x}|. \quad (14.26)$$

Because of the structure of $\hat{N}_k$, Δ_k , and f_k , we have the useful property that

$$\hat{N}_i \Delta_j = \Delta_j, \quad i \geq j, \quad \hat{N}_i f_j = f_j, \quad i \geq j.$$

Without loss of generality, we now assume that the final diagonal matrix D is the identity (i.e., the pivots are all 1); this simplifies the analysis a little and has a negligible effect on the final bounds. Thus (14.25) and (14.26) yield

$$I = \hat{U}_{n+1} = \hat{N}_n \dots \hat{N}_2 U + \sum_{k=2}^n \Delta_k, \quad (14.27)$$

$$\hat{x} = \hat{x}_{n+1} = \hat{N}_n \dots \hat{N}_2 y + \sum_{k=2}^n f_k. \quad (14.28)$$

Now

$$\begin{aligned} |\Delta_k| &\leq \gamma_3 |\hat{N}_k| |\hat{U}_k| \leq \gamma_3 |\hat{N}_k| |\hat{N}_{k-1} \hat{U}_{k-1} + \Delta_{k-1}| \\ &\leq \gamma_3 (1 + \gamma_3) |\hat{N}_k| |\hat{N}_{k-1} \hat{U}_{k-1}| \\ &\leq \dots \leq \gamma_3 (1 + \gamma_3)^{k-2} |\hat{N}_k| \dots |\hat{N}_2| |U|, \end{aligned}$$

and, similarly,

$$|f_k| \leq \gamma_3 (1 + \gamma_3)^{k-2} |\hat{N}_k| \dots |\hat{N}_2| |y|.$$

But defining $\tilde{n}_k = [n_k^T \ 0]^T \in \mathbb{R}^n$, we have

$$\begin{aligned} |\hat{N}_k| \dots |\hat{N}_2| &= I + |\tilde{n}_k| e_k^T + \dots + |\tilde{n}_2| e_2^T \\ &\leq |\hat{N}_n| \dots |\hat{N}_2| = |\hat{N}_n \dots \hat{N}_2| =: |X|, \end{aligned}$$

where $X = U^{-1} + O(u)$ (by (14.27)). Hence

$$\begin{aligned}\sum_{k=2}^n |\Delta_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X||U|, \\ \sum_{k=2}^n |f_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X||y|.\end{aligned}$$

Combining (14.27) and (14.28) we have, for the solution of $Ux = y$,

$$\hat{x} = \left(I - \sum_{k=2}^n \Delta_k\right) U^{-1}y + \sum_{k=2}^n f_k = \left(I - \sum_{k=2}^n \Delta_k\right) x + \sum_{k=2}^n f_k,$$

which gives the componentwise forward error bound

$$|x - \hat{x}| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X|(|U||x| + |y|). \quad (14.29)$$

This is an excellent forward error bound: it says that the error in $\hat{x}$ is no larger than the error we would expect for an approximate solution with a tiny componentwise relative backward error. In other words, the forward error is bounded in the same way as for substitution. However, we have not, and indeed cannot, show that the method is backward stable. The best we can do is to derive from (14.27) and (14.28) the result

$$(U + \Delta U)\hat{x} = y + \Delta y, \quad (14.30a)$$

$$|\Delta U| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||U|, \quad (14.30b)$$

$$|\Delta y| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||y|, \quad (14.30c)$$

using $\hat{N}_2^{-1} \dots \hat{N}_n^{-1} = U + O(u)$. These bounds show that, with respect to the system $Ux = y$, $\hat{x}$ has a normwise backward error bounded approximately by $\kappa_\infty(U)(n-1)\gamma_3$. Hence the backward error can be guaranteed to be small only if U is well conditioned. This agrees with the comments of Peters and Wilkinson [938, 1975] that “the residuals corresponding to the Gauss–Jordan solution are often larger than those corresponding to back-substitution by a factor of order κ .”

A numerical example helps to illustrate the results. We take U to be the upper triangular matrix with 1s on the diagonal and -1 s everywhere above the diagonal ($U = U(1)$ from (8.3)). This matrix has condition number $\kappa_\infty(U) = n2^{n-1}$. Table 14.6 reports the normwise relative backward error $\eta_{U,b}(\hat{x}) = \|U\hat{x} - b\|_\infty / (\|U\|_\infty \|\hat{x}\|_\infty + \|b\|_\infty)$ (see (7.2)) for $b = Ux$, where $x = e/3$. Clearly, GJE is backward unstable for these matrices—the backward errors show a dependence on $\kappa_\infty(U)$. However, the relative distance between $\hat{x}$ and the computed solution from substitution (not shown in the table) is less than $\kappa_\infty(U)u$, which shows that the forward error is bounded by $\kappa_\infty(U)u$, confirming (14.29).

By bringing in the error analysis for the reduction to upper triangular form, we obtain an overall result for GJE.

Table 14.6. GJE for $Ux = b$.

n	$\eta_{U,b}(\hat{x})$	$\kappa_\infty(U)u$
16	2.0e-14	5.8e-11
32	6.4e-10	7.6e-6
64	1.7e-2	6.6e4

Theorem 14.5. Suppose GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular. Then

$$|b - A\hat{x}| \leq 8nu|\hat{L}||\hat{U}||\hat{U}^{-1}||\hat{U}||\hat{x}| + O(u^2), \quad (14.31)$$

$$|x - \hat{x}| \leq 2nu(|A^{-1}||\hat{L}||\hat{U}| + 3|\hat{U}^{-1}||\hat{U}|)|\hat{x}| + O(u^2), \quad (14.32)$$

where $A \approx \hat{L}\hat{U}$ is the factorization computed by GE.

Proof. For the first stage (which is just GE), we have $A + \Delta A_1 = \hat{L}\hat{U}$, $|\Delta A_1| \leq \gamma_n|\hat{L}||\hat{U}|$, and $(\hat{L} + \Delta L)\hat{y} = b$, $|\Delta L| \leq \gamma_n|\hat{L}|$, by Theorems 9.3 and 8.5.

Using (14.30), we obtain

$$(\hat{L} + \Delta L)(\hat{U} + \Delta U)\hat{x} = (\hat{L} + \Delta L)(\hat{y} + \Delta y) = b + (\hat{L} + \Delta L)\Delta y,$$

or $A\hat{x} = b - r$, where

$$r = (\Delta A_1 + \hat{L}\Delta U + \Delta L\hat{U} + \Delta L\Delta U)\hat{x} - (\hat{L} + \Delta L)\Delta y. \quad (14.33)$$

The bound (14.31) follows. To obtain (14.32) we use (14.29) and incorporate the effects of the errors in computing y . Defining x_0 by $\hat{L}\hat{U}x_0 = b$, we have

$$x - \hat{x} = (x - x_0) + (x_0 - \hat{U}^{-1}\hat{y}) + (\hat{U}^{-1}\hat{y} - \hat{x}),$$

where the last term is bounded by (14.29) and the first two terms are easily shown to be bounded by $\gamma_n|A^{-1}||\hat{L}||\hat{U}||\hat{x}| + O(u^2)$. $\square$

Theorem 14.5 shows that the stability of GJE depends not only on the size of $|\hat{L}||\hat{U}|$ (as in GE), but also on the condition of U . The vector $|\hat{U}^{-1}||\hat{U}||\hat{x}|$ is an upper bound for $|\hat{x}|$, and if this bound is sharp then the residual bound is very similar to that for LU factorization. Note that for partial pivoting we have

$$\begin{aligned} \|\hat{U}^{-1}||\hat{U}|\|_\infty &\leq n\|A^{-1}\|_\infty \cdot n\rho_n\|A\|_\infty + O(u) \\ &= n^2\rho_n\kappa_\infty(A) + O(u), \end{aligned}$$

and, similarly, $\|A^{-1}||\hat{L}||\hat{U}|\|_\infty \leq n^2\rho_n\kappa_\infty(A)$, so (14.32) implies

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq 8n^3\rho_n\kappa_\infty(A)u + O(u^2),$$

which shows that GJE with partial pivoting is normwise forward stable.

The bounds in Theorem 14.5 have the pleasing property that they are invariant under row or column scaling of A , though of course if we are using partial pivoting then row scaling can change the pivots and alter the bound.

As mentioned earlier, to obtain bounds for matrix inversion we simply take b to be each of the unit vectors in turn. For example, the residual bound becomes

$$|A\hat{X} - I| \leq 8nu|\hat{L}||\hat{U}||\hat{U}^{-1}||\hat{U}||\hat{X}| + O(u^2).$$

The question arises of whether there is a significant class of matrices for which GJE is backward stable. The next result proves that GJE without pivoting is forward stable for symmetric positive definite matrices and provides a smaller residual bound than might be expected from Theorem 14.5. We make the natural assumption that symmetry is exploited in the elimination.

Corollary 14.6. *Suppose GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite. Then, provided the computed pivots are positive,*

$$\begin{aligned} \|b - A\hat{x}\|_2 &\leq 8n^3 u \kappa_2(A)^{1/2} \|A\|_2 \|x\|_2 + O(u^2), \\ \frac{\|x - \hat{x}\|_2}{\|x\|_2} &\leq 8n^{5/2} u \kappa_2(A) + O(u^2), \end{aligned}$$

where $A \approx \hat{L}\hat{U}$ is the factorization computed by symmetric GE.

Proof. By Theorem 9.3 we have $A + \Delta A = \hat{L}\hat{U}$, where ΔA is symmetric and satisfies $|\Delta A| \leq \gamma_n |\hat{L}||\hat{U}|$. Defining $D = \text{diag}(\hat{U})^{1/2}$ we have, by symmetry, $A + \Delta A = \hat{L}D \cdot D^{-1}\hat{U} \equiv R^T R$. Hence

$$\begin{aligned} \|\hat{U}^{-1}||\hat{U}|\|_2 &= \|R^{-1}D^{-1}||DR|\|_2 \\ &= \|R^{-1}||R|\|_2 \\ &\leq n\kappa_2(R) = n\kappa_2(A + \Delta A)^{1/2} \\ &= n\kappa_2(A)^{1/2} + O(u). \end{aligned}$$

Furthermore, it is straightforward to show that $\|\hat{L}||\hat{U}|\|_2 = \|R^T||R|\|_2 \leq n(1 - n\gamma_n)^{-1} \|A\|_2$. The required bounds follow (note that the first term on the right-hand side of (14.32) dominates). $\square$

Our final result shows that if A is row diagonally dominant then GJE without pivoting is both forward stable and row-wise backward stable (see Table 7.1).

Corollary 14.7. *If GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is row diagonally dominant, then*

$$\begin{aligned} |b - A\hat{x}| &\leq 32n^2 u |A|ee^T|\hat{x}| + O(u^2), \\ \frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} &\leq 4n^3 u (\kappa_\infty(A) + 3) + O(u^2). \end{aligned}$$

Proof. The bounds follow from Theorem 14.5 on noting that U is row diagonally dominant and using Lemma 8.8 to bound $\text{cond}(U)$ and (9.17) to bound $\| |L||U| \|_\infty$. $\square$

14.5. Parallel Inversion Methods

Vavasis [1192, 1989] shows that GEPP is P-complete, a complexity result whose implication is that GEPP cannot be efficiently implemented on a highly parallel computer with a large number of processors. This result provides motivation for looking for non-GE-based methods for solving linear systems on massively parallel machines. In this section we briefly describe some matrix inversion methods whose computational complexity makes them contenders for linear system solution on parallel, and perhaps other, machines.

Csanky [285, 1976] gives a method for inverting an $n \times n$ matrix (and thereby solving a linear system) in $O(\log^2 n)$ time on $O(n^4)$ processors. This method has optimal computational complexity amongst known methods for massively parallel machines, but it involves the use of the characteristic polynomial and it has abysmal numerical stability properties (as can be seen empirically by applying the method to $3I$, for example!).

A more practical method is Newton's method for A^{-1} , which is known as the Schulz iteration [1027, 1933]:

$$X_{k+1} = X_k(2I - AX_k) = (2I - X_kA)X_k.$$

It is known that X_k converges to A^{-1} (or, more generally, to the pseudo-inverse if A is rectangular) if $X_0 = \alpha_0 A^T$ and $0 < \alpha_0 < 2/\|A\|_2^2$ [1056, 1974]. The Schulz iteration is attractive because it involves only matrix multiplication—an operation that can be implemented very efficiently on high-performance computers. The rate of convergence is quadratic, since if $E_k = I - AX_k$ or $E_k = I - X_kA$ then

$$E_{k+1} = E_k^2 = \cdots = E_0^{2^{k+1}}.$$

Like Csanky's method, the Schulz iteration has polylogarithmic complexity, but, unlike Csanky's method, it is numerically stable [1056, 1974]. Unfortunately, for scalar $a > 0$,

$$0 < x_k \ll a^{-1} \Rightarrow x_{k+1} = 2x_k - x_k a x_k \lesssim 2x_k,$$

and since $x_k \rightarrow a^{-1}$, convergence can initially be slow. Overall, about $2 \log_2 \kappa_2(A)$ iterations are required for convergence in floating point arithmetic. Pan and Schreiber [919, 1991] show how to accelerate the convergence by at least a factor 2 via scaling parameters and how to use the iteration to carry out rank and projection calculations. For an interesting application of the Schulz iteration to a sparse matrix possessing a sparse inverse, see [14, 1993], and for its adaptation to Toeplitz-like (low displacement rank) matrices see [917, 2001, Chap. 6] and [918, 1999].

For inverting triangular matrices a divide and conquer method with polylogarithmic complexity is readily derived. See Higham [605, 1995] for details and error analysis.

Another inversion method is one of Strassen, which makes use of his fast matrix multiplication method: see Problem 23.8 and §26.3.2.

14.6. The Determinant

*It may be too optimistic to hope that determinants will
fade out of the mathematical picture in a generation;
their notation alone is a thing of beauty
to those who can appreciate that sort of beauty.*

— E. T. BELL, *Review of "Contributions to the History of Determinants,
1900–1920", by Sir Thomas Muir (1931)*

Like the matrix inverse, the determinant is a quantity that rarely needs to be computed. The common fallacy that the determinant is a measure of ill conditioning is displayed by the observation that if $Q \in \mathbb{R}^{n \times n}$ is orthogonal then $\det(\alpha Q) = \alpha^n \det(Q) = \pm \alpha^n$, which can be made arbitrarily small or large despite the fact that αQ is perfectly conditioned. Of course, we could normalize the matrix before taking its determinant and define, for example,

$$\psi(A) := \frac{1}{\det(D^{-1}A)} = \frac{\det(D)}{\det(A)}, \quad D = \text{diag}(\|A(i, :)\|_2),$$

where $D^{-1}A$ has rows of unit 2-norm. This function ψ is called the Hadamard condition number by Birkhoff [112, 1975], because Hadamard's determinantal inequality (see Problem 14.11) implies that $\psi(A) \geq 1$, with equality if and only if A has mutually orthogonal rows. Unless A is already row equilibrated (see Problem 14.13), $\psi(A)$ does not relate to the conditioning of linear systems in any straightforward way.

As good a way as any to compute the determinant of a general matrix $A \in \mathbb{R}^{n \times n}$ is via GEPP. If $PA = LU$ then

$$\det(A) = \det(P)^{-1} \det(U) = (-1)^r u_{11} \dots u_{nn}, \quad (14.34)$$

where r is the number of row interchanges during the elimination. If we use (14.34) then, barring underflow and overflow, the computed determinant $\hat{d} = fl(\det(A))$ satisfies

$$\hat{d} = \det(\hat{U})(1 + \theta_n),$$

where $|\theta_n| \leq \gamma_n$, so we have a tiny relative perturbation of the exact determinant of a perturbed matrix $A + \Delta A$, where ΔA is bounded in Theorem 9.3. In other words, we have almost the right determinant for a slightly perturbed matrix (assuming the growth factor is small).

However, underflow and overflow in computing the determinant are quite likely, and the determinant itself may not be a representable number. One possibility is to compute $\log |\det(A)| = \sum_{i=1}^n \log |u_{ii}|$; as pointed out by Forsythe and Moler [431, 1967, p. 55], however, the computed sum may be inaccurate due to cancellation. Another approach, used in LINPACK, is to compute and represent $\det(A)$ in the form $y \times 10^e$, where $1 \leq |y| < 10$ and e is an integer exponent.

14.6.1. Hyman's Method

In §1.16 we analysed the evaluation of the determinant of a Hessenberg matrix H by GE. Another method for computing $\det(H)$ is Hyman's method [652, 1957], which is superficially similar to GE but algorithmically different. Hyman's method has an easy derivation in terms of LU factorization that leads to a very short error analysis. Let $H \in \mathbb{R}^{n \times n}$ be an unreduced upper Hessenberg matrix ($h_{i+1,i} \neq 0$ for all i) and write

$$H = \begin{bmatrix} h^T & \eta \\ T & y \end{bmatrix}, \quad H_1 = \begin{bmatrix} T & y \\ h^T & \eta \end{bmatrix}, \quad h, y \in \mathbb{R}^{n-1}, \quad \eta \in \mathbb{R}.$$

The matrix H_1 is H with the first row cyclically permuted to the bottom, so $\det(H_1) = (-1)^{n-1} \det(H)$. Since T is a nonsingular upper triangular matrix, we have the LU factorization

$$H_1 = \begin{bmatrix} I & 0 \\ h^T T^{-1} & 1 \end{bmatrix} \begin{bmatrix} T & y \\ 0 & \eta - h^T T^{-1} y \end{bmatrix} \equiv LU, \quad (14.35)$$

from which we obtain $\det(H_1) = \det(T)(\eta - h^T T^{-1} y)$. Therefore

$$\det(H) = (-1)^{n-1} \det(T)(\eta - h^T T^{-1} y). \quad (14.36)$$

Hyman's method consists of evaluating (14.36) in the natural way: by solving the triangular system $Tx = y$ by back substitution, then forming $\eta - h^T x$ and its product with $\det(T) = h_{21}h_{32} \dots h_{n,n-1}$.

For the error analysis we assume that no underflows or overflows occur. From the backward error analysis for substitution (Theorem 8.5) and for an inner product (see (3.4)) we have immediately, on defining $\mu := \eta - h^T T^{-1} y$,

$$\hat{\mu} = (\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y)(1 + \delta),$$

where

$$|\Delta h| \leq \gamma_{n-1}|h|, \quad |\Delta T| \leq \gamma_{n-1}|T|, \quad |\delta| \leq u.$$

The computed determinant therefore satisfies

$$\hat{d} := fl(\det(H)) = (1 + \theta_n) \det(T)(\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y),$$

where $|\theta_n| \leq \gamma_n$. This is not a backward error result, because only one of the two T s in this expression is perturbed. However, we can write $\det(T) = \det(T + \Delta T)(1 + \theta_{(n-1)^2})$, so that

$$\hat{d} = \det(T + \Delta T)(\eta(1 + \theta_{n^2-n+1}) - (h + \Delta h)^T (T + \Delta T)^{-1} y(1 + \theta_{n^2-n+1})).$$

We conclude that the computed determinant is the exact determinant of a perturbed Hessenberg matrix in which each element has undergone a relative perturbation not exceeding $\gamma_{n^2-n+1} \approx n^2 u$, which is a very satisfactory result. In fact, the constant γ_{n^2-n+1} can be reduced to γ_{2n-1} by a slightly modified analysis; see Problem 14.14.

14.7. Notes and References

Classic references on matrix inversion are Wilkinson's paper *Error Analysis of Direct Methods of Matrix Inversion* [1229, 1961] and his book *Rounding Errors in Algebraic Processes* [1232, 1963]. In discussing Method 1 of §14.2.1, Wilkinson says "*The residual of X as a left-hand inverse may be larger than the residual as a right-hand inverse by a factor as great as $\|L\| \|L^{-1}\| \dots$* We are asserting that the computed X is *almost invariably* of such a nature that $XL - I$ is equally small" [1232, 1963, p. 107]. Our experience concurs with the latter statement. Triangular matrix inversion provides a good example of the value of rounding error analysis: it helps us identify potential instabilities, even if they are rarely manifested in practice, and it shows us what kind of stability is guaranteed to hold.

In [1229, 1961] Wilkinson identifies various circumstances in which triangular matrix inverses are obtained to much higher accuracy than the bounds of this chapter suggest. The results of §8.2 provide some insight. For example, if T is a triangular M -matrix then, as noted after Corollary 8.11, its inverse is computed to high relative accuracy, no matter how ill conditioned L may be.

Sections 14.2 and 14.3 are based closely on Du Croz and Higham [357, 1992].

Method D in §14.3.4 is used by the Hewlett-Packard HP-15C calculator, for which the method's lack of need for extra storage is an important property [570, 1982].

GJE is an old method. It was discovered independently by the geodesist Wilhelm Jordan (1842–1899) (not the mathematician Camille Jordan (1838–1922)) and B.-I. Clasen [16, 1987].

An Algol routine for inverting positive definite matrices by GJE was published in the *Handbook for Automatic Computation* by Bauer and Reinsch [94, 1971]. As a means of solving a single linear system, GJE is 50% more expensive than GE when cost is measured in flops; the reason is that GJE takes $O(n^3)$ flops to solve the upper triangular system that GE solves in n^2 flops. However, GJE has attracted interest for vector computing because it maintains full vector lengths throughout the algorithm. Hoffmann [633, 1987] found that it was faster than GE on the CDC Cyber 205, a now-defunct machine with a relatively large vector startup overhead.

Turing [1166, 1948] gives a simplified analysis of the propagation of errors in GJE, obtaining a forward error bound proportional to $\kappa(A)$. Bauer [91, 1966] does a componentwise forward error analysis of GJE for matrix inversion and obtains a relative error bound proportional to $\kappa_\infty(A)$ for symmetric positive definite A . Bauer's paper is in German and his analysis is not easy to follow. A summary of Bauer's analysis (in English) is given by Meinguet [839, 1969].

The first thorough analysis of the stability of GJE was by Peters and Wilkinson [938, 1975]. Their paper is a paragon of rounding error analysis. Peters and Wilkinson observe the connection with GE, then devote their attention to the "second stage" of GJE, in which $Ux = y$ is solved. They show that each component of x is obtained by solving a lower triangular system, and they deduce that, for each i , $(U + \Delta U_i)x^{(i)} = y + \Delta y_i$, where $|\Delta U_i| \leq \gamma_n |U|$ and $|\Delta y_i| \leq \gamma_n |y|$, and where the i th component of $x^{(i)}$ is the i th component of $\hat{x}$. They then show that $\hat{x}$ has relative error bounded by $2n\kappa_\infty(U)u + O(u^2)$, but that $\hat{x}$ does not necessarily

have a small backward error. The more direct approach used in our analysis is similar to that of Dekker and Hoffmann [303, 1989], who give a normwise analysis of a variant of GJE that uses row pivoting (elimination across rows) and column interchanges.

Peters and Wilkinson [938, 1975] state that “it is well known that Gauss–Jordan is stable” for a diagonally dominant matrix. In the first edition of this book we posed proving this statement as a research problem. In answer to this question, Malyshev [811, 2000] shows that normwise backward stability is easily proved using Theorem 14.5, as shown in Corollary 14.7. Our observation that row-wise backward stability holds is new. Malyshev also shows that GJE without pivoting is backward unstable for matrices diagonally dominant by columns.

Goodnight [512, 1979] describes the use of GJE in statistics for solving least squares problems. Parallel implementation of GJE is discussed by Quintana, Quintana, Sun, and van de Geijn [966, 2001].

Demmel [316, 1993] points out the numerical instability of Csanky’s method. For discussion of the numerical properties of Csanky’s method and the Schulz iteration from a computational complexity viewpoint see Codenotti, Leoncini and Preparata [248, 2001].

For an application of the Hadamard condition number see Burdakov [184, 1997].

Error analysis of Hyman’s method is given by Wilkinson [1228, 1960], [1232, 1963, pp. 147–154], [1233, 1965, pp. 426–431]. Although it dates from the 1950s, Hyman’s method is not obsolete: it has found use in methods designed for high-performance computers; see Ward [1207, 1976], Li and Zeng [784, 1992], and Dubrulle and Golub [359, 1994].

Some fundamental problems in computational geometry can be reduced to testing the sign of the determinant of a matrix, so the question arises of how to be sure that the sign is determined correctly in floating point arithmetic. For work on this problem see Pan and Yu [920, 2001] and the references therein.

Intimately related to the matrix inverse is the adjugate matrix,

$$\text{adj}(A) = ((-1)^{i+j} \det(A_{ji})),$$

where A_{ij} denotes the submatrix of A obtained by deleting row i and column j . Thus the adjugate is the transpose of the matrix of cofactors of A , and for non-singular A , $A^{-1} = \text{adj}(A)/\det(A)$. Stewart [1081, 1998] shows that the adjugate can be well conditioned even when A is ill conditioned, and he shows how $\text{adj}(A)$ can be computed in a numerically stable way from a rank revealing decomposition of A .

14.7.1. LAPACK

LAPACK contains computational routines, but not high-level drivers, for matrix inversion. Routine `xGETRI` computes the inverse of a general square matrix by Method B using an LU factorization computed by `xGETRF`. The corresponding routine for a symmetric positive definite matrix is `xPOTRI`, which uses Method D, with a Cholesky factorization computed by `xPOTRF`. Inversion of a symmetric indefinite matrix is done by `xsYTRI`. Triangular matrix inversion is done by `xTRTRI`,

which uses Method 2C. None of the LAPACK routines computes the determinant, but it is easy for the user to evaluate it after computing an LU factorization.

Problems

14.1. Reflect on this cautionary tale told by Acton [4, 1970, p. 246].

It was 1949 in Southern California. Our computer was a very new CPC (model 1, number 1)—a 1-second-per-arithmetic-operation clunker that was holding the computational fort while an early electronic monster was being coaxed to life in an adjacent room. From a nearby aircraft company there arrived one day a 16×16 matrix of 10-digit numbers whose inverse was desired . . . We labored for two days and, after the usual number of glitches that accompany any strange procedure involving repeated handling of intermediate decks of data cards, we were possessed of an inverse matrix. During the checking operations . . . it was noted that, to eight significant figures, the inverse was the transpose of the original matrix! A hurried visit to the aircraft company to explore the source of the matrix revealed that each element had been laboriously hand computed from some rather simple combinations of sines and cosines of a common angle. It took about 10 minutes to prove that the matrix was, indeed, orthogonal!

14.2. Rework the analysis of the methods of §14.2.2 using the assumptions (13.4) and (13.5), thus catering for possible use of fast matrix multiplication techniques.

14.3. Show that for any nonsingular matrix A ,

$$\kappa(A) \geq \max \left\{ \frac{\|AX - I\|}{\|XA - I\|}, \frac{\|XA - I\|}{\|AX - I\|} \right\}.$$

This inequality shows that the left and right residuals of X as an approximation to A^{-1} can differ greatly only if A is ill conditioned.

14.4. (Mendelsohn [841, 1956]) Find parametrized 2×2 matrices A and X such that the ratio $\|AX - I\|/\|XA - I\|$ can be made arbitrarily large.

14.5. Let X and Y be approximate inverses of $A \in \mathbb{R}^{n \times n}$ that satisfy

$$|A\hat{X} - I| \leq u|A||\hat{X}| \quad \text{and} \quad |\hat{Y}A - I| \leq u|\hat{Y}||A|,$$

and let $\hat{x} = fl(\hat{X}b)$ and $\hat{y} = fl(\hat{Y}b)$. Show that

$$|A\hat{x} - b| \leq \gamma_{n+1}|A||\hat{X}||b| \quad \text{and} \quad |A\hat{y} - b| \leq \gamma_{n+1}|A||\hat{Y}||A||x|.$$

Derive forward error bounds for $\hat{x}$ and $\hat{y}$. Interpret all these bounds.

14.6. What is the inverse of the matrix on the front cover of the *LAPACK Users' Guide* [20, 1999]? (The first two editions gave the inverse on the back cover.) Answer the same question for the *LINPACK Users' Guide* [341, 1979].

14.7. Show that for any matrix having a row or column of 1s, the elements of the inverse sum to 1.

14.8. Let $X = A + iB \in \mathbb{C}^{n \times n}$ be nonsingular, where A and B are real. Show that X^{-1} can be expressed in terms of the inverse of the real matrix of order $2n$,

$$Y = \begin{bmatrix} A & -B \\ B & A \end{bmatrix}.$$

Show that if X is Hermitian positive definite then Y is symmetric positive definite. Compare the economics of real versus complex inversion.

14.9. For a given nonsingular $A \in \mathbb{R}^{n \times n}$ and $X \approx A^{-1}$, it is interesting to ask what is the smallest ϵ such that $X + \Delta X = (A + \Delta A)^{-1}$ with $\|\Delta X\| \leq \epsilon \|X\|$ and $\|\Delta A\| \leq \epsilon \|A\|$. We require $(A + \Delta A)(X + \Delta X) = I$, or

$$A\Delta X + \Delta AX + \Delta A\Delta X = I - AX.$$

It is reasonable to drop the second-order term, leaving a generalized Sylvester equation that can be solved using singular value decompositions of A and X (cf. §16.2). Investigate this approach computationally for a variety of A and methods of inversion.

14.10. For a nonsingular $A \in \mathbb{R}^{n \times n}$ and given integers i and j , under what conditions is $\det(A)$ independent of a_{ij} ? What does this imply about the suitability of $\det(A)$ for measuring conditioning?

14.11. Prove Hadamard's inequality for $A \in \mathbb{C}^{n \times n}$:

$$|\det(A)| \leq \prod_{k=1}^n \|a_k\|_2,$$

where $a_k = A(:, k)$. When is there equality? (Hint: use the QR factorization.)

14.12. (a) Show that if $A^T = QR$ is a QR factorization then the Hadamard condition number $\psi(A) = \prod_{i=1}^n \rho_i / |r_{ii}|$, where $\rho_i = \|R(:, i)\|_2$. (b) Evaluate $\psi(A)$ for $A = U(1)$ (see (8.3)) and for the Pei matrix $A = (\alpha - 1)I + ee^T$.

14.13. (Guggenheimer, Edelman, and Johnson [530, 1995]) (a) Prove that for a nonsingular $A \in \mathbb{R}^{n \times n}$,

$$\kappa_2(A) < \frac{2}{|\det(A)|} \left(\frac{\|A\|_F}{n^{1/2}} \right)^n. \quad (14.37)$$

(Hint: apply the arithmetic-geometric mean inequality to the n numbers $\sigma_1^2/2, \sigma_1^2/2, \sigma_2^2, \dots, \sigma_{n-1}^2$, where the σ_i are the singular values of A .) (b) Deduce that if A has rows of unit 2-norm then

$$\kappa_2(A) < \frac{2}{|\det(A)|} = 2\psi(A),$$

where ψ is the Hadamard condition number. For more on this type of bound see [842, 1997], and for a derivation of (14.37) from a different perspective see [336, 1993].

14.14. Show that Hyman's method for computing $\det(H)$, where $H \in \mathbb{R}^{n \times n}$ is an unreduced upper Hessenberg matrix, computes the exact determinant of $H + \Delta H$ where $|\Delta H| \leq \gamma_{2n-1}|H|$, barring underflow and overflow. What is the effect on the error bound of a diagonal similarity transformation $H \rightarrow D^{-1}HD$, where $D = \text{diag}(d_i)$, $d_i \neq 0$?

14.15. (Godunov, Antonov, Kiriljuk, and Kostin [493, 1993]) Show that if $A \in \mathbb{R}^{n \times n}$ and $\kappa_2(A)\|\Delta A\|_2/\|A\|_2 < 1$ then

$$\frac{|\det(A + \Delta A) - \det(A)|}{\det(A)} \leq \frac{n\kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}}{1 - n\kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}}.$$

Chapter 15

Condition Number Estimation

*Most of LAPACK's condition numbers and error bounds are based on
estimated condition numbers ...*

*The price one pays for using an estimated
rather than an exact condition number is
occasional (but very rare) underestimates of the true error;
years of experience attest to the reliability of our estimators,
although examples where they badly underestimate the error can be constructed.*

— E. ANDERSON et al., *LAPACK Users' Guide* (1999)

*The importance of the counter-examples is that they make clear that
any effort toward proving that the algorithms
always produce useful estimations is fruitless.
It may be possible to prove that the algorithms
produce useful estimations in certain situations, however,
and this should be pursued.*

An effort simply to construct more complex algorithms is dangerous.

— A. K. CLINE and R. K. REW, *A Set of Counter-Examples
to Three Condition Number Estimators* (1983)

Singularity is almost invariably a clue.

— SIR ARTHUR CONAN DOYLE, *The Boscombe Valley Mystery* (1892)

Condition number estimation is the process of computing an inexpensive estimate of a condition number, where “inexpensive” usually means that the cost is an order of magnitude less than would be required to compute the condition number exactly. Condition estimates are required when evaluating forward error bounds for computed solutions to many types of linear algebra problems. An estimate of the condition number that is correct to within a factor 10 is usually acceptable, because it is the magnitude of an error bound that is of interest, not its precise value. In this chapter we are concerned with condition numbers involving A^{-1} , where A is a given nonsingular $n \times n$ matrix; these arise in linear systems $Ax = b$ but also more generally in, for example, least squares problems and eigenvalue problems.

An important question for any condition estimator is whether there exists a “counterexample”: a parametrized matrix for which the quotient “estimate divided by true condition number” can be made arbitrarily small (or large, depending on whether the estimate is a lower bound or an upper bound) by varying a parameter. Intuitively, it might be expected that any condition estimator has counterexamples or, equivalently, that estimating a condition number to within a constant factor independent of the matrix A must cost as much, asymptotically, as computing A^{-1} . This property was conjectured by Demmel [314, 1992] and strong evidence for it is provided by Demmel, Diment, and Malajovich [321, 2001]. In the latter paper it is shown that the cost of computing an estimate of $\|A^{-1}\|$ of guaranteed quality is at least the cost of testing whether the product of two $n \times n$ matrices is zero, and performing this test is conjectured to cost as much as actually computing the product [185, 1997, Problem 16.3].

15.1. How to Estimate Componentwise Condition Numbers

In the perturbation theory of Chapter 7 for linear equations we obtained perturbation bounds that involve the condition numbers

$$\kappa_{E,f}(A, x) = \frac{\|A^{-1}\| \|f\|}{\|x\|} + \|A^{-1}\| \|E\|,$$

$$\text{cond}_{E,f}(A, x) = \frac{\| |A^{-1}|(|E|x| + f) \|_{\infty}}{\|x\|_{\infty}},$$

and their variants. To compute these condition numbers exactly we need to compute A^{-1} , which requires $O(n^3)$ operations, even if we are given a factorization of A . Is it possible to produce reliable estimates of both condition numbers in $O(n^2)$ operations? The answer is yes, but to see why we first need to rewrite the expression for $\text{cond}_{E,f}(A, x)$. Consider the general expression $\| |A^{-1}|d \|_{\infty}$, where d is a given nonnegative vector (thus, $d = f + E|x|$ for $\text{cond}_{E,f}(A, x)$); note that the practical error bound (7.31) is of this form. Writing $D = \text{diag}(d)$ and $e = [1, 1, \dots, 1]^T$, we have

$$\| |A^{-1}|d \|_{\infty} = \| |A^{-1}|De \|_{\infty} = \| |A^{-1}|D \|_{\infty} = \| |A^{-1}|D \|_{\infty}. \quad (15.1)$$

Hence the problem is equivalent to that of estimating $\|B\|_\infty := \|A^{-1}D\|_\infty$. If we can estimate this quantity then we can estimate any of the condition numbers or perturbation bounds for a linear system. There is an algorithm, described in §15.3, that produces reliable order-of-magnitude estimates of $\|B\|_1$, for an arbitrary B , by computing just a few matrix–vector products Bx and B^Ty for carefully selected vectors x and y and then approximating $\|B\|_1 \approx \|Bx\|_1/\|x\|_1$. If we assume that we have a factorization of A (say, $PA = LU$ or $A = QR$), then we can form the required matrix–vector products for $B = A^{-1}D$ in $O(n^2)$ flops. Since $\|B\|_1 = \|B^T\|_\infty$, it follows that we can use the algorithm to estimate $\|A^{-1}d\|_\infty$ in $O(n^2)$ flops.

Before presenting the 1-norm condition estimation algorithm, we describe a more general method that estimates $\|B\|_p$ for any $p \in [1, \infty]$. This more general method is of interest in its own right and provides insight into the special case $p = 1$.

15.2. The p -Norm Power Method

An iterative “power method” for computing $\|A\|_p$ was proposed by Boyd in 1974 [155, 1974]. When $p = 2$ it reduces to the usual power method applied to A^TA . We use the notation $\text{dual}_p(x)$ to denote any vector y of unit q -norm such that equality holds in the Hölder inequality $x^Ty \leq \|x\|_p\|y\|_q$ (this normalization is different from the one we used in (6.3), but is more convenient for our present purposes). Throughout this section, $p \geq 1$ and q is defined by $p^{-1} + q^{-1} = 1$.

Algorithm 15.1 (p -norm power method). Given $A \in \mathbb{R}^{m \times n}$ and $x_0 \in \mathbb{R}^n$, this algorithm computes γ and x such that $\gamma \leq \|A\|_p$ and $\|Ax\|_p = \gamma\|x\|_p$.

```

 $x = x_0/\|x_0\|_p$ 
repeat
     $y = Ax$ 
     $z = A^T \text{dual}_p(y)$ 
    if  $\|z\|_q \leq z^Tx$ 
         $\gamma = \|y\|_p$ 
        quit
    end
     $x = \text{dual}_q(z)$ 
end

```

Cost: $4rmn$ flops (for r iterations).

There are several ways to derive Algorithm 15.1. Perhaps the most natural way is to examine the optimality conditions for maximization of

$$F(x) = \frac{\|Ax\|_p}{\|x\|_p}, \quad x \in \mathbb{R}^n.$$

First, we note that the subdifferential (that is, the set of subgradients) of an arbitrary vector norm $\|\cdot\|$ is given by [417, 1987, p. 379]

$$\partial\|x\| = \{ \lambda : \lambda^T x = \|x\|, \|\lambda\|_D \leq 1 \}.$$

If $x \neq 0$ then $\lambda^T x = \|x\| \Rightarrow \|\lambda\|_D \geq 1$, from the Hölder inequality, and so, if $x \neq 0$,

$$\begin{aligned}\partial\|x\| &= \{\lambda : \lambda^T x = \|x\|, \|\lambda\|_D = 1\} \\ &=: \{\text{dual}(x)\}.\end{aligned}$$

It can also be shown that if A has full rank,

$$\partial\|Ax\| = \{A^T \text{dual}(Ax)\}. \quad (15.2)$$

We assume now that A has full rank, $1 < p < \infty$, and $x \neq 0$. Then it is easy to see that there is a unique vector $\text{dual}_p(x)$, so $\partial\|x\|_p$ has just one element, that is, $\|x\|_p$ is differentiable. Hence we have

$$\nabla F(x) = \frac{A^T \text{dual}_p(Ax)}{\|x\|_p} - \frac{\|Ax\|_p \text{dual}_p(x)}{\|x\|_p^2}. \quad (15.3)$$

The first-order Kuhn–Tucker condition for a local maximum of F is therefore

$$A^T \text{dual}_p(Ax) = \frac{\|Ax\|_p}{\|x\|_p} \text{dual}_p(x).$$

Since $\text{dual}_q(\text{dual}_p(x)) = x/\|x\|_p$ if $p \neq 1, \infty$, this equation can be written

$$x = \frac{\|x\|_p^2}{\|Ax\|_p} \text{dual}_q(A^T \text{dual}_p(Ax)). \quad (15.4)$$

The power method is simply functional iteration applied to this transformed set of Kuhn–Tucker equations (the scale factor $\|x\|_p^2/\|Ax\|_p$ is irrelevant since $F(\alpha x) = F(x)$).

For the 1- and ∞ -norms, which are not everywhere differentiable, a different derivation can be given. The problem can be phrased as one of maximizing the convex function $F(x) := \|Ax\|_p$ over the convex set $S := \{x : \|x\|_p \leq 1\}$. The convexity of F and S ensures that, for any $u \in S$, at least one vector g exists such that

$$F(v) \geq F(u) + g^T(v - u) \quad \text{for all } v \in S. \quad (15.5)$$

Vectors g satisfying (15.5) are called *subgradients* of F (see, for example, [417, 1987, p. 364]). Inequality (15.5) suggests the strategy of choosing a subgradient g and moving from u to a point $v_* \in S$ that maximizes $g^T(v - u)$, that is, a vector that maximizes $g^T v$. Clearly, $v_* = \text{dual}_q(g)$. Since, from (15.2), g has the form $A^T \text{dual}_p(Ax)$, this completes the derivation of the iteration.

For all values of p the power method has the desirable property of generating an increasing sequence of norm approximations.

Lemma 15.2. *In Algorithm 15.1, the vectors from the k th iteration satisfy*

- (a) $z^{k^T} x^k = \|y^k\|_p$, and
- (b) $\|y^k\|_p \leq \|z^k\|_q \leq \|y^{k+1}\|_p \leq \|A\|_p$.

The first inequality in (b) is strict if convergence is not obtained on the k th iteration.

Proof. $z^{k^T}x^k = \text{dual}_p(y^k)^T A x^k = \text{dual}_p(y^k)^T y^k = \|y^k\|_p$. Then

$$\begin{aligned} \|y^k\|_p &= z^{k^T}x^k \\ &\leq \|z^k\|_q \|x^k\|_p = \|z^k\|_q = z^{k^T}x^{k+1} = \text{dual}_p(y^k)^T A x^{k+1} \\ &\leq \|\text{dual}_p(y^k)\|_q \|A x^{k+1}\|_p = \|y^{k+1}\|_p \\ &\leq \|A\|_p. \end{aligned}$$

For the last part, note that, in view of (a), the convergence test “ $\|z^k\|_q \leq z^{k^T}x^k$ ” can be written as “ $\|z^k\|_q \leq \|y^k\|_p$ ”. $\square$

It is clear from Lemma 15.2 (or directly from the Hölder inequality) that the convergence test “ $\|z\|_q \leq z^T x$ ” in Algorithm 15.1 is equivalent to “ $\|z\|_q = z^T x$ ” and, since $\|x\|_p = 1$, this is equivalent to $x = \text{dual}_q(z)$. Thus, although the convergence test compares two scalars, it is actually testing for equality in the vector equation (15.4).

The convergence properties of Algorithm 15.1 merit a careful description. First, in view of Lemma 15.2, the scalars $\gamma_k = \|y^k\|_p$ form an increasing and convergent sequence. This does not necessarily imply that Algorithm 15.1 converges, since the algorithm tests for convergence of the x^k , and these vectors could fail to converge. However, a subsequence of the x^k must converge to a limit, $\bar{x}$ say. Boyd [155, 1974] shows that if $\bar{x}$ is a strong local maximum of F with no zero components, then $x^k \rightarrow \bar{x}$ linearly.

If Algorithm 15.1 converges, it converges to a stationary point of $F(x)$ when $1 < p < \infty$. Thus, instead of the desired global maximum $\|A\|_p$, we may obtain only a local maximum or even a saddle point. When $p = 1$ or ∞ , if the algorithm converges to a point at which F is not differentiable, that point need not even be a stationary point. On the other hand, for $p = 1$ or ∞ Algorithm 15.1 terminates in at most $n + 1$ iterations (assuming that when dual_p or dual_q is not unique an extreme point of the unit ball is taken), since the algorithm moves between the vertices e_i of the unit ball in the 1-norm, increasing F on each stage ($x = \pm e_i$ for $p = 1$, and $\text{dual}_p(y) = \pm e_i$ for $p = \infty$). An example where n iterations are required for $p = 1$ is given in Problem 15.3.

For two special types of matrix, more can be said about Algorithm 15.1.

(1) If $A = xy^T$ (rank 1), the algorithm converges on the second step with $\gamma = \|A\|_p = \|x\|_p \|y\|_q$, whatever x_0 .

(2) Boyd [155, 1974] shows that if A has nonnegative elements, $A^T A$ is irreducible, $1 < p < \infty$, and x_0 has positive elements, then the x^k converge and $\gamma_k \rightarrow \|A\|_p$.

For values of p strictly between 1 and ∞ the convergence behaviour of Algorithm 15.1 is typical of a linearly convergent method: exact convergence is not usually obtained in a finite number of steps and arbitrarily many steps can be required for convergence, as is well known for the 2-norm power method. Fortunately, there is a method for choosing a good starting vector that can be combined with Algorithm 15.1 to produce a reliable norm estimator; see the Notes and References and Problem 15.2.

We now turn our attention to the extreme values of p : 1 and ∞ .

15.3. LAPACK 1-Norm Estimator

Algorithm 15.1 has two remarkable properties when $p = 1$: it almost always converges within four iterations (when $x_0 = [1, 1, \dots, 1]^T$, say) and it frequently yields $\|A\|_1$ exactly. This rapid, finite termination is also obtained for $p = \infty$ and is related to the fact that Algorithm 15.1 moves among the finite set of extreme points of the unit ball. Numerical experiments suggest that the accuracy is about the same for both norms but that slightly more iterations are required on average for $p = \infty$. Hence we will confine our attention to the 1-norm.

The 1-norm version of Algorithm 15.1 was derived independently of the general algorithm by Hager [536, 1984] and can be expressed as follows. The notation $\xi = \text{sign}(y)$ means that $\xi_i = 1$ or -1 according as $y_i \geq 0$ or $y_i < 0$. We now specialize to square matrices.

Algorithm 15.3 (1-norm power method). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and x such that $\gamma \leq \|A\|_1$ and $\|Ax\|_1 = \gamma\|x\|_1$.

```

 $x = n^{-1}e$ 
repeat
   $y = Ax$ 
   $\xi = \text{sign}(y)$ 
   $z = A^T \xi$ 
  if  $\|z\|_\infty \leq z^T x$ 
     $\gamma = \|y\|_1$ 
    quit
  end
   $x = e_j$ , where  $|z_j| = \|z\|_\infty$  (smallest such  $j$ )
end
```

Numerical experiments show that the estimates produced by Algorithm 15.3 are frequently exact ($\gamma = \|A\|_1$), usually “acceptable” ($\gamma \geq \|A\|_1/10$), and sometimes poor ($\gamma < \|A\|_1/10$).

A general class of counterexamples for Algorithm 15.3 is given by the matrices

$$A = I + \theta C,$$

where $Ce = C^T e = 0$ (there are many possible choices for C). For any such matrix, Algorithm 15.3 computes $y = n^{-1}e$, $\xi = e$, $z = e$, and hence the algorithm terminates at the end of the first iteration with

$$\frac{\gamma}{\|A\|_1} = \frac{1}{\|I + \theta C\|_1} \sim \theta^{-1} \quad \text{as } \theta \rightarrow \infty.$$

The problem is that the algorithm stops at a local maximum that can differ from the global one by an arbitrarily large factor.

A more reliable and more robust algorithm is produced by the following modifications of Higham [585, 1988].

Definition of estimate. To overcome most of the poor estimates, γ is redefined as

$$\gamma = \max \left\{ \|y\|_1, \frac{\|c\|_1}{\|b\|_1} \right\},$$

where

$$c = Ab, \quad b_i = (-1)^{i+1} \left(1 + \frac{i-1}{n-1} \right).$$

The vector b is considered likely to “pick out” any large elements of A in those cases where such elements fail to propagate through to y .

Convergence test. The algorithm is limited to a minimum of two and a maximum of five iterations. Further, convergence is declared after computing ξ if the new ξ is parallel to the previous one¹²; this event signals that convergence will be obtained on the current iteration and that the next (and final) multiplication $A^T \xi$ is unnecessary. Convergence is also declared if the new $\|y\|_1$ is no larger than the previous one. This nonincrease of the norm can happen only in finite precision arithmetic and signals the possibility of a vertex e_j being revisited—the onset of “cycling”.

The improved algorithm is as follows. This algorithm is the basis of all the condition number estimation in LAPACK.

Algorithm 15.4 (LAPACK norm estimator). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and $v = Aw$ such that $\gamma \leq \|A\|_1$ with $\|v\|_1/\|w\|_1 = \gamma$ (w is not returned).

```

 $v = A(n^{-1}e)$ 
if  $n = 1$ , quit with  $\gamma = |v_1|$ , end
 $\gamma = \|v\|_1$ 
 $\xi = \text{sign}(v)$ 
 $x = A^T \xi$ 
 $k = 2$ 
repeat
     $j = \min\{i: |x_i| = \|x\|_\infty\}$ 
     $v = Ae_j$ 
     $\bar{\gamma} = \gamma$ 
     $\gamma = \|v\|_1$ 
    if  $\text{sign}(v) = \pm \xi$  or  $\gamma \leq \bar{\gamma}$ , goto (*), end
     $\xi = \text{sign}(v)$ 
     $x = A^T \xi$ 
     $k = k + 1$ 
until  $(\|x\|_\infty = x_j \text{ or } k > 5)$ 
(*)  $x_i = (-1)^{i+1} \left( 1 + \frac{i-1}{n-1} \right), \quad i = 1:n$ 
 $x = Ax$ 
if  $2\|x\|_1/(3n) > \gamma$  then
     $v = x$ 
     $\gamma = 2\|x\|_1/(3n)$ 
end

```

Algorithm 15.4 can still be “defeated”: it returns an estimate 1 for matrices $A(\theta)$ of the form

$$A(\theta) = I + \theta P, \quad \text{where } P = P^T, \quad Pe = 0, \quad Pe_1 = 0, \quad Pb = 0. \quad (15.6)$$

¹²In [585, 1988] and in the algorithm used in LAPACK, only *equality* of the new and old ξ is tested, due to an oversight.

(P can be constructed as $I - Q$, where Q is the orthogonal projection onto $\text{span}\{e, e_1, b\}$.) Indeed, the existence of counterexamples is intuitively obvious since Algorithm 15.4 samples the behaviour of A on fewer than n vectors in $\mathbb{R}^n$. Numerical counterexamples (not parametrized) can be constructed automatically by direct search, as described in §26.3.1. Despite these weaknesses, practical experience with Algorithm 15.4 shows that it is very rare for the estimate to be more than three times smaller than the actual norm, independent of the dimension n . Therefore Algorithm 15.4 is, in practice, a *very reliable* norm estimator. The number of matrix–vector products required is at least 4 and at most 11, and averages between 4 and 5.

There is an analogue of Algorithm 15.3 for complex matrices, in which ξ_i is defined as $y_i/|y_i|$ if $y_i \neq 0$ and 1 otherwise. In the corresponding version of Algorithm 15.4 the test for parallel consecutive ξ vectors is removed, because ξ now has noninteger, complex components and so is unlikely to “repeat”.

MATLAB has a built-in function `rcond` that implements Algorithm 15.4.

It is interesting to look at a performance profile of Algorithm 15.4. A performance profile is a plot of some measure of the performance of an algorithm versus a problem parameter (the idea originates in quadrature—see [800, 1976]). In this case, the natural measure of performance is the underestimation ratio, $\gamma/\|A\|_1$. Figure 15.1 shows the performance profile for a 5×5 matrix $A(\theta)$ of the form (15.6), with P constructed as described above (because of rounding errors in constructing $A(\theta)$ and within the algorithm, the computed norm estimates differ from those that would be produced in exact arithmetic). The jagged nature of the performance curve is typical for algorithms that contain logical tests and branches. Small changes in the parameter θ , which themselves result in different rounding errors, can cause the algorithm to visit different vertices in this example.

15.4. Block 1-Norm Estimator

Algorithm 15.4 has two limitations. First, it offers the user no way to control or improve the accuracy of the estimate. Therefore it is not well suited to applications in which an estimate with one or more correct digits is required. Second, it is based on level-2 BLAS operations, which implies less than optimal efficiency on modern computers.

Higham and Tisseur [624, 2000] derive a block version of the 1-norm power method that iterates with an $n \times t$ matrix, where $t \geq 1$ is a parameter; for $t = 1$, Algorithm 15.3 is recovered. They develop this method into a practical algorithm generalizing Algorithm 15.4. The algorithm has several key features.

- The accuracy and reliability of the estimates it produces generally increase with t , leading quickly to estimates with one or more correct digits.
- The number of iterations varies little with t , so the fast convergence for $t = 1$ is maintained.
- The computational kernels are level-3 BLAS operations for $t > 1$.
- The first column of the starting matrix is $n^{-1}e$ (as in Algorithm 15.4) and the last $t - 1$ columns are chosen randomly, giving the algorithm a statistical

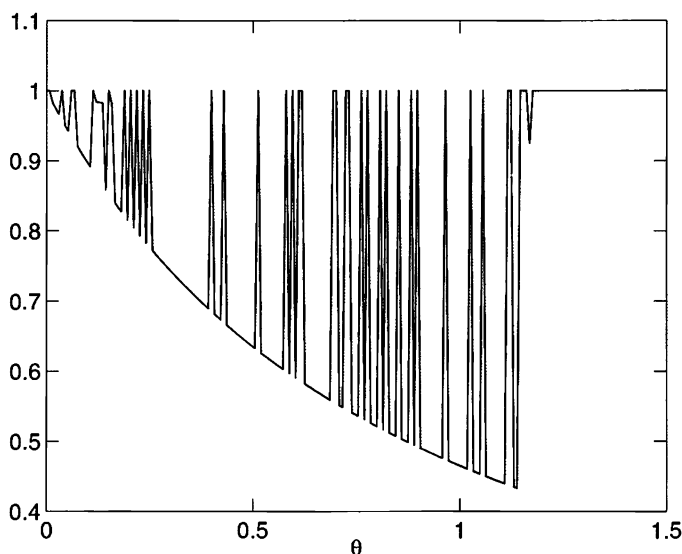


Figure 15.1. Underestimation ratio for Algorithm 15.4 for 5×5 matrix $A(\theta)$ of (15.6) with 200 equally spaced values of $\theta \in [0, 1.5]$.

flavour. Hence if the algorithm is applied repeatedly to a given matrix it may not give the same estimate each time. The randomness lessens the importance of counterexamples, since any counterexample will be valid only for particular starting matrices.

- Working with $t > 1$ columns in the iteration matrix can lead to a vector from the current iteration being parallel to one from the current iteration or an earlier iteration, which results in redundant computation. This redundancy is largely avoided by detecting parallel vectors within the current iteration and the previous one and replacing them by random vectors.
- As for Algorithm 15.4, the algorithm is applicable to both real and complex matrices.

Details of the block algorithm can be found in [624, 2000], and serial and parallel Fortran implementations are described by Cheng and Higham [229, 2001], [230, 2001]. The algorithm is available in MATLAB as function `normest1`, which is invoked by the condition estimation function `condest` (the default is $t = 2$).

15.5. Other Condition Estimators

The first condition estimator to be widely used is the one employed in LINPACK. It was developed by Cline, Moler, Stewart, and Wilkinson [243, 1979]. The idea behind this condition estimator originates with Gragg and Stewart [516, 1976], who were interested in computing an approximate null vector rather than estimating the condition number itself.

We will describe the algorithm as it applies to a triangular matrix $T \in \mathbb{R}^{n \times n}$. There are three steps:

1. Choose a vector d such that $\|y\|$ is as large as possible relative to $\|d\|$, where $T^T y = d$.
2. Solve $Tx = y$.
3. Estimate $\|T^{-1}\| \approx \|x\|/\|y\|$ ($\leq \|T^{-1}\|$).

In LINPACK the norm is the 1-norm, but the algorithm can also be used for the 2-norm or the ∞ -norm. The motivation for step 2 is based on a singular value decomposition analysis. Roughly, if $\|y\|/\|d\|$ ($\approx \|T^{-T}\|$) is large then $\|x\|/\|y\|$ ($\approx \|T^{-1}\|$) will almost certainly be at least as large, and it could be a much better estimate. Notice that $T^T T x = d$, so the algorithm is related to the power method on the matrix $(T^T T)^{-1}$ with the specially chosen starting vector d .

To examine step 1 more closely, suppose that $T = U^T$ is lower triangular and note that the equation $Uy = d$ can be solved by the following column-oriented (saxpy) form of substitution:

```

p(1:n) = 0
for j = n:-1:1
    y_j = (d_j - p_j)/u_jj
(*)   p(1:j-1) = p(1:j-1) + U(1:j-1, j)y(j)
end

```

The idea is to choose the elements of the right-hand side vector d adaptively as the solution proceeds, with $d_j = \pm 1$. At the j th stage of the algorithm $d_n, \dots, d_{j+1}$ have been chosen and $y_n, \dots, y_{j+1}$ are known. The next element $d_j \in \{+1, -1\}$ is chosen so as to maximize a weighted sum of $d_j - p_j$ and the partial sums $p_1, \dots, p_j$, which would be computed during the next execution of statement (*) above. Hence the algorithm looks ahead, trying to gauge the effect of the choice of d_j on future solution components. This heuristic algorithm for choosing d is expressed in detail as follows.

Algorithm 15.5 (LINPACK condition estimator). Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$ and a set of nonnegative weights $\{w_i\}$, this algorithm computes a vector y such that $Uy = d$, where the elements $d_j = \pm 1$ are chosen to make $\|y\|$ large.

```

p(1:n) = 0
for j = n:-1:1
    y_j^+ = (1 - p_j)/u_jj
    y_j^- = (-1 - p_j)/u_jj
    p^+(1:j-1) = p(1:j-1) + U(1:j-1, j)y_j^+(j)
    p^-(1:j-1) = p(1:j-1) + U(1:j-1, j)y_j^-(j)
    if w_j|1 - p_j| + \sum_{i=1}^{j-1} w_i|p_i^+| \geq w_j|1 + p_j| + \sum_{i=1}^{j-1} w_i|p_i^-|
        y_j = y_j^+
    p(1:j-1) = p^+(1:j-1)

```

```

else
     $y_j = y_j^-$ 
     $p(1:j-1) = p^-(1:j-1)$ 
end
end

```

Cost: $4n^2$ flops.

LINPACK takes the weights $w_j \equiv 1$, though another possible (but more expensive) choice would be $w_j = 1/|u_{jj}|$, which corresponds to how p_j is weighted in the expression $y_j = (d_j - p_j)/u_{jj}$.

To estimate $\|A^{-1}\|_1$ for a full matrix A , the LINPACK estimator makes use of an LU factorization of A . Given $PA = LU$, the equations solved are $U^T z = d$, $L^T y = z$, and $Ax = P^T y$, where for the first system d is constructed by the analogue of Algorithm 15.5 for lower triangular matrices; the estimate is $\|x\|_1/\|y\|_1 \approx \|A^{-1}\|_1$. Since d is chosen without reference to L , there is an underlying assumption that any ill condition in A is reflected in U . This assumption may not be true; see Problem 15.4.

In contrast to the LAPACK norm estimator, the LINPACK estimator requires explicit access to the elements of the matrix. Hence the estimator cannot be used to estimate componentwise condition numbers. Furthermore, separate code has to be written for each different type of matrix and factorization. Consequently, while LAPACK has just a single norm estimation routine, which is called by many other routines, LINPACK has multiple versions of its algorithm, each tailored to the specific matrix or factorization.

Several years after the LINPACK condition estimator was developed, several parametrized counterexamples were found by Cline and Rew [244, 1983]. Numerical counterexamples can also be constructed by direct search, as shown in §26.3.1. Despite the existence of these counterexamples the LINPACK estimator has been widely used and is regarded as being almost certain to produce an estimate correct to within a factor 10 in practice.

A 2-norm condition estimator was developed by Cline, Conn, and Van Loan [245, 1982, Algorithm 1]; see also Van Loan [1181, 1987] for another explanation. The algorithm builds on the ideas underlying the LINPACK estimator by using “look-behind” as well as look-ahead. It estimates $\sigma_{\min}(R) = \|R^{-1}\|_2^{-1}$ or $\sigma_{\max}(R) = \|R\|_2$ for a triangular matrix R , where $\sigma_{\min}$ and $\sigma_{\max}$ denote the smallest and largest singular values, respectively. Full matrices can be treated if a factorization $A = QR$ is available (Q orthogonal, R upper triangular), since R and A have the same singular values. The estimator performs extremely well in numerical tests, often producing an estimate that has some correct digits [245, 1982], [582, 1987]. No counterexamples to the estimator were known until Bischof [115, 1990] obtained counterexamples as a by-product of the analysis of a different but related method, mentioned at the end of this section.

We now describe some methods with intrinsic randomness, for which the output of the method depends on the particular random numbers chosen. A natural idea along these lines is to apply the power method to the matrix $(AA^T)^{-1}$ with a randomly chosen starting vector. If a factorization of A is available, the power method vectors can be computed inexpensively by solving linear systems with A and A^T . Analysis based on the singular value decomposition suggests that there

is a high probability that a good estimate of $\|A^{-1}\|_2$ will be obtained. This notion is made precise by Dixon [340, 1983], who proves the following result.

Theorem 15.6 (Dixon). *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and let $\theta > 1$ be a constant. If $x \in \mathbb{R}^n$ is a random vector from the uniform distribution on the unit sphere $S_n = \{y \in \mathbb{R}^n : y^T y = 1\}$, then the inequality*

$$(x^T (AA^T)^{-k} x)^{1/2k} \leq \|A^{-1}\|_2 \leq \theta (x^T (AA^T)^{-k} x)^{1/2k} \quad (15.7)$$

holds with probability at least $1 - 0.8\theta^{-k/2}n^{1/2}$ ($k \geq 1$). $\square$

Note that the left-hand inequality in (15.7) always holds; it is only the right-hand inequality that is in question.

For $k = 1$, (15.7) can be written as

$$\|A^{-1}x\|_2 \leq \|A^{-1}\|_2 \leq \theta \|A^{-1}x\|_2,$$

which suggests the simple estimate $\|A^{-1}\|_2 \approx \|A^{-1}x\|_2$, where x is chosen randomly from the uniform distribution on S_n . Such vectors x can be generated from the formula

$$x_i = z_i / \|z\|_2,$$

where $z_1, \dots, z_n$ are independent random variables from the normal $N(0, 1)$ distribution [744, 1998, §3.4.1, E(6)]. If, for example, $n = 100$ and θ has the rather large value 6400 then inequality (15.7) holds with probability at least 0.9.

In order to take a smaller constant θ , for fixed n and a desired probability, we can use larger values of k . If $k = 2j$ is even then we can simplify (15.7), obtaining

$$\|(AA^T)^{-j}x\|_2^{1/2j} \leq \|A^{-1}\|_2 \leq \theta \|(AA^T)^{-j}x\|_2^{1/2j}, \quad (15.8)$$

and the minimum probability stated by the theorem is $1 - 0.8\theta^{-j}n^{1/2}$. Taking $j = 3$, for the same value $n = 100$ as before, we find that (15.8) holds with probability at least 0.9 for the considerably smaller value $\theta = 4.31$.

Probabilistic condition estimation has not yet been adopted in any major software packages, perhaps because the other techniques work so well. For more on the probabilistic power method approach see Dixon [340, 1983], Higham [582, 1987], and Kuczyński and Woźniakowski [753, 1992] (who also analyse the more powerful Lanczos method with a random starting vector). For a probabilistic condition estimation method of very general applicability see Kenney and Laub [728, 1994] and Gudmundsson, Kenney, and Laub [529, 1995], and for applications see Kenney, Laub, and Reese [726, 1998], [727, 1998].

The condition estimators described above assume that a single estimate is required for a matrix given in its entirety. Condition estimators have also been developed for more specialized situations. Bischof [115, 1990] develops a method for estimating the smallest singular value of a triangular matrix that processes the matrix a row or a column at a time. This “incremental condition estimation” method can be used to monitor the condition of a triangular matrix as it is generated, and so is useful in the context of matrix factorizations such as the QR factorization with column pivoting. The estimator is generalized to sparse

matrices by Bischof, Lewis, and Pierce [116, 1990]. Barlow and Vemulapati [76, 1992] develop a 1-norm incremental condition estimator with look-ahead for sparse matrices.

Condition estimates are also required in applications where a matrix factorization is repeatedly updated as a matrix undergoes low rank changes. Algorithms designed for a recursive least squares problem and employing the Lanczos method are described by Ferng, Golub, and Plemmons [411, 1991]. Pierce and Plemmons [941, 1992] describe an algorithm for use with the Cholesky factorization as the factorization is updated, while Shroff and Bischof [1039, 1992] treat the QR factorization.

15.6. Condition Numbers of Tridiagonal Matrices

For a bidiagonal matrix B , $|B^{-1}| = M(B)^{-1}$ (see §8.3), so the condition numbers $\kappa_{E,f}$ and $\text{cond}_{E,f}$ can be computed exactly with an order of magnitude less work than is required to compute B^{-1} explicitly. This property holds more generally for several types of tridiagonal matrix, as a consequence of the following result. Recall that the LU factors of a tridiagonal matrix are bidiagonal and may be computed using the formulae (9.19).

Theorem 15.7. *If the nonsingular tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$ and $|L||U| = |A|$, then $|U^{-1}||L^{-1}| = |A^{-1}|$.*

Proof. Using the notation of (9.18), $|L||U| = |A| = |LU|$ if and only if, for all i ,

$$|l_i e_{i-1} + u_i| = |l_i| |e_{i-1}| + |u_i|,$$

that is, if

$$\text{sign} \left(\frac{l_i e_{i-1}}{u_i} \right) = 1. \quad (15.9)$$

Using the formulae

$$\begin{aligned} (U^{-1})_{ij} &= \frac{1}{u_j} \prod_{p=i}^{j-1} \left(\frac{-e_p}{u_p} \right), \quad j \geq i, \\ (L^{-1})_{ij} &= \prod_{p=j}^{i-1} (-l_{p+1}), \quad i \geq j, \end{aligned} \quad (15.10)$$

we have

$$\begin{aligned} (U^{-1}L^{-1})_{ij} &= \sum_{k=\max(i,j)}^n (U^{-1})_{ik} (L^{-1})_{kj} \\ &= \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=i}^{k-1} \left(\frac{-e_p}{u_p} \right) \prod_{p=j}^{k-1} (-l_{p+1}) \\ &= \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \end{aligned}$$

$$\begin{aligned}
& \times \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_p} \right) \\
& = \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \\
& \quad \times \frac{1}{u_{\max(i,j)}} \cdot \sum_{k=\max(i,j)}^n \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_{p+1}} \right).
\end{aligned}$$

Thus, in view of (15.9), it is clear that $|U^{-1}L^{-1}|_{ij} = (|U^{-1}||L^{-1}|)_{ij}$, as required. $\square$

Since L and U are bidiagonal, $|U^{-1}| = M(U)^{-1}$ and $|L^{-1}| = M(L)^{-1}$. Hence, if $|A| = |L||U|$, then, from Theorem 15.7,

$$|A^{-1}|y = |U^{-1}||L^{-1}|y = M(U)^{-1}M(L)^{-1}y. \quad (15.11)$$

It follows that we can compute any of the condition numbers or forward error bounds of interest *exactly* by solving two bidiagonal systems. The cost is $O(n)$ flops, as opposed to the $O(n^2)$ flops needed to compute the inverse of a tridiagonal matrix.

When does the condition $|A| = |L||U|$ hold? Theorem 9.12 shows that it holds if the tridiagonal matrix A is symmetric positive definite, totally positive, or an M -matrix. So for these types of matrix we have a very satisfactory way to compute the condition number.

If A is tridiagonal and diagonally dominant by rows, then we can compute in $O(n)$ flops an upper bound for the condition number that is not more than a factor $2n - 1$ too large.

Theorem 15.8. *Suppose the nonsingular, row diagonally dominant tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$. Then, if $y \geq 0$,*

$$\| |U^{-1}||L^{-1}|y \|_{\infty} \leq (2n - 1) \| |A^{-1}|y \|_{\infty}.$$

Proof. We have $L^{-1} = UA^{-1}$, so

$$|U^{-1}||L^{-1}|y \leq |U^{-1}||U||A^{-1}|y,$$

and the result follows on taking norms and applying Lemma 8.8. $\square$

In fact, it is possible to compute $|A^{-1}|y$ exactly in $O(n)$ operations for *any* tridiagonal matrix. This is a consequence of the special form of the inverse of a tridiagonal matrix.

Theorem 15.9 (Ikebe). *Let $A \in \mathbb{R}^{n \times n}$ be tridiagonal and irreducible (that is, $a_{i+1,i}$ and $a_{i,i+1}$ are nonzero for all i). Then there are vectors x , y , p , and q such that*

$$(A^{-1})_{ij} = \begin{cases} x_i y_j, & i \leq j, \\ p_i q_j, & i \geq j. \end{cases} \quad \square$$

This result says that the inverse of an irreducible tridiagonal matrix is the upper triangular part of a rank-1 matrix joined along the diagonal to the lower triangular part of another rank-1 matrix. If A is reducible then it has the block form $\begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}$ (or its transpose), and this blocking can be applied recursively until the diagonal blocks are all irreducible, at which point the theorem can be applied to the diagonal blocks.

The vectors x , y , p , and q in Theorem 15.9 can all be computed in $O(n)$ flops, and this enables the condition numbers and forward error bounds to be computed also in $O(n)$ flops (see Problem 15.6). Unfortunately, the vectors x , y , p , and q can have a huge dynamic range, causing the computation to break down because of overflow or underflow. For example, for the diagonally dominant tridiagonal matrix with $a_{ii} \equiv 4$, $a_{i+1,i} = a_{i,i+1} \equiv 1$, we have $(x_1 = 1) |x_n| \approx \theta^{n-1}$, $|y_1| \approx \theta^{-1}$, and $|y_n| \approx \theta^{-n}$, where $\theta = 2 + \sqrt{3} \approx 3.73$. An ingenious way of exploiting triangular factorizations to compute $\kappa_1(A)$ without explicitly computing the vectors x , y , p , and q , and techniques for avoiding overflow and underflow in these computations, are developed by Dhillon [338, 1998].

15.7. Notes and References

The clever trick (15.1) for converting the norm $\| |A|^{-1} |d| \|_\infty$ into the norm of a matrix with which products are easily formed is due to Arioli, Demmel, and Duff [30, 1989].

The p -norm power method was first derived and analysed by Boyd [155, 1974] and was later investigated by Tao [1127, 1984]. Tao applies the method to an arbitrary mixed subordinate norm $\|A\|_{\alpha,\beta}$ (see (6.6)), while Boyd takes the α - and β -norms to be p -norms (possibly different). Algorithm 15.1 can be converted to estimate $\|A\|_{\alpha,\beta}$ by making straightforward modifications to the norm-dependent terms. An algorithm that estimates $\|A\|_p$ using the power method with a specially chosen starting vector is developed by Higham [598, 1992]; the method for obtaining the starting vector is outlined in Problem 15.2. The estimate produced by this algorithm is always within a factor $n^{1-1/p}$ of $\|A\|_p$ and the cost is about $70n^2$ flops. A MATLAB M-file `pnorm` implementing this method is part of the Matrix Computation Toolbox (see Appendix D).

The finite convergence of the power method for $p = 1$ and $p = \infty$ holds more generally: if the power method is applied to the norm $\|\cdot\|_{\alpha,\beta}$ and one of the α - and β -norms is polyhedral (that is, its unit ball has a finite number of extreme points), then the iteration converges in a finite number of steps. Moreover, under a reasonable assumption, this number of steps can be bounded in terms of the number of extreme points of the unit balls in the α -norm and the dual of the β -norm. See Bartels [85, 1991] and Tao [1127, 1984] for further details.

Hager [536, 1984] gave a derivation of the 1-norm estimator based on subgradients and used the method to estimate $\kappa_1(A)$. That the method is of wide applicability because it accesses A only through matrix-vector products was recognized by Higham, who developed Algorithm 15.4 and its complex analogue and wrote Fortran 77 implementations, which use a reverse communication interface [585, 1988], [591, 1990]. These codes are used in LAPACK, the NAG library, and various other program libraries. Algorithm 15.4 is implemented in ROM on the Hewlett-

Packard HP 48G and HP 48GX calculators (along with several other LAPACK routines), in a form that estimates $\kappa_1(A)$. The Hewlett-Packard implementation is instructive because it shows that condition estimation can be efficient even for small dimensions: on a standard HP 48G, inverting A and estimating its condition number (without being given a factorization of A in either case) both take about 5 seconds for $n = 10$, while for $n = 20$ inversion takes 30 seconds and condition estimation only 20 seconds.

Moler [863, 1978] describes an early version of the LINPACK condition estimator and raises the question of the existence of counterexamples. An early version without look-ahead was incorporated in the Fortran code `decomp` in the book of Forsythe, Malcolm, and Moler [430, 1977].

Matrices for which condition estimators perform poorly can be very hard to find theoretically or with random testing, but for all the estimators described in this chapter they can be found quite easily by applying direct search optimization to the under- or overestimation ratio; see §26.3.1.

Both LINPACK and LAPACK return estimates of the *reciprocal* of the condition number, in a variable $\text{rcond} \leq 1$. Overflow for a very ill conditioned matrix is thereby avoided, and rcond is simply set to zero when singularity is detected.

A simple modification to the LINPACK estimator that can produce a larger estimate is suggested by O’Leary [900, 1980]. For sparse matrices, Grimes and Lewis [526, 1981] suggest a way to reduce the cost of the scaling strategy used in LINPACK to avoid overflow in the condition estimation. Zlatev, Wasniewski, and Schaumburg [1287, 1986] describe their experience in implementing the LINPACK condition estimation algorithm in a software package for sparse matrices.

Stewart [1070, 1980] describes an efficient way to generate random matrices of a given condition number and singular value distribution (see §28.3) and tests the LINPACK estimator on such random matrices.

Condition estimators specialized to the (generalized) Sylvester equation have been developed by Byers [191, 1984], Kågström and Westin [685, 1989], and Kågström and Poromaa [682, 1992].

A survey of condition estimators up to 1987, which includes counterexamples and the results of extensive numerical tests, is given by Higham [582, 1987].

Theorems 15.7 and 15.8 are from Higham [589, 1990]. That $\|A^{-1}\|_\infty$ can be computed in $O(n)$ flops for symmetric positive definite tridiagonal A was first shown in Higham [579, 1986].

Theorem 15.9 has a long history, having been discovered independently in various forms by different authors. The earliest reference we know for the result as stated is Ikebe [658, 1979], where a more general result for Hessenberg matrices is proved. A version of Theorem 15.9 for symmetric tridiagonal matrices was proved by Bukhberger and Emel’yanenko [174, 1973]. The culmination of the many papers on inverses of tridiagonal and Hessenberg matrices is a result of Cao and Stewart on the form of the inverse of a block matrix (A_{ij}) with $A_{ij} = 0$ for $i > j + s$ [202, 1986]; despite the generality of this result, the proof is short and elegant. *Any* banded matrix has an inverse with a special “low rank” structure; the earliest reference on the inverse of a general band matrix is Asplund [40, 1959]. For a recent survey on the inverses of symmetric tridiagonal and block tridiagonal matrices see Meurant [845, 1992].

For symmetric positive definite tridiagonal A the standard way to solve $Ax = b$ is by using a Cholesky or LDL^T factorization, rather than an LU factorization. The LINPACK routine SPTSL uses a “twisted” factorization [927, 2000] resulting from the BABE (“burn at both ends”) algorithm, which eliminates from the middle of the matrix to the top and bottom simultaneously (see [341, 1979, Chap. 7] and Higham [579, 1986]). The results of §15.6 are applicable to all these factorizations, with minor modifications.

15.7.1. LAPACK

Algorithm 15.4 is implemented in routine xLACON, which has a reverse communication interface. The LAPACK routines xPTCON and xPTRFS for symmetric positive definite tridiagonal matrices compute condition numbers using (15.11); the LAPACK routines xGTCN and xGTRFS for general tridiagonal matrices use Algorithm 15.4. LINPACK’s tridiagonal matrix routines do not incorporate condition estimation.

Problems

15.1. Show how to rewrite and approximate $\| |A| |A^{-1}| |A| \|_\infty$ so that it can be estimated using the LAPACK norm estimator (Algorithm 15.4).

15.2. (Higham [598, 1992]) The purpose of this problem is to develop a non-iterative method for choosing a starting vector for Algorithm 15.1. The idea is to choose the components of x in the order $x_1, x_2, \dots, x_n$ in an attempt to maximize $\|Ax\|_p / \|x\|_p$. Suppose $x_1, \dots, x_{k-1}$ satisfying $\|x(1:k-1)\|_p = 1$ have been determined and let $\gamma_{k-1} = \|A(:, 1:k-1)x(1:k-1)\|_p$. We now try to choose x_k , and at the same time revise $x(1:k-1)$, to give the next partial product a larger norm. Defining

$$g(\lambda, \mu) = \|\lambda A(:, 1:k-1)x(1:k-1) + \mu A(:, k)\|_p$$

we set

$$x_k = \mu^*, \quad x(1:k-1) \leftarrow \lambda^* x(1:k-1),$$

where

$$g(\lambda^*, \mu^*) = \max \left\{ g(\lambda, \mu) : \left\| \begin{bmatrix} \lambda \\ \mu \end{bmatrix} \right\|_p = 1 \right\}.$$

Then $\|x(1:k)\|_p = 1$ and

$$\gamma_k = \|A(:, 1:k)x(1:k)\|_p \geq \gamma_{k-1}.$$

Develop this outline into a practical algorithm. What can you prove about the quality of the estimate $\|Ax\|_p / \|x\|_p$ that it produces?

15.3. (Higham [591, 1990]) Let the $n \times n$ symmetric tridiagonal matrix $T_n(\alpha) = (t_{ij})$ be defined by

$$t_{ii} = \begin{cases} 2, & i = 1, \\ i, & 2 \leq i \leq n-1, \\ -t_{n,n-1} + \alpha, & i = n, \end{cases}$$

$$t_{i,i+1} = \begin{cases} -((i+1)/2 - \alpha) & \text{if } i \text{ is odd,} \\ -i/2 & \text{if } i \text{ is even.} \end{cases}$$

For example, $T_6(\alpha)$ is given by

$$\begin{bmatrix} 2 & -(1-\alpha) & & & & \\ -(1-\alpha) & 2 & -1 & & & \\ & -1 & 3 & -(2-\alpha) & & \\ & & -(2-\alpha) & 4 & -2 & \\ & & & -2 & 5 & -(3-\alpha) \\ & & & & -(3-\alpha) & 3 \end{bmatrix}.$$

Note that, for all α , $\|T_n(\alpha)e_{n-1}\|_1 = \|T_n(\alpha)\|_1$. Show that if Algorithm 15.3 is applied to $T_n(\alpha)$ with $0 \leq \alpha < 1$ then $x = e_{i-1}$ on the i th iteration, for $i = 2, \dots, n$, with convergence on the n th iteration. Algorithm 15.4, however, terminates after five iterations with $y^5 = T_n(\alpha)e_4$, and

$$\frac{\|y^5\|_1}{\|T_n(\alpha)\|_1} = \frac{8-\alpha}{2n-2-\alpha} \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

Show that the extra estimate saves the day, so that Algorithm 15.4 returns a final estimate that is within a factor 3 of the true norm, for any $\alpha < 1$.

15.4. Let $PA = LU$ be an LU factorization with partial pivoting of $A \in \mathbb{R}^{n \times n}$. Show that

$$\frac{\|A^{-1}\|_\infty}{2^{n-1}} \leq \|U^{-1}\|_\infty \leq n\|A^{-1}\|_\infty.$$

15.5. Investigate the behaviour of Algorithms 15.3 and 15.4 for

- (a) (Higham [585, 1988]) the Pei matrix, $A = \alpha I + ee^T$ ($\alpha \geq 0$), and for the upper bidiagonal matrix with 1s on the diagonal and the first superdiagonal, and
- (b) (Dhillon [338, 1998]) the inverse of the tridiagonal matrix with zero diagonal and ones on the sub- and superdiagonals.

15.6. (Ikebe [658, 1979], Higham [579, 1986]) Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, tridiagonal, and irreducible. By equating the last columns in $AA^{-1} = I$ and the first rows in $A^{-1}A = I$, show how to compute the vectors x and y in Theorem 15.9 in $O(n)$ flops. Hence obtain an $O(n)$ flops algorithm for computing $\|A^{-1}d\|_\infty$, where $d \geq 0$.

15.7. The representation of Theorem 15.9 for the inverse of nonsingular, tridiagonal, and irreducible $A \in \mathbb{R}^{n \times n}$ involves $4n$ parameters, yet A depends only on $3n - 2$ parameters. Obtain an alternative representation that involves only $3n - 2$ parameters. (Hint: symmetrize the matrix.)

15.8. (RESEARCH PROBLEM) Let $A \in \mathbb{R}^{n \times n}$ be diagonally dominant by rows, let $A = LU$ be an LU factorization, and let $y \geq 0$. What is the maximum size of $\|U^{-1}\|L^{-1}y\|_\infty / \|A^{-1}y\|_\infty$? This is an open problem raised in [589, 1990]. In a small number of numerical experiments with full random matrices the ratio has been found to be less than 2 [589, 1990], [885, 1986].

Chapter 16

The Sylvester Equation

*We must commence, not with a square,
but with an oblong arrangement of terms consisting, suppose,
of m lines and n columns.*

*This will not in itself represent a determinant,
but is, as it were, a Matrix out of which we may form
various systems of determinants by fixing upon a number p ,
and selecting at will p lines and p columns,
the squares corresponding to which may be termed
determinants of the p th order.*

— J. J. SYLVESTER, *Additions to the Articles, "On a New Class of Theorems," and "On Pascal's Theorem"* (1850)

*I have in previous papers defined a "Matrix" as a rectangular array of terms,
out of which different systems of determinants may be engendered,
as from the womb of a common parent;
these cognate determinants being
by no means isolated in their relations to one another,
but subject to certain simple laws of
mutual dependence and simultaneous deperition.*

— J. J. SYLVESTER, *On the Relation Between the Minor Determinants of Linearly Equivalent Quadratic Functions* (1851)

The linear matrix equation

$$AX - XB = C, \quad (16.1)$$

where $A \in \mathbb{R}^{m \times m}$, $B \in \mathbb{R}^{n \times n}$, and $C \in \mathbb{R}^{m \times n}$ are given and $X \in \mathbb{R}^{m \times n}$ is to be determined, is called the *Sylvester equation*. It is of pedagogical interest because it includes as special cases several important linear equation problems:

1. linear system: $Ax = c$,
2. multiple right-hand side linear system: $AX = C$,
3. matrix inversion: $AX = I$,
4. eigenvector corresponding to given eigenvalue b : $(A - bI)x = 0$,
5. commuting matrices: $AX - XA = 0$.

The Sylvester equation arises in its full generality in various applications. For example, the equations

$$\begin{bmatrix} I & -X \\ 0 & I \end{bmatrix} \begin{bmatrix} A & -C \\ 0 & B \end{bmatrix} \begin{bmatrix} I & X \\ 0 & I \end{bmatrix} = \begin{bmatrix} A & AX - XB - C \\ 0 & B \end{bmatrix}$$

show that block-diagonalizing a block triangular matrix is equivalent to solving a Sylvester equation. The Sylvester equation can also be produced from finite difference discretization of a separable elliptic boundary value problem on a rectangular domain, where A and B represent application of a difference operator in the “ y ” and “ x ” directions, respectively [1059, 1991].

That (16.1) is merely a linear system is emphasized by writing it in the form

$$(I_n \otimes A - B^T \otimes I_m) \text{vec}(X) = \text{vec}(C), \quad (16.2)$$

where $A \otimes B := (a_{ij}B)$ is a Kronecker product and the vec operator stacks the columns of a matrix into one long vector. For future reference, we note the useful relation

$$\text{vec}(AXB) = (B^T \otimes A) \text{vec}(X).$$

(See Horn and Johnson [637, 1991, Chap. 4] for a detailed presentation of properties of the Kronecker product and the vec operator.) The $mn \times mn$ coefficient matrix in (16.2) has a very special structure, illustrated for $n = 3$ by

$$\begin{bmatrix} A - b_{11}I & -b_{21}I & -b_{31}I \\ -b_{12}I & A - b_{22}I & -b_{32}I \\ -b_{13}I & -b_{23}I & A - b_{33}I \end{bmatrix}.$$

In dealing with the Sylvester equation it is vital to consider this structure and not treat (16.2) as a general linear system.

Since the mn eigenvalues of $I_n \otimes A - B^T \otimes I_m$ are given by

$$\lambda_{ij}(I_n \otimes A - B^T \otimes I_m) = \lambda_i(A) - \lambda_j(B), \quad i = 1:m, \quad j = 1:n, \quad (16.3)$$

the Sylvester equation is nonsingular precisely when A and B have no eigenvalues in common.

In this chapter we briefly discuss the Schur method for solving the Sylvester equation and summarize its rounding error analysis. Then we determine the backward error for the Sylvester equation, investigate its relationship with the residual, and derive a condition number. All these results respect the structure of the Sylvester equation and are relevant to any solution method. We also consider the special case of the Lyapunov equation and mention how the results extend to generalizations of the Sylvester equation.

16.1. Solving the Sylvester Equation

One way to solve the Sylvester equation is to apply Gaussian elimination with partial pivoting (GEPP) to the “big” system (16.2), but the structure of the coefficient matrix cannot be exploited and the cost is a prohibitive $O(m^3n^3)$ flops. A more efficient method, requiring $O(m^3 + n^3)$ flops, is obtained with the aid of Schur decompositions of A and B . Let A and B have the real Schur decompositions

$$A = URU^T, \quad B = VSV^T, \quad (16.4)$$

where U and V are orthogonal and R and S are quasi-triangular, that is, block triangular with 1×1 or 2×2 diagonal blocks, and with any 2×2 diagonal blocks having complex conjugate eigenvalues. (See Golub and Van Loan [509, 1996, §7.4.1] for more details of the real Schur decomposition.)

With the decompositions (16.4), the Sylvester equation transforms to

$$RZ - ZS = D \quad (Z = U^T X V, \quad D = U^T C V), \quad (16.5)$$

or, equivalently, $Pz = d$, where $P = I_n \otimes R - S^T \otimes I_m$, $z = \text{vec}(Z)$ and $d = \text{vec}(D)$. If R and S are both triangular then P is block triangular with triangular diagonal blocks, so $Pz = d$ can be solved by substitution. Expressed in the notation of (16.5), the solution process take the form of n substitutions: if S is upper triangular then

$$(R - s_{jj}I)Z(:, j) = D(:, j) + Z(:, 1:j-1)S(1:j-1, j), \quad j = 1:n.$$

Suppose now that R and S are quasi-triangular, and for definiteness assume that they are both upper quasi-triangular. Partitioning $Z = (Z_{ij})$ conformally with $R = (R_{ij})$ and $S = (S_{ij})$ we have

$$R_{ii}Z_{ij} - Z_{ij}S_{jj} = D_{ij} - \sum_{k=i+1}^m R_{ik}Z_{kj} + \sum_{k=1}^{j-1} Z_{ik}S_{kj}. \quad (16.6)$$

These equations can be used to determine the blocks of Z working up the block columns from first to last. Since R_{ii} and S_{jj} are both of order 1 or 2, each system (16.6) is a linear system of order 1, 2, or 4 for Z_{ij} ; in the latter two cases it is usually solved by GEPP (or even Gaussian elimination with complete pivoting—see Problem 16.4).

This Schur decomposition method for solving the Sylvester equation is due to Bartels and Stewart [84, 1972]. What can be said about its numerical stability? In the case where R and S are both triangular, Theorem 8.5 shows that

$$(P + \Delta P)\hat{z} = d, \quad |\Delta P| \leq c_{m,n}u|P|, \quad (16.7)$$

where $c_{m,n}$ denotes a constant depending on the dimensions m and n (in fact, we can take $c_{m,n} \approx mn$). Thus $|d - P\bar{z}| \leq c_{m,n}u|P||\bar{z}|$, which implies the weaker inequality

$$|D - (R\hat{Z} - \hat{Z}S)| \leq c_{m,n}u(|R||\hat{Z}| + |\hat{Z}||S|). \quad (16.8)$$

If R or S is quasi-triangular then the error analysis depends on how the systems of dimension 2 or 4 are solved. If GEPP followed by fixed precision iterative refinement is used for each of these systems " $\overline{P}\bar{z} = \bar{d}$ ", and if for each system $\overline{P}$ is not too ill conditioned and the vector $|\overline{P}||\bar{z}|$ is not too badly scaled, then (16.7) and (16.8) remain valid (see §12.2). Otherwise, we have only a normwise bound

$$\|D - (R\hat{Z} - \hat{Z}S)\|_F \leq c'_{m,n}u(\|R\|_F + \|S\|_F)\|\hat{Z}\|_F.$$

Because the transformation of a matrix to Schur form (by the QR algorithm) is a backward stable process¹³ it is true overall that

$$\|C - (A\hat{X} - \hat{X}B)\|_F \leq c''_{m,n}u(\|A\|_F + \|B\|_F)\|\hat{X}\|_F. \quad (16.9)$$

Thus the relative residual is guaranteed to be bounded by a modest multiple of the unit roundoff u .

Golub, Nash, and Van Loan [503, 1979] suggested a modification of the Bartels–Stewart algorithm in which A is reduced only to upper Hessenberg form: $A = UHU^T$. The reduced system $HZ - ZS = D$ can be solved by solving n systems that are either upper Hessenberg or differ from upper Hessenberg form by the addition of an extra subdiagonal. As shown in [503, 1979], the Hessenberg–Schur algorithm has a smaller flop count than the Bartels–Stewart algorithm, with the improvement depending on the relative sizes of m and n . The computed solution $\hat{X}$ again satisfies (16.9).

The use of iterative methods to solve the Sylvester equation has attracted attention recently for applications where A and B are large and sparse [628, 1995], [645, 1992], [1059, 1991], [1202, 1988]. The iterations are usually terminated when an inequality of the form (16.9) holds, so here the size of the relative residual is known a priori (assuming the method converges).

16.2. Backward Error

We saw in the last section that standard methods for solving the Sylvester equation are guaranteed to produce a small relative residual. Does a small relative residual imply a small backward error? The answer to this question for a general linear system is yes (Theorem 7.1). But for the highly structured Sylvester equation the answer must be no, because for the special case of matrix inversion we know that a small residual does not imply a small backward error (§14.1). In this section we investigate the relationship between residual and backward error for the Sylvester equation.

¹³See Golub and Van Loan [509, 1996, §7.5.6]. A proof is outside the scope of this book, but the necessary tools are Lemmas 19.3 and 19.9 about Householder and Givens rotations.

The normwise backward error of an approximate solution Y to (16.1) is defined by

$$\eta(Y) = \min\{\epsilon : (A + \Delta A)Y - Y(B + \Delta B) = C + \Delta C, \|\Delta A\|_F \leq \epsilon\alpha, \|\Delta B\|_F \leq \epsilon\beta, \|\Delta C\|_F \leq \epsilon\gamma\}. \quad (16.10)$$

The tolerances α , β , and γ provide some freedom in how we measure the perturbations. Of most interest is the choice $\alpha = \|A\|_F$, $\beta = \|B\|_F$, $\gamma = \|C\|_F$, for which we will call η the *normwise relative backward error*. The equation $(A + \Delta A)Y - Y(B + \Delta B) = C + \Delta C$ may be written

$$\Delta AY - Y\Delta B - \Delta C = R, \quad (16.11)$$

where the residual $R = C - (AY - YB)$. A small backward error implies a small relative residual since, using the optimal perturbations from (16.10) in (16.11), we have

$$\|R\|_F = \|\Delta AY - Y\Delta B - \Delta C\|_F \leq ((\alpha + \beta)\|Y\|_F + \gamma)\eta(Y). \quad (16.12)$$

To explore the converse question of what the residual implies about the backward error we begin by transforming (16.11) using the SVD of Y (see §6.4), $Y = U\Sigma V^T$, where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal and $\Sigma = \text{diag}(\sigma_i) \in \mathbb{R}^{m \times n}$. The numbers $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ are the singular values of Y and we define, in addition, $\sigma_{\min(m,n)+1} = \dots = \sigma_{\max(m,n)} = 0$. Equation (16.11) transforms to

$$\widetilde{\Delta A}\Sigma - \Sigma\widetilde{\Delta B} - \widetilde{\Delta C} = \widetilde{R}, \quad (16.13)$$

where

$$\widetilde{\Delta A} = U^T \Delta AU, \quad \widetilde{\Delta B} = V^T \Delta BV, \quad \widetilde{\Delta C} = U^T \Delta CV, \quad \widetilde{R} = U^T RV.$$

This is an underdetermined system, with mn equations in $m^2 + n^2 + mn$ unknowns. We can write it in the uncoupled form¹⁴

$$\frac{\widetilde{\Delta a}_{ij}}{\alpha} \cdot \alpha\sigma_j - \beta\sigma_i \cdot \frac{\widetilde{\Delta b}_{ij}}{\beta} - \gamma \cdot \frac{\widetilde{\Delta c}_{ij}}{\gamma} = \widetilde{r}_{ij}, \quad i = 1:m, \quad j = 1:n. \quad (16.14)$$

For each i and j it is straightforward to show that the minimum of $(\alpha^{-1}\widetilde{\Delta a}_{ij})^2 + (\beta^{-1}\widetilde{\Delta b}_{ij})^2 + (\gamma^{-1}\widetilde{\Delta c}_{ij})^2$ subject to (16.14) is attained for

$$\begin{aligned} \frac{\widetilde{\Delta a}_{ij}}{\alpha} &= \frac{\alpha\sigma_j}{\alpha^2\sigma_j^2 + \beta^2\sigma_i^2 + \gamma^2} \widetilde{r}_{ij}, \\ \frac{\widetilde{\Delta b}_{ij}}{\beta} &= \frac{-\beta\sigma_i}{\alpha^2\sigma_j^2 + \beta^2\sigma_i^2 + \gamma^2} \widetilde{r}_{ij}, \\ \frac{\widetilde{\Delta c}_{ij}}{\gamma} &= \frac{-\gamma}{\alpha^2\sigma_j^2 + \beta^2\sigma_i^2 + \gamma^2} \widetilde{r}_{ij}. \end{aligned}$$

¹⁴For notational convenience we extend $\widetilde{\Delta A}$ (if $m < n$) or $\widetilde{\Delta B}$ (if $m > n$) to dimension $m \times n$; the “fictitious” elements will be set to zero by the minimization.

These matrices minimize

$$\left\| \begin{bmatrix} \widetilde{\Delta A} & \widetilde{\Delta B} & \widetilde{\Delta C} \\ \alpha & \beta & \gamma \end{bmatrix} \right\|_F \equiv \left(\left\| \frac{\Delta A}{\alpha} \right\|_F^2 + \left\| \frac{\Delta B}{\beta} \right\|_F^2 + \left\| \frac{\Delta C}{\gamma} \right\|_F^2 \right)^{1/2}.$$

Since $\eta(Y)$ is the minimum value of $\max\{ \|\alpha^{-1}\Delta A\|_F, \|\beta^{-1}\Delta B\|_F, \|\gamma^{-1}\Delta C\|_F \}$, it follows that

$$\frac{\xi}{\sqrt{3}} \leq \eta(Y) \leq \xi, \quad (16.15)$$

where

$$\begin{aligned} \xi &:= \min\{ \left\| \begin{bmatrix} \alpha^{-1}\widetilde{\Delta A} & \beta^{-1}\widetilde{\Delta B} & \gamma^{-1}\widetilde{\Delta C} \end{bmatrix} \right\|_F \} \\ &= \left(\sum_{i=1}^m \sum_{j=1}^n \frac{\widetilde{r}_{ij}^2}{\alpha^2\sigma_j^2 + \beta^2\sigma_i^2 + \gamma^2} \right)^{1/2}. \end{aligned} \quad (16.16)$$

This expression shows that the backward error is approximately equal not to the normwise relative residual $\|R\|_F/((\alpha + \beta)\|X\|_F + \gamma)$, but to a componentwise residual corresponding to the diagonalized equation (16.13).

From (16.15) and (16.16) we deduce that

$$\eta(Y) \leq \mu \frac{\|R\|_F}{(\alpha + \beta)\|Y\|_F + \gamma}, \quad (16.17)$$

where

$$\mu := \frac{(\alpha + \beta)\|Y\|_F + \gamma}{(\alpha^2\sigma_n^2 + \beta^2\sigma_m^2 + \gamma^2)^{1/2}}. \quad (16.18)$$

The scalar $\mu \geq 1$ is an amplification factor that measures by how much, at worst, the backward error can exceed the normwise relative residual. We now examine μ more closely, concentrating on the normwise relative backward error, for which $\alpha = \|A\|_F$, $\beta = \|B\|_F$, and $\gamma = \|C\|_F$.

First, note that if $n = 1$ and $B = 0$, so that the Sylvester equation reduces to a linear system $Ay = c$, then $\sigma_1 = \|y\|_2$ and $\sigma_k = 0$ for $k > 1$, so $\mu = (\|A\|_F\|y\|_2 + \|c\|_2)/(\|A\|_F^2\|y\|_2^2 + \|c\|_2^2)^{1/2}$. Clearly, $1 \leq \mu \leq \sqrt{2}$, and so we recover Theorem 7.1 (for the 2-norm) from (16.12) and (16.17), to within a constant factor.

If $m = n$ then

$$\mu = \frac{(\|A\|_F + \|B\|_F)\|Y\|_F + \|C\|_F}{((\|A\|_F^2 + \|B\|_F^2)\sigma_{\min}(Y)^2 + \|C\|_F^2)^{1/2}}. \quad (16.19)$$

We see that μ is large only when

$$\|Y\|_F \gg \sigma_{\min}(Y) \quad \text{and} \quad \|Y\|_F \gg \frac{\|C\|_F}{\|A\|_F + \|B\|_F}, \quad (16.20)$$

that is, when Y is ill conditioned and Y is a large-normed solution to the Sylvester equation. In the general case, with $m \neq n$, one of σ_m^2 and σ_n^2 is always zero and hence μ can be large for a third reason: A (if $m < n$) or B (if $m > n$) greatly exceeds the rest of the data in norm; in these cases the Sylvester equation is badly

scaled. However, if we set $\alpha = \beta = \|A\|_F + \|B\|_F$, which corresponds to regarding A and B as comprising a single set of data, then bad scaling does not affect μ .

If we allow only A and B to be perturbed in (16.10) (as may be desirable if the right-hand side C is known exactly), then $\gamma = 0$ and (16.19) and (16.20) remain valid with $\|C\|_F$ replaced by zero. In this case $\mu \geq \|Y\|_F \|Y^+\|_2 \approx \kappa_2(Y)$ (for any m and n), so μ is large whenever Y is ill conditioned (and included in this case is matrix inversion). Conditions involving controllability which guarantee that the solution to the Sylvester equation with $m = n$ is nonsingular are given by Hearon [554, 1977], while Datta [292, 1988] gives a determinantal condition for nonsingularity. It is an open problem to derive conditions for the Sylvester equation to have a well-conditioned solution (see Problem 16.5).

The following numerical example illustrates the above analysis. This particular example was carefully chosen so that the entries of A and B are of a simple form, but equally effective examples are easily generated using random, ill-conditioned A and B of dimension $m, n \geq 2$. Let

$$A = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \quad B = A - \alpha \begin{bmatrix} 1 + \alpha & 0 \\ 0 & 1 \end{bmatrix}.$$

Define C by the property that $\text{vec}(C)$ is the singular vector corresponding to the smallest singular value of $I_n \otimes A - B^T \otimes I_m$. With $\alpha = 10^{-6}$, we solved the Sylvester equation in MATLAB by the Bartels–Stewart algorithm and found that the computed solution $\hat{X}$ satisfies

$$\frac{\|R\|_F}{(\|A\|_F + \|B\|_F)\|\hat{X}\|_F + \|C\|_F} = 2.82 \times 10^{-17}, \quad \sigma(\hat{X}) = \{2 \times 10^{18}, 5 \times 10^5\},$$

$$\eta(\hat{X}) \approx \xi = 2.21 \times 10^{-8}, \quad \mu = 5.66 \times 10^{12}.$$

Although $\hat{X}$ has a very acceptable residual (as it must in view of (16.9)), its backward error is eight orders of magnitude larger than is necessary to achieve backward stability. We solved the same Sylvester equation using GEPP on the system (16.2). The relative residual was again less than u , but the backward error was appreciably larger: $\eta(\hat{X}) \approx 1.53 \times 10^{-5}$.

One conclusion we can draw from the analysis is that *standard methods for solving the Sylvester equation are at best conditionally backward stable*, since there exist rounding errors such that $\tilde{r}_{m,n}$ is the only nonzero element of $\tilde{R}$, and then (16.17) is an approximate equality, with μ possibly large.

16.2.1. The Lyapunov Equation

If we put $B = -A^T$ in the Sylvester equation we obtain

$$AX + XA^T = C,$$

which is called the *Lyapunov equation*. This equation plays a major role in control and systems theory and it can be solved using the same techniques as for the Sylvester equation.

If $C = C^T$ then $C = AX + XA^T = X^T A^T + AX^T = C^T$, so X and X^T are both solutions to the Lyapunov equation. If the Lyapunov equation is nonsingular (equivalently, $\lambda_i(A) + \lambda_j(A) \neq 0$ for all i and j , by (16.3)) it therefore has a unique symmetric solution.

We assume that C is symmetric and that Y is a symmetric approximate solution. The definition of backward error is now

$$\eta(Y) = \min\{\epsilon : (A + \Delta A)Y + Y(A + \Delta A)^T = C + \Delta C, \quad \|\Delta A\|_F \leq \epsilon\alpha, \\ \Delta C = \Delta C^T, \quad \|\Delta C\|_F \leq \epsilon\gamma\}.$$

The analogue of (16.11) is $\Delta AY + Y\Delta A^T - \Delta C = R := C - AY - YA^T$. Let $Y = UAU^T$ be a spectral decomposition, with $A = \text{diag}(\lambda_i)$. Then the residual equation transforms to

$$\widetilde{\Delta A}A + A\widetilde{\Delta A}^T - \widetilde{\Delta C} = \widetilde{R},$$

where $\widetilde{\Delta A} = U^T \Delta AU$, $\widetilde{\Delta C} = U^T \Delta CU$, and $\widetilde{R} = U^T RU$. This system can be written in uncoupled form as

$$\frac{\widetilde{\Delta a}_{ij}}{\alpha} \cdot \alpha\lambda_j + \alpha\lambda_i \cdot \frac{\widetilde{\Delta a}_{ji}}{\alpha} - \gamma \cdot \frac{\widetilde{\Delta c}_{ij}}{\gamma} = \widetilde{r}_{ij}, \quad i, j = 1:n. \quad (16.21)$$

We can obtain the minimum value of $\|[\alpha^{-1}\Delta A \quad \gamma^{-1}\Delta C]\|_F$ by minimizing

$$(\alpha^{-1}\widetilde{\Delta a}_{ij})^2 + (\alpha^{-1}\widetilde{\Delta a}_{ji})^2 + 2(\gamma^{-1}\widetilde{\Delta c}_{ij})^2$$

subject to (16.21), for $i, j = 1:n$. The solution is

$$\frac{\widetilde{\Delta a}_{ij}}{\alpha} = \frac{2\alpha\lambda_j}{2\alpha^2(\lambda_i^2 + \lambda_j^2) + \gamma^2} \widetilde{r}_{ij}, \quad \frac{\widetilde{\Delta c}_{ij}}{\gamma} = \frac{-\gamma}{2\alpha^2(\lambda_i^2 + \lambda_j^2) + \gamma^2} \widetilde{r}_{ij}.$$

(Note that $\widetilde{\Delta C}$ is symmetric since $\widetilde{R}$ is.) It follows that $\xi/\sqrt{2} \leq \eta(Y) \leq \xi$, where

$$\xi^2 = \sum_{i,j=1}^n \frac{(4\alpha^2\lambda_j^2 + \gamma^2)\widetilde{r}_{ij}^2}{(2\alpha^2(\lambda_i^2 + \lambda_j^2) + \gamma^2)^2} \leq \sum_{i,j=1}^n \frac{2\widetilde{r}_{ij}^2}{(2\alpha^2(\lambda_i^2 + \lambda_j^2) + \gamma^2)},$$

where the last inequality is usually a good approximation. Comparing with (16.16) we see that respecting the extra structure of the Lyapunov equation has essentially no effect on the backward error.

Finally, the analogue of (16.17) and (16.18) is

$$\eta(Y) \leq \mu \frac{\|R\|_F}{2\alpha\|Y\|_F + \gamma}, \quad \mu := \frac{\sqrt{2}(2\alpha\|Y\|_F + \gamma)}{(4\alpha^2\lambda_*^2 + \gamma^2)^{1/2}},$$

where $\lambda_* = \min_i |\lambda_i|$.

16.3. Perturbation Result

To derive a perturbation result we consider the perturbed Sylvester equation

$$(A + \Delta A)(X + \Delta X) - (X + \Delta X)(B + \Delta B) = C + \Delta C,$$

which, on dropping second-order terms, becomes

$$A\Delta X - \Delta X B = \Delta C - \Delta A X + X \Delta B.$$

This system may be written in the form

$$P \operatorname{vec}(\Delta X) = - \begin{bmatrix} X^T \otimes I_m & -I_n \otimes X & -I_{mn} \end{bmatrix} \begin{bmatrix} \operatorname{vec}(\Delta A) \\ \operatorname{vec}(\Delta B) \\ \operatorname{vec}(\Delta C) \end{bmatrix}, \quad (16.22)$$

where $P = I_n \otimes A - B^T \otimes I_m$. If we measure the perturbations normwise by

$$\epsilon = \max \left\{ \frac{\|\Delta A\|_F}{\alpha}, \frac{\|\Delta B\|_F}{\beta}, \frac{\|\Delta C\|_F}{\gamma} \right\},$$

where α , β , and γ are tolerances as in (16.10), then

$$\frac{\|\Delta X\|_F}{\|X\|_F} \leq \sqrt{3} \Psi \epsilon \quad (16.23)$$

is a sharp bound (to first order in ϵ), where

$$\Psi = \|P^{-1} [\alpha(X^T \otimes I_m) \quad -\beta(I_n \otimes X) \quad -\gamma I_{mn}] \|_2 / \|X\|_F \quad (16.24)$$

is the corresponding condition number for the Sylvester equation. The bound (16.23) can be weakened to

$$\frac{\|\Delta X\|_F}{\|X\|_F} \leq \sqrt{3} \Phi \epsilon, \quad (16.25)$$

where

$$\Phi = \|P^{-1}\|_2 \frac{(\alpha + \beta)\|X\|_F + \gamma}{\|X\|_F}.$$

If $\|P^{-1}\|_2(\alpha + \beta)\epsilon < 1/2$ then twice the upper bound in (16.25) can be shown to be a strict bound for the error. The perturbation bound (16.25) with $\alpha = \|A\|_F$, $\beta = \|B\|_F$, and $\gamma = \|C\|_F$ is the one that is usually quoted in the literature for the Sylvester equation (see [503, 1979] and [569, 1988], for example), and corresponds to applying standard perturbation theory for $Ax = b$ to (16.2). Note that $\|P^{-1}\|_2 = \operatorname{sep}(A, B)^{-1}$, where sep is the *separation* of A and B ,

$$\operatorname{sep}(A, B) = \min_{X \neq 0} \frac{\|AX - XB\|_F}{\|X\|_F}. \quad (16.26)$$

The sep function is an important tool for measuring invariant subspace sensitivity [509, 1996, §7.2.4], [1064, 1973], [1189, 1979].

For the Lyapunov equation, a similar derivation to the one above shows that the condition number is

$$\|(I_n \otimes A + A \otimes I_n)^{-1} [\alpha((X^T \otimes I_n) + (I_n \otimes X)\Pi^T), -\gamma I_{n^2}] \|_2 / \|X\|_F, \quad (16.27)$$

where Π is the vec-permutation matrix, which is defined by the property that $\text{vec}(A^T) = \Pi \text{vec}(A)$.

How much can the bounds (16.23) and (16.25) differ? The answer is by an arbitrary factor. To show this we consider the case where B is normal (or equivalently, A is normal if we transpose the Sylvester equation). We can assume B is in Schur form, thus $B = \text{diag}(\mu_j)$ (with the μ_j possibly complex). Then $P = \text{diag}(A - \mu_{jj}I_m)$, and it is straightforward to show that if $X = [x_1, \dots, x_n]$, and if we approximate the 2-norms in the definitions of Ψ and Φ by Frobenius norms, then

$$\begin{aligned} \Psi^2 \approx & \left(\alpha^2 \sum_{j=1}^n \|x_j\|_2^2 \|(A - \mu_{jj}I_m)^{-1}\|_F^2 + \beta^2 \sum_{j=1}^n \|(A - \mu_{jj}I_m)^{-1}X\|_F^2 \right. \\ & \left. + \gamma^2 \sum_{j=1}^n \|(A - \mu_{jj}I_m)^{-1}\|_F^2 \right) / \|X\|_F^2, \end{aligned}$$

while

$$\Phi^2 \approx \sum_{j=1}^n \|(A - \mu_{jj}I_m)^{-1}\|_F^2 ((\alpha + \beta) + \gamma/\|X\|_F)^2.$$

These formulae show that in general Ψ and Φ will be of similar magnitude, and we know that $\Psi \leq \Phi$ from the definitions. However, Ψ can be much smaller than Φ . For example, suppose that $\gamma = 0$ and

$$\|(A - \mu_{nn}I_m)^{-1}\|_F \gg \max_{j \neq n} \|(A - \mu_{jj}I_m)^{-1}\|_F.$$

Then if

$$\frac{\|x_n\|_2}{\|X\|_F} \ll 1 \quad \text{and} \quad \frac{\|(A - \mu_{nn}I_m)^{-1}X\|_F}{\|X\|_F} \ll \|(A - \mu_{nn}I_m)^{-1}\|_F,$$

we have $\Psi \ll \Phi$. Such examples are easily constructed. To illustrate, let $A = \text{diag}(2, 2, \dots, 2, 1)$ and $B = \text{diag}(1/2, 1/2, \dots, 1/2, 1 - \epsilon)$, with $\epsilon > 0$, so that $A - \mu_{nn}I_m = \text{diag}(1 + \epsilon, 1 + \epsilon, \dots, 1 + \epsilon, \epsilon)$, and let $X = (A - \mu_{nn}I_m)Y$, where $Y = [y, y, \dots, y, 0]$ with $\|(A - \mu_{nn}I_m)y\|_2 = \|A - \mu_{nn}I_m\|_2$ and $\|y\|_2 = 1$. Then, if $\gamma = O(\epsilon)$,

$$\Psi = O(\alpha^2 + \beta^2), \quad \Phi \approx \epsilon^{-1}(\alpha^2 + \beta^2).$$

To summarize, the “traditional” perturbation bound (16.25) for the Sylvester equation can severely overestimate the effect of a perturbation on the data when only A and B are perturbed, because it does not take account of the special structure of the problem. In contrast, the perturbation bound (16.23) does respect the Kronecker structure, and consequently is attainable for any given A , B , and C .

To obtain an a posteriori error bound for a computed solution $\hat{X} := X + \Delta X$ we can set $\Delta A = 0$, $\Delta B = 0$, and $\Delta C = A\hat{X} - \hat{X}B - C = R$ in (16.22), which leads to

$$\frac{\|X - \hat{X}\|_F}{\|X\|_F} \leq \|P^{-1}\|_2 \frac{\|R\|_F}{\|X\|_F}. \quad (16.28)$$

A similar but potentially much smaller bound is described in the next section.

16.4. Practical Error Bounds

For the Sylvester equation we can obtain an analogue of the practical error bound (7.31) by identifying $Ax = b$ with (16.2). For the computed residual of a computed solution $\hat{X}$ we have

$$\begin{aligned} \hat{R} &= fl(C - (fl(A\hat{X}) - fl(\hat{X}B))) = R + \Delta R, \\ |\Delta R| &\leq u|C| + \gamma_{m+2}|A||\hat{X}| + \gamma_{n+2}|\hat{X}||B| =: R_u. \end{aligned}$$

Therefore the bound is

$$\frac{\|X - \hat{X}\|}{\|\hat{X}\|} \leq \frac{\|P^{-1}(|\text{vec}(\hat{R})| + \text{vec}(R_u))\|}{\|\hat{X}\|}, \quad (16.29)$$

where $\|X\| := \max_{i,j} |x_{ij}|$. After transformation by the technique illustrated in (15.1), this bound can be estimated by the LAPACK norm estimator (Algorithm 15.4) at the cost of solving a few linear systems with coefficient matrices $I_n \otimes A - B^T \otimes I_m$ and its transpose—in other words, solving a few Sylvester equations $AX - XB = C$ and $A^T X - XB^T = D$. If the Bartels–Stewart algorithm is used, these solutions can be computed with the aid of the previously computed Schur decompositions of A and B . The condition number Ψ in (16.24) and $\text{sep}(A, B) = \|P^{-1}\|_2^{-1}$ can both be estimated in much the same way; alternatively, the power method can be used (see Ghavimi and Laub [480, 1995]). Other algorithms for efficiently estimating $\text{sep}(A, B)$ given Schur decompositions of A and B are given by Byers [191, 1984] and Kågström and Poromaa [682, 1992].

The attraction of (16.29) is that large elements in the j th column of P^{-1} may be countered by a small j th element of $\text{vec}(\hat{R}) + \text{vec}(R_u)$, making the bound much smaller than (16.28). In this sense (16.29) has better scaling properties than (16.28), although (16.29) is not actually invariant under diagonal scalings of the Sylvester equation.

We give a numerical example to illustrate the advantage of (16.29) over (16.28). Let

$$A = J_3(0), \quad B = J_3(10^{-3}), \quad c_{ij} \equiv 1,$$

where $J_n(\lambda)$ denotes a Jordan block of size n with eigenvalue λ . Solving the Sylvester equation by the Bartels–Stewart algorithm we found that the bounds are

$$(16.28) : 8.00 \times 10^{-3}, \quad (16.29) : 6.36 \times 10^{-15}$$

(where in evaluating (16.28) we replaced R by $|\hat{R}| + R_u$, as in (16.29)). Here, $\text{sep}(A, B) = 1.67 \times 10^{-16}$, and the bound (16.29) is small because relatively large

columns of P^{-1} are nullified by relatively small elements of $|\text{vec}(\widehat{R})| + \text{vec}(R_u)$. For this example, with $\alpha = \|A\|_F$, $\beta = \|B\|_F$, $\gamma = \|C\|_F$, we have

$$\Psi = 7.00 \times 10^9, \quad \Phi = 1.70 \times 10^{16},$$

confirming that the usual perturbation bound (16.25) for the Sylvester equation can be very pessimistic. Furthermore,

$$\frac{\|R\|_F}{(\|A\|_F + \|B\|_F)\|\widehat{X}\|_F + \|C\|_F} = 7.02 \times 10^{-24},$$

$$\sigma(\widehat{X}) = \{6 \times 10^{15}, 5 \times 10^8, 3 \times 10^2\},$$

$$\eta(\widehat{X}) \approx \xi = 1.00 \times 10^{-19}, \quad \mu = 2.26 \times 10^{13},$$

so we have an example where the backward error is small despite a large μ .

16.5. Extensions

The Sylvester equation can be generalized in two main ways. One retains the linearity but adds extra coefficient matrices, yielding the *generalized Sylvester equations*

$$AXB + CXD = E \tag{16.30}$$

and

$$AX - YB = C, \quad DX - YE = F. \tag{16.31}$$

These two forms are equivalent, under conditions on the coefficient matrices [236, 1987]; for example, defining $Z := XB$ and $W := -CX$, (16.30) becomes $AZ - WD = E$, $CZ + WB = 0$. Applications of generalized Sylvester equations include the computation of stable eigendecompositions of matrix pencils [327, 1987], [328, 1988], [683, 1996], [684, 1996] and the implementation of numerical methods for solving implicit ordinary differential equations [391, 1980].

The second generalization incorporates a quadratic term, yielding the *algebraic Riccati equation*

$$AX + XB - XFX + G = 0. \tag{16.32}$$

This general Riccati equation and its symmetric counterpart with $B = A^T$ and F and G symmetric are widely used in control theory.

The backward error results and perturbation theory of this chapter can be generalized in a straightforward way to (16.31) and (16.32). See Kågström [681, 1994] for (16.31) and Ghavimi and Laub [480, 1995] for (16.32). The backward error derivations do not extend to (16.30), because in this equation the coefficient matrices appear nonlinearly.

A variation of the Lyapunov equation called the *discrete-time Lyapunov equation* has the form

$$X - F^T X F = Q,$$

where $F, Q \in \mathbb{R}^{n \times n}$. As in (16.30), the data appears nonlinearly. Ghavimi and Laub [481, 1995] show how to derive an approximation to the backward error by linearizing an equation characterizing the optimal perturbations. Generalized

versions of the Lyapunov equation and the discrete-time Lyapunov equation are considered by Penzl [934, 1998].

Another generalization of the Sylvester equation, mainly of theoretical interest, is

$$\sum_{i=1}^k A_i X B_i = C,$$

where $A_i \in \mathbb{R}^{m \times m}$ and $B_i \in \mathbb{R}^{n \times n}$, $i = 1:k$. See Lancaster [764, 1970] for associated theory.

16.6. Notes and References

This chapter is based on Higham [602, 1993]. The backward error derivations make use of ideas of Ghavimi and Laub [480, 1995].

The Sylvester equation is so named because Sylvester considered the homogeneous version of the equation [1119, 1884].

Bhatia and Rosenthal [109, 1997] give a survey of theoretical results for the Sylvester equation in both finite- and infinite-dimensional spaces.

For details of the role of the Sylvester equation in the eigenproblem see Bai, Demmel, and McKenney [46, 1993], [49, 1993] and the references therein.

Iterative methods that make use of matrix inversion to solve the Sylvester equation are described by Miller [849, 1988] and Roberts [988, 1980].

Hammarling [540, 1982] gives a method for solving the Lyapunov equation $AX + XA^T = -C$ in the case where A has eigenvalues with negative real parts and C is positive semidefinite; his method directly computes the Cholesky factor of the solution (which is indeed symmetric positive definite—see Problem 16.2).

A survey of the vec operator, the Kronecker product, and the vec-permutation matrix is given together with historical comments by Henderson and Searle [561, 1981]. Historical research by Henderson, Pukelsheim, and Searle [560, 1983] indicates that the Kronecker product should be called the Zehfuss product, in recognition of an 1858 paper by Zehfuss that gives a determinantal result involving the product.

The vec-permutation matrix Π (which appears in (16.27)) is given explicitly by

$$\Pi = \sum_{i,j=1}^n (e_i e_j^T) \otimes (e_j e_i^T),$$

and has the property that $(A \otimes B)\Pi = \Pi(B \otimes A)$. It is also known as the commutation matrix [804, 1979], [805, 1979],

Applications of the Lyapunov equation in control theory, including special situations where an approximate solution of low rank is required, are discussed by Hodel [630, 1992]. A much older reference to applications is Barnett and Storey [78, 1968].

Algorithms and software for solving (16.30) are developed by Gardiner, Wette, Laub, Amato, and Moler [457, 1992], [458, 1992].

Perturbation theory for Lyapunov and Riccati equations can be found in the work of Byers [192, 1985], Hewer and Kenney [569, 1988], [724, 1990], and Gahinet, Laub, Kenney, and Hewer [450, 1990].

Chu [236, 1987] determines conditions for the existence of unique solutions to the generalized Sylvester equations (16.30) and (16.31). The appropriate conditions for (16.30) are that the pencils $A + \lambda C$ and $D + \lambda B$ are regular and the spectra of the pencils have an empty intersection, which neatly generalizes the conditions for the Sylvester equation to have a unique solution; the conditions for (16.31) are analogous.

There is much work on algorithms and software for solving the algebraic Riccati equation. For a sampling, see Laub [771, 1979], Arnold and Laub [37, 1984], Byers [193, 1987], Gardiner, and Laub [456, 1991], and Kenney, Laub, and Papadopoulos [729, 1992].

An algorithm for estimating a generalization of sep that occurs in perturbation theory for the generalized Sylvester equation (16.31) is developed by Kågström and Westin [685, 1989].

Another generalization of the Sylvester equation is to take just one equation from (16.31), $AX - YB = C$ ((16.13) is of this form). This equation can be underdetermined or overdetermined, depending on the dimensions of the coefficient matrices. Conditions involving generalized inverses that are both necessary and sufficient for the existence of a solution are given by Baksalary and Kala [58, 1979]. Ziętak examines the inconsistent case [1283, 1985] for one choice of dimensions giving an overdetermined system. Stewart [1073, 1992] shows how to compute a minimum Frobenius norm least squares solution. The even more general equation $AXB + CYD = E$ has also been analysed by Baksalary and Kala [59, 1980], who again give necessary and sufficient conditions for the existence of a solution.

16.6.1. LAPACK

The computations discussed in this chapter can all be done using LAPACK. The Bartels–Stewart algorithm can be implemented by calling `xGEES` to compute the Schur decomposition, using the level-3 BLAS routine `xGEMM` to transform the right-hand side C , calling `xTRSYL` to solve the (quasi-) triangular Sylvester equation, and using `xGEMM` to transform back to the solution X . The error bound (16.29) can be estimated using `xLACON` in conjunction with the above routines.

Routine `xLAS2` solves a real Sylvester equation $AX \pm XB = \sigma C$ in which A and B have dimension 1 or 2 and σ is a scale factor. It is called by `xTRSYL`.

LAPACK also contains a code `xTGSYL` for solving (16.31), for the case where the coefficient matrices are in (quasi-) triangular form. See Kågström and Poromaa [684, 1996] for a description of the underlying algorithm.

Problems

16.1. Show that the Sylvester equation $AX - XA = I$ has no solution.

16.2. (Bellman [101, 1970, §10.18]) Show that if the expression

$$X = - \int_0^\infty e^{At} C e^{Bt} dt$$

exists for all C it represents the unique solution of the Sylvester equation $AX + XB = C$. (Hint: consider the matrix differential equation $dZ/dt = AZ(t) + Z(t)B$, $Z(0) = C$.) Deduce that the Lyapunov equation $AX + XA^T = -C$ has a symmetric positive definite solution if A has eigenvalues with negative real parts and C is symmetric positive definite.

16.3. (Byers and Nash [195, 1987]) Let $A \in \mathbb{R}^{n \times n}$ and consider

$$\text{sep}(A, -A^T) = \min_{X \neq 0} \frac{\|AX + XA^T\|_F}{\|X\|_F}.$$

Show that there exists a minimizer X that is either symmetric or skew-symmetric.

16.4. How would you solve a Sylvester equation $AX - XB = C$ in which A and B are of dimension 1 or 2? Compare your method with the one used in the LAPACK routine `xLAS2`.

16.5. (RESEARCH PROBLEM) Derive conditions for the Sylvester equation to have a well-conditioned solution.

Chapter 17

Stationary Iterative Methods

*I recommend this method to you for imitation.
You will hardly ever again eliminate directly,
at least not when you have more than 2 unknowns.
The indirect [iterative] procedure can be done while half asleep,
or while thinking about other things.*¹⁵

— CARL FRIEDRICH GAUSS, *Letter to C. L. Gerling* (1823)

*The iterative method is commonly called the "Seidel process,"
or the "Gauss–Seidel process."
But, as Ostrowski (1952) points out,
Seidel (1874) mentions the process but advocates not using it.
Gauss nowhere mentions it.*

— GEORGE E. FORSYTHE,
Solving Linear Algebraic Equations Can Be Interesting (1953)

*The spurious contributions in $\text{null}(A)$
grow at worst linearly and
if the rounding errors are small the scheme can be quite effective.*

— HERBERT B. KELLER,
On the Solution of Singular and Semidefinite Linear Systems by Iteration (1965)

¹⁵Gauss refers here to his relaxation method for solving the normal equations. The translation is taken from Forsythe [422, 1951].

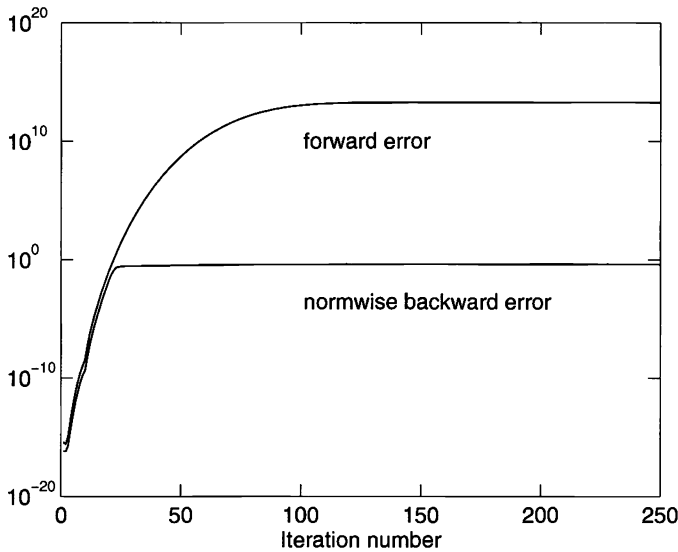
Table 17.1. *Dates of publication of selected iterative methods. Based on Young [1271, 1989].*

1845	Jacobi	Jacobi method
1874	Seidel	Gauss–Seidel method
1910	Richardson	Richardson’s method
1938–1939	Temple	Method of steepest descent
1940s	Various (analysis by Young and Frankel)	Successive overrelaxation (SOR) method
1952	Hestenes and Stiefel	Conjugate gradient method

Iterative methods for solving linear systems have a long history, going back at least to Gauss. Table 17.1 shows the dates of publication of selected methods. It is perhaps surprising, then, that rounding error analysis for iterative methods is not well developed. There are two main reasons for the paucity of error analysis. One is that in many applications accuracy requirements are modest and are satisfied without difficulty, resulting in little demand for error analysis. Certainly there is no point in computing an answer to greater accuracy than that determined by the data, and in scientific and engineering applications the data often has only a few correct digits. The second reason is that rounding error analysis for iterative methods is inherently more difficult than for direct methods, and the bounds that are obtained are harder to interpret.

In this chapter we consider a simple but important class of iterative methods, stationary iterative methods, for which a reasonably comprehensive error analysis can be given. The basic question that our analysis attempts to answer is, “What is the limiting accuracy of a method in floating point arithmetic?” Specifically, “How small can we guarantee that the backward or forward error will be over all iterations $k = 1, 2, \dots$?” Without an answer to this question we cannot be sure that a convergence test of the form $\|b - A\hat{x}_k\| \leq \epsilon$ (say) will ever be satisfied, for any given value of $\epsilon < \|b - Ax_0\|$!

As an indication of the potentially devastating effects of rounding errors we present an example constructed and discussed by Hammarling and Wilkinson [541, 1976]. Here, A is the 100×100 lower bidiagonal matrix with $a_{ii} \equiv 1.5$ and $a_{i,i-1} \equiv 1$, and $b_i \equiv 2.5$. The successive overrelaxation (SOR) method is applied in MATLAB with parameter $\omega = 1.5$, starting with the rounded version of the exact solution x , given by $x_i = 1 - (-2/3)^i$. The forward errors $\|\hat{x}_k - x\|_\infty / \|x\|_\infty$ and the ∞ -norm backward errors $\eta_{A,b}(\hat{x}_k)$ are plotted in Figure 17.1. The SOR method converges in exact arithmetic, since the iteration matrix has spectral radius $1/2$, but in the presence of rounding errors it diverges. The iterate $\hat{x}_{238}$ has a largest element of order 10^{13} , $\hat{x}_{k+2} \equiv \hat{x}_k$ for $k \geq 238$, and for $k > 100$, $\hat{x}_k(60:100) \approx (-1)^k \hat{x}_{100}(60:100)$. The divergence is not a result of ill conditioning of A , since $\kappa_\infty(A) \approx 5$. The reason for the initial rapid growth of the errors in this example is that the iteration matrix is far from normal; this allows the norms of its powers to become very large before they ultimately decay by a factor $\approx 1/2$ with each successive power. The effect of rounding errors is to cause the forward error curve in Figure 17.1 to level off near $k = 100$, instead of decaying to zero as it would

Figure 17.1. *SOR iteration.*

in exact arithmetic. More insight into the initial behaviour of the errors can be obtained using the notion of pseudo-eigenvalues; see §18.3.

17.1. Survey of Error Analysis

Before analysing stationary iterative methods, we briefly survey the published error analysis for iterative methods. For symmetric positive definite systems, Golub [507, 1962] derives both statistical and nonstatistical bounds for the forward error and residual of the Richardson method. Benschop and Ratz [104, 1971] give a statistical analysis of the effect of rounding errors on stationary iteration, under the assumption that the rounding errors are independent random variables with zero mean. Lynn [802, 1964] presents a statistical analysis for the SOR method with a symmetric positive definite matrix.

Hammarling and Wilkinson [541, 1976] give a normwise error analysis for the SOR method. With the aid of numerical examples, they emphasize that while it is the spectral radius of the iteration matrix $M^{-1}N$ that determines the asymptotic rate of convergence, it is the norms of the powers of this matrix that govern the behaviour of the iteration in the early stages. This point is also explained by Trefethen [1154, 1992], using the tool of pseudospectra.

Dennis and Walker [335, 1984] obtain bounds for $\|x - \hat{x}_{k+1}\|/\|x - \hat{x}_k\|$ for stationary iteration as a special case of error analysis of quasi-Newton methods for nonlinear systems. The bounds in [335, 1984] do not readily yield information about normwise or componentwise forward stability.

Bollen [149, 1984] analyses the class of “descent methods” for solving $Ax = b$, where A is required to be symmetric positive definite; these are obtained by iteratively using exact line searches to minimize the quadratic function $F(x) =$

$(A^{-1}b - x)^T A(A^{-1}b - x)$. The choice of search direction $p_k = b - Ax_k =: r_k$ yields the steepest descent method, while $p_k = e_j$ (unit vector), where $|r_k|_j = \|r_k\|_\infty$, gives the Gauss–Southwell method. Bollen shows that both methods are normwise backward stable as long as a condition of the form $c_n \kappa(A)u < 1$ holds. If the p_k are cyclically chosen to be the unit vectors $e_1, e_2, \dots, e_n$ then the Gauss–Seidel method results, but unfortunately no results specific to this method are given in [149, 1984].

Woźniakowski [1256, 1977] shows that the Chebyshev semi-iterative method is normwise forward stable but not normwise backward stable, and in [1257, 1978] he gives a normwise error analysis of stationary iterative methods. Some of the assumptions in [1257, 1978] are difficult to justify, as explained by Higham and Knight [618, 1993].

In [1258, 1980] Woźniakowski analyses a class of conjugate gradient algorithms (which does not include the usual conjugate gradient method). He obtains a forward error bound proportional to $\kappa(A)^{3/2}$ and a residual bound proportional to $\kappa(A)$, from which neither backward nor forward normwise stability can be deduced. We note that as part of the analysis in [1258, 1980] Woźniakowski obtains a residual bound for the steepest descent method that is proportional to $\kappa(A)$, and is therefore much weaker than the bound obtained by Bollen [149, 1984].

Zawilski [1277, 1991] shows that the cyclic Richardson method for symmetric positive definite systems is normwise forward stable provided the parameters are suitably ordered. He also derives a sharp bound for the residual that includes a factor $\kappa(A)$, and which therefore shows that the method is not normwise backward stable.

Arioli and Romani [36, 1992] give a statistical error analysis of stationary iterative methods. They investigate the relations between a statistically defined asymptotic stability factor, ill conditioning of $M^{-1}A$, where $A = M - N$ is the splitting, and the rate of convergence.

Greenbaum [519, 1989] presents a detailed error analysis of the conjugate gradient method, but her concern is with the rate of convergence rather than the attainable accuracy. An excellent survey of work concerned with the effects of rounding error on the conjugate gradient method (and the Lanczos method) is given by Greenbaum and Strakoš in the introduction of [523, 1992]; see also Greenbaum [520, 1994]. Notay [895, 1993] analyses how rounding errors influence the convergence rate of the conjugate gradient method for matrices with isolated eigenvalues at the ends of the spectrum. Van der Vorst [1201, 1990] examines the effect of rounding errors on preconditioned conjugate gradient methods with incomplete Cholesky preconditioners. Greenbaum [521, 1997] analyses the limiting backward error for a general class of iterative methods that compute the residual $b - Ax_k$ by an updating formula rather than explicitly; this class includes the conjugate gradient method and several more recent methods. Numerical stability of the GM-RES method is analysed in [355, 1995], while error analysis of Krylov solvers for symmetric indefinite systems is treated in [1045, 2000].

The analysis given in the remainder of this chapter is from Higham and Knight [618, 1993], [619, 1993], wherein more details are given. Error analysis of Kaczmarz's row-action method is given by Knight [739, 1993].

17.2. Forward Error Analysis

A stationary iterative method has the form

$$Mx_{k+1} = Nx_k + b,$$

where $A = M - N \in \mathbb{R}^{n \times n}$ is nonsingular and M is nonsingular. We assume that the spectral radius $\rho(M^{-1}N) < 1$, so that in exact arithmetic the iteration converges for any starting vector x_0 . We are not concerned with the size of constants in this analysis, so we denote by c_n a constant of order n .

The computed vectors $\hat{x}_k$ satisfy an equality of the form

$$(M + \Delta M_{k+1})\hat{x}_{k+1} = N\hat{x}_k + b + f_k,$$

which we write as

$$M\hat{x}_{k+1} = N\hat{x}_k + b - \xi_k, \quad (17.1)$$

where

$$\xi_k = \Delta M_{k+1}\hat{x}_{k+1} - f_k.$$

We will assume that M is triangular (as is the case for the Jacobi, Gauss–Seidel, SOR, and Richardson iterations), so that $|\Delta M_{k+1}| \leq c'_n u |M|$ and f_k accounts solely for the errors in forming $N\hat{x}_k + b$. Hence

$$|\xi_k| \leq c_n u (|M||\hat{x}_{k+1}| + |N||\hat{x}_k| + |b|) =: \mu_k. \quad (17.2)$$

Solving the recurrence (17.1) we obtain

$$\hat{x}_{m+1} = G^{m+1}x_0 + \sum_{k=0}^m G^k M^{-1}(b - \xi_{m-k}), \quad (17.3)$$

where $G = M^{-1}N$. Since the iteration is stationary at x ,

$$x = G^{m+1}x + \sum_{k=0}^m G^k M^{-1}b, \quad (17.4)$$

and so the error $e_{m+1} := x - \hat{x}_{m+1}$ satisfies

$$e_{m+1} = G^{m+1}e_0 + \sum_{k=0}^m G^k M^{-1}\xi_{m-k}. \quad (17.5)$$

We have

$$|e_{m+1}| \leq |G^{m+1}e_0| + \sum_{k=0}^m |G^k M^{-1}| \mu_{m-k}, \quad (17.6)$$

where μ_k is the bound for ξ_k defined in (17.2). The first term, $|G^{m+1}e_0|$, is the error of the iteration in exact arithmetic and is negligible for large m . The accuracy that can be guaranteed by the analysis is therefore determined by the last term in (17.6), and it is this term on which the rest of the analysis focuses.

At this point we can proceed by using further componentwise inequalities or by using norms. First we consider the norm approach. By taking norms in (17.6) and defining

$$\gamma_x = \sup_k \frac{\|\hat{x}_k\|_\infty}{\|x\|_\infty}, \quad (17.7)$$

we obtain

$$\begin{aligned} \|e_{m+1}\|_\infty &\leq \|G^{m+1}e_0\|_\infty + \max_{0 \leq k \leq m} \|\mu_k\|_\infty \sum_{k=0}^m \|G^k M^{-1}\|_\infty \\ &\leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \gamma_x)(\|M\|_\infty + \|N\|_\infty)\|x\|_\infty \\ &\quad \times \sum_{k=0}^\infty \|G^k M^{-1}\|_\infty, \end{aligned} \quad (17.8)$$

where the existence of the sum is assured by the result of Problem 17.1.

If $\|G\|_\infty = \|M^{-1}N\|_\infty = q < 1$ then (17.8) yields

$$\|e_{m+1}\|_\infty \leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \gamma_x)(\|M\|_\infty + \|N\|_\infty)\|x\|_\infty \frac{\|M^{-1}\|_\infty}{1 - q}.$$

Thus if q is not too close to 1 ($q \leq 0.9$, say), and γ_x and $\|M^{-1}\|_\infty$ are not too large, a small forward error is guaranteed for sufficiently large m .

Of more interest is the following componentwise development of (17.6). Defining

$$\theta_x = \sup_k \max_{1 \leq i \leq n} \left(\frac{|\hat{x}_k|_i}{|x_i|} \right), \quad (17.9)$$

so that $|\hat{x}_k| \leq \theta_x |x|$ for all k , we have from (17.2),

$$|\mu_k| \leq c_n u(1 + \theta_x)(|M| + |N|)|x|. \quad (17.10)$$

Hence (17.6) yields

$$|e_{m+1}| \leq |G^{m+1}e_0| + c_n u(1 + \theta_x) \left(\sum_{k=0}^\infty |G^k M^{-1}| \right) (|M| + |N|)|x|, \quad (17.11)$$

where, again, the existence of the sum is assured by the result of Problem 17.1. Since $A = M - N = M(I - M^{-1}N)$ we have

$$A^{-1} = \left(\sum_{k=0}^\infty (M^{-1}N)^k \right) M^{-1}.$$

The sum in (17.11) is clearly an upper bound for $|A^{-1}|$. Defining $c(A) \geq 1$ by

$$c(A) = \min \left\{ \epsilon : \sum_{k=0}^\infty |(M^{-1}N)^k M^{-1}| \leq \epsilon \left| \sum_{k=0}^\infty (M^{-1}N)^k M^{-1} \right| = \epsilon |A^{-1}| \right\}, \quad (17.12)$$

we have our final bound

$$|e_{m+1}| \leq |G^{m+1}e_0| + c_n u(1 + \theta_x) c(A) |A^{-1}|(|M| + |N|)|x|. \quad (17.13)$$

An interesting feature of stationary iteration methods is that if the elements of M and N are multiples of the elements in the corresponding positions of A , then any scaling of the form $Ax = b \rightarrow D_1 A D_2 \cdot D_2^{-1} x = D_1 b$ (D_i diagonal) leaves the eigenvalues of $M^{-1}N$ unchanged; hence the asymptotic convergence rate is independent of row and column scaling. This scale independence applies to the Jacobi and SOR iterations, but not, for example, to the stationary Richardson iteration, for which $M = I$. One of the benefits of doing a componentwise analysis is that under the above assumptions on M and N the bound (17.13) largely shares the scale independence. In (17.13) the scalar $c(A)$ is independent of the row and column scaling of A , and the term $|A^{-1}|(|M| + |N|)|x|$ scales in the same way as x . Furthermore, θ_x can be expected to depend only mildly on the row and column scaling, because the bound in (17.2) for the rounding error terms has the correct scaling properties.

What can be said about $c(A)$? In general, it can be arbitrarily large. Indeed, $c(A)$ is infinite for the Jacobi and Gauss-Seidel iterations for any $n \geq 3$ if A is the symmetric positive definite matrix with $a_{ij} = \min(i, j)$, because A^{-1} is tridiagonal and $(M^{-1}N)^k M^{-1}$ is not.

If M^{-1} and $M^{-1}N$ both have nonnegative elements then $c(A) = 1$; as we will see in the next section, this condition holds in some important instances.

Some further insight into $c(A)$ can be obtained by examining the case where $M^{-1}N \in \mathbf{C}^{n \times n}$ is diagonal with eigenvalues λ_i . It is easy to show that $c(A) = \max_i |1 - \lambda_i|/(1 - |\lambda_i|)$, so $c(A)$ can be large only if $\rho(M^{-1}N)$ is close to 1. Although $M^{-1}N$ cannot be diagonal for the Jacobi or Gauss-Seidel methods, this formula can be taken as being indicative of the size of $c(A)$ when $M^{-1}N$ is diagonalizable with a well-conditioned matrix of eigenvectors. We therefore have the heuristic inequality, for general A ,

$$c(A) \geq \max_i \frac{|1 - \lambda_i|}{1 - |\lambda_i|}, \quad \lambda_i = \lambda_i(M^{-1}N). \quad (17.14)$$

In practical problems where stationary iteration is used, we would expect $c(A)$ to be of modest size ($O(n)$, say) for two reasons. First, to achieve a reasonable convergence rate, $\rho(M^{-1}N)$ has to be safely less than 1, which implies that the heuristic lower bound (17.14) for $c(A)$ is not too large. Second, even if A is sparse, A^{-1} will usually be full, and so there are unlikely to be zeros on the right-hand side of (17.12). (Such zeros are dangerous because they can make $c(A)$ infinite.)

Note that in (17.13) the only terms that depend on the history of the iteration are $|G^{m+1}e_0|$ and θ_x . In using this bound we can redefine x_0 to be any iterate $\hat{x}_k$, thereby possibly reducing θ_x . This is a circular argument if used to obtain a priori bounds, but it does suggest that the potentially large θ_x term will generally be innocuous. Note that if $x_i = 0$ for some i then θ_x is infinite unless $(\hat{x}_k)_i = 0$ for all k . This difficulty with zero components of x can usually be overcome by redefining

$$\theta_x = \sup_k \max_{1 \leq i \leq n} \frac{(|M| + |N|)|\hat{x}_k|_i}{(|M| + |N|)|x|_i},$$

Table 17.2. *Jacobi method*, $a = 1/2 - 8^{-j}$.

	$\rho(M^{-1}N)$	Iters.	$\text{cond}(A, x)$	$\min_k \phi(\hat{x}_k)$	$\min_k \eta_{A,b}(\hat{x}_k)$
$j = 1$	0.75	90	3.40	2.22e-16	1.27e-16
$j = 2$	0.97	352	4.76	1.78e-15	9.02e-16
$j = 3$	0.996	1974	4.97	1.42e-14	7.12e-15
$j = 4$	1.00	11226	5.00	1.14e-13	5.69e-14
$j = 5$	1.00	55412	5.00	9.10e-13	4.55e-13

for which the above bounds remain valid if θ_x is replaced by $2\theta_x$.

Finally, we note that (17.13) implies

$$\|e_{m+1}\|_\infty \leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \theta_x)c(A) \|A^{-1}(|M| + |N|)|x\|_\infty. \quad (17.15)$$

If $\theta_x c(A) = O(1)$ and $|M| + |N| \leq \alpha|A|$, with $\alpha = O(1)$, this bound is of the form $c_n \text{cond}(A, x)u$ as $m \rightarrow \infty$, and we have componentwise forward stability.

Now we specialize the forward error bound (17.15) to the Jacobi, Gauss–Seidel, and SOR iterations.

17.2.1. Jacobi's Method

For the Jacobi iteration, $M = D = \text{diag}(A)$ and $N = \text{diag}(A) - A$. Hence $|M| + |N| = |M - N| = |A|$, and so (17.15) yields

$$\|e_{m+1}\|_\infty \leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \theta_x)c(A) \|A^{-1}\|_1 \|x\|_\infty. \quad (17.16)$$

If A is an M -matrix then $M^{-1} \geq 0$ and $M^{-1}N \geq 0$, so $c(A) = 1$. Hence in this case we have componentwise forward stability as $m \rightarrow \infty$ if θ_x is suitably bounded.

Woźniakowski [1257, 1978, Ex. 4.1] cites the symmetric positive definite matrix

$$A = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}, \quad 0 < a < \frac{1}{2}, \quad \kappa_2(A) = \frac{1+2a}{1-a}, \quad \rho(M^{-1}N) = 2a,$$

as a matrix for which the Jacobi method can be unstable, in the sense that there exist rounding errors such that no iterate has a relative error bounded by $c_n \kappa_\infty(A)u$. Let us see what our analysis predicts for this example. Straightforward manipulation shows that if $a = 1/2 - \epsilon$ ($\epsilon > 0$), then $c(A) \approx (3\epsilon)^{-1}$, so $c(A) \rightarrow \infty$ as $\epsilon \rightarrow 0$. (The heuristic lower bound (17.14) is approximately $3(2\epsilon)^{-1}$ in this case.) Therefore (17.16) suggests that the Jacobi iteration can be unstable for this matrix. To confirm the instability we applied the Jacobi method to the problem with $x = [1, 1, 1]^T$ and $a = 1/2 - 8^{-j}$, $j = 1:5$. We took a random x_0 with $\|x - x_0\|_2 = 10^{-10}$, and the iteration was terminated when there was no decrease in the norm of the residual for 50 consecutive iterations. Table 17.2 reports the smallest value of $\phi(\hat{x}_k) = \|x - \hat{x}_k\|_\infty / \|x\|_\infty$ over all iterations, for each j ; the number of iterations is shown in the column “Iters.”

Table 17.3. *Jacobi method*, $a = -(1/2 - 8^{-j})$.

	$\rho(M^{-1}N)$	Iters.	$\text{cond}(A, x)$	$\min_k \phi(\hat{x}_k)$	$\min_k \eta_{A,b}(\hat{x}_k)$
$j = 1$	0.75	39	7.00	4.44e-16	5.55e-17
$j = 2$	0.97	273	6.30e1	4.88e-15	7.63e-17
$j = 3$	0.996	1662	5.11e2	4.22e-14	8.24e-17
$j = 4$	1.00	9051	4.09e3	3.41e-13	8.32e-17
$j = 5$	1.00	38294	3.28e4	2.73e-12	8.33e-17

The ratio $\min_k \phi(\hat{x}_k)_{j+1} / \min_k \phi(\hat{x}_k)_j$ takes the values 8.02, 7.98, 8.02, 7.98 for $j = 1:4$, showing excellent agreement with the behaviour predicted by (17.16), since $c(A) \approx 8^j/3$. Moreover, $\theta_x \approx 1$ in these tests and setting $c_n \approx 1$ the bound (17.16) is at most a factor 13.3 larger than the observed error, for each j .

If $-1/2 < a < 0$ then A is an M -matrix and $c(A) = 1$. The bound (17.16) shows that if we set $a = -(1/2 - 8^{-j})$ and repeat the above experiment then the Jacobi method will perform in a componentwise forward stable manner (clearly, $\theta_x \approx 1$ is to be expected). We carried out the modified experiment, obtaining the results shown in Table 17.3. All the $\min_k \phi(\hat{x}_k)_j$ values are less than $\text{cond}(A, x)u$, so the Jacobi iteration is indeed componentwise forward stable in this case. Note that since $\rho(M^{-1}N)$ and $\|M^{-1}N\|_2$ take the same values for a and $-a$, the usual rate of convergence measures cannot distinguish between these two examples.

17.2.2. Successive Overrelaxation

The SOR method can be written in the form $Mx_{k+1} = Nx_k + b$, where

$$M = \frac{1}{\omega}(D + \omega L), \quad N = \frac{1}{\omega}((1 - \omega)D - \omega U),$$

and where $A = D + L + U$, with L and U strictly lower triangular and upper triangular, respectively. The matrix $|M| + |N|$ agrees with $|A|$ everywhere except, possibly, on the diagonal, and the best possible componentwise inequality between these two matrices is

$$|M| + |N| \leq \frac{1 + |1 - \omega|}{\omega} |A| \equiv f(\omega) |A|. \quad (17.17)$$

Note that $f(\omega) = 1$ for $1 \leq \omega \leq 2$, and $f(\omega) \rightarrow \infty$ as $\omega \rightarrow 0$. From (17.15) we have

$$\|e_{m+1}\|_\infty \leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \theta_x) c(A) f(\omega) \|A^{-1}\| \|A\| \|x\|_\infty.$$

If A is an M -matrix and $0 \leq \omega \leq 1$ then $M^{-1} \geq 0$ and $M^{-1}N \geq 0$, so $c(A) = 1$. The Gauss-Seidel method corresponds to $\omega = 1$, and it is interesting to note that for this method the forward error bound has exactly the same form as that for the Jacobi method (though $c(A)$ and θ_x are, of course, different for the two methods).

17.3. Backward Error Analysis

We now turn our attention to bounding the residual vector, $r_k = b - A\hat{x}_k$.

From (17.3) and (17.4) we find that

$$r_{m+1} = AG^{m+1}(x - x_0) + \sum_{k=0}^m AG^k M^{-1} \xi_{m-k}.$$

It is easy to show that $AG^k = H^k A$, where $H := NM^{-1}$ (recall that $G = M^{-1}N$). Therefore

$$r_{m+1} = H^{m+1}r_0 + \sum_{k=0}^m H^k(I - H)\xi_{m-k}. \quad (17.18)$$

Taking norms and using (17.2) gives, similarly to (17.8),

$$\|r_{m+1}\|_\infty \leq \|H^{m+1}r_0\|_\infty + c_n u \sigma (1 + \gamma_x)(\|M\|_\infty + \|N\|_\infty)\|x\|_\infty, \quad (17.19)$$

where

$$\sigma = \left\| \sum_{k=0}^{\infty} |H^k(I - H)| \right\|_\infty.$$

The following bound shows that σ is small if $\|H\|_\infty = q < 1$, with q not too close to 1:

$$\sigma \leq \|I - H\|_\infty \sum_{k=0}^{\infty} \|H\|_\infty^k = \frac{\|I - H\|_\infty}{1 - q}.$$

A potentially much smaller bound can be obtained under the assumption that H is diagonalizable. If $H = XDX^{-1}$, with $D = \text{diag}(\lambda_i)$, then

$$\begin{aligned} \sigma &= \left\| \sum_{k=0}^{\infty} |X(D^k - D^{k+1})X^{-1}| \right\|_\infty \\ &\leq \left\| |X| \left(\sum_{k=0}^{\infty} \text{diag}(|1 - \lambda_i||\lambda_i^k|) \right) |X^{-1}| \right\|_\infty \\ &= \left\| |X| \text{diag} \left(\frac{|1 - \lambda_i|}{1 - |\lambda_i|} \right) |X^{-1}| \right\|_\infty \\ &\leq \kappa_\infty(X) \max_i \frac{|1 - \lambda_i|}{1 - |\lambda_i|}. \end{aligned} \quad (17.20)$$

Note that $\lambda_i = \lambda_i(H) = \lambda_i(NM^{-1}) = \lambda_i(M^{-1}N)$, so we see the reappearance of the term in the heuristic bound (17.14). The bound (17.20) is of modest size if the eigenproblem for H is well conditioned ($\kappa_\infty(X)$ is small) and $\rho(H)$ is not too close to 1. Note that real eigenvalues of H near +1 do not affect the bound for σ , even though they may cause slow convergence.

To summarize, (17.19) shows that, for large m , the normwise backward error $\eta_{A,b}(\hat{x}_m)$ for the ∞ -norm is certainly no larger than

$$c_n u (1 + \gamma_x) \left(\frac{\|M\|_\infty + \|N\|_\infty}{\|A\|_\infty} \right) \sigma.$$

Note that $\|M\|_\infty + \|N\|_\infty \leq 2\|A\|_\infty$ for the Jacobi and Gauss–Seidel methods, and also for the SOR method if $\omega \geq 1$.

A componentwise residual bound can also be obtained, but it does not lead to any identifiable classes of matrix or iteration for which the componentwise relative backward error is small.

To conclude, we return to our numerical examples. For the SOR example at the start of the chapter, $c(A) = O(10^{45})$ and $\sigma = O(10^{30})$, so our error bounds for this problem are all extremely large. In this problem $\max_i |1 - \lambda_i|/(1 - |\lambda_i|) = 3$, where $\lambda_i = \lambda_i(M^{-1}N)$, so (17.14) is very weak; (17.20) is not applicable since $M^{-1}N$ is defective.

For the first numerical example in §17.2.1, Table 17.2 reports the minimum ∞ -norm backward errors $\eta_{A,b}(\hat{x}_k)$. For this problem it is straightforward to show that $\sigma = (1-\epsilon)/\epsilon = 8^j(1-8^{-j})$. The ratios of backward errors for successive values of j are 7.10, 7.89, 7.99, 8.00, so we see excellent agreement with the behaviour predicted by the bounds. Table 17.3 reports the normwise backward errors for the second numerical example in §17.2.1. The backward errors are all less than u , which again is close to what the bounds predict, since it can be shown that $\sigma \leq 5$ for $-1/2 \leq a \leq 0$. In both of the examples of §17.2.1 the componentwise backward error $\omega_{|A|,|b|}(\hat{x}_k) \approx \eta_{A,b}(\hat{x}_k)$, and in our practical experience this behaviour is typical for the Jacobi and SOR iterations.

17.4. Singular Systems

Singular linear systems occur in a variety of applications, including the computation of the stationary distribution vector in a Markov chain [106, 1994], [717, 1983] and the solution of a Neumann boundary value problem by finite difference methods [943, 1976]. Because of the structure and the possibly large dimension of the coefficient matrices in these applications, iterative solution methods are frequently used. An important question is how the rather delicate convergence properties of the iterative methods are affected by rounding errors. In this section we extend the analysis of stationary iteration to singular systems.

17.4.1. Theoretical Background

A useful tool in analysing the behaviour of stationary iteration for a singular system is the *Drazin inverse*. This can be defined, for $A \in \mathbb{R}^{n \times n}$, as the unique matrix A^D such that

$$A^D A A^D = A^D, \quad A A^D = A^D A, \quad \text{and} \quad A^{k+1} A^D = A^k,$$

where $k = \text{index}(A)$. The *index* of A is the smallest nonnegative integer k such that $\text{rank}(A^k) = \text{rank}(A^{k+1})$; it is characterized as the dimension of the largest Jordan block of A with eigenvalue zero. If $\text{index}(A) = 1$ then A^D is also known as the *group inverse* of A and is denoted by $A^\#$. The Drazin inverse is an “equation-solving inverse” precisely when $\text{index}(A) \leq 1$, for then $A A^D A = A$, and so if $Ax = b$ is a consistent system then $A^D b$ is a solution. As we will see, however, the Drazin inverse of the coefficient matrix A itself plays no role in the analysis. The

Drazin inverse can be represented explicitly as follows. If

$$A = P \begin{bmatrix} B & 0 \\ 0 & N \end{bmatrix} P^{-1},$$

where P and B are nonsingular and N has only zero eigenvalues, then

$$A^D = P \begin{bmatrix} B^{-1} & 0 \\ 0 & 0 \end{bmatrix} P^{-1}.$$

Further details of the Drazin inverse can be found in Campbell and Meyer's excellent treatise [199, 1979, Chap. 7].

Let $A \in \mathbb{R}^{n \times n}$ be a singular matrix and consider solving $Ax = b$ by stationary iteration with a splitting $A = M - N$, where M is nonsingular. First, we examine the convergence of the iteration in exact arithmetic. Since any limit point x of the sequence $\{x_k\}$ must satisfy $Mx = Nx + b$, or $Ax = b$, we restrict our attention to consistent linear systems. (For a thorough analysis of stationary iteration for inconsistent systems see Dax [297, 1990].) As in the nonsingular case we have the relation (cf. (17.4)):

$$x_{m+1} = G^{m+1}x_0 + \sum_{i=0}^m G^i M^{-1}b, \quad (17.21)$$

where $G = M^{-1}N$. Since A is singular, G has an eigenvalue 1, so G^m does not tend to zero as $m \rightarrow \infty$, that is, G is not *convergent*. If the iteration is to converge for all x_0 then $\lim_{m \rightarrow \infty} G^m$ must exist. Following Meyer and Plemmons [846, 1977], we call a matrix B for which $\lim_{m \rightarrow \infty} B^m$ exists *semiconvergent*.

We assume from this point on that G is semiconvergent. It is easy to see [106, 1994, Lem. 6.9] that G must have the form

$$G = P \begin{bmatrix} I & 0 \\ 0 & \Gamma \end{bmatrix} P^{-1}, \quad (17.22)$$

where P is nonsingular and $\rho(\Gamma) < 1$. Hence

$$\lim_{m \rightarrow \infty} G^m = P \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix} P^{-1}.$$

To rewrite this limit in terms of G , we note that

$$I - G = P \begin{bmatrix} 0 & 0 \\ 0 & I - \Gamma \end{bmatrix} P^{-1}, \quad (17.23)$$

and, since $I - \Gamma$ is nonsingular,

$$(I - G)^D = P \begin{bmatrix} 0 & 0 \\ 0 & (I - \Gamma)^{-1} \end{bmatrix} P^{-1}. \quad (17.24)$$

Hence

$$\lim_{m \rightarrow \infty} G^m = I - (I - G)^D(I - G). \quad (17.25)$$

To evaluate the limit of the second term in (17.21) we note that, since the system is consistent, $M^{-1}b = M^{-1}Ax = (I - G)x$, and so

$$\begin{aligned}\sum_{i=0}^m G^i M^{-1}b &= \sum_{i=0}^m G^i (I - G)x \\ &= (I - G^{m+1})x \\ &\rightarrow (I - G)^D (I - G)x = (I - G)^D M^{-1}b.\end{aligned}$$

We note in passing that the condition that G is semiconvergent is equivalent to $I - G$ having index 1, in view of (17.23), but that this condition does not imply that $A = M(I - G)$ has index 1.

The conclusion is that if G is semiconvergent, stationary iteration converges to a solution of $Ax = b$ that depends on x_0 :

$$\lim_{m \rightarrow \infty} x_m = (I - (I - G)^D (I - G))x_0 + (I - G)^D M^{-1}b. \quad (17.26)$$

The first term in this limit is in $\text{null}(I - G)$ and the second term is in $\text{range}(I - G)$. To obtain the unique solution in $\text{range}(I - G)$ we should take for x_0 any vector in $\text{range}(I - G)$ ($x_0 = 0$, say).

17.4.2. Forward Error Analysis

We wish to bound $e_{m+1} = x - \hat{x}_{m+1}$, where x is the limit in (17.26) corresponding to the given starting vector x_0 . The analysis proceeds as in the nonsingular case, up to the derivation of equation (17.5):

$$e_{m+1} = G^{m+1}e_0 + \sum_{i=0}^m G^i M^{-1}\xi_{m-i}.$$

As before, the first term, $G^{m+1}e_0$, is negligible for large m , because it is the error after $m + 1$ stages of the exact iteration and this error tends to zero. To obtain a useful bound for the second term, we cannot simply take norms or absolute values, because $\sum_{i=0}^m G^i$ grows unboundedly with m (recall that G has an eigenvalue 1). Our approach is to split the vectors ξ_i according to $\xi_i = \xi_i^{(1)} + \xi_i^{(2)}$, where $M^{-1}\xi_i^{(1)} \in \text{range}(I - G)$ and $M^{-1}\xi_i^{(2)} \in \text{null}(I - G)$; this is a well-defined splitting because $\text{range}(I - G)$ and $\text{null}(I - G)$ are complementary subspaces (since $\text{index}(I - G) = 1$, or equivalently, G is semiconvergent). Using the properties of the splitting the error can be written as

$$\begin{aligned}e_{m+1} &= G^{m+1}e_0 + \sum_{i=0}^m G^i M^{-1}\xi_{m-i}^{(1)} + \sum_{i=0}^m G^i M^{-1}\xi_{m-i}^{(2)} \\ &= G^{m+1}e_0 + \sum_{i=0}^m G^i M^{-1}\xi_{m-i}^{(1)} + M^{-1} \sum_{i=0}^m \xi_{m-i}^{(2)}.\end{aligned}$$

We achieve the required splitting for ξ_i via the formulae

$$\xi_i^{(1)} = MEM^{-1}\xi_i, \quad \xi_i^{(2)} = M(I - E)M^{-1}\xi_i,$$

where

$$E = (I - G)^D(I - G).$$

Hence the error can be written as

$$e_{m+1} = G^{m+1}e_0 + \sum_{i=0}^m G^i E M^{-1} \xi_{m-i} + (I - E)M^{-1} \sum_{i=0}^m \xi_{m-i}. \quad (17.27)$$

Clearly, as $m \rightarrow \infty$ the final term in this expression can become unbounded, but since it grows only linearly in the number of iterations it is unlikely to have a significant effect in applications where stationary iteration converges quickly enough to be of practical use.

Now we bound the term

$$S_m = \sum_{i=0}^m G^i E M^{-1} \xi_{m-i}. \quad (17.28)$$

Using inequality (17.2) and the definition of γ_x in (17.7) and θ_x in (17.9), we have

$$\begin{aligned} \|S_m\|_\infty &\leq c_n u(1 + \gamma_x)(\|M\|_\infty + \|N\|_\infty)\|x\|_\infty \sum_{i=0}^m \|G^i E M^{-1}\|_\infty, \\ |S_m| &\leq c_n u(1 + \theta_x) \sum_{i=0}^m |G^i E M^{-1}|(|M| + |N|)|x|. \end{aligned} \quad (17.29)$$

The convergence of the two infinite sums is assured by the result of Problem 17.1, since by (17.22)–(17.24),

$$\begin{aligned} G^i E &= G^i (I - G)^D(I - G) \\ &= P \begin{bmatrix} I & 0 \\ 0 & \Gamma^i \end{bmatrix} P^{-1} \cdot P \begin{bmatrix} 0 & 0 \\ 0 & (I - \Gamma)^{-1} \end{bmatrix} P^{-1} \cdot P \begin{bmatrix} 0 & 0 \\ 0 & (I - \Gamma) \end{bmatrix} P^{-1} \\ &= P \begin{bmatrix} 0 & 0 \\ 0 & \Gamma^i \end{bmatrix} P^{-1} = (GE)^i \quad (i \geq 1), \end{aligned} \quad (17.30)$$

where $\rho(\Gamma) < 1$.

We conclude that we have the normwise error bound

$$\begin{aligned} \|e_{m+1}\|_\infty &\leq \|G^{m+1}e_0\|_\infty + c_n u(1 + \gamma_x)(\|M\|_\infty + \|N\|_\infty)\|x\|_\infty \\ &\quad \times \left\{ \sum_{i=0}^{\infty} \|G^i E M^{-1}\|_\infty + (m+1)\|(I - E)M^{-1}\|_\infty \right\}. \end{aligned} \quad (17.31)$$

On setting $E = I$ we recover the result (17.8) for the nonsingular case. If we assume that Γ is diagonal, so that P in (17.30) is a matrix of eigenvectors of G , then

$$\sum_{i=0}^{\infty} \|G^i E M^{-1}\|_\infty \leq \kappa_\infty(P) \|M^{-1}\|_\infty \frac{1}{1 - \rho(\Gamma)}.$$

This bound shows that a small forward error is guaranteed if $\kappa(P)\|M^{-1}\| = O(1)$ and the second largest eigenvalue of G is not too close to 1. (It is this subdominant eigenvalue that determines the asymptotic rate of convergence of the iteration.)

Turning to the componentwise case, we see from (17.24) and (17.30) that

$$\sum_{i=0}^{\infty} G^i E = (I - G)^D.$$

Because of the form of the sum in (17.29), this prompts us to define the scalar $c(A) \geq 1$ by

$$c(A) = \min \left\{ \epsilon : \sum_{i=0}^{\infty} |G^i E M^{-1}| \leq \epsilon |(I - G)^D M^{-1}| \right\},$$

in terms of which we have the componentwise error bound

$$\begin{aligned} |e_{m+1}| \leq & |G^{m+1} e_0| + c_n u(1 + \theta_x) \{ c(A) |(I - G)^D M^{-1}| \\ & + (m + 1) |(I - E) M^{-1}| \} (|M| + |N|) |x|. \end{aligned} \quad (17.32)$$

Again, as a special case we have the result for nonsingular A , (17.13).

To what should we compare this bound? A perturbation result for $Ax = b$ is given in [619, 1993] that projects the perturbations of A and b into $\text{range}(I - G)$ and thus can be thought of as gauging the effect of perturbations to the “nonsingular part of the system”. For perturbations of order ϵ it gives an expression

$$\Delta x = (I - G)^D M^{-1} (\Delta b - \Delta A x) + O(\epsilon^2).$$

Hence we can deduce conditions on a stationary iterative method that ensure it is componentwise forward stable, in the sense of yielding a solution whose error is no larger than the uncertainty in x caused by rounding the data. The constants θ_x and $c(A)$ should be bounded by d_n , where d_n denotes a slowly growing function of n ; the inequality $|M| + |N| \leq d'_n |A|$ should hold, as it does for the Jacobi method and for the SOR method when $\omega \in [\beta, 2]$, where β is positive and not too close to zero; and the “exact error” $G^{m+1} e_0$ must decay quickly enough to ensure that the term $(m + 1) |(I - E) M^{-1}|$ does not grow too large before the iteration is terminated.

Numerical results given in [619, 1993] show that the analysis can correctly predict forward and backward stability, and that for certain problems linear growth of the component of the error in $\text{null}(A)$ can indeed cause an otherwise convergent iteration to diverge, even when starting very close to a solution.

17.5. Stopping an Iterative Method

What convergence test should be used to stop an iterative linear equation solver? In this section we explain how backward errors and condition numbers help answer this question. Note first that most iterative methods for solving $Ax = b$ compute all or part of a matrix–vector product $w = Av$ on each iteration, and in floating point arithmetic we have

$$\hat{w} = (A + \Delta A)v, \quad |\Delta A| \leq \gamma_m |A|,$$

where m is the maximum number of nonzeros per row of A . The method therefore cannot distinguish between A and $A + \Delta A$ where $|\Delta A| \leq \gamma_m |A|$, and so there is no point in trying to achieve a componentwise relative backward error less than γ_m . Of course, instability of a method (or simply lack of convergence) may pose further restrictions on how small a backward error can be achieved.

It is worth keeping in mind throughout this discussion that in practical applications accuracy and stability requirements are often quite modest because of large errors or uncertainties in the data, or because the iteration is an “inner step” of some “outer” iterative process. Indeed, one of the advantages of iterative methods is that the slacker the convergence tolerance the less computational effort is required, though the relation between tolerance and work depends very much on the method.

Natural stopping criteria for an iterative method are that some measure of backward error or forward error does not exceed a tolerance, ϵ . We will assume that the residual $r = b - Ay$ is available for each iterate y , and that norms of y , r , and A can be computed or estimated. If r is not computed directly, but is recurred by the method, as, for example, in the conjugate gradient method, then the norm of the computed residual may differ from that of the true residual by several orders of magnitude; clearly, this affects the way that the stopping tests are interpreted.

From Theorem 7.1 we have the following equivalences, for any subordinate matrix norm:

$$\|r\| \leq \epsilon \|b\| \iff Ay = b + \Delta b, \quad \|\Delta b\| \leq \epsilon \|b\|, \quad (17.33a)$$

$$\|r\| \leq \epsilon \|A\| \|y\| \iff (A + \Delta A)y = b, \quad \|\Delta A\| \leq \epsilon \|A\|, \quad (17.33b)$$

$$\begin{aligned} \|r\| \leq \epsilon (\|A\| \|y\| + \|b\|) &\iff (A + \Delta A)y = b + \Delta b, \\ &\|\Delta A\| \leq \epsilon \|A\|, \quad \|\Delta b\| \leq \epsilon \|b\|. \end{aligned} \quad (17.33c)$$

These inequalities remain true with norms replaced by absolute values (Theorem 7.3), but then to evaluate (17.33b) and (17.33c) a matrix–vector product $|A||y|$ must be computed, which is a nontrivial expense in an iterative method.

Of these tests, (17.33c) is preferred in general, assuming it is acceptable to perturb both A and b . Note the importance of including both $\|A\|$ and $\|y\|$ in the test on $\|r\|$; a test $\|r\| \leq \epsilon \|A\|$, though scale independent, does not bound any relative backward error. Test (17.33a) is commonly used in existing codes, but may be very stringent, and possibly unsatisfiable. To see why, note that the residual of the rounded exact solution $fl(x) = x + \Delta x$, $|\Delta x| \leq u|x|$, satisfies, for any absolute norm,

$$\|r\| = \|b - A(x + \Delta x)\| \leq u \|A\| \|x\|,$$

and

$$\frac{\|A\| \|x\|}{\kappa(A)} \leq \|b\| \leq \|A\| \|x\|.$$

If A is ill conditioned and x is a large-normed solution (that is, $\|x\| \approx \|A^{-1}\| \|b\|$), so that $\|b\|$ is close to its lower bound, then (17.33a) is much harder to satisfy than (17.33c).

If the forward error is to be bounded, then, for a nonsingular problem, tests can be derived involving the residual and A^{-1} : the equality $x - y = A^{-1}r$ leads to normwise and componentwise forward error bounds, such as $\|x - y\|/\|y\| \leq \|A^{-1}\| \|r\|/\|y\|$ (cf. Problem 7.2). Since these bounds involve A^{-1} , they are nontrivial to compute. Some iterative methods automatically produce estimates of the extremal eigenvalues, and hence of $\kappa_2(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$. For large, sparse symmetric positive definite matrices $\|A^{-1}\|_2$ can be cheaply estimated using the Lanczos method. Another possibility is to use condition estimation techniques (Chapter 15).

The discussion in this section has inevitably been very general. Other considerations in practice include detecting nonconvergence of a method (due to rounding errors or otherwise), adapting the tests to accommodate a preconditioner (the residual r provided by the method may now be that for the preconditioned system), and using a norm that corresponds to a quantity being minimized by the method (a norm that may be nontrivial to compute). Moreover, it may be possible to design a stopping criterion that relates to the error or residual for the original problem that lead to the linear system; how to do this for the conjugate gradient method applied to linear systems produced by the finite element method is explained by Arioli [34, 2000].

17.6. Notes and References

The Gauss–Seidel method was chosen by Wilkinson [1224, 1948] as an “example of coding” for the ACE. Speaking of his experience at that time at the National Physical Laboratory, he explained that “In general, direct methods have been used on those equations which did not yield readily to relaxation, and hence those solved by direct methods have nearly always been of an ill-conditioned type”.

Stationary iterative methods are relatively easy to program, although there are many issues to consider when complicated data structures or parallel machines are used. A good source of straightforward C, Fortran, and MATLAB implementations of the Jacobi, Gauss–Seidel, and SOR methods, and other nonstationary iterative methods, is the book *Templates for the Solution of Linear Systems* [80, 1994]; the software is available from netlib. The book contains theoretical discussions of the methods, together with practical advice on topics such as data structures and the choice of a stopping criterion. The choice of stopping criteria for iterative methods is also discussed by Arioli, Duff, and Ruiz [35, 1992].

Recent textbooks on iterative methods include those by Axelsson [42, 1994] and Greenbaum [522, 1997]. Numerical linear algebra textbooks that have good coverage of iterative methods include Demmel [317, 1997] and Trefethen and Bau [1156, 1997].

Problems

17.1. Show that if $B \in \mathbb{R}^{n \times n}$ and $\rho(B) < 1$, then the series $\sum_{k=0}^{\infty} |B^k|$ and $\sum_{k=0}^{\infty} \|B^k\|$ are both convergent, where $\|\cdot\|$ is any norm.

Chapter 18

Matrix Powers

Unfortunately, the roundoff errors in the m th power of a matrix, say B^m , are usually small relative to $\|B\|^m$ rather than $\|B^m\|$.

— CLEVE B. MOLER and CHARLES F. VAN LOAN,
Nineteen Dubious Ways to Compute the Exponential of a Matrix (1978)

*It is the size of the hump that matters:
the behavior of $\|p(A\Delta t)^n\| = \|p(A\Delta t)^{t/\Delta t}\|$ for small but nonzero t .
The eigenvalues and the norm, by contrast, give sharp information
only about the limits $t \rightarrow \infty$ or $t \rightarrow 0$.*

— DESMOND J. HIGHAM and LLOYD N. TREFETHEN,
Stiffness of ODEs (1993)

*Many people will go through life powering matrices
and never see anything as dramatic as $J_2(0.99)^k$.*¹⁶

— G. W. STEWART, *Matrix Algorithms. Volume II: Eigensystems* (2001)

¹⁶ $J_2(0.99) = \begin{bmatrix} 0.99 & 1 \\ 0 & 0.99 \end{bmatrix}$.

Powers of matrices occur in many areas of numerical analysis. One approach to proving convergence of multistep methods for solving differential equations is to show that a certain parameter-dependent matrix is uniformly “power bounded” [537, 1991, §V.7], [974, 1992]. Stationary iterative methods for solving linear equations converge precisely when the powers of the iteration matrix converge to zero. And the power method for computing the largest eigenvalue of a matrix computes the action of powers of the matrix on a vector. It is therefore important to understand the behaviour of matrix powers, in both exact and finite precision arithmetic.

It is well known that the powers A^k of $A \in \mathbb{C}^{n \times n}$ tend to zero as $k \rightarrow \infty$ if $\rho(A) < 1$, where ρ is the spectral radius. However, this simple statement does not tell the whole story. Figure 18.1 plots the 2-norms of the first 30 powers of a certain 3×3 matrix with $\rho(A) = 0.75$. The powers do eventually decay, but initially they grow rapidly. (In this and other similar plots, k on the x -axis is plotted against $\|fl(A^k)\|_2$ on the y -axis, and the norm values are joined by straight lines for plotting purposes.) Figure 18.2 plots the 2-norms of the first 250 powers of a 14×14 nilpotent matrix C_{14} discussed by Trefethen and Trummer [1158, 1987] (see §18.2 for details). The plot illustrates the statement of these authors that the matrix is not power bounded in floating point arithmetic, even though its 14th power should be zero.

These examples suggest two important questions.

- For a matrix with $\rho(A) < 1$, how does the sequence $\{\|A^k\|\}$ behave? In particular, what is the size of the “hump” $\max_k \|A^k\|$?
- What conditions on A ensure that $fl(A^k) \rightarrow 0$ as $k \rightarrow \infty$?

We examine these questions in the next two sections.

18.1. Matrix Powers in Exact Arithmetic

In exact arithmetic the limiting behaviour of the powers of $A \in \mathbb{C}^{n \times n}$ is determined by A 's eigenvalues. As already noted, if $\rho(A) < 1$ then $A^k \rightarrow 0$ as $k \rightarrow \infty$; if $\rho(A) > 1$, $\|A^k\| \rightarrow \infty$ as $k \rightarrow \infty$. In the remaining case of $\rho(A) = 1$, $\|A^k\| \rightarrow \infty$ if A has a defective eigenvalue λ such that $|\lambda| = 1$; A^k does not converge if A has a nondefective eigenvalue $\lambda \neq 1$ such that $|\lambda| = 1$ (although the norms of the powers may converge); otherwise, the only eigenvalue of modulus 1 is the nondefective eigenvalue 1 and A^k converges to a nonzero matrix. These statements are easily proved using the Jordan canonical form

$$A = XJX^{-1} \in \mathbb{C}^{n \times n}, \quad (18.1a)$$

where X is nonsingular and

$$J = \text{diag}(J_1, J_2, \dots, J_s), \quad J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \ddots & \ddots & \\ & & \lambda_i & 1 \\ & & & \lambda_i \end{bmatrix} \in \mathbb{C}^{n_i \times n_i}, \quad (18.1b)$$

where $n_1 + n_2 + \dots + n_s = n$. We will call a matrix for which $A^k \rightarrow 0$ as $k \rightarrow \infty$ (or equivalently, $\rho(A) < 1$) a *convergent matrix*.

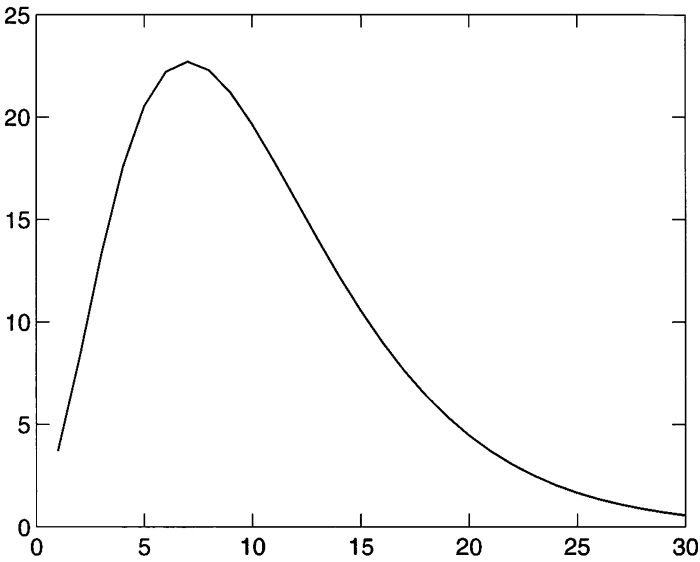


Figure 18.1. A typical hump for a convergent, nonnormal matrix.

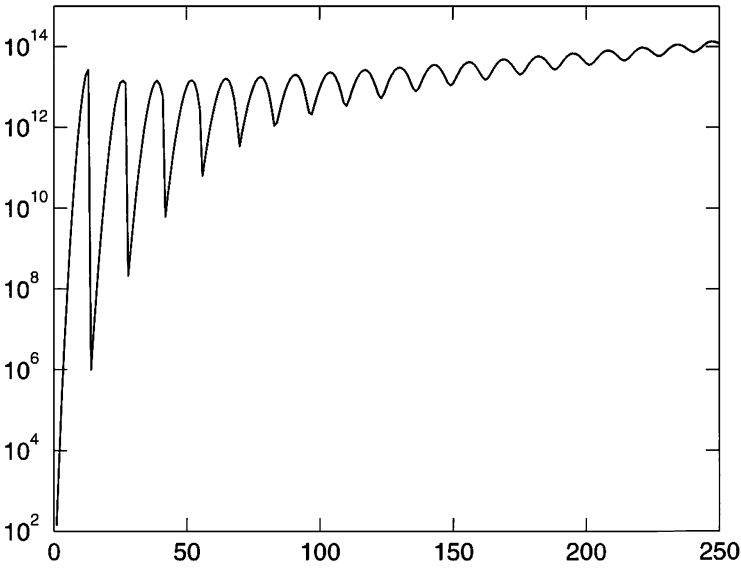


Figure 18.2. Diverging powers of a nilpotent matrix, C_{14} .

The norm of a convergent matrix can be arbitrarily large, as is shown trivially by the scaled Jordan block

$$J = \lambda \begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix}, \quad (18.2)$$

with $|\lambda| < 1$ and $\alpha \gg 1$. While the spectral radius determines the asymptotic rate of growth of matrix powers, the norm influences the initial behaviour of the powers. The interesting result that $\rho(A) = \lim_{k \rightarrow \infty} \|A^k\|^{1/k}$ for any norm (see Horn and Johnson [636, 1985, p. 299], for example) confirms the asymptotic role of the spectral radius. This formula for $\rho(A)$ has actually been considered as a means for computing it; see Wilkinson [1233, 1965, pp. 615–617] and Friedland [445, 1991].

An important quantity is the “hump” $\max_k \|A^k\|$, which can be arbitrarily large for a convergent matrix. Figure 18.1 shows the hump for the 3×3 upper triangular matrix with diagonal entries $3/4$ and off-diagonal entries 2; this matrix has 2-norm 3.57. The shape of the plot is typical of that for a convergent matrix with norm bigger than 1. Note that if A is normal (so that in (18.1a) J is diagonal and X can be taken to be unitary) we have $\|A^k\|_2 = \|\text{diag}(\lambda_i^k)\|_2 = \|A\|_2^k = \rho(A)^k$, so the problem of bounding $\|A^k\|$ is of interest only for nonnormal matrices. The hump phenomenon arises in various areas of numerical analysis. For example, it is discussed for matrix powers in the context of stiff differential equations by D. J. Higham and Trefethen [577, 1993], and by Moler and Van Loan [868, 1978] for the matrix exponential e^{At} with $t \rightarrow \infty$.

More insight into the behaviour of matrix powers can be gained by considering the 2×2 matrix (18.2) with $0 \leq \lambda < 1$ and $\alpha > 0$. We have

$$J^k = \lambda^k \begin{bmatrix} 1 & k\alpha \\ 0 & 1 \end{bmatrix}$$

and

$$\frac{\|J^{k+1}\|_\infty}{\|J^k\|_\infty} = \lambda \frac{1 + (k+1)\alpha}{1 + k\alpha} \approx \lambda \left(1 + \frac{1}{k}\right) \quad \text{for large } k. \quad (18.3)$$

Hence

$$\|J^{k+1}\|_\infty < \|J^k\|_\infty \quad \text{if } k > \frac{\lambda(1+\alpha) - 1}{(1-\lambda)\alpha}.$$

It follows that the norms of the powers can increase for arbitrarily many steps until they ultimately decrease. Moreover, because k^{-1} tends to zero quite slowly as $k \rightarrow \infty$, the rate of convergence of $\|J^k\|_\infty$ to zero can be much slower than the convergence of λ^k to zero (see (18.3)) when λ is close to 1. In other words, nontrivial Jordan blocks retard the convergence to zero.

For this 2×2 matrix, the hump $\max_k \|J^k\|_\infty$ is easily shown to be approximately

$$\rho^{1/\alpha - 1/\log_e \rho} \frac{\alpha}{\log_e \rho},$$

where $\rho = \lambda^{-1} > 1$, this value being attained for $k \approx (\log_e \rho)^{-1} - \alpha^{-1}$. Figure 18.3 displays the norms of the first 400 powers of the matrices with $\lambda = 0.99$ and $\alpha = 0, 0.001, 0.01, 0.1, 1$. The size and location of the hump are complicated expressions even in this simple case. When we generalize to direct sums of larger Jordan

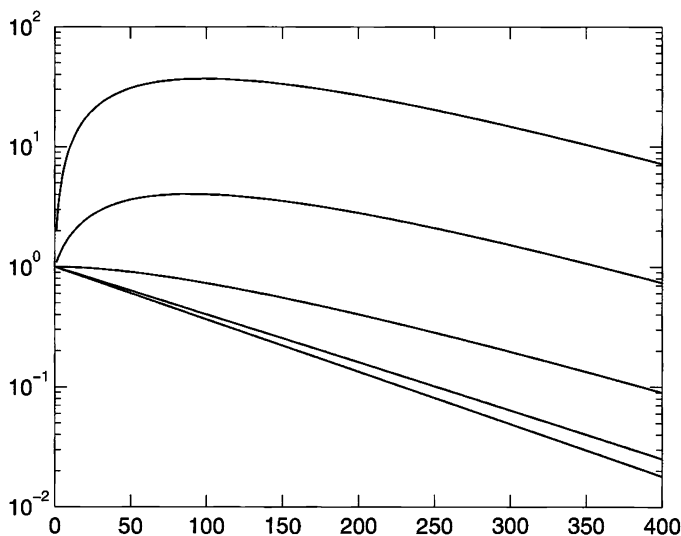


Figure 18.3. Infinity norms of powers of 2×2 matrix J in (18.2), for $\lambda = 0.99$ and $\alpha = 0$ (bottom line) and $\alpha = 10^{-k}$, $k = 0:3$.

blocks and incorporate a similarity transformation, giving (18.1a), the qualitative behaviour of the powers becomes too difficult to describe precisely.

In the rest of this section we briefly survey bounds for $\|A^k\|$, where $\|\cdot\|$ is a consistent norm. First, however, we comment on the condition number $\kappa(X) = \|X\| \|X^{-1}\|$ that appears in various bounds in this chapter. The matrix X in the Jordan form (18.1a) is by no means unique [453, 1959, pp. 220–221], [506, 1976]: if A has distinct eigenvalues (hence J is diagonal) then X can be replaced by XD , for any nonsingular diagonal D , while if A has repeated eigenvalues then X can be replaced by XT , where T is a block matrix with block structure conformal with that of J and which contains some arbitrary upper trapezoidal Toeplitz blocks. We adopt the convention that $\kappa(X)$ denotes the minimum possible value of $\kappa(X)$ over all possible choices of X . This minimum value is not known for general A , and the best we can hope is to obtain a good estimate of it. However, if A has distinct eigenvalues then the results in Theorems 7.5 and 7.7 on diagonal scalings are applicable and enable us to determine (an approximation to) the minimal condition number. Explicit expressions can be given for the minimal 2-norm condition number for $n = 2$; see Young [1270, 1971, §3.8].

A trivial bound is $\|A^k\| \leq \|A\|^k$. A sharper bound can be derived in terms of the *numerical radius*

$$r(A) = \max \left\{ \left| \frac{z^* A z}{z^* z} \right| : 0 \neq z \in \mathbb{C}^n \right\},$$

which is the point of largest modulus in the field of values of A . It is not hard to show that $\|A\|_2/2 \leq r(A) \leq \|A\|_2$ [636, 1985, p. 331]. The (nontrivial) inequality $r(A^k) \leq r(A)^k$ [636, 1985, p. 333] leads to the bound

$$\|A^k\|_2 \leq 2r(A)^k.$$

If A is diagonalizable then, from (18.1a), we have the bound

$$\|A^k\|_p \leq \kappa_p(X)\rho(A)^k, \quad (18.4)$$

for any p -norm. (Since $\rho(A) \leq \|A\|$ for any consistent norm (Problem 6.7), we also have the lower bound $\rho(A)^k \leq \|A^k\|_p$.) This bound is unsatisfactory for two reasons. First, by choosing A to have well-conditioned large eigenvalues and ill-conditioned small eigenvalues we can make the bound arbitrarily pessimistic (see Problem 18.1). Second, it models norms of powers of convergent matrices as monotonically decreasing sequences, which is qualitatively incorrect if there is a large hump.

The Jordan canonical form can also be used to bound the norms of the powers of a defective matrix. If XJX^{-1} is the Jordan canonical form of $\delta^{-1}A$ then

$$\|A^k\|_p \leq \kappa_p(X)(\rho(A) + \delta)^k, \quad (18.5)$$

for all $\delta > 0$. This is a special case of a result of Ostrowski [909, 1973, Thm. 20.1] and the proof is straightforward: We can write $\delta^{-1}A = X(\delta^{-1}D + M)X^{-1}$, where $D = \text{diag}(\lambda_i)$ and M is the off-diagonal part of the Jordan form. Then $A = X(D + \delta M)X^{-1}$, and (18.5) follows by powering and taking norms. An alternative way of writing this bound is

$$\|A^k\|_p \leq \kappa_p(X)\kappa_p(D)(\rho(A) + \delta)^k,$$

where $A = XJX^{-1}$ and $D = \text{diag}(\delta^{n-1}, \delta^{n-2}, \dots, 1)$. Note that this is not the same X as in (18.5): multiplying A by a scalar changes $\kappa(X)$ when A is not diagonalizable. Both bounds suffer from the same problems as the bound (18.4) for diagonalizable matrices.

Another bound in terms of the Jordan canonical form (18.1) of A is given by Gautschi [469, 1953]. For convergent matrices, it can be written in the form

$$\|A^k\|_F \leq c k^{p-1} \rho(A)^k, \quad (18.6)$$

where $p = \max\{n_i : \lambda_i \neq 0\}$ and c is a constant depending only on A (c is not defined explicitly in [469, 1953]). The factor k^{p-1} makes this bound somewhat more effective at predicting the shapes of the actual curve than (18.5), but again c can be unsuitably large.

Since the norm estimation problem is trivial for normal matrices, it is natural to look for bounds that involve a measure of nonnormality. Consider the Schur decomposition $Q^*AQ = D + N$, where N is strictly upper triangular, and let S represent the set of all such N . The nonnormality of A can be measured by Henrici's *departure from normality* [563, 1962]

$$\Delta(A, \|\cdot\|) \equiv \Delta(A) = \min_{N \in S} \|N\|.$$

For the Frobenius norm, Henrici shows that $\|N\|_F$ is independent of the particular Schur form and that

$$\Delta_F(A) = \left(\|A\|_F^2 - \sum_i |\lambda_i|^2 \right)^{1/2} \leq \left(\frac{n^3 - n}{12} \right)^{1/4} \|A^*A - AA^*\|_F^{1/2}.$$

László [770, 1994] has recently shown that $\Delta_F(A)$ is within a constant factor of the distance from A to the nearest normal matrix:

$$\frac{\Delta_F(A)}{\sqrt{n}} \leq \nu(A) \leq \Delta_F(A),$$

where $\nu(A) = \min\{\|E\|_F : A + E \text{ is normal}\}$. Henrici uses the departure from normality to derive the 2-norm bounds

$$\|A^k\|_2 \leq \begin{cases} \sum_{i=0}^{n-1} \binom{k}{i} \rho(A)^{k-i} \Delta_2(A)^i, & \rho(A) > 0, \\ \Delta_2(A)^k, & \rho(A) = 0 \text{ and } k < n. \end{cases} \quad (18.7)$$

Empirical evidence suggests that the first bound in (18.7) can be very pessimistic. However, for normal matrices both the bounds are equalities.

Another bound involving nonnormality is given by Golub and Van Loan [509, 1996, Lem. 7.3.2]. They show that, in the above notation,

$$\|A^k\|_2 \leq (1 + \theta)^{n-1} \left(\rho(A) + \frac{\Delta_F(A)}{1 + \theta} \right)^k$$

for any $\theta \geq 0$. This bound is an analogue of (18.5) with the Schur form replacing the Jordan form. Again, there is equality when A is normal (if we set $\theta = 0$).

To compare bounds based on the Schur form with ones based on the Jordan form we need to compare $\Delta(A)$ with $\kappa(X)$. If A is diagonalizable then [795, 1969, Thm. 4]

$$\kappa_2(X) \geq \left(1 + \frac{\Delta_F(A)^2}{\|A\|_F^2} \right)^{1/2};$$

it can be shown by a 2×2 example that $\min_X \kappa_2(X)$ can exceed $\Delta_F(A)/\|A\|_F$ by an arbitrary factor [224, 1993, §4.2.7], [208, 1996, §9.1.1].

Another tool that can be used to bound the norms of powers is the pseudospectrum of a matrix, popularized by Trefethen [389], [1154, 1992], [1155, 1999]. The ϵ -pseudospectrum of $A \in \mathbb{C}^{n \times n}$ is defined, for a given $\epsilon > 0$, to be the set

$$\Lambda_\epsilon(A) = \{z : z \text{ is an eigenvalue of } A + E \text{ for some } E \text{ with } \|E\|_2 \leq \epsilon\},$$

and it can also be represented, in terms of the resolvent $(zI - A)^{-1}$, as

$$\Lambda_\epsilon(A) = \{z : \|(zI - A)^{-1}\|_2 \geq \epsilon^{-1}\}.$$

As Trefethen notes [1154, 1992], by using the Cauchy integral representation of A^k (which involves a contour integral of the resolvent) one can show that

$$\|A^k\|_2 \leq \epsilon^{-1} \rho_\epsilon(A)^{k+1}, \quad (18.8)$$

where the ϵ -pseudospectral radius

$$\rho_\epsilon(A) = \max\{|z| : z \in \Lambda_\epsilon(A)\}. \quad (18.9)$$

This bound is very similar in flavour to (18.5). The difficulty is transferred from estimating $\kappa(X)$ to choosing ϵ and estimating $\rho_\epsilon(A)$.

Bai, Demmel, and Gu [48, 1997] consider A with $\rho(A) < 1$ and manipulate the Cauchy integral representation of A^k in a slightly different way from Trefethen to produce a bound in terms of the distance to the nearest unstable matrix,

$$\begin{aligned} d(A) &= \min\{ \|\Delta A\|_2 : A + \Delta A \text{ has an eigenvalue of modulus } 1 \} \\ &= \min\{ \epsilon : \rho_\epsilon(A) = 1 \}. \end{aligned}$$

Their bound is

$$\|A^k\|_2 \leq \begin{cases} m\alpha_m(1 - d(A))^m, & m > (1 - d(A))/d(A), \\ 1/d(A), & m \leq (1 - d(A))/d(A), \end{cases}$$

where $e \leq \alpha_m := (1 + 1/m)^{m+1} \leq 4$. Note that $d(A) \leq 1$ when $\rho(A) < 1$, as is easily seen from the Schur decomposition. The distance $d(A)$ is not easy to compute. One approach is a bisection technique of Byers [194, 1988].

Finally, we mention that the Kreiss matrix theorem provides a good estimate of $\sup_{k \geq 0} \|A^k\|$ for a general $A \in \mathbb{C}^{n \times n}$, albeit in terms of an expression that involves the resolvent and is not easy to compute:

$$\phi(A) \leq \sup_{k \geq 0} \|A^k\|_2 \leq ne\phi(A),$$

where $\phi(A) = \sup\{ (|z| - 1)\|(zI - A)^{-1}\|_2 : |z| > 1 \}$ and $e = \exp(1)$. Details and references are given by Wegert and Trefethen [1212, 1994].

18.2. Bounds for Finite Precision Arithmetic

The formulae $A \cdot A^k$ or $A^k \cdot A$ can be implemented in several ways, corresponding to different loop orderings in each individual product, but as long as each product is formed using the standard formula $(AB)_{ij} = \sum_k a_{ik}b_{kj}$, all these variations satisfy the same rounding error bounds. We do not analyse here the use of the binary powering technique, where, for example, A^9 is formed as $A((A^2)^2)^2$; alternate multiplication on the left and right ($fl(A^k) = fl(Afl(A^{k-2})A)$); or fast matrix multiplication techniques such as Strassen's method. None of these methods is equivalent to repeated multiplication in finite precision arithmetic.

We suppose, without loss of generality, that the columns of A^m are computed one at a time, the j th as $fl(A(A(\dots(Ae_j)\dots)))$, where e_j is the j th unit vector. The error analysis for matrix-vector multiplication shows that the j th computed column of A^m satisfies

$$fl(A^m e_j) = (A + \Delta A_1)(A + \Delta A_2) \dots (A + \Delta A_m)e_j, \quad (18.10)$$

where, for both real and complex matrices, we have (Problem 3.7)

$$|\Delta A_i| \leq \gamma_{n+2}|A|. \quad (18.11)$$

It follows that

$$|fl(A^m e_j)| \leq (1 + \gamma_{n+2})^m |A|^m e_j,$$

and so a sufficient condition for convergence of the computed powers is that

$$\rho(|A|) < \frac{1}{1 + \gamma_{n+2}}. \quad (18.12)$$

This result is useful in certain special cases: $\rho(|A|) = \rho(A)$ if A is triangular or has a checkerboard sign pattern (since then $|A| = DAD^{-1}$ where $D = \text{diag}(\pm 1)$); if A is normal then $\rho(|A|) \leq \sqrt{n}\rho(A)$ (this bound being attained for a Hadamard matrix); and in Markov processes, where the a_{ij} are transition probabilities, $|A| = A$. However, in general $\rho(|A|)$ can exceed $\rho(A)$ by an arbitrary factor (see Problem 18.2).

To obtain sharper and more informative results it is necessary to use more information about the matrix. In the following theorem we give a sufficient condition, based on the Jordan canonical form, for the computed powers of a matrix to converge to zero. Although the Jordan form is usually eschewed by numerical analysts because of its sensitivity to perturbations, it is convenient to work with in this application and leads to an informative result.

Theorem 18.1 (Higham and Knight). *Let $A \in \mathbb{C}^{n \times n}$ with the Jordan form (18.1) have spectral radius $\rho(A) < 1$. A sufficient condition for $fl(A^m) \rightarrow 0$ as $m \rightarrow \infty$ is*

$$4t\gamma_{n+2}\kappa_\infty(X)\|A\|_\infty < (1 - \rho(A))^t, \quad (18.13)$$

where $t = \max_i n_i$.

Proof. It is easy to see that if we can find a nonsingular matrix S such that

$$\|S^{-1}AS\|_\infty + \kappa_\infty(S)\|\Delta A_i\|_\infty < 1 \quad (18.14)$$

for all i , then the product

$$(A + \Delta A_1) \dots (A + \Delta A_m) = S(S^{-1}AS + S^{-1}\Delta A_1S) \dots (S^{-1}AS + S^{-1}\Delta A_mS)S^{-1}$$

tends to 0 as $m \rightarrow \infty$. In the rest of the proof we construct such a matrix S for the ΔA_i in (18.10).

Let $P(\epsilon) = \text{diag}(P_1(\epsilon), \dots, P_s(\epsilon))$ where $0 < \epsilon < 1 - \rho(A)$ and

$$P_i(\epsilon) = \text{diag}((1 - |\lambda_i| - \epsilon)^{1-n_i}, (1 - |\lambda_i| - \epsilon)^{2-n_i}, \dots, 1) \in \mathbb{R}^{n_i \times n_i}.$$

Now consider the matrix $P(\epsilon)^{-1}JP(\epsilon)$. Its i th diagonal block is of the form $\lambda_i I + (1 - |\lambda_i| - \epsilon)N$, where the only nonzeros in N are 1s on the first superdiagonal, and so

$$\|P(\epsilon)^{-1}X^{-1}AXP(\epsilon)\|_\infty = \|P(\epsilon)^{-1}JP(\epsilon)\|_\infty \leq \max_i (|\lambda_i| + 1 - |\lambda_i| - \epsilon) = 1 - \epsilon.$$

Defining $S = XP(\epsilon)$, we have $\|S^{-1}AS\|_\infty \leq 1 - \epsilon$ and

$$\kappa_\infty(S) \leq \kappa_\infty(P(\epsilon))\kappa_\infty(X) \leq (1 - \rho(A) - \epsilon)^{1-t}\kappa_\infty(X). \quad (18.15)$$

Now we set $\epsilon = \theta(1 - \rho(A))$ where $0 < \theta < 1$ and we determine θ so that (18.14) is satisfied, that is, so that $\kappa_\infty(S)\|\Delta A_i\|_\infty < \epsilon$ for all i . From (18.11) and (18.15) we have

$$\kappa_\infty(S)\|\Delta A_i\|_\infty \leq \gamma_{n+2}(1 - \theta)^{1-t}(1 - \rho(A))^{1-t}\kappa_\infty(X)\|A\|_\infty.$$

Therefore (18.14) is satisfied if

$$\gamma_{n+2}(1 - \theta)^{1-t}(1 - \rho(A))^{1-t}\kappa_\infty(X)\|A\|_\infty < \theta(1 - \rho(A)),$$

that is, if

$$\gamma_{n+2}\kappa_\infty(X)\|A\|_\infty < (1 - \theta)^{t-1}\theta(1 - \rho(A))^t.$$

If the integer t is greater than 1 then the function $f(\theta) = (1 - \theta)^{t-1}\theta$ has a maximum on $[0, 1]$ at $\theta_* = t^{-1}$ and $f(\theta_*) = (t-1)^{-1}(1-t^{-1})^t$ satisfies $(4(t-1))^{-1} \leq f(\theta_*) < e^{-1}$. We conclude that for all integers $1 \leq t \leq n$,

$$\gamma_{n+2}\kappa_\infty(X)\|A\|_\infty < \frac{1}{4t}(1 - \rho(A))^t$$

is sufficient to ensure that (18.14) holds. $\square$

If A is normal then $\|A\|_2 = \rho(A) < 1$, $t = 1$, and $\kappa_2(X) = 1$, so (18.13) can be written in the form

$$\rho(A) < \frac{1}{1 + c_n u},$$

where c_n is a constant depending on n . This condition is also easily derived by taking 2-norms in (18.10) and (18.11).

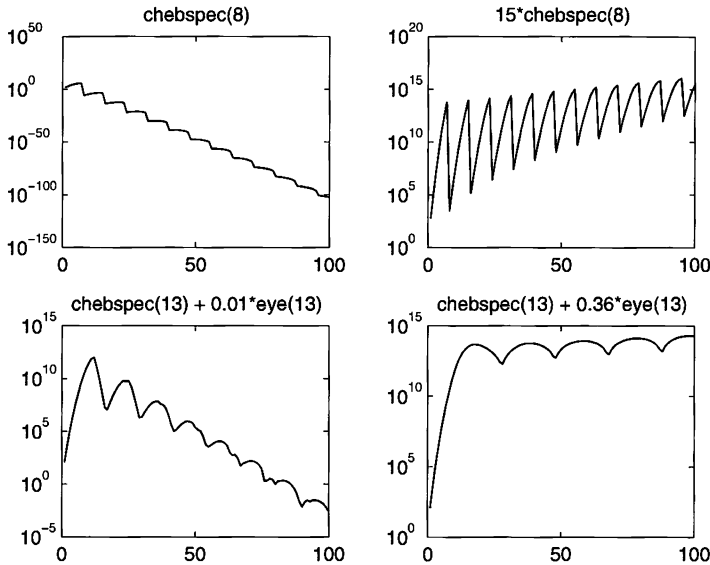
We can show the sharpness of the condition in Theorem 18.1 by using the Chebyshev spectral differentiation matrix $C_n \in \mathbb{R}^{n \times n}$ described by Trefethen and Trummer [1158, 1987]. The matrix C_n arises from degree $n - 1$ polynomial interpolation of n arbitrary data values at n Chebyshev points, including a boundary condition at 1. It is nilpotent and is similar to a single Jordan block of dimension n . We generate C_n in MATLAB using `gallery('chebspec', n)`. Figure 18.4 shows the norms of the powers of four variants of the C_n matrix.

The powers of C_8 converge to zero, while the powers of $15C_8$ diverge. Using a technique for estimating $\kappa_2(X)$ described in [620, 1995], it can be shown that $u\kappa_2(X)\|C_8\|_2 \approx 1.08 \times 10^{-9}$, which is safely less than 1, so that Theorem 18.1 predicts convergence. For $15C_8$ we have $u\kappa_2(X)\|15C_8\|_2 \approx 2.7$, so the theorem correctly does not predict convergence.

Next, for the matrix $A = C_{13} + 0.36I$, whose powers diverge, we have $u\kappa_2(X) \times \|A\|_2/(1 - \rho(A))^{13} \approx 13.05$, and for $A = C_{13} + 0.01I$, whose powers converge, $u\kappa_2(X)\|A\|_2/(1 - \rho(A))^{13} \approx 0.01$, so again the theorem is reasonably sharp.

The plots reveal interesting scalloping patterns in the curves of the norms. For C_8 and $15C_8$ the dips are every eight powers, but the point of first dip and the dipping intervals are altered by adding different multiples of the identity matrix, as shown by the C_{13} examples. Explaining this behaviour is an open problem (see Problem 18.3).

We saw in the last section that the powers of A can be bounded in terms of the pseudospectral radius. Can the pseudospectrum provide information about

Figure 18.4. Computed powers of `chebspec` matrices.

the behaviour of the computed powers? Figure 18.5 shows approximations to the ϵ -pseudospectra for the matrices used in Figure 18.4, where $\epsilon = n\|A\|_2 u$; the (computed) eigenvalues are plotted as crosses “x”. We see that the pseudospectrum lies inside the unit disc precisely when the powers converge to zero.

A heuristic argument based on (18.10) and (18.11) suggests that, if for randomly chosen perturbations ΔA_i with $\|\Delta A_i\| \leq c_n u \|A\|$, most of the eigenvalues of the perturbed matrices lie outside the unit disc, then we can expect a high percentage of the terms $A + \Delta A_i$ in (18.10) to have spectral radius bigger than 1 and hence we can expect the product to diverge. On the other hand, if the $c_n u \|A\|$ -pseudospectrum is wholly contained within the unit disc, each $A + \Delta A_i$ will have spectral radius less than 1 and the product can be expected to converge. (Note, however, that if $\rho(A) < 1$ and $\rho(B) < 1$ it is not necessarily the case that $\rho(AB) < 1$.) To make this heuristic precise, we need an analogue of Theorem 18.1 phrased in terms of the pseudospectrum rather than the Jordan form.

Theorem 18.2 (Higham and Knight). *Suppose that $A \in \mathbb{C}^{n \times n}$ is diagonalizable with $A = X \operatorname{diag}(\lambda_i) X^{-1}$ and has a unique eigenvalue λ_1 of largest modulus. Suppose that $\|X\|_1 = \sum_{i=1}^n |x_{i1}|$ and $\|X^{-1}\|_\infty = \sum_{j=1}^n |y_{1j}|$, where $X^{-1} = (y_{ij})$. If $\rho_\epsilon(A) < 1$ for $\epsilon = c_n u \|A\|_2$, where c_n is a constant depending only on n , then, provided that a certain $O(\epsilon^2)$ term can be ignored, $\lim_{m \rightarrow \infty} fl(A^m) = 0$.*

Proof. It can be shown (see [620, 1995]) that the conditions on $\|X\|_1$ and $\|X^{-1}\|_\infty$ imply there is a perturbation $\tilde{A} = A + \Delta A$ of A with $\|\Delta A\|_2 = \epsilon$ such that

$$\rho(\tilde{A}) \geq \rho(A) + \frac{\kappa_2(X)\epsilon}{n^2} + O(\epsilon^2).$$

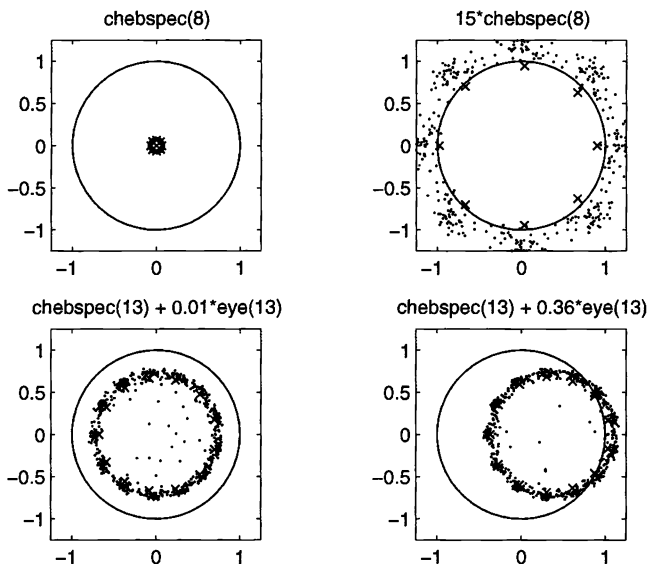


Figure 18.5. Pseudospectra for *chebspec* matrices.

Hence, if $\rho_\epsilon(A) < 1$ then $\rho(A) + \kappa_2(X)\epsilon/n^2 < 1 + O(\epsilon^2)$. Ignoring the $O(\epsilon^2)$ term and rearranging gives

$$c_n u \kappa_2(X) \|A\|_2 / n^2 < 1 - \rho(A).$$

Using Theorem 18.1 we have the required result for $c_n = 4n^2(n+2)$, since $t = 1$. $\square$

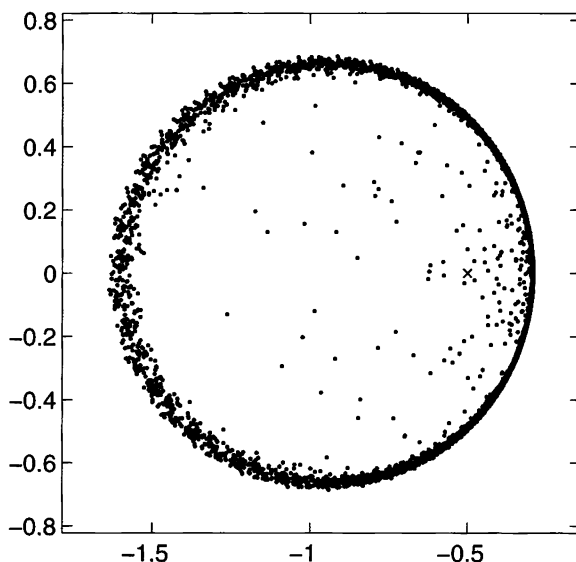
Suppose we compute the eigenvalues of A by a backward stable algorithm, that is, one that yields the exact eigenvalues of $A + E$, where $\|E\|_2 \leq c_n u \|A\|_2$, with c_n a modest constant. (An example of such an algorithm is the QR algorithm [509, 1996, §7.5]). Then the computed spectral radius $\hat{\rho}$ satisfies $\hat{\rho} \leq \rho_{c_n u \|A\|_2}(A)$. In view of Theorem 18.2 we can formulate a rule of thumb—one that bears a pleasing symmetry with the theoretical condition for convergence:

The computed powers of A can be expected to converge to 0 if the spectral radius computed via a backward stable eigensolver is less than 1.

This rule of thumb has also been discussed by Trefethen and Trummer [1158, 1987] and Reichel and Trefethen [978, 1992]. In our experience the rule of thumb is fairly reliable when $\hat{\rho}$ is not too close to 1. For the matrices used in our examples we have, using MATLAB's `eig` function,

$$\begin{aligned} \hat{\rho}(C_8) &= 0.067, & \hat{\rho}(15C_8) &= 0.98, & \hat{\rho}(C_{14}) &= 1.004, \\ \hat{\rho}(C_{13} + 0.01I) &= 0.72, & \hat{\rho}(C_{13} + 0.36I) &= 1.05, \end{aligned}$$

and we observed convergence of the computed powers for C_8 and $C_{13} + 0.01I$ and divergence for the other matrices.

Figure 18.6. *Pseudospectrum for SOR iteration matrix.*

18.3. Application to Stationary Iteration

As we saw in the previous chapter, the errors in stationary iteration satisfy $e_k = (M^{-1}N)^k e_0$, so convergence of the iteration depends on the convergence of $(M^{-1}N)^k$ to zero as $k \rightarrow \infty$. While the errors in stationary iteration are not precisely modelled by the errors in matrix powering, because matrix powers are not formed explicitly, the behaviour of the computed powers $fl((M^{-1}N)^k)$ can be expected to give some insight into the behaviour of stationary iteration.

For the successive overrelaxation (SOR) example at the start of Chapter 17, the matrix $G = M^{-1}N$ is lower triangular with $g_{ij} = 0.5(-1)^{i-j}$. The computed powers of G in MATLAB reach a maximum norm of $\approx 10^{28}$ at $k = 99$ and then decay to zero; the eventual convergence is inevitable in view of the condition (18.12), which clearly is satisfied for this triangular G . An approximation to the $u\|G\|_2$ -pseudospectrum is plotted in Figure 18.6, and we see clearly that part of the pseudospectrum lies outside the unit disk. These facts are consistent with the nonconvergence of the SOR iteration (see Figure 17.1).

That the pseudospectrum of G gives insight into the behaviour of stationary iteration has also been observed by Trefethen [1152, 1990], [1154, 1992] and Chatelin and Frayssé [226, 1992], but no rigorous results about the connection are available.

18.4. Notes and References

This chapter is based closely on Higham and Knight [620, 1995].

The analysis for the powers of the matrix (18.2) is modelled on that of Stewart [1077, 1997], who uses the matrix to construct a Markov chain whose second

largest eigenvalue does not correctly predict the decay of the transient.

For some results on the asymptotic behaviour of the powers of a nonnegative matrix, see Friedland and Schneider [446, 1980].

Another application of matrix powering is in the scaling and squaring method for computing the matrix exponential, which uses the identity $e^A = (e^{A/m})^m$ together with a Taylor or Padé approximation to $e^{A/m}$; see Moler and Van Loan [868, 1978].

Problems

18.1. Let $A \in \mathbb{C}^{n \times n}$ be diagonalizable: $A = X\Lambda X^{-1}$, $\Lambda = \text{diag}(\lambda_i)$. Construct a parametrized example to show that the bound $\|A^k\|_2 \leq \kappa_2(X)\rho(A)^k$ can be arbitrarily weak.

18.2. Show that $\rho(|A|)/\rho(A)$ can be arbitrarily large for $A \in \mathbb{R}^{n \times n}$.

18.3. (RESEARCH PROBLEM) Explain the scalloping patterns in the curves of norms of powers of a matrix, as seen, for example, in Figure 18.4. (Consider exact arithmetic, as the phenomenon is not rounding error dependent.)

18.4. (RESEARCH PROBLEM) Obtain a sharp sufficient condition for $fl(A^k) \rightarrow 0$ in terms of the Schur decomposition of $A \in \mathbb{C}^{n \times n}$ with $\rho(A) < 1$.

Chapter 19

QR Factorization

*Any orthogonal matrix can be written as the product of reflector matrices.
Thus the class of reflections is rich enough for all occasions
and yet each member is characterized by a single vector
which serves to describe its mirror.*

— BERESFORD N. PARLETT, *The Symmetric Eigenvalue Problem* (1998)

*A key observation for understanding the numerical properties of the
modified Gram–Schmidt algorithm is that it can be interpreted as
Householder QR factorization applied to the matrix A
augmented with a square matrix of zero elements on top.
These two algorithms are not only mathematically . . .
but also numerically equivalent.*

*This key observation, apparently by Charles Sheffield,
was relayed to the author in 1968 by Gene Golub.*

— ÅKE BJÖRCK, *Numerics of Gram-Schmidt Orthogonalization* (1994)

*The great stability of unitary transformations in numerical analysis
springs from the fact that both the ℓ_2 -norm
and the Frobenius norm are unitarily invariant.
This means in practice that even when rounding errors are made,
no substantial growth takes place in the
norms of the successive transformed matrices.*

— J. H. WILKINSON,
*Error Analysis of Transformations Based on the
Use of Matrices of the Form $I - 2ww^H$* (1965)

The QR factorization is a versatile computational tool that finds use in linear equation, least squares and eigenvalue problems. It can be computed in three main ways. The Gram–Schmidt process, which sequentially orthogonalizes the columns of A , is the oldest method and is described in most linear algebra textbooks. Givens transformations are preferred when A has a special sparsity structure, such as band or Hessenberg structure. Householder transformations provide the most generally useful way to compute the QR factorization. We explore the numerical properties of all three methods in this chapter. We also examine the use of iterative refinement on a linear system solved with a QR factorization and consider the inherent sensitivity of the QR factorization.

19.1. Householder Transformations

A Householder matrix (also known as a Householder transformation, or Householder reflector) is a matrix of the form

$$P = I - \frac{2}{v^T v} v v^T, \quad 0 \neq v \in \mathbb{R}^n.$$

It enjoys the properties of symmetry and orthogonality, and, consequently, is involutory ($P^2 = I$). The application of P to a vector yields

$$Px = x - \left(\frac{2v^T x}{v^T v} \right) v.$$

Figure 19.1 illustrates this formula and makes it clear why P is sometimes called a Householder reflector: it reflects x about the hyperplane $\text{span}(v)^\perp$.

Householder matrices are powerful tools for introducing zeros into vectors. Consider the question, “Given x and y can we find a Householder matrix P such that $Px = y$?” Since P is orthogonal we clearly require that $\|x\|_2 = \|y\|_2$. Now

$$Px = y \iff x - 2 \left(\frac{v^T x}{v^T v} \right) v = y,$$

and this last equation has the form $\alpha v = x - y$ for some α . But P is independent of the scaling of v , so we can set $\alpha = 1$.

With $v = x - y$ we have

$$v^T v = x^T x + y^T y - 2x^T y,$$

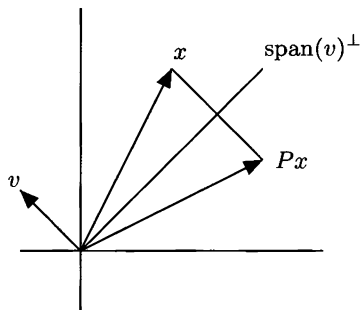
and, since $x^T x = y^T y$,

$$v^T x = x^T x - y^T x = \frac{1}{2} v^T v.$$

Therefore

$$Px = x - v = y,$$

as required. We conclude that, provided $\|x\|_2 = \|y\|_2$, we can find a Householder matrix P such that $Px = y$. (Strictly speaking, we have to exclude the case $x = y$, which would require $v = 0$, making P undefined.)

Figure 19.1. Householder matrix P times vector x .

Normally we choose y to have a special pattern of zeros. The usual choice is $y = \sigma e_1$ where $\sigma = \pm \|x\|_2$, which yields the maximum number of zeros in y . Then

$$v = x - y = x - \sigma e_1.$$

Most textbooks recommend using this formula with the sign chosen to avoid cancellation in the computation of $v_1 = x_1 - \sigma$:

$$\sigma = -\text{sign}(x_1)\|x\|_2, \quad v = x - \sigma e_1. \quad (19.1)$$

This is the approach used by the QR factorization routines in LINPACK [341, 1979] and LAPACK [20, 1999]. The prominence of the sign (19.1) has led to the myth that the other choice of sign is unsuitable. In fact, the other sign is perfectly satisfactory provided that the formula for v_1 is rearranged as follows [924, 1971], [291, 1976], [926, 1998, §6.3.1]:

$$\sigma = \text{sign}(x_1)\|x\|_2, \quad (19.2a)$$

$$v_1 = x_1 - \sigma = \frac{x_1^2 - \|x\|_2^2}{x_1 + \sigma} = \frac{-(x_2^2 + \cdots + x_n^2)}{x_1 + \sigma}. \quad (19.2b)$$

For both choices of sign it is easy to show that $P = I - \beta vv^T$ with

$$\beta = \frac{2}{v^T v} = -\frac{1}{\sigma v_1}.$$

19.2. QR Factorization

A QR factorization of $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ is a factorization

$$A = QR = [Q_1 \quad Q_2] \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1,$$

where $Q \in \mathbb{R}^{m \times m}$ is orthogonal and $R_1 \in \mathbb{R}^{n \times n}$ is upper triangular. The matrix R is called upper trapezoidal, since the term triangular applies only to square

matrices. Depending on the context, either the full factorization $A = QR$ or the “economy size” version $A = Q_1 R_1$ can be called a QR factorization. A quick existence proof of the QR factorization is provided by the Cholesky factorization: if A has full rank and $A^T A = R^T R$ is a Cholesky factorization, then $A = AR^{-1} \cdot R$ is a QR factorization. The QR factorization is unique if A has full rank and we require R to have positive diagonal elements ($A = QD \cdot DR$ is a QR factorization for any $D = \text{diag}(\pm 1)$).

The QR factorization can be computed by premultiplying the given matrix by a suitably chosen sequence of Householder matrices. The process is illustrated for a generic 4×3 matrix as follows:

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{P_1} \left[\begin{array}{c|cc} \times & \times & \times \\ \hline 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{array} \right] \xrightarrow{P_2} \left[\begin{array}{cc|c} \times & \times & \times \\ \hline 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{array} \right] \xrightarrow{P_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R.$$

The general process is adequately described by the k th stage of the reduction to triangular form. With $A_1 = A$ we have, at the start of the k th stage,

$$A_k = \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & x_k & C_k \end{bmatrix}, \quad R_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, \quad x_k \in \mathbb{R}^{m-k+1}, \quad (19.3)$$

where R_{k-1} is upper triangular. Choose a Householder matrix $\tilde{P}_k$ such that $\tilde{P}_k x_k = \sigma e_1$ and embed $\tilde{P}_k$ into an $m \times m$ matrix

$$P_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \tilde{P}_k \end{bmatrix}. \quad (19.4)$$

Then let $A_{k+1} = P_k A_k$. Overall, we obtain $R = P_n P_{n-1} \dots P_1 A =: Q^T A$ ($P_n = I$ if $m = n$).

The Householder matrices P_i are never formed in practice; storage and computations use solely the Householder vector v . For example, to compute A_{k+1} we need to form $\tilde{P}_k C_k$. We can write

$$\tilde{P}_k C_k = (I - \beta v v^T) C_k = C_k - \beta v (v^T C_k), \quad \beta = 2/(v^T v),$$

which shows that the matrix product can be formed as a matrix–vector product followed by an outer product. This approach is much more efficient than forming $\tilde{P}_k$ explicitly and doing a matrix multiplication.

By taking σ nonnegative and switching between the formulae (19.1) and (19.2) according as x_1 is nonpositive and positive, respectively, we can obtain an R factor with nonnegative diagonal elements; this is done in [509, 1996, Algs. 5.1.1, 5.2.1], for example. However, this approach is not recommended for badly row scaled matrices, for reasons explained in §19.4.

The overall cost of the Householder reduction to triangular form is $2n^2(m-n/3)$ flops. Explicit formation of $Q = P_1 P_2 \dots P_n$ can be done in two ways: from left to right or from right to left. The right to left evaluation is the more efficient, because the effective dimension of the intermediate products grows from $m-n$ to m , whereas with the left to right order it is m at each stage. The right to left evaluation requires $4(m^2n - mn^2 + n^3/3)$ flops, or $2n^2(m-n/3)$ flops (the same as for the reduction to triangular form) if only the first n columns of Q are formed. For most applications (such as solving the least squares problem) it suffices to leave Q in factored form.

19.3. Error Analysis of Householder Computations

It is well known that computations with Householder matrices are very stable. Wilkinson showed that the computation of a Householder vector, and the application of a Householder matrix to a given matrix, are both normwise stable, in the sense that the computed Householder vector is very close to the exact one and the computed update is the exact update of a tiny normwise perturbation of the original matrix [1233, 1965, pp. 153–162, 236], [1234, 1965]. Wilkinson also showed that the Householder QR factorization algorithm is normwise backward stable [1233, p. 236]. In this section we give a columnwise error analysis of Householder matrix computations. The columnwise bounds provide extra information over the normwise ones that is essential in certain applications (for example, the analysis of iterative refinement).

In the following analysis it is not worthwhile to evaluate the integer constants in the bounds explicitly, so we make frequent use of the notation

$$\tilde{\gamma}_k = \frac{cku}{1 - cku}$$

introduced in (3.8), where c denotes a small integer constant.

Lemma 19.1. *Let $x \in \mathbb{R}^n$. Consider the following two constructions of $\beta \in \mathbb{R}$ and $v \in \mathbb{R}^n$ such that $Px = \sigma e_1$, where $P = I - \beta vv^T$ is a Householder matrix with $\beta = 2/(v^T v)$:*

% “Usual” choice of sign, (19.1):	% Alternative choice of sign, (19.2):
% $\text{sign}(\sigma) = -\text{sign}(x_1)$.	% $\text{sign}(\sigma) = \text{sign}(x_1)$.
$v = x$	$v = x$
$s = \text{sign}(x_1)\ x\ _2$ % $\sigma = -s$	$s = \text{sign}(x_1)\ x\ _2$ % $\sigma = s$
$v_1 = v_1 + s$	Compute v_1 from (19.2)
$\beta = 1/(sv_1)$	$\beta = 1/(sv_1)$

In floating point arithmetic the computed $\hat{\beta}$ and $\hat{v}$ from both constructions satisfy $\hat{v}(2:n) = v(2:n)$ and

$$\hat{\beta} = \beta(1 + \tilde{\theta}_n), \quad \hat{v}_1 = v_1(1 + \tilde{\theta}_n),$$

where $|\tilde{\theta}_n| \leq \tilde{\gamma}_n$.

Proof. We sketch the proof for the first construction. Each occurrence of δ denotes a different number bounded by $|\delta| \leq u$. We compute $fl(x^T x) = (1 + \theta_n)x^T x$, and then $fl(\|x\|_2) = (1 + \delta)(1 + \theta_n)^{1/2}(x^T x)^{1/2} = (1 + \theta_{n+1})\|x\|_2$ (the latter term $1 + \theta_{n+1}$ is suboptimal, but our main aim is to keep the analysis simple). Hence $\hat{s} = (1 + \theta_{n+1})s$.

For notational convenience, define $w = v_1 + s$. We have $\hat{w} = (v_1 + \hat{s})(1 + \delta) = w(1 + \theta_{n+2})$ (essentially because there is no cancellation in the sum). Hence

$$\begin{aligned}\hat{\beta} &= fl(1/(\hat{s}\hat{w})) = \frac{(1 + \delta)^2}{(1 + \theta_{n+1})s(1 + \theta_{n+2})w} \\ &= \frac{(1 + \delta)^2}{(1 + \theta_{2n+3})sw} = (1 + \theta_{4n+8})\beta.\end{aligned}$$

The proof for the second construction is similar. $\square$

For convenience we will henceforth write Householder matrices in the form $I - vv^T$, which requires $\|v\|_2 = \sqrt{2}$ and amounts to redefining $v := \sqrt{\beta}v$ and $\beta := 1$ in the representation of Lemma 19.1. We can then write, using Lemma 19.1,

$$\hat{v} = v + \Delta v, \quad |\Delta v| \leq \tilde{\gamma}_m |v| \quad (v \in \mathbb{R}^m, \quad \|v\|_2 = \sqrt{2}), \quad (19.5)$$

where, as required for the next two results, the dimension is now m .

The next result describes the application of a Householder matrix to a vector, and is the basis of all the subsequent analysis. In the applications of interest P is defined as in Lemma 19.1, but we will allow P to be an arbitrary Householder matrix. Thus v is an arbitrary, normalized vector, and the only assumption we make is that the computed $\hat{v}$ satisfies (19.5).

Lemma 19.2. *Let $b \in \mathbb{R}^m$ and consider the computation of $y = \hat{P}b = (I - \hat{v}\hat{v}^T)b = b - \hat{v}(\hat{v}^T b)$, where $\hat{v} \in \mathbb{R}^m$ satisfies (19.5). The computed $\hat{y}$ satisfies*

$$\hat{y} = (P + \Delta P)b, \quad \|\Delta P\|_F \leq \tilde{\gamma}_m,$$

where $P = I - vv^T$.

Proof. (Cf. the proof of Lemma 3.9.) We have

$$\hat{w} := fl(\hat{v}(\hat{v}^T b)) = (\hat{v} + \Delta \hat{v})(\hat{v}^T(b + \Delta b)),$$

where $|\Delta \hat{v}| \leq u|\hat{v}|$ and $|\Delta b| \leq \gamma_m |b|$. Hence

$$\hat{w} = (v + \Delta v + \Delta \hat{v})(v + \Delta v)^T(b + \Delta b) =: v(v^T b) + \Delta w,$$

where $|\Delta w| \leq \tilde{\gamma}_m |v||v^T b|$. Then

$$\hat{y} = fl(b - \hat{w}) = b - v(v^T b) - \Delta w + \Delta y_1, \quad |\Delta y_1| \leq u|b - \hat{w}|.$$

We have

$$|-\Delta w + \Delta y_1| \leq u|b| + \tilde{\gamma}_m |v||v^T b|.$$

Hence $\hat{y} = Pb + \Delta y$, where $\|\Delta y\|_2 \leq \tilde{\gamma}_m \|b\|_2$. But then $\hat{y} = (P + \Delta P)b$, where $\Delta P = \Delta y b^T / b^T b$ satisfies $\|\Delta P\|_F = \|\Delta y\|_2 / \|b\|_2 \leq \tilde{\gamma}_m$. $\square$

Next, we consider a sequence of Householder transformations applied to a matrix. Again, each Householder matrix is arbitrary and need have no connection to the matrix to which it is being applied. In the cases of interest, the Householder matrices P_k have the form (19.4), and so are of ever-decreasing effective dimension, but to exploit this property would not lead to any significant improvement in the bounds. Since the P_j are applied to the columns of A , columnwise error bounds are to be expected, and these are provided by the next lemma.

We will assume that

$$r\tilde{\gamma}_m < \frac{1}{2}, \quad (19.6)$$

where r is the number of Householder transformations. We will write the j th columns of A and ΔA as a_j and Δa_j , respectively.

Lemma 19.3. *Consider the sequence of transformations*

$$A_{k+1} = P_k A_k, \quad k = 1: r,$$

where $A_1 = A \in \mathbb{R}^{m \times n}$ and $P_k = I - v_k v_k^T \in \mathbb{R}^{m \times m}$ is a Householder matrix. Assume that the transformations are performed using computed Householder vectors $\hat{v}_k \approx v_k$ that satisfy (19.5). The computed matrix $\hat{A}_{r+1}$ satisfies

$$\hat{A}_{r+1} = Q^T(A + \Delta A), \quad (19.7)$$

where $Q^T = P_r P_{r-1} \dots P_1$ and

$$\|\Delta a_j\|_2 \leq r\tilde{\gamma}_m \|a_j\|_2, \quad j = 1: n. \quad (19.8)$$

In the special case $n = 1$, so that $A \equiv a$, we have $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$ with $\|\Delta Q\|_F \leq r\tilde{\gamma}_m$.

Proof. The j th column of A undergoes the transformations $a_j^{(r+1)} = P_r \dots P_1 a_j$. By Lemma 19.2 we have

$$\hat{a}_j^{(r+1)} = (P_r + \Delta P_r) \dots (P_1 + \Delta P_1) a_j, \quad (19.9)$$

where each ΔP_k depends on j and satisfies $\|\Delta P_k\|_F \leq \tilde{\gamma}_m$. Using Lemma 3.7 we obtain

$$\begin{aligned} \hat{a}_j^{(r+1)} &= Q^T(a_j + \Delta a_j), \\ \|\Delta a_j\|_2 &\leq ((1 + \tilde{\gamma}_m)^r - 1) \|a_j\|_2 \leq \frac{r\tilde{\gamma}_m}{1 - r\tilde{\gamma}_m} \|a_j\|_2 = r\tilde{\gamma}_m' \|a_j\|_2, \end{aligned} \quad (19.10)$$

using Lemma 3.1 and assumption (19.6). Finally, if $n = 1$, so that A is a column vector, then (as in the proof of Lemma 19.2) we can rewrite (19.7) as $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$, where $\Delta Q^T = (Q^T \Delta A) a^T / a^T a$ and $\|\Delta Q\|_F = \|\Delta A\|_2 / \|a\|_2 \leq r\tilde{\gamma}_m$.

□

Recall that columnwise error bounds are easily converted into normwise ones (Lemma 6.6). For example, (19.8) implies $\|\Delta A\|_F \leq r\tilde{\gamma}_m \|A\|_F$.

Lemma 19.3 yields the standard backward error result for Householder QR factorization.

Theorem 19.4. Let $\hat{R} \in \mathbb{R}^{m \times n}$ be the computed upper trapezoidal QR factor of $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) obtained via the Householder QR algorithm (with either choice of sign, (19.1) or (19.2)). Then there exists an orthogonal $Q \in \mathbb{R}^{m \times m}$ such that

$$A + \Delta A = Q\hat{R},$$

where

$$\|\Delta a_j\|_2 \leq \tilde{\gamma}_{mn} \|a_j\|_2, \quad j = 1:n. \quad (19.11)$$

The matrix Q is given explicitly as $Q = (P_n P_{n-1} \dots P_1)^T$, where P_k is the Householder matrix that corresponds to the exact application of the k th step of the algorithm to $\hat{A}_k$.

Proof. This is virtually a direct application of Lemma 19.3, with P_k defined as the Householder matrix that produces zeros below the diagonal in the k th column of the computed matrix $\hat{A}_k$. One subtlety is that we do not explicitly compute the lower triangular elements of $\hat{R}$, but rather set them to zero explicitly. However, it is easy to see that the conclusions of Lemmas 19.2 and 19.3 are still valid in these circumstances; the essential reason is that the elements of $\Delta P b$ in Lemma 19.2 that correspond to elements that are zeroed by the Householder matrix P are forced to be zero, and hence we can set the corresponding rows of ΔP to zero too, without compromising the bound on $\|\Delta P\|_F$. $\square$

We note that for Householder QR factorization $\Delta P_k = 0$ for $k > j$ in (19.9), and consequently the factor $\tilde{\gamma}_{mn}$ in (19.11) can be reduced to $\tilde{\gamma}_{mj}$.

Theorem 19.4 is often stated in the weaker form $\|\Delta A\|_F \leq \tilde{\gamma}_{mn} \|A\|_F$ that is implied by (19.11) (see, e.g., [509, 1996, §5.2.1]). For a matrix whose columns vary widely in norm this normwise bound on ΔA is much weaker than (19.11). For an alternative way to express this backward error result define B by $A = BD_C$, where $D_C = \text{diag}(\|A(:, j)\|_2)$; then the result states that there exists an orthogonal $Q \in \mathbb{R}^{m \times m}$ such that

$$(B + \Delta B)D_C = Q\hat{R}, \quad \|\Delta B(:, j)\|_2 \leq \tilde{\gamma}_{mn}, \quad (19.12)$$

so that $\|\Delta B\|_2 / \|B\|_2 = O(u)$.

Note that the matrix Q in Theorem 19.4 is not computed by the QR factorization algorithm and is of purely theoretical interest. It is the fact that Q is exactly orthogonal that makes the result so useful. When Q is explicitly formed, two questions arise:

1. How close is the computed $\hat{Q}$ to being orthonormal?
2. How large is $A - \hat{Q}\hat{R}$?

Both questions are easily answered using the analysis above.

We suppose that $Q = P_1 P_2 \dots P_n$ is evaluated in the more efficient right to left order. Lemma 19.3 gives (with $A_1 = I_m$)

$$\hat{Q} = Q(I_m + \Delta I), \quad \|\Delta I(:, j)\|_2 \leq \tilde{\gamma}_{mn}, \quad j = 1:n.$$

Hence

$$\|\hat{Q} - Q\|_F \leq \sqrt{n} \tilde{\gamma}_{mn}, \quad (19.13)$$

showing that $\widehat{Q}$ is very close to an orthonormal matrix. Moreover, using Theorem 19.4,

$$\begin{aligned}\|(A - \widehat{Q}\widehat{R})(:,j)\|_2 &= \|(A - Q\widehat{R})(:,j) + ((Q - \widehat{Q})\widehat{R})(:,j)\|_2 \\ &\leq \tilde{\gamma}_{mn}\|a_j\|_2 + \|Q - \widehat{Q}\|_F\|\widehat{R}(:,j)\|_2 \\ &\leq \sqrt{n}\tilde{\gamma}_{mn}\|a_j\|_2.\end{aligned}$$

Thus if Q is replaced by $\widehat{Q}$ in Theorem 19.4, so that $A + \Delta A = \widehat{Q}\widehat{R}$, then the backward error bound remains true with an appropriate increase in the constant.

Finally, we consider use of the QR factorization to solve a linear system. Given a QR factorization of a nonsingular matrix $A \in \mathbb{R}^{n \times n}$, a linear system $Ax = b$ can be solved by forming $Q^T b$ and then solving $Rx = Q^T b$. From Theorem 19.4, the computed $\widehat{R}$ is guaranteed to be nonsingular if $\kappa_2(A)n^{1/2}\tilde{\gamma}_{mn} < 1$.

Theorem 19.5. *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. Suppose we solve the system $Ax = b$ with the aid of a QR factorization computed by the Householder algorithm. The computed $\widehat{x}$ satisfies*

$$(A + \Delta A)\widehat{x} = b + \Delta b,$$

where

$$\|\Delta a_j\|_2 \leq \tilde{\gamma}_{n^2}\|a_j\|_2, \quad j = 1:n, \quad \|\Delta b\|_2 \leq \tilde{\gamma}_{n^2}\|b\|_2.$$

Proof. By Theorem 19.4, the computed upper triangular factor $\widehat{R}$ satisfies $A + \Delta A = Q\widehat{R}$ with $\|\Delta a_j\|_2 \leq \tilde{\gamma}_{n^2}\|a_j\|_2$. By Lemma 19.3, the computed transformed right-hand side satisfies $\widehat{c} = Q^T(b + \Delta b)$, with $\|\Delta b\|_2 \leq \tilde{\gamma}_{n^2}\|b\|_2$. Importantly, the same orthogonal matrix Q appears in the equations involving $\widehat{R}$ and $\widehat{c}$.

By Theorem 8.5, the computed solution $\widehat{x}$ to the triangular system $\widehat{R}x = \widehat{c}$ satisfies

$$(\widehat{R} + \Delta R)\widehat{x} = \widehat{c}, \quad |\Delta R| \leq \gamma_n |\widehat{R}|.$$

Premultiplying by Q yields

$$(A + \Delta A + Q\Delta R)\widehat{x} = b + \Delta b,$$

that is, $(A + \overline{\Delta A})\widehat{x} = b + \Delta b$, where $\overline{\Delta A} = \Delta A + Q\Delta R$. Using $\widehat{R} = Q^T(A + \Delta A)$ we have

$$\begin{aligned}\|\overline{\Delta a}_j\|_2 &\leq \|\Delta a_j\|_2 + \gamma_n \|\widehat{r}_j\|_2 \\ &= \|\Delta a_j\|_2 + \gamma_n \|a_j + \Delta a_j\|_2 \\ &\leq \tilde{\gamma}_{n^2}\|a_j\|_2. \quad \square\end{aligned}$$

The proof of Theorem 19.5 naturally leads to a result in which b is perturbed. However, we can easily modify the proof so that only A is perturbed: the trick is to use the last part of Lemma 19.3 to write $\widehat{c} = (Q + \Delta Q)^T b$, where $\|\Delta Q\|_F \leq \tilde{\gamma}_{n^2}$, and to premultiply by $(Q + \Delta Q)^{-T}$ instead of Q in the middle of the proof. This leads to the result

$$(A + \Delta A)\widehat{x} = b, \quad \|\Delta a_j\|_2 \leq \tilde{\gamma}_{n^2}\|a_j\|_2, \quad j = 1:n. \quad (19.14)$$

An interesting application of Theorem 19.5 is to iterative refinement, as explained in §19.7.

19.4. Pivoting and Row-Wise Stability

It is natural to ask whether a small *row-wise* backward error bound holds for Householder QR factorization, since matrices A for which the rows vary widely in norm occur commonly in weighted least squares problems (see §20.8). In general, the answer is no. However, if column pivoting is used together with row pivoting or row sorting, and the choice of sign (19.1) is used in constructing the Householder vectors, then such a bound does hold. The column pivoting strategy exchanges columns at the start of the k th stage of the factorization to ensure that

$$\|a_k^{(k)}(k:m)\|_2 = \max_{j \geq k} \|a_j^{(k)}(k:m)\|_2. \quad (19.15)$$

In other words, it maximizes the norm of the active part of the pivot column.

Theorem 19.6 (Powell and Reid; Cox and Higham). *Let $\hat{R} \in \mathbb{R}^{m \times n}$ be the computed upper trapezoidal QR factor of $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) obtained via the Householder QR algorithm with column pivoting, with the choice of sign (19.1). Then there exists an orthogonal $Q \in \mathbb{R}^{m \times m}$ such that*

$$(A + \Delta A)\Pi = Q\hat{R},$$

where Π is a permutation matrix that describes the overall effect of the column interchanges and

$$|\Delta a_{ij}| \leq j^2 \tilde{\gamma}_m \alpha_i \max_s |a_{is}|,$$

where

$$\alpha_i = \frac{\max_{j,k} |a_{ij}^{(k)}|}{\max_j |a_{ij}|}$$

and $A_k = (a_{ij}^{(k)})$. The matrix Q is defined as in Theorem 19.4. $\square$

Problem 19.6 indicates the first part of a proof of Theorem 19.6 and gives insight into why column pivoting is necessary to obtain such a result.

Theorem 19.6 shows that the row-wise backward error is bounded by a multiple of $\max_i \alpha_i$. In general, the row-wise growth factors α_i can be arbitrarily large. The size of the α_i is limited by row pivoting and row sorting. With row pivoting, after the column interchange has taken place at the start of the k th stage we interchange rows to ensure that

$$|a_{kk}^{(k)}| = \max_{i \geq k} |a_{ik}^{(k)}|.$$

The alternative strategy of row sorting reorders the rows prior to carrying out the factorization so that

$$\|A(i, :)\|_\infty = \max_{j \geq i} \|A(j, :)\|_\infty, \quad i = 1:m.$$

Note that row interchanges before or during Householder QR factorization have no mathematical effect on the result, because they can be absorbed into the Q factor and the QR factorization is essentially unique. The effect of row interchanges is to change the intermediate numbers that arise during the factorization, and hence to

Table 19.1. *Backward errors for QR factorization with no pivoting, row sorting, and column pivoting on matrix (19.16).*

Pivoting:	None	Row	Column	Row and column
Normwise (η)	2.9e-16	4.2e-16	3.2e-16	1.9e-16
Row-wise (η_R)	2.0e-4	2.7e-4	2.0e-4	4.0-16
$\max_i \alpha_i$	1.4e12	1.0e12	1.4e12	2.0e0

alter the effects of rounding errors. If row pivoting or row sorting is used it can be shown that $\alpha_i \leq \sqrt{m}(1 + \sqrt{2})^{n-1}$ for all i [275, 1998], [951, 1969], and experience shows that the α_i are usually small in practice. Therefore the α_i are somewhat analogous to the growth factor for GEPP. For the alternative choice of sign (19.2) in the Householder vectors, the α_i are unbounded even if row pivoting or row sorting is used and so row-wise stability is lost; see [275, 1998] for an illustrative example.

We give an example to illustrate the results. The matrix, from [1180, 1985], is

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \\ \mu & \mu & \mu \\ \mu & \mu & -\mu \end{bmatrix}. \quad (19.16)$$

We applied Householder QR factorization in MATLAB with $\mu = 10^{12}$, using both no pivoting and combinations of row sorting and column pivoting. The normwise and row-wise backward errors

$$\eta = \frac{\|A\Pi - \widehat{Q}\widehat{R}\|_2}{\|A\|_2}, \quad \eta_R = \max_i \frac{\|(A\Pi - \widehat{Q}\widehat{R})(i, :)\|_2}{\|A(i, :)\|_2}$$

are shown in Table 19.1; here, $\widehat{Q}$ denotes the computed product of the Householder transformations. As expected, the computation is normwise backward stable in every case, but row-wise stability prevails only when both row sorting and column pivoting are used, with the size of the α_i being a good predictor of stability.

19.5. Aggregated Householder Transformations

In Chapter 13 we noted that LU factorization algorithms can be partitioned so as to express the bulk of the computation as matrix-matrix operations (level-3 BLAS). For computations with Householder transformations the same goal can be achieved by aggregating the transformations. This technique is widely used in LAPACK.

One form of aggregation is the “WY” representation of Bischof and Van Loan [117, 1987]. This involves representing the product $Q_r = P_r P_{r-1} \dots P_1$ of r Householder transformations $P_i = I - v_i v_i^T \in \mathbb{R}^{m \times m}$ (where $v_i^T v_i = 2$) in the form

$$Q_r = I + W_r Y_r^T, \quad W_r, Y_r \in \mathbb{R}^{m \times r}.$$

This can be done using the recurrence

$$W_1 = -v_1, \quad Y_1 = v_1, \quad W_i = [W_{i-1} \quad -v_i], \quad Y_i = [Y_{i-1} \quad Q_{i-1}^T v_i]. \quad (19.17)$$

Using the WY representation, a partitioned QR factorization can be developed as follows. Partition $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) as

$$A = [A_1 \quad B], \quad A_1 \in \mathbb{R}^{m \times r}, \quad (19.18)$$

and compute the Householder QR factorization of A_1 ,

$$P_r P_{r-1} \dots P_1 A_1 = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}.$$

The product $P_r P_{r-1} \dots P_1 = I + W_r Y_r^T$ is accumulated using (19.17), as the P_i are generated, and then B is updated according to

$$B \leftarrow (I + W_r Y_r^T) B = B + W_r (Y_r^T B),$$

which involves only level-3 BLAS operations. The process is now repeated on the last $m - r$ rows of B .

When considering numerical stability, two aspects of the WY representation need investigating: its construction and its application. For the construction, we need to show that $\hat{Q}_r := I + \hat{W}_r \hat{Y}_r^T$ satisfies

$$\|\hat{Q}_r \hat{Q}_r^T - I\|_2 \leq d_1 u, \quad (19.19)$$

$$\|\hat{W}_r\|_2 \leq d_2, \quad \|\hat{Y}_r\|_2 \leq d_3, \quad (19.20)$$

for modest constants d_1 , d_2 , and d_3 . Now

$$\hat{Q}_i := I + [\hat{W}_{i-1} \quad -\hat{v}_i] \begin{bmatrix} \hat{Y}_{i-1}^T \\ fl(\hat{v}_i^T \hat{Q}_{i-1}) \end{bmatrix} = \hat{Q}_{i-1} - \hat{v}_i fl(\hat{v}_i^T \hat{Q}_{i-1}).$$

But this last equation is essentially a standard multiplication by a Householder matrix, $\hat{Q}_i = (I - \hat{v}_i \hat{v}_i^T) \hat{Q}_{i-1}$, albeit with less opportunity for rounding errors. It follows from Lemma 19.3 that the near orthogonality of $\hat{Q}_{i-1}$ is inherited by $\hat{Q}_i$; the condition on $\hat{Y}_r$ in (19.20) follows similarly and that on $\hat{W}_r$ is trivial. Note that the condition (19.19) implies that

$$\hat{Q}_r = U_r + \Delta U_r, \quad U_r^T U_r = I, \quad \|\Delta U_r\|_2 \leq d_1 u, \quad (19.21)$$

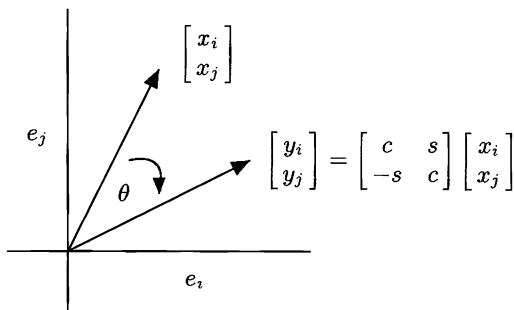
that is, $\hat{Q}$ is close to an exactly orthogonal matrix (see Problem 19.14).

Next we consider the application of $\hat{Q}_r$. Suppose we form $C = \hat{Q}_r B = (I + \hat{W}_r \hat{Y}_r^T) B$ for the B in (19.18), so that

$$\hat{C} = fl(B + fl(\hat{W}_r (\hat{Y}_r^T B))).$$

Analysing this level-3 BLAS-based computation using (19.21) and the very general assumption (13.4) on matrix multiplication (for the 2-norm), it is straightforward to show that

$$\begin{aligned} \hat{C} &= U_r B + \Delta C = U_r (B + U_r^T \Delta C), \\ \|\Delta C\|_2 &\leq [1 + d_1 + d_2 d_3 (1 + c_1(r, m, n - r) \\ &\quad + c_1(m, r, n - r))] u \|B\|_2 + O(u^2). \end{aligned} \quad (19.22)$$

Figure 19.2. *Givens rotation*, $y = G(i, j, \theta)x$.

This result shows that the computed update is an exact orthogonal update of a perturbation of B , where the norm of the perturbation is bounded in terms of the error constants for the level-3 BLAS.

Two conclusions can be drawn. First, algorithms that employ the WY representation with conventional level-3 BLAS are as stable as the corresponding point algorithms. Second, the use of fast BLAS3 for applying the updates affects stability only through the constants in the backward error bounds. The same conclusions apply to the more storage-efficient compact WY representation of Schreiber and Van Loan [1024, 1989], and the variation of Puglisi [959, 1992].

19.6. Givens Rotations

Another way to compute the QR factorization is with Givens rotations. A Givens rotation (or plane rotation) $G(i, j, \theta) \in \mathbb{R}^{n \times n}$ is equal to the identity matrix except that

$$G([i, j], [i, j]) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix},$$

where $c = \cos \theta$ and $s = \sin \theta$. The multiplication $y = G(i, j, \theta)x$ rotates x through θ radians clockwise in the (i, j) plane; see Figure 19.2. Algebraically,

$$y_k = \begin{cases} x_k, & k \neq i, j, \\ cx_i + sx_j, & k = i, \\ -sx_i + cx_j, & k = j, \end{cases}$$

and so $y_j = 0$ if

$$s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}, \quad c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}. \quad (19.23)$$

Givens rotations are therefore useful for introducing zeros into a vector one at a time. Note that there is no need to work out the angle θ , since c and s in (19.23) are all that are needed to apply the rotation. In practice, we would scale the computation to avoid overflow (cf. §27.8).

To compute the QR factorization, Givens rotations are used to eliminate the elements below the diagonal in a systematic fashion. Various choices and orderings of rotations can be used; a natural one is illustrated as follows for a generic 4×3 matrix:

$$\begin{aligned}
 A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} &\xrightarrow{G_{34}} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G_{12}} \\
 \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} &\xrightarrow{G_{34}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{G_{34}} \\
 &\begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R.
 \end{aligned}$$

The operation count for Givens QR factorization of a general $m \times n$ matrix ($m \geq n$) is $3n^2(m - n/3)$ flops, which is 50% more than that for Householder QR factorization. The main use of Givens rotations is to operate on structured matrices—for example, to compute the QR factorization of a tridiagonal or Hessenberg matrix, or to carry out delicate zeroing in updating or downdating problems [509, 1996, §12.5].

Error analysis for Givens rotations is similar to that for Householder matrices—but a little easier. We omit the (straightforward) proof of the first result.

Lemma 19.7. *Let a Givens rotation $G(i, j, \theta)$ be constructed according to (19.23). The computed $\hat{c}$ and $\hat{s}$ satisfy*

$$\hat{c} = c(1 + \theta_4), \quad \hat{s} = s(1 + \theta'_4), \quad (19.24)$$

where $|\theta_4|, |\theta'_4| \leq \gamma_4$. $\square$

Lemma 19.8. *Let $x \in \mathbb{R}^m$ and consider the computation of $y = \hat{G}_{ij}x$, where $\hat{G}_{ij}$ is a computed Givens rotation in the (i, j) plane for which $\hat{c}$ and $\hat{s}$ satisfy (19.24). The computed $\hat{y}$ satisfies*

$$\hat{y} = (G_{ij} + \Delta G_{ij})x, \quad \|\Delta G_{ij}\|_F \leq \sqrt{2}\gamma_6,$$

where G_{ij} is an exact Givens rotation based on c and s in (19.24). All the rows of ΔG_{ij} except the i th and j th are zero.

Proof. The vector $\hat{y}$ differs from x only in elements i and j . We have

$$\hat{y}_i = fl(\hat{c}x_i + \hat{s}x_j) = cx_i(1 + \theta_6) + sx_j(1 + \theta'_6),$$

where $|\theta_6|, |\theta'_6| \leq \gamma_6$, and similarly for $\hat{y}_j$. Hence

$$|\hat{y} - G_{ij}x| \leq \gamma_6 |G_{ij}| |x|,$$

so that $\|\hat{y} - G_{ij}x\|_2 \leq \sqrt{2}\gamma_6\|x\|_2$. We take $\Delta G_{ij} = (\hat{y} - G_{ij}x)x^T/x^T x$. $\square$

For the next result we need the notion of disjoint Givens rotations. Rotations $G_{i_1, j_1}, \dots, G_{i_r, j_r}$ are disjoint if the integers i_s, j_s, i_t , and j_t are distinct for $s \neq t$. Disjoint rotations are “nonconflicting” and therefore commute; it matters neither mathematically nor numerically in which order the rotations are applied. (Disjoint rotations can therefore be applied in parallel, though that is not our interest here.) Our approach is to take a given sequence of rotations and reorder them into groups of disjoint rotations. The reordered algorithm is numerically equivalent to the original one, but allows a simpler error analysis.

As an example of a rotation sequence already ordered into disjoint groups, consider the following sequence and ordering illustrated for a 6×5 matrix:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 1 & \times & \times & \times & \times \\ 2 & 3 & \times & \times & \times \\ 3 & 4 & 5 & \times & \times \\ 4 & 5 & 6 & 7 & \times \\ 5 & 6 & 7 & 8 & 9 \end{bmatrix}.$$

Here, an integer k in position (i, j) denotes that the (i, j) element is eliminated on the k th step by a rotation in the (j, i) plane, and all rotations on the k th step are disjoint. For an $m \times n$ matrix with $m > n$ there are $r = m + n - 2$ stages, and the Givens QR factorization can be written as $W_r W_{r-1} \dots W_1 A = R$, where each W_i is a product of at most n disjoint rotations. It is easy to see that an analogous grouping into disjoint rotations can be done for the scheme illustrated at the start of this section.

Lemma 19.9. *Consider the sequence of transformations*

$$A_{k+1} = W_k A_k, \quad k = 1:r,$$

where $A_1 = A \in \mathbb{R}^{m \times n}$ and each W_k is a product of disjoint Givens rotations. Assume that the individual Givens rotations are performed using computed sine and cosine values related to the exact values defining the W_k by (19.24). Then the computed matrix $\hat{A}_{r+1}$ satisfies

$$\hat{A}_{r+1} = Q^T (A + \Delta A),$$

where $Q^T = W_r W_{r-1} \dots W_1$ and

$$\|\Delta a_j\|_2 \leq \tilde{\gamma}_r \|a_j\|_2, \quad j = 1:n.$$

In the special case $n = 1$, so that $A = a$, we have $\hat{a}^{(r+1)} = (Q + \Delta Q)^T a$ with $\|\Delta Q\|_F \leq \tilde{\gamma}_r$.

Proof. The proof is analogous to that of Lemma 19.3, so we offer only a sketch. First, we consider the j th column of A , a_j , which undergoes the transformations $a_j^{(r+1)} = W_r \dots W_1 a_j$. By Lemma 19.8 and the disjointness of the rotations, we have

$$\hat{a}_j^{(r+1)} = (W_r + \Delta W_r) \dots (W_1 + \Delta W_1) a_j,$$

where each ΔW_k depends on j and satisfies $\|\Delta W_k\|_2 \leq \sqrt{2}\gamma_6$. Using Lemma 3.6 we obtain

$$\begin{aligned}\hat{a}_j^{(r+1)} &= Q^T(a_j + \Delta a_j), \\ \|\Delta a_j\|_2 &\leq ((1 + \sqrt{2}\gamma_6)^r - 1)\|a_j\|_2 = \tilde{\gamma}_r\|a_j\|_2.\end{aligned}\quad (19.25)$$

The result for $n = 1$ is proved as in Lemma 19.3. $\square$

We are now suitably equipped to give a result for Givens QR factorization.

Theorem 19.10. *Let $\hat{R} \in \mathbb{R}^{m \times n}$ be the computed upper trapezoidal QR factor of $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) obtained via the Givens QR algorithm, with any standard choice and ordering of rotations. Then there exists an orthogonal $Q \in \mathbb{R}^{m \times m}$ such that*

$$A + \Delta A = Q\hat{R}, \quad \|\Delta a_j\|_2 \leq \tilde{\gamma}_{m+n-2}\|a_j\|_2, \quad j = 1:n.$$

(The matrix Q is a product of Givens rotations, the k th of which corresponds to the exact application of the k th step of the algorithm to $\hat{A}_k$.) $\square$

It is interesting that the error bounds for QR factorization with Givens rotations are a factor n smaller than those for Householder QR factorization. This appears to be an artefact of the analysis, and we are not aware of any difference in accuracy in practice.

19.7. Iterative Refinement

Consider a nonsingular linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$. Suppose we solve the system using a QR factorization $A = QR$ computed using Householder or Givens transformations (thus, x is obtained by solving $Rx = Q^T b$). Theorem 19.5, and its obvious analogue for Givens rotations, imply a small columnwise relative backward error but not a small componentwise relative backward error. In fact, we know of no nontrivial class of matrices for which Householder or Givens QR factorization is guaranteed to yield a small componentwise relative backward error.

Suppose that we carry out a step of fixed precision iterative refinement, to obtain $\hat{y}$. In order to invoke Theorem 12.4 we need to express the backward error bounds for $\hat{x}$ in the form

$$|b - A\hat{x}| \leq u(G|A||\hat{x}| + H|b|),$$

for suitable nonnegative matrices G and H . For our columnwise backward error bounds this can be done with the aid of Lemma 6.6: (19.14) yields

$$|b - A\hat{x}| \leq |\Delta A||\hat{x}| \leq \tilde{\gamma}_{n^2} ee^T |A||\hat{x}|.$$

Theorem 12.4 can now be invoked with $G \approx n^2 ee^T$ and $H = 0$, giving the conclusion that the componentwise relative backward error $\omega_{|A|,|b|}(\hat{y})$ after one step of iterative refinement will be small provided that A is not too ill conditioned and $|A||\hat{y}|$ is not too badly scaled. This conclusion is similar to that for GEPP, except that for GEPP there is the added requirement that the LU factorization does not suffer large element growth.

The limiting forward error can be bounded by Theorems 12.1 and 12.2, for which $\eta \approx n^2 u \kappa_\infty(A)$.

The performance of QR factorization with fixed precision iterative refinement is illustrated in Tables 12.1–12.3 in §12.2. The performance is as predicted by the analysis. Notice that the initial componentwise relative backward error is large in Table 12.2 but that iterative refinement successfully reduces it to the roundoff level (despite $\text{cond}(A^{-1})\sigma(A, x)$ being huge). It is worth stressing that the QR factorization yielded a small *normwise* relative backward error in each example ($\eta_{A,b}(\hat{x}) < u$, in fact), as we know it must.

19.8. Gram–Schmidt Orthogonalization

The oldest method for computing a QR factorization is the Gram–Schmidt orthogonalization method. It can be derived directly from the equation $A = QR$, where $A, Q \in \mathbb{R}^{m \times n}$ and $R \in \mathbb{R}^{n \times n}$ (Gram–Schmidt does not compute the $m \times m$ matrix Q in the full QR factorization and hence does not provide a basis for the orthogonal complement of $\text{range}(A)$.) Denoting by a_j and q_j the j th columns of A and Q , respectively, we have

$$a_j = \sum_{k=1}^j r_{kj} q_k.$$

Premultiplying by q_i^T yields, since Q has orthonormal columns, $q_i^T a_j = r_{ij}$, $i = 1:j-1$. Further,

$$q_j = q'_j / r_{jj},$$

where

$$q'_j = a_j - \sum_{k=1}^{j-1} r_{kj} q_k, \quad r_{jj} = \|q'_j\|_2.$$

Hence we can compute Q and R a column at a time. To ensure that $r_{jj} > 0$ we require that A has full rank.

Algorithm 19.11 (classical Gram–Schmidt). Given $A \in \mathbb{R}^{m \times n}$ of rank n this algorithm computes the QR factorization $A = QR$, where Q is $m \times n$ and R is $n \times n$, by the Gram–Schmidt method.

```

for  $j = 1:n$ 
  for  $i = 1:j-1$ 
     $r_{ij} = q_i^T a_j$ 
  end
   $q'_j = a_j - \sum_{k=1}^{j-1} r_{kj} q_k$ 
   $r_{jj} = \|q'_j\|_2$ 
   $q_j = q'_j / r_{jj}$ 
end

```

Cost: $2mn^2$ flops ($2n^3/3$ flops more than Householder QR factorization with Q left in factored form).

In the classical Gram–Schmidt method (CGS), a_j appears in the computation only at the j th stage. The method can be rearranged so that as soon as q_j is computed, all the remaining vectors are orthogonalized against q_j . This gives the modified Gram–Schmidt method (MGS).

Algorithm 19.12 (modified Gram–Schmidt). Given $A \in \mathbb{R}^{m \times n}$ of rank n this algorithm computes the QR factorization $A = QR$, where Q is $m \times n$ and R is $n \times n$, by the MGS method.

```

 $a_k^{(1)} = a_k, k = 1:n$ 
for  $k = 1:n$ 
   $r_{kk} = \|a_k^{(k)}\|_2$ 
   $q_k = a_k^{(k)} / r_{kk}$ 
  for  $j = k+1:n$ 
     $r_{kj} = q_k^T a_j^{(k)}$ 
     $a_j^{(k+1)} = a_j^{(k)} - r_{kj} q_k$ 
  end
end
end

```

Cost: $2mn^2$ flops.

It is worth noting that there are two differences between the CGS and MGS methods. The first is the order in which the calculations are performed: in the modified method each remaining vector is updated once on each step instead of having all its updates done together on one step. This is purely a matter of the order in which the operations are performed. Second, and more crucially in finite precision computation, two different (but mathematically equivalent) formulae for r_{kj} are used: in the classical method, $r_{kj} = q_k^T a_j$, which involves the original vector a_j , whereas in the modified method a_j is replaced in this formula by the partially orthogonalized vector $a_j^{(k)}$. Another way to view the difference between the two Gram–Schmidt methods is via representations of an orthogonal projection; see Problem 19.8.

The MGS procedure can be expressed in matrix terms by defining $A_k = [q_1, \dots, q_{k-1}, a_k^{(k)}, \dots, a_n^{(k)}]$. MGS transforms $A_1 = A$ into $A_{n+1} = Q$ by the sequence of transformations $A_k = A_{k+1} R_k$, where R_k is equal to the identity except in the k th row, where it agrees with the final R . For example, if $n = 4$ and $k = 3$,

$$A_3 = \begin{bmatrix} q_1 & q_2 & a_3^{(3)} & a_4^{(3)} \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & q_3 & a_4^{(4)} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & r_{33} & r_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} = A_4 R_3.$$

Thus $R = R_n \dots R_1$.

The Gram–Schmidt methods produce Q explicitly, unlike the Householder and Givens methods, which hold Q in factored form. While this is a benefit, in that no extra work is required to form Q , it is also a weakness, because there is nothing in the methods to force the computed $\hat{Q}$ to be orthonormal in the face of roundoff.

Orthonormality of Q is a consequence of orthogonality relations that are implicit in the methods, and these relations may be vitiated by rounding errors.

Some insight is provided by the case $n = 2$, for which the CGS and MGS methods are identical. Given $a_1, a_2 \in \mathbb{R}^m$ we compute $q_1 = a_1/\|a_1\|_2$, which we will suppose is done exactly, and then we form the unnormalized vector $q_2 = a_2 - (q_1^T a_2)q_1$. The computed vector satisfies

$$\hat{q}_2 = a_2 - q_1^T(a_2 + \Delta a_2)q_1 + \Delta \tilde{q}_2,$$

where

$$|\Delta a_2| \leq \gamma_m |a_2|, \quad |\Delta \tilde{q}_2| \leq u |a_2| + \gamma_2 |q_1^T(a_2 + \Delta a_2)| |q_1|.$$

Hence

$$\hat{q}_2 = q_2 + \Delta q_2, \quad |\Delta q_2| \leq \gamma_m |q_1^T| |a_2| |q_1| + u |a_2| + \gamma_2 (1 + \gamma_m) |q_1^T| |a_2| |q_1|,$$

and so the normalized inner product satisfies

$$\left| q_1^T \frac{\hat{q}_2}{\|\hat{q}_2\|_2} \right| \lesssim (m+2)u \frac{\|a_2\|_2}{\|q_2\|_2} = \frac{(m+2)u}{\sin \angle(a_1, a_2)}, \quad (19.26)$$

where $\angle(a_1, a_2)$ is the angle between a_1 and a_2 . But $\kappa_2(A) \geq \cot \angle(a_1, a_2)$, where $A = [a_1, a_2]$ (Problem 19.9). Hence, for $n = 2$, the loss of orthogonality can be bounded in terms of $\kappa_2(A)$. The same is true in general for the MGS method, as proved by Björck [119, 1967]. A direct proof is quite long and complicated, but a recent approach of Björck and Paige [131, 1992] enables a much shorter derivation; we take this approach here.

The observation that simplifies the error analysis of the MGS method is that the method is equivalent, both mathematically *and numerically*, to Householder QR factorization of the padded matrix $\begin{bmatrix} 0_n \\ A \end{bmatrix} \in \mathbb{R}^{(m+n) \times n}$. To understand this equivalence, consider the Householder QR factorization

$$P^T \begin{bmatrix} 0_n \\ A \end{bmatrix} = \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad P^T = P_n \dots P_2 P_1. \quad (19.27)$$

Let $q_1, \dots, q_n \in \mathbb{R}^m$ be the vectors obtained by applying the MGS method to A . Then it is easy to see that

$$P_1 = I - v_1 v_1^T, \quad v_1 = \begin{bmatrix} -e_1 \\ q_1 \end{bmatrix}, \quad v_1^T v_1 = 2$$

and that the multiplication $A_2 = P_1 \begin{bmatrix} 0_n \\ A \end{bmatrix}$ carries out the first step of the MGS method on A , producing the first row of R and $a_2^{(2)}, \dots, a_n^{(2)}$:

$$A_2 = \begin{matrix} & 1 & & \\ & & n-1 & \\ & & & m \end{matrix} \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1n} \\ 0 & 0 & \dots & 0 \\ 0 & a_2^{(2)} & \dots & a_n^{(2)} \end{bmatrix}.$$

The argument continues in the same way, and we find that

$$P_k = I - v_k v_k^T, \quad v_k = \begin{bmatrix} -e_k \\ q_k \end{bmatrix}, \quad v_k^T v_k = 2, \quad k = 2:n. \quad (19.28)$$

With the Householder–MGS connection established, we are ready to derive error bounds for the MGS method by making use of our existing error analysis for the Householder method.

Theorem 19.13. *Suppose the MGS method is applied to $A \in \mathbb{R}^{m \times n}$ of rank n , yielding computed matrices $\widehat{Q} \in \mathbb{R}^{m \times n}$ and $\widehat{R} \in \mathbb{R}^{n \times n}$. Then there are constants $c_i \equiv c_i(m, n)$ such that*

$$A + \Delta A_1 = \widehat{Q}\widehat{R}, \quad \|\Delta A_1\|_2 \leq c_1 u \|A\|_2, \quad (19.29)$$

$$\|\widehat{Q}^T \widehat{Q} - I\|_2 \leq c_2 u \kappa_2(A) + O((u \kappa_2(A))^2), \quad (19.30)$$

and there exists an orthonormal matrix Q such that

$$A + \Delta A_2 = Q\widehat{R}, \quad \|\Delta A_2(:, j)\|_2 \leq c_3 u \|a_j\|_2, \quad j = 1:n. \quad (19.31)$$

Proof. To prove (19.29) we use the matrix form of the MGS method. For the computed matrices we have

$$\widehat{A}_k = \widehat{A}_{k+1}\widehat{R}_k + \Delta_k, \quad |\Delta_k| \leq u|\widehat{A}_{k+1}||\widehat{R}_k|.$$

Expanding this recurrence, we obtain

$$A = \widehat{Q}\widehat{R} + \Delta_n \widehat{R}_{n-1} \dots \widehat{R}_1 + \Delta_{n-1} \widehat{R}_{n-2} \dots \widehat{R}_1 + \dots + \Delta_2 \widehat{R}_1 + \Delta_1.$$

Hence

$$|A - \widehat{Q}\widehat{R}| \leq u(|\widehat{A}_{n+1}||\widehat{R}_n| \dots |\widehat{R}_1| + \dots + |\widehat{A}_3||\widehat{R}_2||\widehat{R}_1| + |\widehat{A}_2||\widehat{R}_1|), \quad (19.32)$$

and a typical term has the form

$$|\widehat{A}_k||\widehat{R}_{k-1}| \dots |\widehat{R}_1| = |\widehat{q}_1 \dots \widehat{q}_{k-1} \widehat{a}_k^{(k)} \dots \widehat{a}_n^{(k)}| |S_{k-1}|, \quad (19.33)$$

where S_{k-1} agrees with $|\widehat{R}|$ in its first $k-1$ rows and the identity in its last $n-k+1$ rows. Assume for simplicity that $\|\widehat{q}_i\|_2 \equiv 1$ (this does not affect the final result). We have $\widehat{a}_j^{(k+1)} = (I - q_k q_k^T) \widehat{a}_j^{(k)}$, and Lemma 3.9 shows that the computed vector satisfies

$$\|\widehat{a}_j^{(k+1)}\|_2 \leq (1 + 2\gamma_{m+3}) \|\widehat{a}_j^{(k)}\|_2,$$

which implies $\|\widehat{A}_{k+1}\|_F \leq (1 + 2\gamma_{m+3})^k \|A\|_F$ and, from $\widehat{r}_{kj} = fl(\widehat{q}_k^T \widehat{a}_j^{(k)})$, we have $\|\widehat{R}\|_F \leq \sqrt{n}(1 + \gamma_m)(1 + 2\gamma_{m+3})^{n-1} \|A\|_F$. Using (19.32) and exploiting the form of (19.33) we find, after a little working, that

$$\|A - \widehat{Q}\widehat{R}\|_F \leq 4n^2 u \|A\|_F,$$

provided that $(1 + \gamma_m)(1 + 2\gamma_{m+3})^{n-1} < 2$.

To prove the last two parts of the theorem we exploit the Householder–MGS connection. By applying Theorem 19.4 to (19.27) we find that there is an orthogonal $\tilde{P}$ such that

$$\begin{bmatrix} \Delta A_3 \\ A + \Delta A_4 \end{bmatrix} = \tilde{P} \begin{bmatrix} \widehat{R} \\ 0 \end{bmatrix} = \begin{bmatrix} \tilde{P}_{11} \\ \tilde{P}_{21} \end{bmatrix} \widehat{R}, \quad (19.34)$$

with

$$\left\| \begin{bmatrix} \Delta A_3(:, j) \\ \Delta A_4(:, j) \end{bmatrix} \right\|_2 \leq \tilde{\gamma}_{mn} \|a_j\|_2, \quad j = 1:n.$$

This does not directly yield (19.31), since $\tilde{P}_{21}$ is not orthonormal. However, it can be shown that if we define Q to be the nearest orthonormal matrix to $\tilde{P}_{21}$ in the Frobenius norm, then (19.31) holds with $c_3 = (\sqrt{m} + 1)c_4$ (see Problem 19.12).

Now (19.29) and (19.31) yield $\hat{Q} - Q = (\Delta A_1 - \Delta A_2)\hat{R}^{-1}$, so

$$\|\hat{Q} - Q\|_2 \leq (c_1 + \sqrt{n}c_3)u\|A\|_2\|\hat{R}^{-1}\|_2 \leq \frac{c_5 u \kappa_2(A)}{1 - \sqrt{n}c_3 u \kappa_2(A)},$$

where $c_5 = c_1 + \sqrt{n}c_3$ and we have used (19.31) to bound $\|\hat{R}^{-1}\|_2$. This bound implies (19.30) with $c_2 = 2c_5$ (use the first inequality in Problem 19.14). $\square$

We note that (19.30) can be strengthened by replacing $\kappa_2(A)$ in the bound by the minimum over positive diagonal matrices D of $\kappa_2(AD)$. This follows from the observation that in the MGS method the computed $\hat{Q}$ is invariant under scalings $A \leftarrow AD$, at least if D comprises powers of the machine base. As a check, note that the bound in (19.26) for the case $n = 2$ is independent of the column scaling, since $\sin \angle(a_1, a_2)$ is.

Theorem 19.13 tells us three things. First, the computed QR factors from the MGS method have a small residual. Second, the departure from orthonormality of $\hat{Q}$ is bounded by a multiple of $\kappa_2(A)u$, so that $\hat{Q}$ is guaranteed to be nearly orthonormal if A is well conditioned. Finally, $\hat{R}$ is the exact triangular QR factor of a matrix near to A in a columnwise sense, so it is as good an R -factor as that produced by Householder QR factorization applied to A . In terms of the error analysis, the MGS method is weaker than Householder QR factorization *only* in that $\hat{Q}$ is not guaranteed to be nearly orthonormal.

For the CGS method the residual bound (19.29) still holds, but no bound of the form (19.30) holds for $n > 2$ (see Problem 19.10).

Here is a numerical example to illustrate the behaviour of the Gram–Schmidt methods. We take the 25×15 Vandermonde matrix $A = (p_i^{j-1})$, where the p_i are equally spaced on $[0, 1]$. The condition number $\kappa_2(A) = 1.5 \times 10^9$. We have

$$\begin{aligned} \text{CGS: } \|A - \hat{Q}\hat{R}\|_2 &= 5.0 \times 10^{-16}, \quad \|\hat{Q}^T \hat{Q} - I\|_2 = 5.2, \\ \text{MGS: } \|A - \hat{Q}\hat{R}\|_2 &= 1.0 \times 10^{-15}, \quad \|\hat{Q}^T \hat{Q} - I\|_2 = 9.5 \times 10^{-9}. \end{aligned}$$

Both methods produce a small residual for the QR factorization. While CGS produces a $\hat{Q}$ showing no semblance of orthogonality, for MGS we have $\|\hat{Q}^T \hat{Q} - I\|_2 \approx \kappa_2(A)u/17$.

19.9. Sensitivity of the QR Factorization

How do the QR factors of a matrix behave under small perturbations of the matrix? This question was first considered by Stewart [1068, 1977]. He showed that if $A \in \mathbb{R}^{m \times n}$ has rank n and

$$A = QR \quad \text{and} \quad A + \Delta A = (Q + \Delta Q)(R + \Delta R)$$

are QR factorizations, then, for sufficiently small ΔA ,

$$\frac{\|\Delta R\|_F}{\|R\|_F} \leq c_n \kappa_F(A) \frac{\|\Delta A\|_F}{\|A\|_F}, \quad (19.35a)$$

$$\|\Delta Q\|_F \leq c_n \kappa_F(A) \frac{\|\Delta A\|_F}{\|A\|_F}, \quad (19.35b)$$

where c_n is a constant. Here, and throughout this section, we use the “economy size” QR factorization with R a square matrix normalized to have nonnegative diagonal elements. Similar normwise bounds are given by Stewart [1075, 1993] and Sun [1102, 1991], and, for ΔQ only, by Bhatia and Mukherjea [108, 1994] and Sun [1106, 1995].

Columnwise sensitivity analyses have been given by Zha [1278, 1993] and Sun [1103, 1992], [1104, 1992]. Zha’s bounds can be summarized as follows, with the same assumptions and notation as for Stewart’s result above. Let $|\Delta A| \leq \epsilon G|A|$, where G is nonnegative with $\|G\|_2 = 1$. Then, for sufficiently small ϵ ,

$$\begin{aligned} \frac{\|\Delta R\|_2}{\|R\|_2} &\leq c_{m,n} \epsilon \operatorname{cond}(R^{-1}) + O(\epsilon^2), \\ \|\Delta Q\|_2 &\leq c_{m,n} \epsilon \operatorname{cond}(R^{-1}) + O(\epsilon^2), \end{aligned} \quad (19.36)$$

where $c_{m,n}$ is a constant depending on m and n . The quantity $\phi(A) = \operatorname{cond}(R^{-1}) = \| |R| |R^{-1}| \|_2$ can therefore be thought of as a condition number for the QR factorization under the columnwise class of perturbations considered. Note that ϕ is independent of the column scaling of A .

As an application of these bounds, consider a computed QR factorization $A \approx \widehat{Q}\widehat{R}$ obtained via the Householder algorithm, where $\widehat{Q}$ is the computed product of the computed Householder matrices, and let Q be the exact Q -factor of A . Theorem 19.4 shows that $A + \Delta A = \widehat{Q}\widehat{R}$ for an exactly orthogonal $\widehat{Q}$, with $|\Delta A| \leq \tilde{\gamma}_{mn} \epsilon e^T |A|$ (cf. §19.7). Moreover, we know from (19.13) that $\|\widehat{Q} - Q\|_F \leq \sqrt{n} \tilde{\gamma}_{mn}$. Now $Q - \widehat{Q} = (Q - \tilde{Q}) + (\tilde{Q} - \widehat{Q})$ and hence applying (19.36) to the first term we obtain

$$\|Q - \widehat{Q}\|_2 \leq c_{m,n} u \phi(A) + O(u^2). \quad (19.37)$$

In cases where a large $\kappa_F(A)$ is caused by poor column scaling, we can improve the bounds (19.35) by undoing the poor scaling to leave a well-conditioned matrix; the virtue of the columnwise analysis is that it does not require a judicious scaling in order to yield useful results.

19.10. Notes and References

The earliest appearance of Householder matrices is in the book by Turnbull and Aitken [1168, 1932, pp. 102–105]. These authors show that if $\|x\|_2 = \|y\|_2$ ($x \neq -y$) then a unitary matrix of the form $R = \alpha z z^* - I$ (in their notation) can be constructed so that $Rx = y$. They use this result to prove the existence of the Schur decomposition. The first systematic use of Householder matrices for computational purposes was by Householder [643, 1958], who used them to construct the QR factorization. Householder’s motivation was to compute the QR

factorization with fewer arithmetic operations (in particular, fewer square roots) than are required by the use of Givens rotations.

A detailed analysis of different algorithms for constructing a Householder matrix P such that $Px = \sigma e_1$ is given by Parlett [924, 1971].

Tsao [1161, 1975] describes an alternative way to form the product of a Householder matrix with a vector and gives an error analysis. There is no major advantage over the usual approach.

As for Householder matrices, normwise error analysis for Givens rotations was given by Wilkinson [1231, 1963], [1233, 1965, pp. 131–139]. Wilkinson analysed QR factorization by Givens rotations for square matrices [1233, 1965, pp. 240–241], and his analysis was extended to rectangular matrices by Gentleman [474, 1973]. The idea of exploiting disjoint rotations in the error analysis was developed by Gentleman [475, 1975], who gave a normwise analysis that is simpler and produces smaller bounds than Wilkinson's (our normwise bound in Theorem 19.10 is essentially the same as Gentleman's).

For more details of algorithmic and other aspects of Householder and Givens QR factorization, see Golub and Van Loan [509, 1996, §5].

The error analysis in §19.3 is a refined and improved version of analysis that appeared in the technical report [594, 1990] and was quoted without proof in Higham [596, 1991]. The analysis has been reworked for this edition of the book to emphasize the columnwise nature of the backward error bounds.

The need for row and column pivoting in Householder QR factorization for badly row scaled matrices was established by Powell and Reid [951, 1969] and was reported in Lawson and Hanson's 1974 book [775, pp. 103–106, 149]. Theorem 19.6 was originally proved under some additional assumptions by Powell and Reid [951, 1969]. The result as stated is proved by Cox and Higham [275, 1998]; it also follows from a more general result of Higham [614, 2000] that includes Theorems 19.4 and 19.6 as special cases. Björck [128, 1996, p. 169] conjectures that “there is no need to perform row pivoting in Householder QR, provided that the rows are sorted after decreasing row norm before the factorization”. This conjecture was proved by Cox and Higham [275, 1998], who also pointed out that row-wise backward stability is obtained for only one of the two possible choices of sign in the Householder vector.

The WY representation for a product of Householder transformations should not be confused with a genuine block Householder transformation. Schreiber and Parlett [1023, 1988] define, for a given $Z \in \mathbb{R}^{m \times n}$ ($m \geq n$), the “block reflector that reverses the range of Z ” as

$$H = I_m - ZWZ^T, \quad W = 2(Z^T Z)^+ \in \mathbb{R}^{n \times n}.$$

If $n = 1$ this is just a standard Householder transformation. A basic task is as follows: given $E \in \mathbb{R}^{m \times n}$ ($m > n$) find a block reflector H such that

$$HE = \begin{bmatrix} F \\ 0 \end{bmatrix}, \quad F \in \mathbb{R}^{n \times n}.$$

Schreiber and Parlett develop theory and algorithms for block reflectors, in both of which the polar decomposition plays a key role.

Sun and Bischof [1111, 1995] show that any orthogonal matrix can be expressed in the form $Q = I - YSY^T$, even with S triangular, and they explore the properties of this representation.

Another important use of Householder matrices, besides computation of the QR factorization, is to reduce a matrix to a simpler form prior to iterative computation of eigenvalues (Hessenberg or tridiagonal form) or singular values (bidiagonal form). For these two-sided transformations an analogue of Lemma 19.3 holds with normwise bounds (only) on the perturbation. Error analyses of two-sided application of Householder transformations is given by Ortega [907, 1963] and Wilkinson [1230, 1962], [1233, 1965, Chap. 6].

Mixed precision iterative refinement for solution of linear systems by Householder QR factorization is discussed by Wilkinson [1234, 1965, §10], who notes that convergence is obtained as long as a condition of the form $c_n \kappa_2(A)u < 1$ holds.

Fast Givens rotations can be applied to a matrix with half the number of multiplications of conventional Givens rotations, and they do not involve square roots. They were developed by Gentleman [474, 1973] and Hammarling [542, 1974]. Fast Givens rotations are as stable as conventional ones—see the error analysis by Parlett in [926, 1998, §6.8.3], for example—but, for the original formulations, careful monitoring is required to avoid overflow. Rath [973, 1982] investigates the use of fast Givens rotations for performing similarity transformations in solving the eigenproblem. Barlow and Ipsen [74, 1987] propose a class of scaled Givens rotations suitable for implementation on systolic arrays, and they give a detailed error analysis. Anda and Park [19, 1994] develop fast rotation algorithms that use dynamic scaling to avoid overflow.

Rice [984, 1966] was the first to point out that the MGS method produces a more nearly orthonormal matrix than the CGS method in the presence of rounding errors. Björck [119, 1967] gives a detailed error analysis, proving (19.29) and (19.30) but not (19.31), which is an extension of the corresponding normwise result of Björck and Paige [131, 1992]. Björck and Paige give a detailed assessment of MGS versus Householder QR factorization.

The difference between the CGS and MGS methods is indeed subtle. Wilkinson [1239, 1971] admitted that “I used the modified process for many years without even noticing explicitly that I was not performing the classical algorithm.”

The orthonormality of the matrix $\hat{Q}$ from Gram–Schmidt can be improved by reorthogonalization, in which the orthogonalization step of the classical or modified method is iterated. Analyses of Gram–Schmidt with reorthogonalization are given by Abdelmalek [2, 1971], Ruhe [996, 1983], and Hoffmann [634, 1989]. Daniel, Gragg, Kaufman, and Stewart [290, 1976] analyse the use of classical Gram–Schmidt with reorthogonalization for updating a QR factorization after a rank one change to the matrix.

The mathematical and numerical equivalence of the MGS method with Householder QR factorization of the matrix $\begin{bmatrix} 0_n \\ A \end{bmatrix}$ was known in the 1960s (see the Björck quotation at the start of the chapter) and the mathematical equivalence was pointed out by Lawson and Hanson [775, 1995, Ex. 19.39].

A block Gram–Schmidt method is developed by Jalby and Philippe [668, 1991] and error analysis given. See also Björck [127, 1994], who gives an up-to-date

survey of numerical aspects of the Gram–Schmidt method.

For more on Gram–Schmidt methods, including historical comments, see Björck [128, 1996].

More refined (and substantially more complicated) perturbation bounds for the QR factorization than those in §19.9 are given by Chang, Paige and Stewart [221, 1997] and Chang and Paige [219, 2001].

One use of the QR factorization is to orthogonalize a matrix that, because of rounding or truncation errors, has lost its orthogonality; thus we compute $A = QR$ and replace A by Q . An alternative approach is to replace $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) by the nearest orthonormal matrix, that is, the matrix Q that solves $\{\|A - Q\| : Q^T Q = I\} = \min$. For the 2- and Frobenius norms, the optimal Q is the orthonormal polar factor U of A , where $A = UH$ is a *polar decomposition*: $U \in \mathbb{R}^{m \times n}$ has orthonormal columns and $H \in \mathbb{R}^{n \times n}$ is symmetric positive semidefinite. If $m = n$, U is the nearest orthogonal matrix to A in any unitarily invariant norm, as shown by Fan and Hoffman [401, 1955]. Chandrasekaran and Ipsen [216, 1994] show that the QR and polar factors satisfy $\|A - Q\|_{2,F} \leq 5\sqrt{n}\|A - U\|_2$, under the assumptions that A has full rank and columns of unit 2-norm and that R has positive diagonal elements. Sun [1105, 1995] proves a similar result and also obtains a bound for $\|Q - U\|_F$ in terms of $\|A^T A - I\|_F$. Algorithms for maintaining orthogonality in long products of orthogonal matrices, which arise, for example, in subspace tracking problems in signal processing, are analysed by Edelman and Stewart [384, 1993] and Mathias [823, 1996].

Various iterative methods are available for computing the orthonormal polar factor U , and they can be competitive in cost with computation of a QR factorization. For more details on the theory and numerical methods, see Higham [578, 1986], [587, 1989], Higham and Papadimitriou [621, 1994], and the references therein.

A notable omission from this chapter is a treatment of rank-revealing QR factorizations—ones in which the rank of A can readily be determined from R . This topic is not one where rounding errors play a major role, and hence it is outside the scope of this book. Pointers to the literature include Golub and Van Loan [509, 1996, §5.4], Chan and Hansen [212, 1992], and Björck [128, 1996]. Column pivoting in the QR factorization ensures that if A has rank r then only the first r rows of R are nonzero (see Problem 19.5). A perturbation theorem for the QR factorization with column pivoting is given by Higham [588, 1990]; it is closely related to the perturbation theory in §10.3.1 for the Cholesky factorization of a positive semidefinite matrix.

19.10.1. LAPACK

LAPACK contains a rich selection of routines for computing and manipulating the QR factorization and its variants. Routine `xGEQRF` computes the QR factorization $A = QR$ of an $m \times n$ matrix A by the Householder QR algorithm. If $m < n$ (which we ruled out in our analysis, merely to simplify the notation), the factorization takes the form $A = Q[R_1 \ R_2]$, where R_1 is $m \times m$ upper triangular. The matrix Q is represented as a product of Householder transformations and is not formed explicitly. A routine `xORGQR` (or `xUNGQR` in the complex case) is provided to form

all or part of Q , and routine `xORMQR` (or `xUNMQR`) will pre- or postmultiply a matrix by Q or its (conjugate) transpose.

Routine `xGEQPF` computes the QR factorization with column pivoting.

An LQ factorization is computed by `xGELQF`. When A is $m \times n$ with $m \leq n$ it takes the form $A = [L \ 0] Q$. It is essentially the same as a QR factorization of A^T and hence can be used to find the minimum 2-norm solution to an underdetermined system (see §21.1).

LAPACK also computes two nonstandard factorizations of an $m \times n$ A :

$$\text{xGEQLF} : A = Q \begin{bmatrix} 0 \\ L \end{bmatrix}, \quad m \geq n, \quad \text{xGERQF} : A = [0 \ R] Q, \quad m \leq n,$$

where L is lower trapezoidal and R upper trapezoidal.

Problems

19.1. Find the eigenvalues of a Householder matrix and a Givens matrix.

19.2. Let $\hat{P} = I - \hat{\beta}\hat{v}\hat{v}^T$, where $\hat{\beta}$ and $\hat{v}$ are the computed quantities described in Lemma 19.1. Derive a bound for $\|\hat{P}^T \hat{P} - I\|_2$.

19.3. A complex Householder matrix has the form

$$P = I - \beta vv^*,$$

where $0 \neq v \in \mathbb{C}^n$ and $\beta = 2/v^*v$. For given $x, y \in \mathbb{C}^n$, show how to determine, if possible, P so that $Px = y$.

19.4. (Wilkinson [1233, 1965, p. 242]) Let $x \in \mathbb{R}^n$ and let P be a Householder matrix such that $Px = \pm\|x\|_2 e_1$. Let $G_{1,2}, \dots, G_{n-1,n}$ be Givens rotations such that $Qx := G_{1,2} \dots G_{n-1,n} x = \pm\|x\|_2 e_1$. True or false: $P = Q$?

19.5. Show that the R factor produced by QR factorization with column pivoting (see (19.15)) satisfies

$$r_{kk}^2 \geq \sum_{i=k}^j r_{ij}^2, \quad j = k+1:n, \quad k = 1:n,$$

so that, in particular, $|r_{11}| \geq |r_{22}| \geq \dots \geq |r_{nn}|$. (These are the same equations as (10.13), which hold for the Cholesky factorization with complete pivoting—why?)

19.6. (Higham [614, 2000]) Show that in Householder QR factorization applied to $A \in \mathbb{R}^{m \times n}$ the Householder vector v_k from the k th stage constructed according to (19.1) satisfies

$$\sqrt{2}\|a_k^{(k)}(k:m)\|_2 \leq \|v_k\|_2 \leq 2\|a_k^{(k)}(k:m)\|_2. \quad (19.38)$$

Consider now the computation of $\hat{a}_j^{(k+1)} = fl(\hat{P}_k \hat{a}_j^{(k)})$ for $j > k$, where $\hat{P}_k = I - \beta_k \hat{v}_k \hat{v}_k^T$ and $\hat{v}_k$ satisfies

$$\hat{v}_k = v_k + \Delta v_k, \quad |\Delta v_k| \leq \tilde{\gamma}_{m-k} |v_k|,$$

where

$$P_k = I - \beta_k v_k v_k^T$$

is the Householder matrix corresponding to the exact application of the k th stage of the algorithm to the computed matrix $\hat{A}^{(k)}$. Show that

$$\hat{a}_j^{(k+1)} = P_k \hat{a}_j^{(k)} + f_j^{(k)}, \quad (19.39)$$

where $f_j^{(k)}(1:k-1) = 0$ and

$$|f_j^{(k)}| \leq u|\hat{a}_j^{(k)}| + \tilde{\gamma}_{m-k} \frac{\|\hat{a}_j^{(k)}(k:m)\|_2}{\|\hat{a}_k^{(k)}(k:m)\|_2} |v_k|. \quad (19.40)$$

Explain the significance of this result for badly row-scaled problems.

19.7. Let $W \in \mathbb{R}^{m \times m}$ be a product of disjoint Givens rotations. Show that $\|W\|_2 \leq \sqrt{2}$.

19.8. The CGS method and the MGS method applied to $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) compute a QR factorization $A = QR$, $Q \in \mathbb{R}^{m \times n}$. Define the orthogonal projection $P_i = q_i q_i^T$, where $q_i = Q(:, i)$. Show that

$$(I - P_j)(I - P_{j-1}) \dots (I - P_1) = I - P_j - \dots - P_1.$$

Show that the CGS method corresponds to the operations

$$a_j \leftarrow (I - P_j - \dots - P_1)a_j, \quad j = 1:n,$$

while MGS corresponds to

$$a_j \leftarrow (I - P_j)(I - P_{j-1}) \dots (I - P_1)a_j, \quad j = 1:n.$$

19.9. Let $A = [a_1, a_2] \in \mathbb{R}^{m \times 2}$ and denote the angle between a_1 and a_2 by θ , $0 \leq \theta \leq \pi/2$. (Thus, $\cos \theta := |a_1^T a_2| / (\|a_1\|_2 \|a_2\|_2)$.) Show that

$$\kappa_2(A) \geq \frac{\max(\|a_1\|_2, \|a_2\|_2)}{\min(\|a_1\|_2, \|a_2\|_2)} \cot \theta.$$

19.10. (Björck [119, 1967]) Let

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \epsilon & 0 & 0 \\ 0 & \epsilon & 0 \\ 0 & 0 & \epsilon \end{bmatrix},$$

which is a matrix of the form discussed by Läuchli [772, 1961]. Assuming that $\epsilon/(1 + \epsilon^2) = 1$, evaluate the Q matrices produced by the CGS and MGS methods and assess their orthonormality.

19.11. Show that the matrix P in (19.27) has the form

$$P = \begin{bmatrix} 0_n & Q^T \\ Q & I - QQ^T \end{bmatrix},$$

where Q is the matrix obtained from the MGS method applied to A .

19.12. (Björck and Paige [131, 1992]) For any matrices satisfying

$$\begin{bmatrix} \Delta A_1 \\ A + \Delta A_2 \end{bmatrix} = \begin{bmatrix} P_{11} \\ P_{21} \end{bmatrix} R, \quad P_{11}^T P_{11} + P_{21}^T P_{21} = I,$$

where both P_{11} and P_{21} have at least as many rows as columns, show that there exists an orthonormal Q such that $A + \Delta A = QR$, where

$$\Delta A = F\Delta A_1 + \Delta A_2, \quad \|F\|_2 \leq 1.$$

(Hint: use the CS decomposition $P_{11} = UCW^T$, $P_{21} = VSW^T$, where U and V have orthonormal columns, W is orthogonal, and C and S are square, nonnegative diagonal matrices with $C^2 + S^2 = I$. Let $Q = VW^T$. Note, incidentally, that $P_{21} = VW^T \cdot WSW^T$, so $Q = VW^T$ is the orthonormal polar factor of P_{21} and hence is the nearest orthonormal matrix to P_{21} in the 2- and Frobenius norms. For details of the CS decomposition see Golub and Van Loan [509, 1996, §§2.6.4, 8.7.3] and Paige and Wei [913, 1994].)

19.13. We know that Householder QR factorization of $\begin{bmatrix} 0 \\ A \end{bmatrix}$ is equivalent to the MGS method applied to A , and Problem 19.11 shows that the orthonormal matrix Q from MGS is a submatrix of the orthogonal matrix P from the Householder method. Since Householder's method produces a nearly orthogonal P , does it not follow that MGS must also produce a nearly orthonormal Q ?

19.14. (Higham [603, 1994]) Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) have the polar decomposition $A = UH$. Show that

$$\frac{\|A^T A - I\|_2}{1 + \|A\|_2} \leq \|A - U\|_2 \leq \frac{\|A^T A - I\|_2}{1 + \sigma_{\min}(A)}.$$

This result shows that the two measures of orthonormality $\|A^T A - I\|_2$ and $\|A - U\|_2$ are essentially equivalent (cf. (19.30)).

Chapter 20

The Least Squares Problem

*For some time it has been believed that orthogonalizing methods
did not suffer this squaring of the condition number ...
It caused something of a shock, therefore,
when in 1966 Golub and Wilkinson ... asserted that
already the multiplications QA and Qb may produce errors in the solution
containing a factor $\chi^2(A)$.*

— A. VAN DER SLUIS,
Stability of the Solutions of Linear Least Squares Problems (1975)

*Most packaged regression problems do compute a cross-products matrix
and solve the normal equations using a matrix inversion subroutine.
All the programs ... that disagreed
(and some of those that agreed) with the unperturbed solution
tried to solve the normal equations.*

— ALBERT E. BEATON, DONALD B. RUBIN, and JOHN L. BARONE,
*The Acceptability of Regression Solutions:
Another Look at Computational Accuracy* (1976)

*On January 1, 1801 Giuseppe Piazzi discovered the asteroid Ceres.
Ceres was only visible for forty days
before it was lost to view behind the sun ...
Gauss, using three observations, extensive analysis,
and the method of least squares, was able to
determine the orbit with such accuracy that Ceres was
easily found when it reappeared in late 1801.*

— DAVID K. KAHANER, CLEVE B. MOLER, and STEPHEN G. NASH,
Numerical Methods and Software (1989)

In this chapter we consider the least squares (LS) problem $\min_x \|b - Ax\|_2$, where $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) has full rank. We begin by examining the sensitivity of the LS problem to perturbations. Then we examine the stability of methods for solving the LS problem, covering QR factorization methods, the normal equations and seminormal equations methods, and iterative refinement. Finally, we show how to compute the backward error of an approximate LS solution.

We do not develop the basic theory of the LS problem, which can be found in standard textbooks (see, for example, Golub and Van Loan [509, 1996, §5.3]). However, we recall the fundamental result that any solution of the LS problem satisfies the normal equations $A^T A x = A^T b$ (see Problem 20.1). Therefore if A has full rank there is a unique LS solution. More generally, whatever the rank of A the vector $x_{LS} = A^+ b$ is an LS solution, and it is the solution of minimal 2-norm. Here, A^+ is the pseudo-inverse of A (given by $A^+ = (A^T A)^{-1} A^T$ when A has full rank); see Problem 20.3. (For more on the pseudo-inverse see Stewart and Sun [1083, 1990, §3.1].)

20.1. Perturbation Theory

Perturbation theory for the LS problem is, not surprisingly, more complicated than for linear systems, and there are several forms in which bounds can be stated. We begin with a normwise perturbation theorem that is a restatement of a result of Wedin [1211, 1973, Thm. 5.1]. For an arbitrary rectangular matrix A we define the condition number $\kappa_2(A) = \|A\|_2 \|A^+\|_2$. If A has $r = \text{rank}(A)$ nonzero singular values, $\sigma_1 \geq \cdots \geq \sigma_r$, then $\kappa_2(A) = \sigma_1 / \sigma_r$.

Theorem 20.1 (Wedin). *Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) and $A + \Delta A$ both be of full rank, and let*

$$\begin{aligned} \|b - Ax\|_2 &= \min, & r &= b - Ax, \\ \|(b + \Delta b) - (A + \Delta A)y\|_2 &= \min, & s &= b + \Delta b - (A + \Delta A)y, \\ \|\Delta A\|_2 &\leq \epsilon \|A\|_2, & \|\Delta b\|_2 &\leq \epsilon \|b\|_2. \end{aligned}$$

Then, provided that $\kappa_2(A)\epsilon < 1$,

$$\frac{\|x - y\|_2}{\|x\|_2} \leq \frac{\kappa_2(A)\epsilon}{1 - \kappa_2(A)\epsilon} \left(2 + (\kappa_2(A) + 1) \frac{\|r\|_2}{\|A\|_2 \|x\|_2} \right), \quad (20.1)$$

$$\frac{\|r - s\|_2}{\|b\|_2} \leq (1 + 2\kappa_2(A))\epsilon. \quad (20.2)$$

These bounds are approximately attainable.

Proof. We defer a proof until §20.10, since the techniques used in the proof are not needed elsewhere in this chapter. $\square$

The bound (20.1) is usually interpreted as saying that the sensitivity of the LS problem is measured by $\kappa_2(A)$ when the residual is small or zero and by $\kappa_2(A)^2$ otherwise. This means that the sensitivity of the LS problem depends strongly on b as well as A , unlike for a square linear system.

Here is a simple example where the $\kappa_2(A)^2$ effect is seen:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & \delta \\ 0 & 0 \end{bmatrix}, \quad \Delta A = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & \delta/2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad \Delta b = 0.$$

It is a simple exercise to verify that

$$x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad r = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ \frac{2}{5\delta} \end{bmatrix}, \quad s = \begin{bmatrix} 0 \\ -\frac{2}{5} \\ \frac{4}{5} \end{bmatrix}.$$

Since $\kappa_2(A) = 1/\delta$,

$$\begin{aligned} \frac{\|x - y\|_2}{\|x\|_2} &= \frac{2}{5\delta} \lesssim \kappa_2(A)^2 \frac{\|\Delta A\|_2}{\|A\|_2} = \frac{1}{2\delta}, \\ \frac{\|r - s\|_2}{\|b\|_2} &= \frac{1}{\sqrt{5}} \lesssim \kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2} = \frac{1}{2}. \end{aligned}$$

Surprisingly, it is easier to derive componentwise perturbation bounds than normwise ones for the LS problem. The key idea is to express the LS solution and its residual as the solution of the augmented system

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \quad (20.3)$$

which is simply another way of writing the normal equations, $A^T A x = A^T b$. This is a square nonsingular system, so standard techniques can be applied. The perturbed system of interest is

$$\begin{bmatrix} I & A + \Delta A \\ (A + \Delta A)^T & 0 \end{bmatrix} \begin{bmatrix} s \\ y \end{bmatrix} = \begin{bmatrix} b + \Delta b \\ 0 \end{bmatrix}, \quad (20.4)$$

where we assume that

$$|\Delta A| \leq \epsilon E, \quad |\Delta b| \leq \epsilon f. \quad (20.5)$$

From (20.3) and (20.4) we obtain

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} s - r \\ y - x \end{bmatrix} = \begin{bmatrix} \Delta b - \Delta A y \\ -\Delta A^T s \end{bmatrix}.$$

Premultiplying by the inverse of the matrix on the left (obtainable from Problem 13.8) gives

$$\begin{bmatrix} s - r \\ y - x \end{bmatrix} = \begin{bmatrix} I - A A^+ & (A^+)^T \\ A^+ & -(A^T A)^{-1} \end{bmatrix} \begin{bmatrix} \Delta b - \Delta A y \\ -\Delta A^T s \end{bmatrix}. \quad (20.6)$$

Looking at the individual block components we obtain

$$|s - r| \leq \epsilon (|I - A^+ A|(f + E|y|) + |A^+|^T E^T |s|), \quad (20.7)$$

$$|y - x| \leq \epsilon (|A^+|(f + E|y|) + |(A^T A)^{-1}| E^T |s|). \quad (20.8)$$

(Note that $\|I - A A^+\|_2 = \min\{1, m - n\}$, as is easily proved using the SVD.) On taking norms we obtain the desired perturbation result.

Theorem 20.2. *Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) and $A + \Delta A$ be of full rank. For the perturbed LS problem described by (20.4) and (20.5) we have*

$$\frac{\|x - y\|}{\|x\|} \leq \epsilon \frac{\| |A^+| (f + E|y|) \| + \| |(A^T A)^{-1} |E^T| s \|}{\|x\|}, \quad (20.9)$$

$$\frac{\|r - s\|}{\|r\|} \leq \epsilon \frac{\| |I - AA^+| (f + E|y|) \| + \| |A^+|^T E^T |s| \|}{\|r\|}, \quad (20.10)$$

for any absolute norm. These bounds are approximately attainable. $\square$

For a square system, we have $s = 0$, and we essentially recover Theorem 7.4. Note, however, that the bounds contain the perturbed vectors y and s . For theoretical analysis it may be preferable to use alternative bounds in which x and r replace y and s and there is an extra factor

$$\left(1 - \epsilon \left\| \begin{bmatrix} |I - AA^+| & |A^+|^T \\ |A^+| & |(A^T A)^{-1}| \end{bmatrix} \begin{bmatrix} 0 & E \\ E^T & 0 \end{bmatrix} \right\| \right)^{-1},$$

where the term in parentheses is assumed to be positive. For practical computation (20.9) is unsatisfactory because we do not know $s = b + \Delta b - (A + \Delta A)y$. However, as Stewart and Sun observe [1083, 1990, p. 159], $\hat{r} = b - Ay$ is computable and

$$|s| \leq |\hat{r}| + \epsilon(f + E|y|),$$

and using this bound in (20.9) makes only a second-order change.

The componentwise bounds enjoy better scaling properties than the normwise ones. If $E = |A|$ and $f = |b|$ then the bounds (20.7) and (20.8), and to a lesser extent (20.9) and (20.10), are invariant under column scalings $b - Ax \rightarrow b - AD \cdot D^{-1}x$ (D diagonal). Row scaling does affect the componentwise bounds, since it changes the LS solution, but the componentwise bounds are less sensitive to the row scaling than the normwise bounds, in a way that is difficult to make precise.

20.2. Solution by QR Factorization

Let $A \in \mathbb{R}^{m \times n}$, with $m \geq n$ and $\text{rank}(A) = n$. If A has the QR factorization

$$Q^T A = \begin{bmatrix} R \\ 0 \end{bmatrix}$$

then

$$\begin{aligned} \|Ax - b\|_2^2 &= \|Q^T Ax - Q^T b\|_2^2 =: \left\| \begin{bmatrix} Rx - c \\ -d \end{bmatrix} \right\|_2^2 \\ &= \|Rx - c\|_2^2 + \|d\|_2^2. \end{aligned}$$

It follows that the unique LS solution is $x = R^{-1}c$, and the residual $\|b - Ax\|_2 = \|d\|_2$. Thus the LS problem can be solved with relatively little extra work beyond the computation of a QR factorization. Note that Q is not required explicitly;

we just need the ability to apply Q^T to a vector. The cost of the solution is $2n^2(m - n/3)$ flops if we use Householder QR factorization.

It is well known that the Givens and Householder QR factorization algorithms provide a normwise backward stable way to solve the LS problem. The next result expresses this fact for the Householder method and also provides columnwise backward error bounds (essentially the same result holds for the Givens method).

Theorem 20.3. *Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) have full rank and suppose the LS problem $\min_x \|b - Ax\|_2$ is solved using the Householder QR factorization method. The computed solution $\hat{x}$ is the exact LS solution to*

$$\min_x \|(b + \Delta b) - (A + \Delta A)\hat{x}\|_2,$$

where

$$\|\Delta a_j\|_2 \leq \tilde{\gamma}_{mn} \|a_j\|_2, \quad j = 1:n, \quad \|\Delta b\|_2 \leq \tilde{\gamma}_{mn} \|b\|_2.$$

Proof. The proof is a straightforward generalization of the proof of Theorem 19.5 and is left as an exercise (Problem 20.2). $\square$

As for Theorem 19.5 (see (19.14)), Theorem 20.3 remains true if we set $\Delta b \equiv 0$, but in general there is no advantage to restricting the perturbations to A .

Theorem 20.3 is a strong result, but it does not bound the residual of the computed solution, which, after all, is what we are trying to minimize. How close, then, is $\|b - A\hat{x}\|_2$ to $\min_x \|b - Ax\|_2$? We can answer this question using the perturbation theory of §20.1. With $\hat{r} := b + \Delta b - (A + \Delta A)\hat{x}$, $x := x_{LS}$, and $r := b - Ax$, (20.6) yields

$$\hat{r} - r = (I - AA^+)(\Delta b - \Delta A\hat{x}) - (A^+)^T \Delta A^T \hat{r},$$

so that

$$(b - A\hat{x}) - r = -AA^+(\Delta b - \Delta A\hat{x}) - (A^+)^T \Delta A^T \hat{r}.$$

Substituting the bounds for ΔA and Δb from Theorem 20.3, written in the form $|\Delta A| \leq \tilde{\gamma}_{mn} ee^T |A|$, $|\Delta b| \leq \tilde{\gamma}_{mn} ee^T |b|$, and noting that $\|AA^+\|_2 = 1$, we obtain

$$\begin{aligned} \|(b - A\hat{x}) - r\|_2 &\leq \tilde{\gamma}_{mn} (\|ee^T(|b| + |A||x|)\|_2 + \| |A^+|^T |A^T| ee^T |r| \|_2) + O(u^2) \\ &\leq m\tilde{\gamma}_{mn} (\| |b| + |A||x| \|_2 + \text{cond}_2(A^T) \|r\|_2) + O(u^2), \end{aligned}$$

where $\text{cond}_2(A) := \| |A^+| |A| \|_2$. Hence

$$\|b - A\hat{x}\|_2 \leq m\tilde{\gamma}_{mn} \| |b| + |A||x| \|_2 + (1 + m\tilde{\gamma}_{mn} \text{cond}_2(A^T)) \|r\|_2 + O(u^2).$$

This bound contains two parts. The term $m\tilde{\gamma}_{mn} \| |b| + |A||x| \|_2$ is a multiple of the bound for the error in evaluating $fl(b - Ax)$, and so is to be expected. The factor $1 + m\tilde{\gamma}_{mn} \text{cond}_2(A^T)$ will be less than 1.1 (say) provided that $\text{cond}_2(A^T)$ is not too large. Note that $\text{cond}_2(A^T) \leq n\kappa_2(A)$ and $\text{cond}_2(A^T)$ is invariant under column scaling of A ($A \leftarrow A \text{diag}(d_i)$, $d_i \neq 0$). The conclusion is that, unless A is very ill conditioned, the residual $b - A\hat{x}$ will not exceed the larger of the true residual $r = b - Ax$ and a constant multiple of the error in evaluating $fl(r)$ —a very satisfactory result.

20.3. Solution by the Modified Gram–Schmidt Method

The modified Gram–Schmidt (MGS) method can be used to solve the LS problem. However, we must not compute x from $x = R^{-1}(Q^T b)$, because the lack of orthonormality of the computed $\hat{Q}$ would adversely affect the stability. Instead we apply MGS to the augmented matrix $[A \ b]$:

$$[A \ b] = [Q_1 \ q_{n+1}] \begin{bmatrix} R & z \\ 0 & \rho \end{bmatrix}.$$

We have

$$\begin{aligned} Ax - b &= [A \ b] \begin{bmatrix} x \\ -1 \end{bmatrix} = [Q_1 \ q_{n+1}] \begin{bmatrix} Rx - z \\ -\rho \end{bmatrix} \\ &= Q_1(Rx - z) - \rho q_{n+1}. \end{aligned}$$

Since q_{n+1} is orthogonal to the columns of Q_1 , $\|b - Ax\|_2^2 = \|Rx - z\|_2^2 + \rho^2$, so the LS solution is $x = R^{-1}z$. Of course, $z = Q_1^T b$, but z is now computed as part of the MGS procedure instead of as a product between Q^T and b .

Björck [119, 1967] showed that this algorithm is forward stable, in the sense that the forward error $\|x - \hat{x}\|_2 / \|x\|_2$ is as small as that for a normwise backward stable method. It was subsequently shown by Björck and Paige [131, 1992] that the algorithm is, in fact, normwise backward stable (see also Björck [127, 1994]), that is, a normwise result of the form in Theorem 20.3 holds. Moreover, a columnwise result of the form in Theorem 20.3 holds too—see Problem 20.5. Hence the possible lack of orthonormality of $\hat{Q}$ does not impair the stability of the MGS method as a means for solving the LS problem.

20.4. The Normal Equations

The oldest method of solving the LS problem is to form and solve the *normal equations*, $A^T A x = A^T b$. Assuming that A has full rank, we can use the following procedure:

Form $C = A^T A$ and $c = A^T b$.

Compute the Cholesky factorization $C = R^T R$.

Solve $R^T y = c$, $Rx = y$.

Cost: $n^2(m + n/3)$ flops.

If $m \gg n$, the normal equations method requires about half as many flops as the Householder QR factorization approach (or the MGS method). However, it has less satisfactory numerical stability properties. There are two problems. The first is that information may be lost when $\hat{C} = fl(A^T A)$ is formed—essentially because forming the cross product is a squaring operation that increases the dynamic range of the data. A simple example is the matrix

$$A = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \end{bmatrix}, \quad 0 < \epsilon < \sqrt{u},$$

for which

$$A^T A = \begin{bmatrix} 1 + \epsilon^2 & 1 \\ 1 & 1 \end{bmatrix}, \quad fl(A^T A) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Even though A is distance approximately ϵ from a rank-deficient matrix, and hence unambiguously full rank if $\epsilon \approx \sqrt{u/2}$, the computed cross product is singular. In general, whenever $\kappa_2(A) \geq u^{-1/2}$ we can expect $\hat{C}$ to be singular or indefinite, in which case Cholesky factorization is likely to break down (Theorem 10.7).

The second weakness of the normal equations method is more subtle and is explained by a rounding error analysis. In place of $C = A^T A$ and $c = A^T b$ we compute

$$\begin{aligned} \hat{C} &= A^T A + \Delta C_1, & |\Delta C_1| &\leq \gamma_m |A^T| |A|, \\ \hat{c} &= A^T b + \Delta c, & |\Delta c| &\leq \gamma_m |A^T| |b|. \end{aligned}$$

By Theorems 10.3 and 10.4, the computed Cholesky factor $\hat{R}$ and solution $\hat{x}$ satisfy

$$\begin{aligned} \hat{R}^T \hat{R} &= \hat{C} + \Delta C_2, & |\Delta C_2| &\leq \gamma_{n+1} |\hat{R}^T| |\hat{R}|, \\ (\hat{C} + \Delta C_3) \hat{x} &= \hat{c}, & |\Delta C_3| &\leq \gamma_{3n+1} |\hat{R}^T| |\hat{R}|. \end{aligned} \quad (20.11)$$

Overall,

$$\begin{aligned} (A^T A + \Delta A) \hat{x} &= A^T b + \Delta c, & (20.12) \\ |\Delta A| &\leq \gamma_m |A^T| |A| + \gamma_{3n+1} |\hat{R}^T| |\hat{R}|, & |\Delta c| &\leq \gamma_m |A^T| |b|. \end{aligned}$$

By bounding $\| |\hat{R}^T| |\hat{R}| \|_2$ with the aid of (20.11), we find that

$$\|\Delta A\|_2 \leq (mn + 3n^2 + n)u \|A\|_2^2 + O(u^2), \quad (20.13a)$$

$$\|\Delta c\|_2 \leq mn^{1/2}u \|A\|_2 \|b\|_2 + O(u^2). \quad (20.13b)$$

These bounds show that we have solved the normal equations in a backward stable way, as long as $\|A\|_2 \|b\|_2 \approx \|A^T b\|_2$. But if we try to translate this result into a backward error result for the LS problem itself, we find that the best backward error bound contains a factor $\kappa_2(A)$ [623, 1987]. The best forward error bound we can expect, in view of (20.13), is of the form

$$\frac{\|x - \hat{x}\|_2}{\|x\|_2} \lesssim c_{m,n} \kappa_2(A)^2 u \quad (20.14)$$

(since $\kappa_2(A^T A) = \kappa_2(A)^2$). Now we know from Theorem 20.1 that the sensitivity of the LS problem is measured by $\kappa_2(A)^2$ if the residual is large, but by $\kappa_2(A)$ if the residual is small. It follows that the normal equations method has a forward error bound that can be much larger than that possessed by a backward stable method.

A mitigating factor for the normal equations method is that, in view of Theorem 10.6, we can replace (20.14) by the (not entirely rigorous) bound

$$\frac{\|D(x - \hat{x})\|_2}{\|Dx\|_2} \lesssim c_{m,n} \kappa_2(B)^2 u,$$

where $A = BD$, with $D = \text{diag}(\|A(:, i)\|_2)$, so that B has columns of unit 2-norm. Van der Sluis's result (Theorem 7.5) shows that

$$\kappa_2(B) \leq \sqrt{n} \min_F \min_{\text{diagonal}} \kappa_2(AF).$$

Hence the normal equations method is, to some extent, insensitive to poor column scaling of A .

Although numerical analysts almost invariably solve the full rank LS problem by QR factorization, statisticians frequently use the normal equations (though perhaps less frequently than they used to, thanks to the influence of numerical analysts). The normal equations do have a useful role to play. In many statistical problems the regression matrix is contaminated by errors of measurement that are very large relative to the roundoff level; the effects of rounding errors are then likely to be insignificant compared with the effects of the measurement errors, especially if IEEE double precision (as opposed to single precision) arithmetic is used.

The normal equations (NE) versus (Householder) QR factorization debate can be summed up as follows.

- The two methods have a similar computational cost if $m \approx n$, but the NE method is up to twice as fast for $m \gg n$. (This statement assumes that A and b are dense; for details of the storage requirements and computational cost of each method for sparse matrices, see, for example, Björck [128, 1996] and Heath [556, 1984].)
- The QR method is always backward stable. The NE method is guaranteed to be backward stable only if A is well conditioned.
- The forward error from the NE method can be expected to exceed that for the QR method when A is ill conditioned *and* the residual of the LS problem is small.
- The QR method lends itself to iterative refinement, as described in the next section. Iterative refinement can be applied to the NE method, but the rate of convergence inevitably depends on $\kappa_2(A)^2$ instead of $\kappa_2(A)$.

20.5. Iterative Refinement

As for square linear systems, iterative refinement can be used to improve the accuracy and stability of an approximate LS solution. However, for the LS problem there is more than one way to construct an iterative refinement scheme.

By direct analogy with the square system case, we might consider the scheme

1. Compute $r = b - A\hat{x}$.
2. Solve the LS problem $\min_d \|Ad - r\|_2$.
3. Update $y = \hat{x} + d$.

(Repeat from step 1 if necessary, with $\hat{x}$ replaced by y .)

This scheme is certainly efficient—a computed QR factorization (for example) of A can be reused at each step 2. Golub and Wilkinson [505, 1966] investigated this scheme and found that it works well only for nearly consistent systems.

An alternative approach suggested by Björck [118, 1967] is to apply iterative refinement to the augmented system (20.3), so that both x and r are refined simultaneously. Since this is a square, nonsingular system, existing results on the convergence and stability of iterative refinement can be applied, and we would expect the scheme to work well. To make precise statements we need to examine the augmented system method in detail.

For the refinement steps we need to consider an augmented system with an arbitrary right-hand side:

$$r + Ax = f, \quad (20.15a)$$

$$A^T r = g. \quad (20.15b)$$

If A has the QR factorization

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix},$$

where $R \in \mathbb{R}^{n \times n}$, then (20.15) transforms to

$$Q^T r + \begin{bmatrix} R \\ 0 \end{bmatrix} x = Q^T f,$$

$$\begin{bmatrix} R^T & 0 \end{bmatrix} Q^T r = g.$$

This system can be solved as follows:

$$h = R^{-T} g,$$

$$d = Q^T f = \begin{bmatrix} d_1 \\ d_2 \end{bmatrix},$$

$$r = Q \begin{bmatrix} h \\ d_2 \end{bmatrix},$$

$$x = R^{-1}(d_1 - h).$$

The next result describes the effect of rounding errors on the solution process. The bounds are columnwise, but we state them as componentwise inequalities for convenience.

Theorem 20.4. *Let $A \in \mathbb{R}^{m \times n}$ be of full rank $n \leq m$ and suppose the augmented system (20.15) is solved using a Householder QR factorization of A as described above. The computed $\hat{x}$ and $\hat{r}$ satisfy*

$$\begin{bmatrix} I & A + \Delta A_1 \\ (A + \Delta A_2)^T & 0 \end{bmatrix} \begin{bmatrix} \hat{r} \\ \hat{x} \end{bmatrix} = \begin{bmatrix} f + \Delta f \\ g + \Delta g \end{bmatrix},$$

where

$$|\Delta A_i| \leq mn\tilde{\gamma}_m G|A|, \quad i = 1:2,$$

$$|\Delta f| \leq m^{1/2}n\tilde{\gamma}_m(H_1|f| + H_2|\hat{r}|),$$

$$|\Delta g| \leq m^{1/2}n\tilde{\gamma}_m|A^T|H_3|\hat{r}|,$$

with $\|G\|_F = 1$, $\|H_i\|_F = 1$, $i = 1:3$.

Proof. The proof does not involve any new ideas and is rather tedious, so we omit it. $\square$

Consider first fixed precision iterative refinement. Theorem 20.4 implies that the computed solution $(\hat{r}^T, \hat{x}^T)^T$ to (20.15) satisfies

$$\left| \begin{bmatrix} f \\ g \end{bmatrix} - \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} \hat{r} \\ \hat{x} \end{bmatrix} \right| \leq mn\tilde{\gamma}_m \left(\begin{bmatrix} H_2 & G|A| \\ |A^T|(G^T + H_3) & 0 \end{bmatrix} \begin{bmatrix} |\hat{r}| \\ |\hat{x}| \end{bmatrix} + \begin{bmatrix} H_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} |f| \\ |g| \end{bmatrix} \right).$$

Unfortunately, this bound is not of a form that allows us to invoke Theorem 12.4. However, we can apply Theorem 12.3, which tells us that the corrected solution $(\hat{s}^T, \hat{y}^T)^T$ obtained after one step of iterative refinement satisfies

$$\begin{aligned} \left| \begin{bmatrix} b \\ 0 \end{bmatrix} - \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} \hat{s} \\ \hat{y} \end{bmatrix} \right| &\leq mn\tilde{\gamma}_m \begin{bmatrix} H_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} |\hat{r}_1| \\ |\hat{r}_2| \end{bmatrix} + u \begin{bmatrix} I & |A| \\ |A^T| & 0 \end{bmatrix} \begin{bmatrix} |\hat{s}| \\ |\hat{y}| \end{bmatrix} \\ &+ \gamma_{m+n+1} \left(\begin{bmatrix} I & |A| \\ |A^T| & 0 \end{bmatrix} \begin{bmatrix} |\hat{s}| \\ |\hat{y}| \end{bmatrix} + \begin{bmatrix} |b| \\ 0 \end{bmatrix} \right) \\ &+ O(u^2). \end{aligned} \quad (20.16)$$

Here, $(\hat{r}_1^T, \hat{r}_2^T)^T$ denotes the residual of the augmented system corresponding to the original computed solution. We will make two simplifications to the bound (20.16). First, since $(\hat{r}_1^T, \hat{r}_2^T)^T = O(u)$, the first term in the bound may be included in the $O(u^2)$ term. Second, (20.16) yields $|b - \hat{s} - A\hat{y}| = O(u)$ and so $|\hat{s}| \leq |A||\hat{y}| + |b| + O(u)$. With these two simplifications, (20.16) may be written

$$\left| \begin{bmatrix} b \\ 0 \end{bmatrix} - \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} \hat{s} \\ \hat{y} \end{bmatrix} \right| \leq 2\gamma_{m+n+2} \left(\begin{bmatrix} 0 & |A| \\ |A^T| & 0 \end{bmatrix} \begin{bmatrix} |\hat{s}| \\ |\hat{y}| \end{bmatrix} + \begin{bmatrix} |b| \\ 0 \end{bmatrix} \right) + O(u^2).$$

In view of the Oettli–Prager result (Theorem 7.3) this inequality tells us that, asymptotically, the solution $\hat{y}$ produced after one step of fixed precision iterative refinement has a small componentwise relative backward error with respect to the augmented system. However, this backward error allows the two occurrences of A in the augmented system coefficient matrix to be perturbed differently, and thus is not a true componentwise backward error for the LS problem. Nevertheless, the result tells us that iterative refinement can be expected to produce some improvement in stability. Note that the bound in Theorem 20.2 continues to hold if we perturb the two occurrences of A in the augmented system differently. Therefore the bound is applicable to iterative refinement (with $E = |A|$, $f = |b|$), and so we can expect iterative refinement to mitigate the effects of poor row and column scaling of A . Numerical experiments show that these predictions are borne out in practice [596, 1991].

Turning to mixed precision iterative refinement, we would like to apply the analysis of §12.1, with “ $Ax = b$ ” again identified with the augmented system. However, the analysis of §12.1 requires a backward error result in which only the

coefficient matrix is perturbed (see (12.1)). This causes no difficulties because from Theorem 20.4 we can deduce a normwise result (cf. Problem 7.7):

$$\left(\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} + \Delta \right) \begin{bmatrix} \hat{r} \\ \hat{x} \end{bmatrix} = \begin{bmatrix} f \\ g \end{bmatrix}, \quad \|\Delta\|_2 \leq c_{m,n} u \left\| \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \right\|_2.$$

The theory of §12.1 then tells us that mixed precision iterative refinement will converge as long as the condition number of the augmented system matrix is not too large and that the rate of convergence depends on this condition number. How does the condition number of the augmented system relate to that of A ? Consider the matrix that results from the scaling $A \leftarrow \alpha^{-1} A$ ($\alpha > 0$):

$$C(\alpha) = \begin{bmatrix} \alpha I & A \\ A^T & 0 \end{bmatrix}. \quad (20.17)$$

Björck [118, 1967] shows that the eigenvalues of $C(\alpha)$ are

$$\lambda(C(\alpha)) = \begin{cases} \frac{\alpha}{2} \pm \left(\frac{\alpha^2}{4} + \sigma_i^2 \right)^{1/2}, & i = 1:n, \\ \alpha, & m - n \text{ times,} \end{cases} \quad (20.18)$$

where σ_i , $i = 1:n$, are the singular values of A , and that

$$\sqrt{2}\kappa_2(A) \leq \min_{\alpha} \kappa_2(C(\alpha)) \leq 2\kappa_2(A), \quad \max_{\alpha} \kappa_2(C(\alpha)) > \kappa_2(A)^2, \quad (20.19)$$

with $\min_{\alpha} \kappa_2(C(\alpha))$ being achieved for $\alpha = \sigma_n/\sqrt{2}$ (see Problem 20.7). Hence $C(\alpha)$ may be much more ill conditioned than A . However, in our analysis we are at liberty to take $\min_{\alpha} \kappa_2(C(\alpha))$ as the condition number, because scaling the LS problem according to $b - Ax \leftarrow (b - Ax)/\alpha$ does not change the computed solution or the rounding errors in any way (at least not if α is a power of the machine base). Therefore it is $\kappa_2(A)$ that governs the behaviour of mixed precision iterative refinement, irrespective of the size of the LS residual. As Björck [123, 1990] explains, this means that “in a sense iterative refinement is *even more satisfactory* for large residual least-squares problems.” He goes on to explain that “When residuals to the augmented system are accumulated in precision β^{-t_2} , $t_2 \geq 2t_1$, this scheme gives solutions to full single-precision accuracy even though the initial solution may have *no correct significant figures*.”

Iterative refinement can be applied with the MGS method. Björck [118, 1967] gives the details and shows that mixed precision refinement works just as well as it does for Householder’s method.

20.6. The Seminormal Equations

When we use a QR factorization to solve an LS problem $\min_x \|b - Ax\|_2$, the solution x is determined from the equation $Rx = Q^T b$ (or via a similar expression involving Q for the MGS method). But if we need to solve for several right-hand sides that are not all available when the QR factorization is computed, we need to store Q before applying it to the right-hand sides. If A is large and sparse it is

undesirable to store Q , as it can be much more dense than A . We can, however, rewrite the normal equations as

$$R^T R x = A^T b,$$

which are called the *seminormal equations*. The solution x can be determined from these equations without the use of Q . Since the cross product matrix $A^T A$ is not formed explicitly and R is determined stably via a QR factorization, we might expect this approach to be more stable than the normal equations method.

Björck [122, 1987] has done a detailed error analysis of the seminormal equations (SNE) method, under the assumption that R is computed by a backward stable method. His forward error bound for the SNE method is of the same form as that for the normal equations method, involving a factor $\kappa_2(A)^2$. Thus the SNE method is not backward stable. Björck considers applying one step of fixed precision iterative refinement to the SNE method, and he calls the resulting process the corrected seminormal equations (CSNE) method:

$$\begin{aligned} R^T R x &= A^T b \\ r &= b - Ax \\ R^T R w &= A^T r \\ y &= x + w \end{aligned}$$

It is important that the normal equations residual be computed as shown, as $A^T(b - Ax)$, and not as $A^T b - A^T Ax$. Björck derives a forward error bound for the CSNE method that is roughly of the form

$$\frac{\|x - y\|_2}{\|x\|_2} \leq c_{m,n} \left(\kappa_2(A)u \cdot \kappa_2(A)^2 u \left(1 + \frac{\|b\|_2}{\|A\|_2 \|x\|_2} \right) + \frac{\kappa_2(A)^2 u \|r\|_2}{\|A\|_2 \|x\|_2} \right).$$

Hence, if $\kappa_2(A)^2 u \leq 1$, the CSNE method has a forward error bound similar to that for a backward stable method, and the bound is actually smaller than that for the QR method if $\kappa_2(A)^2 u \ll 1$ and r is small. However, the CSNE method is not backward stable for all A .

20.7. Backward Error

Although it has been known since the 1960s that a particular method for solving the LS problem, namely the Householder QR factorization method, yields a small normwise backward error (see §20.2), it was for a long time an open problem to obtain a formula for the backward error of an arbitrary approximate solution. Little progress had been made towards solving this problem until Waldén, Karlson, and Sun [1203, 1995] found an extremely elegant solution. We will denote by $\lambda_{\min}$ and $\sigma_{\min}$ the smallest eigenvalue of a symmetric matrix and the smallest singular value of a general matrix, respectively.

Theorem 20.5 (Waldén, Karlson, and Sun). *Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$), $b \in \mathbb{R}^m$, $0 \neq y \in \mathbb{R}^n$, and $r = b - Ay$. The normwise backward error*

$$\eta_F(y) := \min\{ \|\Delta A, \theta \Delta b\|_F : \|(A + \Delta A)y - (b + \Delta b)\|_2 = \min \} \quad (20.20)$$

is given by

$$\eta_F(y) = \begin{cases} \frac{\|r\|_2}{\|y\|_2} \sqrt{\mu}, & \lambda_* \geq 0, \\ \left(\frac{\|r\|_2^2}{\|y\|_2^2} \mu + \lambda_* \right)^{1/2}, & \lambda_* < 0, \end{cases}$$

where

$$\lambda_* = \lambda_{\min} \left(AA^T - \mu \frac{rr^T}{\|y\|_2^2} \right), \quad \mu = \frac{\theta^2 \|y\|_2^2}{1 + \theta^2 \|y\|_2^2}. \quad \square$$

The backward error (20.20) is not a direct generalization of the usual normwise backward error for square linear systems, because it minimizes $\|[\Delta A, \theta \Delta b]\|_F$ instead of $\max\{\|\Delta A\|_2/\|E\|_2, \|\Delta b\|_2/\|f\|_2\}$. However, the parameter θ allows us some flexibility: taking the limit $\theta \rightarrow \infty$ forces $\Delta b = 0$, giving the case where only A is perturbed.

Theorem 20.5 can be interpreted as saying that if $\lambda_* \geq 0$ then the backward error is the same as that given in Problem 7.8 for a consistent linear system. If $\lambda_* < 0$, however, the nearest perturbed system of which y is the LS solution is inconsistent. A sufficient condition for $\lambda_* < 0$ is $b \notin \text{range}(A)$ (assuming $\mu \neq 0$), that is, the original system is inconsistent.

The formulae given in Theorem 20.5 are unsuitable for computation because they can suffer from catastrophic cancellation when $\lambda_* < 0$. Instead, the following alternative formula derived in [1203, 1995] should be used (see Problem 20.9):

$$\eta_F(y) = \min\{ \phi, \sigma_{\min}([A \quad \phi(I - rr^+)]) \}, \quad \phi = \sqrt{\mu} \frac{\|r\|_2}{\|y\|_2}. \quad (20.21)$$

To illustrate Theorem 20.5, we consider an LS problem with a 25×15 Vandermonde matrix $A = (p_j^{i-1})$, where the p_i are equally spaced on $[0, 1]$, and $b = Ax$ with $x_i = i$ (giving a zero residual problem). The condition number $\kappa_2(A) = 1.47 \times 10^9$. We solved the LS problem in MATLAB in four different ways: by using the NE with Cholesky factorization, via Householder QR factorization, and via the MGS method, using both the stable approach described in §20.3 and the unstable approach in which $Q^T b$ is formed as a matrix–vector product (denoted MGS(bad)). The results, including the norms of the residuals $r = b - A\hat{x}$, are shown in Table 20.1. As would be expected from the analysis in this chapter, the QR and stable MGS methods produce backward stable solutions, but the NE method and the unstable MGS approach do not.

As we saw in the analysis of iterative refinement, sometimes we need to consider the augmented system with different perturbations to A and A^T . The next result shows that from the point of view of normwise perturbations and columnwise perturbations the lack of “symmetry” in the perturbations has little effect on the backward error of y .

Lemma 20.6 (Kielbasiński and Schwetlick). *Let $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) and consider the perturbed augmented system*

$$\begin{bmatrix} I & A + \Delta A_1 \\ (A + \Delta A_2)^T & 0 \end{bmatrix} \begin{bmatrix} s \\ y \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}.$$

Table 20.1. *LS backward errors and residual for Vandermonde system.*

	$\frac{\eta_F}{\ [A, b]\ _F} (\theta = 1)$	$\frac{\eta_F}{\ A\ _F} (\theta = \infty)$	$\ r\ _2$
NE	3.1×10^{-11}	8.6×10^{-10}	2.4×10^{-7}
QR	4.7×10^{-18}	1.3×10^{-16}	3.4×10^{-14}
MGs	5.6×10^{-18}	1.6×10^{-16}	4.0×10^{-14}
MGs(bad)	2.0×10^{-11}	5.5×10^{-10}	4.4×10^{-7}

There is a vector $\hat{s}$ and a perturbation ΔA with

$$\Delta A = G_1 \Delta A_1 + G_2 \Delta A_2, \quad G_1^T = G_1 = G_2^T, \quad G_1 + G_2 = I, \quad (20.22)$$

such that

$$\begin{bmatrix} I & A + \Delta A \\ (A + \Delta A)^T & 0 \end{bmatrix} \begin{bmatrix} \hat{s} \\ y \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix},$$

that is, y solves the LS problem $\min_x \|(A + \Delta A)x - b\|_2$.

Proof. If $s = b - (A + \Delta A_1)y = 0$ we take $\Delta A = \Delta A_1$. Otherwise, we set

$$\Delta A := P \Delta A_2 + (I - P) \Delta A_1 =: \Delta A_1 + PH,$$

where $P = ss^T / s^T s$ and $H = \Delta A_2 - \Delta A_1$. We have

$$\hat{s} := b - (A + \Delta A)y = b - (A + \Delta A_1)y - PHy = \beta s,$$

where $\beta = 1 - s^T Hy / s^T s$. Then

$$\begin{aligned} (A + \Delta A)^T \hat{s} &= (A + \Delta A_2 + (P - I)H)^T \beta s \\ &= \beta ((A + \Delta A_2)^T s + H^T (P - I)s) = 0. \quad \square \end{aligned}$$

Note that (20.22) implies a bound stronger than just $\|\Delta A\|_2 \leq \|\Delta A_1\|_2 + \|\Delta A_2\|_2$:

$$\|\Delta A\|_p \leq (\|\Delta A_1\|_p^2 + \|\Delta A_2\|_p^2)^{1/2}, \quad p = 2, F.$$

Turning to componentwise backward error, the simplest approach is to apply the componentwise backward error $\omega_{E,f}(y)$ to the augmented system (20.3), setting

$$E = \begin{bmatrix} 0 & E_A \\ E_A^T & 0 \end{bmatrix}$$

so as not to perturb the diagonal blocks I and 0 of the augmented system coefficient matrix. However, this approach allows A and A^T to undergo different perturbations ΔA_1 and ΔA_2 with $\Delta A_1 \neq \Delta A_2^T$ and thus does not give a true backward error, and Lemma 20.6 is of no help. This problem can be overcome by using a structured componentwise backward error to force symmetry of the perturbations; see Higham and Higham [574, 1992] for details. One problem remains: as far as the backward error of y is concerned, the vector r in the augmented system

is a vector of free parameters, so to obtain the true componentwise backward error we have to minimize the structure-preserving componentwise backward error over all r . This is a nonlinear optimization problem to which no closed-form solution is known. Experiments show that when y is a computed LS solution, $r = b - Ay$ is often a good approximation to the minimizing r [574, 1992], [596, 1991].

20.8. Weighted Least Squares Problems

In many applications of the LS problem the rows of A have widely varying norms (with corresponding variation in the size of the elements of b), typically because in the underlying model some observations have been given greater weight than others. Such weighted LS problems also arise when the method of weighting is used to approximate the solution to the linearly constrained LS problem; see §20.9.2.

As we saw in §19.4, Householder QR factorization needs to be implemented with column pivoting together with row pivoting or row sorting, *and* with the correct choice of sign in the Householder vectors, if a row-wise backward stable QR factorization is to be obtained. The following result describes the row-wise stability of Householder QR factorization for solving the LS problem.

Theorem 20.7 (Powell and Reid; Cox and Higham). *Suppose the LS problem $\min_x \|b - Ax\|_2$, where $A \in \mathbb{R}^{m \times n}$ is of full rank n , is solved using Householder QR factorization with column pivoting, with the choice of sign (19.1). Then the computed solution $\hat{x}$ is the exact solution to*

$$\min_x \|(b + \Delta b) - (A + \Delta A)x\|_2,$$

where the perturbations satisfy

$$|\Delta a_{ij}| \leq j^2 \tilde{\gamma}_m \alpha_i \max_s |a_{is}|, \quad |\Delta b_i| \leq n^2 \tilde{\gamma}_m \beta_i \max_s (\phi \max_s |a_{is}|, |b_i|),$$

where

$$\alpha_i = \frac{\max_{j,k} |a_{ij}^{(k)}|}{\max_j |a_{ij}|}, \quad \beta_i = \frac{\max(\phi \max_{j,k} |a_{ij}^{(k)}|, |b_i^{(k)}|)}{\max(\phi \max_j |a_{ij}|, |b_i|)},$$

$$\phi = \max_k \frac{\|b^{(k)}(k:m)\|_2}{\|a_k^{(k)}(k:m)\|_2}. \quad \square$$

The theorem shows that row-wise backward stability holds if $\max_i \{\alpha_i, \beta_i\}$ and ϕ are of order 1. With row pivoting or row sorting, $\max_i \{\alpha_i, \beta_i\} \leq \sqrt{m}(1 + \sqrt{2})^{n-1}$. The scalar ϕ is easily seen to be independent of the row ordering and so is beyond our control.

No formula for the row-wise backward error of an approximate LS solution is known (see Problem 20.13).

For a treatment of the weighted LS problem from a forward error perspective see Hough and Vavasis [642, 1997].

20.9. The Equality Constrained Least Squares Problem

We now consider the LS problem with linear equality constraints

$$\text{LSE : } \min_{Bx=d} \|b - Ax\|_2, \quad (20.23)$$

where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times n}$, with $m + p \geq n \geq p$. Note that the condition $m \geq n - p$ ensures that the LSE problem is overdetermined. We will assume that

$$\text{rank}(B) = p, \quad \text{null}(A) \cap \text{null}(B) = \{0\}. \quad (20.24)$$

The assumption that B is of full rank ensures that the system $Bx = d$ is consistent and hence that the LSE problem has a solution. The second condition in (20.24), which is equivalent to the condition that the matrix $[A^T, B^T]^T$ has full rank n , then guarantees that there is a unique solution.

The LSE problem arises in various applications, including the analysis of large-scale structures [66, 1988], [669, 1997], and the solution of the inequality constrained least squares problem [775, 1995, Chap. 23].

20.9.1. Perturbation Theory

We begin with a perturbation result for the LSE problem.

Theorem 20.8 (Eldén; Cox and Higham). *Consider the LSE problem as defined above and a perturbed problem defined by $A + \Delta A$, $b + \Delta b$, $B + \Delta B$, and $d + \Delta d$ with solution y , where*

$$\max \left\{ \frac{\|\Delta A\|_F}{\|A\|_F}, \frac{\|\Delta b\|_2}{\|b\|_2}, \frac{\|\Delta B\|_F}{\|B\|_F}, \frac{\|\Delta d\|_2}{\|d\|_2} \right\} \leq \epsilon.$$

Assume that the condition (20.24) is satisfied by both problems. Then

$$\begin{aligned} \frac{\|\Delta x\|_2}{\|x\|_2} \leq & \epsilon \left[\kappa_A(B) \left(\frac{\|d\|_2}{\|B\|_F \|x\|_2} + 1 \right) + \kappa_B(A) \left(\frac{\|b\|_2}{\|A\|_F \|x\|_2} + 1 \right) \right. \\ & \left. + \kappa_B(A)^2 \left(\frac{\|B\|_F}{\|A\|_F} \|AB_A^+\|_2 + 1 \right) \frac{\|r\|_2}{\|A\|_F \|x\|_2} \right] + O(\epsilon^2), \end{aligned} \quad (20.25)$$

where

$$\kappa_B(A) = \|A\|_F \|(AP)^+\|_2, \quad \kappa_A(B) = \|B\|_F \|B_A^+\|_2,$$

and

$$P = I - B^+B, \quad B_A^+ = (I - (AP)^+A)B^+. \quad \square$$

As a check, we can recover the perturbation bound (20.1) to first order by setting $B = 0$ and $d = 0$ (see Problem 20.11).

The bound (20.25) shows that if the residual r is small or zero, the sensitivity is governed by $\kappa_A(B)$ and $\kappa_B(A)$, otherwise by $\kappa_A(B)$ and $\kappa_B(A)^2 \|B\|_F \|AB_A^+\|_2 / \|A\|_F$. A sufficient condition for the LSE problem to be well conditioned is that B and AP are both well conditioned.

An analogous bound can be derived for Δr that has no direct dependence on $\kappa_A(B)$ and in which $\kappa_B(A)$ appears only to the first power.

20.9.2. Methods

We describe three classes of methods for solving the LSE problem. The *method of weighting* is based on the observation that the solution to the LSE problem (20.23) is the limit of the solution of the unconstrained problem

$$\min \left\| \begin{bmatrix} A \\ \mu B \end{bmatrix} x - \begin{bmatrix} b \\ \mu d \end{bmatrix} \right\|_2 \quad (20.26)$$

as the weight μ tends to infinity. Therefore the LSE solution can be approximated by solving (20.26) for a suitably large weight. For insightful analysis of how to choose the weight see Stewart [1080, 1998, pp. 317–320]. Van Loan [1180, 1985] describes an algorithm that solves (20.26) for a single weight and uses a refinement procedure to approximate the required limit. The algorithm is analysed further by Barlow and Vemulapati [70, 1988], [75, 1992].

Two further classes of methods for solving the LSE problem are *null space methods* and *elimination methods*, with each class having more than one variation. A basic difference between the classes is that the latter QR factorizes the constraint matrix B while the former QR factorizes B^T .

We first describe the null space methods, so-called because they employ an orthogonal basis for the null space of the constraint matrix. We begin with a version based on the generalized QR factorization. The generalized QR factorization was introduced by Hammarling [543, 1987] and Paige [912, 1990] and further analysed by Anderson, Bai, and Dongarra [21, 1992] and is of interest in its own right.

Theorem 20.9 (generalized QR factorization). *Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times n}$ with $m + p \geq n \geq p$. There are orthogonal matrices $Q \in \mathbb{R}^{n \times n}$ and $U \in \mathbb{R}^{m \times m}$ such that*

$$U^T A Q = \begin{matrix} & p & n-p \\ m-n+p & \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \\ n-p & \end{matrix}, \quad B Q = \begin{matrix} & p & n-p \\ p & \begin{bmatrix} S & 0 \end{bmatrix} \\ n-p & \end{matrix}, \quad (20.27)$$

where L_{22} and S are lower triangular. More precisely, we have

$$U^T A Q = \begin{cases} \begin{matrix} & n \\ m-n & \begin{bmatrix} 0 \\ L \end{bmatrix} \\ n & \end{matrix} & \text{if } m \geq n, \\ \begin{matrix} & n-m & m \\ m & \begin{bmatrix} X & L \end{bmatrix} \end{matrix} & \text{if } m < n, \end{cases} \quad (20.28)$$

where L is lower triangular. The assumptions (20.24) are equivalent to S and L_{22} being nonsingular.

Proof. Let

$$Q^T B^T = \begin{bmatrix} S^T \\ 0 \end{bmatrix}$$

be a QR factorization of B^T . We can determine an orthogonal U so that $U^T(AQ)$ has the form (20.28), where L is lower triangular (for example, we can construct

U as a product of suitably chosen Householder transformations). Clearly, B has full rank if and only if S is nonsingular. Partition $Q = [Q_1 \ Q_2]$ conformably with $[S \ 0]$ and assume S is nonsingular. Then, clearly, $\text{null}(B) = \text{range}(Q_2)$. We can write

$$A [Q_1 \ Q_2] = [U_1 \ U_2] \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix},$$

so that $AQ_2 = U_2L_{22}$. It follows that $\text{null}(A) \cap \text{null}(B) = \{0\}$ is equivalent to L_{22} being nonsingular. $\square$

While (20.28) is needed to define the generalized QR factorization precisely, the partitioning of $U^T A Q$ in (20.27) enables us to explain the application to the LSE problem without treating the cases $m \geq n$ and $m < n$ separately.

Using (20.27) the constraint $Bx = d$ may be written

$$Sy_1 = [S \ 0] \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = d, \quad y = Q^T x.$$

Hence the constraint determines $y_1 \in \mathbb{R}^p$ as the solution of the triangular system $Sy_1 = d$ and leaves $y_2 \in \mathbb{R}^{n-p}$ arbitrary. Since

$$\|b - Ax\|_2 = \|c - U^T A Q y\|_2, \quad c = U^T b,$$

we see that we have to find

$$\min_{y_2} \left\| \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} - \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \right\|_2 = \min_{y_2} \left\| \begin{bmatrix} c_1 - L_{11}y_1 \\ (c_2 - L_{21}y_1) - L_{22}y_2 \end{bmatrix} \right\|_2.$$

Therefore y_2 is the solution to the triangular system $L_{22}y_2 = c_2 - L_{21}y_1$. The solution x is recovered from $x = Qy$. We refer to this solution process as the GQR method. It is the method used by the LAPACK driver routine `xGGLSE` [20, 1999]. The operation count for the method is a complicated function of m , n , and p ; for $m \gg n \gg p$ it is approximately $2mn^2$ flops.

The stability of the GQR method is summarized by the following result.

Theorem 20.10 (Cox and Higham). *Suppose the LSE problem (20.23) is solved using the GQR method, where the generalized QR factorization is computed using Householder transformations, and let the assumptions (20.24) be satisfied. Let $\hat{x}$ denote the computed solution.*

(a) $\hat{x} = \bar{x} + \Delta\bar{x}$, where $\bar{x}$ solves

$$\min \{ \|b + \Delta b - (A + \Delta A)x\|_2 : (B + \Delta B)x = d \},$$

where

$$\begin{aligned} \|\Delta\bar{x}\|_2 &\leq \tilde{\gamma}_{np} \|\bar{x}\|_2, & \|\Delta b\|_2 &\leq \tilde{\gamma}_{mn} \|b\|_2, \\ \|\Delta A\|_F &\leq \tilde{\gamma}_{mn} \|A\|_F, & \|\Delta B\|_F &\leq \tilde{\gamma}_{np} \|B\|_F. \end{aligned}$$

(b) $\hat{x}$ solves

$$\min\{\|b + \Delta b - (A + \Delta A)x\|_2 : (B + \Delta B)x = d + \Delta d\},$$

where

$$\begin{aligned}\|\Delta b\|_2 &\leq \tilde{\gamma}_{mn}\|b\|_2 + \tilde{\gamma}_{np}\|A\|_F\|\hat{x}\|_2, & \|\Delta A\|_F &\leq \tilde{\gamma}_{mn}\|A\|_F, \\ \|\Delta B\|_F &\leq \tilde{\gamma}_{np}\|B\|_F, & \|\Delta d\|_2 &\leq \tilde{\gamma}_{np}\|B\|_F\|\hat{x}\|_2. \quad \square\end{aligned}$$

The first part of the theorem says that $\hat{x}$ is very close to the exact solution of a slightly different LSE problem; this is a mixed form of stability. The second part says that $\hat{x}$ exactly solves a perturbed LSE problem in which the perturbations to A and B are tiny but those to b and d can be relatively large when x is large-normed. It is an open problem whether genuine backward stability holds. In any case, the stability of the GQR method can be regarded as quite satisfactory.

The GQR method can be modified to reduce the amount of computation and the modified versions have the same stability properties [276, 1999].

The final class of methods for solving the LSE problem is based on elimination. First, we use QR factorization with column pivoting to factorize

$$B\Pi = Q[R_1 \ R_2], \quad R_1 \in \mathbb{R}^{p \times p} \text{ upper triangular, nonsingular.} \quad (20.29)$$

Note that column pivoting is essential here in order to obtain a nonsingular R_1 . Then, partitioning $\Pi^T x = [\tilde{x}_1^T, \tilde{x}_2^T]^T$, $\tilde{x}_1 \in \mathbb{R}^p$ and substituting the factorization (20.29) into the constraints yields

$$R_1\tilde{x}_1 = Q^T d - R_2\tilde{x}_2.$$

By solving for $\tilde{x}_1$ and partitioning $A\Pi = [\tilde{A}_1, \tilde{A}_2]$, $\tilde{A}_1 \in \mathbb{R}^{m \times p}$ we reduce the LSE problem to the unconstrained problem

$$\min_{\tilde{x}_2} \|(\tilde{A}_2 - \tilde{A}_1 R_1^{-1} R_2)\tilde{x}_2 - (b - \tilde{A}_1 R_1^{-1} Q^T d)\|_2.$$

Solving this unconstrained problem by QR factorization completes the elimination method as originally presented by Björck and Golub [130, 1967] (see also Lawson and Hanson [775, 1995, Chap. 21]). It is instructive to think of the method in terms of transformations on the matrix “ B -over- A ”:

$$\begin{aligned}\begin{bmatrix} B \\ A \end{bmatrix} &= \begin{matrix} p & n-p \\ m & \end{matrix} \begin{bmatrix} B_1 & B_2 \\ A_1 & A_2 \end{bmatrix} \rightarrow \begin{bmatrix} R_1 & R_2 \\ \tilde{A}_1 & \tilde{A}_2 \end{bmatrix} \\ &\rightarrow \begin{bmatrix} R_1 & R_2 \\ 0 & \tilde{A}_2 - \tilde{A}_1 R_1^{-1} R_2 \end{bmatrix} \rightarrow \begin{bmatrix} R_1 & R_2 \\ 0 & R_3 \\ 0 & 0 \end{bmatrix}, \quad (20.30)\end{aligned}$$

where $R_3 \in \mathbb{R}^{(n-p) \times (n-p)}$ is upper triangular. Note that the penultimate transformation is simply the annihilation of $\tilde{A}_1$ by Gaussian elimination. The B -over- A matrix also arises in the method of weighting; see (20.26).

Another elimination method that also produces the factorization (20.30) was derived by Cox and Higham [278, 1999] and independently by Reid [981, 2000].

A row-wise backward error result is available for both elimination methods [278, 1999]. The computed solution exactly solves a perturbed LSE problem for which row-wise backward error bounds involving row-wise growth factors hold for both the objective function and the constraints. If row sorting or row pivoting is used (separately on A and B) then the growth factors have similar behaviour to the α_i in Theorem 19.6.

20.10. Proof of Wedin's Theorem

In this section we give a proof of Theorem 20.1. We define $P_A := AA^+$, the orthogonal projection onto $\text{range}(A)$.

Lemma 20.11. *Let $A, B \in \mathbb{R}^{m \times n}$. If $\text{rank}(A) = \text{rank}(B)$ and $\eta = \|A^+\|_2 \|A - B\|_2 < 1$, then*

$$\|B^+\|_2 \leq \frac{1}{1 - \eta} \|A^+\|_2.$$

Proof. Let $r = \text{rank}(A)$. A standard result on the perturbation of singular values gives

$$\sigma_r(B) \geq \sigma_r(A) - \|A - B\|_2,$$

that is,

$$\|B^+\|_2^{-1} \geq \|A^+\|_2^{-1} - \|A - B\|_2 > 0,$$

which gives the result on rearranging. $\square$

Lemma 20.12. *Let $A, B \in \mathbb{R}^{m \times n}$. If $\text{rank}(A) = \text{rank}(B)$ then*

$$\|P_A(I - P_B)\|_2 = \|P_B(I - P_A)\|_2 \leq \|A - B\|_2 \min\{\|A^+\|_2, \|B^+\|_2\}.$$

Proof. We have

$$\begin{aligned} \|P_B(I - P_A)\|_2 &= \|(P_B(I - P_A))^T\|_2 \\ &= \|(I - P_A)P_B\|_2 \\ &= \|(I - AA^+)BB^+\|_2 \\ &= \|(I - AA^+)(A + (B - A))B^+\|_2 \\ &= \|(I - AA^+)(B - A)B^+\|_2 \\ &\leq \|A - B\|_2 \|B^+\|_2. \end{aligned}$$

The result then follows from the (nontrivial) equality $\|P_A(I - P_B)\|_2 = \|P_B(I - P_A)\|_2$; see, for example, Stewart [1067, 1977, Thm. 2.3] or Stewart and Sun [1083, 1990, Lem. 3.3.5], where proofs that use the CS decomposition are given. $\square$

Proof of Theorem 20.1. Let $B := A + \Delta A$. We have, with $r = b - Ax$,

$$\begin{aligned} y - x &= B^+(b + \Delta b) - x = B^+(r + Ax + \Delta b) - x \\ &= B^+(r + Bx - \Delta Ax + \Delta b) - x \\ &= B^+(r - \Delta Ax + \Delta b) - (I - B^+B)x \\ &= B^+(r - \Delta Ax + \Delta b), \end{aligned} \quad (20.31)$$

since B has full rank. Now

$$B^+r = B^+(BB^+)r = B^+P_B r = B^+P_B(I - P_A)r. \quad (20.32)$$

Applying Lemmas 20.11 and 20.12, we obtain

$$\begin{aligned} \|B^+r\|_2 &\leq \|B^+\|_2(\|B - A\|_2\|A^+\|_2)\|r\|_2 \\ &\leq \frac{\|A^+\|_2}{1 - \|A^+\|_2\|\Delta A\|_2} \|\Delta A\|_2\|A^+\|_2\|r\|_2 \\ &= \frac{\kappa_2(A)^2\epsilon}{1 - \kappa_2(A)\epsilon} \frac{\|r\|_2}{\|A\|_2}. \end{aligned} \quad (20.33)$$

Similarly,

$$\begin{aligned} \|B^+(-\Delta Ax + \Delta b)\|_2 &\leq \frac{\|A^+\|_2}{1 - \kappa_2(A)\epsilon} \epsilon(\|A\|_2\|x\|_2 + \|b\|_2) \\ &= \frac{\kappa_2(A)\epsilon}{1 - \kappa_2(A)\epsilon} \left(1 + \frac{\|b\|_2}{\|A\|_2\|x\|_2}\right) \|x\|_2 \\ &\leq \frac{\kappa_2(A)\epsilon}{1 - \kappa_2(A)\epsilon} \left(2 + \frac{\|r\|_2}{\|A\|_2\|x\|_2}\right) \|x\|_2. \end{aligned} \quad (20.34)$$

The bound for $\|x - y\|_2/\|x\|_2$ is obtained by using inequalities (20.33) and (20.34) in (20.31).

Turning to the residual, using (20.31) we find that

$$\begin{aligned} s - r &= \Delta b + B(x - y) - \Delta Ax \\ &= \Delta b - BB^+(r - \Delta Ax + \Delta b) - \Delta Ax \\ &= (I - BB^+)(\Delta b - \Delta Ax) - BB^+r. \end{aligned}$$

Since $\|I - BB^+\|_2 = \min\{1, m - n\}$,

$$\|r - s\|_2 \leq \|\Delta b\|_2 + \|\Delta Ax\|_2 + \|BB^+r\|_2 \leq \epsilon(\|b\|_2 + \|A\|_2\|x\|_2) + \|BB^+r\|_2.$$

Using (20.32), Lemma 20.12, and $\|BB^+\|_2 = 1$, we obtain

$$\|BB^+r\|_2 = \|BB^+P_B(I - P_A)r\|_2 \leq \|\Delta A\|_2\|A^+\|_2\|r\|_2 \leq \kappa_2(A)\epsilon\|r\|_2.$$

Hence

$$\frac{\|r - s\|_2}{\|b\|_2} \leq \epsilon \left(1 + \frac{\|A\|_2\|x\|_2}{\|b\|_2} + \kappa_2(A) \frac{\|r\|_2}{\|b\|_2}\right) \leq \epsilon(1 + 2\kappa_2(A)).$$

For the attainability, see Wedin [1211, 1973, §6]. $\square$

Note that, as the proof has shown, Wedin's theorem actually holds without any restriction on m and n , provided we define $x = A^+b$ and $y = (A + \Delta A)^+(b + \Delta b)$ when $m < n$ (in which case $r = 0$). We consider underdetermined systems in detail in the next chapter. The original version of Wedin's theorem also requires only $\text{rank}(A) = \text{rank}(A + \Delta A)$ and not that A have full rank.

20.11. Notes and References

The most comprehensive and up-to-date treatment of the LS problem is the book by Björck [128, 1996], which is an updated and expanded version of [124, 1990]. It treats many aspects not considered here, including rank-deficient and modified problems. An early book devoted to numerical aspects of the LS problem was written by Lawson and Hanson [775, 1995] (originally published in 1974), who, together with Stewart [1065, 1973], were the first to present error analysis for the LS problem in textbook form.

The history of the LS problem is described in the statistically oriented book by Farebrother [403, 1988].

The pseudo-inverse A^+ underlies the theory of the LS problem, since the LS solution can be expressed as $x = A^+b$. An excellent reference for perturbation theory of the pseudo-inverse is Stewart and Sun [1083, 1990, §3.3]. The literature on pseudo-inverses is vast, as evidenced by the annotated bibliography of Nashed and Rall [879, 1976], which contains 1,776 references published up to 1976.

Normwise perturbation theory for the LS problem was developed by various authors in the 1960s and 1970s. The earliest analysis was by Golub and Wilkinson [505, 1966], who gave a first-order bound and were the first to recognize the potential $\kappa_2(A)^2$ sensitivity. A nonasymptotic perturbation bound was given by Björck [119, 1967], who worked from the augmented system.

An early set of numerical experiments on the Householder, Gram-Schmidt, and normal equations methods for solving the LS problem was presented by Jordan [679, 1968]; this paper illustrates the incomplete understanding of perturbation theory and error analysis for the LS problem at that time.

van der Sluis [1178, 1975] presents a geometric approach to LS perturbation theory and gives lower bounds for the effect of worst-case perturbations. Golub and Van Loan [509, 1996, Thm. 5.3.1] give a first-order analogue of Theorem 20.1 expressed in terms of the angle θ between b and $\text{range}(A)$ instead of the residual r .

Wei [1213, 1990] gives a normwise perturbation result for the LS problem with a rank-deficient A that allows $\text{rank}(A + \Delta A) > \text{rank}(A)$.

Componentwise perturbation bounds of the form in Theorem 20.2 were first derived by Björck in 1988 and variations have been given by Arioli, Duff, and de Rijk [31, 1989], Björck [125, 1991], and Higham [590, 1990].

Higham [590, 1990] examined the famous test problem from Longley [796, 1967]—a regression problem which has a notoriously ill-conditioned 16×7 coefficient matrix with $\kappa_2(A) \approx 5 \times 10^9$. The inequality (20.8) was found to give tight bounds for the effect of random componentwise relative perturbations of the problem generated in experiments of Beaton, Rubin, and Barone [97, 1976]. Thus

componentwise perturbation bounds are potentially useful in regression analysis as an alternative to the existing statistically based techniques.

The tools required to prove Theorem 20.3 are developed in Wilkinson's book *The Algebraic Eigenvalue Problem* [1233, 1965]. Normwise (rather than columnwise) results of this form were derived informally by Golub and Wilkinson (assuming the use of extended precision inner products) [505, 1966], stated by Wilkinson [1234, 1965, p. 93] and Stewart [1065, 1973], and proved by Lawson and Hanson [775, 1995, Chap. 16].

The idea of using QR factorization to solve the LS problem was mentioned in passing by Householder [643, 1958]. Golub [502, 1965] worked out the details, using Householder QR factorization, and this method is sometimes called "Golub's method". In the same paper, Golub suggested the form of iterative refinement described at the start of §20.5 (which is implemented in a procedure by Businger and Golub [186, 1965]), and showed how to use QR factorization to solve the LSE problem (20.23).

Higham and Stewart [623, 1987] compare the normal equations method with the QR factorization method, with emphasis on aspects relevant to regression problems in statistics.

Foster [433, 1991] proposes a class of methods for solving the LS problem that are intermediate between the normal equations method and the MGS method, and that can be viewed as block MGS algorithms.

The most general analysis of QR factorization methods for solving the LS and related problems is by Björck and Paige [132, 1994], who consider an augmented system with an arbitrary right-hand side (see Problem 21.1) and prove a number of subtle stability results.

Error analysis for solution of the LS problem by the classical Gram–Schmidt method with reorthogonalization is given by Abdelmalek [2, 1971], who obtains a forward error bound as good as that for a backward stable method.

It was Björck [118, 1967] who first recognized that iterative refinement should be applied to the augmented system for best results, and he gave a detailed rounding error analysis for the use of a QR factorization computed by the Householder or MGS methods. Björck and Golub [130, 1967] show how to compute and refine the solution to the LSE problem using Householder transformations, while Björck [120, 1968] gives a similar approach based on the Gram–Schmidt method. In [121, 1978], Björck dispels some misconceptions of statisticians about (mixed precision) iterative refinement for the LS problem; he discusses standard refinement together with two versions of refinement based on the seminormal equations.

Theorem 20.4 and the following analysis are from Higham [596, 1991].

Arioli, Duff, and de Rijk [31, 1989] investigate the application of fixed precision iterative refinement to large, sparse LS problems, taking the basic solver to be the block LDL^T factorization code MA27 [364, 1982] from the Harwell Subroutine Library (applied to the augmented system); in particular, they use scaling of the form (20.17). Björck [126, 1992] determines, via an error analysis for solution of the augmented system by block LDL^T factorization, a choice of α in (20.17) that minimizes a bound on the forward error.

The idea of implementing iterative refinement with a precision that increases on each iteration (see the Notes and References to Chapter 12) can be applied to

the LS problem; see Gluchowska and Smoktunowicz [492, 1990].

The use of the seminormal equations was first suggested by Kahan, in the context of iterative refinement, as explained by Golub and Wilkinson [505, 1966].

Stewart [1069, 1977] discusses the problem of finding the normwise backward error for the LS problem and offers some backward perturbations that are candidates for being of minimal norm. The problem is also discussed by Higham [590, 1990]. Componentwise backward error for the LS problem has been investigated by Arioli, Duff, and de Rijk [31, 1989], Björck [125, 1991], and Higham [590, 1990].

As noted by Gu [527, 1998], the formula (20.21) has the disadvantage that the matrix $[A \ \phi(I - rr^+)]$ has badly scaled columns when ϕ is large, which can cause standard methods for computing the SVD to return an inaccurate computed smallest singular value. However, SVD methods developed in [323, 1999], [614, 2000] do not suffer from this difficulty as they are insensitive to the column scaling. The formula (20.21) requires computation of the SVD of an $m \times (n + m)$ matrix, which can be prohibitively expensive for large problems. Karlson and Waldén [713, 1997] derive upper and lower bounds for the quantity $\eta_F(y)|_{\theta=\infty}$ in which only A is perturbed. Their bounds can be computed in $O(mn)$ operations but the bounds can differ by an arbitrary factor. Gu [527, 1998] derives an approximation to $\eta_F(y)|_{\theta=\infty}$ that differs from it by a factor less than 2 and can be computed in $O(mn^2)$ operations. For large, sparse A , Malyshev and Sadkane [813, 2002] show how to use Lanczos bidiagonalization to compute an upper bound for $\eta_F(y)|_{\theta=\infty}$.

Theorem 20.5 has been extended to the multiple right-hand side LS problem by Sun [1107, 1996]. In [1108, 1997], Sun shows that adding the requirement in (20.20) that y is the solution of minimum 2-norm does not change the backward error if y and b are both nonzero (a result that is, of course, immediate if $A + \Delta A$ has full rank at the minimum).

Lemma 20.6 is from a book by Kielbasiński and Schwetlick, which has been published in German [734, 1988] and Polish [735, 1992] editions, but not in English. The lemma is their Lemma 8.2.11, and can be shown to be equivalent to a result of Stewart [1067, 1977, Thm. 5.3].

Other methods for solving the LS problem not considered in this chapter include those of Peters and Wilkinson [936, 1970], Cline [242, 1973], and Plemmons [944, 1974], all of which begin by computing an LU factorization of the rectangular matrix A . Error bounds for these methods can be derived using results from this chapter and Chapters 9 and 19.

A result similar to Theorem 20.7 was originally proved under some additional assumptions by Powell and Reid [951, 1969]. The result as stated is proved by Cox and Higham [278, 1999, Thm. 4.1], and originally appeared in a slightly weaker form in [275, 1998].

Theorem 20.8 is from Cox and Higham [276, 1999] and is essentially the same perturbation result as that obtained by Eldén [387, 1980], differing only in the assumptions on the perturbations. The bound (20.25) does not yield a condition number for the LSE problem, since it is not attainable. A sharp bound involving Kronecker products is obtained in [276, 1999], but it is more difficult to interpret. That the null space method for solving the LSE problem can be conveniently expressed in terms of the generalized QR factorization was pointed out by Paige [912, 1990] and Anderson, Bai, and Dongarra [21, 1992].

Perturbation bounds for the generalized QR factorization are given by Barlund [82, 1994].

No analogue of Theorem 20.5 is known for the LSE problem, but upper bounds on the normwise backward error of an approximate LSE solution are derived by Cox and Higham [277, 1999], where they are shown by experiment to be of practical use. A backward error formula for the problem of least squares minimization over a sphere, $\min_{\|x\|_2 \leq \alpha} \|b - Ax\|_2$, is given by Malyshev [812, 2001].

Theorem 20.10 is from Cox and Higham [276, 1999].

The use of a weighted QR factorization for solving weighted and/or linearly constrained LS problems is proposed and investigated by Gulliksson and Wedin [533, 1992], [531, 1994], [532, 1995].

In this chapter we have not treated LS problems with linear inequality constraints or quadratic constraints, or generalized LS problems. For coverage of these, see Björck [128, 1996, Chap. 5] and Golub and Van Loan [509, 1996, §§5.6.3, 12.1] and the references therein.

20.11.1. LAPACK

Driver routine `xGELS` solves the full rank LS problem by Householder QR factorization. It caters for multiple right-hand sides, each of which defines a separate LS problem. Thus, `xGELS` solves $\min\{\|B - AX\|_F : X \in \mathbb{R}^{n \times p}\}$, where $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) and $B \in \mathbb{R}^{m \times p}$. This routine does not return any error bounds, but code for computing an error bound (essentially (20.1)) is given in [20, 1999, §4.5]. Iterative refinement is not supported for LS problems in LAPACK.

Driver routines `xGELSY` and `xGELSS`, `xGELSD` solve the rank-deficient LS problem with multiple right-hand sides, using, respectively, a complete orthogonal factorization (computed via QR factorization with column pivoting), the SVD, and a divide and conquer implementation of the SVD.

LAPACK also contains routines for solving the LSE problem (`xGGLSE`, which uses the GQR method) and a generalized form of weighted LS problem (`xGGGLM`). Code for computing an error bound (essentially (20.25)) is given in [20, 1999, §4.6].

Problems

20.1. Show that any solution to the LS problem $\min_x \|b - Ax\|_2$ satisfies the normal equations $A^T A x = A^T b$. What is the geometrical interpretation of these equations?

20.2. Prove Theorem 20.3.

20.3. The pseudo-inverse $X \in \mathbb{R}^{n \times m}$ of $A \in \mathbb{R}^{m \times n}$ can be defined as the unique matrix satisfying the four *Moore–Penrose* conditions

$$\begin{aligned} \text{(i)} \quad & AXA = A, & \text{(ii)} \quad & XAX = X, \\ \text{(iii)} \quad & AX = (AX)^T, & \text{(iv)} \quad & XA = (XA)^T. \end{aligned}$$

Let $A = U\Sigma V^T$ be an SVD, with $\Sigma = \text{diag}(\sigma_i)$ and let $r = \text{rank}(A)$. Show that $X = V \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1}, 0, \dots, 0) U^T$ satisfies (i)–(iv) and hence is the pseudo-inverse of A . Show that $(A^+)^+ = A$.

20.4. Show that the pseudo-inverse A^+ of $A \in \mathbb{R}^{m \times n}$ solves the problem

$$\min_{X \in \mathbb{R}^{n \times m}} \|AX - I_m\|_F.$$

Is the solution unique?

20.5. Prove a result analogous to Theorem 20.3 for the MGS method, as described in §20.3.

20.6. Consider the LS problem $\min_x \|b - Ax\|_2$, where $A \in \mathbb{R}^{m \times n}$. Let $\hat{x}$ be the computed LS solution obtained from the normal equations method and x the exact solution, and define $\hat{r} = b - A\hat{x}$, $r = b - Ax$. Using (20.12) and (20.13) show that a bound holds of the form

$$|r^T r - \hat{r}^T \hat{r}| \leq c_{m,n} u \|x - \hat{x}\|_2 \|A\|_2 (\|A\|_2 \|\hat{x}\|_2 + \|b\|_2) + O(u^2).$$

20.7. Prove (20.18) and (20.19).

20.8. (Waldén, Karlson, and Sun [1203, 1995]) Partially complete the gap in Theorem 20.5 by evaluating $\eta_F(0)$ for the case “ $\theta = \infty$ ”, that is, $\Delta b \equiv 0$.

20.9. Prove (20.21).

20.10. Show that x solves the LSE problem (20.23) if and only if

$$\begin{aligned} A^T(b - Ax) &= B^T \lambda, \\ Bx &= d, \end{aligned}$$

where $\lambda \in \mathbb{R}^p$ is a vector of Lagrange multipliers. These are the normal equations for the LSE problem.

20.11. Show that for the unconstrained LS problem (20.25) reduces to a first-order approximation of (20.1).

20.12. (RESEARCH PROBLEM) For the unconstrained least squares problem Gu [527, 1998] proves that mixed stability implies backward stability. Can this result be extended to the LSE problem?

20.13. (RESEARCH PROBLEM) Derive a computable expression for the row-wise backward error of an approximate LS solution.

Chapter 21

Underdetermined Systems

I'm thinking of two numbers.

Their average is 3.

What are the numbers?

— CLEVE B. MOLER, *The World's Simplest Impossible Problem* (1990)

This problem arises in important algorithms used in mathematical programming . . .

*In these cases, B is usually very large and sparse and,
because of storage difficulties,*

it is often uneconomical to store and access Q_1 . . .

*Sometimes it has been thought that [the seminormal equations method]
could be disastrously worse than [the Q method] . . .*

*It is the purpose of this note to show that such algorithms are
numerically quite satisfactory.*

— C. C. PAIGE, *An Error Analysis of a Method for Solving Matrix Equations* (1973)

Having considered well-determined and overdetermined linear systems, we now turn to the remaining class of linear systems: those that are underdetermined.

21.1. Solution Methods

Consider the underdetermined system $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ with $m \leq n$. The system can be analysed using a QR factorization

$$A^T = Q \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad (21.1)$$

where $Q \in \mathbb{R}^{n \times n}$ is orthogonal and $R \in \mathbb{R}^{m \times m}$ is upper triangular. (We could, alternatively, use an LQ factorization of A , but we will keep to the standard notation.) We have

$$b = Ax = [R^T \quad 0] Q^T x = R^T y_1, \quad (21.2)$$

where

$$y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = Q^T x, \quad y_1 \in \mathbb{R}^m.$$

If A has full rank then $y_1 = R^{-T}b$ is uniquely determined and all solutions of $Ax = b$ are given by

$$x = Q \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}, \quad y_2 \in \mathbb{R}^{n-m} \text{ arbitrary.}$$

The unique solution x_{LS} that minimizes $\|x\|_2$ is obtained by setting $y_2 = 0$. We have

$$x_{LS} = Q \begin{bmatrix} R^{-T}b \\ 0 \end{bmatrix} \quad (21.3)$$

$$= Q \begin{bmatrix} R \\ 0 \end{bmatrix} R^{-1} R^{-T}b = Q \begin{bmatrix} R \\ 0 \end{bmatrix} (R^T R)^{-1}b \quad (21.4)$$

$$= A^T (AA^T)^{-1}b \\ = A^+ b,$$

where $A^+ = A^T (AA^T)^{-1}$ is the pseudo-inverse of A . Hence x_{LS} can be characterized as $x_{LS} = A^T y$, where y solves the *normal equations* $AA^T y = b$.

Equation (21.3) defines one way to compute x_{LS} . We will refer to this method as the “Q method”. When A is large and sparse it is desirable to avoid storing and accessing Q , which can be expensive. An alternative method with this property uses the QR factorization (21.1) but computes x_{LS} as $x_{LS} = A^T y$, where

$$R^T R y = b \quad (21.5)$$

(cf. (21.4)). These latter equations are called the *seminormal equations* (SNE). As the “semi” denotes, however, this method does not explicitly form AA^T , which would be undesirable from the standpoint of numerical stability. Note that equations (21.5) are different from the equations $R^T R x = A^T b$ for an overdetermined least squares (LS) problem, where $A = Q[R^T \quad 0]^T \in \mathbb{R}^{m \times n}$ with $m \geq n$, which are also called seminormal equations (see §20.6).

21.2. Perturbation Theory and Backward Error

A componentwise perturbation result for the minimum 2-norm solution to an underdetermined system is readily obtained.

Theorem 21.1 (Demmel and Higham). *Let $A \in \mathbb{R}^{m \times n}$ ($m \leq n$) be of full rank and $0 \neq b \in \mathbb{R}^m$. Suppose $\|A^+ \Delta A\|_2 < 1$ and*

$$|\Delta A| \leq \epsilon E, \quad |\Delta b| \leq \epsilon f.$$

If x and y are the minimum 2-norm solutions to $Ax = b$ and $(A + \Delta A)y = b + \Delta b$, respectively, then, for any absolute norm,

$$\frac{\|x - y\|}{\|x\|} \leq \left(\| |I - A^+ A| \cdot E^T \cdot |A^{+T} x| \| + \| |A^+| \cdot (f + E|x|) \| \right) \frac{\epsilon}{\|x\|} + O(\epsilon^2). \quad (21.6)$$

For any Hölder p -norm, the bound is attainable to within a constant factor depending on n .

Proof. The perturbed matrix $A + \Delta A = A(I + A^+ \Delta A)$ has full rank, so we can manipulate the equation

$$y = (A + \Delta A)^T ((A + \Delta A)(A + \Delta A)^T)^{-1} (b + \Delta b)$$

to obtain

$$\begin{aligned} y - x &= (I - A^+ A) \Delta A^T (A A^T)^{-1} b + A^+ (\Delta b - \Delta A x) + O(\epsilon^2) \\ &= (I - A^+ A) \Delta A^T A^{+T} x + A^+ (\Delta b - \Delta A x) + O(\epsilon^2). \end{aligned} \quad (21.7)$$

The required bound follows on using absolute value inequalities and taking norms. That the bound is attained to within a constant factor depending on n for Hölder p -norms is a consequence of the fact that the two vectors on the right-hand side of (21.7) are orthogonal. $\square$

Two special cases are worth noting, for later use. We will use the equality $\|I - A^+ A\|_2 = \min\{1, n - m\}$, which can be derived by consideration of the QR factorization (21.1), for example. If $E = |A|H$, where H is a given nonnegative matrix, and $f = |b|$, then we can put (21.6) in the form

$$\frac{\|x - y\|_2}{\|x\|_2} \leq \min\{3, n - m + 2\} \max\{\|H\|_2, 1\} \text{cond}_2(A) \epsilon + O(\epsilon^2), \quad (21.8)$$

where

$$\text{cond}_2(A) = \| |A^+| |A| \|_2.$$

Note that $\text{cond}_2(A)$ is independent of the row scaling of A ($\text{cond}_2(DA) = \text{cond}_2(A)$ for nonsingular diagonal D). If $E = \|A\|_2 e_m e_n^T$ and $f = \|b\|_2 e_m$, where e_m denotes the m -dimensional vector of 1s, then

$$\frac{\|x - y\|_2}{\|x\|_2} \leq \min\{3, n - m + 2\} (mn)^{1/2} \kappa_2(A) \epsilon + O(\epsilon^2). \quad (21.9)$$

The following analogue of Lemma 20.6 will be needed for the error analysis in the next section. It says that if we perturb the two occurrences of A in the normal equations $AA^T x = b$ differently, then the solution of the perturbed system is the solution of normal equations in which there is only *one* perturbation of A and, moreover, this single perturbation is no larger, in the normwise or row-wise sense, than the two perturbations we started with.

Lemma 21.2 (Kielbański and Schwetlick). *Let $A \in \mathbb{R}^{m \times n}$ ($m \leq n$) be of full rank and suppose*

$$(A + \Delta A_1)\bar{x} = b, \quad \bar{x} = (A + \Delta A_2)^T \tilde{y}.$$

Assume that $3 \max(\|A^+ \Delta A_1\|_2, \|A^+ \Delta A_2\|_2) < 1$. Then there is a vector $\tilde{y}$ and a perturbation ΔA with

$$\Delta A = \Delta A_1 G_1 + \Delta A_2 G_2, \quad G_1^T = G_1 = G_2^2, \quad G_1 + G_2 = I,$$

such that

$$(A + \Delta A)\bar{x} = b, \quad \bar{x} = (A + \Delta A)^T \tilde{y},$$

that is, $\bar{x}$ is the minimum 2-norm solution to $(A + \Delta A)x = b$.

Proof. The proof is similar to that of Lemma 20.6, but differs in some details. If $\bar{x} = (A + \Delta A_2)^T \tilde{y} = 0$ we take $\Delta A = \Delta A_2$. Otherwise, we set

$$\Delta A := \Delta A_1 P + \Delta A_2 (I - P) =: \Delta A_2 + H P,$$

where $P = \bar{x} \bar{x}^T / \bar{x}^T \bar{x}$ and $H = \Delta A_1 - \Delta A_2$. We have

$$(A + \Delta A)^T \tilde{y} = (A + \Delta A_2 + H P)^T \tilde{y} = \beta \bar{x},$$

where $\beta = 1 + \bar{x}^T H^T \tilde{y} / \bar{x}^T \bar{x}$, which shows that we need to set $\tilde{y} = \beta^{-1} \bar{x}$. To check that $(A + \Delta A)\bar{x} = b$, we evaluate

$$\begin{aligned} (A + \Delta A)\bar{x} &= (A + \Delta A_1 + H(P - I))\bar{x} \\ &= (A + \Delta A_1)\bar{x} = b, \end{aligned}$$

as required. The vector $\tilde{y}$ is undefined if $\beta = 0$. But

$$\begin{aligned} \beta &= 1 + \frac{\bar{x}^T H^T \tilde{y}}{\bar{x}^T \bar{x}} = 1 + \frac{\bar{x}^T H^T (A + \Delta A_2)^+{}^T \bar{x}}{\bar{x}^T \bar{x}} \\ &\geq 1 - \|(A + \Delta A_2)^+ H\|_2 \\ &\geq 1 - \frac{\|A^+ H\|_2}{1 - \|A^+ \Delta A_2\|_2} \quad (\text{if } \|A^+ \Delta A_2\|_2 < 1) \\ &\geq 1 - \frac{(\|A^+ \Delta A_1\|_2 + \|A^+ \Delta A_2\|_2)}{1 - \|A^+ \Delta A_2\|_2}, \end{aligned}$$

which is positive if $3 \max(\|A^+ \Delta A_1\|_2, \|A^+ \Delta A_2\|_2) < 1$. $\square$

Note that in Lemma 21.2 we have the normwise bound

$$\|\Delta A\|_p \leq (\|\Delta A_1\|_p^2 + \|\Delta A_2\|_p^2)^{1/2}, \quad p = 2, F.$$

A normwise backward error formula for an arbitrary approximate minimum 2-norm solution is given by the following result.

Theorem 21.3 (Sun and Sun). *Let $A \in \mathbb{R}^{m \times n}$ ($m \leq n$), $b \in \mathbb{R}^m$, and $r = b - Ay$. The normwise backward error*

$$\eta_F(y) := \min\{ \|\Delta A, \theta \Delta b\|_F : y \text{ is the minimum 2-norm solution to } (A + \Delta A)y = b + \Delta b \}$$

is given by $\eta_F(0) = \theta \|b\|_2$ and, for $y \neq 0$,

$$\eta_F(y) = \sqrt{\frac{\theta^2 \|y\|_2^2}{1 + \theta^2 \|y\|_2^2} \cdot \frac{\|r\|_2^2}{\|y\|_2^2} + \sigma_m^2(A(I - yy^+))}. \quad \square$$

Note that if $m = n$ then $\sigma_m(A(I - yy^+)) = 0$ and the formula in the theorem reduces to the formula in Problem 7.8 for linear systems.

21.3. Error Analysis

We now consider the stability of the Q method and the SNE method. For both methods we assume that the QR factorization is computed using Householder or Givens transformations.

Before presenting the results we define a measure of stability. The row-wise backward error for a minimum-norm underdetermined system $Ax = b$ is defined as

$$\begin{aligned} \omega^R(y) &:= \min\{ \epsilon : \exists \Delta A \in \mathbb{R}^{m \times n}, \Delta b \in \mathbb{R}^m \text{ with} \\ &\quad \|\Delta A(i, :)\|_2 \leq \epsilon \|A(i, :)\|_2 \text{ and } |\Delta b_i| \leq \epsilon |b_i|, i = 1:n, \\ &\quad \text{s.t. } y \text{ is the min. norm solution to } (A + \Delta A)y = b + \Delta b \}. \end{aligned}$$

Note the requirement in this definition that y be the minimum norm solution; the usual row-wise backward error $\omega_{|A|_{ee^T}, |b|}(y)$ (see (7.7)) is a generally smaller quantity. Let us say that a method is *row-wise backward stable* if it produces a computed solution $\hat{x}$ for which the row-wise backward error $\omega^R(\hat{x})$ is of order u .

Theorem 21.4. *Let $A \in \mathbb{R}^{m \times n}$ with $\text{rank}(A) = m \leq n$, and assume that a condition of the form $\text{cond}_2(A)mn\gamma_n < 1$ holds. Suppose the underdetermined system $Ax = b$ is solved in the minimum 2-norm sense using the Q method. Then the computed solution $\hat{x}$ is the minimum 2-norm solution to $(A + \Delta A)x = b$, where*

$$\|\Delta A(i, :)\|_2 \leq \tilde{\gamma}_{mn} \|A(i, :)\|_2, \quad i = 1:m.$$

Proof. The Q method solves the triangular system $R^T y_1 = b$ and then forms $x = Q[y_1^T, 0]^T$. Assuming the use of Householder QR factorization, from Theorem 19.4 we have that

$$(A + \Delta A_0)^T = Q \begin{bmatrix} \widehat{R} \\ 0 \end{bmatrix},$$

for some orthogonal matrix Q , where $\|\Delta A_0(i, :)\|_2 \leq \tilde{\gamma}_{mn} \|A(i, :)\|_2$, $i = 1:m$. The computed $\widehat{y}_1$ satisfies

$$(\widehat{R} + \Delta \widehat{R})^T \widehat{y}_1 = b, \quad |\Delta \widehat{R}| \leq \gamma_m |\widehat{R}|.$$

From Lemma 19.3, the computed solution $\widehat{x}$ satisfies

$$\widehat{x} = (Q + \Delta Q) \begin{bmatrix} \widehat{y}_1 \\ 0 \end{bmatrix}, \quad \|\Delta Q\|_F \leq m \tilde{\gamma}_n. \quad (21.10)$$

We now rewrite the latter two equations in such a way that we can apply Lemma 21.2:

$$\begin{aligned} (A + \Delta A_1) \widehat{x} &:= [(\widehat{R} + \Delta \widehat{R})^T \quad 0] (Q + \Delta Q)^{-1} \cdot (Q + \Delta Q) \begin{bmatrix} \widehat{y}_1 \\ 0 \end{bmatrix} = b, \\ \widehat{x} &= (Q + \Delta Q) \begin{bmatrix} \widehat{R} \\ 0 \end{bmatrix} \cdot \widehat{R}^{-1} \widehat{y}_1 =: (A + \Delta A_2)^T \bar{y}. \end{aligned}$$

It is straightforward to show that

$$\|\Delta A_k(i, :)\|_2 \leq \tilde{\gamma}_{mn} \|A(i, :)\|_2, \quad i = 1:m, \quad k = 1:2.$$

The result follows on invocation of Lemma 21.2. $\square$

Theorem 21.4 says that the Q method is row-wise backward stable. This is not altogether surprising, since (Householder or Givens) QR factorization for the LS problem enjoys an analogous backward stability result (Theorem 20.3), albeit without the restriction of a minimum norm solution. Applying (21.8) to Theorem 21.4 with $H = ee^T$ we obtain the forward error bound

$$\frac{\|\widehat{x} - x\|_2}{\|x\|_2} \leq n \tilde{\gamma}_{mn} \text{cond}_2(A) + O(u^2). \quad (21.11)$$

The same form of forward error bound (21.11) can be derived for the SNE method [325, 1993]. However, it is not possible to obtain a result analogous to Theorem 21.4, nor even to obtain a residual bound of the form $\|b - A\widehat{x}\|_2 \leq c_{m,n} u \|A\|_2 \|\widehat{x}\|_2$ (which would imply that $\widehat{x}$ solved a nearby system, though $\widehat{x}$ would not necessarily be the minimum norm solution). The method of solution guarantees only that the seminormal equations themselves have a small residual. Thus, as in the context of overdetermined LS problems, the SNE method is not backward stable. A possible way to improve the stability is by iterative refinement, as shown in [325, 1993].

Note that the forward error bound (21.11) is independent of the row scaling of A , since $\text{cond}_2(A)$ is. The bound is therefore potentially much smaller than the bound

$$\frac{\|x - \widehat{x}\|_2}{\|x\|_2} \leq c_{m,n} u \kappa_2(A) + O(u^2),$$

Table 21.1. *Backward errors for underdetermined Vandermonde system.*

	$\frac{\eta_F}{\ [A, b]\ _F} \ (\theta = 1)$
Householder QR	1.6×10^{-17}
MGS with $x := Qy$	1.8×10^{-3}
MGS with x formed stably (see text)	3.4×10^{-17}
SNE method (using Householder QR)	4.7×10^{-14}

obtained by Paige [911, 1973] for the SNE method and by Jennings and Osborne [675, 1974] and Arioli and Laratta [32, 1985, Thm. 4] for the Q method.

Finally, we mention an alternative version of the Q method that is based on the modified Gram–Schmidt (MGS) method. The obvious approach is to compute the QR factorization $A^T = QR$ using MGS ($Q \in \mathbb{R}^{n \times m}$, $R \in \mathbb{R}^{m \times m}$), solve $R^T y = b$, and then form $x = Qy$. Since Q is provided explicitly by the MGS method, the final stage is a full matrix–vector multiplication, unlike for the Householder method. However, because the computed $\hat{Q}$ may depart from orthonormality, this method is unstable in the form described. The formation of $x = Qy$ should instead be done as follows:

```

 $x^{(n)} = 0$ 
for  $k = n:-1:1$ 
     $\alpha_k = q_k^T x^{(k)}$ 
     $x^{(k-1)} = x^{(k)} - (\alpha_k - y_k)q_k$ 
end
 $x = x^{(0)}$ 

```

The recurrence can be written as $x^{(k-1)} = x^{(k)} + y_k q_k - (q_k^T x^{(k)})q_k$, and the last term is zero in exact arithmetic if the q_k are mutually orthogonal. In finite precision arithmetic this correction term has the “magical” effect of making the algorithm stable, in the sense that it satisfies essentially the same result as the Q method in Theorem 21.4; see Björck and Paige [132, 1994].

A numerical example is instructive. Take the 20×30 Vandermonde matrix $A = (p_i^{j-1})$, where the p_i are equally spaced on $[0, 1]$, and let b have elements equally spaced on $[0, 1]$. The condition number $\text{cond}_2(A) = 7.6 \times 10^{12}$. The backward error from Theorem 21.3 is shown in Table 21.1. For A^T , the $\hat{Q}$ supplied by MGS satisfies $\|\hat{Q}^T \hat{Q} - I\|_2 = 9 \times 10^{-3}$, which explains the instability of the “obvious” MGS solver.

21.4. Notes and References

The seminormal equations method was suggested by Gill and Murray [482, 1973] and Saunders [1011, 1972]. Other methods for obtaining minimal 2-norm solutions of underdetermined systems are surveyed by Cline and Plemmons [246, 1976].

Theorem 21.1 is from Demmel and Higham [325, 1993]. The bound (21.9) is well known; it follows from Wedin’s original version of our Theorem 20.1, which

applies to minimum 2-norm underdetermined problems as well as LS problems.

Theorem 21.3 is from Sun and Sun [1110, 1997] and solves a research problem posed in the first edition of this book.

Theorem 21.4 first appeared in the first edition of this book. Demmel and Higham [325, 1993] prove the weaker result that $\hat{x}$ from the Q method is *very close to* a vector $\bar{x}$ that satisfies the criterion for row-wise backward stability, and Lawson and Hanson [775, 1995, Thm. 16.18] give a corresponding result in which $\bar{x}$ satisfies the criterion for general normwise backward stability. The key to showing actual backward stability is the use of Kielbasiński and Schwetlick's lemma, which is a modification of Lemma 8.2.11 in [734, 1988] and [735, 1992] (our Lemma 20.6). Demmel and Higham [325, 1993] also give error analysis for the seminormal equations method.

The new MGS algorithm for solving the minimum norm problem was first suggested by Björck and Paige [131, 1992]; see also Björck [127, 1994].

Arioli and Laratta [33, 1986] give error analysis of QR factorization methods for solving the general problem $\min\{\|x - c\|_2 : Ax = b\}$, where $A \in \mathbb{R}^{m \times n}$ with $m \leq n$.

21.4.1. LAPACK

The same routines that solve the (overdetermined) LS problem also solve underdetermined systems for the solution of minimal 2-norm. Thus, `xGELS` solves a full rank underdetermined system with multiple right-hand sides by the Q method. Routines `xGELSY` and `xGELSS` solve rank-deficient problems with multiple right-hand sides, using, respectively, a complete orthogonal factorization (computed via QR factorization with column pivoting) and the singular value decomposition.

Problems

21.1. (Björck [126, 1992]) Show that the system

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} y \\ x \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix} \quad (21.12)$$

characterizes the solution to the following generalizations of the LS problem and the problem of finding the minimum norm solution to an underdetermined system:

$$\min_x \|b - Ax\|_2^2 + 2c^T x, \quad (21.13)$$

$$\min_y \|y - b\|_2 \quad \text{subject to} \quad A^T y = c. \quad (21.14)$$

Chapter 22

Vandermonde Systems

*We began, 25 years ago, to take up [the conditioning of]
the class of Vandermonde matrices.*

*The original motivation came from unpleasant experiences with the
computation of Gauss type quadrature rules from the
moments of the underlying weight function.*

— WALTER GAUTSCHI, *How (Un)stable Are Vandermonde Systems?* (1990)

*Extreme ill-conditioning of the [Vandermonde] linear systems
will eventually manifest itself as n increases by yielding
an error curve which is not sufficiently levelled on the current reference ...
or more seriously fails to have the correct number of sign changes.*

— M. ALMACANY, C. B. DUNHAM, and J. WILLIAMS,
Discrete Chebyshev Approximation by Interpolating Rationals (1984)

A Vandermonde matrix is defined in terms of scalars $\alpha_0, \alpha_1, \dots, \alpha_n \in \mathbb{C}$ by

$$V = V(\alpha_0, \alpha_1, \dots, \alpha_n) = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha_0 & \alpha_1 & \dots & \alpha_n \\ \vdots & \vdots & & \vdots \\ \alpha_0^n & \alpha_1^n & \dots & \alpha_n^n \end{bmatrix} \in \mathbb{C}^{(n+1) \times (n+1)}.$$

Vandermonde matrices play an important role in various problems, such as in polynomial interpolation. Suppose we wish to find the polynomial $p_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$ that interpolates to the data $(\alpha_i, f_i)_{i=0}^n$, for distinct points α_i , that is, $p_n(\alpha_i) = f_i$, $i = 0:n$. Then the desired coefficient vector $a = [a_0, a_1, \dots, a_n]^T$ is the solution of the dual Vandermonde system

$$V^T a = f \quad (\text{dual}).$$

The primal system

$$Vx = b \quad (\text{primal})$$

represents a moment problem, which arises, for example, when determining the weights for a quadrature rule: given moments b_i find weights x_i such that $\sum_{j=0}^n x_j \alpha_j^i = b_i$, $i = 0:n$.

Because a Vandermonde matrix depends on only $n+1$ parameters and has a great deal of structure, it is possible to perform standard computations with reduced complexity. The easiest algorithm to derive is for matrix inversion.

22.1. Matrix Inversion

Assume that V is nonsingular and let $V^{-1} = W = (w_{ij})_{i,j=0}^n$. The i th row of the equation $WV = I$ may be written

$$\sum_{j=0}^n w_{ij} \alpha_k^j = \delta_{ik}, \quad k = 0:n.$$

These equations specify a fundamental interpolation problem that is solved by the Lagrange basis polynomial:

$$\sum_{j=0}^n w_{ij} x^j = \prod_{\substack{k=0 \\ k \neq i}}^n \left(\frac{x - \alpha_k}{\alpha_i - \alpha_k} \right) =: l_i(x). \quad (22.1)$$

The inversion problem is now reduced to finding the coefficients of $l_i(x)$. It is clear from (22.1) that V is nonsingular iff the α_i are distinct. It also follows from (22.1) that V^{-1} is given explicitly by

$$w_{ij} = \frac{(-1)^{n-j} \sigma_{n-j}(\alpha_0, \dots, \alpha_{i-1}, \alpha_{i+1}, \dots, \alpha_n)}{\prod_{\substack{k=0 \\ k \neq i}}^n (\alpha_i - \alpha_k)}, \quad (22.2)$$

where $\sigma_k(y_1, \dots, y_n)$ denotes the sum of all distinct products of k of the arguments $y_1, \dots, y_n$ (that is, σ_k is the k th elementary symmetric function). An efficient way to find the w_{ij} is first to form the master polynomial

$$\phi(x) = \prod_{k=0}^n (x - \alpha_k) =: \sum_{i=0}^{n+1} a_i x^i,$$

and then to recover each Lagrange polynomial by synthetic division:

$$\begin{aligned} q_i(x) &= \phi(x)/(x - \alpha_i), \\ l_i(x) &= q_i(x)/q_i(\alpha_i). \end{aligned}$$

The scalars $q_i(\alpha_i)$ can be computed by Horner's rule as the coefficients of q_i are formed.

Algorithm 22.1. Given distinct scalars $\alpha_0, \alpha_1, \dots, \alpha_n \in \mathbb{C}$ this algorithm computes $W = (w_{ij})_{i,j=0}^n = V(\alpha_0, \alpha_1, \dots, \alpha_n)^{-1}$.

% Stage 1: Construct the master polynomial.

$a_0 = -\alpha_0; a_1 = 1$

for $k = 1:n$

$a_{k+1} = 1$

 for $j = k:-1:1$

$a_j = a_{j-1} - \alpha_k a_j$

 end

$a_0 = -\alpha_k a_0$

end

% Stage 2: Synthetic division.

for $i = 0:n$

$w_{in} = 1; s = 1$

 for $j = n-1:-1:0$

$w_{ij} = a_{j+1} + \alpha_i w_{i,j+1}$

$s = \alpha_i s + w_{ij}$

 end

$w(i,:) = w(i,:)/s$

end

Cost: $6n^2$ flops.

The $O(n^2)$ complexity is optimal, since the algorithm has n^2 output values, each of which must partake in at least one operation.

Vandermonde matrices have the deserved reputation of being ill conditioned. The ill conditioning is a consequence of the monomials being a poor basis for the polynomials on the real line. A variety of bounds for the condition number of a Vandermonde matrix have been derived. Let $V_n = V(\alpha_0, \alpha_1, \dots, \alpha_{n-1}) \in \mathbb{C}^{n \times n}$. For arbitrary distinct points α_i ,

$$\max_i \prod_{j \neq i} \frac{\max(1, |\alpha_j|)}{|\alpha_i - \alpha_j|} \leq \|V_n^{-1}\|_\infty \leq \max_i \prod_{j \neq i} \frac{1 + |\alpha_j|}{|\alpha_i - \alpha_j|}, \quad (22.3)$$

Table 22.1. *Bounds and estimates for $\kappa(V_n)$.*

	α_i	Bound or estimate	Reference
(V1):	$\frac{1}{i+1}$	$\kappa_\infty(V_n) > n^{n+1}$	[468, 1990]
(V2):	arbitrary real	$\kappa_2(V_n) \geq \left(\frac{2}{n+1}\right)^{1/2} (1 + \sqrt{2})^{n-1}$	[98, 2000]
(V3):	$\alpha_i \geq 0$	$\kappa_2(V_n) \geq \frac{1}{2\sqrt{n+1}} [(1 + \sqrt{2})^{2n} + (1 + \sqrt{2})^{-2n}]$	[98, 2000]
(V4):	equispaced $[0, 1]$	$\kappa_\infty(V_n) \sim (4\pi)^{-1} \sqrt{2} 8^n$	[468, 1990]
(V5):	equispaced $[-1, 1]$	$\kappa_\infty(V_n) \sim \pi^{-1} e^{-\pi/4} (3.1)^n$	[465, 1975]
(V6):	Chebyshev nodes $[-1, 1]$	$\kappa_\infty(V_n) \sim \frac{3^{3/4}}{4} (1 + \sqrt{2})^n$	[465, 1975]
(V7):	roots of unity	$\kappa_2(V_n) = 1$	well known

with equality on the right when $\alpha_j = |\alpha_j|e^{i\theta}$ for all j with a fixed θ (in particular, when $\alpha_j \geq 0$ for all j) [464, 1962], [466, 1978]. Note that the upper and lower bounds differ by at most a factor 2^{n-1} . More specific bounds are given in Table 22.1, on which we now comment.

Bound (V1) and estimate (V4) follow from (22.3). The condition number for the harmonic points $1/(i+1)$ grows faster than $n!$; by contrast, the condition numbers of the notoriously ill-conditioned Hilbert and Pascal matrices grow only exponentially (see §§28.1 and 28.4). For *any* choice of real points the rate of growth is at least exponential (V2), and this rate is achieved for points equally spaced on $[0, 1]$. For points equally spaced on $[-1, 1]$ the condition number grows at a slower exponential rate than that for $[0, 1]$, and the growth rate is slower still for the zeros of the n th degree Chebyshev polynomial (V6). For one set of points the Vandermonde matrix is perfectly conditioned: the roots of unity, for which $V_n/\sqrt{n}$ is unitary.

22.2. Primal and Dual Systems

The standard Vandermonde matrix can be generalized in at least two ways: by allowing confluency of the points α_i and by replacing the monomials by other polynomials. An example of a confluent Vandermonde matrix is

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ \alpha_0 & 1 & 0 & \alpha_1 & 1 \\ \alpha_0^2 & 2\alpha_0 & 2 & \alpha_1^2 & 2\alpha_1 \\ \alpha_0^3 & 3\alpha_0^2 & 6\alpha_0 & \alpha_1^3 & 3\alpha_1^2 \\ \alpha_0^4 & 4\alpha_0^3 & 12\alpha_0^2 & \alpha_1^4 & 4\alpha_1^3 \end{bmatrix}. \quad (22.4)$$

The second, third, and fifth columns are obtained by “differentiating” the previous column. The transpose of a confluent Vandermonde matrix arises in Hermite interpolation; it is nonsingular if the points corresponding to the “nonconfluent columns” are distinct.

A *Vandermonde-like matrix* is defined by $P = (p_i(\alpha_j))_{i,j=0}^n$, where p_i is a polynomial of degree i . The case of practical interest is where the p_i satisfy a three-

term recurrence relation. In the rest of this chapter we will assume that the p_i do satisfy a three-term recurrence relation. A particular application is the solution of certain discrete Chebyshev approximation problems [12, 1984]. Incorporating confluency, we obtain a *confluent Vandermonde-like matrix*, defined by

$$P = P(\alpha_0, \alpha_1, \dots, \alpha_n) = [q_0(\alpha_0), q_1(\alpha_1), \dots, q_n(\alpha_n)] \in \mathbb{C}^{(n+1) \times (n+1)},$$

where the α_i are ordered so that equal points are contiguous, that is,

$$\alpha_i = \alpha_j \quad (i < j) \quad \Rightarrow \quad \alpha_i = \alpha_{i+1} = \dots = \alpha_j, \quad (22.5)$$

and the vectors $q_j(x)$ are defined recursively by

$$q_j(x) = \begin{cases} [p_0(x), p_1(x), \dots, p_n(x)]^T, & \text{if } j = 0 \text{ or } \alpha_j \neq \alpha_{j-1}, \\ \frac{d}{dx} q_{j-1}(x), & \text{otherwise.} \end{cases}$$

For all polynomials and points, P is nonsingular; this follows from the derivation of the algorithms below. One reason for the interest in Vandermonde-like matrices is that for certain polynomials they tend to be better conditioned than Vandermonde matrices (see, for example, Problem 22.5). Gautschi [467, 1983] derives bounds for the condition numbers of Vandermonde-like matrices.

Fast algorithms for solving the confluent Vandermonde-like primal and dual systems $Px = b$ and $P^T a = f$ can be derived under the assumption that the $p_j(x)$ satisfy the three-term recurrence relation

$$p_{j+1}(x) = \theta_j(x - \beta_j)p_j(x) - \gamma_j p_{j-1}(x), \quad j \geq 1, \quad (22.6a)$$

$$p_0(x) = 1, \quad p_1(x) = \theta_0(x - \beta_0)p_0(x), \quad (22.6b)$$

where $\theta_j \neq 0$ for all j . Note that in this chapter γ_i denotes a constant in the recurrence relation and not $iu/(1 - iu)$ as elsewhere in the book. The latter notation is not used in this chapter.

The algorithms exploit the connection with interpolation. Denote by $r(i) \geq 0$ the smallest integer for which $\alpha_{r(i)} = \alpha_{r(i)+1} = \dots = \alpha_i$. Considering first the dual system $P^T a = f$, we note that

$$\psi(x) = \sum_{i=0}^n a_i p_i(x) \quad (22.7)$$

satisfies

$$\psi^{(i-r(i))}(\alpha_i) = f_i, \quad i = 0:n.$$

Thus ψ is a Hermite interpolating polynomial for the data $\{\alpha_i, f_i\}$, and our task is to obtain its representation in terms of the basis $\{p_i(x)\}_{i=0}^n$. As a first step we construct the divided difference form of ψ ,

$$\psi(x) = \sum_{i=0}^n c_i \prod_{j=0}^{i-1} (x - \alpha_j). \quad (22.8)$$

The (confluent) divided differences $c_i = f[\alpha_0, \alpha_1, \dots, \alpha_i]$ may be generated using the recurrence relation

$$f[\alpha_{j-k-1}, \dots, \alpha_j] = \begin{cases} \frac{f[\alpha_{j-k}, \dots, \alpha_j] - f[\alpha_{j-k-1}, \dots, \alpha_{j-1}]}{\alpha_j - \alpha_{j-k-1}}, & \alpha_j \neq \alpha_{j-k-1}, \\ \frac{f_{r(j)+k+1}}{(k+1)!}, & \alpha_j = \alpha_{j-k-1}. \end{cases} \quad (22.9)$$

Now we need to generate the a_i in (22.7) from the c_i in (22.8). The idea is to expand (22.8) using nested multiplication and use the recurrence relations (22.6) to express the results as a linear combination of the p_j . Define

$$q_n(x) = c_n, \quad (22.10)$$

$$q_k(x) = (x - \alpha_k)q_{k+1}(x) + c_k, \quad k = n-1: -1: 0, \quad (22.11)$$

from which $q_0(x) = \psi(x)$. Let

$$q_k(x) = \sum_{j=0}^{n-k} a_{k+j}^{(k)} p_j(x). \quad (22.12)$$

To obtain recurrences for the coefficients $a_j^{(k)}$ we expand the right-hand side of (22.11), giving

$$q_k(x) = (x - \alpha_k) \sum_{j=0}^{n-k-1} a_{k+j+1}^{(k+1)} p_j(x) + c_k.$$

Using the relations, from (22.6),

$$xp_0(x) = \frac{1}{\theta_0} p_1(x) + \beta_0,$$

$$xp_j(x) = \frac{1}{\theta_j} (p_{j+1}(x) + \gamma_j p_{j-1}(x)) + \beta_j p_j(x), \quad j \geq 1,$$

we obtain, for $k = 0: n-2$,

$$\begin{aligned} q_k(x) &= a_{k+1}^{(k+1)} \left(\frac{1}{\theta_0} p_1(x) + \beta_0 \right) \\ &+ \sum_{j=1}^{n-k-1} a_{k+j+1}^{(k+1)} \left(\frac{1}{\theta_j} (p_{j+1}(x) + \gamma_j p_{j-1}(x)) + \beta_j p_j(x) \right) \\ &- \alpha_k \sum_{j=0}^{n-k-1} a_{k+j+1}^{(k+1)} p_j(x) + c_k \\ &= c_k + (\beta_0 - \alpha_k) a_{k+1}^{(k+1)} + \frac{\gamma_1}{\theta_1} a_{k+2}^{(k+1)} \\ &+ \sum_{j=1}^{n-k-2} \left(\frac{1}{\theta_{j-1}} a_{k+j}^{(k+1)} + (\beta_j - \alpha_k) a_{k+j+1}^{(k+1)} + \frac{\gamma_{j+1}}{\theta_{j+1}} a_{k+j+2}^{(k+1)} \right) p_j(x) \end{aligned}$$

$$\begin{aligned}
& + \left(\frac{1}{\theta_{n-k-2}} a_{n-1}^{(k+1)} + (\beta_{n-k-1} - \alpha_k) a_n^{(k+1)} \right) p_{n-k-1}(x) \\
& + \frac{1}{\theta_{n-k-1}} a_n^{(k+1)} p_{n-k}(x),
\end{aligned} \tag{22.13}$$

in which the empty summation is defined to be zero. For the special case $k = n-1$ we have

$$q_{n-1}(x) = c_{n-1} + (\beta_0 - \alpha_{n-1}) a_n^{(n)} + \frac{1}{\theta_0} a_n^{(n)} p_1(x). \tag{22.14}$$

Recurrences for the coefficients $a_j^{(k)}$, $j = k:n$, in terms of $a_j^{(k+1)}$, $j = k+1:n$, follow immediately by comparing (22.12) with (22.13) and (22.14).

In the following algorithm, stage I computes the confluent divided differences and stage II implements the recurrences derived above.

Algorithm 22.2 (dual, $P^T a = f$). Given parameters $\{\theta_j, \beta_j, \gamma_j\}_{j=0}^{n-1}$, a vector f , and points $\alpha(0:n) \in \mathbb{C}^{n+1}$ satisfying (22.5), this algorithm solves the dual confluent Vandermonde-like system $P^T a = f$.

```

% Stage I:
Set  $c = f$ 
for  $k = 0:n-1$ 
     $clast = c_k$ 
    for  $j = k+1:n$ 
        if  $\alpha_j = \alpha_{j-k-1}$  then
             $c_j = c_j / (k+1)$ 
        else
             $temp = c_j$ 
             $c_j = (c_j - clast) / (\alpha_j - \alpha_{j-k-1})$ 
             $clast = temp$ 
        end
    end
end
end

% Stage II:
Set  $a = c$ 
 $a_{n-1} = a_{n-1} + (\beta_0 - \alpha_{n-1}) a_n$ 
 $a_n = a_n / \theta_0$ 
for  $k = n-2:-1:0$ 
     $a_k = a_k + (\beta_0 - \alpha_k) a_{k+1} + (\gamma_1 / \theta_1) a_{k+2}$ 
    for  $j = 1:n-k-2$ 
         $a_{k+j} = a_{k+j} / \theta_{j-1} + (\beta_j - \alpha_k) a_{k+j+1} + (\gamma_{j+1} / \theta_{j+1}) a_{k+j+2}$ 
    end
     $a_{n-1} = a_{n-1} / \theta_{n-k-2} + (\beta_{n-k-1} - \alpha_k) a_n$ 
     $a_n = a_n / \theta_{n-k-1}$ 
end
end

```

Assuming that the values γ_j / θ_j are given (note that γ_j appears only in the terms γ_j / θ_j), the computational cost of Algorithm 22.2 is at most $9n^2/2$ flops.

Table 22.2. *Parameters in the three-term recurrence (22.6).*

Polynomial	θ_j	β_j	γ_j	
Monomials	1	0	0	
Chebyshev	2^*	0	1	$^*\theta_0 = 1$
Legendre*	$\frac{2j+1}{j+1}$	0	$\frac{j}{j+1}$	$^*p_j(1) \equiv 1$
Hermite	2	0	$2j$	
Laguerre	$-\frac{1}{j+1}$	$2j+1$	$\frac{j}{j+1}$	

The vectors c and a have been used for clarity; in fact both can be replaced by f , so that the right-hand side is transformed into the solution without using any extra storage.

Values of θ_j , β_j , γ_j for some polynomials of interest are collected in Table 22.2.

The key to deriving a corresponding algorithm for solving the primal system is to recognize that Algorithm 22.2 implicitly computes a factorization of P^{-T} into the product of $2n$ triangular matrices. In the rest of this chapter we adopt the convention that the subscripts of all vectors and matrices run from 0 to n . In stage I, letting $c^{(k)}$ denote the vector c at the start of the k th iteration of the outer loop, we have

$$c^{(0)} = f, \quad c^{(k+1)} = L_k c^{(k)}, \quad k = 0:n-1. \quad (22.15)$$

The matrix L_k is lower triangular and agrees with the identity matrix in rows 0 to k . The remaining rows can be described, for $k+1 \leq j \leq n$, by

$$e_j^T L_k = \begin{cases} e_j^T / (k+1), & \text{if } \alpha_j = \alpha_{j-k-1}, \\ (e_j^T - e_s^T) / (\alpha_j - \alpha_{j-k-1}), & \text{some } s < j, \text{ otherwise,} \end{cases}$$

where e_j is column j of the identity matrix. Similarly, stage II can be expressed as

$$a^{(n)} = c^{(n)}, \quad a^{(k)} = U_k a^{(k+1)}, \quad k = n-1:-1:0. \quad (22.16)$$

The matrix U_k is upper triangular, it agrees with the identity matrix in rows 0 to $k-1$, and it has zeros everywhere above the first two superdiagonals.

From (22.15) and (22.16) we see that the overall effect of Algorithm 22.2 is to evaluate, step-by-step, the product

$$a = U_0 \dots U_{n-1} L_{n-1} \dots L_0 f \equiv P^{-T} f. \quad (22.17)$$

Taking the transpose of this product we obtain a representation of P^{-1} , from which it is easy to write down an algorithm for computing $x = P^{-1}b$.

Algorithm 22.3 (primal, $Px = b$). Given parameters $\{\theta_j, \beta_j, \gamma_j\}_{j=0}^{n-1}$, a vector b , and points $\alpha(0:n) \in \mathbb{C}^{n+1}$ satisfying (22.5), this algorithm solves the primal confluent Vandermonde-like system $Px = b$.

% Stage I:

Set $d = b$

```

for  $k = 0:n-2$ 
  for  $j = n-k:-1:2$ 
     $d_{k+j} = (\gamma_{j-1}/\theta_{j-1})d_{k+j-2} + (\beta_{j-1} - \alpha_k)d_{k+j-1} + d_{k+j}/\theta_{j-1}$ 
  end
   $d_{k+1} = (\beta_0 - \alpha_k)d_k + d_{k+1}/\theta_0$ 
end
 $d_n = (\beta_0 - \alpha_{n-1})d_{n-1} + d_n/\theta_0$ 
% Stage II:
Set  $x = d$ 
for  $k = n-1:-1:0$ 
   $xlast = 0$ 
  for  $j = n:-1:k+1$ 
    if  $\alpha_j = \alpha_{j-k-1}$  then
       $x_j = x_j/(k+1)$ 
    else
       $temp = x_j/(\alpha_j - \alpha_{j-k-1})$ 
       $x_j = temp - xlast$ 
       $xlast = temp$ 
    end
  end
   $x_k = x_k - xlast$ 
end

```

Algorithm 22.3 has, by construction, the same operation count as Algorithm 22.2.

22.3. Stability

Algorithms 22.2 and 22.3 have interesting stability properties. Depending on the problem parameters, the algorithms can range from being very stable (in either a backward or forward sense) to very unstable.

When the p_i are the monomials and the points α_i are distinct, the algorithms reduce to those of Björck and Pereyra [133, 1970]. Björck and Pereyra found that for the system $Vx = b$ with $\alpha_i = 1/(i+3)$, $b_i = 2^{-i}$, $n = 9$, and on a computer with $u \approx 10^{-16}$,

$$\kappa_{\infty}(V) = 9 \times 10^{13}, \quad \max_i \frac{|x_i - \hat{x}_i|}{|x_i|} = 5u.$$

Thus the computed solution has a tiny componentwise relative error, despite the extreme ill condition of V . Björck and Pereyra comment “It seems as if at least some problems connected with Vandermonde systems, which traditionally have been considered too ill-conditioned to be attacked, actually can be solved with good precision.” This high accuracy can be explained with the aid of the error analysis below.

The analysis can be kept quite short by exploiting the interpretation of the algorithms in terms of matrix–vector products. Because of the inherent duality between Algorithms 22.2 and 22.3, any result for one has an analogue for the other, so we will consider only Algorithm 22.2.

22.3.1. Forward Error

Theorem 22.4. *If no underflows or overflows are encountered then Algorithm 22.2 runs to completion and the computed solution $\hat{a}$ satisfies*

$$|a - \hat{a}| \leq c(n, u) |U_0| \dots |U_{n-1}| |L_{n-1}| \dots |L_0| |f|, \quad (22.18)$$

where $c(n, u) := (1 + u)^{7n} - 1 = 7nu + O(u^2)$.

Proof. First, note that Algorithm 22.2 must succeed in the absence of underflow and overflow, because division by zero cannot occur.

The analysis of the computation of the $c^{(k)}$ vectors is exactly the same as that for the nonconfluent divided differences in §5.3 (see (5.9) and (5.10)). However, we obtain a slightly cleaner error bound by dropping the γ_k notation and instead writing

$$\hat{c}^{(k+1)} = (L_k + \Delta L_k) \hat{c}^{(k)}, \quad |\Delta L_k| \leq [(1 + u)^3 - 1] |L_k|. \quad (22.19)$$

Turning to the equations (22.16), we can regard the multiplication $a^{(k)} = U_k a^{(k+1)}$ as comprising a sequence of three-term inner products. Analysing these in standard fashion we arrive at the equation

$$\hat{a}^{(k)} = (U_k + \Delta U_k) \hat{a}^{(k+1)}, \quad |\Delta U_k| \leq [(1 + u)^4 - 1] |U_k|, \quad (22.20)$$

where we have taken into account the rounding errors in forming $u_{i,i+1}^{(k)} = \beta_j - \alpha_k$ and $u_{i,i+2}^{(k)} = \gamma_{j+1}/\theta_{j+1}$ ($i = k + j$).

Since $\hat{c}^{(0)} = f$, and $\hat{a} = \hat{a}^{(0)}$, (22.19) and (22.20) imply that

$$\hat{a} = (U_0 + \Delta U_0) \dots (U_{n-1} + \Delta U_{n-1}) (L_{n-1} + \Delta L_{n-1}) \dots (L_0 + \Delta L_0) f. \quad (22.21)$$

Applying Lemma 3.8 to (22.21) and using (22.17), we obtain the desired bound for the forward error. $\square$

The product $|U_0| \dots |U_{n-1}| |L_{n-1}| \dots |L_0|$ in (22.18) is an upper bound for $|U_0 \dots U_{n-1} L_{n-1} \dots L_0| = |P^{-T}|$ and is equal to it when there is no subtractive cancellation in the latter product. To gain insight, suppose the points are distinct and consider the case $n = 3$. We have

$$\begin{aligned} P^{-T} &= U_0 U_1 U_2 L_2 L_1 L_0 \\ &\equiv \begin{bmatrix} 1 & \beta_0 - \alpha_0 & \gamma_1/\theta_1 & 0 \\ & \theta_0^{-1} & \beta_1 - \alpha_0 & \gamma_2/\theta_2 \\ & & \theta_1^{-1} & \beta_2 - \alpha_0 \\ & & & \theta_2^{-1} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ & 1 & \beta_0 - \alpha_1 & \gamma_1/\theta_1 \\ & & \theta_0^{-1} & \beta_1 - \alpha_1 \\ & & & \theta_1^{-1} \end{bmatrix} \\ &\quad \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ & 1 & 0 & 0 \\ & & 1 & \beta_0 - \alpha_2 \\ & & & \theta_0^{-1} \end{bmatrix} \begin{bmatrix} 1 & & & \\ 0 & 1 & & \\ 0 & 0 & \frac{1}{\alpha_3 - \alpha_0} & \frac{1}{\alpha_3 - \alpha_0} \\ 0 & 0 & \frac{-1}{\alpha_3 - \alpha_0} & \frac{1}{\alpha_3 - \alpha_0} \end{bmatrix} \\ &\quad \times \begin{bmatrix} 1 & & & \\ 0 & \frac{1}{\alpha_2 - \alpha_0} & \frac{1}{\alpha_2 - \alpha_0} & \\ 0 & \frac{-1}{\alpha_2 - \alpha_0} & \frac{-1}{\alpha_2 - \alpha_0} & \\ 0 & 0 & \frac{1}{\alpha_3 - \alpha_1} & \frac{1}{\alpha_3 - \alpha_1} \end{bmatrix} \end{aligned}$$

$$\times \begin{bmatrix} 1 & & & \\ \frac{-1}{\alpha_1 - \alpha_0} & \frac{1}{\alpha_1 - \alpha_0} & & \\ 0 & \frac{-1}{\alpha_2 - \alpha_1} & \frac{1}{\alpha_2 - \alpha_1} & \\ 0 & 0 & \frac{-1}{\alpha_3 - \alpha_2} & \frac{1}{\alpha_3 - \alpha_2} \end{bmatrix}. \quad (22.22)$$

There is no subtractive cancellation in this product as long as each matrix has the alternating (checkerboard) sign pattern defined, for $A = (a_{ij})$, by $(-1)^{i+j}a_{ij} \geq 0$. This sign pattern holds for the matrices L_i if the points α_i are arranged in increasing order. The matrices U_i have the required sign pattern provided that (in general)

$$\theta_i > 0, \quad \gamma_i \geq 0 \quad \text{for all } i, \quad \text{and} \quad \beta_i - \alpha_k \leq 0 \quad \text{for all } i + k \leq n - 1.$$

In view of Table 22.2 we have the following result.

Corollary 22.5. *If $0 \leq \alpha_0 < \alpha_1 < \cdots < \alpha_n$ then for the monomials, or the Chebyshev, Legendre, or Hermite polynomials,*

$$|a - \hat{a}| \leq c(n, u) |P^{-T}| |f|. \quad \square$$

Corollary 22.5 explains the high accuracy observed by Björck and Pereyra. Note that if

$$|P^{-T}| |f| \leq t_n |P^{-T} f| = t_n |a|$$

then, under the conditions of the corollary, $|\hat{a} - a| \leq c(n, u)t_n|a|$, which shows that the componentwise relative error is bounded by $c(n, u)t_n$. For the problem of Björck and Pereyra it can be shown that $t_n \approx n^4/24$. Another factor contributing to the high accuracy in this problem is that many of the subtractions $\alpha_j - \alpha_{j-k-1} = 1/(j+3) - 1/(j-k+2)$ are performed exactly, in view of Theorem 2.5.

Note that under the conditions of the corollary P^{-T} has the alternating sign pattern, since each of its factors does. Thus if $(-1)^i f_i \geq 0$ then $t_n = 1$, and the corollary implies that $\hat{a}$ is accurate essentially to full machine precision, *independent of the condition number* $\kappa_\infty(P)$. In particular, taking f to be a column of the identity matrix shows that we can compute the inverse of P to high relative accuracy, independent of its condition number.

22.3.2. Residual

Next we look at the residual, $r = f - P^T \hat{a}$. Rewriting (22.21),

$$f = (L_0 + \Delta L_0)^{-1} \cdots (L_{n-1} + \Delta L_{n-1})^{-1} (U_{n-1} + \Delta U_{n-1})^{-1} \cdots (U_0 + \Delta U_0)^{-1} \hat{a}. \quad (22.23)$$

From the proof of Theorem 22.4 and the relation (5.9) it is easy to show that

$$(L_k + \Delta L_k)^{-1} = L_k^{-1} + E_k, \quad |E_k| \leq [(1-u)^{-3} - 1] |L_k^{-1}|.$$

Strictly, an analogous bound for $(U_k + \Delta U_k)^{-1}$ does not hold, since ΔU_k cannot be expressed in the form of a diagonal matrix times U_k . However, it seems reasonable to make a simplifying assumption that such a bound is valid, say

$$(U_k + \Delta U_k)^{-1} = U_k^{-1} + F_k, \quad |F_k| \leq c(n, u) |U_k^{-1}|. \quad (22.24)$$

Then, writing (22.23) as

$$f = (L_0^{-1} + E_0) \dots (L_{n-1}^{-1} + E_{n-1})(U_{n-1}^{-1} + F_{n-1}) \dots (U_0^{-1} + F_0)\hat{a}$$

and invoking Lemma 3.8, we obtain the following result.

Theorem 22.6. *Under the assumption (22.24), the residual of the computed solution $\hat{a}$ from Algorithm 22.2 is bounded by*

$$|f - P^T \hat{a}| \leq d(n, u) |L_0^{-1}| \dots |L_{n-1}^{-1}| |U_{n-1}^{-1}| \dots |U_0^{-1}| |\hat{a}|, \quad (22.25)$$

where $d(n, u) := (1 + c(n, u))^n (1 - u)^{-3n} - 1 = n(c(n, u) + 3u) + O(u^2)$. $\square$

We now specialize to the monomials, for which L_i and U_i are bidiagonal. Assumption (22.24) can be shown to hold with $c(n, u) = (n+1)u + O(u^2)$; see Problem 22.8. For distinct, nonnegative points arranged in increasing order the inverses of L_i and U_i have nonnegative elements and, since $P^T = L_0^{-1} \dots L_{n-1}^{-1} U_{n-1}^{-1} \dots U_0^{-1}$, we obtain from (22.25) the following pleasing result, which guarantees a tiny componentwise relative backward error.

Corollary 22.7. *Let $0 \leq \alpha_0 < \alpha_1 < \dots < \alpha_n$, and consider Algorithm 22.2 for the monomials. The computed solution $\hat{a}$ satisfies*

$$|f - P^T \hat{a}| \leq (n(n+4)u + O(u^2)) |P^T| |\hat{a}|. \quad \square$$

22.3.3. Dealing with Instability

The potential instability of Algorithm 22.2 is illustrated by the following example. Take the Chebyshev polynomials T_i with the points $\alpha_i = \cos(i\pi/n)$ (the extrema of T_n), and define the right-hand side by $f_i = (-1)^i$. The exact solution to $P^T a = f$ is the last column of the identity matrix. Relative errors and residuals are shown in Table 22.3 ($u \approx 10^{-16}$). Even though $\kappa_2(P) \leq 2$ for all n (see Problem 22.7), the forward errors and relative residuals are large and grow with n . The reason for the instability is that there are large intermediate numbers in the algorithm (at the start of stage II for $n = 40$, $\|c\|_\infty$ is of order 10^{15}); hence severe cancellation is necessary to produce the final answer of order 1. Looked at another way, the factorization of P^T used by the algorithm is unstable because it is very sensitive to perturbations in the factors.

How can we overcome this instability? There are two possibilities: prevention and cure. The only means at our disposal for *preventing* instability is to reorder the points α_i . The ordering is arbitrary subject to condition (22.5) being satisfied. Recall that the algorithms construct an LU factorization of P^T in factored form, and note that permuting the rows of P^T is equivalent to reordering the points α_i . A reasonable approach is therefore to take whatever ordering of the points would be produced by GEPP applied to P^T . Since the diagonal elements of U in $P^T = LU$ have the form

$$u_{ii} = h_i \prod_{j=0}^{i-1} (\alpha_i - \alpha_j), \quad i = 0:n,$$

Table 22.3. *Results for dual Chebyshev–Vandermonde-like system.*

n	10	20	30	40
$\frac{\ a - \hat{a}\ _\infty}{\ a\ _\infty}$	2.5×10^{-12}	6.3×10^{-7}	4.7×10^{-2}	1.8×10^3
$\frac{\ f - P^T \hat{a}\ _\infty}{\ P\ _\infty \ \hat{a}\ _\infty + \ f\ _\infty}$	6.0×10^{-13}	1.1×10^{-7}	5.3×10^{-3}	8.3×10^{-2}

where h_i depends only on the θ_i , and since partial pivoting maximizes the pivot at each stage, this ordering of the α_i can be computed at the start of the algorithm in n^2 flops (see Problem 22.9). This ordering is essentially the Leja ordering (5.13) (the difference is that partial pivoting leaves α_0 unchanged).

To attempt to *cure* observed instability we can use iterative refinement in fixed precision. Ordinarily, residual computation for linear equations is trivial, but in this context the coefficient matrix is not given explicitly and computing the residual turns out to be conceptually almost as difficult, and computationally as expensive, as solving the linear system!

To compute the residual for the dual system we need a means for evaluating $\psi(t)$ in (22.7) and its first $k \leq n$ derivatives, where $k = \max_i(i - r(i))$ is the *order of confluency*. Since the polynomials p_j satisfy a three-term recurrence relation we can use an extension of the Clenshaw recurrence formula (which evaluates ψ but not its derivatives). The following algorithm implements the appropriate recurrences, which are given by Smith [1050, 1965] (see Problem 22.10).

Algorithm 22.8 (extended Clenshaw recurrence). This algorithm computes the $k + 1$ values $y_j = \psi^{(j)}(x)$, $0 \leq j \leq k$, where ψ is given by (22.7) and $k \leq n$. It uses a work vector z of order k .

```

Set  $y(0:k) = z(0:k) = 0$ 
 $y_0 = a_n$ 
for  $j = n - 1 : -1 : 0$ 
     $temp = y_0$ 
     $y_0 = \theta_j(x - \beta_j)y_0 - \gamma_{j+1}z_0 + a_j$ 
     $z_0 = temp$ 
    for  $i = 1 : \min(k, n - j)$ 
         $temp = y_i$ 
         $y_i = \theta_j((x - \beta_j)y_i + z_{i-1}) - \gamma_{j+1}z_i$ 
         $z_i = temp$ 
    end
end
 $m = 1$ 
for  $i = 2 : k$ 
     $m = m * i$ 
     $y_i = m * y_i$ 
end

```

Computing the residual using Algorithm 22.8 costs between $3n^2$ flops (for full confluency) and $6n^2$ flops (for the nonconfluent case); recall that Algorithm 22.2 costs at most $9n^2/2$ flops!

The use of iterative refinement can be justified with the aid of Theorem 12.3. For (confluent) Vandermonde matrices, the residuals are formed using Horner's rule and (12.9) holds in view of (5.3) and (5.7). Hence for standard Vandermonde matrices, Theorem 12.3 leads to an asymptotic componentwise backward stability result. A complete error analysis of Algorithm 22.8 is not available for (confluent) Vandermonde-like matrices, but it is easy to see that (12.9) will not always hold. Nevertheless it is clear that a *normwise* bound can be obtained (see Oliver [901, 1977] for the special case of the Chebyshev polynomials) and hence an asymptotic normwise stability result can be deduced from Theorem 12.3. Thus there is theoretical backing for the use of iterative refinement with Algorithm 22.8.

Numerical experiments using Algorithm 22.8 in conjunction with the partial pivoting reordering and fixed precision iterative refinement show that both techniques are effective means for stabilizing the algorithms, but that iterative refinement is likely to fail once the instability is sufficiently severe. Because of its lower cost, the reordering approach is preferable.

Two heuristics are worth noting. Consider a (confluent) Vandermonde-like system $Px = b$. *Heuristic 1*: it is systems with a large normed solution ($\|x\| \approx \|P^{-1}\| \|b\|$) that are solved to high accuracy by the fast algorithms. To produce a large solution the algorithms must sustain little cancellation, and the error analysis shows that cancellation is the main cause of instability. *Heuristic 2*: GEPP is unable to solve accurately Vandermonde systems with a *very* large normed solution ($\|x\| \gg u^{-1}\|b\|/\|P\|$). The pivots for GEPP will tend to satisfy $|u_{ii}| \gtrsim u\|P\|$, so that the computed solution will tend to satisfy $\|\hat{x}\| \leq u^{-1}\|b\|/\|P\|$. A consequence of these two heuristics is that for Vandermonde(-like) systems with a very large-normed solution the fast algorithms will be much more accurate (but no more backward stable) than GEPP. However, we should be suspicious of any framework in which such systems arise; although the solution vector may be obtained accurately (barring overflow), subsequent computations with numbers of such a wide dynamic range will probably themselves be unstable.

22.4. Notes and References

The formulae (22.1) and (22.2), and inversion methods based on these formulae, have been discovered independently by many authors. Traub [1150, 1966, §14] gives a short historical survey, his earliest reference being a 1932 book by Kowalewski. There does not appear to be any published error analysis for Algorithm 22.1 (see Problem 22.3). There is little justification for using the output of the algorithm to solve the primal or dual linear system, as is done in [953, 1992, §2.8]; Algorithms 22.2 and 22.3 are more efficient and almost certainly at least as stable. Calvetti and Reichel [198, 1993] generalize Algorithm 22.1 to Vandermonde-like matrices, but they do not present any error analysis. Gohberg and Olshevsky [495, 1994] give another $O(n^2)$ flops algorithm for inverting a Chebyshev–Vandermonde matrix.

The standard condition number $\kappa(V)$ is not an appropriate measure of sensi-

tivity when only the points α_i are perturbed, because it does not reflect the special structure of the perturbations. Appropriate condition numbers were first derived by Higham [581, 1987] and are comprehensively investigated by Bartels and D. J. Higham [86, 1992]; see Problem 22.11.

Structured backward errors for Vandermonde systems are defined and investigated by Sun [1109, 1998], who obtains upper and lower bounds.

Methods for solving the dual and primal Vandermonde systems have an interesting history. The earliest algorithm was derived by Lyness and Moler [801, 1966] via Neville interpolation; it solves the dual system in $O(n^3)$ flops. The first $O(n^2)$ algorithm was obtained by Ballester and Pereyra [61, 1967]; it computes the LU factors of the Vandermonde matrix and requires $O(n^2)$ elements of storage. Björck and Pereyra [133, 1970] derived the specialization of Algorithms 22.2 and 22.3 to nonconfluent Vandermonde matrices; these algorithms require no storage other than that for the problem data. The algorithms of Björck and Pereyra were generalized by Björck and Elfving to confluent systems [129, 1973], and by Higham to Vandermonde-like systems [584, 1988] and confluent Vandermonde-like systems [595, 1990]. The error analysis in this chapter is based on [595, 1990]. Tang and Golub [1126, 1981] give a block algorithm that requires only real arithmetic to solve a Vandermonde system in which all the points appear in complex conjugate pairs.

Other $O(n^2)$ algorithms for solving Chebyshev–Vandermonde systems are given by Reichel and Opfer [977, 1991] and Calvetti and Reichel [197, 1992]. The former algorithms are progressive, in that they allow the solution to be updated when a new point α_i is added; they generalize progressive algorithms of Björck and Pereyra [133, 1970]. Boros, Kailath, and Olshevsky [151, 1994] use the concept of displacement structure to derive further $O(n^2)$ algorithms for solving Vandermonde and Chebyshev–Vandermonde systems. No error analysis is given in [151, 1994], [197, 1992], or [977, 1991].

The $O(n^2)$ complexity of the algorithms mentioned above for solving Vandermonde-like systems is not optimal. Lu [797, 1994], [798, 1995], [799, 1996] derives $O(n \log n \log p)$ flops algorithms, where p is the number of distinct points. The numerical stability and practical efficiency of the algorithms remain to be determined. Bini and Pan [111, 1994] give an $O(n \log^2 n)$ algorithm for solving a dual Vandermonde system that involves solving related Toeplitz and Hankel systems.

Since Vandermonde systems can be solved in less than $O(n^3)$ flops it is natural to ask whether the $O(mn^2)$ complexity of QR factorization of an $m \times n$ matrix can be bettered for a Vandermonde matrix. QR factorization algorithms with $O(mn)$ flop counts have been derived by Demeure [305, 1989], [306, 1990], and for Vandermonde-like matrices where the polynomials satisfy a three-term recurrence by Reichel [976, 1991]. No error analysis has been published for these algorithms. Demeure's algorithms are likely to be unstable, because they form the normal equations matrix $V^T V$.

Problems

22.1. Derive a modified version of Algorithm 22.1 in which the scale factor $s = q_i(\alpha_i)$ is computed directly as

$$\prod_{\substack{k=0 \\ k \neq i}}^n (\alpha_i - \alpha_k).$$

What is the flop count for this version?

22.2. (Calvetti and Reichel [198, 1993]) Generalize Algorithm 22.1 to the inversion of a Vandermonde-like matrix for polynomials that satisfy a three-term recurrence relation.

22.3. Investigate the stability of Algorithm 22.1 and the modified version of Problem 22.1. (a) Evaluate the left and right residuals of the computed inverses; compare the results with those for GEPP. (b) Show that Algorithm 22.1 always performs subtractions of like-signed numbers and explain the implications for stability. (Does Algorithm 22.1 share the high accuracy properties discussed at the end of §22.3.1?) (c) (RESEARCH PROBLEM) Derive and explore forward error bounds and residual bounds for both algorithms. Extend the analysis to the algorithms of Calvetti and Reichel [198, 1993].

22.4. By summing (22.1) for $i = 0:n$, show that $\sum_{i=0}^n w_{ij} = \delta_{j0}$. What does this imply about the sign pattern of V^{-1} ? What is the sum of all the elements of V^{-1} ?

22.5. Let $T = T(\alpha_0, \alpha_1, \dots, \alpha_n) = (T_i(\alpha_j))_{i,j=0}^n$ be a Chebyshev–Vandermonde matrix (T_j is the Chebyshev polynomial of degree j), with $T^{-1} = (u_{ij})$. Analogously to (22.1) we have $\sum_{j=0}^n u_{ij} T_j(x) = l_i(x)$. Hence $\sum_{j=0}^n w_{ij} x^j = \sum_{j=0}^n u_{ij} T_j(x)$, where $V^{-1} = V(\alpha_0, \alpha_1, \dots, \alpha_n)^{-1} = (w_{ij})$. Show that

$$\sum_{j=0}^n |u_{ij}| \leq \sum_{j=0}^n |w_{ij}|,$$

and hence that

$$\|T^{-1}\|_{\infty} \leq \|V^{-1}\|_{\infty}.$$

(Hint: show that $T = LV$ for a lower triangular matrix L and use the fact that $x^n = \frac{1}{2^{n+1}} \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{k} T_{n-2k}(x)$.)

22.6. Show that for a nonconfluent Vandermonde-like matrix $P = (P_i(\alpha_j))_{i,j=0}^n$, where the p_i satisfy (22.6),

$$\det(P) = \prod_{i=0}^{n-1} \theta_i^{n-i} \prod_{i>j} (\alpha_i - \alpha_j).$$

(Hint: study the structure of (22.22).)

22.7. Show that for the Chebyshev–Vandermonde-like matrix $T = T(\alpha_0, \alpha_1, \dots, \alpha_n)$,

1. $\kappa_2(T) = \sqrt{2}$, for $\alpha_i = \cos((i + \frac{1}{2})\pi/(n+1))$ (zeros of T_{n+1}).

2. $\kappa_2(T) \leq 2$, for $\alpha_i = \cos(i\pi/n)$ (extrema of T_n).

(Hint: use the discrete orthogonality properties of the Chebyshev polynomials; see, e.g., Hamming [545, 1973, pp. 472–473].)

22.8. Let $U \in \mathbb{R}^{n \times n}$ be upper bidiagonal and let ΔU be a perturbation such that $\Delta u_{ij} = \delta_{ij}u_{ij}$. Show that

$$((U + \Delta U)^{-1} - U^{-1})_{ij} = \frac{1}{1 + \delta_{jj}} \left(\prod_{p=i}^{j-1} \frac{-(1 + \delta_{p,p+1})}{1 + \delta_{pp}} \right) (U^{-1})_{ij}.$$

Interpret this equality. Deduce that for $U_k + \Delta U_k$ in (22.24) we have

$$(U_k + \Delta U_k)^{-1} = U_k^{-1} + F_k, \quad |F_k| \leq \frac{(n+1)u}{1 - (n+1)u} |U_k^{-1}|.$$

22.9. Derive an $O(n^2)$ flops algorithm that reorders the distinct points $\alpha_0, \alpha_1, \dots, \alpha_n \in \mathbb{C}$ according to the same permutation that would be produced by GEPP applied to $P^T(\alpha_0, \alpha_1, \dots, \alpha_n)$. (Cf. Problem 5.4.) Can your algorithm ever produce the increasing ordering?

22.10. Derive Algorithm 22.8 by differentiating repeatedly the original Clenshaw recurrence (which is Algorithm 22.8 with $k = 0$) and rescaling so as to consign factorial terms to a cleanup loop at the end. Derive an algorithm for computing the residual for the primal system in a similar way, using recurrences obtained by differentiating (22.6).

22.11. (Higham [581, 1987]) A structured condition number for the primal Vandermonde system $Vx = b$, where $V = V(\alpha_0, \alpha_1, \dots, \alpha_n)$, can be defined by

$$\text{cond}(V) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{\|\Delta x\|_\infty}{\epsilon \|x\|_\infty} : V(\alpha_0 + \Delta\alpha_0, \dots, \alpha_n + \Delta\alpha_n)(x + \Delta x) = b, \right. \\ \left. |\Delta\alpha_i| \leq \epsilon |\alpha_i|, \quad i = 0:n \right\}.$$

Show that

$$\text{cond}(V) = \frac{\|V^{-1} \text{diag}(0, 1, 2, \dots, n)V\|_\infty}{\|x\|_\infty},$$

and derive a corresponding condition number for the dual system $V^T a = f$.

Chapter 23

Fast Matrix Multiplication

A simple but extremely valuable bit of equipment in matrix multiplication consists of two plain cards, with a re-entrant right angle cut out of one or both of them if symmetric matrices are to be multiplied. In getting the element of the i th row and j th column of the product, the i th row of the first factor and the j th column of the second should be marked by a card beside, above, or below it.

— HAROLD HOTELLING, *Some New Methods in Matrix Calculation* (1943)

It was found that multiplication of matrices using punched card storage could be a highly efficient process on the Pilot ACE, due to the relative speeds of the Hollerith card reader used for input (one number per 16 ms.) and the automatic multiplier (2 ms.). While a few rows of one matrix were held in the machine the matrix to be multiplied by it was passed through the card reader. The actual computing and selection of numbers from store occupied most of the time between the passage of successive rows of the cards through the reader, so that the overall time was but little longer than it would have been if the machine had been able to accommodate both matrices.

— MICHAEL WOODGER, *The History and Present Use of Digital Computers at the National Physical Laboratory* (1958)

23.1. Methods

A fast matrix multiplication method forms the product of two $n \times n$ matrices in $O(n^\omega)$ arithmetic operations, where $\omega < 3$. Such a method is more efficient asymptotically than direct use of the definition

$$(AB)_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad (23.1)$$

which requires $O(n^3)$ operations. For over a century after the development of matrix algebra in the 1850s by Cayley, Sylvester, and others, this definition provided the only known method for multiplying matrices. In 1967, however, to the surprise of many, Winograd found a way to exchange half the multiplications for additions in the basic formula [1249, 1968]. The method rests on the identity, for vectors of even dimension n ,

$$x^T y = \sum_{i=1}^{n/2} (x_{2i-1} + y_{2i})(x_{2i} + y_{2i-1}) - \sum_{i=1}^{n/2} x_{2i-1}x_{2i} - \sum_{i=1}^{n/2} y_{2i-1}y_{2i}. \quad (23.2)$$

When this identity is applied to a matrix product AB , with x a row of A and y a column of B , the second and third summations are found to be common to the other inner products involving that row or column, so they can be computed once and reused. Winograd's paper generated immediate practical interest because on the computers of the 1960s floating point multiplication was typically two or three times slower than floating point addition. (On today's machines these two operations are usually similar in cost.)

Shortly after Winograd's discovery, Strassen astounded the computer science community by finding an $O(n^{\log_2 7})$ operations method for matrix multiplication ($\log_2 7 \approx 2.807$). A variant of this technique can be used to compute A^{-1} (see Problem 23.8) and thereby to solve $Ax = b$, both in $O(n^{\log_2 7})$ operations. Hence the title of Strassen's 1969 paper [1093, 1969], which refers to the question of whether Gaussian elimination is asymptotically optimal for solving linear systems.

Strassen's method is based on a circuitous way to form the product of a pair of 2×2 matrices in 7 multiplications and 18 additions, instead of the usual 8 multiplications and 4 additions. As a means of multiplying 2×2 matrices the formulae have nothing to recommend them, but they are valid more generally for block 2×2 matrices. Let A and B be matrices of dimensions $m \times n$ and $n \times p$ respectively, where all the dimensions are even, and partition each of A , B , and $C = AB$ into four equally sized blocks:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \quad (23.3)$$

Strassen's formulae are

$$\begin{aligned}
 P_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\
 P_2 &= (A_{21} + A_{22})B_{11}, \\
 P_3 &= A_{11}(B_{12} - B_{22}), \\
 P_4 &= A_{22}(B_{21} - B_{11}), \\
 P_5 &= (A_{11} + A_{12})B_{22}, \\
 P_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\
 P_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\
 C_{11} &= P_1 + P_4 - P_5 + P_7, \\
 C_{12} &= P_3 + P_5, \\
 C_{21} &= P_2 + P_4, \\
 C_{22} &= P_1 + P_3 - P_2 + P_6.
 \end{aligned} \tag{23.4}$$

Counting the additions (A) and multiplications (M) we find that while conventional multiplication requires

$$mnpM + m(n-1)pA,$$

Strassen's algorithm, using conventional multiplication at the block level, requires

$$\frac{7}{8}mnpM + \left(\frac{7}{8}m(n-2)p + \frac{5}{4}mn + \frac{5}{4}np + \frac{8}{4}mp\right)A.$$

Thus, if m , n , and p are large, Strassen's algorithm reduces the arithmetic by a factor of about $7/8$. The same idea can be used recursively on the multiplications associated with the P_i . In practice, recursion is only performed down to the "crossover" level at which any savings in floating point operations are outweighed by the overheads of a computer implementation.

To state a complete operation count, we suppose that $m = n = p = 2^k$ and that recursion is terminated when the matrices are of dimension $n_0 = 2^r$, at which point conventional multiplication is used. The number of multiplications and additions can be shown to be

$$M(k) = 7^{k-r}8^r, \quad A(k) = 4^r(2^r + 5)7^{k-r} - 6 \cdot 4^k. \tag{23.5}$$

The sum $M(k) + A(k)$ is minimized over all integers r by $r = 3$; interestingly, this value is independent of k . The total operation count for the "optimal" $n_0 = 8$ is less than

$$4 \cdot 7^k = 4 \cdot 2^{\log_2 7^k} = 4(2^k)^{\log_2 7} = 4n^{\log_2 7} = 4n^{2.807\ldots}.$$

Hence, in addition to having a lower exponent, Strassen's method has a reasonable constant.

Winograd found a variant of Strassen's formulae that requires the same number of multiplications but only 15 additions (instead of 18). This variant therefore has slightly smaller constants in the operation count for $n \times n$ matrices. For the product

(23.3) the formulae are

$$\begin{aligned}
 S_1 &= A_{21} + A_{22}, & M_1 &= S_2 S_6, & T_1 &= M_1 + M_2, \\
 S_2 &= S_1 - A_{11}, & M_2 &= A_{11} B_{11}, & T_2 &= T_1 + M_4, \\
 S_3 &= A_{11} - A_{21}, & M_3 &= A_{12} B_{21}, \\
 S_4 &= A_{12} - S_2, & M_4 &= S_3 S_7, \\
 S_5 &= B_{12} - B_{11}, & M_5 &= S_1 S_5, & C_{11} &= M_2 + M_3, \\
 S_6 &= B_{22} - S_5, & M_6 &= S_4 B_{22}, & C_{12} &= T_1 + M_5 + M_6, \\
 S_7 &= B_{22} - B_{12}, & M_7 &= A_{22} S_8, & C_{21} &= T_2 - M_7, \\
 S_8 &= S_6 - B_{21}, & & & C_{22} &= T_2 + M_5.
 \end{aligned} \tag{23.6}$$

Until the late 1980s there was a widespread view that Strassen's method was of theoretical interest only, because of its supposed large overheads for dimensions of practical interest (see, for example, [1028, 1988] and [953, 1992]). However, in 1970 Brent implemented Strassen's algorithm in Algol-W on an IBM 360/67 and concluded that in this environment, and with just one level of recursion, the method runs faster than the conventional method for $n \geq 110$ [158, 1970]. In 1988, Bailey compared his Fortran implementation of Strassen's algorithm for the Cray-2 with the Cray library routine for matrix multiplication and observed speedup factors ranging from 1.45 for $n = 128$ to 2.01 for $n = 2048$ (although 35% of these speedups were due to Cray-specific techniques) [52, 1988]. These empirical results, together with more recent experience of various researchers, show that Strassen's algorithm *is* of practical interest, even for n in the hundreds. Indeed, Fortran codes for (Winograd's variant of) Strassen's method have been supplied with IBM's ESSL library [653, 1988] and Cray's UNICOS library [662, 1989] since the late 1980s.

Strassen's paper raised the question, "What is the minimum exponent ω such that multiplication of $n \times n$ matrices can be done in $O(n^\omega)$ operations?" Clearly, $\omega \geq 2$, since each element of each matrix must partake in at least one operation. It was 10 years before the exponent was reduced below Strassen's $\log_2 7$. A flurry of publications, beginning in 1978 with Pan and his exponent 2.795 [915, 1978], resulted in reduction of the exponent to the current record 2.376, obtained by Coppersmith and Winograd in 1987 [272, 1987]. Figure 23.1 plots exponent versus time of publication (not all publications are represented in the graph); in principle, the graph should extend back to 1850!

Some of the fast multiplication methods are based on a generalization of Strassen's idea to bilinear forms. Let $A, B \in \mathbb{R}^{h \times h}$. A bilinear noncommutative algorithm over $\mathbb{R}$ for multiplying A and B with t "nonscalar multiplications" forms the product $C = AB$ according to

$$c_{ij} = \sum_{k=1}^t w_{h(i-1)+j,k} P_k, \tag{23.7a}$$

$$P_k = \left(\sum_{i,j=1}^h u_{ij}^{(k)} a_{ij} \right) \left(\sum_{i,j=1}^h v_{ij}^{(k)} b_{ij} \right), \quad k = 1:t, \tag{23.7b}$$

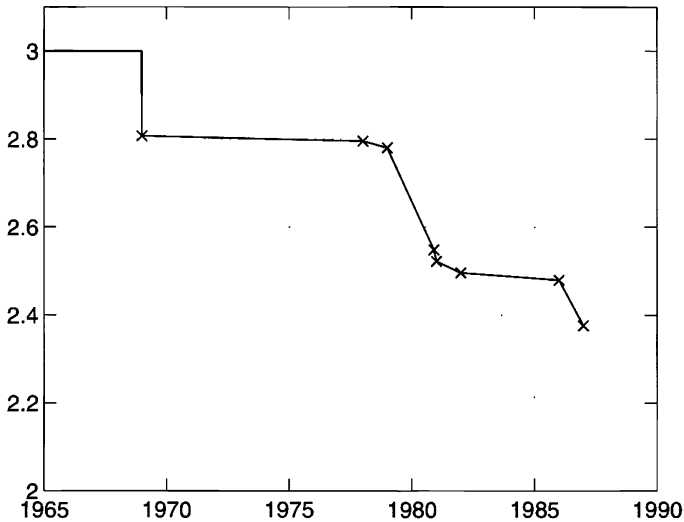


Figure 23.1. *Exponent versus time for matrix multiplication.*

where the elements of the matrices W , $U^{(k)}$, and $V^{(k)}$ are constants. This algorithm can be used to multiply $n \times n$ matrices A and B , where $n = h^k$, as follows: partition A , B , and C into h^2 blocks A_{ij} , B_{ij} , and C_{ij} of size h^{k-1} , then compute $C = AB$ by the bilinear algorithm, with the scalars a_{ij} , b_{ij} , and c_{ij} replaced by the corresponding matrix blocks. (The algorithm is applicable to matrices since, by assumption, the underlying formulae do not depend on commutativity.) To form the t products P_k of $(n/h) \times (n/h)$ matrices, partition them into h^2 blocks of dimension n/h^2 and apply the algorithm recursively. The total number of scalar multiplications required for the multiplication is $t^k = n^\alpha$, where $\alpha = \log_h t$.

Strassen's method has $h = 2$ and $t = 7$. For 3×3 multiplication ($h = 3$), the smallest t obtained so far is 23 [760, 1976]; since $\log_3 23 \approx 2.854 > \log_2 7$, this does not yield any improvement over Strassen's method. The method described in Pan's paper [915, 1978] has $h = 70$ and $t = 143,640$, which yields $\alpha = \log_{70} 143,640 = 2.795 \dots$

In the methods that achieve exponents lower than 2.775, various intricate techniques are used. Laderman, Pan, and Sha [761, 1992] explain that for these methods "very large overhead constants are hidden in the 'O' notation", and that the methods "improve on Strassen's (and even the classical) algorithm only for immense numbers N ".

A further method that is appropriate to discuss in the context of fast multiplication methods, even though it does not reduce the exponent, is a method for efficient multiplication of complex matrices. The clever formula

$$(a + ib)(c + id) = ac - bd + i[(a + b)(c + d) - ac - bd] \quad (23.8)$$

computes the product of two complex numbers using three real multiplications instead of the usual four. Since the formula does not rely on commutativity it extends to matrices. Let $A = A_1 + iA_2$ and $B = B_1 + iB_2$, where $A_j, B_j \in \mathbb{R}^{n \times n}$,

and define $C = C_1 + iC_2 = AB$. Then C can be formed using three real matrix multiplications as

$$\begin{aligned} T_1 &= A_1 B_1, & T_2 &= A_2 B_2, \\ C_1 &= T_1 - T_2, \\ C_2 &= (A_1 + A_2)(B_1 + B_2) - T_1 - T_2, \end{aligned} \tag{23.9}$$

which we will refer to as the “3M method”. This computation involves $3n^3$ scalar multiplications and $3n^3 + 2n^2$ scalar additions. Straightforward evaluation of the conventional formula $C = A_1 B_1 - A_2 B_2 + i(A_1 B_2 + A_2 B_1)$ requires $4n^3$ multiplications and $4n^3 - 2n^2$ additions. Thus, the 3M method requires strictly fewer arithmetic operations than the conventional means of multiplying complex matrices for $n \geq 3$, and it achieves a saving of about 25% for $n \geq 30$ (say). Similar savings occur in the important special case where A or B is triangular. This kind of clear-cut computational saving is rare in matrix computations!

IBM’s ESSL library and Cray’s UNICOS library both contain routines for complex matrix multiplication that apply the 3M method and use Strassen’s method to evaluate the resulting three real matrix products.

23.2. Error Analysis

To be of practical use, a fast matrix multiplication method needs to be faster than conventional multiplication for reasonable dimensions without sacrificing numerical stability. The stability properties of a fast matrix multiplication method are much harder to predict than its practical efficiency and need careful investigation.

The forward error bound (3.13) for conventional computation of $C = AB$, where $A, B \in \mathbb{R}^{n \times n}$, can be written

$$|C - \widehat{C}| \leq nu|A||B| + O(u^2). \tag{23.10}$$

Miller [850, 1975] shows that any polynomial algorithm for multiplying $n \times n$ matrices that satisfies a bound of the form (23.10) (perhaps with a different constant) must perform at least n^3 multiplications. (A polynomial algorithm is essentially one that uses only scalar addition, subtraction, and multiplication.) Hence Strassen’s method, and all other polynomial algorithms with an exponent less than 3, cannot satisfy (23.10). Miller also states, without proof, that any polynomial algorithm in which the multiplications are all of the form $(\sum_{i,j} u_{ij}^{(k)} a_{ij})(\sum_{i,j} v_{ij}^{(k)} b_{ij})$ must satisfy a bound of the form

$$|C - \widehat{C}| \leq f_n u \|A\| \|B\| + O(u^2). \tag{23.11}$$

It follows that any algorithm based on recursive application of a bilinear noncommutative algorithm satisfies (23.11); however, the all-important constant f_n is not specified. These general results are useful because they show us what types of results we can and cannot prove and thereby help to focus our efforts.

In the subsections below we analyse particular methods.

Throughout the rest of this chapter an unsubscripted matrix norm denotes

$$\|A\| := \max_{i,j} |a_{ij}|.$$

As noted in §6.2, this is not a consistent matrix norm, but we do have the bound $\|AB\| \leq n\|A\| \|B\|$ for $n \times n$ matrices.

23.2.1. Winograd's Method

Winograd's method does not satisfy the conditions required for the bound (23.11), since it involves multiplications with operands of the form $a_{ij} + b_{rs}$. However, it is straightforward to derive an error bound.

Theorem 23.1 (Brent). *Let $x, y \in \mathbb{R}^{n \times n}$, where n is even. The inner product computed by Winograd's method satisfies*

$$|x^T y - fl(x^T y)| \leq n\gamma_{n/2+4}(\|x\|_\infty + \|y\|_\infty)^2. \quad (23.12)$$

Proof. A straightforward adaptation of the inner product error analysis in §3.1 produces the following analogue of (3.3):

$$\begin{aligned} fl(x^T y) &= \sum_{i=1}^{n/2} (x_{2i-1} + y_{2i})(x_{2i} + y_{2i-1})(1 + \epsilon_i) \\ &\quad - \sum_{i=1}^{n/2} x_{2i-1}x_{2i}(1 + \alpha_i) - \sum_{i=1}^{n/2} y_{2i-1}y_{2i}(1 + \beta_i), \end{aligned}$$

where the ϵ_i , α_i , and β_i are all bounded in modulus by $\gamma_{n/2+4}$. Hence

$$\begin{aligned} |x^T y - fl(x^T y)| &\leq \gamma_{n/2+4} \sum_{i=1}^{n/2} (|x_{2i-1} + y_{2i}| |x_{2i} + y_{2i-1}| \\ &\quad + |x_{2i-1}x_{2i}| + |y_{2i-1}y_{2i}|) \\ &\leq \gamma_{n/2+4} (n/2) ((\|x\|_\infty + \|y\|_\infty)^2 + \|x\|_\infty^2 + \|y\|_\infty^2) \\ &\leq n\gamma_{n/2+4} (\|x\|_\infty + \|y\|_\infty)^2. \quad \square \end{aligned}$$

The analogue of (23.12) for matrix multiplication is $\|AB - fl(AB)\| \leq n\gamma_{n/2+4} \times (\|A\| + \|B\|)^2$.

Conventional evaluation of $x^T y$ yields the bound (see (3.5))

$$|x^T y - fl(x^T y)| \leq \gamma_n |x|^T |y| \leq n\gamma_n \|x\|_\infty \|y\|_\infty. \quad (23.13)$$

The bound (23.12) for Winograd's method exceeds the bound (23.13) by a factor approximately $\|x\|_\infty / \|y\|_\infty + \|y\|_\infty / \|x\|_\infty$. Therefore Winograd's method is stable if $\|x\|_\infty$ and $\|y\|_\infty$ have similar magnitude, but potentially unstable if they differ widely in magnitude. The underlying reason for the instability is that Winograd's method relies on cancellation of terms $x_{2i-1}x_{2i}$ and $y_{2i-1}y_{2i}$ that can be much larger ($\|x\|_\infty^2$ and $\|y\|_\infty^2$) than the final answer ($|x^T y| \leq \|x\|_\infty \|y\|_1$); therefore the intermediate rounding errors can swamp the desired inner product.

A simple way to avoid the instability is to scale $x \leftarrow \mu x$ and $y \leftarrow \mu^{-1} y$ before applying Winograd's method, where μ , which in practice might be a power of the

machine base to avoid roundoff, is chosen so that $\|x\|_\infty \approx \|y\|_\infty$. When using Winograd's method for a matrix multiplication AB it suffices to carry out a single scaling $A \leftarrow \mu A$ and $B \leftarrow \mu^{-1}B$ such that $\|A\| \approx \|B\|$. If A and B are scaled so that $\tau^{-1} \leq \|A\|/\|B\| \leq \tau$ then

$$\|AB - fl(AB)\| \leq 2(\tau + 1)n\gamma_{n/2+4}\|A\| \|B\|.$$

23.2.2. Strassen's Method

Until recently there was a widespread belief that Strassen's method is numerically unstable. The next theorem, originally proved by Brent [158, 1970], shows that this belief is unfounded.

Theorem 23.2 (Brent). *Let $A, B \in \mathbb{R}^{n \times n}$, where $n = 2^k$. Suppose that $C = AB$ is computed by Strassen's method and that $n_0 = 2^r$ is the threshold at which conventional multiplication is used. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \left[\left(\frac{n}{n_0} \right)^{\log_2 12} (n_0^2 + 5n_0) - 5n \right] u \|A\| \|B\| + O(u^2). \quad (23.14)$$

Proof. We will use without comment the norm inequality $\|AB\| \leq n\|A\| \|B\| = 2^k \|A\| \|B\|$.

Assume that the computed product $\hat{C} \approx AB$ from Strassen's method satisfies

$$\hat{C} = AB + E, \quad \|E\| \leq c_k u \|A\| \|B\| + O(u^2), \quad (23.15)$$

where c_k is a constant. In view of (23.10), (23.15) certainly holds for $n = n_0$, with $c_r = n_0^2$. Our aim is to verify (23.15) inductively and at the same time to derive a recurrence for the unknown constant c_k .

Consider C_{11} in (23.4), and, in particular, its subterm P_1 . Accounting for the errors in matrix addition and invoking (23.15), we obtain

$$\hat{P}_1 = (A_{11} + A_{22} + \Delta_A)(B_{11} + B_{22} + \Delta_B) + E_1,$$

where

$$\begin{aligned} |\Delta_A| &\leq u|A_{11} + A_{22}|, \\ |\Delta_B| &\leq u|B_{11} + B_{22}|, \\ \|E_1\| &\leq c_{k-1}u\|A_{11} + A_{22} + \Delta_A\| \|B_{11} + B_{22} + \Delta_B\| + O(u^2) \\ &\leq 4c_{k-1}u\|A\| \|B\| + O(u^2). \end{aligned}$$

Hence

$$\begin{aligned} \hat{P}_1 &= P_1 + F_1, \\ \|F_1\| &\leq (8 \cdot 2^{k-1} + 4c_{k-1})u\|A\| \|B\| + O(u^2). \end{aligned}$$

Similarly,

$$\hat{P}_4 = A_{22}(B_{21} - B_{11} + \Delta_B) + E_4,$$

where

$$\begin{aligned} |\Delta_B| &\leq u|B_{21} - B_{11}|, \\ \|E_4\| &\leq c_{k-1}u\|A_{22}\|\|B_{21} - B_{11} + \Delta_B\| + O(u^2), \end{aligned}$$

which gives

$$\begin{aligned} \hat{P}_4 &= P_4 + F_4, \\ \|F_4\| &\leq (2 \cdot 2^{k-1} + 2c_{k-1})u\|A\|\|B\| + O(u^2). \end{aligned}$$

Now

$$\hat{C}_{11} = fl(\hat{P}_1 + \hat{P}_4 - \hat{P}_5 + \hat{P}_7),$$

where $\hat{P}_5 =: P_5 + F_5$ and $\hat{P}_7 =: P_7 + F_7$ satisfy exactly the same error bounds as $\hat{P}_4$ and $\hat{P}_1$, respectively. Assuming that these four matrices are added in the order indicated, we have

$$\begin{aligned} \hat{C}_{11} &= C_{11} + \Delta C_{11}, \\ \|\Delta C_{11}\| &\leq u(3\|\hat{P}_1\| + 3\|\hat{P}_4\| + 2\|\hat{P}_5\| + \|\hat{P}_7\|) + \|F_1 + F_4 - F_5 + F_7\| + O(u^2) \\ &\leq 26 \cdot 2^{k-1}u\|A\|\|B\| + 4(5 \cdot 2^{k-1} + 3c_{k-1})u\|A\|\|B\| + O(u^2) \\ &= (46 \cdot 2^{k-1} + 12c_{k-1})u\|A\|\|B\| + O(u^2). \end{aligned}$$

Clearly, the same bound holds for the other three $\|\Delta C_{ij}\|$ terms. Thus, overall,

$$\hat{C} = AB + \Delta C, \quad \|\Delta C\| \leq (46 \cdot 2^{k-1} + 12c_{k-1})u\|A\|\|B\| + O(u^2).$$

A comparison with (23.15) shows that we need to define the c_k by

$$c_k = 12c_{k-1} + 46 \cdot 2^{k-1}, \quad k > r, \quad c_r = 4^r, \quad (23.16)$$

where $c_r = n_0^2$. Solving this recurrence we obtain

$$\begin{aligned} c_k &= 12^{k-r}4^r + 46 \cdot 2^{k-1} \left(\frac{6^{k-r} - 1}{5} \right) \\ &= \left(\frac{n}{n_0} \right)^{\log_2 12} n_0^2 + \frac{46}{5} \frac{n}{2} \left(\left(\frac{n}{n_0} \right)^{\log_2 6} - 1 \right) \\ &\leq \left(\frac{n}{n_0} \right)^{\log_2 12} (n_0^2 + 5n_0) - 5n, \end{aligned}$$

which gives (23.14). $\square$

The forward error bound for Strassen's method is not of the componentwise form (23.10) that holds for conventional multiplication, which we know it cannot be by Miller's result. One unfortunate consequence is that while the scaling $AB \rightarrow (AD)(D^{-1}B)$, where D is diagonal, leaves (23.10) unchanged, it can alter (23.14) by an arbitrary amount.

The reason for the scale dependence is that Strassen's method adds together elements of A matrix-wide (and similarly for B); for example, in (23.4) A_{11} is added to A_{22} , A_{12} , and A_{21} . This intermingling of elements is particularly undesirable when A or B has elements of widely differing magnitudes because it allows large

errors to contaminate small components of the product. This phenomenon is well illustrated by the example

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & \epsilon \\ \epsilon & \epsilon^2 \end{bmatrix},$$

which is evaluated exactly in floating point arithmetic if we use conventional multiplication. However, Strassen's method computes

$$c_{22} = 2(1 + \epsilon^2) + (\epsilon - \epsilon^2) - 1 - (1 + \epsilon).$$

Because c_{22} involves subterms of order unity, the error $c_{22} - \hat{c}_{22}$ will be of order u . Thus the relative error $|c_{22} - \hat{c}_{22}|/|c_{22}| = O(u/\epsilon^2)$, which is much larger than u if ϵ is small. This is an example where Strassen's method does not satisfy the bound (23.10). For another example, consider the product $X = P_{32}E$, where P_n is the $n \times n$ Pascal matrix (see §28.4) and $e_{ij} \equiv 1/3$. With just one level of recursion in Strassen's method we find in MATLAB that $\max_{ij} |x_{ij} - \hat{x}_{ij}|/|x_{ij}|$ is of order 10^{-5} , so that, again, some elements of the computed product have high relative error.

It is instructive to compare the bound (23.14) for Strassen's method with the weakened, normwise version of (23.10) for conventional multiplication:

$$\|C - \hat{C}\| \leq n^2 u \|A\| \|B\| + O(u^2). \quad (23.17)$$

The bounds (23.14) and (23.17) differ only in the constant term. For Strassen's method, the greater the depth of recursion the bigger the constant in (23.14): if we use just one level of recursion ($n_0 = n/2$) then the constant is $3n^2 + 25n$, whereas with full recursion ($n_0 = 1$) the constant is $6n^{\log_2 12} - 5n \approx 6n^{3.585} - 5n$. It is also interesting to note that the bound for Strassen's method (minimal for $n_0 = n$) is not correlated with the operation count (minimal for $n_0 = 8$).

Our conclusion is that Strassen's method has less favourable stability properties than conventional multiplication in two respects: it satisfies a weaker error bound (normwise rather than componentwise) and it has a larger constant in the bound (how much larger depending on n_0).

Another interesting property of Strassen's method is that it always involves some genuine subtractions (assuming that all additions are of nonzero terms). This is easily deduced from the formulae (23.4). This makes Strassen's method unattractive in applications where all the elements of A and B are nonnegative (for example, in Markov processes). Here, conventional multiplication yields low componentwise relative error because, in (23.10), $|A||B| = |AB| = |C|$, yet comparable accuracy cannot be guaranteed for Strassen's method.

An analogue of Theorem 23.2 holds for Winograd's variant of Strassen's method.

Theorem 23.3. *Let $A, B \in \mathbb{R}^{n \times n}$, where $n = 2^k$. Suppose that $C = AB$ is computed by Winograd's variant (23.6) of Strassen's method and that $n_0 = 2^r$ is the threshold at which conventional multiplication is used. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \left[\left(\frac{n}{n_0} \right)^{\log_2 18} (n_0^2 + 6n_0) - 6n \right] u \|A\| \|B\| + O(u^2). \quad (23.18)$$

Proof. The proof is analogous to that of Theorem 23.2, but more tedious. It suffices to analyse the computation of C_{12} , and the recurrence corresponding to (23.16) is

$$c_k = 18c_{k-1} + 89 \cdot 2^{k-1}, \quad k > r, \quad c_r = 4^r. \quad \square$$

Note that the bound for the Winograd–Strassen method has exponent $\log_2 18 \approx 4.170$ in place of $\log_2 12 \approx 3.585$ for Strassen’s method, suggesting that the price to be paid for a reduction in the number of additions is an increased rate of error growth. All the comments above about Strassen’s method apply to the Winograd variant.

Two further questions are suggested by the error analysis:

- How do the actual errors compare with the bounds?
- Which formulae are the more accurate in practice, Strassen’s or Winograd’s variant?

To give some insight we quote results obtained with a single precision Fortran 90 implementation ($u \approx 6 \times 10^{-8}$) of the two methods (the code is easy to write if we exploit the language’s dynamic arrays and recursive procedures). We take random $n \times n$ matrices A and B and look at $\|AB - fl(AB)\|/(\|A\| \|B\|)$ for $n_0 = 1, 2, \dots, 2^k = n$ (note that this is not the relative error, since the denominator is $\|A\| \|B\|$ instead of $\|AB\|$, and note that $n_0 = n$ corresponds to conventional multiplication). Figure 23.2 plots the results for one random matrix of order 1024 from the uniform $[0, 1]$ distribution and another matrix of the same size from the uniform $[-1, 1]$ distribution. The error bound (23.14) for Strassen’s method is also plotted. Two observations can be made.

- Winograd’s variant can be more accurate than Strassen’s formulae, for all n_0 , despite its larger error bound.
- The error bound overestimates the actual error by a factor up to 1.8×10^6 for $n = 1024$, but the variation of the errors with n_0 is roughly as predicted by the bound.

23.2.3. Bilinear Noncommutative Algorithms

Bini and Lotti [110, 1980] have analysed the stability of bilinear noncommutative algorithms in general. They prove the following result.

Theorem 23.4 (Bini and Lotti). *Let $A, B \in \mathbb{R}^{n \times n}$ ($n = h^k$) and let the product $C = AB$ be formed by a recursive application of the bilinear noncommutative algorithm (23.7), which multiplies $h \times h$ matrices using t nonscalar multiplications. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \alpha u n^{\log_h \beta} \log_h n \|A\| \|B\| + O(u^2), \quad (23.19)$$

where α and β are constants that depend on the number of nonzero terms in the matrices U, V , and W that define the algorithm. $\square$

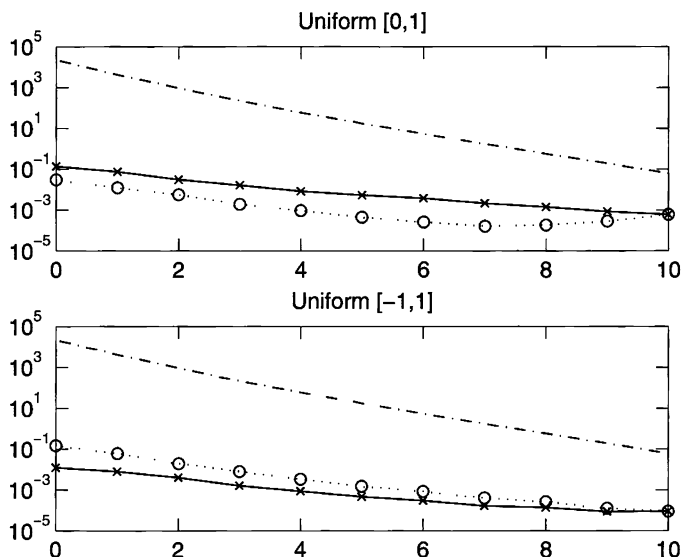


Figure 23.2. Errors for Strassen's method with two random matrices of dimension $n = 1024$. Strassen's formulae: "x", Winograd's variant: "o". X-axis contains $\log_2$ of recursion threshold n_0 , $1 \leq n_0 \leq n$. Dot-dash line is error bound for Strassen's formulae.

The precise definition of α and β is given in [110, 1980]. If we take $k = 1$, so that $h = n$, and if the basic algorithm (23.7) is chosen to be conventional multiplication, then it turns out that $\alpha = n - 1$ and $\beta = n$, so the bound of the theorem becomes $(n - 1)nu\|A\| \|B\| + O(u^2)$, which is essentially the same as (23.17). For Strassen's method, $h = 2$ and $t = 7$, and $\alpha = 5$, $\beta = 12$, so the theorem produces the bound $5un^{\log_2 12} \log_2 n \|A\| \|B\| + O(u^2)$, which is a factor $\log_2 n$ larger than (23.14) (with $n_0 = 1$). This extra weakness of the bound is not surprising given the generality of Theorem 23.4.

Bini and Lotti consider the set of all bilinear noncommutative algorithms that form 2×2 products in 7 multiplications and that employ integer constants of the form $\pm 2^i$, where i is an integer (this set breaks into 26 equivalence classes). They show that Strassen's method has the minimum exponent β in its error bound in this class (namely, $\beta = 12$). In particular, Winograd's variant of Strassen's method has $\beta = 18$, so Bini and Lotti's bound has the same exponent $\log_2 18$ as in Theorem 23.3.

23.2.4. The 3M Method

A simple example reveals a fundamental weakness of the 3M method. Consider the computation of the scalar

$$z = x + iy = (\theta + i/\theta)^2 = \theta^2 - 1/\theta^2 + 2i.$$

In floating point arithmetic, if y is computed in the usual way, as $y = \theta(1/\theta) + (1/\theta)\theta$, then no cancellation occurs and the computed $\hat{y}$ has high relative accuracy:

$|\hat{y} - y|/|y| = O(u)$. The 3M method computes

$$y = \left(\theta + \frac{1}{\theta}\right)\left(\theta + \frac{1}{\theta}\right) - \theta^2 - \frac{1}{\theta^2}.$$

If $|\theta|$ is large this formula expresses a number of order 1 as the difference of large numbers. The computed $\hat{y}$ will almost certainly be contaminated by rounding errors of order $u\theta^2$, in which case the relative error is large: $|\hat{y} - y|/|y| = O(u\theta^2)$. However, if we measure the error in $\hat{y}$ relative to z , then it is acceptably small: $|\hat{y} - y|/|z| = O(u)$. This example suggests that the 3M method may be stable, but in a weaker sense than for conventional multiplication.

To analyse the general case, consider the product $C_1 + iC_2 = (A_1 + iA_2)(B_1 + iB_2)$, where $A_k, B_k, C_k \in \mathbb{R}^{n \times n}$, $k = 1:2$. Using (23.10) we find that the computed product from conventional multiplication,

$$\hat{C} = fl(A_1B_1 - A_2B_2 + i(A_1B_2 + A_2B_1)),$$

satisfies

$$|C_1 - \hat{C}_1| \leq (n+1)u(|A_1||B_1| + |A_2||B_2|) + O(u^2), \quad (23.20)$$

$$|C_2 - \hat{C}_2| \leq (n+1)u(|A_1||B_2| + |A_2||B_1|) + O(u^2). \quad (23.21)$$

For the 3M method C_1 is computed in the conventional way, and so (23.20) holds. It is straightforward to show that $\hat{C}_2$ satisfies

$$|C_2 - \hat{C}_2| \leq (n+4)u[(|A_1| + |A_2|)(|B_1| + |B_2|) + |A_1||B_1| + |A_2||B_2|] + O(u^2). \quad (23.22)$$

Two notable features of the bound (23.22) are as follows. First, it is of a different and weaker form than (23.21); in fact, it exceeds the sum of the bounds (23.20) and (23.21). Second and more pleasing, it retains the property of (23.20) and (23.21) of being invariant under diagonal scalings

$$C = AB \rightarrow D_1AD_2 \cdot D_2^{-1}BD_3 = D_1CD_3, \quad D_j \text{ diagonal},$$

in the sense that the upper bound ΔC_2 in (23.22) scales also according to $D_1\Delta C_2D_3$. (The “hidden” second-order terms in (23.20)–(23.22) are invariant under these diagonal scalings.)

The disparity between (23.21) and (23.22) is, in part, a consequence of the differing numerical cancellation properties of the two methods. It is easy to show that there are always subtractions of like-signed numbers in the 3M method, whereas if A_1, A_2, B_1 , and B_2 have nonnegative elements (for example) then no numerical cancellation takes place in conventional multiplication.

We can define a measure of stability with respect to which the 3M method matches conventional multiplication by taking norms in (23.21) and (23.22). We obtain the weaker bounds

$$\|C_2 - \hat{C}_2\|_\infty \leq 2(n+1)u\|A\|_\infty\|B\|_\infty + O(u^2), \quad (23.23)$$

$$\|C_2 - \hat{C}_2\|_\infty \leq 4(n+4)u\|A\|_\infty\|B\|_\infty + O(u^2) \quad (23.24)$$

(having used $\| |A_1| + |A_2| \|_\infty \leq \sqrt{2} \|A_1 + iA_2\|_\infty$). Combining these with an analogous weakening of (23.20), we find that for both conventional multiplication and the 3M method the computed complex matrix $\hat{C}$ satisfies

$$\|C - \hat{C}\|_\infty \leq c_n u \|A\|_\infty \|B\|_\infty + O(u^2),$$

where $c_n = O(n)$.

The conclusion is that the 3M method produces a computed product $\hat{C}$ whose imaginary part may be contaminated by relative errors much larger than those for conventional multiplication (or, equivalently, much larger than can be accounted for by small componentwise perturbations in the data A and B). However, if the errors are measured relative to $\|A\| \|B\|$, which is a natural quantity to use for comparison when employing matrix norms, then they are just as small as for conventional multiplication.

It is straightforward to show that if the 3M method is implemented using Strassen's method to form the real matrix products, then the computed complex product $\hat{C}$ satisfies the same bound (23.14) as for Strassen's method itself, but with an extra constant multiplier of 6 and with 4 added to the expression inside the square brackets.

23.3. Notes and References

A good introduction to the construction of fast matrix multiplication methods is provided by the papers of Pan [916, 1984] and Laderman, Pan, and Sha [761, 1992].

Harter [549, 1972] shows that Winograd's formula (23.2) is the best of its kind, in a certain precise sense.

How does one derive formulae such as those of Winograd and Strassen, or that in the 3M method? Inspiration and ingenuity seem to be the key. A fairly straightforward, but still not obvious, derivation of Strassen's method is given by Yuval [1276, 1978]. Gustafson and Aluru [534, 1996] develop algorithms that systematically search for fast algorithms, taking advantage of a parallel computer. In an exhaustive search taking 21 hours of computation time on a 256 processor nCUBE 2, they were able to find 12 methods for multiplying 2 complex numbers in 3 multiplications and 5 additions; they could not find a method with fewer additions, thus proving that such a method does not exist. However, they estimate that a search for Strassen's method on a 1024 processor nCUBE 2 would take many centuries, even using aggressive pruning rules, so human ingenuity is not yet redundant!

To obtain a useful implementation of Strassen's method a number of issues have to be addressed, including how to program the recursion, how best to handle rectangular matrices of arbitrary dimension (since the basic method is defined only for square matrices of dimension a power of 2), and how to cater for the extra storage required by the method. These issues are discussed by Bailey [52, 1988], Bailey, Lee, and Simon [56, 1991], Fischer [414, 1974], Higham [592, 1990], Kreczmar [750, 1976], and Spieß [1058, 1976], among others. Douglas, Heroux, Shishman, and Smith [351, 1994] present a portable Fortran implementation of

Winograd's variant of Strassen's method for real and complex matrices, with a level-3 BLAS interface; they take care to use a minimal amount of extra storage (about $2n^3/3$ elements of extra storage is required when multiplying $n \times n$ matrices).

Higham [592, 1990] shows how Strassen's method can be used to produce algorithms for all the level-3 BLAS operations that are asymptotically faster than the conventional algorithms. Most of the standard algorithms in numerical linear algebra remain stable (in an appropriately weakened sense) when fast level-3 BLAS are used. See, for example, Chapter 13, §19.5, and Problems 12.4 and 14.2.

Knight [740, 1995] shows how to choose the recursion threshold to minimize the operation count of Strassen's method for rectangular matrices. He also shows how to use Strassen's method to compute the QR factorization of an $m \times n$ matrix in $O(mn^{1.838})$ operations instead of the usual $O(mn^2)$.

Bailey, Lee, and Simon [56, 1991] substituted their Strassen's method code for a conventionally coded BLAS3 subroutine **SGEMM** and tested LAPACK's LU factorization subroutine **SGETRF** on a Cray Y-MP. They obtained speed improvements for matrix dimensions 1024 and larger.

Kaporin [709, 1999] investigates a fast matrix multiplication formula of Pan from 1972 and shows that it can be implemented so as to be competitive with Strassen's method in terms of both practical efficiency and numerical accuracy. This method deserves further investigation.

The Fortran 95 standard includes an intrinsic function **MATMUL** that returns the product of its two matrix arguments. The standard does not specify which method is to be used for the multiplication. An IBM compiler supports the use of Winograd's variant of Strassen's method, via an optional third argument to **MATMUL** (an extension to Fortran 95) [352, 1994].

Not only multiplication but also division of complex numbers can be done with fewer real multiplications and divisions than by the obvious formulae. The division $(a+ib)/(c+id)$ can be done with six real multiplications and divisions (see (27.1)) and $1/(c+id)$ and $a/(c+id)$ can both be done in four real multiplications and divisions. These operation counts, and that for (23.8), are optimal, as shown by Alt and van Leeuwen [15, 1981] and Lickteig [787, 1987].

Brent was the first to point out the possible instability of Winograd's method [159, 1970]. He presented a full error analysis (including Theorem 23.1) and showed that scaling ensures stability.

An error analysis of Strassen's method was given by Brent in 1970 in an unpublished technical report that has not been widely cited [158, 1970]. Section 23.2.2 is based on Higham [592, 1990].

When AB is to be formed by Strassen's method and the elements of A and B vary widely in magnitude Dumitrescu [367, 1998] suggests computing $D_1(D_1^{-1}A \cdot BD_2^{-1})D_2$, where the diagonal matrices D_1 and D_2 are chosen to equilibrate the rows of A and the columns of B ; he shows that this scaling can improve the accuracy of the result.

According to Knuth, the 3M formula was suggested by P. Ungar in 1963 [744, 1998, p. 706]. It is analogous to a formula of Karatsuba and Ofman [711, 1963] for squaring a $2n$ -digit number using three squarings of n -digit numbers. That three multiplications (or divisions) are *necessary* for evaluating the product of two

complex numbers was proved by Winograd [1250, 1971].

Section 23.2.4 is based on Higham [599, 1992].

The answer to the question, “What method should we use to multiply complex matrices?” depends on the desired accuracy and speed. In a Fortran environment an important factor affecting the speed is the relative efficiency of real and complex arithmetic, which depends on the compiler and the computer (complex arithmetic is automatically converted by the compiler into real arithmetic). For a discussion and some statistics see [599, 1992].

The efficiency of Winograd’s method is very machine dependent. Bjørstad, Manne, Sørveik, and Vajteršić [134, 1992] found the method useful on the MasPar MP-1 parallel machine, on which floating point multiplication takes about three times as long as floating point addition at 64-bit precision. They also implemented Strassen’s method on the MP-1 (using Winograd’s method at the bottom level of recursion) and obtained significant speedups over conventional multiplication for dimensions as small as 256.

As noted in §23.1, Strassen [1093, 1969] gave not only a method for multiplying $n \times n$ matrices in $O(n^{\log_2 7})$ operations, but also a method for inverting an $n \times n$ matrix with the same asymptotic cost. The method is described in Problem 23.8. For more on Strassen’s inversion method see §26.3.2, Bailey and Ferguson [50, 1988], and Balle, Hansen, and Higham [60, 1993].

Problems

23.1. (Knight [740, 1995]) Suppose we have a method for multiplying $n \times n$ matrices in $O(n^\alpha)$ operations, where $2 < \alpha < 3$. Show that if A is $m \times n$ and B is $n \times p$ then the product AB can be formed in $O(n_1^{\alpha-2}n_2n_3)$ operations, where $n_1 = \min(m, n, p)$ and n_2 and n_3 are the other two dimensions.

23.2. Work out the operation count for Winograd’s method applied to $n \times n$ matrices.

23.3. Let $S_n(n_0)$ denote the operation count (additions plus multiplications) for Strassen’s method applied to $n \times n$ matrices, with recursion down to the level of $n_0 \times n_0$ matrices. Assume that n and n_0 are powers of 2. For large n , estimate $S_n(8)/S_n(n)$ and $S_n(1)/S_n(8)$ and explain the significance of these ratios (use (23.5)).

23.4. (Knight [740, 1995]) Suppose that Strassen’s method is used to multiply an $m \times n$ matrix by an $n \times p$ matrix, where $m = a2^j$, $n = b2^j$, $p = c2^j$, and that conventional multiplication is used once any dimension is 2^r or less. Show that the operation count is $\alpha 7^j + \beta 4^j$, where

$$\alpha = 2abc \left(\frac{8}{7}\right)^r + \frac{5}{3}(ab + bc + ac) \left(\frac{4}{7}\right)^r,$$

$$\beta = -\frac{1}{3}(5ab + 5bc + 8ac).$$

Show that by setting $r = 0$ and $a = 1$ a special case of the result of Problem 23.1 is obtained.

23.5. Compare and contrast Winograd's inner product formula for $n = 2$ with the imaginary part of the 3M formula (23.8).

23.6. Prove the error bound described at the end of §23.2.4 for the combination of the 3M method and Strassen's method.

23.7. Two fast ways to multiply complex matrices are (a) to apply the 3M method to the original matrices and to use Strassen's method to form the three real matrix products, and (b) to use Strassen's method with the 3M method applied at the bottom level of recursion. Investigate the merits of the two approaches with respect to speed and storage.

23.8. Strassen [1093, 1969] gives a method for inverting an $n \times n$ matrix in $O(n^{\log_2 7})$ operations. Assume that n is even and write

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad A_{ij} \in \mathbb{R}^{m \times m}, \quad n = 2m.$$

The inversion method is based on the following formulae:

$$\begin{aligned} P_1 &= A_{11}^{-1}, & P_2 &= A_{21}P_1, \\ P_3 &= P_1A_{12}, & P_4 &= A_{21}P_3, \\ P_5 &= P_4 - A_{22}, & P_6 &= P_5^{-1}, \\ A^{-1} &= \begin{bmatrix} P_1 - P_3P_6P_2 & P_3P_6 \\ P_6P_2 & -P_6 \end{bmatrix}. \end{aligned}$$

The matrix multiplications are done by Strassen's method and the inversions determining P_1 and P_6 are done by recursive invocations of the method itself. (a) Verify these formulae, using a block LU factorization of A , and show that they permit the claimed $O(n^{\log_2 7})$ complexity. (b) Show that if A is upper triangular, Strassen's method is equivalent to (the unstable) Method 2B of §14.2.2.

(For a numerical investigation into the stability of Strassen's inversion method, see §26.3.2.)

23.9. Find the inverse of the block upper triangular matrix

$$\begin{bmatrix} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{bmatrix}.$$

Deduce that matrix multiplication can be reduced to matrix inversion.

23.10. (RESEARCH PROBLEM) Carry out extensive numerical experiments to test the accuracy of Strassen's method and Winograd's variant (cf. the results at the end of §23.2.2).

Chapter 24

The Fast Fourier Transform and Applications

*Once the [FFT] method was established
it became clear that it had a long and interesting prehistory
going back as far as Gauss.
But until the advent of computing machines
it was a solution looking for a problem.*

— T. W. KÖRNER, *Fourier Analysis* (1988)

Life as we know it would be very different without the FFT.

— CHARLES F. VAN LOAN, *Computational
Frameworks for the Fast Fourier Transform* (1992)

24.1. The Fast Fourier Transform

The matrix–vector product $y = F_n x$, where

$$F_n = \left(\exp(-2\pi i(r-1)(s-1)/n) \right)_{r,s=1}^n,$$

is the key computation in the numerical evaluation of Fourier transforms. If the product is formed in the obvious way then $O(n^2)$ operations are required. The fast Fourier transform (FFT) is a way to compute y in just $O(n \log n)$ operations. This represents a dramatic reduction in complexity.

The FFT is best understood (at least by a numerical analyst!) by interpreting it as the application of a clever factorization of the discrete Fourier transform (DFT) matrix F_n .

Theorem 24.1 (Cooley–Tukey radix 2 factorization). *If $n = 2^t$ then the DFT matrix F_n may be factorized as*

$$F_n = A_t \dots A_1 P_n, \quad (24.1)$$

where P_n is a permutation matrix and

$$\begin{aligned} A_k &= I_{2^{t-k}} \otimes B_{2^k}, \\ B_{2^k} &= \begin{bmatrix} I_r & \Omega_r \\ I_r & -\Omega_r \end{bmatrix}, \quad r = 2^{k-1}, \\ \Omega_r &= \text{diag}(1, \omega_k, \dots, \omega_k^{r-1}), \quad \omega_k = \exp(-2\pi i/2^k). \end{aligned}$$

Proof. See Van Loan [1182, 1992, Thm. 1.3.3]. $\square$

The theorem shows that we can write $y = F_n x$ as

$$y = A_t \dots A_1 P_n x,$$

which is formed as a sequence of matrix–vector products. It is the sparsity of the A_k (two nonzeros per row) that yields the $O(n \log n)$ operation count.

We will not consider the implementation of the FFT, and therefore we do not need to define the “bit reversing” permutation matrix P_n in (24.1). However, the way in which the weights ω_k^j are computed does affect the accuracy. We will assume that computed weights $\hat{\omega}_k^j$ are used that satisfy, for all j and k ,

$$\hat{\omega}_k^j = \omega_k^j + \epsilon_{kj}, \quad |\epsilon_{kj}| \leq \mu. \quad (24.2)$$

Among the many methods for computing the weights are ones for which we can take $\mu = cu$, $\mu = cu \log j$, and $\mu = cuj$, where c is a constant that depends on the method; see Van Loan [1182, 1992, §1.4].

We are now ready to prove an error bound.

Theorem 24.2. *Let $\hat{y} = fl(F_n x)$, where $n = 2^t$, be computed using the Cooley–Tukey radix 2 FFT, and assume that (24.2) holds. Then*

$$\frac{\|y - \hat{y}\|_2}{\|y\|_2} \leq \frac{t\eta}{1 - t\eta}, \quad \eta := \mu + \gamma_4(\sqrt{2} + \mu).$$

Proof. Note first that $\|A_k\|_2 = \|B_{2^k}\|_2 = \sqrt{2}$ and

$$\| |A_k| \|_2 = \| |B_{2^k}| \|_2 = \left\| \begin{bmatrix} I & I \\ I & I \end{bmatrix} \right\|_2 = 2 = \sqrt{2} \|A_k\|_2. \quad (24.3)$$

Denote by $\hat{A}_k$ the matrix defined in terms of the computed $\hat{\omega}_k^j$. Then

$$\begin{aligned} \hat{y} &= fl(\hat{A}_t \dots \hat{A}_1 P_n x), \\ &= (\hat{A}_t + \Delta \hat{A}_t) \dots (\hat{A}_1 + \Delta \hat{A}_1) P_n x, \end{aligned}$$

where, using the fact that each A_k has only two nonzeros per row, and recalling that we are using complex arithmetic, $|\Delta \hat{A}_k| \leq \gamma_4 |\hat{A}_k|$, and hence

$$\|\Delta \hat{A}_k\|_2 \leq \| |\Delta \hat{A}_k| \|_2 \leq \gamma_4 \| |\hat{A}_k| \|_2.$$

In view of (24.2),

$$\hat{A}_k = A_k + \Delta A_k, \quad \|\Delta A_k\|_2 \leq \sqrt{2} \mu = \mu \|A_k\|_2. \quad (24.4)$$

Hence, using (24.3) and (24.4),

$$\| |\hat{A}_k| \|_2 \leq \| |A_k| \|_2 + \| |\Delta A_k| \|_2 \leq (\sqrt{2} + \mu) \|A_k\|_2.$$

Overall, then,

$$\begin{aligned} \hat{y} &= (A_t + E_t) \dots (A_1 + E_1) P_n x, \\ \|E_k\|_2 &\leq (\mu + \gamma_4(\sqrt{2} + \mu)) \|A_k\|_2 = \eta \|A_k\|_2. \end{aligned}$$

Invoking Lemma 3.6 we find that

$$\begin{aligned} \|y - \hat{y}\|_2 &\leq [(1 + \eta)^t - 1] \|A_t\|_2 \dots \|A_1\|_2 \|P_n\|_2 \|x\|_2 \\ &\leq \frac{t\eta}{1 - t\eta} 2^{t/2} \|x\|_2, \end{aligned} \quad (24.5)$$

using Lemma 3.1 for the second inequality. Finally, because F_n is $\sqrt{n}$ times a unitary matrix ($F_n^* F_n = nI$), $\|x\|_2 = n^{-1/2} \|y\|_2 = 2^{-t/2} \|y\|_2$. $\square$

Theorem 24.2 says that the Cooley–Tukey radix 2 FFT yields a tiny forward error, provided that the weights are computed stably. It follows immediately that the computation is backward stable, since $\hat{y} = y + \Delta y = F_n x + \Delta y$ implies $\hat{y} = F_n(x + \Delta x)$ with $\|\Delta x\|_2 / \|x\|_2 = \|\Delta y\|_2 / \|y\|_2$. If we form $y = F_n x$ by conventional multiplication using the exact F_n , then (Problem 3.7) $|y - \hat{y}| \leq \gamma_{n+2} |F_n| |x|$, so $\|y - \hat{y}\|_2 \leq n^{1/2} \gamma_{n+2} \|y\|_2$. Hence when μ is of order u , the FFT has an error bound

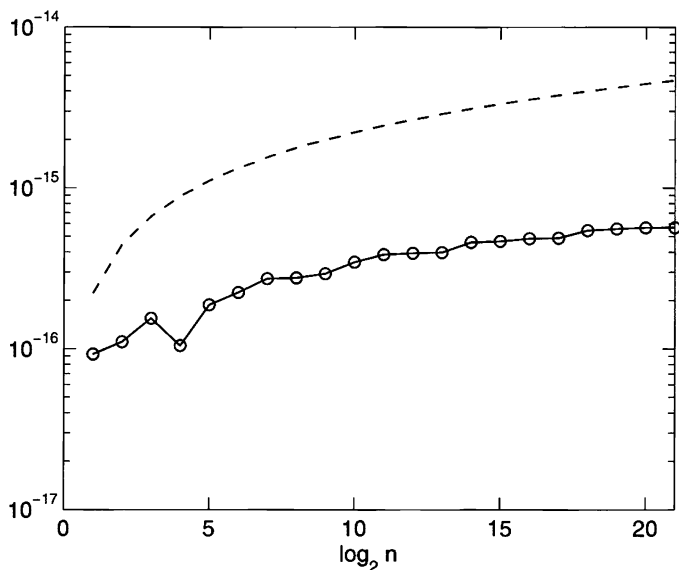


Figure 24.1. Error in FFT followed by inverse FFT ("o"). Dashed line is error bound.

smaller than that for conventional multiplication by a factor even greater than the reduction in complexity of the method. To sum up, the FFT is perfectly stable.

The inverse transform $x = F_n^{-1}y = n^{-1}F_n^*y$ can again be computed in $O(n \log n)$ operations using the Cooley–Tukey radix 2 factorization, and $\|x - \hat{x}\|_2/\|x\|_2$ satisfies the same bound as in Theorem 24.2. (Strictly, we should replace t by $t + 1$ in the bound to account for the rounding error in dividing by n .)

For other variations of the FFT, based on different radices or different factorizations of F_n , results analogous to Theorem 24.2 hold.

A simple way to test the error bounds is to compute the FFT followed by the inverse transform, $\hat{x} = fl(F_n^{-1} fl(F_n x))$, and to evaluate $e_n = \|x - \hat{x}\|_2/\|x\|_2$. Our analysis gives the bound $e_n \leq 2 \log_2 n \eta + O(\eta^2)$. Figure 24.1 plots e_n and the approximate error bound $2 \log_2 n \eta$ for $n = 2^k$, $k = 0:21$, with random x from the normal $N(0, 1)$ distribution (the FFTs were computed using MATLAB's `fft` and `ifft` functions). The errors grow at roughly the same rate as the bound and are on average about a factor of 10 smaller than the bound.

24.2. Circulant Linear Systems

A circulant matrix (or circulant, for short) is a special Toeplitz matrix in which the diagonals wrap around:

$$C = C(c) = \begin{bmatrix} c_1 & c_n & \dots & c_2 \\ c_2 & c_1 & \dots & \vdots \\ \vdots & \ddots & \ddots & c_n \\ c_n & \dots & c_2 & c_1 \end{bmatrix} \in \mathbb{C}^{n \times n}.$$

Circulant matrices have the important property that they are diagonalized by the DFT matrix F_n :

$$F_n C F_n^{-1} = D = \text{diag}(d_i).$$

Moreover, the eigenvalues are given by $d = F_n c$, where $c = [c_1, c_2, \dots, c_n]^T$. Hence a linear system $Cx = b$ can be solved in $O(n \log n)$ operations with the aid of the FFT as follows:

- (1) $d = F_n c$,
- (2) $g = F_n b$,
- (3) $h = D^{-1} g$,
- (4) $x = F_n^{-1} h$.

The computation involves two FFTs, a diagonal scaling, and an inverse FFT. We now analyse the effect of rounding errors. It is convenient to write the result of Theorem 24.2 in the equivalent form (from (24.5))

$$\hat{y} = (F_n + \Delta F_n)x, \quad \|\Delta F_n\|_2 \leq n^{1/2} \frac{t\eta}{1 - t\eta} =: f(n, u). \quad (24.6)$$

Steps (1) and (2) yield

$$\begin{aligned} \hat{d} &= (F_n + \Delta_1)c, & \|\Delta_1\|_2 &\leq f(n, u), \\ \hat{g} &= (F_n + \Delta_2)b, & \|\Delta_2\|_2 &\leq f(n, u). \end{aligned} \quad (24.7)$$

For steps (3) and (4) we have, using Lemma 3.5,

$$\begin{aligned} \hat{h} &= (I + E)\hat{D}^{-1}\hat{g}, & |E| &\leq \sqrt{2}\gamma_4 I, \\ \hat{x} &= (F_n^{-1} + \Delta_3)\hat{h}, & \|\Delta_3\|_2 &\leq n^{-1}f(n, u). \end{aligned}$$

Putting these equations together we have

$$\begin{aligned} \hat{x} &= (F_n^{-1} + \Delta_3)(I + E)\hat{D}^{-1}(F_n + \Delta_2)b \\ &= (I + \Delta_3 F_n)F_n^{-1} \cdot (I + E)\hat{D}^{-1} \cdot F_n(I + F_n^{-1}\Delta_2)b, \end{aligned}$$

or

$$F_n^{-1}\hat{D}(I + E)^{-1}F_n \cdot (I + \Delta_3 F_n)^{-1}\hat{x} = (I + F_n^{-1}\Delta_2)b. \quad (24.8)$$

Note that $F_n^{-1}\hat{D}(I + E)^{-1}F_n$ is a circulant, since $\hat{D}(I + E)^{-1}$ is diagonal. Hence (24.8) can be written as the circulant system

$$C(c + \Delta c)(\hat{x} + \Delta \hat{x}) = b + \Delta b,$$

where

$$\max \left\{ \frac{\|\Delta b\|_2}{\|b\|_2}, \frac{\|\Delta \hat{x}\|_2}{\|\hat{x}\|_2} \right\} \leq \frac{n^{-1/2}f(n, u)}{1 - n^{-1/2}f(n, u)}$$

and, working to first order,

$$\begin{aligned} c + \Delta c &= F_n^{-1}(I + E)^{-1}\hat{d} \\ &= F_n^{-1}(I + E)^{-1}(F_n + \Delta_1)c \\ &\approx c + F_n^{-1}(\Delta_1 - E F_n)c, \end{aligned}$$

so that

$$\|\Delta c\|_2 \leq n^{-1/2}(f(n, u) + \sqrt{2}\gamma_4\sqrt{n})\|c\|_2.$$

We summarize our findings as follows.

Theorem 24.3 (Yalamov). *Let $C \in \mathbb{C}^{n \times n}$ be a circulant and suppose the system $Cx = b$ is solved by the FFT process described above, where the FFT satisfies (24.6). Then the computed solution $\hat{x}$ satisfies $C(c + \Delta c)(\hat{x} + \Delta \hat{x}) = b + \Delta b$, where*

$$\max \left\{ \frac{\|\Delta c\|_2}{\|c\|_2}, \frac{\|\Delta b\|_2}{\|b\|_2}, \frac{\|\Delta \hat{x}\|_2}{\|\hat{x}\|_2} \right\} \leq \eta \log_2 n + 6u + O(u^2). \quad \square$$

The conclusion is that the computed solution from the FFT method for solving a circulant system is very close to the exact solution of a slightly perturbed circulant system. This is a structured mixed forward-backward error result. The computed solution $\hat{x}$ does not, in general, itself solve a nearby circulant system, as can be verified experimentally by computing the “circulant backward error” using techniques from [574, 1992]. The basic reason for the lack of this stronger form of stability is that there are too few parameters in the matrix onto which to throw the backward error.

A forward error bound can be obtained by applying standard perturbation theory to Theorem 24.3: the forward error is bounded by a multiple of $\kappa_2(C)u$. That the forward error can be as large as $\kappa_2(C)u$ is clear from the analysis above, because (24.7) shows that the computed eigenvalue $\min_i |\hat{d}_i|$ is contaminated by an error of order $u \max_i |\hat{d}_i|$.

24.3. Notes and References

For a unified treatment of the many different versions of the FFT, including implementation details, see Van Loan [1182, 1992].

For a comprehensive survey of the discrete Fourier transform see Briggs and Henson [165, 1995].

The Cooley–Tukey radix 2 FFT was presented in [267, 1965], which is one of the most cited mathematics papers of all time [611, 1998, p. 217].

The history of the FFT is discussed by Cooley [265, 1990], [266, 1994] and Cooley and Tukey [268, 1993]. Cooley [265, 1990] states that the earliest known reference to the FFT is an obscure 1866 paper of Gauss in neoclassic Latin, and he recommends that researchers not publish papers in neoclassic Latin!

Theorem 24.2 is not new, but the proof is more concise than most in the literature. In the first edition of this book the bound of Theorem 24.2 had an extra factor $\sqrt{n}$, owing to a less sharp analysis. Early references that derive error bounds using the matrix factorization formulation of the FFT are Gentleman and Sande [473, 1966] and Ramos [971, 1971]. A full list of references for error analysis of the FFT up to 1992 is given by Van Loan [1182, 1992, §1.4]. Schatzman [1015, 1996] and Tasche and Zeuner [1129, 2001] discuss the effect of errors in the weights on the computed FFT.

Linzer [793, 1992] showed that the FFT-based circulant solver is forward stable and posed the question of whether or not the solver is backward stable. Backward

error analysis given in the first edition of this book answered this question positively. Yalamov [1264, 2000] shows that the solver is actually structured mixed forward-backward stable, and Theorem 24.3 incorporates this stronger result.

One application of circulant linear systems is in the preconditioned conjugate gradient method for solving Toeplitz systems. The idea of using a circulant preconditioner was suggested by Strang [1091, 1986], and the theory and practice of this technique is now well developed. For more details see Chan and Ng [211, 1996] and the references therein. A good source of results about circulant matrices is the book by Davis [294, 1979].

Highly efficient implementations of the FFT are available in the FFTW software (the “fastest Fourier transform in the west”) [447, 1998]; see <http://www.fftw.org/>. This software provides automatically optimized C routines for the FFT in one or more dimensions.

Problems

24.1. (Bailey [53, 1993], Percival [935, 2002]) In high-precision multiplication we have two integer n -vectors x and y representing high-precision numbers and we wish to form the terms $z_k = \sum_{j=1}^k x_j y_{k+1-j}$, $k = 1:n$. By padding the vectors with n zeros, these products can be expressed in the form $z_k = \sum_{j=1}^{2n} x_j y_{k+1-j}$, where $k+1-j$ is interpreted as $k+1-j+n$ if $k+1-j$ is negative. These products represent a convolution: a matrix-vector product involving a circulant matrix. Analogously to the linear system solution in §24.2, this product can be evaluated in terms of discrete Fourier transforms as $z = F_n^{-1}(F_n x \cdot F_n y)$, where the dot denotes componentwise multiplication of two vectors. Since x and y are integer vectors, the z_i should also be integers, but in practice they will be contaminated by rounding errors. Obtain a bound on $z - \hat{z}$ and deduce a sufficient condition for the nearest integer vector to $\hat{z}$ to be the exact z .

Chapter 25

Nonlinear Systems and Newton's Method

*Not a single student showed up for Newton's second lecture,
and throughout almost every lecture for the next seventeen years . . .
Newton talked to an empty room.*

— MICHAEL WHITE, *Isaac Newton: The Last Sorcerer* (1997)

*[On Newton's method]
If we start with an approximation to a zero which is
appreciably more accurate than the
limiting accuracy which we have just described,
a single iteration will usually spoil this very good approximation
and produce one with an error which is typical of the limiting accuracy.*
— J. H. WILKINSON, *Rounding Errors in Algebraic Processes* (1963)

25.1. Newton's Method

Newton's method is a key tool in scientific computing for solving nonlinear equations and optimization problems. Our interest in this chapter is in Newton's method for solving algebraic systems of nonlinear equations.

Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be continuously differentiable and denote by J its Jacobian matrix $(\partial F_i / \partial x_j)$. Given a starting vector x_0 , Newton's method for finding a solution of $F(x) = 0$ generates a sequence $\{x_i\}$ defined by

$$J(x_i)(x_{i+1} - x_i) = -F(x_i), \quad i \geq 0. \quad (25.1)$$

The attraction of the method stems from the fact that, under appropriate conditions, it converges rapidly to a solution from any starting vector x_0 sufficiently close to that solution. In particular, if the Jacobian is nonsingular at the solution then the rate of convergence is quadratic [333, 1983, Thm. 5.2.1].

Computationally, Newton's method is implemented as

$$\begin{aligned} \text{solve } J(x_i)d_i &= -F(x_i), \\ x_{i+1} &= x_i + d_i. \end{aligned}$$

In practical computation rounding and other errors are introduced, so the computed iterates $\hat{x}_i$ actually satisfy

$$\hat{x}_{i+1} = \hat{x}_i - (J(\hat{x}_i) + E_i)^{-1} (F(\hat{x}_i) + e_i) + \varepsilon_i, \quad (25.2)$$

where

- e_i is the error made when computing the residual $F(\hat{x}_i)$,
- E_i is the error incurred in forming $J(\hat{x}_i)$ and solving the linear system for d_i , and
- ε_i is the error made when adding the correction $\hat{d}_i$ to $\hat{x}_i$.

Note that the term E_i in (25.2) enables the model to cover modified Newton methods, in which the Jacobian is held constant for several iterations in order to reduce the cost of the method.

The question of interest in this chapter is how the various errors in (25.2) affect the convergence of Newton's method. In particular, it is important to know the limiting accuracy and limiting residual, that is, how small $\|x_* - \hat{x}_i\|$ and $\|F(\hat{x}_i)\|$ are guaranteed to become as n increases, where x_* is the solution to which the iteration would converge in the absence of errors. One might begin an analysis by arguing that (sufficiently small) errors made on a particular iteration are damped out by Newton's method, since the method is locally convergent. However, we are concerned in (25.2) with systematic errors, possibly much larger than the working precision, and how the errors from different iterations interact is not obvious.

In the next section we give general error analysis results for Newton's method in floating point arithmetic that cater for a wide variety of situations: they allow for extended precision in computation of the residual and the use of possibly unstable linear system solvers.

25.2. Error Analysis

We assume that $F(\hat{x}_i)$ is computed in the possibly extended precision $\bar{u} \leq u$ before rounding back to working precision u , and that $\hat{d}_i$ and $\hat{x}_i$ are computed at precision u . Hence we assume that there exists a function ψ depending on F , $\hat{x}_i$, and $\bar{u}$ such that

$$\|e_i\| \leq u\|F(\hat{x}_i)\| + \psi(F, \hat{x}_i, \bar{u}). \quad (25.3)$$

The norm in this chapter denotes any absolute vector norm and the corresponding subordinate matrix norm. Note that standard error analysis shows that $\|e_i\| \leq u\|F(\hat{x}_i)\|$ is the best we can obtain in practice for both mixed and fixed precision. We assume that the error E_i satisfies

$$\|E_i\| \leq u\phi(F, \hat{x}_i, n, u), \quad (25.4)$$

for some function ϕ that reflects both the instability of the linear system solver and the error made when approximating or forming $J(\hat{x}_i)$. In practice, we certainly have $\phi(F, \hat{x}_i, n, u) \geq \|J(\hat{x}_i)\|$. For the error ε_i , we have

$$\|\varepsilon_i\| \leq u(\|\hat{x}_i\| + \|\hat{d}_i\|).$$

Finally, we assume that J is Lipschitz continuous with constant β , that is,

$$\|J(v) - J(w)\| \leq \beta\|v - w\| \quad \text{for all } v, w \in \mathbb{R}^n.$$

The first result bounds the limiting accuracy.

Theorem 25.1 (Tisseur). *Assume that there is an x_* such that $F(x_*) = 0$ and $J_* = J(x_*)$ is nonsingular with*

$$u\kappa(J_*) \leq \frac{1}{8}. \quad (25.5)$$

Assume also that for ϕ in (25.4),

$$u\|J(\hat{x}_i)^{-1}\|\phi(F, \hat{x}_i, n, u) \leq \frac{1}{8} \quad \text{for all } i. \quad (25.6)$$

Then, for all x_0 such that

$$\beta\|J_*^{-1}\|\|x_0 - x_*\| \leq \frac{1}{8}, \quad (25.7)$$

Newton's method in floating point arithmetic generates a sequence $\{\hat{x}_i\}$ whose normwise relative error decreases until the first i for which

$$\frac{\|\hat{x}_{i+1} - x_*\|}{\|x_*\|} \approx \frac{\|J_*^{-1}\|}{\|x_*\|} \psi(F, x_*, \bar{u}) + u. \quad (25.8)$$

Proof. See Tisseur [1138, 2001, §2.2]. $\square$

As a check, we note that in the absence of errors u , $\psi(F, v, \bar{u})$, and $\phi(F, v, n, u)$ are all zero and thus Theorem 25.1 implies local convergence of Newton's method.

In words, Theorem 25.1 says that if $J(x_*)$ is not too ill conditioned, the Jacobian evaluation and the solver are not too inaccurate, the Lipschitz constant β is not too large, and the initial guess x_0 is not too far from x_* , then the limiting accuracy is proportional to the condition of the Jacobian at the solution and the accuracy with which the residual is evaluated. Note that the term ϕ does not appear in (25.8), which shows that *errors in evaluating J and solving the linear system do not affect the limiting accuracy*, provided they are not too large. The details of the analysis in [1138, 2001] show that these latter errors do affect the rate of convergence and that this rate is essentially independent of $\bar{u}$.

The next result bounds the limiting residual.

Theorem 25.2 (Tisseur). *Assume that there is an x_* such that $F(x_*) = 0$, that $J_* = J(x_*)$ is nonsingular, that (25.5) and (25.6) are satisfied, and that the limiting accuracy $g \approx \|J_*^{-1}\|\psi(F, x_*, \bar{u}) + u\|x_*\|$ satisfies $\beta g \|J_*^{-1}\| \leq 1/8$. Then, for all x_0 such that (25.7) holds, the sequence $\{\|F(\hat{x}_i)\|\}$ of residual norms generated by Newton's method in floating point arithmetic decreases until*

$$\|F(\hat{x}_{i+1})\| \approx \psi(F, \hat{x}_i, \bar{u}) + u\|J(\hat{x}_i)\| \|\hat{x}_i\|. \quad (25.9)$$

Proof. See Tisseur [1138, 2001, §2.3]. $\square$

Theorem 25.2 shows that, under very similar conditions to those in Theorem 25.1, the limiting residual is at the level of the error made in computing the residual plus the term $u\|J(\hat{x}_i)\| \|\hat{x}_i\|$. This latter term is inevitable: from the Taylor series

$$F(x_* + \Delta x_*) = F(x_*) + J(x_*)\Delta x_* + O(\|\Delta x_*\|^2),$$

we see that merely rounding the exact solution to $\tilde{x}_* = x_* + \Delta x_*$, $\|\Delta x_*\| \leq u\|x_*\|$ gives

$$\|F(\tilde{x}_*)\| \lesssim \|J(x_*)\| \|\Delta x_*\| \leq u\|J(x_*)\| \|x_*\|.$$

As for the limiting accuracy, the limiting residual does not depend on the errors in evaluating J or in solving the linear systems, and the analysis in [1138, 2001] shows that the rate of decrease of the residual does not depend on $\bar{u}$.

Theorems 25.1 and 25.2 confirm the folklore that Newton's method must be provided with good function values if it is to work well in practice.

25.3. Special Cases and Experiments

A particularly simple but important special case of Newton's method is iterative refinement of linear systems. Iterative refinement, described in Chapter 12, is equivalent to Newton's method with $F(x) = b - Ax : \mathbb{R}^n \rightarrow \mathbb{R}^n$, for which $J(x) = A$, and thus we can take the Lipschitz constant $\beta = 0$.

If the residual $r = F(\hat{x})$ is computed at precision $\bar{u}$ (for example, using the Extended and Mixed Precision BLAS described in §27.10 when $\bar{u} < u$), then for ψ in (25.3) we can take

$$\psi(F, \hat{x}, \bar{u}) = \bar{\gamma}_{n+1}(\|A\| \|\hat{x}\| + \|b\|),$$

where

$$\bar{\gamma}_n = \frac{n\bar{u}}{1 - n\bar{u}}.$$

Theorem 25.1 then shows that if $u\kappa(A)$ is sufficiently less than 1 and if the linear system solver is not too unstable then iterative refinement reduces the relative forward error until

$$\frac{\|\hat{x}_i - x\|}{\|x\|} \approx u + \kappa(A)\bar{\gamma}_{n+1}.$$

If $\bar{u} = u^2$ then the relative error is of order u provided that $n\kappa(A)u \leq 1$.

Under the same assumptions, Theorem 25.2 shows further that the norm of the residual $r = b - Ax$ decreases until

$$\|\hat{r}_i\| \approx \max(\bar{\gamma}_{n+1}, u)(\|A\| \|\hat{x}\| + \|b\|),$$

so that, exploiting the connection between the residual and the backward error in Theorem 7.1, iterative refinement yields a normwise backward error $\eta(\hat{x}) \approx \max(\bar{\gamma}_{n+1}, u)$. These conclusions agree with those in §§12.1 and 12.2, modulo the use of normwise analysis in this chapter rather than componentwise analysis. This analysis of Newton's method therefore loses little by its generality.

To illustrate the analysis numerically we consider the eigenvalue problem $Ax = \lambda x$, which can be posed as the nonlinear system $F(v) = 0$, where

$$F(v) = \begin{bmatrix} (A - \lambda I)x \\ e_s^T x - 1 \end{bmatrix}, \quad v = \begin{bmatrix} x \\ \lambda \end{bmatrix}. \quad (25.10)$$

The last component of F serves to normalize the eigenvector, and here s is some fixed integer. The Jacobian is

$$J(v) = \begin{bmatrix} A - \lambda I & -x \\ e_s^T & 0 \end{bmatrix},$$

for which we can take the Lipschitz constant to be $2\|A\|$. The eigenvalue λ of interest is assumed to be simple, which implies that $J(v)$ is nonsingular at the solution. For the evaluation of F at $\hat{v}_i = [\hat{x}_i \ \hat{\lambda}_i]^T$ we have

$$\psi(F, \hat{v}_i, \bar{u}) = \bar{\gamma}_{n+1}(\|A\| + |\hat{\lambda}_i|)\|\hat{x}_i\|.$$

In a MATLAB experiment we used Newton's method to compute the smallest eigenvalue and corresponding eigenvector of the 10×10 Frank matrix. The Frank matrix [443, 1958] (MATLAB's `gallery('frank')`) is an upper Hessenberg matrix with integer entries and its eigenvalues are notoriously ill conditioned. We obtained the exact eigenpair using 50 digit arithmetic with MATLAB's Symbolic Math Toolbox. For the starting vector v_0 we used the exact eigenpair rounded to 2 binary digits and we took $s = 4$. We implemented the iteration with the residual computed both at the working precision ($\bar{u} = u$) and at twice the working precision ($\bar{u} = u^2$), the latter using the Symbolic Math Toolbox; linear systems were solved by GEPP. The 2-norm relative errors $\|v - \hat{v}_i\|_2 / \|v\|_2$ and residuals $\|F(\hat{v}_i)\|_2$ are shown in Figure 25.1. With $\bar{u} = u$, Newton's method is able to provide a solution with relative error of order 10^{-10} , but with $\bar{u} = u^2$ full accuracy is obtained. The

predictions from the analysis are a little pessimistic, but correctly indicate that doubling the precision will not improve the residual (because $\|J(v)\| \|v\| \approx 10^6$ at the solution, and so the second term in (25.9) dominates), but will achieve a fully accurate solution. For comparison, the eigenpair computed by MATLAB's `eig` command has relative error 3×10^{-8} and residual 5×10^{-10} .

25.4. Conditioning

In order to determine the sensitivity of the solution of a nonlinear system to perturbations in the data it is first necessary to decide what is the data. The problem formulation, $F(x) = 0$, does not explicitly specify the data and hence does not enable us to identify the allowable perturbations of F . We will assume that F depends parametrically on a vector d of data:

$$F(x) \equiv F(x; d), \quad d \in \mathbb{R}^m.$$

In many cases it is clear what are the problem parameters and hence how to define d . For a linear system, $F(x) = b - Ax$, d naturally comprises the elements of A and/or b , while for the eigenproblem (25.10) we can take $d = \text{vec}(A)$. In other cases the choice of d should be guided by the source of errors. For example, if $F(x) = x - \cos(x) \in \mathbb{R}$ then we can assume that

$$fl(F(x)) = (1 + \delta_3)(x - (1 + \delta_2)\cos(x(1 + \delta_1))), \quad |\delta_i| \leq \tilde{\gamma}_1, \quad i = 1:3,$$

under a reasonable assumption on the cosine evaluation (cf. the discussion on page 7). A suitable choice of d is then

$$F(x; d) = x - d_1 \cos x.$$

The reasoning is that the multiplicative factor $1 + \delta_3$ can have little influence when solving $F(x) = 0$, while the $1 + \delta_1$ factor can safely be absorbed into the perturbation in the solution space.

Suppose now that $F(x_*; d) = 0$, that F is a sufficiently smooth function of x and d , and that $F_x(x_*; d)$ is nonsingular, where we write

$$F_x = \left(\frac{\partial F_i}{\partial x_j} \right) \in \mathbb{R}^{n \times n}, \quad F_d = \left(\frac{\partial F_i}{\partial d_j} \right) \in \mathbb{R}^{n \times m}.$$

For $\|\Delta d\|$ sufficiently small the implicit function theorem tells us that there is a unique Δx such that $F(x_* + \Delta x; d + \Delta d) = 0$. A Taylor series expansion about (x_*, d) gives

$$F(x_* + \Delta x; d + \Delta d) = F_x(x_*; d)\Delta x + F_d(x_*; d)\Delta d + O(\max(\|\Delta x\|, \|\Delta d\|)^2),$$

and so

$$\Delta x = -F_x(x_*; d)^{-1}F_d(x_*; d)\Delta d + O(\|\Delta d\|^2),$$

which expresses the perturbation in the solution in terms of the perturbation in the data. It is then clear that the condition number defined by

$$\text{cond}(F, x_*; d) := \limsup_{\epsilon \rightarrow 0} \left\{ \frac{\|\Delta x\|_\infty}{\epsilon \|x_*\|_\infty} : F(x_* + \Delta x; d + \Delta d) = 0, \|\Delta d\| \leq \epsilon \|d\| \right\}$$

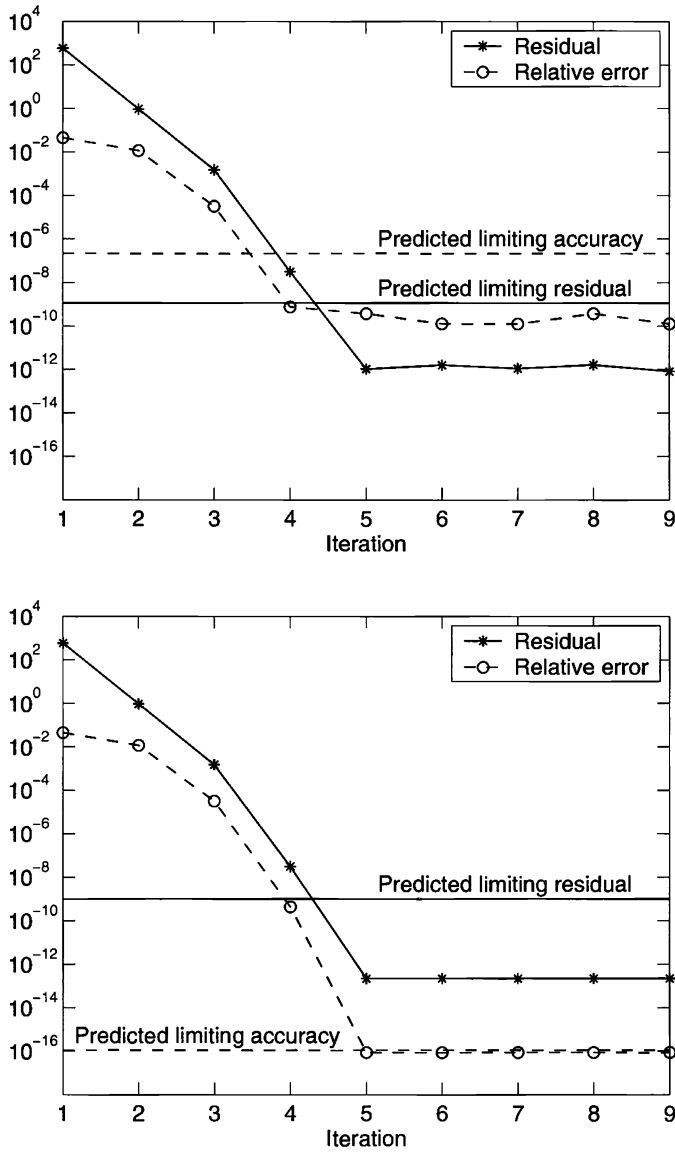


Figure 25.1. Newton's method with $\bar{u} = u$ (top) and $\bar{u} = u^2$ (bottom).

is given by

$$\text{cond}(F, x_*; d) = \|F_x(x_*; d)^{-1} F_d(x_*; d)\| \frac{\|d\|}{\|x_*\|}. \quad (25.11)$$

For the special case of a linear system, $F(x) = b - Ax$, taking $d = \text{vec}(A)$ and the Frobenius norm, we find

$$\text{cond}(F, x_*; d) = \|A^{-1} \begin{bmatrix} x_1 I & x_2 I & \dots & x_n I \end{bmatrix}\|_F \frac{\|A\|_F}{\|x\|_2} = \|A^{-1}\|_F \|A\|_F,$$

which agrees (modulo the nonsubordinate choice of norm) with the condition number $\kappa_{A,0}(A, x)$ in (7.5) for linear systems.

A natural question is how to reconcile the condition number (25.11) with the limiting accuracy (25.8) of Newton's method, since the latter bears no simple relationship with the former. In particular, is it possible that $\|J(x_*)^{-1}\| = \|F_x(x_*; d)\|^{-1}$ be large yet $\text{cond}(F, x_*; d)$ small, in which case Newton's method is potentially unstable? Woźniakowski [1255, 1977] has shown that, under a moderately strong assumption on the errors in the evaluation of F relative to a given set of parameters d , Newton's method does achieve a forward error bounded, to first order, by a multiple of $\text{cond}(F, x_*; d)u$. This result does not imply any weakness in Theorem 25.1, because Theorem 25.1 makes a much weaker assumption on the errors in the evaluation of F and does not involve a parameter vector d . For illustration we consider the example, from [1255, 1977],

$$F(x) = \begin{bmatrix} x_1 - x_2 \\ x_1^2 + \mu x_2^2 - \mu \end{bmatrix}, \quad \mu > 0, \quad (25.12)$$

which has the solution $v = (\mu/(1+\mu))^{1/2} [1 \ 1]^T$. We regard μ as fixed and consider how to parametrize F in a way that reflects the rounding errors in its evaluation. It is straightforward to show that

$$fl(F(x)) = \text{diag}(1 + \delta_1, 1 + \delta_2) \begin{bmatrix} x_1 - x_2 \\ (1 + \delta_3)(x_1^2 + \mu x_2^2) - \mu \end{bmatrix}, \quad (25.13)$$

where $|\delta_1|, |\delta_2| \leq u$, $|\delta_3| \leq \gamma_3$. Hence we take

$$F(x; d) = \begin{bmatrix} x_1 - x_2 \\ d(x_1^2 + \mu x_2^2) - \mu \end{bmatrix},$$

where $d = 1$ is the parameter value of interest. Since

$$F_x(v; 1)^{-1} F_d(v; 1) = \begin{bmatrix} 1 & -1 \\ 2v_1 & 2\mu v_2 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ v_1^2 + \mu v_2^2 \end{bmatrix} = \frac{1}{2}v,$$

it follows that, for any norm,

$$\text{cond}(F, v; 1) = \frac{1}{2} \quad \text{for all } \mu.$$

Thus the problem is very well conditioned, yet the Jacobian $J(v) = F_x(v; 1)$ becomes singular as $\mu \rightarrow 0$ or $\mu \rightarrow \infty$.

We applied Newton's method to (25.12) in MATLAB with $x_0 = [1 \ 2]^T$ and $\mu = 10^8$ and found that the limiting residual was of order 10^{-8} , which is as predicted by Theorem 25.2, since $\|J(v)\| = \|F_x(v; 1)\| \approx 10^8$. It is easy to check that for this problem

$$\psi(F, x_*, \bar{u}) \approx u\mu|x_2^2| \approx u\mu \approx u\|F(x_*)\|,$$

so the limiting accuracy (25.8) is roughly $\kappa(J_*)u$, which is of order 10^{-8} . However, the actual limiting accuracy was approximately u , which reflects the conditioning of the problem rather than the conditioning of the Jacobian $J = F_x$ at the solution. The reason (25.8) overestimates the limiting accuracy is that it does not take account of the nature of the errors in evaluating F (displayed in (25.13)).

Rheinboldt [983, 1976] argues that an appropriate condition number for $F(x) = 0$ on a closed set S is

$$C(F, S) = \frac{\max_{u, v \in S} \|F(u) - F(v)\| / \|u - v\|}{\min_{u, v \in S} \|F(u) - F(v)\| / \|u - v\|}.$$

This quantity directly generalizes the matrix condition number $\kappa(A)$ (see Problem 6.6). If F is smooth and we let S shrink around a solution x_* then $C(F, S)$ tends to $\kappa(J(x_*))$. Rheinboldt's analysis leads him to conclude that “asymptotically near z the conditioning of the nonlinear mapping F and its derivative $F'(z)$ are the same”. There is no contradiction with the analysis above: the condition number cond in (25.11) is a structured condition number whereas $C(F, S)$ corresponds to allowing arbitrary perturbations to F . The example (25.12) shows the importance of exploiting structure in the problem when it is present.

25.5. Stopping an Iterative Method

How to terminate an iteration from a nonlinear equation solver is an important question to which there is no simple answer. Reliable termination is more difficult than in the linear equation case discussed in §17.5, since there is no concept of backward error for a nonlinear equation and the notion of “scale” (how to normalize F) is less clear.

Stopping tests can be based on the norm of the residual, $\|F(x_i)\|$, or the relative error. In a small neighborhood of a solution the residual and relative error can be related, because F is well approximated by a linear function there. If y is sufficiently close to a solution x_* then a Taylor series gives

$$F(y) \approx F(x_*) + J(x_*)(y - x_*) = J(x_*)(y - x_*).$$

It follows that

$$\frac{\|F(y)\|}{\|J(x_*)\|} \lesssim \|x_* - y\| \lesssim \|J(x_*)^{-1}\| \|F(y)\|,$$

which are familiar inequalities in the case of linear equations (see Problem 7.2). For $\|x_* - y\|$ sufficiently small these inequalities are rigorous if the lower bound is multiplied by $1/2$ and the upper bound by 2 [721, 1995, Lem. 4.3.1]. We conclude that the ratio of the relative error and the residual is bounded in terms of the conditioning of the Jacobian at the solution.

In stopping tests the relative error $\|x_{i+1} - x_*\|/\|x_*\|$ is typically approximated by the relative difference $\|x_{i+1} - x_i\|/\|x_{i+1}\|$ between two successive iterates. The latter quantity is actually approximating $\|x_i - x_*\|/\|x_*\|$ rather than $\|x_{i+1} - x_*\|/\|x_*\|$. For

$$x_{i+1} - x_* = (x_{i+1} - x_i) + (x_i - x_*),$$

and for sufficiently fast convergence $\|x_{i+1} - x_*\| \ll \|x_i - x_*\|$ and hence the two terms on the right-hand side are of roughly equal magnitude and opposite signs. To be more precise, consider Newton's method. Quadratic convergence guarantees that

$$\|x_{i+1} - x_*\| \leq c \|x_i - x_*\|^2 \quad (25.14)$$

close to convergence, where c is a constant. Now

$$\begin{aligned} \|x_i - x_*\| &\leq \|x_i - x_{i+1}\| + \|x_{i+1} - x_*\| \\ &\leq \|x_i - x_{i+1}\| + c \|x_i - x_*\|^2. \end{aligned}$$

"Solving" for $\|x_i - x_*\|$ and substituting in (25.14) gives

$$\begin{aligned} \|x_{i+1} - x_*\| &\leq c \left(\frac{\|x_{i+1} - x_i\|}{1 - c \|x_i - x_*\|} \right)^2 \\ &\leq 2c \|x_{i+1} - x_i\|^2, \end{aligned}$$

for small enough $\|x_i - x_*\|$, so the error in x_{i+1} is bounded in terms of the square of $\|x_{i+1} - x_i\|$.

25.6. Notes and References

This chapter is based on Tisseur [1138, 2001, §2.2], who gives the first comprehensive analysis of the limiting accuracy and limiting residual of Newton's method in floating point arithmetic. Earlier work on the subject includes that of Lancaster [763, 1966], Woźniakowski [1255, 1977], Ypma [1273, 1983], and Dennis and Walker [335, 1984].

Related to the results in this chapter is work on inexact Newton methods [304, 1982], which solve the Newton equations $J(x_i)d_i = -F(x_i)$ approximately—by an iterative method, for example. These methods are usually analyzed in terms of a forcing sequence η_i such that the computed x_i satisfy $\|J(x_i)d_i + F(x_i)\| \leq \eta_i \|F(x_i)\|$. The theory of inexact Newton methods does not distinguish the different sources of error and is concerned with the rate of convergence rather than with the limiting accuracy or residual in the presence of persistent rounding errors. For more details of inexact Newton methods see Kelley [721, 1995, Chap. 6] or Nocedal and Wright [894, 1999] and the references therein.

For historical background on Newton's method see Ypma [1275, 1995].

Phrasing the eigenproblem as a nonlinear system and solving it by Newton's method is an old idea. For a detailed discussion see Tisseur [1138, 2001], which contains references to the earlier literature.

Section 25.4 is based on Woźniakowski [1255, 1977]. Related issues are discussed by Chaitin-Chatelin and Frayssé [208, 1996]. The literature contains relatively little on the conditioning of the general nonlinear equations problem and the condition number (25.11) does not seem to be well known.

Problems

25.1. (Descloux [337, 1963]) Consider the iterative process

$$x_{k+1} = G(x_k) + e_k, \quad x_0 \text{ given,}$$

where $G : \mathbb{R}^n \rightarrow \mathbb{R}^n$ satisfies

$$\|G(x) - a\| \leq \beta \|x - a\| \quad \text{for all } x \in \mathbb{R}^n, \quad (25.15)$$

for some $\beta \in (0, 1)$, and where $\|e_k\| \leq \alpha$ for all k . Note that a must satisfy $G(a) = a$.

(a) Show that

$$\begin{aligned} \text{if } \|x_k - a\| &\leq \frac{\alpha}{1-\beta} && \text{then } \|x_{k+1} - a\| &\leq \frac{\alpha}{1-\beta}, \\ \text{if } \|x_k - a\| &> \frac{\alpha}{1-\beta} && \text{then } \|x_{k+1} - a\| &< \|x_k - a\|. \end{aligned}$$

(b) Show that the sequence $\{x_k\}$ is bounded and its points of accumulation x satisfy

$$\|x - a\| \leq \frac{\alpha}{1-\beta}.$$

(c) Explain the practical relevance of the result of (b).

25.2. (RESEARCH PROBLEM) Extend the results reported in this chapter to problems for which the Jacobian is singular at the solution. Useful references are Griewank [525, 1985] and Ypma [1274, 1983].

Chapter 26

Automatic Error Analysis

*Given the pace of technology,
I propose we leave math to the machines and go play outside.*
— CALVIN, *Calvin and Hobbes* by Bill Watterson (1992)

*To analyse a given numerical algorithm we proceed as follows.
A number which measures the effect of roundoff error
is assigned to each set of data.
“Hill-climbing” procedures are then applied to search for
values large enough to signal instability.*
— WEBB MILLER, *Software for Roundoff Analysis* (1975)

*The prospects for effective use of interval arithmetic look very good,
so efforts should be made to increase its availability
and to make it as user-friendly as possible.*
— DONALD E. KNUTH, *The Art of Computer Programming*,
Volume 2, *Seminumerical Algorithms* (1998)

*Despite its wide use,
until quite recently the Nelder-Mead method and its ilk have been
deprecated, scorned, or ignored
by almost all of the mainstream optimization community.*
— MARGARET H. WRIGHT, *Direct Search
Methods: Once Scorned, Now Respectable* (1996)

Automatic error analysis is any process in which we use the computer to help us analyse the accuracy or stability of a numerical computation. The idea of automatic error analysis goes back to the dawn of scientific computing. For example, running error analysis, described in §3.3, is a form of automatic error analysis; it was used extensively by Wilkinson on the ACE machine. Various forms of automatic error analysis have been developed. In this chapter we describe in detail the use of direct search optimization for investigating questions about the stability and accuracy of algorithms. We also describe interval analysis and survey other forms of automatic error analysis.

26.1. Exploiting Direct Search Optimization

Is Algorithm X numerically stable? How large can the growth factor be for Gaussian elimination (GE) with pivoting strategy P? By how much can condition estimator C underestimate the condition number of a matrix? These types of questions are common, as we have seen in this book. Usually, we attempt to answer such questions by a combination of theoretical analysis and numerical experiments with random and nonrandom data. But a third approach can be a valuable supplement to the first two: phrase the question as an optimization problem and apply a direct search method.

A direct search method for the problem

$$\max_{x \in \mathbb{R}^n} f(x), \quad f: \mathbb{R}^n \rightarrow \mathbb{R} \quad (26.1)$$

is a numerical method that attempts to locate a maximizing point using function values only and does not use or approximate derivatives of f . Such methods are usually based on heuristics that do not involve assumptions about the function f . Various direct search methods have been developed; for surveys see Powell [948, 1970], [950, 1998], Swann [1112, 1972], [1113, 1974], Lewis, Torczon, and Trossett [783, 2000], and Wright [1260, 1996]. Most of the methods were developed in the 1960s, in the early years of numerical optimization. For problems in which f is smooth, direct search methods have largely been supplanted by more sophisticated optimization methods that use derivatives (such as quasi-Newton methods and conjugate gradient methods), but they continue to find use in applications where f is not differentiable, or even not continuous. These applications range from chemical analysis [994, 1977], where direct search methods have found considerable use, to the determination of drug doses in the treatment of cancer [105, 1991]; in both applications the evaluation of f is affected by experimental errors. Lack of smoothness of f , and the difficulty of obtaining derivatives when they exist, are characteristic of the optimization problems we consider here.

The use of direct search can be illustrated with the example of the growth factor for GE on $A \in \mathbb{R}^{n \times n}$,

$$\rho_n(A) = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

where the $a_{ij}^{(k)}$ are the intermediate elements generated during the elimination.

We know from §9.3 that the growth factor governs the stability of GE, so for a given pivoting strategy we would like to know how big $\rho_n(A)$ can be.

To obtain an optimization problem of the form (26.1) we let $x = \text{vec}(A) \in \mathbb{R}^{n^2}$, and we define $f(x) := \rho_n(A)$. Then we wish to determine

$$\max_{x \in \mathbb{R}^{n^2}} f(x) \equiv \max_{A \in \mathbb{R}^{n \times n}} \rho_n(A).$$

Suppose, first, that no pivoting is done. Then f is defined and continuous at all points where the elimination does not break down, and it is differentiable except at points where there is a tie for the maximum in the numerator or denominator of the expression defining $\rho_n(A)$. We took $n = 4$ and applied the direct search maximizer MDS (described in §26.2) to $f(x)$, starting with the identity matrix $A = I_4$. After 11 iterations and 433 function evaluations, the maximizer converged¹⁷, having located the matrix¹⁸

$$B = \begin{bmatrix} 1.7846 & -0.2760 & -0.2760 & -0.2760 \\ -3.3848 & 0.7240 & -0.3492 & -0.2760 \\ -0.2760 & -0.2760 & 1.4311 & -0.2760 \\ -0.2760 & -0.2760 & -0.2760 & 0.7240 \end{bmatrix},$$

for which $\rho_4(B) = 1.23 \times 10^5$. (The large growth is a consequence of the submatrix $B(1:3, 1:3)$ being ill conditioned; B itself is well conditioned.) Thus the optimizer readily shows that $\rho_n(A)$ can be very large for GE without pivoting.

Next, consider GE with partial pivoting (GEPP). Because the elimination cannot break down, f is now defined everywhere, but it is usually discontinuous when there is a tie in the choice of pivot element, because then an arbitrarily small change in A can alter the pivot sequence. We applied the maximizer MDS to f , this time starting with the 4×4 instance of the orthogonal matrix¹⁹ A with $a_{ij} = (2/\sqrt{2n+1}) \sin(2ij\pi/(2n+1))$ (cf. (9.12)), for which $\rho_4(A) = 2.32$. After 29 iterations and 1169 function evaluations the maximizer converged to a matrix B with $\rho_4(B) = 5.86$. We used this matrix to start the maximizer AD (described in §26.2); it took 5 iterations and 403 function evaluations to converge to the matrix

$$C = \begin{bmatrix} 0.7248 & 0.7510 & 0.5241 & 0.7510 \\ 0.7317 & 0.1889 & 0.0227 & -0.7510 \\ 0.7298 & -0.3756 & 0.1150 & 0.7511 \\ -0.6993 & -0.7444 & 0.6647 & -0.7500 \end{bmatrix},$$

for which $\rho_4(C) = 7.939$. This is one of the matrices described in Theorem 9.7, modulo the convergence tolerance.

These examples, and others presented below, illustrate the following attractions of using direct search methods to investigate the stability of a numerical computation.

¹⁷In the optimizations of this section we used the convergence tests described in §26.2 with $\text{tol} = 10^{-3}$. There is no guarantee that when convergence is achieved it is to a local maximum; see §26.2.

¹⁸All numbers quoted are rounded to the number of significant figures shown.

¹⁹This matrix is MATLAB's `gallery('orthog', n, 2)`.

(1) The simplest possible formulation of optimization problem is often sufficient to yield useful results. Derivatives are not needed, and direct search methods tend to be insensitive to lack of smoothness in the objective function f . Unboundedness of f is a favourable property—direct search methods usually quickly locate large values of f .

(2) Good progress can often be made from simple starting values, such as an identity matrix. However, prior knowledge of the problem may provide a good starting value that can be substantially improved (as in the partial pivoting example).

(3) Usually it is the global maximum of f in (26.1) that is desired (although it is often sufficient to know that f can exceed a specified value). When a direct search method converges it will, in general, at best have located a *local* maximum—and in practice the maximizer may simply have stagnated, particularly if a slack convergence tolerance is used. However, further progress can often be made by *restarting* the same (or a different) maximizer, as in the partial pivoting example. This is because for methods that employ a simplex (such as the MDS method), the behaviour of the method starting at x_0 is determined not just by x_0 but also by the $n + 1$ vectors in the initial simplex constructed at x_0 .

(4) The numerical information revealed by direct search provides a starting point for further theoretical analysis. For example, the GE experiments above strongly suggest the (well-known) results that $\rho_n(A)$ is unbounded without pivoting and bounded by 2^{n-1} for partial pivoting, and inspection of the numerical data suggests the methods of proof.

When applied to smooth problems the main disadvantages of direct search methods are that they have at best a linear rate of convergence and they are unable to determine the nature of the point at which they terminate (since derivatives are not calculated). These disadvantages are less significant for the problems we consider, where it is not necessary to locate a maximum to high accuracy and objective functions are usually nonsmooth. (Note that these disadvantages are not necessarily shared by methods that implicitly or explicitly *estimate* derivatives using function values, such as methods based on conjugate directions, for which see Powell [948, 1970], [949, 1975]; however, these are not normally regarded as direct search methods.)

A final attraction of direct search is that it can be used to test the correctness of an *implementation* of a stable algorithm. The software in question can be used in its original form and does not have to be translated into some other representation.

26.2. Direct Search Methods

For several years I have been using MATLAB implementations of three direct search methods. The first is the alternating directions (AD) method (also known as the coordinate search method). Given a starting value x it attempts to solve the problem (26.1) by repeatedly maximizing over each coordinate direction in turn:

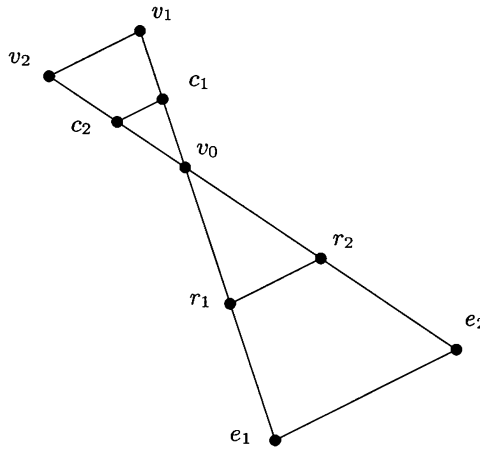


Figure 26.1. *The possible steps in one iteration of the MDS method when $n = 2$.*

```

repeat
    % One iteration comprises a loop over all components of  $x$ .
    for  $i = 1:n$ 
        find  $\alpha$  such that  $f(x + \alpha e_i)$  is maximized (line search)
        set  $x \leftarrow x + \alpha e_i$ 
    end
until converged

```

AD is one of the simplest of all optimization methods and the fundamental weakness that it ignores any interactions between the variables is well known. Despite the poor reputation of AD we have found that it can perform well on the types of problems considered here. In our MATLAB implementation of AD the line search is done using a crude scheme that begins by evaluating $f(x + he_i)$ with $h = 10^{-4}x_i$ (or $h = 10^{-4} \max(\|x\|_\infty, 1)$ if $x_i = 0$); if $f(x + he_i) \leq f(x)$ then the sign of h is reversed. Then if $f(x + he_i) > f(x)$, h is doubled at most 25 times until no further increase in f is obtained. Our convergence test checks for a sufficient relative increase in f between one iteration and the next: convergence is declared when

$$f_k - f_{k-1} \leq \text{tol} |f_{k-1}|, \quad (26.2)$$

where f_k is the highest function value at the end of the k th iteration. The AD method has the very modest storage requirement of just a single n -vector.

The second method is the multidirectional search method (MDS) of Dennis and Torczon. This method employs a simplex, which is defined by $n + 1$ vectors $\{v_i\}_0^n$ in $\mathbb{R}^n$. One iteration in the case $n = 2$ is represented pictorially in Figure 26.1, and may be explained as follows.

The initial simplex is $\{v_0, v_1, v_2\}$ and it is assumed that $f(v_0) = \max_i f(v_i)$. The purpose of an iteration is to produce a new simplex at one of whose vertices f exceeds $f(v_0)$. In the first step the vertices v_1 and v_2 are reflected about v_0 along

the lines joining them to v_0 , yielding r_1 and r_2 and the reflected simplex $\{v_0, r_1, r_2\}$. If this reflection step is successful, that is, if $\max_i f(r_i) > f(v_0)$, then the edges from v_0 to r_i are doubled in length to give an expanded simplex $\{v_0, e_1, e_2\}$. The original simplex is then replaced by $\{v_0, e_1, e_2\}$ if $\max_i f(e_i) > \max_i f(r_i)$, and otherwise by $\{v_0, r_1, r_2\}$. If the reflection step is unsuccessful then the edges $v_0 - v_i$ of the original simplex are shrunk to half their length to give the contracted simplex $\{v_0, c_1, c_2\}$. This becomes the new simplex if $\max_i f(c_i) > \max_i f(v_i)$, in which case the current iteration is complete; otherwise the algorithm jumps back to the reflection step, now working with the contracted simplex. For further details of the MDS method see Dennis and Torczon [334, 1991], Kelley [722, 1999, §8.2], and Torczon [1146, 1989], [1147, 1991].

The MDS method requires at least $2n$ independent function evaluations per iteration, which makes it very suitable for parallel implementation. Generalizations of the MDS method that are even more suitable for parallel computation are described in [334, 1991]. The MDS method requires $O(n^2)$ elements of storage for the simplices, but this can be reduced to $O(n)$ (at the cost of extra bookkeeping) if an appropriate choice of initial simplex is made [334, 1991].

Our implementation of the MDS method provides two possible starting simplices, both of which include the starting point x_0 : a regular one (all sides of equal length) and a right-angled one based on the coordinate axes, both as described by Torczon in [1146, 1989]. The scaling is such that each edge of the regular simplex, or each edge of the right-angled simplex that is joined to x_0 , has length $\max(\|x_0\|_\infty, 1)$. Also as in [1146, 1989], the main termination test halts the computation when the relative size of the simplex is no larger than a tolerance tol , that is, when

$$\frac{1}{\max(1, \|v_0\|_1)} \max_{1 \leq i \leq n} \|v_i - v_0\|_1 \leq \text{tol}. \quad (26.3)$$

Unless otherwise stated, we used $\text{tol} = 10^{-3}$ in (26.2) and (26.3) in all our experiments.

The AD and MDS methods are both examples of pattern search methods. Torczon [1148, 1997] shows that for any pattern search method if the level set of f at the starting vector is compact and f is continuously differentiable on a neighborhood of this level set then every limit point of the sequence of iterates is a stationary point (subject to certain technical conditions on the definition of the method). For additional results specific to the MDS method see Torczon [1147, 1991].

The third method that we have used is the Nelder–Mead direct search method [881, 1965], [722, 1999, §8.1], which also employs a simplex but which is fundamentally different from the MDS method. We omit a description since the method is described in textbooks (see, for example, Gill, Murray, and Wright [486, 1981, §4.2.2], or Press, Teukolsky, Vetterling, and Flannery [953, 1992, §10.4]). Our practical experience of the Nelder–Mead method is that while it can sometimes outperform the MDS method, the MDS method is generally superior for our purposes. No convergence results of the form described above for pattern search methods are available for the Nelder–Mead method, and such results cannot exist in view of an example of McKinnon [837, 1998]; see [762, 1998] for what is known.

It is interesting to note that the MDS method, the Nelder–Mead method, and our particular implementation of the AD method do not exploit the numerical values of f : their only use of f is to compare two function values to see which is the larger!

Our MATLAB implementations of the AD, MDS, and Nelder–Mead direct search methods are in the Matrix Computation Toolbox, described in Appendix D.

26.3. Examples of Direct Search

In this section we give examples of the use of direct search to investigate the behaviour of numerical algorithms.

26.3.1. Condition Estimation

MATLAB's function `rcond` uses the LAPACK condition number estimator, described in Algorithm 15.4, to produce a lower bound $\text{est}(A)$ for $\kappa_1(A)$, where $A \in \mathbb{R}^{n \times n}$. We are interested here in automatically generating counterexamples to `rcond`.

To put the problem in the form of (26.1), we define $x = \text{vec}(A)$ and

$$f(x) = \frac{\kappa_1(A)}{\text{est}(A)} \geq 1.$$

We note that, since the algorithm underlying `rcond` contains tests and branches, there are matrices A for which an arbitrarily small change in A can completely change the condition estimate; hence f has points of discontinuity.

We applied the MDS maximizer to `rcond` starting at the 5×5 version of the $n \times n$ matrix with $a_{ij} = \cos((i-1)(j-1)\pi/(n-1))$ (this is a Chebyshev–Vandermonde matrix, as used in §22.3.3, and is `gallery('orthog', n, -1)` in MATLAB). With $\text{tol} = 10^{-6}$, after 59 iterations and 3326 function evaluations the maximizer had located the (well-scaled) matrix

$$A = \begin{bmatrix} 1.1377 & 1.1653 & 0.8503 & 3.3805 & 1.1377 \\ 1.1377 & 0.7964 & 0.3155 & -0.5736 & -0.7518 \\ 1.1377 & -0.2186 & -0.8608 & 0.1404 & 1.4912 \\ 1.1377 & -0.7683 & -2.6908 & 1.5626 & -1.1289 \\ 1.1377 & 0.5523 & 0.5631 & -0.8623 & 1.4928 \end{bmatrix},$$

for which

$$\kappa_1(A) = 8.6 \times 10^6, \quad \text{est}(A) = 3.4 \times 10^1, \quad \frac{\kappa_1(A)}{\text{est}(A)} = 2.5 \times 10^6.$$

With relatively little effort on our part (most of the effort was spent experimenting with different starting matrices), the maximizer has discovered an example where the condition estimator fails to achieve its objective of producing an estimate correct to within an order of magnitude. The value of direct search maximization in this context is clear: it can readily demonstrate the fallibility of

a condition estimator—a task that can be extremely difficult to accomplish using theoretical analysis or tests with random matrices. Moreover, the numerical examples obtained from direct search may provide a starting point for the construction of parametrized theoretical ones, or for the improvement of a condition estimation algorithm.

In addition to measuring the quality of a single algorithm, direct search can be used to compare two competing algorithms to investigate whether one algorithm performs uniformly better than the other. We applied the MDS maximizer to the function

$$f(x) = \frac{\text{estC}(A)}{\text{estR}(A)},$$

where $\text{estR}(A)$ is the condition estimate from `rcond` and $\text{estC}(A)$ is the condition estimate from MATLAB's `condest`, which implements a block 1-norm estimator (see §15.4). If $f(x) > 1$ then `condest` has produced a larger lower bound for $\kappa_1(A)$ than `rcond`. Starting with the same 5×5 starting matrix as above, the MDS maximizer with $\text{tol} = 10^{-6}$ produced after 1426 function evaluations a matrix A for which $\text{estC}(A) = \kappa_1(A) = 2.5 \times 10^6$ and $f(x) = 5.0 \times 10^5$. With f defined as $f(x) = \text{estR}(A)/\text{estC}(A)$ we tried several different starting matrices but were unable to drive f higher than 2. This experiment confirms what we would expect: that the block estimator (iterating with $n \times 2$ matrices—the default in `condest`) provides condition estimates that can be much better than those from `rcond` (which iterates with vectors) and are rarely much worse.

26.3.2. Fast Matrix Inversion

We recall Strassen's inversion method from Problem 23.8: for

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad A_{ij} \in \mathbb{R}^{m \times m}, \quad n = 2m,$$

it uses the formulae

$$\begin{aligned} P_1 &= A_{11}^{-1}, & P_2 &= A_{21}P_1, \\ P_3 &= P_1A_{12}, & P_4 &= A_{21}P_3, \\ P_5 &= P_4 - A_{22}, & P_6 &= P_5^{-1}, \\ A^{-1} &= \begin{bmatrix} P_1 - P_3P_6P_2 & P_3P_6 \\ P_6P_2 & -P_6 \end{bmatrix}, \end{aligned}$$

where each of the matrix products is formed using Strassen's fast matrix multiplication method. Strassen's inversion method is clearly unstable for general A , because the method breaks down if A_{11} is singular. Indeed Strassen's inversion method has been implemented on a Cray-2 by Bailey and Ferguson [50, 1988] and tested for $n \leq 2048$, and these authors observe empirically that the method has poor numerical stability. Direct search can be used to gain insight into the numerical stability.

With $x = \text{vec}(A) \in \mathbb{R}^{n^2}$, define the stability measure

$$f(x) = \frac{\min\{\|A\hat{X} - I\|_\infty, \|\hat{X}A - I\|_\infty\}}{\|A\|_\infty \|\hat{X}\|_\infty}, \quad (26.4)$$

where $\hat{X}$ is the inverse of A computed using Strassen's inversion method. This definition of f is appropriate because, as shown in Chapter 14, for most conventional matrix inversion methods either the left residual $\hat{X}A - I$ or the right residual $A\hat{X} - I$ is guaranteed to have norm of order $u\|\hat{X}\|\|A\|$. To treat Strassen's inversion method as favourably as possible we use just one level of recursion; thus P_1 and P_6 are computed using GEPP but the multiplications are done with Strassen's method. We applied the MDS maximizer, with $\text{tol} = 10^{-9}$ in (26.3), starting with the 4×4 Vandermonde matrix whose (i, j) element is $((j-1)/3)^{i-1}$. After 34 iterations the maximizer had converged with $f = 0.838$, which represents complete instability. The corresponding matrix A is well conditioned, with $\kappa_2(A) = 82.4$. For comparison, the value of f when A is inverted using Strassen's method with *conventional* multiplication is $f = 6.90 \times 10^{-2}$; this confirms that the instability is not due to the use of fast multiplication techniques—it is inherent in the inversion formulae.

If A is a symmetric positive definite matrix then its leading principal submatrices are no more ill conditioned than the matrix itself, so we might expect Strassen's inversion method to be stable for such matrices. To investigate this possibility we carried out the same maximization as before, except we enforced positive definiteness as follows: when the maximizer generates a vector $x \equiv \text{vec}(B)$, A in (26.4) is defined as $A = B^TB$. Starting with a 4×4 random matrix A with $\kappa_2(A) = 6.71 \times 10^7$, the maximization yielded the value $f = 3.32 \times 10^{-8}$ after 15 iterations, and the corresponding value of f when conventional multiplication is used is $f = 6.61 \times 10^{-11}$ (the "maximizing" matrix A has condition number $\kappa_2(A) = 3.58 \times 10^9$).

The conclusion from these experiments is that Strassen's inversion method cannot be guaranteed to produce a small left or right residual even when A is symmetric positive definite and conventional multiplication is used. Hence the method must be regarded as being fundamentally unstable.

26.3.3. Roots of a Cubic

Explicit formulae can be obtained for the roots of a cubic equation using techniques associated with the 16th century mathematicians del Ferro, Cardano, Tartaglia, and Vieta [156, 1989], [368, 1990], [882, 1998]. The following development is based on Birkhoff and Mac Lane [114, 1977, §5.5].

Any nondegenerate cubic equation can be put in the form $x^3 + ax^2 + bx + c = 0$ by dividing through by the leading coefficient. We will assume that the coefficients are real. The change of variable $x = y - a/3$ eliminates the quadratic term:

$$y^3 + py + q = 0, \quad p = -\frac{a^2}{3} + b, \quad q = \frac{2}{27}a^3 - \frac{ab}{3} + c.$$

Then Vieta's substitution $y = w - p/(3w)$ yields

$$w^3 - \frac{p^3}{27w^3} + q = 0$$

and hence a quadratic equation in w^3 : $(w^3)^2 + qw^3 - p^3/27 = 0$. Hence

$$w^3 = -\frac{q}{2} \pm \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}. \quad (26.5)$$

For either choice of sign, the three cube roots for w yield the roots of the original cubic, on transforming back from w to y to x .

Are these formulae for solving a cubic numerically stable? Direct search provides an easy way to investigate. The variables are the three coefficients a, b, c and for the objective function we take an approximation to the relative error of the computed roots $\hat{z} \in \mathbb{R}^3$. We compute the “exact” roots z using MATLAB’s `roots` function (which uses the QR algorithm to compute the eigenvalues of the companion matrix²⁰) and then our relative error measure is $\min_{\Pi} \|z - \Pi \hat{z}\|_{\infty} / \|z\|_{\infty}$, where we minimize over all six permutations Π .

First, we arbitrarily take the “+” square root in (26.5). With almost any starting vector of coefficients, the MDS maximizer rapidly identifies coefficients for which the computed roots are very inaccurate. For example, starting with $[1, 1, 1]^T$ we are led to the vector

$$[a \ b \ c]^T = [1.732 \ 1 \ 1.2704]^T,$$

for which the computed and “exact” roots are

$$\hat{z} = \begin{bmatrix} -1.5999\text{e}+0 \\ -6.6066\text{e}-2 - 8.8557\text{e}-1i \\ -6.6066\text{e}-2 + 8.8557\text{e}-1i \end{bmatrix}, \quad z = \begin{bmatrix} -1.6026\text{e}+0 \\ -6.4678\text{e}-2 + 8.8798\text{e}-1i \\ -6.4678\text{e}-2 - 8.8798\text{e}-1i \end{bmatrix}.$$

When the cubic is evaluated at the computed roots $\hat{z}$ the results are of order 10^{-2} , whereas they are of order 10^{-15} for z . Since the roots are well separated the problem of computing them is not ill conditioned, so we can be sure that z is an accurate approximation to the exact roots. The conclusion is that the formulae, as programmed, are numerically unstable.

Recalling the discussion for a quadratic equation in §1.8, a likely reason for the observed instability is cancellation in (26.5). Instead of always taking the “+” sign, we therefore take

$$w^3 = -\frac{q}{2} - \text{sign}(q)\sqrt{\frac{q^2}{4} + \frac{p^3}{27}}. \quad (26.6)$$

When the argument of the square root is nonnegative, this formula suffers no cancellation in the subtraction; the same is true when the argument of the square root is negative, because then the square root is pure imaginary. With the use of (26.6), we were unable to locate instability using the objective function described above. However, an alternative objective function can be derived as follows. It is reasonable to ask that the computed roots be the exact roots of a slightly perturbed cubic. Thus each computed root $\hat{z}_i$ should be a root of

$$(1 + \Delta d)x^3 + (a + \Delta a)x^2 + (b + \Delta b)x + c + \Delta c = 0,$$

where $\max(|\Delta a|, |\Delta b|, |\Delta c|, |\Delta d|) / \max(|a|, |b|, |c|, 1)$ is of the order of the unit roundoff. Notice that we are allowing the leading coefficient of unity to be perturbed. Denoting the unperturbed cubic by f , we find that this condition implies

²⁰Edelman and Murakami [383, 1995] and Toh and Trefethen [1144, 1994] analyse the stability of this method of finding polynomial roots; the method is stable.

that

$$\max_i \frac{|f(\hat{z}_i)|}{\max(|a|, |b|, |c|, 1) \sum_{j=0}^3 |\hat{z}_i^j|} \quad (26.7)$$

is of order u . We therefore take the quantity in (26.7) as the function to be maximized. On applying the MDS maximizer with starting vector $[1, 1, 1]^T$, we obtain after 10 iterations an objective function value of 1.2×10^{-11} . The cubic coefficient vector is (to three significant figures)

$$[a \quad b \quad c]^T = [-5.89 \times 10^2 \quad 3.15 \times 10^2 \quad -1.36 \times 10^1],$$

and the computed roots are (ignoring tiny, spurious imaginary parts)

$$\hat{z}^T = [4.75 \times 10^{-2} \quad 4.87 \times 10^{-1} \quad 5.89 \times 10^2].$$

The value of the objective function corresponding to the “exact” roots (computed as described above) is of order 10^{-16} (and the value of the previous relative error objective function for the computed roots is of order 10^{-14}).

The conclusion is that even using (26.6) the formulae for the roots are numerically unstable. However, further theoretical analysis is needed to understand the stability fully; see Problem 26.3.

26.4. Interval Analysis

Interval analysis has been an area of extensive research since the 1960s, and it had been considered earlier by Turing and Wilkinson in the 1940s [1243, 1980, p. 104]. As the name suggests, the idea is to perform arithmetic on intervals $[a, b]$ ($b \geq a$). The aim is to provide as the solution to a problem an interval in which the desired result is guaranteed to lie, which may be particularly appropriate if the initial data is uncertain (and hence can be thought of as an interval).

For the elementary operations, arithmetic on intervals is defined by

$$[a, b] \text{ op } [c, d] = \{x \text{ op } y : x \in [a, b], y \in [c, d]\},$$

and the results are given directly by the formulae

$$\begin{aligned} [a, b] + [c, d] &= [a + c, b + d], \\ [a, b] - [c, d] &= [a - d, b - c], \\ [a, b] * [c, d] &= [\min(ac, ab, bc, bd), \max(ac, ab, bc, bd)], \\ [a, b]/[c, d] &= [a, b] * [1/d, 1/c], \quad 0 \notin [c, d]. \end{aligned}$$

We will use the notation $[x]$ for an interval $[x_1, x_2]$ and we define $\text{width}(x) := x_2 - x_1$.

In floating point arithmetic, an interval containing $fl([x] \text{ op } [y])$ is obtained by rounding computations on the left endpoint to $-\infty$ and those on the right endpoint to $+\infty$ (both these rounding modes are supported in IEEE arithmetic).

The success of an interval analysis depends on whether an answer is produced at all (an interval algorithm will break down with division by zero if it attempts

to divide by an interval containing zero), and on the width of the interval answer. A one-line program that prints “ $[-\infty, \infty]$ ” would be correct, robust, and fast, but useless. Interval analysis is controversial because, in general, narrow intervals cannot be guaranteed. One reason is that when dependencies occur in a calculation, in the sense that a variable appears more than once, final interval lengths can be pessimistic. For example, if $[x] = [1, 2]$ then

$$[x] - [x] = [-1, 1], \quad [x]/[x] = [1/2, 2],$$

whereas the optimal intervals for these calculations are, respectively, $[0, 0]$ and $[1, 1]$. These calculations can be interpreted as saying that there is no additive or multiplicative inverse in interval arithmetic. Another source of overestimation is the wrapping effect: the result of an interval computation in 2 or more dimensions is a rectangular box with sides parallel to the coordinate axes, yet it may enclose an image set oriented at an angle to the axes and of much smaller area than the box.

Successful interval computation usually requires algorithms designed for the purpose [1001, 2001]. An example of an algorithm for which interval arithmetic can be ineffective is GEPP, which in general gives very pessimistic error bounds and is unstable in interval arithmetic even for a well-conditioned matrix [893, 1977]. The basic problem is that the interval sizes grow exponentially. For example, in the 2×2 reduction

$$\begin{bmatrix} 1 & [x] \\ 1 & [y] \end{bmatrix} \rightarrow \begin{bmatrix} 1 & [x] \\ 0 & [y] - [x] \end{bmatrix},$$

if $[x] \approx [y]$ then

$$\text{width}([y] - [x]) \approx \text{width}([x] - [x]) = 2 \text{width}([x]).$$

This type of growth is very likely to happen, unlike the superficially similar phenomenon of element growth in standard GEPP. The poor interval bounds are entirely analogous to the pessimistic results returned by early forward error analyses of GEPP (see §9.13). Nickel [893, 1977] states that “The interval Gauss elimination method is one of the most unfavourable cases of interval computation . . . Nearly all other numerical methods give much better results if transformed to interval methods”. Interval GE is effective, however, for certain special classes of matrices, such as M -matrices.

As already mentioned, there is a large body of literature on interval arithmetic, though, as Skeel notes (in an article that advocates interval arithmetic), “elaborate formalisms and abstruse notation make much of the literature impenetrable to all but the most determined outsiders” [1043, 1989]. Despite the inscrutable nature of much of the interval literature, interval arithmetic is becoming more popular, partly because interval arithmetic software is now more widely available (see below). In fact, interval arithmetic is now quite widely used in science and engineering, for example in computational geometry [751, 2001]. A particular application is in computer-assisted proofs of mathematical results; see Frommer [448, 2001] for an excellent survey and Tucker [1163, 2002] for a recent result concerning the Lorenz attractor.

One of the perceived drawbacks of interval arithmetic has been the difficulty of exploiting high-performance computers when implementing it. Rump [998, 1999]

shows that by using a representation of intervals by the midpoint and radius, interval arithmetic can be implemented entirely using BLAS of all three levels, with major benefits for efficiency.

Good sources of information include various conference proceedings and the journal *Computing*. The earliest book is by Moore [872, 1966], whose later book [873, 1979] is one of the best references on the subject. Nickel [893, 1977] gives an easy-to-read summary of research up to the mid 1970s. A more recent reference is Alefeld and Herzberger [11, 1983]. A short, very readable introduction to interval computations is given by Kearfott [719, 1996].

Yohe [1268, 1979] describes a Fortran 66 package for performing interval arithmetic in which machine-specific features are confined to a few modules. It is designed to work with a precompiler for Fortran called Augment [280, 1979], which allows the user to write programs as though Fortran had extended data types—in this case an `INTERVAL` type. A version of the package that allows arbitrary precision interval arithmetic by incorporating Brent's multiple precision arithmetic package (see §27.9) is described in [1269, 1980]. In [1267, 1979], Yohe describes general principles of implementing nonstandard arithmetic in software, with particular reference to interval arithmetic. Kearfott [718, 1996] presents a Fortran 90 module for interval arithmetic.

Kulisch and Miranker have proposed endowing a computer arithmetic with a super-accurate inner product, that is, a facility to compute an exactly rounded inner product for any two vectors of numbers at the working precision [755, 1981], [756, 1983], [757, 1986]. This idea was implemented in the package ACRITH from IBM, which employs interval arithmetic [139, 1985]. Anomalies in early versions of ACRITH are described by Kahan and LeBlanc [705, 1985] and Jansen and Weidner [673, 1986]. For a readable discussion of interval arithmetic and the super-accurate inner product, see Rall [970, 1991]. Cogent arguments against adopting a super-accurate inner product are given by Demmel [312, 1991].

Fortran and Pascal compilers (Fortran SC and Pascal-XSC) and a C++ class library that support a super-accurate inner product and interval arithmetic have been developed jointly by researchers at IBM and the University of Karlsruhe [140, 1987], [737, 1993], [738, 1992]. A toolbox of routines written in Pascal-XSC for solving basic numerical problems and verifying the results is presented in [544, 1993].

Rump [1000, 1999] has produced a MATLAB toolbox INTLAB that permits interval computations to be carried out elegantly and efficiently in MATLAB. Multiple precision interval arithmetic can be done using the MPFR-based package described in §27.9.

A proposal for a set of interval BLAS is contained in an appendix to the BLAS Technical Forum Standard document (see §C.1) [88, 2002].

Finally, we explain why any attempt to compute highly accurate answers using interval arithmetic of a fixed precision (possibly combined with a super-accurate inner product) cannot always succeed. Suppose we have a sequence of problems to solve, where the output of one is the input to the next: $x_{i+1} = f_i(x_i)$, $i = 1:n$, for smooth functions $f_i : \mathbb{R} \rightarrow \mathbb{R}$. Suppose x_1 is known exactly (interval width zero). Working in finite precision interval arithmetic, we obtain an interval $[x_2 - \alpha_2, x_2 + \beta_2]$ containing x_2 . This is used as input to the computation of f_2 .

Even under the favourable assumption of no rounding errors in the evaluation of f_2 , we obtain an interval answer whose width must be of order

$$|f_2(x_2) - f_2(x_2 \pm \epsilon_2)| \gtrsim |f'_2(x_2)|\epsilon_2, \quad \epsilon_2 = \min(|\alpha_2|, |\beta_2|)$$

(the interval could be much bigger than $f_2(x_2) - f_2(x_2 \pm \epsilon_2)$, depending on the algorithm used to evaluate f_2). In other words, the width of the interval containing x_3 is roughly proportional to the condition number of f_2 . When the output of the f_2 computation is fed into f_3 the interval width is multiplied by f'_3 . The width of the final interval containing x_{n+1} is proportional to the product of the condition numbers of all the functions f_i and if there are enough functions, or if they are conditioned badly enough, the final interval will provide no useful information. The only way to avoid such a failure is to use variable precision arithmetic or to reformulate the problem to escape the product of the condition numbers of the f_i .

26.5. Other Work

In this section we outline other work on automatic error analysis.

First, we describe a linearized forward error expression that underlies several of the methods. Suppose that the function f is evaluated at the data $D = [d_1, \dots, d_n]^T$ by an algorithm g using intermediate variables $d_{n+1}, \dots, d_N$, where for simplicity f and all the d_i are assumed to be real scalars. Each d_k ($k > n$) is assumed to be the result of an elementary operation, $+$, $-$, $*$, $/$ or square root; denote by δ_k the associated rounding error. Then, to first order, the computed solution $\hat{d}_N$ satisfies

$$\hat{d}_N - f(d_1, \dots, d_n) = \sum_{k=n+1}^N \frac{\partial g}{\partial d_k}(D, \delta) \delta_k. \quad (26.8)$$

Langlois [766, 2001] presents a method called CENA which computes the linearization (26.8) of the error and adds this quantity to the computed solution in order to correct it (to first order). The derivatives are computed by automatic differentiation, and the rounding errors δ_k are computed exactly (for addition, subtraction, and multiplication) or closely approximated (for division and square root), using techniques such as that in §4.3 for addition. The method provides a bound for the errors in evaluating the sum in (26.8), making use of running error analysis. For a certain class of “linear algorithms” constructively defined in [766, 2001], which includes the substitution algorithm for solving triangular systems, the relation (26.8) is exact and the CENA method provides a corrected solution with a strict error bound.

In the 1970s, Miller and his co-authors developed methods and software for automatically searching for numerical instability in algebraic processes [851, 1975], [854, 1978], [856, 1980]. In [851, 1975] Miller defines a quantity $\sigma(d)$ that bounds, to first order, the sensitivity of an algorithm to perturbations in the data d and in the intermediate quantities that the algorithm generates; $\sigma(d)$ is essentially the result of applying the triangular inequality to (26.8) and using $|\delta_k| \leq u$. He then

defines the forward stability measure $\rho(d) = \sigma(d)/\kappa(d)$, where $\kappa(d)$ is a condition number for the problem under consideration. The algorithms to be analysed are required to contain no loops or conditional branches and are presented to Miller's Fortran software in a special numeric encoding. The software automatically computes the partial derivatives needed to evaluate $\rho(d)$, and attempts to maximize ρ using the method of alternating directions. Miller gives several examples illustrating the scope of his software; he shows, for example, that it can identify the instability of the classical Gram-Schmidt method for orthogonalizing a set of vectors.

In [854, 1978], [855, 1978] Miller and Spooner extend the work in [851, 1975] in several ways. The algorithm to be analysed is expressed in a Fortran-like language that allows for-loops but not logical tests. The definition of ρ is generalized and a method of computing it is developed that involves solving a generalized eigenvalue problem. The book by Miller and Wrathall [856, 1980] gives a thorough development of the work of [854, 1978], including a description of the graph theory techniques used to compute the partial derivatives, and it provides further examples of the use of the software. The potential of Miller and Spooner's software for exposing numerical instability is clearly demonstrated by the case studies in these references, yet the software has apparently not been widely used. This is probably largely due to the inability of the software to analyse algorithms expressed in Fortran, or any other standard language.

A different approach to algorithm analysis is taken by Larson and Sameh [768, 1978], [769, 1980] and implemented in software by Larson, Pasternak, and Wisniewski [767, 1983]. Here, errors are measured in a relative rather than an absolute sense, and the stability is analysed at *fixed data* instead of attempting to maximize instability over all data; however, the analysis is still linearized.

The idea of applying automatic differentiation to a computational graph to obtain estimates for the forward error in an algorithm is found not only in the references cited above, but also in the automatic differentiation literature; see Rall [969, 1981], Iri [666, 1991], and Kubota [752, 1991], for example.

Hull [646, 1979] discusses the possibility of applying program verification techniques to computations that are subject to rounding errors, with the aim of proving a program "correct". Difficulties include deciding what "correct" means and formulating appropriate assertions. Although he reports some progress, Hull concludes that "there is not a great deal to be gained by trying to apply the techniques of program verification to programs for numerical calculations".

Bliss, Brunet, and Gallopoulos [141, 1992] develop Fortran preprocessor tools for implementing the local relative error approach of Larson and Sameh. Their tools also implement a statistical technique of Brunet and Chatelin [169, 1989], [225, 1990] in which the result of every floating point operation is randomly perturbed and statistical methods are used to measure the effect on the output of the algorithm.

Rowan [995, 1990] develops another way to search for numerical instability. For an algorithm with data d he maximizes $S(d) = e(d)/\kappa(d)$ using a direct search maximizer he has developed called the subplex method (which is based on the Nelder-Mead simplex method). Here, $e(d) = y_{\text{acc}} - \hat{y}$ is an approximation to the forward error in the computed solution $\hat{y}$, where y_{acc} is a more accurate estimate

of the true solution than $\hat{y}$, and the condition number $\kappa(d)$ is estimated using finite difference approximations. The quantity $S(d)$ is a lower bound on the backward error of the algorithm at d . Fortran software given in [995, 1990] implements this “functional stability analysis”. The software takes as input two user-supplied Fortran subprograms; one implements the algorithm to be tested in single precision, and the other provides a more accurate solution, typically by executing the same algorithm in double precision. The examples in [995, 1990] show that Rowan’s software is capable of detecting numerical instability in a wide variety of numerical algorithms.

A technique called “significance arithmetic” was studied by Ashenurst, Metropolis, and others in the 1960s. It involves performing arithmetic on unnormalized numbers in such a way that the number of significant digits in the answers provides estimates of the accuracy of those answers. Significance arithmetic is therefore a form of automatic error analysis. For more details see, for example, Ashenurst and Metropolis [39, 1965], [844, 1977] and Sterbenz [1062, 1974, §7.2]; Sterbenz explains several drawbacks of the technique.

Stoutemyer [1089, 1977] describes the use of the symbolic manipulation package REDUCE to analyse the propagation of data and rounding errors in numerical algorithms.

The CESTAC (permutation-perturbation) method of La Porte and Vignes [170, 1986], [759, 1974], [1196, 1986] takes a statistical approach. It assumes that rounding errors can be modelled by independent random variables and that the global error in a computation can be well approximated by the sum of the local errors. It runs the algorithm in question 2 or 3 times, making a random perturbation of the last bit of the result of each elementary operation, computes the mean M of the final results, and estimates the number of M ’s correct digits using a Student test. This approach is therefore forward error oriented. Chaitin-Chatelin and her co-workers have developed a backward error oriented method called PRECISE. It makes random perturbations in the original data and examines the effect on the residual and the error divided by the residual, producing statistical estimates of various indicators of backward error, forward error, and conditioning. See Brunet [169, 1989], Chatelin and Brunet [225, 1990], and Chaitin-Chatelin and Frayssé [208, 1996]. PRECISE was designed as a tool to explore the behaviour of numerical algorithms as a function of parameters such as mesh size, time step, nonnormality, and nearness to singularity.

Finally, we note that running error analysis (see §3.3) is a form of automatic error analysis and is an attractive alternative to interval arithmetic as a means of computing a posteriori error bounds for almost any numerical algorithm.

26.6. Notes and References

Sections 26.1–26.3.2 and §26.5 are based on Higham [601, 1993].

Another way to solve a cubic is to use Newton’s method to find a real zero, and then to find the other two zeros by solving a quadratic equation. In a detailed investigation, Kahan [693, 1986] finds this iterative method to be preferable to (sophisticated) use of the explicit formulae. Other useful references on the numerical computation of roots of a cubic are Acton [5, 1996, pp. 29–32], Lanczos [765,

1956, Chap. 1], Press, Teukolsky, Vetterling, and Flannery [953, 1992, §5.6], and Uspensky [1175, 1948].

Problems

26.1. Let $A \in \mathbb{R}^{4 \times 4}$ and let $\hat{Q}_{CGS}$ and $\hat{Q}_{MGS}$ be the computed orthogonal QR factors produced by the classical and modified Gram–Schmidt methods, respectively. Use direct search to maximize the functions

$$\begin{aligned} f_1(A) &= \|\hat{Q}_{CGS}^T \hat{Q}_{CGS} - I\|_2, \\ f_2(A) &= \|\hat{Q}_{MGS}^T \hat{Q}_{MGS} - I\|_2, \\ f_3(A) &= f_1(A)/f_2(A). \end{aligned}$$

In order to keep $\kappa_2(A)$ small, try maximizing $f_i(A) - \theta \max(\kappa_2(A) - \mu, 0)$, where θ is a large positive constant and μ is an upper bound on the acceptable condition numbers.

26.2. It is well known that if $A \in \mathbb{R}^{n \times n}$ is nonsingular and $v^T A^{-1} u \neq -1$ then

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u}.$$

This is known as the Sherman–Morrison formula (cf. Problem 13.9). For a history and generalizations see Henderson and Searle [559, 1981]. A natural question is whether this formula provides a stable way to solve a rank-1 perturbed linear system. That is, is the following algorithm stable?

```
% Solve  $Cx := (A + uv^T)x = b$ .
Solve  $Ay = b$  for  $y$ .
Solve  $Az = u$  for  $z$ .
 $x = y - (v^T y)(1 + v^T z)^{-1}z$ 
```

(a) Investigate the stability using direct search. Let both linear systems with coefficient matrix A be solved by GEPP. Take A , u , and v as the data and let the function to be maximized be the normwise backward error $\eta_{C,b}$ in the ∞ -norm.

(b) (RESEARCH PROBLEM) Obtain backward and forward error bounds for the method (for some existing analysis see Yip [1266, 1986]).

26.3. (RESEARCH PROBLEM) Investigate the stability of the formulae of §26.3.3 for computing the roots of a cubic.

26.4. (RESEARCH PROBLEM) Use direct search to try to make progress on some of the research problems in Chapters 9, 10, 11, and 15.

Chapter 27

Software Issues in Floating Point Arithmetic

*The first American Venus probe was lost due to a
program fault caused by the
inadvertent substitution of a statement of the form
DO 3 I = 1.3 for one of the form DO 3 I = 1,3.*

— JIM HORNING, Note on Program Reliability²¹ (1979)

*Numerical subroutines should deliver results that satisfy simple,
useful mathematical laws whenever possible.*

— DONALD E. KNUTH, *The Art of Computer Programming*,
Volume 2, *Seminumerical Algorithms* (1998)

*No method of solving a computational problem is
really available to a user until it is
completely described in an algebraic computing language
and made completely reliable.
Before that, there are indeterminate aspects in any algorithm.*

— GEORGE E. FORSYTHE,
Today's Computational Methods of Linear Algebra (1967)

*The extended precision calculation of pi has substantial application as a
test of the "global integrity" of a supercomputer ...
Such calculations ... are apparently now used routinely
to check supercomputers before they leave the factory.
A large-scale calculation of pi is entirely unforgiving;
it soaks into all parts of the machine and a
single bit awry leaves detectable consequences.*

— J. M. BORWEIN, P. B. BORWEIN, and D. H. BAILEY,
*Ramanujan, Modular Equations, and Approximations to Pi
or How to Compute One Billion Digits of Pi* (1989)

²¹Quoted, with further details, in Tropp [1159, 1984].

In this chapter we discuss some miscellaneous aspects of floating point arithmetic that have an impact on software development.

27.1. Exploiting IEEE Arithmetic

IEEE standard 754 and 854 arithmetic can be exploited in several ways in numerical software, provided that proper access to the arithmetic's features is available in the programming environment. Unfortunately, although virtually all modern floating point processors conform to the IEEE standards, language standards and compilers generally provide poor support (exceptions include the Standard Apple Numerics Environment (SANE) [260, 1988], Apple's PowerPC numerics environment [261, 1994], and Sun's SPARCstation compilers [1101, 1992], [1100, 1992]). We give four examples to show the benefits of IEEE arithmetic on software.

Suppose we wish to evaluate the dual of the vector p -norm ($1 \leq p \leq \infty$), that is, the q -norm, where $p^{-1} + q^{-1} = 1$. In MATLAB notation we simply write `norm(x,1/(1-1/p))`, and the extreme cases $p = 1$ and ∞ correctly yield $q = 1/(1 - 1/p) = \infty$ and 1 in IEEE arithmetic. (Note that the formula $q = p/(p - 1)$ would not work, because `inf/inf` evaluates to a NaN.)

The following code finds the biggest element in an array $a(1:n)$, and elegantly solves the problem of choosing a "sufficiently negative number" with which to initialize the variable `max`:

```
max = -inf
for j = 1:n
    if a_j > max
        max = a_j
    end
end
```

Any unknown or missing elements of the array could be assigned NaNs (`a(j) = NaN`) and the code would (presumably) still work, since a NaN compares as unordered with everything.

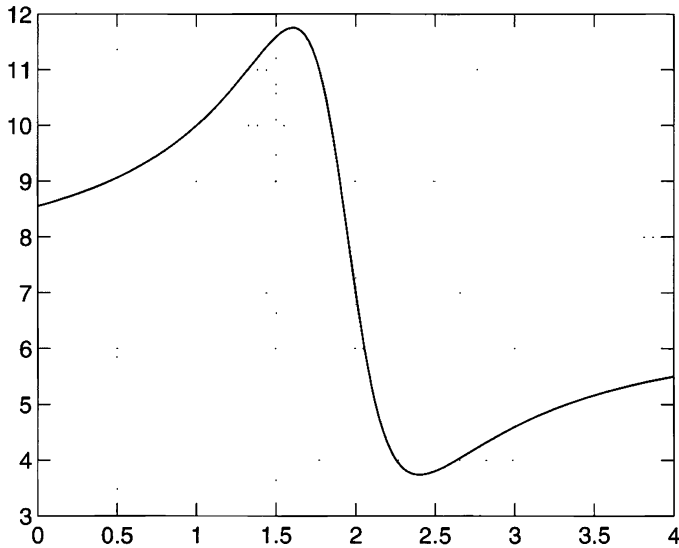
Consider the following continued fraction [691, 1981]:

$$r(x) = 7 - \frac{3}{x - 2 - \frac{1}{x - 7 + \frac{10}{x - 2 - \frac{2}{x - 3}}}},$$

which is plotted over the range $[0, 4]$ in Figure 27.1. Another way to write the rational function $r(x)$ is in the form

$$r(x) = \frac{(((7x - 101)x + 540)x - 1204)x + 958}{(((x - 14)x + 72)x - 151)x + 112},$$

in which the polynomials in the numerator and denominator are written in the form in which they would be evaluated by Horner's rule. Examining these two

Figure 27.1. *Rational function r .*

representations of r , we see that the continued fraction requires fewer arithmetic operations to evaluate (assuming it is evaluated in the obvious “bottom up” fashion) but it incurs division by zero at the four points $x = 1:4$, even though r is well behaved at these points. However, in IEEE arithmetic r evaluates correctly at these points because of the rules of infinity arithmetic. For $x = 10^{77}$, the rational form suffers overflow, while the continued fraction evaluates correctly to 7.0; indeed, in IEEE arithmetic the continued fraction is immune to overflow. Figure 27.2 shows the relative errors made in evaluating r in double precision on an equally spaced grid of points on the range $[0, 4]$ (many of the errors for the continued fraction are zero, hence the gaps in the error curve); clearly, the continued fraction produces the more accurate function values. The conclusion is that in IEEE arithmetic the continued fraction representation is the preferred one for evaluating r .

The exception handling provided by IEEE arithmetic can be used to simplify and speed up certain types of software, provided that the programming environment properly supports these facilities. Recall that the exceptional operations underflow, overflow, divide by zero, invalid operation, and inexact deliver a result, set a flag, and continue. The flags (one for each type of exception) are “sticky”, remaining set until explicitly cleared, and implementations of the IEEE standard are required to provide a means to read and write the flags. Unfortunately, compiler writers have been slow to make exception handling, and some of IEEE arithmetic’s other unusual features, available to the programmer. Kahan has commented that “the fault lies partly with the IEEE standard, which neglected to spell out standard names for the exceptions, their flags, or even ∞ .”

As an example of how exception handling can be exploited, consider the use of the LAPACK norm estimator (Algorithm 15.4) to estimate $\|A^{-1}\|_1$. The algorithm

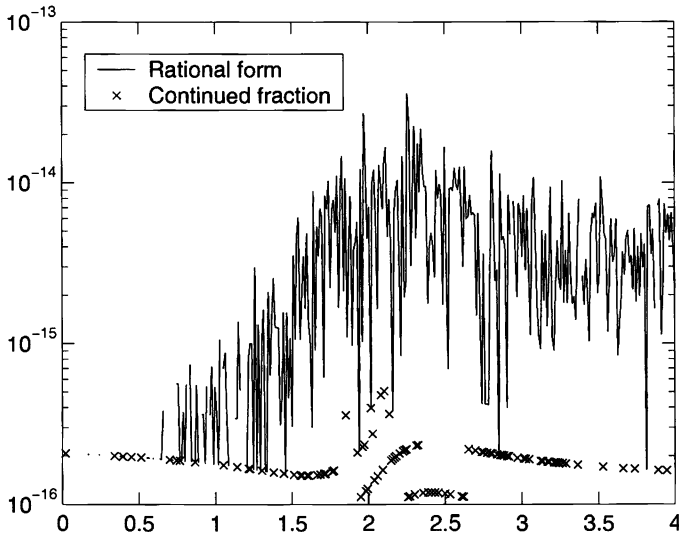


Figure 27.2. Error in evaluating rational function r .

requires the solution of linear systems of the form $Ax = b$ and $A^T y = c$, which is typically done with the aid of an LU factorization $PA = LU$. Solution of the resulting triangular systems is prone to both overflow and division by zero. In LAPACK, triangular systems are solved in this context not with the level-2 Basic Linear Algebra Subprograms (BLAS) routine `xTRSV` but with a routine `xLATRS` that contains elaborate tests and scalings to avoid overflow and division by zero [22, 1991]. In an IEEE arithmetic environment a simpler and potentially faster approach is possible: solve the triangular systems with `xTRSV` and after computing each solution check the flags to see whether any exceptions occurred. Provided that some tests are added to take into account overflow due solely to $\|A\|_1$ being tiny, the occurrence of an exception allows the conclusion that A is singular to working precision; see Demmel and Li [331, 1994] for the details.

For the condition estimator that uses exception handling to be uniformly faster than the code with scaling it is necessary that arithmetic with NaNs, infinities, and subnormal numbers be performed at the same speed as conventional arithmetic. While this requirement is often satisfied, there are machines for which arithmetic on NaNs, infinities, and subnormal numbers is between one and three orders of magnitude slower than conventional arithmetic [331, 1994, Table 2]. Demmel and Li compared the LAPACK condition estimation routines `xxxCON` with modified versions containing exception handling and found the latter to be up to 6 times faster and up to 13 times slower, depending on the machine and the matrix.

Appendices to the IEEE standards 754 and 854 recommend 10 auxiliary functions and predicates to support portability of programs across different IEEE systems. These include `nextafter( $x, y$ )`, which returns the next floating point number to x in the direction of y , and `scalb( $x, n$ )`, which returns $x \times \beta^n$ without explicitly computing β^n , where n is an integer and β the base. Not all implementations of IEEE arithmetic support these functions. In MATLAB, `scalb( $x, n$ )` for $\beta = 2$ is

Table 27.1. *Results from Cholesky factorization.*

Bits	u	Computer	Displacement
128	1e-29	Cray 2	.447440341
64	1e-17	IBM 3090	.44744034 <u>4</u>
64	1e-16	Convex 220	.44744033 <u>9</u>
64	1e-16	IRIS	.44744033 <u>9</u>
64	1e-15	Cray 2	.44744030 <u>3</u>
64	1e-15	Cray Y-MP	.44743 <u>6106</u>

available as `pow2(x,n)`. Portable C versions of six of the functions are presented by Cody and Coonen [254, 1993].

27.2. Subtleties of Floating Point Arithmetic

On some computers of the past, the difference $x - y$ of two machine numbers could evaluate to zero even when $x \neq y$, because of underflow. This made a test such as

```

if  $x \neq y$ 
   $f = f/(x - y)$ 
end

```

unreliable. However, in a system that supports gradual underflow (such as IEEE arithmetic) $x - y$ always evaluates as nonzero when $x \neq y$ (see Problem 2.19). On several models of Cray computer (Cray 1, 2, X-MP, Y-MP, and C90) this code could fail for another reason: they compute $f/(x - y)$ as $f * (1/(x - y))$ and $1/(x - y)$ could overflow.

It is a general principle that one should not test two floating point numbers for equality, but rather use a test such as $|x - y| \leq \text{tol}$ (there are exceptions, such as Algorithm 2 in §1.14.1). Of course, skill may be required to choose an appropriate value for `tol`. A test of this form would correctly avoid the division in the example above when underflow occurs.

27.3. Cray Peculiarities

Carter [206, 1991] describes how a Cray computer at the NASA Ames Laboratories produced puzzling results that were eventually traced to properties of its floating point arithmetic. Carter used Cholesky factorization on a Cray Y-MP to solve a sparse symmetric positive definite system of order 16146 resulting from a finite element model of the National Aerospace Plane. The results obtained for several computers are shown in Table 27.1, where “Displacement” denotes the largest component of the solution and incorrect digits are set in boldface and underlined. Since the coefficient matrix has a condition number of order 10^{11} , the errors in the displacements are consistent with the error bounds for Cholesky factorization.

Given that both the Cray 2 and the Cray Y-MP lack guard digits, it is not surprising that they give a less accurate answer than the other machines with a similar unit roundoff. What is surprising is that, even though both machines use 64-bit binary arithmetic with a 48-bit significand, the result from the Cray Y-MP is much less accurate than that from the Cray 2. The explanation (diagnosed over the telephone to Carter by Kahan, as he scratched on the back of an envelope) lies in the fact that the Cray 2 implementation of subtraction without a guard digit produces more nearly unbiased results (average error zero), while the Cray Y-MP implementation produces biased results, causing $fl(x - y)$ to be too big if $x > y > 0$. Since the inner loop of the Cholesky algorithm contains an operation of the form $a_{ii} = a_{ii} - a_{ij}^2$, the errors in the diagonal elements reinforce as the factorization proceeds on the Cray Y-MP, producing a Cholesky factor with a diagonal that is systematically too large. Similarly, since Carter's matrix has off-diagonal elements that are mostly negative or zero, the Cray Y-MP produces a Cholesky factor with off-diagonal elements that are systematically too large and negative. For the large value of n used by Carter, these two effects in concert are large enough to cause the loss of a further two digits in the answer.

An inexpensive way to improve the stability of the computed solution is to use iterative refinement in fixed precision (see Chapter 12). This was tried by Carter. He found that after one step of refinement the Cray Y-MP solution was almost as accurate as that from the Cray 2 without refinement.

27.4. Compilers

Some of the pitfalls a compiler writer should avoid when implementing floating point arithmetic are discussed by Farnum [404, 1988]. The theme of his paper is that programmers must be able to predict the floating point operations that will be performed when their codes are compiled; this may require, for example, that the compiler writer forgoes certain "optimizations". In particular, compilers should not change groupings specified by parentheses. For example, the two expressions

$$\begin{aligned} &(1.0\text{E-}30 + 1.0\text{E+}30) - 1.0\text{E+}30 \\ &1.0\text{E-}30 + (1.0\text{E+}30 - 1.0\text{E+}30) \end{aligned}$$

will produce different answers on many machines. Farnum explains that

Compiler texts and classes rarely address the peculiar problems of floating-point computation, and research literature on the topic is generally confined to journals read by numerical analysts, not compiler writers. Many production-quality compilers that are excellent in other respects make basic mistakes in their compilation of floating-point, resulting in programs that produce patently absurd results, or worse, reasonable but inaccurate results.

27.5. Determining Properties of Floating Point Arithmetic

Clever algorithms have been devised that attempt to reveal the properties and parameters of a floating point arithmetic. The first algorithms were published by

Malcolm [809, 1972] (see Problem 27.3). These have been improved and extended by other authors. Kahan's **paranoia** code carries out detailed investigations of a computer's floating point arithmetic; there are Basic, C, Modula, Pascal, and Fortran versions, all available from netlib. In addition to computing the arithmetic parameters, **paranoia** tries to determine how well the arithmetic has been implemented (so it can be regarded as a program to test a floating point arithmetic—see the next section). Karpinski [715, 1985] gives an introduction to **paranoia** for the layman, but the best documentation for the code is the output it produces.

Cody has a Fortran routine **machar** for determining 13 parameters associated with a floating point arithmetic system [250, 1988] (an earlier version was published in the book by Cody and Waite [256, 1980]). Cody notes some strange behaviour of compilers and says that he was unable to make his code run correctly on the Cyber 205. A routine based on **machar** is given in *Numerical Recipes* [953, 1992, §20.1].

LAPACK contains a routine **xLAMCH** for determining machine parameters. Because of the difficulty of handling the existing variety of machine arithmetics it is over 850 lines long (including comments and the subprograms it calls). Programs such as **machar**, **paranoia**, and **xLAMCH** are difficult to write; for example, **xLAMCH** tries to determine the overflow threshold without invoking overflow. The Fortran version of **paranoia** handles overflow by requiring the user to restart the program, after which checkpointing information previously written to a file is read to determine how to continue.

Fortran 95 contains environmental inquiry functions, which for **REAL** arguments return the precision (**PRECISION**²²), exponent range (**RANGE**), machine epsilon (**EPSILON**), largest number (**HUGE**), and so on, corresponding to that argument [843, 1999]. The values of these parameters are chosen by the implementor to best fit a model of the machine arithmetic due to Brown [167, 1981] (see §27.7.4). Fortran 95 also contains functions for manipulating floating point numbers: for example, to set or return the exponent or fractional part (**EXPONENT**, **FRACTION**, **SET_EXPONENT**) and to return the spacing of the numbers having the same exponent as the argument (**SPACING**).

27.6. Testing a Floating Point Arithmetic

How can we test whether a particular implementation of floating point arithmetic is correct? It is impractical to test a floating point operation with all possible arguments because there are so many of them—about 10^{19} in IEEE double precision arithmetic, for example (see Problem 2.1)! Special techniques are therefore needed that test with a limited number of arguments.

A package FPV [878, 1986] from NAG Ltd. attempts to verify experimentally that a floating point arithmetic has been correctly implemented according to its specification. FPV must be supplied by the user with the arithmetic parameters β , t , $e_{\min}$, $e_{\max}$, and with the rounding rule; it then attempts to verify that the arithmetic conforms to these parameters by probing for errors. The tests cover the basic arithmetic operations (including square root) but not the elementary

²²Intrinsic function names are shown in parentheses.

functions. FPV, which is available in both Fortran 77 and Pascal versions, adopts an approach used in an earlier program FPTST [1025, 1981], [1026, 1984] by Schryer of AT&T Bell Laboratories: it tests arithmetic operations with a limited set of operands that are regarded as being most likely to reveal errors. This approach is based on the premise that errors are most likely to occur as “edge effects”, induced by operands close to a discontinuity or boundary of the floating point number system (such as a power of the base β).

FPV and FPTST have both revealed implementation errors in floating point arithmetics on commercial computers. Errors detected include multiplication and negation producing unnormalized results, $x * y$ differing from $(-x) * (-y)$, and the product of two numbers greater than 1 being less than 1! Wichmann [1221, 1992] suggests that it was probably revelations such as these that led the UK Ministry of Defence to issue an interim standard prohibiting the use of floating point arithmetic in safety-critical systems.

Verdonk, Cuyt, and Verschaeren [1193, 2001], [1194, 2001] present a tool for testing compliance of a base 2 floating point arithmetic with the principles of the IEEE standard. The tool comprises a driver program and a large set of test vectors and it works for any specified precision and exponent range.

A Fortran package ELEFUNT by Cody and Waite contains programs to test the elementary functions [249, 1982], [256, 1980]; the package is available from netlib. It checks identities such as $\cos(x) = \cos(x/3)(4\cos^2(x/3) - 1)$, taking care to choose arguments x for which the errors in evaluating the identities are negligible. A package CELEFUNT serves the same purpose for the complex elementary functions [252, 1993]. Tang [1122, 1990] develops table-driven techniques for testing the functions exp and log.

27.7. Portability

Software is portable if it can be made to run on different systems with just a few straightforward changes (ideally, we would like to have to make no changes, but this level of portability is often impossible to achieve). Sometimes the word “transportable” is used to describe software that requires certain well-defined changes to run on different machines. For example, Demmel, Dongarra, and Kahan [322, 1992] describe LAPACK as “a transportable way to achieve high efficiency on diverse modern machines”, noting that to achieve high efficiency the BLAS need to be optimized for each machine. A good example of a portable collection of Fortran codes is LINPACK. It contains no machine-dependent constants and uses the PFORT subset of Fortran 77; it uses the level-1 BLAS, so, ideally, optimized BLAS would be used on each machine.

27.7.1. Arithmetic Parameters

Differences between floating point arithmetics cause portability problems. First, what is meant by **REAL** and **DOUBLE PRECISION** in Fortran varies greatly between machines, as shown by the following table:

	u (REAL)	u (DOUBLE PRECISION)	u (EXTENDED)
IBM	$\approx 10^{-7}$	$\approx 10^{-16}$	$\approx 10^{-31}$
Cray	$\approx 10^{-15}$	$\approx 10^{-28}$	

Second, for a given level of precision, u , the various arithmetic parameters such as base, unit roundoff, largest and smallest machine numbers, and the type of rounding, can all vary. Some possible ways for a code to acquire these parameters are as follows.

(1) The parameters are evaluated and embedded in the program in **PARAMETER** and **DATA** statements. This is conceptually the simplest approach, but it is not portable.

(2) Functions are provided that return the machine parameters. Bell Laboratories' PORT library [441, 1978] has three functions:

R1MACH (k)	$1 \leq k \leq 5$	real parameters,
D1MACH (k)	$1 \leq k \leq 5$	double precision parameters,
I1MACH (k)	$1 \leq k \leq 16$	integer parameters.

R1MACH returns the underflow and overflow thresholds, the smallest and largest relative spacing (β^{-t} , β^{1-t} respectively), and $\log_{10} \beta$, where β is the base and t the number of digits in the significand. **I1MACH** returns standard input, output, and error units and further floating point information, such as β , t , and the minimum and maximum exponents $e_{\min}$ and $e_{\max}$. These functions do not carry out any computation; they contain **DATA** statements with the parameters for most common machines in comment lines, and the relevant statements have to be uncommented for a particular machine. This approach is more sensible for a library than the previous one, because only these three routines have to be edited, instead of every routine in the library.

The NAG Library takes a similar approach to PORT. Each of the 18 routines in its X02 chapter returns a different machine constant. For example, **X02AJF** returns the unit roundoff and **X02ALF** returns the largest positive floating point number. These values are determined once and for all when the NAG library is implemented on a particular platform using a routine similar to **paranoia** and **machar**, and they are hard coded into the Chapter X02 routines.

(3) The information is computed at run-time, using algorithms such as those described in §27.5.

On a parallel computer system in which not all processors share precisely the same floating point arithmetic (a possible example is a network of workstations from different manufacturers) the arithmetic parameters can be different for different processors. For a detailed discussion of the problems that can arise in such heterogeneous computing environments and their resolution see [136, 1997].

27.7.2. 2×2 Problems in LAPACK

LAPACK contains 10 or so auxiliary routines for solving linear equation and eigenvalue problems of dimension 2. For example, **SLASV2** computes the singular value decomposition of a 2×2 triangular matrix (see [45, 1993] for a discussion of this routine), **SLAEV2** computes the eigenvalues and eigenvectors of a 2×2 symmetric matrix, and **SLASY2** solves a Sylvester equation $AX - XB = C$ where A and B

have order 1 or 2. These routines need to be reliable and robust because they are called repeatedly by higher-level solvers. Because LAPACK makes minimal assumptions about the underlying floating point arithmetic, the 2×2 routines are nontrivial to write and are surprisingly long: the counts of executable statements are 118 for SLASV2, 75 for SLAEV2, and 235 for SLASY2. If the availability of extended precision arithmetic (possibly simulated using the working precision) or IEEE arithmetic can be assumed, the codes can be simplified significantly. Complicated and less efficient code for these 2×2 problem solvers is a price to be paid for portability across a wide range of floating point arithmetics.

27.7.3. Numerical Constants

How should numerical constants be specified, e.g., for a Gauss quadrature rule or a Runge–Kutta method? Some compilers limit the number of significant digits that can be specified in a DATA statement or an assignment statement. One possibility is to use rational approximations, but again the number of digits required to cater for all precisions may be too large for some compilers. Another scheme is to store a carefully designed table of leading and trailing parts of constants. A constant such as π is best computed via a statement such as `pi = 4.0*atan(1.0)`, if you trust your language's mathematics library.

27.7.4. Models of Floating Point Arithmetic

The model (2.4) of floating point arithmetic contains only one parameter, the unit roundoff. Other more detailed models of floating point arithmetic have been proposed, with the intention of promoting the development of portable mathematical software. A program developed under a model and proved correct for that model has the attraction that it necessarily performs correctly on any machine for which the model is valid.

The first detailed model was proposed by van Wijngaarden [1185, 1966]; it contained 32 axioms and was unsuccessful because it was “mathematically intimidating” and did not cover the CDC 600, an important high-performance computer of that time [697, 1991]. A more recent model is that of Brown [167, 1981]. Brown's model contains four parameters: the base β , the precision t , and the minimum and maximum exponents $e_{\min}$ and $e_{\max}$, together with a number of axioms describing the behaviour of the floating point arithmetic. Aberrant arithmetics are accommodated in the model by penalizing their parameters (for example, reducing t by 1 from its actual machine value if there is no guard digit). Brown builds a substantial theory on the model and gives an example program to compute the 2-norm of a vector, which is accompanied by a correctness proof for the model.

Brown's model is intended to cater to diverse computer arithmetics. In fact, “any behaviour permitted by the axioms of the model is actually exhibited by at least one commercially important computer” [167, 1981, p. 457]. This broadness is a weakness. It means that there are important properties shared by many (but not all) machines that the model cannot reflect, such as the exactness of $fl(x - y)$ described in Theorem 2.4. As Kahan[691, 1981] notes, “Programming for the [IEEE] standard is like programming for one of a family of well-known machines, whereas programming for a model is like programming for a horde of obscure and

ill-understood machines all at once.” Although Brown’s model was used in the Ada language to provide a detailed specification of floating point arithmetic, the model is still somewhat controversial.

Wichmann [1220, 1989] gives a formal specification of floating point arithmetic in the VDM specification language based on a modification of Brown’s model.

The most recent model is the Language Independent Arithmetic (LIA-1) [654, 1994]. The LIA-1 specifies floating point arithmetic far more tightly than Brown’s model. It, too, is controversial; an explanation of flaws in an earlier version (known then as the Language Compatible Arithmetic Standard) was published by Kahan [697, 1991].

For a more detailed critique of models of floating point arithmetic see Priest [955, 1992].

27.8. Avoiding Underflow and Overflow

The radius of the observable universe is estimated as 10^{26} meters. The radius of a proton is about 8×10^{-16} meters. The range $10^{\pm 308}$ provided by IEEE double precision arithmetic might therefore seem more than adequate to avoid underflow and overflow in practical computations. Unfortunately, it is not: intermediate quantities occurring in the computation of representable quantities can overflow or underflow. A simple indication of the problem is the fact that for about half of all machine numbers x , x^2 either underflows or overflows.

A classic example showing how care is needed to avoid underflow and overflow is the evaluation of a vector 2-norm, $\|x\|_2 = (\sum_i |x_i|^2)^{1/2}$. Overflow is avoided by the following algorithm:

```

t =  $\|x\|_\infty$ 
s = 0
for i = 1: n
    s = s + (x(i)/t)2
end
 $\|x\|_2 = t\sqrt{s}$ 

```

The trouble with this algorithm is that it requires n divisions and two passes over the data (the first to evaluate $\|x\|_\infty$), so it is slow. (It also involves more rounding errors than the unscaled evaluation, which could be obviated by scaling by a power of the machine base.) Blue [143, 1978] develops a one-pass algorithm that avoids both overflow and underflow and requires between 0 and n divisions, depending on the data, and he gives an error analysis to show that the algorithm produces an accurate answer. The idea behind the algorithm is simple. The sum of squares is collected in three accumulators, one each for small, medium, and large numbers. After the summation, various logical tests are used to decide how to combine the accumulators to obtain the final answer.

The original, portable implementation of the level-1 BLAS routine `xNRM2` (listed in [341, 1979]) was written by C. L. Lawson in 1978 and, according to Lawson, Hanson, Kincaid, and Krogh [774, 1979], makes use of Blue’s ideas. However, `xNRM2` is not a direct coding of Blue’s algorithm and is extremely difficult to understand—a classic piece of “Fortran spaghetti”! Nevertheless, the routine clearly works and

is reliable, because it has been very widely used without any reported problems. Lawson's version of `xNRM2` has now been superseded in the LAPACK distribution by a concise and elegant version by S. J. Hammarling, which implements a one-pass algorithm; see Problem 27.5.

A special case of the vector 2-norm is the Pythagorean sum, $\sqrt{a^2 + b^2}$, which occurs in many numerical computations. One way to compute it is by a beautiful cubically convergent, square-root-free iteration devised by Moler and Morrisson [867, 1983]; see Problem 27.6. LAPACK has a routine `xLAPY2` that computes $\sqrt{x_1^2 + x_2^2}$ and another routine `xLAPY3` that computes $\sqrt{x_1^2 + x_2^2 + x_3^2}$; both routines avoid overflow by using the algorithm listed at the start of this section with $n = 2$ or 3 .

Pythagorean sums arise in computing the 1-norm of a complex vector:

$$\|x\|_1 = \sum_{i=1}^n \sqrt{(\operatorname{Re} x_i)^2 + (\operatorname{Im} x_i)^2}, \quad x \in \mathbb{C}^n.$$

The level-1 BLAS routine `xCASUM` does not compute the 1-norm, but the more easily computed quantity

$$\sum_{i=1}^n (|\operatorname{Re} x_i| + |\operatorname{Im} x_i|).$$

The reason given by the BLAS developers is that it was assumed that users would expect `xCASUM` to compute a less expensive measure of size than the 2-norm [774, 1979]. This reasoning is sound, but many users have been confused not to receive the true 1-norm. In the BLAS Technical Forum Standard both the true 1- and ∞ -norms and the “real” versions employing $|\operatorname{Re} x_i| + |\operatorname{Im} x_i|$ are included [88, 2002], [137, 2001]. See Problem 6.16 for more on this pseudo 1-norm.

Another example where underflow and overflow can create problems is in complex division. If we use the formula

$$\frac{a + ib}{c + id} = \frac{ac + bd}{c^2 + d^2} + i \frac{bc - ad}{c^2 + d^2},$$

then overflow will occur whenever c or d exceeds the square root of the overflow threshold, even if the quotient $(a + ib)/(c + id)$ does not overflow. Certain Cray and NEC machines implement complex division in this way [322, 1992]; on these machines the exponent range is effectively halved from the point of view of the writer of robust software. Smith [1052, 1962] suggests a simple way to avoid overflow: if $|c| \geq |d|$ use the following formula, obtained by multiplying the numerators and denominators by c^{-1} ,

$$\frac{a + ib}{c + id} = \frac{a + b(dc^{-1})}{c + d(dc^{-1})} + i \frac{b - a(dc^{-1})}{c + d(dc^{-1})}, \quad (27.1)$$

and if $|d| \geq |c|$ use the analogous formula involving d^{-1} . Stewart [1071, 1985] points out that underflow is still possible in these formulae and suggests a more complicated algorithm that avoids both underflow and overflow.

Demmel [308, 1984] discusses in detail the effects of underflow on numerical software and analyses different ways of implementing underflows, the main two of which are flush to zero and gradual underflow (as used in IEEE standard arithmetic). Cody [250, 1988] makes the following observations:

The variety in the treatment of underflow is wondrous. There exist machines in which the exponent wraps around and underflow results in a *large* floating-point number, perhaps with an accompanying sign change. There are also machines on which underflow is replaced by the smallest in magnitude nonzero number, machines on which small positive numbers vanish when added to themselves, and machines on which such small numbers vanish when multiplied by 1.0. Finally, on IEEE-style machines, underflow may be graceful.

27.9. Multiple Precision Arithmetic

If we wish to work at a precision beyond that provided by a Fortran compiler's `DOUBLE PRECISION` (or the quadruple precision supported as an extension by some compilers) there are two basic approaches. The first is to implement higher precision arithmetic using arrays of double precision variables, where the *unevaluated* sum of the elements of an array represents a high-precision variable. Arithmetic can be implemented in this system using techniques such as those of Dekker [302, 1971] (cf. compensated summation, described in §4.3), as refined by Linnainmaa [791, 1981] and Priest [954, 1991], [955, 1992]. These techniques require a “well-behaved” floating point arithmetic, such as IEEE standard arithmetic, and are not straightforward to apply.

A particular case of this first approach is the “double-double” format, in which two double precision variables are used to represent a variable of twice the precision, the first representing the leading digits and the second the trailing digits. A 16-byte double-double format can be implemented portably in IEEE arithmetic [785, 2000], [1038, 1997]. The execution of double-double arithmetic is speeded up by the use of a fused multiply-add (FMA) operation (see §2.6). Quad-double arithmetic, based on four double precision numbers, has also been developed [572, 2001].

The second approach to achieving high precision is to use arrays to represent numbers in “standard” floating point form with a large base and a long significand spread across the elements of an array. All the remaining software described in this section is of the second type.

The first major piece of multiple precision Fortran software was Brent's package [161, 1978], [162, 1978]. This is a portable collection of Fortran 66 subroutines with wide capabilities, including the evaluation of special functions. Brent's package represents multiple precision numbers as arrays of integers and operates on them with integer arithmetic. An interface between Brent's package and the Fortran precompiler Augment [280, 1979] is described by Brent, Hooper, and Yohe [163, 1980]; it makes Brent's package much easier to use.

More recent is Bailey's multiple precision arithmetic package MPFUN [53, 1993], which consists of about 10,000 lines of Fortran 77 code in 87 subprograms.

In MPFUN, a multiprecision (MP) number is a vector of single precision floating point numbers; it represents a number in base 2^{24} (for IEEE arithmetic). Complex multiprecision numbers are also supported.

MPFUN routines are available to perform the basic arithmetic operations, evaluate n th roots and transcendental functions, compare MP numbers, produce a random MP number, solve polynomial equations, and perform other operations. For many of these routines both simple and advanced algorithms are provided; the advanced algorithms are intended for computations at precision levels above 1000 digits. One advanced algorithm is a fast Fourier transform technique for evaluating the convolution involved in a multiplication (see Problem 24.1). Another is used for division: x/y is evaluated as $x * (1/y)$, where $1/y$ is evaluated by Newton's method (see §2.6), which involves only multiplications. An interesting aspect of Newton's method in this context is that it can be implemented with a precision level that doubles at each stage, since the iteration damps out any errors. Another interesting feature of a variable precision environment is that numerical instability can be tackled simply by increasing the precision. As an example, MPFUN does complex multiplication using the 3M method

$$(a + ib)(c + id) = ac - bd + i[(a + b)(c + d) - ac - bd],$$

which uses three real multiplications instead of the usual four. As we saw in §23.2.4, the 3M method produces a computed imaginary part that can have large relative error, but this instability is not a serious obstacle to its use in MPFUN.

Bailey provides a translator that takes Fortran 77 source containing directives in comments that specify the precision level and which variables are to be treated as multiprecision, and produces a Fortran 77 program containing the appropriate multiprecision subroutine calls. He also provides a Fortran 90 version of the package that employs derived data types and operator extensions [54, 1995]. This Fortran 90 version is a powerful and easy-to-use tool for doing high-precision numerical computations.

Bailey's own use of the packages includes high-precision computation of constants such as π [51, 1988] and Euler's constant γ . A practical application to a fluid flow problem is described by Bailey, Krasny, and Pelz [55, 1993]. They found that converting an existing code to use MPFUN routines increased execution times by a factor of about 400 at 56 decimal digit precision, and they comment that this magnitude of increase is fairly typical for multiple precision computation.

Smith [1048, 1991], [1049, 1998] provides Fortran 77 programs for multiple precision real and complex arithmetic that are functionally quite similar to Bailey's MPFUN.

Fortran routines for high-precision computation can also be found in *Numerical Recipes* [953, 1992, §20.6], and high-precision numerical computation is supported by many symbolic manipulation packages, including Maple [222, 1991] and Mathematica [1253, 1999].

GNU MP is a free, portable C library for arbitrary precision arithmetic, operating on integers and rationals as well as floating point numbers. For details see <http://www.swox.com/gmp/>. The MPFR library, developed in France, is a C library for multiprecision floating point computations and is based on the GNU MP library. It provides the four IEEE rounding modes and provides some elementary

functions, all correctly rounded. MPFR is intended to replace GNU MP's existing floating point class (called MPF) in future releases. A multiple precision interval arithmetic library built on MPFR is also available. For details concerning MPFR see <http://www.mpfr.org/>.

A Pascal-like programming language called Turing [635, 1988] developed at the University of Toronto in the 1980s is the basis of an extension called Numerical Turing, developed by Hull and his co-workers [648, 1985]. Numerical Turing includes decimal floating point arithmetic whose precision can be dynamically varied throughout the course of a program, a feature argued for in [647, 1982] and implemented in hardware in [259, 1983].

27.10. Extended and Mixed Precision BLAS

As part of a major effort by the BLAS Technical Forum to develop a new BLAS standard [88, 2002], a set of Extended and Mixed Precision BLAS has been defined [785, 2000]. (See Appendix C for details of the original BLAS.) These BLAS use extended precision internally, defined to mean a precision at least 1.5 times as accurate as double precision and wider than 80 bits. The input and output arguments remain single or double precision variables, but some arguments can be of mixed type (real or complex) as well as mixed precision (single or double). These BLAS provide extended and mixed precision counterparts of selected existing level 1, 2, and 3 BLAS, and the main visible difference is an extra input argument that specifies the precision at which internal computations are to be performed.

A reference implementation is provided that employs the double-double format described in the previous section, giving an extended precision of about 106 bits. For full details of the Extended and Mixed Precision BLAS, including interesting historical background and the approach taken to the tricky issue of testing an implementation, see [785, 2000].

Earlier, an extended precision extension to the level-2 BLAS had been proposed in [347, 1988, App. B].

27.11. Patriot Missile Software Problem

A report from the United States General Accounting Office begins, "On February 25, 1991, a Patriot missile defense system operating at Dhahran, Saudi Arabia, during Operation Desert Storm failed to track and intercept an incoming Scud. This Scud subsequently hit an Army barracks, killing 28 Americans" [1173, 1992]. The report finds that the failure to track the Scud missile was caused by a precision problem in the software.

The computer used to control the Patriot missile was based on a 1970s design and uses 24-bit arithmetic. The Patriot system tracks its targets by measuring the time it takes for radar pulses to bounce back from them. Time is recorded by the system clock in tenths of a second, but is stored as an integer. To enable tracking calculations the time is converted to a 24-bit floating point number. Rounding errors in the time conversions cause shifts in the system's "range gate", which is used to track the target.

Table 27.2. *Effect of extended run time on Patriot missile operation.*

Hours	Seconds	Calculated time (seconds)	Inaccuracy (seconds)	Approximate shift in range gate (meters)
0	0	0	0	0
1	3600	3599.9966	.0034	7
8	28800	28799.9725	.0275	55
20 ^a	72000	71999.9313	.0687	137
48	172800	172799.8352	.1648	330
72	259200	259199.7528	.2472	494
100 ^b	360000	359999.6567 ^c	.3433	687

^aFor continuous operation exceeding 20 hours target is outside range gate.

^bAlpha battery ran continuously for about 100 hours.

^cCorrected value.

On February 11, 1991 the Patriot Project Office received field data identifying a 20% shift in the Patriot system's range gate after the system had been running continuously for 8 hours. This data implied that after 20 consecutive hours of use the system would fail to track and intercept a Scud. Modified software that compensated for the inaccurate time calculations was released on February 16 by army officials. On February 25, Alpha Battery, which was protecting the Dhahran Air Base, had been in continuous operation for over 100 hours. The inaccurate time calculations caused the range gate to shift so much that the system could not track the incoming Scud. On February 26, the next day, the modified software arrived in Dhahran. Table 27.2, taken from [1173, 1992], shows clearly how, with increasing time of operation, the Patriot lost track of its target. Note that the numbers in Table 27.2 are consistent with a relative error of 2^{-20} in the computer's representation of 0.1, this constant being used to convert from the system clock's tenths of a second to seconds (2^{-20} is the relative error introduced by chopping 0.1 to 23 bits after the binary point).

27.12. Notes and References

The issues discussed in this chapter are rarely treated in textbooks. One book that is worth consulting is Miller's *Engineering of Numerical Software* [853, 1984], which is concerned with the designing of reliable numerical software and presents practical examples.

Kahan's paper [688, 1972] is not, as the title suggests, a survey of error analysis, but treats in detail some specific problems, including solution of a quadratic equation and summation, giving error analysis, programs illustrating foibles of Fortran compilers, and ingenious workarounds.

The collection of papers *Improving Floating-Point Programming* [1204, 1990] contains, in addition to an introduction to floating point arithmetic and Brown's model, descriptions of recent work on embedding interval arithmetic with a super-accurate inner product in Pascal and ADA (see §26.4).

There are several algorithms for evaluating a continued fraction; see, for example, Blanch [138, 1964] and Press, Teukolsky, Vetterling, and Flannery [953, 1992, §5.2]. Rounding error analysis for the “bottom up” algorithm is given by Jones and Thron [678, 1974].

A standard set of names and definitions of machine parameters was proposed by the IFIP Working Group on Numerical Software (WG 2.5) [419, 1978], though these do not appear to have been widely adopted.

The use of high-precision computations to investigate the Riemann hypothesis and other mathematical problems and conjectures is described by Varga [1191, 1990], who has made use of Brent’s package [161, 1978], [162, 1978].

Problems

27.1. (a) The following MATLAB code attempts to compute

$$\mu := \min\{\text{floating point } x : fl(1+x) > 1\},$$

under the assumption that the base $\beta = 2$.

```
function x = mu
x = 1;
xp1 = x + 1;
while xp1 > 1
    x = x/2;
    xp1 = x + 1;
end
x = 2*x;
```

On my workstation, running this code gives

```
>> mu
ans =
    2.2204e-016
```

Try this code, or a translation of it into another language, on any machines available to you. Are the answers what you expected?

(b) Under what circumstances might the code in (a) fail to compute μ ? (Hint: consider optimizing compilers.)

(c) What is the value of μ in terms of the arithmetic parameters β and t ? Note: the answer depends subtly on how rounding is performed, and in particular on whether double rounding is in effect; see Higham [597, 1991] and Moler [865, 1990]. On a workstation with an Intel 486DX chip (in which double rounding does occur), the following behaviour is observed in MATLAB 4.2:

```
>> format hex; format compact    % Hexadecimal format.

>> x = 2^(-53) + 2^(-64) + 2^(-105); y = [1+x 1 x]
y =
    3ff0000000000001    3ff0000000000000    3ca0020000000001
```

```
>> x = 2^(-53) + 2^(-64) + 2^(-106); y = [1+x 1 x]
y =
    3ff0000000000000    3ff0000000000000    3ca0020000000000
```

27.2. Show that Smith's formulae (27.1) can be derived by applying Gaussian elimination with partial pivoting to the 2×2 linear system obtained from $(c + id)(x + iy) = a + ib$.

27.3. The following MATLAB function implements an algorithm of Malcolm [809, 1972] for determining the floating point arithmetic parameters β and t .

```
function [beta, t] = param(x)
a = 1;
while (a+1) - a == 1
    a = 2*a;
end
b = 1;
while (a+b) == a
    b = 2*b;
end
beta = (a+b) - a;
t = 1;
a = beta;
while (a+1) - a == 1
    t = t+1;
    a = a*beta;
end
```

Run this code, or a translation into another language, on any machines available to you. Explain how the code works. (Hint: consider the integers that can be represented exactly as floating point numbers.) Under what circumstances might it fail to produce the desired results?

27.4. Hough [641, 1989] formed a 512×512 matrix using the following Fortran random number generator:

```
subroutine matgen(a,lda,n,b,norma)
REAL a(lda,1),b(1),norma
c
    init = 1325
    norma = 0.0
    do 30 j = 1,n
        do 20 i = 1,n
            init = mod(3125*init,65536)
            a(i,j) = (init - 32768.0)/16384.0
            norma = max(a(i,j), norma)
        20 continue
    30 continue
```

He then solved a linear system using this matrix, on a workstation that uses IEEE single precision arithmetic. He found that the program took an inordinate amount of time to run and tracked the problem to underflows. On the system in question

underflows are trapped and the correct result recomputed very slowly in software. Hough picks up the story:

I started printing out the pivots in the program. They started out as normal numbers like 1 or -10 , then suddenly dropped to about $1e-7$, then later to $1e-14$, and then:

k 82 pivot $-1.8666e-20$	k 98 pivot $1.22101e-21$
k 83 pivot $-2.96595e-14$	k 99 pivot $-7.12407e-22$
k 84 pivot $2.46156e-14$	k 100 pivot $-1.75579e-21$
k 85 pivot $2.40541e-14$	k 101 pivot $3.13343e-21$
k 86 pivot $-4.99053e-14$	k 102 pivot $-6.99946e-22$
k 87 pivot $1.7579e-14$	k 103 pivot $3.82048e-22$
k 88 pivot $1.69295e-14$	k 104 pivot $8.05538e-22$
k 89 pivot $-1.56396e-14$	k 105 pivot $-1.18164e-21$
k 90 pivot $1.37869e-14$	k 106 pivot $-6.349e-22$
k 91 pivot $-3.10221e-14$	k 107 pivot $-2.48245e-21$
k 92 pivot $2.35206e-14$	k 108 pivot $-8.89452e-22$
k 93 pivot $1.32175e-14$	k 109 pivot $-8.23235e-22$
k 94 pivot $-7.77593e-15$	k 110 pivot $4.40549e-21$
k 95 pivot $1.34815e-14$	k 111 pivot $1.12387e-21$
k 96 pivot $-1.02589e-21$	k 112 pivot $-4.78853e-22$
k 97 pivot $4.27131e-22$	k 113 pivot $4.38739e-22$
	k 114 pivot $7.3868e-28$

SIGFPE 8: numerical exception, CHK, or TRAP
 stopped at daxpy+0x18c: movl a4@(0xe10),a3@

Explain this behaviour.

27.5. The version of the level-1 BLAS routine `xNRM2` that is distributed with LAPACK implements the following algorithm for computing $\|x\|_2$, $x \in \mathbb{R}^n$:

```

t = 0
s = 1
for i = 1:n
    if |xi| > t
        s = 1 + s (t/xi)2
        t = |xi|
    else
        s = s + (xi/t)2
    end
end
||x||2 = t√s

```

Prove that the algorithm works and describe its properties.

27.6. (Moler and Morrison [867, 1983], Dubrulle [358, 1983]) This MATLAB M-file computes the Pythagorean sum $\sqrt{a^2 + b^2}$ using an algorithm of Moler and Morrison.

```

function p = pythag(a,b)
%PYTHAG      Pythagorean sum.

p = max(abs(a), abs(b));
q = min(abs(a), abs(b));
while 1
    r = (q/p)^2;
    if r+4 == 4, return, end
    s = r/(4+r);
    p = p+2*s*p;
    q = s*q;
    fprintf('p = %19.15e, q = %19.15e\n', p, q)
end

```

The algorithm is immune to underflow and overflow (unless the result overflows), is very accurate, and converges in at most three iterations, assuming the unit roundoff $u \geq 10^{-20}$.

Example:

```

>> p = pythag(3,4); (p-5)/5
p = 4.986301369863014e+000, q = 3.698630136986301e-001
p = 4.999999974188253e+000, q = 5.080526329415358e-004
p = 5.0000000000000001e+000, q = 1.311372652397091e-012
ans =
    1.7764e-016

```

The purpose of this problem is to show that `pythag` is Halley's method applied to a certain equation. Halley's method for solving $f(x) = 0$ is the iteration

$$x_{n+1} = x_n - \frac{f_n/f'_n}{1 - \frac{1}{2}f_n f''_n/(f'_n)^2},$$

where f_n , f'_n , and f''_n denote the values of f and its first two derivatives at x_n .

(a) For given x_0 and y_0 such that $0 < y_0 \leq x_0$, the Pythagorean sum $p = \sqrt{x_0^2 + y_0^2}$ is a root of the equation $f(x) = x^2 - p^2 = 0$. Show that Halley's method applied to this equation gives

$$x_{n+1} = x_n \left(1 + 2 \frac{p^2 - x_n^2}{p^2 + 3x_n^2} \right).$$

Show that if $0 \leq x_0 \leq p = \sqrt{x_0^2 + y_0^2}$ then $x_n \leq x_{n+1} \leq p$ for all n . Deduce that $y_n := \sqrt{p^2 - x_n^2}$ is defined and that

$$x_{n+1} = x_n \left(1 + 2 \frac{y_n^2}{4x_n^2 + y_n^2} \right),$$

$$y_{n+1} = \frac{y_n^3}{4x_n^2 + y_n^2}.$$

Confirm that `pythag` implements a scaled version of these equations.

(b) Show that

$$p - x_{n+1} = \frac{(p - x_n)^3}{p^2 + 3x_n^2},$$

which displays the cubic convergence of the iteration.

27.7. (Incertis [664, 1985]) (a) Given a skew-symmetric matrix $Y \in \mathbb{R}^{n \times n}$ with $\|Y\|_2 \leq 1$, show that there is a real, symmetric X satisfying $X^2 = I + Y^2$ such that $X + Y$ is orthogonal.

(b) Consider the following iteration, for $P_0, Q_0 \in \mathbb{R}^{n \times n}$ with $\|P_0\|_2 \geq \|Q_0\|_2$, which generalizes to matrices the “pythag” iteration in Problem 27.6.

```

for  $k = 1 : \infty$ 
   $R_k = (Q_k P_k^{-1})^2$ 
   $S_k = R_k(4I + R_k)^{-1}$ 
   $P_{k+1} = P_k + 2S_k P_k$ 
   $Q_{k+1} = S_k Q_k$ 
end

```

Show that this iteration can be used to compute X in part (a).

(c) Investigate the convergence of the iteration in (b) for general P_0 and Q_0 .

27.8. Why might we prefer the expression $\sqrt{|x|}/\sqrt{3}$ to $\sqrt{|x|/3}$ in an algorithm intended to be robust in floating point arithmetic?

Chapter 28

A Gallery of Test Matrices

*Many tricks or treats associated with the Hilbert matrix
may seem rather frightening or fascinating.*

— MAN-DUEN CHOI, *Tricks or Treats with the Hilbert Matrix* (1983)

*I start by looking at a 2 by 2 matrix.
Sometimes I look at a 4 by 4 matrix.*

*That's when things get out of control and too hard.
Usually 2 by 2 or 3 by 3 is enough, and I look at them,
and I compute with them, and I try to guess the facts.*

*First, think of a question.
Second, I look at examples, and then third,
guess the facts.*

— PAUL R. HALMOS²³ (1991)

*When people look down on matrices,
remind them of great mathematicians such as
Frobenius, Schur, C. L. Siegel, Ostrowski, Motzkin, Kac, etc.,
who made important contributions to the subject.*

— OLGA TAUSSKY, *How I Became a Torchbearer for Matrix Theory* (1988)

²³From interviews by Albers in [10, 1991].

Ever since the first computer programs for matrix computations were written in the 1940s, researchers have been devising matrices suitable for test purposes and investigating the properties of these matrices. In the 1950s and 1960s it was common for a whole paper to be devoted to a particular test matrix: typically its inverse or eigenvalues would be obtained in closed form.

Early collections of test matrices include those of Newman and Todd [891, 1958] and Rutishauser [1006, 1968]; most of Rutishauser's matrices come from continued fractions or moment problems. Two well-known books present collections of test matrices. Gregory and Karney [524, 1969] deal exclusively with the topic, while Westlake [1217, 1968] gives an appendix of test matrices.

In this chapter we present a gallery of matrices. We describe their properties and explain why they are useful (or not so useful, as the case may be) for test purposes. The coverage is limited, and all the matrices we consider are available in MATLAB. A comprehensive, well-documented collection of parametrized test matrices is contained in MATLAB's `gallery` function (see `help gallery`), and MATLAB has several other special matrices that can be used for test purposes (see `help elmat`).

In addition to the matrices discussed in this chapter, interesting matrices discussed elsewhere in this book include magic squares (Problem 6.4), the Kahan matrix (equation (8.11)), Hadamard matrices (§9.4), and Vandermonde matrices (Chapter 22).

Table 28.1 lists MATLAB functions that can be used to generate many of the test matrices considered in this book.

The matrices described here can be modified in various ways while still preserving some or all of their interesting properties. Among the many ways of constructing new test matrices from old are

- similarity transformations $A \leftarrow X^{-1}AX$,
- unitary transformations $A \leftarrow UAV$, where $U^*U = V^*V = I$,
- Kronecker products $A \leftarrow A \otimes B$ or $B \otimes A$ (for which MATLAB has a function `kron`),
- powers $A \leftarrow A^k$.

28.1. The Hilbert and Cauchy Matrices

The Hilbert matrix $H_n \in \mathbb{R}^{n \times n}$, with elements $h_{ij} = 1/(i + j - 1)$, is perhaps the most famous of all test matrices. It was widely used in the 1950s and 1960s for testing inversion algorithms and linear equation solvers. Its attractions were threefold: it is very ill conditioned for even moderate values of n , formulae are known for the elements of the inverse, and the matrix arises in a practical problem: least squares fitting by a polynomial expressed in the monomial basis.

Despite its past popularity and notoriety, the Hilbert matrix is not a good test matrix. It is too special. Not only is it symmetric positive definite, but it is totally positive. This means, for example, that Gaussian elimination without pivoting is guaranteed to produce a small componentwise relative backward error (as is

Table 28.1. *MATLAB functions for generating matrices discussed in this book.*

<code>compan</code>	Companion matrix
<code>frank</code>	Frank matrix—ill-conditioned eigenvalues
<code>gallery</code>	Large collection of test matrices
<code>gallery('chebspec',...)</code>	Chebyshev spectral differentiation matrix
<code>gallery('clement',...)</code>	Clement matrix—tridiagonal with zero diagonal entries
<code>gallery('kahan',...)</code>	Kahan matrix—upper trapezoidal
<code>gallery('orthog',...)</code>	Orthogonal and nearly orthogonal matrices
<code>gallery('pei',...)</code>	Pei matrix
<code>gallery('randsvd',...)</code>	Random matrix with preassigned singular values and specified bandwidth
<code>gallery('toeppen',...)</code>	Pentadiagonal Toeplitz matrix
<code>gallery('tridiag',...)</code>	Tridiagonal matrix
<code>hadamard</code>	Hadamard matrix
<code>hilb</code>	Hilbert matrix
<code>invhilb</code>	Inverse Hilbert matrix
<code>magic</code>	Magic square
<code>pascal</code>	Pascal matrix
<code>rand</code>	Matrix of uniformly distributed random numbers
<code>randn</code>	Matrix of normally distributed random numbers
<code>toeplitz</code>	Toeplitz matrix
<code>vander</code>	Vandermonde matrix

Cholesky factorization). Thus the Hilbert matrix is not a typical ill-conditioned matrix.

The (i, j) element of the inverse of the Hilbert matrix H_n is the integer

$$(-1)^{i+j}(i+j-1) \binom{n+i-1}{n-j} \binom{n+j-1}{n-i} \binom{i+j-2}{i-1}^2, \quad (28.1)$$

and

$$\det(H_n) = \frac{(1! 2! \dots (n-1)!)^4}{(1! 2! \dots (2n-1)!)^2} \sim 2^{-2n^2}. \quad (28.2)$$

There are many ways to rewrite the formula (28.1). These formulae are best obtained as special cases of those for the Cauchy matrix below.

The Cholesky factor R_n of the inverse of the Hilbert matrix is known explicitly, as is R_n^{-1} :

$$r_{ij} = \frac{\sqrt{2i-1} [(j-1)!]^2}{(i+j-1)!(j-i)!}, \quad j \geq i, \quad (28.3)$$

$$(R_n^{-1})_{ij} = (-1)^{i+j} \sqrt{2j-1} \binom{i+j-2}{i-1} \binom{j-1}{i-1}, \quad j \geq i. \quad (28.4)$$

One interesting application of these formulae is to compute the eigenvalues of H_n as the squares of the singular values of R_n ; if R_n is evaluated from (28.3) and

Table 28.2. *Condition numbers of Hilbert and Pascal matrices.*

n	$\kappa_2(H_n)$	$\kappa_2(P_n)$
2	1.9281e1	6.8541e0
3	5.2406e2	6.1984e1
4	1.5514e4	6.9194e2
5	4.7661e5	8.5175e3
6	1.4951e7	1.1079e5
7	4.7537e8	1.4934e6
8	1.5258e10	2.0645e7
9	4.9315e11	2.9078e8
10	1.6026e13	4.1552e9
11	5.2307e14	6.0064e10
12	1.7132e16	8.7639e11
13	5.6279e17	1.2888e13
14	1.8534e19	1.9076e14
15	6.1166e20	2.8396e15
16	2.0223e22	4.2476e16

the one-sided Jacobi algorithm is used to compute the singular values then high relative accuracy is obtained for every eigenvalue, as shown by Mathias [821, 1995].

The condition number of the Hilbert matrix grows at an exponential rate: $\kappa_2(H_n) \sim \exp(3.5n)$ [1141, 1954]. See Table 28.2 for the first few condition numbers (these were obtained by computing the inverse exactly using MATLAB's Symbolic Math Toolbox [825] and then computing the norm of the numeric representation of the inverse; the numbers given are correct to the figures shown).

It is an interesting fact that the matrix $\tilde{H}_n = (1/(i+j)) \in \mathbb{R}^{n \times n}$ (a submatrix of H_{n+1}) satisfies $\mu_n := \|\tilde{H}_n\|_2 = \pi + O(1/\log n)$ as $n \rightarrow \infty$, as proved by Taussky [1131, 1949]. The convergence to π is very slow: $\mu_{200} = 2.01$, $\mu_{300} = 2.08$, $\mu_{400} = 2.12$.

That H_n^{-1} is known explicitly is not as useful a property for testing an inversion algorithm as it might appear, because H_n cannot be stored exactly in floating point arithmetic. This means that when we attempt to invert H_n we at best invert $H_n + \Delta H$ (the matrix actually stored on the computer), where $|\Delta H| \leq uH_n$, and $(H_n + \Delta H)^{-1}$ can differ greatly from H_n^{-1} , in view of the ill conditioning. A possible way round this difficulty is to start with the integer matrix H_n^{-1} , but its entries are so large that they are exactly representable in IEEE double precision arithmetic only for n less than 13.

The Hilbert matrix is a special case of a Cauchy matrix $C_n \in \mathbb{R}^{n \times n}$, whose elements are $c_{ij} = 1/(x_i + y_j)$, where $x, y \in \mathbb{R}^n$ are given n -vectors (take $x_i = y_i = i - 0.5$ for the Hilbert matrix). The following formulae give the inverse and determinant of C_n , and therefore generalize those for the Hilbert matrix. The

(i, j) element of C_n^{-1} is

$$\frac{\prod_{1 \leq k \leq n} (x_j + y_k)(x_k + y_i)}{(x_j + y_i) \prod_{\substack{1 \leq k \leq n \\ k \neq j}} (x_j - x_k) \prod_{\substack{1 \leq k \leq n \\ k \neq i}} (y_i - y_k)}$$

and

$$\det(C_n) = \frac{\prod_{1 \leq i < j \leq n} (x_j - x_i)(y_j - y_i)}{\prod_{1 \leq i, j \leq n} (x_i + y_j)},$$

the latter formula having been published by Cauchy in 1841 [207, 1841, pp. 151–159]. The LU factors of C_n were found explicitly by Cho [232, 1968]:

$$l_{ij} = \frac{x_j + y_j}{x_i + y_i} \prod_{k=1}^{j-1} \frac{(x_j + y_k)(x_i - x_k)}{(x_i + y_k)(x_j - x_k)}, \quad 1 \leq j < i \leq n,$$

$$u_{ij} = \frac{1}{x_i + y_j} \prod_{k=1}^{i-1} \frac{(x_i - x_k)(y_j - y_k)}{(x_k + y_j)(x_i + y_k)}, \quad 1 \leq i \leq j \leq n.$$

It is known that C_n is totally positive if $0 < x_1 < \dots < x_n$ and $0 < y_1 < \dots < y_n$ (as is true for the Hilbert matrix) [1133, 1962, p. 295]. Interestingly, the sum of all the elements of C_n^{-1} is $\sum_{i=1}^n (x_i + y_i)$ [743, 1997, Ex. 44, §1.2.3]. By exploiting their structure, singular value decompositions of Cauchy matrices can be computed to high accuracy; see Demmel [318, 1999].

28.2. Random Matrices

Random matrices are widely used for test purposes. Perhaps the earliest use of random matrices in numerical analysis was by von Neumann and Goldstine [1200, 1947], [501, 1951], who estimated the size of their error bounds for matrix inversion (see §9.13) for the case of random matrices; to do this, they had to estimate the condition number of a random matrix.

Intuitively, one might expect random matrices to be good at revealing programming errors and unusual behaviour of algorithms, but this expectation is not necessarily correct. For example, Miller [853, 1984, pp. 96–97] describes a mutation experiment involving Fortran codes for Gaussian elimination without pivoting, Gaussian elimination with partial pivoting, and Gauss–Jordan elimination with partial pivoting. For each code, all possible mutants were generated, where a mutant is obtained by making a single typographical change to the source code. All the mutants were tested on a single random linear system $Ax = b$, with known solution, where a_{ij} was chosen from the uniform $[-1, 1]$ distribution. Many mutants were detected by their failure to pass the test of producing a solution with forward error less than a tolerance. However, some mutants passed this test, including all those that solve a system correctly in exact arithmetic; mutants in

the latter class include those that select an incorrect pivot row and thus implement a numerically unstable algorithm. A conclusion to be drawn from Miller's experiment is that random test data can reveal some programming errors, but will not reveal all.

A good example of a problem for which random test matrices are very poor at revealing algorithmic weaknesses is matrix condition number estimation. The popular condition estimation algorithms can yield poor estimates but, in practice, never produce them for a random matrix (see Chapter 15). The role of random matrices here is to indicate the *average* quality of the estimates.

Edelman [376, 1993] summarizes the properties of random matrices well when he says that

What is a mistake is to psychologically link a random matrix with the intuitive notion of a “typical” matrix or the vague concept of “any old matrix.” In contrast, we argue that “random matrices” are very *special* matrices. The larger the size of the matrices the more predictable they are because of the central limit theorem.

Various results are known about the behaviour of matrices with elements from the normal $N(0, 1)$ distribution. Matrices of this type are generated by MATLAB's **randn** function. Let A_n denote a full $n \times n$ matrix, T_n a triangular matrix, and $\tilde{T}_n$ a unit triangular matrix, all with elements from this distribution, and let $E(\cdot)$ be the expectation operator. Then, in the appropriate probabilistic sense, the following results hold as $n \rightarrow \infty$:

$$E(\log(\kappa_2(A_n))) \approx \log n + 1.537 \quad (\text{real data}), \quad (28.5)$$

$$E(\log(\kappa_2(A_n))) \approx \log n + 0.982 \quad (\text{complex data}), \quad (28.6)$$

$$\|A_n\|_2 \approx 2\sqrt{n} \quad (\text{real data}), \quad (28.7)$$

$$\|A_n\|_2 \approx 2\sqrt{2}\sqrt{n} \quad (\text{complex data}), \quad (28.8)$$

$$\rho(A_n) \approx \sqrt{n} \quad (\text{real or complex data}), \quad (28.9)$$

$$\kappa_2(T_n)^{1/n} \approx 2 \quad (\text{real data}), \quad (28.10)$$

$$\kappa_2(\tilde{T}_n)^{1/n} \approx 1.306 \quad (\text{real data}). \quad (28.11)$$

For details of (28.5)–(28.8) see Edelman [371, 1988], and for (28.10), (28.11) see Viswanath and Trefethen [1197, 1998]. Edelman conjectures that the condition number results are true for any distribution with mean 0—in particular, the uniform $[-1, 1]$ distribution used by MATLAB's **rand**. The results (28.5) and (28.6) show that *random matrices from the normal $N(0, 1)$ distribution tend to be very well conditioned*.

The spectral radius result (28.9) has been proved as an upper bound on $\rho(A_n)$ by Geman [471, 1986] for independent identically distributed random variables a_{ij} with zero mean and unit variance, and computer experiments suggest the approximate equality for the standard normal distribution [471, 1986].

A question of interest in eigenvalue applications is how many eigenvalues of a random real matrix are real. The answer has been given by Edelman, Kostlan, and Shub [381, 1994]: denoting by E_n the expected number of real eigenvalues of an $n \times n$ matrix from the normal $N(0, 1)$ distribution,

$$\lim_{n \rightarrow \infty} \frac{E_n}{\sqrt{n}} = \sqrt{\frac{2}{\pi}}.$$

Thus the proportion of real eigenvalues, E_n/n , tends to zero as $n \rightarrow \infty$. Exact formulae for E_n for finite n are also given in [381, 1994].

As a final, and more straightforward, example of the special nature of random matrices, note that a square matrix from the uniform $[0, 1]$ distribution has all positive entries with probability 1, and therefore has a real and positive dominant eigenvalue, by the Perron–Frobenius theory [636, 1985, Chap. 8]. The remaining eigenvalues tend to be of much smaller magnitude than the dominant one [27, 1990].

28.3. “Randsvd” Matrices

By `randsvd`²⁴ we mean a matrix $A \in \mathbb{R}^{m \times n}$ formed as $A = U\Sigma V^T$, where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are random orthogonal matrices and $\Sigma = \text{diag}(\sigma_i)$ ($\sigma_i \geq 0$) is a given matrix of singular values. This type of matrix has a predetermined singular value distribution (and 2-norm condition number), but is, nevertheless, random.

`Randsvd` matrices have been widely used as test matrices, for example for condition estimators [582, 1987], [1070, 1980], and in the LAPACK testing software. Singular value distributions of interest include, with $\alpha := \kappa_2(A) \geq 1$ a parameter,

1. one large singular value: $\sigma_1 = 1$, $\sigma_i = \alpha^{-1}$, $i = 2:n$;
2. one small singular value: $\sigma_i = 1$, $i = 1:n-1$, $\sigma_n = \alpha^{-1}$;
3. geometrically distributed singular values: $\sigma_i = \beta^{1-i}$, $i = 1:n$, where $\beta = \alpha^{1/(n-1)}$;
4. arithmetically distributed singular values: $\sigma_i = 1 - (1 - \alpha^{-1})(i-1)/(n-1)$, $i = 1:n$.

To be precise about what we mean by “random orthogonal matrix” we specify matrices from the Haar distribution, which is a natural distribution over the space of orthogonal matrices, defined in terms of a measure called the Haar measure [874, 1982, §2.1.4]. If $A \in \mathbb{R}^{n \times n}$ has elements from the normal $N(0, \sigma^2)$ distribution and A has the QR factorization $A = QR$, with the factorization normalized so that the diagonal elements of R are positive, then Q is from the Haar distribution, for any variance σ^2 [113, 1979], [1070, 1980]. If we compute Q from this prescription, the cost is $2n^3$ flops. For our `randsvd` application, a more efficient approach is based on the following result of Stewart [1070, 1980].

Theorem 28.1 (Stewart). *Let the independent vectors $x_i \in \mathbb{R}^i$ have elements from the normal $N(0, 1)$ distribution for $i = 1:n-1$. Let $P_i = \text{diag}(I_{n-i}, \bar{P}_i)$, where $\bar{P}_i$ is the Householder transformation that reduces x_i to $r_{ii}e_1$. Then the product $Q = DP_1P_2 \dots P_{n-1}$ is a random orthogonal matrix from the Haar distribution, where $D = \text{diag}(\text{sign}(r_{ii}))$. $\square$*

²⁴In MATLAB matrices of this type are generated by `gallery('randsvd', ...)`.

This result allows us to compute a product form representation of a random $n \times n$ orthogonal matrix from the Haar distribution in $O(n^2)$ flops. If we implicitly compute $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ using the construction in Theorem 28.1, and then form $A = U \Sigma V^T$, exploiting the structure, the cost is $m^3 + n^3$ flops. The alternative, which involves obtaining U and V from the QR factorization of random matrices A , is about twice as expensive.

Note that forming $U \Sigma V^T$ with U and V *single* Householder matrices, as is sometimes done in the literature, is not recommended, as it produces matrices of a very special form: diagonal plus a rank-2 correction.

The construction of **randsvd** matrices is, of course, easily adapted to produce random symmetric matrices $A = Q \Lambda Q^T$ with given eigenvalues.

28.4. The Pascal Matrix

*The numbers in Pascal's triangle satisfy, practically speaking,
infinitely many identities.*

— RONALD L. GRAHAM, DONALD E. KNUTH, and OREN PATASHNIK,
Concrete Mathematics (1994)

A particularly entertaining test matrix is the Pascal matrix $P_n \in \mathbb{R}^{n \times n}$, defined by

$$p_{ij} = \frac{(i+j-2)!}{(i-1)!(j-1)!} = \binom{i+j-2}{j-1}.$$

The rows of Pascal's triangle appear as anti-diagonals of P_n :

```
>> P = pascal(6)
```

P =

1	1	1	1	1	1
1	2	3	4	5	6
1	3	6	10	15	21
1	4	10	20	35	56
1	5	15	35	70	126
1	6	21	56	126	252

The earliest references to the Pascal matrix appear to be in 1958 by Newman and Todd [891, 1958] and by Rutishauser [1007, 1958] (see also Newman [890, 1962, pp. 240–241]); Newman and Todd say that the matrix was introduced to them by Rutishauser. The matrix was independently suggested as a test matrix by Caffney [196, 1963].

Rutishauser [1006, 1968, §8] notes that P_n belongs to the class of moment matrices M whose elements are contour integrals

$$m_{ij} = \int_C z^{j-1} (\bar{z})^{i-1} w(z) dz.$$

All moment matrices corresponding to a positive weight function $w(z)$ on a contour C are Hermitian positive definite (as is easily verified by considering the quadratic form $y^* M y$). The choice $C = [0, 1]$, with weight function $w(z) = 1$, yields the

Hilbert matrix. The Pascal matrix is obtained for C the circle $\{z : |z - 1| = 1\}$ and $w(z) = (2\pi i(z - 1))^{-1}$ (not $w(z) = (2\pi)^{-1}$ as stated in [1006, 1968]); the change of variable $z = 1 + \exp(i\theta)$ yields a moment integral with a positive weight function.

Remarkably, the Cholesky factor of the Pascal matrix again contains the rows of Pascal's triangle, now arranged columnwise:

```
>> R = chol(P)
```

```
R =
```

1	1	1	1	1	1
0	1	2	3	4	5
0	0	1	3	6	10
0	0	0	1	4	10
0	0	0	0	1	5
0	0	0	0	0	1

The scaled and transposed Cholesky factor $S = R^T \text{diag}(1, -1, 1, -1, \dots, (-1)^{n+1})$ is returned by `pascal(n, 1)`:

```
>> S = pascal(6, 1)
```

```
S =
```

1	0	0	0	0	0
1	-1	0	0	0	0
1	-2	1	0	0	0
1	-3	3	-1	0	0
1	-4	6	-4	1	0
1	-5	10	-10	5	-1

It is involutory: $S^2 = I$. This special property leads us to several more properties of $P = P_n$. First, since $P = SS^T$, $P^{-1} = S^{-T}S^{-1} = S^TS$, and so P^{-1} has integer entries (as is also clear from the fact that $\det(P) = \det(R)^2 = 1$). Moreover,

$$P = SS^T = S(S^TS)S^{-1} = SP^{-1}S^{-1},$$

so P and P^{-1} are similar and hence have the same eigenvalues. In other words, the eigenvalues appear in reciprocal pairs. In fact, the characteristic polynomial π_n has a palindromic coefficient vector, which implies the reciprocal eigenvalues property, since $\pi_n(\lambda) = \lambda^n \pi_n(1/\lambda)$. This is illustrated as follows (making use of MATLAB's Symbolic Math Toolbox):

```
>> charpoly(P)
```

```
ans =
```

```
1-351*x+6084*x^2-13869*x^3+6084*x^4-351*x^5+x^6
```

```
>> eig(P)
```

```
ans =
```


0.0030
 0.0643
 0.4893
 2.0436
 15.5535
 332.8463

Since P is symmetric, its eigenvalues are its singular values and so we also have that $\|P\|_2 = \|P^{-1}\|_2$ and $\|P\|_F = \|P^{-1}\|_F$. Now

$$p_{nn} \leq \|P\|_2 \leq (\|P\|_1 \|P\|_\infty)^{1/2} = \left(\frac{2n-1}{n} \right) p_{nn},$$

where for the last equality we used a binomial coefficient summation identity from [517, 1994, §5.1]. Hence, using Stirling's approximation ($n! \sim \sqrt{2\pi n}(n/e)^n$),

$$\kappa_2(P_n) \sim \left(\frac{(2n)!}{(n!)^2} \right)^2 \sim \frac{16^n}{n\pi}.$$

Thus P_n is exponentially ill conditioned as $n \rightarrow \infty$.

It is worth pointing out that it is not hard to generate symmetric positive definite matrices with determinant 1 and the reciprocal root property. Let $X = ZDZ^{-1}$ where Z is nonsingular and $D = \text{diag}(\pm 1) \neq \pm I$. Then $X^2 = I$ and the matrix $A = X^T X$ has the desired properties. If we choose Z lower triangular then X is the Cholesky factor of A up to a column scaling by $\text{diag}(\pm 1)$.

The inverse of the Pascal matrix was found explicitly by Cohen [258, 1975]: the (i, j) element is

$$(-1)^{i-j} \sum_{k=0}^{n-i} \binom{i+k-1}{k} \binom{i+k-1}{i+k-j}, \quad i \geq j.$$

The Pascal matrix can be made singular simply by subtracting 1 from the (n, n) element. To see this, note that

$$\begin{aligned} P - e_n e_n^T &= SS^T - e_n e_n^T = S(I - (Se_n)(Se_n)^T)S^T \\ &= S \text{diag}(1, 1, \dots, 1, 0)S^T. \end{aligned}$$

This perturbation, $\Delta P = -e_n e_n^T$, is far from being the smallest one that makes P singular, which is $\Delta P_{\text{opt}} = -\lambda_n v_n v_n^T$, where λ_n is the smallest eigenvalue of P and v_n is the corresponding unit eigenvector, for $\|\Delta P_{\text{opt}}\|_2 = \lambda_n = \|P^{-1}\|_2^{-1}$ is of order $(n!)^2/(2n)! \sim 4^{-n} \sqrt{n\pi}$, as we saw above.

A more subtle property of the Pascal matrix is that it is totally positive. Karlin [712, 1968, p. 137] shows that the matrix with elements $\binom{i+j-1}{j}$ ($i, j = 0, 1, \dots$) is totally positive; the Pascal matrix is a submatrix of this one and hence is also totally positive. From the total positivity it follows that the Pascal matrix has distinct eigenvalues, which (as we already know from the positive definiteness) are real and positive, and that its i th eigenvector (corresponding to the i th largest eigenvalue) has exactly $i-1$ sign changes [454, 1959, Thm. 13, p. 105].

$T = \text{pascal}(n, 2)$ is obtained by rotating S clockwise through 90 degrees and multiplying by -1 if n is even:

```
>> T = pascal(6, 2)
```

```
T =
```

```

-1    -1    -1    -1    -1    -1
 5     4     3     2     1     0
-10    -6    -3    -1     0     0
 10     4     1     0     0     0
 -5     -1     0     0     0     0
 1      0     0     0     0     0
```

It has the surprising property that it is a cube root of the identity, a property noted by Turnbull [1167, 1929, p. 332]:

```
>> T*T
```

```
ans =
```

```

0      0      0      0      0      1
0      0      0      0     -1     -5
0      0      0      1      4     10
0      0     -1     -3     -6    -10
0      1      2      3      4      5
-1     -1     -1     -1     -1     -1
```

```
>> T*T*T
```

```
ans =
```

```

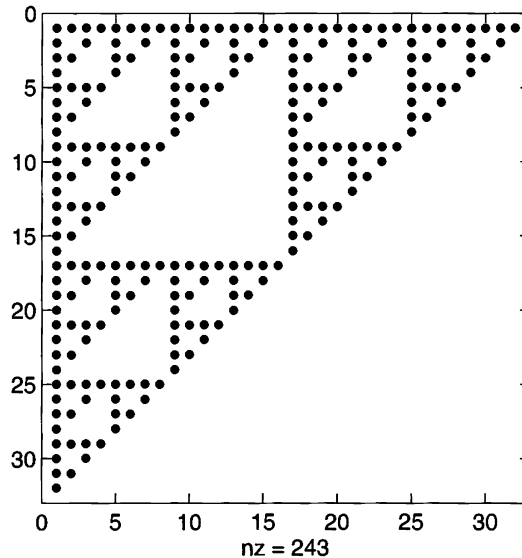
1      0      0      0      0      0
0      1      0      0      0      0
0      0      1      0      0      0
0      0      0      1      0      0
0      0      0      0      1      0
0      0      0      0      0      1
```

Finally, we note that it is trivial to plot an approximation to the Sierpinski gasket [931, 1992, §2.2], [518, 1992] in MATLAB: simply type `spy(rem(pascal(n), 2))`. See Figure 28.1. The picture produced by this command is incorrect for large n , however, because the elements of `pascal(n)` become too large to be exactly representable in floating point arithmetic.

28.5. Tridiagonal Toeplitz Matrices

A tridiagonal Toeplitz matrix has the form

$$T_n(c, d, e) = \begin{bmatrix} d & e & & & \\ c & d & \ddots & & \\ & \ddots & d & e & \\ & & c & d & \\ & & & c & d \end{bmatrix}.$$

Figure 28.1. `spy(rem(pascal(32),2)).`

Such matrices arise, for example, when discretizing partial differential equations or boundary value problems for ordinary differential equations. The eigenvalues are known explicitly [1004, 1947], [1005, 1952], [1143, 1977, pp. 155–156]:

$$d + 2(ce)^{1/2} \cos(k\pi/(n+1)), \quad k = 1:n.$$

The eigenvalues are also known for certain variations of the symmetric matrix $T_n(c, d, c)$ in which the $(1, 1)$ and (n, n) elements are modified; see Gregory and Karney [524, 1969].

The matrix $T_n(-1, 2, -1)$ is minus the well-known second difference matrix, which arises in applying central differences to a second derivative operator. Its inverse has (i, j) element $-i(n-j+1)/(n+1)$ (cf. Theorem 15.9). The condition number satisfies $\kappa_2(T_n) \sim (4/\pi^2)n^2$.

One interesting property of $T_n(c, d, e)$ is that the diagonals of its LU factorization converge as $n \rightarrow \infty$ when T_n is symmetric and diagonally dominant, and this allows some savings in the computation of the LU factorization, as shown by Malcolm and Palmer [810, 1974]. Similar properties hold for cyclic reduction; see Bondeli and Gander [150, 1994].

28.6. Companion Matrices

The companion matrix associated with the characteristic polynomial

$$\det(A - \lambda I) = (-1)^n (\lambda^n - a_{n-1}\lambda^{n-1} - \dots - a_0)$$

of $A \in \mathbb{C}^{n \times n}$ is the matrix

$$C = \begin{bmatrix} a_{n-1} & a_{n-2} & \cdots & \cdots & a_0 \\ 1 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & \ddots & & 0 \\ \vdots & & \ddots & 0 & 0 \\ 0 & \cdots & \cdots & 1 & 0 \end{bmatrix}.$$

In MATLAB C can be computed from A via $C = \text{compan}(\text{poly}(A))$. It is easy to check that C has the same characteristic polynomial as A , and that if λ is an eigenvalue of C then $[\lambda^{n-1}, \lambda^{n-2}, \dots, \lambda, 1]^T$ is a corresponding eigenvector. Since $C - \lambda I$ has rank at least $n-1$ for any λ , C is nonderogatory, that is, in the Jordan form no eigenvalue appears in more than one Jordan block. It follows that A is similar to C only if A is nonderogatory.

There are no explicit formulae for the eigenvalues of C , but, perhaps surprisingly, the singular values have simple representations, as found by Kenney and Laub [725, 1988] (see also Kittaneh [736, 1995]):

$$\begin{aligned} \sigma_1^2 &= \frac{1}{2} \left(\alpha + \sqrt{\alpha^2 - 4a_0^2} \right), \\ \sigma_i^2 &= 1, \quad i = 2:n-1, \\ \sigma_n^2 &= \frac{1}{2} \left(\alpha - \sqrt{\alpha^2 - 4a_0^2} \right), \end{aligned}$$

where $\alpha = 1 + a_0^2 + \cdots + a_{n-1}^2$. These formulae generalize to block companion matrices, as shown by Higham and Tisseur [625, 2001].

The `compan` function is a useful means for generating new test matrices from old. For any $A \in \mathbb{C}^{n \times n}$, `compan(poly(A))` is a nonnormal matrix with the same eigenvalues as A (to be precise, `compan(poly(A))` is normal if and only if $a_0 = 1$ and $a_i = 0$ for $i > 0$).

Companion matrices tend to have interesting pseudospectra, as illustrated in Figure 28.2. For more information on the pseudospectra of companion matrices see Toh and Trefethen [1144, 1994].

28.7. Notes and References

The earliest reference to the Hilbert matrix appears to be [626, 1894], wherein Hilbert obtains the formula (28.2) for $\det(H_n)$.

The formula (28.1) for H_n^{-1} is taken from Choi [233, 1983], who describes various interesting properties of the Hilbert matrix and its infinite analogue. An excellent reference for the derivation of formulae for the inverse of the Cauchy and Hilbert matrices is Knuth [743, 1997, pp. 37–38]. Another reference for the Cauchy matrix is Tyrtshnikov [1172, 1991]. The formulae (28.3) and (28.4) for the Cholesky factor and its inverse are from Choi [233, 1983] and Todd [1141, 1954].

Forsythe and Moler [431, 1967] have a chapter devoted to the Hilbert matrix, in which they describe the underlying least squares problem and discuss numerical

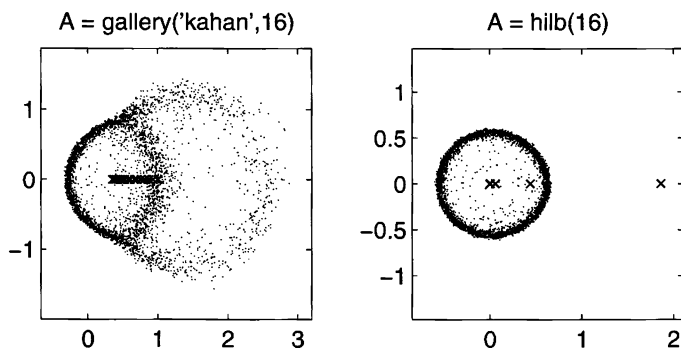


Figure 28.2. *Pseudospectra of $\text{compan}(\text{poly}(A))$.*

computation of the inverse. There have been many papers on the Hilbert matrix; two of particular interest are by Todd [1141, 1954], [1142, 1961].

Other references on the eigenvalues and condition numbers of random matrices include Edelman [372, 1991], [375, 1992] and Kostlan [748, 1992].

Anderson, Olkin, and Underhill [25, 1987] suggest another way to construct random orthogonal matrices from the Haar distribution, based on products of random Givens rotations.

The involutory triangular matrix `pascal(n,1)` arises in the stepsize changing mechanism in an ordinary differential equation code based on backward differentiation formulae; see Shampine and Reichelt [1032, 1997].

We mention some other collections of test matrices. The Harwell-Boeing collection of sparse matrices, largely drawn from practical problems, is presented by Duff, Grimes, and Lewis [362, 1989], [363, 1992]. Bai, Day, Demmel, and Dongarra [44, 1997] present a collection of test matrices for large-scale nonsymmetric eigenvalue problems. Zielke [1281, 1986] gives various parametrized rectangular matrices of fixed dimension with known generalized inverses. Demmel and McKenney [332, 1989] present a suite of Fortran 77 codes for generating random square and rectangular matrices with prescribed singular values, eigenvalues, band structure, and other properties. This suite is part of the testing code for LAPACK (see below), and MATLAB's `gallery('randsvd',...)` is modeled on it.

The Web repository *Matrix Market* [147, 1997] at <http://math.nist.gov/MatrixMarket/> contains a large collection of test matrices, comprising both individual matrices of fixed dimension (usually large and sparse, coming from applications) and matrix generators that can produce matrices of arbitrary dimension. The matrices can be downloaded in a variety of formats. Matrix Market includes matrices from several of the collections mentioned above.

In testing matrix algorithms it can be useful to be able to construct matrices with some given combination of eigenvalues, singular values, and diagonal elements (subject to any necessary constraints on the given data). Algorithms of this type can be summarized as follows.

- *Construct symmetric A with given diagonal elements and given eigenvalues.*
See Chan and Li [209, 1983], [210, 1983] and Zha and Zhang [1280, 1995]

(see also Ikramov [659, 1998]). A special case is the generation of random correlation matrices with specified eigenvalues (a correlation matrix is a symmetric positive semidefinite matrix with ones on the diagonal); see Davies and Higham [293, 2000] and the references therein.

- *Construct A with given eigenvalues and given singular values.* See Chu [238, 2000], and see Kosowski and Smoktunowicz [747, 2000] for the case of triangular matrices with unit diagonal (the latter paper solves a research problem posed in the first edition of this book).
- *Construct A with given singular values and given diagonal elements.* See Chu [237, 1999].

Note that not all these algorithms come with a guarantee that the construction is numerically stable: caveat emptor!

28.7.1. LAPACK

The LAPACK distribution contains a suite of routines for generating test matrices, located in the directory `LAPACK/TESTING/MATGEN` (in Unix notation). These routines (whose names are of the form `xLAzzz`) are used for testing when LAPACK is installed and are not described in the *LAPACK Users' Guide* [20, 1999].

Problems

28.1. Investigate the spectral and pseudospectral properties of pentadiagonal Toeplitz matrices. See Figure 28.3. MATLAB's `gallery('toeppen',...)` generates matrices of this type. References are Beam and Warming [96, 1993] and Reichel and Trefethen [978, 1992].

28.2. (RESEARCH PROBLEM) Two methods for generating a real orthogonal matrix from the Haar distribution are Stewart's method (Theorem 28.1), based on Householder transformations, and a method of Anderson, Olkin, and Underhill [25, 1987], based on Givens rotations. Compare the efficiency of these two methods when used to generate `randsvd` matrices. Investigate the use of the Givens rotations approach to construct random banded matrices with given singular values and random symmetric banded matrices with given eigenvalues, and compare with the technique of generating a full matrix and then using band reduction (as implemented in MATLAB's `gallery('randsvd',...)`).

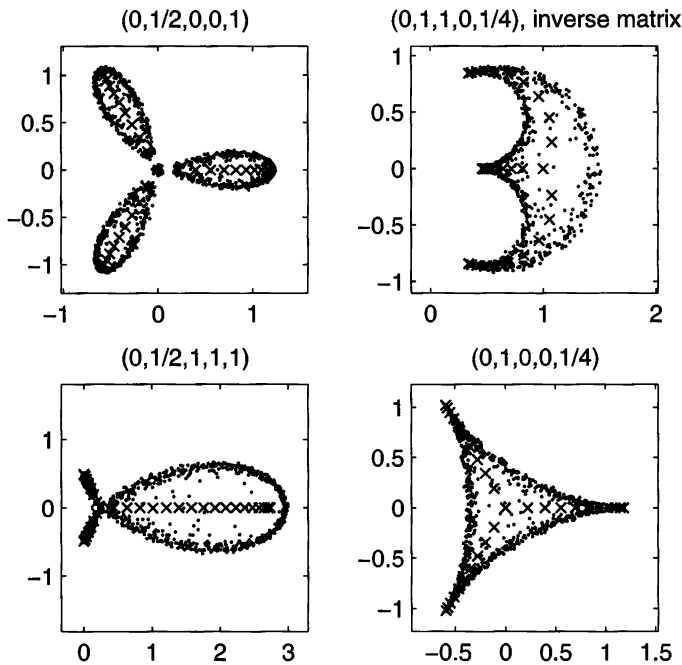


Figure 28.3. *Pseudospectra of 32×32 pentadiagonal Toeplitz matrices, gallery('toeppen', 32, a, b, c, d, e). Shown are the parameters (a, b, c, d, e).*

Appendix A

Solutions to Problems

1.1. Since $\tilde{E}_{\text{rel}}(\hat{x}) = E_{\text{rel}}(\hat{x})|x|/|\hat{x}|$ and $\hat{x} = x(1 \pm E_{\text{rel}}(\hat{x}))$, we have

$$\frac{E_{\text{rel}}(\hat{x})}{1 + E_{\text{rel}}(\hat{x})} \leq \tilde{E}_{\text{rel}}(\hat{x}) \leq \frac{E_{\text{rel}}(\hat{x})}{1 - E_{\text{rel}}(\hat{x})}.$$

Hence if $E_{\text{rel}}(\hat{x}) < 0.01$, say, then there is no difference between E_{rel} and $\tilde{E}_{\text{rel}}$ for practical purposes.

1.2. Nothing can be concluded about the last digit before the decimal point. Evaluating y to higher precision yields

t	y
35	262537412640768743.9999999999925007
40	262537412640768743.999999999992500725972

This shows that the required digit is, in fact, 3. The interesting fact that y is so close to an integer was pointed out by Lehmer [779, 1943], who explains its connection with number theory. It is known that y is irrational [1090, 1991].

1.3. 1. $x/(\sqrt{x+1}+1)$.

2. $2 \sin \frac{x-y}{2} \cos \frac{x+y}{2}$.

3. $(x-y)(x+y)$. Cancellation has not been avoided, but it is now harmless if x and y are known exactly (see also Problem 3.8).

4. $\sin x/(1 + \cos x)$.

5. $c = ((a-b)^2 + ab(2 \sin \theta/2)^2)^{1/2}$.

1.4. $a+ib = (x+iy)^2 = x^2 - y^2 + 2ixy$, so $b = 2xy$ and $a = x^2 - y^2$, giving $x^2 - b^2/(4x^2) = a$, or $4x^4 - 4ax^2 - b^2 = 0$. Hence

$$x^2 = \frac{4a \pm \sqrt{16a^2 + 16b^2}}{8} = \frac{a \pm \sqrt{a^2 + b^2}}{2}.$$

If $a \geq 0$ we use this formula with the plus sign, since $x^2 \geq 0$. If $a < 0$ this formula is potentially unstable, so we use the rewritten form

$$x^2 = \frac{b^2}{2(-a + \sqrt{a^2 + b^2})}.$$

In either case we get two values for x and recover y from $y = b/(2x)$. Note that there are other issues to consider here, such as scaling to avoid overflow.

1.5. A straightforward evaluation of $\log(1+x)$ is not sufficient, since the addition $1+x$ loses significant digits of x when x is small. The following method is effective (for another approach, see Hull, Fairgrieve, and Tang [649, 1994]): calculate $w = 1+x$ and then compute

$$f(x) = \begin{cases} x, & w = 1, \\ \frac{x \log w}{w-1}, & w \neq 1. \end{cases} \quad (\text{A.1})$$

The explanation of why this method works is similar to that for the method in §1.14.1. We have $\hat{w} = (1+x)(1+\delta)$, $|\delta| \leq u$, and if $\hat{w} = 1$ then $|x| \leq u + u^2 + u^3 + \dots$, so from the Taylor series $f(x) = x(1 - x/2 + x^2/3 - \dots)$ we see that $f(x) = x$ is a correctly rounded result. If $\hat{w} \neq 1$ then

$$\hat{f} = \frac{(x \log \hat{w})(1+\epsilon_1)(1+\epsilon_2)}{(\hat{w}-1)(1+\epsilon_3)}(1+\epsilon_4), \quad |\epsilon_i| \leq u.$$

Defining $\hat{w} =: 1+z$,

$$g(\hat{w}) := \frac{\log \hat{w}}{\hat{w}-1} = \frac{\log(1+z)}{z} = 1 - z/2 + z^2/3 - \dots,$$

so if $\hat{w} \approx w \approx 1$ (thus $x \approx 0$) then

$$g(w) - g(\hat{w}) \approx \frac{\hat{w} - w}{2} = \frac{(1+x)\delta}{2} \approx \frac{\delta}{2} \approx \frac{g(w)\delta}{2}.$$

Thus, with $1+\theta := (1+\epsilon_1)(1+\epsilon_2)(1+\epsilon_3)^{-1}(1+\epsilon_4)$,

$$\hat{f} = xg(\hat{w})(1+\theta) \approx xg(w)(1+\theta) = f(x)(1+\theta),$$

showing that $\hat{f}$ is an accurate approximation to $f(x) = \log(1+x)$. Figure A.1 shows MATLAB plots of $\log(1+x)$, using MATLAB's built-in `log` function, and `log1p`, an M-file of Cleve Moler that implements (A.1). Close to the origin the lack of smoothness of the curve for the `log` evaluation is clear.

1.7. From (1.4) we have

$$\begin{aligned} (n-1)V(x+\Delta x) &= \sum_{i=1}^n \left(x_i + \Delta x_i - \bar{x} - n^{-1} \sum_{j=1}^n \Delta x_j \right)^2 \\ &= (n-1)V(x) + 2 \sum_{i=1}^n (x_i - \bar{x}) \left(\Delta x_i - n^{-1} \sum_{j=1}^n \Delta x_j \right) \\ &\quad + \sum_{i=1}^n \left(\Delta x_i - n^{-1} \sum_{j=1}^n \Delta x_j \right)^2 \\ &= (n-1)V(x) + 2 \sum_{j=1}^n (x_j - \bar{x}) \Delta x_j \\ &\quad + \sum_{i=1}^n \left(\Delta x_i - n^{-1} \sum_{j=1}^n \Delta x_j \right)^2. \end{aligned}$$

This yields the following inequality, which is attainable to first order:

$$(n-1)|V(x) - V(x+\Delta x)| \leq 2\epsilon \sum_{i=1}^n |x_i - \bar{x}| |x_i| + \epsilon^2 \sum_{i=1}^n \left(|x_i| + n^{-1} \sum_{j=1}^n |x_j| \right)^2.$$

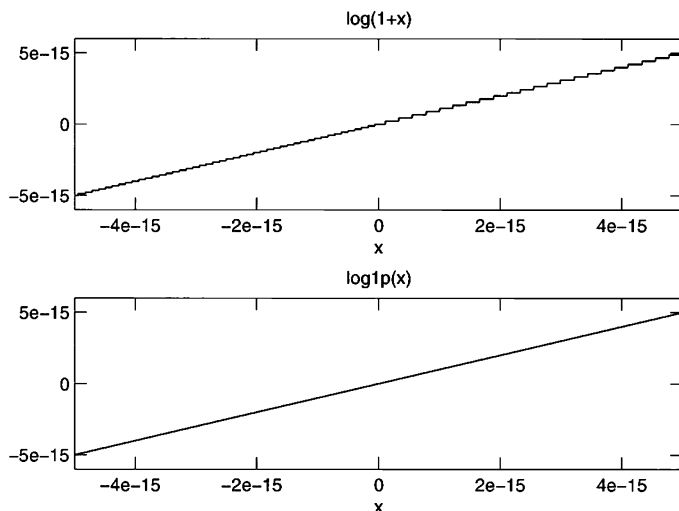


Figure A.1. $\log(1+x)$ evaluated using MATLAB's $\log$ (top) and using the formula (A.1) (bottom).

Dividing by $\epsilon(n-1)V(x)$ and taking the limit as $\epsilon \rightarrow 0$ gives the expression for κ_C . The result for κ_N follows from

$$\left| \sum_{j=1}^n (x_i - \bar{x}) \Delta x_i \right| \leq \left(\sum_{i=1}^n (x_i - \bar{x})^2 \right)^{1/2} \epsilon \|x\|_2 = \epsilon(n-1)^{1/2} V(x)^{1/2} \|x\|_2,$$

where the inequality is attainable (Cauchy-Schwarz), together with the relation $(n-1)V(x) = \|x\|_2^2 - n\bar{x}^2$. That $\kappa_N \geq \kappa_C$ is easily verified.

1.8. The general solution to the recurrence is

$$x_k = \frac{100^{k+1}a + 6^{k+1}b + 5^{k+1}c}{100^k a + 6^k b + 5^k c},$$

where a , b , and c are arbitrary constants. The particular starting values chosen yield $a = 0$, $b = c = 1$, so that

$$x_k = \frac{6^{k+1} + 5^{k+1}}{6^k + 5^k} = 6 - \frac{1}{1 + (6/5)^k}.$$

Rounding errors in the evaluation (and in the representation of x_1 on a binary machine) cause a nonzero “ a ” term to be introduced, and the computed values are approximately of the form

$$\hat{x}_k \approx \frac{100^{k+1}\eta + 6^{k+1} + 5^{k+1}}{100^k\eta + 6^k + 5^k},$$

for a constant η of order the unit roundoff. Hence the computed iterates rapidly converge to 100. Note that resorting to higher precision merely delays the inevitable convergence to 100. Priest [955, 1992, pp. 54–56] gives some interesting observations on the stability of the evaluation of the recurrence.

1.9. Writing

$$C := \text{adj}(A) = \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix},$$

we have

$$\begin{aligned} \hat{x} &= fl\left(\frac{1}{d}Cb\right) \\ &= \frac{1}{d}(C + \Delta C)b, \quad |\Delta C| \leq \gamma_3|C|, \\ &= x + \frac{1}{d}\Delta Cb. \end{aligned}$$

So

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_3 \frac{\|A^{-1}\| \|b\|_\infty}{\|x\|_\infty} \leq \gamma_3 \text{cond}(A, x).$$

Also, $r = b - A\hat{x} = -(1/d)A\Delta Cb$, so $|r| \leq \gamma_3|A||A^{-1}||b|$, which implies the normwise residual bound. Note that the residual bound shows that $\|r\|_\infty/(\|A\|_\infty\|x\|_\infty)$ will be small if x is a large-normed solution ($\|x\|_\infty \approx \|A^{-1}\|_\infty\|b\|_\infty$).

1.10. Let $\hat{m} = fl(\bar{x})$. For any standard summation method we have (see §4.2)

$$\begin{aligned} \hat{m} &= n^{-1} \sum_{i=1}^n x_i(1 + \delta_i), \quad |\delta_i| \leq \gamma_{n-1}, \\ &= m + \Delta m, \quad \Delta m := n^{-1} \sum_{i=1}^n x_i \delta_i. \end{aligned}$$

Then, since $fl((x_i - \hat{m})^2) = (x_i - \hat{m})^2(1 + \alpha_i)^2(1 + \beta_i)$, $|\alpha_i|, |\beta_i| \leq u$, $\hat{V}$ satisfies

$$\begin{aligned} (n-1)\hat{V} &= \sum_{i=1}^n (x_i - \hat{m})^2(1 + \epsilon_i), \quad |\epsilon_i| \leq \gamma_{n+3} \\ &= \sum_{i=1}^n [(x_i - m)^2 - 2\Delta m(x_i - m) + \Delta m^2](1 + \epsilon_i) \\ &= (n-1)V + \sum_{i=1}^n (x_i - m)^2 \epsilon_i \\ &\quad + \sum_{i=1}^n (-2\Delta m(x_i - m) + \Delta m^2)(1 + \epsilon_i). \end{aligned}$$

Hence, defining $\mu := n^{-1} \sum_{i=1}^n |x_i|$, so that $|\Delta m| \leq \gamma_{n-1}\mu$,

$$\begin{aligned} (n-1)|V - \hat{V}| &\leq \gamma_{n+3}(n-1)V + \sum_{i=1}^n |-2\Delta m(x_i - m)\epsilon_i + \Delta m^2(1 + \epsilon_i)| \\ &\leq \gamma_{n+3}(n-1)V + 2\gamma_{n-1}\gamma_{n+3}\mu\sqrt{n(n-1)V} + \gamma_{n-1}^2\mu^2n(1 + \gamma_{n+3}), \end{aligned}$$

that is,

$$\begin{aligned} \frac{|V - \hat{V}|}{V} &\leq \gamma_{n+3} + 2\theta\gamma_{n-1}\gamma_{n+3} + \theta^2\gamma_{n-1}^2(1 + \gamma_{n+3}) \\ &= (n+3)u + O(u^2), \end{aligned}$$

where

$$\theta = \mu\sqrt{\frac{n}{(n-1)V}} \leq \frac{\|x\|_2}{\sqrt{(n-1)V}} = \frac{1}{2}\kappa_N.$$

2.1. There are $1 + 2(e_{\max} - e_{\min} + 1)(\beta - 1)\beta^{t-1}$ normalized numbers (the “1” is for zero), and $2(\beta^{t-1} - 1)$ subnormal numbers. For IEEE arithmetic we therefore have

	Normalized	Subnormal
Single precision	4.3×10^9	1.7×10^7
Double precision	1.8×10^{19}	9×10^{15}

2.2. Without loss of generality suppose $x > 0$. We can write $x = m \times \beta^{e-t}$, where $\beta^{t-1} \leq m < \beta^t$. The next larger floating point number is $x + \Delta x$, where $\Delta x = \beta^{e-t}$, and

$$\beta^{-1}\epsilon_M|x| = \beta^{-t}|x| < \frac{|x|}{m} = |\Delta x| \leq \beta^{e-t}(\beta^{1-t}m) = \epsilon_M|x|.$$

The same upper bound clearly holds for the “next smaller” case, and the lower bound in this case is also easy to see.

2.3. The answer is the same for all adjacent nonzero pairs of single precision numbers. Suppose the numbers are 1 and $1 + \epsilon_M(\text{single}) = 1 + 2^{-23}$. The spacing of the double precision numbers on $[1, 2]$ is 2^{-52} , so the answer is $2^{29} - 1 \approx 5.4 \times 10^8$.

2.4. Inspecting the proof of Theorem 2.2 we see that $|y_i| \geq \beta^{e-1}$, $i = 1, 2$, and so we also have $|fl(x) - x|/|fl(x)| \leq u$, that is, $fl(x) = x/(1 + \delta)$, $|\delta| \leq u$. Note that this is not the same δ as in Theorem 2.2, although it has the same bound, and unlike in Theorem 2.2 there can be equality in this bound for δ .

2.5. The first part is trivial. Since, in binary notation,

$$x = 0.1100\ 1100\ 1100\ 1100\ 1100\ 1100 \mid 1100 \dots \times 2^{-3},$$

we have, rounding to 24 bits,

$$\hat{x} = 0.1100\ 1100\ 1100\ 1100\ 1100\ 1101 \times 2^{-3}.$$

Thus

$$\begin{aligned} \hat{x} - x &= (0.001 - 0.000\overline{1100})_2 \times 2^{-21} \times 2^{-3} \\ &= \left(\frac{1}{8} - \frac{1}{10}\right) \times 2^{-24} = \frac{1}{40} \times 2^{-24}, \end{aligned}$$

and so $(x - \hat{x})/x = -\frac{1}{4} \times 2^{-24} = -\frac{1}{4}u$.

2.6. Since the double precision significand contains 53 bits, $p = 2^{53} = 9007199254740992 \approx 9.01 \times 10^{15}$. For single precision, $p = 2^{24} = 16777216 \approx 1.68 \times 10^7$.

2.7.

1. True, since in IEEE arithmetic $fl(a \text{ op } b)$ is defined to be the rounded value of $a \text{ op } b$, $\text{round}(a \text{ op } b)$, which is the same as $\text{round}(b \text{ op } a)$.
2. True for round to nearest (the default) and round to zero, but false for round to $\pm\infty$.
3. True, because $fl(a + a) := \text{round}(a + a) = \text{round}(2 * a) =: fl(2 * a)$.
4. True: similar to 3.
5. False, in general.

6. True for binary arithmetic. Since the division by 2 is exact, the inequality is equivalent to $2a \leq fl(a+b) \leq 2b$. But $a+b \leq 2b$, so, by the monotonicity of rounding, $fl(a+b) = \text{round}(a+b) \leq \text{round}(2b) = 2b$. The lower bound is verified similarly.

2.8. Examples in 3-digit decimal arithmetic are $fl((5.01 + 5.02)/2) = fl(10.0/2) = 5.0$ and $fl((5.02 + 5.04)/2) = fl(10.1/2) = 5.05$.

The left-hand inequality is immediate, since $fl((b-a)/2) \geq 0$. The right-hand inequality can possibly be violated only if a and b are close (otherwise $(a+b)/2$ is safely less than b , so the inequality will hold). But then, by Theorem 2.5, $fl(b-a)$ is obtained exactly, and the result will have several zero low-order digits in the significand because of cancellation. Consequently, if the base is even the division is done exactly, and the right-hand inequality follows by the monotonicity of rounding. Alternatively, and even more simply, for any base we can argue that $fl((b-a)/2) \leq b-a$, so that $a + fl((b-a)/2) \leq b$ and, again, the result follows by the monotonicity of rounding.

2.9.

$$\begin{aligned}\sqrt{1-2^{-53}} &= 1 - \frac{1}{2} \cdot 2^{-53} - \frac{1}{8} \cdot 2^{-106} - \dots \\ &= 0.\underbrace{11\dots 10}_{53}\underbrace{11\dots 10}_{54}11\dots \quad (\text{binary}).\end{aligned}$$

Rounded directly to 53 bits this yields $1 - 2^{-53}$. But rounded first to 64 bits it yields

$$0.\underbrace{11\dots 11}_{53}\underbrace{00\dots 0}_{10},$$

and when this number is rounded to 53 bits using the round to even rule it yields 1.0.

2.12. The spacing between the floating point numbers in the interval $(1/2, 1]$ is $\epsilon/2$ (cf. Lemma 2.1), so $|1/x - fl(1/x)| \leq \epsilon/4$, which implies that $|1 - xfl(1/x)| \leq \epsilon x/4 < \epsilon/2$. Thus

$$1 - \epsilon/2 < xfl(1/x) < 1 + \epsilon/2.$$

Since the spacing of the floating point numbers just to the right of 1 is ϵ , $xfl(1/x)$ must round to either $1 - \epsilon/2$ or 1.

2.13. If there is no double rounding the answer is 257, 736, 490. For a proof that combines mathematical analysis with computer searching, see Edelman [379, 1994].

2.15. The IEEE standard does not define the results of exponentiation. The choice $0^0 = 1$ can be justified in several ways. For example, if $p(x) = \sum_{j=0}^n a_j x^j$ then $p(0) = a_0 = a_0 \times 0^0$, and the binomial expansion $(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$ yields $1 = 0^0$ for $x = 0$, $y = 1$. For more detailed discussions see Goldberg [496, 1991, p. 32] and Knuth [742, 1992, pp. 406–408].

2.17. If the rounding mode is round to zero or round to $-\infty$ then $fl(2x_{\max}) = x_{\max}$.

2.18. No. A counterexample in 2-digit base 10 arithmetic is $fl(2.0 - 0.91) = fl(1.09) = 1.1$.

2.19. Every floating point number (whether normalized or unnormalized) is a multiple of $\mu = \beta^{e_{\min}-t}$ (although the converse is not true), therefore $x \pm y$ is also a multiple of μ . The subnormal numbers are equally spaced between μ and $\beta^{e_{\min}-1}$, the smallest positive normalized floating point number. But since $fl(x \pm y)$ underflows, $|x \pm y| < \beta^{e_{\min}-1}$, and therefore $x \pm y$ is exactly representable as a subnormal number. Thus $fl(x \pm y) = x \pm y$.

2.20. The function $fl(\sqrt{x})$ maps the set of positive floating point numbers onto a set of floating point numbers with about half as many elements. Hence there exist two distinct floating point numbers x having the same value of $fl(\sqrt{x})$, and so the condition $(\sqrt{x})^2 = |x|$ cannot always be satisfied in floating point arithmetic. The requirement $\sqrt{x^2} = |x|$ is reasonable for base 2, however, and is satisfied in IEEE arithmetic, as we now show.

Without loss of generality, we can assume that $1 \leq x < 2$, since scaling x by a power of 2 does not alter $x/fl(\sqrt{x^2})$. By definition, $fl(\sqrt{x^2})$ is the nearest floating point number to $\sqrt{fl(x^2)} = \sqrt{x^2(1+\delta)}$, $|\delta| \leq u$, and

$$\sqrt{fl(x^2)} = |x|(1+\delta)^{1/2} = |x| \left(1 + \frac{\delta}{\sqrt{1+\delta}+1} \right) = |x| + \theta.$$

Now the spacing of the floating point numbers between 1 and 2 is $\epsilon_M = 2u$, so

$$|\theta| \leq (2 - 2u) \frac{u}{\sqrt{1-u}+1}.$$

Hence $|\theta| < u$ if $u \leq 1/4$ (say), and then $|x|$ is the nearest floating point number to $\sqrt{fl(x^2)}$, so that $fl(\sqrt{fl(x^2)}) = |x|$.

In base 10 floating point arithmetic, the condition $\sqrt{x^2} = |x|$ can be violated. For example, working to 5 significant decimal digits, if $x = 3.1625$ then $fl(x^2) = fl(10.0014\ 0625) = 10.001$, and $fl(\sqrt{10.001}) = fl(3.1624\ 3577\dots) = 3.1624 < x$.

2.21. On a Cray Y-MP the answer is yes, but in base 2 IEEE arithmetic the answer is no. It suffices to demonstrate that $fl(x/\sqrt{x^2}) = \text{sign}(x)$ ($x \neq 0$), which is shown by the proof of Problem 2.20.

2.22. The test “ $x > y$ ” returns false if x or y is a NaN, so the code computes $\max(\text{NaN}, 5) = 5$ and $\max(5, \text{NaN}) = \text{NaN}$, which violates the obvious requirement that $\max(x, y) = \max(y, x)$. Since the test $x \neq x$ identifies a NaN, the following code implements a reasonable definition of $\max(x, y)$:

```
% max(x, y)
if x ≠ x then
    max = y
else if y ≠ y then
    max = x
else if y > x then
    max = y
else
    max = x
end
end
```

A further refinement is to ensure that $\max(-0, +0) = +0$, which is not satisfied by the code above since -0 and $+0$ compare as equal; this requires bit-level programming.

2.23. We give an informal proof; the details are obtained by using the model $fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta)$, but they obscure the argument.

Since a , b , and c are nonnegative, $a + (b + c)$ is computed accurately. Since $c \leq b \leq a$, $c + (a - b)$ and $a + (b - c)$ are the sums of two positive numbers and so are computed

accurately. Since a , b , and c are the lengths of sides of a triangle, $a \leq b + c$; hence, using $c \leq b \leq a$,

$$b \leq a \leq b + c \leq 2b,$$

which implies that $a - b$ is computed exactly, by Theorem 2.5. Hence $c - (a - b)$ is the difference of two exactly represented numbers and so is computed accurately. Thus $fl(A) = fl(\frac{1}{4}\sqrt{\hat{x}})$, where $\hat{x}$ is an accurate approximation to the desired argument x of the square root. It follows that $fl(A)$ is accurate.

2.24. For a machine with a guard digit, $y = x$, by Theorem 2.5 (assuming $2x$ does not overflow). If the machine lacks a guard digit then the subtraction produces x if the last bit of x is zero, otherwise it produces an adjacent floating point number with a zero last bit; in either case the result has a zero last bit. Gu, Demmel, and Dhillon [528, 1994] apply this bit zeroing technique to numbers $d_1, d_2, \dots, d_n$ arising in a divide and conquer bidiagonal SVD algorithm, their motivation being that the differences $d_i - d_j$ can then be computed accurately even on machines without a guard digit.

2.25. The function $f(x) = 3x - 1$ has the single root $x = 1/3$. We can have $fl(f(x)) = 0$ only for $x \approx 1/3$. For $x \approx 1/3$ the evaluation can be summarized as follows:

$$\begin{aligned} x - 0.5 &\approx -\frac{1}{6}, \\ -\frac{1}{6} + x &\approx \frac{1}{6}, \\ \frac{1}{6} - 0.5 &\approx -\frac{1}{3}, \\ -\frac{1}{3} + x &\approx 0. \end{aligned}$$

The first, second, and fourth subtractions are done exactly, by Theorem 2.5. The result of the first subtraction has a zero least-significant bit and the result of the second has two zero least-significant bits; consequently the third subtraction suffers no loss of trailing bits and is done exactly. Therefore $f(x)$ is computed exactly for $x \equiv fl(x)$ near $1/3$. But $fl(x)$ can never equal $1/3$ on a binary machine, so $fl(f(x)) \neq 0$ for all x .

2.26. If x and y are t -digit floating point numbers then xy has $2t - 1$ or $2t$ significant digits. Since the error $fl(xy) - xy$ is less than one ulp in xy (i.e., beyond the t th significant digit), the difference $xy - fl(xy)$ is exactly representable in t digits, and it follows from the definition of FMA that the difference is computed exactly.

2.27. We have

$$\begin{aligned} \hat{w} &= bc(1 + \delta_1), \\ \hat{e} &= (\hat{w} - bc)(1 + \delta_2), \\ (1 + \delta_4)\hat{x} &= (ad - \hat{w})(1 + \delta_3) + \hat{e}, \end{aligned}$$

where $|\delta_i| \leq u$, $i = 1:4$. Hence

$$(1 + \delta_4)\hat{x} = (ad - bc - bc\delta_1)(1 + \delta_3) + bc\delta_1(1 + \delta_2) = x + x\delta_3 - bc\delta_1(\delta_3 - \delta_2),$$

so that

$$|x - \hat{x}| \leq u(|x| + |\hat{x}|) + 2u^2|bc|,$$

which implies high relative accuracy unless $u|bc| \gg |x|$. For comparison, the bound for standard evaluation of $fl(ad - bc)$ is $|x - \hat{x}| \leq \gamma_2(|ad| + |bc|)$.

2.28. We would like (2.8) to be satisfied, that is, we want $\hat{z} = fl(x/y)$ to satisfy

$$\hat{z} = (x/y)(1 + \delta) + \eta, \quad |\delta| \leq u, \quad |\eta| \leq \lambda u. \quad (\text{A.2})$$

This implies that

$$|\hat{z}y - x| = |x\delta + \eta y| \leq u|x| + \lambda u|y|.$$

In place of $r = \hat{z}y - x$ we have to use $\hat{r} = fl(\hat{z}y - x) = \hat{z}y(1 + \epsilon) - x$, where $|\epsilon| \leq u$, and where we can assume the subtraction is exact, since the case of interest is where $\hat{z}y \approx x$ (see Theorem 2.5). Thus $|\hat{r}| = |r + \hat{z}y\epsilon| \leq |r| + u|\hat{z}||y|$, and bounding $|\hat{z}||y|$ using (A.2) we obtain

$$|\hat{r}| \leq |r| + u(1 + u)|x| + \lambda u^2|y| \approx |r| + u|x|.$$

Therefore the convergence test is

$$|\hat{r}| \leq 2u|x| + \lambda u|y|.$$

Since λu underflows to zero it cannot be precomputed, and we should instead compute $\lambda(u|y|)$.

3.1. The proof is by induction. Inductive step: for $\rho_n = +1$,

$$\begin{aligned} \prod_{i=1}^n (1 + \delta_i) &= (1 + \delta_n)(1 + \theta_{n-1}) = 1 + \theta_n, \\ \theta_n &= \delta_n + (1 + \delta_n)\theta_{n-1}, \\ |\theta_n| &\leq u + (1 + u) \frac{(n-1)u}{1 - (n-1)u} \\ &= \frac{u(1 - (n-1)u) + (1 + u)(n-1)u}{1 - (n-1)u} \\ &= \frac{nu}{1 - (n-1)u} \leq \gamma_n. \end{aligned}$$

For $\rho_n = -1$ we find, similarly, that

$$|\theta_n| \leq \frac{nu - (n-1)u^2}{1 - nu + (n-1)u^2} \leq \gamma_n.$$

3.2. This result can be proved by a modification of the proof of Lemma 3.1. But it follows immediately from the penultimate line of the proof of Lemma 3.4.

3.3. The computed iterates $\hat{q}_k$ satisfy

$$(1 + \delta_k)\hat{q}_k = a_k + \frac{b_k}{\hat{q}_{k+1}}(1 + \epsilon_k), \quad |\delta_k|, |\epsilon_k| \leq u.$$

Defining $\hat{q}_k = q_k + e_k$, we have

$$\begin{aligned} q_k + e_k + \delta_k \hat{q}_k &= a_k + \frac{b_k}{q_{k+1} + e_{k+1}} + \frac{b_k \epsilon_k}{\hat{q}_{k+1}} \\ &= a_k + \frac{b_k}{q_{k+1}} - \frac{b_k e_{k+1}}{q_{k+1}(q_{k+1} + e_{k+1})} + \frac{b_k \epsilon_k}{\hat{q}_{k+1}}. \end{aligned}$$

This gives

$$|e_k| \leq u|\hat{q}_k| + \frac{|b_k||e_{k+1}|}{(|\hat{q}_{k+1}| - |e_{k+1}|)|\hat{q}_{k+1}|} + u \frac{|b_k|}{|\hat{q}_{k+1}|}.$$

The running error bound μ can therefore be computed along with the continued fraction as follows:


```

 $q_{n+1} = a_{n+1}$ 
 $f_{n+1} = 0$ 
for  $k = n:-1:0$ 
     $r_k = b_k/q_{k+1}$ 
     $q_k = a_k + r_k$ 
     $f_k = |q_k| + |r_k| + |b_k|f_{k+1}/(|q_{k+1}| - uf_{k+1})|q_{k+1}|$ 
end
 $\mu = uf_0$ 

```

The error bound is valid provided that $|q_{k+1}| - uf_{k+1} > 0$ for all k . Otherwise a more sophisticated approach is necessary (for example, to handle the case where $q_{k+1} = 0$, $q_k = \infty$, and q_{k-1} is finite).

3.4. We prove just the division result and the last result. Let $(1 + \theta_k)/(1 + \theta_j) = 1 + \alpha$. Then $\alpha = (\theta_k - \theta_j)/(1 + \theta_j)$, so

$$\begin{aligned}
 |\alpha| &\leq \frac{\frac{ku}{1-ku} + \frac{ju}{1-ju}}{1 - \frac{ju}{1-ju}} = \frac{(k+j)u - 2kju^2}{(1-2ju)(1-ku)} \\
 &= \frac{(k+j)u - 2kju^2}{1 - (2j+k)u + 2jku^2} \leq \frac{(k+j)u}{1 - (2j+k)u} \leq \gamma_{k+2j}.
 \end{aligned}$$

However, it is easy to verify that, in fact,

$$\frac{(k+j)u - 2kju^2}{(1-2ju)(1-ku)} \leq \frac{(k+j)u}{1 - (k+j)u}$$

if $j \leq k$.

For the last result,

$$\begin{aligned}
 \gamma_k + \gamma_j + \gamma_k \gamma_j &= \frac{ku(1-ju) + ju(1-ku) + kju^2}{(1-ku)(1-ju)} \\
 &= \frac{(k+j)u - kju^2}{1 - (k+j)u + kju^2} \\
 &< \frac{(k+j)u}{1 - (k+j)u} = \gamma_{k+j}.
 \end{aligned}$$

3.5. We have $fl(AB) = AB + \Delta C = (A + \Delta CB^{-1})B =: (A + \Delta A)B$, where $\Delta A = \Delta CB^{-1}$, and $|\Delta C| \leq \gamma_n |A| |B|$ by (3.13). Similarly, $fl(AB) = A(B + \Delta B)$, $|\Delta B| \leq \gamma_n |A^{-1}| |A| |B|$.

3.6. We have

$$|R| = |A\Delta B + \Delta AB + \Delta A\Delta B| \leq 2\epsilon EF + \epsilon^2 EF,$$

which implies

$$\epsilon^2 g_{ij} + 2\epsilon g_{ij} - |r_{ij}| \geq 0 \quad \text{for all } i, j.$$

Solving these inequalities for ϵ gives the required lower bound.

The definition is flawed in general as can be seen from rank considerations. For example if A and B are vectors and $\text{rank}(C) \neq 1$, then no perturbations ΔA and ΔB exist to satisfy $C = (A + \Delta A)(B + \Delta B)!$ Thus we have to use a mixed forward/backward stability definition in which we perturb C by at most ϵ as well as A and B .

3.7. Lemma 3.5 shows that (3.2) holds provided that each $(1 + \delta)^k$ product is replaced by $(1 + \epsilon)(1 + \delta)^{k-1}$, where $|\epsilon| \leq \sqrt{2}\gamma_2$, where the $1 + \epsilon$ corresponds to a multiplication $x_i y_i$. Thus we have

$$\widehat{S}_n = \sum_{i=1}^n x_i y_i (1 + \alpha_i), \quad \text{where} \quad 1 + \alpha_i = (1 + \theta_{n-1}^{(i)})(1 + \epsilon_i).$$

It is easy to show that $|\alpha_i| \leq \gamma_{n+2}$, so the only change required in (3.4) is to replace γ_n by γ_{n+2} . The complex analogue of (3.11) is $\widehat{y} = (A + \Delta A)x$, $|\Delta A| \leq \gamma_{n+2}|A|$.

3.8. We have $fl((x + y)(x - y)) = (x + y)(x - y)(1 + \theta_3)$, $|\theta_3| \leq \gamma_3 = 3u/(1 - 3u)$, so the computed result has small relative error. Moreover, if $y/2 \leq x \leq y$ then $x - y$ is computed exactly, by Theorem 2.5, hence $fl((x + y)(x - y)) = (x + y)(x - y)(1 + \theta_2)$. However, $fl(x^2 - y^2) = x^2(1 + \theta_2) - y^2(1 + \theta'_2)$, so that

$$\left| \frac{fl(x^2 - y^2) - (x^2 - y^2)}{x^2 - y^2} \right| = \left| \frac{x^2\theta_2 - y^2\theta'_2}{x^2 - y^2} \right| \leq \gamma_2 \frac{x^2 + y^2}{|x^2 - y^2|},$$

and we cannot guarantee a small relative error.

If $|x| \gg |y|$ then $fl(x^2 - y^2)$ suffers only two rounding errors, since the error in forming $fl(y^2)$ will not affect the final result, while $fl((x + y)(x - y))$ suffers three rounding errors; in this case $fl(x^2 - y^2)$ is likely to be the more accurate result.

3.9. Assume the result is true for $m - 1$. Now

$$\prod_{j=0}^m (X_j + \Delta X_j) = (X_m + \Delta X_m) \prod_{j=0}^{m-1} (X_j + \Delta X_j),$$

so

$$\begin{aligned} \left\| \prod_{j=0}^m (X_j + \Delta X_j) - \prod_{j=0}^m X_j \right\| &= \left\| X_m \left[\prod_{j=0}^{m-1} (X_j + \Delta X_j) - \prod_{j=0}^{m-1} X_j \right] \right. \\ &\quad \left. + \Delta X_m \prod_{j=0}^{m-1} (X_j + \Delta X_j) \right\| \\ &\leq \|X_m\| \left(\prod_{j=0}^{m-1} (1 + \delta_j) - 1 \right) \prod_{j=0}^{m-1} \|X_j\| \\ &\quad + \delta_m \|X_m\|_2 \prod_{j=0}^{m-1} (\|X_j\| (1 + \delta_j)) \\ &= \left(\prod_{j=0}^m (1 + \delta_j) - 1 \right) \prod_{j=0}^m \|X_j\|. \end{aligned}$$

3.10. Without loss of generality we can suppose that the columns of the product are computed one at a time. With $x_j = A_1 \dots A_k e_j$ we have, using (3.11),

$$fl(x_j) = (A_1 + \Delta A_1) \dots (A_k + \Delta A_k) e_j, \quad \|\Delta A_i\|_2 \leq \sqrt{n} \gamma_n \|A_i\|_2,$$

and so, by Lemma 3.6,

$$\|x_j - fl(x_j)\|_2 \leq [(1 + \sqrt{n} \gamma_n)^k - 1] \|A_1\|_2 \dots \|A_k\|_2.$$

Squaring these inequalities and summing over j yields

$$\|A_1 \dots A_k - fl(A_1 \dots A_k)\|_F \leq \sqrt{n} [(1 + \sqrt{n} \gamma_n)^k - 1] \|A_1\|_2 \dots \|A_k\|_2,$$

which gives the result.

3.11. The computations can be expressed as

$$\begin{aligned} y_1 &= x^2 \\ y_{i+1} &= \sqrt{y_i}, \quad i = 1:m \\ z_1 &= y_{m+1} \\ z_{i+1} &= z_i^2, \quad i = 1:m-1 \end{aligned}$$

We have

$$\begin{aligned} \hat{y}_1 &= x^2(1 + \delta_0) \\ \hat{y}_{i+1} &= \sqrt{\hat{y}_i}(1 + \delta_i), \quad i = 1:m \\ \hat{z}_1 &= \hat{y}_{m+1} \\ \hat{z}_{i+1} &= \hat{z}_i^2(1 + \epsilon_i), \quad i = 1:m-1 \end{aligned}$$

where $|\delta_i|, |\epsilon_i| \leq u$. Solving these recurrences, we find that

$$\hat{y}_{m+1} = (1 + \delta_m)(1 + \delta_{m-1})^{1/2}(1 + \delta_{m-2})^{1/4} \dots (1 + \delta_0)^{1/2^m} y_1^{1/2^m}.$$

It follows that

$$(1 - u)^2 y_1^{\frac{1}{2^m}} \leq \hat{y}_{m+1} \leq (1 + u)^2 y_1^{\frac{1}{2^m}},$$

which shows that $\hat{y}_{m+1}$ differs negligibly from y_{m+1} . For the repeated squarings, however, we find that

$$\begin{aligned} \hat{z}_m &= (1 + \epsilon_{m-1})(1 + \epsilon_{m-2})^2(1 + \epsilon_{m-3})^4 \dots (1 + \epsilon_1)^{2^{m-1}} \hat{z}_1^{2^{m-1}} \\ &= (1 + \theta_{2^m-1}) \hat{z}_1^{2^{m-1}} = (1 + \theta_{2^m-1}) |x|, \end{aligned}$$

where we have used Lemma 3.1. Hence the squarings introduce a relative error that can be approximately as large as $2^m u$. Since $u = 2^{-53}$, this relative error is of order 0.1 for $m = 50$, which explains the observed results for $m = 50$.

3.12. The analysis is just a slight extension of that for an inner product. The analogue of (3.3) is

$$\begin{aligned} \hat{J}(f) &= w_1 f(x_1)(1 + \eta_1)(1 + \theta_n) + w_2 f(x_2)(1 + \eta_2)(1 + \theta'_n) \\ &\quad + w_3 f(x_3)(1 + \eta_3)(1 + \theta_{n-1}) + \dots + w_n f(x_n)(1 + \eta_n)(1 + \theta_2). \end{aligned}$$

Hence

$$|I(f) - \hat{J}(f)| \leq (\eta + \gamma_n + \eta\gamma_n) \sum_{i=1}^n |w_i| |f(x_i)|.$$

Setting $M = \max\{|f(x)| : a \leq x \leq b\}$, we have

$$|I(f) - \hat{J}(f)| \leq M(\eta + \gamma_n + \eta\gamma_n) \sum_{i=1}^n |w_i|. \quad (\text{A.3})$$

Any reasonable quadrature rule designed for polynomial f has $\sum_{i=1}^n w_i = b - a$, so one implication of (A.3) is that it is best not to have weights of large magnitude and varying sign; ideally, $w_i \geq 0$ for all i (as for Gaussian integration rules, for example), so that $\sum_{i=1}^n |w_i| = b - a$.

4.1. A condition number is

$$C(x) = \max \left\{ \left| \frac{S_n(x) - S_n(x + \Delta x)}{\epsilon S_n(x)} \right| : |\Delta x| \leq \epsilon |x| \right\}.$$

It is easy to show that

$$C(x) = \frac{S_n(|x|)}{|S_n(x)|} = \frac{\sum_{i=1}^n |x_i|}{|\sum_{i=1}^n x_i|}.$$

The condition number is 1 if the x_i all have the same sign.

4.2. In the $(i-1)$ st floating point addition the “ 2^{k-t} ” portion of x_i does not propagate into the sum (assuming that the floating point arithmetic uses round to nearest with ties broken by rounding to an even last bit or rounding away from zero), thus there is an error of 2^{k-t} and $\widehat{S}_i = i$. The total error is

$$2^{-t}(1 + 2^2 + 2^4 + \cdots + 2^{2(r-1)}) = 2^{-t} \frac{4^r - 1}{3},$$

while the upper bound of (4.4) is

$$(n-1)u \sum_{i=1}^n |x_i| + O(u^2) \leq 2^r 2^{-t} 2^r + O(2^{-2t}) \approx 2^{-t} 4^r,$$

which agrees with the actual error to within a factor 3; thus the smaller upper bound of (4.3) is also correct to within this factor. The example just quoted is, of course, a very special one, and as Wilkinson [1232, 1963, p. 20] explains, “in order to approach the upper bound as closely as this, not only must each error take its maximum value, but all the terms must be almost equal.”

4.3. With $S_k = \sum_{i=1}^k x_i$, we have

$$\widehat{S}_k = fl(\widehat{S}_{k-1} + x_k) = (\widehat{S}_{k-1} + x_k)(1 + \delta_k), \quad |\delta_k| \leq u, \quad k = 2:n.$$

By repeated use of this relation it follows that

$$\widehat{S}_n = (x_1 + x_2) \prod_{k=2}^n (1 + \delta_k) + \sum_{i=3}^n x_i \prod_{k=i}^n (1 + \delta_k),$$

which yields the required expression for $\widehat{S}_n$. The bound on $|E_n|$ is immediate.

The bound is minimized if the x_i are in increasing order of absolute value. This observation is common in the literature and it is sometimes used to conclude that the increasing ordering is the best one to use. This reasoning is fallacious, because minimizing an error bound is not the same as minimizing the error itself. As (4.3) shows, if we know nothing about the signs of the rounding errors then the “best” ordering to choose is one that minimizes the partial sums.

4.4. Any integer between 0 and 10 inclusive can be produced. For example, $fl(1 + 2 + 3 + 4 + M - M) = 0$, $fl(M - M + 1 + 2 + 3 + 4) = 10$, and $fl(2 + 3 + M - M + 1 + 4) = 5$.

4.5. This method is sometimes promoted on the basis of the argument that it minimizes the amount of cancellation in the computation of S_n . This is incorrect: the “ $\pm$ ” method does not reduce the amount of cancellation—it simply concentrates all the cancellation into one step. Moreover, cancellation is not a bad thing per se, as explained in §1.7.

The “ $\pm$ ” method is an instance of Algorithm 4.1 (assuming that S_+ and S_- are computed using Algorithm 4.1) and it is easy to see that it maximizes $\max_i |T_i|$ over all methods of this form (where, as in §4.2, T_i is the sum computed at the i th stage). Moreover, when $\sum_{i=1}^n |x_i| \gg |\sum_{i=1}^n x_i|$ the value of $\max_i |T_i|$ tends to be much larger for the “ $\pm$ ” method than for the other methods we have considered.

4.6. Suppose, without loss of generality, that $|a| \geq |b|$. By definition, $fl(a + b)$ solves $\min |(a + b) - x|$ over all floating point numbers x . Since a is a floating point number,

$$\text{err}(a, b) \leq |(a + b) - a| = |b|,$$

which gives the first part. Now the least significant nonzero bit of $\text{err}(a, b)$ is no smaller than 1 ulp in b (cf. Figure 4.1), and by the first part, $|\text{err}(a, b)| \leq |b|$. Hence the significand of $\text{err}(a, b)$ is no longer than that of b , which means that it is representable as a floating point number in the absence of overflow.

4.7. The main concern is to evaluate the denominator accurately when the x_i are close to convergence. The bound (4.3) tells us to minimize the partial sums; these are, approximately, for $x_i \approx \xi$, (a) $-\xi$, 0, (b) 0, 0, (c) 2ξ , 0. Hence the error analysis of summation suggests that (b) is the best expression, with (a) a distant second. That (b) is the best choice is confirmed by Theorem 2.5, which shows there will be only one rounding error when the x_i are close to convergence. A further reason to prefer (b) is that it is less prone to overflow than (a) and (c).

4.8. This is, of course, not a practical method, not least because it is very prone to overflow and underflow. However, its error analysis is interesting. Ignoring the error in the log evaluation, and assuming that $\exp$ is evaluated with relative error bounded by u , we have, with $|\delta_i| \leq u$ for all i , and for some δ_{2n}

$$\begin{aligned}\hat{S}_n &= \log(e^{x_1} \dots e^{x_n} (1 + \delta_1) \dots (1 + \delta_{2n-1})) \\ &= \log(e^{x_1} \dots e^{x_n} (1 + \delta_{2n})^{2n-1}) \\ &= x_1 + \dots + x_n + (2n-1) \log(1 + \delta_{2n}) \\ &\approx x_1 + \dots + x_n + (2n-1) \delta_{2n}.\end{aligned}$$

Hence the best relative error bound we can obtain is $|S_n - \hat{S}_n|/|S_n| \lesssim (2n-1)u/|S_n|$. Clearly, this method of evaluation guarantees a small absolute error, but not a small relative error when $|S_n| \ll 1$.

4.9. Method (a) is recursive summation of $a, h, h, \dots, h$. From (4.3) we have $|a + i\hat{h} - \hat{x}_i| \leq [ia + \frac{1}{2}i(i+1)\hat{h}]u + O(u^2)$. Hence, since $\hat{h} = h(1 + \epsilon)$, $|\epsilon| \leq u$,

$$\begin{aligned}|x_i - \hat{x}_i| &\leq |ih - i\hat{h}| + |a + i\hat{h} - \hat{x}_i| \leq ihu + i(|a| + \frac{1}{2}(i+1)h)u + O(u^2) \\ &= i(|a| + \frac{1}{2}(i+3)h)u + O(u^2).\end{aligned}$$

For (b), using the relative error counter notation (3.10), $\hat{x}_i = (a + i\hat{h}\langle 1 \rangle)\langle 1 \rangle = a\langle 1 \rangle + ih\langle 3 \rangle$. Hence

$$|x_i - \hat{x}_i| \leq u|a| + \gamma_3 ih.$$

For (c), $\hat{x}_i = a(1 - \langle 1 \rangle i/n)\langle 2 \rangle + (i/n)b\langle 3 \rangle$, hence

$$|x_i - \hat{x}_i| \leq \gamma_2|a| + \gamma_3|a|i/n + \gamma_3|b|i/n \leq \gamma_3(|a|(1 + i/n) + |b|i/n).$$

The error bound for (b) is about a factor i smaller than that for (a). Note that method (c) is the only one guaranteed to yield $\hat{x}_n = b$ (assuming $fl(n/n) = 1$, as holds in IEEE arithmetic), which may be important when integrating a differential equation to a given end-point.

If $a \geq 0$ then the bounds imply

$$\begin{aligned}(a): \quad |x_i - \hat{x}_i| &\lesssim iu|x_i|, \\ (b): \quad |x_i - \hat{x}_i| &\leq \gamma_3|x_i|, \\ (c): \quad |x_i - \hat{x}_i| &\leq \gamma_3(a + i/n(a+b)) = \gamma_3(a + i/n(b-a+2a)) \\ &= \gamma_3(a + ih + 2ai/n) \leq 3\gamma_3|x_i|.\end{aligned}$$

Thus (b) and (c) provide high relative accuracy for all i , while the relative accuracy of (a) can be expected to degrade as i increases.

5.1. By differentiating the Horner recurrence $q_i = xq_{i+1} + a_i$, $q_n = a_n$, we obtain

$$\begin{aligned} q'_i &= xq'_{i+1} + q_{i+1}, & q'_n &= 0, & q'_{n-1} &= a_n, \\ q''_i &= xq''_{i+1} + 2q'_{i+1}, & q''_n &= q''_{n-1} = 0, & q''_{n-2} &= 2a_n, \\ q'''_i &= xq'''_{i+1} + 3q''_{i+1}, & q'''_n &= q'''_{n-1} = q'''_{n-2} = 0, & q'''_{n-3} &= 6a_n. \end{aligned}$$

The factors 2, 3, ..., can be removed by redefining $r_i^{(k)} = q_i^{(k)}/k!$. Then $r_i^{(k)} = xr_{i+1}^{(k)} + r_{i+1}^{(k-1)}$, $r_{n-k}^{(k)} = a_n$.

5.2. Analysis similar to that for Horner's rule shows that

$$fl(p(x)) = a_0\langle n \rangle + a_1x\langle n+1 \rangle + \cdots + a_nx^n\langle n+1 \rangle.$$

The total number of rounding errors is the same as for Horner's algorithm, but they are distributed more equally among the terms of the polynomial. Horner's rule can be expected to be more accurate when the terms $|a_i x^i|$ decrease rapidly with i , such as when $p(x)$ is the truncation of a rapidly convergent power series. Of course, this algorithm requires twice as many multiplications as Horner's method.

5.3. Accounting for the error in forming y , we have, using the relative error counter notation (3.10),

$$\begin{aligned} fl(p(x)) &= [a_0\langle 1 \rangle + a_2x^2\langle 4 \rangle + \cdots + a_{n-2}x^{n-2}\langle 3m-2 \rangle + a_nx^n\langle 3m \rangle \\ &\quad + x\langle 1 \rangle(a_1\langle 1 \rangle + a_3x^2\langle 4 \rangle + \cdots + a_{2n-1}x^{2n-1}\langle 3m-3 \rangle)]\langle 1 \rangle \\ &= \sum_{i=0}^n a_i x^i (1 + \theta_{3m+1}^{(i)}), \quad |\theta_{3m+1}^{(i)}| \leq \gamma_{3m+1}. \end{aligned}$$

Thus the relative backward perturbations are bounded by $(3n/2+1)u$ instead of $2nu$ for Horner's rule.

5.4. Here is a MATLAB M-file to perform the task.

```
function [a, perm] = leja(a)
%LEJA    LEJA ordering.
%        [A, PERM] = LEJA(A) reorders the points A by the
%        Leja ordering and returns the permutation vector that
%        effects the ordering in PERM.

n = max(size(a));
perm = (1:n)';

% a(1) = max(abs(a)).
[t, i] = max(abs(a));
if i ~= 1
    a([1 i]) = a([i 1]);
    perm([1 i]) = perm([i 1]);
end

p = ones(n,1);
for k=2:n-1
    for i=k:n
```

```

    p(i) = p(i)*(a(i)-a(k-1));
end
[t, i] = max(abs(p(k:n)));
i = i+k-1;
if i ~= k
    a([k i]) = a([i k]);
    p([k i]) = p([i k]);
    perm([k i]) = perm([i k]);
end
end
end

```

5.5. It is easy to show that the computed $\hat{p}$ satisfies $\hat{p} = p(x)(1 + \theta_{2n+1})$, $|\theta_{2n+1}| \leq \gamma_{2n+1}$. Thus $\hat{p}$ has a tiny relative error. Of course, this assumes that the roots x_i are known exactly!

6.1. For $\alpha_{S,2}$, if $A \in \mathbb{C}^{m \times n}$ then, using the Cauchy–Schwarz inequality,

$$\|A\|_S^2 = \left(\sum_{i,j} |a_{ij}| \right)^2 \leq mn \|A\|_F^2 \leq mn \operatorname{rank}(A) \|A\|_2^2.$$

The first inequality is an equality iff $|a_{ij}| \equiv \alpha$, and the second inequality is an equality iff A is a multiple of a matrix with orthonormal columns. If A is real and square, these requirements are equivalent to A being a scalar multiple of a Hadamard matrix. If A is complex and square, the requirements are satisfied by the given Vandermonde matrix, which is $\sqrt{n}$ times a unitary matrix.

6.2. $\|xy^*\| = \max_{\|z\|=1} \|xy^*z\| = \|x\| \max_{\|z\|=1} |y^*z| = \|x\| \|y\|_D$.

6.3. By the Hölder inequality,

$$\operatorname{Re} y^* Ax \leq |y^* Ax| \leq \|y\|_D \|Ax\| = \|y\|_D \|A\| \|x\|. \quad (\text{A.4})$$

We now show that equality is possible throughout (A.4). Let x satisfy $\|A\| = \|Ax\|/\|x\|$ and let y be dual to Ax . Then

$$\operatorname{Re} y^* Ax = y^* Ax = \|y\|_D \|Ax\| = \|y\|_D \|A\| \|x\|,$$

as required. The last part follows by applying this equality to A^* and using the fact that the dual of the dual vector norm is the original norm.

6.4. From (6.19) we have $\|M_n\|_p \leq \mu_n^{1/p} \mu_n^{1-1/p} = \mu_n$. But by taking x in the definition (6.11) to be the vector of all ones, we see that $\|M_n\|_p \geq \mu_n$.

6.5. If $A = PDQ^*$ is an SVD then

$$\begin{aligned}
 \|AB\|_F &= \|PDQ^*B\|_F = \|DQ^*B\|_F \\
 &\leq \|\operatorname{diag}(\max_i \sigma_i) Q^*B\|_F = (\max_i \sigma_i) \|Q^*B\|_F \\
 &= \|A\|_2 \|B\|_F.
 \end{aligned}$$

Similarly, $\|BC\|_F \leq \|B\|_F \|C\|_2$, and these two inequalities together imply the required one.

6.6. By (6.6) and (6.8) it suffices to show that $\|A^{-1}\|_{\beta,\alpha} = (\min_{x \neq 0} \|Ax\|_{\beta}/\|x\|_{\alpha})^{-1}$. We have

$$\begin{aligned}\|A^{-1}\|_{\beta,\alpha} &= \max_{x \neq 0} \frac{\|A^{-1}x\|_{\alpha}}{\|x\|_{\beta}} \\ &= \max_{y \neq 0} \frac{\|y\|_{\alpha}}{\|Ay\|_{\beta}} \quad (y = A^{-1}x) \\ &= \left(\min_{x \neq 0} \|Ax\|_{\beta}/\|x\|_{\alpha}\right)^{-1}.\end{aligned}$$

6.7. Let λ be an eigenvalue of A and x the corresponding eigenvector, and form the matrix $X = [x, x, \dots, x] \in \mathbb{C}^{n \times n}$. Then $AX = \lambda X$, so $|\lambda|\|X\| = \|AX\| \leq \|A\|\|X\|$, showing that $|\lambda| \leq \|A\|$. For a subordinate norm it suffices to take norms in the equation $Ax = \lambda x$.

6.8. The following proof is much simpler than the usual proof based on diagonal scaling to make the off-diagonal of the Jordan matrix small (see, e.g., Horn and Johnson [636, 1985, Lem. 5.6.10]). The proof is from Ostrowski [909, 1973, Thm. 19.3].

Let $\delta^{-1}A$ have the Jordan canonical form $\delta^{-1}A = XJX^{-1}$. We can write $J = \delta^{-1}D + N$, where $D = \text{diag}(\lambda_i)$ and the λ_i are the eigenvalues of A . Then $A = X(D + \delta N)X^{-1}$, so

$$\|A\| := \|X^{-1}AX\|_{\infty} = \|D + \delta N\|_{\infty} \leq \rho(A) + \delta.$$

Note that we actually have $\|A\| = \rho(A) + \delta$ if the largest eigenvalue occurs in a Jordan block of size greater than 1. If A is diagonalizable then with $\delta = 0$ we get $\|A\| = \rho(A)$. The last part of the result is trivial.

6.9. We have $\|A\|_2 \leq \|A\|_F \leq \sqrt{n}\|A\|_2$ (in fact, we can replace $\sqrt{n}$ by $\sqrt{r}$, where $r = \text{rank}(A)$). There is equality on the left when $\sigma_2 = \dots = \sigma_n = 0$, that is, when A has rank 1 ($A = xy^*$) or $A = 0$. There is equality on the right when $\sigma_1 = \dots = \sigma_n = \alpha$, that is, when $A = \alpha Q$ where Q has orthonormal columns, $\alpha \in \mathbb{C}$.

6.10. Let $F = P\Sigma Q^*$ be an SVD. Then

$$\left\| \begin{bmatrix} I & F \\ 0 & I \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} P & 0 \\ 0 & Q \end{bmatrix} \begin{bmatrix} I & \Sigma \\ 0 & I \end{bmatrix} \begin{bmatrix} P^* & 0 \\ 0 & Q^* \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} I & \Sigma \\ 0 & I \end{bmatrix} \right\|_2.$$

But $\begin{bmatrix} I & \Sigma \\ 0 & I \end{bmatrix}$ can be permuted into the form $\text{diag}(D_i)$, where $D_i = \begin{bmatrix} 1 & \sigma_i \\ 0 & 1 \end{bmatrix}$. It is easy to find the singular values of D_i , and the maximum value is attained for $\sigma_1 = \|F\|_2$.

6.11. (a)

$$\|Ax\|_{\beta} = \left\| \sum_j x_j A(:, j) \right\|_{\beta} \leq \max_j \|A(:, j)\|_{\beta} \sum_j |x_j|,$$

with equality for $x = e_k$, where the maximum is attained for $j = k$.

(b)

$$\|Ax\|_{\infty} = \max_i |A(i, :)x| \leq \max_i (\|A(i, :)^*\|_{\alpha}^D \|x\|_{\alpha}) = \|x\|_{\alpha} \max_i \|A(i, :)^*\|_{\alpha}^D.$$

Equality is attained for an x that gives equality in the Hölder inequality involving the k th row of A , where the maximum is attained for $i = k$. Finally, from either formula, $\|A\|_{1,\infty} = \max_{i,j} |a_{ij}|$.

6.12. Using the Cholesky factorization $A = R^*R$,

$$\begin{aligned}\|A\|_{\infty,1} &= \max_{x \neq 0} \frac{\|Ax\|_1}{\|x\|_\infty} = \max\{y^*Ax : \|x\|_\infty = 1, \|y\|_\infty = 1\} \\ &= \max\{(Ry)^*Rx : \|x\|_\infty = 1, \|y\|_\infty = 1\} \\ &= \max\{(Rx)^*Rx : \|x\|_\infty = 1\} \\ &= \max\{x^*Ax : \|x\|_\infty = 1\}.\end{aligned}$$

6.13. H^T is also a Hadamard matrix, so by the duality result (6.21) it suffices to show that $\|H\|_p = n^{1/p}$ for $1 \leq p \leq 2$. Since $|h_{ij}| \equiv 1$, (6.12) gives $\|H\|_p \geq n^{1/p}$. Since $\|H\|_1 = n$ and $\|H\|_2 = n^{1/2}$, (6.20) gives $\|H\|_p \leq n^{1/p}$, and so $\|H\|_p = n^{1/p}$ for $1 \leq p \leq 2$, as required.

6.14. We prove the lower bound in (6.23) and the upper bound in (6.24); the other bounds follow on using $\|A^T\|_p = \|A\|_q$. The lower bound in (6.23) is just (6.12). Now assume that A has at most μ nonzeros per column. Define

$$D_i = \text{diag}(s_{i1}, \dots, s_{in}), \quad s_{ij} = \begin{cases} 1, & a_{ij} \neq 0, \\ 0, & a_{ij} = 0, \end{cases}$$

and note that

$$\sum_i \|D_i x\|_p^p = \sum_{i,j} s_{ij} |x_j|^p = \sum_j |x_j|^p \sum_i s_{ij} \leq \mu \|x\|_p^p.$$

We have

$$\begin{aligned}\|Ax\|_p^p &= \sum_i |A(i, \cdot)x|^p = \sum_i |A(i, \cdot)D_i x|^p \\ &\leq \sum_i \|A(i, \cdot)\|_q^p \|D_i x\|_p^p \\ &\leq \max_i \|A(i, \cdot)\|_q^p \sum_i \|D_i x\|_p^p \\ &\leq \max_i \|A(i, \cdot)\|_q^p \mu \|x\|_p^p,\end{aligned}$$

which gives the upper bound in (6.24).

6.15. The lower bound follows from $\|Ax\|_p / \|x\|_p \leq \| |A| \|_p / \|x\|_p$. From (6.12) we have

$$\| |A| \|_p \leq n^{1-1/p} \max_j \| |A| e_j \|_p = n^{1-1/p} \max_j \| A e_j \|_p \leq n^{1-1/p} \|A\|_p.$$

By (6.21), we also have $\| |A| \|_p = \| |A^T| \|_q \leq n^{1-1/q} \|A^T\|_q = n^{1/p} \|A\|_p$ and the result follows.

6.16. The function ν is not a vector norm because $\nu(\alpha x) \neq |\alpha| \nu(x)$ does not hold for all $\alpha \in \mathbb{C}$, $x \in \mathbb{C}^n$. However, $\nu(\alpha x) \leq \nu(\alpha) \nu(x)$, and the other two norm conditions hold, so it makes sense to define the “subordinate norm”. We have

$$\begin{aligned}\nu(Ax) &= \nu\left(\sum_j x_j A(:, j)\right) \leq \sum_j \nu(x_j A(:, j)) \\ &\leq \sum_j \nu(x_j) \nu(A(:, j)) \leq \max_j \nu(A(:, j)) \sum_j \nu(x_j) \\ &= \max_j \nu(A(:, j)) \nu(x).\end{aligned}$$

There is equality for x_j an appropriate unit vector e_j . Hence $\nu(A) = \max_j \nu(A(:, j))$.

7.1. It is straightforward to obtain from $A(y-x) = \Delta b - \Delta A x + \Delta A(x-y)$ the inequality $(I - \epsilon|A^{-1}|E)|x-y| \leq \epsilon|A^{-1}|(f+E|x|)$. In general, if $B \geq 0$ and $\rho(B) < 1$ then $I - B$ is nonsingular. Since $\rho(\epsilon|A^{-1}|E) \leq \epsilon\| |A^{-1}|E \| < 1$, we can premultiply by $(I - \epsilon|A^{-1}|E)^{-1} \geq 0$ to obtain the bound for $|x-y|$. For the last part,

$$\frac{|x_i - y_i|}{|x_i|} \leq \frac{\epsilon(|A^{-1}|(f+E|x|))_i}{|x_i|} + O(\epsilon^2).$$

7.2. Take norms in $r = A(x-y)$ and $x-y = A^{-1}r$. The result says that the normwise relative error is at least as large as the normwise relative residual and possibly $\kappa(A)$ times as large. Since the upper bound is attainable, the relative residual is not a good predictor of the relative error unless A is very well conditioned.

7.3. Let D_R equilibrate the rows of A , so that $B = D_R A$ satisfies $|B|e = e$. Then

$$\text{cond}(B) = \| |B^{-1}| |B| e \|_\infty = \| |B^{-1}| e \|_\infty = \| B^{-1} \|_\infty = \kappa_\infty(B).$$

Hence $\kappa_\infty(D_R A) = \text{cond}(D_R A) = \text{cond}(A)$, which implies $\kappa_\infty(A) \leq \kappa_\infty(D_R) \text{cond}(A)$. The inequality $\text{cond}(A) \leq \kappa_\infty(A)$ is trivial, and the deduction of (7.15) is immediate.

7.4. The first inequality is trivial. For the second, since $h_{ii} \equiv 1$ and $|h_{ij}| \leq 1$ we have $|H| \geq I$ and $\|H\|_\infty \leq n$. Hence

$$\| |H^{-1}| |H| \|_\infty \geq \| |H^{-1}| \|_\infty = \| H^{-1} \|_\infty \geq \| H^{-1} \|_\infty \frac{\|H\|_\infty}{n} = \frac{\kappa_\infty(H)}{n}.$$

7.5. We have $\Delta x = A^{-1} \Delta b$, which yields $\|\Delta x\|_2 \leq \sigma_n^{-1} \|\Delta b\|_2$. Now

$$\begin{aligned} \|x\|_2^2 &= \|V \Sigma^{-1} U^T b\|_2^2 = \sum_{i=1}^n \frac{(u_i^T b)^2}{\sigma_i^2} \quad (u_i = U(:, i)) \\ &\geq \sum_{i=n+1-k}^n \frac{(u_i^T b)^2}{\sigma_i^2} \\ &\geq \frac{1}{\sigma_{n+1-k}^2} \sum_{i=n+1-k}^n (u_i^T b)^2 = \frac{\|P_k b\|_2^2}{\sigma_{n+1-k}^2}, \end{aligned}$$

which yields the result.

If we take $k = n$ we obtain the bound that would be obtained by applying standard perturbation theory (Theorem 7.2). The gist of this result is that the full $\kappa_2(A)$ magnification of the perturbation will not be felt if b contains a significant component in the subspace $\text{span}(U_k)$ for a k such that $\sigma_{n+1-k} \approx \sigma_n$. This latter condition says that x must be a large-normed solution: $\|x\|_2 \approx \|A^{-1}\|_2 \|b\|_2$.

7.6. (a) Use $\| |A^{-1}| E |x| \|_\infty = \|x\|_1 \| |A^{-1}| |A| e \|_\infty = \|x\|_1 \| |A^{-1}| |A| \|_\infty$. For the upper bound, use $f \leq |A||x|$.

(b) Use $\| |A^{-1}| E |x| \|_\infty = \| |A^{-1}| e e^T |A| |x| \|_\infty = \|A^{-1}\|_\infty \| |A| |x| \|_1$.

7.7. We will prove the result for ω ; the proof for η is entirely analogous. The lower bound is trivial. Let $\epsilon = \omega_{|A|, |b|}(y)$ and $r = b - Ay$. Then $|r| \leq \epsilon(|A||y| + |b|)$ and $(A + \Delta A)y = b + \Delta b$ with $|\Delta A| \leq \epsilon|A|$ and $|\Delta b| \leq \epsilon|b|$. Hence $|b| = |(A + \Delta A)y - \Delta b| \leq (1 + \epsilon)|A||y| + \epsilon|b|$, yielding $|b| \leq (1 - \epsilon)^{-1}(1 + \epsilon)|A||y|$. Thus

$$|r| \leq \epsilon \left(1 + \frac{1 + \epsilon}{1 - \epsilon} \right) |A||y| = \frac{2\epsilon}{1 - \epsilon} |A||y|,$$

which implies the result, by Theorem 7.3.

7.8. The constraint on ΔA and Δb can be written

$$[\Delta A \quad \theta \Delta b] \begin{bmatrix} y \\ -\theta^{-1} \end{bmatrix} = r,$$

and so

$$\|r\|_2 \leq (\|y\|_2^2 + \theta^{-2})^{1/2} \|[\Delta A, \theta \Delta b]\|_2 \leq \theta^{-1} (\theta^2 \|y\|_2^2 + 1)^{1/2} \|[\Delta A, \theta \Delta b]\|_F,$$

which gives a lower bound on $\eta_F(y)$. The bound is attained for the rank-1 perturbation $[\Delta A, \theta \Delta b] = r [y^T, -\theta^{-1}]^T / (\|y\|_2^2 + \theta^{-2})$.

7.9. We have $\Delta x = A^{-1}(\Delta b - \Delta A x) + O(\epsilon^2)$. Therefore $c^T \Delta x = c^T A^{-1}(\Delta b - \Delta A x) + O(\epsilon^2)$, and so

$$\frac{|c^T \Delta x|}{\epsilon |c^T x|} \leq \frac{|c^T A^{-1}|(f + E|x|)}{|c^T x|} + O(\epsilon),$$

this inequality being sharp. Hence

$$\chi_{E,f}(A, x) = \frac{|c^T A^{-1}|(f + E|x|)}{|c^T x|}.$$

The lower bound for $\chi_{|A|,|b|}(A, x)$ follows from the inequalities $|c^T x| = |c^T A^{-1} \cdot Ax| \leq |c^T A^{-1}| |A| |x|$ and $|c^T x| = |c^T A^{-1} b| \leq |c^T A^{-1}| |b|$. A slight modification to the derivation of $\chi_{E,f}(A, x)$ yields

$$\psi_{E,f}(A, x) = \|c^T A^{-1}\|_2 \frac{\|f\|_2 + \|E\|_2 \|x\|_2}{|c^T x|}.$$

7.10. (a) For any $D_1, D_2 \in \mathcal{D}_n$ we have

$$\rho(BC) = \rho(D_1 B C D_1^{-1}) \leq \|D_1 B C D_1^{-1}\|_\infty \leq \|D_1 B D_2\|_\infty \|D_2^{-1} C D_1^{-1}\|_\infty. \quad (\text{A.5})$$

The rest of the proof shows that equality can be attained in this inequality.

Let $x_1 > 0$ be a right Perron vector of BC , so that $BCx_1 = \pi x_1$, where $\pi = \rho(BC) > 0$. Define

$$x_2 = Cx_1, \quad (\text{A.6})$$

so that $x_2 > 0$ and

$$Bx_2 = \pi x_1. \quad (\text{A.7})$$

(We note, incidentally, that x_2 is a right Perron vector of CB : $CBx_2 = \pi x_2$.)

Now define

$$D_1 = \text{diag}(x_1)^{-1}, \quad D_2 = \text{diag}(x_2). \quad (\text{A.8})$$

Then, with $e = [1, 1, \dots, 1]^T$, (A.7) can be written $BD_2 e = \pi D_1^{-1} e$, or $D_1 B D_2 e = \pi e$. Since $D_1 B D_2 > 0$, this gives $\|D_1 B D_2\|_\infty = \pi$. Similarly, (A.6) can be written $D_2 e = C D_1^{-1} e$, or $D_2^{-1} C D_1^{-1} e = e$, which gives $\|D_2^{-1} C D_1^{-1}\|_\infty = 1$. Hence for D_1 and D_2 defined by (A.8) we have $\|D_1 B D_2\|_\infty \|D_2^{-1} C D_1^{-1}\|_\infty = \pi = \rho(BC)$, as required.

Note that for the optimal D_1 and D_2 , $D_1 B D_2$ and $D_2^{-1} C D_1^{-1}$ both have the property that all their rows sums are equal.

(b) Take $B = |A|$ and $C = |A^{-1}|$, and note that $\kappa_\infty(D_1 A D_2) = \|D_1 |A| D_2\|_\infty \times \|D_2^{-1} |A^{-1}| D_1^{-1}\|_\infty$. Now apply (a).

(c) We can choose $F_1 > 0$ and $F_2 > 0$ so that $|A| + tF_1 > 0$ and $|A^{-1}| + tF_2 > 0$ for all $t > 0$. Hence, using (a),

$$\begin{aligned} \inf_{D_1, D_2 \in \mathcal{D}_n} \kappa_\infty(D_1 A D_2) &\leq \inf_{D_1, D_2 \in \mathcal{D}_n} \|D_1(|A| + tF_1) D_2\|_\infty \|D_2^{-1}(|A^{-1}| + tF_2) D_1^{-1}\|_\infty \\ &= \rho((|A| + tF_1)(|A^{-1}| + tF_2)). \end{aligned}$$

Taking the limit as $t \rightarrow 0$ yields the result, using continuity of eigenvalues.

(d) A nonnegative irreducible matrix has a positive Perron vector, from standard Perron–Frobenius theory (see, e.g., Horn and Johnson [636, 1985, Chap. 8]). Therefore the result follows by noting that in the proof of (a) all we need is for D_1 and D_2 in (A.8) to be defined and nonsingular, that is, for x_1 and x_2 to be positive vectors. This is the case if BC and CB are irreducible (since x_2 is a Perron vector of CB).

(e) Using $\kappa_1(A) = \kappa_\infty(A^T)$, $\rho(A) = \rho(A^T)$, and $\rho(AB) = \rho(BA)$, it is easy to show that the results of (a)–(d) remain true with the ∞ -norm replaced by the 1-norm. From $\|A\|_2 \leq (\|A\|_1 \|A\|_\infty)^{1/2}$ it then follows that $\inf_{D_1, D_2 \in \mathcal{D}_n} \kappa_2(D_1 A D_2) \leq \rho(|A| |A^{-1}|)$. In fact, the result in (a) holds for any p -norm, though the optimal D_1 and D_2 depend on the norm; see Bauer [90, 1963, Lem. 1(i)].

7.11. That $\mu'_E(A)$ cannot exceed the claimed expression follows by taking absolute values in the expression $\Delta X = -(A^{-1} \Delta A A^{-1} + A^{-1} \Delta A \Delta X)$. To show it can equal it we need to show that if the maximum is attained for $(i, j) = (r, s)$ then

$$|\Delta x_{rs}| = \epsilon (|A^{-1}| E |A^{-1}|)_{rs}$$

can be attained, to first order in ϵ . Equality is attained for $\Delta A = D_1 E D_2$, where $D_1 = \text{diag}(\text{sign}(A^{-1})_{ri})$, $D_2 = \text{diag}(\text{sign}(A^{-1})_{is})$.

7.12. (a) We need to find a symmetric H satisfying $Hy = b - Ay =: r$. Consider the QR factorization

$$Q^T \begin{bmatrix} y & r \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} \\ 0 & t_{22} \\ \vdots & \vdots \\ 0 & 0 \end{bmatrix}.$$

The constraint can be rewritten $Q^T H Q \cdot Q^T y = Q^T r$, and it is satisfied by $Q^T H Q := \text{diag}(\tilde{H}, 0_{n-2})$ if we can find $\tilde{H} \in \mathbb{R}^{2 \times 2}$ such that

$$\tilde{H}t = u, \quad \tilde{H} = \tilde{H}^T, \quad \text{where } t = \begin{bmatrix} t_{11} \\ 0 \end{bmatrix}, \quad u = \begin{bmatrix} t_{12} \\ t_{22} \end{bmatrix}.$$

We can take $\tilde{H} := (\|u\|_2 / \|t\|_2)P$, where $P \in \mathbb{R}^{2 \times 2}$ is either a suitably chosen Householder matrix, or the identity if t is a positive multiple of u . Then

$$\begin{aligned} \|H\|_2 &= \|\tilde{H}\|_2 = \|u\|_2 / \|t\|_2 \\ &= \|r\|_2 / \|y\|_2 = \|Gy\|_2 / \|y\|_2 \leq \|G\|_2, \end{aligned}$$

and $\|H\|_F = \|\tilde{H}\|_F \leq \sqrt{2} \|\tilde{H}\|_2 \leq \sqrt{2} \|G\|_2 \leq \sqrt{2} \|G\|_F$.

(b) We can assume, without loss of generality, that

$$|y_1| \leq |y_2| \leq \cdots \leq |y_n|. \quad (\text{A.9})$$

Define the off-diagonal of H by $h_{ij} = h_{ji} = g_{ij}$ for $j > i$, and let $h_{11} = g_{11}$. The i th equation of the constraint $Hy = Gy$ will be satisfied if

$$h_{ii}y_i = g_{ii}y_i + \sum_{j=1}^{i-1} (g_{ij} - g_{ji})y_j. \quad (\text{A.10})$$

If $y_i = 0$ set $h_{ii} = 0$; then (A.10) holds by (A.9). Otherwise, set

$$h_{ii} = g_{ii} + \sum_{j=1}^{i-1} (g_{ij} - g_{ji}) \frac{y_j}{y_i}, \quad (\text{A.11})$$

which yields

$$|h_{ii}| \leq \epsilon |a_{ii}| + 2\epsilon \sum_{j=1}^{i-1} |a_{ij}| \leq 3\epsilon |a_{ii}|,$$

by the diagonal dominance.

(c) Let $D = \text{diag}(a_{ii}^{1/2})$ and $A =: D\tilde{A}D$. Then $\tilde{a}_{ii} \equiv 1$ and $|\tilde{a}_{ij}| \leq 1$ for $i \neq j$. The given condition can be written $(\tilde{A} + \tilde{G})\tilde{y} = \tilde{b}$, $|\tilde{G}| \leq \epsilon|\tilde{A}|$, where $\tilde{y} = Dy$ and $\tilde{b} = D^{-1}b$. Defining $\tilde{H}$ as in the proof of (b), we find from (A.11) that $|\tilde{h}_{ii}| \leq \epsilon + 2\epsilon(i-1) \leq (2n-1)\epsilon$, so that $|\tilde{H}| \leq (2n-1)|\tilde{A}|$. With $H := D\tilde{H}D$, we find that $H = H^T$, $(A+H)y = b$, and $|H| \leq (2n-1)\epsilon|A|$.

7.13. By examining the error analysis for an inner product (§3.1), it is easy to see that for any $x \in \mathbb{R}^n$,

$$|fl(Ax) - Ax| \leq \text{diag}(\gamma_{w_i})|A||x|.$$

Hence $\hat{r} = fl(b - A\hat{x})$ satisfies

$$\hat{r} = r + \Delta r, \quad |\Delta r| \leq \Gamma(|A||\hat{x}| + |b|), \quad (\text{A.12})$$

which is used in place of (7.30) to obtain the desired bound.

7.14. (a) Writing $A^{-1} = (\alpha_{ij})$, we have

$$\Delta x_i = - \sum_{j=1}^n \alpha_{ij} \sum_{k=1}^n \epsilon_{jk} e_{jk} x_k + \sum_{j=1}^n \alpha_{ij} \delta_j f_j.$$

By the independence of the ϵ_{ij} and δ_i ,

$$\mathcal{E}(\Delta x_i^2) = \sigma^2 \left(\sum_{j=1}^n \sum_{k=1}^n \alpha_{ij}^2 e_{jk}^2 x_k^2 + \sum_{j=1}^n \alpha_{ij}^2 f_j^2 \right) = \sigma^2 ([A^{-1}][E][x] + [A^{-1}][f])_i.$$

Hence

$$\mathcal{E}(\|\Delta x\|_2^2) = \sum_{i=1}^n \mathcal{E}(\Delta x_i^2) = \sigma^2 \| [A^{-1}][E][x] + [A^{-1}][f] \|_1.$$

(b) A traditional condition number would be based on the maximum of $\|\Delta x\|_2/(\sigma\|x\|_2)$ over all “perturbations to the data of relative size σ ”. The expression $\text{condexp}(A, b)$ uses the “expected perturbation” $\sqrt{\mathcal{E}(\|\Delta x\|_2^2)}/(\sigma\|x\|_2)$ rather than the worst case.

(c) For $e_{ij} \equiv \|A\|_2$ and $f_i \equiv \|b\|_2$ we have

$$[A^{-1}][E][x] + [A^{-1}][f] = (\|A\|_2^2\|x\|_2^2 + \|b\|_2^2)[A^{-1}] \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix},$$

so

$$\frac{(\|[A^{-1}][E][x] + [A^{-1}][f]\|_1)^{1/2}}{\|x\|_2} = \left(\|A\|_2^2 + \frac{\|b\|_2^2}{\|x\|_2^2} \right)^{1/2} \|A^{-1}\|_F.$$

But

$$\kappa_{A,b}(A, x) = \left(\|A\|_2 + \frac{\|b\|_2}{\|x\|_2} \right) \|A^{-1}\|_2.$$

Hence

$$2^{-1/2} \kappa_{A,b}(A, x) \leq \text{condexp}(A, x) \leq \sqrt{n} \kappa_{A,b}(A, x).$$

This inequality shows that when perturbations are measured normwise there is little difference between the average and worst-case condition numbers.

7.15. We have

$$\begin{aligned}\|A \circ A^{-T}\|_2 &= \|D_1 A D_2 \circ D_1^{-1} A^{-T} D_2^{-1}\|_2 = \|(D_1 A D_2) \circ (D_1 A D_2)^{-T}\|_2 \\ &\leq \|D_1 A D_2\|_2 \|(D_1 A D_2)^{-T}\|_2 = \kappa_2(D_1 A D_2).\end{aligned}$$

Note that this result gives a lower bound for the optimal condition number, while Bauer's result in Problem 7.10(c) gives an upper bound. There is equality for diagonal A , trivially. For triangular A we have $\|A \circ A^{-T}\|_2 = 1 = \rho(|A||A^{-1}|)$ and so there is equality in view of Problem 7.10(e).

8.1. Straightforward.

8.2. Let

$$T(\lambda) = \begin{bmatrix} \lambda^{-1} & 1 & 1 \\ 0 & \lambda^{-1} & \lambda^{-1} \\ 0 & 0 & \lambda^{-2} \end{bmatrix}.$$

Then

$$T(\lambda)^{-1} = \begin{bmatrix} \lambda & -\lambda^2 & 0 \\ 0 & \lambda & -\lambda^2 \\ 0 & 0 & \lambda^2 \end{bmatrix}, \quad M(T(\lambda))^{-1} = \begin{bmatrix} \lambda & \lambda^2 & 2\lambda^3 \\ 0 & \lambda & \lambda^2 \\ 0 & 0 & \lambda^2 \end{bmatrix}.$$

As $\lambda \rightarrow \infty$, $\|M(T(\lambda))^{-1}\|/\|T(\lambda)^{-1}\| \sim \lambda$. This 3×3 example can be extended to an $n \times n$ one by padding with an identity matrix.

8.3. The bound follows from Theorem 8.10, since $(M(U)^{-1})_{ij} \leq 2^{j-i-1}$ ($j > i$). Using $\|b\|_\infty \leq \|U\|_\infty \|x\|_\infty \leq n\|x\|_\infty$ we get a similar result to that given by Theorem 8.7.

8.4. Assume T is upper triangular, and write $T = D - U$, where $D = \text{diag}(t_{ii}) \geq 0$ and $U \geq 0$ is strictly upper triangular. Then, using the fact that $(D^{-1}U)^n = 0$,

$$\begin{aligned}|T^{-1}||T| &= (I - D^{-1}U)^{-1}D^{-1} \cdot (D + U) \\ &= \sum_{i=0}^{n-1} (D^{-1}U)^i \cdot (I + D^{-1}U) \\ &= \sum_{i=0}^{n-1} (D^{-1}U)^i + \sum_{i=1}^{n-1} (D^{-1}U)^i.\end{aligned}$$

Now $0 \leq b = Tx = (D - U)x$, so $Dx \geq Ux$, that is, $x \geq D^{-1}Ux$. Hence

$$|T^{-1}||T|x = \sum_{i=0}^{n-1} (D^{-1}U)^i x + \sum_{i=1}^{n-1} (D^{-1}U)^i x \leq (n + n - 1)x,$$

which gives the result, since $x = T^{-1}b \geq 0$.

8.5. Use (8.4).

8.7. (a) Write

$$\|A^{-1}\|_\infty = \max_{x \neq 0} \frac{\|A^{-1}x\|_\infty}{\|x\|_\infty} =: \frac{\|b\|_\infty}{\|y\|_\infty},$$

where $Ab = y$. Let $|b_k| = \|b\|_\infty$. From

$$a_{kk}b_k = y_k - \sum_{j \neq k} a_{kj}b_j,$$

it follows that

$$|a_{kk}||b_k| \leq \|y\|_\infty + \sum_{j \neq k} |a_{kj}||b_j|,$$

and hence that $\|b\|_\infty/\|y\|_\infty \leq 1/\alpha_k \leq 1/(\min_i \alpha_i)$.

(b) Apply the result of (a) to AD and use the inequality $\|A^{-1}\|_\infty \leq \|(AD)^{-1}\|_\infty \|D\|_\infty$.

(c) If A is triangular then we can achieve any diagonal dominance factors β_i we like, by appropriate column scaling. In particular, we can take $\beta_i \equiv 1$ in (b), which requires $M(A)d = e$, where $d = De$. Then $\|D\|_\infty = \|d\|_\infty = \|M(A)^{-1}e\|_\infty$, so the bound is $\|A^{-1}\|_\infty \leq \|M(A)^{-1}e\|_\infty$, as required.

8.8. (a) Using the formula $\det(I + xy^T) = 1 + y^T x$ we have $\det(A + \alpha e_i e_j^T) = \det(A(I + \alpha A^{-1} e_i e_j^T)) = \det(A)(1 + \alpha e_j^T A^{-1} e_i)$. Hence we take $\alpha_{ij} = -(e_j^T A^{-1} e_i)^{-1}$ if $e_j^T A^{-1} e_i \neq 0$; otherwise there is no α_{ij} that makes $A + \alpha_{ij} e_i e_j^T$ singular.

It follows that the “best” place to perturb A to make it singular (the place that gives the smallest α_{ij}) is in the (s, r) position, where the element of largest absolute value of A^{-1} is in the (r, s) position.

(b) The off-diagonal elements of T_n^{-1} are given by $(T_n^{-1})_{ij} = 2^{j-i-1}$. Hence, using part (a), $T_n + \alpha e_n e_1^T$ is singular, where $\alpha = -2^{2-n}$. In fact, T_n is also made singular by subtracting 2^{1-n} from all the elements in the first column.

8.9. Here is Zha’s proof. If $s = 1$ the result is obvious, so assume $s < 1$. Define the n -vectors

$$v^T = \left[0, \quad \dots, \quad 0, \quad \frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right]$$

and

$$u^T = \left[0, \quad \dots, \quad 0, \quad \sqrt{\frac{1+c}{2}}, \frac{-s}{\sqrt{2(1+c)}} \right]$$

and let $\sigma = s^{n-2} \sqrt{1+c}$. It is easy to check that

$$U_n(\theta)v = \sigma u, \quad U_n(\theta)^T u = \sigma v,$$

which shows that σ is a singular value of $U_n(\theta)$. With σ_i denoting the i th largest singular value,

$$\sigma_n(U_n(\theta)) \leq s^{n-1} < \sigma = s^{n-2} \sqrt{1+c}.$$

Now we prove by induction that $\sigma_{n-1}(U_n(\theta)) = \sigma$. For $n = 2$ it is easy to check by direct computation that $\sigma_1(U_2(\theta)) = \sqrt{1+c}$. Using the interlacing property of the singular values [509, 1996, §8.6.1] and the inductive assumption, we have

$$\sigma_{n-2}(U_n(\theta)) \geq \sigma_{n-2}(U_{n-1}(\theta)) = s^{n-3} \sqrt{1+c} > \sigma.$$

Therefore $\sigma_{n-1}(U_n(\theta)) = \sigma$.

8.10. For a solver of this form, it is not difficult to see that

$$|fl(f_i(T, b)) - f_i(T, b)| \leq c_n u \bar{f}_i(M(T), |b|) + O(u^2),$$

where $\bar{f}_i$ denotes f_i with all its coefficients replaced by their absolute values, and where $\bar{f}_i(M(T), |b|)$ is a rational expression consisting entirely of nonnegative terms. This is the required bound expressed in a different notation. An example (admittedly, a contrived one) of a solver that does not satisfy (8.21) is, for a 2×2 lower triangular system $Lx = b$,

$$x_1 = b_1/l_{11}, \quad x_2 = ((b_2 - b_1)(l_{11} + l_{21}) + b_1 l_{11} - b_2 l_{21})/(l_{11} l_{22}).$$

9.1. The proof is by induction. Assume the result is true for matrices of order $n-1$, and suppose

$$\begin{bmatrix} \alpha & a^T \\ b & C \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l & M \end{bmatrix} \begin{bmatrix} \alpha & a^T \\ 0 & V \end{bmatrix} \in \mathbb{R}^{n \times n}$$

is a unique LU factorization. Then $MV = C - la^T$ has a unique LU factorization, and so by the inductive hypothesis has nonsingular leading principal submatrices of order 1 to $n-2$. Thus $v_{ii} \neq 0$, $i = 1:n-2$. If $\alpha = 0$ then $b = 0$ and l is arbitrary subject to $C - la^T$ having an LU factorization. But $C - (l + \Delta l)a^T$ has nonsingular leading principal submatrices of order 1 to $n-2$ for sufficiently small Δl (irrespective of a), so has an LU factorization for sufficiently small Δl . Thus if $\alpha = 0$ we have a contradiction to the uniqueness of the factorization. Hence $\alpha \neq 0$, which completes the proof.

9.2. $A(\sigma)$ fails to have an LU factorization without pivoting only if one of its leading principal submatrices is singular, that is, if $\sigma \in \lambda(A(\sigma)(1:k, 1:k))$, for $k \in \{1, 2, \dots, n-1\}$. There are thus $1 + 2 + \dots + (n-1) = \frac{1}{2}(n-1)n$ “danger” values of σ , which may not all be distinct.

9.3. If $0 \notin F(A)$ then all the principal submatrices of A must be nonsingular, so the result follows by Theorem 9.1. Note that A may have a unique LU factorization even when 0 is in the field of values, as shown by the matrices

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \quad (z = e_2) \quad \text{and} \quad A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix} \quad (z = e_2),$$

so the implication is only one way. Note also that $0 \notin F(A)$ iff $e^{i\theta} A$ has positive definite Hermitian part for some real θ (Horn and Johnson [637, 1991, Thm. 1.3.5]).

9.4. The changes are minor. Denoting by $\hat{P}$ and $\hat{Q}$ the computed permutations, the result of Theorem 9.3 becomes

$$\hat{L}\hat{U} = \hat{P}A\hat{Q} + \Delta A, \quad |\Delta A| \leq \gamma_n |\hat{L}||\hat{U}|,$$

and that of Theorem 9.4 becomes

$$(A + \Delta A)\hat{x} = b, \quad |\Delta A| \leq \gamma_{3n} \hat{P}^T |\hat{L}||\hat{U}| \hat{Q}^T.$$

9.5.

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix} = LU; \quad |L||U| = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

9.6. Since A is nonsingular and every submatrix has a nonnegative determinant, the determinantal inequality shows that $\det(A(1:p, 1:p)) > 0$ for $p = 1:n-1$, which guarantees the existence of an LU factorization (Theorem 9.1). That the elements of L and U are nonnegative follows from (9.2).

GE computes $a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)} = a_{ij}^{(k)} - l_{ik}u_{kj} \leq a_{ij}^{(k)}$, since $l_{ik}, u_{kj} \geq 0$. Thus $a_{ij} = a_{ij}^{(1)} \geq a_{ij}^{(2)} \geq \dots \geq a_{ij}^{(r)}$, $r = \min(i, j)$. For $i > j$, $a_{ij}^{(r)} \geq a_{ij}^{(r+1)} = \dots = a_{ij}^{(n)} = 0$; for $j \geq i$, $a_{ij}^{(r)} = \dots = a_{ij}^{(n)} = u_{ij} \geq 0$. Thus $0 \leq a_{ij}^{(k)} \leq a_{ij}$ for all i, j, k and hence $\rho_n \leq 1$. But $\rho_n \geq 1$, so $\rho_n = 1$.

9.7. The number of square submatrices is

$$\begin{aligned} \sum_{k=1}^n \binom{n}{k}^2 &= \binom{2n}{n} \quad ([517, 1994, \S 5.1]) \\ &= \frac{(2n)!}{(n!)^2} \\ &\sim \frac{\sqrt{2\pi(2n)} (2n/e)^{2n}}{(\sqrt{2\pi n} (n/e)^n)^2} \quad (\text{Stirling's approximation}) \\ &\sim \frac{1}{\sqrt{n}} \left(\frac{2n/e}{n/e} \right)^{2n} \sim \frac{4^n}{\sqrt{n}} = O(4^n). \end{aligned}$$

The number of rectangular submatrices is

$$\sum_{k=1}^n \sum_{l=1}^n \binom{n}{k} \binom{n}{l} = \left(\sum_{k=1}^n \binom{n}{k} \right)^2 = (2^n - 1)^2,$$

which, interestingly, is not many more than the number of square submatrices.

9.8. The given fact implies that JAJ is totally nonnegative. Hence it has an LU factorization $JAJ = LU$ with $L \geq 0$ and $U \geq 0$. This means that $A = (JLJ)(JUJ) \equiv \tilde{L}\tilde{U}$ is an LU factorization, and $|\tilde{L}||\tilde{U}| = LU = JAJ = |A|$.

9.9. With l_j the j th column of L and u_i^T the i th row of U , we have

$$A^{(k)} = A - l_1 u_1^T - \cdots - l_{k-1} u_{k-1}^T = A - L(:, 1:k-1)U(1:k-1, :),$$

hence

$$|A^{(k)}| \leq |A| + |L(:, 1:k-1)||U(1:k-1, :)| \leq |A| + |L||U|.$$

Equating (i, j) elements gives

$$|a_{ij}^{(k)}| \leq |a_{ij}| + \| |L||U| \|_\infty,$$

from which the result follows on dividing by $\max_{i,j} |a_{ij}|$.

9.10. By inspecting the equations in Algorithm 9.2 we see that the computed LU factors satisfy

$$A - \hat{L}\hat{U} = \epsilon \hat{l}_{ij} \hat{u}_{jj} e_i e_j^T.$$

Since $\hat{L}\hat{U}\hat{x} = b$, we have, writing $\alpha = \epsilon \hat{l}_{ij} \hat{u}_{jj}$ and using the Sherman–Morrison formula (Problem 26.2),

$$\hat{x} = (A - \alpha e_i e_j^T)^{-1} b = \left(A^{-1} + \frac{\alpha A^{-1} e_i e_j^T A^{-1}}{1 - \alpha e_j^T A^{-1} e_i} \right) b = x + \frac{\alpha A^{-1} e_i x_j}{1 - \alpha e_j^T A^{-1} e_i}.$$

So

$$x - \hat{x} = - \frac{\epsilon \hat{l}_{ij} \hat{u}_{jj} x_j}{1 - \epsilon \hat{l}_{ij} \hat{u}_{jj} A^{-1}(j, i)} A^{-1}(:, i).$$

The error $x - \hat{x}$ is a rational function of ϵ and is zero if $x_j = 0$, but it will typically be of order $\epsilon \kappa_\infty(A) \|x\|_\infty$.

9.11. $\alpha(B) = \alpha(A)$, and $B^{-1} = \frac{1}{2} \begin{bmatrix} A^{-1} & A^{-1} \\ A^{-1} & -A^{-1} \end{bmatrix}$, so $\beta(B) = \frac{1}{2} \beta(A)$. Hence $\theta(B) = (\alpha(B)\beta(B))^{-1} = 2\theta(A)$. Taking $A = S_n$, $g(2n) \geq \rho_{2n}^c(B) \geq \theta(B) = 2\theta(S_n) = n + 1$.

9.12. First, the size or accuracy of the pivots is not the fundamental issue. The error analysis shows that instability corresponds to large elements in the intermediate matrices or in L or U . Second, in $PAQ = LU$, u_{nn}^{-1} is an element of A^{-1} (see (9.11)), so it is not necessarily true that the pivoting strategy can force u_{nn} to be small. Third, from a practical point of view it is good to obtain small final pivots because they reveal near singularity of the matrix.

9.13. Because the rows of U are the pivot rows, μ_j is the maximum number of times any element in the j th column of A was modified during the reduction. Since the multipliers are bounded by τ^{-1} , the bound for $\max_i |a_{ij}^{(k)}|$ follows easily. Thus

$$\rho_n \leq (1 + \tau^{-1})^{\max_j \mu_j}.$$

10.1. Observe that if $x = \alpha e_i - e_j$, where e_i is the i th unit vector, then for any α we have, using the symmetry of A ,

$$0 < x^T A x = \alpha^2 a_{ii} - 2\alpha a_{ij} + a_{jj}.$$

Since the right-hand side is a quadratic in α , the discriminant must be negative, that is, $4a_{ij}^2 - 4a_{ii}a_{jj} < 0$, which yields the desired result. This result can also be proved using the Cholesky decomposition and the Cauchy-Schwarz inequality: $A = R^T R \Rightarrow a_{ij} = r_i^T r_j \leq \|r_i\|_2 \|r_j\|_2 = (a_{ii}a_{jj})^{1/2}$ (there is strict inequality for $i \neq j$, since $r_i \neq \alpha r_j$). The inequality implies that $|a_{ij}| \leq \max(a_{ii}, a_{jj})$, which shows that $\max_{i,j} |a_{ij}| = \max_i a_{ii}$, that is, the largest element of A lies on the diagonal.

10.2. Compute the Cholesky factorization $A = R^T R$, solve $R^T y = x$, then compute $y^T y$. In addition to being more efficient than forming A^{-1} explicitly, this approach guarantees a nonnegative result.

10.3. Let $s = c - \sum_{i=1}^{k-1} a_i b_i$. By Lemma 8.4,

$$\widehat{s}(1 + \theta_{k-1}^{(0)}) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_{k-1}^{(i)}).$$

Then $\widehat{y} = fl(\sqrt{\widehat{s}}) = \sqrt{\widehat{s}}(1 + \delta)$, $|\delta| \leq u$. Hence

$$\widehat{y}^2(1 + \theta_{k+1}) = \frac{\widehat{y}^2}{(1 + \delta)^2}(1 + \theta_{k-1}^{(0)}) = c - \sum_{i=1}^{k-1} a_i b_i (1 + \theta_{k-1}^{(i)}),$$

as required.

10.4. $A = \begin{bmatrix} \alpha & a^T \\ a & C \end{bmatrix} \Rightarrow A^{(2)} = \begin{bmatrix} \alpha & a^T \\ 0 & B \end{bmatrix}$, where $B = C - aa^T/\alpha$. Let $y = [1, \theta z]^T$. Then $0 < y^T A y = \theta^2 z^T C z + 2\theta z^T a + \alpha$ for all θ and so the discriminant $4(z^T a)^2 - 4\alpha z^T C z < 0$ if $z \neq 0$, that is, $z^T B z = z^T C z - \frac{1}{\alpha}(z^T a)^2 > 0$ (since $\alpha = a_{11} > 0$). This shows that B is positive definite.

By induction, the elimination succeeds (i.e., all pivots are nonzero) and all the reduced submatrices are positive definite. Hence $a_{rr}^{(r)} > 0$ for all r . Therefore

$$a_{kk}^{(r+1)} = a_{kk}^{(r)} - \frac{a_{kr}^{(r)}}{a_{rr}^{(r)}} a_{rk}^{(r)} = a_{kk}^{(r)} - \frac{a_{rk}^{(r)2}}{a_{rr}^{(r)}} \leq a_{kk}^{(r)}.$$

Thus $a_{kk} = a_{kk}^{(1)} \geq \dots \geq a_{kk}^{(k)} > 0$. Since the largest element of a positive definite matrix lies on the diagonal (Problem 10.1), for any i, j, k there exists r such that

$$|a_{ij}^{(k)}| \leq a_{rr}^{(k)} \leq \dots \leq a_{rr}^{(1)} = a_{rr} \leq \max_{i,j} |a_{ij}|,$$

which shows that the growth factor $\rho_n = 1$ (since $\rho_n \geq 1$).

10.5. From (10.8),

$$|\widehat{R}^T| |\widehat{R}| \leq (1 - \gamma_{n+1})^{-1} (a_{ii}^{1/2} a_{jj}^{1/2}) \leq (1 - \gamma_{n+1})^{-1} (\max_i a_{ii}),$$

so

$$\| |\widehat{R}^T| |\widehat{R}| \|_M \leq (1 - \gamma_{n+1})^{-1} \|A\|_M,$$

as required.

10.6. $W = A_{11}^{-1} A_{12} = R_{11}^{-1} R_{12}$, so $RZ = 0$ and hence $AZ = 0$. Z is of dimension $n \times (n - r)$ and of full rank, so it spans $\text{null}(A)$.

10.7. The inequalities (10.13) follow from the easily verified fact that, for $j \geq k$,

$$a_{jj}^{(k)} = \sum_{i=k}^{\min(j,r)} r_{ij}^2.$$

10.8. Examples of indefinite matrices with nonnegative leading principal minors are

$$\begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

A necessary and sufficient condition for definiteness is that all the *principal minors* of A (of which there are 2^{n-1}) be nonnegative (not just the leading principal minors); see, e.g., Horn and Johnson [636, 1985, p. 405] or Mirsky [857, 1961, p. 405] (same page number in both books!).

10.9. For the matrix $Z = [-(R_{11}^{-1} R_{12})^T, I]^T \in \mathbb{R}^{n \times (n-k)}$ we have, from (10.15), $Z^T A Z = S_k(A)$ and so we can take $p = Z e_1$, the first column of Z .

10.10. Theorem 10.3 is applicable only if Cholesky succeeds, which can be guaranteed only if the (suitably scaled) matrix A is not too ill conditioned (Theorem 10.7). Therefore the standard analysis is *not* applicable to positive semidefinite matrices that are very close to being singular. Theorem 10.14 provides a bound on the residual after $\text{rank}(A)$ stages and, in particular, on the computed Schur complement, which would be zero in exact arithmetic. The condition of Theorem 10.7 ensures that all the computed Schur complements are positive definite, so that even if magnification of errors by a factor $\|W\|_2^2$ occurs, it is absorbed by the next part of the Cholesky factor. The proposed appeal to continuity is simply not valid.

10.11. For any nonzero $x \in \mathbb{C}^n$ we have

$$0 < \text{Re} \begin{bmatrix} x_1^* & x_2^* \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Putting $x_1 = -A_{11}^{-1} A_{12} x_2$ we obtain

$$0 < \text{Re} \begin{bmatrix} x_1^* & x_2^* \end{bmatrix} \begin{bmatrix} 0 \\ -A_{21}^* A_{11}^{-1} A_{12} x_2 + A_{22} x_2 \end{bmatrix} = \text{Re} x_2^* (A_{22} - A_{21}^* A_{11}^{-1} A_{12}) x_2,$$

which shows that S is positive definite.

11.1. If a nonsingular pivot matrix E of dimension 1 or 2 cannot be found, then all 1×1 and 2×2 principal submatrices of the symmetric matrix A are singular, and this is easily seen to imply that A is the zero matrix.

11.2. The analysis in §11.1 shows that for a 2×2 pivot E , $\det(E) \leq (\alpha^2 - 1)\mu_0^2$ for complete pivoting and $\det(E) \leq (\alpha^2 - 1)\omega_r^2$ for partial pivoting ($r = 1$) and rook pivoting. Now $\alpha^2 - 1 < 0$ and μ_0 and ω_r are nonzero if a 2×2 pivot is needed. Hence $\det(E) < 0$, which means that E has one positive and one negative eigenvalue. Note that if A is positive definite it follows that all pivots are 1×1 .

If the block diagonal factor has p_+ positive 1×1 diagonal blocks, p_- negative 1×1 diagonal blocks, p_0 zero 1×1 diagonal blocks, and q 2×2 diagonal blocks, then the inertia is $(+, -, 0) = (p_+ + q, p_-, p_0)$.

Denote a 2×2 pivot by

$$E = \begin{bmatrix} a & b \\ b & c \end{bmatrix},$$

and consider partial pivoting. We know $\det(E) = ac - b^2 < 0$ and $|b| \geq |a|$, so the formula $\det(E) = [(a/b)c - b]b$ minimizes the risk of overflow. Similarly, the formula

$$E^{-1} = \frac{1}{b[(a/b)(c/b) - 1]} \begin{bmatrix} c/b & -1 \\ -1 & a/b \end{bmatrix}$$

helps to avoid overflow; this is the formula used in LINPACK's xSIDI and LAPACK's xSYTRI. The same formulae are suitable for rook pivoting and complete pivoting because then $|b| \geq \max(|a|, |c|)$.

11.3. The partial pivoting strategy simplifies as follows: if $|a_{11}| \geq \alpha|a_{21}|$ use a 1×1 pivot a_{11} , if $|a_{22}| \geq \alpha|a_{12}|$ use a 1×1 pivot a_{22} , else use a 2×2 pivot, that is, do nothing.

11.4. There may be interchanges, because the tests $|a_{11}| \geq \alpha\omega_1$ and $\alpha\omega_1^2 \leq |a_{11}|\omega_r$ can both fail, for example for $A = \begin{bmatrix} 1 & \theta \\ \theta & 2\theta^2 \end{bmatrix}$ with $\theta > \alpha^{-1}$. But there can be no 2×2 pivots, as they would be indefinite (Problem 11.2). Therefore the factorization is $PAP^T = LDL^T$ for a diagonal D with positive diagonal entries.

11.7. That the growth factor bound is unchanged is straightforward to check. No 2×2 pivots are used for a positive definite matrix because, as before (Problem 11.2), any 2×2 pivot is indefinite. To show that no interchanges are required for a positive definite matrix we show that the second test, $\alpha\omega_1^2 \leq |a_{11}|\omega_r$, is always passed. The submatrix $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$ is positive definite, so $a_{11}a_{rr} - a_{r1}^2 > 0$. Hence

$$|a_{11}|\omega_r \geq a_{11}a_{rr} > a_{r1}^2 = \omega_1^2 \geq \alpha\omega_1^2,$$

as required.

11.8. Complete pivoting and rook pivoting both yield

$$PAP^T = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & \epsilon \\ 0 & \epsilon & 1 \end{bmatrix} = \begin{bmatrix} 1 & & \\ 1 & 1 & \\ & -\epsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & & \\ & -1 & \\ & & 1 + \epsilon^2 \end{bmatrix} \begin{bmatrix} 1 & 1 & \\ & 1 & -\epsilon \\ & & 1 \end{bmatrix}.$$

Unlike for partial pivoting, $\|L\|_\infty$ is bounded independently of ϵ as $\epsilon \rightarrow 0$.

11.9. (a) The nonsingularity of A follows from the factorization

$$\begin{bmatrix} H & B^T \\ B & -G \end{bmatrix} = \begin{bmatrix} I & 0 \\ BH^{-1} & I \end{bmatrix} \begin{bmatrix} H & B^T \\ 0 & -(G + BH^{-1}B^T) \end{bmatrix},$$

since $G + BH^{-1}B^T$ is symmetric positive definite.

(b) It suffices to show that the first $n + m - 1$ leading principal submatrices of $\Pi^T A \Pi$ are nonsingular for any permutation matrix Π . But these submatrices are of the form $\overline{\Pi}^T A_p \overline{\Pi}$, where A_p is a principal submatrix of A and $\overline{\Pi}$ is a permutation matrix. Any such A_p is of the form

$$A_p = \begin{bmatrix} H_p & B_p^T \\ B_p & -G_p \end{bmatrix},$$

where H_p and G_p are principal submatrices of H and G , respectively, and so are positive definite. Thus A_p is symmetric quasidefinite and hence nonsingular by (a), as required.

(c)

$$AS = \begin{bmatrix} H & -B^T \\ B & G \end{bmatrix},$$

so $(AS + (AS)^T)/2 = \text{diag}(H, G)$, which is symmetric positive definite.

12.1.

$$|A||x| \leq \|A\|_\infty \|x\|_\infty e \leq \|A\|_\infty \|x\|_\infty \frac{|x|}{\min_i |x_i|} = \|A\|_\infty \frac{\max_i |x_i|}{\min_i |x_i|} |x|.$$

12.2. The inequality (12.5) yields, with $\hat{x}_0 := 0$, and dropping the subscripts on the g_i and G_i ,

$$|x - \hat{x}_2| \leq G|x - \hat{x}_1| + g \leq G^2|x - \hat{x}_0| + Gg + g = G^2|x| + (G + I)g.$$

Now $G \approx |F| \leq 2\gamma_n |A^{-1}| |\hat{L}| |\hat{U}| \lesssim 2\gamma_n |A^{-1}| |B| |A|$, where $B = |\hat{L}| |\hat{L}^{-1}|$ can be assumed to have a modest ∞ -norm. Now, using Problem 12.1,

$$\begin{aligned} G^2|x| &\lesssim 2n^2 u^2 |A^{-1}| (B|A| |A^{-1}| B) |A| |x| \\ &\lesssim 2n^2 u^2 \|B|A| |A^{-1}| B\|_\infty \sigma(A, x) |A^{-1}| |A| |x| \\ &\leq 2n^2 \|B\|_\infty^2 \cdot u \text{cond}(A^{-1}) \sigma(A, x) \cdot u |A^{-1}| |A| |x| \\ &\lesssim 2n^2 \|B\|_\infty^2 \cdot u |A^{-1}| |A| |x|, \end{aligned}$$

under the conditions of Theorem 12.4. The term Gg can be bounded in a similar way. The required bound for $\|x - \hat{x}_2\|_\infty / \|x\|_\infty$ follows.

13.1. The equations for the blocks of L and U are $U_{11} = A_{11}$ and

$$\left. \begin{aligned} L_{i,i-1} &= A_{i,i-1} U_{i-1,i-1}^{-1} \\ U_{ii} &= A_{ii} - L_{i,i-1} A_{i-1,i} \end{aligned} \right\} \quad i \geq 2.$$

First, consider the case where A is block diagonally dominant by columns. We prove by induction that

$$\|A_{k,k-1}\| \|U_{k-1,k-1}^{-1}\| \leq 1, \quad k \geq 2.$$

which implies both the required bounds. This inequality clearly holds for $k = 2$; suppose it holds for $k = i$. We have

$$U_{ii} = A_{ii}(I - A_{ii}^{-1} L_{i,i-1} A_{i-1,i}) =: A_{ii}(I - P),$$

where

$$\begin{aligned} \|P\| &\leq \|A_{ii}^{-1}\| (\|A_{i,i-1}\| \|U_{i-1,i-1}^{-1}\|) \|A_{i-1,i}\| \\ &\leq \|A_{ii}^{-1}\| \|A_{i-1,i}\| \\ &\leq 1 - \|A_{ii}^{-1}\| \|A_{i+1,i}\|, \end{aligned}$$

using the block diagonal dominance for the last inequality. Hence

$$\|U_{ii}^{-1}\| \leq \frac{\|A_{ii}^{-1}\|}{1 - \|P\|} \leq \frac{1}{\|A_{i+1,i}\|},$$

as required.

The proof for A block diagonally dominant by rows is similar. The inductive hypothesis is that

$$\|U_{k-1,k-1}^{-1}\| \|A_{k-1,k}\| \leq 1,$$

and with P defined as before, we have

$$\begin{aligned} \|P\| &\leq \|A_{ii}^{-1}\| \|A_{i,i-1}\| \|U_{i-1,i-1}^{-1}\| \|A_{i-1,i}\| \\ &\leq \|A_{ii}^{-1}\| \|A_{i,i-1}\| \\ &\leq 1 - \|A_{ii}^{-1}\| \|A_{i,i+1}\|, \end{aligned}$$

giving $\|U_{ii}^{-1}\| \leq \|A_{i,i+1}\|^{-1}$, as required.

For block diagonal dominance by columns in the ∞ -norm we have $\|L\|_\infty \leq 2$ and $\|U\|_\infty \leq 3\|A\|_\infty$, so block LU factorization is stable. If A is block diagonally dominant by rows, stability is assured if $\|A_{i,i-1}\|/\|A_{i-1,i}\|$ is suitably bounded for all i .

13.2. The block 2×2 matrix

$$\left[\begin{array}{cc|cc} \epsilon & 2 & 1 & 0 \\ 2 & \epsilon & 0 & 1 \\ \hline 1 & 0 & \epsilon & 2 \\ 0 & 1 & 2 & \epsilon \end{array} \right]$$

is block diagonally dominant by rows and columns in the 1- and ∞ -norms for $\epsilon = 1/2$, but is not point diagonally dominant by rows or columns. The block 2×2 matrix

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 2 & 1 \\ 0 & 0 & 1 & 1 \end{array} \right]$$

is point diagonally dominant by rows and columns but not block diagonally dominant by rows or columns in the ∞ -norm or 1-norm.

13.3. No. A counterexample is the first matrix in the solution to Problem 13.2, with $\epsilon = 1/2$, which is clearly not positive definite because the largest element does not lie on the diagonal.

13.4. From (13.2) it can be seen that $(A^{-1})_{21} = -S^{-1}A_{21}A_{11}^{-1}$, where the Schur complement $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$. Hence

$$\|A_{21}A_{11}^{-1}\| \leq n\|S\| \|(A^{-1})_{21}\| \leq n\|S\| \|A^{-1}\|,$$

where the factor n comes from the fact that this norm is not consistent—see §6.2. S is the trailing submatrix that would be obtained after $r-1$ steps of GE. It follows immediately that $\|S\| \leq \rho_n\|A\|$.

For the last part, note that $\|S^{-1}\| \leq \|A^{-1}\|$, because S^{-1} is the $(2, 2)$ block of A^{-1} , as is easily seen from (13.2).

13.5. The proof is similar to that of Problem 8.7(a). We will show that $\|A_{11}^{-T} A_{21}^T\|_\infty \leq 1$. Let $y = A_{11}^{-T} A_{21}^T x$ and let $|y_k| = \|y\|_\infty$. The k th equation of $A_{11}^T y = A_{21}^T x$ gives

$$a_{kk} y_k = \sum_{i=r+1}^n a_{ik} x_{i-r} - \sum_{\substack{i=1 \\ i \neq k}}^r a_{ik} y_i.$$

Hence

$$\|y\|_\infty = |y_k| \leq \|x\|_\infty \sum_{i=r+1}^n \left| \frac{a_{ik}}{a_{kk}} \right| + \|y\|_\infty \sum_{\substack{i=1 \\ i \neq k}}^r \left| \frac{a_{ik}}{a_{kk}} \right|,$$

which yields $\|y\|_\infty \leq \|x\|_\infty$ in view of the column diagonal dominance, as required.

13.7. We have the block LU factorization

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} I & 0 \\ CA^{-1} & I \end{bmatrix} \begin{bmatrix} A & B \\ 0 & D - CA^{-1}B \end{bmatrix},$$

so that

$$\det(X) = \det(A) \det(D - CA^{-1}B).$$

Hence $\det(X) = \det(AD - ACA^{-1}B)$, which equals $\det(AD - CB)$ if C commutes with A .

13.8. The result is obtained by inverting

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} I & 0 \\ CA^{-1} & I \end{bmatrix} \begin{bmatrix} A & B \\ 0 & S \end{bmatrix}.$$

13.9. We have

$$\begin{bmatrix} I & A \\ B & I \end{bmatrix} = \begin{bmatrix} I & 0 \\ B & I \end{bmatrix} \begin{bmatrix} I & A \\ 0 & I - BA \end{bmatrix} = \begin{bmatrix} I & A \\ 0 & I \end{bmatrix} \begin{bmatrix} I - AB & 0 \\ B & I \end{bmatrix}.$$

Inverting both factorizations and equating (1,1) blocks gives the required relation. The Sherman–Morrison–Woodbury formula is obtained by setting $A = T^{-1}U$ and $B = W^{-1}V^T$.

14.2. The new bounds have norms in place of absolute values and the constants are different.

14.3. Immediate from $AX - I = A(XA - I)A^{-1}$.

14.4. With $0 < \epsilon \ll 1$, let

$$A = \begin{bmatrix} \frac{1}{\epsilon} & 1 \\ \frac{1}{\epsilon^2} - 1 & \frac{1}{\epsilon} \end{bmatrix}, \quad X = \begin{bmatrix} 1 - \epsilon + \frac{2}{\epsilon} & -2 - \epsilon \\ 2 - \epsilon + \frac{1}{\epsilon} - \frac{1}{\epsilon^2} & -1 - \epsilon + \frac{1}{\epsilon} \end{bmatrix}.$$

Then

$$AX = \begin{bmatrix} O(\epsilon^{-2}) & O(\epsilon^{-1}) \\ O(\epsilon^{-3}) & O(\epsilon^{-2}) \end{bmatrix}$$

while

$$XA = \begin{bmatrix} 1 + \epsilon & -\epsilon \\ \epsilon & 1 - \epsilon \end{bmatrix}.$$

Hence $\|AX - I\|/\|XA - I\| \rightarrow \infty$ as $\epsilon \rightarrow 0$. Note that in this example *every* element of $AX - I$ is large.

14.5. We have $\hat{x} = fl(\hat{X}b) = \hat{X}b + f_1$, where $|f_1| \leq \gamma_n |\hat{X}| |b|$. Hence $A\hat{x} = A\hat{X}b + Af_1 = b + r_1$, where $|r_1| \leq u|A||\hat{X}||b| + \gamma_n |A||\hat{X}||b| \leq \gamma_{n+1} |A||\hat{X}||b|$. Similarly,

$$\hat{y} = fl(\hat{Y}b) = \hat{Y}b + f_2, \quad |f_2| \leq \gamma_n |\hat{Y}| |b|. \quad (\text{A.13})$$

Hence $A\hat{y} = A\hat{Y}b + Af_2 = A(\hat{Y}A)x + Af_2 = b + r_2$, where

$$|r_2| \leq u|A||\hat{Y}||A||x| + \gamma_n |A||\hat{Y}||b| \leq \gamma_{n+1} |A||\hat{Y}||A||x|.$$

Clearly,

$$|x - \hat{x}| \leq (n+1)u|A^{-1}||A||A^{-1}||b| + O(u^2).$$

From (A.13), $\hat{y} = \hat{Y}Ax + f_2$, so

$$|y - \hat{y}| \leq u|\hat{Y}||A||x| + \gamma_n |\hat{Y}||b| \leq (n+1)u|A^{-1}||A||x| + O(u^2).$$

The first conclusion is that the approximate left inverse yields the smaller residual bound, while the approximate right inverse yields the smaller forward error bound. Therefore which inverse is “better” depends on whether a small backward error or a small forward error is desired. The second conclusion is that neither approximate inverse yields a componentwise backward stable algorithm, despite the favourable assumptions on $\hat{X}$ and $\hat{Y}$. Multiplying by an explicit inverse is simply not a good way to solve a linear system.

14.6. Here is a hint: notice that the matrix on the front cover of the *LAPACK Users' Guide* has the form

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \text{diag}(L, A, P, A, C, K).$$

14.7. If the i th row of A contains all 1s then simply sum the elements in the i th row of the equation $AA^{-1} = I$.

14.8. $(A + iB)(P + iQ) = I$ is equivalent to $AP - BQ = I$ and $AQ + BP = 0$, or

$$\begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} P \\ Q \end{bmatrix} = \begin{bmatrix} I \\ 0 \end{bmatrix},$$

so X^{-1} is obtainable from the first n columns of Y^{-1} . The definiteness result follows from

$$\begin{aligned} (x + iy)^*(A + iB)(x + iy) &= x^T(Ax - By) + y^T(Ay + Bx) \\ &\quad + i[x^T(Ay + Bx) - y^T(Ax - By)] \\ &= \begin{bmatrix} x^T & y^T \end{bmatrix} \begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \end{aligned}$$

where we have used the fact that $A = A^T$ and $B = -B^T$. Doubling the dimension (from X to Y) multiplies the number of flops by a factor of 8, since the flop count for inversion is cubic in the dimension. Yet complex operations should, in theory, cost between about two and eight times as much as real ones (the extremes being for addition and division). The actual relative costs of inverting X and Y depend on the machine and the compiler, so it is not possible to draw any firm conclusions. Note also that Y requires twice the storage of X .

14.10. As in the solution to Problem 8.8, we have

$$\det(A + \alpha e_i e_j^T) = \det(A)(1 + \alpha e_j^T A^{-1} e_i).$$

If $(A^{-1})_{ji} = 0$, this expression is independent of α , and hence $\det(A)$ is independent of a_{ij} . (This result is clearly correct for a triangular matrix.) That $\det(A)$ can be independent of a_{ij} shows that $\det(A)$, or even a scaling of it, is an arbitrarily poor measure of conditioning, for $A + \alpha e_i e_j^T$ approaches a multiple of a rank-1 matrix as $\alpha \rightarrow \infty$.

14.11. That Hadamard's inequality can be deduced from QR factorization was noted by Householder [643, 1958, p. 341]. From $A = QR$ we have $q_k^T A = e_k^T R$, so that

$$|r_{kk}| = |q_k^T a_k| \leq \|a_k\|_2.$$

Hadamard's inequality follows since $|\det(A)| = |\det(R)| = |r_{11} \dots r_{nn}|$. There is equality when $|q_k^T a_k| = \|a_k\|_2$ for $k = 1:n$, that is, when $a_k = \alpha_k q_k$, $k = 1:n$, for some scalars α_k . In other words, there is equality when R in the QR factorization is diagonal.

14.12. (a) Straightforward. (b) For $A = U(1)$, $\psi(A) = \sqrt{n}!$. For the Pei matrix, $\psi(A) = (\alpha^2 + n - 1)^{n/2} / ((n + \alpha - 1)(\alpha - 1)^{n-1})$.

14.13. (a) The geometric mean of $\sigma_1^2/2, \sigma_1^2/2, \sigma_2^2, \dots, \sigma_{n-1}^2$ is

$$\left(\frac{1}{4} \sigma_1^4 \sigma_2^2 \dots \sigma_{n-1}^2 \right)^{1/n} = \left(\frac{1}{4} \frac{\sigma_1^2}{\sigma_n^2} \sigma_1^2 \sigma_2^2 \dots \sigma_n^2 \right)^{1/n} = \left(\frac{1}{2} \kappa_2(A) |\det(A)| \right)^{2/n}.$$

Since the geometric mean does not exceed the arithmetic mean,

$$\left(\frac{1}{2} \kappa_2(A) |\det(A)| \right)^{2/n} \leq \frac{1}{n} (\sigma_1^2 + \dots + \sigma_{n-1}^2) < \frac{1}{n} (\sigma_1^2 + \dots + \sigma_n^2) = \frac{1}{n} \|A\|_F^2,$$

which gives the required bound. (b) is trivial.

14.14. The key observation is that for a triangular system $Tx = b$, $T \in \mathbb{R}^{n \times n}$, the computed solution from substitution satisfies

$$(T + \Delta T)\hat{x} = b + \Delta b, \quad |\Delta T| \leq \gamma_n |T|, \quad \text{diag}(T) = 0, \quad |\Delta b| \leq \gamma_n |b|. \quad (\text{A.14})$$

This result holds for any order of evaluation and is proved (for any particular order of evaluation) using a variation on the proof of Lemma 8.2 in which we do not divide through by the product of $1 + \delta_i$ terms. Using (A.14) we have

$$\hat{d} = (1 + \theta_n) \det(T) (\eta - (h + \Delta h)^T (T + \Delta T)^{-1} (y + \Delta y)),$$

where $|\theta_n| \leq \gamma_n$, $|\Delta y| \leq \gamma_{n-1} |y|$, and $|\Delta h| \leq \gamma_{n-1} |h|$. But, since $\text{diag}(\Delta T) = 0$, $\det(T) = \det(T + \Delta T)$, hence

$$\hat{d} = \det(T + \Delta T) (\eta(1 + \theta_n) - (h + \widetilde{\Delta h})^T (T + \Delta T)^{-1} (y + \Delta y)),$$

where $|\widetilde{\Delta h}| \leq \gamma_{2n-1} |h|$. The conclusion is that $\hat{d} = \det(H + \Delta H)$, where $|\Delta H| \leq \gamma_{2n-1} |H|$.

If $\hat{f} = fl(\det(D^{-1}HD))$ then

$$\hat{f} = \det(D^{-1}HD + \Delta), \quad |\Delta| \leq \gamma_{2n-1} |D^{-1}HD| = \gamma_{2n-1} |D^{-1}| |H| |D|.$$

Thus

$$\hat{f} = \det(D^{-1}(H + D\Delta D^{-1})D) = \det(H + D\Delta D^{-1}), \quad |D\Delta D^{-1}| \leq \gamma_{2n-1} |H|.$$

A diagonal similarity therefore has no effect on the error bound and so there is no point in scaling H before applying Hyman's method.

14.15. We use the fact that $\det(A) = \prod_{i=1}^n \sigma_i(A)$, where $\sigma_i(A)$ is the i th largest singular value of A . We have

$$\frac{\det(A + \Delta A) - \det(A)}{\det(A)} = \prod_{i=1}^n \frac{\sigma_i(A + \Delta A)}{\sigma_i(A)} - 1.$$

Now, using standard singular value inequalities [509, 1996, §8.6],

$$\frac{\sigma_i(A + \Delta A)}{\sigma_i(A)} \leq \frac{\sigma_i(A) + \sigma_1(\Delta A)}{\sigma_i(A)} \leq 1 + \frac{\sigma_1(\Delta A)}{\sigma_n(A)} = 1 + \kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}$$

and

$$\frac{\sigma_i(A + \Delta A)}{\sigma_i(A)} \geq \frac{\sigma_i(A) - \sigma_1(\Delta A)}{\sigma_i(A)} \geq 1 - \frac{\sigma_1(\Delta A)}{\sigma_n(A)} = 1 - \kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}.$$

Hence

$$\prod_{i=1}^n \frac{\sigma_i(A + \Delta A)}{\sigma_i(A)} - 1 = \prod_{i=1}^n (1 + \theta_i) - 1, \quad |\theta_i| \leq \kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}, \quad i = 1:n,$$

and the result follows from Lemma 3.1.

15.1. We have

$$\begin{aligned} \| |A| |A^{-1}| |A| \|_\infty &= \| |A| |A^{-1}| |A| e \|_\infty \\ &= \| |A| |A^{-1}| D_1 e \|_\infty, \quad D_1 = \text{diag}(|A|e) \\ &= \| |A| |A^{-1}| D_1 \|_\infty \\ &\approx \| |A| |A^{-1}| D_1 \|_1 \\ &= \| D_1 |A^{-T}| |A^T| \|_\infty \\ &= \| D_1 |A^{-T}| D_2 \|_\infty, \quad D_2 = \text{diag}(|A^T|e) \\ &= \| D_1 A^{-T} D_2 \|_\infty, \end{aligned}$$

where $\approx$ means “within a factor n of”.

15.4. Straightforward, since L is unit lower triangular with $|l_{ij}| \leq 1$.

15.7. Let $D = \text{diag}(d_j)$, where $d_1 = 1$ and

$$d_j = \prod_{k=1}^{j-1} \frac{a_{k,k+1}}{a_{k+1,k}}, \quad j = 2:n.$$

Then $T = DA$ is tridiagonal, symmetric and irreducible. By applying Theorem 15.9 and using symmetry, we find that

$$(A^{-1})_{ij} = \begin{cases} x_i y_j d_j, & i \leq j, \\ y_i x_j d_j, & i \geq j. \end{cases}$$

There is one degree of freedom in the vectors x and y , which can be expended by setting $x_1 = 1$, say.

16.1. The equation would imply, on taking traces, $0 = \text{trace}(I)$, which is false.

16.2. It is easily checked that the differential equation given in the hint has the solution $Z(t) = e^{At} C e^{Bt}$. Integrating the differential equation between 0 and ∞ gives, assuming that $\lim_{t \rightarrow \infty} Z(t) = 0$,

$$-C = A \left(\int_0^\infty Z(t) dt \right) + \left(\int_0^\infty Z(t) dt \right) B.$$

Hence $-\int_0^\infty Z(t) dt = -\int_0^\infty e^{At} C e^{Bt} dt$ satisfies the Sylvester equation. For the Lyapunov equation, the integral $-\int_0^\infty e^{At} (-C) e^{A^T t} dt$ exists under the assumption on A , and the corresponding quadratic form is easily seen to be positive.

16.3. Since

$$\|AX + XA^T\|_F = \|(AX + XA^T)^T\|_F = \|AX^T + X^T A^T\|_F,$$

X^T is a minimizer if X is. If $X + X^T = 0$ then X is a skew-symmetric minimizer. Otherwise, note that the minimization problem is equivalent to finding the right singular vector corresponding to the smallest singular value $\sigma_{\min}$ of $P = I \otimes A + A \otimes I$. Since $\text{vec}(X)$ and $\text{vec}(X^T)$ (suitably normalized) are both singular vectors corresponding to $\sigma_{\min}$, so is their sum, $\text{vec}(X + X^T) \neq 0$. Thus the symmetric matrix $X + X^T$ is a minimizer.

Byers and Nash [195, 1987] investigate conditions under which a symmetric minimizer exists.

16.4. xLASY2 uses Gaussian elimination with complete pivoting to solve the 2×2 or 4×4 linear system that arises on converting to the Kronecker product form (16.2). For complete pivoting on a 4×4 matrix the growth factor is bounded by 4 (§9.4), versus 8 for partial pivoting. The reasons for using complete pivoting rather than partial pivoting are that, first, the small increase in cost is regarded as worthwhile for a reduction in the error bound by a factor of 2 and, second, complete pivoting reveals the rank better than partial pivoting, enabling better handling of ill-conditioned systems [46, 1993, p. 78].

17.1. Since $\rho(B) < 1$, a standard result (Problem 6.8) guarantees the existence of a consistent norm $\|\cdot\|_\rho$ for which $\|B\|_\rho < 1$. The series $\sum_{k=0}^\infty \|B^k\|_\rho \leq \sum_{k=0}^\infty \|B\|_\rho^k = (1 - \|B\|_\rho)^{-1}$ is clearly convergent, and so, by the equivalence of norms, $\sum_{k=0}^\infty \|B^k\|$ is convergent for any norm.

Since $(|B^k|)_{ij} \leq \|B^k\|_\infty$, the convergence of $\sum_{k=0}^\infty \|B^k\|_\infty$ ensures that of $\sum_{k=0}^\infty |B^k|$. (The convergence of $\sum_{k=0}^\infty |B^k|$ can also be proved directly using the Jordan canonical form.)

18.1. Let X_n be the $n \times n$ version of the upper triangular matrix

$$\begin{bmatrix} 1 & -\theta & -\theta & -\theta \\ & 1 & -\theta & -\theta \\ & & 1 & -\theta \\ & & & 1 \end{bmatrix} D^{-1}, \quad \theta > 0,$$

where $D = \text{diag}(\sqrt{1 + (j-1)\theta^2})$, so that $\|X_n(:, j)\|_2 = 1$, $j = 1:n$. From (8.4), X_n^{-1} is the obvious $n \times n$ analogue of

$$D \begin{bmatrix} 1 & \theta & \theta(\theta+1) & \theta(\theta+1)^2 \\ & 1 & \theta & \theta(\theta+1) \\ & & 1 & \theta \\ & & & 1 \end{bmatrix}.$$

Let $A = \text{diag}(0, \dots, 0, \lambda)$, with $|\lambda| < 1$. Then

$$\begin{aligned}\|A^k\|_2 &= \|X_n A^k X_n^{-1}\|_2 = |\lambda|^k \|X_n e_n e_n^T X_n^{-1}\|_2 \\ &= |\lambda|^k \|X_n(:, n)\|_2 \|X_n^{-1}(n, :)\|_2 = |\lambda|^k \sqrt{1 + (n-1)\theta^2}.\end{aligned}$$

But

$$\kappa_2(X_n)\rho(A)^k \geq |\lambda|^k \max_{i,j} |x_{ij}| \max_{i,j} |(X_n^{-1})_{ij}| \geq |\lambda|^k \theta^{n-1}.$$

Therefore $\kappa_2(X_n)\rho(A)^k/\|A^k\|_2 \rightarrow \infty$ as $\theta \rightarrow \infty$. There is still the question of whether X_n is optimally scaled. We can perturb the zero eigenvalues of A to distinct, sufficiently small numbers so that $\|A^k\|_2$ is essentially unchanged and so that the only freedom in X_n is a diagonal scaling. Since X_n has columns of unit 2-norm, $\min_{F=\text{diag}(f_i)} \kappa_2(X_n F) \geq n^{-1/2} \kappa_2(X_n)$ (Theorem 7.5), so even for the optimally scaled X_n the bound can be arbitrarily poor.

18.2. A simple example is

$$A = \begin{bmatrix} 1 & 2 & 2 \\ 2 & 1 & 2 \\ \alpha & -\alpha & 1 \end{bmatrix},$$

for which $\lambda(A) = \{-1, 1, 3\}$ and $\lambda(|A|) = \{-1, 2 \pm (4\alpha + 1)^{1/2}\}$, so $\rho(|A|)/\rho(A) \rightarrow \infty$ as $\alpha \rightarrow \infty$.

19.1. For the Householder matrix $I - (2/v^T v)vv^T$ there is an eigenvalue -1 with eigenvector v , and there are $n-1$ eigenvalues 1 , with eigenvectors any basis for $\text{span}(v)^\perp$. A Givens matrix $G(i, j, \theta)$ has eigenvalues $e^{\pm i\theta}$, together with $n-2$ eigenvalues 1 .

19.2. Straightforward manipulation shows that a bound holds of the form $cnu + O(u^2)$, where c is a constant of order 10.

19.3. We must have $x^*Px = x^*y$, so x^*y must be real. Also, we need $x^*x = y^*y$ and $x \neq y$. Then, with $v = x - y$, $v^*v = 2v^*x$, so $Px = y$.

LAPACK uses modified Householder matrices $P = I - \tau vv^*$ that are unitary but not Hermitian (τ is not real). The benefit of this approach is that such a P can always be chosen so that $Px = \alpha e_1$, with α real (for this equation to hold for a genuine Householder matrix P it would be necessary that x_1 be real). For more details see Lehoucq [780, 1996].

19.4. False. $\det(P) = -1$ for any Householder matrix and $\det(G) = 1$ for any Givens matrix, so $\det(Q) = 1$. Moreover, whereas P is generally a full, symmetric matrix, Q has some zero entries and is generally nonsymmetric.

19.5. The inequalities follow from the fact that, in the notation of (19.3), $\|x_k\|_2 = |r_{kk}|$ and $\|c_k(:, j)\|_2^2 = \sum_{i=k}^j r_{ij}^2$, the latter equality being a consequence of the invariance of the 2-norm under orthogonal transformations. The last part follows because QR factorization with column pivoting on A is essentially equivalent to Cholesky factorization with complete pivoting applied to $A^T A$.

19.6. The second inequality in (19.38) follows from (19.1) and the first from

$$v_k^T v_k = |2\sigma_k(\sigma_k - a_{kk}^{(k)})| \geq 2\sigma_k^2 = 2\|a_k^{(k)}(k:m)\|_2^2.$$

From the proof of Lemma 19.2 we see that (19.39) holds with $f_j^{(k)}(1:k-1) = 0$ and

$$|f_j^{(k)}| \leq u|\hat{a}_j^{(k)}| + \tilde{\gamma}_{m-k}(|\beta_k||v_k|^T|\hat{a}_j^{(k)}|)|v_k|.$$

Now

$$\begin{aligned} (|\beta_k| |v_k|^T |\widehat{a}_j^{(k)}|) |v_k| &= 2 \frac{|v_k|^T |\widehat{a}_j^{(k)}|}{\|v_k\|_2^2} |v_k| \\ &\leq 2 \frac{\|\widehat{a}_j^{(k)}(k:m)\|_2}{\|v_k(k:m)\|_2} |v_k| \\ &\leq \sqrt{2} \frac{\|\widehat{a}_j^{(k)}(k:m)\|_2}{\|\widehat{a}_k^{(k)}(k:m)\|_2} |v_k|, \end{aligned}$$

using (19.38), and (19.40) follows. Consider a badly row-scaled problem and $k = 1$, and assume that v_1 has the same scaling as $a_j^{(1)}$ (which row pivoting or row sorting both ensure). Then the bound (19.40) shows that the error will scale in the same way as long as $\|\widehat{a}_j^{(1)}(1:m)\|_2 / \|\widehat{a}_1^{(1)}(1:m)\|_2$ is not too large; column pivoting guarantees that this ratio is bounded by 1. For the full development of this argument see Higham [614, 2000].

19.7. If $y = |W|x$ and W comprises r disjoint rotations then, in a rather loose notation,

$$\begin{aligned} \|y\|_2^2 &\leq \sum_{k=1}^r \left\| \begin{bmatrix} c_k & s_k \\ -s_k & c_k \end{bmatrix} \begin{bmatrix} x_{i_k} \\ x_{j_k} \end{bmatrix} \right\|_2^2 + \sum_{k=2r+1}^m x_{i_k}^2 \\ &\leq 2 \sum_{k=1}^{2r} x_{i_k}^2 + \sum_{k=2r+1}^m x_{i_k}^2 \leq 2\|x\|_2^2. \end{aligned}$$

19.8. Straightforward. This problem shows that the CGS and MGS methods correspond to two different ways of representing the orthogonal projection onto $\text{span}\{q_1, \dots, q_j\}$.

19.9. Assume, without loss of generality, that $\|a_1\|_2 \leq \|a_2\|_2$. If E is any matrix such that $A + E$ is rank deficient then

$$0 = \sigma_{\min}(A + E) \geq \sigma_{\min}(A) - \|E\|_2 \quad \Rightarrow \quad \sigma_{\min}(A) \leq \|E\|_2.$$

We take $E = [e_1, 0]$, where e_1 is chosen so that $e_1^T a_1 = 0$ and $a_1 + e_1 = \alpha a_2$, for some α . From Pythagoras's theorem we have that $\|e_1\|_2 = \tan \theta \|a_1\|_2$, and so

$$\sigma_{\min}(A) \leq \tan \theta \|a_1\|_2 = \tan \theta \min(\|a_1\|_2, \|a_2\|_2).$$

Together with the trivial bound $\sigma_{\max}(A) = \|A\|_2 \geq \max(\|a_1\|_2, \|a_2\|_2)$, this yields the result.

19.10. We find that

$$\widehat{Q}_{CGS} = \begin{bmatrix} 1 & 0 & 0 \\ \epsilon & -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}, \quad \widehat{Q}_{MGS} = \begin{bmatrix} 1 & 0 & 0 \\ \epsilon & -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ 0 & 0 & \sqrt{\frac{2}{3}} \end{bmatrix}.$$

For $\widehat{Q}_{MGS}$, $|\widehat{q}_i^T \widehat{q}_j| \leq \epsilon / \sqrt{2}$ for $i \neq j$, but for $\widehat{Q}_{CGS}$, $|\widehat{q}_2^T \widehat{q}_3| = 1/2$. It is easy to see that $fl(1 + \epsilon^2) = 1$ implies $\epsilon \leq \sqrt{u}$ and that $\kappa_2(A) = \epsilon^{-1} \sqrt{3 + \epsilon^2}$. Hence for $\widehat{Q}_{MGS}$, $|\widehat{q}_i^T \widehat{q}_j| \lesssim \kappa_2(A)u / \sqrt{6}$, showing, as expected, that the loss of orthogonality for MGS is bounded in terms of $\kappa_2(A)u$.

19.11. P is the product $P_n \dots P_1$, where P_k is defined in (19.28). Since $v_k^T v_{k-1} = e_k^T e_{k-1} + q_k^T q_{k-1} = 0$, we have

$$P_k P_{k-1} = (I - v_k v_k^T)(I - v_{k-1} v_{k-1}^T) = I - v_k v_k^T - v_{k-1} v_{k-1}^T.$$

Hence

$$P = I - \sum_{k=1}^n v_k v_k^T = I - \sum_{k=1}^n \begin{bmatrix} e_k e_k^T & -e_k q_k^T \\ -q_k e_k^T & q_k q_k^T \end{bmatrix}.$$

This is of the required form, with $Q = [q_1, \dots, q_n]$ the matrix from the MGS method applied to A .

19.12. With Q defined as in the hint,

$$\begin{aligned} \Delta A &= QR - A = (Q - P_{21})R + \Delta A_2 \\ &= (VW^T - P_{21})R + \Delta A_2 \\ &= V(I - S)W^T R + \Delta A_2 \\ &= V(I + S)^{-1}C^2W^T R + \Delta A_2 \\ &= V(I + S)^{-1}W^T \cdot WCU^T \cdot UCW^T \cdot R + \Delta A_2 \\ &= V(I + S)^{-1}W^T \cdot P_{11}^T P_{11} R + \Delta A_2 \\ &= V(I + S)^{-1}W^T \cdot P_{11}^T \Delta A_1 + \Delta A_2, \end{aligned}$$

and the result follows.

19.13. To produce the Householder method's P we have to explicitly form the product of the individual Householder transformations. As long as this is done in a stable way the computed $\hat{P}$ is guaranteed to be nearly orthogonal. MGS's $\hat{Q}$ is formed in an algebraically different way, and the rounding errors in its formation are different from those in the formation of P ; in other words, $\hat{Q}$ is not a submatrix of $\hat{P}$. Consequently, there is no reason to expect $\hat{Q}$ to be nearly orthonormal. Further insight can be gained by a detailed look at the structure of the computed $\hat{P}$; see Björck and Paige [131, 1992, §4].

19.14. It is straightforward to show that $A^T A - I = (A - U)^T (A + U)$. Taking norms gives the lower bound. Since $A + U = U(H + I)$ we have, from the previous relation,

$$(A - U)^T U = (A^T A - I)(H + I)^{-1}.$$

Hence

$$\|A - U\|_2 = \|(A - U)^T U\|_2 \leq \|A^T A - I\|_2 \|(H + I)^{-1}\|_2 \leq \frac{\|A^T A - I\|_2}{1 + \sigma_{\min}(A)},$$

since the eigenvalues of H are the singular values of A . In fact, the result holds for any unitarily invariant norm (but the denominators must be left unchanged).

20.1. One approach is to let x be a solution and y an arbitrary vector, and consider $f(\alpha) = \|A(x + \alpha y) - b\|_2^2$. By expanding this expression it can be seen that if the normal equations are not satisfied then α and y can be chosen so that $f(\alpha) < f(0)$. Alternatively, note that for $f(x) = (b - Ax)^T (b - Ax) = x^T A^T A x - 2b^T A x + b^T b$ we have $\nabla f(x) = 2(A^T A x - A^T b)$ and $\nabla^2 f(x) = 2A^T A$, so any solution of the normal equations is a local minimum of f , and hence a global minimum since f is quadratic. The normal equations can be written as $(b - Ax)^T A = 0$, which shows that the residual $b - Ax$ is orthogonal to $\text{range}(A)$.

20.2. $A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$. The computed upper triangular QR factor $\hat{R}$ satisfies $A + \Delta A = Q \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix}$ with $\|\Delta a_j\|_2 \leq \tilde{\gamma}_{mn} \|a_j\|_2$, by Theorem 19.4. By Lemma 19.3, the computed transformed right-hand side satisfies $\hat{c} = Q^T(b + \Delta b)$, with $\|\Delta b\|_2 \leq \tilde{\gamma}_{mn} \|b\|_2$.

By Theorem 8.5, the computed solution $\hat{x}$ to the triangular system $\hat{R}x = \hat{c}$ satisfies

$$(\hat{R} + \Delta R)\hat{x} = \hat{c}, \quad |\Delta R| \leq \gamma_n |\hat{R}|.$$

So $\hat{x}$ exactly solves the LS problem

$$\begin{bmatrix} \hat{R} + \Delta R \\ 0 \end{bmatrix} x \approx \begin{bmatrix} \hat{c} \\ \hat{d} \end{bmatrix},$$

which is equivalent to the LS problem, on premultiplying by Q ,

$$(A + \overline{\Delta A})x := \left(A + \Delta A + Q \begin{bmatrix} \Delta R \\ 0 \end{bmatrix} \right) x \approx b + \Delta b.$$

We have

$$\begin{aligned} \|\overline{\Delta a_j}\|_2 &\leq \|\Delta a_j\|_2 + \gamma_n \|\hat{r}_j\|_2 \\ &= \|\Delta a_j\|_2 + \gamma_n \|a_j + \Delta a_j\|_2 \\ &\leq \tilde{\gamma}_{mn} \|a_j\|_2. \quad \square \end{aligned}$$

20.3. A straightforward verification.

20.4. Notice that

$$\|AX - I_m\|_F^2 = \|A[x_1, \dots, x_m] - [e_1, \dots, e_m]\|_F^2 = \sum_{i=1}^m \|Ax_i - e_i\|_2^2,$$

which is minimized if $\|Ax_i - e_i\|_2$ is minimized for $i = 1:m$. Thus we have m independent LS problems. The solution is $x_i = A^+ e_i$, $i = 1:m$, that is, $X = A^+ I_m = A^+$. This is the unique solution only if A has full rank, but it is always the unique minimum Frobenius norm solution.

20.5. By Theorem 19.13 there is an orthonormal matrix $[W_1, w_{n+1}] \in \mathbb{R}^{m \times (n+1)}$ such that

$$\begin{bmatrix} A + \Delta A & b + \Delta b \end{bmatrix} = \begin{bmatrix} W_1 & w_{n+1} \end{bmatrix} \begin{bmatrix} \hat{R} & \hat{z} \\ 0 & \hat{\rho} \end{bmatrix},$$

where $\|\Delta a_j\|_2 \leq c_{m,n} u \|a_j\|_2$ and $\|\Delta b\|_2 \leq c_{m,n} u \|b\|_2$. The computed solution $\hat{x}$ to $\hat{R}x = \hat{z}$ satisfies

$$(\hat{R} + \Delta R)\hat{x} = \hat{z}, \quad |\Delta R| \leq \gamma_n |\hat{R}|.$$

Therefore $\hat{x}$ exactly solves the LS problem

$$\begin{bmatrix} \hat{R} + \Delta R \\ 0 \end{bmatrix} x \approx \begin{bmatrix} \hat{z} \\ \hat{\rho} \end{bmatrix},$$

which, on premultiplying by $[W_1, w_{n+1}]$, is equivalent to the LS problem

$$W_1(\hat{R} + \Delta \hat{R})x \approx W_1 \hat{z} + w_{n+1} \hat{\rho},$$

or

$$(A + \overline{\Delta A})x := (A + \Delta A + W_1 \Delta R)x \approx b + \Delta b.$$

We have

$$\|\overline{\Delta a_j}\|_2 \leq \|\Delta a_j\|_2 + \gamma_n \|\hat{r}_j\|_2 \leq c_{m,n} u \|a_j\|_2.$$

20.6. Subtracting $\hat{r}^T \hat{r} = b^T \hat{r} - \hat{x}^T A^T \hat{r}$ from $r^T r = r^T b$, we have

$$\begin{aligned} r^T r - \hat{r}^T \hat{r} &= b^T (r - \hat{r}) + \hat{x}^T A^T \hat{r} \\ &= b^T A(\hat{x} - x) + \hat{x}^T (A^T b - A^T A \hat{x}) \\ &= x^T A^T A(\hat{x} - x) + \hat{x}^T (A^T b - A^T A \hat{x}) \\ &= (x - \hat{x})^T (A^T A \hat{x} - A^T b) \\ &= (x - \hat{x})^T (\Delta c - \Delta A \hat{x}) \quad \text{by (20.12).} \end{aligned}$$

Taking norms gives the result.

20.7. By constructing a block LU factorization it is straightforward to show that

$$\det(C(\alpha) - \lambda I) = (\alpha - \lambda)^{m-n} \det(A^T A + \lambda(\alpha - \lambda)I).$$

Hence the eigenvalues of $C(\alpha)$ are $\lambda = \alpha$ ($m - n$ times) together with the solutions of $\lambda(\alpha - \lambda) = -\sigma_i^2$, namely, $\lambda = \alpha/2 \pm (\alpha^2/4 + \sigma_i^2)^{1/2}$, $i = 1:n$.

Now

$$\sigma_{\min}(C(\alpha)) = \min \left\{ \alpha, \left(\frac{\alpha^2}{4} + \sigma_n^2 \right)^{1/2} - \frac{\alpha}{2} \right\},$$

so the minimum condition number must occur when $\alpha = (\alpha^2/4 + \sigma_n^2)^{1/2} - \alpha/2$. This gives $\alpha = \sigma_n/\sqrt{2}$, for which

$$\kappa_2(C(\alpha)) = \frac{1}{2} + \left(\frac{1}{4} + 2\kappa_2(A)^2 \right)^{1/2}.$$

The lower bound for the maximum is achieved for $\alpha = \sigma_1/\sqrt{2}$.

20.8. Let $y = 0$ and $\|(A + \Delta A)y - b\|_2 = \min$. If $b = 0$ then we can take $\Delta A = 0$. Otherwise, the normal equations tell us that $(A + \Delta A)^T b = 0$, so $\|\Delta A\|_2 \geq \|A^T b\|_2 / \|b\|_2$. This lower bound is achieved, and the normal equations satisfied, for $\Delta A^T = -(A^T b)b^T / b^T b$. Hence, for $\theta = \infty$,

$$\eta_F(0) = \begin{cases} \|A^T b\|_2 / \|b\|_2, & b \neq 0, \\ 0, & b = 0. \end{cases}$$

20.9. For the case $\lambda_* < 0$ we have

$$\begin{aligned} \eta_F(y) &= \lambda_{\min} \left(AA^T - \mu \frac{rr^T}{\|y\|_2^2} + \mu \frac{\|r\|_2^2}{\|y\|_2^2} I \right)^{1/2} \\ &= \lambda_{\min} \left(AA^T + \mu \frac{\|r\|_2^2}{\|y\|_2^2} \left(I - \frac{rr^T}{\|r\|_2^2} \right) \right)^{1/2} \\ &= \lambda_{\min} \left(AA^T + \mu \frac{\|r\|_2^2}{\|y\|_2^2} \left(I - \frac{rr^T}{\|r\|_2^2} \right)^2 \right)^{1/2} \\ &= \sigma_{\min} \left(\left[A \quad \sqrt{\mu} \frac{\|r\|_2}{\|y\|_2} \left(I - \frac{rr^T}{\|r\|_2^2} \right) \right] \right). \end{aligned}$$

20.10. This is a standard result that can be derived by considering the first- and second-order optimality conditions in the theory of constrained optimization.

20.11. Setting $B = 0$ and $d = 0$ and using $\|b\|_2 \leq \|r\|_2 + \|A\|_F \|x\|_2$ we obtain

$$\frac{\|\Delta x\|_2}{\|x\|_2} \leq \epsilon \tilde{\kappa}(A) \left(2 + (\tilde{\kappa}(A) + 1) \frac{\|r\|_2}{\|A\|_F \|x\|_2} \right) + O(\epsilon^2),$$

where $\tilde{\kappa}(A) = \|A\|_F \|A^+\|_2$, which is a first-order approximation of (20.1).

21.1. Setting the gradient of (21.13) to zero gives $A^T Ax - A^T b + c = 0$, which can be written as $y = b - Ax$, $A^T y = c$, which is (21.12). The Lagrangian for (21.14) is $L(y, x) = \frac{1}{2}(y - b)^T(y - b) + x^T(A^T y - c)$. We have $\nabla_y L(y, x) = y - b + Ax$ and $\nabla_x L(y, x) = A^T y - c$. Setting the gradients to zero gives the system (21.12).

22.1. The modified version has the same flop count as the original version.

22.4. The summation gives

$$\sum_{j=0}^n x^j \sum_{i=0}^n w_{ij} = \sum_{i=0}^n l_i(x) \equiv 1,$$

which implies the desired equality. It follows that all columns of V^{-1} except the first must have both positive and negative entries. In particular, $V^{-1} \geq 0$ is not possible. The elements of V^{-1} sum to 1, independently of the points α_i (see also Problem 14.7).

22.5. We have $U(i, :)T(\alpha_0, \dots, \alpha_n) = W(i, :)V(\alpha_0, \dots, \alpha_n)$. But $T = LV$, where L has the form illustrated for $n = 5$ by

$$L = \begin{bmatrix} 1 & & & & \\ 0 & 1 & & & \\ -1 & 0 & 2 & & \\ 0 & -3 & 0 & 4 & \\ 1 & 0 & -8 & 0 & 8 \end{bmatrix}$$

and $Le = e$, so $U(i, :)L = W(i, :)$, or $U(i, :)^T = L^{-T}W(i, :)^T$. But $L^{-T} \geq 0$ by the given x^n formula, so $\|L^{-T}\|_1 = \|L^{-1}\|_\infty = \|L^{-1}e\|_\infty = \|e\|_\infty = 1$, hence $\|U(i, :)\|_1 \leq \|W(i, :)\|_1$.

As an aside, we can evaluate $\|L\|_\infty$ as

$$\|L\|_\infty = |T_n(\sqrt{-1})| = \frac{1}{2}[(1 + \sqrt{2})^n + (1 - \sqrt{2})^n],$$

after a little trigonometric algebra.

22.7. Denote the matrix by C . For the zeros of T_{n+1} we have

$$CC^T = (n+1) \operatorname{diag}\left(1, \frac{1}{2}, \frac{1}{2}, \dots, \frac{1}{2}\right).$$

It follows that $\|C\|_2^2 = \sqrt{n+1}$, $\|C^{-1}\|_2^2 = 2/\sqrt{n+1}$, giving the result.

Extrema of T_n : we have

$$CDC^T = \frac{n}{2}D^{-1}, \quad D = \operatorname{diag}\left(\frac{1}{2}, 1, 1, \dots, 1, \frac{1}{2}\right).$$

Hence $B = CD^{1/2}$ satisfies $BB^T = \frac{n}{2}D^{-1}$, and so $\kappa_2(B) = \kappa_2(D)^{1/2} = \sqrt{2}$. Then $\kappa_2(C) \leq \kappa_2(D^{1/2})\kappa_2(B) = 2$.

22.8. The first expression follows from the formula (15.10) for the inverse of an upper bidiagonal matrix. The expression shows that a small componentwise relative change in a bidiagonal matrix causes only a small componentwise relative change in the inverse. For the second part, we note that in Algorithm 22.2, for the monomials, we have $U_k + \Delta U_k = \operatorname{diag}(1 + \epsilon_i)\tilde{U}_k$, $|\epsilon_i| \leq u$, where $\tilde{U}_k$ agrees with U_k everywhere except in certain superdiagonal elements, where $(\tilde{U}_k)_{i,i+1} = (U_k)_{i,i+1}(1 + \delta_{i,i+1})$, $|\delta_{i,i+1}| \leq u$. The result now follows by applying the first part (and noting that U_k is of dimension $n+1$).

22.9. The increasing ordering is never produced, since the algorithm must choose α_1 to maximize $|\alpha_1 - \alpha_0|$.

22.11. The dual condition number is

$$\frac{\| |V^{-T}| \| V^T \operatorname{diag}(0, 1, 2, \dots, n) a \|_\infty}{\| a \|_\infty}.$$

See Higham [581, 1987] for proofs.

23.1. Consider the case where $m \leq \min(n, p)$, and suppose $n = jm$ and $p = km$ for some integers j and k . Then the multiplication of the two matrices can be split into $m \times m$ blocks:

$$AB = \begin{bmatrix} A_1 & A_2 & \dots & A_j \end{bmatrix} \begin{bmatrix} B_{11} & \dots & B_{1k} \\ \vdots & & \vdots \\ B_{j1} & \dots & B_{jk} \end{bmatrix},$$

which involves a total of jk multiplications of $m \times m$ matrices, each involving $O(m^\alpha)$ operations. Thus the total number of operations is $O(jkm^\alpha)$, or $O(m^{\alpha-2}np)$, as required, and we can show similar results for the cases when n and p are the smallest dimensions.

23.2. $\frac{1}{2}n^3 + n^2$ multiplications and $\frac{3}{2}n^3 + 2n(n-1)$ additions.

23.3. For large $n = 2^k$, $S_n(8)/S_n(n) \approx 1.96 \times (7/8)^k$ and $S_n(1)/S_n(8) \approx 1.79$. The ratio $S_n(8)/S_n(n)$ measures the best possible reduction in the amount of arithmetic by using Strassen's method in place of conventional multiplication. The ratio $S_n(1)/S_n(8)$ measures how much more arithmetic is used by recurring down to the scalar level instead of stopping once the optimal dimension n_0 is reached. Of course, the efficiency of a practical code for Strassen's method also depends on the various non-floating point operations.

23.5. Apart from the differences in stability, the key difference is that Winograd's formula relies on commutativity and so cannot be generalized to matrices.

23.7. Some brief comments are given by Douglas, Heroux, Shishman, and Smith [351, 1994].

23.9. The inverse is

$$\begin{bmatrix} I & -A & AB \\ 0 & I & -B \\ 0 & 0 & I \end{bmatrix}.$$

Hence we can form AB by inverting a matrix of thrice the dimension. This result is not of practical value, but it is useful for computational complexity analysis.

25.1. (a) We have

$$\|x_{k+1} - a\| = \|G(x_k) - a + e_k\| \leq \beta \|x_k - a\| + \alpha. \quad (\text{A.15})$$

If $\|x_k - a\| \leq \alpha/(1 - \beta)$ then

$$\|x_{k+1} - a\| \leq \beta \frac{\alpha}{1 - \beta} + \alpha = \frac{\alpha}{1 - \beta}.$$

If $\|x_k - a\| > \alpha/(1 - \beta)$, so that $\beta \|x_k - a\| < \|x_k - a\| - \alpha$, then, using (A.15),

$$\|x_{k+1} - a\| < (\|x_k - a\| - \alpha) + \alpha = \|x_k - a\|.$$

(b) We break the proof into two cases. First, suppose that $\|x_m - a\| \leq \alpha/(1 - \beta)$ for some m . By part (a), the same inequality holds for all $k \geq m$ and we are finished. Otherwise, $\|x_k - a\| > \alpha/(1 - \beta)$ for all k . By part (a) the positive sequence $\|x_k - a\|$ is monotone decreasing and therefore it converges to a limit l . Suppose that $l = \alpha/(1 - \beta) + \delta$, where $\delta > 0$. Since $\beta < 1$, for some k we must have

$$\|x_k - a\| < \frac{\alpha}{1 - \beta} + \frac{\delta}{\beta}.$$

By (A.15),

$$\|x_{k+1} - a\| < \frac{\beta\alpha}{1 - \beta} + \delta + \alpha = \frac{\alpha}{1 - \beta} + \delta = l,$$

which is a contradiction. Hence $l = \alpha/(1 - \beta)$, as required.

(c) The vectors e_k can be regarded as representing the rounding errors on the k th iteration. The bound in (b) tells us that provided the error vectors are bounded by α , we can expect to achieve a relative error no larger than $\alpha/(1 - \beta)$. In other words, the result gives us an upper bound on the achievable accuracy. When this result is applied to stationary iteration, β is a norm of the iteration matrix; to make β less than 1 we may have to use a scaled norm of the form $\|A\| := \|XAX^{-1}\|$.

26.2. With $n = 3$ and almost any starting data, the backward error can easily be made of order 1, showing that the method is unstable. However, the backward error is found to be roughly of order $\kappa_\infty(A)u$, so the method may have a backward error bound proportional to $\kappa_\infty(A)$ (this is an open question).

27.1. (b) An optimizing compiler might convert the test `xp1 > 1` to `x+1 > 1` and then to `x > 0`. (For a way to stop this conversion in Fortran, see the solution to Problem 27.3.) Then the code would compute a number of order $2^{e_{\min}}$ instead of a number of order 2^{-t} .

27.3. The algorithm is based on the fact that the positive integers that can be exactly represented are $1, 2, \dots, \beta^t$ and

$$\beta^t + \beta, \beta^t + 2\beta, \beta^t + 3\beta, \dots, \beta^{t+1}, \beta^{t+1} + \beta^2, \dots$$

In the interval $[\beta^t, \beta^{t+1}]$ the floating point numbers are spaced β apart. This interval must contain a power of 2, $a = 2^k$. The first while loop finds such an a (or, rather, the floating point representation of such an a) by testing successive powers 2^i to see if both 2^i and $2^i + 1$ are representable. The next while loop adds successive powers of 2 until the next floating point number is found; on subtracting a the base β is produced. Finally t is determined as the smallest power of β for which the distance to the next floating point number exceeds 1.

The routine can fail for at least two reasons. First, an optimizing compiler might simplify the test `while (a+1) - a == 1` to `while 1 == 1`, thus altering the meaning of the program. Second, in the same test the result `(a+1)` might be held in an extra length register and the subtraction done in extra precision. The computation would then reflect this higher precision and not the intended one. We could try to overcome this problem by saving the result `a+1` in a variable, but an optimizing compiler might undo this effort by dispensing with the variable and storing the result in a register. In Fortran, the compiler's unwanted efforts can be nullified by a test of the form `while foo(a+1) - a == 1`, where `foo` is a function that simply returns its argument. The problems caused by the use of extra length registers were discussed by Gentleman and Marovich [476, 1974]; see also Miller [853, 1984, §2.2].

27.4. (This description is based on Schreiber [1022, 1989].) The random number generator in `matgen` repeats after 16384 numbers [744, 1998, §3.2.1.2, Thm. B]. The dimension $n = 512$ divides the period of the generator ($16384 = 512 \times 32$), with the effect that the first 32 columns of the matrix are repeated 16 times ($512 = 32 \times 16$), so the matrix has this structure:

```
B = rand(512,32);
A = [B, B, B, B, B, B, B, B, B, B, B, B, B, B, B, B];
```

Now consider the first 32 steps of Gaussian elimination. We apply 32 transformations to A that have the effect, in exact arithmetic, of making B upper trapezoidal. In floating point arithmetic, they leave a residue of small numbers (about $u \approx 10^{-7}$ in size) in rows 33 onward. Because of the structure of the matrix, identical small numbers occur in each of the 15 blocks of A to the right of the first. Thus, the remaining $(512 - 32) \times (512 - 32)$ submatrix has the same block structure as A (but with 15 block columns). Hence this process repeats every 32 steps:

after 32 steps the elements drop to $\approx 10^{-7}$;
 after 64 steps the elements drop to $\approx 10^{-14}$;
 after 96 steps the elements drop to $\approx 10^{-21}$;
 after 128 steps the elements drop to $\approx 10^{-28}$;
 after 160 steps the elements drop to $\approx 10^{-35}$;
 after 192 steps the elements would drop to $\approx 10^{-42}$,

but that is less than the underflow threshold. The actual pivots do not exactly match the analysis, which is probably due to rank deficiency of one of the submatrices generated. Also, underflow occurs earlier than predicted, apparently because two small numbers (both $O(10^{-21})$) are multiplied in a saxpy operation.

27.5. Let s_i and t_i denote the values of s and t at the start of the i th stage of the algorithm. Then

$$\sum_{k=1}^{i-1} x_k^2 = t_i^2 s_i. \quad (\text{A.16})$$

If $|x_i| > t_i$ then the algorithm uses the relation

$$\sum_{k=1}^i x_k^2 = t_i^2 s_i + x_i^2 = x_i^2 ((t_i/x_i)^2 s_i + 1),$$

so with $s_{i+1} = (t_i/x_i)^2 s_i + 1$ and $t_{i+1} = |x_i|$, (A.16) continues to hold for the next value of i . The same is true trivially in the case $|x_i| \leq t_i$.

This is a one-pass algorithm using n divisions that avoids overflow except, possibly, on the final stage in forming $t\sqrt{s}$, which can overflow only if $\|x\|_2$ does.

27.7. (a) The matrix $A := I + Y^2 = I - Y^T Y$ is symmetric positive semidefinite since $\|Y\|_2 \leq 1$, hence it has a (unique) symmetric positive semidefinite square root X satisfying $X^2 = A = I + Y^2$. The square root X is a polynomial in A (see, for example, Higham [580, 1987]) and hence a polynomial in Y ; therefore X and Y commute, which implies that $(X + Y)^T (X + Y) = (X - Y)(X + Y) = X^2 - Y^2 = I$, as required.

27.8. $|x|/3$ could underflow to zero, or become unnormalized, when $\sqrt{|x|}/\sqrt{3}$ does not.

Appendix B

Acquiring Software

Caveat receptor . . .

Anything free comes with no guarantee!

— JACK DONGARRA and ERIC GROSSE, *Netlib mail header*

In this appendix we provide information on how to acquire software mentioned in the book. First, we describe some basic aspects of the Internet.

B.1. Internet

A huge variety of information and software is available over the Internet, the worldwide combination of interconnected computer networks. The location of a particular object is specified by a URL, which stands for “Uniform Resource Locator”. Examples of URLs are

```
http://www.netlib.org/index.html  
ftp://ftp.netlib.org
```

The first example specifies a World Wide Web server (http = hypertext transfer protocol) together with a file in hypertext format (html = hypertext markup language), while the second specifies an anonymous ftp site. In any URL, the site address may, optionally, be followed by a filename that specifies a particular file. For more details about the Internet see on-line information, or one of the many books on the subject.

B.2. Netlib

Netlib is a repository of freely available mathematical software, documents, and databases of interest to the scientific computing community [350, 1987], [168, 1994]. It includes

- research codes,
- golden oldies (classic programs that are not available in standard libraries),
- the collected algorithms of the ACM,
- program libraries such as EISPACK, LINPACK, LAPACK, and MINPACK.
- back issues of NA-Digest, a weekly digest for the numerical analysis community,
- databases of conferences and performance data for a wide variety of machines.

Netlib also enables the user to download technical reports from certain institutions, to download software and errata for textbooks, and to search a “white pages” database.

Netlib can be accessed in several ways.

- Over the World Wide Web: `http://www.netlib.org`
- By anonymous ftp: `ftp://ftp.netlib.org`

- By electronic mail. For an introduction and master index, send a one-line email message as follows:
`mail netlib@netlib.org`
`send index`

Netlib is mirrored at various sites throughout the world.

B.3. MATLAB

MATLAB is a commercial program sold by The MathWorks, Inc. It runs on a variety of platforms. The MathWorks maintains a collection of user-contributed M-files, which is accessible over the Internet.

For information contact

The MathWorks, Inc.
3 Apple Hill Drive
Natick, MA 01760-2098
USA

Tel: +508-647-7000
Fax: +508-647-7001

Web: www.mathworks.com
Newsgroup: comp.soft-sys.matlab
FTP: [ftp.mathworks.com](ftp://ftp.mathworks.com)
E-mail: info@mathworks.com

B.4. NAG Library and NAGWare F95 Compiler

The Numerical Algorithms Group (NAG) produces a variety of software products. Relevant to this book are the f95 compiler and the NAG Library, a large numerical program library available in several programming languages.

For information contact

NAG Ltd.
Wilkinson House
Jordan Hill Road
Oxford, OX2 8DR
UK

Tel: +44 1865 511245
Fax: +44 1865 310139

email: sales@nag.co.uk
Web: <http://www.nag.co.uk>

NAG has subsidiaries and distributors, whose addresses can be obtained from the above sources.

Appendix C

Program Libraries

Since the programming is likely to be the main bottleneck in the use of an electronic computer we have given a good deal of thought to the preparation of standard routines of considerable generality for the more important processes involved in computation.

By this means we hope to reduce the time taken to code up large-scale computing problems, by building them up, as it were, from prefabricated units.

— J. H. WILKINSON, *The Automatic Computing Engine at the National Physical Laboratory* (1948)

In spite of the self-contained nature of the linear algebra field, experience has shown that even here the preparation of a fully tested set of algorithms is a far greater task than had been anticipated.

— J. H. WILKINSON and C. REINSCH, *Handbook for Automatic Computation: Linear Algebra* (1971)

In this appendix we briefly describe some of the freely available program libraries that have been mentioned in this book. These packages are all available from netlib (see §B.2).

C.1. Basic Linear Algebra Subprograms

The Basic Linear Algebra Subprograms (BLAS) are sets of Fortran primitives for matrix and vector operations. They cover all the common operations in linear algebra. There are three levels, corresponding to the types of object operated upon. In the examples below, x and y are vectors, A, B, C are rectangular matrices, and T is a square triangular matrix. Names of BLAS routines are given in parentheses. The leading “x” denotes the Fortran data type, whose possible values are some or all of

S real
D double precision
C complex
Z complex*16, or double complex

Level 1: [774, 1979] *Vector operations.* Inner product: $x^T y$ (xdot); $y \leftarrow \alpha x + y$ (xAXPY); vector 2-norm $(y^T y)^{1/2}$ (xNRM2); swap vectors $x \leftrightarrow y$ (xSWAP); scale a vector $x \leftarrow \alpha x$ (xSCAL); and other operations.

Level 2: [347, 1988], [348, 1988] *Matrix-vector operations.* Matrix times vector (gaxpy): $y \leftarrow \alpha Ax + \beta y$ (xGEMV); rank-1 update: $A \leftarrow A + \alpha xy^T$ (xGER); triangular solve: $x \leftarrow T^{-1}x$ (xTRSV); and variations on these.

Level 3: [342, 1990], [343, 1990] *Matrix-matrix operations.* Matrix multiplication: $C \leftarrow \alpha AB + \beta C$ (xGEMM); multiple right-hand side triangular solve: $A \leftarrow \alpha T^{-1}A$ (xTRSM); rank- r and rank- $2r$ updates of a symmetric matrix (xSYRK, xSYR2K); and variations on these.

The BLAS are intended to be called in innermost loops of linear algebra codes. Usually, most of the computation in a code that uses BLAS calls is done inside these calls. LINPACK [341, 1979] uses the level-1 BLAS throughout (model Fortran implementations of the level-1 BLAS are listed in [341, 1979, App. D]). LAPACK [20, 1999] exploits all three levels, using the highest possible level at all times.

Each set of BLAS comprises a set of *subprogram specifications* only. The specifications define the parameters to each routine and state what the routine must do, but not how it must do it. Thus the implementor has freedom over the precise implementation details (loop orderings, block algorithms, special code for special cases) and even the method (fast versus conventional matrix multiply), but the implementation is required to be numerically stable, and code that tests the numerical stability is provided with the model implementations [343, 1990], [348, 1988].

For more details on the BLAS and the advantages of using them, see the defining papers listed above, or, for example, [349, 1998] or [509, 1996, Chap. 1].

Automated techniques for producing BLAS tuned to particular machines have been developed in the ATLAS project [1218, 2001].

More recently, a BLAS Technical Forum Standard has been produced that extends the functionality of the original level-1, -2, and -3 BLAS, adding BLAS to handle sparse and banded matrices, further dense matrix computations (many based on LAPACK auxiliary routines), and extended and mixed precision computations [88, 2002], [137, 2001]. It provides bindings for Fortran 95, Fortran 77, and C. For more on the Extended and Mixed Precision BLAS see §27.10. The Web site of the BLAS Technical Forum is <http://www.netlib.org/blas/blast-forum/>

C.2. EISPACK

EISPACK is a collection of Fortran 66 subroutines for computing eigenvalues and eigenvectors of matrices [1047, 1976], [455, 1977]. It contains 58 subroutines and 13 drivers. The subroutines are the basic building blocks for eigensystem computations; they perform such tasks as reduction to Hessenberg form, computation of some or all of the eigenvalues/vectors, and back transformations, for various types of matrix (real, complex, symmetric, banded, etc.). The driver subroutines provide easy access to many of EISPACK's capabilities; they call from one to five other EISPACK subroutines to do the computations.

EISPACK is primarily based on Algol 60 procedures developed in the 1960s by 19 different authors and published in the journal *Numerische Mathematik*. An edited collection of these papers was published in the *Handbook for Automatic Computation* [1246, 1971].

C.3. LINPACK

LINPACK is a collection of Fortran 66 subroutines that analyse and solve linear equations and linear least squares problems [341, 1979]. The package solves linear systems whose matrices are general, banded, symmetric indefinite, symmetric positive definite, triangular, or tridiagonal. In addition, the package computes the QR and singular value decompositions and applies them to least squares problems. All the LINPACK routines use calls to the level-1 BLAS in the innermost loops; thus most of the floating point arithmetic in LINPACK is done within the level-1 BLAS.

C.4. LAPACK

LAPACK [20, 1999] was released on February 29, 1992. As the announcement stated, "LAPACK is a transportable library of Fortran 77 subroutines for solving the most common problems in numerical linear algebra: systems of linear equations, linear least squares problems, eigenvalue problems, and singular value problems. It has been designed to be efficient on a wide range of modern high-performance computers."

LAPACK has been developed since 1987 by a worldwide team of numerical analysts. It can be regarded as a successor to LINPACK and EISPACK, as it

has virtually all their capabilities and much more besides. LAPACK improves on LINPACK and EISPACK in four main respects: speed, accuracy, robustness, and functionality. It was designed at the outset to exploit the level-3 BLAS.

Interfaces from LAPACK (or translations of LAPACK) to other languages are available for Fortran 95 [65, 2001], C, C++, and Java; see the links on the LAPACK home page at <http://www.netlib.org/lapack/>

A related project ScaLAPACK has produced a subset of LAPACK routines redesigned for distributed memory parallel machines [135, 1997].

Other work includes developing codes that take advantage of the careful rounding and exception handling of IEEE arithmetic [331, 1994]. For more details of all these topics see [20, 1999].

LAPACK undergoes regular updates, which are announced on the electronic newsletter NA-Digest. At the time of writing, the current release is version 3.0, last updated May 31, 2000, and the package contains over 1000 routines and over 735,000 lines of Fortran 77 code, including testing and timing code.

Mark 16 onward of the NAG Fortran 77 Library contains much of LAPACK in Chapters F07 and F08.

C.4.1. Structure of LAPACK

The LAPACK routines can be divided into three classes.

The *drivers* solve a complete problem. The *simple* drivers have a minimal specification, while the *expert* drivers have additional capabilities of interest to the sophisticated user. The *computational* routines perform individual tasks such as computing a factorization or reducing a matrix to condensed form; they are called by the drivers. The *auxiliary* routines perform relatively low-level operations such as unblocked factorizations, estimating or computing matrix norms, and solving a triangular system with scaling to prevent overflow.

The driver and computational routines have names of the form *xyyzzz*. The first letter specifies the data type, which is one of S, D, C, and Z. The second two letters refer to the type of matrix. A partial list of types is as follows (there are 27 types in all):

BD	bidiagonal
GB	general band
GE	general
GT	general tridiagonal
HS	upper Hessenberg
OR	(real) orthogonal
PO	symmetric or Hermitian positive definite
PT	symmetric or Hermitian positive definite tridiagonal
SB	(real) symmetric band
ST	(real) symmetric tridiagonal
SY	symmetric
TR	triangular (or quasi-triangular)

The last three characters specify the computation performed.

TRF	factorize
TRS	solve a (multiple right-hand side) linear system using the factorization
CON	estimate $1/\kappa_1(A)$ (or compute it exactly when A is tridiagonal and symmetric positive definite or Hermitian positive definite)
RFS	apply fixed precision iterative refinement and compute the componentwise relative backward error and a forward error bound
TRI	use the factorization to compute A^{-1}
EQU	compute factors to equilibrate the matrix

The auxiliary routines follow a similar naming convention, with most of them having $yy = LA$.

Appendix D

The Matrix Computation Toolbox

The Matrix Computation Toolbox is a collection of MATLAB M-files containing functions for constructing test matrices, computing matrix factorizations, visualizing matrices, and carrying out direct search optimization. Various other miscellaneous functions are also included.

This toolbox supersedes the Test Matrix Toolbox [606, 1995] that was described in the first edition of this book. Most of the test matrices in that toolbox have been incorporated into MATLAB in the `gallery` function. The new toolbox incorporates some of the other routines in the Test Matrix Toolbox and adds several new ones.

Of particular relevance to this book are functions for

- Cholesky factorization with complete pivoting for a symmetric positive semidefinite matrix: `cholp`;
- rounding matrix elements to a specified number of bits—useful for simulating lower precision in experiments: `chop`;
- GE with one of four optional pivoting strategies: no pivoting, partial pivoting, rook pivoting, and complete pivoting: `gep`;
- Gauss–Jordan elimination with no pivoting or partial pivoting for solving a linear system: `gj`;
- generalized QR factorization: `gqr`;
- the classical and modified Gram–Schmidt methods for QR factorization: `gs_c`, `gs_m`;
- block LDL^T factorization of symmetric matrices with partial pivoting or rook pivoting: `ldlt_ssym`;
- block LDL^T factorization of symmetric tridiagonal matrices with Bunch’s pivoting strategy: `ldlt_sytr`;
- block LDL^T factorization of skew-symmetric matrices with Bunch’s partial pivoting strategy: `ldlt_skew`;
- solving the equality constrained least squares problem: `lse`;
- two- and three-dimensional views of pseudospectra: `ps`, `pscont`;

- direct search optimization by the alternating directions method, **ad**, the multidirectional search method, **mds**, and the Nelder–Mead simplex method, **nms**;
- the p -norm of a matrix: **pnorm**;
- Strassen’s method for matrix multiplication and Winograd’s variant: **strassen**, **strassenw**;
- converting between a triangular matrix and the nontrivial elements of its vec (useful in direct search optimization): **treshape**;
- the vec-permutation matrix: **vecperm**.

The Matrix Computation Toolbox is available from

<http://www.ma.man.ac.uk/~higham/mctoolbox>

We summarize the contents of the toolbox in the following tables, which list the M-files by category, with short descriptions.

Demonstration	
mctdemo	Demonstration of Matrix Computation Toolbox.

Test Matrices	
augment	Augmented system matrix.
gfpp	Matrix giving maximal growth factor for Gaussian elimination with partial pivoting.
makejcf	A matrix with specified Jordan canonical form.
rschur	An upper quasi-triangular matrix.
vand	Vandermonde matrix.
vecperm	Vec-permutation matrix.

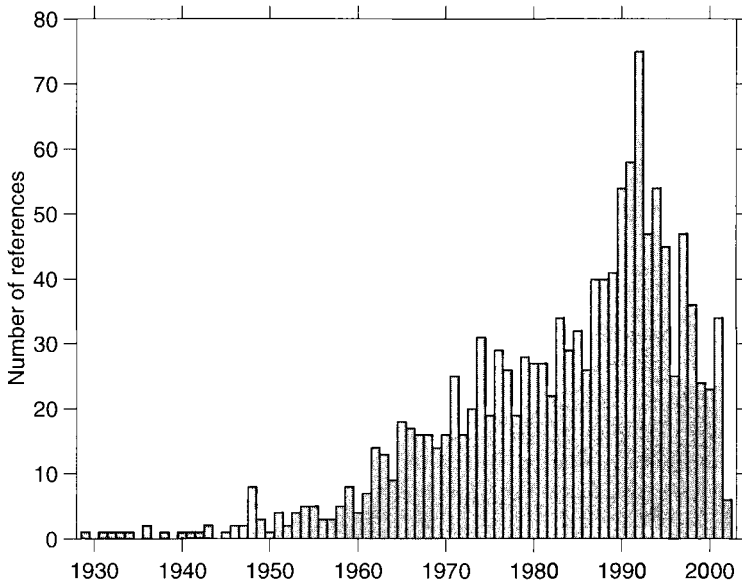
Factorizations and Decompositions	
cholp	Cholesky factorization with complete pivoting of a positive semidefinite matrix.
cod	Complete orthogonal decomposition.
gep	Gaussian elimination with pivoting: none, complete, partial, or rook.
gj	Gauss–Jordan elimination with no pivoting or partial pivoting to solve $Ax = b$.
gqr	Generalized QR factorization.
gs_c	Classical Gram–Schmidt QR factorization.
gs_m	Modified Gram–Schmidt QR factorization.
ldlt_skew	Block LDL^T factorization for a skew-symmetric matrix.
ldlt_symm	Block LDL^T factorization with partial pivoting or rook pivoting for a symmetric indefinite matrix.
ldlt_sytr	Block LDL^T factorization for a symmetric tridiagonal matrix.
matsignt	Matrix sign function of a triangular matrix.
poldec	Polar decomposition.
signm	Matrix sign decomposition.
trap2tri	Unitary reduction of trapezoidal matrix to triangular form.

Visualization	
fv	Field of values (or numerical range).
gersh	Gershgorin disks.
ps	Dot plot of a pseudospectrum.
pscont	Contours and colour pictures of pseudospectra.
see	Pictures of a matrix.

Direct Search Optimization	
adsmx	Alternating directions method.
mdsmx	Multidirectional search method.
mmsmx	Nelder–Mead simplex method.

Miscellaneous	
chop	Round matrix elements.
cpltaxes	Determine suitable axis for plot of complex vector.
dual	Dual vector with respect to Hölder p -norm.
lse	Solve the equality constrained least squares problem.
matrix	Matrix Computation Toolbox information and matrix access by number.
pnorm	Estimate of matrix p -norm ($1 \leq p \leq \infty$).
rootm	P th root of a matrix.
seqcheb	Sequence of points related to Chebyshev polynomials.
seqm	Multiplicative sequence.
show	Display signs of matrix elements.
skewpart	Skew-symmetric (skew-Hermitian) part.
sparsify	Randomly set matrix elements to zero.
strassen	Strassen's fast matrix multiplication algorithm.
strassenw	Strassen's fast matrix multiplication algorithm (Winograd variant).
sub	Principal submatrix.
symmpart	Symmetric (Hermitian) part.
reshape	Reshape vector to or from (unit) triangular matrix.

Bibliography



- The distribution of the year of publication of the references in this bibliography is shown in the graph above.
- Many of the unpublished references are available on the Web and can be found by using Google (www.google.com) to search on author and title. For the more difficult-to-obtain unpublished references a URL is given when known.
- Many of the author's papers are available from <http://www.ma.man.ac.uk/~higham/>
- Many of Kahan's manuscripts are available from <http://http.cs.berkeley.edu/~wkahan>.
- NA Digest is a weekly electronic mail magazine available at <http://www.netlib.org/na-digest-html>

- [1] Jan Ole Aasen. On the reduction of a symmetric matrix to tridiagonal form. *BIT*, 11:233–242, 1971.
- [2] Nabih N. Abdelmalek. Round off error analysis for Gram-Schmidt method and solution of linear least squares problems. *BIT*, 11:345–368, 1971.
- [3] *ACM Turing Award Lectures: The First Twenty Years, 1966–1985*. Addison-Wesley, Reading, MA, USA, 1987. xviii+483 pp. ISBN 0-201-54885-2.
- [4] Forman S. Acton. *Numerical Methods That Work*. Harper and Row, New York, 1970. xviii+541 pp. Reprinted by Mathematical Association of America, Washington, D.C., with new preface and additional problems, 1990. ISBN 0-88385-450-3.
- [5] Forman S. Acton. *Real Computing Made Real: Preventing Errors in Scientific and Engineering Calculations*. Princeton University Press, Princeton, NJ, USA, 1996. xv+259 pp. ISBN 0-691-03663-2.
- [6] Duane A. Adams. A stopping criterion for polynomial root finding. *Comm. ACM*, 10:655–658, 1967.
- [7] Vijay B. Aggarwal and James W. Burgmeier. A round-off error model with applications to arithmetic expressions. *SIAM J. Comput.*, 8(1):60–72, 1979.
- [8] Alan A. Ahac, John J. Buoni, and D. D. Olesky. Stable *LU* factorization of *H*-matrices. *Linear Algebra Appl.*, 99:97–110, 1988.
- [9] J. H. Ahlberg and E. N. Nilson. Convergence properties of the spline fit. *J. Soc. Indust. Appl. Math.*, 11(1):95–104, 1963.
- [10] Paul Halmos by parts (interviews by Donald J. Albers). In *Paul Halmos: Celebrating 50 Years of Mathematics*, John H. Ewing and F. W. Gehring, editors, Springer-Verlag, Berlin, 1991, pages 3–32.
- [11] Göltz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press, New York, 1983. xviii+333 pp. ISBN 0-12-049820-0.
- [12] M. Almacany, C. B. Dunham, and J. Williams. Discrete Chebyshev approximation by interpolating rationals. *IMA J. Numer. Anal.*, 4:467–477, 1984.
- [13] P. Alonso, M. Gasca, and J. M. Peña. Backward error analysis of Neville elimination. *Appl. Numer. Math.*, 23:193–204, 1997.
- [14] B. Alpert, G. Beylkin, R. Coifman, and V. Rokhlin. Wavelet-like bases for the fast solution of second-kind integral equations. *SIAM J. Sci. Comput.*, 14(1):159–184, 1993.
- [15] H. Alt and J. van Leeuwen. The complexity of basic complex operations. *Computing*, 27:205–215, 1981.
- [16] Steven C. Althoen and Renate Mclaughlin. Gauss-Jordan reduction: A brief history. *Amer. Math. Monthly*, 94(2):130–142, 1987.
- [17] Fernando L. Alvarado, Alex Pothén, and Robert S. Schreiber. Highly parallel sparse triangular solution. In *Graph Theory and Sparse Matrix Computations*, J. Alan George, John R. Gilbert, and Joseph W. H. Liu, editors, volume 56 of *IMA Volumes in Mathematics and Its Applications*, Springer-Verlag, New York, 1993, pages 141–158.
- [18] Pierluigi Amodio and Francesca Mazzia. Backward error analysis of cyclic reduction for the solution of tridiagonal systems. *Math. Comp.*, 62(206):601–617, 1994.
- [19] Andrew A. Anda and Haesun Park. Fast plane rotations with dynamic scaling. *SIAM J. Matrix Anal. Appl.*, 15(1):162–174, 1994.

- [20] E. Anderson, Z. Bai, C. H. Bischof, S. Blackford, J. W. Demmel, J. J. Dongarra, J. J. Du Croz, A. Greenbaum, S. J. Hammarling, A. McKenney, and D. C. Sorensen. *LAPACK Users' Guide*. Third edition, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1999. xxvi+407 pp. ISBN 0-89871-447-8.
- [21] E. Anderson, Z. Bai, and J. Dongarra. Generalized QR factorization and its applications. *Linear Algebra Appl.*, 162–164:243–271, 1992.
- [22] Edward Anderson. Robust triangular solves for use in condition estimation. Technical Report CS-91-142, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, August 1991. 35 pp. LAPACK Working Note 36.
- [23] I. J. Anderson. A distillation algorithm for floating-point summation. *SIAM J. Sci. Comput.*, 20(5):1797–1806, 1999.
- [24] T. W. Anderson. *The Statistical Analysis of Time Series*. Wiley, New York, 1971. xiv+704 pp. ISBN 0-471-02900-9.
- [25] T. W. Anderson, I. Olkin, and L. G. Underhill. Generation of random orthogonal matrices. *SIAM J. Sci. Statist. Comput.*, 8(4):625–629, 1987.
- [26] T. Ando. Totally positive matrices. *Linear Algebra Appl.*, 90:165–219, 1987.
- [27] Alan L. Andrew. Eigenvalues and singular values of certain random matrices. *J. Comput. Appl. Math.*, 30:165–171, 1990.
- [28] Anonymous. Le Commandant Cholesky. *Bulletin Géodésique*, pages 159–161, 1922. Translation by Richard W. Cottle (“Major Cholesky”) appears in [806, Appendix] and in NA Digest, Volume 90, Issue 7, 1990.
- [29] Anonymous. James Wilkinson (1919–1986). *Annals of the History of Computing*, 9(2):205–210, 1987. From the introduction: “A series of lightly edited extracts from messages that were sent over various computer networks during the period October 5, 1986–February 13, 1987”.
- [30] M. Arioli, J. W. Demmel, and I. S. Duff. Solving sparse linear systems with sparse backward error. *SIAM J. Matrix Anal. Appl.*, 10(2):165–190, 1989.
- [31] M. Arioli, I. S. Duff, and P. P. M. de Rijk. On the augmented system approach to sparse least-squares problems. *Numer. Math.*, 55:667–684, 1989.
- [32] M. Arioli and A. Laratta. Error analysis of an algorithm for solving an underdetermined linear system. *Numer. Math.*, 46:255–268, 1985.
- [33] M. Arioli and A. Laratta. Error analysis of algorithms for computing the projection of a point onto a linear manifold. *Linear Algebra Appl.*, 82:1–26, 1986.
- [34] Mario Arioli. A stopping criterion for the conjugate gradient algorithm in a finite element method framework. Technical Report 1179, Istituto di Analisi Numerica - C.N.R., Pavia, Italy, 2000. 12 pp.
- [35] Mario Arioli, Iain S. Duff, and Daniel Ruiz. Stopping criteria for iterative solvers. *SIAM J. Matrix Anal. Appl.*, 13(1):138–144, 1992.
- [36] Mario Arioli and Francesco Romani. Stability, convergence, and conditioning of stationary iterative methods of the form $x^{(i+1)} = Px^{(i)} + q$ for the solution of linear systems. *IMA J. Numer. Anal.*, 12:21–30, 1992.
- [37] William F. Arnold and Alan J. Laub. Generalized eigenproblem algorithms and software for algebraic Riccati equations. *Proc. IEEE*, 72(12):1746–1754, 1984.
- [38] Cleve Ashcraft, Roger G. Grimes, and John G. Lewis. Accurate symmetric indefinite linear equation solvers. *SIAM J. Matrix Anal. Appl.*, 20(2):513–561, 1998.

- [39] R. L. Ashenurst and N. Metropolis. Error estimation in computer calculation. *Amer. Math. Monthly*, 72(2):47–58, 1965.
- [40] Edgar Asplund. Inverses of matrices $\{a_{ij}\}$ which satisfy $a_{ij} = 0$ for $j > i + p$. *Math. Scand.*, 7:57–60, 1959.
- [41] John V. Atanasoff. Computing machine for the solution of large systems of linear algebraic equations. Unpublished manuscript, Iowa State College, Ames, IA, USA, August 1940. Reprinted in [972, pp. 315–335].
- [42] Owe Axelsson. *Iterative Solution Methods*. Cambridge University Press, 1994. xiii+654 pp. ISBN 0-521-44524-8.
- [43] Ivo Babuška. Numerical stability in mathematical analysis. In *Proc. IFIP Congress*, Information Processing 68, North-Holland, Amsterdam, The Netherlands, 1969, pages 11–23.
- [44] Zhaojun Bai, David Day, James Demmel, and Jack Dongarra. A test matrix collection for non-Hermitian eigenvalue problems (release 1.0). Technical Report CS-97-355, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, March 1997. 45 pp. LAPACK Working Note 123.
- [45] Zhaojun Bai and James W. Demmel. Computing the generalized singular value decomposition. *SIAM J. Sci. Comput.*, 14(6):1464–1486, 1993.
- [46] Zhaojun Bai and James W. Demmel. On swapping diagonal blocks in real Schur form. *Linear Algebra Appl.*, 186:73–95, 1993.
- [47] Zhaojun Bai, James W. Demmel, Jack J. Dongarra, Antoine Petitet, Howard Robinson, and K. Stanley. The spectral decomposition of nonsymmetric matrices on distributed memory parallel computers. *SIAM J. Sci. Comput.*, 18(5):1446–1461, 1997.
- [48] Zhaojun Bai, James W. Demmel, and Ming Gu. Inverse free parallel spectral divide and conquer algorithms for nonsymmetric eigenproblems. *Numer. Math.*, 76:279–308, 1997.
- [49] Zhaojun Bai, James W. Demmel, and Alan McKenney. On computing condition numbers for the nonsymmetric eigenproblem. *ACM Trans. Math. Software*, 19(2):202–223, 1993.
- [50] D. H. Bailey and H. R. P. Ferguson. A Strassen–Newton algorithm for high-speed parallelizable matrix inversion. In *Proceedings of Supercomputing '88*, IEEE Computer Society Press, New York, 1988, pages 419–424.
- [51] David H. Bailey. The computation of π to 29,360,000 decimal digits using Borweins' quartically convergent algorithm. *Math. Comp.*, 50(181):283–296, 1988.
- [52] David H. Bailey. Extra high speed matrix multiplication on the Cray-2. *SIAM J. Sci. Statist. Comput.*, 9(3):603–607, 1988.
- [53] David H. Bailey. Algorithm 719: Multiprecision translation and execution of FORTRAN programs. *ACM Trans. Math. Software*, 19(3):288–319, 1993.
- [54] David H. Bailey. A Fortran 90-based multiprecision system. *ACM Trans. Math. Software*, 21(4):379–387, 1995.
- [55] David H. Bailey, Robert Krasny, and Richard Pelz. Multiple precision, multiple processor vortex sheet roll-up computation. In *Proceedings of the Sixth SIAM Conference on Parallel Processing for Scientific Computing, Volume I*, Richard F. Sincovec, David E. Keyes, Michael R. Leuze, Linda R. Petzold, and Daniel A. Reed, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1993, pages 52–56.

- [56] David H. Bailey, King Lee, and Horst D. Simon. Using Strassen's algorithm to accelerate the solution of linear systems. *J. Supercomputing*, 4:357–371, 1991.
- [57] David H. Bailey, Horst D. Simon, John T. Barton, and Martin J. Fouts. Floating point arithmetic in future supercomputers. *Internat. J. Supercomputer Appl.*, 3(3):86–90, 1989.
- [58] J. K. Baksalary and R. Kala. The matrix equation $AX - YB = C$. *Linear Algebra Appl.*, 25:41–43, 1979.
- [59] J. K. Baksalary and R. Kala. The matrix equation $AXB + CYD = E$. *Linear Algebra Appl.*, 30:141–147, 1980.
- [60] Susanne M. Balle, Per Christian Hansen, and Nicholas J. Higham. A Strassen-type matrix inversion algorithm for the Connection Machine. Technical Report CNC/1993/028, Centre for Novel Computing, University of Manchester, Manchester, England, October 1993. 29 pp.
- [61] C. Ballester and V. Pereyra. On the construction of discrete approximations to linear differential expressions. *Math. Comp.*, 21:297–302, 1967.
- [62] Randolph E. Bank and Donald J. Rose. Marching algorithms for elliptic boundary value problems. I: The constant coefficient case. *SIAM J. Numer. Anal.*, 14(5):792–829, 1977.
- [63] Yonathan Bard. *Nonlinear Parameter Estimation*. Academic Press, New York, 1974. x+341 pp. ISBN 0-12-078250-2.
- [64] V. Bargmann, D. Montgomery, and J. von Neumann. Solution of linear systems of high order. Report prepared for Navy Bureau of Ordnance, 1946. Reprinted in [1130, pp. 421–477].
- [65] V. A. Barker, L. S. Blackford, J. J. Dongarra, J. J. Du Croz, S. J. Hammarling, M. Marinova, J. Waśniewski, and P. Yalamov. *LAPACK95 Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2001. xviii+258 pp. ISBN 0-89871-504-0.
- [66] J. L. Barlow, N. K. Nichols, and R. J. Plemmons. Iterative methods for equality-constrained least squares problems. *SIAM J. Sci. Statist. Comput.*, 9(5):892–906, 1988.
- [67] Jesse L. Barlow. On the distribution of accumulated roundoff error in floating point arithmetic. In *Proc. 5th IEEE Symposium on Computer Arithmetic*, Ann Arbor, Michigan, 1981, pages 100–105.
- [68] Jesse L. Barlow. *Probabilistic Error Analysis of Floating Point and CRD Arithmetics*. PhD thesis, Northwestern University, Evanston, IL, USA, June 1981.
- [69] Jesse L. Barlow. A note on monitoring the stability of triangular decomposition of sparse matrices. *SIAM J. Sci. Statist. Comput.*, 7(1):166–168, 1986.
- [70] Jesse L. Barlow. Error analysis and implementation aspects of deferred correction for equality constrained least squares problems. *SIAM J. Numer. Anal.*, 25(6):1340–1358, 1988.
- [71] Jesse L. Barlow. Error analysis of a pairwise summation algorithm to compute the sample variance. *Numer. Math.*, 58:583–590, 1991.
- [72] Jesse L. Barlow and E. H. Bareiss. On roundoff error distributions in floating point and logarithmic arithmetic. *Computing*, 34:325–347, 1985.
- [73] Jesse L. Barlow and E. H. Bareiss. Probabilistic error analysis of Gaussian elimination in floating point and logarithmic arithmetic. *Computing*, 34:349–364, 1985.

- [74] Jesse L. Barlow and Ilse C. F. Ipsen. Scaled Givens rotations for the solution of linear least squares problems on systolic arrays. *SIAM J. Sci. Statist. Comput.*, 8(5):716–733, 1987.
- [75] Jesse L. Barlow and Udaya B. Vemulapati. A note on deferred correction for equality constrained least squares problems. *SIAM J. Numer. Anal.*, 29(1):249–256, 1992.
- [76] Jesse L. Barlow and Udaya B. Vemulapati. Rank detection methods for sparse matrices. *SIAM J. Matrix Anal. Appl.*, 13(4):1279–1297, 1992.
- [77] Jesse L. Barlow and Hongyuan Zha. Growth in Gaussian elimination, orthogonal matrices, and the 2-norm. *SIAM J. Matrix Anal. Appl.*, 19(3):807–815, 1998.
- [78] S. Barnett and C. Storey. Some applications of the Lyapunov matrix equation. *J. Inst. Maths. Applics.*, 4:33–42, 1968.
- [79] Geoff Barrett. Formal methods applied to a floating-point number system. *IEEE Trans. Software Engrg.*, 15(5):611–621, 1989.
- [80] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994. xiii+112 pp. ISBN 0-89871-328-5.
- [81] Anders Barrlund. Perturbation bounds for the LDL^H and LU decompositions. *BIT*, 31:358–363, 1991.
- [82] Anders Barrlund. Perturbation bounds for the generalized QR factorization. *Linear Algebra Appl.*, 207:251–271, 1994.
- [83] D. W. Barron and H. P. F. Swinnerton-Dyer. Solution of simultaneous linear equations using a magnetic-tape store. *Comput. J.*, 3(1):28–33, 1960.
- [84] R. H. Bartels and G. W. Stewart. Algorithm 432: Solution of the matrix equation $AX + XB = C$. *Comm. ACM*, 15(9):820–826, 1972.
- [85] Sven G. Bartels. Two topics in matrix analysis: Structured sensitivity for Vandermonde-like systems and a subgradient method for matrix norm estimation. M.Sc. Thesis, Department of Mathematics and Computer Science, University of Dundee, Dundee, Scotland, September 1991.
- [86] Sven G. Bartels and Desmond J. Higham. The structured sensitivity of Vandermonde-like systems. *Numer. Math.*, 62:17–33, 1992.
- [87] Victor Barwell and Alan George. A comparison of algorithms for solving symmetric indefinite systems of linear equations. *ACM Trans. Math. Software*, 2(3):242–251, 1976.
- [88] Basic Linear Algebra Subprograms Technical (BLAST) Forum Standard. *Int. J. High Performance Applications and Supercomputing*, 2002. To appear.
- [89] F. L. Bauer. Optimal scaling of matrices and the importance of the minimal condition. In *Proc. IFIP Congress 1962*, Cicely M. Popplewell, editor, Information Processing 62, North-Holland, Amsterdam, The Netherlands, 1963, pages 198–201.
- [90] F. L. Bauer. Optimally scaled matrices. *Numer. Math.*, 5:73–87, 1963.
- [91] F. L. Bauer. Genauigkeitsfragen bei der Lösung linearer Gleichungssysteme. *Z. Angew. Math. Mech.*, 46(7):409–421, 1966.
- [92] F. L. Bauer. Remarks on optimally scaled matrices. *Numer. Math.*, 13:1–3, 1969.

- [93] F. L. Bauer. Computational graphs and rounding errors. *SIAM J. Numer. Anal.*, 11(1):87–96, 1974.
- [94] F. L. Bauer and C. Reinsch. Inversion of positive definite matrices by the Gauss-Jordan method. In *Linear Algebra*, J. H. Wilkinson and C. Reinsch, editors, volume II of *Handbook for Automatic Computation*, Springer-Verlag, Berlin, 1971, pages 45–49. Contribution I/3.
- [95] F. L. Bauer, J. Stoer, and C. Witzgall. Absolute and monotonic norms. *Numer. Math.*, 3:257–264, 1961.
- [96] Richard M. Beam and Robert F. Warming. The asymptotic spectra of banded Toeplitz and quasi-Toeplitz matrices. *SIAM J. Sci. Comput.*, 14(4):971–1006, 1993.
- [97] Albert E. Beaton, Donald B. Rubin, and John L. Barone. The acceptability of regression solutions: Another look at computational accuracy. *J. Amer. Statist. Assoc.*, 71(353):158–168, 1976.
- [98] Bernhard Beckermann. The condition number of real Vandermonde, Krylov and positive definite Hankel matrices. *Numer. Math.*, 85:553–577, 2000.
- [99] C. Gordon Bell and Allen Newell. *Computer Structures: Readings and Examples*. McGraw-Hill, New York, 1971. xix+668 pp. ISBN 07-004357-4.
- [100] E. T. Bell. Review of “Contributions to the History of Determinants, 1900-1920”, by Sir Thomas Muir. *Amer. Math. Monthly*, 38:161–164, 1931. Reprinted in [397].
- [101] Richard Bellman. *Introduction to Matrix Analysis*. Second edition, McGraw-Hill, New York, 1970. xxiii+403 pp. Reprinted by Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. ISBN 0-89871-399-4.
- [102] Frank Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938.
- [103] Commandant Benoit. Note sur une méthode de résolution des équations normales provenant de l’application de la méthode des moindres carrés à un système d’équations linéaires en nombre inférieur à celui des inconnues. Application de la méthode à la résolution d’un système défini d’équations linéaires (Procédé du Commandant Cholesky). *Bulletin Géodésique* (Toulouse), 2:67–77, 1924. Cited in [28, Cottle’s translation].
- [104] N. F. Benschop and H. C. Ratz. A mean square estimate of the generated roundoff error in constant matrix iterative processes. *J. Assoc. Comput. Mach.*, 18(1):48–62, 1971.
- [105] M. C. Berenbaum. Direct search methods in the optimisation of cancer chemotherapy. *Br. J. Cancer*, 61:101–109, 1991.
- [106] Abraham Berman and Robert J. Plemmons. *Nonnegative Matrices in the Mathematical Sciences*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994. xx+340 pp. Corrected republication, with supplement, of work first published in 1979 by Academic Press. ISBN 0-89871-321-8.
- [107] Rajendra Bhatia. *Matrix Analysis*. Springer-Verlag, New York, 1997. xi+347 pp. ISBN 0-387-94846-5.
- [108] Rajendra Bhatia and Kalyan Mukherjea. Variation of the unitary part of a matrix. *SIAM J. Matrix Anal. Appl.*, 15(3):1007–1014, 1994.
- [109] Rajendra Bhatia and Peter Rosenthal. How and why to solve the operator equation $AX - XB = Y$. *Bull. London Math. Soc.*, 29:1–21, 1997.

- [110] Dario Bini and Grazia Lotti. Stability of fast algorithms for matrix multiplication. *Numer. Math.*, 36:63–72, 1980.
- [111] Dario Bini and Victor Y. Pan. *Polynomial and Matrix Computations. Volume 1: Fundamental Algorithms*. Birkhäuser, Boston, MA, USA, 1994. xvi+415 pp. ISBN 0-8176-3786-9.
- [112] Garrett Birkhoff. Two Hadamard numbers for matrices. *Comm. ACM*, 18(1): 25–29, 1975.
- [113] Garrett Birkhoff and Surender Gulati. Isotropic distributions of test matrices. *J. Appl. Math. Phys. (ZAMP)*, 30:148–158, 1979.
- [114] Garrett Birkhoff and Saunders Mac Lane. *A Survey of Modern Algebra*. Fourth edition, Macmillan, New York, 1977. xi+500 pp. ISBN 0-02-310070-2.
- [115] Christian H. Bischof. Incremental condition estimation. *SIAM J. Matrix Anal. Appl.*, 11(2):312–322, 1990.
- [116] Christian H. Bischof, John G. Lewis, and Daniel J. Pierce. Incremental condition estimation for sparse matrices. *SIAM J. Matrix Anal. Appl.*, 11(4):644–659, 1990.
- [117] Christian H. Bischof and Charles F. Van Loan. The WY representation for products of Householder matrices. *SIAM J. Sci. Statist. Comput.*, 8(1):s2–s13, 1987.
- [118] Åke Björck. Iterative refinement of linear least squares solutions I. *BIT*, 7:257–278, 1967.
- [119] Åke Björck. Solving linear least squares problems by Gram-Schmidt orthogonalization. *BIT*, 7:1–21, 1967.
- [120] Åke Björck. Iterative refinement of linear least squares solutions II. *BIT*, 8:8–30, 1968.
- [121] Åke Björck. Comment on the iterative refinement of least-squares solutions. *J. Amer. Statist. Assoc.*, 73(361):161–166, 1978.
- [122] Åke Björck. Stability analysis of the method of seminormal equations for linear least squares problems. *Linear Algebra Appl.*, 88/89:31–48, 1987.
- [123] Åke Björck. Iterative refinement and reliable computing. In *Reliable Numerical Computation*, M. G. Cox and S. J. Hammarling, editors, Oxford University Press, 1990, pages 249–266.
- [124] Åke Björck. Least squares methods. In *Handbook of Numerical Analysis*, P. G. Ciarlet and J. L. Lions, editors, volume I: Finite Difference Methods—Solution of Equations in $\mathbb{R}^n$, Elsevier/North-Holland, Amsterdam, The Netherlands, 1990.
- [125] Åke Björck. Component-wise perturbation analysis and error bounds for linear least squares solutions. *BIT*, 31:238–244, 1991.
- [126] Åke Björck. Pivoting and stability in the augmented system method. In *Numerical Analysis 1991, Proceedings of the 14th Dundee Conference*, D. F. Griffiths and G. A. Watson, editors, volume 260 of *Pitman Research Notes in Mathematics*, Longman Scientific and Technical, Essex, UK, 1992, pages 1–16.
- [127] Åke Björck. Numerics of Gram-Schmidt orthogonalization. *Linear Algebra Appl.*, 197/198:297–316, 1994.
- [128] Åke Björck. *Numerical Methods for Least Squares Problems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1996. xvii+408 pp. ISBN 0-89871-360-9.
- [129] Åke Björck and Tommy Elfving. Algorithms for confluent Vandermonde systems. *Numer. Math.*, 21:130–137, 1973.

- [130] Åke Björck and Gene H. Golub. Iterative refinement of linear least squares solutions by Householder transformation. *BIT*, 7:322–337, 1967.
- [131] Åke Björck and C. C. Paige. Loss and recapture of orthogonality in the modified Gram–Schmidt algorithm. *SIAM J. Matrix Anal. Appl.*, 13(1):176–190, 1992.
- [132] Åke Björck and C. C. Paige. Solution of augmented linear systems using orthogonal factorizations. *BIT*, 34:1–24, 1994.
- [133] Åke Björck and Victor Pereyra. Solution of Vandermonde systems of equations. *Math. Comp.*, 24(112):893–903, 1970.
- [134] P. Bjørstad, F. Manne, T. Sørsvik, and M. Vajteršić. Efficient matrix multiplication on SIMD computers. *SIAM J. Matrix Anal. Appl.*, 13(1):386–401, 1992.
- [135] L. S. Blackford, J. Choi, A. Cleary, E. D’Azevedo, J. Demmel, I. Dhillon, J. Dongarra, S. Hammarling, G. Henry, A. Petitet, K. Stanley, D. Walker, and R. C. Whaley. *ScaLAPACK Users’ Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. xxvi+325 pp. ISBN 0-89871-397-8.
- [136] L. S. Blackford, A. Cleary, A. Petitet, R. C. Whaley, J. Demmel, I. Dhillon, H. Ren, K. Stanley, J. Dongarra, and S. Hammarling. Practical experience in the numerical dangers of heterogeneous computing. *ACM Trans. Math. Software*, 23(2):133–147, 1997.
- [137] L. S. Blackford, J. Demmel, J. Dongarra, I. Duff, S. Hammarling, G. Henry, M. Heroux, L. Kaufman, A. Lumsdaine, A. Petitet, R. Pozo, K. Remington, and R. C. Whaley. An updated set of Basic Linear Algebra Subprograms (BLAS), February 2001. Submitted to *ACM Trans. Math. Software*. 20 pp.
- [138] G. Blanch. Numerical evaluation of continued fractions. *SIAM Rev.*, 6(4):383–421, 1964.
- [139] J. H. Bleher, A. E. Roeder, and S. M. Rump. ACRITH: High-accuracy arithmetic. An advanced tool for numerical computation. In *Proceedings of the 7th Symposium on Computer Arithmetic*, Kai Hwang, editor, IEEE Computer Society Press, Silver Spring, MD, USA, 1985, pages 318–321.
- [140] J. H. Bleher, S. M. Rump, U. Kulisch, M. Metzger, Ch. Ullrich, and W. Walter. A study of a FORTRAN extension for engineering/scientific computation with access to ACRITH. *Computing*, 39:93–110, 1987.
- [141] B. Bliss, M.-C. Brunet, and E. Gallopoulos. Automatic program instrumentation with applications in performance and error analysis. In *Expert Systems for Scientific Computing*, E. N. Houstis, J. R. Rice, and R. Vichnevetsky, editors, North-Holland, Amsterdam, The Netherlands, 1992, pages 235–260.
- [142] Richard M. Bloch. Mark I calculator. In [552], pages 23–30.
- [143] James L. Blue. A portable Fortran program to find the Euclidean norm of a vector. *ACM Trans. Math. Software*, 4(1):15–23, 1978.
- [144] E. Bodewig. Review of “Rounding-Off Errors in Matrix Processes” by A. M. Turing. *Math. Rev.*, 10:405, 1949.
- [145] Gerd Bohlender. Floating-point computation of functions with maximum accuracy. *IEEE Trans. Comput.*, C-26(7):621–632, 1977.
- [146] Z. Bohte. Bounds for rounding errors in the Gaussian elimination for band systems. *J. Inst. Maths. Applics.*, 16:133–142, 1975.

- [147] Ronald F. Boisvert, Roldan Pozo, Karin Remington, Richard F. Barrett, and Jack J. Dongarra. Matrix Market: A Web resource for test matrix collections. In *Quality of Numerical Software: Assessment and Enhancement*, Ronald F. Boisvert, editor, Chapman and Hall, London, 1997, pages 125–136.
- [148] Daniel Boley, Gene H. Golub, Samy Makar, Nirmal Saxena, and Edward J. McCluskey. Floating point fault tolerance with backward error assertions. *IEEE Trans. Comput.*, 44(2):302–311, 1994.
- [149] Jo A. M. Bollen. Numerical stability of descent methods for solving linear equations. *Numer. Math.*, 43:361–377, 1984.
- [150] S. Bondeli and W. Gander. Cyclic reduction for special tridiagonal systems. *SIAM J. Matrix Anal. Appl.*, 15(1):321–330, 1994.
- [151] T. Boros, T. Kailath, and V. Olshevsky. Fast algorithms for solving Vandermonde and Chebyshev–Vandermonde systems. Manuscript, 1994. 22 pp.
- [152] J. M. Borwein, P. B. Borwein, and D. H. Bailey. Ramanujan, modular equations, and approximations to Pi or how to compute one billion digits of Pi. *Amer. Math. Monthly*, 96(3):201–219, 1989.
- [153] B. V. Bowden. The organization of a typical machine. In *Faster than Thought: A Symposium on Digital Computing Machines*, B. V. Bowden, editor, Pitman, London, 1953, pages 67–77.
- [154] H. J. Bowdler, R. S. Martin, G. Peters, and J. H. Wilkinson. Solution of real and complex systems of linear equations. *Numer. Math.*, 8:217–234, 1966. Also in [1246, pp. 93–110], Contribution I/7.
- [155] David W. Boyd. The power method for ℓ^p norms. *Linear Algebra Appl.*, 9:95–101, 1974.
- [156] Carl B. Boyer. *A History of Mathematics*. Second edition, Wiley, New York, 1989. xviii+762 pp. Revised by Uta C. Merzbach. ISBN 0-471-50357-6.
- [157] Jeff Boyle. An application of Fourier series to the most significant digit problem. *Amer. Math. Monthly*, 101(11):879–886, 1994.
- [158] R. P. Brent. Algorithms for matrix multiplication. Technical Report CS 157, Department of Computer Science, Stanford University, Stanford, CA, USA, March 1970. ii+52 pp.
- [159] R. P. Brent. Error analysis of algorithms for matrix multiplication and triangular decomposition using Winograd’s identity. *Numer. Math.*, 16:145–156, 1970.
- [160] Richard P. Brent. On the precision attainable with various floating-point number systems. *IEEE Trans. Comput.*, C-22(6):601–607, 1973.
- [161] Richard P. Brent. A Fortran multiple-precision arithmetic package. *ACM Trans. Math. Software*, 4(1):57–70, 1978.
- [162] Richard P. Brent. ALGORITHM 524 MP, a Fortran multiple-precision arithmetic package. *ACM Trans. Math. Software*, 4(1):71–81, 1978.
- [163] Richard P. Brent, Judith A. Hooper, and J. Michael Yohe. An AUGMENT interface for Brent’s multiple precision arithmetic package. *ACM Trans. Math. Software*, 6(2):146–149, 1980.
- [164] Claude Brezinski. André Louis Cholesky. In *A Numerical Analysis Conference in Honour of Jean Meinguet*, 1996, pages 45–50. Published as a supplement to the Bulletin of the Belgian Mathematical Society. Also available as Publication ANO-347, Laboratoire d’Analyse Numérique et d’Optimisation, Université des Sciences et Technologies de Lille, Nov. 1995, <http://ano.univ-lille1.fr>.

- [165] William L. Briggs and Van Emden Henson. *The DFT: An Owner's Manual for the Discrete Fourier Transform*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1995. xv+434 pp. ISBN 0-89871-342-0.
- [166] J. L. Britton, editor. *Collected Works of A. M. Turing: Pure Mathematics*. North-Holland, Amsterdam, The Netherlands, 1992. xxii+287 pp. ISBN 0-444-88059-3.
- [167] W. S. Brown. A simple but realistic model of floating-point arithmetic. *ACM Trans. Math. Software*, 7(4):445–480, 1981.
- [168] Shirley V. Browne, Jack J. Dongarra, Stan C. Green, Keith Moore, Thomas H. Rowan, and Reed C. Wade. Netlib services and resources. Report ORNL/TM-12680, Oak Ridge National Laboratory, Oak Ridge, TN, USA, April 1994. 42 pp.
- [169] Marie-Christine Brunet. *Contribution à la Fiabilité de Logiciels Numériques et à L'analyse de Leur Comportement: Une Approche Statistique*. PhD thesis, Université de Paris IX Dauphine, U.E.R. Mathématiques de la Décision, January 1989. viii+214 pp.
- [170] Marie-Christine Brunet and Françoise Chatelin. CESTAC, a tool for a stochastic round-off error analysis in scientific computing. In *Numerical Mathematics and Applications*, R. Vichnevetsky and J. Vignes, editors, Elsevier Science Publishers B.V. (North-Holland), Amsterdam, The Netherlands, 1986, pages 11–20.
- [171] James L. Buchanan and Peter R. Turner. *Numerical Methods and Analysis*. McGraw-Hill, New York, 1992. xv+751 pp. ISBN 0-07-008717-2, 0-07-112922-7 (international paperback edition).
- [172] W. Buchholz. Fingers or fists? (The choice of decimal or binary representation). *Comm. ACM*, 2(12):3–11, 1959.
- [173] W. Buchholz. Origin of the word *byte*. *Annals of the History of Computing*, 3(1): 72, 1981.
- [174] B. Bukhberger and G. A. Emel'yanenko. Methods of inverting tridiagonal matrices. *U.S.S.R. Computational Math. and Math. Phys.*, 13:10–20, 1973.
- [175] James R. Bunch. Analysis of the diagonal pivoting method. *SIAM J. Numer. Anal.*, 8(4):656–680, 1971.
- [176] James R. Bunch. Equilibration of symmetric matrices in the max-norm. *J. Assoc. Comput. Mach.*, 18(4):566–572, 1971.
- [177] James R. Bunch. Partial pivoting strategies for symmetric matrices. *SIAM J. Numer. Anal.*, 11(3):521–528, 1974.
- [178] James R. Bunch. A note on the stable decomposition of skew-symmetric matrices. *Math. Comp.*, 38(158):475–479, 1982.
- [179] James R. Bunch. The weak and strong stability of algorithms in numerical linear algebra. *Linear Algebra Appl.*, 88/89:49–66, 1987.
- [180] James R. Bunch, James W. Demmel, and Charles F. Van Loan. The strong stability of algorithms for solving symmetric linear systems. *SIAM J. Matrix Anal. Appl.*, 10(4):494–499, 1989.
- [181] James R. Bunch and Linda Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Math. Comp.*, 31(137):163–179, 1977.
- [182] James R. Bunch, Linda Kaufman, and Beresford N. Parlett. Decomposition of a symmetric matrix. *Numer. Math.*, 27:95–109, 1976.

- [183] James R. Bunch and Beresford N. Parlett. Direct methods for solving symmetric indefinite systems of linear equations. *SIAM J. Numer. Anal.*, 8(4):639–655, 1971.
- [184] Oleg Burdakov. A greedy algorithm for the optimal basis problem. *BIT*, 37(3): 591–599, 1997.
- [185] Peter Bürgisser, Michael Clausen, and M. Amin Shokrollahi. *Algebraic Complexity Theory*. Springer-Verlag, Berlin, 1997. xxiii+618 pp. ISBN 0-540-60582-7.
- [186] P. Businger and G. H. Golub. Linear least squares solutions by Householder transformations. *Numer. Math.*, 7:269–276, 1965. Also in [1246, pp. 111–118], Contribution I/8.
- [187] P. A. Businger. Matrices which can be optimally scaled. *Numer. Math.*, 12: 346–348, 1968.
- [188] P. A. Businger. Monitoring the numerical stability of Gaussian elimination. *Numer. Math.*, 16:360–361, 1971.
- [189] J. C. Butcher. *The Numerical Analysis of Ordinary Differential Equations: Runge-Kutta and General Linear Methods*. Wiley, Chichester, UK, 1987. xv+512 pp. ISBN 0-471-91046-5.
- [190] B. L. Buzbee, G. H. Golub, and C. W. Nielson. On direct methods for solving Poisson's equations. *SIAM J. Numer. Anal.*, 7(4):627–656, 1970.
- [191] Ralph Byers. A LINPACK-style condition estimator for the equation $AX - XB^T = C$. *IEEE Trans. Automat. Control*, AC-29(10):926–928, 1984.
- [192] Ralph Byers. Numerical condition of the algebraic Riccati equation. In *Linear Algebra and Its Role in Systems Theory*, B. N. Datta, editor, volume 47 of *Contemporary Math.*, American Mathematical Society, Providence, RI, USA, 1985, pages 35–49.
- [193] Ralph Byers. Solving the algebraic Riccati equation with the matrix sign function. *Linear Algebra Appl.*, 85:267–279, 1987.
- [194] Ralph Byers. A bisection method for measuring the distance of a stable matrix to the unstable matrices. *SIAM J. Sci. Statist. Comput.*, 9(5):875–881, 1988.
- [195] Ralph Byers and Stephen Nash. On the singular “vectors” of the Lyapunov operator. *SIAM J. Alg. Discrete Methods*, 8(1):59–66, 1987.
- [196] John Caffney. Another test matrix for determinants and inverses. *Comm. ACM*, 6(6):310, 1963.
- [197] D. Calvetti and L. Reichel. A Chebychev-Vandermonde solver. *Linear Algebra Appl.*, 172:219–229, 1992.
- [198] D. Calvetti and L. Reichel. Fast inversion of Vandermonde-like matrices involving orthogonal polynomials. *BIT*, 33:473–484, 1993.
- [199] S. L. Campbell and C. D. Meyer, Jr. *Generalized Inverses of Linear Transformations*. Pitman, London, 1979. xi+272 pp. Reprinted by Dover, New York, 1991. ISBN 0-486-66693-X.
- [200] Martin Campbell-Kelly. Programming the Pilot ACE: Early programming activity at the National Physical Laboratory. *Annals of the History of Computing*, 3(2): 133–162, 1981.
- [201] Martin Campbell-Kelly. Review of “Alan Turing: The Enigma”, by Andrew Hodges. *Annals of the History of Computing*, 6(2):176–178, 1984.
- [202] Wei-Lu Cao and William J. Stewart. A note on inverses of Hessenberg-like matrices. *Linear Algebra Appl.*, 76:233–240, 1986.

- [203] Ole Caprani. Implementation of a low round-off summation method. *BIT*, 11: 271–275, 1971.
- [204] B. E. Carpenter and R. W. Doran, editors. *A. M. Turing's ACE Report of 1946 and Other Papers*, volume 10 of *Charles Babbage Institute Reprint Series for the History of Computing*. MIT Press, Cambridge, MA, USA, 1986. vii+141 pp. ISBN 0-262-03114-0.
- [205] John W. Carr III. Error analysis in floating point arithmetic. *Comm. ACM*, 2(5): 10–15, 1959.
- [206] Russell Carter. Y-MP floating point and Cholesky factorization. *Int. J. High Speed Computing*, 3(3/4):215–222, 1991.
- [207] A. Cauchy. *Exercices d'Analyse et de Phys. Math.*, volume 2. Paris, 1841. Cited in [1013].
- [208] Françoise Chaitin-Chatelin and Valérie Frayssé. *Lectures on Finite Precision Computations*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1996. xv+235 pp. ISBN 0-89871-358-7.
- [209] N. N. Chan and Kim-Hung Li. Algorithm 115: A FORTRAN subroutine for finding a real symmetric matrix with prescribed diagonal elements and eigenvalues. *Comput. J.*, 26(2):184–186, 1983.
- [210] N. N. Chan and Kim-Hung Li. Diagonal elements and eigenvalues of a real symmetric matrix. *J. Math. Anal. Appl.*, 91:562–566, 1983.
- [211] Raymond H. Chan and Michael K. Ng. Conjugate gradient methods for Toeplitz systems. *SIAM Rev.*, 38(3):427–482, 1996.
- [212] T. F. Chan and P. C. Hansen. Some applications of the rank revealing QR factorization. *SIAM J. Sci. Statist. Comput.*, 13(3):727–741, 1992.
- [213] Tony F. Chan and David E. Foulser. Effectively well-conditioned linear systems. *SIAM J. Sci. Statist. Comput.*, 9(6):963–969, 1988.
- [214] Tony F. Chan, Gene H. Golub, and Randall J. LeVeque. Algorithms for computing the sample variance: Analysis and recommendations. *The American Statistician*, 37(3):242–247, 1983.
- [215] Tony F. Chan and John Gregg Lewis. Computing standard deviations: Accuracy. *Comm. ACM*, 22(9):526–531, 1979.
- [216] Shivkumar Chandrasekaran and Ilse C. F. Ipsen. Backward errors for eigenvalue and singular value decompositions. *Numer. Math.*, 68:215–223, 1994.
- [217] Shivkumar Chandrasekaran and Ilse C. F. Ipsen. On the sensitivity of solution components in linear systems of equations. *SIAM J. Matrix Anal. Appl.*, 16(1): 93–112, 1995.
- [218] Xiao-Wen Chang and Christopher C. Paige. On the sensitivity of the LU factorization. *BIT*, 38(3):486–501, 1998.
- [219] Xiao-Wen Chang and Christopher C. Paige. Componentwise perturbation analyses for the QR factorization. *Numer. Math.*, 88:319–345, 2001.
- [220] Xiao-Wen Chang, Christopher C. Paige, and G. W. Stewart. New perturbation analyses for the Cholesky factorization. *IMA J. Numer. Anal.*, 16(4):457–484, 1996.
- [221] Xiao-Wen Chang, Christopher C. Paige, and G. W. Stewart. Perturbation analyses for the QR factorization. *SIAM J. Matrix Anal. Appl.*, 18(3):775–791, 1997.

- [222] Bruce W. Char, Keith O. Geddes, Gaston H. Gonnet, Benton L. Leong, Michael B. Monagan, and Stephen M. Watt. *Maple V Library Reference Manual*. Springer-Verlag, Berlin, 1991. xxv+698 pp. ISBN 3-540-97592-6.
- [223] Bruce A. Chartres and James C. Geuder. Computable error bounds for direct solution of linear equations. *J. Assoc. Comput. Mach.*, 14(1):63–71, 1967.
- [224] Françoise Chatelin. *Eigenvalues of Matrices*. Wiley, Chichester, UK, 1993. xviii+382 pp. ISBN 0-471-93538-7.
- [225] Françoise Chatelin and Marie-Christine Brunet. A probabilistic round-off error propagation model. Application to the eigenvalue problem. In *Reliable Numerical Computation*, M. G. Cox and S. J. Hammarling, editors, Oxford University Press, 1990, pages 139–160.
- [226] Françoise Chatelin and Valérie Frayssé. Elements of a condition theory for the computational analysis of algorithms. In *Iterative Methods in Linear Algebra*, R. Beauwens and P. de Groen, editors, Elsevier Science Publishers B.V. (North-Holland), Amsterdam, The Netherlands, 1992, pages 15–25.
- [227] Sheung Hun Cheng. *Symmetric Indefinite Matrices: Linear System Solvers and Modified Inertia Problems*. PhD thesis, University of Manchester, Manchester, England, January 1998. 150 pp.
- [228] Sheung Hun Cheng and Nicholas J. Higham. A modified Cholesky algorithm based on a symmetric indefinite factorization. *SIAM J. Matrix Anal. Appl.*, 19(4):1097–1110, 1998.
- [229] Sheung Hun Cheng and Nicholas J. Higham. Implementation for LAPACK of a block algorithm for matrix 1-norm estimation. Numerical Analysis Report No. 393, Manchester Centre for Computational Mathematics, Manchester, England, August 2001. 19 pp. LAPACK Working Note 152.
- [230] Sheung Hun Cheng and Nicholas J. Higham. Parallel implementation of a block algorithm for matrix 1-norm estimation. In *Euro-Par 2001, Parallel Processing*, Rizos Sakellariou, John Keane, John Gurd, and Len Freeman, editors, volume 2150 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 2001, pages 568–577.
- [231] Sheung Hun Cheng, Nicholas J. Higham, Charles S. Kenney, and Alan J. Laub. Approximating the logarithm of a matrix to specified accuracy. *SIAM J. Matrix Anal. Appl.*, 22(4):1112–1125, 2001.
- [232] Choong Yun Cho. On the triangular decomposition of Cauchy matrices. *Math. Comp.*, 2(104):819–825, 1968.
- [233] Man-Duen Choi. Tricks or treats with the Hilbert matrix. *Amer. Math. Monthly*, 90:301–312, 1983.
- [234] Søren Christiansen and Per Christian Hansen. The effective condition number applied to error analysis of certain boundary collocation methods. *J. Comput. Appl. Math.*, 54:15–36, 1994.
- [235] Eleanor Chu and Alan George. A note on estimating the error in Gaussian elimination without pivoting. *ACM SIGNUM Newsletter*, 20(2):2–7, 1985.
- [236] King-wah Eric Chu. The solution of the matrix equations $AXB - CXD = E$ and $(YA - DZ, YC - BZ) = (E, F)$. *Linear Algebra Appl.*, 93:93–105, 1987.
- [237] Moody T. Chu. On constructing matrices with prescribed singular values and diagonal elements. *Linear Algebra Appl.*, 288:11–22, 1999.

- [238] Moody T. Chu. A fast recursive algorithm for constructing matrices with prescribed eigenvalues and singular values. *SIAM J. Numer. Anal.*, 37(3):1004–1020, 2000.
- [239] C. W. Clenshaw. A note on the summation of Chebyshev series. *M.T.A.C.*, 9(51):118–120, 1955.
- [240] C. W. Clenshaw and F. W. J. Olver. Beyond floating point. *J. Assoc. Comput. Mach.*, 31(2):319–328, 1984.
- [241] C. W. Clenshaw, F. W. J. Olver, and P. R. Turner. Level-index arithmetic: An introductory survey. In *Numerical Analysis and Parallel Processing, Lancaster 1987*, Peter R. Turner, editor, volume 1397 of *Lecture Notes in Mathematics*, Springer-Verlag, Berlin, 1989, pages 95–168.
- [242] A. K. Cline. An elimination method for the solution of linear least squares problems. *SIAM J. Numer. Anal.*, 10(2):283–289, 1973.
- [243] A. K. Cline, C. B. Moler, G. W. Stewart, and J. H. Wilkinson. An estimate for the condition number of a matrix. *SIAM J. Numer. Anal.*, 16(2):368–375, 1979.
- [244] A. K. Cline and R. K. Rew. A set of counter-examples to three condition number estimators. *SIAM J. Sci. Statist. Comput.*, 4(4):602–611, 1983.
- [245] Alan K. Cline, Andrew R. Conn, and Charles F. Van Loan. Generalizing the LINPACK condition estimator. In *Numerical Analysis, Mexico 1981*, J. P. Hennart, editor, volume 909 of *Lecture Notes in Mathematics*, Springer-Verlag, Berlin, 1982, pages 73–83.
- [246] R. E. Cline and R. J. Plemmons. l_2 -solutions to underdetermined linear systems. *SIAM Rev.*, 18(1):92–106, 1976.
- [247] William D. Clinger. How to read floating point numbers accurately. *SIGPLAN Notices*, 25(6):92–101, 1990.
- [248] B. Codenotti, M. Leoncini, and F. P. Preparata. The role of arithmetic in fast parallel matrix inversion. *Algorithmica*, 30(4):685–707, 2001.
- [249] W. J. Cody. Implementation and testing of function software. In *Problems and Methodologies in Mathematical Software Production*, Paul C. Messina and Almerico Murli, editors, volume 142 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 1982, pages 24–47.
- [250] W. J. Cody. ALGORITHM 665 MACHAR: A subroutine to dynamically determine machine parameters. *ACM Trans. Math. Software*, 14(4):303–311, 1988.
- [251] W. J. Cody. Floating-point standards—Theory and practice. In *Reliability in Computing: The Role of Interval Methods in Scientific Computing*, Ramon E. Moore, editor, Academic Press, Boston, MA, USA, 1988, pages 99–107.
- [252] W. J. Cody. Algorithm 714. CELEFUNT: A portable test package for complex elementary functions. *ACM Trans. Math. Software*, 19(1):121, 1993.
- [253] W. J. Cody, J. T. Coonen, D. M. Gay, K. Hanson, D. Hough, W. Kahan, R. Karpinski, J. Palmer, F. N. Ris, and D. Stevenson. A proposed radix- and word-length-independent standard for floating-point arithmetic. *IEEE Micro*, 4(4):86–100, 1984.
- [254] W. J. Cody and Jerome T. Coonen. ALGORITHM 722: Functions to support the IEEE standard for binary floating-point arithmetic. *ACM Trans. Math. Software*, 19(4):443–451, 1993.

- [255] William J. Cody, Jr. Static and dynamic numerical characteristics of floating-point arithmetic. *IEEE Trans. Comput.*, C-22(6):598–601, 1973.
- [256] William J. Cody, Jr. and William Waite. *Software Manual for the Elementary Functions*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1980. x+269 pp. ISBN 0-13-822064-6.
- [257] A. M. Cohen. A note on pivot size in Gaussian elimination. *Linear Algebra Appl.*, 8:361–368, 1974.
- [258] A. M. Cohen. The inverse of a Pascal matrix. *Mathematical Gazette*, 59(408): 111–112, 1975.
- [259] Marty S. Cohen, T. E. Hull, and V. Carl Hamacher. CADAC: A controlled-precision decimal arithmetic unit. *IEEE Trans. Comput.*, C-32(4):370–377, 1983.
- [260] Apple Computer. *Apple Numerics Manual*. Second edition, Addison-Wesley, Reading, MA, USA, 1988.
- [261] Apple Computer. *Inside Macintosh: PowerPC Numerics*. Addison-Wesley, Reading, MA, USA, 1994. ISBN 0-201-40728-0.
- [262] P. Concus, G. H. Golub, and G. Meurant. Block preconditioning for the conjugate gradient method. *SIAM J. Sci. Statist. Comput.*, 6(1):220–252, 1985.
- [263] Andrew R. Conn, Nicholas I. M. Gould, and Philippe L. Toint. *LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A)*. Springer-Verlag, Berlin, 1992. xviii+330 pp. ISBN 0-387-55470-X.
- [264] Samuel D. Conte and Carl de Boor. *Elementary Numerical Analysis: An Algorithmic Approach*. Third edition, McGraw-Hill, Tokyo, 1980. xii+432 pp. ISBN 0-07-066228-2.
- [265] James W. Cooley. How the FFT gained acceptance. In *A History of Scientific Computing*, Stephen G. Nash, editor, Addison-Wesley, Reading, MA, USA, 1990, pages 133–140.
- [266] James W. Cooley. Lanczos and the FFT: A discovery before its time. In *Proceedings of the Cornelius Lanczos International Centenary Conference*, J. David Brown, Moody T. Chu, Donald C. Ellison, and Robert J. Plemmons, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994, pages 3–9.
- [267] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Math. Comp.*, 19(90):297–301, 1965.
- [268] James W. Cooley and John W. Tukey. On the origin and publication of the FFT paper. *Current Contents*, (51–52):8–9, 1993.
- [269] Brian A. Coomes, Hüseyin Koçak, and Kenneth J. Palmer. Rigorous computational shadowing of orbits of ordinary differential equations. *Numer. Math.*, 69(4):401–421, 1995.
- [270] Jerome T. Coonen. Underflow and the denormalized numbers. *Computer*, 14: 75–87, 1981.
- [271] J. E. Cope and B. W. Rust. Bounds on solutions of linear systems with inaccurate data. *SIAM J. Numer. Anal.*, 16(6):950–963, 1979.
- [272] Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progressions. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, ACM Press, New York, 1987, pages 1–6.

- [273] Marius Cornea-Hasegan and Bob Norin. IA-64 floating-point operations and the IEEE standard for binary floating-point arithmetic. *Intel Technology Journal*, Q4, 1999. <http://developer.intel.com/technology/itj/>.
- [274] Richard W. Cottle. Manifestations of the Schur complement. *Linear Algebra Appl.*, 8:189–211, 1974.
- [275] Anthony J. Cox and Nicholas J. Higham. Stability of Householder QR factorization for weighted least squares problems. In *Numerical Analysis 1997, Proceedings of the 17th Dundee Biennial Conference*, D. F. Griffiths, D. J. Higham, and G. A. Watson, editors, volume 380 of *Pitman Research Notes in Mathematics*, Addison Wesley Longman, Harlow, Essex, UK, 1998, pages 57–73.
- [276] Anthony J. Cox and Nicholas J. Higham. Accuracy and stability of the null space method for solving the equality constrained least squares problem. *BIT*, 39(1): 34–50, 1999.
- [277] Anthony J. Cox and Nicholas J. Higham. Backward error bounds for constrained least squares problems. *BIT*, 39(2):210–227, 1999.
- [278] Anthony J. Cox and Nicholas J. Higham. Row-wise backward stable elimination methods for the equality constrained least squares problem. *SIAM J. Matrix Anal. Appl.*, 21(1):313–326, 1999.
- [279] M. G. Cox and S. J. Hammarling, editors. *Reliable Numerical Computation*. Oxford University Press, 1990. xvi+339 pp. ISBN 0-19-853564-3.
- [280] Fred D. Crary. A versatile precompiler for nonstandard arithmetics. *ACM Trans. Math. Software*, 5(2):204–217, 1979.
- [281] Prescott D. Crout. A short method for evaluating determinants and solving systems of linear equations with real or complex coefficients. *Trans. Amer. Inst. Elec. Eng.*, 60:1235–1241, 1941.
- [282] Colin W. Cryer. Pivot size in Gaussian elimination. *Numer. Math.*, 12:335–345, 1968.
- [283] Colin W. Cryer. The LU -factorization of totally positive matrices. *Linear Algebra Appl.*, 7:83–92, 1973.
- [284] Colin W. Cryer. Some properties of totally positive matrices. *Linear Algebra Appl.*, 15:1–25, 1976.
- [285] L. Csanky. Fast parallel matrix inversion algorithms. *SIAM J. Comput.*, 5(4): 618–623, 1976.
- [286] A. R. Curtis and J. K. Reid. On the automatic scaling of matrices for Gaussian elimination. *J. Inst. Maths. Applics.*, 10:118–124, 1972.
- [287] George Cybenko and Charles F. Van Loan. Computing the minimum eigenvalue of a symmetric positive definite Toeplitz matrix. *SIAM J. Sci. Statist. Comput.*, 7(1):123–131, 1986.
- [288] G. Dahlquist. On matrix majorants and minorants, with applications to differential equations. *Linear Algebra Appl.*, 52/53:199–216, 1983.
- [289] Germund Dahlquist and Åke Björck. *Numerical Methods*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1974. xviii+573 pp. Translated by Ned Anderson. ISBN 0-13-627315-7.
- [290] J. W. Daniel, W. B. Gragg, L. Kaufman, and G. W. Stewart. Reorthogonalization and stable algorithms for updating the Gram-Schmidt QR factorization. *Math. Comp.*, 30(136):772–795, 1976.

- [291] B. Danloy. On the choice of signs for Householder's matrices. *J. Comput. Appl. Math.*, 2(1):67–69, 1976.
- [292] Karabi Datta. The matrix equation $XA - BX = R$ and its applications. *Linear Algebra Appl.*, 109:91–105, 1988.
- [293] Philip I. Davies and Nicholas J. Higham. Numerically stable generation of correlation matrices and their factors. *BIT*, 40(4):640–651, 2000.
- [294] Philip J. Davis. *Circulant Matrices*. Wiley, New York, 1979. xv+250 pp. ISBN 0-471-05771-1.
- [295] Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Second edition, Academic Press, London, 1984. xiv+612 pp. ISBN 0-12-206360-0.
- [296] Achiya Dax. Partial pivoting strategies for symmetric Gaussian elimination. *Math. Programming*, 22:288–303, 1982.
- [297] Achiya Dax. The convergence of linear stationary iterative processes for solving singular unstructured systems of linear equations. *SIAM Rev.*, 32(4):611–635, 1990.
- [298] Achiya Dax and S. Kaniel. Pivoting techniques for symmetric Gaussian elimination. *Numer. Math.*, 28:221–241, 1977.
- [299] Jane M. Day and Brian Peterson. Growth in Gaussian elimination. *Amer. Math. Monthly*, 95(6):489–513, 1988.
- [300] Carl de Boor and Allan Pinkus. Backward error analysis for totally positive linear systems. *Numer. Math.*, 27:485–490, 1977.
- [301] Lieuwe Sytse de Jong. Towards a formal definition of numerical stability. *Numer. Math.*, 28:211–219, 1977.
- [302] T. J. Dekker. A floating-point technique for extending the available precision. *Numer. Math.*, 18:224–242, 1971.
- [303] T. J. Dekker and W. Hoffmann. Rehabilitation of the Gauss-Jordan algorithm. *Numer. Math.*, 54:591–599, 1989.
- [304] Ron S. Dembo, Stanley C. Eisenstat, and Trond Steihaug. Inexact Newton methods. *SIAM J. Numer. Anal.*, 19(2):400–408, 1982.
- [305] Cédric J. Demeure. Fast QR factorization of Vandermonde matrices. *Linear Algebra Appl.*, 122/3/4:165–194, 1989.
- [306] Cédric J. Demeure. QR factorization of confluent Vandermonde matrices. *IEEE Trans. Acoust., Speech, Signal Processing*, 38(10):1799–1802, 1990.
- [307] James W. Demmel. The condition number of equivalence transformations that block diagonalize matrix pencils. *SIAM J. Numer. Anal.*, 20(3):599–610, 1983.
- [308] James W. Demmel. Underflow and the reliability of numerical software. *SIAM J. Sci. Statist. Comput.*, 5(4):887–919, 1984.
- [309] James W. Demmel. On condition numbers and the distance to the nearest ill-posed problem. *Numer. Math.*, 51:251–289, 1987.
- [310] James W. Demmel. On error analysis in arithmetic with varying relative precision. In *Proceedings of the Eighth Symposium on Computer Arithmetic, Como, Italy*, Mary Jane Irwin and Renato Stefanelli, editors, IEEE Computer Society, Washington, D.C., 1987, pages 148–152.
- [311] James W. Demmel. On floating point errors in Cholesky. Technical Report CS-89-87, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, October 1989. 6 pp. LAPACK Working Note 14.

- [312] James W. Demmel. On the odor of IEEE arithmetic. *NA Digest*, Volume 91, Issue 39, 1991. (Response to a message “IEEE Arithmetic Stinks” in Volume 91, Issue 33.)
- [313] James W. Demmel. The componentwise distance to the nearest singular matrix. *SIAM J. Matrix Anal. Appl.*, 13(1):10–19, 1992.
- [314] James W. Demmel. Open problems in numerical linear algebra. IMA Preprint Series #961, Institute for Mathematics and Its Applications, University of Minnesota, Minneapolis, MN, USA, April 1992. 21 pp. LAPACK Working Note 47.
- [315] James W. Demmel. A specification for floating point parallel prefix. Technical Report CS-92-167, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, July 1992. 8 pp. LAPACK Working Note 49.
- [316] James W. Demmel. Trading off parallelism and numerical stability. In *Linear Algebra for Large Scale and Real-Time Applications*, Marc S. Moonen, Gene H. Golub, and Bart L. De Moor, editors, volume 232 of *NATO ASI Series E*, Kluwer Academic Publishers, Dordrecht, The Netherlands, 1993, pages 49–68.
- [317] James W. Demmel. *Applied Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. xi+419 pp. ISBN 0-89871-389-7.
- [318] James W. Demmel. Accurate singular value decompositions of structured matrices. *SIAM J. Matrix Anal. Appl.*, 21(2):562–580, 1999.
- [319] James W. Demmel. A note on accurate floating point summation. Manuscript, 2001. 5 pp.
- [320] James W. Demmel, Inderjit Dhillon, and Huan Ren. On the correctness of some bisection-like parallel eigenvalue algorithms in floating point arithmetic. *Electron. Trans. Numer. Anal.*, 3:116–149, 1995.
- [321] James W. Demmel, Benjamin Diament, and Gregorio Malajovich. On the complexity of computing error bounds. *Found. Comput. Math.*, 1(1):101–125, 2001.
- [322] James W. Demmel, J. J. Dongarra, and W. Kahan. On designing portable high performance numerical libraries. In *Numerical Analysis 1991, Proceedings of the 14th Dundee Conference*, D. F. Griffiths and G. A. Watson, editors, volume 260 of *Pitman Research Notes in Mathematics*, Longman Scientific and Technical, Essex, UK, 1992, pages 69–84.
- [323] James W. Demmel, Ming Gu, Stanley Eisenstat, Ivan Slapničar, Krešimir Veselić, and Zlatko Drmač. Computing the singular value decomposition with high relative accuracy. *Linear Algebra Appl.*, 299:21–80, 1999.
- [324] James W. Demmel and Nicholas J. Higham. Stability of block algorithms with fast level-3 BLAS. *ACM Trans. Math. Software*, 18(3):274–291, 1992.
- [325] James W. Demmel and Nicholas J. Higham. Improved error bounds for underdetermined system solvers. *SIAM J. Matrix Anal. Appl.*, 14(1):1–14, 1993.
- [326] James W. Demmel, Nicholas J. Higham, and Robert S. Schreiber. Stability of block *LU* factorization. *Numerical Linear Algebra with Applications*, 2(2):173–190, 1995.
- [327] James W. Demmel and Bo Kågström. Computing stable eigendecompositions of matrix pencils. *Linear Algebra Appl.*, 88/89:139–186, 1987.
- [328] James W. Demmel and Bo Kågström. Accurate solutions of ill-posed problems in control theory. *SIAM J. Matrix Anal. Appl.*, 9(1):126–145, 1988.

- [329] James W. Demmel and W. Kahan. Accurate singular values of bidiagonal matrices. *SIAM J. Sci. Statist. Comput.*, 11(5):873–912, 1990.
- [330] James W. Demmel and F. Krückeberg. An interval algorithm for solving systems of linear equations to prespecified accuracy. *Computing*, 34:117–129, 1985.
- [331] James W. Demmel and Xiaoye Li. Faster numerical algorithms via exception handling. *IEEE Trans. Comput.*, 43(8):983–992, 1994.
- [332] James W. Demmel and A. McKenney. A test matrix generation suite. Preprint MCS-P69-0389, Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, IL, USA, March 1989. 16 pp. LAPACK Working Note 9.
- [333] John E. Dennis, Jr. and Robert B. Schnabel. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1983. xiii+378 pp. Reprinted by Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1996. ISBN 0-13-627216-9 (original), 0-89871-364-1 (reprint).
- [334] John E. Dennis, Jr. and Virginia Torczon. Direct search methods on parallel machines. *SIAM J. Optim.*, 1(4):448–474, 1991.
- [335] John E. Dennis, Jr. and Homer F. Walker. Inaccuracy in quasi-Newton methods: Local improvement theorems. *Math. Prog. Study*, 22:70–85, 1984.
- [336] John E. Dennis, Jr. and Henry Wolkowicz. Sizing and least-change secant methods. *SIAM J. Numer. Anal.*, 30(5):1291–1314, 1993.
- [337] J. Descloux. Note on the round-off errors in iterative processes. *Math. Comp.*, 17:18–27, 1963.
- [338] Inderjit Dhillon. Reliable computation of the condition number of a tridiagonal matrix in $O(n)$ time. *SIAM J. Matrix Anal. Appl.*, 19(3):776–796, 1998.
- [339] Harold G. Diamond. Stability of rounded off inverses under iteration. *Math. Comp.*, 32(141):227–232, 1978.
- [340] John D. Dixon. Estimating extremal eigenvalues and condition numbers of matrices. *SIAM J. Numer. Anal.*, 20(4):812–814, 1983.
- [341] J. J. Dongarra, J. R. Bunch, C. B. Moler, and G. W. Stewart. *LINPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1979. ISBN 0-89871-172-X.
- [342] J. J. Dongarra, J. J. Du Croz, I. S. Duff, and S. J. Hammarling. A set of Level 3 basic linear algebra subprograms. *ACM Trans. Math. Software*, 16:1–17, 1990.
- [343] J. J. Dongarra, J. J. Du Croz, I. S. Duff, and S. J. Hammarling. Algorithm 679. A set of Level 3 basic linear algebra subprograms: Model implementation and test programs. *ACM Trans. Math. Software*, 16:18–28, 1990.
- [344] J. J. Dongarra, F. G. Gustavson, and A. Karp. Implementing linear algebra algorithms for dense matrices on a vector pipeline machine. *SIAM Rev.*, 26(1):91–112, 1984.
- [345] Jack Dongarra, Victor Eijkhout, and Piotr Luszczek. Recursive approach in sparse matrix LU factorization. In *Proceedings of the 1st SGI Users Conference*, Cracow, Poland, October 2000, pages 409–418. ACC Cyfronet UMM.
- [346] Jack J. Dongarra. Performance of various computers using standard linear equations software. Technical Report CS-89-85, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, August 2001. 58 pp.

- [347] Jack J. Dongarra, Jeremy J. Du Croz, Sven J. Hammarling, and Richard J. Hanson. An extended set of Fortran basic linear algebra subprograms. *ACM Trans. Math. Software*, 14(1):1–17, 1988.
- [348] Jack J. Dongarra, Jeremy J. Du Croz, Sven J. Hammarling, and Richard J. Hanson. Algorithm 656. An extended set of Fortran basic linear algebra subprograms: Model implementation and test programs. *ACM Trans. Math. Software*, 14(1):18–32, 1988.
- [349] Jack J. Dongarra, Iain S. Duff, Danny C. Sorensen, and Henk A. van der Vorst. *Numerical Linear Algebra for High-Performance Computers*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1998. xviii+342 pp. ISBN 0-89871-428-1.
- [350] Jack J. Dongarra and Eric Grosse. Distribution of mathematical software via electronic mail. *Comm. ACM*, 30(5):403–407, 1987.
- [351] Craig C. Douglas, Michael Heroux, Gordon Sliselman, and Roger M. Smith. GEMMW: A portable level 3 BLAS Winograd variant of Strassen’s matrix–matrix multiply algorithm. *J. Comput. Phys.*, 110:1–10, 1994.
- [352] Craig C. Douglas and Gordon Sliselman. Variants of matrix-matrix multiplication for Fortran-90. *ACM SIGNUM Newsletter*, 29:4–6, 1994.
- [353] Jim Douglas, Jr. Round-off error in the numerical solution of the heat equation. *J. Assoc. Comput. Mach.*, 6:48–58, 1959.
- [354] Thomas C. Doyle. Inversion of symmetric coefficient matrix of positive-definite quadratic form. *M.T.A.C.*, 11:55–58, 1957.
- [355] J. Drkošová, A. Greenbaum, M. Rozložník, and Z. Strakoš. Numerical stability of GMRES. *BIT*, 35:309–330, 1995.
- [356] Zlatko Drmač, Matjaž Omladič, and Krešimir Veselić. On the perturbation of the Cholesky factorization. *SIAM J. Matrix Anal. Appl.*, 15(4):1319–1332, 1994.
- [357] Jeremy J. Du Croz and Nicholas J. Higham. Stability of methods for matrix inversion. *IMA J. Numer. Anal.*, 12:1–19, 1992.
- [358] Augustin A. Dubrulle. A class of numerical methods for the computation of Pythagorean sums. *IBM J. Res. Develop.*, 27(6):582–589, 1983.
- [359] Augustin A. Dubrulle and Gene H. Golub. A multishift QR iteration without computation of the shifts. *Numerical Algorithms*, 7:173–181, 1994.
- [360] I. S. Duff, A. M. Erisman, and J. K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, 1986. xiii+341 pp. ISBN 0-19-853408-6.
- [361] I. S. Duff, N. I. M. Gould, J. K. Reid, J. A. Scott, and K. Turner. The factorization of sparse symmetric indefinite matrices. *IMA J. Numer. Anal.*, 11:181–204, 1991.
- [362] Iain S. Duff, Roger G. Grimes, and John G. Lewis. Sparse matrix test problems. *ACM Trans. Math. Software*, 15(1):1–14, 1989.
- [363] Iain S. Duff, Roger G. Grimes, and John G. Lewis. Users’ guide for the Harwell–Boeing sparse matrix collection (release 1). Report RAL-92-086, Atlas Centre, Rutherford Appleton Laboratory, Didcot, Oxon, UK, December 1992. 84 pp.
- [364] Iain S. Duff and John K. Reid. MA27—A set of Fortran subroutines for solving sparse symmetric sets of linear equations. Technical Report AERE R10533, AERE Harwell Laboratory, July 1982. Published by Her Majesty’s Stationary Office, London.

- [365] Iain S. Duff and John K. Reid. MA47, a Fortran code for direct solution of indefinite sparse symmetric linear systems. Report RAL-95-001, Atlas Centre, Rutherford Appleton Laboratory, Didcot, Oxon, UK, January 1995. 63 pp.
- [366] Iain S. Duff, John K. Reid, Neils Munksgaard, and Hans B. Nielsen. Direct solution of sets of linear equations whose matrix is sparse, symmetric and indefinite. *J. Inst. Maths. Applics.*, 23:235–250, 1979.
- [367] Bogdan Dumitrescu. Improving and estimating the accuracy of Strassen's algorithm. *Numer. Math.*, 79:485–499, 1998.
- [368] William Dunham. *Journey Through Genius: The Great Theorems of Mathematics*. Penguin, New York, 1990. xiii+300 pp. ISBN 0-14-014739-X.
- [369] Paul S. Dwyer. *Linear Computations*. Wiley, New York, 1951. xi+344 pp.
- [370] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- [371] Alan Edelman. Eigenvalues and condition numbers of random matrices. *SIAM J. Matrix Anal. Appl.*, 9(4):543–560, 1988.
- [372] Alan Edelman. The distribution and moments of the smallest eigenvalue of a random matrix of Wishart type. *Linear Algebra Appl.*, 159:55–80, 1991.
- [373] Alan Edelman. The first annual large dense linear system survey. *ACM SIGNUM Newsletter*, 26:6–12, 1991.
- [374] Alan Edelman. The complete pivoting conjecture for Gaussian elimination is false. *The Mathematica Journal*, 2:58–61, 1992.
- [375] Alan Edelman. On the distribution of a scaled condition number. *Math. Comp.*, 58(197):185–190, 1992.
- [376] Alan Edelman. Eigenvalue roulette and random test matrices. In *Linear Algebra for Large Scale and Real-Time Applications*, Marc S. Moonen, Gene H. Golub, and Bart L. De Moor, editors, volume 232 of *NATO ASI Series E*, Kluwer Academic Publishers, Dordrecht, The Netherlands, 1993, pages 365–368.
- [377] Alan Edelman. Large dense numerical linear algebra in 1993: The parallel computing influence. *Internat. J. Supercomputer Appl.*, 7(2):113–128, 1993.
- [378] Alan Edelman. Scalable dense numerical linear algebra in 1994: The multicompiler influence. In *Proceedings of the Fifth SIAM Conference on Applied Linear Algebra*, John G. Lewis, editor, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994, pages 344–348.
- [379] Alan Edelman. When is $x * (1/x) \neq 1$? Manuscript, 1994.
- [380] Alan Edelman. The mathematics of the Pentium division bug. *SIAM Rev.*, 39(1): 54–67, 1997.
- [381] Alan Edelman, Eric Kostlan, and Michael Shub. How many eigenvalues of a random matrix are real? *J. Amer. Math. Soc.*, 7(1):247–267, 1994.
- [382] Alan Edelman and Walter Mascarinas. On the complete pivoting conjecture for a Hadamard matrix of order 12. *Linear and Multilinear Algebra*, 38(3):181–188, 1995.
- [383] Alan Edelman and H. Murakami. Polynomial roots from companion matrix eigenvalues. *Math. Comp.*, 64(210):763–776, 1995.
- [384] Alan Edelman and G. W. Stewart. Scaling for orthogonality. *IEEE Trans. Signal Processing*, 41(4):1676–1677, 1993.

- [385] Editor's note. *SIAM J. Matrix Anal. Appl.*, 12(3), 1991.
- [386] Timo Eirola. Aspects of backward error analysis of numerical ODEs. *J. Comput. Appl. Math.*, 45(1-2):65–73, 1993.
- [387] Lars Eldén. Perturbation theory for the least squares problem with linear equality constraints. *SIAM J. Numer. Anal.*, 17(3):338–350, 1980.
- [388] Samuel K. Eldersveld and Michael A. Saunders. A block-*LU* update for large-scale linear programming. *SIAM J. Matrix Anal. Appl.*, 13(1):191–201, 1992.
- [389] Mark Embree and Lloyd N. Trefethen. Pseudospectra gateway. <http://www.comlab.ox.ac.uk/pseudospectra/>.
- [390] W. H. Enright. A new error-control for initial value solvers. *Appl. Math. Comput.*, 31:288–301, 1989.
- [391] Michael A. Epton. Methods for the solution of $AXD - BXC = E$ and its application in the numerical solution of implicit ordinary differential equations. *BIT*, 20:341–345, 1980.
- [392] A. M. Erisman, R. G. Grimes, J. G. Lewis, W. G. Poole, and H. D. Simon. Evaluation of orderings for unsymmetric sparse matrices. *SIAM J. Sci. Statist. Comput.*, 8(4):600–624, 1987.
- [393] A. M. Erisman and J. K. Reid. Monitoring the stability of the triangular factorization of a sparse matrix. *Numer. Math.*, 22:183–186, 1974.
- [394] Terje O. Espelid. On floating-point summation. *SIAM Rev.*, 37(4):603–607, 1995.
- [395] Christopher Evans. *Interview with J. H. Wilkinson*. Number 10 in *Pioneers of Computing, 60-Minute Recordings of Interviews*. Science Museum, London, 1976.
- [396] Ömer Eğecioğlu, E. Gallopoulos, and Ç. K. Koç. A parallel method for fast and practical high-order Newton interpolation. *BIT*, 30:268–288, 1990.
- [397] John Ewing, editor. *A Century of Mathematics Through the Eyes of the Monthly*. Mathematical Association of America, Washington, D.C., 1994. xi+323 pp. ISBN 0-88385-459-7.
- [398] John H. Ewing and F. W. Gehring, editors. *Paul Halmos: Celebrating 50 Years of Mathematics*. Springer-Verlag, New York, 1991. viii+320 pp. ISBN 3-540-97509-8.
- [399] V. N. Faddeeva. *Computational Methods of Linear Algebra*. Dover, New York, 1959. x+252 pp. ISBN 0-486-60424-1.
- [400] Shaun M. Fallat. Bidiagonal factorizations of totally nonnegative matrices. *Amer. Math. Monthly*, 108(8):697–712, 2001.
- [401] Ky Fan and A. J. Hoffman. Some metric inequalities in the space of matrices. *Proc. Amer. Math. Soc.*, 6:111–116, 1955.
- [402] R. W. Farebrother. A memoir of the life of M. H. Doolittle. *IMA Bulletin*, 23(6/7):102, 1987.
- [403] R. W. Farebrother. *Linear Least Squares Computations*. Marcel Dekker, New York, 1988. xiii+293 pp. ISBN 0-8247-7661-5.
- [404] Charles Farnum. Compiler support for floating-point computation. *Software—Practice and Experience*, 18(7):701–709, 1988.
- [405] Richard J. Fateman. High-level language implications of the proposed IEEE floating-point standard. *ACM Trans. Program. Lang. Syst.*, 4(2):239–257, 1982.

- [406] David G. Feingold and Richard S. Varga. Block diagonally dominant matrices and generalizations of the Gerschgorin circle theorem. *Pacific J. Math.*, 12:1241–1250, 1962.
- [407] Alan Feldstein and Richard Goodman. Loss of significance in floating point subtraction and addition. *IEEE Trans. Comput.*, C-31(4):328–335, 1982.
- [408] Alan Feldstein and Peter Turner. Overflow, underflow, and severe loss of significance in floating-point addition and subtraction. *IMA J. Numer. Anal.*, 6: 241–251, 1986.
- [409] Warren E. Ferguson, Jr. Exact computation of a sum or difference with applications to argument reduction. In *Proc. 12th IEEE Symposium on Computer Arithmetic, Bath, England*, Simon Knowles and William H. McAllister, editors, IEEE Computer Society Press, Los Alamitos, CA, USA, 1995, pages 216–221.
- [410] Warren E. Ferguson, Jr. and Tom Brightman. Accurate and monotone approximations of some transcendental functions. In *Proc. 10th IEEE Symposium on Computer Arithmetic*, Peter Kornerup and David W. Matula, editors, IEEE Computer Society Press, Los Alamitos, CA, USA, 1991, pages 237–244.
- [411] William R. Ferng, Gene H. Golub, and Robert J. Plemmons. Adaptive Lanczos methods for recursive condition estimation. *Numerical Algorithms*, 1(1):1–19, 1991.
- [412] C. T. Fike. Methods of evaluating polynomial approximations in function evaluation routines. *Comm. ACM*, 10(3):175–178, 1967.
- [413] C. T. Fike. *Computer Evaluation of Mathematical Functions*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1968. xii+227 pp.
- [414] Patrick C. Fischer. Further schemes for combining matrix algorithms. In *Automata, Languages and Programming*, Jacques Loeckx, editor, volume 14 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 1974, pages 428–436.
- [415] R. Fletcher. Factorizing symmetric indefinite matrices. *Linear Algebra Appl.*, 14: 257–272, 1976.
- [416] R. Fletcher. Expected conditioning. *IMA J. Numer. Anal.*, 5:247–273, 1985.
- [417] R. Fletcher. *Practical Methods of Optimization*. Second edition, Wiley, Chichester, UK, 1987. xiv+436 pp. ISBN 0-471-91547-5.
- [418] R. Fletcher and M. J. D. Powell. On the modification of LDL^T factorizations. *Math. Comp.*, 28(128):1067–1087, 1974.
- [419] Brian Ford. Parameterization of the environment for transportable numerical software. *ACM Trans. Math. Software*, 4(2):100–103, 1978.
- [420] Anders Forsgren, Philip E. Gill, and Walter Murray. Computing modified Newton directions using a partial Cholesky factorization. *SIAM J. Sci. Comput.*, 16(1): 139–150, 1995.
- [421] Anders Forsgren, Philip E. Gill, and Joseph R. Shinnerl. Stability of symmetric ill-conditioned systems arising in interior methods for constrained optimization. *SIAM J. Matrix Anal. Appl.*, 17(1):187–211, 1996.
- [422] George E. Forsythe. Gauss to Gerling on relaxation. *Mathematical Tables and Other Aids to Computation*, 5:255–258, 1951.
- [423] George E. Forsythe. Solving linear algebraic equations can be interesting. *Bull. Amer. Math. Soc.*, 59(4):299–329, 1953.

- [424] George E. Forsythe. Reprint of a note on rounding-off errors. *SIAM Rev.*, 1(1): 66–67, 1959.
- [425] George E. Forsythe. Algorithm 16: Crout with pivoting. *Comm. ACM*, 3(9): 507–508, 1960.
- [426] George E. Forsythe. Today’s computational methods of linear algebra. *SIAM Rev.*, 9:489–515, 1967.
- [427] George E. Forsythe. Solving a quadratic equation on a computer. In *The Mathematical Sciences: A Collection of Essays*, National Research Council’s Committee on Support of Research in the Mathematical Sciences, editor, MIT Press, Cambridge, MA, USA, 1969, pages 138–152.
- [428] George E. Forsythe. What is a satisfactory quadratic equation solver? In *Constructive Aspects of the Fundamental Theorem of Algebra*, Bruno Dejon and Peter Henrici, editors, Wiley-Interscience, London, 1969, pages 53–61.
- [429] George E. Forsythe. Pitfalls in computation, or why a math book isn’t enough. *Amer. Math. Monthly*, 77:931–956, 1970.
- [430] George E. Forsythe, Michael A. Malcolm, and Cleve B. Moler. *Computer Methods for Mathematical Computations*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1977. xi+259 pp. ISBN 0-13-165332-6.
- [431] George E. Forsythe and Cleve B. Moler. *Computer Solution of Linear Algebraic Systems*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1967. xi+148 pp.
- [432] George E. Forsythe and E. G. Straus. On best conditioned matrices. *Proc. Amer. Math. Soc.*, 6:340–345, 1955.
- [433] Leslie Foster. Modifications of the normal equations method that are numerically stable. In *Numerical Linear Algebra, Digital Signal Processing and Parallel Algorithms*, G. H. Golub and P. M. Van Dooren, editors, volume F70 of *NATO ASI Series*, Springer-Verlag, Berlin, 1991, pages 501–512.
- [434] Leslie V. Foster. Gaussian elimination with partial pivoting can fail in practice. *SIAM J. Matrix Anal. Appl.*, 15(4):1354–1362, 1994.
- [435] Leslie V. Foster. The growth factor and efficiency of Gaussian elimination with rook pivoting. *J. Comput. Appl. Math.*, 86:177–194, 1997. Corrigendum in *J. Comput. Appl. Math.*, 98:177, 1998.
- [436] L. Fox. *An Introduction to Numerical Linear Algebra*. Oxford University Press, 1964. xi+328 pp.
- [437] L. Fox. How to get meaningless answers in scientific computation (and what to do about it). *IMA Bulletin*, 7(10):296–302, 1971.
- [438] L. Fox. All about Jim Wilkinson, with a commemorative snippet on backward error analysis. In *The Contribution of Dr. J. H. Wilkinson to Numerical Analysis*, Symposium Proceedings Series No. 19, The Institute of Mathematics and Its Applications, Southend-On-Sea, Essex, UK, 1978, pages 1–20.
- [439] L. Fox. James Hardy Wilkinson, 1919–1986. *Biographical Memoirs of Fellows of the Royal Society*, 33:671–708, 1987.
- [440] L. Fox, H. D. Huskey, and J. H. Wilkinson. Notes on the solution of algebraic linear simultaneous equations. *Quart. J. Mech. Appl. Math.*, 1:149–173, 1948.
- [441] P. A. Fox, A. D. Hall, and N. L. Schryer. The PORT mathematical subroutine library. *ACM Trans. Math. Software*, 4(2):104–126, 1978.

- [442] Philippe Francois and Jean-Michel Muller. The SCALP perturbation method. In *Proceedings of IMACS '91, 13th World Congress on Computation and Applied Mathematics, Dublin, Volume 1*, 1991, pages 59–60.
- [443] Werner L. Frank. Computing eigenvalues of complex matrices by determinant evaluation and by methods of Danilewski and Wielandt. *J. Soc. Indust. Appl. Math.*, 6:378–392, 1958.
- [444] V. Frayssé. *Reliability of Computer Solutions*. PhD thesis, L'Institut National Polytechnique de Toulouse, Toulouse, France, July 1992. CERFACS Report TH/PA/92/11.
- [445] Shmuel Friedland. Revisiting matrix squaring. *Linear Algebra Appl.*, 154–156: 59–63, 1991.
- [446] Shmuel Friedland and Hans Schneider. The growth of powers of a nonnegative matrix. *SIAM J. Alg. Discrete Methods*, 1(2):185–200, 1980.
- [447] Matteo Frigo and Steven G. Johnson. FFTW: An adaptive software architecture for the FFT. In *Proceedings of International Conference on Acoustics, Speech and Signal Processing, Volume 3*, 1998, pages 1381+.
- [448] Andreas Frommer. Proving conjectures by use of interval arithmetic. In *Perspectives on Enclosure Methods*, A. Facius, U. Kulisch, and R. Lohner, editors, Springer-Verlag, New York, 2001, pages 1–13.
- [449] R. E. Funderlic, M. Neumann, and R. J. Plemmons. LU decompositions of generalized diagonally dominant matrices. *Numer. Math.*, 40:57–69, 1982.
- [450] Pascal M. Gahinet, Alan J. Laub, Charles S. Kenney, and Gary A. Hewan. Sensitivity of the stable discrete-time Lyapunov equation. *IEEE Trans. Automat. Control*, AC-35(11):1209–1217, 1990.
- [451] Shmuel Gal and Boris Bachelis. An accurate elementary mathematical library for the IEEE floating point standard. *ACM Trans. Math. Software*, 17(1):26–45, 1991.
- [452] K. A. Gallivan, R. J. Plemmons, and A. H. Sameh. Parallel algorithms for dense linear algebra computations. *SIAM Rev.*, 32(1):54–135, 1990.
- [453] F. R. Gantmacher. *The Theory of Matrices*, volume one. Chelsea, New York, 1959. x+374 pp. ISBN 0-8284-0131-4.
- [454] F. R. Gantmacher. *The Theory of Matrices*, volume two. Chelsea, New York, 1959. ix+276 pp. ISBN 0-8284-0133-0.
- [455] B. S. Garbow, J. M. Boyle, J. J. Dongarra, and C. B. Moler. *Matrix Eigensystem Routines—EISPACK Guide Extension*, volume 51 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, 1977. viii+343 pp. ISBN 3-540-08254-9.
- [456] Judith D. Gardiner and Alan J. Laub. Parallel algorithms for algebraic Riccati equations. *Internat. J. Control*, 54(6):1317–1333, 1991.
- [457] Judith D. Gardiner, Alan J. Laub, James J. Amato, and Cleve B. Moler. Solution of the Sylvester matrix equation $AXB^T + CXD^T = E$. *ACM Trans. Math. Software*, 18(2):223–231, 1992.
- [458] Judith D. Gardiner, Matthew R. Wette, Alan J. Laub, James J. Amato, and Cleve B. Moler. Algorithm 705: A FORTRAN-77 software package for solving the Sylvester matrix equation $AXB^T + CXD^T = E$. *ACM Trans. Math. Software*, 18(2):232–238, 1992.

- [459] Martin Gardner. *More Mathematical Puzzles and Diversions*. Penguin, New York, 1961. 187 pp. ISBN 0-14-020748-1.
- [460] Harvey L. Garner. A survey of some recent contributions to computer arithmetic. *IEEE Trans. Comput.*, C-25(12):1277–1282, 1976.
- [461] M. Gasca and J. M. Peña. Total positivity and Neville elimination. *Linear Algebra Appl.*, 165:25–44, 1992.
- [462] Noel Gastinel. *Linear Numerical Analysis*. Kershaw Publishing, London, 1983. ix+341 pp. First published in English by Academic Press, London, 1970. ISBN 0-901665-16-9.
- [463] Carl Friedrich Gauss. *Theory of the Combination of Observations Least Subject to Errors. Part One, Part Two, Supplement*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1995. xi+241 pp. Translated from the Latin by G. W. Stewart. ISBN 0-89871-347-1.
- [464] Walter Gautschi. On inverses of Vandermonde and confluent Vandermonde matrices. *Numer. Math.*, 4:117–123, 1962.
- [465] Walter Gautschi. Norm estimates for inverses of Vandermonde matrices. *Numer. Math.*, 23:337–347, 1975.
- [466] Walter Gautschi. On inverses of Vandermonde and confluent Vandermonde matrices III. *Numer. Math.*, 29:445–450, 1978.
- [467] Walter Gautschi. The condition of Vandermonde-like matrices involving orthogonal polynomials. *Linear Algebra Appl.*, 52/53:293–300, 1983.
- [468] Walter Gautschi. How (un)stable are Vandermonde systems? In *Asymptotic and Computational Analysis*, R. Wong, editor, volume 124 of *Lecture Notes in Pure and Applied Mathematics*, Marcel Dekker, New York and Basel, 1990, pages 193–210.
- [469] Werner Gautschi. The asymptotic behaviour of powers of matrices. *Duke Math. J.*, 20:127–140, 1953.
- [470] David M. Gay. Correctly rounded binary-decimal and decimal-binary conversions. Numerical Analysis Manuscript 90-10, AT&T Bell Laboratories, Murray Hill, NJ, USA, November 1990. 16 pp.
- [471] Stuart Geman. The spectral radius of large random matrices. *Ann. Probab.*, 14(4):1318–1328, 1986.
- [472] W. M. Gentleman. An error analysis of Goertzel's (Watt's) method for computing Fourier coefficients. *Comput. J.*, 12:160–165, 1969.
- [473] W. M. Gentleman and G. Sande. Fast Fourier transforms—for fun and profit. In *Fall Joint Computer Conference*, volume 29 of *AFIPS Conference Proceedings*, Spartan Books, Washington, D.C., 1966, pages 563–578.
- [474] W. Morven Gentleman. Least squares computations by Givens transformations without square roots. *J. Inst. Maths. Applics.*, 12:329–336, 1973.
- [475] W. Morven Gentleman. Error analysis of QR decompositions by Givens transformations. *Linear Algebra Appl.*, 10:189–197, 1975.
- [476] W. Morven Gentleman and Scott B. Marovich. More on algorithms that reveal properties of floating point arithmetic units. *Comm. ACM*, 17(5):276–277, 1974.
- [477] Alan George, Khakim D. Ikramov, and Andrey B. Kuchеров. On the growth factor in Gaussian elimination for generalized Higham matrices. *Numerical Linear Algebra with Applications*, 9:107–114, 2002.

- [478] Alan George and Joseph W-H Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1981. xii+324 pp. ISBN 0-13-165274-5.
- [479] A. J. Geurts. A contribution to the theory of condition. *Numer. Math.*, 39:85–96, 1982.
- [480] Ali R. Ghavimi and Alan J. Laub. Backward error, sensitivity, and refinement of computed solutions of algebraic Riccati equations. *Numerical Linear Algebra with Applications*, 2(1):29–49, 1995.
- [481] Ali R. Ghavimi and Alan J. Laub. Residual bounds for discrete-time Lyapunov equations. *IEEE Trans. Automat. Control*, 40(7):1244–1249, 1995.
- [482] P. E. Gill and W. Murray. A numerically stable form of the simplex algorithm. *Linear Algebra Appl.*, 7:99–138, 1973.
- [483] Philip E. Gill, Walter Murray, Dulce B. Ponceleón, and Michael A. Saunders. Preconditioners for indefinite systems arising in optimization. *SIAM J. Matrix Anal. Appl.*, 13(1):292–311, 1992.
- [484] Philip E. Gill, Walter Murray, Michael A. Saunders, and Margaret H. Wright. User's guide for NPSOL (version 4.0): A Fortran package for nonlinear programming. Technical Report SOL 86-2, Department of Operations Research, Stanford University, Stanford, CA, USA, January 1986. 53 pp.
- [485] Philip E. Gill, Walter Murray, Michael A. Saunders, and Margaret H. Wright. Inertia-controlling methods for general quadratic programming. *SIAM Rev.*, 33(1):1–36, 1991.
- [486] Philip E. Gill, Walter Murray, and Margaret H. Wright. *Practical Optimization*. Academic Press, London, 1981. xvi+401 pp. ISBN 0-12-283952-8.
- [487] Philip E. Gill, Michael A. Saunders, and Joseph R. Shinnerl. On the stability of Cholesky factorization for symmetric quasidefinite systems. *SIAM J. Matrix Anal. Appl.*, 17(1):35–46, 1996.
- [488] S. Gill. A process for the step-by-step integration of differential equations in an automatic digital computing machine. *Proc. Cambridge Phil. Soc.*, 47:96–108, 1951.
- [489] T. A. Gillespie. Noncommutative variations on theorems of Marcel Riesz and others. In *Paul Halmos: Celebrating 50 Years of Mathematics*, John H. Ewing and F. W. Gehring, editors, Springer-Verlag, Berlin, 1991, pages 221–236.
- [490] Wallace J. Givens. Numerical computation of the characteristic values of a real symmetric matrix. Technical Report ORNL-1574, Oak Ridge National Laboratory, Oak Ridge, TN, USA, 1954. 107 pp.
- [491] James Glanz. Mathematical logic flushes out the bugs in chip designs. *Science*, 267:332–333, 1995. 20 January.
- [492] J. Gluchowska and A. Smoktunowicz. Solving the linear least squares problem with very high relative accuracy. *Computing*, 45:345–354, 1990.
- [493] S. K. Godunov, A. G. Antonov, O. P. Kiriljuk, and V. I. Kostin. *Guaranteed Accuracy in Numerical Linear Algebra*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 1993. xi+535 pp. ISBN 0-7923-2352-1.
- [494] I. Gohberg and I. Koltracht. Mixed, componentwise, and structured condition numbers. *SIAM J. Matrix Anal. Appl.*, 14(3):688–704, 1993.

- [495] I. Gohberg and V. Olshevsky. Fast inversion of Chebyshev–Vandermonde matrices. *Numer. Math.*, 67(1):71–92, 1994.
- [496] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.
- [497] I. Bennett Goldberg. 27 bits are not enough for 8-digit accuracy. *Comm. ACM*, 10(2):105–106, 1967.
- [498] Moshe Goldberg and E. G. Straus. Multiplicativity of l_p norms for matrices. *Linear Algebra Appl.*, 52/53:351–360, 1983.
- [499] Herman H. Goldstine. *The Computer: From Pascal to von Neumann*. Princeton University Press, Princeton, NJ, USA, 1972. xii+378 pp. 1993 printing with new preface. ISBN 0-691-02367-0.
- [500] Herman H. Goldstine. *A History of Numerical Analysis from the 16th through the 19th Century*. Springer-Verlag, New York, 1977. xiv+348 pp. ISBN 0-387-90277-5.
- [501] Herman H. Goldstine and John von Neumann. Numerical inverting of matrices of high order II. *Proc. Amer. Math. Soc.*, 2:188–202, 1951. Reprinted in [1130, pp. 558–572].
- [502] G. H. Golub. Numerical methods for solving linear least squares problems. *Numer. Math.*, 7:206–216, 1965.
- [503] G. H. Golub, S. Nash, and C. F. Van Loan. A Hessenberg–Schur method for the problem $AX + XB = C$. *IEEE Trans. Automat. Control*, AC-24(6):909–913, 1979.
- [504] G. H. Golub and J. M. Varah. On a characterization of the best ℓ_2 -scaling of a matrix. *SIAM J. Numer. Anal.*, 11(3):472–479, 1974.
- [505] G. H. Golub and J. H. Wilkinson. Note on the iterative refinement of least squares solution. *Numer. Math.*, 9:139–148, 1966.
- [506] G. H. Golub and J. H. Wilkinson. Ill-conditioned eigensystems and the computation of the Jordan canonical form. *SIAM Rev.*, 18(4):578–619, 1976.
- [507] Gene H. Golub. Bounds for the round-off errors in the Richardson second order method. *BIT*, 2:212–223, 1962.
- [508] Gene H. Golub and Charles F. Van Loan. Unsymmetric positive definite linear systems. *Linear Algebra Appl.*, 28:85–97, 1979.
- [509] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Third edition, Johns Hopkins University Press, Baltimore, MD, USA, 1996. xxvii+694 pp. ISBN 0-8018-5413-X (hardback), 0-8018-5414-8 (paperback).
- [510] R. Goodman and A. Feldstein. Round-off error in products. *Computing*, 15: 263–273, 1975.
- [511] R. H. Goodman, A. Feldstein, and J. Bustoz. Relative error in floating-point multiplication. *Computing*, 35:127–139, 1985.
- [512] James H. Goodnight. A tutorial on the SWEEP operator. *The American Statistician*, 33(3):149–158, 1979.
- [513] N. I. M. Gould. On growth in Gaussian elimination with complete pivoting. *SIAM J. Matrix Anal. Appl.*, 12(2):354–361, 1991.
- [514] W. Govaerts and J. D. Pryce. Block elimination with one iterative refinement solves bordered linear systems accurately. *BIT*, 30:490–507, 1990.

- [515] Willy J. F. Govaerts. *Numerical Methods for Bifurcations of Dynamical Equilibria*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2000. xxii+362 pp. ISBN 0-89871-442-7.
- [516] W. B. Gragg and G. W. Stewart. A stable variant of the secant method for solving nonlinear equations. *SIAM J. Numer. Anal.*, 13(6):889–903, 1976.
- [517] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Second edition, Addison-Wesley, Reading, MA, USA, 1994. xiii+657 pp. ISBN 0-201-55802-5.
- [518] Andrew Granville. Zaphod Beeblebrox's brain and the fifty-ninth row of Pascal's triangle. *Amer. Math. Monthly*, 99:318–331, 1992.
- [519] Anne Greenbaum. Behavior of slightly perturbed Lanczos and conjugate-gradient recurrences. *Linear Algebra Appl.*, 113:7–63, 1989.
- [520] Anne Greenbaum. The Lanczos and conjugate gradient algorithms in finite precision arithmetic. In *Proceedings of the Cornelius Lanczos International Centenary Conference*, J. David Brown, Moody T. Chu, Donald C. Ellison, and Robert J. Plemmons, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994, pages 49–60.
- [521] Anne Greenbaum. Estimating the attainable accuracy of recursively computed residual methods. *SIAM J. Matrix Anal. Appl.*, 18(3):535–551, 1997.
- [522] Anne Greenbaum. *Iterative Methods for Solving Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. xiii+220 pp. ISBN 0-89871-396-X.
- [523] Anne Greenbaum and Zdeněk Strakoš. Predicting the behavior of finite precision Lanczos and conjugate gradient computations. *SIAM J. Matrix Anal. Appl.*, 13(1):121–137, 1992.
- [524] Robert T. Gregory and David L. Karney. *A Collection of Matrices for Testing Computational Algorithms*. Wiley, New York, 1969. ix+154 pp. Reprinted with corrections by Robert E. Krieger, Huntington, New York, 1978. ISBN 0-88275-649-4.
- [525] Andreas Griewank. On solving nonlinear equations with simple singularities or nearly singular solutions. *SIAM Rev.*, 27(4):537–563, 1985.
- [526] Roger G. Grimes and John G. Lewis. Condition number estimation for sparse matrices. *SIAM J. Sci. Statist. Comput.*, 2(4):384–388, 1981.
- [527] Ming Gu. Backward perturbation bounds for linear least squares problems. *SIAM J. Matrix Anal. Appl.*, 20(2):363–372, 1998.
- [528] Ming Gu, James W. Demmel, and Inderjit Dhillon. Efficient computation of the singular value decomposition with applications to least squares problems. Technical Report CS-94-257, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, October 1994. 19 pp. LAPACK Working Note 88.
- [529] Thorkell Gudmundsson, Charles S. Kenney, and Alan J. Laub. Small-sample statistical estimates for matrix norms. *SIAM J. Matrix Anal. Appl.*, 16(3):776–792, 1995.
- [530] Heinrich W. Guggenheimer, Alan S. Edelman, and Charles R. Johnson. A simple estimate of the condition number of a linear system. *College Mathematics Journal*, 26(1):2–5, 1995.
- [531] Mårten Gulliksson. Iterative refinement for constrained and weighted linear least squares. *BIT*, 34:239–253, 1994.

- [532] Mårten Gulliksson. Backward error analysis for the constrained and weighted linear least squares problem when using the weighted QR factorization. *SIAM J. Matrix Anal. Appl.*, 16(2):675–687, 1995.
- [533] Mårten Gulliksson and Per-Åke Wedin. Modifying the QR -decomposition to constrained and weighted linear least squares. *SIAM J. Matrix Anal. Appl.*, 13(4): 1298–1313, 1992.
- [534] John L. Gustafson and Srinivas Aluru. Massively parallel searching for better algorithms or, how to do a cross product with five multiplications. *Scientific Programming*, pages 203–213, 1996.
- [535] F. G. Gustavson. Recursion leads to automatic variable blocking for dense linear-algebra algorithms. *IBM Journal of Research and Development*, 41(6):737–755, 1997.
- [536] William W. Hager. Condition estimates. *SIAM J. Sci. Statist. Comput.*, 5(2): 311–316, 1984.
- [537] E. Hairer and G. Wanner. *Solving Ordinary Differential Equations II*. Springer-Verlag, Berlin, 1991. xv+601 pp. ISBN 3-540-53775-9.
- [538] Marshall Hall, Jr. *Combinatorial Theory*. Blaisdell, Waltham, MA, USA, 1967. x+310 pp.
- [539] Hozumi Hamada. A new real number representation and its operation. In *Proceedings of the Eighth Symposium on Computer Arithmetic, Como, Italy*, Mary Jane Irwin and Renato Stefanelli, editors, IEEE Computer Society, Washington, D.C., 1987, pages 153–157.
- [540] S. J. Hammarling. Numerical solution of the stable, non-negative definite Lyapunov equation. *IMA J. Numer. Anal.*, 2:303–323, 1982.
- [541] S. J. Hammarling and J. H. Wilkinson. The practical behaviour of linear iterative methods with particular reference to S.O.R. Report NAC 69, National Physical Laboratory, Teddington, UK, September 1976. 19 pp.
- [542] Sven Hammarling. A note on modifications to the Givens plane rotation. *J. Inst. Maths. Applics.*, 13:215–218, 1974.
- [543] Sven J. Hammarling. The numerical solution of the general Gauss–Markov linear model. In *Mathematics in Signal Processing*, T. S. Durrani, J. B. Abbiss, and J. E. Hudson, editors, Oxford University Press, 1987, pages 451–456.
- [544] Rolf Hammer, Matthias Hocks, Ulrich Kulisch, and Dietmar Ratz. *Numerical Toolbox for Verified Computing I. Basic Numerical Problems: Theory, Algorithms, and Pascal-XSC Programs*. Springer-Verlag, Berlin, 1993. xiii+337 pp. ISBN 3-540-57118-3.
- [545] R. W. Hamming. *Numerical Methods for Scientists and Engineers*. Second edition, McGraw-Hill, New York, 1973. ix+721 pp. ISBN 0-07-025887-2.
- [546] Richard J. Hanson. Aasen’s method for linear systems with self-adjoint matrices. Visual Numerics, Inc., <http://www.vni.com/books/whitepapers/Aasen.html>, July 1997.
- [547] G. H. Hardy. *A Course of Pure Mathematics*. Tenth edition, Cambridge University Press, 1967. xii+509 pp. ISBN 0-521-09227-2.
- [548] G. H. Hardy, J. E. Littlewood, and G. Pólya. *Inequalities*. Second edition, Cambridge University Press, 1952. xii+324 pp.

- [549] Richard Harter. The optimality of Winograd's formula. *Comm. ACM*, 15(5):352, 1972.
- [550] D. J. Hartfiel. Concerning the solution set of $Ax = b$ where $P \leq A \leq Q$ and $p \leq b \leq q$. *Numer. Math.*, 35:355–359, 1980.
- [551] *A Manual of Operation for the Automatic Sequence Controlled Calculator*. Harvard University Press, Cambridge, MA, USA, 1946. Reprinted, with new foreword and introduction, Volume 8 in the Charles Babbage Institute Reprint Series for the History of Computing, MIT Press, Cambridge, MA, USA, 1985. xxxii+561 pp. ISBN 0-262-01084-4.
- [552] *Proceedings of a Symposium on Large-Scale Digital Calculating Machinery*, volume 16 of *The Annals of the Computation Laboratory of Harvard University*. Harvard University Press, Cambridge, MA, USA, 1948. Reprinted, with a new introduction by William Aspray, Volume 7 in the Charles Babbage Institute Reprint Series for the History of Computing, MIT Press, Cambridge, MA, USA, 1985. xxix+302 pp. ISBN 0-262-08152-0.
- [553] John R. Hauser. Handling floating-point exceptions in numeric programs. *ACM Trans. Program. Lang. Syst.*, 18(2):139–174, 1996.
- [554] John Z. Hearon. Nonsingular solutions of $TA - BT = C$. *Linear Algebra Appl.*, 16:57–63, 1977.
- [555] M. T. Heath, G. A. Geist, and J. B. Drake. Early experience with the Intel iPSC/860 at Oak Ridge National Laboratory. Report ORNL/TM-11655, Oak Ridge National Laboratory, Oak Ridge, TN, USA, September 1990. 26 pp.
- [556] Michael T. Heath. Numerical methods for large sparse linear least squares problems. *SIAM J. Sci. Statist. Comput.*, 5(3):497–513, 1984.
- [557] A. S. Hedayat, N. J. A. Sloane, and John Stufken. *Orthogonal Arrays: Theory and Applications*. Springer-Verlag, New York, 1999. xxiii+416 pp. ISBN 0-387-98766-5.
- [558] Piet Hein. *Grooks*. Number 85 in Borgens Pocketbooks. Second edition, Borgens Forlag, Copenhagen, Denmark, 1992. 53 pp. First published in 1966. ISBN 87-418-1079-1.
- [559] H. V. Henderson and S. R. Searle. On deriving the inverse of a sum of matrices. *SIAM Rev.*, 23(1):53–60, 1981.
- [560] Harold V. Henderson, Friedrich Pukelsheim, and Shayle R. Searle. On the history of the Kronecker product. *Linear and Multilinear Algebra*, 14:113–120, 1983.
- [561] Harold V. Henderson and S. R. Searle. The vec-permutation matrix, the vec operator and Kronecker products: A review. *Linear and Multilinear Algebra*, 9: 271–288, 1981.
- [562] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Second edition, Morgan Kaufmann, San Francisco, CA, USA, 1996. xxiii+760+appendices pp. ISBN 1-55860-329-8.
- [563] Peter Henrici. Bounds for iterates, inverses, spectral variation and fields of values of non-normal matrices. *Numer. Math.*, 4:24–40, 1962.
- [564] Peter Henrici. *Discrete Variable Methods in Ordinary Differential Equations*. John Wiley, New York, 1962. xi+407 pp.
- [565] Peter Henrici. *Error Propagation for Difference Methods*. John Wiley, New York, 1963. vi+73 pp.

- [566] Peter Henrici. *Elements of Numerical Analysis*. Wiley, New York, 1964. xv+328 pp.
- [567] Peter Henrici. Test of probabilistic models for the propagation of roundoff errors. *Comm. ACM*, 9(6):409–410, 1966.
- [568] Peter Henrici. A model for the propagation of rounding error in floating arithmetic. In *Interval Mathematics 1980*, Karl L. E. Nickel, editor, Academic Press, New York, 1980, pages 49–73.
- [569] Gary Hewer and Charles Kenney. The sensitivity of the stable Lyapunov equation. *SIAM J. Control Optim.*, 26(2):321–344, 1988.
- [570] *HP-15C Advanced Functions Handbook*. Hewlett-Packard, Portable Computer Division, Corvallis, OR, USA, 1982. 221 pp. Part number 00015-90011 Rev. C.
- [571] *HP 48G Series User's Guide*. Hewlett-Packard, Corvallis Division, Corvallis, OR, USA, 1993. Part number 00048-90126, Edition 3.
- [572] Yozo Hida, Xiaoye S. Li, and David H. Bailey. Algorithms for quad-double precision floating point arithmetic. In *Proc. 15th IEEE Symposium on Computer Arithmetic*, IEEE Computer Society Press, Los Alamitos, CA, USA, 2001, pages 155–162.
- [573] Desmond J. Higham. Condition numbers and their condition numbers. *Linear Algebra Appl.*, 214:193–213, 1995.
- [574] Desmond J. Higham and Nicholas J. Higham. Backward error and condition of structured linear systems. *SIAM J. Matrix Anal. Appl.*, 13(1):162–175, 1992.
- [575] Desmond J. Higham and Nicholas J. Higham. Componentwise perturbation theory for linear systems with multiple right-hand sides. *Linear Algebra Appl.*, 174:111–129, 1992.
- [576] Desmond J. Higham and Nicholas J. Higham. *MATLAB Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2000. xxii+283 pp. ISBN 0-89871-516-4.
- [577] Desmond J. Higham and Lloyd N. Trefethen. Stiffness of ODEs. *BIT*, 33:285–303, 1993.
- [578] Nicholas J. Higham. Computing the polar decomposition—with applications. *SIAM J. Sci. Statist. Comput.*, 7(4):1160–1174, 1986.
- [579] Nicholas J. Higham. Efficient algorithms for computing the condition number of a tridiagonal matrix. *SIAM J. Sci. Statist. Comput.*, 7(1):150–165, 1986.
- [580] Nicholas J. Higham. Computing real square roots of a real matrix. *Linear Algebra Appl.*, 88/89:405–430, 1987.
- [581] Nicholas J. Higham. Error analysis of the Björck-Pereyra algorithms for solving Vandermonde systems. *Numer. Math.*, 50(5):613–632, 1987.
- [582] Nicholas J. Higham. A survey of condition number estimation for triangular matrices. *SIAM Rev.*, 29(4):575–596, 1987.
- [583] Nicholas J. Higham. Computing a nearest symmetric positive semidefinite matrix. *Linear Algebra Appl.*, 103:103–118, 1988.
- [584] Nicholas J. Higham. Fast solution of Vandermonde-like systems involving orthogonal polynomials. *IMA J. Numer. Anal.*, 8:473–486, 1988.
- [585] Nicholas J. Higham. FORTRAN codes for estimating the one-norm of a real or complex matrix, with applications to condition estimation (Algorithm 674). *ACM Trans. Math. Software*, 14(4):381–396, 1988.

- [586] Nicholas J. Higham. The accuracy of solutions to triangular systems. *SIAM J. Numer. Anal.*, 26(5):1252–1265, 1989.
- [587] Nicholas J. Higham. Matrix nearness problems and applications. In *Applications of Matrix Theory*, M. J. C. Gover and S. Barnett, editors, Oxford University Press, 1989, pages 1–27.
- [588] Nicholas J. Higham. Analysis of the Cholesky decomposition of a semi-definite matrix. In *Reliable Numerical Computation*, M. G. Cox and S. J. Hammarling, editors, Oxford University Press, 1990, pages 161–185.
- [589] Nicholas J. Higham. Bounding the error in Gaussian elimination for tridiagonal systems. *SIAM J. Matrix Anal. Appl.*, 11(4):521–530, 1990.
- [590] Nicholas J. Higham. Computing error bounds for regression problems. In *Statistical Analysis of Measurement Error Models and Applications, Contemporary Mathematics 112*, Philip J. Brown and Wayne A. Fuller, editors, American Mathematical Society, Providence, RI, USA, 1990, pages 195–208.
- [591] Nicholas J. Higham. Experience with a matrix norm estimator. *SIAM J. Sci. Statist. Comput.*, 11(4):804–809, 1990.
- [592] Nicholas J. Higham. Exploiting fast matrix multiplication within the level 3 BLAS. *ACM Trans. Math. Software*, 16(4):352–368, 1990.
- [593] Nicholas J. Higham. How accurate is Gaussian elimination? In *Numerical Analysis 1989, Proceedings of the 13th Dundee Conference*, D. F. Griffiths and G. A. Watson, editors, volume 228 of *Pitman Research Notes in Mathematics*, Longman Scientific and Technical, Essex, UK, 1990, pages 137–154.
- [594] Nicholas J. Higham. Iterative refinement enhances the stability of QR factorization methods for solving linear equations. Numerical Analysis Report No. 182, University of Manchester, Manchester, England, April 1990.
- [595] Nicholas J. Higham. Stability analysis of algorithms for solving confluent Vandermonde-like systems. *SIAM J. Matrix Anal. Appl.*, 11(1):23–41, 1990.
- [596] Nicholas J. Higham. Iterative refinement enhances the stability of QR factorization methods for solving linear equations. *BIT*, 31:447–468, 1991.
- [597] Nicholas J. Higham. Three measures of precision in floating point arithmetic. *NA Digest*, Volume 91, Issue 16, 1991. Electronic mail magazine: na.help@na-net.ornl.gov.
- [598] Nicholas J. Higham. Estimating the matrix p -norm. *Numer. Math.*, 62:539–555, 1992.
- [599] Nicholas J. Higham. Stability of a method for multiplying complex matrices with three real matrix multiplications. *SIAM J. Matrix Anal. Appl.*, 13(3):681–687, 1992.
- [600] Nicholas J. Higham. The accuracy of floating point summation. *SIAM J. Sci. Comput.*, 14(4):783–799, 1993.
- [601] Nicholas J. Higham. Optimization by direct search in matrix computations. *SIAM J. Matrix Anal. Appl.*, 14(2):317–333, 1993.
- [602] Nicholas J. Higham. Perturbation theory and backward error for $AX - XB = C$. *BIT*, 33:124–136, 1993.
- [603] Nicholas J. Higham. The matrix sign decomposition and its relation to the polar decomposition. *Linear Algebra Appl.*, 212/213:3–20, 1994.

- [604] Nicholas J. Higham. A survey of componentwise perturbation theory in numerical linear algebra. In *Mathematics of Computation 1943–1993: A Half Century of Computational Mathematics*, Walter Gautschi, editor, volume 48 of *Proceedings of Symposia in Applied Mathematics*, American Mathematical Society, Providence, RI, USA, 1994, pages 49–77.
- [605] Nicholas J. Higham. Stability of parallel triangular system solvers. *SIAM J. Sci. Comput.*, 16(2):400–413, 1995.
- [606] Nicholas J. Higham. The Test Matrix Toolbox for MATLAB (version 3.0). Numerical Analysis Report No. 276, Manchester Centre for Computational Mathematics, Manchester, England, September 1995. 70 pp.
- [607] Nicholas J. Higham. Iterative refinement for linear systems and LAPACK. *IMA J. Numer. Anal.*, 17(4):495–509, 1997.
- [608] Nicholas J. Higham. Stability of the diagonal pivoting method with partial pivoting. *SIAM J. Matrix Anal. Appl.*, 18(1):52–65, 1997.
- [609] Nicholas J. Higham. Stable iterations for the matrix square root. *Numerical Algorithms*, 15(2):227–242, 1997.
- [610] Nicholas J. Higham. Factorizing complex symmetric matrices with positive definite real and imaginary parts. *Math. Comp.*, 67(224):1591–1599, 1998.
- [611] Nicholas J. Higham. *Handbook of Writing for the Mathematical Sciences*. Second edition, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1998. xvi+302 pp. ISBN 0-89871-420-6.
- [612] Nicholas J. Higham. Notes on accuracy and stability of algorithms in numerical linear algebra. In *The Graduate Student's Guide to Numerical Analysis '98*, Mark Ainsworth, Jeremy Levesley, and Marco Marletta, editors, Springer-Verlag, Berlin, 1999, pages 48–82.
- [613] Nicholas J. Higham. Stability of block LDL^T factorization of a symmetric tridiagonal matrix. *Linear Algebra Appl.*, 287:181–189, 1999.
- [614] Nicholas J. Higham. QR factorization with complete pivoting and accurate computation of the SVD. *Linear Algebra Appl.*, 309:153–174, 2000.
- [615] Nicholas J. Higham. Evaluating Padé approximants of the matrix logarithm. *SIAM J. Matrix Anal. Appl.*, 22(4):1126–1135, 2001.
- [616] Nicholas J. Higham and Desmond J. Higham. Large growth factors in Gaussian elimination with pivoting. *SIAM J. Matrix Anal. Appl.*, 10(2):155–164, 1989.
- [617] Nicholas J. Higham and Hyun-Min Kim. Numerical analysis of a quadratic matrix equation. *IMA J. Numer. Anal.*, 20(4):499–519, 2000.
- [618] Nicholas J. Higham and Philip A. Knight. Componentwise error analysis for stationary iterative methods. In *Linear Algebra, Markov Chains, and Queueing Models*, Carl D. Meyer and Robert J. Plemmons, editors, volume 48 of *IMA Volumes in Mathematics and Its Applications*, Springer-Verlag, New York, 1993, pages 29–46.
- [619] Nicholas J. Higham and Philip A. Knight. Finite precision behavior of stationary iteration for solving singular systems. *Linear Algebra Appl.*, 192:165–186, 1993.
- [620] Nicholas J. Higham and Philip A. Knight. Matrix powers in finite precision arithmetic. *SIAM J. Matrix Anal. Appl.*, 16(2):343–358, 1995.
- [621] Nicholas J. Higham and Pythagoras Papadimitriou. A parallel algorithm for computing the polar decomposition. *Parallel Comput.*, 20(8):1161–1173, 1994.

- [622] Nicholas J. Higham and Alex Pothen. Stability of the partitioned inverse method for parallel solution of sparse triangular systems. *SIAM J. Sci. Comput.*, 15(1): 139–148, 1994.
- [623] Nicholas J. Higham and G. W. Stewart. Numerical linear algebra in statistical computing. In *The State of the Art in Numerical Analysis*, A. Iserles and M. J. D. Powell, editors, Oxford University Press, 1987, pages 41–57.
- [624] Nicholas J. Higham and Françoise Tisseur. A block algorithm for matrix 1-norm estimation, with an application to 1-norm pseudospectra. *SIAM J. Matrix Anal. Appl.*, 21(4):1185–1201, 2000.
- [625] Nicholas J. Higham and Françoise Tisseur. Bounds for eigenvalues of matrix polynomials. Numerical Analysis Report 371, Manchester Centre for Computational Mathematics, Manchester, England, January 2001. 16 pp. To appear in *Linear Algebra Appl.*
- [626] David Hilbert. Ein Beitrag zur Theorie des Legendre'schen Polynoms. *Acta Mathematica*, 18:155–159, 1894.
- [627] F. B. Hildebrand. *Introduction to Numerical Analysis*. Second edition, McGraw-Hill, New York, 1974. xiii+669 pp. Reprinted by Dover, New York, 1987. ISBN 0-486-65363-3.
- [628] Marlis Hochbruck and Gerhard Starke. Preconditioned Krylov subspace methods for Lyapunov matrix equations. *SIAM J. Matrix Anal. Appl.*, 16(1):156–171, 1995.
- [629] R. W. Hockney and C. R. Jesshope. *Parallel Computers 2: Architecture, Programming and Algorithms*. Adam Hilger, Bristol, 1988. xv+625 pp. ISBN 0-85274-812-4.
- [630] A. Scottedward Hodel. Recent applications of the Lyapunov equation in control theory. In *Iterative Methods in Linear Algebra*, R. Beauwens and P. de Groen, editors, Elsevier Science Publishers B.V. (North-Holland), Amsterdam, The Netherlands, 1992, pages 217–227.
- [631] Andrew Hodges. *Alan Turing: The Enigma*. Burnett Books, London, 1983. xix+586 pp. 1992 edition with preface, Vintage, London. ISBN 0-09-911641-3.
- [632] Christoph M. Hoffmann. The problems of accuracy and robustness in geometric computation. *Computer*, March:31–41, 1989.
- [633] W. Hoffmann. Solving linear systems on a vector computer. *J. Comput. Appl. Math.*, 18:353–367, 1987.
- [634] W. Hoffmann. Iterative algorithms for Gram-Schmidt orthogonalization. *Computing*, 41:335–348, 1989.
- [635] R. C. Holt and J. R. Cordy. The Turing programming language. *Comm. ACM*, 31(12):1410–1423, 1988.
- [636] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 1985. xiii+561 pp. ISBN 0-521-30586-1.
- [637] Roger A. Horn and Charles R. Johnson. *Topics in Matrix Analysis*. Cambridge University Press, 1991. viii+607 pp. ISBN 0-521-30587-X.
- [638] Jim Horning. Note on program reliability. *ACM SIGSOFT Software Engineering Notes*, 4(4):6, 1979. Cited in [1159].
- [639] Harold Hotelling. Some new methods in matrix calculation. *Ann. Math. Statist.*, 14(1):1–34, 1943.

- [640] David Hough. Applications of the proposed IEEE 754 standard for floating-point arithmetic. *Computer*, 14:70–74, 1981.
- [641] David Hough. Random story. *NA Digest*, Volume 89, Issue 1, 1989.
- [642] Patricia D. Hough and Stephen A. Vavasis. Complete orthogonal decomposition for weighted least squares. *SIAM J. Matrix Anal. Appl.*, 18(2):369–392, 1997.
- [643] Alston S. Householder. Unitary triangularization of a nonsymmetric matrix. *J. Assoc. Comput. Mach.*, 5:339–342, 1958.
- [644] Alston S. Householder. *The Theory of Matrices in Numerical Analysis*. Blaisdell, New York, 1964. xi+257 pp. Reprinted by Dover, New York, 1975. ISBN 0-486-61781-5.
- [645] D. Y. Hu and L. Reichel. Krylov subspace methods for the Sylvester equation. *Linear Algebra Appl.*, 172:283–313, 1992.
- [646] T. E. Hull. Correctness of numerical software. In *Performance Evaluation of Numerical Software*, Lloyd D. Fosdick, editor, North-Holland, Amsterdam, The Netherlands, 1979, pages 3–15.
- [647] T. E. Hull. Precision control, exception handling and a choice of numerical algorithms. In *Numerical Analysis Proceedings, Dundee 1981*, G. A. Watson, editor, volume 912 of *Lecture Notes in Mathematics*, Springer-Verlag, Berlin, 1982, pages 169–178.
- [648] T. E. Hull, A. Abraham, M. S. Cohen, A. F. X. Curley, C. B. Hall, D. A. Penny, and J. T. M. Sawchuk. Numerical TURING. *ACM SIGNUM Newsletter*, 20(3): 26–34, 1985.
- [649] T. E. Hull, Thomas F. Fairgrieve, and Ping Tak Peter Tang. Implementing complex elementary functions using exception handling. *ACM Trans. Math. Software*, 20(2):215–244, 1994.
- [650] T. E. Hull and J. R. Swenson. Tests of probabilistic models for propagation of roundoff errors. *Comm. ACM*, 9(2):108–113, 1966.
- [651] Julian Hunt. Rounding and other approximations for measurements, records and targets. *Mathematics Today*, 33:73–77, 1997.
- [652] M. A. Hyman. Eigenvalues and eigenvectors of general matrices. Presented at the 12th National Meeting of the Association for Computing Machinery, Houston, Texas, 1957. Cited in [1232].
- [653] IBM. *Engineering and Scientific Subroutine Library, Guide and Reference, Release 3*. Fourth Edition (Program Number 5668-863), 1988.
- [654] ISO/IEC 10967-1 (1994-12): *Information Technology—Language Independent Arithmetic—Part 1: Integer and Floating Point Arithmetic*. International Electrotechnical Commission, Geneva, Switzerland, 1994. 92 pp.
- [655] *IEEE Standard for Binary Floating-Point Arithmetic, ANSI/IEEE Standard 754-1985*. Institute of Electrical and Electronics Engineers, New York, 1985. Reprinted in SIGPLAN Notices, 22(2):9–25, 1987.
- [656] *A Radix-Independent Standard for Floating-Point Arithmetic, IEEE Standard 854-1987*. IEEE Computer Society, New York, 1987.
- [657] IEEE Computer Society Microprocessor Standards Committee, Floating-Point Working Group. A proposed standard for binary floating-point arithmetic, Draft 8.0 of IEEE Task P754 (with introductory comments by David Stevenson). *Computer*, 14:51–62, 1981.

- [658] Yasuhiko Ikebe. On inverses of Hessenberg matrices. *Linear Algebra Appl.*, 24: 93–97, 1979.
- [659] Khakim D. Ikramov. A remark on “A note on constructing a symmetric matrix with specified diagonal entries and eigenvalues”. *BIT*, 38(4):807, 1998.
- [660] Khakim D. Ikramov and Andrey B. Kucherov. Bounding the growth factor in Gaussian elimination for Buckley’s class of complex symmetric matrices. *Numerical Linear Algebra with Applications*, 7(5):269–274, 2000.
- [661] ILAS Education Committee. Report on graduate linear algebra courses. Manuscript from the International Linear Algebra Society, November 1993. <http://gauss.technion.ac.il/iic/GRAD-ED.SYLLABI>. 4 pp.
- [662] Cray Research Inc. *UNICOS Math and Scientific Library Reference Manual*. Number SR-2081, Version 5.0, Eagan, MN, USA, 1989.
- [663] D. C. Ince, editor. *Collected Works of A. M. Turing: Mechanical Intelligence*. North-Holland, Amsterdam, The Netherlands, 1992. xix+227 pp. ISBN 0-444-88058-5.
- [664] F. Incertis. A faster method of computing matrix Pythagorean sums. *IEEE Trans. Automat. Control*, AC-30(3):273–275, 1985.
- [665] Ilse C. F. Ipsen and Carl D. Meyer. Uniform stability of Markov chains. *SIAM J. Matrix Anal. Appl.*, 15(4):1061–1074, 1994.
- [666] Masao Iri. History of automatic differentiation and rounding error estimation. In *Automatic Differentiation of Algorithms: Theory, Implementation, and Application*, Andreas Griewank and George F. Corliss, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1991, pages 3–16.
- [667] Eugene Isaacson and Herbert Bishop Keller. *Analysis of Numerical Methods*. Wiley, New York, 1966. xv+541 pp. Reprinted by Dover, New York, 1994. ISBN 0-486-68029-0.
- [668] William Jalby and Bernard Philippe. Stability analysis and improvement of the block Gram–Schmidt algorithm. *SIAM J. Sci. Statist. Comput.*, 12(5):1058–1073, 1991.
- [669] Ho-Jong Jang. A constrained least-squares approach to the rapid reanalysis of structures. *Linear Algebra Appl.*, 265:185–202, 1997.
- [670] M. Jankowski, A. Smoktunowicz, and H. Woźniakowski. A note on floating-point summation of very many terms. *J. Information Processing and Cybernetics-EIK*, 19(9):435–440, 1983.
- [671] M. Jankowski and H. Woźniakowski. Iterative refinement implies numerical stability. *BIT*, 17:303–311, 1977.
- [672] M. Jankowski and H. Woźniakowski. The accurate solution of certain continuous problems using only single precision arithmetic. *BIT*, 25:635–651, 1985.
- [673] Paul Jansen and Peter Weidner. High-accuracy arithmetic software—some tests of the ACRITH problem-solving routines. *ACM Trans. Math. Software*, 12(1): 62–70, 1986.
- [674] A. Jennings. Bounds for the singular values of a matrix. *IMA J. Numer. Anal.*, 2:459–474, 1982.
- [675] L. S. Jennings and M. R. Osborne. A direct error analysis for least squares. *Numer. Math.*, 22:325–332, 1974.

- [676] Mark T. Jones and Merrell L. Patrick. Bunch–Kaufman factorization for real symmetric indefinite banded matrices. *SIAM J. Matrix Anal. Appl.*, 14(2):553–559, 1993.
- [677] Mark T. Jones and Merrell L. Patrick. Factoring symmetric indefinite matrices on high-performance architectures. *SIAM J. Matrix Anal. Appl.*, 15(1):273–283, 1994.
- [678] William B. Jones and W. J. Thron. Numerical stability in evaluating continued fractions. *Math. Comp.*, 28(127):795–810, 1974.
- [679] T. L. Jordan. Experiments on error growth associated with some linear least-squares procedures. *Math. Comp.*, 22:579–588, 1968.
- [680] George Gheverghese Joseph. *The Crest of the Peacock: Non-European Roots of Mathematics*. Penguin, London, 1991. xv+371 pp. ISBN 0-14-012529-9.
- [681] Bo Kågström. A perturbation analysis of the generalized Sylvester equation $(AR - LB, DR - LE) = (C, F)$. *SIAM J. Matrix Anal. Appl.*, 15(4):1045–1060, 1994.
- [682] Bo Kågström and Peter Poromaa. Distributed and shared memory block algorithms for the triangular Sylvester equation with sep^{-1} estimators. *SIAM J. Matrix Anal. Appl.*, 13(1):90–101, 1992.
- [683] Bo Kågström and Peter Poromaa. Computing eigenspaces with specified eigenvalues of a regular matrix pair (A, B) and condition estimation: Theory, algorithms and software. *Numerical Algorithms*, 12:369–407, 1996.
- [684] Bo Kågström and Peter Poromaa. LAPACK-style algorithms and software for solving the generalized Sylvester equation and estimating the separation between regular matrix pairs. *ACM Trans. Math. Software*, 22(1):78–103, 1996.
- [685] Bo Kågström and Lars Westin. Generalized Schur methods with condition estimators for solving the generalized Sylvester equation. *IEEE Trans. Automat. Control*, AC-34(7):745–751, 1989.
- [686] W. Kahan. Further remarks on reducing truncation errors. *Comm. ACM*, 8(1):40, 1965.
- [687] W. Kahan. Numerical linear algebra. *Canadian Math. Bulletin*, 9:757–801, 1966.
- [688] W. Kahan. A survey of error analysis. In *Proc. IFIP Congress, Ljubljana*, Information Processing 71, North-Holland, Amsterdam, The Netherlands, 1972, pages 1214–1239.
- [689] W. Kahan. Implementation of algorithms (lecture notes by W. S. Haugeland and D. Hough). Technical Report 20, Department of Computer Science, University of California, Berkeley, CA, USA, 1973.
- [690] W. Kahan. Interval arithmetic options in the proposed IEEE floating point arithmetic standard. In *Interval Mathematics 1980*, Karl L. E. Nickel, editor, Academic Press, New York, 1980, pages 99–128.
- [691] W. Kahan. Why do we need a floating-point arithmetic standard? Technical report, University of California, Berkeley, CA, USA, February 1981. 41 pp.
- [692] W. Kahan. Mathematics written in sand—the HP-15C, Intel 8087, etc. In *Statistical Computing Section, Proceedings of the American Statistical Association, Toronto*, 1983, pages 12–26.
- [693] W. Kahan. To solve a real cubic equation. Technical Report PAM-352, Center for Pure and Applied Mathematics, University of California, Berkeley, CA, USA, November 1986. 20 pp.

- [694] W. Kahan. Branch cuts for complex elementary functions or much ado about nothing's sign bit. In *The State of the Art in Numerical Analysis*, A. Iserles and M. J. D. Powell, editors, Oxford University Press, 1987, pages 165–211.
- [695] W. Kahan. Doubled-precision IEEE standard 754 floating-point arithmetic. Manuscript, February 1987.
- [696] W. Kahan. How Cray's arithmetic hurts scientific computation (and what might be done about it). Manuscript prepared for the Cray User Group meeting in Toronto, June 1990. 42 pp.
- [697] W. Kahan. Analysis and refutation of the LCAS. *ACM SIGNUM Newsletter*, 26 (3):2–15, 1991.
- [698] W. Kahan. Computer benchmarks versus accuracy. Draft manuscript, June 1994.
- [699] W. Kahan. The improbability of probabilistic error analyses for numerical computations. Manuscript, March 1996. 34 pp.
- [700] W. Kahan. Lecture notes on the status of IEEE Standard 754 for binary floating-point arithmetic. Manuscript, May 1996. 30 pp.
- [701] W. Kahan. Miscalculating area and angles of a needle-like triangle. Manuscript, July 1997. 20 pp.
- [702] W. Kahan. What has the volume of a tetrahedron to do with computer programming languages? Manuscript, April 2001. 31 pp.
- [703] W. Kahan and I. Farkas. Algorithm 168: Newton interpolation with backward divided differences. *Comm. ACM*, 6(4):165, 1963.
- [704] W. Kahan and I. Farkas. Algorithm 169: Newton interpolation with forward divided differences. *Comm. ACM*, 6(4):165, 1963.
- [705] W. Kahan and E. LeBlanc. Anomalies in the IBM ACRITH package. In *Proceedings of the 7th Symposium on Computer Arithmetic*, Kai Hwang, editor, IEEE Computer Society Press, Silver Spring, MD, USA, 1985, pages 322–331.
- [706] W. Kahan and J. Palmer. On a proposed floating-point standard. *ACM SIGNUM Newsletter*, 14:13–21, 1979.
- [707] David K. Kahaner, Cleve B. Moler, and Stephen G. Nash. *Numerical Methods and Software*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1989. xii+495 pp. ISBN 0-13-627258-4.
- [708] Ming-Yang Kao and Jie Wang. Linear-time approximation algorithms for computing numerical summation with provably small errors. *SIAM J. Comput.*, 29 (5):1568–1576, 2000.
- [709] Igor Kaporin. A practical algorithm for faster matrix multiplication. *Numerical Linear Algebra with Applications*, 6:687–700, 1999.
- [710] Ilkka Karasalo. A criterion for truncation of the *QR*-decomposition algorithm for the singular linear least squares problem. *BIT*, 14:156–166, 1974.
- [711] A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics—Doklady*, 7(7):595–596, 1963.
- [712] Samuel Karlin. *Total Positivity*, volume 1. Stanford University Press, Stanford, CA, USA, 1968.
- [713] Rune Karlson and Bertil Waldén. Estimation of optimal backward perturbation bounds for the linear least squares problem. *BIT*, 37(4):862–869, 1997.

- [714] Alan H. Karp and Peter Markstein. High-precision division and square root. *ACM Trans. Math. Software*, 23(4):561–589, 1997.
- [715] Richard Karpinski. Paranoia: A floating-point benchmark. *BYTE*, 10(2):223–235, 1985.
- [716] Tosio Kato. *Perturbation Theory for Linear Operators*. Second edition, Springer-Verlag, Berlin, 1976. xxi+619 pp. ISBN 3-540-97588-5.
- [717] Linda Kaufman. Matrix methods for queuing problems. *SIAM J. Sci. Statist. Comput.*, 4(3):525–552, 1983.
- [718] R. Baker Kearfott. Algorithm 763: INTERVAL_ARITHMETIC: A Fortran 90 module for an interval data type. *ACM Trans. Math. Software*, 22(4):385–392, 1996.
- [719] R. Baker Kearfott. Interval computations: Introduction, uses, and resources. *Euromath Bulletin*, 2(1):95–112, 1996. <http://interval.louisiana.edu/preprints.html>.
- [720] Herbert B. Keller. On the solution of singular and semidefinite linear systems by iteration. *SIAM J. Numer. Anal.*, 2(2):281–290, 1965.
- [721] C. T. Kelley. *Iterative Methods for Linear and Nonlinear Equations*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1995. xiii+165 pp. ISBN 0-89871-352-8.
- [722] C. T. Kelley. *Iterative Methods for Optimization*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1999. xv+180 pp. ISBN 0-89871-433-8.
- [723] William J. Kennedy, Jr. and James E. Gentle. *Statistical Computing*. Marcel Dekker, New York, 1980. xi+591 pp. ISBN 0-8247-6898-1.
- [724] Charles Kenney and Gary Hewer. The sensitivity of the algebraic and differential Riccati equations. *SIAM J. Control Optim.*, 28(1):50–69, 1990.
- [725] Charles Kenney and Alan J. Laub. Controllability and stability radii for companion form systems. *Math. Control Signals Systems*, 1:239–256, 1988.
- [726] Charles Kenney, Alan J. Laub, and M. S. Reese. Statistical condition estimation for linear least squares. *SIAM J. Matrix Anal. Appl.*, 19(4):906–923, 1998.
- [727] Charles Kenney, Alan J. Laub, and M. S. Reese. Statistical condition estimation for linear systems. *SIAM J. Sci. Comput.*, 19(2):566–583, 1998.
- [728] Charles S. Kenney and Alan J. Laub. Small-sample statistical condition estimates for general matrix functions. *SIAM J. Sci. Comput.*, 15(1):36–61, 1994.
- [729] Charles S. Kenney, Alan J. Laub, and Philip M. Papadopoulos. Matrix-sign algorithms for Riccati equations. *IMA J. Math. Control Inform.*, 9:331–344, 1992.
- [730] Thomas H. Kerr. Fallacies in computational testing of matrix positive definiteness/semidefiniteness. *IEEE Trans. Aerospace and Electronic Systems*, 26(2):415–421, 1990.
- [731] Andrzej Kielbasiński. Summation algorithm with corrections and some of its applications. *Math. Stos.*, 1:22–41, 1973. (In Polish, cited in [670] and [672].)
- [732] Andrzej Kielbasiński. Iterative refinement for linear systems in variable-precision arithmetic. *BIT*, 21:97–103, 1981.
- [733] Andrzej Kielbasiński. A note on rounding-error analysis of Cholesky factorization. *Linear Algebra Appl.*, 88/89:487–494, 1987.

- [734] Andrzej Kielbasiński and Hubert Schwetlick. *Numerische Lineare Algebra: Eine Computerorientierte Einführung*. VEB Deutscher, Berlin, 1988. 472 pp. ISBN 3-87144-999-7.
- [735] Andrzej Kielbasiński and Hubert Schwetlick. *Numeryczna Algebra Liniowa: Wprowadzenie do Obliczeń Zautomatyzowanych*. Wydawnictwa Naukowo-Techniczne, Warszawa, 1992. 502 pp. ISBN 83-204-1260-9.
- [736] Fuad Kittaneh. Singular values of companion matrices and bounds on zeros of polynomials. *SIAM J. Matrix Anal. Appl.*, 16(1):333–340, 1995.
- [737] R. Klatte, U. W. Kulisch, C. Lawo, M. Rauch, and A. Wiethoff. *C-XSC: A C++ Class Library for Extended Scientific Computing*. Springer-Verlag, Berlin, 1993. ISBN 0-387-56328-8.
- [738] R. Klatte, U. W. Kulisch, M. Neaga, D. Ratz, and Ch. Ullrich. *PASCAL-XSC—Language Reference with Examples*. Springer-Verlag, Berlin, 1992.
- [739] Philip A. Knight. *Error Analysis of Stationary Iteration and Associated Problems*. PhD thesis, University of Manchester, Manchester, England, September 1993. 135 pp.
- [740] Philip A. Knight. Fast rectangular matrix multiplication and *QR* decomposition. *Linear Algebra Appl.*, 221:69–81, 1995.
- [741] Donald E. Knuth. Evaluation of polynomials by computer. *Comm. ACM*, 5(12):595–599, 1962.
- [742] Donald E. Knuth. Two notes on notation. *Amer. Math. Monthly*, 99(5):403–422, 1992.
- [743] Donald E. Knuth. *The Art of Computer Programming, Volume 1, Fundamental Algorithms*. Third edition, Addison-Wesley, Reading, MA, USA, 1997. xix+650 pp. ISBN 0-201-89683-4.
- [744] Donald E. Knuth. *The Art of Computer Programming, Volume 2, Seminumerical Algorithms*. Third edition, Addison-Wesley, Reading, MA, USA, 1998. xiii+762 pp. ISBN 0-201-89684-2.
- [745] Donald E. Knuth. *Digital Typography*. CSLI Lecture Notes Number 78. Center for the Study of Language and Information, Stanford University, Stanford, CA, USA, 1999. xv+685 pp. ISBN 0-57586-010-4.
- [746] T. W. Körner. *Fourier Analysis*. Cambridge University Press, Cambridge, UK, 1988. xii+591 pp. ISBN 0-521-38991-7.
- [747] Przemysław Kosowski and Alicja Smoktunowicz. On constructing unit triangular matrices with prescribed singular values. *Computing*, 64:279–285, 2000.
- [748] Eric Kostlan. On the spectra of Gaussian matrices. *Linear Algebra Appl.*, 162–164:385–388, 1992.
- [749] Z. V. Kovarik. Compatibility of approximate solutions of inaccurate linear equations. *Linear Algebra Appl.*, 15:217–225, 1976.
- [750] Antoni Kreczmar. On memory requirements of Strassen’s algorithms. volume 45 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 1976, pages 404–407.
- [751] Shankar Krishnan, Mark Foskey, Tim Culver, John Keyser, and Dinesh Manocha. PRECISE: Efficient multiprecision evaluation of algebraic roots and predicates for reliable geometric computation. In *Proc. ACM Symposium on Computational Geometry*, David Eppstein, Dan Halperin, and Dinesh Manocha, editors, ACM Press, 2001, pages 274–283.

- [752] Koichi Kubota. PADRE2, a Fortran precompiler yielding error estimates and second derivatives. In *Automatic Differentiation of Algorithms: Theory, Implementation, and Application*, Andreas Griewank and George F. Corliss, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1991, pages 251–262.
- [753] J. Kuczyński and H. Woźniakowski. Estimating the largest eigenvalue by the power and Lanczos algorithms with a random start. *SIAM J. Matrix Anal. Appl.*, 13(4):1094–1122, 1992.
- [754] H. Kuki and W. J. Cody. A statistical study of the accuracy of floating point number systems. *Comm. ACM*, 16(4):223–230, 1973.
- [755] Ulrich W. Kulisch and Willard L. Miranker. *Computer Arithmetic in Theory and in Practice*. Academic Press, New York, 1981. xiii+249 pp. ISBN 0-12-428650-X.
- [756] Ulrich W. Kulisch and Willard L. Miranker, editors. *A New Approach to Scientific Computation*. Academic Press, New York, 1983. xv+384 pp. ISBN 0-12-428660-7.
- [757] Ulrich W. Kulisch and Willard L. Miranker. The arithmetic of the digital computer: A new approach. *SIAM Rev.*, 28(1):1–40, 1986.
- [758] I. B. Kuperman. *Approximate Linear Algebraic Equations*. Van Nostrand Reinhold, London, 1971. xi+225 pp. ISBN 0-442-04546-8.
- [759] M. La Porte and J. Vignes. Etude statistique des erreurs dans l'arithmétique des ordinateurs; application au contrôle des résultats d'algorithmes numériques. *Numer. Math.*, 23:63–72, 1974.
- [760] J. D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bull. Amer. Math. Soc.*, 82(1):126–128, 1976.
- [761] Julian Laderman, Victor Pan, and Xuan-He Sha. On practical algorithms for accelerated matrix multiplication. *Linear Algebra Appl.*, 162–164:557–588, 1992.
- [762] Jeffrey C. Lagarias, James A. Reeds, Margaret H. Wright, and Paul E. Wright. Convergence properties of the Nelder–Mead simplex method in low dimensions. *SIAM J. Optim.*, 9(1):112–147, 1998.
- [763] Peter Lancaster. Error analysis for the Newton-Raphson method. *Numer. Math.*, 9:55–68, 1966.
- [764] Peter Lancaster. Explicit solutions of linear matrix equations. *SIAM Rev.*, 12(4):544–566, 1970.
- [765] Cornelius Lanczos. *Applied Analysis*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1956. xx+539 pp. Reprinted by Dover, New York, 1988. ISBN 0-486-65656-X.
- [766] Philippe Langlois. Automatic linear correction of rounding errors. *BIT*, 41(3):515–539, 2001.
- [767] John L. Larson, Mary E. Pasternak, and John A. Wisniewski. Algorithm 594: Software for relative error analysis. *ACM Trans. Math. Software*, 9(1):125–130, 1983.
- [768] John L. Larson and Ahmed H. Sameh. Efficient calculation of the effects of roundoff errors. *ACM Trans. Math. Software*, 4(3):228–236, 1978. Errata 5(3):372, 1979.
- [769] John L. Larson and Ahmed H. Sameh. Algorithms for roundoff error analysis—A relative error approach. *Computing*, 24:275–297, 1980.
- [770] Lajos László. An attainable lower bound for the best normal approximation. *SIAM J. Matrix Anal. Appl.*, 15(3):1035–1043, 1994.

- [771] Alan J. Laub. A Schur method for solving algebraic Riccati equations. *IEEE Trans. Automat. Control*, AC-24(6):913–921, 1979.
- [772] Peter L  uchli. Jordan-Elimination und Ausgleichung nach kleinsten Quadraten. *Numer. Math.*, 3:226–240, 1961.
- [773] Simon Lavington. *Early British Computers: The Story of Vintage Computers and the People Who Built Them*. Manchester University Press, 1980. 139 pp. ISBN 0-7190-0803-4.
- [774] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. Basic linear algebra subprograms for Fortran usage. *ACM Trans. Math. Software*, 5(3):308–323, 1979.
- [775] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1995. xii+337 pp. Revised republication of work first published in 1974 by Prentice-Hall. ISBN 0-89871-356-0.
- [776] Lam Lay-Yong and Shen Kangshen. Methods of solving linear equations in traditional China. *Historia Mathematica*, 16(2):107–122, 1989.
- [777] Vincent Lef  vre and Jean-Michel Muller. Worst cases for correct rounding of the elementary functions in double precision. In *Proc. 15th IEEE Symposium on Computer Arithmetic*, IEEE Computer Society Press, Los Alamitos, CA, USA, 2001.
- [778] Vincent Lef  vre, Jean-Michel Muller, and Arnaud Tisserand. Toward correctly rounded transcendentals. *IEEE Trans. Comput.*, 47(11):1235–1243, 1998.
- [779] D. H. Lehmer. Tables to many places of decimals. *Mathematical Tables and Other Aids to Computation*, 1(1):30–31, 1943.
- [780] R. B. Lehoucq. The computation of elementary unitary matrices. *ACM Trans. Math. Software*, 22(4):393–400, 1996.
- [781] Frans Lemeire. Bounds for condition numbers of triangular and trapezoid matrices. *BIT*, 15:58–64, 1975.
- [782] H. Leuprecht and W. Oberaigner. Parallel algorithms for the rounding exact summation of floating point numbers. *Computing*, 28:89–104, 1982.
- [783] Robert Michael Lewis, Virginia J. Torczon, and Michael W. Trossett. Direct search methods: Then and now. *J. Comput. Appl. Math.*, 124:191–207, 2000.
- [784] T. Y. Li and Zhonggang Zeng. Homotopy-determinant algorithm for solving non-symmetric eigenvalue problems. *Math. Comp.*, 59(200):483–502, 1992.
- [785] X. S. Li, J. W. Demmel, D. H. Bailey, G. Henry, Y. Hida, J. Iskandar, W. Kahan, A. Kapur, M. C. Martin, T. Tung, and D. J. Yoo. Design, implementation and testing of Extended and Mixed Precision BLAS. Technical Report CS-00-451, Department of Computer Science, University of Tennessee, Knoxville, TN, USA, October 2000. 61 pp. LAPACK Working Note 149.
- [786] Xiaoye S. Li and James W. Demmel. Making sparse Gaussian elimination scalable by static pivoting. In *Proceedings of SC98: High Performance Networking and Computing Conference, Orlando, Florida*, 1998. CD ROM.
- [787] Thomas Lickteig. The computational complexity of division in quadratic extension fields. *SIAM J. Comput.*, 16(2):278–311, 1987.
- [788] Seppo Linnainmaa. Analysis of some known methods of improving the accuracy of floating-point sums. *BIT*, 14:167–202, 1974.

- [789] Seppo Linnainmaa. Towards accurate statistical estimation of rounding errors in floating-point computations. *BIT*, 15:165–173, 1975.
- [790] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT*, 16:146–160, 1976.
- [791] Seppo Linnainmaa. Software for doubled-precision floating-point computations. *ACM Trans. Math. Software*, 7(3):272–283, 1981.
- [792] Peter Linz. Accurate floating-point summation. *Comm. ACM*, 13(6):361–362, 1970.
- [793] Elliot Linzer. On the stability of transform-based circular deconvolution. *SIAM J. Numer. Anal.*, 29(5):1482–1492, 1992.
- [794] Joseph W. H. Liu. A partial pivoting strategy for sparse symmetric matrix decomposition. *ACM Trans. Math. Software*, 13(2):173–182, 1987.
- [795] Georghios Loizou. Nonnormality and Jordan condition numbers of matrices. *J. Assoc. Comput. Mach.*, 16(4):580–584, 1969.
- [796] J. W. Longley. An appraisal of least squares programs for the electronic computer from the point of view of the user. *J. Amer. Statist. Assoc.*, 62:819–841, 1967.
- [797] Hao Lu. Fast solution of confluent Vandermonde linear systems. *SIAM J. Matrix Anal. Appl.*, 15(4):1277–1289, 1994.
- [798] Hao Lu. Fast algorithms for confluent Vandermonde linear systems and generalized Trummer's problem. *SIAM J. Matrix Anal. Appl.*, 16(2):655–674, 1995.
- [799] Hao Lu. Solution of Vandermonde-like systems and confluent Vandermonde-like systems. *SIAM J. Matrix Anal. Appl.*, 17(1):127–138, 1996.
- [800] J. N. Lyness and J. J. Kaganove. Comments on the nature of automatic quadrature routines. *ACM Trans. Math. Software*, 2(1):65–81, 1976.
- [801] J. N. Lyness and C. B. Moler. Van der Monde systems and numerical differentiation. *Numer. Math.*, 8:458–464, 1966.
- [802] M. Stuart Lynn. On the round-off error in the method of successive over-relaxation. *Math. Comp.*, 18(85):36–49, 1964.
- [803] Allan J. Macleod. Some statistics on Gaussian elimination with partial pivoting. *ACM SIGNUM Newsletter*, 24(2/3):10–14, 1989.
- [804] Jan R. Magnus and Heinz Neudecker. The commutation matrix: Some properties and applications. *Ann. Statist.*, 7(2):381–394, 1979.
- [805] Jan R. Magnus and Heinz Neudecker. *Matrix Differential Calculus with Applications in Statistics and Econometrics*. Revised edition, Wiley, Chichester, UK, 1999. xviii+395 pp. ISBN 0-471-98633-X.
- [806] J. H. Maindonald. *Statistical Computation*. Wiley, New York, 1984. xviii+370 pp. ISBN 0-471-86452-8.
- [807] John Makhoul. Toeplitz determinants and positive semidefiniteness. *IEEE Trans. Signal Processing*, 39(3):743–746, 1991.
- [808] Michael A. Malcolm. On accurate floating-point summation. *Comm. ACM*, 14(11):731–736, 1971.
- [809] Michael A. Malcolm. Algorithms to reveal properties of floating-point arithmetic. *Comm. ACM*, 15(11):949–951, 1972.
- [810] Michael A. Malcolm and John Palmer. A fast method for solving a class of tridiagonal linear systems. *Comm. ACM*, 17(1):14–17, 1974.

- [811] Alexander N. Malyshev. A note on the stability of Gauss-Jordan elimination for diagonally dominant matrices. *Computing*, 65(3):281–284, 2000.
- [812] Alexander N. Malyshev. Optimal backward perturbation bounds for the LSS problem. *BIT*, 41(2):430–432, 2001.
- [813] Alexander N. Malyshev and Miloud Sadkane. Computation of optimal backward perturbation bounds for large sparse linear least squares problems. *BIT*, 41(4):739–747, 2002.
- [814] Thomas A. Manteuffel. An interval analysis approach to rank determination in linear least squares problems. *SIAM J. Sci. Statist. Comput.*, 2(3):335–348, 1981.
- [815] *Maple*. Waterloo Maple Inc., Waterloo, Ontario, Canada. <http://www.maplesoft.com>.
- [816] John Markoff. Circuit flaw causes Pentium chip to miscalculate, Intel admits. *New York Times*, 1994. 24 November.
- [817] R. S. Martin, G. Peters, and J. H. Wilkinson. Iterative refinement of the solution of a positive definite system of equations. *Numer. Math.*, 8:203–216, 1966. Also in [1246, pp. 31–44], Contribution I/2.
- [818] *Mathematica*. Wolfram Research, Inc., Champaign, IL, USA. <http://www.wolfram.com>.
- [819] Gleanings far and near. *Mathematical Gazette*, 22(170):95, 1924.
- [820] Roy Mathias. Matrices with positive definite Hermitian part: Inequalities and linear systems. *SIAM J. Matrix Anal. Appl.*, 13(2):640–654, 1992.
- [821] Roy Mathias. Accurate eigensystem computations by Jacobi methods. *SIAM J. Matrix Anal. Appl.*, 16(3):977–1003, 1995.
- [822] Roy Mathias. The instability of parallel prefix matrix multiplication. *SIAM J. Sci. Comput.*, 16(4):956–973, 1995.
- [823] Roy Mathias. Analysis of algorithms for orthogonalizing products of unitary matrices. *Numerical Linear Algebra with Applications*, 3(2):125–145, 1996.
- [824] *MATLAB*. The MathWorks, Inc., Natick, MA, USA. <http://www.mathworks.com>.
- [825] *Symbolic Math Toolbox Version 2: User's Guide*. The MathWorks, Inc., Natick, MA, USA. Online version.
- [826] Shouichi Matsui and Masao Iri. An overflow/underflow-free floating-point representation of numbers. *J. Information Processing*, 4(3):123–133, 1981.
- [827] R. M. M. Mattheij. Stability of block LU-decompositions of matrices arising from BVP. *SIAM J. Alg. Discrete Methods*, 5(3):314–331, 1984.
- [828] R. M. M. Mattheij. The stability of LU-decompositions of block tridiagonal matrices. *Bull. Austral. Math. Soc.*, 29:177–205, 1984.
- [829] David W. Matula. In-and-out conversions. *Comm. ACM*, 11(1):47–50, 1968.
- [830] David W. Matula. A formalization of floating-point numeric base conversion. *IEEE Trans. Comput.*, C-19(8):681–692, 1970.
- [831] David W. Matula and Peter Kornerup. Finite precision rational arithmetic: Slash number systems. *IEEE Trans. Comput.*, C-34(1):3–18, 1985.
- [832] Charles McCarthy and Gilbert Strang. Optimal conditioning of matrices. *SIAM J. Numer. Anal.*, 10(2):370–388, 1974.

- [833] Daniel D. McCracken and William S. Dorn. *Numerical Methods and Fortran Programming: With Applications in Science and Engineering*. Wiley, New York, 1964. xii+457 pp.
- [834] P. McCullagh and J. A. Nelder. *Generalized Linear Models*. Second edition, Chapman and Hall, London, 1989. xix+511 pp. ISBN 0-412-31760-5.
- [835] B. D. McCullough and H. D. Vinod. The numerical reliability of econometric software. *Journal of Economic Literature*, 37:633–665, 1999.
- [836] William Marshall McKeeman. Algorithm 135: Crout with equilibration and iteration. *Comm. ACM*, 5:553–555, 1962.
- [837] K. I. M. McKinnon. Convergence of the Nelder–Mead simplex method to a non-stationary point. *SIAM J. Optim.*, 9(1):148–158, 1998.
- [838] J. L. Mead, R. E. Renaut, and B. D. Welfert. Stability of a pivoting strategy for parallel Gaussian elimination. *BIT*, 41(3):633–639, 2001.
- [839] Jean Meinguet. On the estimation of significance. In *Topics in Interval Analysis*, Elden Hansen, editor, Oxford University Press, 1969, pages 47–64.
- [840] Jean Meinguet. Refined error analyses of Cholesky factorization. *SIAM J. Numer. Anal.*, 20(6):1243–1250, 1983.
- [841] N. S. Mendelsohn. Some elementary properties of ill conditioned matrices and linear equations. *Amer. Math. Monthly*, 63(5):285–295, 1956.
- [842] Jorma Kaarlo Merikoski, Uoti Arpala, Ari Virtanen, Tin-Yau Tam, and Frank Uhlig. A best upper bound for the 2-norm condition number of a matrix. *Linear Algebra Appl.*, 254:355–365, 1997.
- [843] Michael Metcalf and John K. Reid. *Fortran 90/95 Explained*. Second edition, Oxford University Press, 1999. 358 pp. ISBN 0-19-850558-2.
- [844] N. Metropolis. Methods of significance arithmetic. In *The State of the Art in Numerical Analysis*, David A. H. Jacobs, editor, Academic Press, London, 1977, pages 179–192.
- [845] Gérard Meurant. A review on the inverse of symmetric tridiagonal and block tridiagonal matrices. *SIAM J. Matrix Anal. Appl.*, 13(3):707–728, 1992.
- [846] Carl D. Meyer, Jr. and R. J. Plemmons. Convergent powers of a matrix with applications to iterative methods for singular linear systems. *SIAM J. Numer. Anal.*, 14(4):699–705, 1977.
- [847] H. I. Meyer and B. J. Hollingsworth. A method of inverting large matrices of special form. *M.T.A.C.*, 11:94–97, 1957.
- [848] Victor J. Milenkovic. Verifiable implementations of geometric algorithms using finite precision arithmetic. *Artificial Intelligence*, 37:377–401, 1988.
- [849] D. F. Miller. The iterative solution of the matrix equation $XA + BX + C = 0$. *Linear Algebra Appl.*, 105:131–137, 1988.
- [850] Webb Miller. Computational complexity and numerical stability. *SIAM J. Comput.*, 4(2):97–107, 1975.
- [851] Webb Miller. Software for roundoff analysis. *ACM Trans. Math. Software*, 1(2):108–128, 1975.
- [852] Webb Miller. Graph transformations for roundoff analysis. *SIAM J. Comput.*, 5(2):204–216, 1976.

- [853] Webb Miller. *The Engineering of Numerical Software*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1984. viii+167 pp. ISBN 0-13-279043-2.
- [854] Webb Miller and David Spooner. Software for roundoff analysis, II. *ACM Trans. Math. Software*, 4(4):369–387, 1978.
- [855] Webb Miller and David Spooner. Algorithm 532: Software for roundoff analysis. *ACM Trans. Math. Software*, 4(4):388–390, 1978.
- [856] Webb Miller and Celia Wrathall. *Software for Roundoff Analysis of Matrix Algorithms*. Academic Press, New York, 1980. x+151 pp. ISBN 0-12-497250-0.
- [857] L. Mirsky. *An Introduction to Linear Algebra*. Oxford University Press, 1961. viii+440 pp. Reprinted by Dover, New York, 1990. ISBN 0-486-66434-1.
- [858] Herbert F. Mitchell, Jr. Inversion of a matrix of order 38. *M.T.A.C.*, 3:161–166, 1948.
- [859] Cleve B. Moler. Iterative refinement in floating point. *J. Assoc. Comput. Mach.*, 14(2):316–321, 1967.
- [860] Cleve B. Moler. Matrix computations with Fortran and paging. *Comm. ACM*, 15(4):268–270, 1972.
- [861] Cleve B. Moler. Algorithm 423: Linear equation solver. *Comm. ACM*, 15(4):274, 1972.
- [862] Cleve B. Moler. Cramer’s rule on 2-by-2 systems. *ACM SIGNUM Newsletter*, 9(4):13–14, 1974.
- [863] Cleve B. Moler. Three research problems in numerical linear algebra. In *Numerical Analysis*, G. H. Golub and J. Olinger, editors, volume 22 of *Proceedings of Symposia in Applied Mathematics*, American Mathematical Society, Providence, RI, USA, 1978, pages 1–18.
- [864] Cleve B. Moler. Cleve’s corner: The world’s simplest impossible problem. *The MathWorks Newsletter*, 4(2):6–7, 1990.
- [865] Cleve B. Moler. Technical note: Double-rounding and implications for numeric computations. *The MathWorks Newsletter*, 4:6, 1990.
- [866] Cleve B. Moler. A tale of two numbers. *SIAM News*, 28:1,16, 1995. Also in *MATLAB News and Notes*, Winter 1995, 10–12.
- [867] Cleve B. Moler and Donald Morrison. Replacing square roots by Pythagorean sums. *IBM J. Res. Develop.*, 27(6):577–581, 1983.
- [868] Cleve B. Moler and Charles F. Van Loan. Nineteen dubious ways to compute the exponential of a matrix. *SIAM Rev.*, 20(4):801–836, 1978.
- [869] Ole Møller. Note on quasi double-precision. *BIT*, 5:251–255, 1965.
- [870] Ole Møller. Quasi double-precision in floating point addition. *BIT*, 5:37–50, 1965. See also [869] for remarks on this article.
- [871] J. Strother Moore, Thomas W. Lynch, and Matt Kaufmann. A mechanically checked proof of the AMD5_K86 floating-point division program. *IEEE Trans. Comput.*, 47(9):913–926, 1998.
- [872] Ramon E. Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1966. xi+145 pp.
- [873] Ramon E. Moore. *Methods and Applications of Interval Analysis*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1979. xi+190 pp. ISBN 0-89871-161-4.

- [874] Robb J. Muirhead. *Aspects of Multivariate Statistical Theory*. Wiley, New York, 1982. xix+673 pp. ISBN 0-471-09442-0.
- [875] Jean-Michel Muller. *Arithmétique des Ordinateurs*. Masson, Paris, 1989. 214 pp. In French. Cited in [442]. ISBN 2-225-81689-1.
- [876] Jean-Michel Muller. *Elementary Functions: Algorithms and Implementation*. Birkhäuser, Boston, MA, USA, 1997. xv+204 pp. ISBN 0-8176-3990-X.
- [877] K. H. Müller. Rounding error analysis of Horner's scheme. *Computing*, 30:285–303, 1983.
- [878] FPV: A floating-point validation package. Release I. User's guide. Technical Report NP 1201, NAG Ltd., Oxford, May 1986.
- [879] M. Zuhair Nashed and L. B. Rall. Annotated bibliography on generalized inverses and applications. In *Generalized Inverses and Applications*, M. Zuhair Nashed, editor, Academic Press, New York, 1976, pages 771–1041.
- [880] Larry Neal and George Poole. A geometric analysis of Gaussian elimination. II. *Linear Algebra Appl.*, 173:239–264, 1992.
- [881] J. A. Nelder and R. Mead. A simplex method for function minimization. *Comput. J.*, 7:308–313, 1965.
- [882] David Nelson, editor. *The Penguin Dictionary of Mathematics*. Second edition, Penguin, London, 1998. 461 pp. ISBN 0-14-051342-6.
- [883] A. Neumaier. Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen. *Z. Angew. Math. Mech.*, 54:39–51, 1974.
- [884] A. Neumaier. Inner product rounding error analysis in the presence of underflow. *Computing*, 34:365–373, 1985.
- [885] A. Neumaier. On the comparison of H -matrices with M -matrices. *Linear Algebra Appl.*, 83:135–141, 1986.
- [886] M. Neumann and R. J. Plemmons. Backward error analysis for linear systems associated with inverses of H -matrices. *BIT*, 24:102–112, 1984.
- [887] A. C. R. Newbery. Error analysis for Fourier series evaluation. *Math. Comp.*, 27 (123):639–644, 1973.
- [888] A. C. R. Newbery. Error analysis for polynomial evaluation. *Math. Comp.*, 28 (127):789–793, 1974.
- [889] Simon Newcomb. Note on the frequency of use of the different digits in natural numbers. *Amer. J. Math.*, 4:39–40, 1881.
- [890] Morris Newman. Matrix computations. In *Survey of Numerical Analysis*, John Todd, editor, McGraw-Hill, New York, 1962, pages 222–254.
- [891] Morris Newman and John Todd. The evaluation of matrix inversion programs. *J. Soc. Indust. Appl. Math.*, 6(4):466–476, 1958.
- [892] K. Nickel. Das Kahan-Babuškasche Summierungsverfahren in Triplex-ALGOL 60. *Z. Angew. Math. Mech.*, 50:369–373, 1970.
- [893] Karl Nickel. Interval-analysis. In *The State of the Art in Numerical Analysis*, David A. H. Jacobs, editor, Academic Press, London, 1977, pages 193–225.
- [894] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer-Verlag, New York, 1999. xx+636 pp. ISBN 0-387-98793-2.
- [895] Yvan Notay. On the convergence rate of the conjugate gradients in presence of rounding errors. *Numer. Math.*, 65:301–317, 1993.

- [896] Colm Art O'Cinneide. Entrywise perturbation theory and error analysis for Markov chains. *Numer. Math.*, 65:109–120, 1993.
- [897] W. Oettli. On the solution set of a linear system with inaccurate coefficients. *SIAM J. Numer. Anal.*, 2(1):115–118, 1965.
- [898] W. Oettli and W. Prager. Compatibility of approximate solution of linear equations with given error bounds for coefficients and right-hand sides. *Numer. Math.*, 6:405–409, 1964.
- [899] W. Oettli, W. Prager, and J. H. Wilkinson. Admissible solutions of linear systems with not sharply defined coefficients. *SIAM J. Numer. Anal.*, 2(2):291–299, 1965.
- [900] Dianne Prost O'Leary. Estimating matrix condition numbers. *SIAM J. Sci. Statist. Comput.*, 1(2):205–209, 1980.
- [901] J. Oliver. An error analysis of the modified Clenshaw method for evaluating Chebyshev and Fourier series. *J. Inst. Maths. Applics.*, 20:379–391, 1977.
- [902] J. Oliver. Rounding error propagation in polynomial evaluation schemes. *J. Comput. Appl. Math.*, 5(2):85–97, 1979.
- [903] F. W. J. Olver. A new approach to error arithmetic. *SIAM J. Numer. Anal.*, 15(2):368–393, 1978.
- [904] F. W. J. Olver. Error analysis of complex arithmetic. In *Computational Aspects of Complex Analysis*, volume 102 of *NATO Advanced Study Institute Series C*, D. Reidel, Dordrecht, Holland, 1983, pages 279–292.
- [905] F. W. J. Olver. Error bounds for polynomial evaluation and complex arithmetic. *IMA J. Numer. Anal.*, 6:373–379, 1986.
- [906] F. W. J. Olver and J. H. Wilkinson. *A posteriori* error bounds for Gaussian elimination. *IMA J. Numer. Anal.*, 2:377–406, 1982.
- [907] James M. Ortega. An error analysis of Householder's method for the symmetric eigenvalue problem. *Numer. Math.*, 5:211–225, 1963.
- [908] James M. Ortega. *Numerical Analysis: A Second Course*. Academic Press, New York, 1972. xiii+201 pp. Republished by Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1990. ISBN 0-89871-250-5 (original), 0-89871-250-5 (SIAM).
- [909] A. M. Ostrowski. *Solution of Equations in Euclidean and Banach Spaces*. Academic Press, New York, 1973. xx+412 pp. Third edition of *Solution of Equations and Systems of Equations*. ISBN 0-12530260-6.
- [910] Michael L. Overton. *Numerical Computing with IEEE Floating Point Arithmetic: Including One Theorem, One Rule of Thumb, and One Hundred and One Exercises*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2001. xiv+104 pp. ISBN 0-89871-482-6.
- [911] C. C. Paige. An error analysis of a method for solving matrix equations. *Math. Comp.*, 27(122):355–359, 1973.
- [912] C. C. Paige. Some aspects of generalized QR factorizations. In *Reliable Numerical Computation*, M. G. Cox and S. J. Hammarling, editors, Oxford University Press, 1990, pages 73–91.
- [913] C. C. Paige and M. Wei. History and generality of the CS decomposition. *Linear Algebra Appl.*, 208/209:303–326, 1994.
- [914] C.-T. Pan. On the existence and computation of rank-revealing LU factorizations. *Linear Algebra Appl.*, 316:199–222, 2000.

- [915] Victor Y. Pan. Strassen algorithm is not optimal. Trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix multiplication. In *Proc. 19th Annual Symposium on the Foundations of Computer Science*, Ann Arbor, MI, USA, 1978, pages 166–176.
- [916] Victor Y. Pan. How can we speed up matrix multiplication? *SIAM Rev.*, 26(3): 393–415, 1984.
- [917] Victor Y. Pan. *Structured Matrices and Polynomials: Unified Superfast Algorithms*. Birkhäuser, Boston, MA, USA, 2001. xxiv+278 pp. ISBN 0-8176-4240-4.
- [918] Victor Y. Pan, Sheryl Branham, Rhys E. Rosholt, and Ai-Long Zheng. Newton's iteration for structured matrices. In *Fast Reliable Algorithms for Matrices with Structure*, Thomas Kailath and Ali H. Sayed, editors, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1999, pages 189–210.
- [919] Victor Y. Pan and Robert Schreiber. An improved Newton iteration for the generalized inverse of a matrix, with applications. *SIAM J. Sci. Statist. Comput.*, 12(5):1109–1130, 1991.
- [920] Victor Y. Pan and Y. Yu. Certification of numerical computation of the sign of the determinant of a matrix. *Algorithmica*, 30(4):708–724, 2001.
- [921] Behrooz Parhami. *Computer Arithmetic: Algorithms and Hardware Designs*. Oxford University Press, New York, 2000. xx+490 pp. ISBN 0-19-512583-5.
- [922] Beresford N. Parlett. Laguerre's method applied to the matrix eigenvalue problem. *Math. Comp.*, 18:464–485, 1964.
- [923] Beresford N. Parlett. Matrix eigenvalue problems. *Amer. Math. Monthly*, 72(2): 59–66, 1965.
- [924] Beresford N. Parlett. Analysis of algorithms for reflections in bisectors. *SIAM Rev.*, 13(2):197–208, 1971.
- [925] Beresford N. Parlett. The contribution of J. H. Wilkinson to numerical analysis. In *A History of Scientific Computing*, Stephen G. Nash, editor, Addison-Wesley, Reading, MA, USA, 1990, pages 17–30.
- [926] Beresford N. Parlett. *The Symmetric Eigenvalue Problem*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1998. xxiv+398 pp. Unabridged, amended version of book first published by Prentice-Hall in 1980. ISBN 0-89871-402-8.
- [927] Beresford N. Parlett and Inderjit S. Dhillon. Relatively robust representations of symmetric tridiagonals. *Linear Algebra Appl.*, 309:121–151, 2000.
- [928] Michael S. Paterson and Larry J. Stockmeyer. On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM J. Comput.*, 2(1):60–66, 1973.
- [929] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Second edition, Morgan Kaufmann, San Francisco, CA, USA, 1998. xxix+759+appendices pp. ISBN 1-55860-428-6.
- [930] Vern Paxson. A program for testing IEEE decimal–binary conversion. Manuscript. <ftp://ftp.ee.lbl.gov/testbase-report.ps.Z>, May 1991. 40 pp.
- [931] Heinz-Otto Peitgen, Hartmut Jürgens, and Dietmar Saupe. *Fractals for the Classroom. Part One: Introduction to Fractals and Chaos*. Springer-Verlag, New York, 1992. xiv+450 pp. ISBN 0-387-97041-X.

- [932] J. M. Peña. Pivoting strategies leading to small bounds of the errors for certain linear systems. *IMA J. Numer. Anal.*, 16(2):141–153, 1996.
- [933] J. M. Peña. Pivoting strategies leading to diagonal dominance by rows. *Numer. Math.*, 81:293–304, 1998.
- [934] Thilo Penzl. Numerical solution of generalized Lyapunov equations. *Adv. in Comput. Math.*, 8:33–48, 1998.
- [935] Colin Percival. Rapid multiplication modulo the sum and difference of highly composite numbers. *Math. Comp.*, 2002. To appear.
- [936] G. Peters and J. H. Wilkinson. The least squares problem and pseudo-inverses. *Comput. J.*, 13(3):309–316, 1970.
- [937] G. Peters and J. H. Wilkinson. Practical problems arising in the solution of polynomial equations. *J. Inst. Maths. Applics.*, 8:16–35, 1971.
- [938] G. Peters and J. H. Wilkinson. On the stability of Gauss-Jordan elimination with pivoting. *Comm. ACM*, 18(1):20–24, 1975.
- [939] Karl Petersen. *Ergodic Theory*. Cambridge University Press, Cambridge, UK, 1981. xi+329 pp. ISBN 0-521-23632-0.
- [940] M. Pichat. Correction d’une somme en arithmétique à virgule flottante. *Numer. Math.*, 19:400–406, 1972.
- [941] Daniel J. Pierce and Robert J. Plemmons. Fast adaptive condition estimation. *SIAM J. Matrix Anal. Appl.*, 13(1):274–291, 1992.
- [942] P. J. Plauger. Properties of floating-point arithmetic. *Computer Language*, 5(3):17–22, 1988.
- [943] R. J. Plemmons. Regular splittings and the discrete Neumann problem. *Numer. Math.*, 25:153–161, 1976.
- [944] Robert J. Plemmons. Linear least squares by elimination and MGS. *J. Assoc. Comput. Mach.*, 21(4):581–585, 1974.
- [945] Svatopluk Poljak and Jiří Rohn. Checking robust nonsingularity is NP-hard. *Math. Control Signals Systems*, 6:1–9, 1993.
- [946] Ben Polman. Incomplete blockwise factorizations of (block) H -matrices. *Linear Algebra Appl.*, 90:119–132, 1987.
- [947] George Poole and Larry Neal. The rook’s pivoting strategy. *J. Comput. Appl. Math.*, 123:353–369, 2000.
- [948] M. J. D. Powell. A survey of numerical methods for unconstrained optimization. *SIAM Rev.*, 12(1):79–97, 1970.
- [949] M. J. D. Powell. A view of unconstrained minimization algorithms that do not require derivatives. *ACM Trans. Math. Software*, 1(2):97–107, 1975.
- [950] M. J. D. Powell. Direct search algorithms for optimization calculations. *Acta Numerica*, 7:287–336, 1998.
- [951] M. J. D. Powell and J. K. Reid. On applying Householder transformations to linear least squares problems. In *Proc. IFIP Congress 1968*, North-Holland, Amsterdam, The Netherlands, 1969, pages 122–126.
- [952] Stephen Power. The Cholesky decomposition in Hilbert space. *IMA Bulletin*, 22 (11/12):186–187, 1986.

- [953] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in FORTRAN: The Art of Scientific Computing*. Second edition, Cambridge University Press, 1992. xxvi+963 pp. ISBN 0-521-43064-X.
- [954] Douglas M. Priest. Algorithms for arbitrary precision floating point arithmetic. In *Proc. 10th IEEE Symposium on Computer Arithmetic*, Peter Kornerup and David W. Matula, editors, IEEE Computer Society Press, Los Alamitos, CA, USA, 1991, pages 132–143.
- [955] Douglas M. Priest. *On Properties of Floating Point Arithmetics: Numerical Stability and the Cost of Accurate Computations*. PhD thesis, Mathematics Department, University of California, Berkeley, CA, USA, November 1992. 126 pp. <ftp://ftp.icsi.berkeley.edu/pub/theory/priest-thesis.ps.Z>.
- [956] J. D. Pryce. Round-off error analysis with fewer tears. *IMA Bulletin*, 17:40–47, 1981.
- [957] J. D. Pryce. A new measure of relative error for vectors. *SIAM J. Numer. Anal.*, 21(1):202–215, 1984.
- [958] J. D. Pryce. Multiplicative error analysis of matrix transformation algorithms. *IMA J. Numer. Anal.*, 5:437–445, 1985.
- [959] Chiara Puglisi. Modification of the Householder method based on the compact WY representation. *SIAM J. Sci. Statist. Comput.*, 13(3):723–726, 1992.
- [960] Heinrich Puschmann and Joaquín Cortés. The coordinex problem and its relation to the conjecture of Wilkinson. *Numer. Math.*, 42:291–297, 1983.
- [961] Heinrich Puschmann and Marcelo Nordio. Zwei Unzulässige Verstärkungen der Vermutung von Wilkinson. *Linear Algebra Appl.*, 72:167–176, 1985. In German.
- [962] Gerald D. Quinlan. Round-off error in long-term orbital integrations using multi-step methods. *Celestial Mechanics and Dynamical Astronomy*, 58:339–351, 1994.
- [963] Kevin Quinn. Ever had problems rounding off figures? This stock exchange has. *Wall Street Journal*, 1983. 8 November.
- [964] Thomas Quinn and Scott Tremaine. Roundoff error in long-term planetary orbit integrations. *Astron. J.*, 99(3):1016–1023, 1990.
- [965] Thomas R. Quinn, Scott Tremaine, and Martin Duncan. A three million year integration of the earth's orbit. *Astron. J.*, 101(6):2287–2305, 1991.
- [966] Enrique S. Quintana, Gregorio Quintana, Xiaobai Sun, and Robert van de Geijn. A note on parallel matrix inversion. *SIAM J. Sci. Comput.*, 22(5):1762–1771, 2001.
- [967] Ralph A. Raimi. The peculiar distribution of first digits. *Scientific American*, 221(6):109–120, 1969.
- [968] Ralph A. Raimi. The first digit problem. *Amer. Math. Monthly*, 83:521–538, 1976.
- [969] Louis B. Rall. *Automatic Differentiation: Techniques and Applications*, volume 120 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, 1981. viii+165 pp.
- [970] Louis B. Rall. Tools for mathematical computation. In *Computer Aided Proofs in Analysis*, Kenneth R. Meyer and Dieter S. Schmidt, editors, volume 28 of *IMA Volumes in Mathematics and Its Applications*, Springer-Verlag, New York, 1991, pages 217–228.
- [971] George U. Ramos. Roundoff error analysis of the fast Fourier transform. *Math. Comp.*, 25(116):757–768, 1971.

- [972] Brian Randell, editor. *The Origins of Digital Computers: Selected Papers*. Third edition, Springer-Verlag, Berlin, 1975. xvi+580 pp. ISBN 3-540-11319-3.
- [973] Wolfgang Rath. Fast Givens rotations for orthogonal similarity transformations. *Numer. Math.*, 40:47–56, 1982.
- [974] Satish C. Reddy and Lloyd N. Trefethen. Stability of the method of lines. *Numer. Math.*, 62:235–267, 1992.
- [975] Lothar Reichel. Newton interpolation at Leja points. *BIT*, 30:332–346, 1990.
- [976] Lothar Reichel. Fast QR decomposition of Vandermonde-like matrices and polynomial least squares approximation. *SIAM J. Matrix Anal. Appl.*, 12(3):552–564, 1991.
- [977] Lothar Reichel and Gerhard Opfer. Chebyshev-Vandermonde systems. *Math. Comp.*, 57(196):703–721, 1991.
- [978] Lothar Reichel and Lloyd N. Trefethen. Eigenvalues and pseudo-eigenvalues of Toeplitz matrices. *Linear Algebra Appl.*, 162–164:153–185, 1992.
- [979] J. K. Reid. A note on the stability of Gaussian elimination. *J. Inst. Maths. Applics.*, 8:374–375, 1971.
- [980] J. K. Reid. Sparse matrices. In *The State of the Art in Numerical Analysis*, A. Iserles and M. J. D. Powell, editors, Oxford University Press, 1987, pages 59–85.
- [981] J. K. Reid. Implicit scaling of linear least squares problems. *BIT*, 40(1):146–157, 2000.
- [982] John F. Reiser and Donald E. Knuth. Evading the drift in floating-point addition. *Inform. Process. Lett.*, 3(3):84–87, 1975.
- [983] Werner C. Rheinboldt. On measures of ill-conditioning for nonlinear equations. *Math. Comp.*, 30(133):104–111, 1976.
- [984] John R. Rice. Experiments on Gram-Schmidt orthogonalization. *Math. Comp.*, 20:325–328, 1966.
- [985] John R. Rice. A theory of condition. *SIAM J. Numer. Anal.*, 3(2):287–310, 1966.
- [986] J. L. Rigal and J. Gaches. On the compatibility of a given solution with the data of a linear system. *J. Assoc. Comput. Mach.*, 14(3):543–548, 1967.
- [987] T. G. Robertazzi and S. C. Schwartz. Best “ordering” for floating-point addition. *ACM Trans. Math. Software*, 14(1):101–110, 1988.
- [988] J. D. Roberts. Linear model reduction and solution of the algebraic Riccati equation by use of the sign function. *Internat. J. Control*, 32(4):677–687, 1980. First issued as report CUED/B-Control/TR13, Department of Engineering, University of Cambridge, 1971.
- [989] Jiří Rohn. New condition numbers for matrices and linear systems. *Computing*, 41:167–169, 1989.
- [990] Jiří Rohn. Systems of linear interval equations. *Linear Algebra Appl.*, 126:39–78, 1989.
- [991] Jiří Rohn. Nonsingularity under data rounding. *Linear Algebra Appl.*, 139:171–174, 1990.
- [992] Jiří Rohn. NP-hardness results for some linear and quadratic problems. Technical Report No. 619, Institute of Computer Science, Academy of Sciences of the Czech Republic, Prague, January 1995. 11 pp.

- [993] D. R. Ross. Reducing truncation errors using cascading accumulators. *Comm. ACM*, 8(1):32–33, 1965.
- [994] M. W. Routh, P. A. Swartz, and M. B. Denton. Performance of the super modified simplex. *Analytical Chemistry*, 49(9):1422–1428, 1977.
- [995] Thomas Harvey Rowan. *Functional Stability Analysis of Numerical Algorithms*. PhD thesis, University of Texas at Austin, Austin, TX, USA, May 1990. xii+205 pp.
- [996] Axel Ruhe. Numerical aspects of Gram-Schmidt orthogonalization of vectors. *Linear Algebra Appl.*, 52/53:591–601, 1983.
- [997] Siegfried M. Rump. Structured perturbations and symmetric matrices. *Linear Algebra Appl.*, 278:121–132, 1998.
- [998] Siegfried M. Rump. Fast and parallel interval arithmetic. *BIT*, 39(3):534–554, 1999.
- [999] Siegfried M. Rump. Ill-conditioned matrices are componentwise near to singularity. *SIAM Rev.*, 41(1):102–112, 1999.
- [1000] Siegfried M. Rump. INTLAB—INTERVAL LABoratory. In *Developments in Reliable Computing*, Tibor Csendes, editor, Kluwer Academic Publishers, Dordrecht, The Netherlands, 1999, pages 77–104.
- [1001] Siegfried M. Rump. Self-validating methods. *Linear Algebra Appl.*, 324:3–13, 2001.
- [1002] Siegfried M. Rump. Optimal scaling for P -norms and componentwise distance to singularity. *IMA J. Numer. Anal.*, 2002. To appear.
- [1003] David M. Russinoff. A mechanically checked proof of IEEE compliance of the floating point multiplication, division and square root algorithms of the AMD-K7 processor. *LMS J. Computer Math.*, 1:148–200, 1998.
- [1004] D. E. Rutherford. Some continuant determinants arising in physics and chemistry. *Proc. Royal Soc. Edin.*, 62,A:229–236, 1947.
- [1005] D. E. Rutherford. Some continuant determinants arising in physics and chemistry—II. *Proc. Royal Soc. Edin.*, 63,A:232–241, 1952.
- [1006] H. Rutishauser. On test matrices. In *Programmation en Mathématiques Numériques, Besançon, 1966*, volume 7 (no. 165) of *Éditions Centre Nat. Recherche Sci., Paris*, 1968, pages 349–365.
- [1007] Heinz Rutishauser. Solution of eigenvalue problems with the LR-transformation. In *Further Contributions to the Solution of Simultaneous Linear Equations and the Determination of Eigenvalues*, number 49 in Applied Mathematics Series, National Bureau of Standards, United States Department of Commerce, Washington, D. C., 1958, pages 47–81.
- [1008] Ahmed H. Sameh and Richard P. Brent. Solving triangular systems on a parallel computer. *SIAM J. Numer. Anal.*, 14(6):1101–1113, 1977.
- [1009] Klaus Samelson and Friedrich L. Bauer. Optimale Rechengenauigkeit bei Rechenanlagen mit gleitendem Komma. *Z. Angew. Math. Phys.*, 4:312–316, 1953.
- [1010] J. M. Sanz-Serna and S. Larsson. Shadows, chaos, and saddles. *Appl. Numer. Math.*, 13:181–190, 1993.
- [1011] M. A. Saunders. Large-scale linear programming using the Cholesky factorization. Report CS 252, Department of Computer Science, Stanford University, January 1972.

- [1012] Werner Sautter. Error analysis of Gauss elimination for the best least squares problem. *Numer. Math.*, 30:165–184, 1978.
- [1013] I. Richard Savage and Eugene Lukacs. Tables of inverse of finite segments of the Hilbert matrix. In *Contributions to the Solution of Systems of Linear Equations and the Determination of Eigenvalues*, Olga Taussky, editor, number 39 in Applied Mathematics Series, National Bureau of Standards, United States Department of Commerce, Washington, D. C., 1954, pages 105–108.
- [1014] James B. Scarborough. *Numerical Mathematical Analysis*. Second edition, Johns Hopkins University Press, Baltimore, MD, USA, 1950. xviii+511 pp.
- [1015] James C. Schatzman. Accuracy of the discrete Fourier transform and the fast Fourier transform. *SIAM J. Sci. Comput.*, 17(5):1150–1166, 1996.
- [1016] Charles W. Schelin. Calculator function approximation. *Amer. Math. Monthly*, 90(5):317–325, 1983.
- [1017] R. Scherer and K. Zeller. Shorthand notation for rounding errors. *Computing*, Suppl. 2:165–168, 1980.
- [1018] Hans Schneider and W. Gilbert Strang. Comparison theorems for supremum norms. *Numer. Math.*, 4:15–20, 1962.
- [1019] Hans Schneider and Hans F. Weinberger. On matrices for which norm bounds are attained. *Linear Algebra Appl.*, 275–276:563–577, 1998.
- [1020] J. L. Schonfelder and M. Razaz. Error control with polynomial approximations. *IMA J. Numer. Anal.*, 1:105–114, 1980.
- [1021] Robert S. Schreiber. Block algorithms for parallel machines. In *Numerical Algorithms for Modern Parallel Computer Architectures*, M. H. Schultz, editor, number 13 in IMA Volumes In Mathematics and Its Applications, Springer-Verlag, Berlin, 1988, pages 197–207.
- [1022] Robert S. Schreiber. Hough’s random story explained. *NA Digest*, Volume 89, Issue 3, 1989.
- [1023] Robert S. Schreiber and Beresford N. Parlett. Block reflectors: Theory and computation. *SIAM J. Numer. Anal.*, 25(1):189–205, 1988.
- [1024] Robert S. Schreiber and Charles F. Van Loan. A storage efficient WY representation for products of Householder transformations. *SIAM J. Sci. Statist. Comput.*, 10:53–57, 1989.
- [1025] N. L. Schryer. A test of a computer’s floating-point arithmetic unit. Computing Science Technical Report No. 89, AT&T Bell Laboratories, Murray Hill, NJ, USA, 1981.
- [1026] N. L. Schryer. Determination of correct floating-point model parameters. In *Sources and Development of Mathematical Software*, Wayne R. Cowell, editor, Prentice-Hall, Englewood Cliffs, NJ, USA, 1984, pages 360–366.
- [1027] Günther Schulz. Iterative Berechnung der reziproken Matrix. *Z. Angew. Math. Mech.*, 13:57–59, 1933.
- [1028] Robert Sedgewick. *Algorithms*. Second edition, Addison-Wesley, Reading, MA, USA, 1988. xii+657 pp. ISBN 0-201-06673-4.
- [1029] Charles Severance. An interview with the old man of floating-point. Reminiscences elicited from William Kahan by Charles Severance, February 1998. <http://www.cs.berkeley.edu/~wkahan/ieee754status/754story.html>. A shortened version appears in *Computer*, 31(3):114–115, March 1998.

- [1030] Lawrence F. Shampine. *Numerical Solution of Ordinary Differential Equations*. Chapman and Hall, New York, 1994. x+484 pp. ISBN 0-412-05151-6.
- [1031] Lawrence F. Shampine and Richard C. Allen, Jr. *Numerical Computing: An Introduction*. W. B. Saunders, Philadelphia, PA, USA, 1973. vii+258 pp. ISBN 0-7216-8150-6.
- [1032] Lawrence F. Shampine and Mark W. Reichelt. The MATLAB ODE suite. *SIAM J. Sci. Comput.*, 18(1):1–22, 1997.
- [1033] Alexander Shapiro. Optimally scaled matrices, necessary and sufficient conditions. *Numer. Math.*, 39:239–245, 1982.
- [1034] Alexander Shapiro. Optimal block diagonal l_2 -scaling of matrices. *SIAM J. Numer. Anal.*, 22(1):81–94, 1985.
- [1035] Alexander Shapiro. Upper bounds for nearly optimal diagonal scaling of matrices. *Linear and Multilinear Algebra*, 29:147–147, 1991.
- [1036] H. P. Sharangpani and M. L. Barton. Statistical analysis of floating point flaw in the Pentium processor (1994). Technical report, Intel Corporation, November 1994. 31 pp.
- [1037] David Shepherd and Greg Wilson. Making chips that work. *New Scientist*, pages 61–64, 1989. 13 May.
- [1038] Jonathan Richard Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete and Computational Geometry*, 18(3): 305–363, 1997.
- [1039] Gautam M. Shroff and Christian H. Bischof. Adaptive condition estimation for rank-one updates of QR factorizations. *SIAM J. Matrix Anal. Appl.*, 13(4):1264–1278, 1992.
- [1040] Robert D. Skeel. Scaling for numerical stability in Gaussian elimination. *J. Assoc. Comput. Mach.*, 26(3):494–526, 1979.
- [1041] Robert D. Skeel. Iterative refinement implies numerical stability for Gaussian elimination. *Math. Comp.*, 35(151):817–832, 1980.
- [1042] Robert D. Skeel. Effect of equilibration on residual size for partial pivoting. *SIAM J. Numer. Anal.*, 18(3):449–454, 1981.
- [1043] Robert D. Skeel. Safety in numbers: The boundless errors of numerical computation. Working Document 89-3, Department of Computer Science, University of Illinois, Urbana, IL, USA, 1989. 9 pp.
- [1044] Robert D. Skeel and Jerry B. Keiper. *Elementary Numerical Computing with Mathematica*. McGraw-Hill, New York, 1993. xiv+434 pp. ISBN 0-07-057820-6.
- [1045] Gerard L. G. Sleijpen, Henk A. Van der Vorst, and Jan Modersitzki. Differences in the effects of rounding errors in Krylov solvers for symmetric indefinite linear systems. *SIAM J. Matrix Anal. Appl.*, 22(3):726–751, 2000.
- [1046] Steve Smale. Some remarks on the foundations of numerical analysis. *SIAM Rev.*, 32(2):211–220, 1990.
- [1047] B. T. Smith, J. M. Boyle, J. J. Dongarra, B. S. Garbow, Y. Ikebe, V. C. Klema, and C. B. Moler. *Matrix Eigensystem Routines—EISPACK Guide*, volume 6 of *Lecture Notes in Computer Science*. Second edition, Springer-Verlag, Berlin, 1976. vii+551 pp. ISBN 3-540-06710-8.
- [1048] David M. Smith. Algorithm 693: A FORTRAN package for floating-point multiple-precision arithmetic. *ACM Trans. Math. Software*, 17(2):273–283, 1991.

- [1049] David M. Smith. Algorithm 786: Multiple-precision complex arithmetic and functions. *ACM Trans. Math. Software*, 24(4):359–367, 1998.
- [1050] Francis J. Smith. An algorithm for summing orthogonal polynomial series and their derivatives with applications to curve-fitting and interpolation. *Math. Comp.*, 19:33–36, 1965.
- [1051] Jon M. Smith. *Scientific Analysis on the Pocket Calculator*. Wiley, New York, 1975. xii+380 pp. ISBN 0-471-79997-1.
- [1052] Robert L. Smith. Algorithm 116: Complex division. *Comm. ACM*, 5(8):435, 1962.
- [1053] Alicja Smoktunowicz. A note on the strong componentwise stability of algorithms for solving symmetric linear systems. *Demonstratio Mathematica*, 28(2):443–448, 1995.
- [1054] Alicja Smoktunowicz and Jolanta Sokolnicka. Binary cascades iterative refinement in doubled-mantissa arithmetics. *BIT*, 24:123–127, 1984.
- [1055] James N. Snyder. On the improvement of the solutions to a set of simultaneous linear equations using the ILLIAC. *Mathematical Tables and Other Aids to Computation*, 9:177–184, 1955.
- [1056] Torsten Söderström and G. W. Stewart. On the numerical properties of an iterative method for computing the Moore–Penrose generalized inverse. *SIAM J. Numer. Anal.*, 11(1):61–74, 1974.
- [1057] D. C. Sorensen. Analysis of pairwise pivoting in Gaussian elimination. *IEEE Trans. Comput.*, C-34:274–278, 1985.
- [1058] J. Spieß. Untersuchungen des Zeitgewinns durch neue Algorithmen zur Matrix-Multiplikation. *Computing*, 17:23–36, 1976.
- [1059] Gerhard Starke and Wilhelm Niethammer. SOR for $AX - XB = C$. *Linear Algebra Appl.*, 154–156:355–375, 1991.
- [1060] Guy L. Steele, Jr. and Jon L. White. How to print floating-point numbers accurately. *SIGPLAN Notices*, 25(6):112–126, 1990.
- [1061] Irene A. Stegun and Milton Abramowitz. Pitfalls in computation. *J. Soc. Indust. Appl. Math.*, 4(4):207–219, 1956.
- [1062] Pat H. Sterbenz. *Floating-Point Computation*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1974. xiv+316 pp. ISBN 0-13-322495-3.
- [1063] G. W. Stewart. Error analysis of the algorithm for shifting the zeros of a polynomial by synthetic division. *Math. Comp.*, 25(113):135–139, 1971.
- [1064] G. W. Stewart. Error and perturbation bounds for subspaces associated with certain eigenvalue problems. *SIAM Rev.*, 15(4):727–764, 1973.
- [1065] G. W. Stewart. *Introduction to Matrix Computations*. Academic Press, New York, 1973. xiii+441 pp. ISBN 0-12-670350-7.
- [1066] G. W. Stewart. Modifying pivot elements in Gaussian elimination. *Math. Comp.*, 28(126):537–542, 1974.
- [1067] G. W. Stewart. On the perturbation of pseudo-inverses, projections and linear least squares problems. *SIAM Rev.*, 19(4):634–662, 1977.
- [1068] G. W. Stewart. Perturbation bounds for the QR factorization of a matrix. *SIAM J. Numer. Anal.*, 14(3):509–518, 1977.
- [1069] G. W. Stewart. Research, development, and LINPACK. In *Mathematical Software III*, John R. Rice, editor, Academic Press, New York, 1977, pages 1–14.

- [1070] G. W. Stewart. The efficient generation of random orthogonal matrices with an application to condition estimators. *SIAM J. Numer. Anal.*, 17(3):403–409, 1980.
- [1071] G. W. Stewart. A note on complex division. *ACM Trans. Math. Software*, 11(3): 238–241, 1985.
- [1072] G. W. Stewart. Stochastic perturbation theory. *SIAM Rev.*, 32(4):579–610, 1990.
- [1073] G. W. Stewart. Note on a generalized Sylvester equation. IMA Preprint Series #985, Institute for Mathematics and Its Applications, University of Minnesota, Minneapolis, MN, USA, May 1992. 3 pp.
- [1074] G. W. Stewart. On the early history of the singular value decomposition. *SIAM Rev.*, 35(4):551–566, 1993.
- [1075] G. W. Stewart. On the perturbation of LU, Cholesky, and QR factorizations. *SIAM J. Matrix Anal. Appl.*, 14(4):1141–1145, 1993.
- [1076] G. W. Stewart. Gauss, statistics, and Gaussian elimination. *Journal of Computational and Graphical Statistics*, 4(1):1–11, 1995.
- [1077] G. W. Stewart. On Markov chains with sluggish transients. *Communication in Statistics, Stochastic Models*, 13:85–95, 1997.
- [1078] G. W. Stewart. On the perturbation of LU and Cholesky factors. *IMA J. Numer. Anal.*, 17(1):1–6, 1997.
- [1079] G. W. Stewart. The triangular matrices of Gaussian elimination and related decompositions. *IMA J. Numer. Anal.*, 17(1):7–16, 1997.
- [1080] G. W. Stewart. *Matrix Algorithms. Volume I: Basic Decompositions*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1998. xx+458 pp. ISBN 0-89871-414-1.
- [1081] G. W. Stewart. On the adjugate matrix. *Linear Algebra Appl.*, 283:151–164, 1998.
- [1082] G. W. Stewart. *Matrix Algorithms. Volume II: Eigensystems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2001. xix+469 pp. ISBN 0-89871-503-2.
- [1083] G. W. Stewart and Ji-guang Sun. *Matrix Perturbation Theory*. Academic Press, London, 1990. xv+365 pp. ISBN 0-12-670230-6.
- [1084] Michael Stewart. Cholesky factorization of semidefinite Toeplitz matrices. *Linear Algebra Appl.*, 254:497–525, 1997.
- [1085] Josef Stoer. *Einführung in die Numerische Mathematik I*. Springer-Verlag, Berlin, 1972. ix+250 pp. ISBN 0-387-05750-1.
- [1086] Josef Stoer and R. Bulirsch. *Introduction to Numerical Analysis*. Springer-Verlag, New York, 1980. ix+609 pp. ISBN 0-387-90420-4.
- [1087] Josef Stoer and C. Witzgall. Transformations by diagonal matrices in a normed space. *Numer. Math.*, 4:158–171, 1962.
- [1088] Betty Jane Stone. Best possible ratios of certain matrix norms. *Numer. Math.*, 4:114–116, 1962.
- [1089] David R. Stoutemyer. Automatic error analysis using computer algebraic manipulation. *ACM Trans. Math. Software*, 3(1):26–43, 1977.
- [1090] David R. Stoutemyer. Crimes and misdemeanours in the computer algebra trade. *Notices Amer. Math. Soc.*, 38(7):778–785, 1991.
- [1091] Gilbert Strang. A proposal for Toeplitz matrix calculations. *Studies in Applied Mathematics*, 74:171–176, 1986.

- [1092] Gilbert Strang. *Introduction to Linear Algebra*. Wellesley-Cambridge Press, Wellesley, MA, USA, 1993. viii+472 pp. ISBN 0-9614088-5-5.
- [1093] V. Strassen. Gaussian elimination is not optimal. *Numer. Math.*, 13:354–356, 1969.
- [1094] F. Stummel. Rounding error analysis of elementary numerical algorithms. *Computing*, Suppl. 2:169–195, 1980.
- [1095] F. Stummel. Perturbation theory for evaluation algorithms of arithmetic expressions. *Math. Comp.*, 37(156):435–473, 1981.
- [1096] F. Stummel. Optimal error estimates for Gaussian elimination in floating-point arithmetic. *Z. Angew. Math. Mech.*, 62:T355–T357, 1982.
- [1097] F. Stummel. Strict optimal error estimates for Gaussian elimination. *Z. Angew. Math. Mech.*, 65:T405–T407, 1985.
- [1098] Friedrich Stummel. Forward error analysis of Gaussian elimination, Part I: Error and residual estimates. *Numer. Math.*, 46:365–395, 1985.
- [1099] Friedrich Stummel. Forward error analysis of Gaussian elimination, Part II: Stability theorems. *Numer. Math.*, 46:397–415, 1985.
- [1100] *SPARCompiler FORTRAN: Numerical Computation Guide*. Sun Microsystems, Inc., Mountain View, CA, USA, October 1992. Part No. 800-7097-11, Revision A.
- [1101] *SPARCompiler FORTRAN 2.0.1: User's Guide*. Sun Microsystems, Inc., Mountain View, CA, USA, October 1992. Part No. 800-6552-11, Revision A.
- [1102] Ji-guang Sun. Perturbation bounds for the Cholesky and QR factorizations. *BIT*, 31:341–352, 1991.
- [1103] Ji-guang Sun. Componentwise perturbation bounds for some matrix decompositions. *BIT*, 32(4):702–714, 1992.
- [1104] Ji-guang Sun. Rounding-error and perturbation bounds for the Cholesky and LDL^T factorizations. *Linear Algebra Appl.*, 173:77–97, 1992.
- [1105] Ji-guang Sun. A note on backward perturbations for the Hermitian eigenvalue problem. *BIT*, 35:385–393, 1995.
- [1106] Ji-guang Sun. On perturbation bounds for the QR factorization. *Linear Algebra Appl.*, 215:95–111, 1995.
- [1107] Ji-guang Sun. Optimal backward perturbation bounds for the linear least-squares problem with multiple right-hand sides. *IMA J. Numer. Anal.*, 16(1):1–11, 1996.
- [1108] Ji-guang Sun. On optimal backward perturbation bounds for the linear least squares problem. *BIT*, 37(1):179–188, 1997.
- [1109] Ji-guang Sun. Bounds for the structured backward errors of Vandermonde systems. *SIAM J. Matrix Anal. Appl.*, 20(1):45–59, 1998.
- [1110] Ji-guang Sun and Zheng Sun. Optimal backward perturbation bounds for under-determined systems. *SIAM J. Matrix Anal. Appl.*, 18(2):393–402, 1997.
- [1111] Xiaobai Sun and Christian H. Bischof. A basis-kernel representation of orthogonal matrices. *SIAM J. Matrix Anal. Appl.*, 16(4):1184–1196, 1995.
- [1112] W. H. Swann. Direct search methods. In *Numerical Methods for Unconstrained Optimization*, W. Murray, editor, Academic Press, London, 1972, pages 13–28.

- [1113] W. H. Swann. Constrained optimization by direct search. In *Numerical Methods for Constrained Optimization*, P. E. Gill and W. Murray, editors, Academic Press, London, 1974, pages 191–217.
- [1114] Earl E. Swartzlander, Jr., editor. *Computer Arithmetic*, volume 21 of *Benchmark Papers in Electrical Engineering and Computer Science*. Dowden, Hutchinson and Ross, Stroudsburg, PA, USA, 1980.
- [1115] Earl E. Swartzlander, Jr. and Aristides G. Alexopolous. The sign/logarithm number system. *IEEE Trans. Comput.*, C-24(12):1238–1242, 1975.
- [1116] D. W. Sweeney. An analysis of floating-point addition. *IBM Systems Journal*, 4: 31–42, 1965. Reprinted in [1114, pp. 317–328].
- [1117] J. J. Sylvester. Additions to the articles, “On a New Class of Theorems,” and “On Pascal’s Theorem”. *Philosophical Magazine*, 37:363–370, 1850. Reprinted in [1120, pp. 1451–151].
- [1118] J. J. Sylvester. On the relation between the minor determinants of linearly equivalent quadratic functions. *Philosophical Magazine*, (Fourth Series) 1:295–305, 1851. Reprinted in [1120, pp. 241–250].
- [1119] J. J. Sylvester. Sur l’équation en matrices $px = xq$. *Comptes Rendus de l’Académie des Sciences*, pages 67–71 and 115–116, 1884.
- [1120] *The Collected Mathematical Papers of James Joseph Sylvester*, volume 1 (1837–1853). Cambridge University Press, 1904. xii+650 pp.
- [1121] Ping Tak Peter Tang. Table-driven implementation of the exponential function in IEEE floating-point arithmetic. *ACM Trans. Math. Software*, 15(2):144–157, 1989.
- [1122] Ping Tak Peter Tang. Accurate and efficient testing of the exponential and logarithm functions. *ACM Trans. Math. Software*, 16(3):185–200, 1990.
- [1123] Ping Tak Peter Tang. Table-driven implementation of the logarithm function in IEEE floating-point arithmetic. *ACM Trans. Math. Software*, 16(3):378–400, 1990.
- [1124] Ping Tak Peter Tang. Table-lookup algorithms for elementary functions and their error analysis. In *Proc. 10th IEEE Symposium on Computer Arithmetic*, Peter Kornerup and David W. Matula, editors, IEEE Computer Society Press, Los Alamitos, CA, USA, 1991, pages 232–236.
- [1125] Ping Tak Peter Tang. Table-driven implementation of the $\exp m1$ function in IEEE floating-point arithmetic. *ACM Trans. Math. Software*, 18(2):211–222, 1992.
- [1126] W. P. Tang and G. H. Golub. The block decomposition of a Vandermonde matrix and its applications. *BIT*, 21:505–517, 1981.
- [1127] Pham Dinh Tao. Convergence of a subgradient method for computing the bound norm of matrices. *Linear Algebra Appl.*, 62:163–182, 1984. In French.
- [1128] Pham Dinh Tao. Some methods for computing the maximum of quadratic form on the unit ball of the maximum norm. *Numer. Math.*, 45:377–401, 1984. In French.
- [1129] Manfred Tasche and Hansmartin Zeuner. Worst and average case roundoff error analysis for FFT. *BIT*, 41(3):563–581, 2001.
- [1130] A. H. Taub, editor. *John von Neumann Collected Works*, volume V, Design of Computers, Theory of Automata and Numerical Analysis. Pergamon, Oxford, 1963. ix+784 pp.

- [1131] Olga Taussky. A remark concerning the characteristic roots of the finite segments of the Hilbert matrix. *Quarterly Journal of Mathematics*, 20:80–83, 1949.
- [1132] Olga Taussky. How I became a torchbearer for matrix theory. *Amer. Math. Monthly*, 95(9):801–812, 1988.
- [1133] Olga Taussky and Marvin Marcus. Eigenvalues of finite matrices. In *Survey of Numerical Analysis*, John Todd, editor, McGraw-Hill, New York, 1962, pages 279–313.
- [1134] Henry C. Thacher, Jr. Algorithm 43: Crout with pivoting II. *Comm. ACM*, 4(4):176–177, 1961.
- [1135] Ronald A. Thisted. *Elements of Statistical Computing: Numerical Computation*. Chapman and Hall, New York, 1988. xx+427 pp. ISBN 0-412-01371-1.
- [1136] D'Arcy Wentworth Thompson. *On Growth and Form. The Complete Revised Edition*. Cambridge University Press, 1942. viii+1116 pp. Reprinted by Dover, New York, 1992. ISBN 0-486-67135-6.
- [1137] Martti Tienari. A statistical model of roundoff error for varying length floating-point arithmetic. *BIT*, 10:355–365, 1970.
- [1138] Françoise Tisseur. Newton's method in floating point arithmetic and iterative refinement of generalized eigenvalue problems. *SIAM J. Matrix Anal. Appl.*, 22(4):1038–1057, 2001.
- [1139] Françoise Tisseur and Karl Meerbergen. The quadratic eigenvalue problem. *SIAM Rev.*, 43(2):235–286, 2001.
- [1140] J. Todd. On condition numbers. In *Programmation en Mathématiques Numériques, Besançon, 1966*, volume 7 (no. 165) of *Éditions Centre Nat. Recherche Sci., Paris*, 1968, pages 141–159.
- [1141] John Todd. The condition of the finite segments of the Hilbert matrix. In *Contributions to the Solution of Systems of Linear Equations and the Determination of Eigenvalues*, Olga Taussky, editor, number 39 in Applied Mathematics Series, National Bureau of Standards, United States Department of Commerce, Washington, D. C., 1954, pages 109–116.
- [1142] John Todd. Computational problems concerning the Hilbert matrix. *J. Res. Nat. Bur. Standards-B*, 65(1):19–22, 1961.
- [1143] John Todd. *Basic Numerical Mathematics, Vol. 2: Numerical Algebra*. Birkhäuser, Basel, and Academic Press, New York, 1977. 216 pp. ISBN 0-12-692402-3.
- [1144] Kim-Chuan Toh and Lloyd N. Trefethen. Pseudozeros of polynomials and pseudospectra of companion matrices. *Numer. Math.*, 68(3):403–425, 1994.
- [1145] Sivan Toledo. Locality of reference in LU decomposition with partial pivoting. *SIAM J. Matrix Anal. Appl.*, 18(4):1065–1081, 1997.
- [1146] Virginia J. Torczon. *Multi-Directional Search: A Direct Search Algorithm for Parallel Machines*. PhD thesis, Rice University, Houston, TX, USA, May 1989. vii+85 pp.
- [1147] Virginia J. Torczon. On the convergence of the multidirectional search algorithm. *SIAM J. Optim.*, 1(1):123–145, 1991.
- [1148] Virginia J. Torczon. On the convergence of pattern search algorithms. *SIAM J. Optim.*, 7(1):1–25, 1997.

- [1149] L. Tornheim. Maximum third pivot for Gaussian reduction. Technical report, Calif. Research Corp., Richmond, CA, 1965. Cited in [282].
- [1150] J. F. Traub. Associated polynomials and uniform methods for the solution of linear problems. *SIAM Rev.*, 8(3):277–301, 1966.
- [1151] Lloyd N. Trefethen. Three mysteries of Gaussian elimination. *ACM SIGNUM Newsletter*, 20:2–5, 1985.
- [1152] Lloyd N. Trefethen. Approximation theory and numerical linear algebra. In *Algorithms for Approximation II*, J. C. Mason and M. G. Cox, editors, Chapman and Hall, London, 1990, pages 336–360.
- [1153] Lloyd N. Trefethen. The definition of numerical analysis. *SIAM News*, 25:6 and 22, 1992. Reprinted in *IMA Bulletin*, 29 (3/4), pp. 47–49, 1993.
- [1154] Lloyd N. Trefethen. Pseudospectra of matrices. In *Numerical Analysis 1991, Proceedings of the 14th Dundee Conference*, D. F. Griffiths and G. A. Watson, editors, volume 260 of *Pitman Research Notes in Mathematics*, Longman Scientific and Technical, Essex, UK, 1992, pages 234–266.
- [1155] Lloyd N. Trefethen. Spectra and pseudospectra. In *The Graduate Student's Guide to Numerical Analysis '98*, Mark Ainsworth, Jeremy Levesley, and Marco Marletta, editors, Springer-Verlag, Berlin, 1999, pages 217–250.
- [1156] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. xii+361 pp. ISBN 0-89871-361-7.
- [1157] Lloyd N. Trefethen and Robert S. Schreiber. Average-case stability of Gaussian elimination. *SIAM J. Matrix Anal. Appl.*, 11(3):335–360, 1990.
- [1158] Lloyd N. Trefethen and Manfred R. Trummer. An instability phenomenon in spectral methods. *SIAM J. Numer. Anal.*, 24(5):1008–1023, 1987.
- [1159] Henry S. Tropp. FORTRAN anecdotes. *Annals of the History of Computing*, 6 (1):59–64, 1984.
- [1160] Henry S. Tropp. Origin of the term *bit*. *Annals of the History of Computing*, 6 (2):152–155, 1984.
- [1161] Nai-kuan Tsao. A note on implementing the Householder transformation. *SIAM J. Numer. Anal.*, 12(1):53–58, 1975.
- [1162] Nai-kuan Tsao. A simple approach to the error analysis of division-free numerical algorithms. *IEEE Trans. Comput.*, C-32(4):343–351, 1983.
- [1163] Warwick Tucker. A rigorous ODE solver and Smale's 14th problem. *Found. Comput. Math.*, 2(1):53–117, 2002.
- [1164] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Math. Soc.*, 42:230–265, 1936.
- [1165] A. M. Turing. Proposal for development in the Mathematics Division of an Automatic Computing Engine (ACE). Report E.882, Executive Committee, National Physical Laboratory, Teddington, UK, 1945. Reprinted in [204, pp. 20–105] and [663, pp. 1–86].
- [1166] A. M. Turing. Rounding-off errors in matrix processes. *Quart. J. Mech. Appl. Math.*, 1:287–308, 1948. Reprinted in [166] with summary and notes (including corrections).
- [1167] H. W. Turnbull. *The Theory of Determinants, Matrices, and Invariants*. Blackie, London and Glasgow, 1929. xvi+338 pp.

- [1168] H. W. Turnbull and A. C. Aitken. *An Introduction to the Theory of Canonical Matrices*. Blackie, London and Glasgow, 1932. xiii+200 pp. Reprinted with appendix, 1952.
- [1169] Peter R. Turner. The distribution of leading significant digits. *IMA J. Numer. Anal.*, 2:407–412, 1982.
- [1170] Peter R. Turner. Further revelations on L.S.D. *IMA J. Numer. Anal.*, 4:225–231, 1984.
- [1171] Peter R. Turner. Will the “real” real arithmetic please stand up? *Notices Amer. Math. Soc.*, 38(4):298–304, 1991.
- [1172] Evgenij E. Tyrtysnikov. Cauchy-Toeplitz matrices and some applications. *Linear Algebra Appl.*, 149:1–18, 1991.
- [1173] Patriot missile defense: Software problem led to system failure at Dhahran, Saudi Arabia. Report GAO/IMTEC-92-26, Information Management and Technology Division, United States General Accounting Office, Washington, D.C., February 1992. 16 pp.
- [1174] Minoru Urabe. Roundoff error distribution in fixed-point multiplication and a remark about the rounding rule. *SIAM J. Numer. Anal.*, 5(2):202–210, 1968.
- [1175] J. V. Uspensky. *Theory of Equations*. McGraw-Hill, New York, 1948. vii+353 pp.
- [1176] A. van der Sluis. Condition numbers and equilibration of matrices. *Numer. Math.*, 14:14–23, 1969.
- [1177] A. van der Sluis. Condition, equilibration and pivoting in linear algebraic systems. *Numer. Math.*, 15:74–86, 1970.
- [1178] A. van der Sluis. Stability of the solutions of linear least squares problems. *Numer. Math.*, 23:241–254, 1975.
- [1179] Charles F. Van Loan. A note on the evaluation of matrix polynomials. *IEEE Trans. Automat. Control*, AC-24(2):320–321, 1979.
- [1180] Charles F. Van Loan. On the method of weighting for equality-constrained least-squares problems. *SIAM J. Numer. Anal.*, 22(5):851–864, 1985.
- [1181] Charles F. Van Loan. On estimating the condition of eigenvalues and eigenvectors. *Linear Algebra Appl.*, 88/89:715–732, 1987.
- [1182] Charles F. Van Loan. *Computational Frameworks for the Fast Fourier Transform*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1992. xiii+273 pp. ISBN 0-89871-285-8.
- [1183] Ursula van Rienen. *Numerical Methods in Computational Electrodynamics: Linear Systems in Practical Applications*. Number 12 in Lecture Notes in Computational Science and Engineering. Springer-Verlag, Berlin, 2001. xiii+375 pp. ISBN 3-540-67629-5.
- [1184] M. van Veldhuizen. A note on partial pivoting and Gaussian elimination. *Numer. Math.*, 29:1–10, 1977.
- [1185] A. van Wijngaarden. Numerical analysis as an independent science. *BIT*, 6:66–81, 1966.
- [1186] Robert J. Vanderbei. Symmetric quasi-definite matrices. *SIAM J. Optim.*, 5(1):100–113, 1995.
- [1187] J. M. Varah. On the solution of block-tridiagonal systems arising from certain finite-difference equations. *Math. Comp.*, 26(120):859–868, 1972.

- [1188] J. M. Varah. A lower bound for the smallest singular value of a matrix. *Linear Algebra Appl.*, 11:3–5, 1975.
- [1189] J. M. Varah. On the separation of two matrices. *SIAM J. Numer. Anal.*, 16(2): 216–222, 1979.
- [1190] Richard S. Varga. On diagonal dominance arguments for bounding $\|A^{-1}\|_{\infty}$. *Linear Algebra Appl.*, 14:211–217, 1976.
- [1191] Richard S. Varga. *Scientific Computation on Mathematical Problems and Conjectures*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1990. vi+122 pp. ISBN 0-89871-257-2.
- [1192] S. A. Vavasis. Gaussian elimination with partial pivoting is P-complete. *SIAM J. Disc. Math.*, 2:413–423, 1989.
- [1193] Brigitte Verdonk, Annie Cuyt, and Dennis Verschaeren. A precision and range independent tool for testing floating-point arithmetic I: Basic operations, square root and remainder. *ACM Trans. Math. Software*, 27(1):92–118, 2001.
- [1194] Brigitte Verdonk, Annie Cuyt, and Dennis Verschaeren. A precision and range independent tool for testing floating-point arithmetic II: Conversions. *ACM Trans. Math. Software*, 27(1):119–140, 2001.
- [1195] Frank M. Verzuh. The solution of simultaneous linear equations with the aid of the 602 calculating punch. *M.T.A.C.*, 3:453–462, 1949.
- [1196] J. Vignes and R. Alt. An efficient stochastic method for round-off error analysis. In *Accurate Scientific Computations, Proceedings, 1985*, Willard L. Miranker and Richard A. Toupin, editors, volume 235 of *Lecture Notes in Computer Science*, Springer-Verlag, Berlin, 1986, pages 183–205.
- [1197] D. Viswanath and L. N. Trefethen. Condition numbers of random triangular matrices. *SIAM J. Matrix Anal. Appl.*, 19(2):564–581, 1998.
- [1198] Emil Vitasek. The numerical stability in solution of differential equations. In *Conference on the Numerical Solution of Differential Equations*, J. Ll. Morris, editor, volume 109 of *Lecture Notes in Mathematics*, Springer-Verlag, Berlin, 1969, pages 87–111.
- [1199] I. V. Viten'ko. Optimum algorithms for adding and multiplying on computers with a floating point. *U.S.S.R. Computational Math. and Math. Phys.*, 8(5):183–195, 1968.
- [1200] John von Neumann and Herman H. Goldstine. Numerical inverting of matrices of high order. *Bull. Amer. Math. Soc.*, 53:1021–1099, 1947. Reprinted in [1130, pp. 479–557].
- [1201] H. A. Van Der Vorst. The convergence behaviour of preconditioned CG and CG-S in the presence of rounding errors. In *Preconditioned Conjugate Gradient Methods*, Owe Axelsson and Lily Yu. Kolotilina, editors, volume 1457 of *Lecture Notes in Mathematics*, Springer-Verlag, Berlin, 1990, pages 126–136.
- [1202] Eugene L. Wachpress. Iterative solution of the Lyapunov matrix equation. *Appl. Math. Lett.*, 1(1):87–90, 1988.
- [1203] Bertil Waldén, Rune Karlson, and Ji-guang Sun. Optimal backward perturbation bounds for the linear least squares problem. *Numerical Linear Algebra with Applications*, 2(3):271–286, 1995.
- [1204] Peter J. L. Wallis, editor. *Improving Floating-Point Programming*. Wiley, London, 1990. xvi+191 pp. ISBN 0-471-92437-7.

- [1205] W. D. Wallis. Hadamard matrices. In *Combinatorial and Graph-Theoretical Problems in Linear Algebra*, Richard A. Brualdi, Shmuel Friedland, and Victor Klee, editors, volume 50 of *IMA Volumes in Mathematics and Its Applications*, Springer-Verlag, New York, 1993, pages 235–243.
- [1206] W. D. Wallis, Anne Penfold Street, and Jennifer Seberry Wallis. *Combinatorics: Room Squares, Sum-Free Sets, Hadamard Matrices*, volume 292 of *Lecture Notes in Mathematics*. Springer-Verlag, Berlin, 1972. 508 pp. ISBN 3-540-06035-9.
- [1207] Robert C. Ward. The QR algorithm and Hyman's method on vector computers. *Math. Comp.*, 30(133):132–142, 1976.
- [1208] Willis H. Ware, editor. Soviet computer technology—1959. *Comm. ACM*, 3(3): 131–166, 1960.
- [1209] G. A. Watson. An algorithm for optimal ℓ_2 scaling of matrices. *IMA J. Numer. Anal.*, 11:481–492, 1991.
- [1210] J. H. M. Wedderburn. *Lectures on Matrices*, volume 17 of *American Mathematical Society Colloquium Publications*. American Mathematical Society, Providence, RI, USA, 1934. vii+205 pp.
- [1211] Per-Åke Wedin. Perturbation theory for pseudo-inverses. *BIT*, 13:217–232, 1973.
- [1212] Elias Wegert and Lloyd N. Trefethen. From the Buffon needle problem to the Kreiss matrix theorem. *Amer. Math. Monthly*, 101(2):132–139, 1994.
- [1213] Musheng Wei. Perturbation of the least squares problem. *Linear Algebra Appl.*, 141:177–182, 1990.
- [1214] N. Weiss, G. W. Wasilkowski, H. Woźniakowski, and M. Shub. Average condition number for solving linear equations. *Linear Algebra Appl.*, 83:79–102, 1986.
- [1215] Burton Wendroff. *Theoretical Numerical Analysis*. Academic Press, New York, 1966. xi+239 pp.
- [1216] Wilhelm Werner. Polynomial interpolation: Lagrange versus Newton. *Math. Comp.*, 43(167):205–217, 1984.
- [1217] Joan R. Westlake. *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations*. Wiley, New York, 1968.
- [1218] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. Automated empirical optimization of software and the ATLAS project. *Parallel Comput.*, 27(1–2):3–25, 2001.
- [1219] Michael White. *Isaac Newton: The Last Sorcerer*. Fourth Estate, London, 1997. 402 pp. ISBN 1-85702-706-X.
- [1220] B. A. Wichmann. Towards a formal specification of floating point. *Comput. J.*, 32:432–436, 1989.
- [1221] B. A. Wichmann. A note on the use of floating point in critical systems. *Comput. J.*, 35(1):41–44, 1992.
- [1222] M. V. Wilkes. Arithmetic on the EDSAC. *IEEE Annals of the History of Computing*, 19(1):13–15, 1997.
- [1223] J. H. Wilkinson. The Automatic Computing Engine at the National Physical Laboratory. *Proc. Roy. Soc. London Ser. A*, 195:285–286, 1948.
- [1224] J. H. Wilkinson. Progress report on the Automatic Computing Engine. Report MA/17/1024, Mathematics Division, Department of Scientific and Industrial Research, National Physical Laboratory, Teddington, UK, April 1948. 127 pp.

- [1225] J. H. Wilkinson. Linear algebra on the Pilot ACE. In *Automatic Digital Computation*, Her Majesty's Stationery Office, London, 1954. Reprinted in [1248, pp. 337–344].
- [1226] J. H. Wilkinson. The Pilot ACE. In *Automatic Digital Computation*, Her Majesty's Stationery Office, London, 1954, pages 5–14. Reprinted in [99, pp. 193–199] and [1248, pp. 219–228].
- [1227] J. H. Wilkinson. The use of iterative methods for finding the latent roots and vectors of matrices. *Mathematical Tables and Other Aids to Computation*, 9:184–191, 1955.
- [1228] J. H. Wilkinson. Error analysis of floating-point computation. *Numer. Math.*, 2: 319–340, 1960.
- [1229] J. H. Wilkinson. Error analysis of direct methods of matrix inversion. *J. Assoc. Comput. Mach.*, 8:281–330, 1961.
- [1230] J. H. Wilkinson. Error analysis of eigenvalue techniques based on orthogonal transformations. *J. Soc. Indust. Appl. Math.*, 10(1):162–195, 1962.
- [1231] J. H. Wilkinson. Plane-rotations in floating-point arithmetic. In *Experimental Arithmetic, High Speed Computing and Mathematics*, volume 15 of *Proceedings of Symposia in Applied Mathematics*, American Mathematical Society, Providence, RI, USA, 1963, pages 185–198.
- [1232] J. H. Wilkinson. *Rounding Errors in Algebraic Processes*. Notes on Applied Science No. 32, Her Majesty's Stationery Office, London, 1963. vi+161 pp. Also published by Prentice-Hall, Englewood Cliffs, NJ, USA. Reprinted by Dover, New York, 1994. ISBN 0-486-67999-3.
- [1233] J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Oxford University Press, 1965. xviii+662 pp. ISBN 0-19-853403-5 (hardback), 0-19-853418-3 (paperback).
- [1234] J. H. Wilkinson. Error analysis of transformations based on the use of matrices of the form $I - 2ww^H$. In *Error in Digital Computation*, Louis B. Rall, editor, volume 2, Wiley, New York, 1965, pages 77–101.
- [1235] J. H. Wilkinson. *Błędy Zaokrąglenia w Procesach Algebraicznych*. PWW, Warszawa, 1967. Polish translation of [1232].
- [1236] J. H. Wilkinson. A priori error analysis of algebraic processes. In *Proc. International Congress of Mathematicians, Moscow 1966*, I. G. Petrovsky, editor, Mir Publishers, Moscow, 1968, pages 629–640.
- [1237] J. H. Wilkinson. *Rundungsfehler*. Springer-Verlag, Berlin, 1969. German translation of [1232].
- [1238] J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Nauka, U.S.S.R. Academy of Sciences, 1970. 564 pp. Russian translation of [1233].
- [1239] J. H. Wilkinson. Modern error analysis. *SIAM Rev.*, 13(4):548–568, 1971.
- [1240] J. H. Wilkinson. Some comments from a numerical analyst. *J. Assoc. Comput. Mach.*, 18(2):137–147, 1971. Reprinted in [3].
- [1241] J. H. Wilkinson. The classical error analyses for the solution of linear systems. *IMA Bulletin*, 10(5/6):175–180, 1974.
- [1242] J. H. Wilkinson. Numerical linear algebra on digital computers. *IMA Bulletin*, 10(9/10):354–356, 1974.

- [1243] J. H. Wilkinson. Turing's work at the National Physical Laboratory and the construction of Pilot ACE, DEUCE, and ACE. In *A History of Computing in the Twentieth Century: A Collection of Essays*, N. Metropolis, J. Howlett, and Gian-Carlo Rota, editors, Academic Press, New York, 1980, pages 101–114.
- [1244] J. H. Wilkinson. The state of the art in error analysis. In *NAG Newsletter 2/85*, Numerical Algorithms Group, Oxford, UK, November 1985, pages 5–28.
- [1245] J. H. Wilkinson. Error analysis revisited. *IMA Bulletin*, 22(11/12):192–200, 1986.
- [1246] J. H. Wilkinson and C. Reinsch, editors. *Linear Algebra*, volume II of *Handbook for Automatic Computation*. Springer-Verlag, Berlin, 1971. ix+439 pp. ISBN 3-540-05414-6.
- [1247] James H. Wilkinson. The perfidious polynomial. In *Studies in Numerical Analysis*, G. H. Golub, editor, volume 24 of *Studies in Mathematics*, Mathematical Association of America, Washington, D.C., 1984, pages 1–28.
- [1248] M. R. Williams and Martin Campbell-Kelly, editors. *The Early British Computer Conferences*, volume 14 of *Charles Babbage Institute Reprint Series for the History of Computing*. MIT Press, Cambridge, MA, USA, 1989. xvi+508 pp. ISBN 0-262-23136-0.
- [1249] S. Winograd. A new algorithm for inner product. *IEEE Trans. Comput.*, C-18: 693–694, 1968.
- [1250] S. Winograd. On multiplication of 2×2 matrices. *Linear Algebra Appl.*, 4:381–388, 1971.
- [1251] Jack Woehr. A conversation with William Kahan. *Dr. Dobb's Journal*, 271:18–32, 1997.
- [1252] Jack M. Wolfe. Reducing truncation errors by programming. *Comm. ACM*, 7(6): 355–356, 1964.
- [1253] Stephen Wolfram. *The Mathematica Book*. Fourth edition, Wolfram Media, Champaign, IL, USA and Cambridge University Press, Cambridge, UK, 1999. xxvi+1470 pp. ISBN 1-57955-004-5 (Wolfram), 0-521-64314-7 (Cambridge).
- [1254] Michael Woodger. The history and present use of digital computers at the National Physical Laboratory. *Process Control and Automation*, pages 437–442, 1958. Reprinted in [204, pp. 125–140].
- [1255] H. Woźniakowski. Numerical stability for solving nonlinear equations. *Numer. Math.*, 27:373–390, 1977.
- [1256] H. Woźniakowski. Numerical stability of the Chebyshev method for the solution of large linear systems. *Numer. Math.*, 28:191–209, 1977.
- [1257] H. Woźniakowski. Roundoff-error analysis of iterations for large linear systems. *Numer. Math.*, 30:301–314, 1978.
- [1258] H. Woźniakowski. Roundoff-error analysis of a new class of conjugate-gradient algorithms. *Linear Algebra Appl.*, 29:507–529, 1980.
- [1259] Margaret H. Wright. Interior methods for constrained optimization. *Acta Numerica*, pages 341–407, 1992.
- [1260] Margaret H. Wright. Direct search methods: Once scorned, now respectable. In *Numerical Analysis 1995, Proceedings of the 16th Dundee Conference*, D. F. Griffiths and G. A. Watson, editors, volume 344 of *Pitman Research Notes in Mathematics*, Addison Wesley Longman, Harlow, Essex, UK, 1996, pages 191–208.

- [1261] Stephen J. Wright. A collection of problems for which Gaussian elimination with partial pivoting is unstable. *SIAM J. Sci. Statist. Comput.*, 14(1):231–238, 1993.
- [1262] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1997. xx+289 pp. ISBN 0-89871-382-X.
- [1263] P. Y. Yalamov. Graphs and stability of algorithms. Preprint N27, University of Rousse, Bulgaria, February 1995. 17 pp.
- [1264] Plamen Y. Yalamov. On the almost strong stability of the circular deconvolution algorithm. *SIAM J. Matrix Anal. Appl.*, 22(2):358–363, 2000.
- [1265] Man-Chung Yeung and Tony. F. Chan. Probabilistic analysis of Gaussian elimination without pivoting. *SIAM J. Matrix Anal. Appl.*, 18(2):499–517, 1997.
- [1266] E. L. Yip. A note on the stability of solving a rank- p modification of a linear system by the Sherman–Morrison–Woodbury formula. *SIAM J. Sci. Statist. Comput.*, 7(3):507–513, 1986.
- [1267] J. M. Yohe. Implementing nonstandard arithmetics. *SIAM Rev.*, 21(1):34–56, 1979.
- [1268] J. M. Yohe. Software for interval arithmetic: A reasonably portable package. *ACM Trans. Math. Software*, 5(1):50–63, 1979.
- [1269] J. M. Yohe. Portable software for interval arithmetic. *Computing*, Suppl. 2:211–229, 1980.
- [1270] David M. Young. *Iterative Solution of Large Linear Systems*. Academic Press, New York, 1971. xxiv+570 pp. ISBN 0-12-773050-8.
- [1271] David M. Young. A historical overview of iterative methods. *Computer Physics Communications*, 53:1–17, 1989.
- [1272] David M. Young and Robert T. Gregory. *A Survey of Numerical Mathematics*, volume 1. Addison-Wesley, Reading, MA, USA, 1972. x+492+appendices pp. Reprinted by Dover, New York, 1988. ISBN 0-486-65691-8.
- [1273] Tjalling J. Ypma. The effect of rounding errors on Newton-like methods. *IMA J. Numer. Anal.*, 3:109–118, 1983.
- [1274] Tjalling J. Ypma. Finding a multiple zero by transformations and Newton-like methods. *SIAM Rev.*, 25(3):365–378, 1983.
- [1275] Tjalling J. Ypma. Historical development of the Newton–Raphson method. *SIAM Rev.*, 37(4):531–551, 1995.
- [1276] Gideon Yuval. A simple proof of Strassen’s result. *Inform. Process. Lett.*, 7(6):285–286, 1978.
- [1277] Adam T. Zawilski. Numerical stability of the cyclic Richardson iteration. *Numer. Math.*, 60:251–290, 1991.
- [1278] Hongyuan Zha. A componentwise perturbation analysis of the QR decomposition. *SIAM J. Matrix Anal. Appl.*, 14(4):1124–1131, 1993.
- [1279] Hongyuan Zha. Problem 10312. *Amer. Math. Monthly*, 100(5):499, 1993.
- [1280] Hongyuan Zha and Zhenyue Zhang. A note on constructing a symmetric matrix with specified diagonal entries and eigenvalues. *BIT*, 35:448–452, 1995.
- [1281] G. Zielke. Report on test matrices for generalized inverses. *Computing*, 36:105–162, 1986.

- [1282] Gerhard Zielke. Some remarks on matrix norms, condition numbers, and error estimates for linear equations. *Linear Algebra Appl.*, 110:29–41, 1988.
- [1283] K. Ziętak. On a particular case of the inconsistent linear matrix equation $AX + YB = C$. *Linear Algebra Appl.*, 66:249–258, 1985.
- [1284] Abraham Ziv. Relative distance—An error measure in round-off error analysis. *Math. Comp.*, 39(160):563–569, 1982.
- [1285] Abraham Ziv. Fast evaluation of elementary mathematical functions with correctly rounded last bit. *ACM Trans. Math. Software*, 17(3):410–423, 1991.
- [1286] Abraham Ziv. Converting approximate error bounds into exact ones. *Math. Comp.*, 64(209):265–277, 1995.
- [1287] Zahari Zlatev, Jerzy Wasniewski, and Kjeld Schaumburg. Condition number estimators in a sparse matrix software. *SIAM J. Sci. Statist. Comput.*, 7(4):1175–1189, 1986.

Name Index

Science is organized knowledge.

— HERBERT SPENCER, *Essays On Education* (1861)

A suffix “t” after a page number denotes a table, “f” a figure, “n” a footnote, and “q” a quotation.

- Aasen, Jan Ole, 214, 222, 224, 227
 Abdelmalek, Nabih N., 376, 403
 Abramowitz, Milton, 30
 Acton, Forman S., 30, 195 q, 283 q, 486
 Adams, Duane A., 103
 Aelfric, xxi q
 Aggarwal, Vijay B., 76
 Ahac, Alan A., 190
 Ahlberg, J. H., 154
 Aitken, Alexander Craig, 91, 374
 Albers, Donald J., 511 n
 Alefeld, Göltz, 483
 Alexopolous, Aristides G., 49
 Allen, Jr., Richard C., 76
 Almacany, Montaha, 415 q
 Alt, H., 447
 Aluru, Srinivas, 446
 Alvarado, Fernando L., 153
 Amato, James J., 317
 Amodio, Pierluigi, 190
 Anda, Andrew A., 376
 Anderson, Edward, 287 q, 397, 404
 Anderson, Iain J., 88
 Anderson, T. W., 524, 525
 Ando, T., 188
 Antonov, A. G., 285
 Arioli, Mario, 130, 301, 324, 337, 402–404, 413, 414
 Arnold, William F., 318
 Ashcraft, Cleve, 188, 219, 227
 Ashenhurst, R. L., 486
 Asplund, Edgar, 302
 Atanasoff, John V., 139 q
 Axelsson, Owe, 337
 Babuška, Ivo, 90
 Bachelis, Boris, 50
 Bai, Zhaojun, 317, 346, 397, 404, 524
 Bailey, David H., 436, 446–448, 457, 478, 489 q, 501, 502
 Baksalary, J. K., 318
 Balle, Susanne M., 448
 Ballester, C., 429
 Bank, Randolph E., 257
 Bareiss, E. H., 49, 189
 Bargmann, V., 183
 Barlow, Jesse L., 29, 47, 49, 181, 189, 299, 376, 397
 Barnett, S., 317
 Barone, John L., 381 q, 402
 Barrett, Geoff, 56
 Barrlund, Anders, 181, 190, 209, 405
 Bartels, R. H., 307
 Bartels, Sven G., 129, 301, 429
 Bau III, David, 169, 337
 Bauer, F. L., 53, 76, 107, 114, 127, 133, 135, 177, 281, 547
 Beam, Richard M., 525
 Beaton, Albert E., 381 q, 402
 Bell, E. T., 279 q
 Bellman, Richard, 318
 Benford, Frank, 47
 Benôit, Commandant, 209
 Benschop, N. F., 323
 Berman, Abraham, 133, 152
 Bhatia, Rajendra, 317, 374
 Bini, Dario, 429, 443, 444
 Birkhoff, Garrett, 279, 479
 Bischof, Christian H., 297–299, 363, 376

- Björck, Åke, 76, 241, 353 q, 371, 375–377, 379, 380, 386, 388, 389, 391, 391 q, 392, 399, 402–405, 413, 414, 423, 423 q, 425, 429, 565
 Bjørstad, Petter, 448
 Blanch, G., 505
 Bliss, B., 485
 Blue, James L., 499
 Bodewig, E., 185
 Bohlender, Gerd, 88
 Bohte, Z., 173
 Boley, Daniel, 242
 Bollen, Jo A. M., 323
 Bondeli, S., 522
 Boros, T., 429
 Borwein, J. M., 489 q
 Borwein, P. B., 489 q
 Bowden, B. V., 195 q
 Bowdler, H. J., 188
 Boyd, David W., 289, 291, 301
 Boyle, Jeff, 47
 Brent, Richard P., 47, 150, 151, 436, 439, 440, 447, 483, 501, 505
 Brezinski, Claude, 209
 Briggs, William L., 456
 Brightman, Tom, 56
 Brown, W. S., 495, 498, 498 q
 Brunet, Marie-Christine, 485, 486
 Buchan, John, 57 q
 Buchanan, James L., 152
 Buchholz, W., 56
 Bukhberger, B., 302
 Bulirsch, R., 76, 187
 Bunch, James R., 129, 133, 136, 213 q, 215, 216, 221, 225–227, 231 q
 Buoni, John J., 190
 Burdakov, Oleg, 282
 Burgmeier, James W., 76
 Businger, Peter A., 133, 180, 403
 Butcher, J. C., 90
 Byers, Ralph, 302, 315, 318, 319, 346, 562
 Caffney, John, 518
 Calvetti, D., 428–430
 Calvin (and Hobbes), 471 q
 Campbell, S. L., 332
 Campbell-Kelly, Martin, 245 q
 Cao, Wei-Lu, 302
 Caprani, Ole, 90
 Cardano, Geronimo, 479
 Carr III, John W., 53
 Carter, Russell, 493, 494
 Cauchy, Augustin-Louis, 515
 Cayley, Arthur, 434
 Chaitin-Chatelin, Françoise, 351, 468, 485, 486
 Chan, N. N., 524
 Chan, Raymond H., 457
 Chan, Tony F., 11 q, 29, 133, 134, 189, 377
 Chandrasekaran, Shivkumar, 123, 129, 377
 Chang, Xiao-Wen, 190, 209, 377
 Chartres, Bruce A., 186
 Chatelin, Françoise, *see* Chaitin-Chatelin, Françoise
 Cheng, Sheung Hun, 219, 221, 224, 227, 295
 Cho, Choong Yun, 515
 Choi, Man-Duen, 511 q, 523
 Cholesky, André-Louis, 209
 Christiansen, Søren, 133
 Chu, Eleanor, 181
 Chu, King-wah Eric, 318
 Chu, Moody T., 525
 Clasen, B.-I., 281
 Clenshaw, C. W., 49, 102
 Cline, Alan K., 287 q, 295, 297, 404
 Cline, R. E., 413
 Clinger, William D., 57
 Codenotti, B., 282
 Cody, Jr., William J., 51, 55, 56, 493, 495, 496, 501 q
 Cohen, A. M., 169, 170, 520
 Concus, P., 257
 Conn, Andrew R., 297
 Conte, Samuel D., 187
 Cooley, James W., 456
 Coomes, Brian A., 29
 Coonen, Jerome T., 55, 493
 Cope, J. E., 132
 Coppersmith, Don, 436
 Cortés, Joaquín, 189
 Cottle, Richard W., 209
 Cox, Anthony J., 362, 375, 395, 396, 398, 400, 404, 405
 Crout, Prescott D., 187
 Cryer, Colin W., 169, 170, 188, 189
 Csanky, L., 278
 Curtis, A. R., 190
 Cuyt, Annie, 496

- Cybenko, George, 210
 Dahlquist, Germund, 76, 147
 Daniel, J. W., 376
 Datta, Karabi, 311
 Davies, Philip I., 525
 Davis, Philip J., 90, 457
 Dax, Achiya, 227, 332
 Day, David, 524
 Day, Jane M., 189
 de Boor, Carl, 186–188
 de Jong, Lieuwe Sytse, 29
 de Rijk, P. P. M., 402–404
 Dekker, T. J., 53, 84, 282, 501
 del Ferro, Scipione, 479
 Demeure, Cédric J., 429
 Demmel, James W., 42, 49, 56, 57, 77,
 90, 115, 128–130, 136, 152, 182,
 188, 191, 198–200, 209, 232,
 242, 249–253, 256, 257, 282,
 288, 301, 317, 337, 346, 409,
 413, 414, 483, 492, 496, 501,
 515, 524, 534
 Dennis, Jr., John E., 323, 468, 475, 476
 Descloux, J., 469
 Dhillon, Inderjit S., 56, 301, 304, 534
 Diamant, Benjamin, 288
 Diamond, Harold G., 57
 Dixon, John D., 298
 Dongarra, Jack J., 185, 187, 226, 231 q,
 232, 256, 397, 404, 496, 524,
 573 q
 Doolittle, Myrick Hascall, 187
 Dorn, William S., 76, 90, 102
 Douglas, Craig C., 446, 569
 Douglas, Jr., Jim, 185
 Doyle, Sir Arthur Conan, 287 q
 Drake, J. B., 259 q
 Drmač, Zlatko, 210
 Du Croz, Jeremy J., 262, 269, 281
 Dubrulle, Augustin A., 282, 507
 Duff, Iain S., 131, 187, 193, 226, 227,
 256, 301, 337, 402–404, 524
 Dumitrescu, Bogdan, 447
 Duncan, Martin, 87
 Dunham, C. B., 415 q
 Dwyer, Paul S., 157 q
 Eckart, Carl, 114
 Edelman, Alan, 59, 150, 169, 170, 189,
 191, 284, 377, 480 n, 516, 516 q,
 524, 532
 Egecioglu, Ömer, 103
 Eijkhout, Victor, 232
 Eirola, Timo, 29
 Eldén, Lars, 396, 404
 Eldersveld, Samuel K., 257
 Elfving, Tommy, 429
 Emel'yanenko, G. A., 302
 Enright, Wayne H., 29
 Erisman, A. M., 180, 181, 186, 193
 Espelid, Terje O., 90
 Faddeeva, V. N., 188
 Fairgrieve, Thomas F., 50, 528
 Fallat, Shaun M., 189
 Fan, Ky, 377
 Farebrother, R. W., 402
 Farkas, I., 103
 Farnum, Charles, 494, 494 q
 Fateman, Richard J., 55
 Feingold, David G., 257
 Feldstein, A., 57
 Ferguson, H. R. P., 448, 478
 Ferguson, Jr., Warren E., 45, 56
 Ferng, William R., 299
 Fike, C. T., 93 q, 104
 Fischer, Patrick C., 446
 Flannery, Brian P., 476, 487, 505
 Fletcher, Roger, 133, 136, 188, 210, 227
 Forsgren, Anders, 210, 226
 Forsythe, George E., 29, 29 q, 30, 48,
 53, 76, 86, 126, 133, 152, 186,
 188, 190, 235, 240, 242, 245 q,
 259 q, 260, 279, 302, 321 q, 489 q,
 523
 Foster, Leslie V., 159, 167, 170, 403
 Foulser, David E., 133, 134
 Fox, L., xxix q, xxx, 30, 31 q, 105 n, 184,
 186
 Francois, Philippe, 32
 Frayssé, Valérie, 351, 468, 486
 Friedland, Shmuel, 342, 352
 Frommer, Andreas, 482
 Funderlic, R. E., 190
 Gaches, J., 120, 132
 Gahinet, Pascal M., 318
 Gal, Shmuel, 50
 Gallivan, K. A., 180
 Gallopoulos, E., 103, 485
 Gander, Walter, 522
 Gantmacher, F. R., 161
 Gardiner, Judith D., 317, 318

- Gardner, Martin, 115 q
 Garner, Harvey L., 57
 Gasca, M., 180, 189
 Gastinel, Noel, 111, 114
 Gauss, Carl Friedrich, 1 q, 187, 215, 321 q,
 381, 456
 Gautschi, Walter, 415 q, 419
 Gautschi, Werner, 344
 Gay, David M., 57
 Geist, G. A., 259 q
 Geman, Stuart, 516
 Gentle, James E., 29
 Gentleman, W. Morven, 103, 375, 376,
 456, 570
 George, Alan, 181, 209
 Geuder, James C., 186
 Ghavimi, Ali R., 315–317
 Gill, Philip E., 210, 226, 227, 229, 413,
 476
 Gill, S., 83
 Givens, Wallace J., 29, 61 q
 Gluchowska, J., 404
 Godunov, S. K., 285
 Gohberg, I., 128, 129, 428
 Goldberg, David, 35 q, 53, 85, 532
 Goldberg, I. Bennett, 57
 Goldstine, Herman H., 1 n, 29, 48, 183,
 184, 188, 259 q, 260, 515
 Golub, Gene H., xxviii, 11 q, 24, 29, 115,
 132, 133, 172, 187, 208, 231 q,
 256, 257, 282, 299, 307, 308,
 308 n, 323, 345, 375, 377, 380–
 382, 389, 399, 402–405, 429
 Goodman, R., 57
 Goodnight, James H., 282
 Gould, Nicholas I. M., 170, 189
 Govaerts, Willy J. F., 241
 Gragg, W. B., 295, 376
 Graham, Ronald L., 79 q, 518 q
 Greenbaum, Anne, 324, 337
 Gregory, Robert T., 29, 512, 522
 Griewank, Andreas, 469
 Grimes, Roger G., 188, 219, 227, 302,
 524
 Grosse, Eric, 573 q
 Gu, Ming, 346, 404, 406, 534
 Gudmundsson, Thorkell, 298
 Guggenheimer, Heinrich W., 284
 Gulliksson, Mårten, 405
 Gustafson, John L., 446
 Gustavson, F. G., 187
 Hager, William W., 292, 301
 Hall, Jr., Marshall, 168
 Halmos, Paul R., 511 q
 Hamada, Hozumi, 49
 Hammarling, Sven J., 317, 322, 323, 376,
 397, 500
 Hamming, R. W., 53, 431
 Hansen, Per Christian, 133, 377, 448
 Hanson, Richard J., 375, 376, 399, 402,
 403, 414, 499
 Harter, Richard, 446
 Hartfiel, D. J., 132
 Hauser, John R., 57, 59
 Hearon, John Z., 311
 Heath, Michael T., 259 q, 388
 Hedayat, A. S., 168
 Hein, Piet, 31 q, 115 q
 Helvin, Marie, 153 q
 Henderson, Harold V., 317, 487
 Hennessy, John L., 35 n, 53
 Henrici, Peter, 48, 49, 61 q, 77, 344
 Henson, Van Emden, 456
 Heroux, Michael, 446, 569
 Herzberger, Jürgen, 483
 Hewer, Gary, 318
 Higham, Desmond J., 114, 128, 129, 136,
 167, 187, 339 q, 342, 394, 429
 Higham, Nicholas J., 65, 90, 128, 129,
 132, 151–153, 167, 187, 190,
 210, 219, 227, 228, 240–242,
 249–253, 256, 257, 262, 269,
 278, 281, 292, 294, 295, 298,
 301–304, 317, 324, 347, 349,
 351, 362, 375, 377, 378, 380,
 394–396, 398, 400, 402–405, 409,
 413, 414, 429, 431, 446–448,
 486, 505, 523, 525, 564, 569,
 571
 Hilbert, David, 523
 Hildebrand, F. B., 28, 35 q
 Hodel, A. Scottedward, 317
 Hodges, Andrew, xxix
 Hoffman, A. J., 377
 Hoffmann, Christoph M., 29
 Hoffmann, W., 281, 282, 376
 Hooper, Judith A., 501
 Horn, Roger A., 107, 115, 137, 152, 306,
 342, 543, 547, 551, 554
 Horning, Jim, 489 q
 Hotelling, Harold, 183, 433 q
 Hough, David, 43, 506, 507

- Hough, Patricia D., 395
 Householder, Alston S., 2, 105 q, 115, 147, 161, 374, 403, 560
 Hull, T. E., 48, 50, 485, 485 q, 503, 528
 Hunt, Julian, 54
 Huskey, H. D., 184
 Hyman, M. A., 280

 Ikebe, Yasuhiko, 300, 302, 304
 Ikramov, Khakim D., 209, 525
 Incertis, F., 509
 Ipsen, Ilse C. F., 123, 129, 133, 376, 377
 Iri, Masao, 49, 485
 Isaacson, Eugene, 186, 257

 Jalby, William, 376
 Jankowski, M., 85, 240, 241
 Jansen, Paul, 483
 Jennings, A., 152
 Jennings, L. S., 413
 Johnson, Charles R., 107, 115, 137, 152, 284, 306, 342, 543, 547, 551, 554
 Johnson, Samuel, 667
 Jones, Mark T., 227
 Jones, William B., 505
 Jordan, Camille, 281
 Jordan, T. L., 402
 Jordan, Wilhelm, 281

 Kågström, Bo, 302, 315, 316, 318
 Kahan, William M. (Velvel), 1 q, 26, 29, 30, 32, 42, 43 q, 44 q, 45, 47 q, 49, 55, 58, 59 q, 60, 69, 78, 83–86, 88, 103, 111, 114, 124, 149, 152, 157 q, 226, 242, 404, 483, 486, 491 q, 494–496, 498 q, 499, 504
 Kahaner, David K., 381 q
 Kailath, T., 429
 Kala, R., 318
 Kaniel, S., 227
 Kaporin, Igor, 447
 Karasalo, Ilkka, 152
 Karatsuba, A., 447
 Karlin, Samuel, 520
 Karlson, Rune, 392, 404, 406
 Karney, David L., 512, 522
 Karp, A., 187
 Karpinski, Richard, 52, 495
 Kato, Tosio, 115

 Kaufman, Linda, 213 q, 216, 226, 227, 376
 Kaufmann, Matt, 56
 Kearfott, R. Baker, 483
 Keiper, Jerry B., 31
 Keller, Herbert Bishop, 186, 257, 321 q
 Kelley, C. T., 468, 476
 Kennedy, Jr., William J., 29
 Kenney, Charles S., 298, 318, 523
 Kerr, Thomas H., 210
 Kielbasiński, Andrzej, 77, 85, 209, 241, 393, 404, 410, 414
 Kincaid, D. R., 499
 Kiriljuk, O. P., 285
 Kittaneh, Fuad, 523
 Knight, Philip A., 324, 347, 349, 351, 447, 448
 Knuth, Donald E., xxiii, xxviii, 49, 53, 54, 61 q, 79 q, 84, 85, 104, 447, 471 q, 489 q, 518 q, 523, 532
 Koç, Ç K., 103
 Koçak, Hüseyin, 29
 Koltracht, I., 128, 129
 Körner, T. W., 451 q
 Kornerup, Peter, 49
 Kosowski, Przemysław, 525
 Kostin, V. I., 285
 Kostlan, Eric, 516, 524
 Kovarik, Z. V., 132
 Kowalewski, G., 428
 Krasny, Robert, 502
 Kreczmar, Antoni, 446
 Krogh, F. T., 499
 Krückeberg, F., 191
 Kubota, Koichi, 485
 Kucherov, Andrey B., 209
 Kuczyński, J., 298
 Kuki, H., 56
 Kulisch, Ulrich W., 483
 Kuperman, I. B., 132

 La Porte, M., 486
 La Touche, Mrs., 79 q, 80
 Laderman, Julian, 437, 446
 Lagrange, Joseph Louis, 215
 Lancaster, Peter, 317, 468
 Lanczos, Cornelius, 486
 Langlois, Philippe, 484
 Laratta, A., 413, 414
 Larson, John L., 485
 Larsson, S., 29
 László, Lajos, 345

- Laub, Alan J., 298, 315–318, 523
 Läuchli, Peter, 379
 Lawson, Charles L., 375, 376, 399, 402, 403, 414, 499
 LeBlanc, E., 483
 Lee, King, 446, 447
 Lefèvre, Vincent, 50
 Lehmer, D. H., 527
 Lehoucq, Richard B., 563
 Lemeire, Frans, 152
 Leoncini, M., 282
 Leuprecht, H., 88
 LeVeque, Randall J., 11 q, 29
 Lewis, John G., 29, 188, 219, 227, 299, 302, 524
 Lewis, Robert Michael, 472
 Li, Kim-Hung, 524
 Li, T. Y., 282
 Li, Xiaoye S., 232, 492
 Lickteig, Thomas, 447
 Linnainmaa, Seppo, 49, 76, 84, 90, 501
 Linz, Peter, 90
 Linzer, Elliot, 456
 Liu, Joseph W. H., 209, 227
 Longley, James W., 402
 Lotti, Grazia, 443, 444
 Lu, Hao, 429
 Luszczek, Piotr, 232
 Lynch, Thomas W., 56
 Lyness, J. N., 429
 Lynn, M. Stuart, 323

 Mac Lane, Saunders, 479
 Macleod, Allan J., 189
 Makhoul, John, 210
 Malajovich, Gregorio, 288
 Malcolm, Michael A., 29, 86, 88, 188, 245 q, 260, 302, 495, 506, 522
 Malyshev, Alexander N., 152, 282, 404, 405
 Manne, F., 448
 Manteuffel, Thomas A., 152
 Marovich, Scott B., 570
 Martin, R. S., 188
 Mascarenhas, Walter, 189
 Mathias, Roy, 152, 208, 377, 514
 Matsui, Shouichi, 49
 Mattheij, R. M. M., 257
 Matula, David W., 49, 53, 57
 Mazzia, Francesca, 190
 McCarthy, Charles, 133
 McCracken, Daniel D., 76, 90, 102

 McKeeman, William Marshall, 188, 240
 McKenney, Alan, 317, 524
 McKinnon, K. I. M., 476
 Mead, J. L., 180
 Meinguet, Jean, 209, 281
 Mendelsohn, N. S., 283
 Metropolis, N., 486
 Meurant, Gérard, 257, 302
 Meyer, Jr., Carl D., 133, 332
 Milenkovic, Victor J., 29
 Miller, D. F., 317
 Miller, Webb, 76, 104, 438, 441, 471 q, 484, 485, 504, 515, 570
 Miranker, Willard L., 483
 Mirsky, L., 554
 Moler, Cleve B., 29, 30, 55, 76, 86, 152, 186, 188, 190, 231 q, 235, 240, 242, 245 q, 259 q, 260, 279, 295, 302, 317, 339 q, 342, 352, 381 q, 407 q, 429, 500, 505, 507, 523
 Møller, Ole, 83
 Montgomery, D., 183
 Moore, J. Strother, 56
 Moore, Ramon E., 483
 Morrison, Donald, 500, 507
 Mukherjea, Kalyan, 374
 Muller, Jean-Michel, 32, 50
 Müller, K. H., 102
 Murakami, H., 480 n
 Murray, Walter, 210, 226, 227, 413, 476

 Nash, Stephen G., 308, 319, 381 q, 562
 Nashed, M. Zuhair, 402
 Neal, Larry, 188
 Neumaier, A., 57, 85
 Neumann, M., 190
 Newbery, A. C. R., 102, 103
 Newcomb, Simon, 47
 Newman, Morris, 512, 518
 Ng, Michael K., 457
 Nickel, Karl, 85, 482 q, 483
 Nilson, E. N., 154
 Nocedal, Jorge, 226, 468
 Nordio, Marcelo, 189
 Notay, Yvan, 324

 Oberaigner, W., 88
 O'Cinneide, Colm Art, 133
 Oettli, W., 122, 132
 Ofman, Yu., 447
 O'Leary, Dianne Prost, 302
 Olesky, D. D., 190

- Oliver, J., 103, 428
 Olkin, Ingram, 524, 525
 Olshevsky, V., 428, 429
 Olver, F. W. J., 49, 69, 77, 102, 187
 Omladič, Matjaž, 210
 Opfer, Gerhard, 429
 Ortega, James M., 376
 Osborne, M. R., 413
 Ostrowski, A. M., 344, 543
 Overton, Michael L., 35 q
- Paige, Christopher C., 190, 209, 371,
 376, 377, 380, 386, 397, 403,
 404, 407 q, 413, 414, 565
 Palmer, John, 522
 Palmer, Kenneth J., 29
 Pan, Ching-Tsuan, 183
 Pan, Victor Y., 278, 282, 429, 436, 437,
 446
 Papadimitriou, Pythagoras, 377
 Papadopoulos, Philip M., 318
 Parhami, Behrooz, 56
 Park, Haesun, 376
 Parlett, Beresford N., xxviii, xxx, 24,
 30, 55, 76, 79 q, 184, 215, 226,
 353 q, 375, 376
 Pasternak, Mary E., 485
 Patashnik, Oren, 79 q, 518 q
 Paterson, Michael S., 102
 Patrick, Merrell L., 227
 Patterson, David A., 35 n, 53
 Paxson, Vern, 57
 Pelz, Richard, 502
 Peña, J. M., 152, 179, 180, 189, 190
 Penzl, Thilo, 317
 Percival, Colin, 457
 Pereyra, Victor, 423, 423 q, 425, 429
 Peters, G., 103, 188, 275, 281, 282, 404
 Peterson, Brian, 189
 Philippe, Bernard, 376
 Pichat, M., 88
 Pierce, Daniel J., 299
 Pinkus, Allan, 186, 188
 Plemmons, Robert J., 133, 152, 180, 190,
 299, 332, 404, 413
 Poljak, Svatopluk, 128
 Polman, Ben, 257
 Ponceleón, Dulce B., 227
 Poole, George, 188
 Poromaa, Peter, 302, 315, 318
 Pothén, Alex, 153
- Powell, M. J. D., 210, 362, 375, 395, 404,
 472, 474
 Power, Stephen, 209
 Prager, W., 122, 132
 Preparata, F. P., 282
 Press, William H., 476, 487, 505
 Priest, Douglas M., 28, 29, 54, 87–89,
 92, 499, 501, 529
 Pryce, J. D., 69, 241
 Puglisi, Chiara, 365
 Pukelsheim, Friedrich, 317
 Puschmann, Heinrich, 189
- Quinlan, Gerald D., 87
 Quinn, Thomas R., 87
 Quintana, Enrique S., 282
 Quintana, Gregorio, 282
- Rabinowitz, Philip, 90
 Raimi, Ralph A., 48
 Rall, Louis B., 402, 483, 485
 Ramos, George U., 456
 Rath, Wolfgang, 376
 Ratz, H. C., 323
 Razaz, M., 102
 Reese, M. S., 298
 Reichel, Lothar, 103, 350, 428–430, 525
 Reichelt, Mark W., 524
 Reid, John K., 180, 181, 186, 190, 193,
 227, 362, 375, 395, 400, 404
 Reinsch, C., xxix, 281, 577 q
 Reiser, John F., 54
 Ren, Huan, 56
 Renaut, R. E., 180
 Rew, R. K., 287 q, 297
 Rheinboldt, Werner C., 467, 467 q
 Rice, John R., 29, 376
 Rigal, J. L., 120, 132
 Robertazzi, T. G., 89
 Roberts, J. D., 317
 Rohn, Jiří, 116, 128, 135
 Romani, Francesco, 324
 Rose, Donald J., 257
 Rosenthal, Peter, 317
 Ross, D. R., 88
 Rowan, Thomas Harvey, 485, 486
 Rubin, Donald B., 381 q, 402
 Ruhe, Axel, 376
 Ruiz, Daniel, 337
 Rump, Siegfried M., 127–129, 191, 482,
 483
 Russinoff, David M., 56

- Rust, B. W., 132
 Rutishauser, Heinz, 512, 518

 Sadkane, Miloud, 404
 Sameh, Ahmed H., 150, 151, 180, 485
 Samelson, Klaus, 53
 Sande, G., 456
 Sanz-Serna, J. M., 29
 Saunders, Michael A., 226, 227, 229, 257, 413
 Sautter, Werner, 186
 Scarborough, James B., 28, 54
 Schatzman, James C., 456
 Schaumburg, Kjeld, 302
 Schelin, Charles W., 50
 Scherer, R., 76
 Schneider, Hans, 112, 115, 352
 Schonfelder, J. L., 102
 Schreiber, Robert S., 153, 168, 180, 245 q, 250–253, 256, 257, 278, 365, 375, 571
 Schryer, N. L., 496
 Schulz, Günther, 183, 278
 Schwartz, S. C., 89
 Schwetlick, Hubert, 77, 393, 404, 410, 414
 Searle, Shayle R., 317, 487
 Sha, Xuan-He, 437, 446
 Shampine, Lawrence F., 29, 76, 86, 524
 Shannon, Claude E., 56
 Shapiro, Alexander, 133
 Shepherd, David, 56
 Shewchuk, Jonathan Richard, 29, 91
 Shinnerl, Joseph R., 226, 229
 Shroff, Gautam M., 299
 Shub, Michael, 133, 516
 Simon, Horst D., 446, 447
 Skeel, Robert D., 31, 123, 133, 178, 186, 187, 190, 231 q, 235, 240, 482 q
 Slishman, Gordon, 446, 569
 Sloane, N. J. A., 168
 Smale, Steve, 2n
 Smith, David M., 502
 Smith, Francis J., 103, 427
 Smith, Jon M., 32
 Smith, Robert L., 500, 506
 Smith, Roger M., 446, 569
 Smoktunowicz, Alicja, 85, 129, 136, 241, 404, 525
 Snyder, James N., 231 q, 240
 Sokolnicka, Jolanta, 241

 Sorensen, Danny C., 180, 187, 226, 228, 256
 Sørevik, T., 448
 Spencer, Herbert, 657
 Spieß, J., 446
 Spooner, David, 485
 Steele, Jr., Guy L., 57
 Stegun, Irene A., 30
 Sterbenz, Pat H., 29, 30, 45, 53, 56, 486
 Stewart, G. W. (Pete), xxviii, 68, 102, 107, 115, 119 q, 126, 133, 139 q, 152, 163, 187, 190, 209, 231 q, 240, 241, 282, 295, 302, 307, 318, 339 q, 351, 373, 374, 376, 377, 382, 384, 397, 400, 402–404, 500, 517, 525
 Stewart, Michael, 210
 Stewart, William J., 302
 Stockmeyer, Larry J., 102
 Stoer, Josef, 76, 107, 114, 116, 187
 Stone, Betty Jane, 114
 Storey, C., 317
 Stoutemyer, David R., 486
 Strakoš, Zdeněk, 324
 Strang, Gilbert, 13 q, 112, 133, 457
 Strassen, Volker, 278, 434, 448, 449
 Straus, E. G., 126, 133
 Street, Anne Penfold, 168
 Stufken, John, 168
 Stummel, Friedrich, 30, 76, 187
 Sun, Ji-guang, 107, 115, 119 q, 126, 129, 133, 181, 190, 201, 209, 374, 377, 382, 384, 392, 400, 402, 404, 406, 411, 414, 429
 Sun, Xiaobai, 282, 376
 Sun, Zheng, 411, 414
 Swann, W. H., 472
 Swartzlander, Jr., Earl E., 49
 Sweeney, D. W., 56
 Swenson, J. R., 48
 Sylvester, James Joseph, 305 q, 317, 434

 Tang, Ping Tak Peter, 50, 496, 528
 Tang, W. P., 429
 Tao, Pham Dinh, 116, 301
 Tartaglia, Niccolo, 479
 Tasche, Manfred, 456
 Taussky, Olga, 511 q, 514
 Teukolsky, Saul A., 476, 487, 505
 Thacher, Jr., Henry C., 188
 Thisted, Ronald A., 29
 Thompson, Sir D'arcy Wentworth, 1 q

- Thron, W. J., 505
 Tienari, Martti, 49
 Tisserand, Arnaud, 50
 Tisseur, Françoise, 294, 461, 462, 468, 523
 Todd, John, 114, 115, 512, 518, 523, 524
 Toh, Kim-Chuan, 480 n, 523
 Toledo, Sivan, 248
 Torczon, Virginia J., 472, 475, 476
 Tornheim, L., 169, 170
 Traub, J. F., 428
 Trefethen, Lloyd N., 5 q, 157 q, 168, 169, 180, 323, 337, 339 q, 340, 342, 345, 346, 348, 350, 351, 480 n, 516, 523, 525
 Tremaine, Scott, 87
 Tropp, Henry S., 489 n
 Trossett, Michael W., 472
 Trummer, Manfred R., 340, 348, 350
 Tsao, Nai-kuan, 76, 375
 Tucker, Warwick, 482
 Tukey, John W., 56, 456
 Turing, Alan Mathison, xxix, xxx, 29, 114, 119 q, 184, 185, 281, 481
 contributions in 1948 paper "Rounding-off errors...", 184–185, 281
 Turnbull, H. W., 374, 521
 Turner, Peter R., 47, 49, 152
 Tyrtshnikov, Evgenij E., 523
- Underhill, L. G., 524, 525
 Ungar, Peter, 447
 Urabe, Minoru, 57
 Uspensky, J. V., 487
- Vajtersić, M., 448
 van de Geijn, Robert, 282
 van der Sluis, A., 125, 126, 190, 199, 381 q, 402
 Van der Vorst, Henk A., 187, 226, 256, 324
 van Leeuwen, J., 447
 Van Loan, Charles F., xxviii, 24, 102, 115, 129, 132, 136, 172, 187, 208, 210, 228, 231 q, 256, 297, 307, 308, 308 n, 339 q, 342, 345, 352, 363, 365, 375, 377, 380, 382, 397, 402, 405, 451 q, 452, 456
 van Veldhuizen, M., 189
 van Wijngaarden, A., 498
 Vanderbei, Robert J., 229
- Varah, James M., 133, 154, 257
 Varga, Richard S., 147, 154, 257, 505
 Vavasis, Stephen A., 278, 395
 Vemulapati, Udaya B., 299, 397
 Verdonk, Brigitte, 496
 Verschaeren, Dennis, 496
 Veselić, Krešimir, 210
 Vetterling, William T., 476, 487, 505
 Vieta, Franciscus, 479
 Vignes, J., 486
 Viswanath, D., 516
 Vitasek, Emil, 90
 Viten'ko, I. V., 86
 von Neumann, John, 29, 48, 183, 184, 188, 260, 515
- Waite, William, 495, 496
 Waldén, Bertil, 392, 404, 406
 Walker, Homer F., 323, 468
 Wallis, Jennifer Seberry, 168
 Wallis, W. D., 168
 Ward, Robert C., 282
 Warming, Robert F., 525
 Wasilkowski, G. W., 133
 Wasniewski, Jerzy, 302
 Watson, G. A., 133
 Watterson, Bill, 471
 Wedderburn, J. H. M., 105 q
 Wedin, Per-Åke, 382, 400, 402, 405, 413
 Wegert, Elias, 346
 Wei, Musheng, 380, 402
 Weidner, Peter, 483
 Weinberger, Hans F., 115
 Weiss, N., 133
 Welfert, B. D., 180
 Wendroff, Burton, 186, 190
 Werner, Wilhelm, 103
 Westin, Lars, 302, 318
 Westlake, Joan R., 512
 Wette, Matthew R., 317
 White, Jon L., 57
 White, Michael, 459 q
 Wichmann, B. A., 496, 499
 Wilkinson, J. H., xxv, xxviii–xxx, 12, 22, 27 q, 29, 29 q, 30, 35 q, 48, 52, 53, 55, 61 q, 66, 67, 67 q, 69, 76, 77, 90, 91, 93 q, 94, 102, 103, 105 q, 132, 139 q, 151, 152, 165, 167, 169, 172, 177 q, 180, 183–190, 195 q, 199, 200, 209, 232, 240, 252, 275, 281, 282, 295, 322, 323, 337 q, 342,

- 353 q, 357, 375, 376, 376 q, 378,
381, 389, 402–404, 459 q, 472,
481, 539, 577 q
- first program for Gaussian elimina-
tion, 188
- on the purpose of a priori error anal-
ysis, 195 q
- solving linear systems on desk cal-
culator in 1940s, 184
- user participation in a computation,
27
- Williams, Jack, 415 q
- Wilson, Greg, 56
- Winograd, Shmuel, 434–436, 448
- Wisniewski, John A., 485
- Witzgall, C., 107, 114, 116
- Wolfe, Jack M., 88
- Woodger, Michael, 157 q, 433 q
- Woźniakowski, H., 85, 133, 240, 241, 298,
324, 328, 466, 468
- Wrathall, Celia, 485
- Wright, Margaret H., 226, 471 q, 472,
476
- Wright, Stephen J., 167, 226, 468
- Yalamov, Plamen Y., 76, 456, 457
- Yeung, Man-Chung, 189
- Yip, E. L., 487
- Yohe, J. Michael, 483, 501
- Young, David M., 29, 322 t, 343
- Young, Gale, 114
- Ypma, Tjalling Y., 468, 469
- Yu, Y., 282
- Yuval, Gideon, 446
- Zawilski, Adam T., 324
- Zehfuss, Johann Georg, 317
- Zeller, K., 76
- Zeng, Z., 282
- Zeuner, Hansmartin, 456
- Zha, Hongyuan, 154, 189, 374, 524
- Zhang, Zhenyue, 524
- Zielke, Gerhard, 114, 524
- Ziętak, K., 318
- Ziv, Abraham, 50, 56, 76
- Zlatev, Zahari, 302

Subject Index

*Knowledge is of two kinds.
We know a subject ourselves,
or we know where we can find information upon it.*
— SAMUEL JOHNSON, *Boswell's Life of Johnson* (1775)

A suffix “t” after a page number denotes a table, “f” a figure, “n” a footnote, and “q” a quotation. Mathematical symbols and Greek letters are indexed as if they were spelled out. The solution to a problem is not indexed if the problem itself is indexed under the same term.

- Aasen's method, 222–225
 - growth factor, 224
- absolute error, 3
- absolute norm, 107
- accuracy versus precision, 6, 28
- ACRITH, 483
- adjugate matrix, 282
- Aitken extrapolation, 91
- alternating directions method, 474–475
- ATLAS, 579
- Augment precompiler, 483, 501
- augmented system, 383
 - iterative refinement on, 389
 - scaling and conditioning, 391
- Automatic Computing Engine (ACE), 53, 185 t, 188, 337
- automatic differentiation, 485
- automatic error analysis, 471–487, *see also* interval analysis; running error analysis
 - condition estimation, 477–478
 - roots of a cubic, 479–481
 - Strassen's inversion method, 478–479
 - using direct search optimization, 472–474
- back substitution, 140
- backward error, 6–7
 - columnwise, 122
 - componentwise, 122, 128
 - evaluating, 130
 - componentwise relative, 122
 - definition, 6
 - least squares problem, 392–395, 406
 - linear system
 - Oettli–Prager theorem, 122
 - Rigal–Gaches theorem, 12, 120
 - Lyapunov equation, 311–312
 - mixed forward-backward error, 7, 456
 - normwise, 120
 - normwise relative, 120
 - preserving symmetric structure, 394–395
 - row-wise, 122
 - structured, 129
 - Sylvester equation, 308–311
 - symmetric structure, preserving, 136
 - underdetermined system, 411
- backward error analysis
 - development by Wilkinson, 29–30, 185–186
 - in differential equations, 29
 - motivation, 6
 - not a panacea, 1 q
 - purpose, 65, 195 q
- backward stability
 - componentwise, 129
 - definition, 7

- normwise, 129
- row-wise, 130
- banded matrix, growth factor, 173
- Bartels–Stewart method, 307–308
- base, of floating point arithmetic, 36
- Bauer’s scaling theorem, 127, 133
- bilinear noncommutative matrix multiplication algorithm, 436–437
 - error analysis, 443–444
- binary–decimal conversion, 57
- bit, 56
- BLAS (Basic Linear Algebra Subprograms), 578
 - Extended and Mixed Precision, 64, 241, 462, 503, 579
 - fast level 3, 447
 - Technical Forum Standard, 503, 579
 - ∞ NRM2 (2-norm), 499–500, 507
- block algorithm
 - advantages of, 245 q
 - definition, 246
- block diagonal dominance, 251–255, 257
 - and block LU factorization, 252–255
 - definition, 251
- block LDL^T factorization (of symmetric matrix), 214–222
 - complete pivoting and its stability, 215–216
 - for tridiagonal matrix, 221–222
 - growth factor
 - complete pivoting, 216
 - partial pivoting, 218
 - rook pivoting, 221
 - partial pivoting and its stability, 216–219
 - rook pivoting and its stability, 219–221
- block LDL^T factorization (of skew-symmetric matrix), 225–226
 - growth factor, 226
- block LU factorization, 246, 247
 - computation, 247
 - error analysis, 250–256
 - existence and uniqueness, 247
 - stability
 - for (point) diagonally dominant matrix, 254–255
 - for block diagonally dominant matrix, 251–255
 - for block tridiagonal matrix, 257
 - for symmetric positive definite matrix, 255–256
- Bunch–Kaufman partial pivoting strategy, 216–219
- Bunch–Parlett complete pivoting strategy, 215–216
- byte, 56
- calculator, displaying words on, 32
- cancellation, 9–10, 27
 - in summation, 83, 539
 - not a bad thing, 10
 - of rounding errors, 19–22
- Cauchy matrix, 514–515
 - determinant, 515
 - inverse, 515
 - LU factors, 515
- Cauchy–Schwarz inequality, 106
- CELEFUNT package, 496
- CESTAC, 486
- Chebyshev spectral differentiation matrix, 340, 348
- Cholesky factorization, 196
 - computation of, 197
 - conditions for success in floating point, 200
 - error analysis, 197–200
 - existence and uniqueness, 196
 - perturbation bounds, 201
 - semidefinite matrix
 - complete pivoting, 202
 - computation of, 202
 - error analysis, 205–208
 - existence and uniqueness, 201
 - perturbation theory, 203–205
 - termination criteria, 207–208
- chopping, 54
- circulant matrix, 454
- circulant system, error analysis for solution by FFT, 454–456
- colon notation, 2
- commutation matrix, 317
- companion matrix, 522–523
 - singular values, 523
- comparison matrix, 145
- compensated summation, 83–88
- complete pivoting, 158
 - early use of, 188
 - fallacious criticism of, 193
 - for symmetric indefinite matrices, 215–216
 - growth factor, 169–170, 189

- conjecture proved false, 170
- use of, in practice, 188, 562
- complex arithmetic, error analysis, 71–73, 77
- complex number
 - division without overflow, 500, 506
 - square root of, 32
- componentwise relative error, 4
- condition number
 - distance to singularity and, 111, 114, 127
 - estimation, 287–304
 - asymptotic cost, 288
 - block 1-norm estimator, 294–295
 - counterexamples, 287 q, 288, 292–294, 297, 302
 - counterexamples by direct search, 477–478
 - for tridiagonal matrices, 299–301
 - incremental, 298
 - LAPACK estimator, 292–294, 477–478
 - LINPACK estimator, 295–297
 - probabilistic methods, 297–298
 - general theory, 29
 - Hadamard, 279, 282, 284
 - minimizing by scaling, 123, 125–127, 133
 - of function, 8
 - of linear system
 - componentwise, 123
 - normwise, 121
 - of matrix (rectangular), 382
 - of matrix (square), 109, 110, 114
 - of nonlinear system, 464–467
 - of summation, 91
 - Skeel's, 123
- conjugate gradient method, 324, 336
 - circulant preconditioner, 457
- continued fraction
 - algorithms and error analysis, 505
 - evaluating in IEEE arithmetic, 490–491
 - running error bound, 77
- convergent matrix, 332, 340
- conversion, binary–decimal, 57
- correct significant digits, 3–4, 28
- correlation matrix, 525
- Cramer's rule, (in)stability of, 13–14, 30, 33
- Cray computers
 - adoption of IEEE arithmetic, 44
 - arithmetic on, 35 q, 493
 - puzzling results from Cray Y-MP and Cray 2, 493–494
 - UNICOS library, 436, 438
- Crout's method, 163
- CS decomposition, 380, 400
- cubic equation
 - Newton's method, 486–487
 - stability of explicit formulae for roots, 479–481
- cyclic reduction, 190
- denormalized numbers, *see* subnormal numbers
- departure from normality (Henrici's), 344–345
- determinant, 279–280
 - computation of, 279–280
 - of upper Hessenberg matrix, 24–25
 - perturbation bound, 285
 - testing sign of, 282
- diagonal dominance, 172, *see also* block diagonal dominance
- diagonally dominant matrix
 - and block LU factorization, 254–255
- Gaussian elimination, error bounds for, 177
- growth factor, bound for, 172
- inverse, bound for, 154
- inverse by Gauss–Jordan elimination, error bound for, 277
- nonsingularity of, 190
- triangular, bound for cond, 144
- tridiagonal
 - bound for inverse, 300
 - bound for LU factors, 175
- differential equations, *see* ordinary differential equations
- direct search optimization methods, 474–477
- discretization error, 5
- distance to singularity
 - componentwise, 127
 - normwise, 111
- divided differences, 21, 99–101
 - confluent, 419–420
- Doolittle's method, 162–163, 187
- double rounding, 43, 58
- Drazin inverse, 331–332
- drift, in floating point arithmetic, 54

- dual norm, 107
- dual vector, 107
- effective conditioning, 133
- EISPACK, 579
- ELEFUNT package, 496
- elementary functions, 50
- equilibration, 123, 126, 178
- error
 - absolute, 3
 - backward, *see* backward error
 - forward, *see* forward error
 - mixed forward-backward error, 7, 456
 - relative, 3, 4
 - sources of, 5–6
- error analysis, *see* rounding error analysis
- ESSL library (IBM), 436, 438
- Euler's method, with compensated summation, 86
- expm1 function ($e^x - 1$), 30
- fan-in algorithm, 152
 - for summation, 80
 - for triangular system solution, 149–151
- fast Fourier transform, 451–457
 - Cooley–Tukey factorization of DFT matrix, 452
 - error bound, 453
 - for solving circulant systems, 454–456
 - inverse transform, 454
- fast matrix multiplication, 433–449
 - bilinear noncommutative algorithm, 436–437
 - deriving methods, 446
 - error analysis, 438–446
 - in the level-3 BLAS, 447
 - Miller's error results, 438
 - record exponent, 436
 - Strassen's method, 434–436
 - Winograd's variant, 435–436, 442–443
 - 3M method for complex multiplication, 437–438
 - Winograd's method, 434
- FFT, *see* fast Fourier transform
- FFTW, 457
- fixed point arithmetic, 53
- fl operator (rounding), 38
- floating point arithmetic, 35–60
 - alternatives to, 49
 - banned from safety-critical systems, 496
 - base, 36
 - choice of, 47, 56
 - binary–decimal conversion, 57
 - chopping, 54
 - compiler optimization, dangers of, 494
 - complex arithmetic, error analysis, 71–73
 - determining properties of, 494–495
 - drift in, 54
 - earliest subroutines, 35 q
 - elementary functions, 50
 - formal $\overline{\text{op}}$ algebra, 54–55
 - fused multiply-add operation, 46–47, 60
 - gradual underflow, *see* gradual underflow
 - guard digit, *see* guard digit
 - IEEE arithmetic, *see* IEEE arithmetic
 - Language Independent Arithmetic (LIA-1), 499
 - mantissa, 36 n
 - model, 54, 498–499
 - Brown's, 495, 498–499
 - complex arithmetic, 71
 - standard, 40
 - with underflow, 56–57
 - without guard digit, 44
 - monotonic, 56
 - multiple precision, 501–503
 - parameters for selected machines, 37 t
 - parameters in software, specifying, 496–497
 - representation error, 47
 - rounding, *see* rounding; double rounding
 - significand, 36
 - software issues, 489–509
 - speed of operations (relative), 56
 - subnormal numbers, 37, 42, 492
 - subtraction done exactly, 45
 - testing accuracy of, 51–52
 - testing correctness of, 495–496
 - unit roundoff, 3, 38
 - wobbling precision, 39, 47

- floating point coprocessors, 42
- floating point numbers
 - characterization, 36
 - normalized, 36
 - spacing between, 37
 - subnormal, 37, 42
 - testing for equality, 493
- flop, 3
- Fortran 95
 - environmental inquiry functions, 495
 - matmul, 447
- forward error, 6–7
 - definition, 6
 - for linear system, 12
 - linearized expression for, 484
 - mixed forward-backward error, 7, 456
- forward stability
 - componentwise, 130
 - definition, 9
 - normwise, 130
- forward substitution, 141
- Fourier matrix, 168
- FPV (floating point verification) package, 495–496
- Frank matrix, 463
- Frobenius norm, 107
- fused multiply-add operation, 46–47, 60

- γ_n (error constant)
 - definition, 63
 - properties, 67
- $\tilde{\gamma}_n$ (error constant), 68
- Gauss–Jordan elimination, 273–277, 281–282
 - algorithm, 273
 - error analysis, 273–277
- Gauss–Seidel method, 321 q, 329
- Gaussian elimination, 158–163, *see also* LU factorization
 - a posteriori stability tests, 180–181
 - complete pivoting, 158, *see also* complete pivoting
 - computer programs
 - first, 188
 - history of, 188
 - connection with LU factorization, 161
 - error analysis, 163–166
 - history of, 183–187
 - growth factor, 165–173, *see also* growth factor
 - in ancient China, 187
 - loop orderings, 187
 - need for pivoting, 158
 - on diagonally dominant matrix, 170–172
 - on Hessenberg matrix, 24–25
 - on tridiagonal matrix, 174
 - pairwise elimination, 180, 189
 - parallel variants of, 179–180
 - partial pivoting, 158, 162, *see also* partial pivoting
 - pessimism of its accuracy in 1940s, 183
 - pivoting strategy, choice of, 178–179
 - rook pivoting, 159, *see also* rook pivoting
 - row-wise error bounds, 177
 - scaling, row and column, 177–179
 - threshold pivoting, 193
 - use by Gauss, 187
 - versus Cramer’s rule, 13–14
 - without pivoting, instability of, 15
- GE, *see* Gaussian elimination
- Gelfand’s problem, 48
- generalized QR factorization, 397–398
- geometric computation, accuracy of algorithms in, 29
- GEPP, *see* Gaussian elimination, partial pivoting
- Givens rotation, 365
 - disjoint rotations, 367–368, 379
 - fast, 376
- GNU MP library, 502
- gradual underflow, 38, 42, 45, 56
- Gram–Schmidt method, 369–373
 - classical, 369
 - error analysis, 371, 373
 - modified, 370
 - connection with Householder QR factorization, 353 q, 371–372, 376
 - error analysis, 371–373
 - error analysis for application to LS problem, 386
 - stability, 24
 - reorthogonalization, 376
- group inverse, 331
- growth factor, 165–173
 - a posteriori estimates for, 180–181
 - define using exact or computed quan-

- titles?, 165, 189
 - for banded matrix, 173
 - for block LDL^T factorization, 216, 218, 221, 226
 - for complete pivoting, 169–170
 - for diagonally dominant matrix, 172
 - for partial pivoting, 166–173
 - large growth in practical problems, 167
 - for random matrices, 189
 - for rook pivoting, 170
 - for tridiagonal matrix, 173
 - for upper Hessenberg matrix, 172
 - lower bound for, 167
 - maximization by direct search, 472–473
 - numerical maximization for complete pivoting, 170, 189
 - statistical model of, 168
- guard digit, 44
 - test for, 52
- Haar distribution, random orthogonal matrix from, 517–518
- Hadamard condition number, 279, 282, 284
- Hadamard matrix, 116, 168, 170, 193
- Hadamard's inequality, 284
- Halley's method, 508
- Harwell–Boeing sparse matrix collection, 524
- Heron's formula, 45
- Hessenberg matrix
 - determinant of, 24–25, 30, 280
 - Gaussian elimination, 24–25, 30
 - growth factor for, 172
- Hewlett-Packard HP 48G calculator
 - condition estimator, 302
 - exhausting its range and precision, 15–16
- hidden bit, 41
- Hilbert matrix, 512–515, 523–524
 - Cholesky factor, 513
 - determinant, 513
 - inverse, 513
- Hölder inequality, 106, 107
- Horner's method, 94–104
 - for derivatives, 96–99
 - for rational function, 26
 - running error bound, 95–96, 103
- Hough's underflow story, 506–507
- Householder transformation, 354–355
 - aggregated (WY representation), 363–365
 - block, 375
 - choice of sign in, 355
 - error analysis, 357–363
 - history of, 374–375
 - in QR factorization, 355–357
- Hyman's method, 30, 280, 285
- IBM, ESSL library, 436, 438
- IEEE arithmetic, 39, 41–43
 - double rounding, 43, 58
 - exception handling, 41, 491–492
 - exceptions, 41–42
 - exploiting in software, 490–493
 - extended formats, 42
 - gradual underflow, 42
 - implementation using formal methods, 55–56
 - ∞ , 42, 490, 492
 - NaN, 41–42, 490, 492
 - parameters, 37 t, 41
 - recommended auxiliary functions, 492–493
 - rounding modes, 41
 - signed zeros, 42
 - Standard 754, 41
 - Standard 854, 43
 - subnormal numbers, 42, 492
- index of a matrix, 331
- inertia, 214 n
- ∞ (IEEE arithmetic), 42, 490, 492
- inner product
 - error analysis, 62–65
 - in extended precision, 64
 - reducing constant in error bound, 63
- Intel
 - Itanium chip, 46
 - Pentium chip, division bug, 55
- Internet, 574
- interval analysis, 41, 190, 481–484
 - dependencies, 482
 - fallibility, 483–484
 - Gaussian elimination, 482
 - super-accurate inner product, 483
- interval arithmetic, *see* interval analysis
- INTLAB, 483
- inverse iteration, 24
- inverse matrix, 259–285
 - bound using diagonal dominance, 154

- Csanky's method for, 278
- error analysis
 - for Gauss–Jordan elimination, 273–277
 - for LU factorization, 267–271
 - for triangular matrix, 262–267
- high-accuracy computation of, 281
- in solving $Ax = b$, stability, 260
- left and right residuals, 261
- Newton's method for, 278
- perturbation theory, 127–128
- Schulz iteration, 278
- Strassen's method for, 448–449, 478–479
- times for computation on early computers, 272 t
- triangular
 - bounds for, 147–149
 - methods for, 262–267
 - why not to compute, 260
- involuntary matrix, 519
- irreducible matrix, 127
- Itanium chip, 46
- iterative methods, *see also* stationary iterative methods
 - dates of publication, 322 t
 - error analysis, 325–335
 - survey of, 323–324
 - stopping criteria, 335–337, 467–468
- iterative refinement
 - backward error analysis, 235–239, 463
 - condition number estimate from, 242
 - for least squares problem, 388–391, 403
 - for square linear system, 27, 184, 231–243, 462–463
 - with QR factorization, 368–369
 - for Vandermonde system, 427–428
 - forward error analysis, 232–235, 463
 - in fixed precision, 179, 234, 494
 - in mixed precision, 234
 - LAPACK convergence test, 241
 - practical issues, 241–242
- Jacobi method, forward error analysis, 328–329
- Jordan canonical form, 340
- Kahan matrix, 149, 205
 - second smallest singular value, 154
- Kreiss matrix theorem, 346
- Kronecker product, 306, 317
- LANCELOT, 170, 189
- LAPACK, 579–581
 - block LDL^T factorization (of symmetric matrix), 228
 - block and partitioned LU factorization, 257
 - Cholesky factorization, 210–211
 - condition number estimation, 292–294, 303
 - forward error bound for linear systems, 131
 - iterative refinement, 241–242
 - least squares problem, 405
 - linear system, 191
 - LU factorization, 178, 191–192, 257
 - matrix 1-norm estimator, 292–294, 477–478
 - matrix inversion, 282–283
 - QR factorization, 377–378
 - Sylvester equation, 318
 - test matrix generation, 525
 - triangular systems, 153
 - 2×2 problems, solving, 497–498
 - underdetermined system, 414
 - xLAMCH for determining machine parameters, 495
- Latin, neoclassic, publishing papers in, 456
- LDL^T factorization, 197
- least significant digit, 36
- least squares problem, 382
 - augmented system, 383
 - scaling and conditioning of, 391
 - backward error, 392–395, 406
 - iterative refinement, 388–391, 403
 - linear equality constrained, 396–400
 - backward error, 405
 - elimination method, 399–400
 - method of weighting, 397
 - null space method, 397–399
 - perturbation theory, 396
 - linear inequality constrained, 405
 - Longley test problem, 402
 - modified Gram–Schmidt, error analysis, 386
 - normal equations, 382, 405
 - error analysis, 386–388
 - versus QR factorization, 388
 - perturbation theory, 382–384

- QR factorization, error analysis, 384–385
- quadratic constrained, 405
- seminormal equations, 391–392
- weighted, 395
- Leja ordering, 100, 101, 103, 104, 427
- level index arithmetic, 49
- linear system
 - large dense, in applications, 191
 - perturbation theory, 119–137
 - practical forward error bounds, 131
 - records for largest solved, 191 t
 - scaling before Gaussian elimination, 177–179, 190
 - times for solution on early computers, 185 t
- LINPACK, 579
 - Cholesky factorization of semidefinite matrix, 207
 - condition estimator, 295–297, 302
 - iterative refinement, 241
 - LU factorization, 178
 - matrix inversion, 264, 268, 269, 271
 - tridiagonal system solution, 303
- $\log(1+x)$, accurate evaluation, 32
- logarithmic distribution of numbers, 47, 49
- Longley test problem, 402
- LU factorization, 161, *see also* Gaussian elimination
 - a posteriori stability tests, 180–181
 - block, *see* block LU factorization
 - complete pivoting, 158, *see also* complete pivoting
 - CROUT's method, 163
 - determinantal formulae for factors, 161
 - Doolittle's method, 162–163
 - error analysis, 163–166
 - history of, 183–187
 - existence and uniqueness, 161
 - for nonsymmetric positive definite matrix, 208–209
 - growth factor, 165–173, *see also* growth factor
 - loop orderings, 187
 - of diagonally dominant matrix, 170–172
 - of Hessenberg matrix, 24–25
 - of tridiagonal matrix, 174
 - parallel variants of, 179–180
 - partial pivoting, 158, 162, *see also* partial pivoting
 - partitioned, 246
 - error analysis of, 249–250
 - perturbation bounds, 181–182
 - pivoting strategy, choice of, 178–179
 - rank-revealing, 182–183
 - recursively partitioned, 248
 - rook pivoting, 159, *see also* rook pivoting
 - scaling, row and column, 177–179
 - stability for M -matrix, 190
 - threshold pivoting, 193
 - versus Cramer's rule, 13–14
 - without pivoting, instability of, 15
- Lyapunov equation, 311
 - backward error, 311–312
 - discrete-time, 316–317
- M -matrix, 145, 152
 - stability of LU factorization, 190
 - triangular, 145–147
- machar code, 495
- machine epsilon, 37
- magic square matrix, p norm of, 115
- mantissa, 36 n
- Maple, 6, 170, 502
- Markov chain, perturbation analysis for, 133
- Mathematica, 6, 170, 502
- MATLAB, 3, 575
 - compan, 513 t, 523
 - condest, xxii, 295
 - eig, 464
 - eps, 39
 - fft, 454
 - frank, 513 t
 - gallery, 512, 513 t, 583
 - gallery('chebspec',...), 348, 513 t
 - gallery('clement',...), 239, 513 t
 - gallery('frank',...), 463
 - gallery('kahan',...), 513 t
 - gallery('orthog',...), 239, 473 n, 477, 513 t
 - gallery('pei',...), 513 t
 - gallery('randsvd',...), 513 t, 517 n, 524, 525
 - gallery('toeppen',...), 513 t, 525
 - gallery('tridiag',...), 513 t
 - hadamard, 513 t
 - hilb, 513 t

- ifft, 454
- invhlib, 513t
- inv, 261, 268
- magic, 513t
- normest1, 295
- pascal, 513t, 524
- pow2, 493
- rand, 513t, 516
- randn, 513t, 516
- rcond, 294
- realmax, 39
- realmin, 39
- roots, 480
- toeplitz, 513t
- vander, 513t
- Matrix Computation Toolbox, 583–585
- Symbolic Math Toolbox, 3, 6, 463, 514, 519
- matrix
 - adjugate, 282
 - block diagonally dominant, 251
 - Cauchy, 514–515
 - circulant, 454
 - commutation, 317
 - companion, 522–523
 - comparison, 145
 - condition number, 109, 110, 114, 382
 - confluent Vandermonde-like, 419
 - convergent, 332, 340
 - correlation, 525
 - diagonally dominant, 172
 - distance to singularity, 111, 127
 - Drazin inverse, 331–332
 - Fourier, 168
 - Frank, 463
 - group inverse, 331
 - Hadamard, 116, 168, 170, 193
 - Hilbert, 512–515, 523–524
 - inversion, 259–285, *see also* inverse matrix
 - involutary, 519
 - irreducible, 127
 - Kahan, 149, 154, 205
 - M -matrix, 145, 152
 - magic square, 115
 - moment, 518
 - nonsymmetric positive definite, 208
 - Pascal, 518–521
 - Pei, 284, 304
 - polynomials, 102
 - powers of, 339–352, *see also* powers of a matrix
 - pseudo-inverse, 382, 402, 405, 408
 - random, 515–518
 - randsvd, 517–518, 525
 - second difference, 522
 - semiconvergent, 332
 - skew-symmetric, 214, 225
 - submatrices, number of, 192
 - Sylvester's introduction of term, 305
 - symmetric indefinite, 214
 - symmetric positive definite, 196
 - symmetric positive semidefinite, 201
 - symmetric quasidefinite, 229
 - test, 511–525
 - totally nonnegative, 164
 - tridiagonal, 174–176
 - Toeplitz, 521–522
 - Vandermonde, 416
 - Vandermonde-like, 418
 - vec-permutation, 314, 317
- Matrix Market, 524
- matrix multiplication
 - backward error, 77
 - error analysis, 69–71, 78
 - fast methods, 433–449
- matrix norm, *see* norm
- meaningless answers, why you might get them, 30–31
- misconceptions, of floating point arithmetic, 28
- mixed forward-backward error, 7, 456
- modified Gram-Schmidt method, *see* Gram-Schmidt method, modified
- moment matrix, 518
- monotone norm, 107
- monotonicity
 - of floating point arithmetic, 56
 - of rounding, 38
- Moore-Penrose conditions, 405
- most significant digit, 36
- multidirectional search method, 475–477
- multiple precision arithmetic, 501–503
 - Bailey's package MPFUN, 501–502
 - Brent's package, 483, 501
 - GNU MP library, 502
- mutation testing, 515–516
- NAG Library, 575
 - LAPACK in, 580
 - machine constants, 497

- NaN (not a number), 41–42, 490, 492
- Nelder–Mead simplex method, 476–477
- netlib, 574–575
- Neville elimination, 180, 189
- Newton's method, 460
 - for eigenproblem, 463–464
 - for matrix inverse, 278
 - for reciprocation, 46
 - inexact, 468
 - limiting accuracy, 461
 - limiting residual, 462
 - sources of error in, 460
 - stopping criteria, 467–468
- Newton–Schulz iteration, 183
- nonsymmetric positive definite matrix, 208
 - LU factorization, stability of, 208–209
- norm, 105–117
 - $\|\cdot\|_{\alpha,\beta}$, explicit formulae for, 116
 - absolute, 107
 - consistent, 108
 - dual, 107
 - Frobenius, 107
 - Hölder inequality, 106
 - matrix, 107–112
 - norm equivalence constants, 109 t
 - matrix p -norm, 112–114
 - of magic square matrix, 115
 - monotone, 107
 - subordinate matrix, 108, 109
 - 2-norm, evaluation without overflow, 499–500
 - unitarily invariant, 108
 - vector, 106–107
 - norm equivalence constants, 109 t
- normal distribution, 3
- normal equations, 382, 405
 - error analysis, 386–388
- notation, explanation of, 2–3, 67–69
- 0^0 , definition, 59
- NPSOL, 189
- numerical analysis, definition, 5–6
- numerical radius, 343
- numerical stability
 - definition, 7, 29
 - for linear equation solvers, 129–130
- Oettli–Prager backward error theorem, 122
- ordinary differential equations
 - accuracy of mesh point formation, 92
 - backward error in, 29
 - Euler's method with compensated summation, 86
- outer product, error analysis, 64–65
- overflow, 16, 38
 - avoiding, 499–501
- p -norm power method, 289–291, 301–302
- pairwise (fan-in) summation, 80
- pairwise elimination, 180, 189
- parallel prefix operation, 103, 152
- paranoia code, 495
- partial pivoting, 158, 162
 - early use of, 188
 - for skew-symmetric matrices, 225
 - for symmetric indefinite matrices, 216–219
 - growth factor, 166–173
 - large growth in practical problems, 167
 - threshold pivoting, 193
- partitioned algorithm, definition, 246
- partitioned LU factorization, 246
 - error analysis, 249–250
- Pascal matrix, 518–521
 - Cholesky factor, 519
 - eigenvalue reciprocity, 519
 - inverse, 520
 - total positivity, 520
- Patriot missile software problem, 503–504
- Pei matrix, 284, 304
- Pentium chip, division bug, 55
- performance profile, for LAPACK norm estimator, 294
- perturbation theory
 - by calculus, 132
 - linear systems, 119–137
 - statistical, 133, 136
 - Sylvester equation, 313–315
- π (π), high-precision calculation as computer test, 489 q
- polar decomposition, 377, 380
- polynomials, 93–104, *see also* Horner's method
 - divided differences, 99–101
 - fast evaluation schemes, 103–104
 - matrix, evaluation of, 102
 - Newton form, 99–101

- PORT library, machine constants, 497
- portability of software, 496–499
- positive definite matrix, *see* nonsymmetric positive definite matrix; symmetric positive definite matrix
- power method, 22
 - for matrix 1-norm estimation, 292–295
 - for matrix p -norm estimation, 289–291, 301–302
- powers of a matrix, 339–352
 - behaviour of stationary iteration, 351
 - departure from normality, 344–345
 - hump, 342–343
 - in exact arithmetic, 340–346
 - in finite precision arithmetic, 346–350
 - pseudospectrum, 345–346, 349–350
 - role of spectral radius, 340–342
- PRECISE, 486
- precision
 - effect of increasing, 17–19
 - versus accuracy, 6, 28
- program verification, applied to error analysis, 485
- pseudo-inverse, 382, 402, 405, 408
- pseudospectral radius, 345
- pseudospectrum, 345–346, 349–350
 - of companion matrix, 523
- Pythagorean sum, 500, 507–509
- QR factorization, 355
 - column pivoting, 362, 378
 - generalized, 397–398
 - Givens, 365–366
 - cancellation of errors in, 21–22
 - error analysis, 366–368
 - Householder, 355–357
 - error analysis, 359–363
 - error analysis for application to LS problem, 384–385
 - error analysis for partitioned (WY representation), 363–365
 - iterative refinement for linear system, 368–369
 - perturbation theory, 373–374
 - rank-revealing, 377
 - row pivoting, 362
 - row sorting, 362
- quadratic equation, solving, 10–11, 29
- quadrature
 - accuracy of grid formation, 92
 - error bound for evaluation of rule, 78
- quasidefinite matrix, *see* symmetric quasidefinite matrix
- random matrices, 515–518
 - condition number of, 516
 - correlation matrices, 525
 - expected number of real eigenvalues, 516–517
 - orthogonal, 517, 524
 - spectral radius of, 516
 - tend to be well conditioned, 516
 - 2-norm of, 516
 - with given singular values, 517–518
- randsvd matrix, 517–518, 524, 525
- range reduction, 51
- RCOND condition estimator (LAPACK, LINPACK, MATLAB), 302, 477–478
- recursively partitioned LU factorization, 248
- relative error, 3, 4
 - componentwise, 4
- relative error counter, $\langle k \rangle$, 68
- relative precision, 69
- relative residual, 12
- research problems, 92, 104, 193, 212, 229, 242, 304, 319, 352, 406, 430, 449, 469, 487, 525
- residual, relative, 12
- Riccati equation, algebraic, 316
- Rigal–Gaches backward error theorem, 120
- rook pivoting, 159–160, 188
 - average number of comparisons, 160, 220
 - for symmetric indefinite matrices, 219–221
 - growth factor, 170
- rounding, 4, 38
 - dealing with ties, 38, 54
 - modes in IEEE arithmetic, 41
 - monotonicity of, 38
 - to even versus to odd, 54
- rounding error analysis
 - automatic, 471–487
 - demystified, 74–76
 - graphs in, 76
 - model
 - standard, 40

- with underflow, 56–57
 - without guard digit, 44
- notation, 67–69
- ordering of operations, effect of, 70, 141, 142
- purpose of, 65, 195 q
- statistical approach, 48–49
- rounding errors
 - accumulation of, 14
 - are not random, 25–26, 48
 - beneficial effects of, 22–24
 - cancellation of, 19–22
 - in subtraction, 45
 - statistical assumptions on, 48–49
- rules of thumb
 - condition for computed powers of matrix to converge to zero, 350
 - forward error related to backward error and condition number, 9
 - relative speed of floating point operations, 56
 - square root of constants in error bound, 48
- Runge–Kutta method, 83, 90
- running error analysis, 66, 486
 - for continued fraction, 77
 - for Horner's method, 95–96, 103
 - for inner product, 65–67
- sample variance, *see* variance
- ScaLAPACK, 580
- scaling a linear system before Gaussian elimination, 177–179, 190
- scaling to minimize the condition number, 123, 125–127, 178
- Schulz iteration, 183, 278
- Schur complement, 203, 209, 215, 246, 252
 - perturbation bounds for symmetric positive semidefinite matrix, 203–208
- second difference matrix, 522
- semiconvergent matrix, 332
- seminormal equations
 - for least squares problem, 391–392
 - for underdetermined system, 408
- separation (sep), of two matrices, 313
- Sherman–Morrison formula, 190, 487
- Sherman–Morrison–Woodbury formula, 258, 558
- Sierpinski gasket, 521
- significance arithmetic, 486
- significand, 36
- significant digits
 - correct, 3–4, 28
 - least and most significant, 36
- singular value decomposition (SVD), 114–115
- skew-symmetric matrix, 214, 225
- software
 - avoiding underflow and overflow, 499–501
 - effects of underflow, 501
 - issues in floating point arithmetic, 489–509
 - portability, 496–499
 - specifying arithmetic parameters, 496–497
 - specifying numerical constants, 498
- SOR method, forward error analysis, 329
- square root, of complex number, 32
- stability, *see* backward stability; forward stability
- stable algorithms, designing, 26–28
- stationary iterative methods, 321–337
 - and powers of a matrix, 351
 - backward error analysis, 330–331
 - forward error analysis, 325–329
 - singular systems, 333–335
 - Jacobi method, 328–329
 - scale independence, 327
 - singular systems, theory for, 331–333
 - SOR method, 329
 - stopping criteria, 335–337
- statistics, *see also* variance
 - computational references, 29
- sticky bit, 41
- Strassen's method, 434–436
 - accuracy compared with conventional multiplication, 441–442
 - error analysis, 440–443
 - error versus operation count, 14
 - for inversion, 448–449, 478–479
 - implementation issues, 446–447
 - Winograd's variant, 435–436, 442–443
- subdifferential, of a vector norm, 289
- subgradient, 290
- subnormal numbers, 37, 42, 492
- substitution, back and forward, 140–141
- subtraction, exactness in floating point arithmetic, 45

- successive overrelaxation method, *see* SOR method
- summation, 79–92
 - choice of method: summary, 89–90
 - compensated and applications, 83–88
 - condition number, 91
 - criterion for minimizing error, 82
 - distillation algorithms, 88
 - doubly compensated, 87–88
 - error analysis, 81–83
 - insertion method, 80
 - pairwise (fan-in), 80
 - recursive, 80
 - ordering in, 17, 82–83
 - statistical error estimates, 88–89
- SVD (singular value decomposition), 114–115
- Sylvester equation, 306
 - backward error, 308–311
 - Bartels–Stewart method, 307–308
 - generalizations, 316–318
 - perturbation theory, 313–315
 - practical error bounds, 315–316
 - solution methods, 307–308
- symbolic manipulation package, 6
- Symbolic Math Toolbox, 3, 6, 463, 514, 519
- symmetric indefinite factorization, *see* block LDL^T factorization (of symmetric matrix)
- symmetric indefinite matrix, 214
- symmetric positive definite matrix, 196
 - and block LU factorization, 255–256
 - practical test for, 210
- symmetric positive semidefinite matrix, 201
 - determinantal conditions for, 211
- symmetric quasidefinite matrix, 229
- synthetic division, 96
- tablemaker's dilemma, 4, 50
- test
 - for accuracy of floating point arithmetic, 51–52
 - for guard digit, 52
- test matrices, 511–525, *see also* MATLAB; matrix; random matrices
 - Matrix Computation Toolbox, 583–585
 - Harwell–Boeing collection, 524
 - Matrix Market, 524
- 3M method, 437–438, 447–448, 502
 - error analysis, 444–446
- threshold pivoting, 193
- Toeplitz matrix
 - pseudospectra, 525, 526 f
 - tridiagonal, 521–522
- totally nonnegative matrix, 164, 520
 - LU factorization, 164–165, 188
 - row scaling in, 179
 - test for, 188
- transformations, well conditioned, 27
- triangular matrix
 - bounds for inverse, 147–149
 - condition numbers, 142–143
 - inversion methods
 - blocked, 265–267
 - unblocked, 262–264
 - M*-matrix, 145–147
- triangular systems, 139–155
 - accurate solution of, 139 q, 143, 144, 147
 - conditioning, 144–145
 - fan-in algorithm, 149–151
 - partitioned inverse method, 153
 - substitution
 - backward error analysis, 140–142
 - forward error analysis, 142–147
- tridiagonal matrix, 174–176
 - condition number estimation, 299–301
 - growth factor, 173
 - LU factorization, 174
 - error analysis of, 174–176
 - structure of inverse, 300–302
 - Toeplitz, 521–522
- truncation error, 5
- Turing Award of the ACM, xxix, 55
- Turing programming language, 503
 - Numerical Turing, 503
- twisted factorization, 303
- 2 × 2 problems, reliable solution of, 497–498
- ulp (unit in last place), 39
- uncertainty, in data, 5
- underdetermined system, 408
 - backward error
 - normwise, 411
 - row-wise, 411
 - modified Gram–Schmidt, 413

- perturbation theory, 409–411
 - Q method (QR factorization), 408
 - error analysis, 411–413
 - seminormal equations, 408
 - error analysis, 412
- underflow, 16, 38
 - avoiding, 499–501
 - effects on software, 501
 - gradual, 38, 42, 45, 56
 - model for error analysis, 56–57
- UNICOS library (Cray), 436, 438
- unit roundoff, 3, 38
- update formula, involving small correction, 27

- van der Sluis's theorem, 125
- Vancouver Stock Exchange, inaccurate index, 54
- Vandermonde matrix
 - bounds and estimates for condition number, 417–418
 - definition, 416
 - inverse, 416–418
 - inversion algorithm, 417
 - LU factorization in factored form, 422
 - QR factorization, 429
 - structured condition number, 428–429
- Vandermonde system, 415–431
 - accuracy independent of condition number, 425
 - algorithm
 - for dual, 421
 - for primal, 422–423
 - for residual of confluent system, 427
 - backward error analysis, 425–426
 - complexity results, 429
 - curing instability, 427–428
 - forward error analysis, 424–425
 - history of solution methods, 429
 - preventing instability, 426–427
 - structured backward error, 429
- Vandermonde-like matrix
 - confluent, definition, 419
 - definition, 418
 - determinant, 430
- variance
 - algorithms for computing, 11–12, 29
 - condition numbers for, 32
 - error bound for two-pass formula, 33
 - vec operator, 306
 - vec-permutation matrix, 314, 317, 584
 - Venus probe, loss due to program bug, 489q

 - Wedin's least squares perturbation theorem, 382
 - proof, 400–402
 - Winograd's method, 434, 448
 - error analysis, 439–440
 - scaling for stability, 439–440
 - wobbling precision, 39, 47
 - WY representation of product of Householder matrices, 363–365
- 0^0 , definition, 59

Accuracy and Stability of Numerical Algorithms gives a thorough, up-to-date treatment of the behavior of numerical algorithms in finite precision arithmetic. It combines algorithmic derivations, perturbation theory, and rounding error analysis, all enlivened by historical perspective and informative quotations.

This second edition expands and updates the coverage of the first edition (1996) and includes numerous improvements to the original material. Two new chapters treat symmetric indefinite systems and skew-symmetric systems, and nonlinear systems and Newton's method. An expanded treatment of Gaussian elimination incorporates rook pivoting and additional error bounds. Other new topics include rank-revealing LU factorizations, weighted and constrained least squares problems, and the fused multiply-add operation found on some modern computer architectures.

Although not designed specifically as a textbook, this new edition is a suitable reference for an advanced course. It can also be used by instructors at all levels as a supplementary text from which to draw examples, historical perspective, statements of results, and exercises.

From reviews of the first edition:

"This definitive source on the accuracy and stability of numerical algorithms is quite a bargain and a worthwhile addition to the library of any statistician heavily involved in computing."

— Robert L. Strawderman, *Journal of the American Statistical Association*, March 1999.

"This text may become the new 'Bible' about accuracy and stability for the solution of systems of linear equations. It covers 688 pages carefully collected, investigated, and written ... One will find that this book is a very suitable and comprehensive reference for research in numerical linear algebra, software usage and development, and for numerical linear algebra courses."

— N. Köckler, *Zentralblatt für Mathematik*, Band 847/96.

"Nick Higham has assembled an enormous amount of important and useful material in a coherent, readable form. His book belongs on the shelf of anyone who has more than a casual interest in rounding error and matrix computations."

— G. W. Stewart, *SIAM Review*, March 1997.



Nicholas J. Higham is Richardson Professor of Applied Mathematics at the University of Manchester, England. He is the author of more than 80 publications and is a member of the editorial boards of *Foundations of Computational Mathematics*, the *IMA Journal of Numerical Analysis*, *Linear Algebra and Its Applications*, and the *SIAM Journal on Matrix Analysis and Applications*.

For more information about SIAM books, journals, conferences, memberships, or activities, contact:

siam

Society for Industrial and Applied Mathematics
3600 University City Science Center
Philadelphia, PA 19104-2688
215-382-9800 • Fax 215-386-7999
siam@siam.org • www.siam.org

ISBN 0-89871-521-0



9 780898 715217 90000

BKOT0080